

Contents

第二十三册简介：AI Infra、大模型基础设施与平台工程	35
本书定位	35
为什么需要这本书	35
适合读者	35
学完后应具备的能力	35
内容结构	35
阅读建议	35
与其他书的关系	36
第二十三册：AI Infra、大模型基础设施与平台工程	36
本书定位	36
阅读入口	36
适合读者	36
章节目录	36
第一部分：AI Infra 总览与硬件基础	36
第二部分：集群、网络、存储与调度	36
第三部分：大模型训练平台工程	37
第四部分：大模型推理平台工程	37
第五部分：数据、模型与实验平台	37
第六部分：可观测性、可靠性与成本治理	37
第七部分：AI Infra 系统设计与开放题	38
与其他书的关系	38
第一批优先扩写章节	38
第 1 章 AI Infra 总览：从模型 API 到基础设施底座	38
1.0 本讲资料边界与第二轮精修口径	38
1.1 一个简单 API 背后的复杂系统	39
1.2 AI Infra 包含哪些模块	39
1.2.1 计算基础设施	39
1.2.2 网络基础设施	40
1.2.3 存储基础设施	40
1.2.4 调度与资源管理	40
1.2.5 训练平台	40
1.2.6 推理平台	41
1.2.7 数据平台	41
1.2.8 模型与 Artifact 平台	41
1.2.9 评估与实验平台	41
1.2.10 可观测性、安全和成本治理	42
1.3 AI Infra 和 MLOps、LLMOps、Platform Engineering 的关系	42
1.3.1 MLOps	42
1.3.2 LLMOps	42
1.3.3 Platform Engineering	42
1.3.4 AI Infra	43
1.4 为什么算法岗也要懂 AI Infra	43
1.4.1 训练问题经常不是算法问题	43
1.4.2 推理效果和推理系统有关	43
1.4.3 成本是模型方案的一部分	43
1.5 AI Infra 的核心矛盾	43
1.5.1 性能与成本	44
1.5.2 利用率与隔离性	44
1.5.3 灵活性与标准化	44
1.5.4 速度与治理	44
1.6 一个完整 AI Infra 视图	44
1.7 面试中如何讲 AI Infra 总览	45

1.8 AI Infra 总览审计指标与最小 demo	45
1.9 常见误区	47
1.10 面试题	48
题 1: AI Infra 和普通后端基础设施有什么不同?	48
题 2: 为什么 GPU 多不一定训练快?	48
题 3: 训练平台和推理平台的关注点有什么不同?	48
题 4: AI Infra 为什么需要成本治理?	48
题 5: 为什么可观测性对 AI Infra 特别重要?	48
1.11 小练习	48
1.12 本章小结	49
第 2 章 AI Infra、MLOps、LLMOps、Platform Engineering 的边界	49
2.0 本讲资料边界与第二轮精修口径	49
2.1 为什么边界容易混乱	49
2.2 一张总览图	50
2.3 AI Infra 的核心边界	50
2.4 MLOps 的核心边界	51
2.5 LLMOps 的核心边界	51
2.6 Platform Engineering 的核心边界	52
2.7 DevOps、SRE 和 AI Infra 的关系	52
2.8 Data Platform 和 AI Infra 的关系	52
2.9 Model Platform 和 AI Infra 的关系	53
2.10 一个例子: 训练一个新模型	53
2.11 一个例子: 上线一个 RAG 应用	54
2.12 常见组织形态	55
2.13 如何判断一个需求属于哪一层	55
2.14 面试回答模板	55
2.15 边界审计指标与最小 demo	56
2.16 常见误区	58
2.17 面试题	59
题 1: AI Infra 和 MLOps 的区别是什么?	59
题 2: LLMOps 相比 MLOps 多了什么?	59
题 3: Platform Engineering 在 AI Infra 中的价值是什么?	59
题 4: 为什么说 AI Infra 不是简单的 DevOps?	59
题 5: 如果一个 RAG 应用回答质量下降, 应该从哪些层排查?	59
2.18 小练习	59
2.19 本章小结	60
第二章: 硬件、集群、网络、存储与调度	60
第 3 章 GPU、NPU、TPU 与 AI 加速器基础	60
3.0 本讲资料边界与第二轮精修口径	60
3.1 为什么大模型需要 AI 加速器	61
3.2 GPU 是什么	61
3.3 NPU 是什么	62
3.4 TPU 是什么	62
3.5 AI ASIC 与通用 GPU 的差异	63
3.6 AI 加速器的关键指标	63
3.6.1 峰值算力	63
3.6.2 显存容量	63
3.6.3 显存带宽	64
3.6.4 互联带宽	64
3.6.5 低精度支持	64
3.6.6 软件生态	64
3.7 为什么 CUDA 生态重要	65
3.8 训练和推理对硬件的需求不同	65

3.8.1 训练更关注什么	65
3.8.2 推理更关注什么	65
3.9 云上加速器与自建集群	66
3.10 加速器选型的思考框架	66
3.11 加速器容量审计指标与最小 demo	66
3.12 常见误区	70
3.13 面试题	70
题 1: 为什么大模型训练主要使用 GPU?	70
题 2: GPU、NPU、TPU 的区别是什么?	70
题 3: 为什么不能只看峰值 TFLOPS 选 GPU?	70
题 4: 训练和推理对加速器的需求有什么不同?	70
题 5: 为什么软件生态是硬件选型的重要因素?	70
3.14 小练习	71
3.15 本章小结	71
第三章: 大模型训练平台工程	71
第四章: 大模型推理平台工程	71
第 4 章 显存、HBM、PCIe、NVLink、NVSwitch 与带宽瓶颈	71
4.0 本讲资料边界与第二轮精修口径	72
4.1 为什么带宽这么重要	72
4.2 显存是什么	72
4.3 训练时显存为什么消耗大	73
4.4 推理时 KV cache 为什么重要	73
4.5 HBM 是什么	73
4.6 显存容量和显存带宽的区别	74
4.7 PCIe 是什么	74
4.8 NVLink 是什么	75
4.9 NVSwitch 是什么	75
4.10 单机多卡里的拓扑问题	75
4.11 多机训练里的网络带宽	76
4.12 带宽瓶颈怎么判断	76
4.12.1 显存带宽瓶颈	76
4.12.2 PCIe 瓶颈	76
4.12.3 GPU 间互联瓶颈	77
4.12.4 机间网络瓶颈	77
4.13 如何缓解带宽瓶颈	77
4.13.1 缓解显存容量压力	77
4.13.2 缓解显存带宽压力	77
4.13.3 缓解 PCIe 压力	78
4.13.4 缓解 GPU 间通信压力	78
4.13.5 缓解机间网络压力	78
4.14 面试中如何回答带宽瓶颈	78
4.15 带宽瓶颈审计指标与最小 demo	78
4.16 常见误区	82
4.17 面试题	82
题 1: 显存容量和显存带宽有什么区别?	82
题 2: 为什么训练比推理更吃显存?	82
题 3: KV cache 为什么会限制推理并发?	82
题 4: PCIe、NVLink、NVSwitch 分别解决什么问题?	82
题 5: 如何排查多机训练扩展效率差?	83
4.18 小练习	83
4.19 本章小结	83
第 5 章 GPU 利用率、MFU、HFU 与训练效率指标	83

5.0 本讲资料边界与第二轮精修口径	84
5.1 为什么只看 GPU 利用率不够	84
5.2 常见训练效率指标	84
5.3 tokens/s 和 samples/s	85
5.4 step time	85
5.5 MFU 是什么	85
5.6 HFU 是什么	86
5.7 MFU 和 GPU 利用率的差别	86
5.8 扩展效率 Scaling Efficiency	87
5.9 通信占比	87
5.10 数据加载和 I/O 指标	87
5.11 Checkpoint 对效率的影响	88
5.12 训练效率分析方法	88
5.13 常见诊断场景	89
5.13.1 GPU 利用率低	89
5.13.2 GPU 利用率高但 MFU 低	89
5.13.3 单卡快，多卡慢	89
5.13.4 step time 抖动大	89
5.14 一个训练效率面板应该有什么	90
5.15 面试中如何回答训练效率问题	90
5.16 训练效率审计指标与最小 demo	90
5.17 常见误区	94
5.18 面试题	95
题 1: GPU utilization 和 MFU 有什么区别?	95
题 2: HFU 和 MFU 的区别是什么?	95
题 3: 为什么 LLM 训练更常用 tokens/s, 而不是 samples/s?	95
题 4: 多机训练扩展效率低怎么办?	95
题 5: 训练速度很快但 loss 不正常, 应该怎么看?	95
5.19 小练习	95
5.20 本章小结	95
第五章: 数据、模型与实验平台	96
第六章: 可观测性、可靠性、成本与安全	96
第 6 章 大模型任务画像: 预训练、SFT、RLHF、评估、推理和 Agent	96
6.0 本讲资料边界与第二轮精修口径	96
6.1 什么是任务画像	96
6.2 预训练任务画像	97
6.2.1 资源特征	97
6.2.2 主要瓶颈	97
6.2.3 平台诉求	97
6.3 SFT 任务画像	98
6.3.1 资源特征	98
6.3.2 主要瓶颈	98
6.3.3 平台诉求	98
6.4 RLHF / RLAIIF 任务画像	99
6.4.1 资源特征	99
6.4.2 主要瓶颈	99
6.4.3 平台诉求	99
6.5 评估任务画像	100
6.5.1 资源特征	100
6.5.2 主要瓶颈	100
6.5.3 平台诉求	100
6.6 推理任务画像	100
6.6.1 资源特征	101

6.6.2 主要瓶颈	101
6.6.3 平台诉求	101
6.7 RAG 任务画像	101
6.7.1 资源特征	101
6.7.2 主要瓶颈	102
6.7.3 平台诉求	102
6.8 Agent 任务画像	102
6.8.1 资源特征	102
6.8.2 主要瓶颈	103
6.8.3 平台诉求	103
6.9 多模态任务画像	103
6.10 不同任务的资源画像对比	103
6.11 调度策略如何随任务变化	104
6.12 监控指标如何随任务变化	104
6.13 成本模型如何随任务变化	105
6.14 面试中如何回答任务画像问题	106
6.15 大模型任务画像审计指标与最小 demo	106
6.16 常见误区	113
6.17 面试题	114
题 1: 为什么预训练对 AI Infra 要求最高?	114
题 2: SFT 和预训练在平台诉求上有什么不同?	114
题 3: 推理任务为什么和训练任务的基础设施要求不同?	114
题 4: RAG 的瓶颈为什么不一定在 LLM?	114
题 5: Agent 工作负载为什么难以做成本治理?	114
6.18 小练习	114
6.19 本章小结	115

第七章：AI Infra 系统设计面试 115

第 7 章 GPU 集群架构：单机多卡、多机多卡和机柜级互联 115

7.0 本讲资料边界与第二轮精修口径	115
7.1 为什么需要 GPU 集群	116
7.2 GPU 集群的基本层次	116
7.3 单机多卡架构	117
7.4 单机多卡拓扑为什么重要	117
7.5 PCIe-only 多卡机器	118
7.6 NVLink / NVSwitch 多卡机器	118
7.7 多机多卡架构	119
7.8 多机训练中的通信模式	119
7.9 机柜级 GPU 互联	120
7.10 Fat-tree、Dragonfly 和其他网络拓扑	120
7.11 GPU 集群中的存储位置	120
7.12 GPU 集群中的控制面和数据面	121
7.13 故障域设计	121
7.14 拓扑感知调度	122
7.15 GPU 集群资源池设计	122
7.16 GPU 集群容量规划	123
7.17 一个 GPU 集群架构示例	123
7.18 GPU 集群拓扑审计指标与最小 demo	123
7.19 面试中如何回答 GPU 集群架构	129
7.20 常见误区	129
7.21 面试题	129
题 1: 单机多卡和多机多卡的主要区别是什么?	129
题 2: 为什么 NVSwitch 对大模型训练有价值?	129
题 3: 为什么 tensor parallel 通常更适合同机放置?	130
题 4: 什么是拓扑感知调度?	130

题 5: 为什么推理资源池最好和训练资源池隔离?	130
7.22 小练习	130
7.23 本章小结	130
第 8 章 网络基础: InfiniBand、RoCE、以太网和集合通信	130
8.0 本讲资料边界与第二轮精修口径	131
8.1 为什么大模型训练需要高速网络	131
8.2 网络指标怎么看	131
8.2.1 带宽	131
8.2.2 延迟	132
8.2.3 抖动	132
8.2.4 丢包和重传	132
8.2.5 拓扑和拥塞	132
8.3 RDMA 是什么	132
8.4 InfiniBand 是什么	132
8.5 RoCE 是什么	133
8.6 普通以太网能不能训练大模型	133
8.7 InfiniBand、RoCE、以太网对比	134
8.8 集合通信是什么	134
8.9 AllReduce	134
8.10 AllGather 和 ReduceScatter	135
8.11 NCCL 是什么	135
8.12 Ring 和 Tree 通信	135
8.13 通信和并行策略的关系	136
8.14 网络瓶颈的表现	136
8.15 网络问题排查路径	136
8.16 如何优化通信	137
8.17 网络与调度的关系	137
8.18 网络通信审计指标与最小 demo	138
8.19 面试中如何回答网络问题	143
8.20 常见误区	144
8.21 面试题	144
题 1: 为什么多机训练需要高速网络?	144
题 2: RDMA 和 GPUDirect RDMA 的价值是什么?	144
题 3: AllReduce 在数据并行中做什么?	144
题 4: FSDP / ZeRO 为什么会增加通信?	144
题 5: 如何优化多机通信?	144
8.22 小练习	145
8.23 本章小结	145
第 9 章 存储体系: 本地盘、共享文件系统、对象存储和数据湖	145
9.0 本讲资料边界与第二轮精修口径	145
9.1 大模型为什么依赖存储	146
9.2 存储指标怎么看	146
9.3 本地盘	147
9.4 共享文件系统	147
9.5 对象存储	147
9.6 数据湖	148
9.7 数据格式对训练性能的影响	149
9.8 小文件问题	149
9.9 训练数据供给链路	149
9.10 Checkpoint 存储需求	150
9.11 模型权重存储和加载	150
9.12 Artifact 存储	151
9.13 存储一致性和元数据	151
9.14 存储权限和合规	151

9.15 常见存储瓶颈	152
9.15.1 数据读取慢	152
9.15.2 Checkpoint 保存慢	152
9.15.3 模型加载慢	152
9.15.4 存储成本失控	152
9.16 存储优化策略	153
9.17 存储体系审计指标与最小 demo	153
9.18 面试中如何回答存储体系设计	159
9.19 常见误区	159
9.20 面试题	159
题 1: 本地盘、共享文件系统、对象存储分别适合什么场景?	159
题 2: 为什么小文件会拖慢训练?	159
题 3: 训练数据供给链路有哪些瓶颈?	159
题 4: checkpoint 存储要注意什么?	160
题 5: 模型加载慢怎么优化?	160
9.21 小练习	160
9.22 本章小结	160
第 10 章 Checkpoint 存储、加载、恢复和生命周期管理	160
10.0 本讲资料边界与第二轮精修口径	161
10.1 为什么 checkpoint 这么重要	161
10.2 Checkpoint 里到底有什么	161
10.3 模型权重 checkpoint 和训练状态 checkpoint	162
10.3.1 模型权重 checkpoint	162
10.3.2 训练状态 checkpoint	162
10.4 为什么 checkpoint 会很大	162
10.5 Checkpoint 保存频率	163
10.6 同步 checkpoint 和异步 checkpoint	163
10.6.1 同步 checkpoint	163
10.6.2 异步 checkpoint	163
10.7 分片 checkpoint	164
10.8 Manifest 和 metadata	164
10.9 Commit 语义	165
10.10 Checkpoint 加载	165
10.11 恢复训练的一致性	166
10.12 Rank 数变化和弹性恢复	166
10.13 Checkpoint 与抢占式调度	166
10.14 Checkpoint 生命周期管理	167
10.15 Checkpoint 成本治理	167
10.16 Checkpoint 监控指标	168
10.17 常见故障和排查	168
10.17.1 Checkpoint 保存很慢	168
10.17.2 Checkpoint 恢复失败	168
10.17.3 恢复后 loss 异常	169
10.18 Checkpoint 恢复测试	169
10.19 Checkpoint 生命周期审计指标与最小 demo	170
10.20 面试中如何回答 checkpoint 系统设计	178
10.21 常见误区	178
10.22 面试题	179
题 1: 模型权重 checkpoint 和训练状态 checkpoint 有什么区别?	179
题 2: 为什么大模型 checkpoint 很大?	179
题 3: 为什么 checkpoint 需要 manifest?	179
题 4: 异步 checkpoint 有什么风险?	179
题 5: 恢复后 loss 异常可能是什么原因?	179
10.23 小练习	179
10.24 本章小结	179

第 11 章 容器化基础：Docker、镜像、驱动、CUDA 和依赖环境	180
11.0 本讲资料边界与第二轮精修口径	180
11.1 为什么 AI Infra 需要容器化	180
11.2 Docker 是什么	181
11.3 镜像、容器和仓库	181
11.4 镜像分层	181
11.5 GPU 容器和普通容器的区别	182
11.6 驱动、CUDA 和 PyTorch 的关系	182
11.7 CUDA 兼容性	183
11.8 依赖管理	183
11.9 训练镜像和推理镜像的区别	184
11.10 镜像体积问题	184
11.11 镜像构建流水线	185
11.12 镜像安全	185
11.13 容器和多租户隔离	185
11.14 环境可复现性	186
11.15 常见环境问题	186
11.15.1 容器里看不到 GPU	186
11.15.2 CUDA 版本不匹配	186
11.15.3 NCCL 通信失败	187
11.15.4 本地能跑，集群不能跑	187
11.16 容器环境审计指标与最小 demo	187
11.17 面试中如何回答容器化设计	194
11.18 常见误区	194
11.19 面试题	195
题 1：Docker 镜像和容器有什么区别？	195
题 2：为什么 GPU 容器仍然依赖宿主机驱动？	195
题 3：为什么要记录镜像 digest？	195
题 4：训练镜像和推理镜像有什么不同？	195
题 5：容器里 NCCL 通信失败可能是什么原因？	195
11.20 小练习	195
11.21 本章小结	195
第 12 章 Kubernetes 与 GPU 资源管理	196
12.0 本讲资料边界与第二轮精修口径	196
12.1 Kubernetes 在 AI Infra 中的位置	196
12.2 Kubernetes 的基本对象	197
12.3 Kubernetes 如何表达资源	197
12.4 Device Plugin	197
12.5 GPU Operator	198
12.6 GPU 调度的基本流程	198
12.7 Kubernetes 默认调度的局限	198
12.8 Gang Scheduling	199
12.9 GPU 碎片化	199
12.10 拓扑感知调度	200
12.11 Node Label、Taint 和 Affinity	200
12.12 GPU 共享、MIG 和时间片	200
12.13 GPU 资源申请设计	201
12.14 K8s 上运行训练任务	201
12.15 K8s 上运行推理服务	202
12.16 Namespace、Quota 和多租户	202
12.17 GPU 监控	202
12.18 常见问题和排查	203
12.18.1 Pod 一直 Pending	203
12.18.2 Pod 启动后看不到 GPU	203
12.18.3 GPU 空闲但任务排队	203

12.18.4 训练任务部分 worker 启动失败	203
12.19 Kubernetes GPU 资源管理审计指标与最小 demo	204
12.20 面试中如何回答 K8s 与 GPU 管理	211
12.21 常见误区	212
12.22 面试题	212
题 1: Kubernetes 如何识别 GPU?	212
题 2: 为什么分布式训练需要 gang scheduling?	212
题 3: 什么是 GPU 碎片化?	212
题 4: K8s 默认调度器为什么不够?	212
题 5: MIG 适合什么场景?	212
12.23 小练习	213
12.24 本章小结	213
第 13 章 训练任务调度：队列、优先级、抢占、公平性和配额	213
13.0 本讲资料边界与第二轮精修口径	213
13.1 为什么训练调度很难	214
13.2 调度系统要解决什么问题	214
13.3 训练任务的资源请求	214
13.4 队列 Queue	215
13.5 优先级 Priority	215
13.6 配额 Quota	216
13.7 公平性 Fairness	216
13.8 抢占 Preemption	217
13.9 抢占策略如何设计	217
13.10 Gang Scheduling	217
13.11 Backfilling	218
13.12 GPU 碎片治理	218
13.13 拓扑感知调度	218
13.14 调度与 checkpoint 的关系	219
13.15 调度与成本治理	219
13.16 调度指标	219
13.17 训练调度治理审计指标与最小 demo	220
13.18 常见调度故障	229
13.18.1 大任务长期排队	229
13.18.2 小任务体验差	229
13.18.3 抢占过多	229
13.18.4 GPU 利用率高但产出低	229
13.19 面试中如何回答训练调度系统设计	230
13.20 常见误区	230
13.21 面试题	230
题 1: 为什么大模型训练需要 gang scheduling?	230
题 2: 什么是 GPU 碎片化? 如何缓解?	230
题 3: 抢占策略应该考虑哪些因素?	230
题 4: 配额和公平性有什么区别?	230
题 5: 为什么调度系统要知道 checkpoint 信息?	231
13.22 小练习	231
13.23 本章小结	231
第 14 章 多租户隔离：资源隔离、权限隔离和数据隔离	231
14.0 本讲资料边界与第二轮精修口径	231
14.1 什么是多租户	232
14.2 多租户隔离的层次	232
14.3 资源隔离	233
14.4 硬隔离和软隔离	233
14.5 权限隔离	234
14.6 数据隔离	234

14.7 数据分类与使用策略	235
14.8 网络隔离	235
14.9 运行时隔离	235
14.10 镜像和供应链隔离	236
14.11 密钥隔离	236
14.12 日志、Trace 和可见性隔离	237
14.13 成本隔离和归因	237
14.14 生产环境和实验环境隔离	237
14.15 跨租户共享	238
14.16 多租户隔离架构示例	238
14.17 多租户隔离审计指标与最小 demo	239
14.18 面试中如何回答多租户隔离	247
14.19 常见误区	248
14.20 面试题	248
题 1: AI Infra 多租户隔离包括哪些层?	248
题 2: 为什么数据隔离不只是文件权限?	248
题 3: 生产推理为什么要和实验训练隔离?	248
题 4: 如何避免密钥泄露?	248
题 5: 硬隔离和软隔离如何选择?	249
14.21 小练习	249
14.22 本章小结	249
第 15 章 集群容量规划: GPU 数量、网络带宽、存储吞吐和利用率	249
15.0 本讲资料边界与第二轮精修口径	249
15.1 为什么容量规划重要	250
15.2 容量规划不是只算 GPU	250
15.3 从任务画像开始	251
15.3.1 关键公式与容量规划速查	251
15.4 GPU 数量估算	252
15.5 平均负载和峰值负载	252
15.6 GPU 型号规划	253
15.7 训练容量规划	253
15.8 推理容量规划	254
15.9 网络容量规划	254
15.10 存储容量和吞吐规划	255
15.11 利用率目标	255
15.12 冗余和故障预留	256
15.13 增长预测	256
15.14 云上、混合云和自建容量	256
15.15 容量规划指标体系	257
15.16 常见容量规划错误	257
15.17 集群容量规划审计指标与最小 demo	258
15.18 面试中如何回答容量规划题	263
15.19 常见误区	264
15.20 面试题	264
题 1: 如何用 GPU-hours 估算训练容量?	264
题 2: 推理容量规划和训练容量规划有什么不同?	264
题 3: 为什么容量规划要区分 GPU 型号?	264
题 4: 为什么不能追求长期 100% GPU 利用率?	264
题 5: 容量 dashboard 应该看哪些指标?	264
15.21 小练习	264
15.22 本章小结	265
第 16 章 训练平台总览: 从脚本训练到平台化训练	265
16.0 本讲资料边界与第二轮精修口径	265
16.1 为什么需要训练平台	266

16.2 从脚本训练到平台化训练	266
16.3 训练平台的核心目标	267
16.4 训练平台总体架构	267
16.5 TrainingJob 是什么	268
16.6 任务提交入口	269
16.7 配置校验	269
16.8 资源调度	269
16.9 分布式训练启动	270
16.10 数据接入	270
16.11 Checkpoint 管理	270
16.12 日志、指标和事件	271
16.13 实验追踪	271
16.14 模型和 Artifact 注册	272
16.15 失败恢复	272
16.16 权限、安全和审计	272
16.17 成本归因	273
16.18 训练平台成熟度	273
16.19 训练平台审计指标与最小 demo	274
16.20 面试中如何回答训练平台总览	281
16.21 常见误区	281
16.22 面试题	282
题 1: 训练平台和训练脚本的区别是什么?	282
题 2: TrainingJob 应该包含哪些字段?	282
题 3: 为什么训练平台需要实验追踪?	282
题 4: 训练失败后平台应该如何处理?	282
题 5: 为什么模型产物不能只是一个文件路径?	282
题 6: 如何判断一个训练平台是否达到可上线门槛?	282
16.23 小练习	283
16.24 本章小结	283
第 17 章 训练任务提交系统: 配置、镜像、代码、数据和启动命令	283
17.0 本讲资料边界与第二轮精修口径	283
17.1 为什么任务提交系统重要	284
17.2 提交系统的核心职责	284
17.3 提交入口	284
17.4 TrainingJob Schema	285
17.5 基本信息	286
17.6 镜像字段	286
17.7 代码字段	287
17.8 数据字段	287
17.9 启动命令	288
17.10 训练配置	288
17.11 分布式配置	289
17.12 资源配置	289
17.13 Checkpoint 配置	289
17.14 日志和指标配置	290
17.15 队列和优先级配置	290
17.16 提交后的状态机	290
17.17 提交前校验和提交后校验	291
17.18 幂等性和重复提交	291
17.19 模板和复用	292
17.20 训练任务提交审计指标与最小 demo	292
17.21 面试中如何回答任务提交系统设计	302
17.22 常见误区	302
17.23 面试题	302
题 1: TrainingJob schema 为什么重要?	302

题 2: 为什么要记录镜像 digest 和代码 commit?	302
题 3: 提交系统需要校验哪些内容?	302
题 4: 为什么提交 API 需要幂等?	303
题 5: 为什么训练配置要记录最终生效版本?	303
17.24 小练习	303
17.25 本章小结	303
第 18 章 分布式训练启动器: torchrun、DeepSpeed、Megatron 和 Ray	303
18.0 本讲资料边界与第二轮精修口径	304
18.1 为什么需要启动器	304
18.2 分布式训练的基本概念	304
18.2.1 Rank	304
18.2.2 Local Rank	305
18.2.3 World Size	305
18.2.4 Master Addr 和 Master Port	305
18.3 启动器要设置哪些环境变量	305
18.4 torchrun	305
18.5 DeepSpeed Launcher	306
18.6 Megatron 启动方式	307
18.7 Ray	307
18.8 MPI 和其他启动方式	308
18.9 平台如何统一封装启动器	308
18.10 启动器和调度系统的关系	308
18.11 启动器和网络配置	309
18.12 启动器和日志聚合	309
18.13 启动器和失败处理	309
18.14 Elastic Training	310
18.15 启动器选择建议	310
18.16 分布式启动器审计指标与最小 demo	310
18.17 面试中如何回答分布式启动器	316
18.18 常见误区	317
18.19 面试题	317
题 1: rank、local rank 和 world size 分别是什么?	317
题 2: torchrun 主要解决什么问题?	317
题 3: DeepSpeed 和 torchrun 有什么区别?	317
题 4: Megatron 为什么强调 tensor parallel 和 pipeline parallel?	317
题 5: Ray 在训练平台里适合什么场景?	317
18.20 小练习	317
18.21 本章小结	318
第 19 章 训练配置管理: 超参、版本、环境和可复现性	318
19.0 本讲资料边界与第二轮精修口径	318
19.1 为什么训练配置管理重要	318
19.2 配置包含哪些内容	319
19.2.1 模型配置	319
19.2.2 数据配置	319
19.2.3 优化器配置	319
19.2.4 训练批次配置	320
19.2.5 精度配置	320
19.2.6 并行配置	320
19.2.7 Checkpoint 配置	321
19.2.8 评估配置	321
19.2.9 环境配置	321
19.2.10 平台配置	321
19.3 配置来源	321
19.4 配置覆盖顺序	322

19.5 最终生效配置	322
19.6 配置版本化	323
19.7 配置校验	323
19.7.1 Schema 校验	323
19.7.2 一致性校验	323
19.7.3 资源校验	324
19.7.4 策略校验	324
19.8 超参搜索和配置管理	324
19.9 随机种子和可复现性	324
19.10 环境版本记录	325
19.11 配置和 checkpoint 恢复	325
19.12 配置和模型发布	326
19.13 配置权限和审批	326
19.14 配置 diff 和实验对比	326
19.15 常见配置事故	327
19.15.1 使用了错误数据版本	327
19.15.2 命令行覆盖没人记录	327
19.15.3 batch size 算错	327
19.15.4 恢复 checkpoint 后效果异常	327
19.16 训练配置管理审计指标与最小 demo	327
19.17 面试中如何回答配置管理	335
19.18 常见误区	336
19.19 面试题	336
题 1: 为什么只保存 YAML 不够?	336
题 2: 训练配置应该包含哪些类型?	336
题 3: 配置校验有哪些层次?	336
题 4: 恢复 checkpoint 时为什么要校验配置?	336
题 5: 配置 diff 有什么价值?	336
19.20 小练习	337
19.21 本章小结	337

第 20 章 日志、指标、事件和训练状态追踪	337
20.0 本讲资料边界与第二轮精修口径	337
20.1 为什么训练可观测性重要	338
20.2 日志、指标、事件、Trace 的区别	338
20.3 训练日志	338
20.4 Rank 维度日志	339
20.5 日志脱敏和权限	339
20.6 训练指标	340
20.6.1 模型指标	340
20.6.2 性能指标	340
20.6.3 资源指标	340
20.6.4 平台指标	340
20.7 指标采集方式	341
20.8 指标粒度	341
20.9 事件 Events	341
20.10 训练状态机	342
20.11 Attempt 和重试追踪	342
20.12 Trace 设计	343
20.13 Dashboard 设计	343
20.14 告警设计	344
20.15 异常检测	344
20.16 日志、指标和实验追踪的关系	345
20.17 数据留存和成本	345
20.18 训练可观测性审计指标与最小 demo	345
20.19 面试中如何回答训练可观测性	353

20.20 常见误区	353
20.21 面试题	353
题 1: logs、metrics、events、traces 有什么区别?	353
题 2: 训练平台应该采集哪些关键指标?	353
题 3: 为什么需要 attempt_id?	353
题 4: 训练 dashboard 应该包含什么?	354
题 5: 为什么日志需要权限和脱敏?	354
20.22 小练习	354
20.23 本章小结	354
第 21 章 Checkpoint 策略: 频率、保留、异步保存和断点恢复	354
21.0 本讲资料边界与第二轮精修口径	355
21.1 Checkpoint 策略要解决什么问题	355
21.2 保存频率的 trade-off	355
21.3 RPO 和 RTO	355
21.4 按 step 保存还是按时间保存	356
21.5 保存哪些内容	356
21.5.1 Resume checkpoint	357
21.5.2 Eval checkpoint	357
21.5.3 Release checkpoint	357
21.6 保留策略	357
21.7 同步保存	358
21.8 异步保存	358
21.9 异步保存的一致性问题	358
21.10 分片保存策略	359
21.11 断点恢复策略	359
21.12 Checkpoint 与抢占联动	359
21.13 Checkpoint 与评估联动	360
21.14 Checkpoint 与模型发布联动	360
21.15 Checkpoint 监控和告警	360
21.16 常见策略错误	361
21.17 Checkpoint 策略审计指标与最小 demo	361
21.18 面试中如何回答 checkpoint 策略	368
21.19 常见误区	368
21.20 面试题	368
题 1: Checkpoint 保存频率如何选择?	368
题 2: Resume checkpoint、eval checkpoint、release checkpoint 有什么区别?	368
题 3: 异步 checkpoint 的关键风险是什么?	369
题 4: 可抢占训练任务为什么必须配 checkpoint?	369
题 5: 为什么不能只保留最新 checkpoint?	369
21.21 小练习	369
21.22 本章小结	369
第 22 章 训练容错: 节点失败、通信失败、OOM 和自动重试	369
22.0 本讲资料边界与第二轮精修口径	370
22.1 为什么训练容错重要	370
22.2 故障分类	370
22.3 可恢复故障和不可恢复故障	371
22.4 节点失败	371
22.5 GPU 故障	372
22.6 通信失败	372
22.7 OOM	373
22.8 数据故障	373
22.9 Checkpoint 故障	373
22.10 数值稳定性故障	374
22.11 自动重试策略	374

22.12 从 checkpoint 恢复	375
22.13 故障诊断报告	375
22.14 故障隔离和黑名单	376
22.15 容错和用户体验	376
22.16 训练容错审计指标与最小 demo	376
22.17 面试中如何回答训练容错	384
22.18 常见误区	384
22.19 面试题	384
题 1: 哪些训练故障适合自动重试?	384
题 2: OOM 应该如何处理?	384
题 3: NCCL timeout 如何排查?	384
题 4: 从 checkpoint 恢复后要验证什么?	384
题 5: 故障诊断报告应该包含什么?	385
22.20 小练习	385
22.21 本章小结	385
第 23 章 训练数据供给: 流式读取、缓存、shuffle 和数据局部性	385
23.0 本讲资料边界与第二轮精修口径	385
23.1 为什么数据供给重要	386
23.2 训练数据供给链路	386
23.3 离线预处理和在线预处理	387
23.4 数据格式和 shard	387
23.5 Dataset Manifest	388
23.6 流式读取	388
23.7 本地缓存	389
23.8 分布式缓存	389
23.9 Shuffle	390
23.10 多 rank 数据切分	390
23.11 数据局部性	390
23.12 Dataloader 性能	391
23.13 坏样本处理	391
23.14 数据供给监控	392
23.15 数据供给和 checkpoint 恢复	392
23.16 训练数据供给审计指标与最小 demo	392
23.17 面试中如何回答训练数据供给	400
23.18 常见误区	401
23.19 面试题	401
题 1: 为什么大规模训练要做数据 shard?	401
题 2: Dataset manifest 有什么作用?	401
题 3: 流式读取和本地缓存如何取舍?	401
题 4: shuffle 在流式训练中如何做?	401
题 5: 如何判断 GPU 在等数据?	401
23.20 小练习	401
23.21 本章小结	402
第 24 章 训练平台权限、安全和审计	402
24.0 本讲资料边界与第二轮精修口径	402
24.1 训练平台为什么高风险	403
24.2 安全目标	403
24.3 身份模型	403
24.4 RBAC 和 ABAC	404
24.5 数据权限	404
24.6 数据等级传播	404
24.7 资源权限	405
24.8 镜像安全	405
24.9 代码安全	405

24.10 密钥管理	406
24.11 网络安全	406
24.12 日志安全	406
24.13 模型产物权限	407
24.14 高风险操作审批	407
24.15 审计日志	408
24.16 安全事件响应	408
24.17 训练平台安全审计指标与最小 demo	408
24.18 面试中如何回答训练平台安全	416
24.19 常见误区	416
24.20 面试题	417
题 1: 训练平台有哪些身份?	417
题 2: 为什么训练产物要继承数据等级?	417
题 3: 镜像安全要做哪些事?	417
题 4: 哪些训练操作属于高风险操作?	417
题 5: 审计日志应该记录什么?	417
24.21 小练习	417
24.22 本章小结	418
第 25 章 训练平台系统设计面试题	418
25.0 本讲资料边界与第二轮精修口径	418
25.1 面试题描述	418
25.2 需求澄清	419
25.3 核心目标	419
25.4 总体架构	419
25.5 TrainingJob 设计	420
25.6 提交流程	420
25.7 调度设计	420
25.8 分布式启动设计	421
25.9 数据层设计	421
25.10 Checkpoint 设计	422
25.11 可观测性设计	422
25.12 容错设计	423
25.13 安全和审计设计	423
25.14 成本治理	423
25.15 一个完整流程	424
25.16 面试回答模板	424
25.17 系统设计审计指标与最小 demo	425
25.18 常见扣分回答	434
25.19 面试题	434
题 1: 训练平台的核心对象是什么?	434
题 2: 为什么不能只用 Kubernetes Job?	434
题 3: 训练平台如何保证可复现?	435
题 4: 训练平台如何处理失败?	435
题 5: 训练平台如何做多租户治理?	435
25.20 小练习	435
25.21 本章小结	435
第 26 章 推理平台总览：从单模型服务到多模型平台	435
26.0 本讲资料边界与第二轮精修口径	436
26.1 从单模型服务开始	436
26.2 推理平台要解决什么问题	437
26.3 推理和训练的差异	437
26.4 推理平台总体架构	437
26.5 推理平台的核心指标	438
26.6 Prefill 和 Decode	438

26.7 KV Cache 为什么关键	439
26.8 从单模型服务到多模型平台	439
26.9 Model Registry 的作用	439
26.10 Model Router 的作用	440
26.11 Runtime 的作用	440
26.12 Batching 为什么重要	441
26.13 自动扩缩容不是只看 GPU 利用率	441
26.14 冷启动和热启动	441
26.15 灰度、A/B 测试和回滚	442
26.16 限流、熔断、超时和降级	442
26.17 安全和治理	442
26.18 成本治理	443
26.19 一个完整请求流程	443
26.20 推理平台总览审计指标与最小 demo	443
26.21 常见误区	453
26.22 面试回答模板	453
26.23 小练习	453
26.24 本章小结	454
第 27 章 模型服务运行时: Triton、vLLM、TGI、SGLang 与自研 runtime	454
27.0 本讲资料边界与第二轮精修口径	454
27.1 什么是模型服务运行时	454
27.2 Runtime 和平台层的边界	455
27.3 一个 Runtime 的内部结构	455
27.4 Triton 的定位	456
27.5 vLLM 的定位	456
27.6 TGI 的定位	457
27.7 SGLang 的定位	457
27.8 TensorRT-LLM 的定位	457
27.9 自研 Runtime 什么时候值得做	458
27.10 Runtime 选型维度	458
27.11 性能指标怎么比较	459
27.12 Runtime 的平台集成方式	459
27.13 Runtime 指标必须暴露什么	460
27.14 Runtime 的稳定性问题	460
27.15 Runtime 和量化	461
27.16 Runtime 和分布式推理	461
27.17 模型服务运行时审计指标与最小 demo	462
27.18 面试常见追问	467
27.19 小练习	467
27.20 本章小结	468
第 28 章 Prefill、Decode、KV Cache 与推理资源画像	468
28.0 本讲资料边界与第二轮精修口径	468
28.1 一次大模型生成请求发生了什么	468
28.2 Prefill 是什么	469
28.3 Decode 是什么	469
28.4 Prefill 和 Decode 的核心差异	469
28.5 TTFT 和 TPOT	470
28.6 KV Cache 是什么	470
28.7 KV Cache 为什么占显存	470
28.8 一个直观例子	471
28.9 推理资源画像	471
28.10 为什么 QPS 不够用	471
28.11 Prefill 的优化方向	472
28.12 Decode 的优化方向	472

28.13 KV Cache 的管理策略	472
28.14 PagedAttention 的直觉	473
28.15 长上下文的特殊问题	473
28.16 多租户下的 KV Cache 问题	473
28.17 Prefill/Decode 分离	474
28.18 如何定位推理慢	474
28.19 Prefill/Decode/KV 资源画像审计指标与最小 demo	475
28.20 面试常见追问	480
28.21 小练习	480
28.22 本章小结	480
第 29 章 Continuous Batching、PagedAttention 和队列调度	481
29.0 本讲资料边界与第二轮精修口径	481
29.1 为什么需要 batching	481
29.2 静态 batching 的问题	481
29.3 Continuous Batching 的直觉	482
29.4 Continuous Batching 的收益	482
29.5 Continuous Batching 的代价	482
29.6 PagedAttention 解决什么问题	482
29.7 PagedAttention 和 Continuous Batching 的关系	483
29.8 队列调度为什么重要	483
29.9 一个简单调度流程	484
29.10 调度目标之间的冲突	484
29.11 常见队列策略	484
29.12 Token Budget 调度	485
29.13 长短请求混跑的问题	485
29.14 Prefill 和 Decode 的调度冲突	485
29.15 公平性调度	486
29.16 Admission Control	486
29.17 Streaming 对调度的影响	487
29.18 指标设计	487
29.19 常见故障模式	487
29.20 Continuous Batching / PagedAttention 调度审计指标与最小 demo	488
29.21 面试常见追问	492
29.22 小练习	493
29.23 本章小结	493
第 30 章 模型路由：模型选择、能力路由、成本路由和降级路由	493
30.0 本讲资料边界与第二轮精修口径	493
30.1 为什么需要模型路由	494
30.2 模型路由和负载均衡的区别	494
30.3 模型路由的输入信息	495
30.4 模型路由的输出结果	495
30.5 显式模型选择	495
30.6 能力路由	496
30.7 成本路由	496
30.8 质量路由	497
30.9 延迟路由	497
30.10 降级路由	497
30.11 Fallback Chain	498
30.12 灰度路由	498
30.13 多地域路由	499
30.14 Router 的实时信号	499
30.15 Router 的配置管理	499
30.16 Router 的可观测性	500
30.17 Router 的风险	500

30.18 一个 Router 决策流程	500
30.19 模型路由审计指标与最小 demo	501
30.20 面试常见追问	509
30.21 小练习	510
30.22 本章小结	510
第 31 章 缓存体系：prompt cache、KV cache、语义缓存和结果缓存	511
31.0 本讲资料边界与第二轮精修口径	511
31.1 为什么大模型推理需要缓存	511
31.2 四类常见缓存	511
31.3 Prompt Cache 是什么	512
31.4 Prompt Cache 的命中条件	512
31.5 Prefix Cache	512
31.6 KV Cache 回顾	513
31.7 KV Cache 的生命周期	513
31.8 语义缓存是什么	513
31.9 语义缓存的收益和风险	513
31.10 结果缓存是什么	514
31.11 缓存 Key 如何设计	514
31.12 缓存粒度	514
31.13 缓存与安全隔离	515
31.14 缓存与一致性	515
31.15 缓存与随机性	516
31.16 缓存与流式输出	516
31.17 缓存驱逐策略	516
31.18 缓存指标	516
31.19 缓存策略如何落地	517
31.20 缓存体系审计指标与最小 demo	517
31.21 常见误区	526
31.22 面试常见追问	526
31.23 小练习	527
31.24 本章小结	527
第 32 章 自动扩缩容：QPS、延迟、队列长度和 GPU 利用率	528
32.0 本讲资料边界与第二轮精修口径	528
32.1 为什么推理服务需要自动扩缩容	528
32.2 普通 HPA 为什么不够	528
32.3 扩缩容目标	529
32.4 QPS 指标的局限	529
32.5 延迟指标	529
32.6 队列长度和等待时间	530
32.7 GPU 利用率的误区	530
32.8 KV Cache 水位	530
32.9 Token 吞吐指标	531
32.10 冷启动问题	531
32.11 热池和预热	531
32.12 扩容策略	532
32.13 缩容策略	532
32.14 Draining 机制	532
32.15 多模型扩缩容	533
32.16 多租户扩缩容	533
32.17 扩缩容和路由联动	533
32.18 扩缩容控制闭环	534
32.19 如何避免扩缩容振荡	534
32.20 自动扩缩容审计指标与最小 demo	534
32.21 常见误区	539

32.22 面试常见追问	540
32.23 小练习	540
32.24 本章小结	540
第 33 章 限流、熔断、重试、超时和降级	541
33.0 本讲资料边界与第二轮精修口径	541
33.1 为什么推理平台需要保护机制	541
33.2 五类保护机制	541
33.3 限流解决什么问题	542
33.4 限流维度	542
33.5 QPS 限流为什么不够	542
33.6 Token Bucket 和 Leaky Bucket	543
33.7 Admission Control 和限流的区别	543
33.8 熔断解决什么问题	543
33.9 熔断状态	543
33.10 熔断触发条件	544
33.11 重试的危险	544
33.12 什么错误可以重试	544
33.13 重试策略	545
33.14 超时为什么重要	545
33.15 多层超时	545
33.16 Timeout Budget	545
33.17 降级解决什么问题	546
33.18 常见降级方式	546
33.19 降级策略要可观测	546
33.20 保护机制之间的组合	546
33.21 保护策略审计指标与最小 demo	547
33.22 常见误区	555
33.23 面试常见追问	556
33.24 小练习	556
33.25 本章小结	557
第 34 章 灰度发布、A/B 测试、回滚和模型版本管理	557
34.0 本讲资料边界与第二轮精修口径	557
34.1 为什么模型发布更复杂	557
34.2 模型版本管理的基本对象	558
34.3 ModelVersion 应该记录什么	558
34.4 发布状态机	558
34.5 发布前校验	559
34.6 灰度发布是什么	559
34.7 灰度维度	559
34.8 稳定分桶	560
34.9 A/B 测试是什么	560
34.10 A/B 测试和灰度的区别	560
34.11 模型发布指标	560
34.12 Guardrail 指标	561
34.13 回滚机制	561
34.14 回滚触发条件	562
34.15 兼容性问题	562
34.16 Prompt 版本也要管理	562
34.17 Runtime 配置也要管理	563
34.18 发布审计	563
34.19 一个完整发布流程	563
34.20 发布治理审计指标与最小 demo	564
34.21 常见误区	570
34.22 面试常见追问	570

34.23 小练习	571
34.24 本章小结	571
第 35 章 推理平台系统设计面试题	571
35.0 本讲资料边界与第二轮精修口径	572
35.1 面试题拆解	572
35.2 先做需求澄清	572
35.3 定义核心目标	572
35.4 总体架构	573
35.5 请求链路	573
35.6 Runtime 设计重点	574
35.7 Prefill、Decode 和 KV Cache 怎么讲	574
35.8 模型路由设计	574
35.9 缓存设计	575
35.10 自动扩缩容设计	575
35.11 限流和稳定性设计	576
35.12 灰度发布和回滚设计	576
35.13 多租户设计	577
35.14 可观测性设计	577
35.15 成本治理设计	577
35.16 安全设计	578
35.17 系统瓶颈分析	578
35.18 面试回答模板	578
35.19 系统设计审计指标与最小 demo	579
35.20 常见扣分回答	583
35.21 面试官可能继续追问	584
35.22 小练习	584
35.23 本章小结	584
第 36 章 数据平台与训练数据供给链路	585
36.0 本讲资料边界与第二轮精修口径	585
36.1 为什么需要数据平台	585
36.2 数据平台要解决什么问题	585
36.3 训练数据供给链路总览	586
36.4 数据来源	586
36.5 数据接入 Ingestion	587
36.6 Raw Data Lake	587
36.7 数据清洗	587
36.8 去重 Deduplication	588
36.9 PII 脱敏和安全过滤	588
36.10 数据质量评分	589
36.11 Dataset Builder	589
36.12 Dataset Manifest	590
36.13 数据格式	590
36.14 Tokenization 在数据链路中的位置	590
36.15 Sharding	591
36.16 数据读取链路	591
36.17 数据缓存	592
36.18 Shuffle 设计	592
36.19 数据权限	592
36.20 数据血缘	593
36.21 数据供给和训练平台的接口	593
36.22 数据平台供给链路审计指标与最小 demo	593
36.23 常见故障	601
36.24 面试常见追问	601
36.25 小练习	601

36.26 本章小结	602
第 37 章 数据版本管理、数据血缘和数据质量监控	602
37.0 本讲资料边界与第二轮精修口径	602
37.1 为什么数据版本管理重要	602
37.2 数据版本和代码版本的区别	603
37.3 一个 Dataset Version 应该包含什么	603
37.4 Manifest 版本化	603
37.5 不可变数据版本	604
37.6 增量版本	604
37.7 数据血缘是什么	605
37.8 数据血缘图	605
37.9 为什么血缘重要	606
37.10 数据删除和影响分析	606
37.11 数据质量监控为什么重要	606
37.12 数据质量指标	607
37.13 分布漂移监控	608
37.14 Dataset Diff Report	608
37.15 数据质量门禁	608
37.16 Train/Eval 泄漏监控	609
37.17 标注数据质量监控	609
37.18 合成数据质量监控	610
37.19 数据质量告警	610
37.20 数据版本和训练任务联动	610
37.21 数据版本和模型版本联动	611
37.22 常见系统设计	611
37.23 数据版本、血缘与质量监控审计指标和最小 demo	611
37.24 常见误区	619
37.25 面试常见追问	619
37.26 小练习	620
37.27 本章小结	620
第 38 章 模型仓库：权重、tokenizer、配置、adapter 和量化版本	620
38.0 本讲资料边界与第二轮精修口径	620
38.1 为什么需要模型仓库	621
38.2 模型仓库解决什么问题	621
38.3 模型仓库中的核心对象	621
38.4 Model 和 ModelVersion	622
38.5 ModelVersion 应该记录什么	622
38.6 权重文件管理	623
38.7 Safetensors 为什么常见	623
38.8 Tokenizer 管理	623
38.9 Model Config 管理	624
38.10 Adapter 管理	624
38.11 Merge 后模型	625
38.12 量化版本管理	625
38.13 Runtime 兼容性	626
38.14 模型血缘	626
38.15 评估报告关联	626
38.16 发布状态管理	627
38.17 权限控制	627
38.18 安全扫描	627
38.19 生命周期管理	627
38.20 模型仓库和推理平台的接口	628
38.21 模型仓库和训练平台的接口	628
38.22 模型仓库和评估平台的接口	628

38.23 一个模型仓库系统架构	629
38.24 模型仓库审计指标和最小 demo	629
38.25 常见误区	637
38.26 面试常见追问	638
38.27 小练习	638
38.28 本章小结	638
第 39 章 Artifact 管理: dataset、checkpoint、eval report 和 deployment package	639
39.0 本讲资料边界与第二轮精修口径	639
39.1 什么是 Artifact	639
39.2 为什么 Artifact 管理重要	639
39.3 Artifact 的基本元数据	640
39.4 Artifact 类型	640
39.5 Dataset Artifact	641
39.6 Checkpoint Artifact	641
39.7 Checkpoint 元数据	642
39.8 Eval Report Artifact	642
39.9 Deployment Package Artifact	643
39.10 Release Manifest	643
39.11 Artifact 血缘	643
39.12 Artifact Store 和 Metadata Store	644
39.13 Checksum 和完整性校验	644
39.14 Artifact 权限	645
39.15 Artifact 生命周期	645
39.16 Retention Policy	645
39.17 Artifact Promotion	646
39.18 Artifact 和实验追踪的关系	646
39.19 Artifact 和回滚	646
39.20 Artifact 命名和路径规范	647
39.21 Artifact 管理系统架构	647
39.22 Artifact 管理审计指标和最小 demo	647
39.23 常见误区	656
39.24 面试常见追问	657
39.25 小练习	657
39.26 本章小结	657
第 40 章 实验追踪: 参数、指标、日志、样本和代码版本	657
40.0 本讲资料边界与第二轮精修口径	658
40.1 为什么需要实验追踪	658
40.2 实验追踪记录什么	658
40.3 Run 的概念	659
40.4 参数追踪	659
40.5 指标追踪	660
40.6 日志追踪	660
40.7 样本级结果追踪	660
40.8 代码版本追踪	661
40.9 数据版本追踪	661
40.10 模型和 checkpoint 追踪	662
40.11 环境追踪	662
40.12 实验对比	663
40.13 实验搜索和筛选	663
40.14 Tags 和 Notes	664
40.15 大模型实验的特殊性	664
40.16 Prompt 实验追踪	664
40.17 Eval 实验追踪	665
40.18 成本追踪	665

40.19 实验状态管理	665
40.20 实验追踪系统架构	666
40.21 和训练平台的集成	666
40.22 和评估平台的集成	666
40.23 实验追踪审计指标和最小 demo	667
40.24 常见误区	674
40.25 面试常见追问	675
40.26 小练习	675
40.27 本章小结	675
第 41 章 评估平台：离线批量评测、人工评测和在线评测	675
41.0 本讲资料边界与第二轮精修口径	676
41.1 为什么需要评估平台	676
41.2 评估平台的核心目标	676
41.3 评估类型总览	677
41.4 离线批量评测	677
41.5 Eval Dataset 管理	678
41.6 指标定义	678
41.7 LLM-as-Judge	679
41.8 人工评测	680
41.9 人工评测质量控制	680
41.10 Pairwise 评测	681
41.11 在线评测	681
41.12 在线指标	681
41.13 发布门禁	682
41.14 评估报告	682
41.15 Slice Evaluation	683
41.16 回归评测	683
41.17 评估任务调度	684
41.18 评估缓存	684
41.19 评估可观测性	685
41.20 评估平台架构	685
41.21 评估平台审计指标和最小 demo	686
41.22 常见误区	695
41.23 面试常见追问	695
41.24 小练习	696
41.25 本章小结	696
第 42 章 Feature Store、Embedding Store 与向量索引基础设施	696
42.0 本讲资料边界与第二轮精修口径	697
42.1 为什么需要这些基础设施	697
42.2 Feature Store 是什么	697
42.3 什么是特征	698
42.4 Offline Store 和 Online Store	698
42.5 Training-Serving Skew	699
42.6 Point-in-Time Correctness	699
42.7 Embedding Store 是什么	700
42.8 Embedding 为什么要版本化	700
42.9 向量索引是什么	701
42.10 ANN 检索	701
42.11 HNSW 的直觉	702
42.12 IVF 和 PQ 的直觉	702
42.13 向量索引的元数据过滤	702
42.14 Chunking 和 Embedding	703
42.15 Re-indexing	703
42.16 双索引切换	704

42.17 Embedding Pipeline	704
42.18 Retrieval Service	705
42.19 向量检索质量指标	705
42.20 Feature / Embedding 血缘	706
42.21 多租户和权限	706
42.22 Feature Store、Embedding Store 和 RAG	707
42.23 常见系统架构	707
42.24 特征向量基础设施审计指标和最小 demo	707
42.25 常见误区	717
42.26 面试常见追问	717
42.27 小练习	718
42.28 本章小结	718
第 43 章 RAG/Agent 平台中的知识库、工具和 trace 存储	718
43.0 本讲资料边界与第二轮精修口径	718
43.1 为什么需要 RAG/Agent 平台存储	719
43.2 RAG 平台中的核心资产	719
43.3 Knowledge Base 设计	720
43.4 Document 设计	720
43.5 Chunk 设计	721
43.6 文档解析和清洗存储	722
43.7 知识库同步	722
43.8 权限和 ACL	722
43.9 Retrieval Trace	723
43.10 Prompt Assembly Trace	723
43.11 Citation 存储	724
43.12 Agent 平台中的核心资产	724
43.13 Agent Definition	725
43.14 Tool Definition	725
43.15 Tool Permission	726
43.16 Tool Call Trace	726
43.17 Execution Trace	727
43.18 Trace 回放	727
43.19 Memory 存储	728
43.20 Run State 存储	728
43.21 安全与审计	728
43.22 成本追踪	729
43.23 Trace 存储的隐私问题	729
43.24 Trace 采样	730
43.25 RAG/Agent 平台存储架构	730
43.26 RAG/Agent 平台存储审计指标和最小 demo	731
43.27 常见误区	747
43.28 面试常见追问	747
43.29 小练习	748
43.30 本章小结	748
第 44 章 AI Infra 可观测性总览: metrics、logs、traces、events	748
44.0 本讲资料边界与第二轮精修口径	748
44.1 为什么 AI Infra 更需要可观测性	749
44.2 可观测性的四类信号	749
44.3 Metrics 是什么	750
44.4 Logs 是什么	751
44.5 Traces 是什么	751
44.6 Events 是什么	752
44.7 四类信号如何协作	752
44.8 训练可观测性	753

44.9 推理可观测性	753
44.10 数据平台可观测性	754
44.11 模型和实验可观测性	754
44.12 RAG/Agent 可观测性	755
44.13 成本可观测性	756
44.14 统一 ID 体系	756
44.15 标签体系	757
44.16 高基数问题	757
44.17 告警设计	757
44.18 Dashboard 设计	758
44.19 日志脱敏	759
44.20 Trace 采样	759
44.21 可观测性平台架构	759
44.22 AI Infra 可观测性审计指标和最小 demo	760
44.23 常见误区	771
44.24 面试常见追问	771
44.25 小练习	772
44.26 本章小结	772
第 45 章 训练故障定位: loss 异常、hang、OOM、通信慢和 I/O 慢	772
45.0 本讲资料边界与第二轮精修口径	773
45.1 训练故障的常见类型	773
45.2 排查总原则	773
45.3 先看任务生命周期事件	774
45.4 Loss 异常怎么排查	774
45.5 Loss NaN	775
45.6 Loss 不下降	775
45.7 Hang 住怎么排查	776
45.8 OOM 怎么排查	776
45.9 GPU OOM 的常见解决方案	777
45.10 通信慢怎么排查	777
45.11 NCCL 错误	778
45.12 I/O 慢怎么排查	778
45.13 GPU 利用率低	779
45.14 Checkpoint 故障	780
45.15 Resume 后异常	780
45.16 数据问题导致的训练异常	781
45.17 配置变更排查	781
45.18 最小复现	782
45.19 故障定位需要的工具	782
45.20 故障复盘	782
45.21 训练故障定位审计指标和最小 demo	782
45.22 常见误区	794
45.23 面试常见追问	794
45.24 小练习	794
45.25 本章小结	794
第 46 章 推理故障定位: TTFT、TPOT、吞吐下降和错误率上升	795
46.0 本讲资料边界与第二轮精修口径	795
46.1 推理故障的常见类型	795
46.2 推理请求链路回顾	796
46.3 先看故障范围	796
46.4 TTFT 升高怎么排查	797
46.5 TPOT 升高怎么排查	797
46.6 p99 延迟升高	798
46.7 吞吐下降怎么排查	799

46.8 错误率上升怎么排查	799
46.9 Timeout 上升	800
46.10 KV Cache 不足	800
46.11 模型加载失败	801
46.12 Streaming 中断	802
46.13 Cache 命中率下降	802
46.14 路由异常	803
46.15 工具调用失败	803
46.16 质量异常怎么排查	804
46.17 发布后故障	805
46.18 推理故障排查 Runbook	805
46.19 应急处理手段	805
46.20 推理故障定位审计指标和最小 demo	806
46.21 常见误区	818
46.22 面试常见追问	818
46.23 小练习	819
46.24 本章小结	819
第 47 章 SLO、SLA、错误预算和生产值班体系	819
47.0 本讲资料边界与第二轮精修口径	819
47.1 SLI、SLO、SLA 的区别	820
47.2 为什么 AI Infra 需要 SLO	820
47.3 AI Infra 常见 SLI	821
47.4 推理 SLO 示例	821
47.5 训练 SLO 示例	822
47.6 数据平台 SLO 示例	823
47.7 错误预算是什么	823
47.8 错误预算怎么用	824
47.9 Burn Rate	824
47.10 SLO 不能太多	825
47.11 告警和 SLO 的关系	825
47.12 生产值班体系	826
47.13 事故分级	826
47.14 On-call Runbook	827
47.15 推理事故处理流程	827
47.16 训练事故处理流程	828
47.17 值班中的权限	828
47.18 变更管理	828
47.19 事故复盘	829
47.20 错误预算和发布策略	829
47.21 SLO 和成本的关系	830
47.22 SLO 值班体系审计指标和最小 demo	830
47.23 常见误区	841
47.24 面试常见追问	842
47.25 小练习	842
47.26 本章小结	842
第 48 章 成本治理：GPU 成本、存储成本、网络成本和推理成本	842
48.0 本讲资料边界与第二轮精修口径	843
48.1 为什么 AI Infra 成本治理重要	843
48.2 成本治理解决什么问题	843
48.3 成本分类	844
48.4 GPU 成本	844
48.5 GPU 成本指标	845
48.6 训练成本治理	845
48.7 推理成本	846

48.8 推理成本优化	846
48.9 存储成本	847
48.10 存储成本指标	847
48.11 存储成本优化	848
48.12 网络成本	848
48.13 网络成本优化	848
48.14 成本归因	849
48.15 标签和成本中心	850
48.16 预算和配额	850
48.17 成本告警	851
48.18 成本和 SLO 的权衡	851
48.19 Unit Economics	851
48.20 成本 Dashboard	852
48.21 成本治理系统架构	852
48.22 成本治理审计指标和最小 demo	853
48.23 常见误区	863
48.24 面试常见追问	864
48.25 小练习	864
48.26 本章小结	864
第 49 章 资源利用率优化：混部、抢占、弹性训练和低优先级队列	864
49.0 本讲资料边界与第二轮精修口径	865
49.1 为什么资源利用率难	865
49.2 利用率指标	865
49.3 有效利用率	866
49.4 资源碎片	866
49.5 减少碎片的方法	867
49.6 混部是什么	867
49.7 混部风险	868
49.8 安全混部策略	868
49.9 抢占是什么	869
49.10 抢占设计	869
49.11 低优先级队列	869
49.12 弹性训练是什么	870
49.13 弹性训练的难点	870
49.14 弹性训练策略	871
49.15 推理资源利用率优化	871
49.16 Warm Pool 优化	872
49.17 资源预留	872
49.18 配额和公平性	873
49.19 利用率优化和可靠性	873
49.20 调度器能力	874
49.21 利用率 Dashboard	874
49.22 资源利用率治理审计指标和最小 demo	875
49.23 常见误区	887
49.24 面试常见追问	888
49.25 小练习	888
49.26 本章小结	888
第 50 章 安全治理：密钥、权限、数据合规、供应链和模型访问控制	889
50.0 本讲资料边界与第二轮精修口径	889
50.1 为什么 AI Infra 安全更复杂	889
50.2 安全治理范围	889
50.3 身份认证	890
50.4 权限控制	890
50.5 RBAC 和 ABAC	891

50.6 密钥管理	891
50.7 训练任务中的密钥	892
50.8 数据合规	893
50.9 数据分级	893
50.10 PII 和敏感信息	893
50.11 多租户隔离	894
50.12 模型访问控制	895
50.13 模型输出安全	895
50.14 RAG 权限安全	896
50.15 Agent 工具安全	896
50.16 供应链安全	897
50.17 镜像安全	897
50.18 模型 artifact 安全	898
50.19 运行时安全	898
50.20 日志和 Trace 安全	899
50.21 审计	899
50.22 安全事件响应	900
50.23 安全治理系统架构	900
50.24 安全治理审计指标和最小 demo	901
50.25 常见误区	913
50.26 面试常见追问	913
50.27 小练习	913
50.28 本章小结	914
第 51 章 审计、日志留存、事故复盘和变更管理	914
51.0 本讲资料边界与第二轮精修口径	914
51.1 为什么 AI Infra 需要审计	914
51.2 审计和普通日志的区别	915
51.3 审计日志应该记录什么	915
51.4 需要审计的操作	916
51.5 审计日志存储要求	916
51.6 日志留存策略	917
51.7 日志留存的权衡	917
51.8 日志分级	918
51.9 日志脱敏和最小化	918
51.10 Trace 留存	919
51.11 事故复盘的价值	919
51.12 事故复盘模板	920
51.13 时间线很重要	920
51.14 根因分析	921
51.15 行动项管理	921
51.16 变更管理为什么重要	922
51.17 变更记录应包含什么	922
51.18 高风险变更	923
51.19 变更冻结	923
51.20 变更和 SLO	923
51.21 审计、复盘、变更的闭环	924
51.22 系统架构	924
51.23 审计变更治理指标和最小 demo	924
51.24 常见误区	935
51.25 面试常见追问	936
51.26 小练习	936
51.27 本章小结	936
第 52 章 设计一个大模型训练平台	937
52.0 本讲资料边界与第二轮精修口径	937

52.1 题目理解	937
52.2 先做需求澄清	937
52.3 核心目标	938
52.4 核心对象: TrainingJob	938
52.5 TrainingJob 示例	939
52.6 总体架构	939
52.7 提交系统设计	939
52.8 队列和配额	940
52.9 调度系统设计	940
52.10 分布式启动器	941
52.11 数据供给设计	941
52.12 Checkpoint 设计	942
52.13 容错和重试	942
52.14 可观测性设计	942
52.15 实验追踪联动	943
52.16 模型仓库联动	943
52.17 安全和审计	943
52.18 成本治理	944
52.19 Web Console 设计	944
52.20 API / CLI / SDK 设计	944
52.21 状态机设计	944
52.22 训练平台系统设计指标和最小 demo	945
52.23 关键 trade-off	953
52.24 常见故障和应对	954
52.25 面试回答模板	954
52.26 常见扣分点	954
52.27 小练习	955
52.28 本章小结	955
第 53 章 设计一个高并发大模型推理平台	955
53.0 本讲资料边界与第二轮精修口径	955
53.1 题目理解	955
53.2 需求澄清	956
53.3 核心目标	956
53.4 总体架构	956
53.5 API Gateway	957
53.6 多租户鉴权和配额	957
53.7 Model Router	957
53.8 Cache Layer	958
53.9 Admission Controller	958
53.10 Runtime Cluster	958
53.11 Prefill、Decode 和 KV Cache	959
53.12 Continuous Batching	959
53.13 队列调度	959
53.14 自动扩缩容	960
53.15 稳定性保护	960
53.16 灰度发布和回滚	960
53.17 可观测性设计	961
53.18 安全和审计	961
53.19 成本治理	961
53.20 高可用设计	962
53.21 状态和数据模型	962
53.22 推理平台系统设计指标和最小 demo	962
53.23 关键 trade-off	968
53.24 面试回答模板	968
53.25 常见扣分点	968

53.26 小练习	969
53.27 本章小结	969
第 54 章 设计一个企业级 LLMOps 平台	969
54.0 本讲资料边界与第二轮精修口径	969
54.1 题目理解	970
54.2 LLMOps 和 MLOps 的区别	970
54.3 需求澄清	970
54.4 核心目标	970
54.5 总体架构	971
54.6 Model Gateway	971
54.7 Prompt Registry	971
54.8 Prompt 发布流程	972
54.9 Knowledge Base 管理	972
54.10 RAG Pipeline	972
54.11 Tool Registry	973
54.12 Agent Runtime	973
54.13 权限和安全	973
54.14 Eval Platform 集成	974
54.15 Release System	974
54.16 Trace 和可观测性	974
54.17 用户反馈闭环	975
54.18 成本治理	975
54.19 多环境管理	975
54.20 Web Console 设计	976
54.21 API / SDK / CLI	976
54.22 核心数据模型	976
54.23 LLMOps 平台系统设计指标和最小 demo	977
54.24 关键 trade-off	982
54.25 面试回答模板	983
54.26 常见扣分点	983
54.27 小练习	983
54.28 本章小结	983
第 55 章 设计一个多租户 GPU 集群调度系统	984
55.0 本讲资料边界与第二轮精修口径	984
55.1 题目理解	984
55.2 需求澄清	984
55.3 核心目标	985
55.4 总体架构	985
55.5 资源模型	985
55.6 任务模型	986
55.7 多租户队列	986
55.8 配额设计	986
55.9 公平性策略	987
55.10 Gang Scheduling	987
55.11 拓扑感知调度	987
55.12 Placement 策略	988
55.13 资源碎片治理	988
55.14 抢占机制	988
55.15 低优先级队列	989
55.16 推理服务调度	989
55.17 异构 GPU 调度	989
55.18 调度事件和可观测性	989
55.19 成本和利用率	990
55.20 与 Kubernetes 的关系	990

55.21 多租户 GPU 调度系统设计指标和最小 demo	990
55.22 关键 trade-off	995
55.23 面试回答模板	996
55.24 常见扣分点	996
55.25 小练习	996
55.26 本章小结	997
第 56 章 设计一个大模型评估与实验平台	997
56.0 本讲资料边界与第二轮精修口径	997
56.1 题目理解	997
56.2 需求澄清	998
56.3 核心目标	998
56.4 总体架构	998
56.5 Eval Dataset Registry	998
56.6 Eval Suite	999
56.7 Eval Job	999
56.8 Inference Runner	999
56.9 Metric Engine	1000
56.10 LLM-as-Judge 服务	1000
56.11 Human Review System	1000
56.12 Sample-level Result Store	1001
56.13 Slice Evaluation	1001
56.14 Experiment Tracker 集成	1001
56.15 Eval Report	1002
56.16 Quality Gate	1002
56.17 Online A/B 测试	1002
56.18 评估缓存	1003
56.19 权限和安全	1003
56.20 成本治理	1003
56.21 可观测性	1003
56.22 核心数据模型	1004
56.23 评估实验平台系统设计指标和最小 demo	1004
56.24 关键 trade-off	1008
56.25 面试回答模板	1008
56.26 常见扣分点	1009
56.27 小练习	1009
56.28 本章小结	1009
第 57 章 设计一个模型仓库与发布系统	1010
57.0 本讲资料边界与第二轮精修口径	1010
57.1 题目理解	1010
57.2 需求澄清	1010
57.3 核心目标	1011
57.4 总体架构	1011
57.5 核心对象	1011
57.6 Model 和 ModelVersion	1011
57.7 ModelVersion 元数据	1012
57.8 Artifact Store	1012
57.9 Artifact 校验	1013
57.10 模型血缘	1013
57.11 Adapter 和 Merge 管理	1013
57.12 量化版本管理	1013
57.13 发布状态机	1014
57.14 Quality Gate	1014
57.15 审批流程	1014
57.16 Release Manifest	1015

57.17 灰度发布	1015
57.18 A/B 测试	1015
57.19 回滚设计	1016
57.20 Deployment Controller	1016
57.21 权限控制	1016
57.22 安全和审计	1016
57.23 生命周期管理	1017
57.24 可观测性	1017
57.25 模型仓库与发布系统设计指标和最小 demo	1017
57.26 核心 trade-off	1022
57.27 面试回答模板	1022
57.28 常见扣分点	1023
57.29 小练习	1023
57.30 本章小结	1023
第 58 章 设计一个 AI Infra 可观测性平台	1023
58.0 本讲资料边界与第二轮精修口径	1024
58.1 题目理解	1024
58.2 需求澄清	1024
58.3 核心目标	1024
58.4 总体架构	1025
58.5 Metrics Pipeline	1025
58.6 Logs Pipeline	1026
58.7 Traces Pipeline	1026
58.8 Events Pipeline	1026
58.9 Cost Pipeline	1027
58.10 统一 ID 体系	1027
58.11 Correlation Layer	1028
58.12 训练可观测性设计	1028
58.13 推理可观测性设计	1028
58.14 RAG/Agent 可观测性设计	1028
58.15 数据和评估可观测性	1029
58.16 告警系统	1029
58.17 SLO 和错误预算	1029
58.18 Dashboard 设计	1030
58.19 隐私和权限	1030
58.20 高基数控制	1031
58.21 数据留存和成本	1031
58.22 Incident 集成	1031
58.23 变更关联	1031
58.24 AI Infra 可观测性平台系统设计指标和最小 demo	1032
58.25 核心 trade-off	1036
58.26 面试回答模板	1037
58.27 常见扣分点	1037
58.28 小练习	1037
58.29 本章小结	1037
第 59 章 AI Infra 面试高频问题与标准回答	1038
59.0 本讲资料边界与第二轮精修口径	1038
59.1 AI Infra 是什么	1039
59.2 训练平台怎么设计	1039
59.3 为什么 TrainingJob 不是普通 K8s Job	1039
59.4 为什么需要 Gang Scheduling	1039
59.5 GPU 集群调度怎么设计	1039
59.6 GPU 利用率低怎么排查	1040
59.7 Loss NaN 怎么排查	1040

59.8 推理平台怎么设计	1040
59.9 为什么不能只看 QPS	1040
59.10 TTFT 和 TPOT	1040
59.11 KV Cache 为什么重要	1041
59.12 Continuous Batching	1041
59.13 模型路由和负载均衡	1041
59.14 自动扩缩容怎么设计	1041
59.15 RAG 平台怎么设计	1041
59.16 Agent 平台怎么设计	1041
59.17 模型仓库怎么设计	1042
59.18 评估平台怎么设计	1042
59.19 可观测性平台怎么设计	1042
59.20 成本治理怎么做	1042
59.21 安全治理怎么做	1042
59.22 多租户隔离怎么做	1043
59.23 训练故障定位	1043
59.24 推理故障定位	1043
59.25 SLO 和错误预算	1043
59.26 回答系统设计题的结构	1043
59.27 面试常用公式速记	1044
59.28 AI Infra 面试准备度指标和最小 demo	1044
59.29 高频扣分点总结	1049
59.30 小练习	1049
59.31 本章小结	1049
第 60 章 AI Infra 未来趋势：Serverless GPU、异构算力、边缘推理和自治运维	1050
60.0 本讲资料边界与第二轮精修口径	1050
60.1 为什么要看趋势	1050
60.2 趋势一：Serverless GPU	1050
60.3 Serverless GPU 的难点	1051
60.4 Serverless 推理	1051
60.5 趋势二：异构算力	1051
60.6 异构算力调度	1052
60.7 Runtime 生态会继续分化	1052
60.8 趋势三：边缘推理	1052
60.9 云边协同	1052
60.10 趋势四：推理和训练更加一体化	1053
60.11 趋势五：更强的评估与质量治理	1053
60.12 趋势六：Agent Infra 成为新基础设施	1053
60.13 趋势七：自治运维	1054
60.14 AIOps 在 AI Infra 中的特殊性	1054
60.15 趋势八：成本成为一等公民	1054
60.16 趋势九：安全和合规内建	1054
60.17 趋势十：标准化和互操作	1055
60.18 AI Infra 工程师未来能力模型	1055
60.19 未来趋势判断指标和最小 demo	1055
60.20 面试开放题回答思路	1064
60.21 常见误区	1064
60.22 给读者的最后建议	1065
60.23 第二十三册总结	1065
60.24 本章小结	1065

第二十三册简介：AI Infra、大模型基础设施与平台工程

本书定位

第二十三册专门讲 AI Infra，也就是支撑大模型训练、推理、评估、数据、Agent 和多模态系统运行的基础设施与平台工程。

如果说第五册讲训练算法和训练流程，第六册讲推理优化和部署，第十一册讲系统设计面试，那么本书聚焦更底层、更平台化的问题：GPU 集群怎么建设，训练任务怎么调度，推理平台怎么承载高并发，数据和 checkpoint 怎么存储，监控告警怎么设计，成本怎么控制，故障怎么排查。

为什么需要这本书

大模型能力不是只由模型结构和算法决定，也强依赖基础设施。

同样的模型和数据，在不同基础设施上可能出现完全不同的训练效率、稳定性、成本和上线体验。真实团队里，AI Infra 决定了模型能不能训得动、推得快、服务稳、成本可控、事故可排查。

对于算法岗候选人来说，不需要变成纯 SRE 或云原生专家，但必须理解 AI Infra 的核心约束和 trade-off。顶级大模型算法岗面试中，很多系统设计、训练 debug、推理优化和项目深挖问题，本质都离不开 AI Infra。

适合读者

1. 想从算法视角理解 GPU 集群、训练平台和推理平台的候选人。
2. 想准备大模型系统设计、训练平台、推理平台和 MLOps 面试的工程师。
3. 想理解为什么训练慢、任务排队、GPU 利用率低、推理延迟高和成本失控的学习者。
4. 想从模型算法扩展到平台工程、AI Infra 或大模型工程架构方向的人。

学完后应具备的能力

1. 能解释 AI Infra 的核心组成：计算、存储、网络、调度、数据、平台、监控和安全。
2. 能分析 GPU 利用率、显存、通信、I/O、checkpoint 和任务调度对训练效率的影响。
3. 能设计一个基础的大模型训练平台，覆盖资源申请、任务提交、容错、日志、监控和 checkpoint。
4. 能设计一个推理服务平台，覆盖模型路由、自动扩缩容、batching、缓存、限流和灰度发布。
5. 能建立可观测性体系，定位训练卡死、节点故障、网络抖动、推理延迟和服务错误。
6. 能从成本、可靠性、安全和团队协作角度评价 AI Infra 方案。

内容结构

1. AI Infra 总览与硬件基础。
2. GPU 集群、网络、存储和调度。
3. 大模型训练平台工程。
4. 大模型推理平台工程。
5. 数据、特征、artifact 和实验平台。
6. 可观测性、可靠性、安全与成本治理。
7. AI Infra 系统设计面试。

阅读建议

建议先学习第一册、第五册和第六册，理解训练与推理主线后再读本书。本书也适合和第十一册系统设计、第十九册实战宝典、第二十册 Agent Harness 一起学习。

阅读时要特别关注：每个基础设施组件解决什么瓶颈，它带来什么成本，出了问题如何定位，以及面试中如何把方案讲成可运行、可扩展、可观测、可治理的系统。

与其他书的关系

第二十三册是全书系的基础设施底座。

1. 第五册讲训练流程，本书讲支撑训练流程的平台和集群。
2. 第六册讲推理优化，本书讲推理平台、服务治理和生产运维。
3. 第七册讲评估方法，本书讲评估平台、实验追踪和批量评测基础设施。
4. 第九册讲数据工程，本书讲数据湖、对象存储、数据版本和训练数据供给链路。
5. 第十一册讲系统设计面试，本书提供更底层的 AI Infra 系统设计素材。
6. 第十九册讲实战事故，本书系统整理事故背后的平台层原因。

第二十三册：AI Infra、大模型基础设施与平台工程

本书定位

本书是“大模型算法岗系统面试二十四部曲”的第二十三册，专门讲 AI Infra、大模型基础设施与平台工程。

AI Infra 是连接算法、工程和生产环境的底座。它覆盖 GPU/加速器集群、网络、存储、调度、训练平台、推理平台、数据平台、实验平台、可观测性、成本治理、安全治理和系统设计。

本书重点回答：

1. AI Infra 到底包含哪些模块，和 MLOps、LLMOps、云原生有什么关系。
2. GPU 集群、网络、存储和调度如何影响训练效率和稳定性。
3. 如何设计大模型训练平台，支持任务提交、资源隔离、容错、checkpoint、日志和监控。
4. 如何设计大模型推理平台，支持高并发、低延迟、高吞吐、灰度发布和成本控制。
5. 如何建设数据、模型 artifact、实验追踪和评估平台。
6. 如何做 AI Infra 可观测性、故障定位、SLO、容量规划和成本优化。
7. 面试中如何系统设计一个 AI Infra 平台，而不是只说 Kubernetes 或 GPU。

阅读入口

1. [简介.md](#)：本书定位、适合读者、阅读建议和与其他书的关系。
2. 本文件：全书章节目录和阅读顺序。

适合读者

1. 已经理解大模型训练和推理，但想补齐平台工程视角的读者。
2. 想准备 AI Infra、训练平台、推理平台、MLOps、LLMOps 和系统设计面试的候选人。
3. 想理解 GPU 利用率、任务调度、网络通信、checkpoint、推理延迟和成本优化的工程师。
4. 想从算法工程师成长为能负责完整大模型系统的技术负责人。

章节目录

第一部分：AI Infra 总览与硬件基础

1. [AI Infra 总览：从模型 API 到基础设施底座](#)
2. [AI Infra、MLOps、LLMOps、Platform Engineering 的边界](#)
3. [GPU、NPU、TPU 与 AI 加速器基础](#)
4. [显存、HBM、PCIe、NVLink、NVSwitch 与带宽瓶颈](#)
5. [GPU 利用率、MFU、HFU 与训练效率指标](#)
6. [大模型任务画像：预训练、SFT、RLHF、评估、推理和 Agent](#)

第二部分：集群、网络、存储与调度

7. [GPU 集群架构：单机多卡、多机多卡和机柜级互联](#)
8. [网络基础：InfiniBand、RoCE、以太网和集合通信](#)

9. 存储体系：本地盘、共享文件系统、对象存储和数据湖
10. Checkpoint 存储、加载、恢复和生命周期管理
11. 容器化基础：Docker、镜像、驱动、CUDA 和依赖环境
12. Kubernetes 与 GPU 资源管理
13. 训练任务调度：队列、优先级、抢占、公平性和配额
14. 多租户隔离：资源隔离、权限隔离和数据隔离
15. 集群容量规划：GPU 数量、网络带宽、存储吞吐和利用率

第三部分：大模型训练平台工程

16. 训练平台总览：从脚本训练到平台化训练
17. 训练任务提交系统：配置、镜像、代码、数据和启动命令
18. 分布式训练启动器：torchrun、DeepSpeed、Megatron 和 Ray
19. 训练配置管理：超参、版本、环境和可复现性
20. 日志、指标、事件和训练状态追踪
21. Checkpoint 策略：频率、保留、异步保存和断点恢复
22. 训练容错：节点失败、通信失败、OOM 和自动重试
23. 训练数据供给：流式读取、缓存、shuffle 和数据局部性
24. 训练平台权限、安全和审计
25. 训练平台系统设计面试题

第四部分：大模型推理平台工程

26. 推理平台总览：从单模型服务到多模型平台
27. 模型服务运行时：Triton、vLLM、TGI、SGLang 与自研 runtime
28. Prefill、Decode、KV Cache 与推理资源画像
29. Continuous Batching、PagedAttention 和队列调度
30. 模型路由：模型选择、能力路由、成本路由和降级路由
31. 缓存体系：prompt cache、KV cache、语义缓存和结果缓存
32. 自动扩缩容：QPS、延迟、队列长度和 GPU 利用率
33. 限流、熔断、重试、超时和降级
34. 灰度发布、A/B 测试、回滚和模型版本管理
35. 推理平台系统设计面试题

第五部分：数据、模型与实验平台

36. 数据平台与训练数据供给链路
37. 数据版本管理、数据血缘和数据质量监控
38. 模型仓库：权重、tokenizer、配置、adapter 和量化版本
39. Artifact 管理：dataset、checkpoint、eval report 和 deployment package
40. 实验追踪：参数、指标、日志、样本和代码版本
41. 评估平台：离线批量评测、人工评测和在线评测
42. Feature Store、Embedding Store 与向量索引基础设施
43. RAG/Agent 平台中的知识库、工具和 trace 存储

第六部分：可观测性、可靠性与成本治理

44. AI Infra 可观测性总览：metrics、logs、traces、events
45. 训练故障定位：loss 异常、hang、OOM、通信慢和 I/O 慢
46. 推理故障定位：TTFT、TPOT、吞吐下降和错误率上升
47. SLO、SLA、错误预算和生产值班体系
48. 成本治理：GPU 成本、存储成本、网络成本和推理成本
49. 资源利用率优化：混部、抢占、弹性训练和低优先级队列
50. 安全治理：密钥、权限、数据合规、供应链和模型访问控制
51. 审计、日志留存、事故复盘和变更管理

第七部分：AI Infra 系统设计与开放题

52. [设计一个大模型训练平台](#)
53. [设计一个高并发大模型推理平台](#)
54. [设计一个企业级 LLMOps 平台](#)
55. [设计一个多租户 GPU 集群调度系统](#)
56. [设计一个大模型评估与实验平台](#)
57. [设计一个模型仓库与发布系统](#)
58. [设计一个 AI Infra 可观测性平台](#)
59. [AI Infra 面试高频问题与标准回答](#)
60. [AI Infra 未来趋势：Serverless GPU、异构算力、边缘推理和自治运维](#)

与其他书的关系

1. 第五册讲大模型训练流程，本书讲训练平台、集群、调度和容错。
2. 第六册讲推理优化与部署，本书讲推理平台、多模型服务、弹性扩缩容和生产治理。
3. 第七册讲评估实验方法，本书讲评估平台和实验追踪基础设施。
4. 第九册讲数据工程，本书讲数据平台、对象存储、数据版本和训练数据供给。
5. 第十一册讲系统设计面试，本书补齐 AI Infra 系统设计素材。
6. 第十九册讲实战坑，本书讲这些坑背后的基础设施原因和平台化防线。
7. 第二十册讲 Agent Harness，本书讲支撑 Agent 服务运行的底层平台能力。

第一批优先扩写章节

1. 第 1 章：AI Infra 总览。
2. 第 5 章：GPU 利用率、MFU、HFU 与训练效率指标。
3. 第 13 章：训练任务调度。
4. 第 16 章：训练平台总览。
5. 第 21 章：Checkpoint 策略。
6. 第 26 章：推理平台总览。
7. 第 29 章：Continuous Batching、PagedAttention 和队列调度。
8. 第 44 章：AI Infra 可观测性总览。
9. 第 48 章：成本治理。
10. 第 52 章：设计一个大模型训练平台。

第 1 章 AI Infra 总览：从模型 API 到基础设施底座

很多人第一次接触大模型时，看到的是一个很简单的接口：输入 prompt，调用模型 API，得到回答。

但在真实公司里，模型 API 背后不是一个黑盒，而是一整套基础设施：GPU 集群、网络、存储、调度系统、训练平台、推理平台、数据平台、模型仓库、评估平台、监控告警、权限审计、成本治理和事故处理。

这些东西合起来，就是 AI Infra。

先记住一句话：

AI Infra 是支撑大模型从数据、训练、评估、部署到在线服务稳定运行的基础设施和平台工程体系。

如果说算法决定“模型能不能变聪明”，那么 AI Infra 决定“模型能不能训得动、推得快、跑得稳、成本可控、出了问题能定位”。

1.0 本讲资料边界与第二轮精修口径

本讲是第二十三册入口章，目标不是把 Kubernetes、Slurm、DCGM、OpenTelemetry、MLflow 或某一家云厂商平台逐一讲成教程，而是建立 AI Infra 的分层地图和面试表达框架。

第二轮精修时，我按 WRITING_PLAN.md 做了资料校准，主要参考公开官方资料中的稳定边界：Kubernetes device plugin 说明了加速器等厂商设备如何暴露给集群调度；Slurm 是 HPC 场景常见的作业调度和资源管理系统；

NVIDIA DCGM 代表 GPU 遥测、健康检查和诊断能力；OpenTelemetry 把 metrics、logs、traces 作为可观测性信号；MLflow Tracking 代表实验记录、参数、指标和 artifact 追踪的一类平台能力。

这些资料只用于校准 “AI Infra 应覆盖哪些层”，不能反过来推出 “AI Infra 等于 Kubernetes” 或 “AI Infra 等于某个实验平台”。本讲新增的公式和 demo 也只做教学审计：帮助你把模块清单转成可检查的指标，而不是替代真实公司的容量规划、调度器实现、SRE 体系或安全合规系统。

1.1 一个简单 API 背后的复杂系统

用户看到的可能只是：

`POST /v1/chat/completions`

但请求背后可能经过：

1. API Gateway 做认证、限流和路由。
2. 模型路由器选择合适模型和版本。
3. 推理调度器把请求放入队列。
4. 推理引擎做 prefill、decode 和 batching。
5. GPU 执行 attention、MLP、采样和 KV cache 读写。
6. 监控系统记录 TTFT、TPOT、错误率、GPU 利用率。
7. 日志系统记录请求、trace 和异常。
8. 成本系统统计 token、GPU 时间和租户账单。
9. 安全系统检查权限、敏感词、数据留存策略。

训练一个模型更复杂。一个预训练任务可能涉及：

1. 数据从对象存储或数据湖读取。
2. 数据清洗、去重、分片和 shuffle。
3. 训练任务提交到集群调度系统。
4. 容器拉取镜像和依赖环境。
5. 多机多卡建立通信组。
6. GPU 计算 forward 和 backward。
7. 网络传输梯度、参数或激活。
8. checkpoint 定期保存到存储。
9. 监控系统采集 loss、吞吐、显存、网络和 I/O。
10. 节点失败后恢复训练。

所以，大模型不是只运行在代码里，而是运行在基础设施里。

1.2 AI Infra 包含哪些模块

AI Infra 可以粗略分成十个模块。

1.2.1 计算基础设施

包括 GPU、NPU、TPU、CPU、内存、显存、HBM、PCIe、NVLink、NVSwitch 等。

它决定：

1. 单步训练能放多大 batch。
2. 推理能承载多少并发。
3. 模型能否放进显存。
4. 训练和推理的理论上限。

面试里常见问题是：为什么 GPU 利用率低？为什么显存够但吞吐上不去？为什么多机训练比单机慢很多？这些都和计算基础设施有关。

1.2.2 网络基础设施

包括 InfiniBand、RoCE、以太网、交换机、网卡、拓扑、RDMA 和集合通信。

它决定：

1. 多机训练的通信效率。
2. AllReduce、AllGather、ReduceScatter 的耗时。
3. 参数并行和 pipeline 并行的稳定性。
4. checkpoint、数据加载和分布式推理的传输瓶颈。

大模型训练不是 GPU 越多越快。如果网络跟不上，GPU 会互相等待，整体效率会很差。

1.2.3 存储基础设施

包括本地 SSD、共享文件系统、对象存储、数据湖、缓存层和 checkpoint 存储。

它决定：

1. 训练数据能否稳定供给。
2. checkpoint 保存和恢复速度。
3. 数据集版本是否可追踪。
4. 多任务并发读取是否会打爆存储。

很多训练慢，不是模型慢，而是数据读不动。

1.2.4 调度与资源管理

包括 Kubernetes、Slurm、Ray、队列系统、配额、优先级、抢占、公平调度和 gang scheduling。

它决定：

1. 谁能使用 GPU。
2. 任务什么时候启动。
3. 多租户如何公平共享资源。
4. 训练任务失败后如何恢复。
5. 低优先级任务如何利用碎片资源。

AI Infra 里的调度比普通 Web 服务更难，因为大模型训练通常需要一组 GPU 同时到位，不能只给一部分资源就启动。

1.2.5 训练平台

训练平台把“写脚本跑训练”变成“可提交、可复现、可监控、可恢复”的平台能力。

它包括：

1. 任务提交。
2. 镜像和环境管理。
3. 代码版本管理。
4. 数据版本管理。
5. 超参配置。
6. 分布式启动器。
7. 日志和指标。
8. checkpoint 策略。
9. 自动重试和容错。
10. 权限和审计。

训练平台的目标不是让训练脚本消失，而是让训练过程可管理。

1.2.6 推理平台

推理平台负责把模型稳定服务给线上用户。

它包括：

1. 模型加载和运行时。
2. Continuous batching。
3. KV cache 管理。
4. 模型路由。
5. 自动扩缩容。
6. 限流、熔断、超时和降级。
7. 灰度发布和回滚。
8. 多模型服务。
9. 监控和告警。
10. 成本统计。

训练关注吞吐和稳定收敛，推理关注延迟、并发、吞吐、可用性和成本。

1.2.7 数据平台

数据平台负责数据采集、清洗、存储、版本、血缘、质量监控和访问控制。

在大模型场景里，数据平台尤其重要，因为数据规模大、来源杂、质量差异大，而且训练结果高度依赖数据。

它需要回答：

1. 这批数据从哪里来？
2. 有没有去重？
3. 是否包含敏感信息？
4. 使用了哪个版本？
5. 训练集和评估集是否泄漏？
6. 数据质量变化是否影响模型表现？

1.2.8 模型与 Artifact 平台

Artifact 是训练和部署过程中的产物。

包括：

1. 模型权重。
2. tokenizer。
3. 配置文件。
4. checkpoint。
5. adapter。
6. 量化版本。
7. eval report。
8. deployment package。

模型仓库不是简单网盘。它要管理版本、血缘、权限、签名、兼容性、发布状态和回滚。

1.2.9 评估与实验平台

评估平台负责离线评测、人工评测、在线 A/B 测试和回归测试。

实验平台负责记录：

1. 使用了什么代码。
2. 使用了什么数据。
3. 使用了什么模型初始化。
4. 使用了什么超参。

5. 得到了什么指标。
6. 产生了什么 artifact。

没有实验追踪，团队很快会陷入“这次为什么变好了”“上次那个模型是怎么训的”这种混乱。

1.2.10 可观测性、安全和成本治理

可观测性包括 metrics、logs、traces、events 和 profiles。

安全包括身份、权限、密钥、数据隔离、镜像安全、供应链安全和审计。

成本治理包括 GPU 成本、存储成本、网络成本、推理 token 成本和闲置资源成本。

AI Infra 的生产化水平，很大程度体现在这些“看起来不酷但非常关键”的能力上。

1.3 AI Infra 和 MLOps、LLMOps、Platform Engineering 的关系

这些词经常混在一起，需要区分。

1.3.1 MLOps

MLOps 关注机器学习模型从开发到上线的生命周期管理。

典型内容包括：

1. 数据版本。
2. 特征管理。
3. 训练流水线。
4. 模型注册。
5. 部署发布。
6. 监控回滚。

MLOps 更早服务于传统机器学习和深度学习。

1.3.2 LLMOps

LLMOps 是面向大模型的生命周期管理。

相比 MLOps，它更强调：

1. Prompt 管理。
2. RAG 知识库。
3. 大模型评测。
4. Agent trace。
5. 工具调用日志。
6. 模型供应商管理。
7. token 成本。
8. 安全和内容治理。

LLMOps 更贴近大模型应用层。

1.3.3 Platform Engineering

Platform Engineering 关注为内部开发者提供自助式平台。

它的目标是让算法工程师、应用工程师和业务团队不用直接面对底层复杂性。

例如：

1. 自助提交训练任务。
2. 自助部署模型服务。
3. 自助查看指标和日志。

4. 自助申请数据和权限。
5. 自助创建评测任务。

1.3.4 AI Infra

AI Infra 更底层，也更综合。

它覆盖硬件、集群、网络、存储、调度、平台、治理和运维。

可以这样理解：

AI Infra 是底座。

ML0ps / LLM0ps 是生命周期管理方法和平台能力。

Platform Engineering 是把这些能力产品化给内部用户使用的工程方式。

1.4 为什么算法岗也要懂 AI Infra

很多算法同学会问：我又不是 SRE，为什么要懂 GPU 集群、调度和存储？

原因很现实。

1.4.1 训练问题经常不是算法问题

例如：

1. loss 突然变 NaN，可能是混合精度、数据异常或通信问题。
2. 训练吞吐下降，可能是数据加载慢、网络拥塞或 checkpoint 阻塞。
3. 多机扩展效率差，可能是并行策略和网络拓扑不匹配。
4. 任务经常失败，可能是节点稳定性、驱动版本或容器环境问题。

如果完全不懂 Infra，很容易把所有问题都误判为模型或代码问题。

1.4.2 推理效果和推理系统有关

同一个模型，不同推理平台可能表现完全不同：

1. batching 策略影响延迟和吞吐。
2. KV cache 策略影响并发和显存。
3. 模型路由影响成本和质量。
4. 超时策略影响用户体验。
5. 降级策略影响可用性。

算法岗如果不了解推理平台，就很难设计可落地的模型方案。

1.4.3 成本是模型方案的一部分

大模型项目里，成本不是财务部门的问题，而是技术方案的一部分。

一个模型如果效果提升 1%，但推理成本增加 5 倍，未必值得上线。

一个训练方案如果理论上更好，但需要占用整个集群两周，也可能不现实。

优秀算法工程师需要能回答：这个方案效果、成本、延迟和资源占用之间如何权衡。

1.5 AI Infra 的核心矛盾

AI Infra 不是堆机器，而是在多个目标之间做 trade-off。

1.5.1 性能与成本

更大的 GPU 集群、更快的网络、更高性能的存储都能提升效率，但成本很高。

问题是：哪些瓶颈值得花钱解决？哪些可以通过软件优化解决？

例如：

1. 如果训练瓶颈是数据读取，买更多 GPU 没用。
2. 如果推理瓶颈是 KV cache 显存，增加 CPU 不解决问题。
3. 如果网络慢，模型并行策略可能需要调整。

1.5.2 利用率与隔离性

为了提高资源利用率，平台希望混部、抢占和共享资源。

为了保证稳定性，平台又需要隔离、配额和优先级。

两者天然冲突。

例如低优先级训练任务可以使用空闲 GPU，但高优先级任务来了以后是否抢占？抢占后 checkpoint 如何恢复？这些都是调度系统要解决的问题。

1.5.3 灵活性与标准化

算法团队希望自由选择框架、镜像、依赖、启动方式和实验配置。

平台团队希望标准化，方便运维、监控、安全和复现。

好的平台不是禁止灵活性，而是在关键边界上标准化：任务描述、资源申请、日志格式、checkpoint 位置、指标上报和权限模型。

1.5.4 速度与治理

研发希望快速试验和上线。

企业需要权限、审计、合规、成本控制和变更管理。

AI Infra 的成熟度体现在：在不拖慢研发太多的情况下，把治理嵌入平台默认流程。

1.6 一个完整 AI Infra 视图

可以用下面的图理解：

Users / Teams

- > Portal / CLI / SDK
- > Training Platform
- > Inference Platform
- > Data Platform
- > Eval Platform
- > Model Registry / Artifact Store
- > Scheduler / Resource Manager
- > GPU Cluster / Network / Storage
- > Observability / Security / Cost Governance

从上往下看：用户通过平台入口提交任务。

从下往上看：底层硬件资源被抽象成训练、推理、数据和评估能力。

横向看：监控、安全和成本治理贯穿所有层。

1.7 面试中如何讲 AI Infra 总览

如果面试官问：

你怎么理解 AI Infra？它包含哪些模块？

可以这样回答：

我理解 AI Infra 是支撑大模型训练、推理、评估和应用运行的基础设施与平台工程体系。

底层包括 GPU/加速器集群、网络、存储和调度系统；中间包括训练平台、推理平台、数据平台、模型仓库、实验追踪和评估平台；上

它解决的问题不是单纯把模型跑起来，而是让模型训得动、推得快、服务稳、可复现、可观测、可扩展、成本可控。

如果面试官继续追问：

AI Infra 和 MLOps / LLMops 有什么区别？

可以回答：

AI Infra 更偏底座和平台，覆盖硬件、集群、网络、存储、调度、训练和推理服务。MLOps / LLMops 更偏模型生命周期管理，包括数

1.8 AI Infra 总览审计指标与最小 demo

总览章节最容易写成“列一堆模块名”。第二轮精修里，更推荐把它转成一组可审计问题：这个平台是否真的覆盖了训练、推理、数据、评估、可观测性、安全、成本和开发者体验？它是否把 AI Infra、MLOps、LLMops 和 Platform Engineering 的边界讲清楚？算法团队和平台团队是否能用同一张指标表沟通？

先定义一个 AI Infra 总览审计样本：

$x_i = (c_i, n_i, s_i, q_i, t_i, r_i, d_i, a_i, e_i, o_i, g_i, p_i, z_i)$

其中， c_i 表示计算和加速器准备度， n_i 表示网络和通信准备度， s_i 表示存储、数据供给和 checkpoint 准备度， q_i 表示调度和资源治理， t_i 表示训练平台可复现性， r_i 表示推理平台 SLO 能力， d_i 表示数据平台血缘， a_i 表示模型与 artifact 治理， e_i 表示评估和实验追踪， o_i 表示可观测性， g_i 表示安全治理， p_i 表示成本和容量治理， z_i 表示边界、协作和开发者自助能力等标签。

对第 j 个审计维度，统一写成通过率：

$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(x_i)=1]$

其中， N 是审计样本数， $g_j(x_i)=1$ 表示样本 x_i 通过第 j 个检查。

常见资源效率可以用 GPU 忙碌时间占比做直觉起点：

$U_{\mathrm{gpu}} = \frac{T_{\mathrm{busy}}}{T_{\mathrm{wall}}}$

但面试中要强调：GPU utilization 只是粗粒度忙碌指标，不能单独证明训练有效率。还要结合 MFU、tokens/sec、step time、通信时间、数据加载时间、checkpoint 阻塞和失败率。

成本治理可以先用单位成功任务成本表达：

$K_{\mathrm{success}} = \frac{K_{\mathrm{gpu}} + K_{\mathrm{storage}} + K_{\mathrm{network}} + K_{\mathrm{ops}}}{N_{\mathrm{success}}}$

其中， K_{gpu} 、 K_{storage} 、 K_{network} 和 K_{ops} 分别代表 GPU、存储、网络和运维成本， N_{success} 是通过质量门禁的成功训练、评估或推理任务数。这个公式的重点不是精确财务建模，而是提醒你：只算 GPU 小时或 token 单价，会低估 AI Infra 的真实成本。

最后给一个总览门禁：

$G_{\mathrm{ai_infra}} = \mathbf{1}[\min_j C_j \geq \tau_j \wedge R_{p0} = 0]$

其中， τ_j 是每个维度的最低通过阈值， R_{p0} 表示 P0 级硬阻断数量。只要关键维度覆盖不足，或者存在没有缓解方案的 P0 风险，总览门禁就不应该通过。

下面是一个 0 依赖 Python demo。它不模拟真实集群，只把“AI Infra 总览回答是否完整”变成一张 toy 审计表：

```

METRICS = [
    "compute_accelerator_readiness",
    "network_communication_readiness",
    "storage_data_checkpoint_readiness",
    "scheduler_resource_governance",
    "training_platform_reproducibility",
    "inference_platform_slo_readiness",
    "data_platform_lineage_quality",
    "model_artifact_registry_governance",
    "eval_experiment_tracking_coverage",
    "observability_signal_coverage",
    "security_governance_coverage",
    "cost_capacity_governance",
    "developer_self_service_readiness",
    "ai_infra_boundary_clarity",
    "mlops_llmops_platform_boundary_clarity",
    "algorithm_infra_collaboration_readiness",
]

def make_case(name, failed_metric=None, p0=False):
    flags = {metric: True for metric in METRICS}
    if failed_metric is not None:
        flags[failed_metric] = False
    return {"name": name, "flags": flags, "p0": p0}

def build_cases():
    bad_cases = [
        ("gpu_capacity_missing_bad", "compute_accelerator_readiness"),
        ("network_ignored_bad", "network_communication_readiness"),
        ("storage_checkpoint_missing_bad", "storage_data_checkpoint_readiness"),
        ("scheduler_no_quota_bad", "scheduler_resource_governance"),
        ("training_not_reproducible_bad", "training_platform_reproducibility"),
        ("inference_no_slo_bad", "inference_platform_slo_readiness"),
        ("data_lineage_missing_bad", "data_platform_lineage_quality"),
        ("artifact_registry_missing_bad", "model_artifact_registry_governance"),
        ("eval_tracking_missing_bad", "eval_experiment_tracking_coverage"),
        ("observability_missing_bad", "observability_signal_coverage"),
        ("security_audit_missing_bad", "security_governance_coverage"),
        ("cost_capacity_missing_bad", "cost_capacity_governance"),
        ("no_self_service_bad", "developer_self_service_readiness"),
        ("kubernetes_only_bad", "ai_infra_boundary_clarity"),
        ("boundary_confused_bad", "mlops_llmops_platform_boundary_clarity"),
        ("algorithm_infra_silo_bad", "algorithm_infra_collaboration_readiness"),
    ]

    cases = [make_case("complete_ai_infra_stack")]
    cases.extend(make_case(name, metric, p0=True) for name, metric in bad_cases)
    return cases

def audit_ai_infra_overview(cases, threshold=0.95):
    metrics = {}
    for metric in METRICS:

```

```

    passed = sum(1 for case in cases if case["flags"][metric])
    metrics[metric] = round(passed / len(cases), 3)

failed_cases = [
    case["name"]
    for case in cases
    if case["p0"] or any(not case["flags"][metric] for metric in METRICS)
]
failed_gates = [
    metric for metric, score in metrics.items() if score < threshold
]
hard_blocker_count = sum(1 for case in cases if case["p0"])
gate_pass = not failed_gates and hard_blocker_count == 0

return {
    "metrics": metrics,
    "hard_blocker_count": hard_blocker_count,
    "failed_cases": failed_cases,
    "failed_gates": failed_gates,
    "ai_infra_gate_pass": gate_pass,
}

cases = build_cases()
report = audit_ai_infra_overview(cases)

smoke = {
    "complete_case_passes": "complete_ai_infra_stack" not in report["failed_cases"],
    "caught_kubernetes_only": "kubernetes_only_bad" in report["failed_cases"],
    "caught_network_ignored": "network_ignored_bad" in report["failed_cases"],
    "caught_inference_no_slo": "inference_no_slo_bad" in report["failed_cases"],
    "caught_boundary_confused": "boundary_confused_bad" in report["failed_cases"],
}

print("smoke=", smoke)
print("metrics=", report["metrics"])
print("hard_blocker_count=", report["hard_blocker_count"])
print("failed_cases=", report["failed_cases"])
print("failed_gates=", report["failed_gates"])
print("ai_infra_gate_pass=", report["ai_infra_gate_pass"])

```

这段 demo 的设计故意让 16 个 bad case 分别打穿 16 个维度，因此每个覆盖率都是 $16/17=0.941$ ，低于 0.95 阈值。它想表达的不是“真实平台必须有 16 个指标”，而是面试中不能只说“我们用了 Kubernetes”或“我们买了 GPU”。一个可上线的 AI Infra 总览回答，需要同时覆盖计算、网络、存储、调度、训练、推理、数据、artifact、评估、可观测性、安全、成本、开发者体验和跨团队协作。

1.9 常见误区

误区一：AI Infra 等于 Kubernetes。

Kubernetes 只是资源管理和容器编排的一部分。AI Infra 还包括 GPU、网络、存储、训练平台、推理平台、数据平台、评估平台、可观测性和治理。

误区二：AI Infra 等于买 GPU。

GPU 是基础，但没有调度、网络、存储、平台和运维，GPU 很容易变成昂贵但低利用率的资源。

误区三：算法岗不用懂 Infra。

大模型训练和推理问题经常跨越算法、系统和平台边界。懂 Infra 能帮助你更快定位问题，也能设计更可落地的方案。

误区四：推理平台只是把模型部署成服务。

推理平台还要处理 batching、KV cache、模型路由、限流、自动扩缩容、灰度、回滚、监控和成本。

误区五：监控是上线之后再补。

大模型系统如果没有可观测性，出了问题很难定位。监控、日志、trace 和事件应该从平台设计一开始就纳入。

1.10 面试题

题 1：AI Infra 和普通后端基础设施有什么不同？

答：AI Infra 更依赖 GPU/加速器、显存、高速网络、大规模数据读取、checkpoint 存储和模型运行时。训练任务通常长时间运行、资源需求大、需要 gang scheduling 和容错；推理服务则需要处理 token 级延迟、KV cache、batching 和模型版本治理。

题 2：为什么 GPU 多不一定训练快？

答：训练速度还受网络通信、并行策略、数据读取、存储吞吐、checkpoint、CPU 预处理和调度影响。如果通信或 I/O 成为瓶颈，增加 GPU 只会增加等待时间，扩展效率可能下降。

题 3：训练平台和推理平台的关注点有什么不同？

答：训练平台关注资源申请、分布式启动、数据供给、checkpoint、日志、指标、容错和可复现性。推理平台关注模型加载、请求路由、延迟、吞吐、batching、KV cache、限流、熔断、灰度、回滚和成本。

题 4：AI Infra 为什么需要成本治理？

答：GPU、存储、网络和推理 token 都很贵。没有成本治理，容易出现 GPU 闲置、低利用率训练、重复实验、无效 checkpoint、过度扩容和高成本模型滥用。成本治理帮助团队在效果、延迟和资源之间做合理权衡。

题 5：为什么可观测性对 AI Infra 特别重要？

答：大模型系统链路长、组件多、故障原因复杂。训练慢可能来自数据、网络、GPU、框架或存储；推理慢可能来自队列、prefill、decode、KV cache 或路由。没有 metrics、logs、traces 和 events，就很难定位问题。

1.11 小练习

练习一：画出你理解的大模型训练平台架构图。

要求：至少包含用户入口、任务提交、调度器、GPU 集群、数据存储、checkpoint、日志和监控。

练习二：画出你理解的大模型推理平台架构图。

要求：至少包含 API Gateway、模型路由、推理引擎、batching、KV cache、自动扩缩容、监控和成本统计。

练习三：分析一个 GPU 利用率低的问题。

假设训练任务 GPU 利用率只有 35%，请列出至少 8 个可能原因，并说明你会先看哪些指标。

练习四：区分 AI Infra、MLOps、LLMOps 和 Platform Engineering。

要求：用自己的话分别解释它们关注什么，并举一个具体例子。

1.12 本章小结

本章是第二十三册的入口。

你需要掌握：

1. AI Infra 是支撑大模型训练、推理、评估和应用运行的基础设施与平台工程体系。
2. 模型 API 背后包含 GPU、网络、存储、调度、推理引擎、监控、安全和成本治理。
3. AI Infra 主要模块包括计算、网络、存储、调度、训练平台、推理平台、数据平台、模型仓库、评估平台和治理体系。
4. AI Infra 和 MLOps、LLMOps、Platform Engineering 有交集，但 AI Infra 更偏底层和平台底座。
5. 算法岗也要懂 AI Infra，因为训练效率、推理延迟、稳定性和成本都与基础设施密切相关。
6. AI Infra 的核心 trade-off 包括性能与成本、利用率与隔离性、灵活性与标准化、速度与治理。

后面的章节会逐步拆开这些模块：先讲 GPU、显存、网络 and 存储，再讲调度、训练平台、推理平台、数据平台、可观测性、成本治理和系统设计。

第 2 章 AI Infra、MLOps、LLMOps、Platform Engineering 的边界

上一章我们从整体上认识了 AI Infra：它是支撑大模型训练、推理、评估和应用运行的基础设施与平台工程体系。

但在真实团队里，你会经常听到几个相近的词：AI Infra、MLOps、LLMOps、Platform Engineering、DevOps、Data Platform、Model Platform。它们有交集，但不是同一个东西。

如果边界讲不清，面试时很容易出现两种问题：要么把 AI Infra 说成 Kubernetes，要么把 LLMOps 说成 prompt 管理工具。

本章目标是把这些概念拆清楚。

先记住一句话：

AI Infra 更像底座，MLOps / LLMOps 更像模型生命周期管理，Platform Engineering 更像把底座能力产品化给内部团队使用的方法。

2.0 本讲资料边界与第二轮精修口径

本讲讨论的是概念边界和职责切分，不是给某个组织画固定组织架构，也不是定义行业统一岗位名称。真实公司里，AI Infra、ML Platform、LLMOps、Data Platform、Model Platform、SRE 和 Platform Engineering 经常会重叠，重要的是交付物、接口和责任边界可审计。

第二轮精修时，我按 WRITING_PLAN.md 做了资料校准，主要参考公开官方资料中的稳定边界：Google Cloud 的 MLOps 资料强调 ML 系统开发与运维、自动化、监控、CI/CD/CT、数据和模型验证；MLflow 的 LLM / Agent 文档把 tracing、evaluation、prompt management、AI gateway、agent serving 和 monitoring 放在 LLM 应用工程能力里；CNCF Platforms White Paper 把平台定义成面向内部用户的一组能力、接口、模板、自助 API 和一致体验；OpenTelemetry 用 traces、metrics、logs、baggage 等 signals 描述可观测性基础；Google SRE 的 SLI / SLO 框架强调可靠性目标要由用户关心的指标驱动。

因此，本章新增内容采用“职责边界审计”口径：不说某个工具天然属于某个团队，而是看它解决的是资源运行、模型生命周期、大模型应用生命周期、内部开发者体验、数据底座、模型资产管理还是可靠性治理问题。

2.1 为什么边界容易混乱

这些概念混乱，主要有四个原因。

第一，它们都和“让模型上线”有关。

训练、评估、部署、监控、回滚、数据、权限、成本，每个概念都会碰到其中一部分。

第二，大模型改变了传统机器学习平台的边界。

过去 MLOps 主要服务传统 ML 模型，例如特征工程、模型训练、模型部署。大模型出现后，又增加了 prompt、RAG、Agent、工具调用、token 成本、多模型路由、人工评测和安全治理。

第三，公司组织架构不同。

有的公司 AI Infra 团队负责 GPU 集群和训练平台；有的公司叫 ML Platform；有的公司把推理平台放在在线服务团队；有的公司把 LLMOps 做成应用平台。

第四，厂商命名不统一。

云厂商、模型厂商、开源项目和企业内部平台会用不同名字包装相似能力。

所以不要死记名字，要看它解决的问题。

2.2 一张总览图

可以用下面的分层理解：

Business / Product / AI Applications

- > LLMops: prompt, RAG, Agent, eval, model routing, safety
- > MLOps: data, training pipeline, experiment, model registry, deployment
- > Platform Engineering: portal, CLI, SDK, self-service, golden path
- > AI Infra: GPU cluster, network, storage, scheduler, runtime, observability, security, cost
- > Hardware / Cloud / Data Center

这张图不是严格上下级关系，而是强调侧重点。

1. AI Infra 关心底层资源和平台能力。
2. MLOps 关心机器学习生命周期。
3. LLMOps 关心大模型应用生命周期。
4. Platform Engineering 关心内部开发者体验和自助化。

2.3 AI Infra 的核心边界

AI Infra 的关键词是“资源、平台、运行时、治理”。

它负责让大模型工作负载有地方跑、跑得稳、跑得快、可观测、可治理。

典型范围包括：

1. GPU / NPU / TPU 集群。
2. 高速网络和集合通信。
3. 本地盘、共享文件系统、对象存储和数据湖。
4. Kubernetes、Slurm、Ray 等调度系统。
5. 多租户资源配额和优先级。
6. 训练平台。
7. 推理平台。
8. 模型运行时和 serving runtime。
9. 日志、指标、trace、event 和 profile。
10. 权限、安全、审计和成本治理。

AI Infra 关注的问题通常是：

1. GPU 为什么闲着？
2. 多机训练为什么扩展效率低？
3. checkpoint 为什么保存很慢？
4. 推理请求为什么排队？
5. p95 延迟为什么突然升高？
6. 哪些团队占用了最多 GPU？
7. 哪个模型版本造成错误率上升？
8. 如何让训练任务失败后自动恢复？

一句话：AI Infra 管的是 “底座能不能支撑 AI 工作负载”。

2.4 MLOps 的核心边界

MLOps 的关键词是 “机器学习生命周期”。

它关注一个模型从数据到训练、评估、部署、监控和迭代的完整流程。

典型范围包括：

1. 数据集版本管理。
2. 特征工程和 Feature Store。
3. 训练 pipeline。
4. 实验追踪。
5. 模型注册和模型仓库。
6. 模型评估。
7. 模型部署。
8. 在线监控。
9. 数据漂移和模型漂移检测。
10. 回滚和再训练。

传统 MLOps 场景里，模型可能是推荐模型、投放模型、风控模型、CV 模型或小型 NLP 模型。

MLOps 关注的问题通常是：

1. 这次训练用了哪个数据版本？
2. 超参和代码版本是什么？
3. 实验指标有没有提升？
4. 模型是否通过上线门禁？
5. 线上数据分布是否变化？
6. 模型效果下降后如何触发再训练？
7. 如何回滚到上一个稳定模型？

一句话：MLOps 管的是 “机器学习模型生命周期是否可复现、可发布、可监控、可迭代”。

2.5 LLMOps 的核心边界

LLMOps 是大模型时代的模型和应用生命周期管理。

它继承 MLOps 的一部分，但又有明显新增内容。

典型范围包括：

1. Prompt 管理。
2. Prompt 版本和回归测试。
3. RAG 知识库管理。
4. Embedding 和向量索引管理。
5. 大模型评测。
6. 人工评测和偏好数据。
7. 模型路由和多模型选择。
8. Agent trace。
9. 工具调用日志和安全策略。
10. token 成本治理。
11. 内容安全和输出审核。
12. provider 管理。

LLMOps 关注的问题通常是：

1. 这个 prompt 版本是否比上一个好？
2. RAG 检索结果是否相关？
3. 模型回答是否有 hallucination？

4. Agent 调用了哪些工具？
5. 工具调用是否越权？
6. token 成本是否超预算？
7. 哪个模型更适合当前任务？
8. 安全策略是否拦截了风险输出？

一句话：LLMOps 管的是 “大模型应用从 prompt、RAG、Agent 到评估和上线的生命周期”。

2.6 Platform Engineering 的核心边界

Platform Engineering 的关键词是 “内部开发者平台” 和 “自助化”。

它不是某个 AI 专属概念，而是一种工程组织方式。

它的目标是把复杂基础设施包装成内部团队容易使用的产品。

典型能力包括：

1. Portal。
2. CLI。
3. SDK。
4. 模板和最佳实践。
5. 自助申请资源。
6. 自助提交任务。
7. 自助部署服务。
8. 自助查看日志和指标。
9. 权限申请和审批流。
10. 标准化 golden path。

Platform Engineering 关注的问题通常是：

1. 算法工程师能否自己提交训练任务？
2. 应用工程师能否自己部署模型服务？
3. 团队是否必须找平台同学手工开资源？
4. 新项目是否有标准模板？
5. 平台是否把安全和监控作为默认能力？
6. 开发者是否知道任务失败原因？

一句话：Platform Engineering 管的是 “如何让内部用户高效、安全、标准化地使用平台能力”。

2.7 DevOps、SRE 和 AI Infra 的关系

DevOps 强调开发和运维协作，关注自动化交付、CI/CD、环境一致性和发布效率。

SRE 强调可靠性工程，关注 SLO、错误预算、监控告警、事故响应和容量规划。

AI Infra 会借鉴 DevOps 和 SRE，但 AI 工作负载有特殊性：

1. GPU 资源昂贵且稀缺。
2. 训练任务长时间运行。
3. 分布式训练需要多节点同时可用。
4. checkpoint 和数据吞吐压力大。
5. 推理延迟和 token 生成过程强相关。
6. 模型效果不是普通服务健康检查能完全衡量。
7. 数据、模型和 prompt 都需要版本治理。

所以 AI Infra 不是简单把 Web 服务那套搬过来，而是把 DevOps / SRE 思想适配到 AI 工作负载。

2.8 Data Platform 和 AI Infra 的关系

Data Platform 关注数据采集、计算、存储、治理和服务。

典型组件包括：

1. 数据湖。
2. 数仓。
3. 流式计算。
4. 离线计算。
5. 数据质量。
6. 数据血缘。
7. 权限和合规。

AI Infra 依赖 Data Platform，因为训练数据、评估数据、RAG 文档和业务日志都来自数据平台。

但 AI Infra 还会额外关注 AI 特有的数据问题：

1. 大规模样本读取吞吐。
2. 训练数据分片和 shuffle。
3. 数据与 checkpoint 的版本对应关系。
4. 训练集和评估集污染。
5. embedding 索引更新。
6. 多模态数据存储。

一句话：Data Platform 提供数据底座，AI Infra 把数据底座接入训练、评估、推理和 Agent 工作负载。

2.9 Model Platform 和 AI Infra 的关系

Model Platform 通常指围绕模型资产的管理平台。

包括：

1. 模型注册。
2. 模型版本。
3. 权重存储。
4. tokenizer 和配置管理。
5. adapter 和 LoRA 管理。
6. 量化版本。
7. 模型评测报告。
8. 模型发布状态。
9. 权限和审批。

它和 AI Infra 的关系是：

1. AI Infra 提供存储、训练和推理底座。
2. Model Platform 管理模型 artifact 和生命周期。
3. 推理平台从 Model Platform 拉取模型版本。
4. 评估平台把评测结果回写到 Model Platform。

在成熟系统里，模型不是文件路径，而是带 metadata、权限、血缘、评估结果和发布状态的资产。

2.10 一个例子：训练一个新模型

假设团队要训练一个新的客服大模型。

AI Infra 负责：

1. 提供 GPU 集群。
2. 提供网络和存储。
3. 调度训练任务。
4. 提供训练平台。
5. 采集训练指标和日志。
6. 保存 checkpoint。
7. 处理节点失败和重试。

MLOps 负责：

1. 记录数据版本。
2. 记录训练配置。
3. 记录实验指标。
4. 注册模型版本。
5. 管理评估和上线流程。

LLMOps 负责：

1. 评测对话质量。
2. 管理 prompt 模板。
3. 测试 RAG 效果。
4. 跟踪安全和幻觉指标。
5. 管理上线后的 token 成本。

Platform Engineering 负责：

1. 提供自助提交入口。
2. 提供标准训练模板。
3. 提供查看日志和指标的页面。
4. 提供权限申请流程。
5. 降低用户使用底层平台的复杂度。

同一件事，不同层关注点不同。

2.11 一个例子：上线一个 RAG 应用

假设团队要上线一个企业知识库问答系统。

AI Infra 负责：

1. 推理服务资源。
2. Embedding 服务。
3. 向量索引运行环境。
4. 存储和网络。
5. 监控、日志、限流和成本统计。

MLOps 可能负责：

1. embedding 模型版本。
2. reranker 模型版本。
3. 模型评估和发布。

LLMOps 负责：

1. prompt 版本。
2. RAG 召回评估。
3. 答案正确性评估。
4. 引用来源检查。
5. Agent 或工具调用 trace。
6. 内容安全和输出审核。

Platform Engineering 负责：

1. 提供知识库接入模板。
2. 提供应用创建向导。
3. 提供默认监控面板。
4. 提供灰度发布流程。
5. 提供开发者文档和 SDK。

这个例子能说明：LLMOps 更接近大模型应用层，而 AI Infra 提供运行底座。

2.12 常见组织形态

不同公司会有不同团队拆分。

小团队里，一个平台团队可能同时负责：

1. GPU 集群。
2. 训练平台。
3. 推理平台。
4. LLMOps 工具。
5. 监控和成本。

中型团队可能拆成：

1. Infra 团队：集群、网络、存储、调度。
2. Training Platform 团队：训练任务、checkpoint、实验追踪。
3. Inference Platform 团队：模型服务、路由、batching、扩缩容。
4. LLMOps 团队：prompt、RAG、评估、Agent trace。
5. Data Platform 团队：数据湖、数据质量、数据权限。

大型公司可能进一步拆分出：

1. GPU Fleet 团队。
2. Networking 团队。
3. Storage 团队。
4. Scheduler 团队。
5. Model Runtime 团队。
6. Eval Platform 团队。
7. Safety Platform 团队。
8. Cost Governance 团队。

面试时不要纠结名称，而要能说明每个团队解决什么问题。

2.13 如何判断一个需求属于哪一层

可以用下面的问题判断。

如果问题是“资源在哪里、怎么调度、怎么跑稳”，通常属于 AI Infra。

如果问题是“模型从训练到上线怎么管理”，通常属于 MLOps。

如果问题是“prompt、RAG、Agent、工具调用和大模型评测怎么管理”，通常属于 LLMOps。

如果问题是“内部用户怎么自助使用这些能力”，通常属于 Platform Engineering。

举例：

1. GPU 利用率低：AI Infra。
2. 训练实验不可复现：MLOps。
3. Prompt 改动导致线上回答变差：LLMOps。
4. 算法同学提交训练任务太复杂：Platform Engineering。
5. 模型权重版本混乱：MLOps / Model Platform。
6. RAG 检索质量下降：LLMOps / Data Platform。
7. 推理 p95 延迟升高：AI Infra / Inference Platform。
8. 线上模型回答有安全风险：LLMOps / Safety Platform。

2.14 面试回答模板

如果面试官问：

AI Infra、MLOps、LLMOps、Platform Engineering 有什么区别？

可以这样回答：

我会从关注点区分。

AI Infra 更偏底层基础设施和平台能力，负责 GPU 集群、网络、存储、调度、训练平台、推理平台、可观测性、安全和成本治理，目

MLOps 更偏机器学习生命周期，负责数据版本、训练 pipeline、实验追踪、模型注册、评估、部署、监控和回滚，目标是让模型开发

LLMOps 是大模型应用生命周期，除了继承 MLOps 一部分能力，还重点关注 prompt、RAG、Agent、工具调用、模型路由、大模型评测

Platform Engineering 是一种产品化内部平台的方式，把上述能力通过 Portal、CLI、SDK、模板和自助流程提供给算法和应用团队使

这几个概念有交集，但不是同义词。AI Infra 是底座，MLOps / LLMOps 是生命周期管理，Platform Engineering 是面向内部用户的交

2.15 边界审计指标与最小 demo

边界题最容易答成“背名词”。更可靠的方式是把每个需求映射到主责层、协作层、交付物和门禁。

先定义一个需求或事故样本：

$b_i = (u_i, y_i, h_i, a_i, m_i, l_i, p_i, d_i, o_i, s_i, c_i, z_i)$

其中， u_i 是用户或需求场景， y_i 是应该主责的层， h_i 是当前回答或系统设计给出的主责层， a_i 是 AI Infra 资源运行职责， m_i 是 MLOps 生命周期职责， l_i 是 LLMOps 应用生命周期职责， p_i 是 Platform Engineering 开发者体验职责， d_i 是 Data / Model Platform 资产职责， o_i 是可观测性和 SLO 证据， s_i 是安全、权限和成本治理， c_i 是跨团队接口， z_i 是标签、风险等级和复盘信息。

分类错误率可以写成：

$R_{\{\mathrm{route}\}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[h_i \neq y_i]$

第 j 个边界维度的覆盖率仍然写成：

$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(b_i) = 1]$

如果要把职责矩阵写清楚，可以用：

$A_{\{k,m\}} \in \{0,1\}$

其中， $A_{\{k,m\}} = 1$ 表示第 k 个平台层对第 m 个能力有主责或硬门禁责任。注意：主责不代表独占。比如模型 registry 可以属于 MLOps / Model Platform，但它要和 AI Infra 的存储、推理平台的模型加载、评估平台的报告和权限系统打通。

边界门禁可以写成：

$G_{\{\mathrm{boundary}\}} = \mathbf{1}[\left[\min_j C_j \geq \tau_j \ \& \ R_{\{\mathrm{route}\}} = 0 \ \& \ P_0 = 0\right]]$

其中， τ_j 是各维度最低通过率， P_0 是 P0 级边界风险数量。边界风险的典型形式是：出了事故没人主责、上线前缺硬门禁、平台接口没有 owner、模型版本和数据版本断链、安全策略只写在文档里、成本和 SLO 无法归因。

下面是一个 0 依赖 Python demo。它把本章概念做成 toy 边界审计表：

```
METRICS = [
    "ai_infra_scope_accuracy",
    "mlops_lifecycle_accuracy",
    "llmops_application_accuracy",
    "platform_engineering_dx_accuracy",
    "devops_sre_boundary_accuracy",
    "data_platform_boundary_accuracy",
    "model_platform_boundary_accuracy",
    "primary_owner_clarity",
    "interface_contract_coverage",
    "artifact_lineage_handoff",
    "observability_slo_handoff",
```



```

    "security_cost_governance_handoff",
    "lifecycle_stage_mapping",
    "anti_tool_name_confusion",
    "incident_routing_accuracy",
    "collaboration_handoff_readiness",
]

def make_case(name, expected_owner, actual_owner=None, failed_metric=None, p0=False):
    flags = {metric: True for metric in METRICS}
    if failed_metric is not None:
        flags[failed_metric] = False
    return {
        "name": name,
        "expected_owner": expected_owner,
        "actual_owner": actual_owner or expected_owner,
        "flags": flags,
        "p0": p0,
    }

def build_cases():
    bad_cases = [
        ("gpu_queue_called_mlops_bad", "AI Infra", "MLOps", "ai_infra_scope_accuracy"),
        ("model_registry_no_lifecycle_bad", "MLOps", "AI Infra", "mlops_lifecycle_accuracy"),
        ("llmops_prompt_only_bad", "LLMOps", "Prompt Tool", "llmops_application_accuracy"),
        ("platform_as_devops_rename_bad", "Platform Engineering", "DevOps", "platform_engineering_dx_accuracy"),
        ("slo_missing_for_serving_bad", "SRE", "AI Infra", "devops_sre_boundary_accuracy"),
        ("rag_doc_freshness_misrouted_bad", "Data Platform", "LLMOps", "data_platform_boundary_accuracy"),
        ("weight_file_no_model_platform_bad", "Model Platform", "Object Storage", "model_platform_boundary_accuracy"),
        ("no_primary_owner_bad", "AI Infra", "Unknown", "primary_owner_clarity"),
        ("no_interface_contract_bad", "Platform Engineering", "Ad Hoc Ticket", "interface_contract_coverage"),
        ("artifact_lineage_broken_bad", "MLOps", "Unknown", "artifact_lineage_handoff"),
        ("trace_slo_not_connected_bad", "SRE", "Monitoring Only", "observability_slo_handoff"),
        ("cost_security_not_handoff_bad", "AI Infra", "Finance", "security_cost_governance_handoff"),
        ("lifecycle_stage_missing_bad", "MLOps", "Script", "lifecycle_stage_mapping"),
        ("tool_name_confusion_bad", "LLMOps", "Kubernetes", "anti_tool_name_confusion"),
        ("rag_incident_wrong_route_bad", "LLMOps", "AI Infra", "incident_routing_accuracy"),
        ("team_handoff_missing_bad", "Platform Engineering", "Manual Ops", "collaboration_handoff_readiness"),
    ]

    cases = [make_case("complete_boundary_map", "Boundary Board")]
    cases.extend(
        make_case(name, expected, actual, metric, p0=True)
        for name, expected, actual, metric in bad_cases
    )
    return cases

def audit_boundary(cases, threshold=0.95):
    metrics = {}
    for metric in METRICS:
        passed = sum(1 for case in cases if case["flags"][metric])
        metrics[metric] = round(passed / len(cases), 3)

```

```

misrouted_cases = [
    case["name"]
    for case in cases
    if case["expected_owner"] != case["actual_owner"]
]
failed_cases = [
    case["name"]
    for case in cases
    if case["p0"] or any(not case["flags"][metric] for metric in METRICS)
]
failed_gates = [
    metric for metric, score in metrics.items() if score < threshold
]
hard_blocker_count = sum(1 for case in cases if case["p0"])
route_error_rate = round(len(misrouted_cases) / len(cases), 3)
gate_pass = not failed_gates and not misrouted_cases and hard_blocker_count == 0

return {
    "metrics": metrics,
    "route_error_rate": route_error_rate,
    "hard_blocker_count": hard_blocker_count,
    "misrouted_cases": misrouted_cases,
    "failed_cases": failed_cases,
    "failed_gates": failed_gates,
    "boundary_gate_pass": gate_pass,
}

```

```

cases = build_cases()
report = audit_boundary(cases)

```

```

smoke = {
    "complete_case_passes": "complete_boundary_map" not in report["failed_cases"],
    "caught_mlops_as_ai_infra": "gpu_queue_called_mlops_bad" in report["failed_cases"],
    "caught_prompt_only_llmops": "llmops_prompt_only_bad" in report["failed_cases"],
    "caught_platform_as_devops": "platform_as_devops_rename_bad" in report["failed_cases"],
    "caught_rag_incident_misroute": "rag_incident_wrong_route_bad" in report["failed_cases"],
}

```

```

print("smoke=", smoke)
print("metrics=", report["metrics"])
print("route_error_rate=", report["route_error_rate"])
print("hard_blocker_count=", report["hard_blocker_count"])
print("misrouted_cases=", report["misrouted_cases"])
print("failed_cases=", report["failed_cases"])
print("failed_gates=", report["failed_gates"])
print("boundary_gate_pass=", report["boundary_gate_pass"])

```

这段 demo 里有 1 个完整样本和 16 个边界错误样本，所以每个维度的覆盖率是 16/17=0.941，低于 0.95 阈值。它想训练的并不是固定答案，而是边界判断方法：先识别主责层，再列协作层，再说明交付物和门禁。面试时只要能把事故路由、artifact 血缘、trace / SLO、安全成本和开发者自助讲清楚，就不会停留在名词解释。

2.16 常见误区

误区一：MLOps 等于 AI Infra。

MLOps 更关注模型生命周期，AI Infra 更关注底层资源和平台能力。训练平台可能同时属于两者交集，但二者侧重点不同。

误区二：LLMOps 只是 prompt 管理。

Prompt 管理只是 LLMOps 的一部分。LLMOps 还包括 RAG、评估、Agent trace、工具调用、安全、模型路由和 token 成本。

误区三：Platform Engineering 是换个名字的 DevOps。

Platform Engineering 更强调把基础设施能力做成内部产品，提供自助化、标准模板和开发者体验。

误区四：AI Infra 团队只负责买卡和维护集群。

成熟 AI Infra 还要负责训练平台、推理平台、可观测性、容量规划、成本治理和安全治理。

误区五：边界一定要绝对清晰。

真实组织里边界会重叠。重要的是职责清楚、接口清楚、交付物清楚，而不是名字完全统一。

2.17 面试题

题 1：AI Infra 和 MLOps 的区别是什么？

答：AI Infra 更关注底层资源和平台能力，例如 GPU 集群、网络、存储、调度、训练平台、推理平台和可观测性。MLOps 更关注模型生命周期，例如数据版本、训练 pipeline、实验追踪、模型注册、评估、部署、监控和回滚。训练平台和模型仓库可能是两者交集。

题 2：LLMOps 相比 MLOps 多了什么？

答：LLMOps 增加了大模型应用特有能力和能力，例如 prompt 管理、RAG 知识库、embedding 和向量索引、Agent trace、工具调用日志、大模型评测、内容安全、模型路由、provider 管理和 token 成本治理。

题 3：Platform Engineering 在 AI Infra 中的价值是什么？

答：它把复杂的底层能力封装成内部开发者可自助使用的平台，例如 Portal、CLI、SDK、训练模板、部署模板、日志面板和权限流程。这样算法和应用团队不用每次都找平台团队手工操作，可以更快、更标准、更安全地使用 AI Infra。

题 4：为什么说 AI Infra 不是简单的 DevOps？

答：AI 工作负载有特殊性：GPU 稀缺且昂贵，训练任务长时间运行，分布式训练需要多节点同时可用，checkpoint 和数据吞吐压力大，推理有 token 级延迟和 KV cache 管理，模型效果也需要评估。普通 Web 服务 DevOps 经验需要适配这些特点。

题 5：如果一个 RAG 应用回答质量下降，应该从哪些层排查？

答：可以从 LLMOps 排查 prompt、检索、rerank、上下文拼接和评测；从 Data Platform 排查文档更新、数据质量和索引版本；从 AI Infra 排查 embedding 服务、向量索引服务、推理延迟和错误率；从 Model Platform 排查模型版本是否变化。

2.18 小练习

练习一：给下面需求分类：GPU 任务排队太久、prompt 改动无回归测试、模型版本无法回滚、算法同学不会写 Kubernetes YAML、RAG 引用文档过期。

要求：分别判断主要属于 AI Infra、MLOps、LLMOps、Platform Engineering、Data Platform 或 Model Platform 的哪一类。

练习二：设计一个内部 AI 平台首页。

要求：从 Platform Engineering 角度列出用户最需要的入口，例如提交训练任务、部署模型服务、查看实验、查看评估、申请权限、查看成本。

练习三：画出一个大模型应用从开发到上线的生命周期。

要求：标出哪些步骤属于 MLOps，哪些属于 LLMOps，哪些依赖 AI Infra。

练习四：假设你是 AI Infra 负责人，如何和 Data Platform、Safety Platform、应用团队分工？

要求：写出每个团队的职责边界和接口。

2.19 本章小结

本章拆清了 AI Infra、MLOps、LLMOps、Platform Engineering 的边界。

你需要掌握：

1. AI Infra 更关注底层资源、平台运行时、可观测性、安全和成本治理。
2. MLOps 更关注机器学习模型生命周期，包括数据、训练、实验、模型注册、部署和回滚。
3. LLMOps 更关注大模型应用生命周期，包括 prompt、RAG、Agent、工具调用、评估、安全和 token 成本。
4. Platform Engineering 更关注把平台能力产品化，让内部团队自助、安全、标准化使用。
5. DevOps、SRE、Data Platform、Model Platform 都与 AI Infra 有交集，但关注点不同。
6. 面试时不要纠结名词，而要讲清楚每层解决什么问题、交付什么能力、和其他层如何协作。

下一章开始，我们进入硬件基础，讲 GPU、NPU、TPU 与 AI 加速器。

第二章：硬件、集群、网络、存储与调度

待扩写。

本章将覆盖 GPU/AI 加速器、显存、NVLink、NVSwitch、InfiniBand、RoCE、对象存储、共享文件系统、Kubernetes、队列调度、多租户隔离和容量规划。

第 3 章 GPU、NPU、TPU 与 AI 加速器基础

大模型训练和推理离不开 AI 加速器。你可以把加速器理解成专门为大规模矩阵计算、张量计算和并行计算设计的硬件。

很多同学一听到 AI Infra，就先想到“GPU 很贵”“A100、H100 很强”。但面试里不能只背型号。你需要理解：为什么大模型需要加速器？GPU、NPU、TPU 有什么共同点和差异？为什么显存、带宽、互联、软件生态和调度同样重要？

先记住一句话：

AI 加速器的核心价值，是用高度并行的计算单元和高带宽内存，加速大模型中海量矩阵乘法、attention、MLP 和数据搬运。

3.0 本讲资料边界与第二轮精修口径

本讲是硬件基础入口，不追逐某一代 GPU、TPU、NPU 或云实例的最新参数，也不把某个厂商的峰值算力写成通用结论。硬件规格变化很快，面试中更重要的是建立稳定判断框架：计算峰值、显存容量、显存带宽、互联、低精度格式、软件栈、通信库、调试工具、成本和迁移风险。

第二轮精修时，我按 WRITING_PLAN.md 做了资料校准，主要参考公开官方资料中的稳定边界：NVIDIA CUDA 文档把 CUDA 定义为 GPU 上的并行计算平台和编程模型；NVIDIA NCCL 文档强调 NCCL 是面向 GPU 间通信的 collective / point-to-point 通信库，并支持 PCIe、NVLink、InfiniBand 等互联；Google Cloud TPU 文档把 TPU 定义为面向机器学习工作负载的 ASIC，核心是快速矩阵运算、MXU、HBM、slice、topology 和 Pod；AMD ROCm 文档说明 ROCm 是面向 AMD GPU 的开源软件栈，覆盖 HIP、OpenCL、OpenMP 和深度学习框架兼容。

因此，本章新增内容只写稳定抽象：GPU、NPU、TPU 和 AI ASIC 都是加速器选择的一部分，差异不只在“谁的 TFLOPS 更高”，而在真实工作负载能否落到硬件、内存、互联和软件生态上。

3.1 为什么大模型需要 AI 加速器

大模型的主要计算来自矩阵和张量操作。

例如 Transformer 里最核心的计算包括：

1. Q、K、V 线性投影。
2. Attention score 计算。
3. Attention value 聚合。
4. MLP 中的矩阵乘法。
5. LayerNorm、激活函数和残差连接。
6. 反向传播中的梯度计算。

这些操作有两个特点：

1. 计算量巨大。
2. 并行度很高。

CPU 擅长复杂控制逻辑和通用任务，但大规模矩阵乘法不是 CPU 的强项。GPU 和其他 AI 加速器则有大量并行计算单元，可以同时处理大量乘加操作。

简单对比：

CPU：少量强核心，适合复杂控制和通用任务。

GPU：大量并行核心，适合矩阵、向量和图形/张量计算。

AI ASIC：面向 AI 算子专门设计，追求特定工作负载的效率。

大模型不是不能在 CPU 上跑，而是效率和成本通常无法接受。

3.2 GPU 是什么

GPU 最初是图形处理器，用来做图形渲染。后来大家发现图形计算和深度学习都大量依赖并行矩阵运算，所以 GPU 成为深度学习训练的主力。

现代数据中心 GPU 通常包含：

1. 大量计算核心。
2. Tensor Core 或类似矩阵加速单元。
3. 高带宽显存，例如 HBM。
4. 高速互联，例如 NVLink。
5. 支持 CUDA、cuDNN、NCCL 等软件栈。

GPU 的优势：

1. 通用性强。
2. 软件生态成熟。
3. 深度学习框架支持完善。
4. 适合训练和推理。
5. 可编程性较好。

GPU 的限制：

1. 成本高。
2. 功耗高。
3. 供应紧张。
4. 多机通信复杂。
5. 对软件栈版本敏感。

面试中提到 GPU，不要只说“算力强”，还要说“显存、带宽、互联和生态”。

3.3 NPU 是什么

NPU 通常指 Neural Processing Unit，也就是神经网络处理器。

不同厂商对 NPU 的定义不完全一致。有的 NPU 是手机端 AI 加速单元，有的是数据中心 AI 加速卡，有的是云厂商自研 AI 芯片。

NPU 的共同目标是：针对神经网络算子做专门优化。

它可能更强调：

1. 矩阵乘法加速。
2. 低精度计算。
3. 能效比。
4. 端侧推理。
5. 特定框架和算子融合。

NPU 的优势：

1. 对特定模型或算子能效高。
2. 在端侧设备上可以降低功耗。
3. 在云侧可能降低单位推理成本。
4. 可以和厂商软硬件栈深度结合。

NPU 的挑战：

1. 软件生态通常不如 CUDA 成熟。
2. 算子兼容性可能有限。
3. 模型迁移需要适配。
4. 调试和性能分析工具可能不完善。
5. 多机训练生态可能不如主流 GPU 成熟。

所以，NPU 不是“比 GPU 更高级”，而是面向特定场景的加速器选择。

3.4 TPU 是什么

TPU 是 Tensor Processing Unit，最早由 Google 面向 TensorFlow 和大规模机器学习工作负载设计。

TPU 的核心思想是用专门的矩阵乘法单元加速张量计算。

它通常强调：

1. 大规模矩阵乘法吞吐。
2. 高效低精度计算。
3. Pod 级互联。
4. 与 XLA 编译器结合。
5. 面向大规模训练和推理的整体系统设计。

TPU 的优势：

1. 在适配良好的模型和框架下效率很高。
2. 大规模集群互联能力强。
3. 编译器可以做图级优化。
4. 适合大规模训练工作负载。

TPU 的挑战：

1. 生态和使用方式与 GPU 不同。
2. 需要适配 XLA 和相关框架。
3. 某些动态模型或自定义算子迁移成本高。
4. 对非 Google 云环境不一定方便。

面试里可以把 TPU 理解成“专门面向张量计算和大规模 ML 工作负载的加速器体系”，而不是简单理解成“另一种 GPU”。

3.5 AI ASIC 与通用 GPU 的差异

ASIC 是 Application-Specific Integrated Circuit, 也就是专用集成电路。

AI ASIC 是为 AI 工作负载设计的专用芯片。

和 GPU 相比:

1. GPU 更通用, 生态成熟。
2. ASIC 更专用, 可能能效更高。
3. GPU 更适合快速变化的模型研究。
4. ASIC 更适合稳定、高规模、可预测的工作负载。
5. GPU 的框架支持通常更好。
6. ASIC 的迁移和调试成本可能更高。

一个简单判断:

如果模型和算子变化快, 优先考虑生态成熟的 GPU。

如果工作负载稳定、规模巨大、成本敏感, 可以考虑专用加速器。

3.6 AI 加速器的关键指标

选择加速器时, 不能只看 “峰值算力”。

需要看以下指标。

3.6.1 峰值算力

峰值算力表示理论最大计算能力, 常见单位有 TFLOPS、PFLOPS。

但峰值算力不等于真实训练速度。

原因:

1. 算子不一定能跑满硬件。
2. 数据搬运可能成为瓶颈。
3. 显存容量可能限制 batch size。
4. 多机通信可能导致等待。
5. 框架和 kernel 优化影响很大。

所以面试中不要只拿峰值算力比较硬件。

3.6.2 显存容量

显存容量决定模型参数、梯度、优化器状态、activation 和 KV cache 能不能放下。

训练时显存主要被以下内容占用:

1. 参数。
2. 梯度。
3. 优化器状态。
4. activation。
5. 临时 buffer。

推理时显存主要被以下内容占用:

1. 模型权重。
2. KV cache。
3. batch 中间状态。
4. runtime buffer。

显存不够会直接限制模型大小、batch size、context length 和并发。

3.6.3 显存带宽

显存带宽决定数据从显存读写到计算单元的速度。

大模型很多操作既受计算限制，也受内存带宽限制。

尤其是推理 decode 阶段，每次生成 token 都要读取大量权重和 KV cache，显存带宽非常关键。

3.6.4 互联带宽

多 GPU 和多机训练需要大量通信。

互联包括：

1. PCIe。
2. NVLink。
3. NVSwitch。
4. InfiniBand。
5. RoCE。

互联影响：

1. 数据并行梯度同步。
2. 张量并行中间结果交换。
3. pipeline 并行 stage 间传输。
4. checkpoint 和参数加载。

如果互联慢，多卡训练可能变成“多卡一起等”。

3.6.5 低精度支持

大模型训练和推理常用 FP16、BF16、FP8、INT8、INT4 等格式。

硬件是否支持这些格式，会影响：

1. 训练速度。
2. 推理吞吐。
3. 显存占用。
4. 数值稳定性。
5. 量化部署成本。

例如 BF16 在训练中很常见，因为它比 FP16 有更大的指数范围，数值稳定性通常更好。

3.6.6 软件生态

软件生态包括：

1. 驱动。
2. 编译器。
3. kernel 库。
4. 通信库。
5. 框架支持。
6. profiler。
7. 调试工具。
8. 社区经验。

硬件再强，如果软件生态不成熟，真实落地成本会很高。

3.7 为什么 CUDA 生态重要

CUDA 是 NVIDIA 的并行计算平台和编程模型。

在大模型生态中，CUDA 的重要性不只是“能写 GPU 程序”。

它背后有一整套生态：

1. cuDNN。
2. cuBLAS。
3. NCCL。
4. TensorRT。
5. Triton 编译器生态。
6. PyTorch 和 TensorFlow 深度支持。
7. 大量开源项目默认优化 NVIDIA GPU。
8. 完整 profiler 和调试工具。

这也是为什么很多团队即使关注其他加速器，也会把 CUDA 生态作为重要对照。

面试中可以说：

AI 加速器选型不只看芯片峰值算力，还要看框架、算子库、通信库、调试工具和开源生态。如果生态不成熟，迁移和调优成本可能抵

3.8 训练和推理对硬件的需求不同

训练和推理都需要加速器，但关注点不同。

3.8.1 训练更关注什么

训练关注：

1. 计算吞吐。
2. 显存容量。
3. 多卡互联。
4. 多机网络。
5. checkpoint I/O。
6. 数值稳定性。
7. 分布式训练生态。

训练需要保存梯度、优化器状态和 activation，所以显存压力比推理更大。

训练还需要反向传播和参数更新，通信量也更大。

3.8.2 推理更关注什么

推理关注：

1. TTFT。
2. TPOT。
3. 吞吐。
4. 并发。
5. KV cache 显存。
6. 显存带宽。
7. 低精度和量化。
8. 单位 token 成本。

推理尤其关注 decode 阶段的效率。很多时候推理不是峰值算力不够，而是显存带宽、KV cache 和调度策略限制了吞吐。

3.9 云上加速器与自建集群

企业使用 AI 加速器有两种常见方式：云上租用和自建集群。

云上优点：

1. 启动快。
2. 弹性好。
3. 不需要自己维护机房。
4. 可以尝试多种硬件。
5. 适合早期探索和弹性需求。

云上缺点：

1. 长期成本可能高。
2. 资源不一定稳定可得。
3. 网络拓扑和性能可控性有限。
4. 数据合规和安全需要额外评估。
5. 深度定制能力有限。

自建优点：

1. 长期大规模使用成本可能更可控。
2. 网络、存储、调度和安全可深度定制。
3. 资源可预测。
4. 适合稳定大规模训练和推理。

自建缺点：

1. 初始投入大。
2. 建设周期长。
3. 运维复杂。
4. 容量规划难。
5. 硬件迭代风险高。

面试回答不要绝对说云上好或自建好，要看规模、合规、成本、资源稳定性和团队运维能力。

3.10 加速器选型的思考框架

选择 GPU、NPU、TPU 或其他加速器，可以从八个维度评估：

1. 工作负载：训练、推理、微调、embedding、rerank 还是多模态。
2. 模型规模：参数量、context length、batch size。
3. 性能目标：吞吐、延迟、训练时间。
4. 显存需求：权重、activation、KV cache、优化器状态。
5. 通信需求：单机多卡还是多机多卡。
6. 软件生态：框架、算子、通信库和 profiler。
7. 成本：采购、租用、电力、运维和迁移成本。
8. 风险：供应、兼容性、团队经验和厂商锁定。

一个简化模板：

我不会只看峰值算力，而会从工作负载、显存、带宽、互联、低精度支持、软件生态、成本和运维风险综合选型。

3.11 加速器容量审计指标与最小 demo

硬件选型题不能只说“这张卡 TFLOPS 更高”。你需要把模型、batch、context length、精度、并行策略、训练 / 推理目标和硬件指标放到同一张表里。

先定义一个加速器选型审计样本：

$h_i = (w_i, m_i, b_i, \ell_i, r_i, c_i, u_i, e_i, p_i, z_i)$

其中, w_i 是工作负载类型, m_i 是模型规模, b_i 是 batch 和并发, ℓ_i 是 context length, r_i 是需要的训练或推理 runtime, c_i 是计算峰值和低精度能力, u_i 是显存容量、带宽和互联, e_i 是软件生态, p_i 是成本、电力、供应和云上 / 自建约束, z_i 是风险标签。

训练状态显存可以先用粗略下限估算：

$$M_{\{\mathrm{train}\}} \approx N, (B_{\{\mathrm{param}\}} + B_{\{\mathrm{grad}\}} + B_{\{\mathrm{opt}\}}) + M_{\{\mathrm{act}\}} + M_{\{\mathrm{tmp}\}}$$

其中, N 是参数量, B_{param} 是每个参数的存储字节数, B_{grad} 是每个梯度的字节数, B_{opt} 是 optimizer state 的字节数, M_{act} 是 activation 显存, M_{tmp} 是临时 buffer。AdamW 训练时 optimizer state 往往比权重本身更大, 所以“模型权重能放下”不等于“训练能跑”。

推理 KV cache 显存可以写成：

$$M_{\{\mathrm{kv}\}} = 2LBSH_{\{\mathrm{kv}\}}D_hB_{\{\mathrm{elem}\}}$$

其中, L 是层数, B 是 batch 或并发序列数, S 是序列长度, H_{kv} 是 KV head 数, D_h 是 head dimension, B_{elem} 是每个元素字节数, 前面的 2 表示 K 和 V。这个公式解释了为什么长上下文和高并发推理很容易被 KV cache 卡住。

判断算子更像 compute-bound 还是 memory-bound, 可以用 roofline 的简化直觉：

$$F_{\{\mathrm{achievable}\}} \leq \min(F_{\{\mathrm{peak}\}}, I B_{\{\mathrm{mem}\}})$$

其中, F_{peak} 是峰值算力, B_{mem} 是显存带宽, I 是 arithmetic intensity, 也就是每读取 1 字节能做多少 FLOPs。 I 很低时, 即使峰值算力很高, 也会被显存带宽限制。

多卡训练还要看通信暴露比例：

$$R_{\{\mathrm{comm}\}} = \frac{T_{\{\mathrm{comm}\}}}{T_{\{\mathrm{compute}\}} + T_{\{\mathrm{comm}\}} + T_{\{\mathrm{io}\}}}$$

如果 R_{comm} 很高, 问题通常不在单卡 TFLOPS, 而在并行策略、互联、通信库、拓扑或 batch / bucket 配置。

下面是一个 0 依赖 Python demo。它用 toy 数字估算训练显存、KV cache、roofline 上限, 并做加速器选型门禁：

```
METRICS = [
    "peak_compute_fit",
    "memory_capacity_fit",
    "memory_bandwidth_fit",
    "interconnect_bandwidth_fit",
    "low_precision_support",
    "software_stack_maturity",
    "kernel_library_readiness",
    "distributed_communication_readiness",
    "training_memory_budget",
    "inference_kv_cache_budget",
    "workload_hardware_fit",
    "cloud_self_build_decision",
    "cost_power_capacity_awareness",
    "profiling_observability_readiness",
    "fallback_portability_plan",
    "selection_risk_governance",
]
```

```
def gib(num_bytes):
    return num_bytes / (1024 ** 3)
```

```
def training_state_gib(params_billion, param_bytes, grad_bytes, opt_bytes):
    params = params_billion * 1_000_000_000
```

```

    return round(gib(params * (param_bytes + grad_bytes + opt_bytes)), 2)

def kv_cache_gib(layers, batch, seq_len, kv_heads, head_dim, elem_bytes):
    total_bytes = 2 * layers * batch * seq_len * kv_heads * head_dim * elem_bytes
    return round(gib(total_bytes), 2)

def roofline_tflops(peak_tflops, memory_tbps, arithmetic_intensity):
    ridge = peak_tflops / memory_tbps
    achievable = min(peak_tflops, arithmetic_intensity * memory_tbps)
    bound = "compute_bound" if arithmetic_intensity >= ridge else "memory_bound"
    return {
        "ridge_flop_per_byte": round(ridge, 2),
        "achievable_tflops": round(achievable, 2),
        "bound": bound,
    }

def make_case(name, failed_metric=None, p0=False):
    flags = {metric: True for metric in METRICS}
    if failed_metric is not None:
        flags[failed_metric] = False
    return {"name": name, "flags": flags, "p0": p0}

def build_cases():
    bad_cases = [
        ("peak_tflops_only_bad", "peak_compute_fit"),
        ("memory_too_small_bad", "memory_capacity_fit"),
        ("decode_bandwidth_ignored_bad", "memory_bandwidth_fit"),
        ("multi_gpu_interconnect_ignored_bad", "interconnect_bandwidth_fit"),
        ("fp8_int8_missing_bad", "low_precision_support"),
        ("immature_software_stack_bad", "software_stack_maturity"),
        ("kernel_library_gap_bad", "kernel_library_readiness"),
        ("collective_comm_gap_bad", "distributed_communication_readiness"),
        ("training_state_oom_bad", "training_memory_budget"),
        ("kv_cache_oom_bad", "inference_kv_cache_budget"),
        ("workload_mismatch_bad", "workload_hardware_fit"),
        ("cloud_self_build_unreasoned_bad", "cloud_self_build_decision"),
        ("cost_power_ignored_bad", "cost_power_capacity_awareness"),
        ("no_profiler_metrics_bad", "profiling_observability_readiness"),
        ("no_portability_plan_bad", "fallback_portability_plan"),
        ("vendor_lockin_unowned_bad", "selection_risk_governance"),
    ]
    cases = [make_case("complete_accelerator_plan")]
    cases.extend(make_case(name, metric, p0=True) for name, metric in bad_cases)
    return cases

def audit_accelerator_selection(cases, threshold=0.95):
    metrics = {}
    for metric in METRICS:
        passed = sum(1 for case in cases if case["flags"][metric])
        metrics[metric] = round(passed / len(cases), 3)

```

```

failed_cases = [
    case["name"]
    for case in cases
    if case["p0"] or any(not case["flags"][metric] for metric in METRICS)
]
failed_gates = [
    metric for metric, score in metrics.items() if score < threshold
]
hard_blocker_count = sum(1 for case in cases if case["p0"])
gate_pass = not failed_gates and hard_blocker_count == 0
return {
    "metrics": metrics,
    "hard_blocker_count": hard_blocker_count,
    "failed_cases": failed_cases,
    "failed_gates": failed_gates,
    "accelerator_gate_pass": gate_pass,
}

capacity_examples = {
    "weights_70b_bf16_gib": round(gib(70_000_000_000 * 2), 2),
    "train_state_70b_gib": training_state_gib(70, 2, 2, 8),
    "kv_cache_gib": kv_cache_gib(
        layers=80,
        batch=8,
        seq_len=8192,
        kv_heads=8,
        head_dim=128,
        elem_bytes=2,
    ),
    "roofline": roofline_tflops(
        peak_tflops=1000,
        memory_tbps=3.35,
        arithmetic_intensity=80,
    ),
}

cases = build_cases()
report = audit_accelerator_selection(cases)
smoke = {
    "complete_case_passes": "complete_accelerator_plan" not in report["failed_cases"],
    "caught_peak_only": "peak_tflops_only_bad" in report["failed_cases"],
    "caught_training_oom": "training_state_oom_bad" in report["failed_cases"],
    "caught_kv_oom": "kv_cache_oom_bad" in report["failed_cases"],
    "caught_vendor_lockin": "vendor_lockin_unowned_bad" in report["failed_cases"],
}

print("capacity_examples=", capacity_examples)
print("smoke=", smoke)
print("metrics=", report["metrics"])
print("hard_blocker_count=", report["hard_blocker_count"])
print("failed_cases=", report["failed_cases"])
print("failed_gates=", report["failed_gates"])
print("accelerator_gate_pass=", report["accelerator_gate_pass"])

```

这段 demo 故意把 16 个 bad case 分别打穿 16 个维度，因此每个维度覆盖率都是 $16/17=0.941$ 。前面的 capacity_examples 还会展示三个直觉：70B 权重的 BF16 存储只是推理权重下限；训练状态会因为梯度和 optimizer state 明显放大；长上下文推理的 KV cache 可以单独吃掉大量显存。面试时你不需要背具体硬件型号，但要能把这些量级关系讲清楚。

3.12 常见误区

误区一：峰值算力越高，训练一定越快。

真实训练速度还受显存、带宽、通信、I/O、kernel 优化和并行策略影响。

误区二：GPU 只要数量够就行。

多 GPU 需要互联、调度、网络、存储和分布式训练栈配合。数量多但系统差，效率可能很低。

误区三：NPU 或 TPU 一定比 GPU 更适合 AI。

要看具体工作负载、软件生态、迁移成本和团队经验。专用硬件在适配良好时效率高，但不一定通用。

误区四：推理主要看算力。

推理尤其是 decode 阶段经常受显存带宽、KV cache 和调度限制。

误区五：硬件选型和算法无关。

模型结构、精度格式、context length、并行策略和量化方式都会影响硬件需求。

3.13 面试题

题 1：为什么大模型训练主要使用 GPU？

答：大模型训练包含大量矩阵乘法和张量操作，并行度很高。GPU 有大量并行计算单元、Tensor Core、高带宽显存和成熟的软件生态，适合加速 forward、backward 和优化器计算。相比 CPU，GPU 在这类工作负载上的吞吐和能效更高。

题 2：GPU、NPU、TPU 的区别是什么？

答：GPU 是通用并行计算加速器，生态成熟，适合多种训练和推理任务。NPU 通常指面向神经网络算子优化的处理器，可能在特定场景有更好能效，但生态和兼容性取决于厂商。TPU 是面向张量计算和大规模 ML 工作负载设计的加速器体系，通常与特定编译器和云生态结合更深。

题 3：为什么不能只看峰值 TFLOPS 选 GPU？

答：峰值 TFLOPS 是理论上限，真实性能还受显存容量、显存带宽、互联带宽、低精度支持、kernel 优化、通信库、数据 I/O 和并行策略影响。很多任务瓶颈不是计算，而是内存、通信或存储。

题 4：训练和推理对加速器的需求有什么不同？

答：训练需要 forward、backward、梯度和优化器状态，显存压力和通信压力更大，关注训练吞吐、扩展效率和稳定性。推理主要关注延迟、吞吐、并发、KV cache、显存带宽、低精度和单位 token 成本。

题 5：为什么软件生态是硬件选型的重要因素？

答：大模型落地依赖框架、算子库、通信库、编译器、profiler 和调试工具。硬件理论性能再强，如果生态不成熟，模型迁移、算子适配、性能调优和故障排查成本都会很高。

3.14 小练习

练习一：比较两张加速卡。

要求：不要只比较峰值算力，还要列出显存容量、显存带宽、互联、低精度支持、软件生态、成本和适用场景。

练习二：分析一个推理吞吐低的问题。

假设 GPU 峰值算力很高，但 decode 吞吐不高，请列出可能原因，包括显存带宽、KV cache、batching 和调度。

练习三：设计一个加速器选型表。

要求：列出训练、SFT、embedding、rerank、在线推理、离线批量推理等场景分别关注哪些硬件指标。

练习四：讨论云上 GPU 和自建 GPU 集群的取舍。

要求：从成本、弹性、合规、运维、资源稳定性和性能可控性角度分析。

3.15 本章小结

本章讲了 GPU、NPU、TPU 与 AI 加速器基础。

你需要掌握：

1. 大模型需要加速器，是因为矩阵乘法和张量计算规模巨大且并行度高。
2. GPU 的优势是通用性、成熟生态、高带宽显存和强大的矩阵计算能力。
3. NPU 和 TPU 是面向神经网络或张量计算优化的加速器，但生态、迁移和适配成本需要评估。
4. 选型不能只看峰值算力，还要看显存容量、显存带宽、互联、低精度支持、软件生态和成本。
5. 训练更关注吞吐、显存、通信和容错；推理更关注延迟、并发、KV cache、带宽和单位 token 成本。
6. 云上和自建各有优缺点，要结合规模、合规、成本和团队能力判断。

下一章我们会继续往硬件内部走，重点讲显存、HBM、PCIe、NVLink、NVSwitch 与带宽瓶颈。

第三章：大模型训练平台工程

待扩写。

本章将覆盖训练任务提交、分布式训练启动、配置管理、日志指标、checkpoint、容错恢复、训练数据供给、权限审计和训练平台系统设计。

第四章：大模型推理平台工程

待扩写。

本章将覆盖模型服务 runtime、prefill/decode、KV Cache、continuous batching、PagedAttention、模型路由、缓存、自动扩缩容、限流熔断、灰度发布和推理平台系统设计。

第 4 章 显存、HBM、PCIe、NVLink、NVSwitch 与带宽瓶颈

上一章讲了 GPU、NPU、TPU 与 AI 加速器基础。很多同学理解加速器时，第一反应是“算力”。但在大模型训练和推理中，真正限制性能的经常不是算力，而是显存容量、显存带宽和设备之间的数据传输。

这章我们讲硬件里最容易被低估的一组概念：显存、HBM、PCIe、NVLink、NVSwitch 和带宽瓶颈。

先记住一句话：

大模型性能不只取决于能算多快，还取决于数据能不能及时送到计算单元，以及多张卡之间能不能高效交换数据。

4.0 本讲资料边界与第二轮精修口径

本章讨论显存、HBM、PCIe、NVLink、NVSwitch 和多机网络的稳定工程概念，不绑定某个 GPU 型号、服务器 SKU、云厂商实例名或某一代互联数字。

第二轮精修时，资料边界按官方公开材料校准：NVIDIA Hopper 架构资料说明高端 GPU 会把 HBM、PCIe、NVLink 和 NVSwitch 放在同一套加速器系统设计里；NCCL 文档把 GPU 间 collective 和 point-to-point 通信抽象成拓扑感知通信库；GPUDirect RDMA 文档强调 GPU 与网卡、存储或其他 PCIe peer 直接交换数据仍依赖硬件、驱动、拓扑和权限配置；PCI-SIG 规格库只作为 PCIe 是标准化互联规范的依据，不把某一代 PCIe 的峰值带宽写成所有机器的实际带宽。

因此，本章只抽象四类面试稳定结论：

1. HBM 解决 GPU 计算单元和显存之间的高带宽供给问题。
2. PCIe 解决 CPU、GPU、网卡和存储设备间的通用连接问题，但通常不是高频 GPU 张量通信的最优路径。
3. NVLink / NVSwitch 解决单机多 GPU 之间更高带宽、更均匀拓扑的通信问题。
4. 多机训练还要看 InfiniBand / RoCE、网卡拓扑、NCCL 路由、存储和 checkpoint I/O。

4.1 为什么带宽这么重要

大模型的计算过程可以简化成两件事：

1. 从显存读取权重、activation、KV cache 等数据。
2. 在计算单元上做矩阵乘法、attention、MLP 等计算。

如果计算单元很强，但数据送不过来，计算单元就会等待。

这就像厨房里厨师很多、炉灶很多，但食材供应很慢，整体出菜速度还是上不去。

在 AI Infra 中，常见带宽包括：

1. GPU 内部计算单元和显存之间的带宽。
2. GPU 和 CPU 之间通过 PCIe 的带宽。
3. GPU 和 GPU 之间通过 NVLink 的带宽。
4. 多 GPU 之间通过 NVSwitch 的交换带宽。
5. 多机之间通过 InfiniBand 或 RoCE 的网络带宽。
6. GPU 和存储之间间接形成的数据读取带宽。

带宽瓶颈可能发生在任意一层。

4.2 显存是什么

显存是 GPU 上的高速内存，用来存放模型计算需要的数据。

训练时显存里通常有：

1. 模型参数。
2. 梯度。
3. 优化器状态。
4. activation。
5. 临时 buffer。
6. 通信 buffer。

推理时显存里通常有：

1. 模型权重。
2. KV cache。
3. 当前 batch 的中间状态。
4. runtime 需要的临时 buffer。
5. 量化或解量化相关数据。

显存容量决定了几个关键上限：

1. 能放多大的模型。
2. 能支持多长上下文。
3. 能支持多大的 batch。
4. 能支持多少并发请求。
5. 训练时是否需要使用 ZeRO、offload、activation checkpointing 等技术。

显存不够不是小问题，很多时候它会直接决定系统方案。

4.3 训练时显存为什么消耗大

训练比推理更吃显存，因为训练不只需要模型权重。

以常见 Adam 优化器为例，训练时需要：

1. 参数本身。
2. 参数梯度。
3. Adam 的一阶动量。
4. Adam 的二阶动量。
5. forward 中间 activation。
6. backward 临时张量。

如果使用混合精度，还可能有 FP16/BF16 参数和 FP32 master weight。

因此，一个模型推理能放进一张卡，不代表训练也能放进一张卡。

粗略理解：

推理显存：权重 + KV cache + runtime buffer

训练显存：权重 + 梯度 + 优化器状态 + activation + runtime buffer

所以训练平台经常需要分布式策略，例如数据并行、张量并行、pipeline 并行、ZeRO、FSDP 和 checkpointing。

4.4 推理时 KV cache 为什么重要

推理阶段尤其要关注 KV cache。

Transformer 自回归生成时，每生成一个新 token，都需要利用前面 token 的 Key 和 Value。为了避免重复计算，系统会把历史 token 的 K/V 存下来，这就是 KV cache。

KV cache 的大小和几个因素有关：

1. batch size。
2. sequence length。
3. layer 数量。
4. hidden size。
5. attention head 数量。
6. KV head 数量。
7. 数据类型，例如 FP16、BF16、INT8。

长上下文和高并发会快速放大 KV cache。

这也是为什么推理平台里，经常出现“模型权重放得下，但并发上不去”。瓶颈可能不是权重，而是 KV cache 占满显存。

4.5 HBM 是什么

HBM 是 High Bandwidth Memory，高带宽内存。

现代高端 AI GPU 通常使用 HBM，而不是普通内存。

HBM 的特点：

1. 带宽非常高。

2. 离计算芯片很近。
3. 通过 3D 堆叠提高数据传输能力。
4. 成本高。
5. 容量仍然有限。

为什么 HBM 重要？

因为大模型不仅需要算，还需要不断读写权重、activation、KV cache 和中间结果。

如果 HBM 带宽不够，计算核心就会等数据。

面试里可以这样说：

HBM 的价值在于提供远高于普通内存的显存带宽，让矩阵计算和推理 decode 阶段有足够的供给。但 HBM 容量和成本都有限，所以

4.6 显存容量和显存带宽的区别

显存容量和显存带宽经常被混淆。

显存容量回答的是：能放多少数据？

显存带宽回答的是：单位时间能搬多少数据？

举例：

1. 模型权重太大放不下，这是容量问题。
2. batch 稍微变大就 OOM，这是容量问题。
3. decode 阶段 GPU 利用率不高但延迟高，可能是带宽问题。
4. 大量读取 KV cache 导致吞吐上不去，可能是带宽问题。

简单类比：

容量像仓库大小。

带宽像仓库门口的出入速度。

仓库很大但门很窄，取货慢。门很宽但仓库很小，东西放不下。

4.7 PCIe 是什么

PCIe 是 CPU、GPU、网卡、存储设备之间常见的高速连接总线。

在 AI 服务器中，PCIe 常用于：

1. CPU 和 GPU 通信。
2. GPU 和网卡通信。
3. GPU 和 NVMe 存储间接通信。
4. 多设备挂载和数据传输。

PCIe 的问题是：相比 GPU 内部 HBM 带宽和 NVLink，PCIe 带宽通常低得多，延迟也更高。

因此，如果训练或推理频繁在 CPU 内存和 GPU 显存之间搬数据，就可能严重拖慢性能。

常见场景：

1. 数据加载太慢，CPU 到 GPU 拷贝阻塞。
2. offload 把参数或优化器状态放到 CPU，PCIe 成为瓶颈。
3. 多 GPU 之间只能通过 PCIe 通信，通信速度不如 NVLink。
4. GPU Direct RDMA 配置不当，网络数据绕 CPU，增加开销。

面试中要记住：PCIe 可以连接设备，但不适合把大量高频张量通信都压在上面。

4.8 NVLink 是什么

NVLink 是 NVIDIA 提供的 GPU 高速互联技术。

它的目标是让 GPU 和 GPU 之间以更高带宽、更低延迟交换数据。

相比 PCIe, NVLink 更适合:

1. 张量并行中的中间结果交换。
2. 多 GPU 之间参数和激活传输。
3. 高速 P2P 访问。
4. 单机多卡训练和推理。

为什么它重要?

因为大模型经常需要把一个模型切到多张 GPU 上。

例如张量并行时, 一个矩阵乘法被拆到多张卡上, 每一步都可能需要 AllReduce 或 AllGather。如果 GPU 间互联慢, 计算会被通信拖住。

NVLink 可以显著缓解单机多卡通信瓶颈。

4.9 NVSwitch 是什么

NVSwitch 可以理解为 GPU 之间的高速交换芯片。

如果只有点对点 NVLink, GPU 之间的连接拓扑可能不完全均匀。有些 GPU 之间通信快, 有些要绕路。

NVSwitch 的目标是让多张 GPU 之间形成更高带宽、更均匀的互联结构。

它的价值:

1. 多 GPU 之间通信更均衡。
2. 支持更大规模单机 GPU 互联。
3. 降低拓扑差异对并行策略的影响。
4. 提升模型并行和集合通信效率。

可以简单理解:

NVLink 像 GPU 之间的高速公路。

NVSwitch 像高速公路交换枢纽。

4.10 单机多卡里的拓扑问题

不是所有“8 卡机器”都一样。

你需要看 GPU 之间怎么连接。

常见拓扑可能是:

1. 全部 GPU 通过 PCIe 连接。
2. 部分 GPU 之间有 NVLink。
3. 多 GPU 通过 NVSwitch 互联。
4. GPU 和网卡之间距离不同。

拓扑会影响:

1. 哪些 GPU 适合放在同一个 tensor parallel group。
2. 哪些 GPU 之间通信更快。
3. 多机通信是否能通过最近的网卡出去。
4. NCCL 选择什么通信路径。

训练慢的时候, 不能只看“我有 8 张卡”, 还要看这 8 张卡之间怎么连。

4.11 多机训练里的网络带宽

当训练扩展到多机时，瓶颈从单机内部互联扩展到机间网络。

常见网络包括：

1. InfiniBand。
2. RoCE。
3. 高速以太网。

多机训练中常见通信：

1. 数据并行的梯度 AllReduce。
2. ZeRO / FSDP 的参数 shard 同步。
3. 张量并行跨机通信。
4. pipeline 并行 stage 间 activation 传输。
5. checkpoint 保存和加载。

如果网络慢或抖动，GPU 会等待通信完成。

表现可能是：

1. GPU 利用率周期性下降。
2. step time 变长。
3. 某些 rank 明显慢。
4. NCCL timeout。
5. 训练 hang。

这就是为什么大模型集群网络设计非常重要。

4.12 带宽瓶颈怎么判断

判断带宽瓶颈，不能只看一个指标。

可以按层排查。

4.12.1 显存带宽瓶颈

可能现象：

1. GPU 算力利用率不高。
2. 显存读写利用率高。
3. decode 阶段吞吐低。
4. batch 增大后吞吐提升有限。

常见原因：

1. 权重读取频繁。
2. KV cache 读写压力大。
3. kernel 没有充分融合。
4. 算子 memory-bound。

4.12.2 PCIe 瓶颈

可能现象：

1. CPU 到 GPU 拷贝时间长。
2. 数据加载占 step time 比例高。
3. offload 后速度显著下降。
4. GPU 等待 host 数据。

常见原因：

1. dataloader 太慢。
2. pinned memory 没用好。
3. CPU 预处理太重。
4. 参数或优化器状态频繁 offload。

4.12.3 GPU 间互联瓶颈

可能现象：

1. 多卡比单卡扩展效率差。
2. tensor parallel 性能差。
3. NCCL 通信占比高。
4. 某些 GPU 之间通信明显慢。

常见原因：

1. 只走 PCIe，没有 NVLink。
2. 拓扑不均匀。
3. parallel group 划分不合理。
4. NCCL 参数或驱动配置问题。

4.12.4 机间网络瓶颈

可能现象：

1. 多机扩展效率低。
2. rank 间 step time 差异大。
3. AllReduce 时间长。
4. NCCL timeout 或 hang。

常见原因：

1. 网络带宽不足。
2. 网络拥塞。
3. RDMA 配置问题。
4. 跨机张量并行过重。
5. 拓扑感知调度不足。

4.13 如何缓解带宽瓶颈

不同瓶颈有不同手段。

4.13.1 缓解显存容量压力

可以使用：

1. 混合精度训练。
2. 参数分片。
3. ZeRO / FSDP。
4. activation checkpointing。
5. gradient accumulation。
6. 更小 batch 或 sequence length。
7. 量化推理。
8. KV cache 压缩或分页管理。

4.13.2 缓解显存带宽压力

可以使用：

1. kernel fusion。
2. FlashAttention。
3. 更高效的 attention kernel。
4. 量化降低数据读取量。
5. 改善 batch 和序列调度。
6. 减少不必要的数据拷贝。

4.13.3 缓解 PCIe 压力

可以使用：

1. 提前预取数据。
2. 使用 pinned memory。
3. 减少 CPU-GPU 往返。
4. 优化 dataloader。
5. 减少频繁 offload。
6. 使用 GPUDirect 相关能力。

4.13.4 缓解 GPU 间通信压力

可以使用：

1. 拓扑感知分组。
2. 优化 tensor parallel 和 pipeline parallel 划分。
3. 使用更高带宽互联。
4. 通信计算重叠。
5. 优化 NCCL 配置。

4.13.5 缓解机间网络压力

可以使用：

1. 减少跨机 tensor parallel。
2. 把通信密集 group 放在同机。
3. 使用更适合的并行策略。
4. 拓扑感知调度。
5. 隔离高优先级训练网络。
6. 监控网络拥塞和错误包。

4.14 面试中如何回答带宽瓶颈

如果面试官问：

为什么 GPU 算力很强，但训练或推理还是慢？

可以这样回答：

大模型性能不只由峰值算力决定，还受显存容量、显存带宽、GPU 间互联、机间网络、数据 I/O 和软件 kernel 影响。

训练时如果 activation、梯度和优化器状态占满显存，就需要分片或 checkpointing；如果多卡通信慢，AllReduce、AllGather 或 t

推理时模型权重可能放得下，但 KV cache 会随着 batch 和 context length 增长，占用大量显存；decode 阶段也经常受显存带宽限制

4.15 带宽瓶颈审计指标与最小 demo

带宽题不能只说“用 NVLink 会更快”。你需要能把显存容量、HBM 带宽、PCIe 拷贝、GPU 间互联、机间网络、存储和 checkpoint 放进同一张审计表。

先定义一个带宽瓶颈审计样本：

$$q_i=(w_i,m_i,k_i,p_i,n_i,s_i,c_i,o_i,z_i)$$

其中, w_i 是工作负载, m_i 是显存对象和容量预算, k_i 是 kernel 与 HBM 读写模式, p_i 是 PCIe / CPU-GPU 拷贝, n_i 是 NVLink / NVSwitch / 机间网络拓扑, s_i 是存储和 checkpoint I/O, c_i 是并行策略与通信量, o_i 是观测指标, z_i 是风险标签。

显存总预算可以写成:

$$M_{\mathrm{vram}}=M_{\mathrm{weight}}+M_{\mathrm{grad}}+M_{\mathrm{opt}}+M_{\mathrm{act}}+M_{\mathrm{kv}}+M_{\mathrm{comm}}$$

推理场景中, M_{grad} 和 M_{opt} 通常为 0, 但 M_{kv} 会随着并发和上下文长度增长。训练场景中, M_{grad} 、 M_{opt} 、 M_{act} 和 M_{comm} 往往让显存压力远大于权重本身。

任意链路的有效带宽可以用传输数据量除以耗时估算:

$$B_{\mathrm{eff}}=\frac{D}{T}$$

其中, D 是实际搬运的数据量, T 是这段搬运花费的时间。注意 B_{eff} 通常低于规格峰值, 因为有协议开销、拓扑绕行、kernel 调度、并发争用和软件栈开销。

显存带宽是否限制算子, 可以继续使用 roofline 直觉:

$$F_{\mathrm{achievable}}\leq \min(F_{\mathrm{peak}}, I B_{\mathrm{hbm}})$$

其中, I 是 arithmetic intensity。decode、embedding、归一化、部分小 batch kernel 和频繁 KV cache 访问经常更接近 memory-bound。

通信暴露比例可以写成:

$$R_{\mathrm{comm}}=\frac{T_{\mathrm{comm}}}{T_{\mathrm{compute}}+T_{\mathrm{comm}}+T_{\mathrm{io}}}$$

PCIe 或 offload 暴露比例可以写成:

$$R_{\mathrm{pcie}}=\frac{T_{\mathrm{pcie}}}{T_{\mathrm{step}}}$$

多卡扩展效率可以写成:

$$E_n=\frac{T_1}{nT_n}$$

其中, T_1 是单卡 step time, T_n 是 n 卡 step time。 E_n 很低时, 要检查并行策略、拓扑、通信、I/O 和 load balance, 而不是只说 “GPU 不够强”。

最后可以定义一个带宽门禁:

$$G_{\mathrm{bandwidth}}=\mathbf{1}\left[\min_j C_j\geq \tau_j \ \text{and} \ R_{\mathrm{comm}}\leq \rho_{\mathrm{comm}} \ \text{and} \ R_{\mathrm{pcie}}\leq \rho_{\mathrm{pcie}}\right]$$

其中, C_j 是第 j 个带宽审计维度覆盖率, ρ_{comm} 和 ρ_{pcie} 是可接受暴露比例阈值, P_0 是 P0 级硬阻断数量。

下面是一个 0 依赖 Python demo。它用 toy 数字估算 KV cache、HBM / PCIe 传输时间、ring all-reduce 通信时间和多卡扩展效率, 再用 bad case 检查带宽审计门禁。

```
METRICS = [
    "vram_capacity_accounting",
    "hbm_bandwidth_model",
    "pcie_transfer_awareness",
    "nvlink_topology_awareness",
    "nvswitch_all_to_all_awareness",
    "inter_node_network_awareness",
    "kv_cache_growth_accounting",
    "training_state_memory_accounting",
    "communication_volume_accounting",
    "topology_aware_parallel_group",
    "dataloader_storage_io_awareness",
    "checkpoint_io_awareness",
    "offload_penalty_awareness",
```

```

    "overlap_fusion_optimization",
    "observability_metric_coverage",
    "bandwidth_bottleneck_gate",
]

def gib(num_bytes):
    return num_bytes / (1024 ** 3)

def bytes_from_gib(num_gib):
    return num_gib * (1024 ** 3)

def transfer_seconds(num_gib, bandwidth_gbps):
    return round(bytes_from_gib(num_gib) / (bandwidth_gbps * 1_000_000_000), 3)

def kv_cache_gib(layers, batch, seq_len, kv_heads, head_dim, elem_bytes):
    total_bytes = 2 * layers * batch * seq_len * kv_heads * head_dim * elem_bytes
    return round(gib(total_bytes), 2)

def training_state_gib(params_billion, param_bytes, grad_bytes, opt_bytes):
    params = params_billion * 1_000_000_000
    return round(gib(params * (param_bytes + grad_bytes + opt_bytes)), 2)

def ring_allreduce_seconds(num_gib, bandwidth_gbps, world_size):
    traffic_multiplier = 2 * (world_size - 1) / world_size
    return round(traffic_multiplier * bytes_from_gib(num_gib) / (bandwidth_gbps * 1_000_000_000), 3)

def scale_efficiency(single_gpu_step_s, multi_gpu_step_s, world_size):
    return round(single_gpu_step_s / (world_size * multi_gpu_step_s), 3)

def make_case(name, failed_metric=None, p0=False):
    flags = {metric: True for metric in METRICS}
    if failed_metric is not None:
        flags[failed_metric] = False
    return {"name": name, "flags": flags, "p0": p0}

def build_cases():
    bad_cases = [
        ("vram_objects_missing_bad", "vram_capacity_accounting"),
        ("hbm_bandwidth_ignored_bad", "hbm_bandwidth_model"),
        ("pcie_copy_hidden_bad", "pcie_transfer_awareness"),
        ("nvlink_topology_unknown_bad", "nvlink_topology_awareness"),
        ("nvswitch_assumed_bad", "nvswitch_all_to_all_awareness"),
        ("inter_node_network_ignored_bad", "inter_node_network_awareness"),
        ("kv_cache_growth_missing_bad", "kv_cache_growth_accounting"),
        ("training_state_underestimated_bad", "training_state_memory_accounting"),
        ("allreduce_volume_missing_bad", "communication_volume_accounting"),
    ]

```



```

        ("parallel_group_not_topology_aware_bad", "topology_aware_parallel_group"),
        ("dataloader_storage_unmeasured_bad", "dataloader_storage_io_awareness"),
        ("checkpoint_io_blocking_bad", "checkpoint_io_awareness"),
        ("offload_over_pcie_unbounded_bad", "offload_penalty_awareness"),
        ("no_overlap_or_fusion_plan_bad", "overlap_fusion_optimization"),
        ("metrics_missing_bad", "observability_metric_coverage"),
        ("no_bandwidth_gate_bad", "bandwidth_bottleneck_gate"),
    ]
    cases = [make_case("complete_bandwidth_plan")]
    cases.extend(make_case(name, metric, p0=True) for name, metric in bad_cases)
    return cases

def audit_bandwidth(cases, threshold=0.95):
    metrics = {}
    for metric in METRICS:
        passed = sum(1 for case in cases if case["flags"][metric])
        metrics[metric] = round(passed / len(cases), 3)

    failed_cases = [
        case["name"]
        for case in cases
        if case["p0"] or any(not case["flags"][metric] for metric in METRICS)
    ]
    failed_gates = [
        metric for metric, score in metrics.items() if score < threshold
    ]
    hard_blocker_count = sum(1 for case in cases if case["p0"])
    gate_pass = not failed_gates and hard_blocker_count == 0
    return {
        "metrics": metrics,
        "hard_blocker_count": hard_blocker_count,
        "failed_cases": failed_cases,
        "failed_gates": failed_gates,
        "bandwidth_gate_pass": gate_pass,
    }

bandwidth_examples = {
    "weights_70b_bf16_gib": round(gib(70_000_000_000 * 2), 2),
    "train_state_70b_adamw_gib": training_state_gib(70, 2, 2, 8),
    "kv_cache_8x8192_gib": kv_cache_gib(80, 8, 8192, 8, 128, 2),
    "hbm_read_140gib_at_3000gbps_s": transfer_seconds(140, 3000),
    "pcie_copy_16gib_at_64gbps_s": transfer_seconds(16, 64),
    "nvlink_ring_allreduce_8gib_at_900gbps_s": ring_allreduce_seconds(8, 900, 8),
    "pcie_ring_allreduce_8gib_at_64gbps_s": ring_allreduce_seconds(8, 64, 8),
    "scale_efficiency_8gpu": scale_efficiency(8.0, 1.4, 8),
}

cases = build_cases()
report = audit_bandwidth(cases)
smoke = {
    "complete_case_passes": "complete_bandwidth_plan" not in report["failed_cases"],
    "caught_pcie_copy": "pcie_copy_hidden_bad" in report["failed_cases"],
    "caught_topology_gap": "parallel_group_not_topology_aware_bad" in report["failed_cases"],
}

```

```

    "caught_kv_growth": "kv_cache_growth_missing_bad" in report["failed_cases"],
    "caught_offload_penalty": "offload_over_pcie_unbounded_bad" in report["failed_cases"],
}

```

```

print("bandwidth_examples=", bandwidth_examples)
print("smoke=", smoke)
print("metrics=", report["metrics"])
print("hard_blocker_count=", report["hard_blocker_count"])
print("failed_cases=", report["failed_cases"])
print("failed_gates=", report["failed_gates"])
print("bandwidth_gate_pass=", report["bandwidth_gate_pass"])

```

这段 demo 故意让 16 个 bad case 分别打穿 16 个审计维度，因此每个维度覆盖率都是 $16/17=0.941$ 。bandwidth_examples 也展示了三个面试直觉：同样是 8 GiB 通信量，走高带宽 GPU 互联和走 PCIe 的耗时量级不同；70B 训练状态显存远大于权重本身；长上下文推理的 KV cache 会把“权重能放下”这个判断变得不够。

4.16 常见误区

误区一：显存越大，速度一定越快。

显存容量大能放更多数据，但速度还取决于显存带宽、计算能力和通信效率。

误区二：GPU 利用率低一定是模型代码写得差。

也可能是数据加载慢、PCIe 拷贝慢、通信慢、checkpoint 阻塞或调度问题。

误区三：单机 8 卡都一样。

不同机器的 PCIe、NVLink、NVSwitch 拓扑可能不同，通信性能差异很大。

误区四：推理只要模型权重能放下就行。

长上下文和高并发下，KV cache 可能比你想象中更快占满显存。

误区五：offload 总能解决显存问题。

Offload 可以降低显存占用，但会增加 CPU-GPU 或存储-GPU 数据传输，可能让 PCIe 或 I/O 成为瓶颈。

4.17 面试题

题 1：显存容量和显存带宽有什么区别？

答：显存容量表示能存多少数据，例如模型权重、activation、KV cache。显存带宽表示单位时间能读写多少数据。容量不足会导致 OOM 或 batch/context 受限，带宽不足会导致计算单元等待数据，吞吐和延迟变差。

题 2：为什么训练比推理更吃显存？

答：训练除了模型权重，还要存梯度、优化器状态、activation 和 backward 临时张量。使用 Adam 时还需要一阶和二阶动量。推理通常主要存权重、KV cache 和 runtime buffer，所以同一模型能推理不代表能训练。

题 3：KV cache 为什么会限制推理并发？

答：KV cache 随 batch size、sequence length、layer 数、hidden size 和数据类型增长。长上下文和高并发会占用大量显存。即使模型权重能放下，KV cache 也可能让并发受限或触发 OOM。

题 4：PCIe、NVLink、NVSwitch 分别解决什么问题？

答：PCIe 是 CPU、GPU、网卡和存储设备之间的通用高速连接。NVLink 是 GPU 之间的高速点对点互联，带宽和延迟通常优于 PCIe。NVSwitch 是 GPU 之间的交换互联，让多 GPU 通信更均匀、更高带宽，适合大规模单机多卡通信。

题 5：如何排查多机训练扩展效率差？

答：先看 step time 分解，确认计算、通信、数据加载、checkpoint 各占多少。再看 NCCL 通信时间、rank 间差异、网络带宽和错误、GPU 利用率、拓扑分布和并行策略。常见原因包括网络带宽不足、拓扑不合理、跨机 tensor parallel 过重、数据 I/O 慢和通信计算没有重叠。

4.18 小练习

练习一：估算推理显存占用。

要求：列出模型权重、KV cache、batch 中间状态和 runtime buffer，并说明哪些因素会随 context length 增长。

练习二：分析一个训练慢的问题。

假设 8 卡训练 GPU 利用率只有 45%，请从 HBM、PCIe、NVLink、数据加载、NCCL 和 checkpoint 角度列出排查路径。

练习三：比较两种服务器拓扑。

假设一台机器 8 卡只通过 PCIe 互联，另一台机器 8 卡通过 NVSwitch 互联。说明它们在 tensor parallel 和数据并行下的差异。

练习四：设计一个带宽监控面板。

要求：包含 GPU 显存读写、PCIe 传输、GPU 间通信、机间网络、存储读取和 checkpoint 写入指标。

4.19 本章小结

本章讲了显存、HBM、PCIe、NVLink、NVSwitch 与带宽瓶颈。

你需要掌握：

1. 大模型性能不只取决于峰值算力，还取决于数据搬运能力。
2. 显存容量决定能放多大模型、多大 batch、多长上下文和多少并发。
3. 显存带宽决定数据能否及时供给计算单元。
4. HBM 是高带宽显存，对训练和推理都很关键。
5. PCIe 连接 CPU、GPU、网卡和存储，但高频张量通信容易受限。
6. NVLink 提供 GPU 间高速互联，NVSwitch 提供更均匀的多 GPU 交换互联。
7. 多机训练还会受到 InfiniBand、RoCE 等机间网络影响。
8. 排查性能问题要分层看显存、PCIe、GPU 间互联、机间网络和存储 I/O。

下一章我们会讲 GPU 利用率、MFU、HFU 与训练效率指标，帮助你用指标量化训练是否真的跑得好。

第 5 章 GPU 利用率、MFU、HFU 与训练效率指标

前面我们讲了 GPU、显存、HBM、PCIe、NVLink 和 NVSwitch。接下来要回答一个非常工程化的问题：怎么判断一套训练系统跑得好不好？

很多人第一反应是看 GPU 利用率。GPU 利用率当然重要，但它不是唯一指标，也不是最准确的训练效率指标。大模型训练里还经常讨论 MFU、HFU、tokens/s、step time、吞吐、扩展效率、通信占比和 I/O 占比。

先记住一句话：

GPU 利用率告诉你 GPU 忙不忙，MFU/HFU 更接近告诉你模型训练到底用了多少理论算力，训练效率要结合吞吐、通信、I/O、显存和稳定性一起看。

5.0 本讲资料边界与第二轮精修口径

本章讨论训练效率指标的的稳定工程口径，不把某个监控面板字段、某个 GPU 型号、某个 profiler 的事件名或某篇论文里的具体数值写成通用标准。

第二轮精修时，资料边界按官方和论文公开材料校准：NVIDIA DCGM / Nsight Systems 代表了 GPU 利用率、显存、SM、带宽、kernel timeline 和系统级 trace 的常见观测入口；PyTorch profiler 代表了框架层 CPU / CUDA activity、算子耗时、shape、memory 和 trace 的观测入口；Megatron-LM 和 PaLM 等大模型训练论文常用 MFU / FLOPs utilization 说明大规模训练有效算力利用情况；前文带宽瓶颈章节已经说明 GPU utilization 需要和 HBM、PCIe、NVLink / NVSwitch、网络、存储和 checkpoint 一起解释。

因此，本章只抽象四类稳定结论：

1. GPU utilization 是入口指标，不能单独证明训练效率高。
2. tokens/s、step time 和 loss 曲线是端到端训练是否快且正确的基础证据。
3. MFU / HFU 需要明确 FLOPs 估算口径、硬件峰值口径和统计窗口。
4. 训练效率排查必须同时看通信、I/O、checkpoint、rank skew、padding、重计算、kernel 和扩展效率。

5.1 为什么只看 GPU 利用率不够

GPU 利用率通常表示一段时间内 GPU 是否有 kernel 在运行。

例如你看到：

GPU Utilization: 95%

这说明 GPU 大部分时间都在执行任务。但这不等于训练效率一定高。

原因有几个：

1. GPU 忙，不代表算子高效。
2. 可能运行的是低效 kernel。
3. 可能计算单元忙，但显存带宽成为瓶颈。
4. 可能有大量小 kernel，调度开销很大。
5. 可能通信和计算没有充分重叠。
6. 可能 batch 太小，吞吐仍然低。
7. 可能有些 rank 很慢，整体被 straggler 拖住。

反过来，GPU 利用率低也不一定是模型代码错了，可能是数据加载、checkpoint、网络通信或调度问题。

所以 GPU 利用率是入口指标，不是结论。

5.2 常见训练效率指标

大模型训练常见指标包括：

1. GPU utilization：GPU 忙碌程度。
2. memory utilization：显存占用比例。
3. memory bandwidth utilization：显存带宽利用情况。
4. step time：每个训练 step 耗时。
5. tokens/s：每秒处理 token 数。
6. samples/s：每秒处理样本数。
7. TFLOPS/GPU：每张 GPU 实际计算吞吐。
8. MFU：Model FLOPs Utilization。
9. HFU：Hardware FLOPs Utilization。
10. scaling efficiency：多卡扩展效率。
11. communication overhead：通信占比。
12. dataloader time：数据加载耗时。
13. checkpoint time：checkpoint 保存耗时。

这些指标要结合起来看。

例如：

1. GPU utilization 高，但 tokens/s 低，可能是 kernel 低效或 batch 太小。
2. tokens/s 高，但 loss 异常，说明快但可能不对。
3. MFU 低，但 GPU utilization 高，可能是大量非有效模型计算。
4. step time 抖动大，可能是 I/O、网络或集群干扰。

5.3 tokens/s 和 samples/s

在大模型训练中，tokens/s 通常比 samples/s 更有意义。

因为不同样本长度可能差异很大。一个样本可以是 100 token，也可以是 4096 token。

tokens/s 表示每秒处理多少 token。

常见口径：

$\text{global_tokens_per_second} = \text{global_batch_size} * \text{sequence_length} / \text{step_time}$

如果使用 packed sequence 或动态长度，计算会更复杂，需要按实际 token 数统计。

samples/s 表示每秒处理多少样本。

它适合样本长度相对稳定的任务，但在 LLM 训练中不如 tokens/s 稳定。

面试里可以说：

LLM 训练通常更关注 tokens/s，因为模型计算量主要和 token 数、序列长度、模型大小有关，samples/s 容易被样本长度差异误导。

5.4 step time

step time 是一个训练 step 的耗时。

一个 step 通常包括：

1. 数据读取和预处理。
2. CPU 到 GPU 数据拷贝。
3. forward。
4. loss 计算。
5. backward。
6. 梯度同步。
7. optimizer step。
8. 日志记录。
9. checkpoint 或 eval，如果该 step 触发。

分析 step time 时，不能只看总时间，要拆分。

例如：

$\text{step_time} = \text{dataloader} + \text{h2d_copy} + \text{forward} + \text{backward} + \text{communication} + \text{optimizer} + \text{logging} + \text{checkpoint}$

如果总 step time 是 2 秒，其中通信 0.8 秒，说明通信是大头。如果 dataloader 0.6 秒，说明数据供给可能拖慢训练。

5.5 MFU 是什么

MFU 是 Model FLOPs Utilization，模型 FLOPs 利用率。

它想回答：模型训练实际完成的有效计算，占硬件理论峰值算力的比例是多少。

粗略公式：

$\text{MFU} = \text{actual_model_flops_per_second} / \text{theoretical_peak_flops}$

其中 `actual_model_flops_per_second` 通常根据模型结构、参数量、token 数和 step time 估算。

MFU 越高，说明模型有效计算越接近硬件理论上限。

MFU 的价值：

1. 比 GPU utilization 更接近有效训练计算。
2. 可以比较不同训练配置的效率。
3. 可以判断硬件是否被模型计算充分利用。
4. 可以帮助定位瓶颈是否在通信、I/O 或 kernel。

但 MFU 也不是绝对准确，因为 FLOPs 估算依赖模型结构和统计口径。

5.6 HFU 是什么

HFU 是 Hardware FLOPs Utilization，硬件 FLOPs 利用率。

它更关注硬件实际执行了多少 FLOPs，占理论峰值的比例。

可以粗略理解：

$$\text{HFU} = \text{hardware_executed_flops_per_second} / \text{theoretical_peak_flops}$$

MFU 和 HFU 的区别在于：

1. MFU 更关注模型有效计算。
2. HFU 更关注硬件实际执行计算。
3. HFU 可能包含一些对模型训练不直接有用的额外计算。
4. MFU 更适合衡量训练算法和系统整体效率。

如果 HFU 高但 MFU 低，可能说明硬件在忙，但有很多开销不转化为有效模型训练。

例如：

1. 重计算过多。
2. padding 太多。
3. 低效 kernel。
4. 不必要的数据格式转换。
5. 额外同步和通信导致有效计算比例低。

5.7 MFU 和 GPU 利用率的差别

GPU utilization 回答：GPU 有没有在干活？

MFU 回答：GPU 干的活里，有多少是有效模型计算，而且接近理论峰值多少？

举个例子：

场景 A：GPU 利用率 95%，MFU 25%。

场景 B：GPU 利用率 90%，MFU 45%。

场景 B 可能更好，因为它把更多硬件能力转化成了有效模型计算。

GPU 利用率高但 MFU 低的常见原因：

1. 小 kernel 太多。
2. kernel fusion 不充分。
3. padding 浪费严重。
4. 通信等待多。
5. activation recompute 过多。
6. batch 或 sequence 配置不合适。
7. attention kernel 不高效。

5.8 扩展效率 Scaling Efficiency

扩展效率衡量多卡训练是否真的带来接近线性的加速。

假设单卡 tokens/s 是 1000，8 卡理想情况应该是 8000 tokens/s。

如果实际只有 5600 tokens/s，那么扩展效率是：

$$\begin{aligned}\text{scaling_efficiency} &= \text{actual_throughput} / \text{ideal_throughput} \\ &= 5600 / 8000 \\ &= 70\%\end{aligned}$$

扩展效率下降的常见原因：

1. 梯度同步成本增加。
2. 张量并行通信成本高。
3. pipeline bubble。
4. 数据加载跟不上。
5. 网络拥塞。
6. 拓扑不合理。
7. rank 间负载不均。
8. checkpoint 同步阻塞。

多卡训练不是卡越多越快。超过一定规模后，如果通信和调度没有优化，收益会递减。

5.9 通信占比

分布式训练中，通信占比非常关键。

常见通信包括：

1. 数据并行 AllReduce。
2. FSDP / ZeRO 的 AllGather 和 ReduceScatter。
3. 张量并行的 AllReduce / AllGather。
4. pipeline 并行的 activation 传输。
5. 参数广播。

通信占比可以这样理解：

$$\text{communication_ratio} = \text{communication_time} / \text{step_time}$$

如果通信占比很高，优化方向可能是：

1. 调整并行策略。
2. 增大计算粒度。
3. 通信计算重叠。
4. 拓扑感知分组。
5. 减少跨机通信。
6. 优化 NCCL 配置。

面试中要强调：分布式训练性能很大程度取决于通信效率，而不是单卡算力。

5.10 数据加载和 I/O 指标

训练慢也可能是数据供给慢。

需要关注：

1. dataloader time。
2. CPU preprocessing time。
3. storage read throughput。
4. cache hit rate。
5. CPU 到 GPU 拷贝时间。

6. batch 组装耗时。
7. tokenizer 或解码耗时。

如果 dataloader 慢，可能出现：

1. GPU 利用率周期性下降。
2. step time 抖动。
3. CPU 使用率高。
4. 存储吞吐打满。
5. 某些 worker 成为瓶颈。

常见优化：

1. 预处理离线化。
2. 数据分片。
3. 本地缓存。
4. 提高 dataloader worker 数。
5. 使用 streaming dataset。
6. 减少训练时动态解析。
7. 使用 pinned memory 和异步拷贝。

5.11 Checkpoint 对效率的影响

Checkpoint 是训练可靠性的保障，但也会影响效率。

如果每隔一段时间保存 checkpoint，可能出现：

1. step time 周期性尖刺。
2. GPU 等待 I/O。
3. 存储带宽被打满。
4. 多个训练任务同时保存，造成拥塞。
5. checkpoint 文件过多，管理成本上升。

常见指标：

1. checkpoint save time。
2. checkpoint load time。
3. checkpoint size。
4. checkpoint frequency。
5. checkpoint failure rate。
6. storage write throughput。

优化方向：

1. 异步 checkpoint。
2. 分片 checkpoint。
3. 降低保存频率。
4. 只保留关键版本。
5. 使用更高吞吐存储。
6. 避免全量任务同时保存。

5.12 训练效率分析方法

分析训练效率，可以按这个顺序：

1. 看端到端吞吐：tokens/s、step time。
2. 看 GPU 利用率和显存占用。
3. 看 MFU / HFU。
4. 看 step time 分解。
5. 看通信占比。
6. 看数据加载和 I/O。

7. 看 checkpoint 影响。
8. 看 rank 间差异。
9. 看扩展效率。
10. 看 loss 是否正常。

为什么最后还要看 loss?

因为训练快但训练错没有意义。

例如混合精度配置错误可能让训练速度很快，但 loss 不稳定。数据 pipeline 有 bug 也可能让吞吐很好，但模型学不到东西。

5.13 常见诊断场景

5.13.1 GPU 利用率低

可能原因：

1. 数据加载慢。
2. CPU 预处理慢。
3. CPU 到 GPU 拷贝慢。
4. batch 太小。
5. checkpoint 阻塞。
6. 通信等待。
7. rank 间负载不均。
8. 调度系统导致资源干扰。

5.13.2 GPU 利用率高但 MFU 低

可能原因：

1. kernel 低效。
2. 小算子太多。
3. padding 浪费。
4. attention 实现低效。
5. 重计算开销过高。
6. 数据类型转换过多。
7. 计算不是主要瓶颈。

5.13.3 单卡快，多卡慢

可能原因：

1. 通信占比高。
2. 网络带宽不足。
3. 拓扑不合理。
4. 并行策略不合适。
5. NCCL 配置问题。
6. 某些 rank 数据加载慢。
7. checkpoint 或日志同步阻塞。

5.13.4 step time 抖动大

可能原因：

1. 数据读取不稳定。
2. 共享存储抖动。
3. 网络拥塞。
4. 周期性 checkpoint。

5. 其他任务资源干扰。
6. 动态 padding 导致 batch 计算量变化。
7. 垃圾回收或日志写入阻塞。

5.14 一个训练效率面板应该有什么

成熟训练平台应该提供训练效率面板。

至少包含：

1. loss 曲线。
2. learning rate 曲线。
3. tokens/s。
4. step time。
5. GPU utilization。
6. 显存占用。
7. 显存带宽利用。
8. MFU / HFU。
9. 通信时间。
10. dataloader time。
11. checkpoint time。
12. rank 间 step time 分布。
13. 网络吞吐。
14. 存储吞吐。
15. 错误和重试事件。

如果面板只有 loss 和 GPU utilization，定位问题会非常困难。

5.15 面试中如何回答训练效率问题

如果面试官问：

一个大模型训练任务 GPU 利用率只有 40%，你怎么排查？

可以这样答：

我不会只看 GPU utilization 一个指标，会先看 step time 分解和 tokens/s。

第一步看数据加载是否慢，包括 dataloader time、CPU 预处理、存储读取和 CPU 到 GPU 拷贝。

第二步看通信是否慢，包括 NCCL 时间、AllReduce/AllGather 占比、rank 间 step time 差异和网络带宽。

第三步看显存和 kernel，包括 batch 是否太小、padding 是否严重、attention kernel 是否高效、是否有大量小 kernel。

第四步看 checkpoint、日志和评估是否周期性阻塞。

最后结合 MFU/HFU、扩展效率和 loss 曲线判断是系统瓶颈、配置问题还是模型训练本身的问题。

如果面试官问：

GPU 利用率 95%，是否说明训练效率很好？

可以回答：

不一定。GPU utilization 只说明 GPU 大部分时间有 kernel 在跑，不代表这些计算都是有效模型计算。还要看 tokens/s、MFU、HFU、

5.16 训练效率审计指标与最小 demo

训练效率题不能只说“GPU 利用率 95%”。你需要能把吞吐、FLOPs、step time、通信、I/O、checkpoint、rank skew 和 loss 正确性放进同一个审计表。

先定义一个训练效率审计样本：

$$e_i = (u_i, t_i, f_i, h_i, c_i, d_i, r_i, p_i, l_i, z_i)$$

其中， u_i 是 GPU / memory / bandwidth utilization, t_i 是 tokens/s、samples/s 和 step time, f_i 是模型 FLOPs 估算, h_i 是硬件峰值和实际执行 FLOPs, c_i 是通信时间和通信量, d_i 是 dataloader、存储、H2D 和 checkpoint I/O, r_i 是 rank 间差异, p_i 是 padding、重计算和 kernel 效率, l_i 是 loss / grad / numerics 正确性, z_i 是风险标签。

端到端吞吐可以写成：

$$Q_{\{\mathrm{tok}\}} = \frac{B_{\{\mathrm{global}\}} S_{\{\mathrm{eff}\}}}{T_{\{\mathrm{step}\}}}$$

其中， B_{global} 是 global batch size, S_{eff} 是每个样本实际参与训练的有效 token 数, T_{step} 是 step time。如果使用 packing 或动态长度, $B_{\mathrm{global}} S_{\mathrm{eff}}$ 应替换成真实非 padding token 数。

step time 分解可以写成：

$$T_{\{\mathrm{step}\}} = T_{\{\mathrm{data}\}} + T_{\{\mathrm{h2d}\}} + T_{\{\mathrm{fwd}\}} + T_{\{\mathrm{bwd}\}} + T_{\{\mathrm{comm}\}} + T_{\{\mathrm{other}\}}$$

MFU 可以写成：

$$U_{\{\mathrm{mfu}\}} = \frac{F_{\{\mathrm{model}\}}}{T_{\{\mathrm{step}\}}} \{N_{\{\mathrm{gpu}\}} F_{\{\mathrm{peak}\}}\}$$

其中， F_{model} 是一个 step 的模型有效 FLOPs, N_{gpu} 是 GPU 数量, F_{peak} 是单 GPU 理论峰值 FLOPs/s。粗略 decoder-only 训练估算常用 $F_{\mathrm{model}} \approx 6NT$, 其中 N 是参数量, T 是 step 内有效 token 数。这个估算适合快速量级判断, 不应替代 profiler 和模型结构级 FLOPs 统计。

HFU 可以写成：

$$U_{\{\mathrm{hfu}\}} = \frac{F_{\{\mathrm{hw}\}}}{T_{\{\mathrm{step}\}}} \{N_{\{\mathrm{gpu}\}} F_{\{\mathrm{peak}\}}\}$$

其中， F_{hw} 是硬件实际执行 FLOPs。若 U_{hfu} 明显高于 U_{mfu} , 说明硬件做了不少非模型有效计算, 例如 padding、重计算、低效 kernel、格式转换或额外同步。

通信、I/O 和扩展效率可以分别写成：

$$R_{\{\mathrm{comm}\}} = \frac{T_{\{\mathrm{comm}\}}}{T_{\{\mathrm{step}\}}}$$

$$R_{\{\mathrm{io}\}} = \frac{T_{\{\mathrm{data}\}} + T_{\{\mathrm{h2d}\}} + T_{\{\mathrm{ckpt}\}}}{T_{\{\mathrm{step}\}}}$$

$$E_n = \frac{Q_n}{nQ_1}$$

其中， Q_n 是 n 卡吞吐, Q_1 是单卡吞吐。 E_n 低时, 要优先看通信、I/O、拓扑、batch 粒度和 rank skew。

最后可以定义训练效率门禁：

$$G_{\{\mathrm{eff}\}} = \mathbf{1} \left[U_{\{\mathrm{mfu}\}} \geq \tau_{\{\mathrm{mfu}\}} \wedge E_n \geq \tau_{\{\mathrm{scale}\}} \wedge R_{\{\mathrm{comm}\}} \leq \tau_{\{\mathrm{comm}\}} \wedge R_{\{\mathrm{io}\}} \leq \tau_{\{\mathrm{io}\}} \right]$$

下面是一个 0 依赖 Python demo。它用 toy step profile 计算 tokens/s、MFU、HFU、通信占比、I/O 占比、rank skew 和扩展效率, 并用 bad case 检查训练效率门禁。

```
METRICS = [
    "tokens_throughput_accounting",
    "step_time_breakdown_coverage",
    "gpu_utilization_interpretation",
    "mfu_estimation",
    "hfu_estimation",
    "model_flops_accounting",
    "hardware_peak_accounting",
    "communication_ratio_tracking",
    "io_dataloader_tracking",
    "checkpoint_overhead_tracking",
    "rank_skew_detection",
    "scaling_efficiency_tracking",
    "padding_waste_awareness",
```

```

    "recompute_overhead_awareness",
    "loss_correctness_coupling",
    "efficiency_gate",
]

def peta_flops(flops):
    return round(flops / 1_000_000_000_000_000, 3)

def model_flops(params_billion, tokens):
    return 6 * params_billion * 1_000_000_000 * tokens

def utilization(flops, step_time_s, gpu_count, peak_tflops_per_gpu):
    peak_flops_per_s = gpu_count * peak_tflops_per_gpu * 1_000_000_000_000
    return round((flops / step_time_s) / peak_flops_per_s, 3)

def step_total(parts):
    return round(sum(parts.values()), 3)

def ratio(value, total):
    return round(value / total, 3)

def rank_skew(rank_step_times):
    avg = sum(rank_step_times) / len(rank_step_times)
    return {
        "min": min(rank_step_times),
        "max": max(rank_step_times),
        "avg": round(avg, 3),
        "skew_ratio": round((max(rank_step_times) - min(rank_step_times)) / avg, 3),
    }

def scaling_efficiency(single_gpu_tps, multi_gpu_tps, gpu_count):
    return round(multi_gpu_tps / (single_gpu_tps * gpu_count), 3)

def make_case(name, failed_metric=None, p0=False):
    flags = {metric: True for metric in METRICS}
    if failed_metric is not None:
        flags[failed_metric] = False
    return {"name": name, "flags": flags, "p0": p0}

def build_cases():
    bad_cases = [
        ("tokens_per_second_missing_bad", "tokens_throughput_accounting"),
        ("no_step_breakdown_bad", "step_time_breakdown_coverage"),
        ("gpu_utilization_as_final_answer_bad", "gpu_utilization_interpretation"),
        ("mfu_not_estimated_bad", "mfu_estimation"),
        ("hfu_not_separated_bad", "hfu_estimation"),
    ]

```

```

        ("model_flops_wrong_bad", "model_flops_accounting"),
        ("peak_flops_unknown_bad", "hardware_peak_accounting"),
        ("communication_ratio_missing_bad", "communication_ratio_tracking"),
        ("dataloader_io_unmeasured_bad", "io_dataloader_tracking"),
        ("checkpoint_spike_ignored_bad", "checkpoint_overhead_tracking"),
        ("rank_skew_hidden_bad", "rank_skew_detection"),
        ("scaling_efficiency_missing_bad", "scaling_efficiency_tracking"),
        ("padding_waste_ignored_bad", "padding_waste_awareness"),
        ("recompute_overhead_ignored_bad", "recompute_overhead_awareness"),
        ("fast_but_loss_wrong_bad", "loss_correctness_coupling"),
        ("no_efficiency_gate_bad", "efficiency_gate"),
    ]
    cases = [make_case("complete_training_efficiency_plan")]
    cases.extend(make_case(name, metric, p0=True) for name, metric in bad_cases)
    return cases

def audit_efficiency(cases, threshold=0.95):
    metrics = {}
    for metric in METRICS:
        passed = sum(1 for case in cases if case["flags"][metric])
        metrics[metric] = round(passed / len(cases), 3)

    failed_cases = [
        case["name"]
        for case in cases
        if case["p0"] or any(not case["flags"][metric] for metric in METRICS)
    ]
    failed_gates = [
        metric for metric, score in metrics.items() if score < threshold
    ]
    hard_blocker_count = sum(1 for case in cases if case["p0"])
    gate_pass = not failed_gates and hard_blocker_count == 0
    return {
        "metrics": metrics,
        "hard_blocker_count": hard_blocker_count,
        "failed_cases": failed_cases,
        "failed_gates": failed_gates,
        "efficiency_gate_pass": gate_pass,
    }

global_tokens = 262_144
step_parts = {
    "dataloader": 0.25,
    "h2d_copy": 0.08,
    "forward": 0.75,
    "backward": 1.05,
    "communication": 0.45,
    "optimizer": 0.25,
    "checkpoint": 0.12,
    "misc": 0.05,
}
step_time = step_total(step_parts)
model_step_flops = model_flops(params_billion=7, tokens=global_tokens)

```

```

hardware_step_flops = 14_640_000_000_000_000
tokens_per_second = round(global_tokens / step_time)

efficiency_examples = {
    "step_time_s": step_time,
    "tokens_per_second": tokens_per_second,
    "model_step_pfllops": peta_flops(model_step_flops),
    "mfu": utilization(model_step_flops, step_time, gpu_count=8, peak_tflops_per_gpu=1000),
    "hfu": utilization(hardware_step_flops, step_time, gpu_count=8, peak_tflops_per_gpu=1000),
    "communication_ratio": ratio(step_parts["communication"], step_time),
    "io_ratio": ratio(step_parts["dataloader"] + step_parts["h2d_copy"] + step_parts["checkpoint"], step_time),
    "rank_skew": rank_skew([2.85, 2.9, 2.98, 3.0, 3.05, 3.1, 3.3, 3.42]),
    "scaling_efficiency_8gpu": scaling_efficiency(12_000, tokens_per_second, 8),
}

cases = build_cases()
report = audit_efficiency(cases)
smoke = {
    "complete_case_passes": "complete_training_efficiency_plan" not in report["failed_cases"],
    "caught_gpu_util_only": "gpu_utilization_as_final_answer_bad" in report["failed_cases"],
    "caught_mfu_missing": "mfu_not_estimated_bad" in report["failed_cases"],
    "caught_rank_skew": "rank_skew_hidden_bad" in report["failed_cases"],
    "caught_fast_wrong_loss": "fast_but_loss_wrong_bad" in report["failed_cases"],
}

print("efficiency_examples=", efficiency_examples)
print("smoke=", smoke)
print("metrics=", report["metrics"])
print("hard_blocker_count=", report["hard_blocker_count"])
print("failed_cases=", report["failed_cases"])
print("failed_gates=", report["failed_gates"])
print("efficiency_gate_pass=", report["efficiency_gate_pass"])

```

这段 demo 故意让 16 个 bad case 分别打穿 16 个审计维度，因此每个维度覆盖率都是 16/17=0.941。efficiency_examples 里的 mfu 低于 hfu，说明硬件执行 FLOPs 里有一部分没有变成模型有效 FLOPs；communication_ratio、io_ratio、rank_skew 和 scaling_efficiency_8gpu 则帮助你判断瓶颈更像通信、I/O、负载不均还是扩展效率不足。

5.17 常见误区

误区一：GPU 利用率越高越好。

高利用率是好信号，但不是充分条件。还要看有效吞吐和 MFU。

误区二：tokens/s 高就一定好。

如果 loss 异常、数据错误或训练不稳定，tokens/s 再高也没意义。

误区三：MFU 是精确真理。

MFU 依赖 FLOPs 估算口径，不同模型和实现可能有差异。它适合比较和诊断，但不能脱离上下文。

误区四：多卡扩展效率低一定是网络问题。

也可能是并行策略、batch 太小、rank 不均、checkpoint、数据加载或代码同步问题。

误区五：训练效率只归平台团队负责。

训练效率同时受模型结构、数据 pipeline、并行策略、kernel、硬件、网络、存储和调度影响，需要算法和平台一起优化。

5.18 面试题

题 1: GPU utilization 和 MFU 有什么区别?

答: GPU utilization 表示 GPU 是否忙, 通常看一段时间内是否有 kernel 运行。MFU 表示模型有效 FLOPs 占硬件理论峰值 FLOPs 的比例, 更接近训练有效计算效率。GPU utilization 高不代表 MFU 高。

题 2: HFU 和 MFU 的区别是什么?

答: HFU 更关注硬件实际执行 FLOPs 占峰值的比例, MFU 更关注模型有效计算 FLOPs 占峰值的比例。HFU 高但 MFU 低, 可能说明硬件在忙, 但有不少计算没有转化为有效模型训练。

题 3: 为什么 LLM 训练更常用 tokens/s, 而不是 samples/s?

答: LLM 样本长度差异很大, samples/s 容易误导。tokens/s 更直接反映模型处理的 token 数和计算量, 适合比较训练吞吐。

题 4: 多机训练扩展效率低怎么办?

答: 先计算 scaling efficiency, 再看 step time 分解。重点检查通信占比、NCCL 时间、rank 间差异、网络带宽、拓扑、并行策略、batch size、数据加载和 checkpoint。优化方向包括拓扑感知分组、通信计算重叠、调整并行策略、减少跨机通信和优化 I/O。

题 5: 训练速度很快但 loss 不正常, 应该怎么看?

答: 说明性能指标不能脱离训练正确性。需要检查数据 pipeline、label、mask、loss 计算、混合精度、梯度裁剪、学习率、分布式同步和 checkpoint resume 是否正确。

5.19 小练习

练习一: 设计一个训练效率 dashboard。

要求: 包含 tokens/s、step time、GPU utilization、MFU、通信占比、dataloader time、checkpoint time、rank 差异和 loss 曲线。

练习二: 分析一个 GPU 利用率低的问题。

假设 64 卡训练 GPU 利用率只有 45%, 请写出从数据、通信、显存、kernel、checkpoint 和调度六个角度的排查清单。

练习三: 计算扩展效率。

单卡吞吐 1200 tokens/s, 8 卡实际吞吐 7200 tokens/s, 计算扩展效率, 并解释可能瓶颈。

练习四: 比较两个训练配置。

配置 A: GPU utilization 96%, MFU 28%, tokens/s 10k。配置 B: GPU utilization 88%, MFU 42%, tokens/s 14k。你认为哪个更好? 为什么?

5.20 本章小结

本章讲了 GPU 利用率、MFU、HFU 与训练效率指标。

你需要掌握:

1. GPU utilization 只说明 GPU 忙不忙, 不等于训练效率高。
2. tokens/s 比 samples/s 更适合衡量 LLM 训练吞吐。
3. step time 要拆成数据加载、forward、backward、通信、优化器、checkpoint 等部分。
4. MFU 衡量模型有效 FLOPs 利用率, HFU 衡量硬件 FLOPs 利用率。
5. GPU utilization 高但 MFU 低, 说明可能存在低效 kernel、padding、重计算或非有效开销。

6. 多卡训练要看扩展效率和通信占比。
7. 训练效率排查要同时看吞吐、显存、通信、I/O、checkpoint、rank 差异和 loss。

下一章我们会讲大模型任务画像：预训练、SFT、RLHF、评估、推理和 Agent，帮助你理解不同任务对 AI Infra 的资源需求差异。

第五章：数据、模型与实验平台

待扩写。

本章将覆盖数据版本、数据血缘、模型仓库、artifact 管理、实验追踪、评估平台、embedding store、向量索引和 RAG/Agent 平台存储。

第六章：可观测性、可靠性、成本与安全

待扩写。

本章将覆盖 metrics、logs、traces、events、训练故障定位、推理故障定位、SLO/SLA、成本治理、资源利用率优化、安全治理、审计和事故复盘。

第 6 章 大模型任务画像：预训练、SFT、RLHF、评估、推理和 Agent

前面几章讲了 AI Infra 的总览、概念边界、AI 加速器、显存带宽和训练效率指标。本章要补一个非常重要的视角：不同大模型任务，对基础设施的要求完全不同。

预训练、SFT、RLHF、评估、推理、RAG、Agent 看起来都在“用大模型”，但它们的资源画像、瓶颈、调度方式、监控指标和故障模式差异很大。

先记住一句话：

AI Infra 不是为“模型”服务，而是为不同 AI 工作负载服务；不同工作负载的计算、显存、网络、存储、延迟和可靠性要求完全不同。

6.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，资料口径按“稳定工作负载画像”处理，而不是按某个厂商平台、某个云产品 SKU 或某个调度器字段来写死。预训练部分参考 Megatron-LM 等大规模语言模型训练公开资料中对数据并行、张量并行、流水并行、GPU 集群和通信瓶颈的描述；SFT / RLHF 部分参考 InstructGPT 一类对齐流程中 SFT、reward model、policy 优化和 rollout 的多阶段结构；评估部分参考 OpenAI Evals 这类把模型或系统行为做成可复现实验的开源评估框架；推理部分参考 vLLM / PagedAttention 一类推理系统中 KV cache、continuous batching 和请求级延迟的稳定工程抽象。

需要注意三点：

1. 本章不把某个训练框架、推理框架、Agent runtime 或向量数据库写成唯一标准，只抽象它们共同暴露出的资源、调度、观测和治理问题。
2. RAG 和 Agent 的任务画像依赖业务链路，不能只按 LLM 推理成本估算；检索、工具、权限、trace、人工接管和失败重试都可能成为主瓶颈。
3. 任务画像的目标不是给每类任务贴标签，而是把“该配什么资源、用什么队列、看什么指标、怎么算成本、失败后怎么恢复”变成可审计的输入。

6.1 什么是任务画像

任务画像就是描述一个任务对基础设施的需求特征。

可以从这些维度看：

1. 任务目标: 训练、微调、评估、推理还是 Agent 执行。
2. 运行时长: 分钟级、小时级、天级还是周级。
3. 资源规模: 单卡、多卡、单机多卡、多机多卡。
4. 计算强度: 矩阵计算多不多。
5. 显存压力: 参数、activation、KV cache 占用如何。
6. 网络压力: 是否需要大量跨卡通信。
7. 存储压力: 是否频繁读数据、写 checkpoint。
8. 延迟要求: 离线任务还是在线服务。
9. 容错要求: 失败能不能重试, 是否允许中断。
10. 成本敏感度: 是否需要极致压低单位成本。

理解任务画像后, 你才能设计合理的平台和调度策略。

6.2 预训练任务画像

预训练是从大量通用数据中训练基础模型。

它的特点是:

1. 数据规模极大。
2. 训练时间长。
3. GPU 数量多。
4. 分布式训练复杂。
5. checkpoint 体积大。
6. 成本高。
7. 容错非常重要。

6.2.1 资源特征

预训练通常需要:

1. 大规模 GPU 集群。
2. 高速互联网络。
3. 高吞吐数据读取。
4. 高可靠 checkpoint 存储。
5. 稳定的调度和运行环境。

预训练任务可能连续运行几天、几周甚至更久。一次节点故障、网络抖动或存储异常, 都可能造成巨大成本浪费。

6.2.2 主要瓶颈

预训练常见瓶颈:

1. GPU 利用率低。
2. 多机通信慢。
3. 数据加载跟不上。
4. checkpoint 保存慢。
5. 节点故障导致训练中断。
6. loss 异常或训练发散。
7. 集群排队时间长。

6.2.3 平台诉求

预训练平台需要重点支持:

1. 大规模分布式启动。
2. gang scheduling。
3. 拓扑感知调度。
4. checkpoint 自动保存和恢复。

5. 节点故障检测。
6. 训练指标和日志聚合。
7. 数据版本和实验追踪。
8. 成本统计。

预训练是 AI Infra 能力的压力测试。

6.3 SFT 任务画像

SFT 是 Supervised Fine-Tuning, 也就是监督微调。

它通常在基础模型上, 用指令数据、对话数据或任务数据做微调。

相比预训练, SFT 通常:

1. 数据规模更小。
2. 训练时间更短。
3. 资源规模更小。
4. 实验迭代更频繁。
5. 更关注数据质量和效果评估。

6.3.1 资源特征

SFT 可能使用:

1. 单机多卡。
2. 少量多机。
3. LoRA / QLoRA 等参数高效微调。
4. 多实验并行。

它不像预训练那样一定追求超大规模, 但更强调快速迭代。

6.3.2 主要瓶颈

SFT 常见瓶颈:

1. 小实验太多, 调度排队影响效率。
2. 数据版本混乱。
3. prompt / template 改动不可追踪。
4. 显存不够导致 batch 很小。
5. 多个实验结果难比较。
6. 评估集污染或不稳定。

6.3.3 平台诉求

SFT 平台更需要:

1. 快速任务提交。
2. 实验追踪。
3. 数据集版本管理。
4. 参数和配置管理。
5. 模型 artifact 管理。
6. 自动评估。
7. LoRA / adapter 管理。
8. 低优先级资源池。

SFT 的核心不是单次训练最大规模, 而是高质量、高频率、可复现的迭代。

6.4 RLHF / RLAIF 任务画像

RLHF 是 Reinforcement Learning from Human Feedback, RLAIF 是 Reinforcement Learning from AI Feedback。

它们用于偏好对齐和行为优化。

这类任务比 SFT 更复杂，因为它通常涉及多个模型和多个阶段。

例如 RLHF 可能包含：

1. SFT 模型。
2. Reward Model。
3. Policy Model。
4. Reference Model。
5. 采样生成。
6. 偏好数据。
7. PPO、DPO 或其他优化算法。

6.4.1 资源特征

RLHF / RLAIF 的资源画像比较特殊：

1. 同时有训练和推理。
2. 需要大量 rollout 或样本生成。
3. 需要 reward 评分。
4. 可能需要多个模型同时加载。
5. pipeline 更复杂。
6. 对实验追踪要求高。

和预训练相比，它不一定 GPU 总量最大，但编排复杂度更高。

6.4.2 主要瓶颈

常见瓶颈：

1. rollout 生成慢。
2. reward model 评分慢。
3. 多模型显存占用高。
4. pipeline 中某个阶段成为瓶颈。
5. 偏好数据质量不稳定。
6. 训练过程指标难解释。
7. 实验不可复现。

6.4.3 平台诉求

RLHF / RLAIF 平台需要：

1. 多阶段 pipeline 编排。
2. 训练和推理资源联合调度。
3. 数据、模型、reward、policy 版本追踪。
4. rollout 样本管理。
5. 人工或 AI 反馈数据管理。
6. 安全和偏好评估。
7. 复杂实验追踪。

这类任务是 MLOps、LLMOps 和 AI Infra 的交叉区域。

6.5 评估任务画像

评估任务用于判断模型是否真的变好。

评估可以分为：

1. 离线自动评估。
2. 人工评估。
3. LLM-as-judge。
4. 安全评估。
5. 回归评估。
6. 在线 A/B 测试。

6.5.1 资源特征

评估任务通常：

1. 推理请求多。
2. 样本集多。
3. 模型版本多。
4. 并发可控。
5. 对可复现性要求高。
6. 对结果存储和分析要求高。

评估不一定需要训练级大规模 GPU，但会消耗大量推理资源。

6.5.2 主要瓶颈

评估常见瓶颈：

1. 推理成本高。
2. 多模型评测排队。
3. benchmark 版本混乱。
4. 评测结果不可复现。
5. judge 模型不稳定。
6. 评估样本污染。
7. 人工评估周期长。

6.5.3 平台诉求

评估平台需要：

1. benchmark 管理。
2. eval job 调度。
3. 多模型批量推理。
4. 评估结果存储。
5. 指标对比和可视化。
6. 样本级错误分析。
7. 安全评估和回归测试。
8. 成本预算控制。

评估任务的关键不是“跑完”，而是“可比较、可复现、可解释”。

6.6 推理任务画像

推理是把模型服务给用户或业务系统。

它和训练最大的区别是：推理通常在线服务，有明确延迟和可用性要求。

6.6.1 资源特征

推理任务关注：

1. TTFT。
2. TPOT。
3. QPS。
4. 并发。
5. KV cache。
6. 显存带宽。
7. batching。
8. 自动扩缩容。
9. 单位 token 成本。

推理还有明显流量波动。白天和夜间、工作日和节假日、活动期间和普通期间，负载可能差异很大。

6.6.2 主要瓶颈

推理常见瓶颈：

1. 队列等待时间长。
2. TTFT 高。
3. decode 慢。
4. KV cache 爆显存。
5. batch 调度不合理。
6. 热点模型资源不足。
7. 长上下文请求拖慢短请求。
8. 扩容太慢。
9. 成本过高。

6.6.3 平台诉求

推理平台需要：

1. 模型服务 runtime。
2. continuous batching。
3. KV cache 管理。
4. 模型路由。
5. 限流、熔断和降级。
6. 自动扩缩容。
7. 灰度发布和回滚。
8. 请求级 trace。
9. 成本统计。
10. SLO 监控。

推理任务的核心是质量、延迟、吞吐、可用性和成本之间的平衡。

6.7 RAG 任务画像

RAG 是 Retrieval-Augmented Generation，检索增强生成。

RAG 不只是模型推理，还包含知识库、embedding、向量检索、rerank 和上下文拼接。

6.7.1 资源特征

RAG 涉及：

1. 文档解析。
2. 文本切分。

3. embedding 生成。
4. 向量索引构建。
5. 在线检索。
6. rerank。
7. LLM 生成。
8. 引用和溯源。

因此 RAG 的瓶颈可能不在 LLM，而在检索链路。

6.7.2 主要瓶颈

RAG 常见瓶颈：

1. 文档更新延迟。
2. embedding 生成慢。
3. 向量索引构建慢。
4. 检索召回差。
5. rerank 延迟高。
6. 上下文过长。
7. 引用来源不准确。
8. 权限过滤复杂。

6.7.3 平台诉求

RAG 平台需要：

1. 知识库管理。
2. 文档解析 pipeline。
3. embedding 服务。
4. 向量索引服务。
5. 检索质量评估。
6. 权限过滤。
7. 引用追踪。
8. RAG trace。
9. 数据更新监控。

RAG 是 LLM Ops 和 Data Platform 深度交叉的典型场景。

6.8 Agent 任务画像

Agent 任务比普通推理更复杂，因为它可能包含多轮思考、工具调用、记忆读写、RAG、代码执行、浏览器操作和多 Agent 协作。

6.8.1 资源特征

Agent 任务通常：

1. 请求链路长。
2. 工具调用多。
3. 延迟不稳定。
4. token 消耗难预测。
5. 需要 trace。
6. 需要权限控制。
7. 失败模式复杂。
8. 可能需要人工接管。

普通推理是“一次模型调用”，Agent 可能是“一串模型调用和工具调用”。

6.8.2 主要瓶颈

Agent 常见瓶颈：

1. 工具调用延迟高。
2. 多轮模型调用成本高。
3. 上下文越来越长。
4. 任务循环。
5. 工具失败后不会恢复。
6. 权限检查复杂。
7. trace 数据量大。
8. 结果不可复现。

6.8.3 平台诉求

Agent 平台需要：

1. Agent Runtime。
2. Tool / MCP Gateway。
3. Memory Store。
4. Trace Store。
5. 权限和审计。
6. 步数和成本预算。
7. 任务状态机。
8. 人工接管。
9. Agent eval。

Agent 工作负载对 AI Infra 的要求不仅是算力，还包括状态、工具、权限、trace 和治理。

6.9 多模态任务画像

多模态任务包括图像、音频、视频、文本的联合训练和推理。

它的特点是数据更重、预处理更复杂、存储和带宽压力更大。

常见场景：

1. 图文理解。
2. OCR。
3. 图像生成。
4. 视频理解。
5. 视频生成。
6. 语音识别。
7. 语音合成。

多模态任务的基础设施压力：

1. 原始数据体积大。
2. 解码和预处理重。
3. 存储吞吐要求高。
4. GPU 显存压力大。
5. batch 组装复杂。
6. 评估更难自动化。

例如视频任务可能不是 GPU 算不动，而是视频解码、帧采样和数据读取拖慢训练。

6.10 不同任务的资源画像对比

可以用下面的表总结：

任务	计算压力	显存压力	网络压力	存储压力	延迟要求	关键指标
预训练	很高	很高	很高	很高	低	tokens/s、MFU、loss、checkpoint
SFT	中到高	中到高	中	中	低	迭代速度、eval、可复现性
RLHF/RLAIF	高	高	中到高	中	中	rollout 吞吐、reward 质量、稳定性
评估	中	中	低到中	中	低到中	评测吞吐、成本、可复现性
推理	中到高	高	中	低到中	很高	TTFT、TPOT、QPS、成本
RAG	中	中	低到中	高	高	检索延迟、召回、引用准确性
Agent	不稳定	不稳定	中	中	中到高	成功率、步数、成本、trace
多模态	高	高	中	很高	视场景而定	数据吞吐、预处理、质量指标

这张表不是绝对标准，但能帮助你在面试中快速建立分析框架。

6.11 调度策略如何随任务变化

不同任务应该使用不同调度策略。

预训练：

1. 需要大规模连续资源。
2. 适合 gang scheduling。
3. 需要高优先级和稳定资源。
4. 需要拓扑感知。

SFT：

1. 实验多、规模小到中等。
2. 适合队列化和配额管理。
3. 可以使用低优先级或可抢占资源。
4. 需要快速启动。

评估：

1. 可以批量排队。
2. 可以使用低峰资源。
3. 需要成本预算。
4. 需要结果可复现。

推理：

1. 需要在线 SLO。
2. 需要弹性扩缩容。
3. 不能随意抢占。
4. 需要隔离高优先级流量。

Agent：

1. 需要控制最大步数。
2. 需要工具调用预算。
3. 需要任务状态和超时。
4. 高风险动作需要人工确认。

调度系统如果不区分任务画像，很容易出现资源错配。

6.12 监控指标如何随任务变化

预训练要重点看：

1. loss。
2. tokens/s。

3. MFU。
4. step time。
5. 通信占比。
6. checkpoint 状态。
7. 节点健康。

SFT 要重点看：

1. eval score。
2. loss。
3. 训练配置。
4. 数据版本。
5. 实验对比。

评估要重点看：

1. benchmark 版本。
2. 模型版本。
3. 样本级结果。
4. 评测成本。
5. judge 一致性。

推理要重点看：

1. QPS。
2. TTFT。
3. TPOT。
4. p95 / p99 延迟。
5. 错误率。
6. 队列长度。
7. GPU 利用率。
8. KV cache 使用率。

Agent 要重点看：

1. 任务成功率。
2. 平均步数。
3. 工具调用次数。
4. token 成本。
5. 失败原因。
6. trace 完整性。
7. 人工接管率。

指标设计必须服务于任务画像。

6.13 成本模型如何随任务变化

预训练成本主要来自：

1. GPU 小时。
2. 网络和存储。
3. 失败重跑。
4. checkpoint 存储。

SFT 成本主要来自：

1. 多实验并行。
2. 低利用率小任务。
3. 重复训练。
4. 评估调用。

评估成本主要来自：

1. 批量推理。
2. judge 模型调用。
3. 人工评估。
4. 结果存储。

推理成本主要来自：

1. GPU 常驻。
2. token 生成。
3. KV cache 显存。
4. 峰值容量冗余。
5. 多模型部署。

Agent 成本主要来自：

1. 多轮模型调用。
2. 工具调用。
3. RAG 检索。
4. 长上下文。
5. 失败重试和循环。

成本治理必须按任务拆分，否则很难找到真正的浪费来源。

6.14 面试中如何回答任务画像问题

如果面试官问：

预训练、SFT、推理和 Agent 对基础设施的要求有什么不同？

可以这样回答：

我会从任务画像分析，包括计算、显存、网络、存储、延迟、容错和成本。

预训练是长周期、大规模、多机多卡任务，重点是 GPU 吞吐、MFU、网络通信、数据供给、checkpoint 和容错。

SFT 规模通常小一些，但实验频繁，重点是快速提交、数据版本、实验追踪、模型 artifact 和自动评估。

推理是在线服务，重点是 TTFT、TPOT、QPS、KV cache、batching、自动扩缩容、限流降级和单位 token 成本。

Agent 是多轮模型调用加工具调用，重点不只是算力，还包括任务状态、工具权限、trace、步数预算、成本控制和失败恢复。

所以 AI Infra 不能用一种调度和监控方式服务所有任务，需要按任务画像设计资源池、调度策略、监控指标和成本模型。

6.15 大模型任务画像审计指标与最小 demo

前面的章节讲的是定性画像。真实平台设计里，还要把任务画像转成可检查的数据结构，否则很容易停留在“预训练很重、推理要低延迟、Agent 很复杂”这种口号层面。

可以把第 i 个工作负载样本写成：

$a_i = (w_i, s_i, r_i, c_i, m_i, n_i, d_i, l_i, o_i, g_i, z_i)$

其中， w_i 是 workload type， s_i 是阶段， r_i 是运行时长或生命周期， c_i 是计算需求， m_i 是显存和内存需求， n_i 是网络需求， d_i 是数据和存储需求， l_i 是延迟或 SLO， o_i 是观测信号， g_i 是成本和治理， z_i 是风险。

更工程化一点，可以把资源画像写成向量：

$\mathbf{v}_i = (C_i, M_i, N_i, D_i, L_i, R_i, K_i, S_i)$

其中， C_i 是 compute， M_i 是 memory， N_i 是 network， D_i 是 data / storage， L_i 是 latency， R_i 是 reliability， K_i 是 cost， S_i 是 security / governance。

离线批任务的总耗时可以粗略拆成：

$$T_i = T_{\{\mathrm{queue}\}, i} + T_{\{\mathrm{data}\}, i} + T_{\{\mathrm{compute}\}, i} + T_{\{\mathrm{comm}\}, i} + T_{\{\mathrm{io}\}, i} + T_{\{\mathrm{checkpoint}\}, i}$$

这个式子提醒你：预训练、SFT、评估这类任务不是只看 GPU compute。排队、数据读取、通信、checkpoint I/O 和自动评估都会进入端到端时间。

在线推理或 Agent 链路可以用 SLO 通过率表达：

$$R_{\{\mathrm{slo}\}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\mathrm{TTFT}_i \leq \tau_{\{\mathrm{ttft}\}} \wedge \mathrm{TPOT}_i \leq \tau_{\{\mathrm{tpot}\}}]$$

其中， TTFT_i 是第 i 个请求的首 token 延迟， TPOT_i 是每输出 token 延迟， E_i 是错误标记。只看平均延迟通常不够，最好同时看 p95 / p99、错误率、队列等待和请求 trace。

成本也要按任务拆开：

$$K_i = K_{\{\mathrm{gpu}\}, i} + K_{\{\mathrm{storage}\}, i} + K_{\{\mathrm{network}\}, i} + K_{\{\mathrm{human}\}, i} + K_{\{\mathrm{api}\}, i}$$

预训练的 K_{gpu} 可能最大；评估和 RLHF 可能有 K_{human} 或 judge / reward 调用；Agent 和 RAG 可能因为工具调用、检索、长上下文和重试让 K_{api} 上升。

最后，可以把任务画像门禁写成：

$$G_{\{\mathrm{task}\}} = \mathbf{1}[\left[\min_j C_j \geq \tau_j \wedge R_{\{\mathrm{slo}\}} \geq \tau_{\{\mathrm{slo}\}} \wedge P_0 = 0 \right]]$$

其中， C_j 是第 j 个审计指标覆盖率， τ_j 是最低覆盖率阈值， P_0 是 P0 级风险数量。这个门禁不是让所有任务都满足同一套资源指标，而是要求每类任务必须有匹配自己的资源、调度、监控、成本和风险说明。

下面这个 0 依赖 demo 演示如何把任务画像写成可审计规则。它故意构造 1 个完整样本和 16 个坏样本，让每个审计维度各失败一次。

```
import copy
```

```
METRICS = [
    "workload_type_classification",
    "resource_shape_completeness",
    "pretraining_profile_accuracy",
    "sft_iteration_profile_accuracy",
    "rlhf_pipeline_profile_accuracy",
    "evaluation_reproducibility_profile",
    "serving_slo_profile",
    "rag_freshness_retrieval_profile",
    "agent_tool_runtime_profile",
    "multimodal_resource_profile",
    "scheduler_policy_fit",
    "observability_metric_fit",
    "cost_model_fit",
    "artifact_lineage_fit",
    "safety_governance_fit",
    "task_profile_gate",
]
```

```
def slo_pass_rate(requests, ttft_limit_ms, tpot_limit_ms):
    passed = 0
    for req in requests:
        ok = (
            req["ttft_ms"] <= ttft_limit_ms
            and req["tpot_ms"] <= tpot_limit_ms
            and req["errors"] == 0
        )
        passed += int(ok)
```

```

return passed / len(requests)

def batch_job_time_s(parts):
    return sum(parts.values())

def request_cost_usd(prompt_tokens, output_tokens, input_per_m, output_per_m, tool_cost):
    return (
        prompt_tokens * input_per_m / 1_000_000
        + output_tokens * output_per_m / 1_000_000
        + tool_cost
    )

def build_profiles():
    complete = {
        "name": "complete",
        "workload_type": "mixed_llm_platform",
        "stages": ["pretraining", "sft", "rlhf", "eval", "serving", "rag", "agent"],
        "resources": {"C": 8, "M": 8, "N": 7, "D": 8, "L": 7, "R": 8, "K": 7, "S": 8},
        "pretraining": {
            "tokens_per_second": 100_000,
            "mfu": 0.45,
            "checkpoint_minutes": 30,
            "data_version": "pretrain-v42",
            "topology_aware": True,
        },
        "sft": {
            "dataset_version": "sft-v7",
            "template_version": "chat-template-v4",
            "eval_suite": "sft-regression-v3",
            "adapter_artifacts": True,
        },
        "rlhf": {
            "sft_model": "sft-ckpt-7",
            "reward_model": "rm-3",
            "policy_model": "policy-9",
            "reference_model": "ref-9",
            "rollout_samples": 50_000,
            "preference_data_version": "pref-v5",
            "safety_eval": True,
        },
        "evaluation": {
            "benchmark_version": "eval-v12",
            "model_version": "model-v8",
            "sample_outputs_stored": True,
            "judge_seed": 123,
            "replayable": True,
        },
        "serving": {
            "requests": [
                {"ttft_ms": 420, "tpot_ms": 31, "errors": 0},
                {"ttft_ms": 610, "tpot_ms": 42, "errors": 0},
                {"ttft_ms": 780, "tpot_ms": 55, "errors": 0},
            ]
        }
    }

```

```

        {"ttft_ms": 930, "tpot_ms": 62, "errors": 0},
    ],
    "ttft_limit_ms": 1_000,
    "tpot_limit_ms": 80,
    "min_slo_rate": 0.95,
    "kv_cache_tracked": True,
    "autoscaling": True,
},
"rag": {
    "index_version": "kb-v12",
    "embedding_model": "embed-v3",
    "document_lag_minutes": 12,
    "citation_trace": True,
    "permission_filter": True,
},
"agent": {
    "trace_id": "trace-001",
    "max_steps": 12,
    "tool_budget_usd": 0.05,
    "state_machine": True,
    "human_escalation": True,
    "tool_spans": 5,
},
"multimodal": {
    "modalities": ["text", "image", "audio"],
    "max_payload_mb": 64,
    "preprocess_queue": True,
    "quality_eval": True,
},
"scheduler": {
    "policy_by_workload": {
        "pretraining": "gang",
        "sft": "quota",
        "eval": "batch",
        "serving": "slo",
        "agent": "deadline",
    },
    "quota": True,
},
"observability": {
    "metrics": ["tokens/s", "mfu", "ttft", "tpot", "tool_latency", "cost"],
    "logs": True,
    "traces": True,
    "events": True,
},
"cost": {"gpu": 1500, "storage": 80, "network": 35, "human": 200, "api": 120, "budget": 2200},
"lineage": {
    "dataset": "dataset-v7",
    "code": "git-sha",
    "image": "train-image-v4",
    "model": "model-v8",
    "eval_report": "eval-report-v12",
},
"governance": {
    "owner": "ai-platform",

```

```

        "risk_review": True,
        "pii_policy": True,
        "access_control": True,
    },
    "gate": {"enabled": True},
}

def bad_case(name, mutator):
    profile = copy.deepcopy(complete)
    profile["name"] = name
    mutator(profile)
    return profile

bad_cases = [
    bad_case("generic_ai_job_bad", lambda p: p.update({"workload_type": "generic"})),
    bad_case("resource_vector_incomplete_bad", lambda p: p["resources"].pop("S")),
    bad_case("pretraining_no_checkpoint_bad", lambda p: p["pretraining"].update({"checkpoint_minutes": None})),
    bad_case("sft_no_dataset_version_bad", lambda p: p["sft"].update({"dataset_version": ""})),
    bad_case("rlhf_rollout_ignored_bad", lambda p: p["rlhf"].update({"rollout_samples": 0})),
    bad_case("eval_not_reproducible_bad", lambda p: p["evaluation"].update({"replayable": False})),
    bad_case(
        "serving_no_ttft_tpot_bad",
        lambda p: p["serving"].update({"requests": [{"ttft_ms": 1400, "tpot_ms": 120, "errors": 0}]})),
    bad_case("rag_freshness_ignored_bad", lambda p: p["rag"].update({"document_lag_minutes": 720})),
    bad_case("agent_tool_trace_missing_bad", lambda p: p["agent"].update({"trace_id": ""})),
    bad_case("multimodal_payload_unbounded_bad", lambda p: p["multimodal"].update({"max_payload_mb": 2048})),
    bad_case("scheduler_one_queue_bad", lambda p: p["scheduler"].update({"policy_by_workload": {"default": "fifo"}})),
    bad_case("observability_generic_gpu_only_bad", lambda p: p["observability"].update({"metrics": ["gpu_utilization"]})),
    bad_case("cost_token_only_bad", lambda p: p["cost"].pop("human")),
    bad_case("artifact_lineage_missing_bad", lambda p: p["lineage"].update({"eval_report": ""})),
    bad_case("safety_governance_missing_bad", lambda p: p["governance"].update({"risk_review": False})),
    bad_case("task_profile_gate_missing_bad", lambda p: p["gate"].update({"enabled": False})),
]

return [complete] + bad_cases

def check_workload_type(profile):
    allowed = {"pretraining", "sft", "rlhf", "eval", "serving", "rag", "agent", "multimodal", "mixed_llm_platform"}
    return profile.get("workload_type") in allowed

def check_resource_shape(profile):
    return all(k in profile["resources"] for k in ["C", "M", "N", "D", "L", "R", "K", "S"])

def check_pretraining(profile):
    p = profile["pretraining"]
    return bool(p["tokens_per_second"] > 0 and p["mfu"] > 0 and p["checkpoint_minutes"] and p["topology_aware"])

def check_sft(profile):
    p = profile["sft"]
    return bool(p["dataset_version"] and p["template_version"] and p["eval_suite"] and p["adapter_artifacts"])

```

```

def check_rlhf(profile):
    p = profile["rlhf"]
    required = ["sft_model", "reward_model", "policy_model", "reference_model", "preference_data_version"]
    return all(p[k] for k in required) and p["rollout_samples"] > 0 and p["safety_eval"]

def check_evaluation(profile):
    p = profile["evaluation"]
    return bool(p["benchmark_version"] and p["model_version"] and p["sample_outputs_stored"] and p["replayable"])

def check_serving(profile):
    p = profile["serving"]
    rate = slo_pass_rate(p["requests"], p["ttft_limit_ms"], p["tpot_limit_ms"])
    return rate >= p["min_slo_rate"] and p["kv_cache_tracked"] and p["autoscaling"]

def check_rag(profile):
    p = profile["rag"]
    return bool(p["document_lag_minutes"] <= 60 and p["index_version"] and p["citation_trace"] and p["permission_filters"])

def check_agent(profile):
    p = profile["agent"]
    return bool(p["trace_id"] and p["max_steps"] > 0 and p["tool_budget_usd"] > 0 and p["state_machine"] and p["tool_calls"])

def check_multimodal(profile):
    p = profile["multimodal"]
    return bool(p["modalities"] and p["max_payload_mb"] <= 128 and p["preprocess_queue"] and p["quality_eval"])

def check_scheduler(profile):
    policies = profile["scheduler"]["policy_by_workload"]
    return all(k in policies for k in ["pretraining", "sft", "eval", "serving", "agent"]) and profile["scheduler"]["config"]

def check_observability(profile):
    o = profile["observability"]
    needed = {"tokens/s", "mfu", "ttft", "tpot", "tool_latency", "cost"}
    return needed.issubset(set(o["metrics"])) and o["logs"] and o["traces"] and o["events"]

def check_cost(profile):
    c = profile["cost"]
    needed = ["gpu", "storage", "network", "human", "api", "budget"]
    return all(k in c for k in needed) and sum(c[k] for k in needed[:-1]) <= c["budget"]

def check_lineage(profile):
    l = profile["lineage"]
    return all(l[k] for k in ["dataset", "code", "image", "model", "eval_report"])

```

```
def check_governance(profile):
    g = profile["governance"]
    return bool(g["owner"] and g["risk_review"] and g["pii_policy"] and g["access_control"])
```

```
def check_gate(profile):
    return profile["gate"]["enabled"]
```

```
CHECKS = {
    "workload_type_classification": check_workload_type,
    "resource_shape_completeness": check_resource_shape,
    "pretraining_profile_accuracy": check_pretraining,
    "sft_iteration_profile_accuracy": check_sft,
    "rlhf_pipeline_profile_accuracy": check_rlhf,
    "evaluation_reproducibility_profile": check_evaluation,
    "serving_slo_profile": check_serving,
    "rag_freshness_retrieval_profile": check_rag,
    "agent_tool_runtime_profile": check_agent,
    "multimodal_resource_profile": check_multimodal,
    "scheduler_policy_fit": check_scheduler,
    "observability_metric_fit": check_observability,
    "cost_model_fit": check_cost,
    "artifact_lineage_fit": check_lineage,
    "safety_governance_fit": check_governance,
    "task_profile_gate": check_gate,
}
```

```
def audit_task_profile(profiles):
    case_failures = {}
    for profile in profiles:
        failures = [name for name, check in CHECKS.items() if not check(profile)]
        case_failures[profile["name"]] = failures

    metrics = {}
    for name, check in CHECKS.items():
        metrics[name] = round(sum(int(check(p)) for p in profiles) / len(profiles), 3)

    failed_cases = [name for name, failures in case_failures.items() if failures]
    return {
        "metrics": metrics,
        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "task_profile_gate_pass": not failed_cases and min(metrics.values()) >= 0.95,
    }
```

```
profiles = build_profiles()
profile_by_name = {profile["name"]: profile for profile in profiles}
audit = audit_task_profile(profiles)
```

```
task_profile_examples = {
    "pretraining_time_h": round(batch_job_time_s({
        "queue": 240,
```



```

        "data": 900,
        "compute": 3600,
        "comm": 600,
        "io": 180,
        "eval": 120,
    }) / 3600, 2),
    "sft_time_h": round(batch_job_time_s({
        "queue": 60,
        "data": 180,
        "compute": 900,
        "comm": 60,
        "io": 60,
        "eval": 90,
    }) / 3600, 2),
    "serving_slo_pass_rate": round(slo_pass_rate([
        {"ttft_ms": 500, "tpot_ms": 40, "errors": 0},
        {"ttft_ms": 700, "tpot_ms": 60, "errors": 0},
        {"ttft_ms": 1200, "tpot_ms": 70, "errors": 0},
        {"ttft_ms": 800, "tpot_ms": 95, "errors": 0},
    ], 1000, 80), 2),
    "agent_cost_usd": round(request_cost_usd(12_000, 4_000, 1.5, 6.0, 0.02), 3),
    "complete_cost_usd": sum(profile_by_name["complete"]["cost"][k] for k in ["gpu", "storage", "network", "human", "
]),

smoke = {
    "complete_case_passes": all(check(profile_by_name["complete"]) for check in CHECKS.values()),
    "caught_pretraining_checkpoint_gap": not check_pretraining(profile_by_name["pretraining_no_checkpoint_bad"]),
    "caught_rlhf_rollout_gap": not check_rlhf(profile_by_name["rlhf_rollout_ignored_bad"]),
    "caught_serving_slo_gap": not check_serving(profile_by_name["serving_no_ttft_tpot_bad"]),
    "caught_agent_trace_gap": not check_agent(profile_by_name["agent_tool_trace_missing_bad"]),
}

print(f"task_profile_examples={task_profile_examples}")
print(f"smoke={smoke}")
print(f"metrics={audit['metrics']}")
print(f"hard_blocker_count={audit['hard_blocker_count']}")
print(f"failed_cases={audit['failed_cases']}")
print(f"task_profile_gate_pass={audit['task_profile_gate_pass']}")

```

一组典型输出是：

```

task_profile_examples={'pretraining_time_h': 1.57, 'sft_time_h': 0.38, 'serving_slo_pass_rate': 0.5, 'agent_cost_usd':
smoke={'complete_case_passes': True, 'caught_pretraining_checkpoint_gap': True, 'caught_rlhf_rollout_gap': True, 'caught
metrics={'workload_type_classification': 0.941, 'resource_shape_completeness': 0.941, 'pretraining_profile_accuracy': (
hard_blocker_count=16
failed_cases=['generic_ai_job_bad', 'resource_vector_incomplete_bad', 'pretraining_no_checkpoint_bad', 'sft_no_dataset
task_profile_gate_pass=False

```

这个 demo 的重点不在规则本身，而在思维方式：平台团队要把“任务不同”落到数据结构和门禁上。预训练缺 checkpoint 不能过；RLHF 不统计 rollout 不能过；评估不可复现不能过；推理没有 TTFT / TPOT 不能过；Agent 没 trace 不能过；成本只按 token 估算也不能过。面试中如果能把这些差异说成审计指标，而不是只背概念表，会更接近真实平台设计。

6.16 常见误区

误区一：所有大模型任务都按训练任务设计。

推理和 Agent 有在线延迟、状态和权限要求，不能用纯训练平台思路解决。

误区二：SFT 只是小规模预训练。

SFT 更强调实验迭代、数据质量、评估和可复现性，不只是资源规模变小。

误区三：评估任务不重要，可以随便跑。

评估结果直接影响模型选择和上线决策，必须管理 benchmark、模型版本、结果和成本。

误区四：推理只要部署模型就行。

推理还要处理 batching、KV cache、限流、扩缩容、灰度、回滚、监控和成本。

误区五：Agent 只是多调用几次模型。

Agent 还涉及工具、权限、状态、trace、失败恢复和人工接管。

6.17 面试题

题 1：为什么预训练对 AI Infra 要求最高？

答：预训练通常数据规模大、训练时间长、GPU 数量多、通信复杂、checkpoint 体积大、成本高。任何节点、网络、存储或数据问题都可能导致巨大损失，所以需要强调分布式调度、拓扑感知、数据供给、checkpoint 和容错。

题 2：SFT 和预训练在平台诉求上有什么不同？

答：预训练更关注大规模稳定运行和吞吐效率，SFT 更关注快速实验迭代、数据版本、配置管理、模型 artifact、自动评估和可复现性。SFT 任务规模通常较小，但数量更多、变化更快。

题 3：推理任务为什么和训练任务的基础设施要求不同？

答：推理是在线服务，关注 TTFT、TPOT、QPS、p95/p99 延迟、KV cache、batching、自动扩缩容、限流降级和可用性。训练更关注吞吐、通信、checkpoint 和收敛稳定性。

题 4：RAG 的瓶颈为什么不一定在 LLM？

答：RAG 包含文档解析、embedding、向量索引、检索、rerank、权限过滤和上下文拼接。瓶颈可能出现在文档更新、索引构建、检索延迟、rerank 延迟或引用质量，而不仅是 LLM 生成。

题 5：Agent 工作负载为什么难以做成本治理？

答：Agent 的调用链不固定，可能多轮调用模型、检索、工具、代码执行和其他 Agent。步数、上下文长度、失败重试和工具延迟都不稳定，所以需要设置最大步数、token 预算、工具调用预算、超时和 trace。

6.18 小练习

练习一：为预训练、SFT、推理、RAG、Agent 分别设计 5 个关键监控指标。

练习二：给一个企业 AI 平台设计资源池。

要求：至少包含预训练资源池、SFT 资源池、评估资源池、推理资源池和低优先级资源池，并说明调度策略。

练习三：分析一个 Agent 成本失控问题。

假设某个 Agent 平均每个任务调用模型 30 次、工具 15 次，成本过高。请从任务拆解、步数限制、模型路由、缓存、工具失败和上下文裁剪角度分析。

练习四：设计一个评估平台任务画像。

要求：说明它需要哪些资源、如何调度、如何记录模型版本和 benchmark 版本、如何控制成本。

6.19 本章小结

本章讲了大模型任务画像。

你需要掌握：

1. AI Infra 服务的是不同 AI 工作负载，而不是抽象的“模型”。
2. 预训练是长周期、大规模、高成本任务，重点是吞吐、通信、数据供给、checkpoint 和容错。
3. SFT 更关注高频实验、数据版本、配置管理、评估和可复现性。
4. RLHF / RLAIIF 同时包含训练、推理、reward 和 rollout，编排复杂度高。
5. 评估任务需要可复现、可比较、可解释和成本可控。
6. 推理任务关注在线延迟、吞吐、KV cache、扩缩容和单位 token 成本。
7. RAG 任务把 LLM、检索、embedding、索引、权限和引用串在一起。
8. Agent 任务关注状态、工具、权限、trace、步数预算和失败恢复。
9. 调度策略、监控指标和成本模型都要按任务画像设计。

下一章开始进入第二部分，讲 GPU 集群架构：单机多卡、多机多卡和机柜级互联。

第七章：AI Infra 系统设计面试

待扩写。

本章将集中训练 AI Infra 系统设计题，包括训练平台、推理平台、LLMOps 平台、GPU 集群调度、评估平台、模型仓库、可观测性平台和开放趋势题。

第 7 章 GPU 集群架构：单机多卡、多机多卡和机柜级互联

前面我们讲了单张加速器、显存带宽和训练效率指标。从本章开始进入第二部分：集群、网络、存储与调度。

大模型训练和推理很少只依赖单张 GPU。真实系统通常是单机多卡、多机多卡，甚至机柜级、集群级互联。GPU 集群架构决定了训练能扩到多大、通信效率如何、推理能承载多少并发、故障影响范围多大，以及调度系统该如何分配资源。

先记住一句话：

GPU 集群不是把很多卡堆在一起，而是计算、显存、互联、网络、存储、电力、散热和调度共同组成的系统。

7.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，资料口径按“稳定集群架构抽象”处理，而不是按某一代 GPU 服务器、某个云实例、某个交换机型号或某个调度器实现写死。单机多卡部分参考 NVIDIA HGX / DGX 资料中对 NVLink、NVSwitch、PCIe、CPU、NIC 和本地 NVMe 的系统边界；多机多卡部分参考 NCCL 对 collective communication、PCIe、NVLink / NVSwitch、InfiniBand / RoCE 和多节点通信的工程边界；GPU 到网卡亲和性部分参考 GPUDirect RDMA 文档对 PCIe 拓扑、驱动、权限和 peer-to-peer 访问条件的说明；机柜级 / 集群级部分只抽象 scale-out fabric、oversubscription、故障域、电力散热和调度感知，不把任何单一厂商的数值写成通用标准。

需要注意三点：

1. 本章关注 GPU cluster topology 和 placement，不重复上一章的“任务画像”总览，也不提前展开下一章的 InfiniBand、RoCE 和集合通信细节。
2. NVLink / NVSwitch 是 scale-up 互联，主要解决同机或同一高速互联域内 GPU 之间通信；InfiniBand / RoCE / Ethernet 是 scale-out 互联，主要解决节点之间和机柜之间通信。
3. 集群架构的核心不是“GPU 越多越好”，而是让并行策略、通信模式、故障域、数据位置、资源池和调度策略匹配。

7.1 为什么需要 GPU 集群

单张 GPU 再强，也会遇到限制。

主要限制包括：

1. 模型参数放不下。
2. batch size 不够大。
3. context length 受限。
4. 训练时间太长。
5. 推理并发不足。
6. 单卡故障会中断任务。
7. 多实验并行需要更多资源。

为了突破这些限制，需要多张 GPU 共同工作。

多 GPU 可以带来：

1. 更大的总显存。
2. 更高的总算力。
3. 更高训练吞吐。
4. 更高推理并发。
5. 更灵活的资源池。

但多 GPU 也带来新问题：通信、拓扑、调度、同步、故障和成本。

7.2 GPU 集群的基本层次

可以把 GPU 集群分成四个层次：

单 GPU

- > 单机多 GPU
- > 多机多 GPU
- > 机柜级 / 集群级 GPU 互联

每一层的瓶颈不同。

单 GPU 关注：

1. 显存容量。
2. HBM 带宽。
3. Tensor Core 性能。
4. kernel 效率。

单机多 GPU 关注：

1. GPU 间互联。
2. PCIe / NVLink / NVSwitch 拓扑。
3. 单机 CPU、内存和网卡配置。
4. NUMA 和设备亲和性。

多机多 GPU 关注：

1. 机间网络。
2. RDMA。
3. 集合通信。
4. 拓扑感知调度。
5. 分布式训练容错。

机柜级 / 集群级关注：

1. 交换机拓扑。
2. 机柜内和机柜间带宽。
3. 电力和散热。

4. 故障域。
5. 容量规划。

7.3 单机多卡架构

单机多卡是最常见的 GPU 服务器形态。

一台机器可能有 4 张、8 张或更多 GPU。

典型组件包括：

1. 多张 GPU。
2. CPU。
3. 系统内存。
4. PCIe Root Complex。
5. NVLink / NVSwitch。
6. 网卡。
7. 本地 NVMe。
8. 电源和散热系统。

单机多卡的优势：

1. 部署简单。
2. 通信延迟低于多机。
3. 故障域相对小。
4. 适合 SFT、推理、小规模训练。
5. 调试比多机更容易。

单机多卡的限制：

1. GPU 数量有限。
2. 总显存有限。
3. 单机故障会影响整机任务。
4. 机内拓扑可能不均匀。
5. CPU、内存、PCIe 和网卡可能成为瓶颈。

7.4 单机多卡拓扑为什么重要

不是所有 8 卡机器都一样。

拓扑决定 GPU 之间通信路径。

常见情况：

1. GPU 只通过 PCIe 通信。
2. 某些 GPU 对之间有 NVLink。
3. 所有 GPU 通过 NVSwitch 高速互联。
4. GPU 到网卡距离不同。
5. GPU 分属不同 CPU socket。

如果拓扑不均匀，调度和并行策略就要考虑设备亲和性。

例如：

1. tensor parallel group 应优先放在高速互联的 GPU 上。
2. 数据加载要尽量靠近对应 GPU。
3. 网卡和 GPU 的 NUMA 亲和性会影响多机通信。
4. 多个任务共享同一机器时，不能随便切分 GPU。

一个常见错误是：只申请“4 张 GPU”，但不关心这 4 张 GPU 是否在同一个高速互联域里。

7.5 PCIe-only 多卡机器

PCIe-only 机器中，GPU 之间主要通过 PCIe 和 CPU Root Complex 通信。

优点：

1. 成本相对低。
2. 架构通用。
3. 适合通信较少的任务。

缺点：

1. GPU 间通信带宽较低。
2. 延迟高于 NVLink。
3. 张量并行效率可能差。
4. 多卡训练扩展效率受限。

适合场景：

1. 数据并行为主的小规模训练。
2. LoRA / SFT。
3. 多个独立推理实例。
4. embedding 批处理。
5. 通信不密集的离线任务。

不适合场景：

1. 大规模 tensor parallel。
2. 需要频繁跨卡通信的模型并行。
3. 极致低延迟多卡推理。

7.6 NVLink / NVSwitch 多卡机器

高端训练服务器通常配备 NVLink 和 NVSwitch。

它们的目标是提高 GPU 间通信带宽，降低拓扑不均匀性。

适合场景：

1. 大模型预训练。
2. 张量并行。
3. pipeline 并行。
4. 大模型单机推理。
5. 高并发长上下文推理。

优势：

1. GPU 间通信更快。
2. 多卡模型并行效率更高。
3. 拓扑更均匀。
4. 更适合大模型切分。

限制：

1. 成本高。
2. 功耗高。
3. 供应和运维要求高。
4. 仍然受单机 GPU 数量和总显存限制。

面试中可以说：

如果任务主要是数据并行，PCIe-only 机器可能够用；如果任务需要频繁 GPU 间通信，例如 tensor parallel 或大模型推理切分，NVLink / NVSwitch 多卡机器更合适。

7.7 多机多卡架构

当单机多卡无法满足模型规模或训练时间要求，就需要多机多卡。

多机多卡由多台 GPU 服务器组成，每台服务器内部有多张 GPU，服务器之间通过高速网络连接。

典型结构：

Node 1: GPU x 8

Node 2: GPU x 8

Node 3: GPU x 8

...

Nodes connected by InfiniBand / RoCE / Ethernet

多机多卡的优势：

1. 可以扩展到更多 GPU。
2. 总显存更大。
3. 训练吞吐更高。
4. 可以承载更大模型。
5. 可以支持更大实验规模。

多机多卡的挑战：

1. 机间网络成为关键瓶颈。
2. 分布式训练复杂。
3. 故障概率更高。
4. 调度需要 gang scheduling。
5. checkpoint 和恢复更复杂。
6. 集群利用率更难优化。

多机多卡不是简单把单机复制多份，而是进入分布式系统问题。

7.8 多机训练中的通信模式

多机训练常见通信模式包括：

1. AllReduce。
2. AllGather。
3. ReduceScatter。
4. Broadcast。
5. Point-to-point。

数据并行主要依赖 AllReduce 或 ReduceScatter。

FSDP / ZeRO 会频繁做参数 shard 的 AllGather 和梯度 ReduceScatter。

张量并行可能在每层都需要跨卡通信。

pipeline 并行需要在 stage 之间传递 activation。

因此，通信模式和并行策略强相关。

如果把通信密集的 parallel group 跨机放置，网络压力会很大。

一个经验原则：

尽量把通信最密集的并行维度放在带宽最高、延迟最低的互联范围内。

例如 tensor parallel 通常更适合同机 GPU，data parallel 可以跨机扩展。

7.9 机柜级 GPU 互联

当集群规模继续扩大，单台机器不再是主要视角，机柜成为重要单位。

一个机柜可能包含：

1. 多台 GPU 服务器。
2. 高速交换机。
3. 存储或缓存节点。
4. 管理网络。
5. 电力和散热设施。

机柜级设计要考虑：

1. 机柜内带宽。
2. 机柜间带宽。
3. oversubscription ratio。
4. 故障域。
5. 电力密度。
6. 冷却能力。
7. 维护便利性。

如果机柜内通信快、机柜间通信慢，调度系统就应该尽量把同一个大任务放在同一机柜或相邻机柜内。

这就是拓扑感知调度的意义。

7.10 Fat-tree、Dragonfly 和其他网络拓扑

大规模集群需要网络拓扑。

常见拓扑包括：

1. Fat-tree。
2. Dragonfly。
3. Torus / Mesh。
4. Clos 网络。

你不需要在算法面试里深入讲交换机设计，但要理解核心问题：网络拓扑决定任意两台机器之间的带宽、延迟和拥塞概率。

关注点包括：

1. 是否全双工。
2. 是否有阻塞。
3. 是否 oversubscribe。
4. 多任务并发时是否互相干扰。
5. 某个链路故障影响多大。

对于大模型训练，网络不是普通后台服务的网络。集合通信会产生高度同步的大流量，对拓扑和拥塞非常敏感。

7.11 GPU 集群中的存储位置

GPU 集群不只需要计算和网络，还需要数据和 checkpoint。

常见存储位置：

1. 本地 NVMe。
2. 共享文件系统。
3. 对象存储。
4. 数据湖。
5. 分布式缓存。

不同位置适合不同用途。

本地 NVMe:

1. 速度快。
2. 适合缓存热点数据。
3. 但容量有限，节点故障后数据可能不可用。

共享文件系统:

1. 使用方便。
2. 多节点可访问。
3. 但高并发下可能成为瓶颈。

对象存储:

1. 容量大。
2. 成本相对低。
3. 适合长期存储数据集和 checkpoint。
4. 但直接训练读取可能延迟较高。

AI Infra 常用策略是：对象存储做源数据，本地或分布式缓存做训练热数据。

7.12 GPU 集群中的控制面和数据面

可以把 GPU 集群分成控制面和数据面。

控制面负责管理:

1. 任务提交。
2. 调度决策。
3. 资源分配。
4. 权限认证。
5. 配额管理。
6. 任务状态。
7. 监报告警。

数据面负责实际运行:

1. GPU 计算。
2. GPU 间通信。
3. 数据读取。
4. checkpoint 写入。
5. 模型推理。

控制面故障可能导致新任务无法提交，但不一定影响正在运行的任务。

数据面故障会直接影响训练或推理。

成熟平台要区分这两类故障，并设计不同恢复策略。

7.13 故障域设计

GPU 集群会出现故障。

常见故障包括:

1. 单张 GPU 故障。
2. 整机故障。
3. 网卡故障。
4. 交换机故障。
5. 存储故障。
6. 电力或散热故障。

7. 驱动或系统软件故障。

故障域是指一个故障会影响的范围。

例如：

1. 单卡故障影响单机上的某些任务。
2. 整机故障影响整台机器上的任务。
3. 机柜交换机故障可能影响一个机柜。
4. 共享存储故障可能影响大量任务。

设计集群时要避免单点故障，并让调度系统知道故障域。

例如高优先级训练任务可以跨故障域放置，降低单点故障影响。但通信密集任务又希望放得近。这里存在 trade-off。

7.14 拓扑感知调度

拓扑感知调度是 GPU 集群调度的关键能力。

它不是简单找够 N 张 GPU，而是找“合适的一组 GPU”。

它需要考虑：

1. GPU 是否在同一机器。
2. GPU 之间是否有 NVLink / NVSwitch。
3. GPU 到网卡距离。
4. 机器之间是否在同一机柜。
5. 网络路径是否拥塞。
6. 存储数据是否靠近计算节点。
7. 故障域是否合理。

例如一个 8 卡 tensor parallel 任务，最好分配到同一台 NVSwitch 机器，而不是分散到 8 台机器各一张卡。

一个 256 卡预训练任务，最好分配到网络拓扑紧密的一组节点，而不是横跨整个集群随机分配。

7.15 GPU 集群资源池设计

企业通常会把 GPU 集群划分成不同资源池。

常见资源池：

1. 预训练资源池。
2. SFT 资源池。
3. 推理资源池。
4. 评估资源池。
5. 低优先级资源池。
6. 交互式开发资源池。

为什么要分资源池？

1. 任务画像不同。
2. SLO 不同。
3. 抢占策略不同。
4. 成本核算不同。
5. 故障影响不同。

例如在线推理资源池不能被低优先级训练任务随意抢占。预训练资源池需要大规模连续资源，而交互式开发资源池更关注快速启动。

7.16 GPU 集群容量规划

容量规划要回答：需要多少 GPU、什么类型 GPU、多少网络带宽、多少存储吞吐。

需要考虑：

1. 预训练计划。
2. SFT 实验数量。
3. 推理峰值流量。
4. 评估任务量。
5. Agent 和 RAG 工作负载。
6. 资源利用率目标。
7. 冗余和故障预留。
8. 未来增长。

容量规划不能只按平均负载做，否则峰值会排队严重。

也不能只按峰值买满，否则大量资源闲置。

这就是为什么需要调度、抢占、混部和成本治理。

7.17 一个 GPU 集群架构示例

一个简化企业 GPU 集群可以这样设计：

```
User / Platform Portal / CLI
-> Scheduler / Queue / Quota Manager
-> Resource Pools
    -> Training Pool
        -> NVSwitch GPU Nodes
        -> High-speed Fabric
        -> Checkpoint Storage
    -> Inference Pool
        -> Serving Nodes
        -> Load Balancer
        -> Autoscaler
    -> Eval / Batch Pool
        -> Preemptible GPU Nodes
        -> Object Storage
-> Observability
-> Security / Audit
-> Cost Governance
```

这个架构强调：

1. 训练、推理、评估资源池分开。
2. 调度系统统一管理配额和优先级。
3. 训练池关注拓扑和通信。
4. 推理池关注 SLO 和扩缩容。
5. 评估池可使用低优先级资源。
6. 监控、安全和成本贯穿所有资源池。

7.18 GPU 集群拓扑审计指标与最小 demo

前面讲的是架构直觉。真实平台设计里，还需要把 GPU 集群拓扑写成可检查的数据结构，否则调度系统只能看到“空闲 GPU 数量”，看不到这些 GPU 是否适合同一个任务。

可以把第 i 个集群候选资源组写成：

$g_i=(n_i,u_i,p_i,\ell_i,a_i,r_i,b_i,s_i,f_i,c_i,o_i,z_i)$

其中, n_i 是节点数量, u_i 是单节点 GPU 数, p_i 是 PCIe / NUMA 拓扑, ℓ_i 是 NVLink / NVSwitch 等 scale-up 互联, a_i 是 GPU 到 NIC 的亲 and 性, r_i 是 rack / pod 位置, b_i 是机内和机间带宽, s_i 是存储与 checkpoint 路径, f_i 是故障域, c_i 是电力散热容量, o_i 是观测指标, z_i 是风险。

总 GPU 和总显存可以先粗略写成:

$$G_{\{\mathrm{total}\}} = N_{\{\mathrm{node}\}} G_{\{\mathrm{node}\}}$$

$$M_{\{\mathrm{total}\}} = G_{\{\mathrm{total}\}} M_{\{\mathrm{gpu}\}}$$

其中, N_{node} 是节点数, G_{node} 是单节点 GPU 数, M_{gpu} 是单 GPU 显存容量。这个公式只说明资源上限, 不代表这些显存一定能被一个模型像单卡一样自由使用; 跨 GPU 使用显存要付通信代价。

通信耗时可以用简化模型表达:

$$T_{\{\mathrm{comm}\}} \approx \alpha H + \frac{V}{B_{\{\mathrm{eff}\}}}$$

其中, α 是单跳或一次通信启动延迟, H 是路径跳数, V 是通信量, B_{eff} 是有效带宽。这个式子提醒你: 跨机、跨机柜、拥塞链路和 oversubscription 会同时影响延迟和带宽。

通信放置是否合理, 可以看跨慢链路通信比例:

$$R_{\{\mathrm{slow}\}} = \frac{V_{\{\mathrm{slow}\}}}{V_{\{\mathrm{total}\}}}$$

如果 tensor parallel 或专家并行的大量通信被放到跨机链路上, R_{slow} 会升高, 扩展效率通常会下降。

网络 oversubscription 可以粗略写成:

$$O_{\{\mathrm{net}\}} = \frac{B_{\{\mathrm{down}\}}}{B_{\{\mathrm{up}\}}}$$

其中, B_{down} 是下行接入总带宽, B_{up} 是上行汇聚带宽。 $O_{\mathrm{net}}=1$ 接近无阻塞, $O_{\mathrm{net}}>1$ 表示存在超卖; 是否能接受取决于任务通信模式和并发负载。

故障域影响可以写成:

$$F_{\{\mathrm{rack}\}} = \frac{G_{\{\mathrm{rack}\}}}{G_{\{\mathrm{total}\}}}$$

其中, G_{rack} 是单机柜 GPU 数。这个比例越大, 单个机柜故障的影响越大; 但把任务跨太多故障域铺开, 又可能增加通信成本。

最后, 可以把 GPU 集群门禁写成:

$$G_{\{\mathrm{cluster}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \text{ \& } R_{\{\mathrm{slow}\}} \leq \rho_{\{\mathrm{slow}\}} \text{ \& } O_{\{\mathrm{net}\}} \leq \rho_{\{\mathrm{over}\}} \right]$$

其中, C_j 是第 j 个拓扑审计指标覆盖率, τ_j 是最低覆盖率阈值, ρ_{slow} 是慢链路通信比例阈值, ρ_{over} 是 oversubscription 阈值, P_0 是 P0 级风险数量。

下面这个 0 依赖 demo 演示如何把 GPU 集群拓扑、并行组放置和资源池隔离写成审计规则。它故意构造 1 个完整样本和 16 个坏样本, 让每个关键维度各失败一次。

```
import copy
```

```
METRICS = [
    "scale_up_domain_fit",
    "pcie_numa_locality",
    "nvlink_nvswitch_locality",
    "gpu_nic_affinity",
    "inter_node_fabric_readiness",
    "rack_locality_awareness",
    "oversubscription_awareness",
    "collective_communication_fit",
    "parallel_group_placement",
    "storage_checkpoint_locality",
    "fault_domain_isolation",
    "power_cooling_capacity_fit",
```

```

    "resource_pool_isolation",
    "topology_aware_scheduling",
    "observability_topology_coverage",
    "gpu_cluster_gate",
]

def ring_allreduce_time_s(payload_gib, ranks, bandwidth_gbps):
    bytes_total = payload_gib * (1024 ** 3)
    traffic = 2 * (ranks - 1) / ranks * bytes_total
    return traffic / (bandwidth_gbps * 1_000_000_000)

def total_hbm_tib(nodes, gpus_per_node, hbm_gib_per_gpu):
    return nodes * gpus_per_node * hbm_gib_per_gpu / 1024

def oversubscription_ratio(down_gbps, up_gbps):
    return down_gbps / up_gbps

def rack_fault_blast_radius(gpus_per_rack, total_gpus):
    return gpus_per_rack / total_gpus

def build_cluster_cases():
    complete = {
        "name": "complete",
        "nodes": 16,
        "gpus_per_node": 8,
        "hbm_gib_per_gpu": 80,
        "scale_up": {"domain": "node_nvswitch", "gpus": 8, "fits_task": True},
        "pcie": {"numa_mapped": True, "root_complex_known": True},
        "nvlink": {"present": True, "nvswitch": True, "topology_known": True},
        "gpu_nic": {"affinity_mapped": True, "gdr_ready": True},
        "fabric": {"type": "infiniband_or_roce", "rdma": True, "bandwidth_gbps": 200, "lossless_or_cc": True},
        "rack": {"rack_locality": True, "nodes_per_rack": 4, "rack_spread_limit": 2},
        "network": {"down_gbps": 3200, "up_gbps": 3200, "oversubscription_known": True},
        "collective": {"modeled_ops": ["allreduce", "allgather", "reducescatter"], "nccl_tested": True},
        "parallel_groups": {"tensor_parallel_within_node": True, "data_parallel_cross_node": True},
        "storage": {"checkpoint_nearby": True, "data_cache": True, "write_bandwidth_gbps": 80},
        "fault_domain": {"rack_aware": True, "single_rack_fraction": 0.25, "checkpoint_recoverable": True},
        "power_cooling": {"power_kw_required": 120, "power_kw_available": 150, "cooling_ready": True},
        "resource_pools": {"training": "isolated", "inference": "isolated", "eval": "preemptible"},
        "scheduler": {"topology_aware": True, "gang": True, "quota": True},
        "observability": {"metrics": ["pcie", "nvlink", "nic", "nccl", "rack", "storage"], "traces": True},
        "gate": {"enabled": True},
    }

def bad_case(name, mutator):
    case = copy.deepcopy(complete)
    case["name"] = name
    mutator(case)
    return case

```

```

bad_cases = [
    bad_case("scale_up_domain_missing_bad", lambda c: c["scale_up"].update({"fits_task": False})),
    bad_case("pcie_numa_unknown_bad", lambda c: c["pcie"].update({"numa_mapped": False})),
    bad_case("nvlink_nvswitch_ignored_bad", lambda c: c["nvlink"].update({"topology_known": False})),
    bad_case("gpu_nic_affinity_missing_bad", lambda c: c["gpu_nic"].update({"affinity_mapped": False})),
    bad_case("inter_node_fabric_unready_bad", lambda c: c["fabric"].update({"rdma": False})),
    bad_case("rack_locality_missing_bad", lambda c: c["rack"].update({"rack_locality": False})),
    bad_case("oversubscription_unknown_bad", lambda c: c["network"].update({"oversubscription_known": False})),
    bad_case("collective_comm_unmodeled_bad", lambda c: c["collective"].update({"modeled_ops": []})),
    bad_case("parallel_group_random_bad", lambda c: c["parallel_groups"].update({"tensor_parallel_within_node": False})),
    bad_case("storage_checkpoint_remote_bad", lambda c: c["storage"].update({"checkpoint_nearby": False})),
    bad_case("fault_domain_single_rack_bad", lambda c: c["fault_domain"].update({"single_rack_fraction": 1.0})),
    bad_case("power_cooling_overbooked_bad", lambda c: c["power_cooling"].update({"power_kw_available": 100})),
    bad_case("resource_pool_mixed_bad", lambda c: c["resource_pools"].update({"inference": "shared_with_training"})),
    bad_case("topology_scheduler_disabled_bad", lambda c: c["scheduler"].update({"topology_aware": False})),
    bad_case("observability_no_topology_metrics_bad", lambda c: c["observability"].update({"metrics": ["gpu_utilization"]})),
    bad_case("gpu_cluster_gate_missing_bad", lambda c: c["gate"].update({"enabled": False})),
]
return [complete] + bad_cases

def check_scale_up(case):
    return case["scale_up"]["fits_task"] and case["scale_up"]["gpus"] <= case["gpus_per_node"]

def check_pcie(case):
    return case["pcie"]["numa_mapped"] and case["pcie"]["root_complex_known"]

def check_nvlink(case):
    return case["nvlink"]["present"] and case["nvlink"]["topology_known"]

def check_gpu_nic(case):
    return case["gpu_nic"]["affinity_mapped"] and case["gpu_nic"]["gdr_ready"]

def check_fabric(case):
    f = case["fabric"]
    return f["rdma"] and f["bandwidth_gbps"] >= 100 and f["lossless_or_cc"]

def check_rack(case):
    return case["rack"]["rack_locality"] and case["rack"]["rack_spread_limit"] >= 1

def check_oversubscription(case):
    n = case["network"]
    return n["oversubscription_known"] and oversubscription_ratio(n["down_gbps"], n["up_gbps"]) <= 2.0

def check_collective(case):
    ops = set(case["collective"]["modeled_ops"])
    return {"allreduce", "allgather", "reducescatter"}.issubset(ops) and case["collective"]["nccl_tested"]

```

```

def check_parallel_groups(case):
    return case["parallel_groups"]["tensor_parallel_within_node"] and case["parallel_groups"]["data_parallel_cross_no

def check_storage(case):
    s = case["storage"]
    return s["checkpoint_nearby"] and s["data_cache"] and s["write_bandwidth_gbps"] >= 40

def check_fault_domain(case):
    f = case["fault_domain"]
    return f["rack_aware"] and f["single_rack_fraction"] <= 0.5 and f["checkpoint_recoverable"]

def check_power_cooling(case):
    p = case["power_cooling"]
    return p["power_kw_available"] >= p["power_kw_required"] and p["cooling_ready"]

def check_resource_pools(case):
    pools = case["resource_pools"]
    return pools["training"] == "isolated" and pools["inference"] == "isolated" and pools["eval"] in {"preemptible",

def check_scheduler(case):
    s = case["scheduler"]
    return s["topology_aware"] and s["gang"] and s["quota"]

def check_observability(case):
    needed = {"pcie", "nvlink", "nic", "nccl", "rack", "storage"}
    return needed.issubset(set(case["observability"]["metrics"])) and case["observability"]["traces"]

def check_gate(case):
    return case["gate"]["enabled"]

CHECKS = {
    "scale_up_domain_fit": check_scale_up,
    "pcie_numa_locality": check_pcie,
    "nvlink_nvswitch_locality": check_nvlink,
    "gpu_nic_affinity": check_gpu_nic,
    "inter_node_fabric_readiness": check_fabric,
    "rack_locality_awareness": check_rack,
    "oversubscription_awareness": check_oversubscription,
    "collective_communication_fit": check_collective,
    "parallel_group_placement": check_parallel_groups,
    "storage_checkpoint_locality": check_storage,
    "fault_domain_isolation": check_fault_domain,
    "power_cooling_capacity_fit": check_power_cooling,
    "resource_pool_isolation": check_resource_pools,
    "topology_aware_scheduling": check_scheduler,
    "observability_topology_coverage": check_observability,

```

```

    "gpu_cluster_gate": check_gate,
}

```

```

def audit_gpu_cluster(cases):
    case_failures = {}
    for case in cases:
        failures = [name for name, check in CHECKS.items() if not check(case)]
        case_failures[case["name"]] = failures

    metrics = {}
    for name, check in CHECKS.items():
        metrics[name] = round(sum(int(check(case)) for case in cases) / len(cases), 3)

    failed_cases = [name for name, failures in case_failures.items() if failures]
    return {
        "metrics": metrics,
        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "gpu_cluster_gate_pass": not failed_cases and min(metrics.values()) >= 0.95,
    }

```

```

cases = build_cluster_cases()
case_by_name = {case["name"]: case for case in cases}
complete = case_by_name["complete"]
total_gpus = complete["nodes"] * complete["gpus_per_node"]

```

```

cluster_examples = {
    "total_gpus": total_gpus,
    "total_hbm_tib": round(total_hbm_tib(complete["nodes"], complete["gpus_per_node"], complete["hbm_gib_per_gpu"]),
    "scale_up_allreduce_8gib_s": round(ring_allreduce_time_s(8, 8, 900), 3),
    "scale_out_allreduce_8gib_s": round(ring_allreduce_time_s(8, total_gpus, 200), 3),
    "rack_fault_blast_radius": round(rack_fault_blast_radius(32, total_gpus), 2),
    "oversubscription_ratio": round(oversubscription_ratio(3200, 1600), 2),
}

```

```

smoke = {
    "complete_case_passes": all(check(complete) for check in CHECKS.values()),
    "caught_pcie_numa_gap": not check_pcie(case_by_name["pcie_numa_unknown_bad"]),
    "caught_gpu_nic_gap": not check_gpu_nic(case_by_name["gpu_nic_affinity_missing_bad"]),
    "caught_parallel_group_gap": not check_parallel_groups(case_by_name["parallel_group_random_bad"]),
    "caught_resource_pool_mixing": not check_resource_pools(case_by_name["resource_pool_mixed_bad"]),
}

```

```

audit = audit_gpu_cluster(cases)
print(f"cluster_examples={cluster_examples}")
print(f"smoke={smoke}")
print(f"metrics={audit['metrics']}")
print(f"hard_blocker_count={audit['hard_blocker_count']}")
print(f"failed_cases={audit['failed_cases']}")
print(f"gpu_cluster_gate_pass={audit['gpu_cluster_gate_pass']}")

```

一组典型输出是：

```

cluster_examples={'total_gpus': 128, 'total_hbm_tib': 10.0, 'scale_up_allreduce_8gib_s': 0.017, 'scale_out_allreduce_8gib_s': 0.017, 'rack_fault_blast_radius': 2.0, 'oversubscription_ratio': 2.0}

```



```
smoke={'complete_case_passes': True, 'caught_pcie_numa_gap': True, 'caught_gpu_nic_gap': True, 'caught_parallel_group_g
metrics={'scale_up_domain_fit': 0.941, 'pcie_numa_locality': 0.941, 'nvlink_nvswitch_locality': 0.941, 'gpu_nic_affinity
hard_blocker_count=16
failed_cases=['scale_up_domain_missing_bad', 'pcie_numa_unknown_bad', 'nvlink_nvswitch_ignored_bad', 'gpu_nic_affinity
gpu_cluster_gate_pass=False
```

这个 demo 的重点是把 “GPU 集群架构” 从机房图变成可验证的 placement 规则。单机内要证明 PCIe / NUMA、NVLink / NVSwitch 和 GPU-NIC 亲和性；多机要证明 fabric、collective、rack locality 和 over-subscription；平台层要证明并行组放置、资源池隔离、故障域、电力散热、存储 checkpoint 和拓扑观测都过线。否则即使 GPU 数量足够，训练也可能因为通信、调度或故障域设计失败而跑不稳。

7.19 面试中如何回答 GPU 集群架构

如果面试官问：

如何设计一个支撑大模型训练和推理的 GPU 集群？

可以这样回答：

我会先按任务画像拆资源池。预训练需要大规模连续 GPU、高速网络、checkpoint 存储和拓扑感知调度；SFT 需要快速启动和实验追踪。

底层集群分为单机多卡和多机多卡。单机内要关注 PCIe、NVLink、NVSwitch、GPU 到网卡的 NUMA 亲和性；多机要关注 InfiniBand/RoCE。

调度系统不能只找够 GPU 数量，还要拓扑感知，保证通信密集任务放在高速互联范围内。平台还要提供监控、日志、成本、权限、ch

7.20 常见误区

误区一：GPU 集群就是很多 GPU。

真正的集群还包括网络、存储、调度、电力、散热、监控、安全和成本治理。

误区二：只要 GPU 数量够，训练就能线性加速。

通信、拓扑、数据加载、checkpoint 和调度都会影响扩展效率。

误区三：8 卡机器都一样。

PCIe-only、NVLink、NVSwitch 拓扑差异会显著影响模型并行性能。

误区四：训练和推理可以随便混用同一资源池。

训练和推理的任务画像不同。在线推理需要 SLO，不能被训练任务干扰。

误区五：调度只需要看空闲 GPU 数量。

大模型任务需要拓扑感知、故障域感知、优先级、配额和数据位置感知。

7.21 面试题

题 1：单机多卡和多机多卡的主要区别是什么？

答：单机多卡主要受机内 PCIe、NVLink、NVSwitch、CPU 和网卡拓扑影响，通信延迟较低，适合 SFT、小规模训练和单机推理。多机多卡可以扩展到更大规模，但依赖机间网络、集合通信、拓扑感知调度和容错，复杂度更高。

题 2：为什么 NVSwitch 对大模型训练有价值？

答：NVSwitch 提供更均匀、更高带宽的 GPU 间互联，减少 GPU 间拓扑差异，提升张量并行、pipeline 并行和多卡推理的通信效率。对通信密集的大模型任务尤其有价值。

题 3: 为什么 tensor parallel 通常更适合同机放置?

答: tensor parallel 往往在模型每层都需要跨卡通信, 通信频率高、延迟敏感。把 tensor parallel group 放在同机 NVLink/NVSwitch 范围内, 可以减少跨机网络通信, 提高效率。

题 4: 什么是拓扑感知调度?

答: 拓扑感知调度不是只分配指定数量 GPU, 而是根据 GPU 之间互联、机器位置、机柜网络、GPU 到网卡亲和性、故障域和数据位置, 选择最适合任务的一组资源。它能提高训练效率并降低通信瓶颈。

题 5: 为什么推理资源池最好和训练资源池隔离?

答: 推理是在线服务, 有明确延迟和可用性 SLO; 训练任务通常长时间运行、资源占用大、可能产生网络和 I/O 干扰。如果混用同一资源池, 训练可能影响推理延迟和稳定性。因此通常需要隔离或至少设置严格优先级和资源保障。

7.22 小练习

练习一: 画一个 8 卡单机拓扑图。

要求: 分别画出 PCIe-only 和 NVSwitch 两种情况, 并说明 tensor parallel 性能差异。

练习二: 设计一个 128 卡预训练任务的资源分配策略。

要求: 说明如何选择节点、是否要求同一机柜、如何考虑网络拓扑和故障域。

练习三: 设计一个企业 GPU 资源池。

要求: 至少包含预训练、SFT、推理、评估和交互式开发资源池, 并说明每个资源池的优先级和抢占策略。

练习四: 分析一个多机训练扩展效率低的问题。

要求: 从单机拓扑、机间网络、parallel group 划分、NCCL、数据加载和 checkpoint 角度列排查清单。

7.23 本章小结

本章讲了 GPU 集群架构。

你需要掌握:

1. GPU 集群不是堆卡, 而是计算、互联、网络、存储、调度、电力、散热和治理组成的系统。
2. 单机多卡关注 PCIe、NVLink、NVSwitch、NUMA 和 GPU 到网卡亲和性。
3. 多机多卡关注机间网络、集合通信、拓扑感知调度和容错。
4. tensor parallel 等通信密集任务应优先放在高速互联范围内。
5. 机柜级设计要考虑机柜内外带宽、故障域、电力和散热。
6. GPU 集群通常需要按任务画像划分训练、推理、评估、SFT 和低优先级资源池。
7. 调度系统不能只看 GPU 数量, 还要看拓扑、故障域、优先级、配额和数据位置。

下一章我们会深入讲网络基础: InfiniBand、RoCE、以太网和集合通信。

第 8 章 网络基础: InfiniBand、RoCE、以太网和集合通信

上一章讲了 GPU 集群架构。本章继续往下拆: 多机多卡为什么这么依赖网络? InfiniBand、RoCE、以太网有什么区别? 集合通信为什么会成为大模型训练的关键瓶颈?

很多训练问题表面上是 “GPU 利用率低” “训练 hang” “扩展效率差”, 本质却是网络问题。对于大模型训练, 网络不是普通后端服务里的辅助组件, 而是训练主路径的一部分。

先记住一句话:

多机大模型训练的网络不是只负责传文件，而是在每个 step 中参与参数、梯度、activation 和 optimizer state 的同步。

8.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，资料口径按“AI 集群网络稳定概念”处理，而不是按某个交换机型号、云产品网络规格、网卡 SKU 或 NCCL 环境变量写死。InfiniBand 部分参考 IBTA 对高带宽、低延迟、RDMA fabric 的公开定义；RoCE 部分参考 RDMA over Converged Ethernet 的基本边界，以及 NVIDIA / 以太网数据中心文档中对 lossless fabric、PFC、ECN、拥塞控制和丢包敏感性的工程说明；集合通信部分参考 NVIDIA NCCL 对 AllReduce、AllGather、ReduceScatter、Broadcast、InfiniBand / RoCE / GPUDirect RDMA 和拓扑选择的边界；GPUDirect RDMA 部分延续上一章的 GPU-NIC PCIe 拓扑和直接访问 GPU 显存的边界。

需要注意三点：

1. 本章只讲网络基础和集合通信，不把某个厂商的峰值带宽、交换机端口数、NCCL 参数或云实例网络指标写成通用结论。
2. InfiniBand、RoCE 和普通以太网不是简单“谁更高级”的关系，而是性能、成本、运维复杂度、拥塞控制、团队能力和任务通信强度的取舍。
3. 集合通信瓶颈要和并行策略一起分析。数据并行、FSDP / ZeRO、张量并行和 pipeline 并行对应不同通信频率、通信量和 tail latency 敏感度。

8.1 为什么大模型训练需要高速网络

单机多卡可以依赖 NVLink / NVSwitch 做 GPU 间通信。多机训练时，GPU 分散在不同服务器上，就必须通过机间网络通信。

这些通信可能发生在每个训练 step，甚至每个 Transformer layer。

常见场景：

1. 数据并行同步梯度。
2. FSDP / ZeRO 拉取参数分片。
3. 张量并行交换中间结果。
4. pipeline 并行传递 activation。
5. checkpoint 写入远端存储。
6. 分布式推理交换 KV cache 或中间状态。

如果网络慢，GPU 就会等通信完成。

这会导致：

1. step time 变长。
2. GPU 利用率下降。
3. MFU 下降。
4. 多机扩展效率差。
5. 训练任务偶发 timeout。
6. 某些 rank 拖慢整体任务。

所以，大模型集群里的网络是计算系统的一部分。

8.2 网络指标怎么看

理解训练网络，需要关注几个指标。

8.2.1 带宽

带宽表示单位时间能传多少数据。

常见单位：Gbps、GB/s。

注意 bit 和 byte 的区别：

8 bits = 1 byte

400 Gbps $\approx$ 50 GB/s 理论上限

实际有效带宽会低于理论值，因为有协议开销、拥塞、拓扑、通信模式和实现效率。

8.2.2 延迟

延迟表示一次通信从发起到收到响应需要多久。

小消息通信更敏感于延迟，大消息通信更敏感于带宽。

分布式训练中既有大数据传输，也有频繁同步点，因此带宽和延迟都重要。

8.2.3 抖动

抖动表示延迟或吞吐不稳定。

训练是同步过程，一个 rank 抖动可能拖慢所有 rank。

8.2.4 丢包和重传

网络丢包会导致重传和延迟上升。

在 RoCE 这类基于以太网的 RDMA 场景里，丢包和拥塞控制配置尤其关键。

8.2.5 拓扑和拥塞

即使单链路带宽很高，如果多任务共享同一交换路径，也可能拥塞。

训练任务的集合通信会形成同步大流量，对拓扑非常敏感。

8.3 RDMA 是什么

RDMA 是 Remote Direct Memory Access，远程直接内存访问。

它允许一台机器直接访问另一台机器的内存，尽量绕过远端 CPU，减少拷贝和内核协议栈开销。

RDMA 的价值：

1. 降低延迟。
2. 提高吞吐。
3. 降低 CPU 开销。
4. 提升分布式训练通信效率。

在 GPU 集群中，还会关注 GPUDirect RDMA。

GPUDirect RDMA 的目标是让网卡直接和 GPU 显存交互，减少数据在 CPU 内存中中转。

简化对比：

普通路径：GPU -> CPU memory -> NIC -> network -> NIC -> CPU memory -> GPU

GPUDirect RDMA: GPU memory -> NIC -> network -> NIC -> GPU memory

这对大规模训练非常重要，因为减少一次额外拷贝就能节省大量时间。

8.4 InfiniBand 是什么

InfiniBand 是高性能计算和 AI 集群中常见的高速网络技术。

它强调：

1. 高带宽。
2. 低延迟。

3. RDMA 原生支持。
4. 拥塞控制能力。
5. 面向 HPC 和大规模训练优化。

InfiniBand 常用于大规模 GPU 集群，尤其是对通信敏感的预训练任务。

优势：

1. 性能强。
2. 延迟低。
3. 训练生态成熟。
4. 与 NCCL 等通信库结合较好。

挑战：

1. 成本高。
2. 运维要求高。
3. 设备和生态相对专用。
4. 网络规划和故障排查复杂。

面试中可以说：InfiniBand 更像为高性能计算和同步通信设计的网络，而不是普通业务网络。

8.5 RoCE 是什么

RoCE 是 RDMA over Converged Ethernet，也就是在以太网上跑 RDMA。

它的目标是利用以太网生态，同时获得接近 RDMA 的低延迟和高吞吐。

RoCE 的优势：

1. 可以复用以太网生态。
2. 成本和通用性可能更好。
3. 带宽可以很高。
4. 适合云厂商和企业数据中心演进。

RoCE 的挑战：

1. 对网络配置要求高。
2. 对拥塞控制敏感。
3. 丢包会严重影响性能。
4. PFC、ECN 等配置复杂。
5. 排查难度高。

RoCE 网络要尽量构建 lossless 或低丢包环境，否则 RDMA 性能会受到很大影响。

不要简单认为“以太网 + RDMA = 自动高性能”。RoCE 的工程质量非常依赖配置和运维。

8.6 普通以太网能不能训练大模型

普通以太网也可以训练模型，但适用范围有限。

适合：

1. 小规模训练。
2. 单机多卡为主的任务。
3. 通信不密集的 SFT。
4. 离线 batch 推理。
5. 评估任务。

不适合：

1. 大规模预训练。
2. 强依赖跨机 tensor parallel 的任务。
3. 高频同步通信。

4. 对扩展效率要求很高的任务。

普通以太网的问题：

1. 延迟较高。
2. CPU 协议栈开销大。
3. 拥塞和丢包影响明显。
4. 集合通信效率不如专用高性能网络。

所以是否能用普通以太网，要看任务画像和性能目标。

8.7 InfiniBand、RoCE、以太网对比

可以这样对比：

网络	优点	挑战	适合场景
InfiniBand	高带宽、低延迟、RDMA 成熟	成本高、运维复杂	大规模预训练、HPC、高通信任务
RoCE	复用以太网生态、支持 RDMA	对配置和拥塞控制敏感	大规模云数据中心、AI 集群
普通以太网	通用、便宜、易运维	延迟高、通信效率有限	小规模训练、评估、普通服务

面试里不要说某一种一定最好，要根据规模、预算、团队能力、任务通信强度和现有数据中心条件判断。

8.8 集合通信是什么

集合通信是多个进程或设备之间的集体数据交换。

大模型训练中常见集合通信包括：

1. AllReduce。
2. AllGather。
3. ReduceScatter。
4. Broadcast。
5. Gather。
6. Scatter。

它们不是普通的一对一请求，而是所有 rank 共同参与的同步操作。

这意味着：

1. 最慢的 rank 会拖慢所有 rank。
2. 网络抖动会放大成 step time 抖动。
3. 拓扑不合理会显著降低效率。
4. 通信库实现非常关键。

8.9 AllReduce

AllReduce 的作用是：每个 rank 都有一份数据，先做 reduce 聚合，再把结果分发给所有 rank。

数据并行训练中，常见用法是同步梯度。

例如 4 张卡各自算出梯度：

```
rank0: grad0
rank1: grad1
rank2: grad2
rank3: grad3
```

AllReduce 后，每张卡都得到：

```
grad = grad0 + grad1 + grad2 + grad3
```

再除以 world size，就得到平均梯度。

AllReduce 是数据并行扩展效率的关键。

如果 AllReduce 慢，GPU 就会等梯度同步。

8.10 AllGather 和 ReduceScatter

AllGather 的作用是：每个 rank 持有一部分数据，通信后所有 rank 都获得完整数据。

ReduceScatter 的作用是：先 reduce，再把结果分片分给不同 rank。

FSDP / ZeRO 中经常使用这两类通信。

例如参数被分片存储在不同 GPU 上，某一层计算前可能需要 AllGather 拉齐参数。反向传播后，梯度可能通过 ReduceScatter 分散回各个 rank。

这能降低显存占用，但会增加通信。

这就是典型 trade-off：

用通信换显存。

8.11 NCCL 是什么

NCCL 是 NVIDIA Collective Communications Library，常用于多 GPU 和多机训练的集合通信。

它负责高效实现 AllReduce、AllGather、Broadcast 等操作。

NCCL 会根据硬件拓扑选择通信路径，例如：

1. 单机内走 NVLink / NVSwitch。
2. 机间走 InfiniBand / RoCE。
3. 根据拓扑构建 ring 或 tree。
4. 尽量优化带宽和延迟。

训练中出现 NCCL timeout、NCCL hang、通信占比高，是非常常见的问题。

面试中需要知道 NCCL 是分布式训练通信栈的关键组件。

8.12 Ring 和 Tree 通信

集合通信常见实现包括 ring 和 tree。

Ring AllReduce 可以理解为所有 rank 组成一个环，数据沿环传递和归约。

优点：

1. 带宽利用好。
2. 适合大消息。
3. 实现成熟。

缺点：

1. 延迟可能随 rank 数增加。
2. 对某个慢 rank 敏感。

Tree 通信像树结构汇聚和广播。

优点：

1. 延迟较低。
2. 适合某些小消息或大规模场景。

缺点：

1. 带宽利用和拓扑匹配需要优化。
2. 某些节点可能成为瓶颈。

实际通信库会根据消息大小、拓扑和硬件选择不同算法。

8.13 通信和并行策略的关系

不同并行策略对应不同通信压力。

数据并行：

1. 每个 step 同步梯度。
2. 通信频率相对低。
3. 通信量和参数量相关。

张量并行：

1. 每层可能通信。
2. 延迟敏感。
3. 更适合同机高速互联。

Pipeline 并行：

1. stage 间传 activation。
2. 有 pipeline bubble。
3. 需要平衡各 stage 负载。

FSDP / ZeRO：

1. 参数、梯度、优化器状态分片。
2. 降低显存占用。
3. 增加 AllGather 和 ReduceScatter。

所以选并行策略时，必须结合网络拓扑。

8.14 网络瓶颈的表现

网络瓶颈可能表现为：

1. 多机扩展效率低。
2. GPU 利用率周期性下降。
3. step time 抖动。
4. NCCL timeout。
5. 训练 hang。
6. 某些 rank 明显慢。
7. AllReduce 时间变长。
8. 网络端口错误计数增加。
9. 重传增加。
10. p99 通信延迟升高。

不要只看平均带宽。同步训练对尾延迟非常敏感。

8.15 网络问题排查路径

排查网络问题可以按层走。

第一层：训练指标。

1. step time 是否变长。
2. 通信占比是否升高。
3. 哪些 rank 慢。
4. 是否有 NCCL timeout。

第二层：通信库。

1. NCCL 日志。
2. 通信算法选择。
3. 环或树构建是否合理。
4. 是否走了预期网卡。
5. 是否启用 GPUDirect RDMA。

第三层：节点和网卡。

1. 网卡带宽。
2. 错误包。
3. 重传。
4. PCIe 带宽。
5. GPU 到网卡亲和性。

第四层：交换机和拓扑。

1. 链路拥塞。
2. oversubscription。
3. 交换机端口错误。
4. 跨机柜路径。
5. 多任务流量干扰。

第五层：调度。

1. 任务是否被分配得太分散。
2. 通信密集 group 是否跨机柜。
3. 是否和其他大任务共享瓶颈链路。

网络排查需要训练平台、集群网络和调度系统共同配合。

8.16 如何优化通信

常见优化方向：

1. 使用更高性能网络，例如 InfiniBand 或高质量 RoCE。
2. 启用 GPUDirect RDMA。
3. 优化 NCCL 参数。
4. 拓扑感知调度。
5. 把 tensor parallel 放在同机。
6. 减少跨机通信密集操作。
7. 通信计算重叠。
8. 梯度累积，降低同步频率。
9. 压缩通信数据。
10. 避免多个大任务争抢同一链路。

这些优化不是越多越好，要结合任务画像。

例如梯度累积可以减少同步频率，但会改变有效 batch size 和训练动态，需要算法侧一起评估。

8.17 网络与调度的关系

网络优化不只是网络团队的事，调度也很关键。

调度系统如果随机分配节点，可能把一个训练任务分散到网络距离很远的位置。

结果是：

1. 通信路径长。
2. 跨交换机流量多。
3. 和其他任务互相干扰。

4. 扩展效率下降。

拓扑感知调度可以让任务优先使用：

1. 同机 GPU。
2. 同机柜节点。
3. 同一网络 fabric 下的节点。
4. 拥塞较低的路径。
5. GPU 到网卡亲和性更好的组合。

这就是为什么 AI Infra 里，调度系统必须理解硬件拓扑。

8.18 网络通信审计指标与最小 demo

前面讲的是网络概念。真实排查里，还要把“网络好不好”变成可检查指标，否则很容易只盯着一个 400 Gbps 或 800 Gbps 的纸面带宽。

可以把第 i 个网络通信样本写成：

$$n_i = (b_i, \ell_i, j_i, r_i, g_i, f_i, c_i, a_i, t_i, e_i, s_i, z_i)$$

其中， b_i 是 bandwidth， ℓ_i 是 latency， j_i 是 jitter， r_i 是 RDMA / GPUDirect RDMA 状态， g_i 是 collective group， f_i 是 fabric 类型， c_i 是 congestion / lossless 配置， a_i 是通信算法， t_i 是拓扑路径， e_i 是错误和重传， s_i 是调度放置， z_i 是风险。

带宽单位先要说清楚：

$$B_{\mathrm{GiB/s}} = \frac{B_{\mathrm{Gbps}}}{8} \cdot 10^9 \cdot 2^{30}$$

其中，Gbps 是十进制 bit/s，GiB/s 是二进制 GiB/s。面试和排障中经常有人把 bit 和 byte 混用，导致估算差 8 倍。

大消息传输的有效耗时可以写成：

$$T_{\mathrm{xfer}} = \frac{D}{\eta B_{\mathrm{line}}}$$

其中， D 是传输数据量， B_{line} 是链路理论带宽， η 是有效带宽系数。协议开销、拥塞、拓扑、通信库和消息大小都会让 $\eta < 1$ 。

通信耗时可以粗略拆成：

$$T_{\mathrm{msg}} \approx \alpha H + \frac{D}{B_{\mathrm{eff}}} + T_{\mathrm{queue}} + T_{\mathrm{retx}}$$

其中， α 是单跳或启动延迟， H 是路径跳数， D 是数据量， B_{eff} 是有效带宽， T_{queue} 是拥塞排队， T_{retx} 是丢包重传带来的额外时间。

Ring AllReduce 的每个 rank 近似通信量可以写成：

$$V_{\mathrm{ring}} \approx 2 \frac{P-1}{P} D$$

其中， P 是 rank 数， D 是每个 rank 参与归约的数据大小。这个公式帮助你估算“参数越大、rank 越多、链路越慢，AllReduce 越贵”。

AllGather 和 ReduceScatter 的每个 rank 近似通信量可以写成：

$$V_{\mathrm{gather}} \approx \frac{P-1}{P} D$$

其中， D 是 gather 后完整张量大小。FSDP / ZeRO 省显存，但会把这类通信放进更多 layer 或 micro-step。

同步训练对 tail latency 很敏感，可以用 jitter ratio 和通信占比辅助判断：

$$J_{\mathrm{lat}} = \frac{L_{\mathrm{p99}}}{L_{\mathrm{p50}}}$$

$$R_{\mathrm{comm}} = \frac{T_{\mathrm{comm}}}{T_{\mathrm{step}}}$$

丢包或重传风险可以写成：

$$R_{\mathrm{retx}} = \frac{N_{\mathrm{retx}}}{N_{\mathrm{packet}}}$$

最后，可以把网络通信门禁写成：

$$G_{\{\mathrm{net}\}} = \mathbf{1} \setminus \left[\min_j C_j \geq \tau_j \wedge R_{\{\mathrm{comm}\}} \leq \rho_{\{\mathrm{comm}\}} \wedge J_{\{\mathrm{net}\}} \right]$$

其中, C_j 是第 j 个网络审计指标覆盖率, R_{comm} 是通信占比, J_{lat} 是延迟抖动比例, R_{retx} 是重传比例, P_0 是 P0 级风险数量。

下面这个 0 依赖 demo 演示如何把网络、RDMA、collective、NCCL、rank skew 和调度放置写成审计规则。它故意构造 1 个完整样本和 16 个坏样本, 让每个关键维度各失败一次。

```
import copy
```

```
METRICS = [
    "bandwidth_unit_accounting",
    "latency_jitter_tracking",
    "rdma_capability_fit",
    "gpudirect_rdma_path",
    "infiniband_fabric_readiness",
    "roce_congestion_losslessness",
    "ethernet_fallback_scope",
    "collective_operation_modeling",
    "nccl_topology_runtime_fit",
    "allreduce_cost_estimation",
    "allgather_reducescatter_cost",
    "rank_straggler_detection",
    "packet_error_retransmit_tracking",
    "topology_congestion_awareness",
    "scheduler_network_locality",
    "network_communication_gate",
]
```

```
def gbps_to_gib_s(gbps):
    return gbps * 1_000_000_000 / 8 / (1024 ** 3)

def transfer_time_s(gib, gbps, efficiency):
    bytes_total = gib * (1024 ** 3)
    bytes_per_s = gbps * 1_000_000_000 / 8 * efficiency
    return bytes_total / bytes_per_s

def ring_allreduce_time_s(gib, ranks, gbps, efficiency):
    traffic_gib = 2 * (ranks - 1) / ranks * gib
    return transfer_time_s(traffic_gib, gbps, efficiency)

def allgather_time_s(gib_after_gather, ranks, gbps, efficiency):
    traffic_gib = (ranks - 1) / ranks * gib_after_gather
    return transfer_time_s(traffic_gib, gbps, efficiency)

def jitter_ratio(p99_us, p50_us):
    return p99_us / p50_us

def retransmit_rate(retransmits, packets):
    return retransmits / packets
```

```

def build_network_cases():
    complete = {
        "name": "complete",
        "bandwidth": {"gbps": 400, "unit_checked": True, "effective_efficiency": 0.8},
        "latency": {"p50_us": 8, "p99_us": 18, "jitter_tracked": True},
        "rdma": {"enabled": True, "validated": True},
        "gdr": {"enabled": True, "gpu_nic_affinity": True},
        "infiniband": {"used_for_pretraining": True, "subnet_health": True, "link_width_ok": True},
        "roce": {"pfc": True, "ecn": True, "cc": True, "loss_rate": 0.0},
        "ethernet": {"scope": "eval_and_small_sft", "large_pretraining": False},
        "collectives": {"ops": ["allreduce", "allgather", "reducescatter"], "message_sizes_known": True},
        "nccl": {"topology_logged": True, "expected_nics": True, "gdr_level_checked": True},
        "allreduce": {"modeled": True, "payload_gib": 8, "ranks": 64},
        "sharded_ops": {"allgather_modeled": True, "reducescatter_modeled": True},
        "rank_times_s": [3.0, 3.05, 3.1, 3.02],
        "packets": {"errors": 0, "retransmits": 5, "total": 1_000_000, "tracked": True},
        "topology": {"rack_local": True, "oversubscription_known": True, "congestion_hotspots": []},
        "scheduler": {"network_locality": True, "tp_within_fast_domain": True},
        "gate": {"enabled": True},
    }

def bad_case(name, mutator):
    case = copy.deepcopy(complete)
    case["name"] = name
    mutator(case)
    return case

bad_cases = [
    bad_case("bandwidth_unit_confused_bad", lambda c: c["bandwidth"].update({"unit_checked": False})),
    bad_case("latency_jitter_ignored_bad", lambda c: c["latency"].update({"p99_us": 120, "jitter_tracked": False})),
    bad_case("rdma_missing_bad", lambda c: c["rdma"].update({"enabled": False})),
    bad_case("gdr_path_missing_bad", lambda c: c["gdr"].update({"enabled": False})),
    bad_case("ib_fabric_unverified_bad", lambda c: c["infiniband"].update({"subnet_health": False})),
    bad_case("roce_no_pfc_ecn_bad", lambda c: c["roce"].update({"pfc": False, "ecn": False, "loss_rate": 0.02})),
    bad_case("ethernet_used_for_pretraining_bad", lambda c: c["ethernet"].update({"large_pretraining": True})),
    bad_case("collective_unmodeled_bad", lambda c: c["collectives"].update({"ops": []})),
    bad_case("nccl_topology_unknown_bad", lambda c: c["nccl"].update({"topology_logged": False})),
    bad_case("allreduce_cost_missing_bad", lambda c: c["allreduce"].update({"modeled": False})),
    bad_case("allgather_reducescatter_ignored_bad", lambda c: c["sharded_ops"].update({"allgather_modeled": False})),
    bad_case("rank_straggler_hidden_bad", lambda c: c.update({"rank_times_s": [3.0, 3.1, 4.8, 3.0]})),
    bad_case("packet_errors_untracked_bad", lambda c: c["packets"].update({"tracked": False})),
    bad_case("topology_congestion_ignored_bad", lambda c: c["topology"].update({"oversubscription_known": False})),
    bad_case("scheduler_network_blind_bad", lambda c: c["scheduler"].update({"network_locality": False})),
    bad_case("network_gate_missing_bad", lambda c: c["gate"].update({"enabled": False})),
]

return [complete] + bad_cases

def check_bandwidth(case):
    b = case["bandwidth"]
    return b["unit_checked"] and b["gbps"] > 0 and 0 < b["effective_efficiency"] <= 1

```

```

def check_latency(case):
    l = case["latency"]
    return l["jitter_tracked"] and jitter_ratio(l["p99_us"], l["p50_us"]) <= 4

def check_rdma(case):
    return case["rdma"]["enabled"] and case["rdma"]["validated"]

def check_gdr(case):
    return case["gdr"]["enabled"] and case["gdr"]["gpu_nic_affinity"]

def check_infiniband(case):
    ib = case["infiniband"]
    return ib["used_for_pretraining"] and ib["subnet_health"] and ib["link_width_ok"]

def check_roce(case):
    r = case["roce"]
    return r["pfc"] and r["ecn"] and r["cc"] and r["loss_rate"] <= 0.001

def check_ethernet(case):
    e = case["ethernet"]
    return e["scope"] in {"eval_and_small_sft", "batch_inference"} and not e["large_pretraining"]

def check_collectives(case):
    ops = set(case["collectives"]["ops"])
    return {"allreduce", "allgather", "reducescatter"}.issubset(ops) and case["collectives"]["message_sizes_known"]

def check_nccl(case):
    n = case["nccl"]
    return n["topology_logged"] and n["expected_nics"] and n["gdr_level_checked"]

def check_allreduce(case):
    a = case["allreduce"]
    return a["modeled"] and a["payload_gib"] > 0 and a["ranks"] > 1

def check_sharded_ops(case):
    s = case["sharded_ops"]
    return s["allgather_modeled"] and s["reducescatter_modeled"]

def check_rank_straggler(case):
    times = case["rank_times_s"]
    return max(times) / (sum(times) / len(times)) <= 1.2

def check_packets(case):
    p = case["packets"]

```

```

    return p["tracked"] and p["errors"] == 0 and retransmit_rate(p["retransmits"], p["total"]) <= 0.001

def check_topology(case):
    t = case["topology"]
    return t["rack_local"] and t["oversubscription_known"] and not t["congestion_hotspots"]

def check_scheduler(case):
    s = case["scheduler"]
    return s["network_locality"] and s["tp_within_fast_domain"]

def check_gate(case):
    return case["gate"]["enabled"]

CHECKS = {
    "bandwidth_unit_accounting": check_bandwidth,
    "latency_jitter_tracking": check_latency,
    "rdma_capability_fit": check_rdma,
    "gpudirect_rdma_path": check_gdr,
    "infiniband_fabric_readiness": check_infiniband,
    "roce_congestion_losslessness": check_roce,
    "ethernet_fallback_scope": check_ethernet,
    "collective_operation_modeling": check_collectives,
    "nccl_topology_runtime_fit": check_nccl,
    "allreduce_cost_estimation": check_allreduce,
    "allgather_reducescatter_cost": check_sharded_ops,
    "rank_straggler_detection": check_rank_straggler,
    "packet_error_retransmit_tracking": check_packets,
    "topology_congestion_awareness": check_topology,
    "scheduler_network_locality": check_scheduler,
    "network_communication_gate": check_gate,
}

def audit_network(cases):
    case_failures = {}
    for case in cases:
        failures = [name for name, check in CHECKS.items() if not check(case)]
        case_failures[case["name"]] = failures

    metrics = {}
    for name, check in CHECKS.items():
        metrics[name] = round(sum(int(check(case)) for case in cases) / len(cases), 3)

    failed_cases = [name for name, failures in case_failures.items() if failures]
    return {
        "metrics": metrics,
        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "network_gate_pass": not failed_cases and min(metrics.values()) >= 0.95,
    }

```

```

cases = build_network_cases()
case_by_name = {case["name"]: case for case in cases}
complete = case_by_name["complete"]
network_examples = {
    "gbps_400_to_gib_s": round(gbps_to_gib_s(400), 2),
    "send_16gib_at_400gbps_80eff_s": round(transfer_time_s(16, 400, 0.8), 3),
    "ring_allreduce_8gib_64ranks_200gbps_75eff_s": round(ring_allreduce_time_s(8, 64, 200, 0.75), 3),
    "allgather_8gib_64ranks_200gbps_75eff_s": round(allgather_time_s(8, 64, 200, 0.75), 3),
    "jitter_ratio": round(jitter_ratio(complete["latency"]["p99_us"], complete["latency"]["p50_us"]), 2),
    "retransmit_rate": round(retransmit_rate(complete["packets"]["retransmits"], complete["packets"]["total"]), 6),
}

smoke = {
    "complete_case_passes": all(check(complete) for check in CHECKS.values()),
    "caught_unit_confusion": not check_bandwidth(case_by_name["bandwidth_unit_confused_bad"]),
    "caught_roce_loss_gap": not check_roce(case_by_name["roce_no_pfc_ecn_bad"]),
    "caught_rank_straggler": not check_rank_straggler(case_by_name["rank_straggler_hidden_bad"]),
    "caught_scheduler_blindness": not check_scheduler(case_by_name["scheduler_network_blind_bad"]),
}

audit = audit_network(cases)
print(f"network_examples={network_examples}")
print(f"smoke={smoke}")
print(f"metrics={audit['metrics']}")
print(f"hard_blocker_count={audit['hard_blocker_count']}")
print(f"failed_cases={audit['failed_cases']}")
print(f"network_gate_pass={audit['network_gate_pass']}")

```

一组典型输出是：

```

network_examples={'gbps_400_to_gib_s': 46.57, 'send_16gib_at_400gbps_80eff_s': 0.429, 'ring_allreduce_8gib_64ranks_200gbps_75eff_s': 0.06}
smoke={'complete_case_passes': True, 'caught_unit_confusion': True, 'caught_roce_loss_gap': True, 'caught_rank_straggler': True, 'caught_scheduler_blindness': True}
metrics={'bandwidth_unit_accounting': 0.941, 'latency_jitter_tracking': 0.941, 'rdma_capability_fit': 0.941, 'gpudirect_capability_fit': 0.941, 'rdma_path_missing_bad': False, 'gdr_path_missing_bad': False}
hard_blocker_count=16
failed_cases=['bandwidth_unit_confused_bad', 'latency_jitter_ignored_bad', 'rdma_missing_bad', 'gdr_path_missing_bad']
network_gate_pass=False

```

这个 demo 的重点是把“网络慢”拆成可定位的证据链。先确认 bit / byte 和有效带宽，再看 latency、jitter、RDMA、GPUDirect RDMA、InfiniBand / RoCE / Ethernet 适用范围；然后把 AllReduce、AllGather、ReduceScatter 的通信量估出来，结合 NCCL 拓扑、rank straggler、错误包、重传、拥塞路径和调度放置一起判断。只说“网卡是 400G，所以网络没问题”在大模型训练排障里是不够的。

8.19 面试中如何回答网络问题

如果面试官问：

为什么多机训练扩展效率很差？你怎么排查？

可以这样回答：

我会先看 step time 分解，确认通信占比是否升高。然后看 NCCL 日志和 profiler，定位是 AllReduce、AllGather 还是 ReduceScatter。

接着看 rank 间差异，判断是否有 straggler。再检查网络层，包括网卡带宽、错误包、重传、RDMA/GPUDirect 是否生效、GPU 到网卡延迟。

最后看调度和并行策略：tensor parallel 是否跨机，任务是否被分散到拓扑较远的节点，是否和其他大任务共享瓶颈链路。优化方向包括调整拓扑、调整并行策略、调整调度策略。

如果面试官问：

InfiniBand 和 RoCE 有什么区别？

可以回答：

InfiniBand 是面向高性能计算的专用高速网络，RDMA 支持成熟，低延迟、高带宽，常用于大规模训练集群，但成本和运维复杂度较高。

RoCE 是在以太网上承载 RDMA，能复用以太网生态，成本和通用性可能更好，但对拥塞控制、PFC/ECN、丢包和运维配置非常敏感。两

8.20 常见误区

误区一：网络只影响数据加载，不影响训练计算。

多机训练中，网络参与梯度、参数和 activation 同步，是训练主路径的一部分。

误区二：带宽高就一定快。

延迟、抖动、拥塞、丢包、拓扑、通信算法和 tail latency 都会影响训练。

误区三：RoCE 是普通以太网，不需要特殊配置。

RoCE 对拥塞控制和低丢包要求很高，配置不好性能会很差。

误区四：NCCL timeout 一定是代码 bug。

也可能是网络抖动、某个 rank 慢、网卡故障、拓扑问题或资源干扰。

误区五：调度和网络无关。

调度决定任务放在哪些节点上，直接影响通信路径、拥塞和扩展效率。

8.21 面试题

题 1：为什么多机训练需要高速网络？

答：多机训练中，不同 GPU 分布在不同服务器上，需要通过网络同步梯度、参数分片、中间 activation 或 pipeline 数据。这些通信可能发生在每个 step，甚至每层。如果网络慢，GPU 会等待通信，导致 step time 变长、MFU 下降和扩展效率变差。

题 2：RDMA 和 GPUDirect RDMA 的价值是什么？

答：RDMA 允许远程直接内存访问，减少 CPU 和内核协议栈开销，降低延迟、提高吞吐。GPUDirect RDMA 进一步让网卡直接访问 GPU 显存，减少 CPU 内存中转，对大规模 GPU 通信非常重要。

题 3：AllReduce 在数据并行中做什么？

答：每个 GPU 计算本地 mini-batch 梯度后，需要把各 GPU 的梯度聚合并同步。AllReduce 会对所有 rank 的梯度做 reduce，并把结果分发回所有 rank，让每个 GPU 使用相同的平均梯度更新参数。

题 4：FSDP / ZeRO 为什么会增加通信？

答：FSDP / ZeRO 通过分片参数、梯度和优化器状态降低显存占用。但计算某层时可能需要 AllGather 拉取参数，反向传播后需要 ReduceScatter 分发梯度。这是用通信换显存的 trade-off。

题 5：如何优化多机通信？

答：可以从网络、通信库、并行策略和调度四层优化。网络层使用高性能 IB/RoCE 和 GPUDirect RDMA；通信库层优化 NCCL 参数；并行策略层减少跨机 tensor parallel、做通信计算重叠；调度层做拓扑感知，避免把通信密集任务分散到远距离节点。

8.22 小练习

练习一：画出数据并行 AllReduce 的流程。

要求：用 4 个 rank 举例，说明每个 rank 计算本地梯度后如何得到平均梯度。

练习二：比较 InfiniBand、RoCE 和普通以太网。

要求：从带宽、延迟、RDMA、成本、运维复杂度和适用场景分析。

练习三：分析 NCCL timeout。

假设一个 64 卡训练任务偶发 NCCL timeout，请列出至少 8 个可能原因，并按训练指标、NCCL、网卡、交换机、调度分层排查。

练习四：设计一个网络监控面板。

要求：包含网卡吞吐、错误包、重传、RDMA 状态、NCCL 通信时间、rank 差异、交换机拥塞和跨机柜流量。

8.23 本章小结

本章讲了 AI 集群网络基础。

你需要掌握：

1. 多机训练中，网络是训练主路径，不只是传文件。
2. 网络指标包括带宽、延迟、抖动、丢包、重传、拓扑和拥塞。
3. RDMA 和 GPUDirect RDMA 可以降低通信开销，提高 GPU 间跨机通信效率。
4. InfiniBand 性能强但成本和运维复杂度高。
5. RoCE 复用以太网生态，但对拥塞控制和低丢包要求高。
6. 普通以太网适合小规模或低通信任务，不适合高强度大规模预训练。
7. 集合通信包括 AllReduce、AllGather、ReduceScatter 等，是分布式训练的关键。
8. NCCL 是多 GPU 集合通信的重要库。
9. 网络瓶颈要从训练指标、通信库、网卡、交换机、拓扑和调度多层排查。

下一章我们会讲存储体系：本地盘、共享文件系统、对象存储和数据湖。

第 9 章 存储体系：本地盘、共享文件系统、对象存储和数据湖

上一章讲了网络。网络决定多机 GPU 之间能不能高效同步，存储则决定数据、checkpoint、模型权重和各种 artifact 能不能稳定、高效、可追踪地被读写。

很多训练慢、训练抖动、checkpoint 保存失败、模型加载慢的问题，本质都是存储问题。

先记住一句话：

AI Infra 的存储系统不只是放文件，而是支撑训练数据供给、checkpoint 恢复、模型版本管理、评估结果留存和多租户数据治理的基础设施。

9.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，资料口径按“AI 训练与推理存储体系的稳定抽象”处理，而不是按某个云产品、某个共享文件系统、某个对象存储 API 或某个数据湖表格式写死。高性能训练存储部分参考 FSx for Lustre / Lustre 类共享并行文件系统对高性能计算和机器学习 workload 的边界；对象存储部分参考 Amazon S3 对 bucket / object、容量、持久性、一致性和访问控制的通用抽象；GPU 数据路径部分参考 NVIDIA GPUDirect Storage 对 storage 和 GPU memory 直接 DMA、减少 CPU bounce buffer 的边界；数据格式和数据湖部分参考 Apache Parquet 的列式格式、Apache Iceberg 的 metadata / snapshot / manifest 这类表格式治理口径。

需要注意三点：

1. 本章只讲存储体系分层、访问模式、checkpoint、artifact、数据湖和治理，不把下一章专门的 checkpoint 生命周期细节提前展开。

2. 本地 NVMe、共享文件系统、对象存储和数据湖不是互相替代关系，而是分别承担热缓存、POSIX 易用性、权威源 / 归档和数据治理。
3. 存储瓶颈不能只看容量和平均吞吐，还要看 metadata、IOPS、小文件、缓存命中、恢复时间、一致性、权限、生命周期和成本。

9.1 大模型为什么依赖存储

大模型系统里，存储承载很多类型的数据。

包括：

1. 原始训练数据。
2. 清洗后的训练数据。
3. 数据索引和 metadata。
4. tokenizer 产物。
5. checkpoint。
6. 模型权重。
7. adapter / LoRA。
8. 量化模型。
9. eval report。
10. 训练日志。
11. 推理日志。
12. RAG 文档和向量索引。
13. Agent trace。

这些数据的访问模式不同。

例如：

1. 训练数据需要高吞吐顺序或随机读取。
2. checkpoint 需要大文件高吞吐写入和恢复读取。
3. 模型权重需要部署时快速加载。
4. 日志需要持续写入和查询。
5. RAG 文档需要更新、索引和权限过滤。

所以 AI 存储体系不能只用一种存储解决所有问题。

9.2 存储指标怎么看

常见指标包括：

1. 容量：能存多少数据。
2. 吞吐：单位时间能读写多少数据。
3. IOPS：每秒能处理多少 I/O 操作。
4. 延迟：一次读写要多久。
5. 并发：能支持多少任务同时读写。
6. 可用性：存储服务是否稳定。
7. 持久性：数据丢失概率。
8. 一致性：写入后读取能否看到最新数据。
9. 成本：每 TB 存储和访问成本。
10. 权限：谁能读写哪些数据。

不同任务关注指标不同。

训练数据读取更关注吞吐和并发。

小文件场景更关注 IOPS 和 metadata 性能。

checkpoint 更关注大文件写入吞吐、恢复速度和可靠性。

对象归档更关注容量和成本。

9.3 本地盘

本地盘通常指机器本地的 SSD / NVMe 存储。

优点：

1. 读写速度快。
2. 延迟低。
3. 不依赖远端存储网络。
4. 适合做缓存。
5. 可以缓解共享存储压力。

缺点：

1. 容量有限。
2. 节点故障后数据可能不可用。
3. 数据分布在不同机器上，管理复杂。
4. 不适合作为唯一长期存储。

适合场景：

1. 训练数据本地缓存。
2. 临时数据。
3. 解压后的数据 shard。
4. 中间结果。
5. 热点模型权重缓存。

本地盘常见用法是：远端对象存储或数据湖保存源数据，本地 NVMe 保存训练热数据。

9.4 共享文件系统

共享文件系统允许多台机器像访问本地目录一样访问同一套文件。

常见用途：

1. 共享代码。
2. 共享数据集。
3. 保存 checkpoint。
4. 保存日志和中间结果。
5. 多节点访问同一个路径。

优点：

1. 使用简单。
2. POSIX 语义友好。
3. 多节点共享方便。
4. 改造现有训练脚本成本低。

缺点：

1. 高并发下 metadata 压力大。
2. 大规模训练可能打爆吞吐。
3. 小文件过多性能差。
4. 故障影响范围大。
5. 成本和运维复杂度可能高。

共享文件系统适合易用性要求高的场景，但大规模预训练不能盲目依赖单一共享目录。

9.5 对象存储

对象存储以 object 为单位存储数据，常见形式是 bucket + key。

它不完全等同于文件系统。

优点：

1. 容量大。
2. 成本相对低。
3. 持久性强。
4. 易于跨服务访问。
5. 适合长期保存数据集、checkpoint 和 artifact。

缺点：

1. 延迟通常高于本地盘。
2. POSIX 语义不完整。
3. 小文件访问效率可能低。
4. 直接用于高频训练读取可能不理想。
5. 列目录和 metadata 操作可能昂贵或慢。

适合场景：

1. 原始数据保存。
2. 训练数据归档。
3. checkpoint 长期存储。
4. 模型权重仓库后端。
5. eval report 和 artifact 保存。

常见架构是：对象存储做权威源，本地缓存或分布式缓存做训练加速。

9.6 数据湖

数据湖是企业统一存储大规模结构化、半结构化和非结构化数据的平台。

它通常包含：

1. 原始数据层。
2. 清洗数据层。
3. 特征或训练数据层。
4. metadata。
5. 数据血缘。
6. 数据质量。
7. 权限和合规。

对于大模型，数据湖可以管理：

1. 网页文本。
2. 代码数据。
3. 文档数据。
4. 多模态数据。
5. 对话数据。
6. 评估数据。
7. RAG 知识库数据。

数据湖的价值不只是存储容量，而是数据治理。

它回答：

1. 这批数据从哪里来？
2. 是否有权限使用？
3. 是否包含敏感信息？
4. 清洗规则是什么？
5. 训练使用的是哪个版本？
6. 数据质量是否变化？

9.7 数据格式对训练性能的影响

训练数据不只是“存在那里”，格式也很重要。

常见格式包括：

1. JSON / JSONL。
2. Parquet。
3. Arrow。
4. WebDataset / tar shards。
5. TFRecord。
6. 自定义二进制格式。

JSONL 易读、调试方便，但解析成本高，体积可能较大。

Parquet 适合列式分析和数据工程，但训练读取要看访问模式。

WebDataset 或 shard 格式适合大规模顺序读取，减少小文件问题。

自定义二进制格式可能性能好，但可维护性和工具生态要评估。

训练系统常见优化是把原始数据离线预处理成训练友好的 shard，减少训练时动态解析。

9.8 小文件问题

小文件是训练存储中的常见坑。

如果一个数据集有几千万个小文件，会导致：

1. metadata 查询压力巨大。
2. 打开文件开销高。
3. 共享文件系统压力大。
4. 对象存储请求成本高。
5. dataloader 等待 I/O。

解决方法：

1. 合并成 shard。
2. 使用 tar / WebDataset。
3. 使用 Parquet / Arrow。
4. 建立索引文件。
5. 使用本地缓存。
6. 预取和顺序读取。

面试中如果提到训练数据供给，一定要想到小文件问题。

9.9 训练数据供给链路

一个典型训练数据供给链路：

Raw Data

- > Data Lake / Object Storage
- > Cleaning / Dedup / Filtering
- > Tokenization / Packing
- > Training Shards
- > Local Cache / Distributed Cache
- > Dataloader
- > GPU

每一层都可能成为瓶颈。

例如：

1. 清洗太慢，数据版本延迟。
2. tokenization 在线执行，CPU 成为瓶颈。
3. shard 太小，小文件问题严重。
4. 缓存命中率低，远端存储压力大。
5. dataloader worker 不够。
6. CPU 到 GPU 拷贝慢。

成熟平台会尽量把重预处理放到离线阶段，让训练阶段只做高效读取。

9.10 Checkpoint 存储需求

Checkpoint 是训练恢复的关键。

它通常包含：

1. 模型参数。
2. 优化器状态。
3. scheduler 状态。
4. random seed。
5. dataloader 状态。
6. 分布式 rank 信息。
7. 训练 step。
8. 配置和 metadata。

大模型 checkpoint 可能非常大。

存储系统需要支持：

1. 高吞吐写入。
2. 分片保存。
3. 快速恢复读取。
4. 元数据一致性。
5. 保留策略。
6. 权限控制。
7. 跨区域或跨集群复制。

checkpoint 不是随便把文件写出去就完了，后续恢复是否可靠同样重要。

9.11 模型权重存储和加载

推理平台上线模型时，需要加载模型权重。

如果模型很大，加载时间可能成为发布和扩容瓶颈。

常见问题：

1. 模型权重文件太大。
2. 多个实例同时拉取，打爆对象存储。
3. 冷启动慢。
4. 本地缓存失效。
5. 权重版本混乱。
6. tokenizer、config 和权重不匹配。

优化方法：

1. 节点本地缓存模型权重。
2. 分层缓存。
3. 预热模型。
4. 分片加载。
5. 模型仓库统一管理版本。
6. 发布前做完整性校验。

模型加载是推理平台 SLO 的一部分，不能只看请求处理时延。

9.12 Artifact 存储

大模型系统有很多 artifact。

包括：

1. dataset version。
2. training config。
3. checkpoint。
4. final model。
5. tokenizer。
6. LoRA / adapter。
7. quantized model。
8. eval report。
9. deployment package。
10. trace sample。

Artifact 存储要解决：

1. 版本管理。
2. 血缘关系。
3. 权限控制。
4. 生命周期管理。
5. 可复现性。
6. 审计。

如果 artifact 管理混乱，团队很快会不知道哪个模型来自哪个数据和配置。

9.13 存储一致性和元数据

存储系统不只是数据块，还包括 metadata。

metadata 包括：

1. 文件名。
2. 大小。
3. 修改时间。
4. 版本。
5. checksum。
6. owner。
7. 权限。
8. 数据血缘。

训练和部署非常依赖 metadata 正确性。

例如：

1. checkpoint 文件写了一半但 metadata 标记完成，会导致恢复失败。
2. 模型权重版本和 config 不匹配，会导致推理错误。
3. 数据集版本指向错了，会导致实验不可复现。

所以大文件写入通常需要 commit 语义：先写临时文件，校验完成后再标记为可用。

9.14 存储权限和合规

大模型数据可能包含敏感信息、版权内容、业务机密或用户数据。

存储系统需要支持：

1. 身份认证。
2. 访问控制。
3. 数据分类。
4. 加密存储。
5. 访问日志。
6. 数据留存策略。
7. 删除和过期策略。
8. 跨租户隔离。

AI Infra 中，数据权限不是训练脚本自己解决的，而应该由平台和存储治理统一支持。

9.15 常见存储瓶颈

9.15.1 数据读取慢

可能原因：

1. 远端对象存储延迟高。
2. 小文件太多。
3. 数据没有 shard。
4. 缓存命中率低。
5. dataloader worker 不够。
6. CPU 解码或 tokenization 太慢。
7. 共享文件系统 metadata 压力大。

9.15.2 Checkpoint 保存慢

可能原因：

1. checkpoint 太大。
2. 所有 rank 同时写。
3. 存储写入吞吐不足。
4. metadata 操作多。
5. 网络到存储路径拥塞。
6. 没有异步保存。

9.15.3 模型加载慢

可能原因：

1. 权重文件太大。
2. 冷启动无缓存。
3. 多副本同时下载。
4. 对象存储限流。
5. 权重分片不合理。
6. 校验和转换耗时。

9.15.4 存储成本失控

可能原因：

1. checkpoint 保留过多。
2. 重复数据集。
3. 中间 artifact 无生命周期管理。
4. 日志无限增长。
5. 冷数据没有降级存储。

9.16 存储优化策略

常见优化：

1. 数据预处理离线化。
2. 小文件合并成 shard。
3. 本地 NVMe 缓存。
4. 分布式缓存。
5. 顺序读取和预取。
6. 提高 dataloader 并发。
7. 使用异步 checkpoint。
8. checkpoint 分片保存。
9. 模型权重预热。
10. 设置 artifact 生命周期策略。
11. 对冷数据做归档降级。
12. 使用 checksum 和 commit 标记保证完整性。

优化时要注意：缓存能提升速度，但会带来一致性和容量管理问题。

9.17 存储体系审计指标与最小 demo

前面讲的是存储组件和常见瓶颈。真实平台设计里，还要把“该用哪种存储、够不够快、能不能恢复、成本是否可控”变成可检查指标。

可以把第 i 个存储工作负载样本写成：

$$s_i=(u_i,c_i,q_i,m_i,h_i,k_i,r_i,a_i,g_i,l_i,z_i)$$

其中， u_i 是用途，例如 training data、checkpoint、model weights、artifact 或 RAG documents； c_i 是容量需求， q_i 是吞吐和 IOPS， m_i 是 metadata 压力， h_i 是 cache / hot data， k_i 是 checkpoint 或恢复需求， r_i 是可靠性和一致性， a_i 是 artifact lineage， g_i 是 governance， l_i 是 lifecycle / cost， z_i 是风险。

有效读写吞吐可以写成：

$$B_{\mathrm{eff}}=\frac{D}{T}$$

其中， D 是实际读写数据量， T 是端到端耗时。它比纸面带宽更有用，因为包括协议、metadata、并发、网络 and 客户端实现的影响。

训练 I/O 占比可以写成：

$$R_{\mathrm{io}}=\frac{T_{\mathrm{data loader}}+T_{\mathrm{storage}}+T_{\mathrm{h2d}}}{T_{\mathrm{step}}}$$

如果 R_{io} 很高，GPU 可能不是算不动，而是在等数据。

小文件 metadata 放大可以写成：

$$A_{\mathrm{meta}}=\frac{N_{\mathrm{file}}}{D_{\mathrm{TiB}}}$$

其中， N_{file} 是文件数， D_{TiB} 是数据集 TiB 数。文件数越多，metadata、open、stat、list 和对象请求开销越容易压垮共享文件系统或对象存储。

缓存命中率可以写成：

$$H_{\mathrm{cache}}=\frac{D_{\mathrm{cache}}}{D_{\mathrm{total}}}$$

其中， D_{cache} 是从本地或分布式缓存命中的字节数， D_{total} 是总读取字节数。缓存命中率低时，远端对象存储或共享文件系统压力会上升。

Checkpoint 保存和恢复时间可以粗略写成：

$$T_{\mathrm{ckpt}}=\frac{S_{\mathrm{ckpt}}}{\eta B_{\mathrm{write}}}+T_{\mathrm{meta}}+T_{\mathrm{commit}}$$

$$T_{\mathrm{restore}}=\frac{S_{\mathrm{ckpt}}}{\eta B_{\mathrm{read}}}+T_{\mathrm{validate}}+T_{\mathrm{requeue}}$$

其中, S_{ckpt} 是 checkpoint 大小, B_{write} 和 B_{read} 是写入和读取带宽, η 是有效带宽系数, T_{meta} 是 metadata 开销, T_{commit} 是 checksum / commit 标记开销, T_{validate} 是恢复校验开销, T_{requeue} 是重新排队或重启开销。

存储成本可以拆成:

$$K_{\{\mathrm{storage}\}} = K_{\{\mathrm{capacity}\}} + K_{\{\mathrm{request}\}} + K_{\{\mathrm{egress}\}} + K_{\{\mathrm{metadata}\}} + K_{\{\mathrm{amplification}\}}$$

最后, 可以把存储体系门禁写成:

$$G_{\{\mathrm{storage}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \wedge R_{\{\mathrm{io}\}} \leq \rho_{\{\mathrm{io}\}} \wedge T_{\{\mathrm{restore}\}} \leq P_0 \right]$$

其中, C_j 是第 j 个存储审计指标覆盖率, R_{io} 是训练 I/O 占比, T_{restore} 是恢复耗时, P_0 是 P0 级风险数量。

下面这个 0 依赖 demo 演示如何把训练数据、checkpoint、模型权重和 artifact 存储写成审计规则。它故意构造 1 个完整样本和 16 个坏样本, 让每个关键维度各失败一次。

```
import copy
```

```
METRICS = [
    "capacity_tier_fit",
    "throughput_iops_fit",
    "local_nvme_cache_fit",
    "shared_fs_metadata_fit",
    "object_store_authority_fit",
    "data_lake_governance_fit",
    "training_shard_format_fit",
    "small_file_amplification_control",
    "dataloader_cache_hit_tracking",
    "checkpoint_write_recovery_fit",
    "model_weight_load_cache_fit",
    "artifact_lineage_metadata_fit",
    "consistency_commit_integrity",
    "security_compliance_fit",
    "lifecycle_cost_governance",
    "storage_system_gate",
]
```

```
def throughput_gib_s(gib, seconds):
    return gib / seconds
```

```
def transfer_time_s(gib, bandwidth_gib_s, efficiency):
    return gib / (bandwidth_gib_s * efficiency)
```

```
def checkpoint_time_s(size_gib, bandwidth_gib_s, efficiency, metadata_s, commit_s):
    return transfer_time_s(size_gib, bandwidth_gib_s, efficiency) + metadata_s + commit_s
```

```
def restore_time_s(size_gib, bandwidth_gib_s, efficiency, validate_s, requeue_s):
    return transfer_time_s(size_gib, bandwidth_gib_s, efficiency) + validate_s + requeue_s
```

```
def metadata_amplification(files, dataset_tib):
    return files / dataset_tib
```

```

def cache_hit_rate(cache_gib, total_gib):
    return cache_gib / total_gib

def monthly_storage_cost_usd(hot_tib, cold_tib, hot_per_tib, cold_per_tib, request_cost):
    return hot_tib * hot_per_tib + cold_tib * cold_per_tib + request_cost

def build_storage_cases():
    complete = {
        "name": "complete",
        "capacity": {"hot_tib": 500, "cold_tib": 1500, "growth_tib_per_month": 120, "tiered": True},
        "performance": {"read_gib_s": 120, "write_gib_s": 80, "iops": 800_000, "measured": True},
        "local_cache": {"nvme_tib": 64, "hit_rate": 0.88, "eviction_policy": "lru"},
        "shared_fs": {"metadata_ops": 40_000, "small_file_pressure": "controlled", "quota": True},
        "object_store": {"authoritative": True, "versioning": True, "direct_training_read": False},
        "data_lake": {"lineage": True, "quality_checks": True, "permission_tags": True},
        "format": {"sharded": True, "format": "webdataset_or_parquet", "avg_shard_gib": 2.0},
        "small_files": {"file_count": 50_000, "dataset_tib": 100, "sharded": True},
        "dataloader": {"cache_hit_tracked": True, "dataloader_wait_s": 0.25, "step_time_s": 4.0},
        "checkpoint": {"size_gib": 3200, "write_gib_s": 80, "restore_gib_s": 100, "restore_tested": True},
        "model_load": {"local_weight_cache": True, "warmup": True, "max_load_s": 180},
        "artifact": {"dataset_version": "ds-v8", "config": "cfg-v5", "model": "m-v9", "eval_report": "eval-v4"},
        "commit": {"checksum": True, "atomic_commit": True, "manifest": True},
        "security": {"iam": True, "encryption": True, "audit_log": True, "tenant_isolation": True},
        "lifecycle": {"retention_policy": True, "archive_cold_data": True, "monthly_budget_usd": 20_000},
        "gate": {"enabled": True},
    }

def bad_case(name, mutator):
    case = copy.deepcopy(complete)
    case["name"] = name
    mutator(case)
    return case

bad_cases = [
    bad_case("capacity_tier_missing_bad", lambda c: c["capacity"].update({"tiered": False})),
    bad_case("throughput_iops_unknown_bad", lambda c: c["performance"].update({"measured": False})),
    bad_case("local_cache_missing_bad", lambda c: c["local_cache"].update({"nvme_tib": 0, "hit_rate": 0.0})),
    bad_case("shared_fs_metadata_overload_bad", lambda c: c["shared_fs"].update({"small_file_pressure": "high"})),
    bad_case("object_store_as_posix_bad", lambda c: c["object_store"].update({"direct_training_read": True})),
    bad_case("data_lake_governance_missing_bad", lambda c: c["data_lake"].update({"lineage": False})),
    bad_case("training_format_raw_json_bad", lambda c: c["format"].update({"sharded": False, "format": "raw_json"})),
    bad_case("small_files_unsharded_bad", lambda c: c["small_files"].update({"file_count": 5_000_000, "sharded": False})),
    bad_case("cache_hit_untracked_bad", lambda c: c["dataloader"].update({"cache_hit_tracked": False})),
    bad_case("checkpoint_restore_untested_bad", lambda c: c["checkpoint"].update({"restore_tested": False})),
    bad_case("model_weight_cold_start_bad", lambda c: c["model_load"].update({"local_weight_cache": False})),
    bad_case("artifact_lineage_missing_bad", lambda c: c["artifact"].update({"eval_report": ""})),
    bad_case("commit_checksum_missing_bad", lambda c: c["commit"].update({"checksum": False})),
    bad_case("storage_security_missing_bad", lambda c: c["security"].update({"audit_log": False})),
    bad_case("lifecycle_policy_missing_bad", lambda c: c["lifecycle"].update({"retention_policy": False})),
    bad_case("storage_gate_missing_bad", lambda c: c["gate"].update({"enabled": False})),
]
return [complete] + bad_cases

```

```

def check_capacity(case):
    c = case["capacity"]
    return c["tiered"] and c["hot_tib"] > 0 and c["cold_tib"] > 0 and c["growth_tib_per_month"] > 0

def check_performance(case):
    p = case["performance"]
    return p["measured"] and p["read_gib_s"] >= 50 and p["write_gib_s"] >= 40 and p["iops"] >= 100_000

def check_local_cache(case):
    c = case["local_cache"]
    return c["nvme_tib"] > 0 and c["hit_rate"] >= 0.7 and bool(c["eviction_policy"])

def check_shared_fs(case):
    s = case["shared_fs"]
    return s["metadata_ops"] > 0 and s["small_file_pressure"] == "controlled" and s["quota"]

def check_object_store(case):
    o = case["object_store"]
    return o["authoritative"] and o["versioning"] and not o["direct_training_read"]

def check_data_lake(case):
    d = case["data_lake"]
    return d["lineage"] and d["quality_checks"] and d["permission_tags"]

def check_format(case):
    f = case["format"]
    return f["sharded"] and f["format"] in {"webdataset_or_parquet", "arrow"} and f["avg_shard_gib"] >= 0.5

def check_small_files(case):
    s = case["small_files"]
    return s["sharded"] and metadata_amplification(s["file_count"], s["dataset_tib"]) <= 10_000

def check_data_loader(case):
    d = case["data_loader"]
    return d["cache_hit_tracked"] and d["data_loader_wait_s"] / d["step_time_s"] <= 0.15

def check_checkpoint(case):
    c = case["checkpoint"]
    write_s = checkpoint_time_s(c["size_gib"], c["write_gib_s"], 0.75, 20, 10)
    restore_s = restore_time_s(c["size_gib"], c["restore_gib_s"], 0.75, 20, 10)
    return c["restore_tested"] and write_s <= 120 and restore_s <= 120

def check_model_load(case):

```

```

m = case["model_load"]
return m["local_weight_cache"] and m["warmup"] and m["max_load_s"] <= 300

def check_artifact(case):
    a = case["artifact"]
    return all(a[k] for k in ["dataset_version", "config", "model", "eval_report"])

def check_commit(case):
    c = case["commit"]
    return c["checksum"] and c["atomic_commit"] and c["manifest"]

def check_security(case):
    s = case["security"]
    return s["iam"] and s["encryption"] and s["audit_log"] and s["tenant_isolation"]

def check_lifecycle(case):
    l = case["lifecycle"]
    cost = monthly_storage_cost_usd(500, 1500, 23, 4, 500)
    return l["retention_policy"] and l["archive_cold_data"] and cost <= l["monthly_budget_usd"]

def check_gate(case):
    return case["gate"]["enabled"]

CHECKS = {
    "capacity_tier_fit": check_capacity,
    "throughput_iops_fit": check_performance,
    "local_nvme_cache_fit": check_local_cache,
    "shared_fs_metadata_fit": check_shared_fs,
    "object_store_authority_fit": check_object_store,
    "data_lake_governance_fit": check_data_lake,
    "training_shard_format_fit": check_format,
    "small_file_amplification_control": check_small_files,
    "dataloader_cache_hit_tracking": check_dataloader,
    "checkpoint_write_recovery_fit": check_checkpoint,
    "model_weight_load_cache_fit": check_model_load,
    "artifact_lineage_metadata_fit": check_artifact,
    "consistency_commit_integrity": check_commit,
    "security_compliance_fit": check_security,
    "lifecycle_cost_governance": check_lifecycle,
    "storage_system_gate": check_gate,
}

def audit_storage(cases):
    case_failures = {}
    for case in cases:
        failures = [name for name, check in CHECKS.items() if not check(case)]
        case_failures[case["name"]] = failures

```

```

metrics = {}
for name, check in CHECKS.items():
    metrics[name] = round(sum(int(check(case)) for case in cases) / len(cases), 3)

failed_cases = [name for name, failures in case_failures.items() if failures]
return {
    "metrics": metrics,
    "hard_blocker_count": len(failed_cases),
    "failed_cases": failed_cases,
    "storage_gate_pass": not failed_cases and min(metrics.values()) >= 0.95,
}

cases = build_storage_cases()
case_by_name = {case["name"]: case for case in cases}
complete = case_by_name["complete"]
storage_examples = {
    "scan_4096gib_at_120gibs_s": round(transfer_time_s(4096, 120, 0.8), 1),
    "checkpoint_write_s": round(checkpoint_time_s(3200, 80, 0.75, 20, 10), 1),
    "checkpoint_restore_s": round(restore_time_s(3200, 100, 0.75, 20, 10), 1),
    "metadata_amp_files_per_tib": int(metadata_amplification(complete["small_files"]["file_count"], complete["small_files"]["file_size"]), 1),
    "cache_hit_rate": complete["local_cache"]["hit_rate"],
    "monthly_storage_cost_usd": monthly_storage_cost_usd(500, 1500, 23, 4, 500),
}

smoke = {
    "complete_case_passes": all(check(complete) for check in CHECKS.values()),
    "caught_small_files": not check_small_files(case_by_name["small_files_unsharded_bad"]),
    "caught_checkpoint_restore_gap": not check_checkpoint(case_by_name["checkpoint_restore_untested_bad"]),
    "caught_commit_gap": not check_commit(case_by_name["commit_checksum_missing_bad"]),
    "caught_lifecycle_gap": not check_lifecycle(case_by_name["lifecycle_policy_missing_bad"]),
}

audit = audit_storage(cases)
print(f"storage_examples={storage_examples}")
print(f"smoke={smoke}")
print(f"metrics={audit['metrics']}")
print(f"hard_blocker_count={audit['hard_blocker_count']}")
print(f"failed_cases={audit['failed_cases']}")
print(f"storage_gate_pass={audit['storage_gate_pass']}")

```

一组典型输出是：

```

storage_examples={'scan_4096gib_at_120gibs_s': 42.7, 'checkpoint_write_s': 83.3, 'checkpoint_restore_s': 72.7, 'metadata_amp_files_per_tib': 1, 'cache_hit_rate': 0.941, 'monthly_storage_cost_usd': 500}
smoke={'complete_case_passes': True, 'caught_small_files': True, 'caught_checkpoint_restore_gap': True, 'caught_commit_gap': True, 'caught_lifecycle_gap': True}
metrics={'capacity_tier_fit': 0.941, 'throughput_iops_fit': 0.941, 'local_nvme_cache_fit': 0.941, 'shared_fs_metadata_fit': 0.941}
hard_blocker_count=16
failed_cases=['capacity_tier_missing_bad', 'throughput_iops_unknown_bad', 'local_cache_missing_bad', 'shared_fs_metadata_fit_missing_bad']
storage_gate_pass=False

```

这个 demo 的重点是把“存储体系”拆成可验证证据链：容量要分 hot / cold 层级，训练数据要有 shard 和 cache，shared filesystem 要控制 metadata 压力，对象存储适合作权威源但不能被当成高频 POSIX 文件系统，checkpoint 必须证明写得出也恢复得回，模型权重要有缓存和预热，artifact 要有血缘，写入要有 checksum / commit，权限和生命周期成本要能审计。

9.18 面试中如何回答存储体系设计

如果面试官问：

大模型训练平台的存储体系怎么设计？

可以这样回答：

我会把存储分层设计。

对象存储或数据湖作为权威源，保存原始数据、清洗数据、checkpoint、模型权重和 artifact，提供容量、持久性、权限和数据治理。

训练前把数据离线清洗、去重、tokenize，并转换成训练友好的 shard，减少小文件和在线解析开销。

训练节点使用本地 NVMe 或分布式缓存保存热点数据，提升读取吞吐，避免每个 step 都访问远端存储。

checkpoint 使用分片和异步保存，写入完成后通过 checksum 和 commit metadata 标记可恢复版本，同时设置保留策略和生命周期管理。

模型仓库管理权重、tokenizer、config、adapter 和 eval report，推理平台通过缓存和预热降低模型加载时间。

横向还要有权限、审计、数据血缘、成本统计和监控。

9.19 常见误区

误区一：存储只是把文件放起来。

AI 存储还要支持高吞吐读取、checkpoint 恢复、模型版本、artifact 血缘、权限审计和成本治理。

误区二：共享文件系统最方便，所以所有任务都用它。

共享文件系统易用，但在大规模小文件和高并发场景可能成为瓶颈。

误区三：对象存储容量大，所以训练直接从对象存储读就行。

对象存储适合权威源和归档，但直接高频训练读取可能有延迟和请求开销，需要缓存或 shard 优化。

误区四：checkpoint 保存成功就没问题。

还要验证能否恢复，metadata 是否一致，版本是否完整，保留策略是否合理。

误区五：存储成本不重要。

大模型 checkpoint、数据集、日志和中间 artifact 很容易快速膨胀，没有生命周期管理会造成巨大浪费。

9.20 面试题

题 1：本地盘、共享文件系统、对象存储分别适合什么场景？

答：本地盘速度快、延迟低，适合热点数据缓存和临时数据；共享文件系统使用方便，适合多节点共享代码、数据和中间结果，但高并发和小文件场景可能瓶颈明显；对象存储容量大、持久性强、成本相对低，适合原始数据、checkpoint、模型权重和 artifact 的长期保存。

题 2：为什么小文件会拖慢训练？

答：小文件会带来大量 metadata 查询、打开文件和对象请求开销，导致共享文件系统或对象存储压力很大。dataloader 会等待 I/O，GPU 利用率下降。解决方法包括合并 shard、使用 WebDataset / Parquet / Arrow、建立索引和本地缓存。

题 3：训练数据供给链路有哪些瓶颈？

答：可能瓶颈包括数据清洗慢、在线 tokenization 慢、shard 设计不合理、小文件过多、远端存储延迟高、缓存命中率低、dataloader worker 不够、CPU 到 GPU 拷贝慢等。

题 4: checkpoint 存储要注意什么?

答: 要关注保存吞吐、恢复速度、分片策略、异步保存、metadata 一致性、checksum 校验、保留策略、权限控制和恢复测试。checkpoint 不是只要写成功, 还必须能可靠恢复。

题 5: 模型加载慢怎么优化?

答: 可以做本地缓存、分层缓存、模型预热、分片加载、对象存储限流保护、发布前完整性校验、避免多副本同时冷启动, 并通过模型仓库统一管理权重、tokenizer 和 config 版本。

9.21 小练习

练习一: 设计一个训练数据存储链路。

要求: 从原始数据到训练 shard, 再到本地缓存和 dataloader, 画出完整链路。

练习二: 分析一个 GPU 利用率周期性下降的问题。

假设每隔 1000 step GPU 利用率掉到 0, 持续 3 分钟。请从 checkpoint、存储吞吐、metadata 和异步保存角度分析。

练习三: 设计一个 checkpoint 生命周期策略。

要求: 说明保存频率、保留数量、关键版本、冷数据归档和删除策略。

练习四: 设计一个模型 artifact schema。

要求: 至少包含 model_id、version、weights_uri、tokenizer_uri、config_uri、base_model、dataset_version、eval_report、owner、created_at 和 checksum。

9.22 本章小结

本章讲了 AI Infra 中的存储体系。

你需要掌握:

1. AI 存储承载训练数据、checkpoint、模型权重、artifact、日志、RAG 文档和 trace。
2. 本地盘适合低延迟缓存, 共享文件系统适合多节点共享, 对象存储适合大容量长期保存, 数据湖适合数据治理。
3. 存储指标包括容量、吞吐、IOPS、延迟、并发、可用性、持久性、一致性、成本和权限。
4. 数据格式、小文件、shard 和缓存会显著影响训练吞吐。
5. checkpoint 需要关注保存、加载、metadata、checksum、恢复测试和生命周期管理。
6. 模型权重加载会影响推理冷启动和扩容速度。
7. Artifact 管理要支持版本、血缘、权限、生命周期和可复现性。
8. 存储优化要结合本地缓存、分布式缓存、异步写入、预处理、生命周期策略和数据治理。

下一章我们会专门讲 Checkpoint 存储、加载、恢复和生命周期管理。

第 10 章 Checkpoint 存储、加载、恢复和生命周期管理

上一章讲了存储体系。本章专门讲 checkpoint。

Checkpoint 是大模型训练平台里最关键、也最容易被低估的能力之一。它看起来只是“保存一下模型”, 但在真实大规模训练中, checkpoint 决定了训练失败后能不能恢复、恢复到哪里、恢复是否正确、存储成本是否可控、实验是否可复现。

先记住一句话:

Checkpoint 不是一个模型文件, 而是训练状态的可恢复快照; 它的价值不在于保存成功, 而在于能可靠、快速、正确地恢复。

10.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，资料口径按“大模型训练平台 checkpoint 生命周期的稳定抽象”处理，而不是绑定某个框架、某个对象存储厂商或某套内部平台实现。分布式 checkpoint 部分参考 PyTorch Distributed Checkpoint 对分布式 state_dict 保存、加载和异步保存的抽象；工程接口部分参考 DeepSpeed 对训练 checkpoint 保存、加载和 ZeRO checkpoint 的接口边界；对象存储部分参考 Amazon S3 对强一致读取、对象生命周期和保留策略的通用口径；平台治理部分沿用上一章存储体系的 manifest、checksum、commit、权限、审计和成本分层。

需要注意三点：

1. 本章讲训练状态 checkpoint 的保存、加载、恢复、一致性和生命周期，不把下一章容器、镜像、驱动和 CUDA 依赖管理提前展开。
2. 模型权重 checkpoint、训练状态 checkpoint、发布 artifact 和归档对象不是同一个概念，面试中要先说明用途、状态内容、恢复要求和权限边界。
3. checkpoint 系统不能只证明“写入成功”，还要证明分片完整、manifest 可解释、commit 语义清楚、恢复演练通过、保留策略可控、成本和权限可审计。

10.1 为什么 checkpoint 这么重要

大模型训练时间长、成本高、故障概率高。

一个预训练任务可能运行数天甚至数周，使用成百上千张 GPU。期间可能遇到：

1. GPU 故障。
2. 节点宕机。
3. 网络 timeout。
4. 存储异常。
5. 训练进程 crash。
6. 驱动或容器问题。
7. 调度抢占。
8. 人为停止任务。

如果没有 checkpoint，任务失败就要从头开始，成本巨大。

Checkpoint 的作用：

1. 故障后恢复训练。
2. 支持抢占式调度。
3. 保存重要训练阶段。
4. 支持模型评估和发布。
5. 支持实验复现。
6. 支持回滚。
7. 支持训练过程分析。

所以 checkpoint 是训练可靠性的基础。

10.2 Checkpoint 里到底有什么

很多人以为 checkpoint 只包含模型参数。

真实训练 checkpoint 通常包含：

1. 模型参数。
2. 优化器状态。
3. 学习率 scheduler 状态。
4. 当前 global step。
5. epoch 或样本位置。
6. random seed。
7. dataloader 状态。
8. gradient scaler 状态。

9. 分布式训练状态。
10. 并行策略 metadata。
11. tokenizer 和 config 引用。
12. 训练代码版本。
13. 数据版本。
14. 训练参数。

如果只保存模型参数，通常只能用于推理或继续微调，不一定能精确恢复训练。

精确恢复训练需要保存完整训练状态。

10.3 模型权重 checkpoint 和训练状态 checkpoint

要区分两类 checkpoint。

10.3.1 模型权重 checkpoint

它主要包含模型参数。

用途：

1. 推理部署。
2. 模型评估。
3. 下游微调。
4. 模型发布。

它通常不包含完整优化器状态。

10.3.2 训练状态 checkpoint

它包含恢复训练需要的完整状态。

用途：

1. 断点续训。
2. 故障恢复。
3. 抢占恢复。
4. 精确复现实验。

它通常比模型权重 checkpoint 大很多，因为优化器状态可能非常大。

面试中要强调：用于部署的 checkpoint 和用于恢复训练的 checkpoint 不是一回事。

10.4 为什么 checkpoint 会很大

大模型 checkpoint 大，主要因为：

1. 参数量大。
2. 优化器状态大。
3. 分布式分片多。
4. 多精度状态并存。
5. 需要保存 metadata。

以 Adam 为例，每个参数通常有：

1. 参数本身。
2. 一阶动量。
3. 二阶动量。
4. 可能还有 FP32 master weight。

所以训练状态 checkpoint 可能是模型权重大小的数倍。

这也是为什么 checkpoint 会给存储带来巨大压力。

10.5 Checkpoint 保存频率

保存频率是一个 trade-off。

保存太频繁：

1. 训练被 I/O 打断。
2. 存储吞吐压力大。
3. checkpoint 数量爆炸。
4. 成本高。

保存太少：

1. 故障后回退太多 step。
2. 浪费训练算力。
3. 抢占恢复代价高。
4. 丢失重要中间版本。

选择保存频率时要考虑：

1. 训练任务规模。
2. 平均故障间隔。
3. checkpoint 保存耗时。
4. 存储成本。
5. 可接受的恢复点目标。
6. 评估和发布需求。

可以用一个简单问题判断：如果现在任务失败，最多能接受损失多少训练时间？

10.6 同步 checkpoint 和异步 checkpoint

10.6.1 同步 checkpoint

同步 checkpoint 是训练暂停，等待 checkpoint 写完，再继续训练。

优点：

1. 实现简单。
2. 一致性容易保证。
3. 恢复逻辑清楚。

缺点：

1. 训练会被阻塞。
2. 大模型保存耗时长。
3. GPU 可能等待 I/O。
4. 周期性 step time 尖刺明显。

10.6.2 异步 checkpoint

异步 checkpoint 是训练继续跑，后台线程或独立进程负责写 checkpoint。

优点：

1. 减少训练阻塞。
2. 提高 GPU 利用率。
3. 降低 step time 尖刺。

缺点：

1. 实现复杂。
2. 要处理内存占用。
3. 要保证写入一致性。
4. 失败状态更复杂。
5. 可能和训练争抢 I/O 或内存带宽。

异步 checkpoint 不是简单开线程写文件，而是一个一致性和资源隔离问题。

10.7 分片 checkpoint

大模型通常不能让一个 rank 写一个巨大文件。

分片 checkpoint 会把状态拆成多个 shard。

常见分片方式：

1. 按 rank 分片。
2. 按 layer 分片。
3. 按参数 shard 分片。
4. 按 optimizer state 分片。
5. 按 tensor parallel / data parallel 维度分片。

分片 checkpoint 的好处：

1. 单文件更小。
2. 多 rank 并行写入。
3. 恢复时可以并行读取。
4. 更适合分布式训练状态。

挑战：

1. metadata 更复杂。
2. 需要保证所有 shard 完整。
3. rank 数变化时恢复更复杂。
4. 文件数量多可能造成 metadata 压力。

因此分片 checkpoint 必须配套 manifest。

10.8 Manifest 和 metadata

Manifest 是 checkpoint 的目录和说明书。

它应该记录：

1. checkpoint_id。
2. global_step。
3. created_at。
4. model_version。
5. training_config。
6. data_version。
7. parallelism_config。
8. shard 列表。
9. 每个 shard 的大小。
10. checksum。
11. 保存状态。
12. 是否可恢复。

示例：

```
{  
  "checkpoint_id": "ckpt_00012000",  
  "global_step": 12000,  
}
```

```

"status": "committed",
"parallelism": {
  "tensor_parallel": 8,
  "pipeline_parallel": 4,
  "data_parallel": 16
},
"shards": [
  {
    "name": "rank_000_state.pt",
    "size_bytes": 10737418240,
    "checksum": "sha256:..."
  }
]
}

```

没有 manifest，恢复系统就不知道哪些文件属于同一个一致版本。

10.9 Commit 语义

Checkpoint 写入必须避免 “半成品被当成可用版本”。

常见做法：

1. 写入临时目录。
2. 所有 shard 写完。
3. 校验 checksum。
4. 写 manifest。
5. 原子标记 committed。
6. 清理旧临时文件。

简化流程：

```

writing -> validating -> committed
                -> failed

```

恢复时只允许读取 committed checkpoint。

如果读取 writing 状态的 checkpoint，很容易恢复失败或训练状态不一致。

10.10 Checkpoint 加载

加载 checkpoint 不是简单读文件。

需要做：

1. 找到目标 checkpoint。
2. 检查 manifest。
3. 校验 shard 完整性。
4. 加载模型参数。
5. 加载优化器状态。
6. 加载 scheduler 状态。
7. 恢复 random seed。
8. 恢复 dataloader 位置。
9. 恢复分布式状态。
10. 验证配置兼容性。

常见加载失败原因：

1. shard 缺失。
2. checksum 不匹配。
3. 模型结构变化。

4. 并行策略变化。
5. 优化器配置变化。
6. 代码版本不兼容。
7. tokenizer 或 config 不匹配。

所以 checkpoint 加载需要严格校验，不应该静默忽略错误。

10.11 恢复训练的一致性

恢复训练要回答一个问题：恢复后是否和没中断时尽可能一致？

影响一致性的因素：

1. 模型参数。
2. 优化器状态。
3. scheduler 状态。
4. random seed。
5. dataloader 顺序。
6. gradient scaler。
7. 分布式 rank 映射。
8. dropout 随机性。
9. 数据 shuffle 状态。
10. 代码和依赖版本。

如果只恢复模型参数，不恢复 optimizer 和 scheduler，训练曲线可能发生变化。

如果不恢复 dataloader 状态，可能重复或跳过数据。

如果不恢复 random seed，严格复现会失败。

10.12 Rank 数变化和弹性恢复

有时恢复训练时 GPU 数量可能变化。

例如原来 128 卡训练，恢复时只有 64 卡。

这会带来问题：

1. checkpoint shard 和 rank 数绑定。
2. tensor parallel 维度可能不兼容。
3. pipeline stage 划分可能变化。
4. optimizer state 需要重新分片。
5. global batch size 可能变化。

弹性恢复需要支持 checkpoint resharding。

Resharding 是把原有 checkpoint 分片重新组织成新的并行布局。

这很有价值，但实现复杂。

面试里可以说：基础版本先要求相同并行配置恢复；高级版本支持 resharding 和弹性训练。

10.13 Checkpoint 与抢占式调度

抢占式调度可以提高集群利用率。

低优先级训练任务使用空闲 GPU，高优先级任务来了以后被抢占。

要支持抢占，必须有可靠 checkpoint。

抢占前最好：

1. 给任务发送 preemption notice。

2. 任务保存 checkpoint。
3. 标记任务可恢复。
4. 释放资源。
5. 后续重新排队恢复。

如果没有 checkpoint，抢占会浪费大量训练进度。

所以 checkpoint 是弹性调度和资源利用率优化的基础能力。

10.14 Checkpoint 生命周期管理

Checkpoint 不能无限保留。

生命周期管理要回答：

1. 保存哪些 checkpoint?
2. 保留多久?
3. 哪些是关键 checkpoint?
4. 哪些可以删除?
5. 哪些要归档到低成本存储?
6. 删除前是否需要审批?

常见策略：

1. 保留最近 N 个 checkpoint。
2. 每隔固定步数保留一个里程碑版本。
3. 最优 eval 分数 checkpoint 永久保留或长期保留。
4. 最终发布模型保留。
5. 失败或临时 checkpoint 定期清理。
6. 冷 checkpoint 迁移到低成本存储。

一个例子：

最近 5 个 checkpoint：保留在高性能存储。

每 10k step checkpoint：保留 30 天。

最佳 eval checkpoint：长期保留。

发布版本：长期保留并加权限保护。

临时失败 checkpoint：7 天后删除。

10.15 Checkpoint 成本治理

Checkpoint 成本容易失控。

原因：

1. 单个 checkpoint 很大。
2. 保存频率高。
3. 多个实验并行。
4. 优化器状态巨大。
5. 没有删除策略。
6. 多份重复保存。
7. 日志和 artifact 一起膨胀。

成本治理方法：

1. 设置默认保留策略。
2. 区分训练恢复 checkpoint 和发布权重。
3. 冷热分层存储。
4. 压缩或去重。
5. 定期生成成本报告。
6. 按团队或项目归因。

7. 对超预算实验告警。

成本治理不是只删文件，而是建立可解释的保留规则。

10.16 Checkpoint 监控指标

训练平台应该监控 checkpoint。

关键指标：

1. save_duration。
2. load_duration。
3. checkpoint_size。
4. checkpoint_frequency。
5. save_failure_rate。
6. load_failure_rate。
7. committed_checkpoint_count。
8. temporary_checkpoint_count。
9. storage_write_throughput。
10. storage_read_throughput。
11. restore_success_rate。
12. time_since_last_checkpoint。

告警场景：

1. 长时间没有成功 checkpoint。
2. checkpoint 保存时间突然变长。
3. checkpoint 失败率上升。
4. 存储空间接近上限。
5. 恢复失败。
6. 临时 checkpoint 堆积。

如果训练任务已经跑了 12 小时但没有成功 checkpoint，这是严重风险。

10.17 常见故障和排查

10.17.1 Checkpoint 保存很慢

可能原因：

1. checkpoint 太大。
2. 所有 rank 同时写。
3. 存储吞吐不足。
4. 小文件太多。
5. metadata 服务瓶颈。
6. 网络到存储路径拥塞。
7. 同一时间多个任务保存。

排查：

1. 看保存耗时分布。
2. 看每个 shard 大小。
3. 看存储吞吐。
4. 看 metadata 延迟。
5. 看网络吞吐。
6. 看是否周期性和其他任务冲突。

10.17.2 Checkpoint 恢复失败

可能原因：

1. shard 缺失。
2. manifest 不完整。
3. checksum 不匹配。
4. 代码版本变化。
5. 并行配置变化。
6. optimizer 状态不兼容。
7. tokenizer / config 不匹配。

排查：

1. 只读取 committed checkpoint。
2. 校验 manifest。
3. 校验 checksum。
4. 检查版本兼容性。
5. 在小规模环境做恢复测试。

10.17.3 恢复后 loss 异常

可能原因：

1. optimizer 状态没恢复。
2. scheduler 状态没恢复。
3. random seed 没恢复。
4. dataloader 状态错。
5. mixed precision scaler 状态丢失。
6. 数据重复或跳过。
7. 代码逻辑变化。

排查：

1. 比较恢复前后 loss。
2. 检查 global step。
3. 检查学习率。
4. 检查数据位置。
5. 检查 optimizer state。
6. 用短任务做中断恢复测试。

10.18 Checkpoint 恢复测试

很多团队只测试保存，不测试恢复。

这是危险的。

平台应该定期做恢复测试：

1. 启动一个训练任务。
2. 训练若干 step。
3. 保存 checkpoint。
4. 模拟中断。
5. 从 checkpoint 恢复。
6. 对比 loss、step、学习率和数据位置。
7. 验证继续训练正常。

高级测试还可以包括：

1. rank 故障恢复。
2. 节点故障恢复。
3. checkpoint shard 缺失测试。
4. 并行策略变化恢复。
5. 存储短暂失败恢复。

Checkpoint 能不能用，必须通过恢复测试证明。

10.19 Checkpoint 生命周期审计指标与最小 demo

可以把 checkpoint 生命周期从“保存文件”写成一组可审计对象。

一个训练状态 checkpoint 样本可以抽象为：

$c_i = (m_i, o_i, p_i, s_i, w_i, d_i, r_i, a_i, v_i, k_i, l_i, g_i, z_i)$

其中， m_i 是 manifest， o_i 是训练状态对象， p_i 是并行配置， s_i 是 shard 布局， w_i 是写入路径， d_i 是校验摘要， r_i 是恢复测试， a_i 是异步保存策略， v_i 是版本和血缘， k_i 是保留与成本策略， l_i 是权限和审计日志， g_i 是监控与 SLO， z_i 是最终门禁结果。

分片大小可以先用：

$S_{\{\mathrm{shard}\}} = \frac{S_{\{\mathrm{ckpt}\}}}{N_{\{\mathrm{shard}\}}}$

其中， S_{ckpt} 是总 checkpoint 大小， N_{shard} 是 shard 数。真实系统还要考虑 tensor 大小、rank 映射、对象存储请求数、metadata 压力和恢复并发度。

保存耗时可以拆成：

$T_{\{\mathrm{save}\}} = \frac{S_{\{\mathrm{ckpt}\}}}{B_{\{\mathrm{write}\}}} + T_{\{\mathrm{sync}\}} + T_{\{\mathrm{meta}\}} + T_{\{\mathrm{commit}\}}$

其中， B_{write} 是写入带宽， η_w 是有效带宽系数， T_{sync} 是冻结或拷贝一致快照的时间， T_{meta} 是 manifest / metadata 写入开销， T_{commit} 是 checksum 校验和 committed 标记开销。

如果使用异步 checkpoint，训练可见阻塞不是总写入时间，而更接近：

$T_{\{\mathrm{visible}\}} = \max(0, T_{\{\mathrm{save}\}}(1 - \gamma_{\{\mathrm{overlap}\}}))$

其中， $\gamma_{\mathrm{overlap}}$ 是写入与训练重叠比例。这个比例不能只靠理论估计，必须从 step time 尖刺、I/O 争抢和后台队列积压中实测。

恢复耗时可以写成：

$T_{\{\mathrm{restore}\}} = \frac{S_{\{\mathrm{ckpt}\}}}{B_{\{\mathrm{read}\}}} + T_{\{\mathrm{validate}\}} + T_{\{\mathrm{rebuild}\}} + T_{\{\mathrm{requeue}\}}$

其中， B_{read} 是读取带宽， η_r 是有效读取系数， T_{validate} 是 checksum / manifest 校验时间， T_{rebuild} 是重建并行状态或 reshard 的时间， T_{requeue} 是重新排队、拉起容器和恢复训练进程的时间。

保存间隔决定最大丢失进度：

$W_{\{\mathrm{lost}\}} \leq I_{\{\mathrm{ckpt}\}}$

其中， I_{ckpt} 是两次成功 checkpoint 之间的时间间隔。抢占式训练、长周期预训练和不稳定集群要把 I_{ckpt} 作为平台 SLO，而不是只看“平均每天保存几次”。

生命周期成本可以写成：

$K_{\{\mathrm{ckpt}\}} = \sum_{a=1}^A S_a P_a T_a + K_{\{\mathrm{request}\}} + K_{\{\mathrm{egress}\}} + K_{\{\mathrm{ops}\}}$

其中， S_a 是第 a 类 checkpoint 的容量， P_a 是对应存储层级单价， T_a 是保留时长， K_{request} 是对象请求成本， K_{egress} 是跨区或出站成本， K_{ops} 是校验、复制、扫描、人工审批和运维成本。

最后，可以把 checkpoint 生命周期门禁写成：

$G_{\{\mathrm{ckpt}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \ \text{and} \ T_{\{\mathrm{restore}\}} \leq \tau_{\{\mathrm{restore}\}} \ \text{and} \ W_{\{\mathrm{lost}\}} \leq \omega \right]$

其中， C_j 是第 j 个 checkpoint 审计指标覆盖率， τ_j 是覆盖率阈值， τ_{restore} 是恢复 SLO， ω 是最大可接受丢失进度， P_0 是 P0 级风险数量。

下面这个 0 依赖 demo 演示如何把 checkpoint 生命周期写成审计规则。它故意构造 1 个完整样本和 16 个坏样本，让每个关键维度各失败一次。

```
import copy
```

```
METRICS = [  
    "checkpoint_object_completeness",  
    "shard_layout_fit",  
    "async_save_overlap",  
    "write_bandwidth_fit",  
    "metadata_commit_integrity",  
    "checksum_manifest_validation",  
    "restore_replay_readiness",  
    "dataloader_rng_state_capture",  
    "distributed_rank_state_capture",  
    "retention_policy_fit",  
    "lifecycle_cost_governance",  
    "cross_region_replication_fit",  
    "security_access_control",  
    "resume_slo_tracking",  
    "failure_drill_coverage",  
    "checkpoint_lifecycle_gate",  
]
```

```
def shard_size_gib(total_gib, shard_count):  
    return total_gib / shard_count
```

```
def transfer_time_s(gib, bandwidth_gib_s, efficiency):  
    return gib / (bandwidth_gib_s * efficiency)
```

```
def save_time_s(size_gib, bandwidth_gib_s, efficiency, sync_s, metadata_s, commit_s):  
    return transfer_time_s(size_gib, bandwidth_gib_s, efficiency) + sync_s + metadata_s + commit_s
```

```
def visible_save_overhead_s(total_save_s, overlap_ratio):  
    return max(0.0, total_save_s * (1.0 - overlap_ratio))
```

```
def restore_time_s(size_gib, bandwidth_gib_s, efficiency, validate_s, rebuild_s, requeue_s):  
    return transfer_time_s(size_gib, bandwidth_gib_s, efficiency) + validate_s + rebuild_s + requeue_s
```

```
def monthly_checkpoint_cost_usd(hot_tib, cold_tib, hot_per_tib, cold_per_tib, request_cost):  
    return hot_tib * hot_per_tib + cold_tib * cold_per_tib + request_cost
```

```
def build_checkpoint_cases():  
    complete = {  
        "name": "complete",  
        "object": {  
            "type": "training_state",  
            "model_state": True,  
            "optimizer_state": True,  
            "scheduler_state": True,
```

```

    "global_step": True,
    "scaler_state": True,
    "code_data_version": True,
},
"shard_layout": {
    "total_gib": 3200,
    "expected_shards": 64,
    "actual_shards": 64,
    "manifest_shards": 64,
    "max_shard_gib": 64,
    "parallelism_match": True,
},
"async_save": {
    "mode": "async",
    "snapshot_frozen": True,
    "overlap_ratio": 0.82,
    "io_isolated": True,
    "pending_limit": 2,
},
"write_path": {
    "write_gib_s": 80,
    "efficiency": 0.75,
    "sync_s": 5,
    "metadata_s": 20,
    "commit_s": 5,
    "save_slo_s": 120,
    "measured": True,
},
"metadata": {
    "manifest": True,
    "atomic_commit": True,
    "status": "committed",
    "temp_prefix": True,
},
"checksum": {
    "all_shards": True,
    "algorithm": "sha256",
    "validate_on_load": True,
},
"restore": {
    "restore_tested": True,
    "read_gib_s": 100,
    "efficiency": 0.75,
    "validate_s": 20,
    "rebuild_s": 10,
    "requeue_s": 20,
    "restore_slo_s": 120,
},
"rng_state": {
    "python": True,
    "numpy": True,
    "torch": True,
    "cuda": True,
    "sampler": True,
},

```

```

"rank_state": {
    "tensor_parallel": True,
    "pipeline_parallel": True,
    "data_parallel": True,
    "zero_partition": True,
    "grad_accum_step": True,
},
"retention": {
    "recent_n": 5,
    "milestone_days": 30,
    "best_eval_keep": True,
    "release_keep": True,
    "temp_failed_days": 7,
    "keep_all": False,
},
"cost": {
    "hot_tib": 120,
    "cold_tib": 480,
    "hot_per_tib": 23,
    "cold_per_tib": 4,
    "request_cost": 320,
    "monthly_budget_usd": 6000,
    "archive_cold": True,
    "budget_owner": True,
},
"replication": {
    "object_store": True,
    "cross_region_release": True,
    "restore_copy_tested": True,
},
"security": {
    "iam": True,
    "encryption": True,
    "audit_log": True,
    "tenant_isolation": True,
},
"slo": {
    "tracked": True,
    "save_p95_s": 90,
    "save_slo_s": 120,
    "restore_p95_s": 100,
    "restore_slo_s": 120,
    "time_since_last_ckpt_min": 12,
    "max_interval_min": 15,
},
"drill": {
    "automated": True,
    "restore_test_age_days": 7,
    "failure_modes": 4,
},
"gate": {"enabled": True},
}

```

```

def bad_case(name, mutator):
    case = copy.deepcopy(complete)

```

```
case["name"] = name
mutator(case)
return case
```

```
bad_cases = [
    bad_case("weights_only_checkpoint_bad", lambda c: c["object"].update({"type": "model_weights", "optimizer_state_dict": None})),
    bad_case("single_huge_file_bad", lambda c: c["shard_layout"].update({"actual_shards": 1, "manifest_shards": 1000000})),
    bad_case("blocking_save_bad", lambda c: c["async_save"].update({"mode": "sync", "overlap_ratio": 0.0})),
    bad_case("write_bandwidth_unknown_bad", lambda c: c["write_path"].update({"measured": False})),
    bad_case("metadata_marked_before_commit_bad", lambda c: c["metadata"].update({"atomic_commit": False})),
    bad_case("checksum_missing_bad", lambda c: c["checksum"].update({"all_shards": False})),
    bad_case("restore_never_tested_bad", lambda c: c["restore"].update({"restore_tested": False})),
    bad_case("rng_state_missing_bad", lambda c: c["rng_state"].update({"torch": False})),
    bad_case("rank_state_missing_bad", lambda c: c["rank_state"].update({"zero_partition": False})),
    bad_case("keep_everything_forever_bad", lambda c: c["retention"].update({"keep_all": True, "temp_failed_days": 999999999})),
    bad_case("cold_archive_never_used_bad", lambda c: c["cost"].update({"archive_cold": False})),
    bad_case("no_replication_bad", lambda c: c["replication"].update({"cross_region_release": False})),
    bad_case("open_bucket_bad", lambda c: c["security"].update({"iam": False})),
    bad_case("resume_slo_missing_bad", lambda c: c["slo"].update({"tracked": False})),
    bad_case("failure_drill_missing_bad", lambda c: c["drill"].update({"automated": False})),
    bad_case("checkpoint_gate_missing_bad", lambda c: c["gate"].update({"enabled": False})),
]

return [complete] + bad_cases
```

```
def check_object(case):
    obj = case["object"]
    required = ["model_state", "optimizer_state", "scheduler_state", "global_step", "scaler_state", "code_data_version"]
    return obj["type"] == "training_state" and all(obj[k] for k in required)
```

```
def check_shard_layout(case):
    layout = case["shard_layout"]
    expected = layout["expected_shards"]
    shard_gib = shard_size_gib(layout["total_gib"], max(layout["actual_shards"], 1))
    return (
        layout["actual_shards"] == expected
        and layout["manifest_shards"] == expected
        and shard_gib <= layout["max_shard_gib"]
        and layout["parallelism_match"]
    )
```

```
def check_async(case):
    async_save = case["async_save"]
    return (
        async_save["mode"] == "async"
        and async_save["snapshot_frozen"]
        and async_save["overlap_ratio"] >= 0.7
        and async_save["io_isolated"]
        and async_save["pending_limit"] <= 2
    )
```

```
def check_write(case):
```

```

path = case["write_path"]
total_s = save_time_s(
    case["shard_layout"]["total_gib"],
    path["write_gib_s"],
    path["efficiency"],
    path["sync_s"],
    path["metadata_s"],
    path["commit_s"],
)
return path["measured"] and total_s <= path["save_slo_s"]

def check_metadata(case):
    meta = case["metadata"]
    return meta["manifest"] and meta["atomic_commit"] and meta["status"] == "committed" and meta["temp_prefix"]

def check_checksum(case):
    checksum = case["checksum"]
    return checksum["all_shards"] and checksum["algorithm"] == "sha256" and checksum["validate_on_load"]

def check_restore(case):
    restore = case["restore"]
    total_s = restore_time_s(
        case["shard_layout"]["total_gib"],
        restore["read_gib_s"],
        restore["efficiency"],
        restore["validate_s"],
        restore["rebuild_s"],
        restore["requeue_s"],
    )
    return restore["restore_tested"] and total_s <= restore["restore_slo_s"]

def check_rng(case):
    return all(case["rng_state"].values())

def check_rank_state(case):
    return all(case["rank_state"].values())

def check_retention(case):
    retention = case["retention"]
    return (
        retention["recent_n"] >= 3
        and retention["milestone_days"] >= 7
        and retention["best_eval_keep"]
        and retention["release_keep"]
        and 0 < retention["temp_failed_days"] <= 14
        and not retention["keep_all"]
    )

```

```

def check_cost(case):
    cost = case["cost"]
    monthly = monthly_checkpoint_cost_usd(
        cost["hot_tib"],
        cost["cold_tib"],
        cost["hot_per_tib"],
        cost["cold_per_tib"],
        cost["request_cost"],
    )
    return cost["archive_cold"] and cost["budget_owner"] and monthly <= cost["monthly_budget_usd"]

def check_replication(case):
    repl = case["replication"]
    return repl["object_store"] and repl["cross_region_release"] and repl["restore_copy_tested"]

def check_security(case):
    return all(case["security"].values())

def check_slo(case):
    slo = case["slo"]
    return (
        slo["tracked"]
        and slo["save_p95_s"] <= slo["save_slo_s"]
        and slo["restore_p95_s"] <= slo["restore_slo_s"]
        and slo["time_since_last_ckpt_min"] <= slo["max_interval_min"]
    )

def check_drill(case):
    drill = case["drill"]
    return drill["automated"] and drill["restore_test_age_days"] <= 14 and drill["failure_modes"] >= 3

def check_gate(case):
    return case["gate"]["enabled"]

CHECKS = {
    "checkpoint_object_completeness": check_object,
    "shard_layout_fit": check_shard_layout,
    "async_save_overlap": check_async,
    "write_bandwidth_fit": check_write,
    "metadata_commit_integrity": check_metadata,
    "checksum_manifest_validation": check_checksum,
    "restore_replay_readiness": check_restore,
    "dataloader_rng_state_capture": check_rng,
    "distributed_rank_state_capture": check_rank_state,
    "retention_policy_fit": check_retention,
    "lifecycle_cost_governance": check_cost,
    "cross_region_replication_fit": check_replication,
    "security_access_control": check_security,
    "resume_slo_tracking": check_slo,

```



```

    "failure_drill_coverage": check_drill,
    "checkpoint_lifecycle_gate": check_gate,
}

```

```

def audit_checkpoint_lifecycle(cases):
    case_failures = {}
    for case in cases:
        failures = [name for name, check in CHECKS.items() if not check(case)]
        case_failures[case["name"]] = failures

    metrics = {}
    for name, check in CHECKS.items():
        metrics[name] = round(sum(int(check(case)) for case in cases) / len(cases), 3)

    failed_cases = [name for name, failures in case_failures.items() if failures]
    return {
        "metrics": metrics,
        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "checkpoint_gate_pass": not failed_cases and min(metrics.values()) >= 0.95,
    }

```

```

cases = build_checkpoint_cases()
case_by_name = {case["name"]: case for case in cases}
complete = case_by_name["complete"]
write_path = complete["write_path"]
restore_path = complete["restore"]
total_save_s = save_time_s(
    complete["shard_layout"]["total_gib"],
    write_path["write_gib_s"],
    write_path["efficiency"],
    write_path["sync_s"],
    write_path["metadata_s"],
    write_path["commit_s"],
)
checkpoint_examples = {
    "shard_size_gib": round(shard_size_gib(3200, 64), 1),
    "save_3200gib_s": round(total_save_s, 1),
    "async_visible_overhead_s": round(visible_save_overhead_s(total_save_s, complete["async_save"]["overlap_ratio"]), 1),
    "restore_3200gib_s": round(restore_time_s(3200, restore_path["read_gib_s"], restore_path["efficiency"], 20, 10, 2), 1),
    "max_lost_work_min": complete["slo"]["max_interval_min"],
    "monthly_retention_cost_usd": monthly_checkpoint_cost_usd(120, 480, 23, 4, 320),
}

smoke = {
    "complete_case_passes": all(check(complete) for check in CHECKS.values()),
    "caught_weights_only": not check_object(case_by_name["weights_only_checkpoint_bad"]),
    "caught_single_shard": not check_shard_layout(case_by_name["single_huge_file_bad"]),
    "caught_commit_gap": not check_metadata(case_by_name["metadata_marked_before_commit_bad"]),
    "caught_restore_gap": not check_restore(case_by_name["restore_never_tested_bad"]),
    "caught_cost_gap": not check_cost(case_by_name["cold_archive_never_used_bad"]),
}

```

```
audit = audit_checkpoint_lifecycle(cases)
print(f"checkpoint_examples={checkpoint_examples}")
print(f"smoke={smoke}")
print(f"metrics={audit['metrics']}")
print(f"hard_blocker_count={audit['hard_blocker_count']}")
print(f"failed_cases={audit['failed_cases']}")
print(f"checkpoint_gate_pass={audit['checkpoint_gate_pass']}")
```

```
checkpoint_examples={'shard_size_gib': 50.0, 'save_3200gib_s': 83.3, 'async_visible_overhead_s': 15.0, 'restore_3200gib_s': 100.0, 'smoke':{'complete_case_passes': True, 'caught_weights_only': True, 'caught_single_shard': True, 'caught_commit_gap': True}, 'metrics':{'checkpoint_object_completeness': 0.941, 'shard_layout_fit': 0.941, 'async_save_overlap': 0.941, 'write_bandwidth_gib_s': 0.941, 'hard_blocker_count': 16, 'failed_cases':['weights_only_checkpoint_bad', 'single_huge_file_bad', 'blocking_save_bad', 'write_bandwidth_unknown_bad'], 'checkpoint_gate_pass': False}
```

10.20 面试中如何回答 checkpoint 系统设计

如何设计大模型训练平台的 checkpoint 系统?

可以这样回答：

我会先区分两类 checkpoint: 用于推理和发布的模型权重 checkpoint, 以及用于断点续训的训练状态 checkpoint。后者需要保存模型

存储上采用分片 checkpoint, 每个 rank 或参数 shard 并行写入, 并用 manifest 记录 shard、checksum、并行配置、数据版本和训练

为了降低训练阻塞，可以支持异步 checkpoint，但要处理一致性、内存占用和 I/O 干扰。平台需要监控 save/load duration、失败率

生命周期上保留最近 N 个 checkpoint、关键 milestone、最佳 eval 和发布版本，临时和失败 checkpoint 自动清理，冷版本迁移到低

10.21 常见误区

训练恢复需要完整训练状态，不只是模型参数。

误区二：保存成功就等于可恢复。

必须验证 manifest、checksum、配置兼容性，并做恢复测试。

误区三：checkpoint 越频繁越好。

频繁保存会增加 I/O、阻塞训练和存储成本，需要权衡恢复点目标和性能。

误区四：异步 checkpoint 一定更好。

异步保存能减少阻塞，但会带来一致性、内存占用和失败状态复杂度。

误区五：checkpoint 可以无限保留。

大模型 checkpoint 成本极高，必须做生命周期和成本治理。

10.22 面试题

题 1: 模型权重 checkpoint 和训练状态 checkpoint 有什么区别?

答: 模型权重 checkpoint 主要保存模型参数, 适合推理、评估和下游微调。训练状态 checkpoint 保存恢复训练所需的完整状态, 包括模型参数、优化器状态、scheduler、global step、随机种子、dataloader 状态和分布式 metadata, 适合断点续训和故障恢复。

题 2: 为什么大模型 checkpoint 很大?

答: 因为除了模型参数, 还可能包含梯度、优化器状态、一阶动量、二阶动量、FP32 master weight、分布式分片和 metadata。使用 Adam 时训练状态通常远大于纯模型权重。

题 3: 为什么 checkpoint 需要 manifest?

答: 分布式 checkpoint 通常由多个 shard 组成。Manifest 记录 checkpoint id、global step、并行配置、shard 列表、大小、checksum、数据版本和状态。没有 manifest, 系统很难判断哪些文件组成同一个一致可恢复版本。

题 4: 异步 checkpoint 有什么风险?

答: 异步保存可能增加内存占用, 和训练争抢 I/O, 出现写入失败或状态不一致。必须有清晰的 snapshot、校验、commit 语义和失败处理, 不能让半成品 checkpoint 被恢复使用。

题 5: 恢复后 loss 异常可能是什么原因?

答: 可能是 optimizer 状态、scheduler、random seed、dataloader 状态、mixed precision scaler 或数据位置没有正确恢复, 也可能是代码、配置、tokenizer 或并行策略变化导致不兼容。

10.23 小练习

练习一: 设计一个 checkpoint manifest。

要求: 包含 checkpoint_id、global_step、model_config、parallelism_config、data_version、shards、checksum、status 和 created_at。

练习二: 设计一个 checkpoint 生命周期策略。

要求: 说明最近 checkpoint、milestone checkpoint、最佳 eval checkpoint、发布 checkpoint 和失败临时 checkpoint 分别保留多久。

练习三: 分析 checkpoint 保存慢。

假设一个 256 卡训练任务每次保存 checkpoint 要 20 分钟, 请从 shard、存储吞吐、metadata、网络、同步阻塞和并发任务角度排查。

练习四: 设计一个恢复测试流程。

要求: 模拟训练中断, 从 checkpoint 恢复, 并验证 step、loss、学习率、数据位置和 optimizer state 是否正确。

10.24 本章小结

本章讲了 checkpoint 存储、加载、恢复和生命周期管理。

你需要掌握:

1. Checkpoint 是训练状态的可恢复快照, 不只是模型文件。
2. 模型权重 checkpoint 和训练状态 checkpoint 用途不同。
3. 训练状态需要保存参数、优化器、scheduler、step、随机状态、dataloader 和分布式 metadata。
4. 保存频率需要在训练阻塞、存储成本和故障恢复损失之间权衡。
5. 分片 checkpoint 需要 manifest、checksum 和 commit 语义。

6. 恢复训练要关注配置兼容性和状态一致性。
7. 弹性恢复需要 resharding, 但实现复杂。
8. Checkpoint 是抢占式调度和容错训练的基础。
9. 生命周期管理和成本治理非常重要。
10. Checkpoint 必须定期做恢复测试, 不能只测试保存。

下一章我们会讲容器化基础: Docker、镜像、驱动、CUDA 和依赖环境。

第 11 章 容器化基础: Docker、镜像、驱动、CUDA 和依赖环境

前面我们讲了 GPU 集群、网络、存储和 checkpoint。本章讲一个训练平台和推理平台每天都会遇到的问题: 环境。

大模型项目里, 很多故障不是算法错了, 而是环境不一致: CUDA 版本不对、驱动不匹配、PyTorch 版本不同、NCCL 配置变化、Python 依赖漂移、系统库缺失、镜像里混入了临时修改。

容器化就是为了解决这些问题的一种基础能力。

先记住一句话:

容器化的核心价值不是“把程序打包”, 而是把训练和推理环境变成可复现、可分发、可调度、可审计的运行单元。

11.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时, 资料口径按“AI 训练与推理容器环境的稳定抽象”处理, 而不是绑定某个 Kubernetes 集群、某个镜像仓库、某个 CUDA 镜像 tag 或某家云厂商实现。镜像构建部分参考 Docker 官方对 Dockerfile best practices、多阶段构建、基础镜像 digest pin 和 CI 测试的建议; GPU 容器部分参考 NVIDIA Container Toolkit 对 runtime / CDI、GPU device 暴露、driver capabilities 和容器内 GPU 可见性的边界; CUDA 兼容性部分参考 NVIDIA CUDA compatibility 对 driver 与 CUDA Toolkit / runtime 的兼容关系; 镜像安全部分参考 Docker Scout / 镜像扫描、签名、SBOM 和 registry 权限治理的通用口径。

需要注意三点:

1. 本章讲容器镜像、GPU runtime、驱动 / CUDA / 框架兼容、依赖锁定、安全扫描和环境可复现, 不把下一章 Kubernetes device plugin、Pod 调度和 GPU 资源管理提前展开。
2. 容器不是虚拟机, 也不是绝对安全边界。GPU 容器仍依赖宿主机驱动、内核、设备挂载、网络和多租户权限策略。
3. 训练镜像、推理镜像、基础镜像和项目镜像要分层治理; 复现时不能只记录 tag, 必须记录 digest、驱动、CUDA、框架、依赖、代码、数据和启动命令。

11.1 为什么 AI Infra 需要容器化

没有容器化时, 训练任务可能直接跑在机器环境里。

这会带来很多问题:

1. 不同机器 Python 版本不同。
2. CUDA 版本不同。
3. PyTorch 版本不同。
4. 系统库版本不同。
5. 用户手工安装依赖互相污染。
6. 任务迁移到另一台机器后跑不起来。
7. 环境修改无法追踪。
8. 线上推理和离线训练环境不一致。

容器化可以把代码运行需要的用户态环境打包起来, 让任务在不同机器上更一致地运行。

在 AI Infra 里, 容器化支撑:

1. 训练任务提交。
2. 推理服务部署。
3. 评估任务运行。
4. 多租户隔离。
5. 依赖复现。
6. 镜像安全扫描。
7. 灰度发布和回滚。

11.2 Docker 是什么

Docker 是常见的容器技术。

你可以把 Docker 镜像理解成一个打包好的运行环境。

它通常包含：

1. 基础操作系统文件。
2. Python 或其他语言运行时。
3. CUDA 用户态库。
4. 深度学习框架。
5. 系统依赖。
6. Python 包。
7. 训练或推理代码。
8. 启动脚本。

容器运行时会从镜像启动容器。

容器不是虚拟机。它共享宿主机内核，但有自己的文件系统、进程空间、环境变量和资源限制。

11.3 镜像、容器和仓库

几个概念要区分。

镜像 image：静态包，包含运行环境。

容器 container：镜像运行起来后的进程环境。

镜像仓库 registry：存放镜像的服务。

tag：镜像标签，例如 `train:cuda12.1-pytorch2.2`。

digest：镜像内容哈希，比 tag 更可靠。

为什么 digest 重要？

因为 tag 可以被覆盖。

例如今天 `latest` 指向一个镜像，明天可能指向另一个镜像。为了复现训练，最好记录镜像 digest。

11.4 镜像分层

Docker 镜像由多层组成。

例如：

```
base OS
-> CUDA runtime
-> PyTorch
-> system dependencies
-> python packages
-> project code
-> startup scripts
```

分层的好处：

1. 复用公共层。
2. 加快构建和分发。
3. 减少重复存储。
4. 便于组织基础镜像和业务镜像。

AI 平台常见做法：

1. 平台维护基础镜像。
2. 算法团队基于基础镜像构建项目镜像。
3. 推理平台维护 serving 镜像。
4. 安全团队扫描镜像漏洞。

不要让每个项目从零构建所有依赖，否则镜像会混乱且难维护。

11.5 GPU 容器和普通容器的区别

普通容器主要使用 CPU、内存和文件系统。

GPU 容器还需要访问宿主机 GPU 设备。

这涉及：

1. NVIDIA driver。
2. CUDA 用户态库。
3. GPU device file。
4. container runtime hook。
5. NCCL 和通信库。
6. MIG 或 GPU 分配策略。

一个关键点：驱动通常在宿主机上，容器里放的是用户态 CUDA 库和框架。

因此 GPU 容器不是完全自包含的，它仍然依赖宿主机驱动。

11.6 驱动、CUDA 和 PyTorch 的关系

这是 AI 环境最容易出错的地方。

可以这样理解：

1. NVIDIA Driver：宿主机驱动，负责和 GPU 硬件交互。
2. CUDA Driver API：驱动接口。
3. CUDA Runtime / Toolkit：用户态 CUDA 库和开发工具。
4. cuDNN、cuBLAS、NCCL：深度学习常用库。
5. PyTorch / TensorFlow：深度学习框架。

版本关系很重要。

常见问题：

1. 宿主机驱动太旧，不支持容器里的 CUDA。
2. PyTorch 编译的 CUDA 版本和运行环境不兼容。
3. NCCL 版本不匹配导致通信异常。
4. cuDNN 版本变化导致性能或行为差异。
5. 容器里有多多个 CUDA 路径，实际加载了错误版本。

面试中可以用一句话总结：

容器可以打包用户态依赖，但不能忽略宿主机 GPU 驱动和内核环境；CUDA、驱动、框架和通信库需要形成兼容矩阵。

11.7 CUDA 兼容性

CUDA 兼容性不是简单 “版本越新越好”。

需要考虑：

1. GPU 架构是否支持。
2. 驱动版本是否支持。
3. PyTorch 是否支持。
4. 自定义 CUDA kernel 是否兼容。
5. Triton kernel 是否兼容。
6. NCCL 是否兼容。
7. 推理 runtime 是否兼容。

平台常见做法：

1. 维护少数标准 CUDA 版本。
2. 建立驱动、CUDA、PyTorch、NCCL 兼容矩阵。
3. 禁止随意使用未经验证的组合。
4. 新版本先灰度验证。
5. 保留旧版本镜像用于复现历史实验。

标准化比 “大家自由装” 更可靠。

11.8 依赖管理

AI 项目依赖很多。

包括：

1. Python 版本。
2. PyTorch / TensorFlow。
3. Transformers。
4. tokenizers。
5. DeepSpeed。
6. Megatron。
7. vLLM / TGI / SGLang。
8. Triton。
9. FlashAttention。
10. xFormers。
11. NCCL。
12. 系统库。

依赖管理常见问题：

1. 没有锁版本。
2. 依赖自动升级导致行为变化。
3. 开发环境和训练环境不同。
4. 训练环境和推理环境不同。
5. 镜像构建不可复现。
6. 私有包版本缺失。

建议：

1. 锁定关键依赖版本。
2. 记录镜像 digest。
3. 使用可复现构建流程。
4. 训练、评估、推理环境分别管理。
5. 对基础镜像做版本化。
6. 镜像构建后跑 smoke test。

11.9 训练镜像和推理镜像的区别

训练镜像和推理镜像不一定一样。

训练镜像通常包含：

1. 训练框架。
2. 分布式训练库。
3. 数据处理工具。
4. 实验追踪 SDK。
5. 调试工具。
6. profiler。
7. 编译工具。

推理镜像通常更关注：

1. serving runtime。
2. 模型加载。
3. 高性能推理库。
4. 安全加固。
5. 镜像体积。
6. 冷启动速度。
7. 稳定性。

训练镜像可以相对重一些，因为它需要调试和实验。

推理镜像应该尽量精简、可审计、启动快。

不要直接把实验训练镜像拿去线上推理。

11.10 镜像体积问题

AI 镜像很容易变得巨大。

原因：

1. CUDA 库大。
2. PyTorch 大。
3. 编译工具多。
4. Python 依赖多。
5. 缓存没有清理。
6. 模型权重误打进镜像。
7. 多版本依赖共存。

镜像过大带来的问题：

1. 拉取慢。
2. 任务启动慢。
3. 推理扩容慢。
4. 镜像仓库存储成本高。
5. 节点磁盘压力大。

优化方法：

1. 使用基础镜像分层。
2. 清理构建缓存。
3. 多阶段构建。
4. 不把大模型权重打进镜像。
5. 训练和推理镜像拆分。
6. 只保留必要依赖。
7. 节点预拉取常用镜像。

11.11 镜像构建流水线

成熟平台应该有镜像构建流水线。

典型流程：

```
Code / Dockerfile
-> Build
-> Dependency Lock Check
-> Unit Test / Smoke Test
-> Security Scan
-> Push Registry
-> Record Digest
-> Deploy / Submit Job
```

Smoke test 可以包括：

1. Python 能否启动。
2. CUDA 是否可用。
3. PyTorch 是否能看到 GPU。
4. NCCL 简单通信是否正常。
5. 关键包是否能 import。
6. 模型小样本推理是否成功。

镜像不是构建成功就能用，必须验证能在目标集群运行。

11.12 镜像安全

镜像是供应链安全的重要入口。

风险包括：

1. 基础镜像有漏洞。
2. Python 包有漏洞。
3. 镜像里包含密钥。
4. 镜像里包含敏感数据。
5. 使用未审核第三方包。
6. 镜像 tag 被覆盖。
7. 用户以 root 权限运行容器。
8. 容器逃逸风险。

安全措施：

1. 镜像漏洞扫描。
2. 禁止把密钥写进镜像。
3. 使用镜像签名。
4. 记录 digest。
5. 限制 root 权限。
6. 基础镜像定期更新。
7. 私有 registry 权限控制。
8. 构建日志审计。

AI 镜像经常包含大量第三方包，更需要供应链治理。

11.13 容器和多租户隔离

容器提供一定隔离，但不是强安全边界。

隔离包括：

1. 文件系统隔离。

2. 进程隔离。
3. 网络隔离。
4. 资源限制。
5. 环境变量隔离。

但 GPU、多租户和高权限挂载会带来额外风险。

例如：

1. 容器挂载了宿主机目录。
2. 容器以 privileged 模式运行。
3. 多租户共享同一 GPU。
4. 任务能访问不该访问的数据卷。
5. 环境变量泄露 token。

因此容器化要配合 Kubernetes、权限系统、密钥管理、网络策略和审计一起使用。

11.14 环境可复现性

训练复现需要记录环境。

至少记录：

1. 镜像 digest。
2. Python 版本。
3. CUDA 版本。
4. PyTorch 版本。
5. NCCL 版本。
6. 关键包版本。
7. 驱动版本。
8. GPU 型号。
9. 启动命令。
10. 环境变量。
11. 代码 commit。
12. 数据版本。

没有这些信息，几个月后很难复现实验。

实验追踪平台应该自动记录环境 metadata，而不是靠人手写。

11.15 常见环境问题

11.15.1 容器里看不到 GPU

可能原因：

1. 没有正确配置 GPU runtime。
2. Kubernetes 没有分配 GPU。
3. device plugin 异常。
4. 驱动问题。
5. 容器权限问题。

11.15.2 CUDA 版本不匹配

可能原因：

1. 宿主机驱动太旧。
2. 容器 CUDA 版本太新。
3. PyTorch 编译版本不匹配。
4. 系统加载了错误 CUDA 路径。

11.15.3 NCCL 通信失败

可能原因：

1. NCCL 版本不兼容。
2. 网卡配置不正确。
3. 容器网络限制。
4. RDMA 设备没有挂载进容器。
5. 环境变量配置错误。

11.15.4 本地能跑，集群不能跑

可能原因：

1. 本地环境和镜像不同。
2. 依赖没有写入镜像。
3. 使用了本地绝对路径。
4. 数据挂载路径不同。
5. 集群驱动和 CUDA 组合不同。
6. 启动命令或环境变量不同。

11.16 容器环境审计指标与最小 demo

可以把容器环境从“一个 Dockerfile”写成一组可审计对象。

一个 AI 容器环境样本可以抽象为：

$e_i = (b_i, d_i, g_i, c_i, l_i, p_i, s_i, r_i, m_i, n_i, a_i, z_i)$

其中， b_i 是基础镜像和 layer 信息， d_i 是镜像 digest， g_i 是 GPU runtime 和设备可见性， c_i 是 CUDA / driver / framework 兼容矩阵， l_i 是依赖锁定信息， p_i 是构建流水线和 smoke test， s_i 是镜像扫描、签名和 SBOM， r_i 是 registry 权限， m_i 是运行时 metadata， n_i 是多租户挂载和网络边界， a_i 是审计日志， z_i 是最终门禁结果。

镜像拉取和冷启动时间可以粗略拆成：

$$T_{\mathrm{pull}} = \frac{S_{\mathrm{img}}}{\eta B_{\mathrm{reg}}} + T_{\mathrm{unpack}}$$

$$T_{\mathrm{start}} = T_{\mathrm{pull}} + T_{\mathrm{init}} + T_{\mathrm{smoke}}$$

其中， S_{img} 是需要拉取的镜像大小， B_{reg} 是 registry 到节点的有效带宽， η 是有效传输系数， T_{unpack} 是解压和写入本地镜像层的时间， T_{init} 是进程初始化时间， T_{smoke} 是启动前 smoke test 的时间。

镜像层复用率可以写成：

$$R_{\mathrm{layer}} = \frac{S_{\mathrm{cached}}}{S_{\mathrm{layer}}}$$

其中， S_{cached} 是节点已缓存的 layer 大小， S_{layer} 是镜像总 layer 大小。复用率越高，任务启动和推理扩容时越不容易被镜像拉取拖慢。

依赖锁定覆盖率可以写成：

$$C_{\mathrm{lock}} = \frac{N_{\mathrm{pinned}}}{N_{\mathrm{dep}}}$$

其中， N_{pinned} 是锁定版本或 hash 的关键依赖数量， N_{dep} 是关键依赖总数。对 CUDA、PyTorch、NCCL、Triton、FlashAttention、serving runtime 和私有 wheel，缺少锁定会直接影响可复现性。

GPU 容器兼容性可以写成一个硬门禁：

$$G_{\mathrm{cuda}} = \mathbf{1} \left[v_{\mathrm{driver}} \geq v_{\mathrm{min}}(u_{\mathrm{cuda}}) \wedge C_{\mathrm{fw}} = 1 \right]$$

其中， v_{driver} 是宿主机驱动版本， $v_{\mathrm{min}}(u_{\mathrm{cuda}})$ 是容器内 CUDA 用户态版本所需的最低驱动口径， C_{fw} 表示框架、NCCL、cuDNN、Triton / 自定义 kernel 版本兼容， C_{gpu} 表示 GPU 架构和 device 暴露正确。

最后，可以把容器环境门禁写成：

$$G_{\{\mathrm{container}\}} = \mathbf{1} \setminus \left[\min_j C_j \geq \tau_j \text{ \& } T_{\{\mathrm{start}\}} \leq \tau_{\{\mathrm{start}\}} \text{ \& } V_{\{\mathrm{crit}\}} \right]$$

其中, C_j 是第 j 个容器环境审计指标覆盖率, τ_j 是覆盖率阈值, τ_{start} 是冷启动 SLO, V_{crit} 是 critical 漏洞数量, P_0 是 P0 级风险数量。

下面这个 0 依赖 demo 演示如何把容器环境写成审计规则。它故意构造 1 个完整样本和 16 个坏样本, 让每个关键维度各失败一次。

```
import copy
```

```
METRICS = [
    "image_digest_reproducibility",
    "base_layer_cache_fit",
    "cuda_driver_framework_compatibility",
    "gpu_runtime_visibility",
    "dependency_lock_coverage",
    "training_serving_image_separation",
    "image_size_startup_fit",
    "build_pipeline_smoke_test",
    "security_scan_signing",
    "secret_data_exclusion",
    "runtime_hardenig_fit",
    "registry_access_governance",
    "environment_metadata_capture",
    "multi_tenant_mount_isolation",
    "nccl_rdma_runtime_fit",
    "container_environment_gate",
]
```

```
def layer_cache_hit(cached_gib, total_layer_gib):
    return cached_gib / total_layer_gib
```

```
def pull_time_s(size_gib, bandwidth_gib_s, efficiency, unpack_s):
    return size_gib / (bandwidth_gib_s * efficiency) + unpack_s
```

```
def cold_start_time_s(size_gib, bandwidth_gib_s, efficiency, unpack_s, init_s, smoke_s):
    return pull_time_s(size_gib, bandwidth_gib_s, efficiency, unpack_s) + init_s + smoke_s
```

```
def lock_coverage(pinned, total):
    return pinned / total
```

```
def build_container_cases():
    complete = {
        "name": "complete",
        "image": {
            "tag": "train:cuda12.4-py310",
            "digest_pinned": True,
            "base_digest_pinned": True,
            "tag_immutable": True,
            "layers_total_gib": 18,
            "cached_layers_gib": 14,
```

```

},
"cuda": {
  "host_driver": 550,
  "min_driver": 545,
  "cuda_user": "12.4",
  "framework_cuda": "12.4",
  "gpu_arch_supported": True,
  "library_set_consistent": True,
},
"gpu_runtime": {
  "runtime_hook": True,
  "device_plugin": True,
  "visible_devices": "GPU-uuid",
  "driver_capabilities": {"compute", "utility"},
  "gpu_smoke_test": True,
},
"dependency": {
  "lockfile": True,
  "pinned": 42,
  "total": 42,
  "private_wheel_hashes": True,
},
"image_role": {
  "role": "training",
  "serving_from_training": False,
  "serving_slimmed": True,
},
"startup": {
  "image_size_gib": 24,
  "registry_gib_s": 2.0,
  "efficiency": 0.8,
  "unpack_s": 12,
  "init_s": 8,
  "smoke_s": 10,
  "startup_slo_s": 50,
},
"build": {
  "ci": True,
  "smoke_tests": {"python", "cuda", "torch", "nccl", "import"},
  "digest_recorded": True,
  "sbom": True,
},
"security": {
  "scan": True,
  "signed": True,
  "critical_vulns": 0,
  "high_vulns": 0,
},
"secret_data": {
  "secrets_in_image": 0,
  "sensitive_data_files": 0,
  "model_weights_in_image": False,
},
"hardening": {
  "run_as_root": False,

```

```

        "privileged": False,
        "readonly_rootfs": True,
        "dropped_caps": True,
    },
    "registry": {
        "private": True,
        "rbac": True,
        "retention": True,
        "audit_log": True,
    },
    "metadata": {
        "image_digest": True,
        "python": True,
        "cuda": True,
        "driver": True,
        "torch": True,
        "nccl": True,
        "command": True,
        "env": True,
        "code_commit": True,
        "data_version": True,
    },
    "mounts": {
        "host_path_wildcard": False,
        "tenant_volume_scope": True,
        "secret_env_masked": True,
    },
    "nccl": {
        "rdma_devices_mounted": True,
        "ib_visible": True,
        "network_mode_ok": True,
        "nccl_smoke_test": True,
    },
    "gate": {"enabled": True},
}

```

```

def bad_case(name, mutator):
    case = copy.deepcopy(complete)
    case["name"] = name
    mutator(case)
    return case

```

```

bad_cases = [
    bad_case("tag_only_latest_bad", lambda c: c["image"].update({"digest_pinned": False})),
    bad_case("layer_cache_miss_bad", lambda c: c["image"].update({"cached_layers_gib": 2})),
    bad_case("old_driver_bad", lambda c: c["cuda"].update({"host_driver": 520})),
    bad_case("gpu_runtime_missing_bad", lambda c: c["gpu_runtime"].update({"runtime_hook": False})),
    bad_case("dependency_unpinned_bad", lambda c: c["dependency"].update({"lockfile": False, "pinned": 30})),
    bad_case("training_image_as_serving_bad", lambda c: c["image_role"].update({"serving_from_training": True})),
    bad_case("huge_image_cold_start_bad", lambda c: c["startup"].update({"image_size_gib": 80})),
    bad_case("smoke_test_missing_bad", lambda c: c["build"].update({"smoke_tests": {"python", "import"}})),
    bad_case("unsigned_vulnerable_image_bad", lambda c: c["security"].update({"signed": False, "critical_vulns": 1})),
    bad_case("secret_baked_in_bad", lambda c: c["secret_data"].update({"secrets_in_image": 1})),
    bad_case("root_privileged_bad", lambda c: c["hardening"].update({"run_as_root": True, "privileged": True})),
    bad_case("open_registry_bad", lambda c: c["registry"].update({"private": False})),

```

```

        bad_case("metadata_missing_bad", lambda c: c["metadata"].update({"driver": False})),
        bad_case("wildcard_host_mount_bad", lambda c: c["mounts"].update({"host_path_wildcard": True})),
        bad_case("rdma_not_visible_bad", lambda c: c["nccl"].update({"rdma_devices_mounted": False})),
        bad_case("container_gate_missing_bad", lambda c: c["gate"].update({"enabled": False})),
    ]
    return [complete] + bad_cases

def check_image_digest(case):
    image = case["image"]
    return image["digest_pinned"] and image["base_digest_pinned"] and image["tag_immutable"]

def check_layer_cache(case):
    image = case["image"]
    return layer_cache_hit(image["cached_layers_gib"], image["layers_total_gib"]) >= 0.6

def check_cuda(case):
    cuda = case["cuda"]
    return (
        cuda["host_driver"] >= cuda["min_driver"]
        and cuda["cuda_user"] == cuda["framework_cuda"]
        and cuda["gpu_arch_supported"]
        and cuda["library_set_consistent"]
    )

def check_gpu_runtime(case):
    runtime = case["gpu_runtime"]
    return (
        runtime["runtime_hook"]
        and runtime["device_plugin"]
        and bool(runtime["visible_devices"])
        and {"compute", "utility"}.issubset(runtime["driver_capabilities"])
        and runtime["gpu_smoke_test"]
    )

def check_dependency(case):
    dep = case["dependency"]
    return dep["lockfile"] and lock_coverage(dep["pinned"], dep["total"]) >= 0.95 and dep["private_wheel_hashes"]

def check_image_role(case):
    role = case["image_role"]
    return not role["serving_from_training"] and role["serving_slimmed"]

def check_startup(case):
    startup = case["startup"]
    start_s = cold_start_time_s(
        startup["image_size_gib"],
        startup["registry_gib_s"],
        startup["efficiency"],

```

```

        startup["unpack_s"],
        startup["init_s"],
        startup["smoke_s"],
    )
    return startup["image_size_gib"] <= 30 and start_s <= startup["startup_slo_s"]

def check_build(case):
    build = case["build"]
    required = {"python", "cuda", "torch", "nccl", "import"}
    return build["ci"] and required.issubset(build["smoke_tests"]) and build["digest_recorded"] and build["sbom"]

def check_security(case):
    sec = case["security"]
    return sec["scan"] and sec["signed"] and sec["critical_vulns"] == 0 and sec["high_vulns"] == 0

def check_secret_data(case):
    data = case["secret_data"]
    return data["secrets_in_image"] == 0 and data["sensitive_data_files"] == 0 and not data["model_weights_in_image"]

def check_hardening(case):
    hard = case["hardening"]
    return not hard["run_as_root"] and not hard["privileged"] and hard["readonly_rootfs"] and hard["dropped_caps"]

def check_registry(case):
    return all(case["registry"].values())

def check_metadata(case):
    return all(case["metadata"].values())

def check_mounts(case):
    mounts = case["mounts"]
    return not mounts["host_path_wildcard"] and mounts["tenant_volume_scope"] and mounts["secret_env_masked"]

def check_nccl(case):
    return all(case["nccl"].values())

def check_gate(case):
    return case["gate"]["enabled"]

CHECKS = {
    "image_digest_reproducibility": check_image_digest,
    "base_layer_cache_fit": check_layer_cache,
    "cuda_driver_framework_compatibility": check_cuda,
    "gpu_runtime_visibility": check_gpu_runtime,
    "dependency_lock_coverage": check_dependency,

```



```

    "training_serving_image_separation": check_image_role,
    "image_size_startup_fit": check_startup,
    "build_pipeline_smoke_test": check_build,
    "security_scan_signing": check_security,
    "secret_data_exclusion": check_secret_data,
    "runtime_hardening_fit": check_hardening,
    "registry_access_governance": check_registry,
    "environment_metadata_capture": check_metadata,
    "multi_tenant_mount_isolation": check_mounts,
    "nccl_rdma_runtime_fit": check_nccl,
    "container_environment_gate": check_gate,
}

```

```

def audit_container_environment(cases):
    case_failures = {}
    for case in cases:
        failures = [name for name, check in CHECKS.items() if not check(case)]
        case_failures[case["name"]] = failures

    metrics = {}
    for name, check in CHECKS.items():
        metrics[name] = round(sum(int(check(case)) for case in cases) / len(cases), 3)

    failed_cases = [name for name, failures in case_failures.items() if failures]
    return {
        "metrics": metrics,
        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "container_gate_pass": not failed_cases and min(metrics.values()) >= 0.95,
    }

```

```

cases = build_container_cases()
case_by_name = {case["name"]: case for case in cases}
complete = case_by_name["complete"]
startup = complete["startup"]
image = complete["image"]
container_examples = {
    "layer_cache_hit": round(layer_cache_hit(image["cached_layers_gib"], image["layers_total_gib"]), 3),
    "pull_24gib_s": round(pull_time_s(24, startup["registry_gib_s"], startup["efficiency"], startup["unpack_s"]), 1),
    "cold_start_24gib_s": round(cold_start_time_s(24, startup["registry_gib_s"], startup["efficiency"], startup["unpack_s"]), 1),
    "lock_coverage": round(lock_coverage(complete["dependency"]["pinned"], complete["dependency"]["total"]), 3),
    "driver_margin": complete["cuda"]["host_driver"] - complete["cuda"]["min_driver"],
    "critical_vulns": complete["security"]["critical_vulns"],
}

```

```

smoke = {
    "complete_case_passes": all(check(complete) for check in CHECKS.values()),
    "caught_tag_only": not check_image_digest(case_by_name["tag_only_latest_bad"]),
    "caught_old_driver": not check_cuda(case_by_name["old_driver_bad"]),
    "caught_missing_gpu_runtime": not check_gpu_runtime(case_by_name["gpu_runtime_missing_bad"]),
    "caught_unpinned_deps": not check_dependency(case_by_name["dependency_unpinned_bad"]),
    "caught_secret": not check_secret_data(case_by_name["secret_baked_in_bad"]),
}

```

```

audit = audit_container_environment(cases)
print(f"container_examples={container_examples}")
print(f"smoke={smoke}")
print(f"metrics={audit['metrics']}")
print(f"hard_blocker_count={audit['hard_blocker_count']}")
print(f"failed_cases={audit['failed_cases']}")
print(f"container_gate_pass={audit['container_gate_pass']}")

```

一组典型输出是：

```

container_examples={'layer_cache_hit': 0.778, 'pull_24gib_s': 27.0, 'cold_start_24gib_s': 45.0, 'lock_coverage': 1.0, '
smoke={'complete_case_passes': True, 'caught_tag_only': True, 'caught_old_driver': True, 'caught_missing_gpu_runtime':
metrics={'image_digest_reproducibility': 0.941, 'base_layer_cache_fit': 0.941, 'cuda_driver_framework_compatibility':
hard_blocker_count=16
failed_cases=['tag_only_latest_bad', 'layer_cache_miss_bad', 'old_driver_bad', 'gpu_runtime_missing_bad', 'dependency
container_gate_pass=False

```

这个 demo 的重点是把容器环境拆成可验证证据链：digest 和基础镜像要可复现，layer cache 和镜像体积影响冷启动，CUDA / driver / framework 要形成兼容矩阵，GPU runtime 必须真的能看到设备，依赖要锁版本，训练和推理镜像要分离，构建流水线要跑 smoke test，镜像要扫描和签名，密钥和模型权重不能打进镜像，多租户挂载、registry 权限、环境 metadata、NCCL / RDMA 和最终门禁都要能审计。

11.17 面试中如何回答容器化设计

如果面试官问：

大模型训练平台为什么需要容器化？如何设计镜像体系？

可以这样回答：

容器化的核心是让训练和推理环境可复现、可分发、可调度和可审计。大模型环境依赖复杂，包括 CUDA、cuDNN、NCCL、PyTorch、De

我会设计分层镜像体系：平台团队维护经过验证的基础镜像，包含 OS、CUDA、cuDNN、NCCL 和 PyTorch；算法团队基于基础镜像添加

镜像构建流程要包含依赖锁定、smoke test、GPU 可用性测试、NCCL 测试、安全扫描、签名和 registry 权限控制。训练任务提交时记

11.18 常见误区

误区一：用了 Docker 就完全可复现。

容器仍依赖宿主机驱动、GPU、内核和设备挂载。还需要记录镜像 digest、驱动版本、代码和数据版本。

误区二：镜像 tag 足够可靠。

Tag 可以被覆盖，复现时应该记录 digest。

误区三：训练镜像可以直接用于线上推理。

训练镜像通常很重，包含调试和编译工具。推理镜像应该更精简、安全、启动快。

误区四：CUDA 版本越新越好。

新版本要和驱动、框架、NCCL、自定义 kernel 和推理 runtime 兼容，必须经过验证。

误区五：容器就是安全边界。

容器不是绝对安全边界，多租户场景还需要权限、网络策略、密钥管理、审计和宿主机安全。

11.19 面试题

题 1: Docker 镜像和容器有什么区别?

答: 镜像是静态的运行环境包, 包含文件系统和依赖; 容器是镜像运行后的进程环境。镜像可以存到 registry, 容器在具体机器上启动运行。

题 2: 为什么 GPU 容器仍然依赖宿主机驱动?

答: GPU 驱动通常安装在宿主机上, 负责和硬件交互。容器里主要包含 CUDA 用户态库和框架。容器需要通过 GPU runtime 和设备挂载访问宿主机 GPU, 因此驱动、CUDA、框架需要兼容。

题 3: 为什么要记录镜像 digest?

答: Tag 可以被覆盖, 不能保证长期指向同一内容。Digest 是镜像内容哈希, 能唯一标识镜像版本, 有利于实验复现、审计和回滚。

题 4: 训练镜像和推理镜像有什么不同?

答: 训练镜像通常包含训练框架、分布式库、调试工具、profiler 和编译工具; 推理镜像更关注 serving runtime、启动速度、镜像体积、安全加固和稳定性。推理镜像应尽量精简。

题 5: 容器里 NCCL 通信失败可能是什么原因?

答: 可能是 NCCL 版本不兼容、网卡或 RDMA 设备没有挂载、容器网络策略限制、环境变量配置错误、驱动/CUDA 不匹配、GPU 到网卡亲和性或集群网络问题。

11.20 小练习

练习一: 设计一个训练基础镜像。

要求: 列出 OS、CUDA、cuDNN、NCCL、PyTorch、DeepSpeed、FlashAttention、调试工具和 smoke test。

练习二: 设计一个推理镜像优化方案。

要求: 说明如何减少镜像体积、加快冷启动、避免把模型权重打进镜像、做安全扫描和预热。

练习三: 排查“本地能跑, 集群不能跑”。

要求: 从镜像、依赖、驱动、CUDA、路径、数据挂载、环境变量和权限角度列出检查清单。

练习四: 设计一个镜像构建流水线。

要求: 包含构建、依赖锁定、GPU smoke test、安全扫描、签名、推送 registry 和记录 digest。

11.21 本章小结

本章讲了容器化基础。

你需要掌握:

1. 容器化让训练和推理环境可复现、可分发、可调度、可审计。
2. Docker 镜像是静态环境包, 容器是镜像运行后的实例。
3. GPU 容器仍依赖宿主机驱动, CUDA、驱动、框架和 NCCL 需要兼容。
4. 镜像 tag 不够可靠, 复现应记录 digest。
5. 训练镜像和推理镜像关注点不同, 不能简单混用。
6. 镜像体积会影响任务启动和推理扩容。
7. 镜像构建应包含依赖锁定、smoke test、安全扫描、签名和审计。
8. 容器不是绝对安全边界, 多租户还需要权限、网络、密钥和审计。
9. 实验追踪应记录镜像、驱动、CUDA、依赖、代码、数据和启动命令。

下一章我们会讲 Kubernetes 与 GPU 资源管理。

第 12 章 Kubernetes 与 GPU 资源管理

上一章讲了容器化。本章进入容器编排和资源管理：Kubernetes 如何管理 GPU？为什么很多 AI 平台会基于 Kubernetes 构建？为什么 Kubernetes 又不能直接解决所有大模型调度问题？

很多候选人一被问到 AI Infra，就回答“用 Kubernetes”。这不够。Kubernetes 是重要底座，但大模型训练和推理还需要 GPU device plugin、调度扩展、队列、配额、拓扑感知、gang scheduling、抢占、checkpoint 和多租户治理。

先记住一句话：

Kubernetes 能管理容器和基础资源，但大模型 GPU 资源管理还需要在它之上补齐 AI 工作负载特有的调度、拓扑、队列和治理能力。

12.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，资料口径按“Kubernetes 上的 GPU 资源管理稳定抽象”处理，而不是绑定某个云厂商、某个集群发行版、某个训练平台或某个调度器插件版本。Kubernetes 侧重点参考官方 device plugin、extended resource、ResourceQuota、node affinity、taint / toleration 和 topology spread 等文档口径；NVIDIA 侧重点参考 NVIDIA Kubernetes device plugin 对 GPU、MIG、time-slicing / MPS 共享模式的说明，以及 GPU Operator 对 driver、container toolkit、device plugin、DCGM exporter、MIG manager 和 GPU feature discovery 的管理边界。

需要注意三点：

1. Kubernetes 上游已有 gang scheduling 的 alpha 能力文档，但这不等于所有生产集群默认可用。真实平台仍然要明确是否启用上游能力，或用 Volcano、Kueue、YuniKorn、自研调度层等方式补齐队列、配额、gang、抢占和公平性。
2. 本章讲 GPU resource expression、device plugin、GPU Operator、MIG / 共享、拓扑感知、namespace / quota、多租户和监控排障；更细的队列、优先级、抢占、公平性和配额策略放到下一章展开。
3. 这里的 demo 是教学审计脚本，不调用真实 Kubernetes API。它帮助你把“GPU 能不能被 K8s 管起来”拆成可检查字段，真实平台还需要从 kube-state-metrics、scheduler events、device plugin logs、DCGM、训练平台数据库和审计日志采集证据。

12.1 Kubernetes 在 AI Infra 中的位置

Kubernetes，简称 K8s，是容器编排系统。

它负责：

1. 启动容器。
2. 调度 Pod 到节点。
3. 管理服务发现。
4. 管理资源 request / limit。
5. 管理配置和密钥。
6. 做健康检查。
7. 支持滚动升级。
8. 提供声明式 API。

在 AI Infra 中，K8s 常作为底层调度和运行平台。

训练平台、推理平台、评估平台可以把任务转换成 K8s 资源，例如 Pod、Job、Deployment、StatefulSet 或自定义 CRD。

但 K8s 默认更偏通用容器编排，不是专门为大规模分布式训练设计的。

12.2 Kubernetes 的基本对象

理解 K8s GPU 管理前，需要知道几个对象。

Pod: K8s 调度的最小单位，里面可以包含一个或多个容器。

Node: 集群中的机器。

Deployment: 管理无状态服务副本，适合推理服务。

Job: 运行一次性任务，适合离线任务。

StatefulSet: 管理有状态服务。

ConfigMap: 配置。

Secret: 密钥。

Service: 服务发现和负载均衡入口。

Namespace: 逻辑隔离空间。

CRD: 自定义资源定义，用来扩展 K8s API。

AI 平台通常会定义自己的 CRD，例如 TrainingJob、InferenceService、RayCluster、ModelDeployment。

12.3 Kubernetes 如何表达资源

K8s 使用 request 和 limit 表达资源需求。

例如 CPU 和内存：

```
resources:
  requests:
    cpu: "8"
    memory: "64Gi"
  limits:
    cpu: "8"
    memory: "64Gi"
```

GPU 通常作为 extended resource 表达：

```
resources:
  limits:
    nvidia.com/gpu: 4
```

这里的意思是这个 Pod 需要 4 张 NVIDIA GPU。

注意：GPU 通常是整数资源。默认情况下，K8s 不会像 CPU 那样自然切分 GPU。

如果要做 GPU 共享、MIG、时间片或虚拟化，需要额外机制。

12.4 Device Plugin

K8s 本身不知道某台机器上有哪些 GPU。

Device Plugin 负责把设备资源注册给 K8s。

NVIDIA GPU 常用 NVIDIA device plugin。

它负责：

1. 发现节点上的 GPU。
2. 向 kubelet 注册 GPU 资源。
3. 把 GPU 暴露给容器。
4. 设置设备文件。

5. 设置环境变量。
6. 支持 MIG 等模式。

没有 device plugin, K8s 调度器无法正确识别 GPU 资源。

Device Plugin 是 GPU 容器调度的关键桥梁。

12.5 GPU Operator

GPU Operator 是用于自动化管理 GPU 软件栈的工具。

它通常可以管理：

1. NVIDIA driver。
2. Container toolkit。
3. Device plugin。
4. DCGM exporter。
5. MIG manager。
6. GPU feature discovery。

它的价值是减少手工安装和维护 GPU 节点软件栈的复杂度。

但在大型生产集群里，团队仍然需要严格控制驱动版本、升级节奏和兼容矩阵。

不要以为用了 operator 就不用理解驱动、CUDA 和 device plugin。

12.6 GPU 调度的基本流程

一个 GPU Pod 的调度流程大致是：

1. 用户提交任务。
2. 平台生成 Pod / Job / CRD。
3. Pod 声明需要 GPU 数量。
4. K8s scheduler 查找有足够 GPU 的节点。
5. Pod 被绑定到节点。
6. kubelet 启动容器。
7. device plugin 分配 GPU 设备。
8. 容器内通过 CUDA 访问 GPU。

这个流程适合简单任务。

但大模型训练需要的不只是“找一台有 8 张空闲 GPU 的机器”。

它还需要：

1. 多个 Pod 同时启动。
2. 多节点 gang scheduling。
3. 拓扑感知。
4. 队列和优先级。
5. 抢占和恢复。
6. 配额和公平性。
7. 多租户隔离。

这些通常需要扩展。

12.7 Kubernetes 默认调度的局限

K8s 默认调度器对普通服务很强，但对大模型训练有局限。

主要包括：

1. 不天然支持 gang scheduling。

2. 对 GPU 拓扑理解有限。
3. 不天然理解训练任务整体性。
4. 队列和公平性能力有限。
5. GPU 碎片化问题明显。
6. 多租户成本治理需要额外系统。
7. 对分布式训练失败恢复支持有限。

例如一个 64 卡训练任务需要 8 台 8 卡机器同时启动。如果 K8s 只调度到了 6 台，任务无法正常开始，但资源可能已经被占住。

这就是 gang scheduling 要解决的问题。

12.8 Gang Scheduling

Gang scheduling 是指一组相关 Pod 要么一起调度成功，要么都不启动。

分布式训练非常需要它。

截至当前主流 Kubernetes 口径，gang scheduling 不是所有生产集群默认开启的基础能力。即使上游有 alpha 能力，训练平台也要把“是否支持 gang、是否有最小可用 worker 数、失败后是否释放已占资源、是否和队列 / quota / priority 联动”写进调度门禁，而不是只假设 scheduler 会自动理解分布式训练的整体性。

原因：

1. 所有 worker 需要同时参与训练。
2. 缺少部分 worker 任务无法启动。
3. 部分 Pod 先占资源会造成资源浪费。
4. rank 初始化依赖完整 world size。

例如：

任务需要 16 个 worker，每个 worker 8 GPU。

如果只能调度 12 个 worker，任务不能开始。

没有 gang scheduling，可能出现资源被部分 Pod 占住但训练无法进行的情况。

AI 平台常用 Volcano、Kueue、YuniKorn 或自研调度层来补齐这类能力。

12.9 GPU 碎片化

GPU 碎片化是集群利用率的大问题。

例如集群中有很多节点，每个节点 8 张 GPU。

如果很多任务各占 1 到 2 张 GPU，可能导致每台机器都剩几张卡，但没有一台机器能满足一个需要完整 8 卡的任务。

这就是碎片化。

碎片化影响：

1. 大任务排队。
2. GPU 利用率看似不低但可用性差。
3. 调度等待时间变长。
4. 拓扑变差。

缓解方法：

1. bin packing。
2. 按任务类型划分资源池。
3. 小任务集中放置。
4. 大任务预留完整节点。
5. 抢占低优先级碎片任务。
6. 定期整理或重调度。

12.10 拓扑感知调度

第 7 章讲过拓扑感知，这里结合 K8s 再看。

GPU 拓扑包括：

1. 同一节点内 GPU 是否通过 NVLink / NVSwitch 连接。
2. GPU 属于哪个 NUMA 域。
3. GPU 到网卡距离。
4. 节点在哪个机柜。
5. 节点之间网络路径如何。
6. 数据是否靠近节点。

K8s 默认调度器对这些细节理解有限。

AI 调度层需要根据任务特点选择资源。

例如：

1. tensor parallel 任务优先同机 NVSwitch GPU。
2. 多机训练优先同机柜节点。
3. 数据密集任务优先靠近缓存节点。
4. 推理服务要考虑 GPU 型号和显存大小。

拓扑感知调度能显著影响训练效率。

12.11 Node Label、Taint 和 Affinity

K8s 提供一些基础调度能力。

Node label 可以标记节点属性：

```
gpu-type=A100
gpu-count=8
network=ib
zone=rack-a
```

Node selector 可以选择节点。

Affinity 可以表达亲和性和反亲和性。

Taint 和 toleration 可以控制哪些 Pod 能调度到某类节点。

例如：

1. 训练任务调度到 GPU 节点。
2. 推理任务调度到推理资源池。
3. 高优先级任务调度到专用节点。
4. 普通任务不能调度到保留节点。

这些能力是基础，但复杂 AI 调度通常还需要自定义调度器或调度扩展。

12.12 GPU 共享、MIG 和时间片

GPU 默认通常按整卡分配。

但有些场景想共享 GPU。

常见方式：

1. MIG。
2. 时间片共享。
3. MPS。
4. 自定义 GPU 虚拟化。

MIG 是 Multi-Instance GPU，可以把支持的 GPU 切成多个硬件隔离实例。

适合：

1. 小模型推理。
2. 评估任务。
3. 交互式开发。
4. 需要隔离的小负载。

不适合：

1. 大模型训练。
2. 需要完整显存的任务。
3. 通信密集多卡任务。

时间片共享可以提高利用率，但隔离性和性能稳定性不如硬件隔离。

推理服务中可以考虑 GPU 共享，预训练通常不适合随意共享。

12.13 GPU 资源申请设计

训练平台不应该让用户直接写复杂 YAML。

更好的方式是提供任务配置：

```
resources:
  gpu_type: A100-80G
  gpu_count: 64
  gpu_per_node: 8
  network: ib
  priority: high
  preemptible: false
```

平台再把它转换成 K8s 资源和调度约束。

资源申请应该表达：

1. GPU 型号。
2. GPU 数量。
3. 每节点 GPU 数。
4. 显存需求。
5. 网络需求。
6. 存储需求。
7. 优先级。
8. 是否可抢占。
9. 预计运行时长。
10. 项目和租户。

这样调度系统才能做更好的决策。

12.14 K8s 上运行训练任务

在 K8s 上运行分布式训练，常见方式：

1. PyTorchJob。
2. MPIJob。
3. RayJob。
4. DeepSpeed launcher + Job。
5. 自研 TrainingJob CRD。

关键问题：

1. master 地址如何发现。

2. worker rank 如何分配。
3. world size 如何设置。
4. worker 如何同时启动。
5. 某个 worker 失败怎么办。
6. checkpoint 如何保存。
7. 日志如何聚合。
8. 任务状态如何展示。

K8s 提供基础运行能力，但训练语义通常由上层 operator 或平台补齐。

12.15 K8s 上运行推理服务

推理服务通常用 Deployment、StatefulSet 或专门的 InferenceService。

关注点：

1. 模型加载。
2. GPU 绑定。
3. 服务发现。
4. 自动扩缩容。
5. 滚动升级。
6. 灰度发布。
7. 健康检查。
8. 请求路由。
9. GPU 利用率和延迟监控。
10. 冷启动。

推理服务和训练任务不同，它是在线服务，要满足 SLO。

所以推理资源池通常需要更严格的隔离和优先级。

12.16 Namespace、Quota 和多租户

K8s 可以用 namespace 做逻辑隔离。

ResourceQuota 可以限制资源使用。

例如：

1. 每个团队最多使用多少 GPU。
2. 每个 namespace 最多创建多少 Pod。
3. 限制 CPU、内存和存储。
4. 限制高优先级任务数量。

但 AI 多租户还需要更细能力：

1. 按 GPU 型号配额。
2. 按资源池配额。
3. 按项目成本统计。
4. 按任务优先级排队。
5. 按团队公平分享。
6. 数据权限隔离。
7. 镜像和密钥权限隔离。

所以 namespace 和 quota 是基础，不是完整多租户方案。

12.17 GPU 监控

K8s 需要配合 GPU 监控。

常见监控组件包括 DCGM exporter、Prometheus、Grafana 等。

关键指标：

1. GPU utilization。
2. 显存使用。
3. 显存带宽。
4. GPU temperature。
5. power usage。
6. ECC error。
7. PCIe throughput。
8. GPU process。
9. Pod 到 GPU 的映射。
10. GPU 分配率。

AI 平台还要把 GPU 指标和任务关联起来。

例如：

```
team_a / job_123 / pod_worker_0 / gpu_3 / utilization=92%
```

否则你只能看到某张卡忙，不知道是谁在用。

12.18 常见问题和排查

12.18.1 Pod 一直 Pending

可能原因：

1. GPU 不足。
2. 指定 GPU 型号没有资源。
3. node selector 太严格。
4. taint 没有 toleration。
5. quota 不够。
6. gang scheduling 条件不满足。
7. 镜像拉取失败。

12.18.2 Pod 启动后看不到 GPU

可能原因：

1. device plugin 异常。
2. GPU runtime 配置错误。
3. 容器没有分配 GPU。
4. 驱动问题。
5. MIG 配置不正确。

12.18.3 GPU 空闲但任务排队

可能原因：

1. GPU 碎片化。
2. 任务需要完整节点。
3. 配额不足。
4. 优先级较低。
5. 拓扑约束不满足。
6. 资源池隔离导致不可用。

12.18.4 训练任务部分 worker 启动失败

可能原因：

1. 没有 gang scheduling。
2. 镜像拉取速度不一致。
3. 某些节点环境异常。
4. 网络或 DNS 问题。
5. rank 配置错误。
6. checkpoint 挂载失败。

12.19 Kubernetes GPU 资源管理审计指标与最小 demo

把 Kubernetes GPU 管理讲清楚，不能只说“Pod 里写 `nvidia.com/gpu`”。更工程化的说法是：平台要证明 GPU 设备能注册、资源请求能表达、调度约束能落地、分布式训练能整体启动、碎片和拓扑能被治理、租户配额能被约束、监控和排障能把 Pod 映射回具体 GPU。

可以把一个 GPU 任务或资源池审计样本写成：

$k_i = (p_i, n_i, d_i, g_i, m_i, q_i, a_i, t_i, r_i, o_i, s_i, z_i)$

其中， p_i 是 Pod / Job / CRD 画像， n_i 是节点与 GPU 拓扑， d_i 是 device plugin 状态， g_i 是 GPU request / limit， m_i 是 MIG / 共享策略， q_i 是 namespace / quota， a_i 是 affinity / taint / toleration， t_i 是 topology 证据， r_i 是训练或推理运行时， o_i 是监控观测， s_i 是排障事件， z_i 是最终门禁。

统一覆盖率可以写成：

$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(k_i)=1]$

其中， $g_j(k_i)=1$ 表示样本 i 通过第 j 个 Kubernetes GPU 管理检查。

GPU 分配率可以写成：

$U_{\{\mathrm{gpu}\}} = \frac{G_{\{\mathrm{allocated}\}}}{G_{\{\mathrm{total}\}}}$

其中， $G_{\mathrm{allocated}}$ 是已经分配给 Pod / Job 的 GPU 数， G_{total} 是资源池总 GPU 数。

碎片化比例可以写成：

$R_{\{\mathrm{frag}\}} = 1 - \frac{\sum_n \lfloor g_n / g_{\{\mathrm{job}\}} \rfloor}{\sum_n g_n}$

其中， g_n 是节点 n 上的空闲 GPU 数， g_{job} 是目标任务每个放置单元需要的 GPU 数。这个值越高，说明“总空闲 GPU 看起来够，但能满足完整任务形状的 GPU 越少”。

Gang scheduling 可行性可以写成：

$C_{\{\mathrm{gang}\}} = \mathbf{1}[\sum_n \lfloor g_n / g_{\{\mathrm{pernode}\}} \rfloor \geq N_{\{\mathrm{worker}\}}]$

其中， g_{pernode} 是每个 worker 需要的单节点 GPU 数， N_{worker} 是 worker 数量。

租户配额占用率可以写成：

$Q_{\{\mathrm{tenant}\}} = \frac{G_{\{\mathrm{used}\}}}{G_{\{\mathrm{quota}\}}}$

其中， G_{used} 是租户当前已用 GPU， G_{quota} 是租户配额。

拓扑得分可以写成：

$S_{\{\mathrm{topo}\}} = w_1 C_{\{\mathrm{node}\}} + w_2 C_{\{\mathrm{nic}\}} + w_3 C_{\{\mathrm{rack}\}} + w_4 C_{\{\mathrm{storage}\}}$

其中， C_{node} 、 C_{nic} 、 C_{rack} 、 C_{storage} 分别表示节点内 GPU 互联、GPU-NIC 亲和性、机柜 locality 和存储 locality 是否满足任务画像， w_i 是权重。

最后，可以把 Kubernetes GPU 资源管理门禁写成：

$G_{\{\mathrm{k8s_gpu}\}} = \mathbf{1}[\min_j C_j \geq \tau_j \wedge R_{\{\mathrm{frag}\}} \leq \rho \wedge Q_{\{\mathrm{tenant}\}} \leq 1]$

其中， τ_j 是每个检查项的最低覆盖率， ρ 是碎片化阈值， P_0 是 P0 级风险数量。

下面这个 0 依赖 demo 演示如何把 Kubernetes GPU 管理拆成审计规则。它故意构造 1 个完整样本和 16 个坏样本，让每个关键维度各失败一次。

```
import copy
```

```
METRICS = [  
    "device_plugin_readiness",  
    "gpu_extended_resource_fit",  
    "gpu_request_limit_integrity",  
    "gang_scheduling_readiness",  
    "fragmentation_control",  
    "topology_aware_placement",  
    "node_label_affinity_fit",  
    "taint_toleration_fit",  
    "mig_sharing_policy_fit",  
    "quota_namespace_governance",  
    "training_operator_readiness",  
    "inference_service_readiness",  
    "gpu_monitoring_mapping",  
    "multi_tenant_isolation",  
    "pending_troubleshooting_coverage",  
    "kubernetes_gpu_gate",  
]
```

```
def gpu_allocation_ratio(allocated, total):  
    return allocated / total
```

```
def fragmentation_ratio(free_gpus_by_node, gpus_per_job):  
    allocatable = sum((free // gpus_per_job) * gpus_per_job for free in free_gpus_by_node)  
    total_free = sum(free_gpus_by_node)  
    return 1 - allocatable / total_free
```

```
def gang_feasible(free_gpus_by_node, gpu_per_node, workers):  
    available_workers = sum(free // gpu_per_node for free in free_gpus_by_node)  
    return available_workers >= workers
```

```
def tenant_quota_ratio(used_gpu, quota_gpu):  
    return used_gpu / quota_gpu
```

```
def topology_score(components, weights):  
    return round(sum(components[name] * weights[name] for name in weights), 3)
```

```
def build_kubernetes_gpu_cases():  
    complete = {  
        "name": "complete",  
        "device_plugin": {  
            "ready": True,  
            "resource_name": "nvidia.com/gpu",  
            "node_capacity_reported": True,  
            "allocatable_reported": True,  
        },  
    },
```

```

"resource": {
  "extended_resource_declared": True,
  "limits_gpu": 8,
  "requests_gpu": 8,
  "requests_equal_limits": True,
  "total_requested_gpu": 64,
  "available_gpu": 128,
},
"gang": {
  "enabled": True,
  "min_available": 8,
  "worker_count": 8,
  "gpu_per_worker": 8,
  "partial_allocation_allowed": False,
},
"fragmentation": {
  "free_gpus_by_node": [8, 8, 8, 8, 4, 4],
  "gpus_per_job": 8,
  "threshold": 0.35,
  "bin_packing": True,
  "large_job_pool": True,
},
"topology": {
  "components": {"node": 1.0, "nic": 1.0, "rack": 0.8, "storage": 0.8},
  "weights": {"node": 0.4, "nic": 0.25, "rack": 0.2, "storage": 0.15},
  "threshold": 0.8,
  "gpu_nic_affinity_known": True,
  "nvlink_domain_known": True,
},
"placement": {
  "labels": {"gpu-type": "A100-80G", "network": "ib", "pool": "train"},
  "required_labels": {"gpu-type": "A100-80G", "network": "ib"},
  "node_affinity": True,
  "label_governance": True,
},
"taints": {
  "dedicated_taints": {"gpu-pool"},
  "tolerations": {"gpu-pool"},
  "toleration_scope": True,
},
"sharing": {
  "workload": "pretraining",
  "mode": "exclusive",
  "mig_policy": "none",
  "time_slicing": False,
  "mps": False,
},
"quota": {
  "namespace": "team-a",
  "resource_quota": True,
  "per_gpu_type_quota": True,
  "used_gpu": 48,
  "quota_gpu": 64,
  "cost_attribution": True,
},

```

```

"training_runtime": {
    "training_crd": True,
    "rank_env": True,
    "service_discovery": True,
    "checkpoint_uri": True,
},
"inference_runtime": {
    "serving_controller": True,
    "autoscaling": True,
    "slo_metrics": True,
    "health_check": True,
    "cold_start_budget": True,
},
"monitoring": {
    "dcmg_exporter": True,
    "prometheus": True,
    "pod_gpu_mapping": True,
    "ecc_error": True,
    "allocation_metric": True,
},
"isolation": {
    "namespace": True,
    "rbac": True,
    "network_policy": True,
    "secret_scope": True,
    "registry_scope": True,
},
"troubleshooting": {
    "playbook": True,
    "pending_reasons": {
        "gpu_insufficient",
        "quota_exceeded",
        "node_affinity",
        "taint_blocked",
        "image_pull",
        "gang_waiting",
    },
},
"gate": {"enabled": True, "thresholds_ok": True},
}

```

```

def bad_case(name, mutator):
    case = copy.deepcopy(complete)
    case["name"] = name
    mutator(case)
    return case

```

```

bad_cases = [
    bad_case("device_plugin_missing_bad", lambda c: c["device_plugin"].update({"ready": False})),
    bad_case("gpu_resource_not_declared_bad", lambda c: c["resource"].update({"extended_resource_declared": False})),
    bad_case("request_limit_mismatch_bad", lambda c: c["resource"].update({"requests_equal_limits": False})),
    bad_case("no_gang_scheduling_bad", lambda c: c["gang"].update({"enabled": False})),
    bad_case("fragmented_cluster_bad", lambda c: c["fragmentation"].update({"free_gpus_by_node": [4, 4, 4, 4, 4]})),
    bad_case("topology_ignored_bad", lambda c: c["topology"].update({"gpu_nic_affinity_known": False})),
    bad_case("label_affinity_missing_bad", lambda c: c["placement"].update({"node_affinity": False})),

```

```

        bad_case("toleration_missing_bad", lambda c: c["taints"].update({"tolerations": set()})),
        bad_case("mig_for_pretraining_bad", lambda c: c["sharing"].update({"mode": "time_slicing", "time_slicing": True})),
        bad_case("quota_missing_bad", lambda c: c["quota"].update({"resource_quota": False})),
        bad_case("training_operator_missing_bad", lambda c: c["training_runtime"].update({"training_crd": False})),
        bad_case("inference_slo_missing_bad", lambda c: c["inference_runtime"].update({"slo_metrics": False})),
        bad_case("gpu_monitoring_unmapped_bad", lambda c: c["monitoring"].update({"pod_gpu_mapping": False})),
        bad_case("tenant_isolation_missing_bad", lambda c: c["isolation"].update({"rbac": False})),
        bad_case("pending_reason_unknown_bad", lambda c: c["troubleshooting"].update({"pending_reasons": {"gpu_insufficient": True}})),
        bad_case("kubernetes_gpu_gate_missing_bad", lambda c: c["gate"].update({"enabled": False})),
    ]
    return [complete] + bad_cases

def check_device_plugin(case):
    plugin = case["device_plugin"]
    return (
        plugin["ready"]
        and plugin["resource_name"] == "nvidia.com/gpu"
        and plugin["node_capacity_reported"]
        and plugin["allocatable_reported"]
    )

def check_gpu_resource(case):
    resource = case["resource"]
    return (
        resource["extended_resource_declared"]
        and resource["limits_gpu"] > 0
        and resource["total_requested_gpu"] <= resource["available_gpu"]
    )

def check_request_limit(case):
    resource = case["resource"]
    return resource["requests_equal_limits"] and resource["requests_gpu"] == resource["limits_gpu"]

def check_gang(case):
    gang = case["gang"]
    return (
        gang["enabled"]
        and gang["min_available"] == gang["worker_count"]
        and not gang["partial_allocation_allowed"]
    )

def check_fragmentation(case):
    frag = case["fragmentation"]
    return (
        fragmentation_ratio(frag["free_gpus_by_node"], frag["gpus_per_job"]) <= frag["threshold"]
        and frag["bin_packing"]
        and frag["large_job_pool"]
    )

```



```

def check_topology(case):
    topo = case["topology"]
    return (
        topology_score(topo["components"], topo["weights"]) >= topo["threshold"]
        and topo["gpu_nic_affinity_known"]
        and topo["nvlink_domain_known"]
    )

def check_label_affinity(case):
    placement = case["placement"]
    labels = placement["labels"]
    required = placement["required_labels"]
    labels_match = all(labels.get(key) == value for key, value in required.items())
    return placement["node_affinity"] and placement["label_governance"] and labels_match

def check_taints(case):
    taints = case["taints"]
    return taints["dedicated_taints"].issubset(taints["tolerations"]) and taints["toleration_scope"]

def check_sharing(case):
    sharing = case["sharing"]
    if sharing["workload"] == "pretraining":
        return (
            sharing["mode"] == "exclusive"
            and sharing["mig_policy"] == "none"
            and not sharing["time_slicing"]
            and not sharing["mps"]
        )
    return sharing["mode"] in {"exclusive", "mig", "time_slicing", "mps"}

def check_quota(case):
    quota = case["quota"]
    return (
        quota["resource_quota"]
        and quota["per_gpu_type_quota"]
        and tenant_quota_ratio(quota["used_gpu"], quota["quota_gpu"]) <= 1
        and quota["cost_attribution"]
    )

def check_training_runtime(case):
    return all(case["training_runtime"].values())

def check_inference_runtime(case):
    return all(case["inference_runtime"].values())

def check_monitoring(case):
    return all(case["monitoring"].values())

```

```

def check_isolation(case):
    return all(case["isolation"].values())

def check_troubleshooting(case):
    required = {
        "gpu_insufficient",
        "quota_exceeded",
        "node_affinity",
        "taint_blocked",
        "image_pull",
        "gang_waiting",
    }
    trouble = case["troubleshooting"]
    return trouble["playbook"] and required.issubset(trouble["pending_reasons"])

def check_gate(case):
    return case["gate"]["enabled"] and case["gate"]["thresholds_ok"]

CHECKS = {
    "device_plugin_readiness": check_device_plugin,
    "gpu_extended_resource_fit": check_gpu_resource,
    "gpu_request_limit_integrity": check_request_limit,
    "gang_scheduling_readiness": check_gang,
    "fragmentation_control": check_fragmentation,
    "topology_aware_placement": check_topology,
    "node_label_affinity_fit": check_label_affinity,
    "taint_tolerant_fit": check_taints,
    "mig_sharing_policy_fit": check_sharing,
    "quota_namespace_governance": check_quota,
    "training_operator_readiness": check_training_runtime,
    "inference_service_readiness": check_inference_runtime,
    "gpu_monitoring_mapping": check_monitoring,
    "multi_tenant_isolation": check_isolation,
    "pending_troubleshooting_coverage": check_troubleshooting,
    "kubernetes_gpu_gate": check_gate,
}

def audit_kubernetes_gpu_management(cases):
    case_failures = {}
    for case in cases:
        failures = [name for name, check in CHECKS.items() if not check(case)]
        case_failures[case["name"]] = failures

    metrics = {}
    for name, check in CHECKS.items():
        metrics[name] = round(sum(int(check(case)) for case in cases) / len(cases), 3)

    failed_cases = [name for name, failures in case_failures.items() if failures]
    return {
        "metrics": metrics,

```

```

        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "kubernetes_gpu_gate_pass": not failed_cases and min(metrics.values()) >= 0.95,
    }

cases = build_kubernetes_gpu_cases()
case_by_name = {case["name"]: case for case in cases}
complete = case_by_name["complete"]
examples = {
    "gpu_allocation_ratio": round(gpu_allocation_ratio(56, 80), 3),
    "fragmentation_ratio": round(fragmentation_ratio([8, 4, 4, 2, 2], 8), 3),
    "gang_feasible": gang_feasible([8, 8, 8, 4], 8, 3),
    "tenant_quota_ratio": round(tenant_quota_ratio(48, 64), 3),
    "topology_score": topology_score(
        {"node": 1.0, "nic": 1.0, "rack": 0.5, "storage": 1.0},
        {"node": 0.4, "nic": 0.25, "rack": 0.2, "storage": 0.15},
    ),
}

smoke = {
    "complete_case_passes": all(check(complete) for check in CHECKS.values()),
    "caught_device_plugin_gap": not check_device_plugin(case_by_name["device_plugin_missing_bad"]),
    "caught_fragmentation": not check_fragmentation(case_by_name["fragmented_cluster_bad"]),
    "caught_topology_gap": not check_topology(case_by_name["topology_ignored_bad"]),
    "caught_quota_gap": not check_quota(case_by_name["quota_missing_bad"]),
    "caught_pending_gap": not check_troubleshooting(case_by_name["pending_reason_unknown_bad"]),
}

audit = audit_kubernetes_gpu_management(cases)
print(f"kubernetes_gpu_examples={examples}")
print(f"smoke={smoke}")
print(f"metrics={audit['metrics']}")
print(f"hard_blocker_count={audit['hard_blocker_count']}")
print(f"failed_cases={audit['failed_cases']}")
print(f"kubernetes_gpu_gate_pass={audit['kubernetes_gpu_gate_pass']}")

```

一组典型输出是：

```

kubernetes_gpu_examples={'gpu_allocation_ratio': 0.7, 'fragmentation_ratio': 0.6, 'gang_feasible': True, 'tenant_quota_ratio': 0.75, 'topology_score': 0.55}
smoke={'complete_case_passes': True, 'caught_device_plugin_gap': True, 'caught_fragmentation': True, 'caught_topology_gap': True, 'caught_quota_gap': True, 'caught_pending_gap': True}
metrics={'device_plugin_readiness': 0.941, 'gpu_extended_resource_fit': 0.941, 'gpu_request_limit_integrity': 0.941, 'gang_feasibility': 0.941, 'node_label_affinity_taint_toleration': 0.941, 'namespace_quota_cost_attribution': 0.941, 'request_limit_mismatch': 0.941, 'tenant_quota_ratio': 0.75, 'topology_score': 0.55}
hard_blocker_count=16
failed_cases=['device_plugin_missing_bad', 'gpu_resource_not_declared_bad', 'request_limit_mismatch_bad', 'no_gang_scheduler_bad']
kubernetes_gpu_gate_pass=False

```

这个 demo 的重点是把 Kubernetes GPU 管理拆成可验证证据链：device plugin 要就绪，GPU extended resource 要正确声明，request / limit 不能含糊，分布式训练要有 gang 语义，碎片化和拓扑要进入调度决策，node label / affinity / taint / toleration 要可解释，MIG / time slicing 要匹配工作负载，namespace / quota / 成本归因要约束租户，训练和推理运行时要有各自的 CRD / SLO / health 证据，GPU 监控必须能映射到 Pod，最后还要有 Pending 原因排障和统一门禁。

12.20 面试中如何回答 K8s 与 GPU 管理

如果面试官问：

如何用 Kubernetes 管理大模型训练和推理的 GPU 资源？

可以这样回答：

Kubernetes 可以作为容器编排和资源管理底座。GPU 通过 device plugin 注册为 extended resource，Pod 通过 nvidia.com/gpu 申请

但默认 K8s 调度器不足以完整支持大模型训练。分布式训练需要 gang scheduling，避免部分 worker 占住资源但任务无法启动；还需

训练任务可以通过 PyTorchJob、MPIJob、RayJob 或自研 TrainingJob CRD 表达；推理服务可以用 Deployment、StatefulSet 或 Infe

12.21 常见误区

误区一：用了 Kubernetes 就解决了 GPU 调度。

K8s 只是底座，大模型还需要 gang scheduling、拓扑感知、队列、配额、抢占和多租户治理。

误区二：GPU 和 CPU 一样可以随便切分。

GPU 通常按整卡分配，MIG、时间片和共享需要额外机制，且适用场景有限。

误区三：Pod Pending 就一定没有 GPU。

也可能是配额、node selector、taint、优先级、拓扑约束或 gang scheduling 条件不满足。

误区四：Namespace 就等于多租户隔离。

Namespace 是逻辑隔离，还需要资源配额、权限、网络策略、数据隔离、镜像权限和成本归因。

误区五：训练和推理可以使用同一套调度策略。

训练关注吞吐、gang scheduling 和 checkpoint，推理关注 SLO、扩缩容和稳定性。

12.22 面试题

题 1：Kubernetes 如何识别 GPU？

答：K8s 通过 device plugin 机制识别 GPU。NVIDIA device plugin 会发现节点上的 GPU，并向 kubelet 注册 nvidia.com/gpu 这类 extended resource。Pod 通过资源声明申请 GPU，kubelet 启动容器时把对应 GPU 设备暴露给容器。

题 2：为什么分布式训练需要 gang scheduling？

答：分布式训练需要多个 worker 同时参与。如果只启动部分 worker，任务无法正常开始，但资源可能被占住。Gang scheduling 保证一组 Pod 要么全部调度成功，要么都不启动，避免资源浪费和任务卡死。

题 3：什么是 GPU 碎片化？

答：GPU 碎片化是指很多节点上都有少量空闲 GPU，但无法满足需要完整节点或大量连续 GPU 的任务。例如每台 8 卡机器都剩 2 张卡，但一个 8 卡任务无法启动。解决方法包括 bin packing、资源池划分、小任务集中放置、抢占和重调度。

题 4：K8s 默认调度器为什么不够？

答：默认调度器对普通容器服务足够，但不天然理解大模型训练的整体性、GPU 拓扑、gang scheduling、通信密集任务、队列公平性、抢占恢复和多租户成本治理。这些需要调度扩展或上层平台补齐。

题 5：MIG 适合什么场景？

答：MIG 适合把支持的 GPU 切成多个硬件隔离实例，常用于小模型推理、评估、交互式开发和隔离小负载。不适合需要完整显存和高带宽通信的大模型训练或通信密集多卡任务。

12.23 小练习

练习一：写一个 GPU Pod 资源申请示例。

要求：声明需要 2 张 GPU，并说明 device plugin 的作用。

练习二：设计一个 TrainingJob CRD。

要求：包含 gpu_type、gpu_count、gpu_per_node、priority、queue、image、command、checkpoint_uri 和 data_uri。

练习三：分析 Pod Pending。

假设一个 64 卡训练任务一直 Pending，请从 GPU 数量、完整节点、quota、taint、node selector、gang scheduling、拓扑和优先级角度排查。

练习四：设计一个 GPU 碎片化治理策略。

要求：包含 bin packing、小任务资源池、大任务预留、抢占和监控指标。

12.24 本章小结

本章讲了 Kubernetes 与 GPU 资源管理。

你需要掌握：

1. Kubernetes 是 AI Infra 常见底座，但不是完整 AI 调度系统。
2. GPU 通过 device plugin 注册为 extended resource。
3. GPU Operator 可以管理驱动、device plugin、监控等节点组件。
4. 默认 K8s 调度器对大模型训练有局限。
5. 分布式训练需要 gang scheduling。
6. GPU 碎片化会影响大任务启动和集群利用率。
7. 拓扑感知调度需要考虑 NVLink/NVSwitch、NUMA、网卡、机柜和数据位置。
8. MIG、时间片和 GPU 共享适合部分小负载，不适合所有任务。
9. Namespace 和 ResourceQuota 是多租户基础，但还需要更完整的权限、配额、队列和成本治理。
10. 平台层应该为用户屏蔽复杂 YAML，提供面向任务的资源申请接口。

下一章我们会讲训练任务调度：队列、优先级、抢占、公平性和配额。

第 13 章 训练任务调度：队列、优先级、抢占、公平性和配额

上一章讲了 Kubernetes 与 GPU 资源管理。Kubernetes 能把 Pod 调度到节点上，但大模型训练平台不能只依赖默认调度。真实场景里，GPU 贵、任务多、团队多、优先级不同、训练时长不确定、大任务需要成组资源、小任务容易造成碎片。

训练任务调度就是解决这些问题的核心系统。

先记住一句话：

大模型训练调度的目标不是简单“谁先来谁先跑”，而是在稀缺 GPU 资源上平衡吞吐、等待时间、公平性、优先级、成本和任务成功率。

13.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，资料口径按“大模型训练调度治理的稳定抽象”处理，而不是绑定某个 Kubernetes 发行版、云厂商、内部平台或单一调度器实现。Kubernetes 侧参考 PriorityClass、Pod priority / preemption、ResourceQuota 和 scheduler framework 的稳定口径；Kueue 侧参考 LocalQueue、ClusterQueue、cohort 借用、preemption、admission fair sharing 和 topology-aware scheduling 的官方设计；Volcano 侧参考 queue、gang scheduling、priority、preemption、reclaimable、guarantee / deserved / weight 等调度治理概念；公平性部分参考 dominant resource fairness 的多资源公平思想。

需要注意三点：

1. 本章讲队列、优先级、配额、公平性、抢占、gang scheduling、backfilling、碎片治理、拓扑感知、checkpoint 和调度指标。第 12 章已经讲过 Kubernetes 与 GPU 资源表达，本章重点是训练任务调度策略和治理闭环。
2. Kueue、Volcano、YuniKorn、自研调度层和云厂商训练平台的 API 不完全相同。本章只抽象稳定工程问题：如何 admission、如何排队、如何公平、如何借用、如何抢占、如何恢复、如何观测。
3. 这里的 demo 是教学审计脚本，不调用真实 scheduler API。真实平台需要从 queue CRD、scheduler events、admission records、checkpoint metadata、cost ledger、DCGM / Prometheus 指标和训练平台数据库采集证据。

13.1 为什么训练调度很难

普通后端服务调度通常关注 CPU、内存和副本数。

大模型训练调度更难，原因包括：

1. GPU 昂贵且稀缺。
2. 任务资源需求差异大。
3. 大任务需要多节点同时可用。
4. 任务运行时间长。
5. 训练失败恢复依赖 checkpoint。
6. 不同团队有配额和优先级。
7. GPU 型号和拓扑影响性能。
8. 小任务会造成 GPU 碎片。
9. 低优先级任务希望利用空闲资源。
10. 线上推理不能被训练任务干扰。

因此训练调度不是 K8s 默认调度的简单应用，而是 AI Infra 的核心能力。

13.2 调度系统要解决什么问题

训练调度系统至少要回答：

1. 任务能不能运行？
2. 任务什么时候运行？
3. 任务应该用哪些 GPU？
4. 谁的任务优先？
5. 多个团队如何公平共享资源？
6. 高优先级任务来了怎么办？
7. 低优先级任务能不能被抢占？
8. 抢占后如何恢复？
9. 如何减少碎片？
10. 如何提高整体 GPU 利用率？

这些问题对应队列、优先级、配额、抢占、公平性和拓扑感知。

13.3 训练任务的资源请求

一个训练任务不应该只声明“我要 8 张 GPU”。

更完整的资源请求包括：

1. GPU 类型。
2. GPU 数量。
3. 每节点 GPU 数量。
4. 显存需求。
5. 网络需求。
6. 存储需求。
7. 预计运行时间。
8. 优先级。

9. 队列。
10. 是否可抢占。
11. checkpoint 路径。
12. 租户和项目。

示例：

```
job:
  name: pretrain-7b-v3
  queue: pretrain
  priority: high
  resources:
    gpu_type: A100-80G
    gpu_count: 128
    gpu_per_node: 8
    network: ib
  preemptible: false
  checkpoint_uri: s3://ckpts/pretrain-7b-v3/
```

资源请求越结构化，调度系统越能做正确决策。

13.4 队列 Queue

队列是训练调度的基本组织方式。

常见队列：

1. pretrain queue。
2. sft queue。
3. eval queue。
4. research queue。
5. production queue。
6. low-priority queue。

队列可以有不同策略：

1. 最大 GPU 配额。
2. 最大并发任务数。
3. 优先级规则。
4. 是否允许抢占。
5. 是否允许使用空闲资源。
6. 默认 checkpoint 要求。

例如：

1. production queue 优先级最高，不允许被抢占。
2. research queue 有固定配额。
3. low-priority queue 可以使用空闲 GPU，但可被抢占。

队列让资源治理从“单个任务”提升到“团队和业务类型”。

13.5 优先级 Priority

优先级决定任务竞争资源时谁先运行。

优先级可以来自：

1. 业务重要性。
2. 截止时间。
3. 用户角色。
4. 项目等级。

5. 任务类型。
6. 是否线上故障修复。
7. 是否阻塞发布。

常见优先级：

critical > high > normal > low > best-effort

但优先级不能无限滥用。

否则所有人都申请 high，系统就失去意义。

需要配套：

1. 优先级审批。
2. 每队列高优任务额度。
3. 使用审计。
4. 超额告警。
5. 成本归因。

13.6 配额 Quota

配额用于控制团队或项目可使用的资源。

配额可以按多种维度定义：

1. GPU 数量。
2. GPU 小时。
3. GPU 型号。
4. 队列。
5. 并发任务数。
6. 存储容量。
7. 推理实例数。
8. 月度成本预算。

例如：

team_a:

A100-80G: 128 GPUs

H100: 32 GPUs

monthly_budget: 100000 GPU-hours

配额的目标不是限制创新，而是避免某个团队占满集群，让资源使用可预期、可治理。

13.7 公平性 Fairness

公平性回答：多个团队同时需要 GPU 时，如何分配才合理？

常见策略：

1. FIFO。
2. Weighted fair sharing。
3. Dominant Resource Fairness。
4. 配额内优先。
5. 借用空闲资源。
6. 超额可抢占。

FIFO 最简单，但不一定公平。

如果一个超大任务排在前面，小任务可能都被堵住。

Weighted fair sharing 可以给不同团队设置权重。

例如核心项目权重更高，研究项目权重较低。

DRF 适合多资源场景，例如 GPU、CPU、内存、网络 and 存储同时考虑。

公平性不是平均分，而是按业务重要性、配额、历史使用和资源需求综合平衡。

13.8 抢占 Preemption

抢占是指高优先级任务需要资源时，中断低优先级任务并释放资源。

抢占的价值：

1. 提高高优先级任务响应速度。
2. 让低优先级任务利用空闲资源。
3. 提高集群整体利用率。
4. 支持弹性资源池。

抢占的代价：

1. 被抢占任务损失进度。
2. 需要 checkpoint 支持。
3. 可能增加存储压力。
4. 用户体验变差。
5. 调度逻辑复杂。

好的抢占策略需要：

1. 只抢占可抢占任务。
2. 尽量选择损失最小的任务。
3. 提前通知任务保存 checkpoint。
4. 记录抢占原因。
5. 支持恢复排队。
6. 避免频繁抖动。

没有 checkpoint 的任务不适合随便抢占。

13.9 抢占策略如何设计

选择被抢占任务时，可以考虑：

1. 任务优先级。
2. 是否可抢占。
3. 已运行时间。
4. 距离上次 checkpoint 多久。
5. 释放资源是否足够。
6. 是否影响同一团队公平性。
7. 是否会造成更多碎片。
8. 用户或项目权重。

一个合理策略：

优先抢占低优先级、可抢占、最近有 checkpoint、释放后能满足高优任务、且损失最小的任务。

抢占不应该只看“谁优先级低”，还要看实际释放效果和恢复成本。

13.10 Gang Scheduling

分布式训练需要 gang scheduling。

一个任务可能有多个 worker，每个 worker 一个 Pod。

如果不能同时启动，任务无法正常运行。

Gang scheduling 要求：

1. 资源满足整体任务需求。
2. worker 要么全部调度成功。
3. 要么全部等待。
4. 避免部分 worker 占住资源。

例如 64 卡任务需要 8 个 8 卡 worker。调度系统必须确认 8 个 worker 都能放下，才启动任务。这对大任务非常关键。

13.11 Backfilling

Backfilling 是利用大任务等待期间的空闲资源运行小任务。

例如一个大任务需要 128 卡，但当前只有 96 卡可用。它还要等一段时间。

此时可以运行一些短小任务，只要它们不会影响大任务预计启动时间。

Backfilling 的价值：

1. 提高 GPU 利用率。
2. 减少资源空转。
3. 改善小任务体验。

风险：

1. 小任务超时会阻塞大任务。
2. 预计运行时间不准。
3. 需要可抢占或严格截止。

适合配合低优先级队列和 checkpoint 使用。

13.12 GPU 碎片治理

GPU 碎片会导致明明有空闲 GPU，但大任务启动不了。

治理方法：

1. 小任务集中放置。
2. 大任务优先完整节点。
3. 按任务类型划分资源池。
4. bin packing。
5. 空间预留。
6. 抢占和重排。
7. 限制小任务随意占用整机关键资源。

例如：

1. 1 卡和 2 卡任务集中放到专门资源池。
2. 8 卡训练任务优先使用完整 8 卡节点。
3. 预训练资源池不接受零散小任务。

碎片治理直接影响大任务等待时间。

13.13 拓扑感知调度

训练调度不能只看 GPU 数量。

它还要看：

1. GPU 型号。
2. 显存大小。
3. NVLink / NVSwitch。
4. GPU 到网卡亲和性。

5. 节点所在机柜。
6. 网络拥塞。
7. 数据缓存位置。
8. 故障域。

例如 tensor parallel 任务需要高速 GPU 间通信，最好放同机 NVSwitch。

大规模 data parallel 可以跨机，但最好在高一高带宽 fabric 内。

拓扑感知调度可以提升 MFU 和扩展效率。

13.14 调度与 checkpoint 的关系

调度和 checkpoint 强相关。

抢占需要 checkpoint。

故障恢复需要 checkpoint。低优先级队列需要 checkpoint。弹性训练需要 checkpoint。

调度系统应该知道：

1. 任务是否支持 checkpoint。
2. 最近一次 checkpoint 时间。
3. checkpoint 路径。
4. 恢复预计耗时。
5. 抢占后损失多少进度。

没有这些信息，调度系统无法做理性抢占。

13.15 调度与成本治理

调度系统直接影响成本。

成本浪费来自：

1. GPU 空闲。
2. 大任务排队导致项目延期。
3. 低优先级任务占用昂贵 GPU。
4. 任务失败重跑。
5. 训练效率低。
6. checkpoint 太频繁。
7. 资源申请过大。

调度可以优化：

1. 提高利用率。
2. 使用低优先级队列。
3. 限制超额资源申请。
4. 让短任务利用碎片资源。
5. 按团队归因 GPU 小时。
6. 对低效任务告警。

调度不是只决定“跑不跑”，也决定“贵不贵”。

13.16 调度指标

训练调度系统应该监控：

1. queue_length。
2. pending_time。
3. running_jobs。
4. GPU_allocated。

5. GPU_utilization。
6. GPU_fragmentation。
7. preemption_count。
8. checkpoint_before_preemption_rate。
9. quota_usage。
10. fair_share_ratio。
11. job_success_rate。
12. job_failure_rate。
13. average_wait_time_by_queue。
14. GPU_hours_by_team。

这些指标帮助平台回答：

1. 谁在等资源？
2. 哪些队列拥堵？
3. 哪些团队超额？
4. 是否存在大量碎片？
5. 抢占是否过多？
6. GPU 是否真正被有效使用？

13.17 训练调度治理审计指标与最小 demo

训练调度系统不能只看 GPU utilization。一个好的调度系统要证明：任务画像完整、队列规则清楚、优先级不滥用、配额不过载、公平性可解释、gang 能保证整体启动、backfilling 不阻塞保留任务、抢占有 checkpoint 兜底、碎片和拓扑可控、失败后能重排、成本可以归因。

可以把一个调度审计样本写成：

$s_i = (j_i, q_i, p_i, a_i, f_i, b_i, e_i, c_i, t_i, o_i, z_i)$

其中， j_i 是训练任务资源画像， q_i 是队列配置， p_i 是优先级治理， a_i 是 admission / gang 状态， f_i 是公平性与配额， b_i 是 backfilling 策略， e_i 是抢占与恢复， c_i 是 checkpoint 耦合， t_i 是拓扑和碎片， o_i 是观测指标， z_i 是最终门禁。

统一覆盖率可以写成：

$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(s_i)=1]$

队列等待时间可以按队列统计 P95：

$W_{\{q, \mathrm{p95}\}} = \mathrm{P95}(t_{\{\mathrm{start}\}} - t_{\{\mathrm{sub}\}} \mid q)$

租户配额占用率可以写成：

$Q_u = \frac{G_{\{u, \mathrm{used}\}}}{G_{\{u, \mathrm{quota}\}}}$

其中， G_{u_used} 是租户 u 当前已用 GPU， G_{u_quota} 是租户 GPU 配额。

多资源公平性可以用 dominant share 近似：

$D_u = \max_r \frac{A_{\{u, r\}}}{C_r}$

其中， $A_{u, r}$ 是租户 u 使用的第 r 类资源， C_r 是该资源总容量。DRF 的直觉是让多个租户的 dominant share 尽量接近其权重，而不是只看单一 GPU 数。

抢占损失可以估算为：

$L_j = G_j(t_{\{\mathrm{now}\}} - t_{\{\mathrm{ckpt}\}})$

其中， G_j 是被抢占任务占用的 GPU 数， $t_{\mathrm{now}} - t_{\mathrm{ckpt}}$ 是距离上次 checkpoint 的时间。单位可以是 GPU-minute 或 GPU-hour。

Backfilling 的安全条件可以写成：

$B_j = \mathbf{1}[\left[t_{\{\mathrm{now}\}} + T_j \leq t_{\{\mathrm{reserve}\}}\right]]$

其中, T_j 是小任务预计运行时长, t_{reserve} 是大任务保留资源的预计启动时间。

GPU 碎片化比例仍然可以写成:

$$R_{\{\mathrm{frag}\}} = 1 - \frac{\sum_n \lfloor g_n / g_{\{\mathrm{job}\}} \rfloor}{\sum_n g_n}$$

最后, 可以把训练调度治理门禁写成:

$$G_{\{\mathrm{train_sched}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \ \& \ W_{\{q, \mathrm{p95}\}} \leq \omega_q \ \& \ Q_u \leq 1 \right]$$

其中, τ_j 是每个检查项的覆盖率阈值, ω_q 是队列等待时间 SLO, ρ 是碎片化阈值, P_0 是 P0 级调度风险数量。

下面这个 0 依赖 demo 演示如何把训练调度治理拆成审计规则。它故意构造 1 个完整样本和 16 个坏样本, 让每个关键维度各失败一次。

```
import copy
import math
```

```
METRICS = [
    "workload_resource_shape",
    "queue_policy_coverage",
    "priority_governance",
    "quota_usage_control",
    "fair_share_accounting",
    "gang_scheduling_fit",
    "backfilling_safety",
    "preemption_checkpoint_safety",
    "fragmentation_control",
    "topology_aware_scheduling",
    "checkpoint_scheduler_coupling",
    "cost_attribution_control",
    "scheduler_observability",
    "failure_requeue_readiness",
    "anti_starvation_control",
    "training_scheduler_gate",
]
```

```
def percentile(values, pct):
    ordered = sorted(values)
    rank = math.ceil(len(ordered) * pct / 100) - 1
    rank = max(0, min(rank, len(ordered) - 1))
    return ordered[rank]

def quota_ratio(used_gpu, quota_gpu):
    return used_gpu / quota_gpu

def dominant_share(usage, capacity):
    return max(usage[name] / capacity[name] for name in usage)

def weighted_share_ratio(used_gpu, total_gpu, weight):
    return (used_gpu / total_gpu) / weight
```

```

def preemption_loss_gpu_min(now_min, last_checkpoint_min, gpu_count):
    return (now_min - last_checkpoint_min) * gpu_count

def backfill_safe(now_min, runtime_min, reserved_start_min):
    return now_min + runtime_min <= reserved_start_min

def fragmentation_ratio(free_gpus_by_node, gpus_per_job):
    allocatable = sum((free // gpus_per_job) * gpus_per_job for free in free_gpus_by_node)
    total_free = sum(free_gpus_by_node)
    return 1 - allocatable / total_free

def topology_score(components, weights):
    return round(sum(components[name] * weights[name] for name in weights), 3)

def build_training_scheduler_cases():
    complete = {
        "name": "complete",
        "job": {
            "gpu_type": "A100-80G",
            "gpu_count": 64,
            "gpu_per_node": 8,
            "network": "ib",
            "storage": "shared_fs",
            "queue": "pretrain",
            "priority": "high",
            "preemptible": False,
            "checkpoint_uri": "s3://ckpt/pretrain-7b/",
            "estimated_runtime_min": 720,
        },
        "queue": {
            "quota_gpu": 128,
            "max_running_jobs": 8,
            "priority_classes": {"critical", "high", "normal", "low"},
            "preemption_policy": "lower-priority-with-checkpoint",
            "borrow_limit_gpu": 32,
            "lending_enabled": True,
        },
        "priority": {
            "allowed_classes": {"critical", "high", "normal", "low", "best-effort"},
            "approval_required": {"critical", "high"},
            "approved": True,
            "high_used": 3,
            "high_limit": 5,
            "audit_log": True,
        },
        "quota": {
            "used_gpu": 96,
            "quota_gpu": 128,
            "used_gpu_hours": 18000,
            "budget_gpu_hours": 24000,
            "per_gpu_type_quota": True,

```

```

},
"fairness": {
  "tracked": True,
  "tenant_gpu_used": 40,
  "cluster_gpu_total": 160,
  "tenant_weight": 0.35,
  "weighted_share_limit": 1.0,
  "usage": {"gpu": 40, "cpu": 240, "memory": 1200},
  "capacity": {"gpu": 160, "cpu": 1600, "memory": 6400},
  "dominant_share_limit": 0.35,
},
"gang": {
  "enabled": True,
  "worker_count": 8,
  "min_available": 8,
  "all_or_nothing": True,
  "partial_allocation_allowed": False,
},
"backfill": {
  "enabled": True,
  "now_min": 100,
  "short_job_runtime_min": 20,
  "reserved_start_min": 130,
  "deadline_enforced": True,
  "short_job_preemptible": True,
},
"preemption": {
  "candidate_preemptible": True,
  "candidate_priority": "low",
  "higher_priority_waiting": True,
  "checkpoint_supported": True,
  "now_min": 240,
  "last_checkpoint_min": 225,
  "gpu_count": 8,
  "max_loss_gpu_min": 180,
  "reason_logged": True,
  "restore_requeue": True,
  "min_runtime_protection": True,
},
"fragmentation": {
  "free_gpus_by_node": [8, 8, 8, 8, 4, 4],
  "gpus_per_job": 8,
  "threshold": 0.35,
  "bin_packing": True,
  "large_job_pool": True,
},
"topology": {
  "components": {"node": 1.0, "nic": 1.0, "rack": 0.8, "storage": 0.8},
  "weights": {"node": 0.4, "nic": 0.25, "rack": 0.2, "storage": 0.15},
  "threshold": 0.8,
  "topology_profile": True,
  "scheduler_uses_topology": True,
},
"checkpoint": {
  "scheduler_knows_uri": True,

```

```

        "last_checkpoint_age_min": 15,
        "max_lost_min": 20,
        "restore_estimate_min": 12,
        "resume_slo_min": 30,
    },
    "cost": {
        "gpu_hours_by_team": True,
        "queue_cost_center": True,
        "overrequest_detection": True,
        "idle_gpu_alert": True,
    },
    "observability": {
        "queue_length": True,
        "pending_p95": True,
        "running_jobs": True,
        "gpu_allocated": True,
        "fragmentation": True,
        "preemption_count": True,
        "fair_share": True,
        "success_rate": True,
    },
    "failure": {
        "requeue_on_failure": True,
        "backoff_policy": True,
        "event_reason": True,
        "terminal_status": True,
    },
    "starvation": {
        "max_wait_slo": True,
        "aging_boost": True,
        "priority_abuse_guard": True,
        "queue_sla": True,
    },
    "gate": {"enabled": True, "thresholds_ok": True, "p0_risks": 0},
}

```

```

def bad_case(name, mutator):
    case = copy.deepcopy(complete)
    case["name"] = name
    mutator(case)
    return case

```

```

bad_cases = [
    bad_case("resource_shape_missing_bad", lambda c: c["job"].update({"network": None})),
    bad_case("queue_policy_missing_bad", lambda c: c["queue"].update({"preemption_policy": None})),
    bad_case("priority_unbounded_bad", lambda c: c["priority"].update({"high_used": 8})),
    bad_case("quota_overrun_bad", lambda c: c["quota"].update({"used_gpu": 160})),
    bad_case("fairness_untracked_bad", lambda c: c["fairness"].update({"tracked": False})),
    bad_case("gang_disabled_bad", lambda c: c["gang"].update({"enabled": False})),
    bad_case("unsafe_backfilling_bad", lambda c: c["backfill"].update({"short_job_runtime_min": 45})),
    bad_case("preemption_without_checkpoint_bad", lambda c: c["preemption"].update({"checkpoint_supported": False})),
    bad_case("fragmentation_uncontrolled_bad", lambda c: c["fragmentation"].update({"free_gpus_by_node": [4, 4, 4]})),
    bad_case("topology_ignored_bad", lambda c: c["topology"].update({"scheduler_uses_topology": False})),
    bad_case("checkpoint_uncoupled_bad", lambda c: c["checkpoint"].update({"scheduler_knows_uri": False})),
    bad_case("cost_unattributed_bad", lambda c: c["cost"].update({"gpu_hours_by_team": False})),

```



```

        bad_case("observability_missing_bad", lambda c: c["observability"].update({"pending_p95": False})),
        bad_case("failure_requeue_missing_bad", lambda c: c["failure"].update({"requeue_on_failure": False})),
        bad_case("starvation_no_guard_bad", lambda c: c["starvation"].update({"aging_boost": False})),
        bad_case("scheduler_gate_missing_bad", lambda c: c["gate"].update({"enabled": False})),
    ]
    return [complete] + bad_cases

def check_workload_shape(case):
    job = case["job"]
    required = ["gpu_type", "gpu_count", "gpu_per_node", "network", "queue", "priority", "checkpoint_uri"]
    return all(job.get(key) for key in required) and job["gpu_count"] % job["gpu_per_node"] == 0

def check_queue_policy(case):
    queue = case["queue"]
    return (
        queue["quota_gpu"] > 0
        and queue["max_running_jobs"] > 0
        and bool(queue["priority_classes"])
        and bool(queue["preemption_policy"])
        and queue["borrow_limit_gpu"] >= 0
        and queue["lending_enabled"]
    )

def check_priority(case):
    priority = case["priority"]
    job_priority = case["job"]["priority"]
    return (
        job_priority in priority["allowed_classes"]
        and priority["approved"]
        and priority["high_used"] <= priority["high_limit"]
        and priority["audit_log"]
    )

def check_quota(case):
    quota = case["quota"]
    return (
        quota_ratio(quota["used_gpu"], quota["quota_gpu"]) <= 1
        and quota["used_gpu_hours"] <= quota["budget_gpu_hours"]
        and quota["per_gpu_type_quota"]
    )

def check_fairness(case):
    fair = case["fairness"]
    return (
        fair["tracked"]
        and weighted_share_ratio(fair["tenant_gpu_used"], fair["cluster_gpu_total"], fair["tenant_weight"])
        <= fair["weighted_share_limit"]
        and dominant_share(fair["usage"], fair["capacity"]) <= fair["dominant_share_limit"]
    )

```

```

def check_gang(case):
    gang = case["gang"]
    return (
        gang["enabled"]
        and gang["min_available"] == gang["worker_count"]
        and gang["all_or_nothing"]
        and not gang["partial_allocation_allowed"]
    )

def check_backfill(case):
    backfill = case["backfill"]
    return (
        backfill["enabled"]
        and backfill_safe(backfill["now_min"], backfill["short_job_runtime_min"], backfill["reserved_start_min"])
        and backfill["deadline_enforced"]
        and backfill["short_job_preemptible"]
    )

def check_preemption(case):
    preemption = case["preemption"]
    loss = preemption_loss_gpu_min(
        preemption["now_min"],
        preemption["last_checkpoint_min"],
        preemption["gpu_count"],
    )
    return (
        preemption["candidate_preemptible"]
        and preemption["candidate_priority"] == "low"
        and preemption["higher_priority_waiting"]
        and preemption["checkpoint_supported"]
        and loss <= preemption["max_loss_gpu_min"]
        and preemption["reason_logged"]
        and preemption["restore_requeue"]
        and preemption["min_runtime_protection"]
    )

def check_fragmentation(case):
    frag = case["fragmentation"]
    return (
        fragmentation_ratio(frag["free_gpus_by_node"], frag["gpus_per_job"]) <= frag["threshold"]
        and frag["bin_packing"]
        and frag["large_job_pool"]
    )

def check_topology(case):
    topo = case["topology"]
    return (
        topology_score(topo["components"], topo["weights"]) >= topo["threshold"]
        and topo["topology_profile"]
        and topo["scheduler_uses_topology"]
    )

```

```

)

def check_checkpoint(case):
    checkpoint = case["checkpoint"]
    return (
        checkpoint["scheduler_knows_uri"]
        and checkpoint["last_checkpoint_age_min"] <= checkpoint["max_lost_min"]
        and checkpoint["restore_estimate_min"] <= checkpoint["resume_slo_min"]
    )

def check_cost(case):
    return all(case["cost"].values())

def check_observability(case):
    return all(case["observability"].values())

def check_failure_requeue(case):
    return all(case["failure"].values())

def check_starvation(case):
    return all(case["starvation"].values())

def check_gate(case):
    gate = case["gate"]
    return gate["enabled"] and gate["thresholds_ok"] and gate["p0_risks"] == 0

CHECKS = {
    "workload_resource_shape": check_workload_shape,
    "queue_policy_coverage": check_queue_policy,
    "priority_governance": check_priority,
    "quota_usage_control": check_quota,
    "fair_share_accounting": check_fairness,
    "gang_scheduling_fit": check_gang,
    "backfilling_safety": check_backfill,
    "preemption_checkpoint_safety": check_preemption,
    "fragmentation_control": check_fragmentation,
    "topology_aware_scheduling": check_topology,
    "checkpoint_scheduler_coupling": check_checkpoint,
    "cost_attribution_control": check_cost,
    "scheduler_observability": check_observability,
    "failure_requeue_readiness": check_failure_requeue,
    "anti_starvation_control": check_starvation,
    "training_scheduler_gate": check_gate,
}

def audit_training_scheduler(cases):
    case_failures = {}

```

```

for case in cases:
    failures = [name for name, check in CHECKS.items() if not check(case)]
    case_failures[case["name"]] = failures

metrics = {}
for name, check in CHECKS.items():
    metrics[name] = round(sum(int(check(case)) for case in cases) / len(cases), 3)

failed_cases = [name for name, failures in case_failures.items() if failures]
failed_gates = [name for name, value in metrics.items() if value < 1.0]
return {
    "metrics": metrics,
    "hard_blocker_count": len(failed_cases),
    "failed_cases": failed_cases,
    "failed_gates": failed_gates,
    "training_scheduler_gate_pass": not failed_cases and min(metrics.values()) >= 0.95,
}

cases = build_training_scheduler_cases()
case_by_name = {case["name"]: case for case in cases}
complete = case_by_name["complete"]
scheduler_examples = {
    "queue_pending_p95_min": percentile([12, 18, 25, 31, 44], 95),
    "quota_ratio": round(quota_ratio(96, 128), 3),
    "dominant_share": round(dominant_share({"gpu": 48, "cpu": 220}, {"gpu": 100, "cpu": 400}), 3),
    "weighted_share_ratio": round(weighted_share_ratio(40, 160, 0.35), 3),
    "preemption_loss_gpu_min": preemption_loss_gpu_min(240, 225, 8),
    "backfill_safe": backfill_safe(100, 20, 130),
    "fragmentation_ratio": round(fragmentation_ratio([8, 4, 4, 2, 2], 8), 3),
    "topology_score": topology_score(
        {"node": 1.0, "nic": 1.0, "rack": 0.5, "storage": 1.0},
        {"node": 0.4, "nic": 0.25, "rack": 0.2, "storage": 0.15},
    ),
}

smoke = {
    "complete_case_passes": all(check(complete) for check in CHECKS.values()),
    "caught_priority_abuse": not check_priority(case_by_name["priority_unbounded_bad"]),
    "caught_quota_overrun": not check_quota(case_by_name["quota_overrun_bad"]),
    "caught_unsafe_backfill": not check_backfill(case_by_name["unsafe_backfilling_bad"]),
    "caught_checkpointless_preemption": not check_preemption(case_by_name["preemption_without_checkpoint_bad"]),
    "caught_starvation_gap": not check_starvation(case_by_name["starvation_no_guard_bad"]),
}

audit = audit_training_scheduler(cases)
print(f"scheduler_examples={scheduler_examples}")
print(f"smoke={smoke}")
print(f"metrics={audit['metrics']}")
print(f"hard_blocker_count={audit['hard_blocker_count']}")
print(f"failed_cases={audit['failed_cases']}")
print(f"failed_gates={audit['failed_gates']}")
print(f"training_scheduler_gate_pass={audit['training_scheduler_gate_pass']}")

```

一组典型输出是：

```
scheduler_examples={'queue_pending_p95_min': 44, 'quota_ratio': 0.75, 'dominant_share': 0.55, 'weighted_share_ratio': 0.5},
smoke={'complete_case_passes': True, 'caught_priority_abuse': True, 'caught_quota_ouerrun': True, 'caught_unsafe_backfill': True},
metrics={'workload_resource_shape': 0.941, 'queue_policy_coverage': 0.941, 'priority_governance': 0.941, 'quota_usage_control': 0.941},
hard_blocker_count=16
failed_cases=['resource_shape_missing_bad', 'queue_policy_missing_bad', 'priority_unbounded_bad', 'quota_ouerrun_bad', 'smoke'],
failed_gates=['workload_resource_shape', 'queue_policy_coverage', 'priority_governance', 'quota_usage_control', 'fair_share'],
training_scheduler_gate_pass=False
```

这个 demo 的重点是把训练调度从“谁先跑”拆成可验证证据链：任务资源画像要完整，队列要有配额和借用策略，优先级要有审批和审计，fair share 和 dominant share 要能解释，gang 要保证整体启动，backfilling 不能影响保留任务，抢占必须依赖 checkpoint，碎片和拓扑要被调度器使用，checkpoint、成本、观测、失败重排、防饥饿和最终门禁都要能审计。

13.18 常见调度故障

13.18.1 大任务长期排队

可能原因：

1. 资源不足。
2. GPU 碎片化。
3. 配额不足。
4. 优先级太低。
5. 需要特定 GPU 型号。
6. 拓扑要求太严格。
7. gang scheduling 无法满足。

13.18.2 小任务体验差

可能原因：

1. FIFO 被大任务堵住。
2. 没有 backfilling。
3. 队列策略不合理。
4. 小任务和大任务混用资源池。
5. 交互式资源池不足。

13.18.3 抢占过多

可能原因：

1. 高优任务过多。
2. 低优先级资源池策略不合理。
3. 没有最小运行时间保护。
4. checkpoint 频率不匹配。
5. 资源预留不足。

13.18.4 GPU 利用率高但产出低

可能原因：

1. 低效任务占用资源。
2. 训练任务 MFU 低。
3. 大量失败重试。
4. 数据加载慢。
5. 通信拓扑差。
6. 用户申请资源过量。

13.19 面试中如何回答训练调度系统设计

如果面试官问：

如何设计一个大模型训练任务调度系统？

可以这样回答：

我会在 Kubernetes 之上设计面向 AI 任务的调度层。用户提交 TrainingJob，声明 GPU 类型、数量、每节点 GPU 数、网络需求、优先级。调度层按队列管理任务，每个队列有配额、优先级和公平分享策略。分布式训练使用 gang scheduling，保证 worker 要么全部启动，资源选择时做拓扑感知，考虑 GPU 型号、NVLink/NVSwitch、GPU 到网卡亲和性、机柜网络、故障域和数据位置。为了提高利用率，抢占方面，只抢占低优先级、可抢占、最近有 checkpoint 且释放资源有效的任务，并记录抢占原因和恢复状态。治理方面，提供团队配额、公平性、成本归因、GPU 碎片监控、队列等待时间、GPU 小时统计和低效任务告警。

13.20 常见误区

误区一：训练调度就是 FIFO。

FIFO 简单但容易被大任务堵住，也无法表达优先级和公平性。

误区二：GPU 利用率高就说明调度好。

还要看任务产出、等待时间、公平性、MFU、失败率和成本。

误区三：抢占越多利用率越高。

抢占会带来进度损失和用户体验问题，需要 checkpoint 和策略控制。

误区四：配额就是固定不能超。

成熟系统可以允许借用空闲资源，但超额资源要可抢占。

误区五：调度不用理解网络拓扑。

大模型训练通信密集，拓扑会直接影响训练效率。

13.21 面试题

题 1：为什么大模型训练需要 gang scheduling？

答：分布式训练需要所有 worker 同时参与。如果只启动部分 worker，任务无法开始，但资源会被占住。Gang scheduling 保证任务所需的一组 Pod 要么全部调度成功，要么全部等待，避免资源浪费和任务卡死。

题 2：什么是 GPU 碎片化？如何缓解？

答：GPU 碎片化是指集群中有空闲 GPU，但分散在不同节点或拓扑位置，无法满足大任务需要完整节点或连续资源的需求。缓解方法包括 bin packing、小任务集中放置、大任务资源池、抢占、backfilling 和拓扑感知调度。

题 3：抢占策略应该考虑哪些因素？

答：应考虑任务优先级、是否可抢占、最近 checkpoint 时间、已运行时间、释放资源是否满足高优任务、恢复成本、团队公平性和碎片影响。不能只看优先级低就抢占。

题 4：配额和公平性有什么区别？

答：配额是资源使用上限或保障，例如某团队最多使用多少 GPU；公平性是多个团队同时竞争资源时如何分配，例如按权重、历史使用、dominant resource 或队列策略分配。配额是规则，公平性是调度目标。

题 5：为什么调度系统要知道 checkpoint 信息？

答：因为抢占、故障恢复和弹性调度都依赖 checkpoint。调度系统需要知道任务是否支持 checkpoint、最近 checkpoint 时间、恢复路径和恢复成本，才能选择损失最小的抢占对象。

13.22 小练习

练习一：设计一个队列策略。

要求：包含 pretrain、sft、eval、low-priority 四个队列，分别定义优先级、配额和抢占规则。

练习二：设计一个抢占算法。

要求：输入高优任务资源需求和当前运行任务列表，输出应该抢占哪些任务，并说明评分规则。

练习三：分析一个大任务排队问题。

假设一个 128 卡任务排队 2 天仍无法启动，请从资源总量、碎片、配额、拓扑、优先级和 gang scheduling 角度排查。

练习四：设计一个调度 dashboard。

要求：包含队列长度、等待时间、GPU 分配率、碎片率、抢占次数、配额使用率和团队 GPU 小时。

13.23 本章小结

本章讲了训练任务调度。

你需要掌握：

1. 大模型训练调度要平衡吞吐、等待时间、公平性、优先级、成本和成功率。
2. 训练任务资源请求应包含 GPU 类型、数量、每节点 GPU、网络、优先级、队列和 checkpoint。
3. 队列用于组织不同业务和任务类型。
4. 优先级需要配套审批和审计，避免滥用。
5. 配额和公平性用于多团队资源治理。
6. 抢占能提高高优任务响应和资源利用率，但依赖 checkpoint。
7. 分布式训练需要 gang scheduling。
8. Backfilling 可以利用大任务等待期间的空闲资源。
9. GPU 碎片治理对大任务启动非常重要。
10. 调度系统要理解拓扑、checkpoint、成本和任务画像。

下一章我们会讲多租户隔离：资源隔离、权限隔离和数据隔离。

第 14 章 多租户隔离：资源隔离、权限隔离和数据隔离

前面我们讲了 Kubernetes、GPU 管理和训练任务调度。本章讲企业 AI Infra 里非常现实的问题：多个团队、多个项目、多个用户共用同一套 GPU 集群时，如何隔离？

如果没有多租户隔离，AI 平台很快会出现资源被抢占、数据被误读、权限失控、成本无法归因、日志混乱、镜像不可信、线上推理被实验任务干扰等问题。

先记住一句话：

多租户隔离不是只建几个 namespace，而是资源、身份、权限、数据、网络、镜像、密钥、审计和成本的系统性治理。

14.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，资料口径按“AI Infra 多租户隔离的稳定治理抽象”处理，而不是绑定某个云厂商账号体系、某个 Kubernetes 发行版、某个 service mesh 或某个内部平台实现。Kubernetes 侧重点参考官方 multi-tenancy、Namespace、ResourceQuota、LimitRange、RBAC、ServiceAccount、NetworkPolicy、

Secret、Pod Security Standards 和静态数据加密等文档口径；容器和供应链侧沿用前文对镜像 digest、扫描、签名、registry 权限和运行时加固的边界；数据治理侧只抽象分类、授权、使用目的、血缘、产物继承和审计，不写成某个厂商的 IAM / DLP / KMS 私有配置。

需要注意三点：

1. Namespace 是多租户治理入口，不是完整隔离边界。只建 namespace 但不配 resource quota、RBAC、network policy、secret scope、data gateway、日志脱敏和审计，仍然不是可靠多租户。
2. AI Infra 的多租户隔离比普通 Web 服务更复杂，因为训练数据可能流入 checkpoint、模型权重、eval report、日志、trace、embedding、RAG 文档索引和下游微调产物。
3. 这里的 demo 是教学审计脚本，不调用真实 Kubernetes API。真实平台需要从 IdP、RBAC / ABAC policy engine、Kubernetes admission、scheduler、data access gateway、secret manager、registry、observability、audit log 和 cost ledger 采集证据。

14.1 什么是多租户

租户可以是：

1. 一个业务团队。
2. 一个项目组。
3. 一个部门。
4. 一个外部客户。
5. 一个环境，例如 dev、staging、prod。

多租户就是多个租户共享同一套平台能力。

共享的好处：

1. 提高 GPU 利用率。
2. 降低重复建设成本。
3. 统一平台能力。
4. 统一监控和治理。
5. 统一安全和审计。

共享的风险：

1. 资源互相抢。
2. 数据互相看见。
3. 网络互相访问。
4. 密钥泄露。
5. 镜像污染。
6. 成本无法归因。
7. 故障影响扩大。

多租户隔离的目标，就是在共享资源的同时控制这些风险。

14.2 多租户隔离的层次

AI Infra 多租户隔离至少包括八层：

1. 资源隔离。
2. 权限隔离。
3. 数据隔离。
4. 网络隔离。
5. 运行时隔离。
6. 镜像和供应链隔离。
7. 密钥隔离。
8. 审计和成本隔离。

其中任何一层缺失，都可能出问题。

例如资源隔离做得好，但数据权限没做好，仍然可能泄露训练数据。

又比如数据权限做得好，但密钥以环境变量泄露到日志里，也会出事。

14.3 资源隔离

资源隔离回答：每个租户能使用多少 GPU、CPU、内存、存储和网络？

常见资源：

1. GPU 数量。
2. GPU 型号。
3. GPU 小时。
4. CPU。
5. 内存。
6. 本地盘。
7. 共享存储容量。
8. 网络带宽。
9. 推理 QPS。
10. 队列并发。

资源隔离手段：

1. Namespace。
2. ResourceQuota。
3. Queue quota。
4. PriorityClass。
5. LimitRange。
6. 独立资源池。
7. 调度队列。
8. 成本预算。

资源隔离不是所有资源完全物理隔开，而是明确租户的保障、上限和超额使用规则。

14.4 硬隔离和软隔离

资源隔离可以分成硬隔离和软隔离。

硬隔离：

1. 专属节点。
2. 专属 GPU 资源池。
3. 独立网络。
4. 独立存储 bucket。
5. 独立 Kubernetes 集群。

优点是隔离强，缺点是利用率可能低、成本高。

软隔离：

1. 共享集群。
2. namespace 隔离。
3. 配额限制。
4. 优先级队列。
5. 抢占策略。
6. 逻辑权限控制。

优点是利用率高，缺点是治理复杂。

企业常见做法是混合：生产推理、高敏数据和关键任务硬隔离；普通训练和实验软隔离。

14.5 权限隔离

权限隔离回答：谁可以提交任务、访问数据、使用模型、查看日志、修改配置、发布服务？

常见权限对象：

1. 用户。
2. 用户组。
3. Service Account。
4. 训练任务。
5. 推理服务。
6. Agent。
7. 平台管理员。

常见权限动作：

1. submit_job。
2. cancel_job。
3. read_logs。
4. read_dataset。
5. write_checkpoint。
6. deploy_model。
7. access_secret。
8. change_quota。
9. approve_release。

权限模型通常需要 RBAC 和 ABAC 结合。

RBAC 按角色授权，例如 admin、developer、viewer。

ABAC 按属性授权，例如项目、数据等级、环境、任务类型、资源池。

14.6 数据隔离

数据隔离是 AI Infra 多租户里最重要的一层。

需要隔离的数据包括：

1. 原始训练数据。
2. 清洗数据。
3. 评估数据。
4. RAG 文档。
5. checkpoint。
6. 模型权重。
7. 日志。
8. trace。
9. prompt。
10. 用户请求。

数据隔离要解决：

1. 谁能读？
2. 谁能写？
3. 谁能复制？
4. 谁能导出？
5. 谁能用于训练？
6. 谁能用于评估？
7. 是否允许跨租户共享？

尤其要注意：训练任务一旦能读数据，就可能把数据记进模型参数、checkpoint 或日志里。

所以数据权限不仅是读文件权限，还要考虑数据使用目的和下游产物权限。

14.7 数据分类与使用策略

企业数据通常需要分类。

例如：

1. public。
2. internal。
3. confidential。
4. restricted。
5. regulated。

不同分类对应不同策略：

1. 是否允许训练。
2. 是否允许导出。
3. 是否允许跨区域。
4. 是否需要脱敏。
5. 是否需要审批。
6. 产出的模型是否继承数据等级。

例如使用 restricted 数据训练出的模型，可能也应该被标记为 restricted，不能随便发布到所有租户。

这叫数据等级传播。

14.8 网络隔离

网络隔离回答：不同租户的任务能不能互相访问？能不能访问外网？能不能访问生产数据库？

常见手段：

1. Kubernetes NetworkPolicy。
2. VPC / subnet。
3. Service mesh policy。
4. 防火墙。
5. egress 控制。
6. 私有 endpoint。
7. DNS 限制。

AI 任务网络风险包括：

1. 训练任务访问不该访问的数据服务。
2. 容器向外发送敏感数据。
3. 不同租户 Pod 互相访问。
4. 调试端口暴露。
5. 推理服务被非授权调用。

默认策略应该是最小开放，而不是默认互通。

14.9 运行时隔离

运行时隔离包括容器、Pod、节点和 GPU 层面的隔离。

需要关注：

1. 容器是否 privileged。
2. 是否挂载宿主机路径。
3. 是否允许 root。
4. 是否共享 IPC / PID namespace。
5. 是否共享 GPU。
6. 是否使用 MIG。
7. 是否隔离本地缓存。

8. 是否限制系统调用。

GPU 多租户尤其复杂。

整卡分配隔离更强，但利用率可能低。

MIG 可以提供硬件级切分，适合部分推理和开发负载。

时间片共享利用率高，但性能隔离和安全隔离较弱。

选择哪种方式要看任务风险和稳定性要求。

14.10 镜像和供应链隔离

租户提交的镜像可能不可信。

风险包括：

1. 镜像包含恶意代码。
2. 镜像包含漏洞包。
3. 镜像里写入密钥。
4. 镜像 tag 被覆盖。
5. 镜像来源不明。
6. 基础镜像过旧。

治理措施：

1. 私有镜像仓库。
2. 镜像扫描。
3. 镜像签名。
4. 只允许可信 registry。
5. 基础镜像白名单。
6. 记录 digest。
7. 禁止高风险镜像运行。
8. 构建流水线审计。

多租户平台不能让任何用户随便拉任意镜像在 GPU 节点上运行。

14.11 密钥隔离

AI 任务经常需要访问：

1. 数据库。
2. 对象存储。
3. 模型仓库。
4. 外部 API。
5. 评估服务。
6. 私有代码仓库。

这需要密钥。

密钥隔离要避免：

1. 把密钥写进镜像。
2. 把密钥写进代码。
3. 把密钥打印到日志。
4. 多租户共享同一个高权限密钥。
5. 任务结束后密钥仍然有效。

更好的做法：

1. 使用短期凭证。
2. 使用 Service Account。

3. 按任务注入最小权限密钥。
4. 密钥自动轮换。
5. 访问审计。
6. 日志脱敏。

14.12 日志、Trace 和可见性隔离

日志和 trace 也可能包含敏感信息。

例如：

1. 训练样本片段。
2. 用户 prompt。
3. 模型输出。
4. 数据路径。
5. 密钥片段。
6. 错误堆栈中的参数。
7. Agent 工具调用结果。

因此日志隔离要控制：

1. 谁能看任务日志。
2. 谁能看推理请求日志。
3. 谁能看 Agent trace。
4. 日志保留多久。
5. 是否需要脱敏。
6. 是否允许导出。

不要默认所有平台管理员都能随便看所有用户数据日志。至少要有审计和最小权限。

14.13 成本隔离和归因

多租户平台必须能回答：谁用了多少资源，花了多少钱？

需要归因的资源：

1. GPU 小时。
2. CPU 和内存。
3. 存储容量。
4. 网络流量。
5. 推理 token。
6. checkpoint 存储。
7. 评估任务。
8. Agent 工具调用。

成本归因维度：

1. tenant。
2. team。
3. project。
4. user。
5. job。
6. model。
7. environment。

没有成本隔离，就无法做预算、配额和优化。

14.14 生产环境和实验环境隔离

生产推理和实验训练最好隔离。

原因:

1. 生产有 SLO。
2. 实验任务不稳定。
3. 实验镜像可能不安全。
4. 训练任务会占用网络和存储。
5. 生产数据权限更敏感。
6. 变更流程不同。

隔离方式:

1. 独立 namespace。
2. 独立节点池。
3. 独立资源池。
4. 独立网络策略。
5. 独立权限审批。
6. 独立发布流程。

关键生产服务可以使用硬隔离资源池。

14.15 跨租户共享

完全隔离并不总是最佳。

有些资源需要共享:

1. 基础模型。
2. 公共数据集。
3. 公共 embedding 服务。
4. 公共评估 benchmark。
5. 基础镜像。
6. 工具平台。

共享要有规则:

1. 谁维护?
2. 谁能使用?
3. 是否只读?
4. 是否可复制?
5. 使用后产物如何定级?
6. 成本如何分摊?

共享不是把权限全打开, 而是把共享资源产品化、版本化、权限化。

14.16 多租户隔离架构示例

一个企业 AI 平台可以这样设计:

User / Team

- > Identity Provider
- > AI Platform Portal / CLI
- > Policy Engine
- > Queue / Quota / Scheduler
- > Kubernetes Namespace / Resource Pool
- > Data Access Gateway
- > Secret Manager
- > Image Registry
- > Observability / Audit / Cost

关键点:

1. 身份统一从 IdP 来。
2. Policy Engine 统一判断权限。
3. Scheduler 管资源配额和队列。
4. Data Access Gateway 管数据访问。
5. Secret Manager 管短期凭证。
6. Image Registry 管可信镜像。
7. Observability 管日志、审计和成本。

14.17 多租户隔离审计指标与最小 demo

多租户隔离不能只靠“每个团队一个 namespace”。更工程化的做法是把资源、身份、权限、数据、网络、运行时、镜像、密钥、日志、成本和共享资源都变成可检查证据。

可以把一个租户或任务审计样本写成：

$m_i = (r_i, a_i, d_i, n_i, u_i, g_i, k_i, l_i, c_i, o_i, s_i, z_i)$

其中， r_i 是资源与配额， a_i 是身份与权限， d_i 是数据与 artifact， n_i 是网络策略， u_i 是运行时安全， g_i 是镜像供应链， k_i 是密钥范围， l_i 是日志和 trace， c_i 是成本归因， o_i 是生产 / 实验环境边界， s_i 是跨租户共享规则， z_i 是最终门禁。

统一覆盖率可以写成：

$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(m_i)=1]$

资源配额占用率可以写成：

$Q_u = \frac{G_{u, \text{used}}}{G_{u, \text{quota}}}$

其中， $G_{u, \text{used}}$ 是租户 u 当前使用的 GPU， $G_{u, \text{quota}}$ 是租户 GPU 配额。

未授权访问拦截率可以写成：

$B_{\{\text{auth}\}} = \frac{N_{\{\text{blocked}\}}}{N_{\{\text{unauth}\}}}$

其中， N_{blocked} 是被策略阻断的未授权访问次数， N_{unauth} 是未授权访问尝试总数。多租户系统里，这个值应接近 1。

数据等级传播可以写成：

$L_{\{\text{artifact}\}} \geq \max_{d \in D_{\{\text{train}\}}} L_d$

其中， L_d 是训练数据等级， L_{artifact} 是 checkpoint、模型权重、embedding index 或 eval report 的等级。直觉是：使用高敏数据训练出的产物不能自动降级。

密钥暴露率可以写成：

$R_{\{\text{secret}\}} = \frac{N_{\{\text{secret_exposed}\}}}{N_{\{\text{secret}\}}}$

成本归因覆盖率可以写成：

$C_{\{\text{cost}\}} = \frac{K_{\{\text{attributed}\}}}{K_{\{\text{total}\}}}$

最终，多租户隔离门禁可以写成：

$G_{\{\text{tenant}\}} = \mathbf{1}[\min_j C_j \geq \tau_j \wedge Q_u \leq 1 \wedge B_{\{\text{auth}\}} = 1 \wedge R_{\{\text{secret}\}} = 1]$

其中， τ_j 是每个隔离维度的最低覆盖率， γ 是成本归因最低覆盖率， P_0 是 P0 级隔离风险数量。

下面这个 0 依赖 demo 演示如何把多租户隔离拆成审计规则。它故意构造 1 个完整样本和 16 个坏样本，让每个关键维度各失败一次。

```
import copy
```

```
METRICS = [
    "resource_quota_isolation",
```

```

    "identity_namespace_binding",
    "rbac_abac_permission_fit",
    "data_access_boundary",
    "data_classification_lineage",
    "network_policy_isolation",
    "runtime_security_boundary",
    "image_supply_chain_boundary",
    "secret_scope_rotation",
    "logs_trace_redaction",
    "cost_attribution_coverage",
    "prod_experiment_separation",
    "cross_tenant_sharing_governance",
    "audit_evidence_readiness",
    "blast_radius_control",
    "multi_tenant_isolation_gate",
]

DATA_LEVEL = {
    "public": 0,
    "internal": 1,
    "confidential": 2,
    "restricted": 3,
    "regulated": 4,
}

def quota_ratio(used_gpu, quota_gpu):
    return used_gpu / quota_gpu

def unauthorized_block_rate(blocked, attempts):
    return blocked / attempts

def secret_exposure_rate(exposed, total):
    return exposed / total

def cost_attribution_rate(attributed_cost, total_cost):
    return attributed_cost / total_cost

def blast_radius_ratio(affected_tenants, total_tenants):
    return affected_tenants / total_tenants

def max_data_level(levels):
    return max(DATA_LEVEL[level] for level in levels)

def build_multi_tenant_cases():
    complete = {
        "name": "complete",
        "resource": {

```



```

    "namespace": "team-a-prod",
    "resource_quota": True,
    "limit_range": True,
    "queue_quota": True,
    "gpu_used": 48,
    "gpu_quota": 64,
    "gpu_type_quota": True,
    "overage_policy": "preemptible-borrow",
},
"identity": {
    "idp_group": "team-a",
    "tenant_label": "team-a",
    "namespace_tenant": "team-a",
    "service_account_per_job": True,
    "workload_identity": True,
},
"permission": {
    "rbac": True,
    "abac": True,
    "least_privilege": True,
    "wildcard_actions": False,
    "admin_separation": True,
    "approval_for_quota_change": True,
},
"data": {
    "tenant_dataset_scope": True,
    "unauthorized_attempts": 5,
    "unauthorized_blocked": 5,
    "purpose_bound": True,
    "export_approval": True,
    "copy_policy": True,
},
"classification": {
    "input_levels": ["internal", "restricted"],
    "artifact_level": "restricted",
    "checkpoint_labeled": True,
    "model_labeled": True,
    "eval_report_labeled": True,
    "lineage_recorded": True,
},
"network": {
    "default_deny": True,
    "tenant_ingress_policy": True,
    "egress_allowlist": True,
    "prod_data_endpoint_private": True,
    "debug_port_blocked": True,
},
"runtime": {
    "privileged": False,
    "host_path": False,
    "run_as_root": False,
    "pod_security_enforced": True,
    "gpu_sharing_policy": "exclusive-or-mig",
    "local_cache_scoped": True,
},

```

```

"image": {
  "trusted_registry": True,
  "digest_pinned": True,
  "signed": True,
  "scan_passed": True,
  "base_allowlist": True,
  "critical_vulns": 0,
},
"secret": {
  "secret_manager": True,
  "tenant_scoped": True,
  "short_lived": True,
  "rotation": True,
  "secrets_exposed": 0,
  "secret_total": 10,
  "log_masking": True,
},
"logs": {
  "tenant_log_scope": True,
  "pii_redaction": True,
  "prompt_redaction": True,
  "trace_permission": True,
  "retention_policy": True,
  "export_audit": True,
},
"cost": {
  "tenant": "team-a",
  "project": "policy-llm",
  "cost_center": "cc-123",
  "attributed_cost": 9800,
  "total_cost": 10000,
  "gpu_hours_by_job": True,
  "storage_cost_by_artifact": True,
},
"environment": {
  "prod_namespace": "team-a-prod",
  "experiment_namespace": "team-a-dev",
  "separate_node_pool": True,
  "separate_release_flow": True,
  "prod_slo_guard": True,
  "experiment_image_review": True,
},
"sharing": {
  "shared_resource_owner": True,
  "versioned": True,
  "read_only_by_default": True,
  "copy_requires_approval": True,
  "usage_logged": True,
  "cost_split_rule": True,
},
"audit": {
  "policy_decision_log": True,
  "data_access_log": True,
  "secret_access_log": True,
  "image_admission_log": True,

```

```

        "cost_ledger": True,
        "reviewable_trace": True,
    },
    "blast_radius": {
        "affected_tenants": 1,
        "total_tenants": 20,
        "max_ratio": 0.1,
        "fault_domain_labeled": True,
        "kill_switch": True,
    },
    "gate": {"enabled": True, "thresholds_ok": True, "p0_risks": 0},
}

```

```

def bad_case(name, mutator):
    case = copy.deepcopy(complete)
    case["name"] = name
    mutator(case)
    return case

```

```

bad_cases = [
    bad_case("resource_quota_missing_bad", lambda c: c["resource"].update({"resource_quota": False})),
    bad_case("identity_namespace_mismatch_bad", lambda c: c["identity"].update({"namespace_tenant": "team-b"})),
    bad_case("permission_overbroad_bad", lambda c: c["permission"].update({"wildcard_actions": True})),
    bad_case("data_cross_access_bad", lambda c: c["data"].update({"unauthorized_blocked": 3})),
    bad_case("classification_not_propagated_bad", lambda c: c["classification"].update({"artifact_level": "internal"})),
    bad_case("network_default_allow_bad", lambda c: c["network"].update({"default_deny": False})),
    bad_case("runtime_privileged_bad", lambda c: c["runtime"].update({"privileged": True})),
    bad_case("image_untrusted_bad", lambda c: c["image"].update({"signed": False})),
    bad_case("secret_shared_long_lived_bad", lambda c: c["secret"].update({"tenant_scoped": False, "short_lived": True})),
    bad_case("logs_unredacted_bad", lambda c: c["logs"].update({"pii_redaction": False})),
    bad_case("cost_unattributed_bad", lambda c: c["cost"].update({"attributed_cost": 7000})),
    bad_case("prod_experiment_mixed_bad", lambda c: c["environment"].update({"separate_node_pool": False})),
    bad_case("sharing_unversioned_bad", lambda c: c["sharing"].update({"versioned": False})),
    bad_case("audit_missing_bad", lambda c: c["audit"].update({"data_access_log": False})),
    bad_case("blast_radius_large_bad", lambda c: c["blast_radius"].update({"affected_tenants": 6})),
    bad_case("tenant_gate_missing_bad", lambda c: c["gate"].update({"enabled": False})),
]
return [complete] + bad_cases

```

```

def check_resource(case):
    resource = case["resource"]
    return (
        resource["resource_quota"]
        and resource["limit_range"]
        and resource["queue_quota"]
        and quota_ratio(resource["gpu_used"], resource["gpu_quota"]) <= 1
        and resource["gpu_type_quota"]
        and resource["overage_policy"] in {"deny", "preemptible-borrow"}
    )

```

```

def check_identity(case):
    identity = case["identity"]
    return (

```

```

        identity["idp_group"] == identity["tenant_label"]
    and identity["namespace_tenant"] == identity["tenant_label"]
    and identity["service_account_per_job"]
    and identity["workload_identity"]
)

def check_permission(case):
    permission = case["permission"]
    return (
        permission["rbac"]
    and permission["abac"]
    and permission["least_privilege"]
    and not permission["wildcard_actions"]
    and permission["admin_separation"]
    and permission["approval_for_quota_change"]
)

def check_data(case):
    data = case["data"]
    return (
        data["tenant_dataset_scope"]
    and unauthorized_block_rate(data["unauthorized_blocked"], data["unauthorized_attempts"]) == 1
    and data["purpose_bound"]
    and data["export_approval"]
    and data["copy_policy"]
)

def check_classification(case):
    cls = case["classification"]
    return (
        DATA_LEVEL[cls["artifact_level"]] >= max_data_level(cls["input_levels"])
    and cls["checkpoint_labeled"]
    and cls["model_labeled"]
    and cls["eval_report_labeled"]
    and cls["lineage_recorded"]
)

def check_network(case):
    return all(case["network"].values())

def check_runtime(case):
    runtime = case["runtime"]
    return (
        not runtime["privileged"]
    and not runtime["host_path"]
    and not runtime["run_as_root"]
    and runtime["pod_security_enforced"]
    and runtime["gpu_sharing_policy"] in {"exclusive", "exclusive-or-mig"}
    and runtime["local_cache_scoped"]
)

```

```

def check_image(case):
    image = case["image"]
    return (
        image["trusted_registry"]
        and image["digest_pinned"]
        and image["signed"]
        and image["scan_passed"]
        and image["base_allowlist"]
        and image["critical_vulns"] == 0
    )

def check_secret(case):
    secret = case["secret"]
    return (
        secret["secret_manager"]
        and secret["tenant_scoped"]
        and secret["short_lived"]
        and secret["rotation"]
        and secret_exposure_rate(secret["secrets_exposed"], secret["secret_total"]) == 0
        and secret["log_masking"]
    )

def check_logs(case):
    return all(case["logs"].values())

def check_cost(case):
    cost = case["cost"]
    return (
        bool(cost["tenant"])
        and bool(cost["project"])
        and bool(cost["cost_center"])
        and cost_attribution_rate(cost["attributed_cost"], cost["total_cost"]) >= 0.95
        and cost["gpu_hours_by_job"]
        and cost["storage_cost_by_artifact"]
    )

def check_environment(case):
    env = case["environment"]
    return (
        env["prod_namespace"] != env["experiment_namespace"]
        and env["separate_node_pool"]
        and env["separate_release_flow"]
        and env["prod_slo_guard"]
        and env["experiment_image_review"]
    )

def check_sharing(case):
    return all(case["sharing"].values())

```

```

def check_audit(case):
    return all(case["audit"].values())

def check_blast_radius(case):
    blast = case["blast_radius"]
    return (
        blast_radius_ratio(blast["affected_tenants"], blast["total_tenants"]) <= blast["max_ratio"]
        and blast["fault_domain_labeled"]
        and blast["kill_switch"]
    )

def check_gate(case):
    gate = case["gate"]
    return gate["enabled"] and gate["thresholds_ok"] and gate["p0_risks"] == 0

CHECKS = {
    "resource_quota_isolation": check_resource,
    "identity_namespace_binding": check_identity,
    "rbac_abac_permission_fit": check_permission,
    "data_access_boundary": check_data,
    "data_classification_lineage": check_classification,
    "network_policy_isolation": check_network,
    "runtime_security_boundary": check_runtime,
    "image_supply_chain_boundary": check_image,
    "secret_scope_rotation": check_secret,
    "logs_trace_redaction": check_logs,
    "cost_attribution_coverage": check_cost,
    "prod_experiment_separation": check_environment,
    "cross_tenant_sharing_governance": check_sharing,
    "audit_evidence_readiness": check_audit,
    "blast_radius_control": check_blast_radius,
    "multi_tenant_isolation_gate": check_gate,
}

def audit_multi_tenant_isolation(cases):
    case_failures = {}
    for case in cases:
        failures = [name for name, check in CHECKS.items() if not check(case)]
        case_failures[case["name"]] = failures

    metrics = {}
    for name, check in CHECKS.items():
        metrics[name] = round(sum(int(check(case)) for case in cases) / len(cases), 3)

    failed_cases = [name for name, failures in case_failures.items() if failures]
    failed_gates = [name for name, value in metrics.items() if value < 1.0]
    return {
        "metrics": metrics,
        "hard_blocker_count": len(failed_cases),
    }

```

```

    "failed_cases": failed_cases,
    "failed_gates": failed_gates,
    "multi_tenant_gate_pass": not failed_cases and min(metrics.values()) >= 0.95,
}

cases = build_multi_tenant_cases()
case_by_name = {case["name"]: case for case in cases}
complete = case_by_name["complete"]
tenant_examples = {
    "quota_ratio": round(quota_ratio(48, 64), 3),
    "unauthorized_block_rate": round(unauthorized_block_rate(5, 5), 3),
    "artifact_level_ok": DATA_LEVEL["restricted"] >= max_data_level(["internal", "restricted"]),
    "secret_exposure_rate": round(secret_exposure_rate(0, 10), 3),
    "cost_attribution_rate": round(cost_attribution_rate(9800, 10000), 3),
    "blast_radius_ratio": round(blast_radius_ratio(1, 20), 3),
}

smoke = {
    "complete_case_passes": all(check(complete) for check in CHECKS.values()),
    "caught_cross_data_access": not check_data(case_by_name["data_cross_access_bad"]),
    "caught_classification_gap": not check_classification(case_by_name["classification_not_propagated_bad"]),
    "caught_network_default_allow": not check_network(case_by_name["network_default_allow_bad"]),
    "caught_secret_gap": not check_secret(case_by_name["secret_shared_long_lived_bad"]),
    "caught_blast_radius": not check_blast_radius(case_by_name["blast_radius_large_bad"]),
}

audit = audit_multi_tenant_isolation(cases)
print(f"tenant_examples={tenant_examples}")
print(f"smoke={smoke}")
print(f"metrics={audit['metrics']}")
print(f"hard_blocker_count={audit['hard_blocker_count']}")
print(f"failed_cases={audit['failed_cases']}")
print(f"failed_gates={audit['failed_gates']}")
print(f"multi_tenant_gate_pass={audit['multi_tenant_gate_pass']}")

```

一组典型输出是：

```

tenant_examples={'quota_ratio': 0.75, 'unauthorized_block_rate': 1.0, 'artifact_level_ok': True, 'secret_exposure_rate': 0.0, 'cost_attribution_rate': 0.98, 'blast_radius_ratio': 0.05}
smoke={'complete_case_passes': True, 'caught_cross_data_access': True, 'caught_classification_gap': True, 'caught_network_default_allow': True, 'caught_secret_gap': True, 'caught_blast_radius': True}
metrics={'resource_quota_isolation': 0.941, 'identity_namespace_binding': 0.941, 'rbac_abac_permission_fit': 0.941, 'data_access_boundaries': 0.941, 'secret_exposure_rate': 0.0, 'cost_attribution_rate': 0.98, 'blast_radius_ratio': 0.05}
hard_blocker_count=16
failed_cases=['resource_quota_missing_bad', 'identity_namespace_mismatch_bad', 'permission_overbroad_bad', 'data_cross_access_boundaries_bad']
failed_gates=['resource_quota_isolation', 'identity_namespace_binding', 'rbac_abac_permission_fit', 'data_access_boundaries']
multi_tenant_gate_pass=False

```

这个 demo 的重点是把多租户隔离从“namespace 分组”拆成可验证证据链：资源配额要生效，身份和 namespace 要绑定，RBAC / ABAC 要最小权限，数据访问要按租户和用途阻断，数据等级要传播到 checkpoint / 模型 / eval report，网络默认拒绝，运行时不能 privileged / hostPath / root，镜像要可信，密钥要短期和租户化，日志和 trace 要脱敏，成本要归因，生产和实验要隔离，跨租户共享要有 owner、版本和审计，最终还要能控制故障影响范围。

14.18 面试中如何回答多租户隔离

如果面试官问：

如何设计一个多租户 GPU 训练平台？

可以这样回答：

我会从资源、权限、数据、网络、运行时、镜像、密钥、审计和成本几个层面设计隔离。

资源层用 namespace、queue、quota、priority 和资源池隔离不同团队，关键生产推理和高敏任务使用独立节点池或硬隔离资源池。

权限层接入统一身份系统，用 RBAC + ABAC 控制谁能提交任务、访问数据、读取日志、部署模型和修改配额。

数据层按数据分类和项目授权控制访问，训练数据、checkpoint、模型权重、RAG 文档、日志和 trace 都要有权限和审计。使用敏感数据训练模型时，数据要加密。

网络层通过 NetworkPolicy、VPC、egress 控制和私有 endpoint 限制租户间互访和外部访问。运行时层限制 privileged、hostPath、

镜像和密钥层使用私有 registry、镜像扫描、签名、digest、Secret Manager 和短期凭证，避免密钥写入镜像或日志。

最后通过审计和成本系统记录谁在什么时候用了什么资源、访问了什么数据、产生了什么模型和成本。

14.19 常见误区

误区一：namespace 就是多租户隔离。

Namespace 只是逻辑分组，还需要资源、权限、网络、数据、密钥和审计隔离。

误区二：容器就是强安全边界。

容器隔离有限，privileged、hostPath、共享 GPU 和高权限密钥都可能破坏隔离。

误区三：只隔离训练数据就够了。

Checkpoint、模型权重、日志、trace、prompt 和评估结果都可能包含敏感信息。

误区四：共享资源就是所有人可读。

共享需要 owner、版本、权限、审计、成本分摊和使用策略。

误区五：成本归因是财务问题。

成本归因直接影响配额、公平性、资源优化和平台治理，是 AI Infra 的核心能力。

14.20 面试题

题 1：AI Infra 多租户隔离包括哪些层？

答：包括资源隔离、权限隔离、数据隔离、网络隔离、运行时隔离、镜像和供应链隔离、密钥隔离、日志审计隔离和成本隔离。

题 2：为什么数据隔离不只是文件权限？

答：训练任务读取数据后，数据可能进入 checkpoint、模型权重、日志、trace 或评估结果。使用敏感数据训练出的模型也可能继承敏感等级。所以要控制数据使用目的和下游产物权限。

题 3：生产推理为什么要和实验训练隔离？

答：生产推理有 SLO 和稳定性要求，实验训练可能不稳定、占用网络和存储、使用未审核镜像或访问不同数据。混用会造成性能干扰和安全风险。

题 4：如何避免密钥泄露？

答：不要把密钥写进镜像、代码或日志。使用 Secret Manager、短期凭证、Service Account、最小权限、自动轮换、日志脱敏和访问审计。

题 5：硬隔离和软隔离如何选择？

答：高敏数据、生产推理、关键业务和强合规场景适合硬隔离，例如专属资源池或独立集群。普通训练、评估和研究任务可用软隔离，通过 namespace、quota、queue、policy 和审计提高资源利用率。

14.21 小练习

练习一：设计一个团队级租户模型。

要求：包含用户、项目、namespace、queue、quota、dataset permission、model permission 和 cost center。

练习二：设计一个数据等级传播规则。

要求：说明 restricted 数据训练出的 checkpoint、模型权重、eval report 应该如何定级和授权。

练习三：排查跨租户数据泄露风险。

假设某用户在日志里看到了其他团队的数据片段，请从日志权限、数据挂载、任务身份、共享目录、trace 和脱敏策略排查。

练习四：设计一个多租户成本报表。

要求：按团队、项目、用户、任务、GPU 类型、存储和推理 token 展示成本。

14.22 本章小结

本章讲了 AI Infra 多租户隔离。

你需要掌握：

1. 多租户隔离不是 namespace，而是系统性治理。
2. 资源隔离要覆盖 GPU、CPU、内存、存储、网络、队列、推理 QPS 和成本。
3. 权限隔离需要 RBAC + ABAC。
4. 数据隔离要覆盖训练数据、checkpoint、模型权重、日志、trace、RAG 文档和评估结果。
5. 敏感数据训练出的模型和 artifact 可能需要继承数据等级。
6. 网络隔离要控制租户互访、外部访问和生产数据访问。
7. 运行时隔离要限制 privileged、hostPath、root、共享 GPU 等风险。
8. 镜像、密钥、日志和审计都是多租户安全的重要部分。
9. 成本隔离和归因是配额、公平性和资源优化的基础。

下一章我们会讲集群容量规划：GPU 数量、网络带宽、存储吞吐和利用率。

第 15 章 集群容量规划：GPU 数量、网络带宽、存储吞吐和利用率

前面几章讲了 GPU 集群、网络、存储、Kubernetes、调度和多租户隔离。本章是第二部分的收尾：如何规划一个 AI 集群到底需要多少资源？

很多团队一开始会问：“我们要买多少 GPU？”这个问题本身不完整。真正要问的是：为了支撑哪些任务画像、在什么 SLO、成本和增长假设下，需要多少 GPU、什么型号、多少网络带宽、多少存储吞吐、多少冗余，以及如何保证利用率？

先记住一句话：

AI 集群容量规划不是按预算买卡，而是按工作负载、性能目标、增长预期、故障冗余和成本约束设计一整套计算、网络、存储和调度容量。

15.0 本讲资料边界与第二轮精修口径

第二轮精修时，本章按 WRITING_PLAN.md 的要求做了联网资料校准，主要参考 Kubernetes 对 resource request / limit、extended resource、ResourceQuota 和 HorizontalPodAutoscaler 的稳定口径，参考 NVIDIA DCGM

对 GPU profiling metrics、SM activity、显存带宽和 PCIe 传输指标的定义，参考 Google SRE 对 overload、按资源而不是只按 QPS 建模、quota、降级和限流的工程经验。

本章只抽象 AI Infra 集群容量规划的稳定工程问题：如何把 GPU 数量、GPU 型号、训练队列、推理峰值、网络、存储、利用率、故障冗余、增长预测和成本预算放进同一张容量表。它不绑定某个云厂商报价、某一代 GPU 性能参数、某个内部调度器实现或某个固定采购流程。

本章第二轮补强重点有三点：

1. 把原来文字化的 GPU-hours、推理容量、网络和存储估算改成可复用公式。
2. 明确容量规划不能只看平均 GPU utilization，而要看 SLO、峰值、余量、成本和预测偏差。
3. 新增一个 0 依赖 Python demo，用 toy case 演示如何把完整容量计划和 16 类失败案例做成容量规划门禁。

15.1 为什么容量规划重要

容量规划做不好，会出现两类问题。

第一类是资源不足：

1. 训练任务长期排队。
2. 推理高峰延迟上升。
3. 评估任务无法按时完成。
4. 新项目无法启动。
5. 抢占过于频繁。
6. 团队为了资源互相冲突。

第二类是资源浪费：

1. GPU 闲置。
2. 高端 GPU 被低价值任务占用。
3. 网络和存储过度配置。
4. checkpoint 和日志存储膨胀。
5. 推理实例过度冗余。
6. 成本高但产出低。

容量规划的目标是在“缺资源”和“浪费资源”之间找到可治理的平衡。

15.2 容量规划不是只算 GPU

AI 集群容量包括：

1. GPU 数量。
2. GPU 型号和显存。
3. CPU 和内存。
4. 机内互联。
5. 机间网络。
6. 本地盘。
7. 共享存储。
8. 对象存储。
9. checkpoint 吞吐。
10. 模型加载带宽。
11. 推理服务容量。
12. 监控和日志容量。

只买 GPU 不配网络和存储，会导致 GPU 等通信或等数据。

只规划训练不规划推理，会导致模型训出来但上线扛不住流量。

只规划平均负载不规划峰值，会导致关键时刻排队或超时。

15.3 从任务画像开始

容量规划第一步是拆任务画像。

至少要分：

1. 预训练。
2. SFT。
3. RLHF / RLAIIF。
4. 离线评估。
5. 在线推理。
6. RAG / embedding。
7. Agent。
8. 交互式开发。

每类任务要估算：

1. 每个任务需要多少 GPU。
2. 任务运行多久。
3. 每月任务数量。
4. 是否有截止时间。
5. 是否可抢占。
6. 是否需要高端 GPU。
7. 是否需要高速网络。
8. 是否需要大量存储读写。

没有任务画像，容量规划就是拍脑袋。

15.3.1 关键公式与容量规划速查

先把一段时间窗口内的训练需求折算成 GPU-hours：

$$H_{\{\mathrm{gpu}\}} = \sum_{k=1}^K n_k g_k t_k r_k$$

其中， k 是任务类型， n_k 是窗口内任务数， g_k 是单任务 GPU 数， t_k 是单任务小时数， r_k 是重试、失败恢复和实验返工系数。

每张 GPU 在窗口内能提供的有效小时数为：

$$C_{\{\mathrm{one}\}} = D \cdot 24 \cdot u_{\{\mathrm{eff}\}}$$

其中， D 是窗口天数， u_{eff} 是目标有效利用率，不是 DCGM 或 nvidia-smi 里看到的瞬时 GPU utilization。容量规划中更关心 “这张卡是否在完成有价值工作”。

考虑峰值、故障、碎片和增长之后，训练 GPU 需求可以写成：

$$G_{\{\mathrm{need}\}} = \left\lceil \frac{H_{\{\mathrm{gpu}\}}}{D \cdot 24 \cdot u_{\{\mathrm{eff}\}}} \right\rceil (1 + h_{\{\mathrm{peak}\}} + h_{\{\mathrm{fail}\}} + h_{\{\mathrm{frag}\}} + h_{\{\mathrm{growth}\}})$$

其中， h_{peak} 是峰值余量， h_{fail} 是故障和维护余量， h_{frag} 是调度碎片余量， h_{growth} 是增长余量。

推理实例数不能只按平均 QPS 算。一个简化估算是：

$$I_{\{\mathrm{serve}\}} = \left\lceil \frac{\lambda_{\{\mathrm{peak}\}} (L_{\{\mathrm{in}\}} + L_{\{\mathrm{out}\}})}{\mu_{\{\mathrm{inst}\}}} \right\rceil \eta_{\{\mathrm{slo}\}}$$

其中， λ_{peak} 是峰值 QPS， L_{in} 和 L_{out} 分别是输入和输出 token 数， μ_{inst} 是单实例在目标延迟下可稳定提供的 token/s， η_{slo} 是为了满足 TTFT、TPOT 和 P99 延迟保留的折扣系数。

训练网络容量至少要覆盖关键 collective 的通信量：

$$B_{\{\mathrm{net}\}} \geq \frac{V_{\{\mathrm{comm}\}}}{T_{\{\mathrm{step}\}}} \eta_{\{\mathrm{net}\}}$$

其中， V_{comm} 是每步通信量， T_{step} 是目标 step time， η_{net} 是网络有效效率。实际规划还要看 RDMA、GPU-NIC 亲和性、机柜内外 oversubscription、重传和 rank skew。

存储吞吐下限可以取多条路径的最大值：

$$B_{\mathrm{store}} \geq \max \left(B_{\mathrm{data}}, \frac{S_{\mathrm{ckpt}}}{T_{\mathrm{ckpt}}}, \frac{S_{\mathrm{model}}}{T_{\mathrm{load}}} \right)$$

其中, B_{data} 是训练数据读取吞吐, $S_{\mathrm{ckpt}} / T_{\mathrm{ckpt}}$ 是 checkpoint 写入吞吐, $S_{\mathrm{model}} / T_{\mathrm{load}}$ 是模型权重加载吞吐。

利用率目标必须留下弹性:

$$u_{\mathrm{target}} \leq 1 - h_{\mathrm{burst}} - h_{\mathrm{maint}} - h_{\mathrm{fail}}$$

如果长期目标利用率高到没有 burst、维护和故障余量, 表面上省钱, 实际会导致排队、抢占、推理扩容失败和故障恢复困难。

滚动预测要追踪预测偏差:

$$E_{\mathrm{forecast}} = \frac{|D_{\mathrm{actual}} - D_{\mathrm{forecast}}|}{D_{\mathrm{forecast}}}$$

其中, D_{actual} 是实际需求, D_{forecast} 是预测需求。容量评审不是写完一次文档, 而是每月或每季度用预测偏差修正下一轮采购、云上弹性和配额策略。

最后可以把容量规划门禁写成:

$$G_{\mathrm{capacity}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \ \& \ G_{\mathrm{actual}} \geq G_{\mathrm{need}} \ \& \ I_{\mathrm{actual}} \leq I_{\mathrm{max}} \right]$$

其中, C_j 是第 j 个治理检查的覆盖率, G_{actual} 是实际可用 GPU 数, I_{actual} 是实际推理实例数, K_{est} 是估算成本, K_{budget} 是预算, P_0 是硬阻断数量。

15.4 GPU 数量估算

GPU 数量可以从 GPU-hours 估算。

例如一个任务需要 64 张 GPU 训练 48 小时, 那么消耗:

$$64 * 48 = 3072 \text{ GPU-hours}$$

如果一个月有 20 个类似任务, 需要:

$$3072 * 20 = 61440 \text{ GPU-hours}$$

如果集群希望平均有效利用率为 60%, 一个月按 30 天计算, 每张 GPU 可提供:

$$30 * 24 * 0.60 = 432 \text{ effective GPU-hours}$$

所需 GPU 数约为:

$$61440 / 432 \approx 143 \text{ GPUs}$$

这只是粗略估算, 还要加上:

1. 峰值需求。
2. 排队容忍度。
3. 故障冗余。
4. 调度碎片。
5. 资源池隔离。
6. 增长预期。

15.5 平均负载和峰值负载

只按平均负载规划会低估容量。

训练任务可能集中在:

1. 发布前。
2. 论文截止前。
3. 大版本迭代前。
4. 数据刷新后。
5. 新模型启动阶段。

推理流量也有峰值：

1. 白天高峰。
2. 活动期间。
3. 新产品发布。
4. 批量任务集中提交。
5. 外部事件触发。

容量规划要同时看：

1. average demand。
2. p95 demand。
3. peak demand。
4. burst duration。
5. 可延迟任务比例。
6. 可降级任务比例。

训练任务可以排队，推理请求通常不能长时间排队。这两类容量要分开考虑。

15.6 GPU 型号规划

不是所有 GPU 都适合所有任务。

需要按任务匹配 GPU 型号。

高端大显存 GPU 适合：

1. 预训练。
2. 大模型 SFT。
3. 长上下文推理。
4. 大 batch 推理。
5. 张量并行任务。

中端 GPU 适合：

1. 小模型 SFT。
2. embedding。
3. rerank。
4. 离线评估。
5. 小模型推理。

低成本或可抢占资源适合：

1. 实验探索。
2. 批量评估。
3. 数据处理。
4. 低优先级训练。

容量规划不能只问“买多少卡”，还要问“每种卡买多少”。

15.7 训练容量规划

训练容量规划要考虑：

1. 预训练计划。
2. SFT 并发实验数。
3. RLHF rollout 需求。
4. 评估任务穿插。
5. 失败重试。
6. checkpoint 开销。
7. 调度碎片。
8. 团队配额。

一个训练容量表可以这样设计：

任务类型	单任务 GPU	单任务时长	月任务数	GPU-hours	是否可抢占
7B SFT	8	10h	80	6400	是
70B SFT	64	24h	10	15360	否
Eval	4	6h	200	4800	是
Pretrain	256	120h	2	61440	否

表格能帮助你从需求倒推容量，而不是凭感觉估算。

15.8 推理容量规划

推理容量规划和训练不同。

训练按 GPU-hours 估算较多，推理要按流量和延迟估算。

关键指标：

1. QPS。
2. 输入 token 长度。
3. 输出 token 长度。
4. TTFT。
5. TPOT。
6. 并发。
7. batch size。
8. KV cache 显存。
9. p95 / p99 延迟。
10. 可用性目标。

推理容量粗略流程：

1. 估算每秒输入和输出 token。
2. 测量单卡或单实例吞吐。
3. 根据延迟 SLO 确定可接受 batch。
4. 估算需要多少实例。
5. 加上峰值和冗余。
6. 规划自动扩缩容。

推理不能只看平均 QPS。长上下文请求、长输出请求和短请求的资源消耗差异很大。

15.9 网络容量规划

网络容量规划要看通信模式。

训练网络需求取决于：

1. 数据并行规模。
2. 张量并行是否跨机。
3. FSDP / ZeRO 通信量。
4. Pipeline 并行 stage 放置。
5. checkpoint 写入路径。
6. 数据读取路径。

网络规划要考虑：

1. 单节点网卡带宽。
2. 机柜内带宽。
3. 机柜间带宽。
4. oversubscription ratio。

5. 多任务并发拥塞。
6. RDMA 支持。
7. 网络故障冗余。

如果预训练任务要求高扩展效率，网络不能只按普通业务流量规划。

集合通信会造成同步大流量，尾延迟会拖慢整个训练。

15.10 存储容量和吞吐规划

存储要同时规划容量和吞吐。

容量包括：

1. 原始数据。
2. 清洗数据。
3. 训练 shard。
4. checkpoint。
5. 模型权重。
6. eval report。
7. 日志和 trace。
8. RAG 文档和索引。

吞吐包括：

1. 训练数据读取吞吐。
2. checkpoint 写入吞吐。
3. checkpoint 恢复读取吞吐。
4. 模型权重加载吞吐。
5. 多任务并发读写吞吐。

例如 checkpoint 容量估算：

单个 checkpoint 大小 * 保留数量 * 实验数量

如果单个训练状态 checkpoint 是 2TB，每个实验保留 10 个，10 个实验并发，就是：

$$2\text{TB} * 10 * 10 = 200\text{TB}$$

这还不包括模型发布权重、日志和中间数据。

15.11 利用率目标

GPU 利用率不是越高越好。

如果目标利用率设得太高，例如长期 95%，会导致：

1. 新任务排队严重。
2. 高优先级任务没有弹性空间。
3. 推理峰值无法扩容。
4. 维护和故障恢复困难。
5. 抢占频繁。

如果利用率太低，成本浪费。

合理目标要按资源池区分：

1. 预训练资源池：追求较高利用率，但保留维护窗口。
2. 推理资源池：需要冗余，不能压满。
3. 评估资源池：可以使用低优先级资源，提高利用率。
4. 交互式资源池：需要快速响应，利用率不能太满。

容量规划要区分“分配率”和“有效利用率”。GPU 被分配了不代表有效训练。

15.12 冗余和故障预留

生产系统必须预留冗余。

原因：

1. GPU 故障。
2. 节点维护。
3. 网络故障。
4. 存储故障。
5. 推理峰值。
6. 突发任务。
7. 灰度发布。

冗余策略：

1. N+1 节点冗余。
2. 推理服务多副本。
3. 跨机柜部署。
4. 关键 checkpoint 多副本。
5. 高优任务预留资源。
6. 低优任务可抢占。

冗余会降低表面利用率，但提高可靠性和业务连续性。

15.13 增长预测

AI 资源需求增长通常很快。

容量规划要看：

1. 模型规模增长。
2. context length 增长。
3. 用户流量增长。
4. 团队数量增长。
5. 实验数量增长。
6. 数据规模增长。
7. Agent 调用链增长。
8. 多模态数据增长。

常见做法：

1. 做 3 个月、6 个月、12 个月预测。
2. 按保守、中性、激进三种场景规划。
3. 设置触发扩容阈值。
4. 定期回顾实际用量和预测偏差。

容量规划不是一次性文档，而是持续滚动过程。

15.14 云上、混合云和自建容量

容量来源可以是：

1. 自建集群。
2. 公有云 GPU。
3. 混合云。
4. 外部推理 API。
5. 低优先级或竞价资源。

自建适合稳定长期大规模需求。

云上适合弹性峰值、临时实验和快速扩容。

混合云适合把基础负载放自建，把突发负载放云上。

但混合云要注意：

1. 数据迁移成本。
2. 网络延迟。
3. 安全和合规。
4. 镜像和环境一致性。
5. 成本监控。
6. 调度系统统一视图。

15.15 容量规划指标体系

一个 AI 集群容量 dashboard 应该包含：

1. GPU 总量。
2. GPU 分配率。
3. GPU 有效利用率。
4. 各队列 pending time。
5. 各租户 GPU-hours。
6. GPU 碎片率。
7. 抢占次数。
8. 推理 QPS 和延迟。
9. 推理实例利用率。
10. 网络吞吐和拥塞。
11. 存储吞吐。
12. checkpoint 容量。
13. 日志和 trace 增长。
14. 成本趋势。
15. 预测需求和实际需求偏差。

没有这些指标，容量规划只能靠感觉。

15.16 常见容量规划错误

错误一：只按当前需求买 GPU。

没有考虑增长、峰值和故障冗余。

错误二：只买最高端 GPU。

很多评估、embedding、小模型推理和开发任务不需要最高端 GPU。

错误三：只看 GPU 数量，不看网络和存储。

训练可能被通信和 I/O 拖慢。

错误四：把训练和推理混在一起规划。

训练可以排队，推理需要 SLO 和冗余。

错误五：追求 100% 利用率。

这会牺牲弹性、可靠性和高优任务响应。

错误六：不做成本归因。

不知道谁在用资源，就无法优化。

15.17 集群容量规划审计指标与最小 demo

下面这个 demo 不模拟真实 GPU 集群，而是演示容量规划评审应该怎么结构化：一个完整样本要同时通过任务画像、GPU 数量、GPU 型号、训练队列、推理峰值、网络、存储、生命周期、利用率余量、故障冗余、增长预测、成本、资源池隔离、quota / burst 和观测回填；16 个 bad case 分别故意打破一个门禁。

```
from math import ceil
```

```
METRICS = [  
    "workload_forecast_coverage",  
    "gpu_count_capacity_fit",  
    "gpu_type_mix_fit",  
    "training_queue_slo_fit",  
    "serving_peak_slo_fit",  
    "network_bandwidth_capacity_fit",  
    "storage_throughput_capacity_fit",  
    "storage_capacity_lifecycle_fit",  
    "utilization_headroom_fit",  
    "failure_redundancy_fit",  
    "growth_forecast_fit",  
    "cost_budget_fit",  
    "pool_separation_fit",  
    "quota_burst_governance",  
    "observability_forecast_feedback",  
    "cluster_capacity_planning_gate",  
]
```

```
def percentile(values, pct):  
    ordered = sorted(values)  
    if not ordered:  
        return 0.0  
    index = int(round((len(ordered) - 1) * pct / 100.0))  
    return ordered[index]
```

```
def required_training_gpus(case):  
    base = case["monthly_gpu_hours"] / (  
        case["days"] * 24.0 * case["target_effective_utilization"]  
    )  
    multiplier = (  
        1.0  
        + case["peak_headroom"]  
        + case["failure_headroom"]  
        + case["fragmentation_headroom"]  
        + case["growth_headroom"]  
    )  
    return int(ceil(base * multiplier))
```

```
def required_serving_instances(case):  
    peak_tokens_per_s = (  
        case["peak_qps"] * (case["input_tokens_p95"] + case["output_tokens_p95"])  
    )  
    stable_tokens_per_s = (  
        case["stable_qps"] * (case["input_tokens_p95"] + case["output_tokens_p95"])
```

```

        case["instance_token_capacity_per_s"] * case["slo_efficiency"]
    )
    return int(ceil(peak_tokens_per_s / stable_tokens_per_s))

def required_network_gibs(case):
    return case["comm_gib_per_step"] / (
        case["target_step_seconds"] * case["network_efficiency"]
    )

def required_storage_gibs(case):
    checkpoint_write = (
        case["checkpoint_tib"] * 1024.0 / case["checkpoint_window_seconds"]
    )
    model_load = case["model_tib"] * 1024.0 / case["model_load_window_seconds"]
    return max(case["data_read_gibs"], checkpoint_write, model_load)

def checkpoint_capacity_tib(case):
    return (
        case["checkpoint_tib"]
        * case["retained_checkpoints"]
        * case["concurrent_experiments"]
    )

def complete_case():
    return {
        "name": "complete_capacity_plan",
        "workload_profiles": True,
        "monthly_gpu_hours": 61440,
        "days": 30,
        "target_effective_utilization": 0.60,
        "peak_headroom": 0.20,
        "failure_headroom": 0.10,
        "fragmentation_headroom": 0.08,
        "growth_headroom": 0.15,
        "actual_gpus": 220,
        "gpu_type_mix_ok": True,
        "queue_wait_minutes": [22, 30, 36, 40, 44],
        "queue_slo_minutes": 60,
        "peak_qps": 42,
        "input_tokens_p95": 900,
        "output_tokens_p95": 220,
        "instance_token_capacity_per_s": 9000,
        "slo_efficiency": 0.72,
        "actual_serving_instances": 10,
        "serving_p99_seconds": 1.6,
        "serving_p99_slo_seconds": 2.0,
        "comm_gib_per_step": 120,
        "target_step_seconds": 40,
        "network_efficiency": 0.75,
        "provisioned_network_gibs": 5.2,
        "data_read_gibs": 3.5,
    }

```

```

"checkpoint_tib": 6.0,
"checkpoint_window_seconds": 1500,
"model_tib": 1.5,
"model_load_window_seconds": 300,
"provisioned_storage_gibs": 8.0,
"storage_measured": True,
"retained_checkpoints": 12,
"concurrent_experiments": 8,
"total_checkpoint_capacity_tib": 900,
"lifecycle_policy": True,
"burst_headroom": 0.18,
"maintenance_headroom": 0.08,
"n_plus_one_ready": True,
"forecast_period_months": 12,
"forecast_error": 0.09,
"estimated_monthly_cost_usd": 470000,
"monthly_budget_usd": 550000,
"pool_separation": True,
"quota_ledger": True,
"burst_credits": True,
"observability_metrics": {
    "gpu_allocated",
    "gpu_effective_utilization",
    "queue_wait_p95",
    "serving_p99",
    "network_throughput",
    "storage_throughput",
    "checkpoint_growth",
    "forecast_error",
    "cost_by_tenant",
    "fragmentation_ratio",
},
"forecast_review_days": 30,
"capacity_gate_defined": True,
}

```

```

def build_cases():
    cases = [complete_case()]
    mutations = [
        ("workload_missing_bad", {"workload_profiles": False}),
        ("gpu_count_underplanned_bad", {"actual_gpus": 180}),
        ("gpu_type_mismatch_bad", {"gpu_type_mix_ok": False}),
        ("queue_slo_missing_bad", {"queue_wait_minutes": [70, 80, 95]}),
        ("serving_peak_ignored_bad", {"actual_serving_instances": 6}),
        ("network_underprovisioned_bad", {"provisioned_network_gibs": 2.0}),
        ("storage_throughput_unknown_bad", {"storage_measured": False}),
        ("lifecycle_capacity_missing_bad", {"total_checkpoint_capacity_tib": 500}),
        ("target_util_too_high_bad", {"target_effective_utilization": 0.88}),
        ("no_failure_redundancy_bad", {"n_plus_one_ready": False}),
        ("growth_forecast_missing_bad", {"forecast_period_months": 1}),
        ("cost_budget_missing_bad", {"estimated_monthly_cost_usd": 620000}),
        ("pools_mixed_bad", {"pool_separation": False}),
        ("burst_quota_uncontrolled_bad", {"burst_credits": False}),
        ("observability_missing_bad", {"observability_metrics": {"gpu_allocated"}}),
    ]

```

```

        ("capacity_gate_missing_bad", {"capacity_gate_defined": False}),
    ]
    for name, patch in mutations:
        item = complete_case()
        item["name"] = name
        item.update(patch)
        cases.append(item)
    return cases

def evaluate_case(case):
    required_gpus = required_training_gpus(case)
    required_instances = required_serving_instances(case)
    network_need = required_network_gibs(case)
    storage_need = required_storage_gibs(case)
    checkpoint_need = checkpoint_capacity_tib(case)
    max_safe_util = 1.0 - (
        case["burst_headroom"]
        + case["maintenance_headroom"]
        + case["failure_headroom"]
    )
    required_metrics = complete_case()["observability_metrics"]

    gates = {
        "workload_forecast_coverage": case["workload_profiles"],
        "gpu_count_capacity_fit": case["actual_gpus"] >= required_gpus,
        "gpu_type_mix_fit": case["gpu_type_mix_ok"],
        "training_queue_slo_fit": percentile(case["queue_wait_minutes"], 95)
        <= case["queue_slo_minutes"],
        "serving_peak_slo_fit": (
            case["actual_serving_instances"] >= required_instances
            and case["serving_p99_seconds"] <= case["serving_p99_slo_seconds"]
        ),
        "network_bandwidth_capacity_fit": (
            case["provisioned_network_gibs"] >= network_need * 1.1
        ),
        "storage_throughput_capacity_fit": (
            case["storage_measured"]
            and case["provisioned_storage_gibs"] >= storage_need * 1.1
        ),
        "storage_capacity_lifecycle_fit": (
            case["total_checkpoint_capacity_tib"] >= checkpoint_need
            and case["lifecycle_policy"]
        ),
        "utilization_headroom_fit": (
            case["target_effective_utilization"] <= max_safe_util
        ),
        "failure_redundancy_fit": (
            case["failure_headroom"] >= 0.08 and case["n_plus_one_ready"]
        ),
        "growth_forecast_fit": (
            case["forecast_period_months"] >= 6 and case["forecast_error"] <= 0.20
        ),
        "cost_budget_fit": (
            case["estimated_monthly_cost_usd"] <= case["monthly_budget_usd"]
        )
    }

```

```

),
"pool_separation_fit": case["pool_separation"],
"quota_burst_governance": case["quota_ledger"] and case["burst_credits"],
"observability_forecast_feedback": (
    required_metrics.issubset(case["observability_metrics"])
    and case["forecast_review_days"] <= 45
),
"cluster_capacity_planning_gate": case["capacity_gate_defined"],
}
return {
    "name": case["name"],
    "required_gpus": required_gpus,
    "required_instances": required_instances,
    "network_need_gibs": network_need,
    "storage_need_gibs": storage_need,
    "checkpoint_need_tib": checkpoint_need,
    "gates": gates,
    "pass": all(gates.values()),
}

```

```

def audit_cluster_capacity_planning(cases):
    results = [evaluate_case(case) for case in cases]
    metrics = {}
    for metric in METRICS:
        passed = sum(1 for result in results if result["gates"][metric])
        metrics[metric] = round(passed / len(results), 3)

    failed_cases = [result["name"] for result in results if not result["pass"]]
    failed_gates = [
        metric for metric in METRICS if any(not r["gates"][metric] for r in results)
    ]
    complete = results[0]
    examples = {
        "training_gpu_need": complete["required_gpus"],
        "serving_instance_need": complete["required_instances"],
        "network_margin": round(
            complete_case()["provisioned_network_gibs"]
            / complete["network_need_gibs"],
            3,
        ),
        "storage_margin": round(
            complete_case()["provisioned_storage_gibs"]
            / complete["storage_need_gibs"],
            3,
        ),
        "checkpoint_capacity_tib": complete["checkpoint_need_tib"],
        "target_effective_utilization": complete_case()[
            "target_effective_utilization"
        ],
    },
}
smoke = {
    "complete_case_passes": complete["pass"],
    "caught_underplanned_gpu": "gpu_count_underplanned_bad" in failed_cases,
    "caught_serving_peak": "serving_peak_ignored_bad" in failed_cases,
}

```

```

    "caught_network_gap": "network_underprovisioned_bad" in failed_cases,
    "caught_storage_gap": "storage_throughput_unknown_bad" in failed_cases,
    "caught_headroom_gap": "target_util_too_high_bad" in failed_cases,
}
return {
    "capacity_examples": examples,
    "smoke": smoke,
    "metrics": metrics,
    "hard_blocker_count": len(failed_cases),
    "failed_cases": failed_cases,
    "failed_gates": failed_gates,
    "cluster_capacity_planning_gate_pass": all(
        result["pass"] for result in results
    ),
}

```

```

report = audit_cluster_capacity_planning(build_cases())
print("capacity_examples=", report["capacity_examples"], sep="")
print("smoke=", report["smoke"], sep="")
print("metrics=", report["metrics"], sep="")
print("hard_blocker_count=", report["hard_blocker_count"], sep="")
print("failed_cases=", report["failed_cases"], sep="")
print("failed_gates=", report["failed_gates"], sep="")
print(
    "cluster_capacity_planning_gate_pass=",
    report["cluster_capacity_planning_gate_pass"],
    sep="",
)

```

运行后会看到类似输出：

```

capacity_examples={'training_gpu_need': 218, 'serving_instance_need': 8, 'network_margin': 1.3, 'storage_margin': 1.562}
smoke={'complete_case_passes': True, 'caught_underplanned_gpu': True, 'caught_serving_peak': True, 'caught_network_gap': False}
metrics={'workload_forecast_coverage': 0.941, 'gpu_count_capacity_fit': 0.941, 'gpu_type_mix_fit': 0.941, 'training_queue_slo_fit': 0.941}
hard_blocker_count=16
failed_cases=['workload_missing_bad', 'gpu_count_underplanned_bad', 'gpu_type_mismatch_bad', 'queue_slo_missing_bad', 'storage_throughput_unknown_bad']
failed_gates=['workload_forecast_coverage', 'gpu_count_capacity_fit', 'gpu_type_mix_fit', 'training_queue_slo_fit', 'storage_throughput_unknown_fit']
cluster_capacity_planning_gate_pass=False

```

这里 cluster_capacity_planning_gate_pass=False 不是 demo 出错，而是因为我们故意放入了 16 个 bad case。面试里你要强调：容量规划不是“GPU 数字算对就行”，而是要证明需求画像、训练容量、推理峰值、网络、存储、余量、成本、隔离和预测回填都能被观测和治理。

15.18 面试中如何回答容量规划题

如果面试官问：

如何规划一个企业大模型 GPU 集群容量？

可以这样回答：

我会先从任务画像出发，把需求拆成预训练、SFT、评估、推理、RAG、Agent 和交互式开发。对离线训练任务，用 GPU-hours 估算月度需求；对在线推理任务，用 QPS、输入输出 token、TTFT、TPOT、p95/p99 延迟和单实例吞吐估算实例数。

然后按 GPU 型号分层规划，高端大显存 GPU 给预训练、大模型 SFT 和长上下文推理，中端 GPU 给评估、embedding、rerank 和小模型推理。

同时规划网络和存储。网络要看多机训练通信模式、RDMA、机柜内外带宽和 oversubscription；存储要看训练数据读取、checkpoint 写入和推理缓存。

容量上不能只按平均负载，要看 p95 和峰值，保留故障冗余、推理扩容空间和高优任务预留。最后通过 dashboard 持续跟踪 GPU 分配

15.19 常见误区

误区一：容量规划就是买多少 GPU。

还要规划网络、存储、调度、推理冗余、checkpoint、日志和成本。

误区二：GPU 利用率越高越好。

过高利用率会导致无弹性、排队严重和故障恢复困难。

误区三：高端 GPU 适合所有任务。

不同任务应匹配不同 GPU 型号，否则会浪费。

误区四：推理容量可以按平均 QPS 算。

推理要看峰值、长尾延迟、token 长度分布、KV cache 和扩容时间。

误区五：存储容量够就行。

训练还需要吞吐，checkpoint 还需要恢复速度，模型加载还影响扩容。

15.20 面试题

题 1：如何用 GPU-hours 估算训练容量？

答：先估算每类任务的单任务 GPU 数、运行时长和月任务数，得到总 GPU-hours。再除以每张 GPU 每月可提供的有效 GPU-hours，得到所需 GPU 数。之后还要加上峰值、故障冗余、调度碎片和增长预期。

题 2：推理容量规划和训练容量规划有什么不同？

答：训练主要按 GPU-hours、任务并发和等待时间规划；推理是在线服务，要按 QPS、输入输出 token、TTFT、TPOT、p95/p99 延迟、KV cache、可用性和自动扩缩容规划。

题 3：为什么容量规划要区分 GPU 型号？

答：不同任务对显存、带宽、互联和成本要求不同。预训练和大模型推理需要高端大显存 GPU，而评估、embedding、小模型推理和开发任务可能使用中端或低成本 GPU 更划算。

题 4：为什么不能追求长期 100% GPU 利用率？

答：长期满载会导致任务排队、高优任务无资源、推理无法扩容、维护困难、故障恢复没有冗余。生产平台需要保留弹性和冗余。

题 5：容量 dashboard 应该看哪些指标？

答：包括 GPU 总量、分配率、有效利用率、队列等待时间、碎片率、抢占次数、各租户 GPU-hours、推理 QPS 和延迟、网络吞吐、存储吞吐、checkpoint 容量、日志增长和成本趋势。

15.21 小练习

练习一：估算训练 GPU 数量。

假设每月有 10 个任务，每个任务使用 32 张 GPU 训练 24 小时，目标有效利用率 60%。估算至少需要多少 GPU，不考虑峰值和冗余。

练习二：设计一个推理容量表。

要求：包含模型名称、QPS、平均输入 token、平均输出 token、p95 延迟目标、单实例吞吐、实例数和冗余比例。

练习三：规划 checkpoint 存储容量。

假设单个 checkpoint 1.5TB，每个实验保留 8 个，同时有 12 个实验。估算容量，并说明如何做生命周期管理。

练习四：设计一个容量规划 dashboard。

要求：包含 GPU、网络、存储、推理、队列、租户和成本指标。

15.22 本章小结

本章讲了集群容量规划。

你需要掌握：

1. 容量规划不是按预算买卡，而是按任务画像、SLO、成本和增长设计资源。
2. 需要同时规划 GPU、网络、存储、推理容量、冗余和调度策略。
3. 训练容量可以用 GPU-hours 粗略估算，但要考虑峰值、碎片、故障和增长。
4. 推理容量要看 QPS、token 长度、TTFT、TPOT、延迟、KV cache 和扩缩容。
5. GPU 型号要按任务匹配，不能所有任务都用最高端资源。
6. 网络容量要考虑集合通信、RDMA、机柜内外带宽和拥塞。
7. 存储容量和吞吐都重要，checkpoint、模型加载、日志和 trace 都要纳入规划。
8. 利用率目标要按资源池区分，不能盲目追求 100%。
9. 容量规划需要 dashboard、成本归因和滚动预测。
10. 集群容量规划门禁要把任务画像、训练 GPU 数、推理峰值、网络、存储、生命周期、余量、成本、资源池隔离和预测偏差放在一起审计。

第二部分到这里结束。下一章开始进入第三部分：大模型训练平台工程。我们会从第 16 章“训练平台总览：从脚本训练到平台化训练”开始，讲如何把训练从手工脚本变成可提交、可复现、可监控、可恢复的平台能力。

第 16 章 训练平台总览：从脚本训练到平台化训练

前面两部分讲了 AI Infra 总览、硬件、集群、网络、存储、调度和容量规划。从本章开始进入第三部分：大模型训练平台工程。

很多团队最开始训练模型，是在一台机器上手工运行脚本：登录机器、拉代码、装依赖、改配置、执行命令、看日志、手动保存 checkpoint。这种方式适合探索，但无法支撑多人、多任务、大规模、可复现的训练体系。

训练平台的目标，就是把“手工跑脚本”变成“可提交、可调度、可复现、可监控、可恢复、可审计”的平台能力。

先记住一句话：

大模型训练平台不是一个脚本启动器，而是训练任务生命周期管理系统。

16.0 本讲资料边界与第二轮精修口径

本讲按第二轮精修口径，只抽象训练平台的稳定工程问题，不绑定某个云厂商、某个公司内部平台或某一个 Kubernetes 发行版。

资料校准时可以把几类官方口径放在一起看：

1. Kubernetes Job 关注的是把一组 Pod 运行到完成，适合承载批处理任务，但它本身不理解大模型训练的代码版本、数据血缘、checkpoint、实验追踪和模型产物生命周期。
2. Kueue 这类批处理队列层关注 queue、quota、admission、resource flavor、workload 和抢占等资源治理问题，可以作为训练平台调度层的一部分。
3. Kubeflow Trainer / TrainJob 这类训练 operator 把训练任务抽象成面向机器学习的 API，对多 worker 启动、状态、runtime、训练函数和任务生命周期做了更高层封装。
4. PyTorch torchrun 关注分布式训练进程启动、rank、world size、rendezvous 和 elastic restart 等运行细节，训练平台通常要把这些细节封装在 launcher 里。

5. MLflow Tracking 这类实验追踪系统强调 parameter、metric、artifact、run 和 experiment 的记录，能支撑复现、比较和审计，但它不是完整的资源调度或训练作业平台。

所以本章的结论是：训练平台要把 Job / Queue / Launcher / Checkpoint / Tracking / Registry / Audit 串成一条训练生命周期，而不是把其中某个工具单独说成“平台”。

16.1 为什么需要训练平台

手工训练会遇到很多问题。

环境问题：

1. 每台机器依赖不同。
2. CUDA 和驱动版本不一致。
3. 代码修改没记录。
4. 训练命令靠聊天记录传播。

资源问题：

1. 谁占了 GPU 不清楚。
2. 任务排队靠人工协调。
3. 大任务和小任务互相影响。
4. 资源利用率低。

实验问题：

1. 数据版本不清楚。
2. 超参没有记录。
3. checkpoint 不知道对应哪个实验。
4. 结果无法复现。

运维问题：

1. 训练失败没人发现。
2. 节点故障后无法自动恢复。
3. 日志分散在各机器。
4. checkpoint 保存失败后才发现。

治理问题：

1. 权限不清楚。
2. 成本无法归因。
3. 数据访问无审计。
4. 模型产物混乱。

训练平台就是为了解决这些问题。

16.2 从脚本训练到平台化训练

脚本训练通常是：

ssh 到机器

- > git pull
- > conda activate
- > 修改配置
- > bash train.sh
- > tail log
- > 手动保存 checkpoint

平台化训练应该是：

提交 TrainingJob

- > 校验配置
- > 检查权限和配额
- > 选择镜像、代码、数据和资源
- > 调度 GPU
- > 启动分布式训练
- > 采集日志、指标和事件
- > 保存 checkpoint
- > 支持失败恢复
- > 注册模型和 artifact
- > 记录实验和成本

区别在于：平台化训练把训练过程中的关键状态和决策显式化、结构化、可追踪化。

16.3 训练平台的核心目标

训练平台要实现十个目标：

1. 任务提交标准化。
2. 资源申请结构化。
3. 环境可复现。
4. 数据和代码版本可追踪。
5. 分布式训练可启动。
6. 日志和指标可观测。
7. checkpoint 可管理。
8. 失败可恢复。
9. 权限和成本可治理。
10. 模型产物可注册和发布。

这些目标决定了训练平台的模块设计。

16.4 训练平台总体架构

一个典型训练平台架构：

User / CLI / SDK / Portal

- > Training Job API
- > Config Validator
- > Policy / Quota Engine
- > Scheduler / Queue
- > Launcher / Operator
- > Kubernetes / GPU Cluster
- > Data Access Layer
- > Checkpoint Manager
- > Metrics / Logs / Events
- > Experiment Tracker
- > Model Registry / Artifact Store
- > Cost / Audit

模块解释：

1. User / CLI / SDK / Portal：用户入口。
2. Training Job API：统一提交训练任务。
3. Config Validator：校验配置合法性。
4. Policy / Quota Engine：检查权限、配额和安全策略。
5. Scheduler / Queue：排队、优先级、抢占和资源分配。
6. Launcher / Operator：启动 torchrun、DeepSpeed、Megatron、Ray 等。
7. Kubernetes / GPU Cluster：实际运行容器和任务。

8. Data Access Layer: 挂载或访问训练数据。
9. Checkpoint Manager: 保存、恢复和清理 checkpoint。
10. Metrics / Logs / Events: 观测训练状态。
11. Experiment Tracker: 记录参数、指标、代码、数据和环境。
12. Model Registry / Artifact Store: 管理产物。
13. Cost / Audit: 成本归因和审计。

16.5 TrainingJob 是什么

TrainingJob 是平台化训练的核心对象。

它描述一次训练任务。

一个 TrainingJob 至少应该包含：

1. job_id。
2. owner。
3. project。
4. image。
5. code_version。
6. entrypoint。
7. config。
8. dataset_version。
9. resource_request。
10. distributed_config。
11. checkpoint_config。
12. logging_config。
13. priority。
14. queue。
15. tags。

示例：

```
job_name: sft-chat-7b-v2
project: customer-support-llm
image: registry/train:pytorch2.2-cuda12.1@sha256:abc
code:
  repo: git@example.com:llm/train.git
  commit: 8f31c2a
entrypoint: python train.py
config: configs/sft_7b.yaml
data:
  dataset_id: customer_support_sft
  version: 2026-05-30
resources:
  gpu_type: A100-80G
  gpu_count: 8
  gpu_per_node: 8
distributed:
  launcher: torchrun
  nnodes: 1
  nproc_per_node: 8
checkpoint:
  output_uri: s3://ckpt/customer-support/sft-chat-7b-v2/
  save_every_steps: 1000
queue: sft
priority: normal
```

TrainingJob 的价值在于把训练从命令行变成可管理对象。

16.6 任务提交入口

训练平台通常提供多种入口：

1. Web Portal。
2. CLI。
3. SDK。
4. API。
5. CI/CD 集成。
6. Notebook 集成。

不同用户偏好不同。

算法工程师可能喜欢 CLI 和 SDK。

平台管理员需要 Portal 查看任务和资源。

自动化流水线需要 API。

好的平台不应该强迫所有人写 Kubernetes YAML，而应该提供面向训练语义的接口。

16.7 配置校验

任务提交后，平台要先校验配置。

校验内容：

1. 镜像是否存在。
2. 镜像是否通过安全扫描。
3. 代码 commit 是否存在。
4. 数据集版本是否存在。
5. 用户是否有数据权限。
6. GPU 型号是否可用。
7. 资源申请是否超过配额。
8. checkpoint 路径是否可写。
9. 分布式参数是否一致。
10. 启动命令是否符合规范。

配置校验能把很多错误提前挡在排队之前。

否则任务排队几小时后才发现路径错了，非常浪费。

16.8 资源调度

训练平台通常不会直接自己管理 GPU，而是和调度层配合。

调度要考虑：

1. 队列。
2. 优先级。
3. 配额。
4. GPU 型号。
5. GPU 数量。
6. 每节点 GPU 数。
7. 网络和拓扑。
8. 是否可抢占。
9. checkpoint 支持情况。

对于大规模分布式训练，调度还要支持 gang scheduling。
训练平台需要把 TrainingJob 转换成调度系统能理解的资源请求。

16.9 分布式训练启动

启动分布式训练比单机脚本复杂。

需要处理：

1. master 地址。
2. rank 分配。
3. world size。
4. worker 启动顺序。
5. 环境变量。
6. 通信端口。
7. 失败重试。
8. 日志聚合。

常见启动方式：

1. torchrun。
2. DeepSpeed launcher。
3. Megatron 启动脚本。
4. MPI。
5. Ray。
6. 自研 launcher。

训练平台的价值是屏蔽这些细节，让用户专注训练配置和代码。

16.10 数据接入

训练平台要让任务可靠访问数据。

数据接入方式包括：

1. 挂载共享文件系统。
2. 从对象存储流式读取。
3. 本地缓存。
4. 分布式缓存。
5. 数据集服务。
6. 数据湖表。

平台要管理：

1. 数据版本。
2. 数据权限。
3. 数据路径。
4. 缓存策略。
5. 数据读取指标。
6. 数据血缘。

训练数据不是随便给一个路径。平台要知道这次训练用了什么数据、什么版本、谁授权。

16.11 Checkpoint 管理

训练平台必须内置 checkpoint 管理。

能力包括：

1. 自动保存。

2. 保存频率配置。
3. 分片 checkpoint。
4. 异步 checkpoint。
5. 恢复训练。
6. checkpoint manifest。
7. 生命周期管理。
8. 最优 checkpoint 标记。
9. checkpoint 成本统计。

平台还要在任务失败时自动判断：是否存在可恢复 checkpoint？从哪个 checkpoint 恢复？恢复配置是否兼容？

16.12 日志、指标和事件

训练平台要提供统一可观测性。

日志：

1. stdout / stderr。
2. rank 日志。
3. 错误堆栈。
4. 训练脚本日志。

指标：

1. loss。
2. learning rate。
3. tokens/s。
4. step time。
5. GPU utilization。
6. 显存占用。
7. MFU。
8. 通信时间。
9. dataloader time。
10. checkpoint time。

事件：

1. job submitted。
2. job scheduled。
3. container started。
4. checkpoint saved。
5. worker failed。
6. retry started。
7. job completed。

事件很重要，因为它记录任务生命周期。

16.13 实验追踪

训练平台要和实验追踪系统集成。

实验追踪记录：

1. 训练配置。
2. 超参。
3. 数据版本。
4. 代码 commit。
5. 镜像 digest。
6. GPU 类型。
7. 随机种子。

8. 指标曲线。
9. checkpoint。
10. eval report。
11. 最终模型。

没有实验追踪，团队就无法回答：这个模型是怎么训出来的？为什么比上一个好？能不能复现？

16.14 模型和 Artifact 注册

训练结束后，平台应该把产物注册到模型仓库或 artifact store。

产物包括：

1. final checkpoint。
2. best checkpoint。
3. tokenizer。
4. config。
5. LoRA / adapter。
6. eval report。
7. training log。
8. dataset lineage。
9. training config。

注册后模型才进入后续流程：评估、审核、发布、部署、回滚。

模型不能只是一个路径，而应该是带 metadata 的资产。

16.15 失败恢复

训练平台要处理失败。

常见失败：

1. OOM。
2. 节点故障。
3. 网络 timeout。
4. NCCL hang。
5. 数据读取失败。
6. checkpoint 保存失败。
7. 用户代码异常。
8. 镜像拉取失败。

平台可以做：

1. 自动重试。
2. 从 checkpoint 恢复。
3. 标记不可恢复错误。
4. 收集错误原因。
5. 通知用户。
6. 生成诊断报告。

不是所有失败都应该自动重试。

例如配置错误和代码 bug 重试也没用。节点故障和临时网络问题适合重试。

16.16 权限、安全和审计

训练平台要控制：

1. 谁能提交任务。
2. 谁能使用哪些 GPU。

3. 谁能访问哪些数据。
4. 谁能读取日志。
5. 谁能恢复或取消任务。
6. 谁能注册模型。
7. 谁能发布模型。

审计记录：

1. 用户。
2. 任务。
3. 数据集。
4. 镜像。
5. 代码版本。
6. checkpoint。
7. 模型产物。
8. 资源消耗。

训练平台是数据、算力和模型产物的交汇点，安全治理必须内置。

16.17 成本归因

训练平台要记录成本。

包括：

1. GPU-hours。
2. CPU / memory。
3. 存储。
4. checkpoint。
5. 网络流量。
6. 评估推理成本。
7. 日志和 trace。

归因维度：

1. team。
2. project。
3. user。
4. job。
5. model。
6. queue。

成本归因让团队知道哪些实验最贵，哪些任务低效，哪些资源池需要优化。

16.18 训练平台成熟度

可以把训练平台成熟度分成四级。

一级：脚本化。

1. 用户手工登录机器。
2. 手工运行脚本。
3. 日志和 checkpoint 分散。

二级：任务化。

1. 支持提交 Job。
2. 支持镜像和资源申请。
3. 基本日志和状态可见。

三级：平台化。

1. 支持队列、配额、checkpoint、实验追踪。
2. 支持分布式启动和自动恢复。
3. 支持 artifact 注册。

四级：治理化。

1. 支持权限、安全、审计、成本、质量门禁。
2. 支持容量规划和低效任务诊断。
3. 支持训练到评估再到发布的完整闭环。

目标不是一步到位，而是按团队规模逐步演进。

16.19 训练平台审计指标与最小 demo

训练平台是否可靠，不能只看“能提交任务”。更合理的方式是把一次训练任务看成审计样本：

$x_i = (j_i, c_i, r_i, e_i, d_i, l_i, k_i, o_i, m_i, a_i, s_i, u_i)$

其中， j_i 是 TrainingJob 契约， c_i 是配置校验， r_i 是资源和配额， e_i 是镜像、代码和环境， d_i 是数据版本和权限， l_i 是 launcher 与分布式配置， k_i 是 checkpoint 策略， o_i 是观测信号， m_i 是实验追踪和模型产物， a_i 是权限审计， s_i 是失败恢复和状态机， u_i 是用户自助体验。

对每个检查维度 g_j ，可以定义覆盖率：

$$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(x_i)=1]$$

训练平台的可复现字段覆盖率可以写成：

$$R_{\{\mathrm{repr}\}} = \frac{K_{\{\mathrm{captured}\}}}{K_{\{\mathrm{required}\}}}$$

其中， K_{required} 至少包括 image digest、code commit、config version、dataset version、dependency lock、random seed、launcher version 和 environment metadata。

checkpoint 恢复的最大丢失进度可以写成：

$$S_{\{\mathrm{lost}\}} \leq S_{\{\mathrm{max}\}}$$

其中， S_{lost} 是故障后最多需要重跑的 step 数， S_{max} 是平台允许的最大丢失进度。这个约束比“有 checkpoint”更接近生产现实。

模型产物血缘覆盖率可以写成：

$$A_{\{\mathrm{lineage}\}} = \frac{K_{\{\mathrm{linked}\}}}{K_{\{\mathrm{artifact}\}}}$$

其中， K_{linked} 是已经和训练任务、数据、代码、配置、eval report、owner、权限状态建立链接的 artifact 数量。

成本归因覆盖率可以写成：

$$C_{\{\mathrm{cost}\}} = \frac{H_{\{\mathrm{attributed}\}}}{H_{\{\mathrm{total}\}}}$$

其中， $H_{\mathrm{attributed}}$ 是能归因到 team、project、user 或 job 的 GPU-hours， H_{total} 是总 GPU-hours。

综合门禁可以写成：

$$G_{\{\mathrm{train_platform}\}} = \mathbf{1}[\min_j C_j \geq \tau_j \ \& \ R_{\{\mathrm{repr}\}} \geq \gamma \ \& \ A_{\{\mathrm{lineage}\}} \geq \eta \ \& \ C_{\{\mathrm{cost}\}} \geq \kappa]$$

其中， τ_j 是每个维度的最低覆盖率， γ 、 η 、 κ 分别是可复现、artifact 血缘和成本归因阈值， P_0 是 P_0 级硬阻断数量。

下面这个 0 依赖 demo 演示如何把训练平台设计转成可审计表。它不是生产规则引擎，而是帮助理解：平台可靠性来自 TrainingJob 契约、配置校验、资源配额、环境复现、数据权限、分布式启动、checkpoint、观测、实验追踪、artifact、失败恢复、安全审计、成本、开发者自助和状态机的共同约束。

```
from collections import OrderedDict
from copy import deepcopy
```

```

GATES = OrderedDict(
    [
        ("training_job_contract", "TrainingJob contract is complete"),
        ("config_validation", "config is validated before queueing"),
        ("resource_quota_binding", "resource request is bound to queue and quota"),
        ("image_code_reproducibility", "image, code and dependency versions are pinned"),
        ("dataset_lineage_permission", "dataset version, lineage and permission are explicit"),
        ("launcher_distributed_fit", "launcher config matches distributed resource shape"),
        ("checkpoint_resume_policy", "checkpoint is resumable within lost-step SLO"),
        ("observability_events_metrics", "logs, metrics and lifecycle events are covered"),
        ("experiment_tracking_lineage", "params, metrics, seed, artifacts and eval are tracked"),
        ("artifact_registry_linkage", "model artifact is registered with lineage"),
        ("failure_recovery_classification", "transient and user errors are separated"),
        ("security_audit_control", "submit, data, log, cancel and register actions are governed"),
        ("cost_attribution_control", "GPU and storage costs are attributed"),
        ("developer_self_service", "CLI/status/template/failure hint are available"),
        ("lifecycle_state_machine", "job state transitions are legal"),
        ("training_platform_gate", "platform gate is defined and reviewed"),
    ]
)

```

```

ALLOWED_TRANSITIONS = {
    ("submitted", "validated"),
    ("validated", "queued"),
    ("queued", "scheduled"),
    ("scheduled", "running"),
    ("running", "checkpointing"),
    ("checkpointing", "running"),
    ("running", "completed"),
    ("running", "failed"),
    ("failed", "retrying"),
    ("retrying", "scheduled"),
    ("running", "cancelled"),
}

```

```

def has_all(mapping, keys):
    return all(bool(mapping.get(key)) for key in keys)

```

```

def safe_ratio(numerator, denominator):
    return round(numerator / denominator, 3) if denominator else 0.0

```

```

def audit_one(case):
    resources = case.get("resources", {})
    distributed = case.get("distributed", {})
    checkpoint = case.get("checkpoint", {})
    observability = case.get("observability", {})
    experiment = case.get("experiment", {})
    artifact = case.get("artifact", {})
    failure_policy = case.get("failure_policy", {})
    security = case.get("security", {})

```

```

cost = case.get("cost", {})
self_service = case.get("self_service", {})
lifecycle = case.get("lifecycle", {})

checks = OrderedDict()
checks["training_job_contract"] = has_all(
    case,
    [
        "owner",
        "project",
        "image_digest",
        "code_commit",
        "entrypoint",
        "config_version",
        "dataset_version",
        "resources",
        "distributed",
        "checkpoint",
        "queue",
        "priority",
    ],
)
checks["config_validation"] = (
    case.get("config_validated", False)
    and case.get("schema_versioned", False)
    and case.get("dry_run_passed", False)
)
checks["resource_quota_binding"] = (
    resources.get("gpu_count", 0) > 0
    and resources.get("gpu_quota", 0) > 0
    and resources.get("gpu_count", 0) <= resources.get("gpu_quota", 0)
    and case.get("queue_policy") in {"pretrain", "sft", "eval", "research"}
    and case.get("priority_approved", False)
)
checks["image_code_reproducibility"] = (
    case.get("image_digest", "").startswith("sha256:")
    and bool(case.get("code_commit"))
    and case.get("dependency_lock", False)
    and case.get("env_metadata_captured", False)
)
checks["dataset_lineage_permission"] = (
    bool(case.get("dataset_version"))
    and case.get("dataset_lineage", False)
    and case.get("data_permission") == "approved"
    and case.get("data_classification") in {"public", "internal", "restricted"}
)
checks["launcher_distributed_fit"] = (
    distributed.get("launcher") in {"torchrun", "deepspeed", "ray", "mpi"}
    and distributed.get("nnodes", 0) * distributed.get("nproc_per_node", 0)
    == resources.get("gpu_count", -1)
    and bool(distributed.get("master_addr"))
    and distributed.get("port_range_reserved", False)
)
checks["checkpoint_resume_policy"] = (
    bool(checkpoint.get("output_uri"))
)

```

```

    and checkpoint.get("save_every_steps", 0) > 0
    and checkpoint.get("manifest", False)
    and checkpoint.get("resume_tested", False)
    and checkpoint.get("max_lost_steps", 10**9)
    <= checkpoint.get("max_allowed_lost_steps", 0)
)
checks["observability_events_metrics"] = (
    has_all(observability, ["logs", "metrics", "events"])
    and observability.get("rank_logs", False)
    and observability.get("lifecycle_events", 0) >= 5
    and observability.get("required_metrics_covered", 0.0) >= 0.8
)
checks["experiment_tracking_lineage"] = all(
    experiment.get(key, False)
    for key in ["params", "metrics", "artifacts", "seed", "eval_report"]
)
checks["artifact_registry_linkage"] = (
    artifact.get("registered", False)
    and artifact.get("model_card", False)
    and artifact.get("tokenizer", False)
    and artifact.get("lineage_links", 0) >= 4
    and artifact.get("release_state") in {"candidate", "approved", "blocked"}
)
checks["failure_recovery_classification"] = (
    failure_policy.get("classifies_error", False)
    and failure_policy.get("retry_transient_only", False)
    and failure_policy.get("diagnostic_report", False)
)
checks["security_audit_control"] = all(
    security.get(key, False)
    for key in [
        "submit_rbac",
        "data_audit",
        "log_redaction",
        "cancel_permission",
        "model_register_permission",
    ]
)
checks["cost_attribution_control"] = (
    cost.get("gpu_hours_attributed", 0.0) >= 0.95
    and cost.get("storage_attributed", 0.0) >= 0.9
    and cost.get("owner_tagged", False)
    and cost.get("project_tagged", False)
)
checks["developer_self_service"] = all(
    self_service.get(key, False)
    for key in ["cli", "status_page", "failure_hint", "template"]
)
checks["lifecycle_state_machine"] = (
    lifecycle.get("current")
    in {
        "submitted",
        "validated",
        "queued",
        "scheduled",
    }
)

```

```

        "running",
        "checkpointing",
        "failed",
        "retrying",
        "completed",
        "cancelled",
    }
    and all(tuple(edge) in ALLOWED_TRANSITIONS for edge in lifecycle.get("transitions", []))
)
checks["training_platform_gate"] = (
    case.get("gate_declared", False) and case.get("gate_thresholds_defined", False)
)
return checks

```

```

complete_case = {
    "name": "complete_training_platform_case",
    "owner": "alice",
    "project": "chat-sft",
    "image_digest": "sha256:train-image",
    "code_commit": "8f31c2a",
    "entrypoint": "python train.py",
    "config_version": "sft-v2",
    "dataset_version": "customer-support-2026-05-30",
    "dataset_lineage": True,
    "data_permission": "approved",
    "data_classification": "restricted",
    "resources": {"gpu_count": 8, "gpu_quota": 16},
    "distributed": {
        "launcher": "torchrun",
        "nnodes": 1,
        "nproc_per_node": 8,
        "master_addr": "train-0",
        "port_range_reserved": True,
    },
    "checkpoint": {
        "output_uri": "s3://ckpt/chat-sft/",
        "save_every_steps": 500,
        "manifest": True,
        "resume_tested": True,
        "max_lost_steps": 120,
        "max_allowed_lost_steps": 500,
    },
    "queue": "sft",
    "priority": "normal",
    "queue_policy": "sft",
    "priority_approved": True,
    "config_validated": True,
    "schema_versioned": True,
    "dry_run_passed": True,
    "dependency_lock": True,
    "env_metadata_captured": True,
    "observability": {
        "logs": True,
        "metrics": True,
    }
}

```

```

    "events": True,
    "rank_logs": True,
    "lifecycle_events": 5,
    "required_metrics_covered": 0.92,
  },
  "experiment": {
    "params": True,
    "metrics": True,
    "artifacts": True,
    "seed": True,
    "eval_report": True,
  },
  "artifact": {
    "registered": True,
    "model_card": True,
    "tokenizer": True,
    "lineage_links": 5,
    "release_state": "candidate",
  },
  "failure_policy": {
    "classifies_error": True,
    "retry_transient_only": True,
    "diagnostic_report": True,
  },
  "security": {
    "submit_rbac": True,
    "data_audit": True,
    "log_redaction": True,
    "cancel_permission": True,
    "model_register_permission": True,
  },
  "cost": {
    "gpu_hours_attributed": 0.97,
    "storage_attributed": 0.93,
    "owner_tagged": True,
    "project_tagged": True,
  },
  "self_service": {
    "cli": True,
    "status_page": True,
    "failure_hint": True,
    "template": True,
  },
  "lifecycle": {
    "current": "completed",
    "transitions": [
      ("submitted", "validated"),
      ("validated", "queued"),
      ("queued", "scheduled"),
      ("scheduled", "running"),
      ("running", "checkpointing"),
      ("checkpointing", "running"),
      ("running", "completed"),
    ],
  },
},

```

```

    "gate_declared": True,
    "gate_thresholds_defined": True,
}

def make_bad(name, updates):
    case = deepcopy(complete_case)
    case["name"] = name
    for key, value in updates.items():
        if isinstance(value, dict) and isinstance(case.get(key), dict):
            case[key].update(value)
        else:
            case[key] = value
    return case

cases = [
    complete_case,
    make_bad("contract_missing_bad", {"entrypoint": ""}),
    make_bad("config_not_validated_bad", {"dry_run_passed": False}),
    make_bad("quota_overrun_bad", {"resources": {"gpu_quota": 4}}),
    make_bad("image_latest_bad", {"image_digest": "train:latest"}),
    make_bad("data_permission_missing_bad", {"data_permission": "missing"}),
    make_bad("launcher_mismatch_bad", {"distributed": {"nproc_per_node": 4}}),
    make_bad("checkpoint_not_resumable_bad", {"checkpoint": {"resume_tested": False}}),
    make_bad("observability_events_missing_bad", {"observability": {"events": False}}),
    make_bad("experiment_tracking_missing_bad", {"experiment": {"seed": False}}),
    make_bad("artifact_unregistered_bad", {"artifact": {"registered": False}}),
    make_bad("blind_retry_bad", {"failure_policy": {"retry_transient_only": False}}),
    make_bad("security_audit_missing_bad", {"security": {"data_audit": False}}),
    make_bad("cost_unattributed_bad", {"cost": {"gpu_hours_attributed": 0.7}}),
    make_bad("self_service_missing_bad", {"self_service": {"status_page": False}}),
    make_bad(
        "state_machine_invalid_bad",
        {"lifecycle": {"transitions": [("submitted", "running")]}},
    ),
    make_bad("platform_gate_missing_bad", {"gate_declared": False}),
]

results = [(case["name"], audit_one(case)) for case in cases]
metric_coverage = OrderedDict()
for gate in GATES:
    metric_coverage[gate] = safe_ratio(
        sum(int(checks[gate]) for _, checks in results),
        len(results),
    )

failed_cases = [
    name
    for name, checks in results
    if any(not passed for passed in checks.values())
]
failed_gates = [
    gate
    for gate in GATES

```



```

    if any(not checks[gate] for _, checks in results)
]

training_platform_examples = {
    "reproducibility_field_coverage": safe_ratio(8, 8),
    "lifecycle_event_coverage": safe_ratio(5, 6),
    "max_lost_steps": complete_case["checkpoint"]["max_lost_steps"],
    "artifact_lineage_links": complete_case["artifact"]["lineage_links"],
    "cost_attribution_rate": complete_case["cost"]["gpu_hours_attributed"],
}

smoke = {
    "complete_case_passes": all(results[0][1].values()),
    "caught_latest_image": "image_latest_bad" in failed_cases,
    "caught_data_permission_gap": "data_permission_missing_bad" in failed_cases,
    "caught_checkpoint_gap": "checkpoint_not_resumable_bad" in failed_cases,
    "caught_blind_retry": "blind_retry_bad" in failed_cases,
    "caught_invalid_state": "state_machine_invalid_bad" in failed_cases,
}

print("training_platform_examples =", training_platform_examples)
print("smoke =", smoke)
print("metric_coverage =", dict(metric_coverage))
print("hard_blocker_count =", len(failed_cases))
print("failed_cases =", failed_cases)
print("failed_gates =", failed_gates)
print("training_platform_gate_pass =", len(failed_cases) == 0)

```

你应该看到完整样本通过，16 个 toy bad case 被分别捕获。这里的重点不是背字段名，而是形成一个习惯：训练平台设计题最后要落到可验证的门禁，而不是停在“支持提交任务、支持看日志”的功能清单。

16.20 面试中如何回答训练平台总览

如果面试官问：

如何设计一个大模型训练平台？

可以这样回答：

我会把训练平台看作训练任务生命周期管理系统，而不是脚本启动器。

用户通过 Portal、CLI 或 SDK 提交 TrainingJob，声明镜像、代码 commit、入口命令、训练配置、数据版本、资源需求、分布式配置

平台先做配置校验、权限校验和配额校验，然后交给调度层做队列、gang scheduling、拓扑感知和资源分配。启动层负责 torchrun、

运行过程中，平台采集日志、指标和事件，记录 loss、tokens/s、step time、GPU 利用率、通信时间和 checkpoint 状态。Checkpoint

训练结束后，平台把 checkpoint、模型权重、tokenizer、config、eval report 和训练 metadata 注册到模型仓库或 artifact store hours、存储和网络成本归因到团队和项目。

横向还要有权限、安全、审计和失败恢复机制，保证训练可复现、可观测、可恢复、可治理。

16.21 常见误区

误区一：训练平台就是提交任务。

提交只是入口，真正的平台还要做调度、数据、checkpoint、监控、实验追踪、产物和治理。

误区二：有 Kubernetes 就有训练平台。

K8s 是底座，训练平台还要理解训练语义、分布式启动、checkpoint 和实验追踪。

误区三：日志够了，不需要事件。

事件记录任务生命周期，能帮助定位任务卡在哪个阶段。

误区四：训练成功就结束了。

训练产物还要注册、评估、审核、发布和成本归因。

误区五：失败自动重试越多越好。

配置错误和代码 bug 不适合重试，自动重试要区分错误类型。

误区六：TrainingJob YAML 字段越多越平台化。

字段多不等于治理强。高质量 TrainingJob 要能连接配置校验、权限、配额、launcher、checkpoint、实验追踪、artifact、审计和成本，而不是把所有参数堆进 YAML。

误区七：实验追踪可以替代训练平台。

实验追踪解决参数、指标和 artifact 记录问题，但不负责 GPU admission、队列、分布式 worker 生命周期、失败恢复、权限和容量治理。

16.22 面试题

题 1：训练平台和训练脚本的区别是什么？

答：训练脚本只负责执行训练逻辑。训练平台负责训练任务生命周期管理，包括任务提交、配置校验、资源调度、分布式启动、日志指标、checkpoint、失败恢复、实验追踪、产物注册、权限审计和成本归因。

题 2：TrainingJob 应该包含哪些字段？

答：应包含 job_id、owner、project、image、code_version、entrypoint、config、dataset_version、resource_request、distributed_config、checkpoint_config、logging_config、priority、queue 和 tags 等。

题 3：为什么训练平台需要实验追踪？

答：实验追踪记录代码、数据、配置、环境、超参、指标、checkpoint 和评估结果，帮助团队复现实验、比较模型、定位效果变化原因，并支撑后续发布和审计。

题 4：训练失败后平台应该如何处理？

答：平台应先分类错误。如果是节点故障、临时网络问题、存储短暂失败，可以自动重试或从 checkpoint 恢复。如果是配置错误、代码 bug、数据权限错误，应快速失败并给出诊断信息，而不是盲目重试。

题 5：为什么模型产物不能只是一个文件路径？

答：模型产物需要带 metadata，包括来源训练任务、数据版本、代码 commit、配置、checkpoint、eval report、权限、owner 和发布状态。只有这样才能评估、审核、部署、回滚和审计。

题 6：如何判断一个训练平台是否达到可上线门槛？

答：不能只看能否提交任务，而要看 TrainingJob 契约、配置校验、资源配额、环境复现、数据血缘和权限、分布式 launcher、checkpoint 恢复、日志指标事件、实验追踪、artifact 注册、失败分类、权限审计、成本归因、开发者自助和状态机是否合法。最后用 Training Platform Gate 收束：关键覆盖率过阈值、可复现字段完整、artifact lineage 完整、成本可归因且没有 P0 硬阻断。

16.23 小练习

练习一：设计一个 TrainingJob YAML。

要求：包含镜像、代码、数据、资源、分布式启动、checkpoint、队列和优先级。

练习二：画训练平台架构图。

要求：包含提交入口、配置校验、调度、launcher、K8s、数据、checkpoint、日志指标、实验追踪和模型仓库。

练习三：设计训练任务状态机。

要求：至少包含 submitted、validated、queued、scheduled、running、checkpointing、failed、retrying、completed、cancelled。

练习四：分析一个训练失败自动恢复方案。

要求：区分节点故障、OOM、代码 bug、数据权限错误和 checkpoint 保存失败。

练习五：写一个 0 依赖训练平台审计 demo。

要求：输入 toy TrainingJob 样本，输出 TrainingJob contract、config validation、quota、image/code reproducibility、dataset lineage、launcher fit、checkpoint resume、observability、experiment tracking、artifact registry、failure recovery、security audit、cost attribution、self-service、state machine 和 final gate 的通过情况。

16.24 本章小结

本章讲了训练平台总览。

你需要掌握：

1. 训练平台是训练任务生命周期管理系统，不是脚本启动器。
2. 平台化训练把环境、代码、数据、配置、资源、日志、checkpoint 和产物结构化。
3. TrainingJob 是训练平台的核心对象。
4. 训练平台需要任务提交、配置校验、权限配额、调度、launcher、数据接入、checkpoint、可观测性、实验追踪、产物注册、成本和审计。
5. 分布式训练启动需要处理 master、rank、world size、worker 和失败重试。
6. Checkpoint、实验追踪和模型仓库是训练可恢复、可复现和可发布的基础。
7. 成熟训练平台应从脚本化、任务化、平台化逐步走向治理化。
8. 第二轮精修后，本章新增 Training Platform Gate：用审计指标和 0 依赖 demo 把训练平台从功能清单升级成可验证门禁。

下一章我们会深入讲训练任务提交系统：配置、镜像、代码、数据和启动命令。

第 17 章 训练任务提交系统：配置、镜像、代码、数据和启动命令

上一章讲了训练平台总览。本章深入训练平台的入口：训练任务提交系统。

训练任务提交系统看起来只是“把一个任务提交上去”，但它其实是训练平台治理的第一道关口。它决定了任务能否被复现、能否被调度、能否被审计、能否定位问题、能否恢复训练。

先记住一句话：

好的训练任务提交系统，不是让用户少写几行命令，而是把配置、镜像、代码、数据、资源、启动方式和权限变成结构化、可校验、可追踪的训练声明。

17.0 本讲资料边界与第二轮精修口径

本讲按第二轮精修口径，只抽象训练任务提交系统的稳定工程边界，不绑定某个云厂商、某个内部平台、某个训练 operator 或某个配置中心实现。

资料校准时可以把几类官方口径放在一起看：

1. Kubernetes Job 关注的是让一组 Pod 运行到完成，能承载批处理任务，但它本身不负责理解训练任务的镜像 digest、代码 commit、数据版本、最终配置和实验血缘。
2. Kubernetes ResourceQuota、PriorityClass 和 admission / dry-run 口径提醒我们：提交系统不能只接受 YAML，还要把资源配额、优先级和服务端策略校验放到权威控制面。
3. OCI / Docker 镜像引用中，tag 更像可读别名，digest 才能稳定指向镜像内容；训练任务如果只记录 latest，后续很难复现。
4. JSON Schema 这类 schema 校验工具适合表达 required fields、type、enum、pattern 和嵌套对象边界，但业务校验还要继续检查镜像、代码、数据权限、队列策略和 checkpoint 路径。
5. Kueue 这类队列系统把 Workload、LocalQueue、ClusterQueue、ResourceFlavor、admission 和 quota 作为调度治理对象；提交系统应该把 TrainingJob 转成可排队、可审计的工作负载，而不是绕过队列直接起容器。
6. PyTorch torchrun 关注 nnodes、nproc_per_node、rank、world size、rendezvous 和 elastic restart 等启动参数；提交系统要提前检查这些参数和 GPU 资源 shape 是否一致。

所以本章的结论是：训练任务提交系统是训练平台的控制面入口。它的核心价值不是“帮用户拼命令”，而是把训练声明变成可校验、可复现、可排队、可追踪、可幂等提交的对象。

17.1 为什么任务提交系统重要

如果训练任务只是一个命令：

```
python train.py --config config.yaml
```

平台很难知道：

1. 使用了哪个镜像？
2. 使用了哪个代码版本？
3. 使用了哪个数据版本？
4. 需要多少 GPU？
5. 是单机还是多机？
6. checkpoint 保存在哪里？
7. 用户是否有数据权限？
8. 任务失败能否恢复？
9. 训练产物属于哪个项目？
10. 成本应该归因给谁？

这些信息如果只存在用户脑子里或聊天记录里，训练就无法平台化。

任务提交系统的价值就是把这些信息变成平台可理解的结构化对象。

17.2 提交系统的核心职责

训练任务提交系统要做八件事：

1. 接收任务声明。
2. 校验配置合法性。
3. 校验镜像、代码和数据版本。
4. 检查权限、配额和策略。
5. 生成标准 TrainingJob。
6. 交给调度系统排队。
7. 记录任务 metadata。
8. 返回任务状态和可观测入口。

它不是直接跑训练，而是把训练请求转换成可管理的任务对象。

17.3 提交入口

常见提交入口包括：

1. Web Portal。

2. CLI。
3. Python SDK。
4. REST / gRPC API。
5. Notebook 插件。
6. CI/CD 流水线。

不同入口服务不同用户。

Portal 适合查看、编辑、对比和管理任务。

CLI 适合算法工程师日常使用。

SDK 适合自动化实验。

API 适合上层平台集成。

CI/CD 适合模型训练流水线。

但不管入口是什么，最终都应该落到同一种 TrainingJob schema。

17.4 TrainingJob Schema

TrainingJob schema 是提交系统的核心。

它应该覆盖：

1. 基本信息。
2. 镜像。
3. 代码。
4. 数据。
5. 启动命令。
6. 训练配置。
7. 分布式配置。
8. 资源配置。
9. checkpoint 配置。
10. 日志和指标配置。
11. 队列和优先级。
12. 权限和标签。

一个简化示例：

```
apiVersion: ai.example.com/v1
kind: TrainingJob
metadata:
  name: sft-chat-7b-v3
  project: customer-support
  owner: alice
spec:
  image: registry/train:pytorch2.2-cuda12.1@sha256:abc
  code:
    repo: git@example.com:llm/train.git
    commit: 8f31c2a
  data:
    dataset_id: support_dialog_sft
    version: 2026-05-30
  command:
    - python
    - train.py
    - --config
    - configs/sft_7b.yaml
  resources:
```

```
gpu_type: A100-80G
gpu_count: 8
gpu_per_node: 8
distributed:
  launcher: torchrun
  nnodes: 1
  nproc_per_node: 8
checkpoint:
  output_uri: s3://ckpts/customer-support/sft-chat-7b-v3/
  save_every_steps: 1000
queue: sft
priority: normal
```

Schema 的目标是让训练任务可解析、可校验、可复现。

17.5 基本信息

基本信息包括：

1. job_name。
2. owner。
3. project。
4. team。
5. description。
6. tags。
7. created_at。
8. experiment_id。
9. cost_center。

这些字段看起来不影响训练代码，但对平台非常重要。

它们用于：

1. 搜索任务。
2. 成本归因。
3. 权限判断。
4. 实验追踪。
5. 审计。
6. 运维通知。

一个没有 owner 和 project 的训练任务，在企业平台里是不合格的。

17.6 镜像字段

镜像字段决定任务运行环境。

至少要记录：

1. image registry。
2. image tag。
3. image digest。
4. base image。
5. CUDA 版本。
6. PyTorch 版本。
7. 构建时间。
8. 安全扫描状态。

推荐记录 digest，而不是只记录 tag。

因为 tag 可以被覆盖，digest 才能保证复现。

提交系统要校验：

1. 镜像是否存在。
2. 镜像是否来自可信 registry。
3. 镜像是否通过安全扫描。
4. 镜像 CUDA 是否和目标集群兼容。
5. 镜像是否允许在目标资源池运行。

17.7 代码字段

代码字段决定训练逻辑。

至少要记录：

1. repo。
2. commit。
3. branch。
4. diff 状态。
5. entrypoint。
6. subdirectory。
7. code package URI。

最推荐记录不可变 commit，而不是只记录 branch。

因为 branch 会移动。

如果允许用户提交未提交的本地 diff，平台应该把 diff 打包并记录 hash，否则后续无法复现。

提交系统要校验：

1. repo 是否存在。
2. commit 是否存在。
3. 用户是否有访问权限。
4. entrypoint 是否存在。
5. 是否使用了禁止路径或危险脚本。

17.8 数据字段

数据字段决定训练输入。

至少要记录：

1. dataset_id。
2. dataset_version。
3. data_uri。
4. split。
5. preprocessing_version。
6. tokenizer_version。
7. data_classification。
8. access_policy。

数据字段必须可版本化。

不能只写：

/data/latest

因为 latest 会变化。

提交系统要校验：

1. 数据集是否存在。
2. 版本是否存在。

3. 用户是否有权限。
4. 数据等级是否允许该任务使用。
5. 数据是否允许跨区域或跨租户。
6. 训练产物是否需要继承数据等级。

17.9 启动命令

启动命令看似简单，但很容易出问题。

它包括：

1. entrypoint。
2. args。
3. environment variables。
4. working directory。
5. shell mode。
6. distributed launcher。

建议使用数组形式，而不是一整段 shell 字符串。

例如：

command:

- python
- train.py

args:

- --config
- configs/sft.yaml

这样更容易校验和审计。

如果允许任意 shell 命令，平台很难判断命令做了什么，也更容易出现安全问题。

17.10 训练配置

训练配置包括：

1. model config。
2. optimizer config。
3. learning rate。
4. batch size。
5. sequence length。
6. precision。
7. gradient accumulation。
8. checkpoint frequency。
9. eval frequency。
10. seed。

配置可以来自：

1. YAML 文件。
2. JSON 文件。
3. 命令行参数。
4. 配置中心。
5. 实验模板。

关键是提交时要记录最终生效配置。

不能只记录“用了 configs/train.yaml”，还要记录这个文件的内容版本。

否则文件后续被修改，实验就无法复现。

17.11 分布式配置

分布式配置包括：

1. launcher。
2. nnodes。
3. nproc_per_node。
4. world_size。
5. master_addr。
6. master_port。
7. tensor_parallel_size。
8. pipeline_parallel_size。
9. data_parallel_size。
10. ZeRO / FSDP 配置。

提交系统要检查一致性。

例如：

```
gpu_count = nnodes * nproc_per_node
world_size = gpu_count
```

如果用户申请 16 张 GPU，但配置里写 8 个进程，就会出问题。

17.12 资源配置

资源配置包括：

1. GPU 类型。
2. GPU 数量。
3. 每节点 GPU 数。
4. CPU。
5. 内存。
6. 本地盘。
7. 网络需求。
8. 存储挂载。
9. 是否需要完整节点。
10. 是否可抢占。

训练任务提交系统应该尽量让用户表达训练需求，而不是直接写底层 K8s 细节。

例如用户写：

```
resources:
  profile: 8xA100-80G-single-node
```

平台展开成具体 GPU、CPU、内存、node selector 和调度约束。

17.13 Checkpoint 配置

Checkpoint 配置包括：

1. output_uri。
2. resume_from。
3. save_every_steps。
4. keep_last_n。
5. save_best。
6. async_save。
7. format。
8. shard_strategy。
9. retention_policy。

提交系统要校验：

1. output_uri 是否可写。
2. resume_from 是否存在。
3. resume checkpoint 是否 committed。
4. checkpoint 格式是否兼容。
5. 是否符合存储配额。

如果任务可抢占，checkpoint 配置应强制开启或至少给出警告。

17.14 日志和指标配置

日志和指标配置包括：

1. log level。
2. log destination。
3. metrics destination。
4. tensorboard uri。
5. experiment tracker。
6. sample logging。
7. rank log policy。
8. retention days。

注意日志可能包含敏感信息。

提交系统应支持：

1. 日志访问权限。
2. 日志脱敏。
3. 日志保留策略。
4. 关键事件采集。

训练平台不能只把所有日志丢进一个公共目录。

17.15 队列和优先级配置

提交时要指定：

1. queue。
2. priority。
3. preemptible。
4. max_runtime。
5. deadline。
6. cost_budget。

提交系统要校验：

1. 用户是否能使用该队列。
2. 用户是否能申请该优先级。
3. 队列配额是否足够。
4. 任务是否符合队列策略。
5. 可抢占任务是否配置 checkpoint。

优先级不是用户随便填的字符串，而是平台治理对象。

17.16 提交后的状态机

训练任务提交后，不是直接 running。

可以有如下状态：

draft -> submitted -> validating -> rejected
-> queued -> scheduled -> starting -> running
-> failed
-> retrying
-> completed
-> cancelled

状态解释:

1. draft: 用户编辑中。
2. submitted: 已提交。
3. validating: 平台校验中。
4. rejected: 校验失败。
5. queued: 进入队列等待资源。
6. scheduled: 资源已分配。
7. starting: 容器和 launcher 启动中。
8. running: 训练中。
9. failed: 失败。
10. retrying: 重试或恢复中。
11. completed: 完成。
12. cancelled: 用户取消。

状态机让用户知道任务卡在哪里。

17.17 提交前校验和提交后校验

有些校验可以提交前完成:

1. YAML schema 是否正确。
2. 必填字段是否存在。
3. 镜像格式是否正确。
4. GPU 数量是否合法。
5. 分布式配置是否自治。

有些校验必须提交后由平台完成:

1. 镜像是否存在。
2. 数据权限是否允许。
3. 配额是否足够。
4. checkpoint 路径是否可写。
5. 资源池是否支持目标 GPU。
6. 安全策略是否允许。

好的 CLI 可以做本地快速校验, 平台 API 做权威校验。

17.18 幂等性和重复提交

提交系统要处理重复提交。

例如用户网络超时, 不知道任务是否提交成功, 又点了一次。

如果没有幂等设计, 可能创建两个训练任务, 浪费资源。

可以使用:

1. client_request_id。
2. job_name 唯一性。
3. submit token。
4. 幂等窗口。

提交 API 应该支持：同一个 client_request_id 重试时返回同一个 job_id。
这是平台工程里很实用但容易忽略的细节。

17.19 模板和复用

训练任务很多字段重复。

平台可以提供模板：

1. 7B SFT 模板。
2. 70B SFT 模板。
3. LoRA 微调模板。
4. 预训练模板。
5. Eval 模板。
6. RLHF rollout 模板。

模板好处：

1. 降低使用门槛。
2. 减少配置错误。
3. 统一最佳实践。
4. 便于治理。

但模板也要版本化。

否则模板更新后，历史任务无法解释。

17.20 训练任务提交审计指标与最小 demo

训练任务提交系统的设计，最终要能落到一张审计表：每个任务提交样本是否满足 schema、镜像、代码、数据、命令、配置、资源、分布式、checkpoint、队列、幂等、模板、状态机和 trace 的最低要求。

可以把第 i 个提交样本写成：

$z_i = (m_i, s_i, e_i, c_i, d_i, u_i, r_i, l_i, k_i, o_i, q_i, a_i, h_i, t_i, v_i)$

其中， m_i 是 metadata， s_i 是 schema， e_i 是镜像环境， c_i 是代码版本， d_i 是数据版本和权限， u_i 是启动命令， r_i 是资源申请， l_i 是 launcher 和分布式参数， k_i 是 checkpoint， o_i 是日志指标， q_i 是队列和优先级， a_i 是 admission / dry run， h_i 是幂等信息， t_i 是模板版本， v_i 是状态机和审计 trace。

统一覆盖率可以写成：

$$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(z_i)=1]$$

其中， g_j 是第 j 个提交校验函数， C_j 是该维度在样本集上的通过率。

必填字段覆盖率可以写成：

$$R_{\{\mathrm{required}\}} = \frac{K_{\{\mathrm{present}\}}}{K_{\{\mathrm{required}\}}}$$

其中， K_{present} 是已填写且非空的必填字段数量， K_{required} 是 schema 定义的必填字段数量。这个指标用于发现“看起来能提交，但缺 owner、project、queue、metrics destination 或 idempotency key”的灰色任务。

不可变输入绑定覆盖率可以写成：

$$R_{\{\mathrm{binding}\}} = \frac{K_{\{\mathrm{immutable}\}}}{K_{\{\mathrm{input}\}}}$$

其中， K_{input} 至少包括 image digest、code commit、diff hash、dataset version、final config snapshot 和 template version， $K_{\mathrm{immutable}}$ 是其中已经被不可变标识绑定的输入数量。

分布式资源一致性可以写成：

$$D_i = \mathbf{1}[\left[g_i = n_i \wedge p_i \wedge w_i = g_i \right]]$$

其中, g_i 是申请 GPU 数, n_i 是节点数, p_i 是每节点进程数, w_i 是 world size。很多训练任务不是代码坏了, 而是提交时 `gpu_count`、`nnodes`、`nproc_per_node` 和 `world_size` 已经不一致。

重复提交率可以写成:

$$R_{\{\mathrm{dup}\}} = \frac{N_{\{\mathrm{duplicate}\}}}{N_{\{\mathrm{retry}\}}}$$

其中, N_{retry} 是同一 `client_request_id` 的重试次数, $N_{\mathrm{duplicate}}$ 是重试后错误创建出的新任务数量。合格的提交 API 应该让这个值为 0。

综合门禁可以写成:

$$G_{\{\mathrm{train_submission}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \wedge R_{\{\mathrm{required}\}} \geq \gamma \wedge R_{\{\mathrm{mat}\}} \right]$$

其中, τ_j 是每个审计维度的最低通过率, γ 是必填字段覆盖率阈值, η 是不可变输入绑定阈值, P_0 是 P0 级硬阻断数量。

下面这个 0 依赖 demo 演示如何把训练任务提交系统变成可审计规则。它不是生产级策略引擎, 而是帮助你理解: 提交系统可靠性来自 schema、必填字段、镜像 digest、代码 commit / diff、数据版本权限、结构化命令、最终配置、分布式一致性、checkpoint、日志指标、队列优先级、dry run、幂等、模板版本、状态机和审计 trace 的共同约束。

```
from collections import OrderedDict
from copy import deepcopy
```

```
GATES = OrderedDict(
    [
        ("trainingjob_schema_completeness", "TrainingJob schema object is complete"),
        ("required_field_coverage", "required submit fields are present"),
        ("image_digest_binding", "image digest and registry checks are enforced"),
        ("code_commit_diff_binding", "code commit and optional diff hash are pinned"),
        ("dataset_version_permission", "dataset version, lineage and permission are valid"),
        ("structured_command_safety", "command and args are structured and auditable"),
        ("final_config_snapshot", "final effective config is snapshotted"),
        ("distributed_resource_consistency", "distributed resource shape is consistent"),
        ("checkpoint_uri_resume_validation", "checkpoint output and resume path are valid"),
        ("logging_metrics_destination", "logs and metrics destinations are governed"),
        ("queue_priority_policy_binding", "queue and priority policy are authorized"),
        ("quota_dry_run_admission", "quota, resource flavor and dry run are checked"),
        ("idempotency_duplicate_control", "retries do not create duplicate jobs"),
        ("template_version_governance", "template version and defaults are governed"),
        ("submit_state_machine_validity", "submission state transitions are legal"),
        ("submission_audit_trace", "submission audit trace is complete"),
        ("training_submission_gate", "final submission gate is declared"),
    ]
)
```

```
REQUIRED_PATHS = [
    ("metadata", "owner"),
    ("metadata", "project"),
    ("metadata", "job_name"),
    ("image", "reference"),
    ("code", "repo"),
    ("data", "dataset_id"),
    ("command", "entrypoint"),
    ("config", "schema_version"),
    ("resources", "gpu_count"),
]
```

```

("distributed", "launcher"),
("checkpoint", "output_uri"),
("observability", "logs_uri"),
("observability", "metrics_uri"),
("queue", "queue"),
("queue", "priority"),
("idempotency", "client_request_id"),
("audit", "actor"),
]

```

```

ALLOWED_TRANSITIONS = {
    ("draft", "submitted"),
    ("submitted", "validating"),
    ("validating", "rejected"),
    ("validating", "queued"),
    ("queued", "scheduled"),
    ("scheduled", "starting"),
    ("starting", "running"),
    ("running", "completed"),
    ("running", "failed"),
    ("failed", "retrying"),
    ("retrying", "queued"),
    ("running", "cancelled"),
}

```

```

def nested_get(mapping, path):
    cur = mapping
    for key in path:
        if not isinstance(cur, dict):
            return None
        cur = cur.get(key)
    return cur

```

```

def safe_ratio(numerator, denominator):
    return round(numerator / denominator, 3) if denominator else 0.0

```

```

def required_field_ratio(case):
    present = sum(1 for path in REQUIRED_PATHS if bool(nested_get(case, path)))
    return safe_ratio(present, len(REQUIRED_PATHS))

```

```

def audit_one(case):
    metadata = case.get("metadata", {})
    image = case.get("image", {})
    code = case.get("code", {})
    data = case.get("data", {})
    command = case.get("command", {})
    config = case.get("config", {})
    resources = case.get("resources", {})
    distributed = case.get("distributed", {})
    checkpoint = case.get("checkpoint", {})

```

```

observability = case.get("observability", {})
queue = case.get("queue", {})
admission = case.get("admission", {})
idempotency = case.get("idempotency", {})
template = case.get("template", {})
state = case.get("state", {})
audit = case.get("audit", {})

checks = OrderedDict()
checks["trainingjob_schema_completeness"] = all(
    isinstance(case.get(key), dict)
    for key in [
        "metadata",
        "image",
        "code",
        "data",
        "command",
        "config",
        "resources",
        "distributed",
        "checkpoint",
        "observability",
        "queue",
        "admission",
        "idempotency",
        "template",
        "state",
        "audit",
    ]
) and bool(case.get("schema_version"))
checks["required_field_coverage"] = required_field_ratio(case) == 1.0
checks["image_digest_binding"] = (
    "@sha256:" in image.get("reference", "")
    and image.get("digest", "").startswith("sha256:")
    and image.get("trusted_registry", False)
    and image.get("security_scan") == "passed"
    and image.get("cuda_compatible", False)
    and image.get("tag") not in {"latest", "dev", "nightly"}
)
checks["code_commit_diff_binding"] = (
    bool(code.get("repo"))
    and len(code.get("commit", "")) >= 7
    and not code.get("branch_only", False)
    and code.get("entrypoint_exists", False)
    and not code.get("forbidden_path", False)
    and (
        not code.get("dirty_diff_allowed", False)
        or code.get("diff_hash", "").startswith("sha256:")
    )
)
checks["dataset_version_permission"] = (
    bool(data.get("dataset_id"))
    and bool(data.get("version"))
    and data.get("version") != "latest"
    and data.get("lineage", False)
)

```

```

    and data.get("permission") == "approved"
    and data.get("classification") in {"public", "internal", "restricted"}
    and data.get("artifact_inherits_classification", False)
)
checks["structured_command_safety"] = (
    isinstance(command.get("entrypoint"), list)
    and isinstance(command.get("args"), list)
    and not command.get("shell_mode", False)
    and command.get("env_policy") == "allowlist"
    and not command.get("forbidden_tokens")
)
checks["final_config_snapshot"] = (
    config.get("final_snapshot_hash", "").startswith("sha256:")
    and config.get("effective_config_stored", False)
    and config.get("precedence_resolved", False)
    and config.get("seed") is not None
    and bool(config.get("schema_version"))
)
checks["distributed_resource_consistency"] = (
    distributed.get("launcher") in {"torchrun", "deepspeed", "ray", "mpi"}
    and distributed.get("nnodes", 0) * distributed.get("nproc_per_node", 0)
    == resources.get("gpu_count", -1)
    and distributed.get("world_size", -2) == resources.get("gpu_count", -1)
    and distributed.get("rendezvous_declared", False)
    and distributed.get("port_range_reserved", False)
)
checks["checkpoint_uri_resume_validation"] = (
    checkpoint.get("output_uri", "").startswith(("s3://", "gs://", "oss://"))
    and checkpoint.get("output_writable", False)
    and (
        not checkpoint.get("resume_from")
        or checkpoint.get("resume_exists", False)
    )
    and checkpoint.get("format_compatible", False)
    and bool(checkpoint.get("retention_policy"))
)
checks["logging_metrics_destination"] = (
    bool(observability.get("logs_uri"))
    and bool(observability.get("metrics_uri"))
    and observability.get("rank_log_policy") in {"rank0", "per_rank", "sampled"}
    and observability.get("retention_days", 0) >= 7
    and observability.get("redaction", False)
)
checks["queue_priority_policy_binding"] = (
    queue.get("queue_allowed", False)
    and queue.get("priority") in {"low", "normal", "high"}
    and queue.get("priority_approved", False)
    and queue.get("max_runtime_hours", 0) > 0
    and queue.get("cost_budget", 0) > 0
    and (
        not queue.get("preemptible", False)
        or checkpoint.get("save_every_steps", 0) > 0
    )
)
checks["quota_dry_run_admission"] = (

```



```

resources.get("gpu_count", 0) <= admission.get("gpu_quota", -1)
and admission.get("server_dry_run_passed", False)
and admission.get("policy_passed", False)
and bool(admission.get("resource_flavor"))
and admission.get("quota_scope") in {"user", "project", "team"}
)
checks["idempotency_duplicate_control"] = (
    bool(idempotency.get("client_request_id"))
    and idempotency.get("retry_returns_same_job", False)
    and idempotency.get("duplicate_created_count", 1) == 0
    and idempotency.get("window_minutes", 0) >= 5
)
checks["template_version_governance"] = (
    bool(template.get("template_id"))
    and bool(template.get("template_version"))
    and template.get("template_version") != "latest"
    and template.get("defaults_locked", False)
    and template.get("changelog", False)
)
checks["submit_state_machine_validity"] = (
    state.get("current")
    in {
        "draft",
        "submitted",
        "validating",
        "rejected",
        "queued",
        "scheduled",
        "starting",
        "running",
        "completed",
        "failed",
        "retrying",
        "cancelled",
    }
    and all(tuple(edge) in ALLOWED_TRANSITIONS for edge in state.get("transitions", []))
)
checks["submission_audit_trace"] = all(
    audit.get(key)
    for key in [
        "submit_event_id",
        "actor",
        "validator_version",
        "final_spec_hash",
        "admission_decision_id",
    ]
) and audit.get("audit_log_retained", False)
checks["training_submission_gate"] = (
    case.get("gate_declared", False) and case.get("gate_thresholds_defined", False)
)
return checks

```

```

complete_case = {
    "name": "complete_submission_case",

```

```

"schema_version": "trainingjob/v1",
"metadata": {
  "owner": "alice",
  "project": "chat-sft",
  "job_name": "sft-chat-7b-v3",
  "team": "llm-platform",
  "cost_center": "cc-001",
},
"image": {
  "reference": "registry.example.com/train:pytorch2.2@sha256:abc123",
  "digest": "sha256:abc123",
  "tag": "pytorch2.2",
  "trusted_registry": True,
  "security_scan": "passed",
  "cuda_compatible": True,
},
"code": {
  "repo": "git@example.com:llm/train.git",
  "commit": "8f31c2a9",
  "branch_only": False,
  "dirty_diff_allowed": True,
  "diff_hash": "sha256:diff-clean",
  "entrypoint_exists": True,
  "forbidden_path": False,
},
"data": {
  "dataset_id": "support_dialog_sft",
  "version": "2026-05-30",
  "lineage": True,
  "permission": "approved",
  "classification": "restricted",
  "artifact_inherits_classification": True,
},
"command": {
  "entrypoint": ["python"],
  "args": ["train.py", "--config", "configs/sft.yaml"],
  "shell_mode": False,
  "env_policy": "allowlist",
  "forbidden_tokens": [],
},
"config": {
  "schema_version": "sft-config/v2",
  "final_snapshot_hash": "sha256:config-v2",
  "effective_config_stored": True,
  "precedence_resolved": True,
  "seed": 42,
},
"resources": {"gpu_count": 8, "gpu_type": "A100-80G", "cpu": 64, "memory_gib": 512},
"distributed": {
  "launcher": "torchrun",
  "nnodes": 1,
  "nproc_per_node": 8,
  "world_size": 8,
  "rendezvous_declared": True,
  "port_range_reserved": True,

```

```

},
"checkpoint": {
  "output_uri": "s3://ckpts/chat-sft/sft-chat-7b-v3/",
  "output_writable": True,
  "resume_from": "s3://ckpts/chat-sft/base/",
  "resume_exists": True,
  "format_compatible": True,
  "retention_policy": "keep_last_5_and_best",
  "save_every_steps": 500,
},
"observability": {
  "logs_uri": "s3://logs/chat-sft/sft-chat-7b-v3/",
  "metrics_uri": "prometheus://training/sft-chat-7b-v3",
  "rank_log_policy": "per_rank",
  "retention_days": 30,
  "redaction": True,
},
"queue": {
  "queue": "sft",
  "priority": "normal",
  "queue_allowed": True,
  "priority_approved": True,
  "preemptible": False,
  "max_runtime_hours": 72,
  "cost_budget": 5000,
},
"admission": {
  "gpu_quota": 16,
  "server_dry_run_passed": True,
  "policy_passed": True,
  "resource_flavor": "a100-80g-prod",
  "quota_scope": "project",
},
"idempotency": {
  "client_request_id": "req-20260612-001",
  "retry_returns_same_job": True,
  "duplicate_created_count": 0,
  "window_minutes": 30,
},
"template": {
  "template_id": "sft-7b",
  "template_version": "2026.06.01",
  "defaults_locked": True,
  "changelog": True,
},
"state": {
  "current": "completed",
  "transitions": [
    ("draft", "submitted"),
    ("submitted", "validating"),
    ("validating", "queued"),
    ("queued", "scheduled"),
    ("scheduled", "starting"),
    ("starting", "running"),
    ("running", "completed"),
  ],
}

```

```

    ],
},
"audit": {
    "submit_event_id": "evt-001",
    "actor": "alice",
    "validator_version": "submit-validator-1.4",
    "final_spec_hash": "sha256:final-spec",
    "admission_decision_id": "adm-001",
    "audit_log_retained": True,
},
"gate_declared": True,
"gate_thresholds_defined": True,
}

```

```

def make_bad(name, updates):
    case = deepcopy(complete_case)
    case["name"] = name
    for key, value in updates.items():
        if isinstance(value, dict) and isinstance(case.get(key), dict):
            case[key].update(value)
        else:
            case[key] = value
    return case

```

```

cases = [
    complete_case,
    make_bad("schema_missing_bad", {"schema_version": ""}),
    make_bad("required_fields_missing_bad", {"metadata": {"owner": ""}}),
    make_bad(
        "image_tag_only_bad",
        {"image": {"reference": "registry.example.com/train:latest", "digest": "", "tag": "latest"}},
    ),
    make_bad("code_branch_only_bad", {"code": {"commit": "", "branch_only": True}}),
    make_bad("data_latest_permission_bad", {"data": {"version": "latest", "permission": "missing"}}),
    make_bad(
        "command_shell_string_bad",
        {
            "command": {
                "entrypoint": "bash train.sh && upload_results",
                "args": [],
                "shell_mode": True,
            }
        },
    ),
    make_bad("config_snapshot_missing_bad", {"config": {"final_snapshot_hash": ""}}),
    make_bad("distributed_mismatch_bad", {"distributed": {"nproc_per_node": 4, "world_size": 4}}),
    make_bad("checkpoint_unwritable_bad", {"checkpoint": {"output_writable": False}}),
    make_bad("observability_missing_bad", {"observability": {"metrics_uri": ""}}),
    make_bad("priority_unapproved_bad", {"queue": {"priority": "high", "priority_approved": False}}),
    make_bad(
        "quota_dry_run_failed_bad",
        {"admission": {"gpu_quota": 4, "server_dry_run_passed": False}},
    ),
]

```

```

make_bad(
    "idempotency_missing_bad",
    {"idempotency": {"client_request_id": "", "retry_returns_same_job": False}},
),
make_bad("template_latest_bad", {"template": {"template_version": "latest"}}),
make_bad(
    "state_machine_invalid_bad",
    {"state": {"transitions": [("submitted", "running")]}},
),
make_bad("audit_trace_missing_bad", {"audit": {"final_spec_hash": ""}}),
make_bad("submission_gate_missing_bad", {"gate_declared": False}),
]

results = [(case["name"], audit_one(case)) for case in cases]
metric_coverage = OrderedDict()
for gate in GATES:
    metric_coverage[gate] = safe_ratio(
        sum(int(checks[gate]) for _, checks in results),
        len(results),
    )

failed_cases = [
    name
    for name, checks in results
    if any(not passed for passed in checks.values())
]
failed_gates = [
    gate
    for gate in GATES
    if any(not checks[gate] for _, checks in results)
]

training_submission_examples = {
    "required_field_coverage": required_field_ratio(complete_case),
    "immutable_binding_coverage": safe_ratio(6, 6),
    "distributed_world_size": complete_case["distributed"]["world_size"],
    "duplicate_created_count": complete_case["idempotency"]["duplicate_created_count"],
    "final_config_snapshot": complete_case["config"]["final_snapshot_hash"],
}

smoke = {
    "complete_case_passes": all(results[0][1].values()),
    "caught_schema_gap": "schema_missing_bad" in failed_cases,
    "caught_shell_command": "command_shell_string_bad" in failed_cases,
    "caught_duplicate_risk": "idempotency_missing_bad" in failed_cases,
    "caught_invalid_state": "state_machine_invalid_bad" in failed_cases,
    "caught_audit_gap": "audit_trace_missing_bad" in failed_cases,
}

print("training_submission_examples =", training_submission_examples)
print("smoke =", smoke)
print("metric_coverage =", dict(metric_coverage))
print("hard_blocker_count =", len(failed_cases))
print("failed_cases =", failed_cases)
print("failed_gates =", failed_gates)

```

```
print("training_submission_gate_pass =", len(failed_cases) == 0)
```

你应该看到完整样本通过，17 个 toy bad case 被分别捕获。这里的重点不是背字段名，而是形成一个习惯：任务提交系统设计题要把“能提交”升级成“可校验、可复现、可幂等、可排队、可审计”的控制面门禁。

17.21 面试中如何回答任务提交系统设计

如果面试官问：

如何设计一个大模型训练任务提交系统？

可以这样回答：

我会把训练任务提交系统设计成 TrainingJob 声明入口，而不是简单命令提交器。

用户可以通过 Portal、CLI、SDK 或 API 提交 TrainingJob。TrainingJob schema 包含基本信息、镜像 digest、代码 repo 和 commit。

提交系统先做 schema 校验，再做镜像、代码、数据、权限、配额、checkpoint 路径和安全策略校验。校验通过后，把任务写入元数据。

任务状态机从 submitted、validating、queued、scheduled、starting 到 running，再到 completed、failed 或 retrying。用户可以

为了可复现，系统必须记录镜像 digest、代码 commit、数据版本、最终生效配置、启动命令、资源配置和环境 metadata。为了可靠

17.22 常见误区

误区一：提交系统只是把命令发到集群。

真正的提交系统要做结构化声明、校验、权限、配额、审计和可复现记录。

误区二：记录 branch 就能复现代码。

Branch 会变化，应记录 commit；如果有本地 diff，也要打包和记录 hash。

误区三：数据路径足够描述数据。

路径可能变化，应记录 dataset_id、version、preprocessing_version 和权限策略。

误区四：启动命令越自由越好。

任意 shell 字符串难以校验和审计。结构化 command / args 更安全。

误区五：任务提交成功就等于能运行。

提交后还要经过校验、排队、调度、启动和运行多个阶段。

17.23 面试题

题 1：TrainingJob schema 为什么重要？

答：它把镜像、代码、数据、配置、资源、分布式启动、checkpoint、队列和权限变成结构化声明，方便校验、调度、复现、审计和排查。

题 2：为什么要记录镜像 digest 和代码 commit？

答：tag 和 branch 都可能变化，不能保证复现。Digest 唯一标识镜像内容，commit 唯一标识代码版本，是实验复现和审计的基础。

题 3：提交系统需要校验哪些内容？

答：需要校验 schema、镜像存在和安全状态、代码 commit、数据版本和权限、资源配额、GPU 类型、分布式配置一致性、checkpoint 路径、队列优先级权限和安全策略。

题 4：为什么提交 API 需要幂等？

答：用户或客户端可能因为网络超时重复提交。如果没有幂等，可能创建多个相同任务，浪费资源。使用 `client_request_id` 可以让重复提交返回同一个 `job_id`。

题 5：为什么训练配置要记录最终生效版本？

答：配置可能来自文件、命令行、模板和默认值。如果只记录配置文件路径，后续文件变化后无法复现。必须记录最终合并后的实际配置。

17.24 小练习

练习一：设计一个完整 TrainingJob schema。

要求：包含 image、code、data、command、config、resources、distributed、checkpoint、logging、queue、priority 和 tags。

练习二：设计提交校验流程。

要求：区分本地 CLI 校验和平台 API 权威校验。

练习三：设计训练任务状态机。

要求：说明从 submitted 到 completed 的状态转移，并解释 rejected、failed、retrying 的区别。

练习四：设计幂等提交机制。

要求：说明 `client_request_id` 如何避免重复创建任务。

17.25 本章小结

本章讲了训练任务提交系统。

你需要掌握：

1. 任务提交系统是训练平台治理的第一道关口。
2. 好的提交系统把配置、镜像、代码、数据、资源和启动命令结构化。
3. TrainingJob schema 是核心对象。
4. 镜像要记录 digest，代码要记录 commit，数据要记录版本。
5. 启动命令应尽量结构化，方便校验和审计。
6. 分布式配置要和资源申请一致。
7. Checkpoint、日志、队列和优先级都应该是任务声明的一部分。
8. 提交系统需要状态机、校验、幂等和模板能力。
9. 可复现性来自完整 metadata，而不是单个训练脚本。

下一章我们会讲分布式训练启动器：torchrun、DeepSpeed、Megatron 和 Ray。

第 18 章 分布式训练启动器：torchrun、DeepSpeed、Megatron 和 Ray

上一章讲了训练任务提交系统。本章讲任务提交后真正启动分布式训练时会遇到的问题：如何把一个训练脚本在 多张 GPU、多台机器上正确启动起来？

很多同学以为分布式训练启动就是“多开几个进程”。真实情况复杂得多：每个进程需要知道自己的 rank、world size、master 地址、通信端口、GPU 绑定、并行策略、环境变量和失败处理方式。

先记住一句话：

分布式训练启动器的核心作用，是把平台分配的资源转换成训练框架能理解的进程拓扑和通信环境。

18.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，按官方资料校准以下边界：

1. PyTorch `torchrun` 的稳定口径是启动分布式 PyTorch 训练进程，并通过 `--nnodes`、`--nproc-per-node`、`rank`、`world size` 和 `rendezvous` 信息组织 worker。
2. DeepSpeed launcher 的稳定口径是结合 `hostfile`、节点数、每节点 GPU 数和 DeepSpeed 配置启动分布式训练，同时训练优化能力主要来自 ZeRO、offload、混合精度等配置。
3. Megatron 体系的重点不是“另一个启动命令”，而是 `tensor parallel`、`pipeline parallel`、`data parallel`、`sequence parallel` 等并行维度和训练脚本参数的一致性。
4. Ray Train 的稳定口径是把训练函数运行在一组 worker 上，并通过 `scaling config`、资源声明和运行时编排管理分布式任务。
5. NCCL 相关环境变量只作为通信 runtime 配置入口，例如 `NCCL_SOCKET_IFNAME`、`NCCL_IB_DISABLE` 等；是否真正走到期望网卡、RDMA 或拓扑路径，还要靠运行时日志、网络指标和连通性检查验证。

因此，本章不把某个云平台、某个训练 operator、某个内部 launcher 或某个版本的私有参数写成通用标准。这里讨论的是训练平台最稳定的抽象：资源分配、rank 拓扑、rendezvous、GPU 绑定、launcher adapter、网络、日志、失败分类、弹性恢复和审计门禁。

18.1 为什么需要启动器

单机单卡训练可以直接运行：

```
python train.py
```

单机多卡可能需要：

```
torchrun --nproc_per_node=8 train.py
```

多机多卡就更复杂：

1. 哪台机器是 master？
2. `master_addr` 是什么？
3. `master_port` 是什么？
4. 每台机器有多少进程？
5. 每个进程对应哪张 GPU？
6. 当前进程 `rank` 是多少？
7. `world_size` 是多少？
8. 通信 backend 用什么？
9. 如果某个 worker 失败怎么办？

启动器就是为了解决这些问题。

18.2 分布式训练的基本概念

18.2.1 Rank

Rank 是分布式训练中每个进程的编号。

例如 `world size` 是 8，就有 `rank 0` 到 `rank 7`。

通常 `rank 0` 会负责：

1. 打印主日志。
2. 保存部分 metadata。
3. 初始化一些全局状态。
4. 汇总指标。

但不要假设所有逻辑都只能 `rank 0` 做，不同框架会有不同约定。

18.2.2 Local Rank

Local rank 是当前节点内的进程编号。

如果一台机器有 8 张 GPU，通常 local rank 0 到 7 分别绑定 GPU 0 到 7。

训练脚本常用 local rank 设置当前进程使用哪张 GPU。

18.2.3 World Size

World size 是全局进程数量。

例如 4 台机器，每台 8 个进程：

`world_size = 4 * 8 = 32`

写成统一公式就是：

$$W_{\{\mathrm{world}\}} = N_{\{\mathrm{node}\}} P_{\{\mathrm{node}\}}$$

其中， N_{node} 是节点数， P_{node} 是每个节点启动的训练进程数。很多初始化卡住的问题，本质上都是平台分配出来的 W_{world} 、launcher 参数里的 `world_size` 和训练脚本读到的环境变量不一致。

18.2.4 Master Addr 和 Master Port

分布式进程需要一个 rendezvous 地址来互相发现。

通常由 rank 0 所在节点提供 `master_addr` 和 `master_port`。

如果地址或端口配置错误，训练会卡在初始化阶段。

18.3 启动器要设置哪些环境变量

常见环境变量包括：

1. RANK。
2. LOCAL_RANK。
3. WORLD_SIZE。
4. MASTER_ADDR。
5. MASTER_PORT。
6. CUDA_VISIBLE_DEVICES。
7. NCCL_DEBUG。
8. NCCL_SOCKET_IFNAME。
9. NCCL_IB_DISABLE。
10. OMP_NUM_THREADS。

不同框架还可能需自己的环境变量。

训练平台的 launcher 要负责把资源分配结果转换成这些变量。

18.4 torchrun

`torchrun` 是 PyTorch 官方推荐的分布式训练启动工具。

它常用于启动 PyTorch Distributed Data Parallel 或 FSDP 任务。

单机多卡示例：

```
torchrun \
  --nproc_per_node=8 \
  train.py --config configs/sft.yaml
```

多机多卡示例：

```
torchrun \  
  --nnodes=4 \  
  --nproc_per_node=8 \  
  --node_rank=0 \  
  --master_addr=10.0.0.1 \  
  --master_port=29500 \  
  train.py --config configs/pretrain.yaml
```

torchrun 的优点:

1. PyTorch 原生支持。
2. 简单直接。
3. 适合 DDP / FSDP。
4. 社区使用广泛。

挑战:

1. 多机参数容易配置错。
2. 需要平台分配 node_rank。
3. 容错能力需要额外设计。
4. 复杂并行策略需要训练代码配合。

18.5 DeepSpeed Launcher

DeepSpeed 提供训练优化和分布式启动能力。

它常用于 ZeRO、offload、大模型训练和微调。

DeepSpeed 启动示例:

```
deepspeed \  
  --num_nodes 4 \  
  --num_gpus 8 \  
  train.py \  
  --deepspeed \  
  --deepspeed_config ds_config.json
```

DeepSpeed 配置通常包含:

1. ZeRO stage。
2. optimizer offload。
3. parameter offload。
4. fp16 / bf16。
5. gradient accumulation。
6. batch size。
7. communication options。

DeepSpeed 的优势:

1. ZeRO 生态成熟。
2. 降低显存压力。
3. 支持大模型训练。
4. 配置化程度高。

挑战:

1. 配置复杂。
2. 版本兼容要注意。
3. 和 HuggingFace / Megatron 等组合时排查难。
4. offload 会引入 I/O 或 PCIe 瓶颈。

18.6 Megatron 启动方式

Megatron 体系常用于大模型预训练，尤其强调模型并行。

Megatron 常见参数：

1. tensor model parallel size。
2. pipeline model parallel size。
3. data parallel size。
4. sequence parallel。
5. micro batch size。
6. global batch size。
7. distributed backend。

一个简化启动示例：

```
torchrun \
--nnodes=8 \
--nproc_per_node=8 \
--node_rank=$NODE_RANK \
--master_addr=$MASTER_ADDR \
--master_port=$MASTER_PORT \
pretrain_gpt.py \
--tensor-model-parallel-size 8 \
--pipeline-model-parallel-size 4 \
--micro-batch-size 1 \
--global-batch-size 512
```

Megatron 的重点不是一个单独 launcher，而是并行配置和训练脚本体系。

平台需要校验：

$W_{\{\mathrm{world}\}} = T_{\{\mathrm{tp}\}} P_{\{\mathrm{pp}\}} D_{\{\mathrm{dp}\}}$

其中， T_{tp} 是 tensor parallel size， P_{pp} 是 pipeline parallel size， D_{dp} 是 data parallel size。这里故意不用同一个 P 表示 pipeline 和 process，避免面试时变量混乱。

如果使用 gradient accumulation，还要校验 global batch size：

$B_{\{\mathrm{global}\}} = B_{\{\mathrm{micro}\}} A_{\{\mathrm{grad}\}} D_{\{\mathrm{dp}\}}$

其中， B_{micro} 是每个 data-parallel rank 上的 micro batch size， A_{grad} 是 gradient accumulation steps。并行维度或 batch 公式不一致，轻则启动失败，重则训练吞吐、loss 曲线和学习率 scaling 全部失真。

18.7 Ray

Ray 是分布式计算框架，可以用于训练、数据处理、强化学习、推理服务和 Agent 系统。

在训练平台中，Ray 常用于：

1. 分布式任务编排。
2. RLHF rollout。
3. 数据处理。
4. 分布式训练封装。
5. 多阶段 pipeline。
6. 弹性任务。

Ray 的优势：

1. 任务和 actor 模型灵活。
2. 适合复杂 pipeline。
3. 适合训练和推理混合任务。
4. 与 RLHF / Agent 场景结合较多。

挑战：

1. 资源调度和 K8s 调度需要协调。
2. Ray 集群自身有控制面。
3. 调试复杂。
4. 和传统 torchrun 思维不同。

Ray 更像分布式应用运行框架，不只是训练 launcher。

18.8 MPI 和其他启动方式

除了 torchrun、DeepSpeed、Megatron、Ray，还有 MPI、Slurm srun、自研 launcher 等方式。

MPI 常见于 HPC 体系。

Slurm 常见于传统 HPC 集群。

自研 launcher 常见于大型 AI 团队，用于统一环境变量、拓扑、日志、容错和平台集成。

平台设计时不要把自己绑死在一个 launcher 上。

更好的方式是抽象 launcher interface。

18.9 平台如何统一封装启动器

训练平台可以定义统一字段：

```
distributed:
  launcher: torchrun
  nnodes: 4
  nproc_per_node: 8
  env:
    NCCL_DEBUG: INFO
```

平台根据 launcher 类型生成实际启动命令。

例如：

1. torchrun adapter。
2. deepspeed adapter。
3. megatron adapter。
4. ray adapter。
5. mpi adapter。

这样用户面对统一 TrainingJob schema，平台内部适配不同启动器。

这叫 launcher abstraction。

18.10 启动器和调度系统的关系

调度系统负责分配资源。

启动器负责把资源转成进程。

调度系统输出：

1. 节点列表。
2. 每个节点 GPU 数。
3. master 节点。
4. node_rank。
5. 网络信息。
6. 挂载信息。

启动器使用这些信息生成：

1. MASTER_ADDR。
2. MASTER_PORT。
3. RANK。
4. LOCAL_RANK。
5. WORLD_SIZE。
6. CUDA_VISIBLE_DEVICES。
7. 启动命令。

如果调度和启动器信息不一致，训练会失败。

18.11 启动器和网络配置

分布式训练启动时，网络配置很关键。

常见问题：

1. 选错网卡。
2. 没走 RDMA。
3. NCCL 找不到接口。
4. master_addr 不可达。
5. 端口冲突。
6. DNS 解析慢。
7. NetworkPolicy 阻断通信。

平台可以提供默认网络配置：

1. 自动选择训练网卡。
2. 设置 NCCL_SOCKET_IFNAME。
3. 检查 RDMA 设备。
4. 分配 master_port。
5. 做连通性检测。

不要把这些细节全部交给用户手写。

18.12 启动器和日志聚合

分布式训练有多个 rank，日志很多。

平台要决定：

1. 是否收集所有 rank 日志。
2. rank 0 日志是否单独突出显示。
3. worker 失败日志如何聚合。
4. 日志是否按节点和 rank 查询。
5. 日志中敏感信息是否脱敏。

如果只看 rank 0 日志，可能漏掉某个 worker 的真实错误。

如果收集所有日志但没有索引，用户也很难排查。

18.13 启动器和失败处理

启动阶段常见失败：

1. 镜像拉取失败。
2. 代码不存在。
3. 依赖缺失。
4. master_addr 不可达。
5. 端口冲突。

6. rank 数不一致。
7. NCCL 初始化失败。
8. 某个 worker OOM。
9. 数据挂载失败。

平台应该区分：

1. 启动前校验失败。
2. 容器启动失败。
3. 分布式初始化失败。
4. 训练运行失败。

不同阶段给用户不同诊断信息。

18.14 Elastic Training

Elastic training 是指训练任务可以适应节点数量变化或故障恢复。

PyTorch Elastic 等机制可以支持一定程度的弹性。

价值：

1. 节点故障后不用完全失败。
2. 支持抢占和恢复。
3. 支持资源动态变化。

挑战：

1. 训练代码要支持。
2. checkpoint 要兼容。
3. global batch size 可能变化。
4. 数据 shard 要重新分配。
5. 收敛行为可能变化。

弹性训练很有吸引力，但不是所有任务都能无痛支持。

18.15 启动器选择建议

可以按任务选择：

1. 普通 PyTorch DDP / FSDP: torchrun。
2. HuggingFace + ZeRO: DeepSpeed。
3. 大规模 GPT 预训练: Megatron 体系。
4. RLHF / rollout / 多阶段 pipeline: Ray。
5. HPC 集群: MPI / Slurm。
6. 企业统一平台: 自研 launcher abstraction。

这不是绝对规则。

关键要看训练代码生态、并行策略、团队经验和平台集成成本。

18.16 分布式启动器审计指标与最小 demo

把启动器做成平台能力时，不能只问“命令能不能跑”。更合理的验收口径是：资源、rank、rendezvous、GPU 绑定、launcher adapter、框架配置、网络、日志、失败处理、弹性恢复、checkpoint 和调度交接是否都能被审计。

一个启动器审计样本可以写成：

```
l_i=(r_i,n_i,p_i,w_i,m_i,b_i,e_i,g_i,c_i,o_i,f_i,z_i)
```

其中, r_i 是资源分配, n_i 是节点列表, p_i 是每节点进程数, w_i 是 world size, m_i 是 rendezvous, b_i 是 GPU 绑定, e_i 是环境变量, g_i 是 launcher adapter, c_i 是框架配置, o_i 是日志和观测, f_i 是失败和恢复策略, z_i 是审计 trace。

第 j 个检查维度的覆盖率可以写成:

$$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(l_i)=1]$$

训练平台的启动器门禁可以写成:

$$G_{\{\mathrm{launcher}\}} = \mathbf{1}[\left[\min_j C_j \geq \tau_j \ \wedge \ W_{\{\mathrm{cfg}\}} = W_{\{\mathrm{env}\}} \ \wedge \ P_0 = 0\right]]$$

其中, W_{cfg} 是配置层声明的 world size, W_{env} 是实际注入到 worker 环境里的 world size, P_0 是 P0 级启动风险数量。直觉上, launcher gate 不是替代 torchrun、DeepSpeed、Megatron 或 Ray, 而是证明平台生成的启动拓扑和每个框架实际看到的拓扑一致。

下面这个 0 依赖 demo 演示如何把分布式启动器检查做成可运行审计。它不启动真实训练进程, 只检查 toy 配置里最容易导致分布式训练 hang、错绑 GPU、并行维度不一致、日志缺失或弹性恢复不安全的字段。

```
from collections import OrderedDict
from copy import deepcopy
```

```
def complete_case():
    return OrderedDict(
        name="complete_launcher",
        launcher="torchrun",
        adapter="torchrun",
        nodes=["node-a", "node-b"],
        gpus_per_node=4,
        nproc_per_node=4,
        allocated_world_size=8,
        world_size=8,
        ranks=list(range(8)),
        local_ranks=[[0, 1, 2, 3], [0, 1, 2, 3]],
        cuda_visible_devices=[[0, 1, 2, 3], [0, 1, 2, 3]],
        master_addr="10.0.0.10",
        master_port=29500,
        torchrun=OrderedDict(
            nnodes=2,
            nproc_per_node=4,
            node_rank_assigned=True,
            rdzv_backend="c10d",
        ),
        deepspeed=OrderedDict(
            config_path="ds_config.json",
            zero_stage=2,
            hostfile=True,
        ),
        megatron=OrderedDict(
            tensor_parallel=2,
            pipeline_parallel=2,
            data_parallel=2,
            micro_batch=2,
            grad_accum=8,
            global_batch=32,
        ),
        ray=OrderedDict(
            workers=8,
```

```

        worker_gpus=1,
        scaling_config=True,
    ),
    network=OrderedDict(
        nccl_socket_ifname="ib0",
        rdma_ready=True,
        master_reachable=True,
    ),
    logs=OrderedDict(
        rank_logs="all",
        rank_index=True,
        failed_worker_collected=True,
    ),
    failure=OrderedDict(
        stage="distributed_init",
        classified=True,
        retry_policy="checkpoint_resume",
    ),
    elastic=OrderedDict(
        enabled=True,
        checkpoint_compatible=True,
        data_reshard=True,
    ),
    checkpoint=OrderedDict(
        output_uri="s3://train-ckpt/job-18",
        resume_tested=True,
        launcher_state=True,
    ),
    scheduler=OrderedDict(
        node_list_bound=True,
        node_rank_bound=True,
        gpu_binding_bound=True,
    ),
    audit=OrderedDict(
        trace_id="trace-launcher-001",
        final_command_hash="sha256:cmd-v1",
        env_snapshot=True,
    ),
    gate=True,
)

```

```

def make_bad(name, mutate):
    case = deepcopy(complete_case())
    case["name"] = name
    mutate(case)
    return case

```

```

def resource_world_size_consistency(case):
    expected = len(case["nodes"]) * case["nproc_per_node"]
    return (
        case["world_size"] == expected
        and case["allocated_world_size"] == expected
        and case["nproc_per_node"] <= case["gpus_per_node"]
    )

```



```

)

def rank_local_rank_mapping(case):
    expected_ranks = list(range(case["world_size"]))
    expected_local = list(range(case["nproc_per_node"]))
    return (
        case["ranks"] == expected_ranks
        and all(ranks == expected_local for ranks in case["local_ranks"])
    )

def rendezvous_endpoint_readiness(case):
    port = case["master_port"]
    return (
        bool(case["master_addr"])
        and isinstance(port, int)
        and 1024 <= port <= 65535
        and case["network"]["master_reachable"]
    )

def gpu_binding_visibility(case):
    need = case["nproc_per_node"]
    for devices in case["cuda_visible_devices"]:
        if len(devices) < need or len(set(devices)) != len(devices):
            return False
    return True

def launcher_adapter_fit(case):
    supported = {"torchrun", "deepspeed", "megatron", "ray", "mpi"}
    return case["launcher"] in supported and case["adapter"] == case["launcher"]

def torchrun_argument_fit(case):
    args = case["torchrun"]
    return (
        args["nnodes"] == len(case["nodes"])
        and args["nproc_per_node"] == case["nproc_per_node"]
        and args["node_rank_assigned"]
        and bool(args["rdzv_backend"])
    )

def deepspeed_config_fit(case):
    cfg = case["deepspeed"]
    return bool(cfg["config_path"]) and cfg["zero_stage"] in {0, 1, 2, 3} and cfg["hostfile"]

def megatron_parallel_consistency(case):
    cfg = case["megatron"]
    model_parallel_world = (
        cfg["tensor_parallel"] * cfg["pipeline_parallel"] * cfg["data_parallel"]
    )

```

```

global_batch = cfg["micro_batch"] * cfg["grad_accum"] * cfg["data_parallel"]
return model_parallel_world == case["world_size"] and global_batch == cfg["global_batch"]

def ray_runtime_fit(case):
    cfg = case["ray"]
    return (
        cfg["scaling_config"]
        and cfg["workers"] == case["world_size"]
        and cfg["worker_gpus"] == 1
    )

def network_nccl_readiness(case):
    net = case["network"]
    return bool(net["nccl_socket_ifname"]) and net["rdma_ready"] and net["master_reachable"]

def log_rank_aggregation(case):
    logs = case["logs"]
    return logs["rank_logs"] == "all" and logs["rank_index"] and logs["failed_worker_collected"]

def failure_stage_classification(case):
    known = {"preflight", "container_start", "distributed_init", "training_runtime"}
    failure = case["failure"]
    return failure["classified"] and failure["stage"] in known and bool(failure["retry_policy"])

def elastic_training_safety(case):
    elastic = case["elastic"]
    if not elastic["enabled"]:
        return True
    return elastic["checkpoint_compatible"] and elastic["data_reshard"]

def checkpoint_launcher_coupling(case):
    ckpt = case["checkpoint"]
    return bool(ckpt["output_uri"]) and ckpt["resume_tested"] and ckpt["launcher_state"]

def scheduler_launcher_handoff(case):
    handoff = case["scheduler"]
    return (
        handoff["node_list_bound"]
        and handoff["node_rank_bound"]
        and handoff["gpu_binding_bound"]
    )

def launcher_audit_trace(case):
    audit = case["audit"]
    return bool(audit["trace_id"]) and bool(audit["final_command_hash"]) and audit["env_snapshot"]

```

```
def distributed_launcher_gate(case):
    return case["gate"] is True
```

```
GATES = OrderedDict(
    resource_world_size_consistency=resource_world_size_consistency,
    rank_local_rank_mapping=rank_local_rank_mapping,
    rendezvous_endpoint_readiness=rendezvous_endpoint_readiness,
    gpu_binding_visibility=gpu_binding_visibility,
    launcher_adapter_fit=launcher_adapter_fit,
    torchrun_argument_fit=torchrun_argument_fit,
    deepspeed_config_fit=deepspeed_config_fit,
    megatron_parallel_consistency=megatron_parallel_consistency,
    ray_runtime_fit=ray_runtime_fit,
    network_nccl_readiness=network_nccl_readiness,
    log_rank_aggregation=log_rank_aggregation,
    failure_stage_classification=failure_stage_classification,
    elastic_training_safety=elastic_training_safety,
    checkpoint_launcher_coupling=checkpoint_launcher_coupling,
    scheduler_launcher_handoff=scheduler_launcher_handoff,
    launcher_audit_trace=launcher_audit_trace,
    distributed_launcher_gate=distributed_launcher_gate,
)
```

```
CASES = [
    complete_case(),
    make_bad("world_size_mismatch_bad", lambda c: c.__setitem__("allocated_world_size", 7)),
    make_bad("duplicate_rank_bad", lambda c: c.__setitem__("ranks", [0, 1, 2, 3, 4, 5, 6, 6])),
    make_bad("missing_rendezvous_bad", lambda c: c.__setitem__("master_addr", "")),
    make_bad("gpu_binding_mismatch_bad", lambda c: c.__setitem__("cuda_visible_devices", [[0, 1, 2], [0, 1, 2, 3]])),
    make_bad("wrong_launcher_adapter_bad", lambda c: c.__setitem__("adapter", "deepspeed")),
    make_bad("torchrun_arg_missing_bad", lambda c: c["torchrun"].__setitem__("node_rank_assigned", False)),
    make_bad("deepspeed_config_bad", lambda c: c["deepspeed"].__setitem__("zero_stage", 5)),
    make_bad("megatron_parallel_mismatch_bad", lambda c: c["megatron"].__setitem__("data_parallel", 1)),
    make_bad("ray_runtime_missing_bad", lambda c: c["ray"].__setitem__("scaling_config", False)),
    make_bad("nccl_network_missing_bad", lambda c: c["network"].__setitem__("nccl_socket_ifname", "")),
    make_bad("logs_rank0_only_bad", lambda c: c["logs"].__setitem__("rank_logs", "rank0")),
    make_bad("failure_unclassified_bad", lambda c: c["failure"].__setitem__("classified", False)),
    make_bad("elastic_without_safety_bad", lambda c: c["elastic"].__setitem__("checkpoint_compatible", False)),
    make_bad("checkpoint_uncoupled_bad", lambda c: c["checkpoint"].__setitem__("launcher_state", False)),
    make_bad("scheduler_handoff_missing_bad", lambda c: c["scheduler"].__setitem__("node_rank_bound", False)),
    make_bad("audit_trace_missing_bad", lambda c: c["audit"].__setitem__("env_snapshot", False)),
    make_bad("launcher_gate_missing_bad", lambda c: c.__setitem__("gate", False)),
]
```

```
def audit_one(case):
    return OrderedDict((name, bool(fn(case))) for name, fn in GATES.items())
```

```
def metric_coverage(audits):
    total = len(audits)
    return OrderedDict(
        (name, round(sum(result[name] for result in audits.values()) / total, 3))
    )
```

```

        for name in GATES
    )

AUDITS = OrderedDict((case["name"], audit_one(case)) for case in CASES)
METRICS = metric_coverage(AUDITS)
FAILED_CASES = [
    name for name, result in AUDITS.items() if not all(result.values())
]
FAILED_GATES = [name for name, value in METRICS.items() if value < 1.0]

examples = OrderedDict(
    world_size=complete_case()["world_size"],
    torchrun_nnodes=complete_case()["torchrun"]["nnodes"],
    torchrun_nproc_per_node=complete_case()["torchrun"]["nproc_per_node"],
    megatron_world_size=(
        complete_case()["megatron"]["tensor_parallel"]
        * complete_case()["megatron"]["pipeline_parallel"]
        * complete_case()["megatron"]["data_parallel"]
    ),
    global_batch=complete_case()["megatron"]["global_batch"],
)
smoke = OrderedDict(
    complete_case_passes=all(AUDITS["complete_launcher"].values()),
    caught_world_size_mismatch=not AUDITS["world_size_mismatch_bad"]["resource_world_size_consistency"],
    caught_megatron_mismatch=not AUDITS["megatron_parallel_mismatch_bad"]["megatron_parallel_consistency"],
    caught_network_gap=not AUDITS["nccl_network_missing_bad"]["network_nccl_readiness"],
    caught_elastic_gap=not AUDITS["elastic_without_safety_bad"]["elastic_training_safety"],
)
launcher_gate_pass = not FAILED_CASES and all(value >= 1.0 for value in METRICS.values())

print("distributed_launcher_examples=", dict(examples))
print("smoke=", dict(smoke))
print("metrics=", dict(METRICS))
print("hard_blocker_count=", len(FAILED_CASES))
print("failed_cases=", FAILED_CASES)
print("failed_gates=", FAILED_GATES)
print("distributed_launcher_gate_pass=", launcher_gate_pass)

```

这段 demo 的重点不是把真实平台实现完，而是建立面试和工程审计的语言：启动器问题要能落到 world_size 一致性、rank 映射、rendezvous、GPU 绑定、launcher adapter、DeepSpeed / Megatron / Ray 配置、NCCL 网络、rank 日志、失败阶段、elastic checkpoint、调度交接和审计 trace。只要这些字段不可见，平台就很难解释“任务为什么卡住、为什么只某个 rank OOM、为什么重试后又重复失败”。

18.17 面试中如何回答分布式启动器

如果面试官问：

训练平台如何支持 torchrun、DeepSpeed、Megatron 和 Ray？

可以这样回答：

我会在训练平台里抽象统一的 launcher interface。用户在 TrainingJob 里声明 launcher 类型、nnodes、nproc_per_node、并行策略

torchrun 适合 PyTorch DDP/FSDP，DeepSpeed 适合 ZeRO 和大模型微调，Megatron 适合大规模模型并行预训练，Ray 更适合 RLHF、ro

平台要统一处理 master_addr、master_port、rank、local_rank、world_size、NCCL 配置、日志聚合、失败诊断和 checkpoint 恢复

18.18 常见误区

误区一：分布式启动就是多开几个 Python 进程。

还要正确设置 rank、local rank、world size、master 地址、GPU 绑定和通信环境。

误区二：torchrun、DeepSpeed、Megatron、Ray 是互相替代的同类工具。

它们边界不同。torchrun 是 PyTorch 分布式启动器，DeepSpeed 是训练优化框架和启动工具，Megatron 是大模型并行训练体系，Ray 是分布式应用框架。

误区三：启动失败都是代码 bug。

也可能是资源分配、网络、端口、NCCL、环境变量、数据挂载或镜像问题。

误区四：平台只要支持一种 launcher 就够。

不同团队和任务可能依赖不同生态，平台最好通过 adapter 支持多种 launcher。

误区五：弹性训练可以自动解决所有故障。

弹性训练需要代码、checkpoint、数据分片和收敛策略配合，不是免费能力。

18.19 面试题

题 1：rank、local rank 和 world size 分别是什么？

答：rank 是全局进程编号，local rank 是节点内进程编号，world size 是全局进程总数。local rank 常用于绑定当前节点内的 GPU，world size 用于初始化分布式通信。

题 2：torchrun 主要解决什么问题？

答：torchrun 用于启动 PyTorch 分布式训练进程，设置多进程、多节点相关环境，帮助训练脚本初始化 distributed process group。它常用于 DDP 和 FSDP。

题 3：DeepSpeed 和 torchrun 有什么区别？

答：torchrun 主要是 PyTorch 原生分布式启动工具。DeepSpeed 不只是启动器，还提供 ZeRO、offload、混合精度等训练优化能力，适合降低显存压力和训练大模型。

题 4：Megatron 为什么强调 tensor parallel 和 pipeline parallel？

答：大模型参数和计算可能无法放在单张 GPU 上，Megatron 通过 tensor parallel 和 pipeline parallel 把模型切分到多张 GPU 上训练，提高可训练模型规模和吞吐。

题 5：Ray 在训练平台里适合什么场景？

答：Ray 适合复杂分布式任务编排，例如 RLHF rollout、多阶段训练 pipeline、数据处理、训练和推理混合任务。它更像分布式应用框架，不只是训练启动器。

18.20 小练习

练习一：写出 2 节点、每节点 8 GPU 的 torchrun 启动参数。

要求：说明 nnodes、nproc_per_node、node_rank、master_addr、master_port 和 world_size。

练习二：设计一个 launcher adapter 接口。

要求：输入 TrainingJob、节点列表和资源分配，输出启动命令和环境变量。

练习三：排查分布式初始化卡住。

要求：从 master_addr、端口、rank、world_size、网络、NCCL、DNS 和 NetworkPolicy 角度排查。

练习四：比较 torchrun、DeepSpeed、Megatron 和 Ray。

要求：分别说明适用任务、优势、限制和平台集成注意点。

18.21 本章小结

本章讲了分布式训练启动器。

你需要掌握：

1. 分布式训练启动器把资源分配结果转换成进程拓扑和通信环境。
2. rank、local rank、world size、master_addr、master_port 是基础概念。
3. torchrun 适合 PyTorch DDP / FSDP。
4. DeepSpeed 适合 ZeRO、offload 和大模型微调。
5. Megatron 适合大规模模型并行预训练。
6. Ray 适合复杂分布式 pipeline、RLHF、rollout 和 Agent 场景。
7. 平台最好用 launcher abstraction 统一封装不同启动器。
8. 启动器要和调度、网络、日志、失败处理和 checkpoint 集成。
9. 分布式启动失败需要按资源、环境、网络、rank 和框架分层排查。

下一章我们会讲训练配置管理：超参、版本、环境和可复现性。

第 19 章 训练配置管理：超参、版本、环境和可复现性

上一章讲了分布式训练启动器。本章讲训练平台里另一个容易被低估的问题：配置管理。

大模型训练配置非常多：模型结构、数据版本、batch size、learning rate、optimizer、precision、并行策略、checkpoint、评估频率、随机种子、环境变量。任何一个字段变化，都可能导致训练结果不同。

先记住一句话：

训练配置管理的核心不是把参数写进 YAML，而是记录、校验、版本化和冻结一次训练真正生效的全部条件。

19.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，按官方资料校准以下边界：

1. Hydra / OmegaConf 的稳定口径是配置组合、覆盖、结构化配置和命令行 override；本章借用它们的思想讲“来源、覆盖顺序和最终配置”，不绑定某个具体框架。
2. MLflow Tracking 的稳定口径是把一次 run 的 parameters、metrics、artifacts、code version 和 tags 等记录下来；本章借用它讲实验追踪和复现证据。
3. PyTorch reproducibility 文档强调，随机种子、确定性算法和环境记录能提升复现性，但跨平台、跨版本或不同算子路径不保证完全 bitwise 一致；本章因此区分“严格复现”和“统计复现”。
4. Pydantic / JSON Schema / typed config 的稳定口径是结构化校验和类型约束；本章只抽象 schema、字段类型、枚举、范围和跨字段一致性，不要求读者必须使用某个库。
5. 配置管理的工程目标不是保存一个 YAML 文件，而是冻结最终生效配置、记录字段来源、绑定代码 / 数据 / 镜像 / 环境 / 随机种子 / checkpoint / 实验追踪，并支持 diff、审批、恢复和发布。

因此，本章不把某个配置中心、实验平台、内部 SDK 或云厂商参数写成通用标准。这里讨论的是训练平台最稳定的抽象：配置来源、覆盖顺序、最终快照、版本绑定、schema 校验、跨字段一致性、资源策略、随机性边界、resume 兼容、发布耦合和审计门禁。

19.1 为什么训练配置管理重要

没有配置管理时，团队经常遇到这些问题：

1. 不知道某个模型是怎么训出来的。
2. 同一个配置重跑结果不一样。

3. 配置文件被改了，历史实验无法复现。
4. 命令行覆盖了 YAML，但没人记录。
5. 默认参数变化导致结果变化。
6. 训练代码和配置不兼容。
7. 分布式并行参数写错。
8. checkpoint 恢复时配置不一致。

配置管理的目标是让每次训练都能回答：

1. 用了什么配置？
2. 配置从哪里来？
3. 哪些字段被覆盖了？
4. 最终生效值是什么？
5. 配置和代码、数据、环境是否匹配？
6. 以后能否复现？

19.2 配置包含哪些内容

大模型训练配置可以分成十类。

19.2.1 模型配置

包括：

1. model_type。
2. hidden_size。
3. num_layers。
4. num_attention_heads。
5. vocab_size。
6. max_position_embeddings。
7. rope 配置。
8. activation。
9. normalization。
10. dropout。

模型配置决定模型结构。

如果模型结构和 checkpoint 不匹配，加载会失败或产生错误。

19.2.2 数据配置

包括：

1. dataset_id。
2. dataset_version。
3. train / eval split。
4. data sampling weight。
5. preprocessing_version。
6. tokenizer_version。
7. packing 策略。
8. max sequence length。

数据配置变化经常比模型配置变化更影响效果。

19.2.3 优化器配置

包括：

1. optimizer type。

2. learning rate。
3. beta1 / beta2。
4. weight decay。
5. epsilon。
6. gradient clipping。
7. warmup steps。
8. scheduler type。
9. min learning rate。

优化器配置影响收敛和稳定性。

19.2.4 训练批次配置

包括：

1. micro batch size。
2. global batch size。
3. gradient accumulation steps。
4. sequence length。
5. tokens per batch。
6. drop last。

这些字段要彼此一致。

例如：

$$B_{\mathrm{global}} = B_{\mathrm{micro}} A_{\mathrm{grad}} D_{\mathrm{dp}}$$

其中， B_{micro} 是每个 data-parallel rank 上的 micro batch size， A_{grad} 是 gradient accumulation steps， D_{dp} 是 data parallel size。这个公式不只是数学关系，也是配置校验规则：如果用户声明的 global batch 和平台计算出的 global batch 不一致，学习率、吞吐、显存和 loss 曲线解释都会被误导。

19.2.5 精度配置

包括：

1. FP32。
2. FP16。
3. BF16。
4. FP8。
5. loss scaling。
6. mixed precision policy。

精度配置影响速度、显存和数值稳定性。

19.2.6 并行配置

包括：

1. data parallel size。
2. tensor parallel size。
3. pipeline parallel size。
4. sequence parallel。
5. ZeRO stage。
6. FSDP policy。
7. activation checkpointing。

并行配置要和 GPU 数量、模型结构、启动器配置匹配。

19.2.7 Checkpoint 配置

包括:

1. save_every_steps。
2. keep_last_n。
3. save_best。
4. async_save。
5. resume_from。
6. checkpoint_format。
7. shard_strategy。

Checkpoint 配置影响可靠性、成本和恢复能力。

19.2.8 评估配置

包括:

1. eval_every_steps。
2. eval_dataset_version。
3. metric list。
4. generation config。
5. judge model。
6. eval batch size。

评估配置决定指标是否可比较。

19.2.9 环境配置

包括:

1. image digest。
2. CUDA version。
3. PyTorch version。
4. NCCL version。
5. Python version。
6. environment variables。
7. driver version。

环境配置是复现训练的基础。

19.2.10 平台配置

包括:

1. queue。
2. priority。
3. resource profile。
4. node selector。
5. storage mount。
6. logging policy。
7. retry policy。
8. timeout。

平台配置决定任务如何运行在基础设施上。

19.3 配置来源

训练配置可能来自多个地方:

1. 默认值。
2. 基础 YAML。
3. 任务模板。
4. 命令行参数。
5. 环境变量。
6. 配置中心。
7. 代码内部默认值。
8. 平台自动注入字段。

这会带来一个问题：最终到底哪个值生效？

例如 learning rate 可能在 YAML 里是 $1e-5$ ，命令行覆盖成 $2e-5$ ，代码默认是 $3e-5$ 。

如果平台只记录 YAML，就会误导复现。

19.4 配置覆盖顺序

平台应该明确配置覆盖顺序。

例如：

系统默认值

- < 基础模板
- < 项目模板
- < 用户 YAML
- < 命令行 override
- < 平台强制策略

平台强制策略可能包括：

1. 禁止使用未审核数据。
2. 强制 checkpoint。
3. 强制日志采集。
4. 强制安全环境变量。
5. 限制最大运行时长。

覆盖顺序必须可解释，否则用户不知道为什么配置变了。

19.5 最终生效配置

最终生效配置是训练真正使用的配置快照。

它应该在任务启动前生成并冻结。

包含：

1. 合并后的所有字段。
2. 每个字段来源。
3. 被覆盖字段记录。
4. 平台注入字段。
5. 配置 hash。
6. 生成时间。

示例：

```
{
  "learning_rate": {
    "value": 0.00002,
    "source": "cli_override"
  },
  "global_batch_size": {
    "value": 512,
```

```

    "source": "user_yaml"
  },
  "checkpoint.save_every_steps": {
    "value": 1000,
    "source": "platform_policy"
  }
}

```

复现实验时，应使用最终生效配置，而不是原始配置文件。

抽象地说，最终生效配置可以写成：

$c^{\star} = M(c_{\text{default}}, c_{\text{template}}, c_{\text{yaml}}, c_{\text{cli}}, c_{\text{env}}, c_{\text{platform_policy}})$

其中， M 是平台定义的 merge / override 函数。配置管理真正要记录的不是某一个输入文件，而是 $c_{\star}$ 的每个字段最终值、字段来源和生成时使用的 merge 规则版本。

19.6 配置版本化

配置也应该版本化。

版本化对象包括：

1. 训练模板。
2. YAML 配置文件。
3. 最终生效配置。
4. 默认参数集合。
5. 平台策略版本。

为什么平台策略版本也要记录？

因为平台可能自动注入或修改配置。

例如某天平台把默认 checkpoint 频率从 1000 step 改成 500 step，如果不记录策略版本，历史任务就难以解释。

19.7 配置校验

配置校验分为几类。

19.7.1 Schema 校验

检查字段类型和必填项。

例如：

1. learning_rate 必须是正数。
2. gpu_count 必须是整数。
3. precision 只能是允许值。
4. dataset_version 不能为空。

19.7.2 一致性校验

检查字段之间是否一致。

例如：

1. global_batch_size 是否等于 micro_batch * accumulation * data_parallel。
2. world_size 是否等于 GPU 总数。
3. tensor_parallel * pipeline_parallel 是否能整除 world_size。
4. checkpoint resume 是否和模型结构匹配。

19.7.3 资源校验

检查配置是否可能跑得动。

例如：

1. sequence length 太长是否会 OOM。
2. batch size 是否超过显存。
3. GPU 型号是否支持 BF16 / FP8。
4. 资源请求是否满足并行配置。

19.7.4 策略校验

检查是否符合平台规则。

例如：

1. 高敏数据是否允许训练。
2. 未审核镜像是否禁止使用。
3. 可抢占任务是否配置 checkpoint。
4. 高优先级是否需要审批。

19.8 超参搜索和配置管理

超参搜索会产生大量配置变体。

常见超参：

1. learning rate。
2. batch size。
3. warmup ratio。
4. weight decay。
5. dropout。
6. LoRA rank。
7. sequence length。
8. sampling ratio。

平台需要支持：

1. 配置模板。
2. 参数 sweep。
3. 实验分组。
4. 结果对比。
5. 失败任务过滤。
6. 最优配置记录。

超参搜索不能只生成一堆任务，还要能追踪每个任务对应的配置差异。

19.9 随机种子和可复现性

复现训练通常需要记录随机种子。

包括：

1. Python random seed。
2. NumPy seed。
3. PyTorch seed。
4. CUDA seed。
5. data shuffle seed。
6. dropout seed。

但记录 seed 不代表完全可复现。

原因：

1. 某些 GPU 算子非确定性。
2. 多机通信顺序可能变化。
3. 数据加载 worker 顺序可能变化。
4. 混合精度数值误差。
5. checkpoint 恢复位置差异。

所以要区分严格可复现和统计可复现。

严格可复现要求更高，成本也更高。

19.10 环境版本记录

配置管理不能只记录训练参数，还要记录环境。

至少包括：

1. image digest。
2. code commit。
3. Python version。
4. CUDA version。
5. PyTorch version。
6. NCCL version。
7. driver version。
8. GPU type。
9. launcher version。
10. platform version。

环境变化可能导致：

1. 性能变化。
2. 数值结果变化。
3. kernel 行为变化。
4. 通信行为变化。
5. checkpoint 兼容性变化。

所以环境版本是配置管理的一部分。

19.11 配置和 checkpoint 恢复

恢复 checkpoint 时，配置必须兼容。

需要检查：

1. 模型结构是否一致。
2. tokenizer 是否一致。
3. 并行策略是否兼容。
4. optimizer 配置是否一致。
5. scheduler 状态是否匹配。
6. precision 是否兼容。
7. dataset position 是否可恢复。

有些字段可以变化，例如日志路径。

有些字段不能随便变，例如 hidden size、layer 数、vocab size。

平台应该把配置字段分成：

1. immutable。
2. resumable-compatible。

3. runtime-only。

这样恢复时才能做自动校验。

19.12 配置和模型发布

模型发布也需要配置。

发布模型时要知道：

1. 模型结构。
2. tokenizer。
3. generation config。
4. safety config。
5. quantization config。
6. serving config。
7. eval config。
8. 训练数据 lineage。

训练配置和推理配置不完全一样，但它们有关联。

如果训练时使用了特殊 tokenizer 或 RoPE scaling，推理时必须匹配。

否则模型加载成功，也可能输出异常。

19.13 配置权限和审批

有些配置不是任何人都能改。

例如：

1. 使用高敏数据。
2. 使用生产资源池。
3. 申请高优先级。
4. 关闭日志采集。
5. 禁用 checkpoint。
6. 使用未审核镜像。
7. 调大最大运行时间。

这些配置应该触发策略检查或审批。

配置管理和权限系统要结合，而不是只做格式校验。

19.14 配置 diff 和实验对比

实验对比时，最重要的是知道差异在哪里。

平台应该支持配置 diff。

例如比较两个实验：

```
learning_rate: 1e-5 -> 2e-5
global_batch_size: 256 -> 512
dataset_version: v3 -> v4
precision: fp16 -> bf16
```

配置 diff 可以帮助判断效果变化来自哪里。

没有配置 diff，实验对比会变成猜测。

19.15 常见配置事故

19.15.1 使用了错误数据版本

表现：效果突然变化，复现失败。

原因：使用了 latest 路径，数据被覆盖。

防线：强制 dataset_version。

19.15.2 命令行覆盖没人记录

表现：YAML 看起来一样，但结果不同。

原因：启动命令中加了 override。

防线：记录最终生效配置。

19.15.3 batch size 算错

表现：loss 不稳定或吞吐异常。

原因：micro batch、accumulation、data parallel 不一致。

防线：一致性校验。

19.15.4 恢复 checkpoint 后效果异常

表现：恢复后 loss 跳变。

原因：optimizer、scheduler、seed、数据位置或配置不一致。

防线：恢复兼容性校验。

19.16 训练配置管理审计指标与最小 demo

配置管理在平台里应该是一个审计对象，而不是一个文件上传功能。一个配置审计样本可以写成：

$q_i = (s_i, m_i, d_i, o_i, b_i, p_i, k_i, e_i, r_i, h_i, a_i, z_i)$

其中， s_i 是配置来源和覆盖顺序， m_i 是模型配置， d_i 是数据配置， o_i 是优化器和 scheduler， b_i 是 batch / 并行配置， p_i 是 precision 和资源约束， k_i 是 checkpoint / resume， e_i 是环境版本， r_i 是随机性和复现边界， h_i 是超参 sweep 和配置 diff， a_i 是权限审批， z_i 是审计 trace。

第 j 个配置管理维度的覆盖率可以写成：

$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(q_i)=1]$

必填字段覆盖率可以写成：

$R_{\{\mathrm{required}\}} = \frac{K_{\{\mathrm{present}\}}}{K_{\{\mathrm{required}\}}}$

可复现证据覆盖率可以写成：

$R_{\{\mathrm{repr}\}} = \frac{K_{\{\mathrm{captured}\}}}{K_{\{\mathrm{required}\}}}$

checkpoint 恢复兼容性可以抽象为：

$G_{\{\mathrm{resume}\}} = \mathbf{1}[\forall k \in I_{\{\mathrm{immutable}\}}, c^{\star}_{\{k\}} = c_{\{\mathrm{ckpt}\}}^{\{k\}}]$

其中， $I_{\mathrm{immutable}}$ 是恢复时不能变化的字段集合，例如模型结构、tokenizer、vocab、并行切分、optimizer / scheduler 状态和数据游标等。训练配置门禁可以写成：

$G_{\{\mathrm{config}\}} = \mathbf{1}[\min_j C_j \geq \tau_j \wedge R_{\{\mathrm{required}\}} \geq \gamma \wedge R_{\{\mathrm{repr}\}} \geq \delta]$

下面这个 0 依赖 demo 演示如何把训练配置管理变成可运行审计。它不依赖真实 Hydra、MLflow 或 Pydantic，只用 toy 字段模拟“最终配置快照、字段来源、版本绑定、跨字段一致性、resume 兼容、权限审批和配置 diff”。

```

from collections import OrderedDict
from copy import deepcopy

```

```

REQUIRED_FIELDS = [
    "model.model_type",
    "model.hidden_size",
    "model.num_layers",
    "data.dataset_id",
    "data.dataset_version",
    "optimizer.learning_rate",
    "batch.micro_batch",
    "batch.grad_accum",
    "batch.global_batch",
    "parallel.data_parallel",
    "checkpoint.save_every_steps",
    "environment.image_digest",
    "environment.code_commit",
    "reproducibility.seed_python",
    "experiment.run_id",
    "audit.final_config_hash",
]

```

```

REPRO_FIELDS = [
    "environment.image_digest",
    "environment.code_commit",
    "environment.python_version",
    "environment.cuda_version",
    "environment.pytorch_version",
    "environment.nccl_version",
    "data.dataset_version",
    "reproducibility.seed_python",
    "reproducibility.seed_numpy",
    "reproducibility.seed_torch",
    "reproducibility.seed_data_loader",
    "audit.final_config_hash",
]

```

```

IMMUTABLE_RESUME_FIELDS = [
    "model.model_type",
    "model.hidden_size",
    "model.num_layers",
    "model.vocab_size",
    "data.tokenizer_version",
    "parallel.tensor_parallel",
    "parallel.pipeline_parallel",
    "parallel.data_parallel",
]

```

```

def get_path(case, path):
    current = case
    for part in path.split("."):
        if not isinstance(current, dict) or part not in current:
            return None

```



```

        current = current[part]
    return current

```

```

def complete_case():
    return OrderedDict(
        name="complete_config",
        sources=OrderedDict(
            order=["default", "base_template", "project_yaml", "cli", "env", "platform_policy"],
            merge_rule_version="merge-v3",
            source_map_complete=True,
            override_log=[
                ("optimizer.learning_rate", "project_yaml", "cli"),
                ("checkpoint.save_every_steps", "project_yaml", "platform_policy"),
            ],
        ),
        final_config=OrderedDict(
            frozen=True,
            hash="sha256:final-config-v7",
            generated_before_start=True,
        ),
        model=OrderedDict(
            model_type="gpt",
            hidden_size=4096,
            num_layers=32,
            vocab_size=128000,
            max_position_embeddings=8192,
        ),
        data=OrderedDict(
            dataset_id="sft_mix",
            dataset_version="2026-06-01",
            tokenizer_version="tok-v5",
            permission="approved",
        ),
        optimizer=OrderedDict(
            type="adamw",
            learning_rate=2e-5,
            weight_decay=0.1,
            scheduler="cosine",
        ),
        batch=OrderedDict(
            micro_batch=4,
            grad_accum=8,
            global_batch=256,
        ),
        parallel=OrderedDict(
            data_parallel=8,
            tensor_parallel=2,
            pipeline_parallel=2,
            world_size=32,
        ),
        precision=OrderedDict(
            dtype="bf16",
            hardware_supported=True,
        ),
    ),

```

```

checkpoint=OrderedDict(
    save_every_steps=1000,
    resume_from="s3://ckpt/run-123/step-4000",
    resume_compat_checked=True,
    ckpt_config=OrderedDict(
        model_type="gpt",
        hidden_size=4096,
        num_layers=32,
        vocab_size=128000,
        tokenizer_version="tok-v5",
        tensor_parallel=2,
        pipeline_parallel=2,
        data_parallel=8,
    ),
    state_captured=["optimizer", "scheduler", "rng", "dataloader_cursor"],
),
environment=OrderedDict(
    image_digest="sha256:image-v4",
    code_commit="abc1234",
    python_version="3.11.9",
    cuda_version="12.4",
    pytorch_version="2.5.1",
    nccl_version="2.21",
    driver_version="550.54",
),
reproducibility=OrderedDict(
    seed_python=7,
    seed_numpy=7,
    seed_torch=7,
    seed_cuda=7,
    seed_dataloader=2026,
    deterministic_mode="statistical",
    nondeterminism_notes=["distributed collectives may vary across versions"],
),
sweep=OrderedDict(
    sweep_id="lr-sweep-17",
    trial_id="trial-003",
    parent_config_hash="sha256:base-config",
    changed_fields=["optimizer.learning_rate", "batch.global_batch"],
),
experiment=OrderedDict(
    run_id="run-20260612-001",
    params_logged=True,
    artifacts_logged=True,
    final_config_logged=True,
    code_data_env_logged=True,
),
release=OrderedDict(
    tokenizer_linked=True,
    generation_config_linked=True,
    eval_config_linked=True,
    serving_config_linked=True,
),
permissions=OrderedDict(
    sensitive_data_approved=True,

```

```

        production_queue_approved=True,
        disable_checkpoint_requested=False,
        high_priority_approved=True,
    ),
    diff=OrderedDict(
        base_config_hash="sha256:base-config",
        diff_fields=["optimizer.learning_rate", "batch.global_batch", "data.dataset_version"],
        human_readable=True,
    ),
    audit=OrderedDict(
        trace_id="trace-config-001",
        actor="alice",
        validator_version="validator-v4",
        final_config_hash="sha256:final-config-v7",
        policy_decision="allow",
    ),
    gate=True,
)

```

```

def make_bad(name, mutate):
    case = deepcopy(complete_case())
    case["name"] = name
    mutate(case)
    return case

```

```

def coverage(case, fields):
    present = sum(get_path(case, field) not in (None, "", []) for field in fields)
    return present / len(fields)

```

```

def config_source_order_defined(case):
    src = case["sources"]
    return (
        src["order"][-1] == "platform_policy"
        and bool(src["merge_rule_version"])
        and src["source_map_complete"]
        and bool(src["override_log"])
    )

```

```

def final_config_snapshot_frozen(case):
    cfg = case["final_config"]
    return cfg["frozen"] and cfg["generated_before_start"] and bool(cfg["hash"])

```

```

def required_field_coverage(case):
    return coverage(case, REQUIRED_FIELDS) == 1.0

```

```

def schema_type_validation(case):
    return (
        isinstance(case["optimizer"]["learning_rate"], float)
        and case["optimizer"]["learning_rate"] > 0
    )

```

```

        and case["precision"]["dtype"] in {"fp32", "fp16", "bf16", "fp8"}
        and isinstance(case["parallel"]["world_size"], int)
        and case["parallel"]["world_size"] > 0
    )

def batch_parallel_consistency(case):
    batch = case["batch"]
    parallel = case["parallel"]
    expected_batch = batch["micro_batch"] * batch["grad_accum"] * parallel["data_parallel"]
    expected_world = (
        parallel["data_parallel"]
        * parallel["tensor_parallel"]
        * parallel["pipeline_parallel"]
    )
    return batch["global_batch"] == expected_batch and parallel["world_size"] == expected_world

def immutable_version_binding(case):
    env = case["environment"]
    data = case["data"]
    return (
        env["image_digest"].startswith("sha256:")
        and bool(env["code_commit"])
        and data["dataset_version"] not in {"", "latest"}
        and data["permission"] == "approved"
    )

def environment_version_capture(case):
    env = case["environment"]
    needed = ["python_version", "cuda_version", "pytorch_version", "nccl_version", "driver_version"]
    return all(env.get(key) for key in needed)

def reproducibility_seed_boundary(case):
    rep = case["reproducibility"]
    seeds = ["seed_python", "seed_numpy", "seed_torch", "seed_dataloader"]
    return (
        all(isinstance(rep.get(seed), int) for seed in seeds)
        and rep["deterministic_mode"] in {"strict", "statistical"}
        and bool(rep["nondeterminism_notes"])
    )

def experiment_tracking_linkage(case):
    exp = case["experiment"]
    return (
        bool(exp["run_id"])
        and exp["params_logged"]
        and exp["artifacts_logged"]
        and exp["final_config_logged"]
        and exp["code_data_env_logged"]
    )

```

```

def hyperparameter_sweep_trace(case):
    sweep = case["sweep"]
    return (
        bool(sweep["sweep_id"])
        and bool(sweep["trial_id"])
        and bool(sweep["parent_config_hash"])
        and bool(sweep["changed_fields"])
    )

def checkpoint_resume_compatibility(case):
    ckpt = case["checkpoint"]
    if not ckpt["resume_from"]:
        return True
    if not ckpt["resume_compat_checked"]:
        return False
    ckpt_cfg = ckpt["ckpt_config"]
    for field in IMMUTABLE_RESUME_FIELDS:
        left = get_path(case, field)
        right = ckpt_cfg[field.split(".")[1]]
        if left != right:
            return False
    required_state = {"optimizer", "scheduler", "rng", "dataloader_cursor"}
    return required_state.issubset(set(ckpt["state_captured"]))

def release_config_coupling(case):
    release = case["release"]
    return (
        release["tokenizer_linked"]
        and release["generation_config_linked"]
        and release["eval_config_linked"]
        and release["serving_config_linked"]
    )

def config_permission_approval(case):
    perm = case["permissions"]
    return (
        perm["sensitive_data_approved"]
        and perm["production_queue_approved"]
        and perm["high_priority_approved"]
        and not perm["disable_checkpoint_requested"]
    )

def config_diff_readiness(case):
    diff = case["diff"]
    return bool(diff["base_config_hash"]) and bool(diff["diff_fields"]) and diff["human_readable"]

def config_audit_trace(case):
    audit = case["audit"]
    return (

```

```

        bool(audit["trace_id"])
        and bool(audit["actor"])
        and bool(audit["validator_version"])
        and audit["final_config_hash"] == case["final_config"]["hash"]
        and audit["policy_decision"] == "allow"
    )

def training_config_gate(case):
    return case["gate"] is True

GATES = OrderedDict(
    config_source_order_defined=config_source_order_defined,
    final_config_snapshot_frozen=final_config_snapshot_frozen,
    required_field_coverage=required_field_coverage,
    schema_type_validation=schema_type_validation,
    batch_parallel_consistency=batch_parallel_consistency,
    immutable_version_binding=immutable_version_binding,
    environment_version_capture=environment_version_capture,
    reproducibility_seed_boundary=reproducibility_seed_boundary,
    experiment_tracking_linkage=experiment_tracking_linkage,
    hyperparameter_sweep_trace=hyperparameter_sweep_trace,
    checkpoint_resume_compatibility=checkpoint_resume_compatibility,
    release_config_coupling=release_config_coupling,
    config_permission_approval=config_permission_approval,
    config_diff_readiness=config_diff_readiness,
    config_audit_trace=config_audit_trace,
    training_config_gate=training_config_gate,
)

CASES = [
    complete_case(),
    make_bad("source_order_missing_bad", lambda c: c["sources"].__setitem__("source_map_complete", False)),
    make_bad("final_config_not_frozen_bad", lambda c: c["final_config"].__setitem__("frozen", False)),
    make_bad("required_field_missing_bad", lambda c: c["checkpoint"].__setitem__("save_every_steps", None)),
    make_bad("schema_type_bad", lambda c: c["optimizer"].__setitem__("learning_rate", -1.0)),
    make_bad("batch_parallel_mismatch_bad", lambda c: c["batch"].__setitem__("global_batch", 128)),
    make_bad("mutable_version_bad", lambda c: c["data"].__setitem__("dataset_version", "latest")),
    make_bad("environment_missing_bad", lambda c: c["environment"].__setitem__("nccl_version", "")),
    make_bad("seed_boundary_missing_bad", lambda c: c["reproducibility"].__setitem__("nondeterminism_notes", [])),
    make_bad("experiment_tracking_missing_bad", lambda c: c["experiment"].__setitem__("final_config_logged", False)),
    make_bad("sweep_trace_missing_bad", lambda c: c["sweep"].__setitem__("parent_config_hash", "")),
    make_bad("resume_incompatible_bad", lambda c: c["checkpoint"]["ckpt_config"].__setitem__("hidden_size", 2048)),
    make_bad("release_config_uncoupled_bad", lambda c: c["release"].__setitem__("serving_config_linked", False)),
    make_bad("approval_missing_bad", lambda c: c["permissions"].__setitem__("production_queue_approved", False)),
    make_bad("diff_missing_bad", lambda c: c["diff"].__setitem__("diff_fields", [])),
    make_bad("audit_trace_missing_bad", lambda c: c["audit"].__setitem__("final_config_hash", "sha256:other")),
    make_bad("config_gate_missing_bad", lambda c: c.__setitem__("gate", False)),
]

def audit_one(case):
    return OrderedDict((name, bool(fn(case))) for name, fn in GATES.items())

```

```

def metric_coverage(audits):
    total = len(audits)
    return OrderedDict(
        (name, round(sum(result[name] for result in audits.values()) / total, 3))
        for name in GATES
    )

complete = complete_case()
audits = OrderedDict((case["name"], audit_one(case)) for case in CASES)
metrics = metric_coverage(audits)
failed_cases = [name for name, result in audits.items() if not all(result.values())]
failed_gates = [name for name, value in metrics.items() if value < 1.0]

config_examples = OrderedDict(
    effective_batch=complete["batch"]["micro_batch"]
    * complete["batch"]["grad_accum"]
    * complete["parallel"]["data_parallel"],
    world_size=complete["parallel"]["data_parallel"]
    * complete["parallel"]["tensor_parallel"]
    * complete["parallel"]["pipeline_parallel"],
    required_field_coverage=round(coverage(complete, REQUIRED_FIELDS), 3),
    reproducibility_field_coverage=round(coverage(complete, REPRO_FIELDS), 3),
    final_config_hash=complete["final_config"]["hash"],
)
smoke = OrderedDict(
    complete_case_passes=all(audits["complete_config"].values()),
    caught_batch_mismatch=not audits["batch_parallel_mismatch_bad"]["batch_parallel_consistency"],
    caught_latest_dataset=not audits["mutable_version_bad"]["immutable_version_binding"],
    caught_resume_mismatch=not audits["resume_incompatible_bad"]["checkpoint_resume_compatibility"],
    caught_audit_hash_gap=not audits["audit_trace_missing_bad"]["config_audit_trace"],
)
training_config_gate_pass = not failed_cases and all(value >= 1.0 for value in metrics.values())

print("training_config_examples=", dict(config_examples))
print("smoke=", dict(smoke))
print("metrics=", dict(metrics))
print("hard_blocker_count=", len(failed_cases))
print("failed_cases=", failed_cases)
print("failed_gates=", failed_gates)
print("training_config_gate_pass=", training_config_gate_pass)

```

这段 demo 的重点不是替代真实配置框架，而是建立平台验收口径：配置管理要能证明覆盖顺序、最终快照、必填字段、类型校验、batch / 并行一致性、不可变版本绑定、环境记录、随机性边界、实验追踪、sweep trace、resume 兼容、发布耦合、权限审批、配置 diff 和审计 trace 都可见。否则同一个模型“看起来配置一样”，其实可能数据、命令行 override、平台策略、环境或 checkpoint 状态已经变了。

19.17 面试中如何回答配置管理

如果面试官问：

训练平台如何做配置管理和实验复现？

可以这样回答：

我会把训练配置分成模型、数据、优化器、batch、精度、并行、checkpoint、评估、环境和平台配置。配置可以来自默认值、模板、最终配置要记录每个字段的值和来源，生成 hash，并和镜像 digest、代码 commit、数据版本、环境版本、随机种子、启动命令一起提交时做 schema 校验、一致性校验、资源校验和策略校验，例如检查 batch size 公式、world size、并行维度、数据权限和 checkpoint 恢复 checkpoint 时，平台要检查模型结构、tokenizer、并行策略、optimizer、scheduler 和数据位置是否兼容。实验对比时提供配

19.18 常见误区

误区一：配置管理就是保存 YAML。

YAML 只是来源之一，真正重要的是最终生效配置和字段来源。

误区二：记录随机种子就能完全复现。

多 GPU、非确定性算子、通信顺序和数据加载都可能影响严格复现。

误区三：默认参数不用记录。

默认参数变化也会影响结果，必须记录最终值。

误区四：环境不是配置。

镜像、CUDA、PyTorch、NCCL、驱动和 GPU 型号都会影响结果和性能。

误区五：配置自由度越高越好。

高风险配置需要策略校验和审批。

19.19 面试题

题 1：为什么只保存 YAML 不够？

答：配置可能来自默认值、模板、YAML、命令行、环境变量和平台策略。YAML 不一定等于最终生效配置。复现需要记录合并后的最终配置和每个字段来源。

题 2：训练配置应该包含哪些类型？

答：包括模型配置、数据配置、优化器配置、batch 配置、精度配置、并行配置、checkpoint 配置、评估配置、环境配置和平台配置。

题 3：配置校验有哪些层次？

答：包括 schema 校验、一致性校验、资源校验和策略校验。比如字段类型是否合法、batch size 是否一致、显存是否可能够用、是否有权限使用数据。

题 4：恢复 checkpoint 时为什么要校验配置？

答：如果模型结构、tokenizer、并行策略、optimizer、scheduler、precision 或数据位置不兼容，恢复后可能加载失败、loss 异常或训练不可复现。

题 5：配置 diff 有什么价值？

答：配置 diff 能明确两个实验的差异，帮助判断效果变化来自 learning rate、batch size、数据版本、precision 还是其他配置，避免凭感觉分析。

19.20 小练习

练习一：设计最终生效配置 schema。

要求：每个字段包含 value、source、overridden_by、policy_applied 和 hash。

练习二：设计配置覆盖顺序。

要求：包含系统默认、基础模板、项目模板、用户 YAML、命令行 override 和平台强制策略。

练习三：设计 batch size 一致性校验。

要求：校验 micro_batch_size、gradient_accumulation_steps、data_parallel_size 和 global_batch_size。

练习四：分析一次复现失败。

假设同一 YAML 重跑结果不同，请从代码、数据、环境、随机种子、默认参数、命令行 override 和非确定性算子角度排查。

19.21 本章小结

本章讲了训练配置管理。

你需要掌握：

1. 配置管理的核心是记录、校验、版本化和冻结最终生效配置。
2. 训练配置包括模型、数据、优化器、batch、精度、并行、checkpoint、评估、环境和平台配置。
3. 配置可能来自多个来源，必须定义覆盖顺序。
4. 最终生效配置要记录字段值、来源、覆盖关系和 hash。
5. 配置版本化要包含模板、默认值和平台策略版本。
6. 配置校验包括 schema、一致性、资源和策略校验。
7. 可复现性不仅需要随机种子，还需要代码、数据、环境、配置和依赖记录。
8. checkpoint 恢复和模型发布都依赖配置兼容性。
9. 配置 diff 是实验分析的重要工具。

下一章我们会讲日志、指标、事件和训练状态追踪。

第 20 章 日志、指标、事件和训练状态追踪

上一章讲了训练配置管理。本章讲训练平台的可观测性：日志、指标、事件和训练状态追踪。

大模型训练经常跑很久、资源很贵、链路很长。没有可观测性，任务失败时你只能看到“挂了”；任务变慢时你不知道是 GPU、网络、数据、checkpoint 还是代码问题；模型效果变化时你也不知道训练过程中发生过什么。

先记住一句话：

训练可观测性的目标不是收集更多日志，而是让平台能回答任务现在在哪、是否健康、为什么慢、为什么失败、是否可恢复、成本花在哪里。

20.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，资料口径主要对齐几类公开工程资料：OpenTelemetry 对 traces、metrics、logs、events 和 profiles 等观测信号的抽象；Prometheus 对 counter、gauge、histogram 等指标类型的定义；Kubernetes Job / Pod 生命周期、失败重试和完成状态的控制面语义；MLflow Tracking 对 run、params、metrics、artifacts 和 experiment 的实验追踪口径；PyTorch profiler 对训练算子耗时、显存和长任务 profiling 的性能诊断口径。

这里不绑定某个商业监控系统、日志系统、Dashboard 产品、告警平台或云厂商实现。正文只抽象大模型训练平台里稳定的能力：结构化日志、rank 维度日志、指标类型和维度、生命周期事件、训练状态机、attempt / retry、trace span、实验追踪链接、权限脱敏、留存成本、告警和自动异常检测。换成 Prometheus + Grafana、OpenTelemetry + Jaeger、ELK / Loki、MLflow / TensorBoard、Kubernetes event 或自研控制面时，底层采集方式可以不同，但审计问题是相同的：每个训练任务能否被定位、下钻、关联、恢复、复盘和治理。

第二轮精修重点放在三个方面：

1. 把 logs、metrics、events、traces、profiles 和 experiment tracking 的边界讲清楚，避免把 “有 stdout” 误认为 “可观测”。
2. 给出可审计公式，能量化 rank 日志覆盖、指标维度覆盖、状态迁移合法性、trace 阶段覆盖和 checkpoint 新鲜度。
3. 增加一个 0 依赖 Python demo，把训练可观测性从概念清单变成可以运行的门禁检查。

20.1 为什么训练可观测性重要

训练任务不同于普通短任务。

它有几个特点：

1. 运行时间长。
2. 消耗 GPU 成本高。
3. 多节点多进程。
4. 故障原因复杂。
5. 中间状态重要。
6. 结果需要复现。
7. 失败恢复依赖 checkpoint。

没有可观测性，会出现：

1. 任务卡住没人知道。
2. GPU 空转没人发现。
3. loss 异常但继续烧钱。
4. checkpoint 保存失败很久才发现。
5. 某个 rank 慢拖住全部任务。
6. 训练失败后无法定位原因。
7. 实验对比缺少过程证据。

训练平台必须把可观测性作为核心能力，而不是上线后补日志。

20.2 日志、指标、事件、Trace 的区别

这几个概念经常混用，需要区分。

日志 logs：描述发生了什么，通常是文本或结构化记录。

指标 metrics：可聚合的数值时间序列，例如 loss、GPU utilization、tokens/s。

事件 events：任务生命周期中的关键状态变化，例如 job scheduled、checkpoint saved、worker failed。

Trace：跨组件、跨阶段的链路追踪，用来串起一次任务从提交到完成的全过程。

简单理解：

Logs：具体细节

Metrics：数值趋势

Events：状态变化

Traces：因果链路

训练平台四者都需要。

20.3 训练日志

训练日志包括：

1. stdout / stderr。
2. rank 日志。
3. worker 日志。

4. launcher 日志。
5. dataloader 日志。
6. checkpoint 日志。
7. framework 日志。
8. 错误堆栈。

日志应该包含基本上下文：

1. job_id。
2. attempt_id。
3. node_id。
4. pod_id。
5. rank。
6. timestamp。
7. log_level。
8. component。

如果日志里没有 job_id 和 rank，排查分布式训练会非常痛苦。

20.4 Rank 维度日志

分布式训练有很多 rank。

常见问题：rank 0 日志正常，但 rank 17 已经报错。

所以日志系统要支持：

1. 按 job 查看所有日志。
2. 按 rank 过滤日志。
3. 按节点过滤日志。
4. 聚合错误堆栈。
5. 高亮第一个失败 rank。
6. 对 rank 0 主日志特殊展示。

一个实用设计是：默认展示 rank 0 关键日志，但错误排查时自动提示其他 rank 的异常。

20.5 日志脱敏和权限

训练日志可能包含敏感信息。

例如：

1. 数据样本片段。
2. 文件路径。
3. 用户输入。
4. API token。
5. 数据库连接串。
6. 模型输出。
7. 业务字段。

平台需要：

1. 日志访问权限。
2. 日志脱敏。
3. 日志保留策略。
4. 日志导出审计。
5. 敏感字段检测。

不要把所有训练日志默认暴露给所有平台用户。

20.6 训练指标

训练指标可以分成四类。

20.6.1 模型指标

包括：

1. loss。
2. eval loss。
3. perplexity。
4. accuracy。
5. reward。
6. safety score。
7. task-specific metric。

这些指标告诉我们模型是否在学习。

20.6.2 性能指标

包括：

1. tokens/s。
2. samples/s。
3. step time。
4. MFU。
5. HFU。
6. dataloader time。
7. forward time。
8. backward time。
9. optimizer time。
10. communication time。

这些指标告诉我们训练是否高效。

20.6.3 资源指标

包括：

1. GPU utilization。
2. GPU memory used。
3. GPU memory bandwidth。
4. CPU utilization。
5. host memory。
6. network throughput。
7. storage throughput。
8. disk usage。
9. power。
10. temperature。

这些指标告诉我们资源是否健康。

20.6.4 平台指标

包括：

1. queue wait time。
2. scheduling time。
3. image pull time。

4. startup time。
5. checkpoint save time。
6. checkpoint load time。
7. retry count。
8. preemption count。
9. cost。

这些指标告诉我们平台是否高效。

20.7 指标采集方式

指标来源包括：

1. 训练脚本主动上报。
2. Framework callback。
3. 日志解析。
4. GPU exporter。
5. Node exporter。
6. DCGM exporter。
7. Kubernetes metrics。
8. 存储和网络监控。
9. 平台控制面事件。

模型指标通常由训练脚本上报。

资源指标通常由 exporter 采集。

平台指标通常由训练平台控制面记录。

成熟平台要把这些指标用 job_id、rank、node、project 关联起来。

20.8 指标粒度

指标粒度要平衡成本和可诊断性。

粒度太粗：看不出问题。

粒度太细：存储成本高，查询慢。

常见维度：

1. job_id。
2. attempt_id。
3. rank。
4. node。
5. GPU id。
6. step。
7. timestamp。
8. component。

例如 GPU utilization 只按集群平均看没用，必须能下钻到 job、node、GPU。

loss 曲线要按 step 对齐，而不是只按时间。

20.9 事件 Events

事件记录任务生命周期关键变化。

常见事件：

1. JobSubmitted。
2. JobValidated。

3. JobQueued。
4. JobScheduled。
5. ImagePullStarted。
6. ContainerStarted。
7. DistributedInitStarted。
8. TrainingStarted。
9. CheckpointSaved。
10. EvalStarted。
11. WorkerFailed。
12. RetryStarted。
13. JobCompleted。
14. JobFailed。
15. JobCancelled。

事件应该是结构化的。

示例：

```
{
  "event_type": "CheckpointSaved",
  "job_id": "job_123",
  "attempt_id": "attempt_2",
  "step": 12000,
  "checkpoint_uri": "s3://ckpts/job_123/step_12000",
  "duration_ms": 83000,
  "timestamp": "2026-05-30T12:00:00Z"
}
```

事件比日志更适合做状态追踪和自动化处理。

20.10 训练状态机

训练平台应该有状态机。

常见状态：

```
submitted -> validating -> queued -> scheduled -> starting -> initializing -> running
                                                    -> checkpointing
                                                    -> evaluating
                                                    -> retrying
                                                    -> completed
                                                    -> failed
                                                    -> cancelled
```

状态机价值：

1. 用户知道任务在哪个阶段。
2. 平台知道是否可以重试。
3. 调度系统知道资源是否释放。
4. 运维可以定位卡点。
5. 审计可以复盘任务过程。

没有状态机，训练平台只是日志堆积。

20.11 Attempt 和重试追踪

一个 job 可能有多个 attempt。

例如第一次运行节点故障，第二次从 checkpoint 恢复。

需要区分：

1. job_id: 逻辑任务。
2. attempt_id: 某次运行尝试。
3. run_id: 具体执行实例。

为什么重要？

因为指标和日志需要知道属于哪次尝试。

例如 loss 曲线应该能标注：step 8000 后任务失败，attempt 2 从 checkpoint 恢复。

20.12 Trace 设计

训练 trace 可以串起整个任务生命周期。

一个 trace 包含：

1. submit span。
2. validate span。
3. queue span。
4. schedule span。
5. image pull span。
6. startup span。
7. distributed init span。
8. training span。
9. checkpoint span。
10. eval span。
11. artifact registration span。

Trace 可以回答：

1. 任务为什么启动慢？
2. 排队花了多久？
3. 镜像拉取花了多久？
4. 分布式初始化卡了多久？
5. checkpoint 保存是否拖慢训练？

训练 trace 不一定要像微服务 trace 那样记录每个函数调用，但要记录关键阶段耗时。

20.13 Dashboard 设计

训练 dashboard 应该分层。

任务概览：

1. 当前状态。
2. 运行时长。
3. 队列等待时间。
4. 当前 step。
5. 最近 checkpoint。
6. 当前 loss。
7. GPU 数量。
8. 成本估计。

训练曲线：

1. loss。
2. learning rate。
3. tokens/s。
4. step time。
5. eval metrics。

资源面板：

1. GPU utilization。
2. 显存。
3. CPU。
4. 网络。
5. 存储。

诊断面板：

1. rank 差异。
2. 通信时间。
3. dataloader time。
4. checkpoint time。
5. 错误事件。

如果一个 dashboard 只有 loss 曲线，它不是训练平台 dashboard。

20.14 告警设计

训练平台需要告警。

常见告警：

1. loss NaN。
2. loss 长时间不下降。
3. GPU utilization 长时间过低。
4. step time 突然上升。
5. dataloader time 过高。
6. communication time 过高。
7. checkpoint 长时间未成功。
8. worker failed。
9. retry count 过多。
10. 存储空间不足。
11. 成本超过预算。

告警要避免过度打扰。

可以按严重程度分级：

1. info。
2. warning。
3. critical。

例如 loss NaN 是 critical，GPU 利用率略低可能只是 warning。

20.15 异常检测

平台可以做一些自动异常检测。

例如：

1. loss spike。
2. step time spike。
3. tokens/s 下降。
4. GPU utilization 异常低。
5. 某个 rank 明显慢。
6. checkpoint 保存时间异常长。
7. 数据读取吞吐下降。

异常检测可以基于：

1. 固定阈值。
2. 同任务历史。

3. 同类任务基线。
4. 滑动窗口统计。
5. 简单规则模型。

不需要一开始就复杂，很多问题用规则就能发现。

20.16 日志、指标和实验追踪的关系

实验追踪关注实验结果和配置。

可观测性关注运行过程。

二者要打通。

例如实验追踪中应该能跳转到：

1. 训练日志。
2. loss 曲线。
3. GPU 利用率。
4. checkpoint 事件。
5. 失败原因。
6. 评估报告。

这样分析模型效果时，不只看最终指标，还能看到训练过程是否健康。

20.17 数据留存和成本

日志和指标也有成本。

需要设计留存策略：

1. 高频指标保留多久？
2. 聚合指标保留多久？
3. 失败任务日志保留多久？
4. 成功任务日志保留多久？
5. 高敏日志是否更短留存？
6. 是否归档到低成本存储？

常见策略：

1. 原始高频指标保留 7 到 30 天。
2. 降采样指标保留更久。
3. 关键事件长期保留。
4. 发布模型关联日志长期保留。
5. 临时实验日志定期清理。

可观测性也需要生命周期管理。

20.18 训练可观测性审计指标与最小 demo

训练可观测性不能只靠“看起来有 Dashboard”。平台至少要能证明：日志有上下文、所有 rank 都可下钻、指标能按 job / attempt / rank / node / step 关联、事件和状态机合法、trace 能覆盖关键阶段、告警能发现高风险异常、日志权限和留存成本可治理。

可以把一次训练可观测性审计样本写成：

`o_i=(l_i,r_i,m_i,d_i,e_i,s_i,a_i,t_i,b_i,w_i,x_i,u_i,p_i,k_i,z_i)`

其中，`l_i` 是结构化日志字段，`r_i` 是 rank 日志覆盖，`m_i` 是指标类型覆盖，`d_i` 是指标维度，`e_i` 是生命周期事件，`s_i` 是状态机路径，`a_i` 是 attempt / retry 记录，`t_i` 是 trace span，`b_i` 是 dashboard 分层，`w_i` 是告警规则，`x_i` 是异常检测规则，`u_i` 是实验追踪链接，`p_i` 是权限脱敏，`k_i` 是成本归因，`z_i` 是最终可观测性门禁。

统一覆盖率仍然可以写成：

$$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(o_i)=1]$$

rank 日志覆盖率可以写成：

$$R_{\{\mathrm{rank}\}} = \frac{N_{\{\mathrm{logged}\}}}{N_{\{\mathrm{rank}\}}}$$

其中， N_{logged} 表示可查询日志的 rank 数， N_{rank} 表示本次任务总 rank 数。对分布式训练来说， $R_{\mathrm{rank}}=1$ 还不够，还要能定位第一个失败 rank、慢 rank 和非 rank 0 的异常堆栈。

训练 step time 分解可以写成：

$$T_{\{\mathrm{step}\}} = T_{\{\mathrm{data}\}} + T_{\{\mathrm{fwd}\}} + T_{\{\mathrm{bwd}\}} + T_{\{\mathrm{comm}\}} + T_{\{\mathrm{optim}\}} + T_{\{\mathrm{other}\}}$$

如果 Dashboard 只有 loss 曲线，没有 T_{data} 、 T_{comm} 、 T_{ckpt} 、rank skew 和 GPU / network / storage 指标，就很难解释“为什么慢”。

状态机迁移合法性可以写成：

$$V_{\{\mathrm{state}\}} = \frac{1}{K} \sum_{k=1}^K \mathbf{1}[(s_{k-1}, s_k) \in E_{\{\mathrm{allowed}\}}]$$

其中， E_{allowed} 是平台允许的状态迁移集合。例如 `queued -> scheduled -> starting -> initializing -> running` 合法，而 `completed -> running`、`failed -> completed` 或 `running -> queued` 通常说明状态机或事件处理有 bug。

trace 阶段覆盖率可以写成：

$$R_{\{\mathrm{span}\}} = \frac{K_{\{\mathrm{observed}\}}}{K_{\{\mathrm{required}\}}}$$

其中， K_{required} 是 submit、validate、queue、schedule、image pull、startup、distributed init、training、checkpoint、eval、artifact registration 等必需阶段数量， K_{observed} 是实际 trace 中能看到的阶段数量。

checkpoint 新鲜度可以写成：

$$A_{\{\mathrm{ckpt}\}} = t_{\{\mathrm{now}\}} - t_{\{\mathrm{last}\}}$$

如果 A_{ckpt} 超过阈值，且任务仍在长时间运行，就要触发告警：这可能意味着 checkpoint 保存失败、存储不可写、rank 卡住或控制面事件丢失。

训练可观测性综合门禁可以写成：

$$G_{\{\mathrm{obs}\}} = \mathbf{1}[\min_j C_j \geq \tau_j \ \& \ R_{\{\mathrm{rank}\}} \geq \gamma \ \& \ V_{\{\mathrm{state}\}}=1 \ \& \ P_0=0]$$

其中， P_0 是 P0 级观测硬阻断数量，例如日志缺 job / attempt / rank、只有 rank 0 日志、状态机非法、trace 断链、日志泄露敏感信息、checkpoint 长时间无成功事件但没有告警。

下面这个 demo 不依赖第三方库。它用一条完整训练任务和 16 条坏样本，模拟训练可观测性门禁如何发现常见问题。

```
from collections import OrderedDict
```

```
REQUIRED_LOG_FIELDS = {
    "job_id",
    "attempt_id",
    "node_id",
    "pod_id",
    "rank",
    "timestamp",
    "level",
    "component",
}
```

```
REQUIRED_METRIC_GROUPS = {"model", "performance", "resource", "platform"}
REQUIRED_METRIC_DIMS = {"job_id", "attempt_id", "rank", "node", "step", "timestamp"}
REQUIRED_EVENT_TYPES = {
    "JobSubmitted",
```

```

    "JobValidated",
    "JobQueued",
    "JobScheduled",
    "TrainingStarted",
    "CheckpointSaved",
    "WorkerFailed",
    "RetryStarted",
    "JobCompleted",
}
REQUIRED_EVENT_FIELDS = {"event_type", "job_id", "attempt_id", "timestamp"}
ALLOWED_TRANSITIONS = {
    ("submitted", "validating"),
    ("validating", "queued"),
    ("queued", "scheduled"),
    ("scheduled", "starting"),
    ("starting", "initializing"),
    ("initializing", "running"),
    ("running", "checkpointing"),
    ("checkpointing", "running"),
    ("running", "evaluating"),
    ("evaluating", "running"),
    ("running", "retrying"),
    ("retrying", "queued"),
    ("running", "completed"),
    ("running", "failed"),
    ("running", "cancelled"),
}
REQUIRED_TRACE_SPANS = {
    "submit",
    "validate",
    "queue",
    "schedule",
    "image_pull",
    "startup",
    "distributed_init",
    "training",
    "checkpoint",
    "eval",
    "artifact_registration",
}
REQUIRED_DASHBOARD_SECTIONS = {"overview", "training_curve", "resource", "diagnostic", "cost"}
REQUIRED_ALERT_RULES = {
    "loss_nan",
    "gpu_low",
    "step_time_spike",
    "checkpoint_stale",
    "worker_failed",
}
REQUIRED_ANOMALY_RULES = {
    "loss_spike",
    "step_time_spike",
    "rank_straggler",
    "checkpoint_slow",
}
REQUIRED_EXPERIMENT_LINKS = {

```

```

    "run_id",
    "params",
    "metrics",
    "artifacts",
    "config_hash",
    "checkpoint_uri",
}
REQUIRED_RETENTION_POLICY = {
    "raw_metrics",
    "downsampled_metrics",
    "failed_logs",
    "release_logs",
    "archive",
}
REQUIRED_PRIVACY_CONTROLS = {"rbac", "redaction", "export_audit", "sensitive_detector"}
REQUIRED_COST_FIELDS = {"gpu_hours", "storage_gib_day", "network_gib", "owner", "project"}

GATE_ORDER = [
    "structured_log_context",
    "rank_log_coverage",
    "metric_taxonomy_coverage",
    "metric_dimension_scope",
    "event_schema_lifecycle",
    "state_machine_validity",
    "attempt_retry_traceability",
    "trace_phase_coverage",
    "dashboard_diagnostic_readiness",
    "alert_rule_readiness",
    "anomaly_detection_rules",
    "experiment_tracking_linkage",
    "retention_cost_governance",
    "privacy_redaction_access",
    "cost_attribution_signal",
    "training_observability_gate",
]

def has_all(values, required):
    return set(values) >= set(required)

def ratio(numerator, denominator):
    if denominator == 0:
        return 0.0
    return numerator / denominator

def rounded(value):
    return round(value, 3)

def transitions_are_valid(states):
    if len(states) < 2:
        return False

```

```

    return all((src, dst) in ALLOWED_TRANSITIONS for src, dst in zip(states, states[1:]))

def structured_events_are_valid(events):
    event_types = {event.get("event_type") for event in events}
    fields_ok = all(set(event) >= REQUIRED_EVENT_FIELDS for event in events)
    return fields_ok and event_types >= REQUIRED_EVENT_TYPES

def rank_log_coverage(case):
    ranks = set(case.get("ranks", []))
    logged_ranks = set(case.get("logged_ranks", []))
    if not ranks:
        return 0.0
    return ratio(len(ranks & logged_ranks), len(ranks))

def step_time_breakdown_ms(case):
    return sum(case["step_breakdown_ms"].values())

def checkpoint_age_min(case):
    return case["now_min"] - case["last_checkpoint_min"]

def audit_one(case):
    gates = OrderedDict()
    gates["structured_log_context"] = has_all(case["log_fields"], REQUIRED_LOG_FIELDS)
    gates["rank_log_coverage"] = (
        rank_log_coverage(case) >= 0.99
        and set(case["error_ranks_detected"]) >= set(case["error_ranks"])
    )
    gates["metric_taxonomy_coverage"] = has_all(case["metric_groups"], REQUIRED_METRIC_GROUPS)
    gates["metric_dimension_scope"] = has_all(case["metric_dimensions"], REQUIRED_METRIC_DIMS)
    gates["event_schema_lifecycle"] = structured_events_are_valid(case["events"])
    gates["state_machine_validity"] = transitions_are_valid(case["state_events"])
    gates["attempt_retry_traceability"] = bool(case["attempt_id"]) and case["retry_policy_recorded"]
    gates["trace_phase_coverage"] = has_all(case["trace_spans"], REQUIRED_TRACE_SPANS)
    gates["dashboard_diagnostic_readiness"] = has_all(
        case["dashboard_sections"], REQUIRED_DASHBOARD_SECTIONS
    )
    gates["alert_rule_readiness"] = has_all(case["alert_rules"], REQUIRED_ALERT_RULES)
    gates["anomaly_detection_rules"] = has_all(case["anomaly_rules"], REQUIRED_ANOMALY_RULES)
    gates["experiment_tracking_linkage"] = has_all(
        case["experiment_links"], REQUIRED_EXPERIMENT_LINKS
    )
    gates["retention_cost_governance"] = has_all(
        case["retention_policy"], REQUIRED_RETENTION_POLICY
    )
    gates["privacy_redaction_access"] = has_all(case["privacy_controls"], REQUIRED_PRIVACY_CONTROLS)
    gates["cost_attribution_signal"] = has_all(case["cost_fields"], REQUIRED_COST_FIELDS)
    gates["training_observability_gate"] = bool(case["observability_gate"])

    return {
        "name": case["name"],

```

```

    "gates": gates,
    "rank_log_coverage": rounded(rank_log_coverage(case)),
    "trace_phase_coverage": rounded(ratio(len(set(case["trace_spans"])), len(REQUIRED_TRACE_SPANS))),
    "state_transitions_valid": transitions_are_valid(case["state_events"]),
    "step_time_ms": rounded(step_time_breakdown_ms(case)),
    "checkpoint_age_min": checkpoint_age_min(case),
}

```

```

def clone(base, name, **changes):

```

```

    copied = {}
    for key, value in base.items():
        if isinstance(value, list):
            copied[key] = list(value)
        elif isinstance(value, dict):
            copied[key] = dict(value)
        elif isinstance(value, set):
            copied[key] = set(value)
        else:
            copied[key] = value
    copied["name"] = name
    copied.update(changes)
    return copied

```

```

complete_case = {
    "name": "complete_case",
    "job_id": "job_train_obs_001",
    "attempt_id": "attempt_2",
    "log_fields": REQUIRED_LOG_FIELDS,
    "ranks": list(range(8)),
    "logged_ranks": list(range(8)),
    "error_ranks": [6],
    "error_ranks_detected": [6],
    "metric_groups": REQUIRED_METRIC_GROUPS,
    "metric_dimensions": REQUIRED_METRIC_DIMS,
    "events": [
        {
            "event_type": event_type,
            "job_id": "job_train_obs_001",
            "attempt_id": "attempt_2",
            "timestamp": "2026-06-12T12:00:00Z",
            "step": 1200,
        }
        for event_type in REQUIRED_EVENT_TYPES
    ],
    "state_events": [
        "submitted",
        "validating",
        "queued",
        "scheduled",
        "starting",
        "initializing",
        "running",
        "checkpointing",
    ],
}

```

```

        "running",
        "evaluating",
        "running",
        "completed",
    ],
    "retry_policy_recorded": True,
    "trace_spans": REQUIRED_TRACE_SPANS,
    "dashboard_sections": REQUIRED_DASHBOARD_SECTIONS,
    "alert_rules": REQUIRED_ALERT_RULES,
    "anomaly_rules": REQUIRED_ANOMALY_RULES,
    "experiment_links": REQUIRED_EXPERIMENT_LINKS,
    "retention_policy": REQUIRED_RETENTION_POLICY,
    "privacy_controls": REQUIRED_PRIVACY_CONTROLS,
    "cost_fields": REQUIRED_COST_FIELDS,
    "observability_gate": True,
    "now_min": 240,
    "last_checkpoint_min": 228,
    "step_breakdown_ms": {
        "data": 180.0,
        "fwd": 260.0,
        "bwd": 520.0,
        "comm": 310.0,
        "optim": 90.0,
        "ckpt": 30.0,
        "log": 10.0,
    },
},
}

cases = [
    complete_case,
    clone(complete_case, "log_context_missing_bad", log_fields={"job_id", "timestamp", "level"}),
    clone(
        complete_case,
        "rank0_only_bad",
        logged_ranks=[0],
        error_ranks=[3],
        error_ranks_detected=[],
    ),
    clone(complete_case, "metric_taxonomy_missing_bad", metric_groups={"model"}),
    clone(
        complete_case,
        "metric_dimension_unscoped_bad",
        metric_dimensions={"job_id", "timestamp"},
    ),
    clone(
        complete_case,
        "event_schema_unstructured_bad",
        events=[{"event_type": "WorkerFailed", "message": "rank 3 exited"}],
    ),
    clone(complete_case, "state_machine_invalid_bad", state_events=["submitted", "running", "queued"]),
    clone(complete_case, "attempt_id_missing_bad", attempt_id="", retry_policy_recorded=False),
    clone(complete_case, "trace_phase_missing_bad", trace_spans={"submit", "training", "eval"}),
    clone(complete_case, "dashboard_loss_only_bad", dashboard_sections={"training_curve"}),
    clone(complete_case, "alert_rule_missing_bad", alert_rules={"loss_nan"}),
    clone(complete_case, "anomaly_rule_missing_bad", anomaly_rules=set()),

```

```

clone(complete_case, "experiment_tracking_unlinked_bad", experiment_links={"run_id"}),
clone(complete_case, "retention_policy_missing_bad", retention_policy=set()),
clone(complete_case, "privacy_redaction_missing_bad", privacy_controls={"rbac"}),
clone(complete_case, "cost_attribution_missing_bad", cost_fields={"owner"}),
clone(complete_case, "observability_gate_missing_bad", observability_gate=False),
]

audits = [audit_one(case) for case in cases]
metrics = OrderedDict()
for gate_name in GATE_ORDER:
    passed = sum(1 for audit in audits if audit["gates"][gate_name])
    metrics[gate_name] = rounded(ratio(passed, len(audits)))

failed_cases = [
    audit["name"]
    for audit in audits
    if not all(audit["gates"].values())
]
failed_gates = [name for name, value in metrics.items() if value < 1.0]

complete_audit = audits[0]
examples = {
    "rank_log_coverage": complete_audit["rank_log_coverage"],
    "trace_phase_coverage": complete_audit["trace_phase_coverage"],
    "state_transitions_valid": complete_audit["state_transitions_valid"],
    "step_time_ms": complete_audit["step_time_ms"],
    "checkpoint_age_min": complete_audit["checkpoint_age_min"],
    "linked_run_id": "run_train_obs_001",
}
smoke = {
    "complete_case_passes": all(complete_audit["gates"].values()),
    "caught_rank0_only": "rank0_only_bad" in failed_cases,
    "caught_invalid_state": "state_machine_invalid_bad" in failed_cases,
    "caught_missing_trace": "trace_phase_missing_bad" in failed_cases,
    "caught_privacy_gap": "privacy_redaction_missing_bad" in failed_cases,
    "caught_gate_gap": "observability_gate_missing_bad" in failed_cases,
}

print("training_observability_examples=", examples, sep="")
print("smoke=", smoke, sep="")
print("metrics=", dict(metrics), sep="")
print("hard_blocker_count=", len(failed_cases), sep="")
print("failed_cases=", failed_cases, sep="")
print("failed_gates=", failed_gates, sep="")
print("training_observability_gate_pass=", all(value >= 0.99 for value in metrics.values()), sep="")

```

一组期望输出类似：

```

training_observability_examples={'rank_log_coverage': 1.0, 'trace_phase_coverage': 1.0, 'state_transitions_valid': True,
smoke={'complete_case_passes': True, 'caught_rank0_only': True, 'caught_invalid_state': True, 'caught_missing_trace': True,
metrics={'structured_log_context': 0.941, 'rank_log_coverage': 0.941, 'metric_taxonomy_coverage': 0.941, 'metric_dimension_scope': 0.941,
hard_blocker_count=16
failed_cases=['log_context_missing_bad', 'rank0_only_bad', 'metric_taxonomy_missing_bad', 'metric_dimension_unscoped_bad',
failed_gates=['structured_log_context', 'rank_log_coverage', 'metric_taxonomy_coverage', 'metric_dimension_scope', 'observability_gate_missing_bad',
training_observability_gate_pass=False

```

这个 demo 想说明：训练可观测性不是日志平台、指标平台、trace 平台或实验平台中的某一个，而是把它们通

过 `job_id`、`attempt_id`、`rank`、`step`、状态机、`checkpoint`、`run` 和 `artifact` 串成同一条证据链。面试时如果能把“慢、挂、贵、不可恢复、不可复现”都落到这组证据链上，回答就会比只列工具名扎实很多。

20.19 面试中如何回答训练可观测性

如果面试官问：

如何设计大模型训练平台的日志、指标和状态追踪？

可以这样回答：

我会把训练可观测性分成 `logs`、`metrics`、`events` 和 `traces`。

`Logs` 用于排查具体错误，需要按 `job`、`attempt`、`node`、`pod`、`rank`、`component` 结构化采集，并支持 `rank` 维度查询和日志脱敏。

`Metrics` 分为模型指标、性能指标、资源指标和平台指标。模型指标包括 `loss`、`eval score`；性能指标包括 `tokens/s`、`step time`、`MFU`。

`Events` 记录任务生命周期，比如 `submitted`、`validated`、`queued`、`scheduled`、`started`、`checkpoint_saved`、`worker_failed`、`retry`。

平台需要训练状态机和 `attempt` 概念，区分逻辑 `job` 和多次运行尝试。`Dashboard` 要展示任务状态、训练曲线、资源使用、`rank` 差异。

20.20 常见误区

误区一：可观测性就是收集 `stdout`。

`stdout` 只是日志的一部分，还需要指标、事件、`trace` 和状态机。

误区二：只看 `rank 0` 日志就够。

分布式训练中错误可能发生在任意 `rank`。

误区三：有 `loss` 曲线就能判断训练健康。

还要看吞吐、`step time`、GPU、通信、数据加载和 `checkpoint`。

误区四：日志越多越好。

日志太多会增加成本和噪声，还可能泄露敏感信息。

误区五：成功任务不需要保留事件。

事件是审计、复现和性能分析的重要依据。

20.21 面试题

题 1：logs、metrics、events、traces 有什么区别？

答：logs 记录具体细节，metrics 是可聚合的数值时间序列，events 记录任务生命周期状态变化，traces 串起跨组件和阶段的因果链路。训练平台四者都需要。

题 2：训练平台应该采集哪些关键指标？

答：包括模型指标 `loss`、`eval score`；性能指标 `tokens/s`、`step time`、`MFU`、`communication time`、`dataloader time`；资源指标 GPU utilization、显存、CPU、网络、存储；平台指标 `queue time`、`startup time`、`checkpoint time`、`retry count` 和 `cost`。

题 3：为什么需要 attempt_id？

答：同一个 `job` 可能失败后重试或从 `checkpoint` 恢复。`attempt_id` 区分不同运行尝试，帮助关联日志、指标、事件和恢复过程，避免把多次运行混在一起。

题 4：训练 dashboard 应该包含什么？

答：应包含任务状态、运行时间、当前 step、最近 checkpoint、loss 和 learning rate 曲线、tokens/s、step time、GPU 和显存、rank 差异、通信时间、dataloader time、checkpoint time、错误事件和成本估计。

题 5：为什么日志需要权限和脱敏？

答：训练日志可能包含样本内容、用户输入、模型输出、路径、密钥片段和业务字段。如果不控制权限和脱敏，可能造成数据泄露。

20.22 小练习

练习一：设计一个训练任务事件 schema。

要求：包含 event_type、job_id、attempt_id、rank、step、timestamp、message 和 metadata。

练习二：设计一个训练 dashboard。

要求：分为任务概览、训练曲线、资源面板和诊断面板。

练习三：分析一个训练变慢问题。

假设 step time 从 1.2s 增加到 2.5s，请从 dataloader、通信、checkpoint、GPU、网络 and 存储指标排查。

练习四：设计日志留存策略。

要求：区分高频原始指标、降采样指标、失败任务日志、发布模型关联日志和临时实验日志。

20.23 本章小结

本章讲了日志、指标、事件和训练状态追踪。

你需要掌握：

1. 训练可观测性要回答任务在哪、是否健康、为什么慢、为什么失败、是否可恢复和成本花在哪里。
2. Logs、metrics、events、traces 各自解决不同问题。
3. 分布式训练日志要支持 rank、node、pod、attempt 维度查询。
4. 指标应覆盖模型、性能、资源和平台四类。
5. 事件和状态机用于追踪任务生命周期。
6. Attempt 用于区分同一 job 的多次运行尝试。
7. Trace 可以分析提交、排队、调度、启动、训练、checkpoint 等阶段耗时。
8. Dashboard 要同时展示训练曲线、资源、诊断和成本。
9. 告警和异常检测能减少 GPU 空烧和训练事故。
10. 日志、指标和事件也需要权限、脱敏和留存策略。

下一章我们会讲 Checkpoint 策略：频率、保留、异步保存和断点恢复。

第 21 章 Checkpoint 策略：频率、保留、异步保存和断点恢复

第 10 章我们讲过 checkpoint 的存储、加载、恢复和生命周期管理。本章回到训练平台工程视角，重点讲 checkpoint 策略：训练时应该多久保存一次？保留哪些？异步保存怎么做？抢占和失败恢复如何联动？

Checkpoint 策略不是越频繁越好，也不是越省越好。它是在训练成本、存储成本、恢复损失、评估需求和平台复杂度之间做平衡。

先记住一句话：

Checkpoint 策略的目标，是用可接受的 I/O 和存储成本，把训练失败后的损失控制在可接受范围内，并支持评估、回滚、抢占和发布。

21.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，资料口径主要对齐几类公开工程资料：PyTorch Distributed Checkpoint 对分布式 state_dict、planner、storage writer、异步保存和加载的抽象；DeepSpeed checkpointing 对模型、优化器、scheduler、ZeRO 分片状态保存和恢复的接口边界；Kubernetes priority / preemption、Pod 终止宽限期和任务生命周期对可抢占训练的控制面影响；对象存储生命周期策略对 checkpoint 保留、归档和删除的治理口径。

这里不把某个框架 API、某种对象存储、某个调度器或某家云厂商的参数写成通用标准。正文只抽象 checkpoint 策略的稳定问题：RPO/RTO、保存触发、保存内容分层、同步 / 异步保存、一致 snapshot、临时目录、checksum、manifest commit、分片元数据、恢复选择、抢占联动、评估联动、发布联动、监控告警、保留成本和恢复演练。

第二轮精修重点放在三个方面：

1. 把“多久保存一次”从经验参数升级为 RPO、RTO、保存阻塞比例和最大丢失进度的可计算策略。
2. 把 resume、eval、release checkpoint 分清，避免所有 checkpoint 都保存完整训练状态或只保留 latest。
3. 增加一个 0 依赖 Python demo，把 checkpoint 策略从口头 checklist 变成能发现坏策略的门禁。

21.1 Checkpoint 策略要解决什么问题

Checkpoint 策略要回答八个问题：

1. 什么时候保存？
2. 保存什么内容？
3. 保存到哪里？
4. 同步保存还是异步保存？
5. 保留多少个？
6. 哪些 checkpoint 是关键版本？
7. 失败后从哪个 checkpoint 恢复？
8. 如何控制存储成本？

这些问题如果没有平台策略，就会变成每个训练脚本各自实现，最后无法治理。

21.2 保存频率的 trade-off

保存频率是最核心的策略。

保存太频繁：

1. 训练被 I/O 打断。
2. checkpoint 存储膨胀。
3. metadata 压力大。
4. 多任务同时保存导致存储拥塞。
5. 成本上升。

保存太少：

1. 故障后损失大量训练进度。
2. 抢占恢复代价高。
3. 找不到中间版本评估。
4. 训练异常后可回滚点少。

一个合理问题是：

如果任务现在失败，我最多能接受损失多少 GPU-hours？

保存频率应该围绕这个目标设计。

21.3 RPO 和 RTO

可以借用两个可靠性概念。

RPO: Recovery Point Objective, 恢复点目标。

它表示最多能接受丢失多少训练进度。

例如 RPO = 30 minutes, 表示任务失败后最多回退 30 分钟训练。

RTO: Recovery Time Objective, 恢复时间目标。

它表示从失败到恢复训练需要多久。

例如 RTO = 15 minutes, 表示故障后 15 分钟内恢复运行。

Checkpoint 策略同时影响 RPO 和 RTO。

1. 保存越频繁, RPO 越小。
2. checkpoint 加载越快, RTO 越小。
3. checkpoint 越大, RTO 可能越大。
4. 存储越慢, 保存和恢复都更慢。

面试中讲 checkpoint 策略时, 用 RPO/RTO 会显得很工程化。

21.4 按 step 保存还是按时间保存

常见保存方式有两种。

按 step 保存:

每 1000 step 保存一次

优点:

1. 和训练进度对齐。
2. 便于评估和比较。
3. 实现简单。

缺点:

1. step time 变化时, 真实时间间隔不稳定。
2. 如果 step 很慢, RPO 可能变大。

按时间保存:

每 30 分钟保存一次

优点:

1. RPO 更容易控制。
2. 适合长时间任务。

缺点:

1. checkpoint step 不规整。
2. 和 eval milestone 对齐较差。

实际平台可以同时支持: 按 step、按时间、按里程碑保存。

21.5 保存哪些内容

不同 checkpoint 用途不同。

可以分三类:

21.5.1 Resume checkpoint

用于断点续训。

包含：

1. 模型参数。
2. 优化器状态。
3. scheduler。
4. random state。
5. dataloader state。
6. 分布式状态。

体积大，但恢复能力强。

21.5.2 Eval checkpoint

用于评估。

可以只包含模型权重和必要 config。

体积相对小，方便评估和对比。

21.5.3 Release checkpoint

用于发布或部署。

需要包含：

1. 模型权重。
2. tokenizer。
3. config。
4. generation config。
5. eval report。
6. metadata。

发布 checkpoint 要更严格校验。

平台应该区分这三类，不要所有 checkpoint 都保存完整训练状态。

21.6 保留策略

保留策略决定哪些 checkpoint 留下。

常见策略：

1. 保留最近 N 个。
2. 每隔 M step 保留一个 milestone。
3. 保留最优 eval checkpoint。
4. 保留最终 checkpoint。
5. 保留发布版本。
6. 删除失败或未提交 checkpoint。
7. 冷版本归档到低成本存储。

一个实用策略：

最近 5 个 resume checkpoint：高性能存储保留。

每 10k step milestone：保留 30 天。

best eval checkpoint：长期保留。

release checkpoint：长期保留并加权限保护。

failed temporary checkpoint：7 天后清理。

保留策略要按任务类型区分。

预训练和 SFT 的策略不一定相同。

21.7 同步保存

同步保存是训练暂停，等待 checkpoint 写完。

适合：

1. 小模型。
2. checkpoint 不大。
3. 一致性要求高。
4. 平台实现早期。

优点：

1. 逻辑简单。
2. 一致性容易保证。
3. 恢复点明确。

缺点：

1. GPU 等 I/O。
2. step time 有尖刺。
3. 大模型保存时间长。
4. 多任务同时保存会造成拥塞。

同步保存适合作为基础能力，但大规模训练通常需要进一步优化。

21.8 异步保存

异步保存是把 checkpoint 写入从训练主循环中解耦。

常见做法：

1. 训练进程生成快照。
2. 后台线程或进程写入存储。
3. 训练继续往前跑。
4. 后台完成后提交 manifest。

优点：

1. 减少训练阻塞。
2. 降低 GPU 空等。
3. 提高训练吞吐。

缺点：

1. 实现复杂。
2. 需要额外内存或磁盘缓冲。
3. 可能和训练争抢 I/O。
4. 要处理失败和一致性。
5. 不能让半成品被恢复。

异步保存的关键是 snapshot 和 commit 语义。

21.9 异步保存的一致性问题的

异步保存最大风险是保存内容不一致。

例如模型参数已经进入 step 1001，但 optimizer state 还是 step 1000。

防护方法：

1. 在确定边界生成一致 snapshot。
2. snapshot 完成后训练才能继续修改对应状态。
3. 写入临时目录。
4. 所有 shard 写完后校验 checksum。
5. 最后提交 manifest。
6. 恢复时只读取 committed checkpoint。

异步 checkpoint 不是简单把 `save()` 放到线程里。

21.10 分片保存策略

大模型 checkpoint 通常分片保存。

分片策略包括：

1. 按 rank 保存。
2. 按 tensor shard 保存。
3. 按 layer 保存。
4. 参数和 optimizer state 分开保存。
5. 模型权重和训练状态分开保存。

分片策略影响：

1. 保存速度。
2. 加载速度。
3. metadata 压力。
4. rank 数变化恢复。
5. 跨框架兼容性。

如果未来需要 resharding，分片格式要尽量带完整 metadata。

21.11 断点恢复策略

断点恢复要决定从哪个 checkpoint 恢复。

常见策略：

1. 从最新 committed checkpoint 恢复。
2. 从用户指定 checkpoint 恢复。
3. 从最近健康 checkpoint 恢复。
4. 从 best checkpoint 继续微调。
5. 从 milestone checkpoint 回滚。

恢复前要校验：

1. checkpoint 是否完整。
2. 配置是否兼容。
3. 数据版本是否可访问。
4. 并行策略是否匹配。
5. 权限是否允许。
6. 存储路径是否可读。

恢复后要记录事件：

```
JobResumedFromCheckpoint(step=12000, checkpoint_id=ckpt_12000)
```

21.12 Checkpoint 与抢占联动

可抢占任务必须有 checkpoint 策略。

抢占前最好：

1. 发送 preemption notice。
2. 触发一次 checkpoint。
3. 等待 checkpoint committed。
4. 标记任务为 preempted。
5. 释放资源。

如果通知时间很短，可能来不及保存完整 checkpoint。

因此可抢占队列应该要求：

1. 周期性 checkpoint。
2. 最近 checkpoint 不超过某个时间阈值。
3. checkpoint 保存时间可控。

调度系统抢占时应该优先选择最近有 checkpoint 的任务，减少损失。

21.13 Checkpoint 与评估联动

很多训练平台会在 checkpoint 后触发评估。

流程：

save checkpoint -> run eval -> record metrics -> mark best checkpoint

这样可以自动找到 best checkpoint。

注意：

1. 不一定每个 checkpoint 都要评估。
2. 评估会消耗推理资源。
3. 评估配置要版本化。
4. best checkpoint 需要保护。
5. eval 失败不一定代表训练失败。

训练平台要把 checkpoint 和 eval report 关联起来。

21.14 Checkpoint 与模型发布联动

发布模型通常来自某个 checkpoint。

发布前要做：

1. 权重转换。
2. tokenizer 和 config 校验。
3. 推理加载测试。
4. 评估报告检查。
5. 安全审核。
6. metadata 注册。
7. 权限标记。

发布 checkpoint 和训练 resume checkpoint 不同。

发布版本应该更稳定、更小、更可审计。

21.15 Checkpoint 监控和告警

Checkpoint 策略必须可观测。

监控指标：

1. save_duration。
2. load_duration。
3. checkpoint_size。

4. time_since_last_checkpoint。
5. save_failure_rate。
6. committed_count。
7. temporary_count。
8. storage_usage。
9. restore_success_rate。
10. preemption_without_recent_checkpoint_count。

告警：

1. 长时间未成功 checkpoint。
2. checkpoint 保存失败。
3. 保存耗时突然上升。
4. 存储容量接近上限。
5. 临时 checkpoint 堆积。
6. 恢复失败。

Checkpoint 没有监控，就不是真正可靠。

21.16 常见策略错误

错误一：所有 checkpoint 都保存完整训练状态。

会导致存储成本过高。应该区分 resume、eval、release。

错误二：只保留最新一个 checkpoint。

如果最新 checkpoint 损坏或训练已经发散，就没有回滚点。

错误三：异步保存没有 commit 语义。

可能恢复到半成品 checkpoint。

错误四：可抢占任务 checkpoint 太少。

抢占后损失大，用户体验差。

错误五：没有恢复测试。

保存成功不代表能恢复。

21.17 Checkpoint 策略审计指标与最小 demo

Checkpoint 策略不能只写成“每 1000 step 保存一次”。平台至少要能证明：保存频率符合 RPO，恢复耗时符合 RTO，保存阻塞比例可接受，resume / eval / release 三类 checkpoint 分清，异步保存有一致 snapshot 和 commit 语义，保留策略能控制成本，抢占、评估、发布、告警和恢复演练都能和 checkpoint 串起来。

可以把一次 checkpoint 策略审计样本写成：

$p_i = (r_i, f_i, s_i, a_i, m_i, h_i, u_i, q_i, e_i, v_i, l_i, c_i, o_i, d_i, z_i)$

其中， r_i 是 RPO / RTO 目标， f_i 是保存触发策略， s_i 是保存内容分层， a_i 是异步保存策略， m_i 是 manifest / commit 语义， h_i 是分片元数据， u_i 是恢复选择策略， q_i 是抢占联动， e_i 是评估联动， v_i 是发布联动， l_i 是保留生命周期， c_i 是存储成本， o_i 是监控告警， d_i 是恢复演练， z_i 是最终策略门禁。

统一覆盖率可以写成：

$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(p_i) = 1]$

按 step 保存的最大丢失时间可以粗略写成：

$A_{\{\mathrm{step}\}} = \frac{I_{\{\mathrm{step}\}} T_{\{\mathrm{step}\}}}{60}$

其中， I_{step} 是 step 间隔， T_{step} 是单 step 秒数。按时间保存的最大丢失时间就是 $A_{\mathrm{time}} = I_{\mathrm{time}}$ 。混合策略下，可以取更保守的触发上界：

$$A_{\mathrm{lost}} = \min(A_{\mathrm{step}}, A_{\mathrm{time}})$$

如果有抢占通知，还要比较通知窗口和 checkpoint 完成时间：

$$G_{\mathrm{preempt}} = \mathbf{1} \setminus \left[T_{\mathrm{notice}} \geq T_{\mathrm{snapshot}} + T_{\mathrm{commit}} \right]$$

同步保存的阻塞比例可以写成：

$$R_{\mathrm{block}} = \frac{T_{\mathrm{block}}}{I_{\mathrm{step}} T_{\mathrm{step}} + T_{\mathrm{block}}}$$

T_{block} 是训练主循环真正停住等待 checkpoint 的时间。异步保存不是把 T_{block} 变成 0，而是把主循环阻塞从完整写入时间降低到 snapshot、buffer、metadata 或必要同步边界。

保留成本可以写成：

$$K_{\mathrm{retain}} = \sum_{k=1}^K N_k S_k D_k P_k + K_{\mathrm{request}} + K_{\mathrm{ops}}$$

其中， N_k 是第 k 类 checkpoint 保留个数， S_k 是单个大小， D_k 是保留天数， P_k 是对应存储层的单位成本。

Checkpoint 策略门禁可以写成：

$$G_{\mathrm{ckpt_strategy}} = \mathbf{1} \setminus \left[A_{\mathrm{lost}} \leq \mathrm{RPO} \ \wedge \ T_{\mathrm{restore}} \leq \mathrm{RT} \right]$$

下面这个 demo 不依赖第三方库。它用一条完整策略和 16 条坏样本，模拟平台如何审计 checkpoint 频率、保留、异步保存、抢占、评估、发布和恢复演练。

```
from collections import OrderedDict
```

```
GATE_ORDER = [
    "rpo_rto_budget_defined",
    "save_trigger_policy",
    "checkpoint_scope_separation",
    "async_snapshot_consistency",
    "manifest_commit_integrity",
    "shard_metadata_resharding",
    "restore_selection_policy",
    "preemption_checkpoint_coupling",
    "evaluation_best_linkage",
    "release_checkpoint_readiness",
    "retention_tier_lifecycle",
    "storage_cost_budget",
    "checkpoint_monitoring_alerts",
    "restore_drill_coverage",
    "permission_release_protection",
    "checkpoint_strategy_gate",
]

REQUIRED_SCOPES = {
    "resume": {"model", "optimizer", "scheduler", "rng", "dataloader", "distributed", "config"},
    "eval": {"model", "config", "tokenizer", "metadata"},
    "release": {"model", "tokenizer", "config", "generation_config", "eval_report", "metadata"},
}

REQUIRED_TRIGGERS = {"step", "time", "milestone", "preemption"}

REQUIRED_ASYNC_CONTROLS = {
    "snapshot_boundary",
    "buffer_budget",
    "temp_path",
    "checksum",
    "manifest_commit",
    "committed_only",
}
```

```

    "failure_cleanup",
}
REQUIRED_MANIFEST_FIELDS = {"checkpoint_id", "global_step", "shards", "checksums", "state_kind", "status"}
REQUIRED_SHARD_METADATA = {"rank_map", "tensor_shards", "global_step", "format_version", "reshard_policy"}
REQUIRED_RESTORE_POLICY = {
    "latest_committed",
    "health_check",
    "config_compat",
    "data_access",
    "permission_check",
    "event_record",
}
REQUIRED_PREEMPTION_POLICY = {
    "notice_handler",
    "on_notice_checkpoint",
    "freshness_gate",
    "scheduler_uses_freshness",
}
REQUIRED_EVAL_LINKS = {"eval_after_milestone", "eval_config_version", "best_protection", "report_link"}
REQUIRED_RELEASE_LINKS = {
    "conversion_test",
    "tokenizer_config_check",
    "load_smoke",
    "eval_report_required",
    "security_review",
    "registry_metadata",
}
REQUIRED_MONITORING = {
    "save_duration",
    "load_duration",
    "checkpoint_age",
    "failure_rate",
    "temporary_count",
    "storage_usage",
    "restore_success",
    "preempt_without_recent",
}
REQUIRED_PERMISSIONS = {"release_protected", "rbac", "audit_event", "archive_policy"}
REQUIRED_RETENTION_KINDS = {"resume", "milestone", "best_eval", "release", "failed_temporary"}

TIER_PRICE_PER_GIB_DAY = {
    "hot": 0.004,
    "warm": 0.0015,
    "archive": 0.0004,
    "protected": 0.0025,
}

def has_all(values, required):
    return set(values) >= set(required)

def ratio(numerator, denominator):
    if denominator == 0:
        return 0.0

```

```

    return numerator / denominator

def rounded(value):
    return round(value, 3)

def max_lost_minutes(case):
    step_minutes = case["step_interval"] * case["step_seconds"] / 60.0
    return min(step_minutes, case["time_interval_minutes"])

def blocking_ratio(case):
    interval_seconds = case["step_interval"] * case["step_seconds"]
    return ratio(case["main_loop_block_seconds"], interval_seconds + case["main_loop_block_seconds"])

def retention_cost(case):
    total = case.get("request_ops_cost", 0.0)
    for item in case["retention_items"]:
        price = TIER_PRICE_PER_GIB_DAY[item["tier"]]
        total += item["count"] * item["size_gib"] * item["days"] * price
    return total

def retention_kinds(case):
    return {item["kind"] for item in case["retention_items"]}

def scopes_are_separated(case):
    scopes = case["checkpoint_scopes"]
    return all(has_all(scopes.get(kind, set()), fields) for kind, fields in REQUIRED_SCOPES.items())

def audit_one(case):
    lost = max_lost_minutes(case)
    cost = retention_cost(case)
    gates = OrderedDict()
    gates["rpo_rto_budget_defined"] = (
        case["rpo_minutes"] > 0
        and case["rto_minutes"] > 0
        and lost <= case["rpo_minutes"]
        and case["restore_minutes"] <= case["rto_minutes"]
    )
    gates["save_trigger_policy"] = has_all(case["save_triggers"], REQUIRED_TRIGGERS)
    gates["checkpoint_scope_separation"] = scopes_are_separated(case)
    gates["async_snapshot_consistency"] = (
        not case["async_enabled"] or has_all(case["async_controls"], REQUIRED_ASYNC_CONTROLS)
    )
    gates["manifest_commit_integrity"] = has_all(case["manifest_fields"], REQUIRED_MANIFEST_FIELDS)
    gates["shard_metadata_resharding"] = has_all(case["shard_metadata"], REQUIRED_SHARD_METADATA)
    gates["restore_selection_policy"] = has_all(case["restore_policy"], REQUIRED_RESTORE_POLICY)
    gates["preemption_checkpoint_coupling"] = (
        has_all(case["preemption_policy"], REQUIRED_PREEMPTION_POLICY)
        and case["checkpoint_age_minutes"] <= case["preemption_freshness_minutes"]
    )

```

```

)
gates["evaluation_best_linkage"] = has_all(case["eval_links"], REQUIRED_EVAL_LINKS)
gates["release_checkpoint_readiness"] = has_all(case["release_links"], REQUIRED_RELEASE_LINKS)
gates["retention_tier_lifecycle"] = has_all(retention_kinds(case), REQUIRED_RETENTION_KINDS)
gates["storage_cost_budget"] = cost <= case["storage_budget"]
gates["checkpoint_monitoring_alerts"] = has_all(case["monitoring"], REQUIRED_MONITORING)
gates["restore_drill_coverage"] = (
    case["restore_drill"]["scheduled"]
    and case["restore_drill"]["pass_rate"] >= 0.99
    and case["restore_drill"]["last_days_ago"] <= 30
)
gates["permission_release_protection"] = has_all(case["permissions"], REQUIRED_PERMISSIONS)
gates["checkpoint_strategy_gate"] = bool(case["strategy_gate"])

return {
    "name": case["name"],
    "gates": gates,
    "max_lost_minutes": rounded(lost),
    "blocking_ratio": rounded(blocking_ratio(case)),
    "retention_cost": round(cost, 1),
    "restore_minutes": case["restore_minutes"],
    "retention_kinds": sorted(retention_kinds(case)),
}

```

```

def clone(base, name, **changes):
    copied = {}
    for key, value in base.items():
        if isinstance(value, dict):
            copied[key] = {
                inner_key: set(inner_value) if isinstance(inner_value, set) else dict(inner_value)
                if isinstance(inner_value, dict)
                else inner_value
                for inner_key, inner_value in value.items()
            }
        elif isinstance(value, list):
            copied[key] = [dict(item) if isinstance(item, dict) else item for item in value]
        elif isinstance(value, set):
            copied[key] = set(value)
        else:
            copied[key] = value
    copied["name"] = name
    copied.update(changes)
    return copied

```

```

complete_case = {
    "name": "complete_case",
    "rpo_minutes": 30,
    "rto_minutes": 20,
    "step_interval": 1000,
    "step_seconds": 1.5,
    "time_interval_minutes": 30,
    "restore_minutes": 14,
    "main_loop_block_seconds": 18,
}

```

```

"save_triggers": REQUIRED_TRIGGERS,
"checkpoint_scopes": REQUIRED_SCOPES,
"async_enabled": True,
"async_controls": REQUIRED_ASYNC_CONTROLS,
"manifest_fields": REQUIRED_MANIFEST_FIELDS,
"shard_metadata": REQUIRED_SHARD_METADATA,
"restore_policy": REQUIRED_RESTORE_POLICY,
"preemption_policy": REQUIRED_PREEMPTION_POLICY,
"checkpoint_age_minutes": 18,
"preemption_freshness_minutes": 30,
"eval_links": REQUIRED_EVAL_LINKS,
"release_links": REQUIRED_RELEASE_LINKS,
"retention_items": [
    {"kind": "resume", "count": 5, "size_gib": 3200, "days": 7, "tier": "hot"},
    {"kind": "milestone", "count": 8, "size_gib": 800, "days": 30, "tier": "warm"},
    {"kind": "best_eval", "count": 2, "size_gib": 250, "days": 180, "tier": "warm"},
    {"kind": "release", "count": 1, "size_gib": 240, "days": 365, "tier": "protected"},
    {"kind": "failed_temporary", "count": 3, "size_gib": 100, "days": 7, "tier": "archive"},
],
"request_ops_cost": 25.0,
"storage_budget": 1500.0,
"monitoring": REQUIRED_MONITORING,
"restore_drill": {"scheduled": True, "pass_rate": 1.0, "last_days_ago": 14},
"permissions": REQUIRED_PERMISSIONS,
"strategy_gate": True,
}

cases = [
    complete_case,
    clone(complete_case, "rpo_rto_missing_bad", rpo_minutes=0, rto_minutes=0),
    clone(complete_case, "save_trigger_missing_bad", save_triggers={"step"}),
    clone(
        complete_case,
        "scope_mixed_bad",
        checkpoint_scopes={
            "resume": {"model"},
            "eval": {"model"},
            "release": {"model"},
        },
    ),
    clone(complete_case, "async_snapshot_missing_bad", async_controls={"temp_path", "checksum"}),
    clone(complete_case, "manifest_commit_missing_bad", manifest_fields={"checkpoint_id", "status"}),
    clone(complete_case, "shard_metadata_missing_bad", shard_metadata={"rank_map"}),
    clone(complete_case, "restore_policy_latest_bad", restore_policy={"latest_committed"}),
    clone(
        complete_case,
        "preemption_uncoupled_bad",
        preemption_policy={"notice_handler"},
        checkpoint_age_minutes=120,
    ),
    clone(complete_case, "eval_unlinked_bad", eval_links={"eval_after_milestone"}),
    clone(complete_case, "release_unready_bad", release_links={"registry_metadata"}),
    clone(
        complete_case,
        "retention_latest_only_bad",

```

```

        retention_items=[{"kind": "resume", "count": 1, "size_gib": 3200, "days": 7, "tier": "hot"}],
    ),
    clone(complete_case, "storage_cost_unbounded_bad", storage_budget=500.0),
    clone(complete_case, "monitoring_missing_bad", monitoring={"save_duration"}),
    clone(
        complete_case,
        "restore_drill_missing_bad",
        restore_drill={"scheduled": False, "pass_rate": 0.0, "last_days_ago": 999},
    ),
    clone(complete_case, "permission_release_open_bad", permissions={"archive_policy"}),
    clone(complete_case, "strategy_gate_missing_bad", strategy_gate=False),
]

audits = [audit_one(case) for case in cases]
metrics = OrderedDict()
for gate_name in GATE_ORDER:
    passed = sum(1 for audit in audits if audit["gates"][gate_name])
    metrics[gate_name] = rounded(ratio(passed, len(audits)))

failed_cases = [
    audit["name"]
    for audit in audits
    if not all(audit["gates"].values())
]
failed_gates = [name for name, value in metrics.items() if value < 1.0]
complete_audit = audits[0]

examples = {
    "max_lost_minutes": complete_audit["max_lost_minutes"],
    "restore_minutes": complete_audit["restore_minutes"],
    "blocking_ratio": complete_audit["blocking_ratio"],
    "retention_cost": complete_audit["retention_cost"],
    "retention_kinds": complete_audit["retention_kinds"],
}
smoke = {
    "complete_case_passes": all(complete_audit["gates"].values()),
    "caught_rpo_gap": "rpo_rto_missing_bad" in failed_cases,
    "caught_scope_gap": "scope_mixed_bad" in failed_cases,
    "caught_async_gap": "async_snapshot_missing_bad" in failed_cases,
    "caught_preemption_gap": "preemption_uncoupled_bad" in failed_cases,
    "caught_restore_drill_gap": "restore_drill_missing_bad" in failed_cases,
}

print("checkpoint_strategy_examples=", examples, sep="")
print("smoke=", smoke, sep="")
print("metrics=", dict(metrics), sep="")
print("hard_blocker_count=", len(failed_cases), sep="")
print("failed_cases=", failed_cases, sep="")
print("failed_gates=", failed_gates, sep="")
print("checkpoint_strategy_gate_pass=", all(value >= 0.99 for value in metrics.values()), sep="")

```

一组期望输出类似：

```

checkpoint_strategy_examples={'max_lost_minutes': 25.0, 'restore_minutes': 14, 'blocking_ratio': 0.012, 'retention_cost': 0.012, 'retention_kinds': ['hot', 'warm', 'cold'], 'retention_items': [{'kind': 'resume', 'count': 1, 'size_gib': 3200, 'days': 7, 'tier': 'hot'}], 'storage_cost_unbounded_bad': 500.0, 'monitoring_missing_bad': {'save_duration': 1000}, 'restore_drill_missing_bad': {'scheduled': False, 'pass_rate': 0.0, 'last_days_ago': 999}, 'permission_release_open_bad': {'archive_policy': 'archive_policy'}, 'strategy_gate_missing_bad': False}
smoke={'complete_case_passes': True, 'caught_rpo_gap': True, 'caught_scope_gap': True, 'caught_async_gap': True, 'caught_preemption_gap': True, 'caught_restore_drill_gap': True}
metrics={'rpo_rto_budget_defined': 0.941, 'save_trigger_policy': 0.941, 'checkpoint_scope_separation': 0.941, 'async_snapshot_missing': 0.941, 'preemption_uncoupled': 0.941, 'scope_mixed': 0.941, 'rpo_rto_missing': 0.941, 'restore_drill_missing': 0.941, 'storage_cost_unbounded': 0.941, 'monitoring_missing': 0.941}
hard_blocker_count=0
failed_cases=[]
failed_gates=[]
checkpoint_strategy_gate_pass=True

```

```
hard_blocker_count=16
```

```
failed_cases=['rpo_rto_missing_bad', 'save_trigger_missing_bad', 'scope_mixed_bad', 'async_snapshot_missing_bad', 'man
```

```
failed_gates=['rpo_rto_budget_defined', 'save_trigger_policy', 'checkpoint_scope_separation', 'async_snapshot_consiste
```

```
checkpoint_strategy_gate_pass=False
```

这个 demo 想说明：checkpoint 策略的关键不是“保存成功”四个字，而是把 RPO/RTO、保存阻塞、异步一致性、分片元数据、恢复选择、抢占、评估、发布、留存、成本、权限、监控和恢复演练放进同一个策略门禁。大模型训练平台如果没有这张表，checkpoint 很容易从可靠性能力变成存储成本和恢复风险的来源。

21.18 面试中如何回答 checkpoint 策略

如果面试官问：

大模型训练平台如何设计 checkpoint 策略？

可以这样回答：

我会先按用途区分 checkpoint：resume checkpoint 用于断点续训，包含模型、优化器、scheduler、随机状态和 dataloader 状态；

保存频率要根据 RPO/RTO 设计，可以支持按 step、按时间和 milestone 保存。保存太频繁会阻塞训练和增加存储成本，太少会导致故障

大模型 checkpoint 应采用分片保存，并用 manifest、checksum 和 commit 语义保证一致性。异步 checkpoint 可以减少训练阻塞，但

保留策略上，保留最近 N 个 resume checkpoint、周期性 milestone、best eval checkpoint 和 release checkpoint，临时失败 check

调度层要和 checkpoint 联动：可抢占任务必须有周期 checkpoint，抢占前尽量触发保存，恢复时从最新 committed checkpoint 加载

21.19 常见误区

误区一：checkpoint 越频繁越好。

频繁保存会拖慢训练并增加存储成本，要按 RPO/RTO 权衡。

误区二：checkpoint 只为故障恢复服务。

它还服务评估、回滚、抢占、发布和实验分析。

误区三：异步保存一定无损。

异步保存会带来一致性和资源竞争问题，需要 snapshot 和 commit 语义。

误区四：保留 latest 就够。

latest 可能损坏或对应坏训练状态，需要保留 milestone 和 best checkpoint。

误区五：能保存就能恢复。

必须做恢复测试和配置兼容校验。

21.20 面试题

题 1：Checkpoint 保存频率如何选择？

答：要根据可接受的恢复进度损失和恢复时间来选择，也就是 RPO/RTO。保存太频繁会增加 I/O 和存储成本，保存太少会导致故障后回退太多。可以支持按 step、按时间和 milestone 混合策略。

题 2：Resume checkpoint、eval checkpoint、release checkpoint 有什么区别？

答：Resume checkpoint 用于断点续训，包含完整训练状态；eval checkpoint 用于评估，通常只需要模型权重和必要配置；release checkpoint 用于发布和推理，需要权重、tokenizer、config、评估报告和 metadata。

题 3: 异步 checkpoint 的关键风险是什么?

答: 关键风险是不一致和半成品恢复。必须在一致边界生成 snapshot, 写入临时目录, 校验所有 shard 后提交 manifest, 并且恢复时只读取 committed checkpoint。

题 4: 可抢占训练任务为什么必须配 checkpoint?

答: 抢占会中断任务, 如果没有 checkpoint, 训练进度会大量丢失。可抢占任务应周期性保存 checkpoint, 抢占前尽量触发保存, 恢复时从最新 committed checkpoint 继续。

题 5: 为什么不能只保留最新 checkpoint?

答: 最新 checkpoint 可能损坏, 也可能对应训练发散后的状态。保留 milestone、best eval 和 release checkpoint 可以支持回滚、评估和发布。

21.21 小练习

练习一: 设计一个 checkpoint 策略。

要求: 给一个 70B SFT 任务设计保存频率、保留策略、best checkpoint 和 release checkpoint 规则。

练习二: 设计异步 checkpoint 流程。

要求: 画出 snapshot、写临时目录、checksum、manifest、commit 和失败清理流程。

练习三: 设计抢占联动。

要求: 说明可抢占任务收到 preemption notice 后如何保存 checkpoint, 调度系统如何选择被抢占任务。

练习四: 分析 checkpoint 成本失控。

要求: 从保存频率、完整训练状态、保留策略、实验数量和冷存储归档角度分析。

21.22 本章小结

本章讲了 checkpoint 策略。

你需要掌握:

1. Checkpoint 策略要平衡训练成本、存储成本、恢复损失、评估需求和发布需求。
2. RPO/RTO 是设计保存频率的重要框架。
3. 可以按 step、按时间和 milestone 保存。
4. 要区分 resume、eval、release checkpoint。
5. 保留策略应包含最近 N 个、milestone、best eval、release 和临时失败清理。
6. 异步 checkpoint 需要一致 snapshot、临时目录、checksum、manifest 和 commit 语义。
7. 分片保存影响保存速度、加载速度、metadata 和 resharding。
8. 可抢占任务必须和 checkpoint 策略联动。
9. Checkpoint 应和评估、发布、监控和恢复测试打通。

下一章我们会讲训练容错: 节点失败、通信失败、OOM 和自动重试。

第 22 章 训练容错: 节点失败、通信失败、OOM 和自动重试

上一章讲了 checkpoint 策略。本章继续讲训练平台的可靠性: 训练任务失败时, 平台应该怎么办?

大模型训练任务运行时间长、资源多、组件多。节点、GPU、网络、存储、代码、数据、配置任意一处出问题, 都可能导致任务失败。训练平台不能只把状态标成 failed, 它应该尽量识别故障类型、判断是否可恢复、自动重试或从 checkpoint 恢复, 并给用户可行动的诊断信息。

先记住一句话:

训练容错的核心不是无限重试，而是故障分类、损失控制、自动恢复和清晰诊断。

22.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，资料口径主要对齐几类公开工程资料：PyTorch Elastic / torchrun 对 max_restarts、rendezvous、worker failure、agent failure、node failure 和 membership change 的分布式恢复口径；Kubernetes Job 对 restartPolicy、backoffLimit、podFailurePolicy、Pod 失败计数和指数退避的控制面语义；NVIDIA NCCL troubleshooting 对通信环境、网络接口、共享内存、GPU Direct RDMA、debug 日志和异步错误处理的排障口径；PyTorch CUDA memory management 对 caching allocator、memory_allocated、memory_reserved、max_memory_allocated、memory snapshot 和 OOM 调参的口径。

这里不把某个训练平台、某个云厂商、某个 GPU 型号、某个调度器插件或某个 NCCL 环境变量组合写成通用答案。正文只抽象训练容错里的稳定问题：故障分类、retryable / non-retryable 判断、attempt 预算、backoff、checkpoint 恢复、节点替换、GPU 健康隔离、通信 hang 检测、OOM 处理、数据和 checkpoint 故障、数值稳定性、诊断报告、黑名单、用户可见动作和审计 trace。

第二轮精修重点放在三个方面：

1. 把“自动重试”改写为有边界的故障分类和恢复策略，明确哪些错误可以重试，哪些错误应该快速失败。
2. 补充可计算指标：最大丢失 GPU-hours、重试成功率、无效重试率、OOM 显存余量、rank hang 心跳间隔和恢复连续性。
3. 增加一个 0 依赖 Python demo，把训练容错从经验 checklist 变成可运行的门禁检查。

22.1 为什么训练容错重要

预训练任务可能使用几百张 GPU 跑几天。

如果任务因为一个节点故障中断，并且无法恢复，损失巨大。

训练容错的价值：

1. 降低失败重跑成本。
2. 提高 GPU 利用率。
3. 支持长周期训练。
4. 支持抢占式调度。
5. 降低运维压力。
6. 改善用户体验。
7. 让故障可复盘。

容错不是只靠训练脚本，也需要平台、调度、checkpoint、监控和存储共同配合。

22.2 故障分类

训练失败可以分为几大类：

1. 基础设施故障。
2. 资源不足。
3. 通信故障。
4. 存储和数据故障。
5. 用户代码故障。
6. 配置故障。
7. 数值稳定性故障。
8. 平台控制面故障。

不同故障处理方式不同。

例如：

1. 节点故障适合重调度后从 checkpoint 恢复。
2. OOM 可能需要调整 batch 或显存策略，盲目重试没用。

3. 数据权限错误应直接失败，不该自动重试。
4. 临时网络抖动可以重试。
5. loss NaN 需要诊断数值问题。

故障分类是自动恢复的前提。

22.3 可恢复故障和不可恢复故障

可以先粗分为两类。

可恢复故障：

1. 节点宕机。
2. GPU 短暂故障。
3. 网络短暂抖动。
4. 存储短暂不可用。
5. Pod 被驱逐。
6. 可抢占任务被抢占。
7. 镜像拉取临时失败。

不可恢复或不宜自动重试故障：

1. 代码语法错误。
2. 配置字段错误。
3. 数据权限错误。
4. checkpoint 格式不兼容。
5. 持续 OOM。
6. 模型结构和权重不匹配。
7. loss 持续 NaN。

自动重试要谨慎。错误分类不准，会导致 GPU 空烧。

22.4 节点失败

节点失败包括：

1. 机器宕机。
2. kubelet 异常。
3. GPU 掉卡。
4. 驱动异常。
5. 本地盘损坏。
6. 节点网络断开。

表现：

1. Pod 变成 failed 或 unknown。
2. 某个 rank 失联。
3. NCCL timeout。
4. 训练 hang。
5. GPU 指标消失。

处理策略：

1. 标记节点不可调度。
2. 收集节点诊断信息。
3. 释放任务资源。
4. 选择新节点。
5. 从最近 checkpoint 恢复。
6. 记录恢复事件。

如果任务不支持 checkpoint，只能失败或从头重跑。

22.5 GPU 故障

GPU 故障可能包括：

1. ECC error。
2. Xid error。
3. GPU reset。
4. 显存错误。
5. 温度过高。
6. 功耗异常。
7. GPU 不可见。

平台需要采集 GPU 健康指标。

如果某张 GPU 多次导致任务失败，应自动隔离该 GPU 或节点。

处理策略：

1. 记录 GPU 错误码。
2. 标记节点 unhealthy。
3. 禁止新任务调度。
4. 触发硬件维修流程。
5. 任务从 checkpoint 恢复到其他节点。

不要让坏 GPU 反复坑不同任务。

22.6 通信失败

通信失败是多机训练高频问题。

常见表现：

1. NCCL timeout。
2. NCCL unhandled error。
3. distributed init 卡住。
4. AllReduce 时间暴涨。
5. 某个 rank hang。
6. 网络重传增加。

可能原因：

1. 网络抖动。
2. 网卡故障。
3. RDMA 配置错误。
4. 防火墙或 NetworkPolicy 阻断。
5. 端口冲突。
6. rank 配置错误。
7. 某个 worker OOM 后退出，其他 worker 等通信。

处理策略：

1. 区分初始化失败还是运行中失败。
2. 检查第一个失败 rank。
3. 收集 NCCL 日志。
4. 检查网络指标。
5. 对临时故障自动重试。
6. 对配置错误快速失败。

通信失败不能只看最后报错，因为最后报错的 rank 不一定是根因。

22.7 OOM

OOM 是训练常见失败。

包括：

1. GPU OOM。
2. CPU memory OOM。
3. container memory limit OOM。
4. shared memory 不足。

GPU OOM 常见原因：

1. batch size 太大。
2. sequence length 太长。
3. activation 太多。
4. ZeRO/FSDP 配置不合理。
5. gradient accumulation 配错。
6. 没开启 activation checkpointing。
7. KV cache 或 eval 过程占用过高。

OOM 是否适合自动重试？

通常不适合直接原样重试。

更好的策略：

1. 失败并给出建议。
2. 自动降低 micro batch 后重试，但必须用户允许。
3. 建议开启 activation checkpointing。
4. 建议使用 ZeRO/FSDP。
5. 提示显存峰值和失败 step。

盲目重试 OOM 只会重复失败。

22.8 数据故障

数据故障包括：

1. 数据路径不存在。
2. 数据权限不足。
3. 数据文件损坏。
4. shard 缺失。
5. 读取超时。
6. tokenizer 不匹配。
7. 样本格式错误。
8. 数据中有异常值。

处理策略：

1. 提交前校验数据版本。
2. 启动前做小样本读取测试。
3. 训练中记录坏样本位置。
4. 对临时读取失败重试。
5. 对格式错误快速失败。
6. 生成数据诊断报告。

数据错误如果被吞掉，可能导致模型质量问题，而不只是训练失败。

22.9 Checkpoint 故障

Checkpoint 故障包括：

1. 保存失败。
2. 加载失败。
3. shard 缺失。
4. checksum 不匹配。
5. manifest 不完整。
6. 存储空间不足。
7. 权限不足。
8. 恢复后 loss 异常。

处理策略：

1. 保存失败要告警。
2. 不允许恢复 uncommitted checkpoint。
3. 加载前校验 manifest。
4. 失败后尝试上一个健康 checkpoint。
5. 定期做恢复测试。
6. 记录 checkpoint 失败事件。

Checkpoint 是容错基础，它自己也需要容错。

22.10 数值稳定性故障

数值故障包括：

1. loss NaN。
2. loss inf。
3. gradient norm 爆炸。
4. 参数变成 NaN。
5. mixed precision overflow。
6. reward 异常。

可能原因：

1. learning rate 太大。
2. 数据异常。
3. loss scaling 配置错误。
4. FP16 不稳定。
5. 梯度裁剪缺失。
6. 初始化或模型结构问题。

处理策略：

1. 立即停止或暂停任务。
2. 保存诊断状态。
3. 标记异常 step。
4. 从上一个健康 checkpoint 回滚。
5. 提示可能配置原因。

数值故障不应该简单自动重试，因为继续训练可能浪费资源。

22.11 自动重试策略

自动重试要有边界。

重试策略可以包含：

1. max_retry_count。
2. retry_backoff。
3. retryable_error_types。
4. non_retryable_error_types。
5. resume_from_checkpoint。

6. node_blacklist。
7. timeout。

示例：

```
retry:
  max_attempts: 3
  retryable_errors:
    - node_failure
    - transient_network_error
    - image_pull_timeout
  non_retryable_errors:
    - config_error
    - permission_denied
    - repeated_oom
  backoff_seconds: 300
```

自动重试必须记录 attempt_id，避免日志和指标混乱。

22.12 从 checkpoint 恢复

从 checkpoint 恢复流程：

1. 找到最新 committed checkpoint。
2. 校验 manifest 和 checksum。
3. 检查配置兼容性。
4. 重新申请资源。
5. 启动新 attempt。
6. 加载 checkpoint。
7. 恢复 step、optimizer、scheduler、random state。
8. 记录恢复事件。

恢复后要观察：

1. loss 是否连续。
2. learning rate 是否正确。
3. 数据位置是否正确。
4. step 是否正确。
5. GPU 利用率是否正常。

恢复成功不只是进程跑起来，还要训练状态正确。

22.13 故障诊断报告

训练失败后，平台应该生成诊断报告。

报告包括：

1. 失败时间。
2. 失败阶段。
3. 首个失败 rank。
4. 错误类型。
5. 错误摘要。
6. 相关日志链接。
7. 相关指标截图。
8. 最近 checkpoint。
9. 是否已重试。
10. 建议动作。

示例建议：

错误类型: GPU OOM

失败 rank: rank 3

失败 step: 420

显存峰值: 79.4GB / 80GB

建议: 降低 micro_batch_size, 开启 activation checkpointing, 或使用 ZeRO stage 3。

诊断报告能显著降低用户和平台团队沟通成本。

22.14 故障隔离和黑名单

如果某个节点或 GPU 多次导致失败, 平台应该隔离。

机制包括:

1. node blacklist。
2. GPU blacklist。
3. 自动 cordon 节点。
4. 标记 unhealthy。
5. 禁止新任务调度。
6. 通知运维维修。

同时要避免误伤。

例如一次用户代码 OOM 不应该把节点拉黑。

只有硬件错误、系统错误或多任务重复失败才适合隔离。

22.15 容错和用户体验

容错系统要让用户知道发生了什么。

用户应该能看到:

1. 任务失败原因。
2. 是否自动重试。
3. 第几次 attempt。
4. 从哪个 checkpoint 恢复。
5. 损失了多少 step。
6. 当前是否需要用户操作。

不要只显示:

Job failed

这对用户没有帮助。

22.16 训练容错审计指标与最小 demo

这一节把前面的经验规则收敛成一个可检查的训练容错审计表。真实平台可以接入 Kubernetes Job 事件、PyTorch Elastic attempt、NCCL 日志、GPU 健康事件、PyTorch CUDA OOM 统计、checkpoint manifest 和训练状态机; 教学 demo 里先用 toy 字段模拟这些证据。

一个训练容错样本可以抽象成:

$f_i = (c_i, r_i, a_i, b_i, k_i, n_i, g_i, h_i, o_i, d_i, p_i, s_i, u_i, t_i, z_i)$

其中, c_i 是故障分类, r_i 是 retryable / non-retryable 判断, a_i 是 attempt 预算, b_i 是 backoff, k_i 是 checkpoint 恢复证据, n_i 是节点替换证据, g_i 是 GPU 健康隔离证据, h_i 是 rank heartbeat / hang 证据, o_i 是 OOM 处理策略, d_i 是数据权限和数据读取证据, p_i 是 checkpoint committed / manifest / checksum 证据, s_i 是数值稳定性证据, u_i 是用户可见动作, t_i 是诊断 trace, z_i 是最终门禁。

统一覆盖率仍然可以写成:

$$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(f_i)=1]$$

其中， N 是审计样本数， $g_j(f_i)=1$ 表示第 i 个样本通过第 j 个训练容错检查。

失败后的最大丢失 GPU-hours 可以写成：

$$L_{\{\mathrm{gpu}\}} = G \frac{t_{\{\mathrm{fail}\}} - t_{\{\mathrm{ckpt}\}}}{60}$$

其中， G 是训练使用的 GPU 数， t_{fail} 和 t_{ckpt} 用分钟表示， L_{gpu} 越大，说明 checkpoint 间隔、失败检测或恢复路径越贵。

重试成功率和无效重试率可以写成：

$$R_{\{\mathrm{success}\}} = \frac{N_{\{\mathrm{recovered}\}}}{N_{\{\mathrm{retry}\}}}$$

$$R_{\{\mathrm{waste}\}} = \frac{N_{\{\mathrm{bad_retry}\}}}{N_{\{\mathrm{retry}\}}}$$

前者看自动恢复是否真的有效，后者看平台是否在对不可恢复故障浪费 GPU。

OOM 显存余量可以写成：

$$H_{\{\mathrm{mem}\}} = 1 - \frac{M_{\{\mathrm{peak}\}}}{M_{\{\mathrm{limit}\}}}$$

如果 H_{mem} 接近 0，说明任务已经贴着显存上限运行；原样重试通常没有意义。

rank hang 的心跳年龄可以写成：

$$A_{\{\mathrm{hb}\}} = t_{\{\mathrm{now}\}} - t_{\{\mathrm{last_heartbeat}\}}$$

如果 A_{hb} 超过阈值，要结合首个失败 rank、NCCL 日志、worker 退出状态、节点健康和 OOM 事件判断根因，不能只把所有 hang 都归因于网络。

从 checkpoint 恢复后的连续性可以写成：

$$V_{\{\mathrm{resume}\}} = \mathbf{1}[\ell_{\{\mathrm{after}\}} - \ell_{\{\mathrm{before}\}}| \leq \epsilon_{\ell} \wedge s_{\{\mathrm{resume}\}} - s_{\{\mathrm{before}\}} \leq \epsilon_s]$$

其中， ℓ 是 loss， s 是 step。恢复成功不是进程重新启动，而是 loss、step、learning rate、optimizer、scheduler、RNG 和 dataloader cursor 都合理连续。

最终训练容错门禁可以写成：

$$G_{\{\mathrm{fault}\}} = \mathbf{1}[\min_j C_j \geq \tau_j \wedge L_{\{\mathrm{gpu}\}} \leq B_{\{\mathrm{loss}\}} \wedge R_{\{\mathrm{waste}\}} \leq \tau_R]$$

这个门禁背后的含义是：训练容错不是“失败了再拉起来”，而是故障分类、重试预算、checkpoint 恢复、节点和 GPU 隔离、通信 hang 诊断、OOM 快速失败、数值异常回滚、诊断报告和用户动作都可审计。

下面的 0 依赖 Python demo 演示一个最小训练容错审计器。它不是生产平台实现，而是把面试中最容易说散的规则变成可运行检查。

```
from copy import deepcopy
```

```
GATES = [
    "fault_taxonomy_defined",
    "retryability_classification",
    "retry_budget_bounded",
    "backoff_policy_defined",
    "checkpoint_resume_readiness",
    "node_failure_reschedule",
    "gpu_health_isolation",
    "communication_hang_detection",
    "oom_handling_policy",
    "data_permission_fast_fail",
    "checkpoint_commit_safety",
    "numeric_instability_stop_rollback",
    "diagnosis_report_evidence",
```

```

    "blacklist_precision",
    "user_action_clarity",
    "training_fault_tolerance_gate",
]

```

```

def base_case():
    return {
        "name": "complete_fault_tolerance_case",
        "fault_type": "node_failure",
        "stage": "runtime",
        "taxonomy": {
            "node_failure": ["runtime", "infrastructure"],
            "gpu_xid": ["runtime", "hardware"],
            "transient_network_error": ["distributed_init", "runtime"],
            "oom": ["runtime", "resource"],
            "permission_denied": ["preflight", "data"],
            "checkpoint_uncommitted": ["resume", "checkpoint"],
            "loss_nan": ["runtime", "numeric"],
        },
        "retryable": True,
        "retryable_errors": ["node_failure", "gpu_xid", "transient_network_error", "image_pull_timeout"],
        "non_retryable_errors": ["config_error", "permission_denied", "repeated_oom", "checkpoint_uncommitted", "loss_nan"],
        "attempt": 2,
        "max_attempts": 3,
        "backoff_seconds": 300,
        "min_backoff_seconds": 60,
        "max_backoff_seconds": 1800,
        "jitter": True,
        "checkpoint": {
            "latest_committed": True,
            "manifest_valid": True,
            "checksum_valid": True,
            "config_compatible": True,
            "restore_uncommitted": False,
            "resume_fields": ["model", "optimizer", "scheduler", "random_state", "data_loader_cursor"],
        },
        "node": {
            "failed_node": "node-a",
            "rescheduled_node": "node-b",
            "cordoned_nodes": ["node-a"],
            "worker_group_restarted": True,
        },
        "gpu": {
            "error_codes": ["Xid 79"],
            "isolation_policy": True,
            "node_marked_unhealthy": True,
            "repeated_hardware_failures": 2,
            "false_positive_guard": True,
            "blacklist_user_code_oom": False,
        },
        "communication": {
            "heartbeat_age_s": 12,
            "heartbeat_timeout_s": 60,
            "first_failed_rank": 3,
        },
    }

```

```

    "nccl_logs_collected": True,
    "worker_exit_status_collected": True,
},
"oom": {
    "kind": "none",
    "blind_retry": False,
    "peak_gib": 78.0,
    "limit_gib": 80.0,
    "auto_batch_decrease_requires_opt_in": True,
    "recommendations": ["lower micro_batch_size", "enable activation checkpointing", "use FSDP or ZeRO"],
},
"data": {
    "permission_checked": True,
    "retry_permission_denied": False,
    "sample_read_test": True,
},
"numeric": {
    "non_finite_detected": False,
    "blind_retry_on_nan": False,
    "rollback_checkpoint": True,
    "diagnostics_saved": True,
},
"diagnosis_report": {
    "fields": [
        "failed_at",
        "stage",
        "first_failed_rank",
        "error_type",
        "checkpoint_id",
        "attempt_id",
        "log_links",
        "metric_links",
        "recommended_action",
    ]
},
"user_action": {
    "visible_status": "retrying",
    "attempt_visible": True,
    "checkpoint_visible": True,
    "lost_steps_visible": True,
    "recommendation": "Platform retried on a healthy node from committed checkpoint ckpt-1020.",
},
"history": {"retry_total": 5, "recovered": 4, "bad_retries": 1},
"gpu_count": 64,
"fail_time_min": 186,
"checkpoint_time_min": 180,
"loss_before": 1.920,
"loss_after": 1.921,
"loss_epsilon": 0.01,
"checkpoint_step": 1000,
"step_after": 1020,
"delta_step": 20,
"training_fault_tolerance_gate": True,
}

```

```

def contains_all(values, required):
    return set(required).issubset(set(values))

def evaluate(case):
    required_resume = ["model", "optimizer", "scheduler", "random_state", "dataloader_cursor"]
    required_report = [
        "failed_at",
        "stage",
        "first_failed_rank",
        "error_type",
        "checkpoint_id",
        "attempt_id",
        "log_links",
        "metric_links",
        "recommended_action",
    ]

    retryable_expected = case["fault_type"] in case["retryable_errors"]
    communication = case["communication"]
    checkpoint = case["checkpoint"]
    node = case["node"]
    gpu = case["gpu"]
    oom = case["oom"]
    numeric = case["numeric"]

    checks = {}
    checks["fault_taxonomy_defined"] = (
        case["fault_type"] in case["taxonomy"]
        and case["stage"] in case["taxonomy"][case["fault_type"]]
    )
    checks["retryability_classification"] = (
        case["retryable"] == retryable_expected
        and not (case["retryable"] and case["fault_type"] in case["non_retryable_errors"])
    )
    checks["retry_budget_bounded"] = (
        isinstance(case["max_attempts"], int)
        and 1 <= case["max_attempts"] <= 5
        and 1 <= case["attempt"] <= case["max_attempts"]
    )
    checks["backoff_policy_defined"] = (
        case["min_backoff_seconds"] <= case["backoff_seconds"] <= case["max_backoff_seconds"]
        and case["jitter"]
    )
    checks["checkpoint_resume_readiness"] = (
        checkpoint["latest_committed"]
        and checkpoint["manifest_valid"]
        and checkpoint["checksum_valid"]
        and checkpoint["config_compatible"]
        and contains_all(checkpoint["resume_fields"], required_resume)
    )
    checks["node_failure_reschedule"] = (
        case["fault_type"] != "node_failure"
        or (

```

```

        node["worker_group_restarted"]
        and node["failed_node"] != node["rescheduled_node"]
        and node["failed_node"] in node["cordoned_nodes"]
    )
)
checks["gpu_health_isolation"] = (
    not gpu["error_codes"]
    or (
        gpu["isolation_policy"]
        and gpu["node_marked_unhealthy"]
        and gpu["repeated_hardware_failures"] >= 2
    )
)
checks["communication_hang_detection"] = (
    communication["heartbeat_age_s"] <= communication["heartbeat_timeout_s"]
    or (
        communication["first_failed_rank"] is not None
        and communication["nccl_logs_collected"]
        and communication["worker_exit_status_collected"]
    )
)
checks["oom_handling_policy"] = (
    oom["kind"] == "none"
    or (
        oom["kind"] in ["gpu", "cpu", "container"]
        and not oom["blind_retry"]
        and (oom["recommendations"] or oom["auto_batch_decrease_requires_opt_in"])
    )
)
checks["data_permission_fast_fail"] = (
    case["data"]["permission_checked"]
    and case["data"]["sample_read_test"]
    and not case["data"]["retry_permission_denied"]
)
checks["checkpoint_commit_safety"] = (
    checkpoint["latest_committed"]
    and checkpoint["manifest_valid"]
    and checkpoint["checksum_valid"]
    and not checkpoint["restore_uncommitted"]
)
checks["numeric_instability_stop_rollback"] = (
    not numeric["blind_retry_on_nan"]
    and (not numeric["non_finite_detected"] or (numeric["rollback_checkpoint"] and numeric["diagnostics_saved"]))
)
checks["diagnosis_report_evidence"] = contains_all(case["diagnosis_report"]["fields"], required_report)
checks["blacklist_precision"] = (
    gpu["false_positive_guard"]
    and not gpu["blacklist_user_code_oom"]
    and (not gpu["error_codes"] or gpu["repeated_hardware_failures"] >= 2)
)
checks["user_action_clarity"] = (
    bool(case["user_action"]["visible_status"])
    and case["user_action"]["attempt_visible"]
    and case["user_action"]["checkpoint_visible"]
    and case["user_action"]["lost_steps_visible"]
)

```

```

        and bool(case["user_action"]["recommendation"])
    )
    checks["training_fault_tolerance_gate"] = bool(case["training_fault_tolerance_gate"])
    return checks

def summarize_examples(case):
    history = case["history"]
    retry_total = max(history["retry_total"], 1)
    lost_gpu_hours = case["gpu_count"] * (case["fail_time_min"] - case["checkpoint_time_min"]) / 60
    oom_headroom = 1 - oom_ratio(case)
    resume_continuity = (
        abs(case["loss_after"] - case["loss_before"]) <= case["loss_epsilon"]
        and case["step_after"] == case["checkpoint_step"] + case["delta_step"]
    )
    return {
        "lost_gpu_hours": round(lost_gpu_hours, 3),
        "retry_success_rate": round(history["recovered"] / retry_total, 3),
        "waste_retry_rate": round(history["bad_retries"] / retry_total, 3),
        "oom_headroom": round(oom_headroom, 3),
        "heartbeat_age_s": case["communication"]["heartbeat_age_s"],
        "resume_continuity": resume_continuity,
    }

def oom_ratio(case):
    limit = max(case["oom"]["limit_gib"], 1e-9)
    return case["oom"]["peak_gib"] / limit

def mutate(name, editor):
    case = deepcopy(base_case())
    case["name"] = name
    editor(case)
    return case

def build_cases():
    cases = [base_case()]
    cases.append(mutate("fault_taxonomy_missing_bad", lambda c: c.update({"fault_type": "mystery_exit", "retryable":
    cases.append(mutate("retryability_wrong_bad", lambda c: c.update({"fault_type": "permission_denied", "stage": "pr
    cases.append(mutate("retry_budget_missing_bad", lambda c: c.update({"max_attempts": None})))
    cases.append(mutate("backoff_missing_bad", lambda c: c.update({"backoff_seconds": 0, "jitter": False})))
    cases.append(mutate("checkpoint_resume_missing_bad", lambda c: c["checkpoint"].update({"resume_fields": ["model"]
    cases.append(mutate("node_failure_not_rescheduled_bad", lambda c: c["node"].update({"rescheduled_node": "node-a",
    cases.append(mutate("gpu_health_not_isolated_bad", lambda c: c["gpu"].update({"isolation_policy": False, "node_ma
    cases.append(mutate("nccl_hang_unclassified_bad", lambda c: c["communication"].update({"heartbeat_age_s": 180, "f
    cases.append(mutate("oom_blind_retry_bad", lambda c: c["oom"].update({"kind": "gpu", "blind_retry": True, "recomm
    cases.append(mutate("data_permission_retried_bad", lambda c: c["data"].update({"retry_permission_denied": True})))
    cases.append(mutate("checkpoint_uncommitted_recovered_bad", lambda c: c["checkpoint"].update({"restore_uncommite
    cases.append(mutate("numeric_nan_retried_bad", lambda c: c["numeric"].update({"non_finite_detected": True, "blind
    cases.append(mutate("diagnosis_report_missing_bad", lambda c: c["diagnosis_report"].update({"fields": ["failed_at
    cases.append(mutate("blacklist_false_positive_bad", lambda c: c["gpu"].update({"blacklist_user_code_oom": True, "
    cases.append(mutate("user_action_unclear_bad", lambda c: c["user_action"].update({"visible_status": "", "recommen
    cases.append(mutate("fault_tolerance_gate_missing_bad", lambda c: c.update({"training_fault_tolerance_gate": Fals

```

return cases

```
def main():
    cases = build_cases()
    evaluations = {case["name"]: evaluate(case) for case in cases}
    metrics = {
        gate: round(sum(1 for case in cases if evaluations[case["name"]][gate]) / len(cases), 3)
        for gate in GATES
    }
    failed_cases = [
        case["name"]
        for case in cases
        if not all(evaluations[case["name"]].values())
    ]
    failed_gates = [
        gate
        for gate in GATES
        if any(not evaluations[case["name"]][gate] for case in cases)
    ]
    complete = cases[0]
    smoke = {
        "complete_case_passes": all(evaluations[complete["name"]].values()),
        "caught_oom_blind_retry": not evaluations["oom_blind_retry_bad"]["oom_handling_policy"],
        "caught_uncommitted_checkpoint": not evaluations["checkpoint_uncommitted_recovered_bad"]["checkpoint_commit_timeout"],
        "caught_nan_retry": not evaluations["numeric_nan_retried_bad"]["numeric_instability_stop_rollback"],
        "caught_blacklist_false_positive": not evaluations["blacklist_false_positive_bad"]["blacklist_precision"],
        "caught_gate_gap": not evaluations["fault_tolerance_gate_missing_bad"]["training_fault_tolerance_gate"],
    }
    training_fault_tolerance_gate_pass = not failed_cases and all(v >= 1.0 for v in metrics.values())

    print("training_fault_tolerance_examples=", summarize_examples(complete))
    print("smoke=", smoke)
    print("metrics=", metrics)
    print("hard_blocker_count=", len(failed_cases))
    print("failed_cases=", failed_cases)
    print("failed_gates=", failed_gates)
    print("training_fault_tolerance_gate_pass=", training_fault_tolerance_gate_pass)
```

```
if __name__ == "__main__":  
    main()
```

预期输出类似：

```
training_fault_tolerance_examples= {'lost_gpu_hours': 6.4, 'retry_success_rate': 0.8, 'waste_retry_rate': 0.2, 'oom_headroom': 1.0, 'smoke= {'complete_case_passes': True, 'caught_oom_blind_retry': True, 'caught_uncommitted_checkpoint': True, 'caught_nanosync': True, 'metrics= {'fault_taxonomy_defined': 0.941, 'retryability_classification': 0.941, 'retry_budget_bounded': 0.941, 'backoff_policy_defined': 0.941, 'hard_blocker_count= 16
failed_cases= ['fault_taxonomy_missing_bad', 'retryability_wrong_bad', 'retry_budget_missing_bad', 'backoff_missing_bad', 'backoff_policy_defined_bad']
failed_gates= ['fault_taxonomy_defined', 'retryability_classification', 'retry_budget_bounded', 'backoff_policy_defined']
training fault tolerance gate pass= False
```

面试里可以把这个 demo 压缩成一句话：训练容错的工程质量，要用“可恢复故障恢复成功、不可恢复故障快速失败、恢复后状态连续、坏节点和坏 GPU 不反复伤害任务、用户能看到明确动作”来证明，而不是用“平台会自动重试”来证明。

22.17 面试中如何回答训练容错

如果面试官问：

大模型训练平台如何做容错和自动重试？

可以这样回答：

我会先做故障分类，而不是所有失败都自动重试。故障可以分为节点/GPU 故障、通信故障、存储和数据故障、OOM、用户代码错误、酉

对节点故障、临时网络问题、Pod 驱逐、镜像拉取超时这类可恢复故障，可以自动重试，并从最新 committed checkpoint 恢复。对配

平台需要 checkpoint manager 支持 manifest、checksum、committed 状态和恢复校验；调度层重新分配健康节点；观测系统记录 att

自动重试要有 max_attempts、backoff、retryable error list 和 node blacklist。任务恢复后要检查 loss、step、learning rate 和

22.18 常见误区

误区一：自动重试越多越可靠。

盲目重试会浪费 GPU。必须区分可恢复和不可恢复错误。

误区二：有 checkpoint 就一定能恢复。

Checkpoint 可能不完整、不兼容或恢复后状态异常，必须校验和测试。

误区三：OOM 可以直接重试。

原样重试通常还会 OOM，需要调整 batch、显存策略或配置。

误区四：NCCL timeout 一定是网络问题。

也可能是某个 rank OOM、节点故障、配置不一致或端口问题。

误区五：失败日志足够定位问题。

还需要指标、事件、rank 信息、节点健康和 checkpoint 状态。

22.19 面试题

题 1：哪些训练故障适合自动重试？

答：节点故障、Pod 驱逐、临时网络抖动、存储短暂不可用、镜像拉取超时、可抢占任务中断等适合自动重试。配置错误、权限错误、代码 bug、持续 OOM、checkpoint 不兼容和数值发散不适合盲目重试。

题 2：OOM 应该如何处理？

答：先区分 GPU OOM、CPU OOM 和容器内存 OOM。GPU OOM 通常需要降低 micro batch、减少 sequence length、开启 activation checkpointing、使用 ZeRO/FSDP 或更大显存 GPU。平台可以给建议，不能简单原样重试。

题 3：NCCL timeout 如何排查？

答：先看第一个失败 rank 和 NCCL 日志，再查是否有 worker OOM 或节点失联。然后看网络、RDMA、端口、rank/world size 配置、NetworkPolicy、网卡错误和通信指标。NCCL timeout 可能是通信问题，也可能是其他 rank 先失败导致。

题 4：从 checkpoint 恢复后要验证什么？

答：要验证 checkpoint manifest 和 checksum、配置兼容性、global step、optimizer、scheduler、random state、dataloader 位置、loss 连续性和 learning rate 是否正确。

题 5：故障诊断报告应该包含什么？

答：包括失败时间、失败阶段、首个失败 rank、错误类型、错误摘要、相关日志和指标、最近 checkpoint、重试次数、恢复状态和建议动作。

22.20 小练习

练习一：设计一个错误分类表。

要求：把 `node_failure`、`gpu_error`、`nccl_timeout`、`oom`、`permission_denied`、`config_error`、`loss_nan` 分成 `retryable` 和 `non_retryable`。

练习二：设计自动重试策略。

要求：包含 `max_attempts`、`backoff`、`retryable_errors`、`non_retryable_errors`、`checkpoint` 恢复和 `node blacklist`。

练习三：分析一次 NCCL timeout。

要求：从 rank、OOM、节点、网络、端口、NCCL 环境变量和调度拓扑角度排查。

练习四：设计故障诊断报告模板。

要求：包含错误摘要、根因假设、证据、相关日志、相关指标和建议动作。

22.21 本章小结

本章讲了训练容错。

你需要掌握：

1. 训练容错的核心是故障分类、损失控制、自动恢复和清晰诊断。
2. 可恢复故障和不可恢复故障要区别处理。
3. 节点和 GPU 故障适合重调度并从 `checkpoint` 恢复。
4. 通信失败要结合 rank、NCCL、网络和其他 worker 状态排查。
5. OOM 通常不适合原样重试，需要调整资源或配置。
6. 数据和 `checkpoint` 故障要有校验、重试和诊断。
7. 数值稳定性故障需要停止、诊断和可能回滚。
8. 自动重试要有边界、`backoff`、`attempt_id` 和错误类型策略。
9. 从 `checkpoint` 恢复后要验证训练状态连续性。
10. 故障诊断报告能显著提升平台用户体验。

下一章我们会讲训练数据供给：流式读取、缓存、shuffle 和数据局部性。

第 23 章 训练数据供给：流式读取、缓存、shuffle 和数据局部性

上一章讲了训练容错。本章讲训练平台中另一个决定训练效率的关键环节：数据供给。

很多训练任务 GPU 利用率低，不是模型算得慢，而是数据喂得慢。大模型训练需要持续把海量 token、样本、图像、视频或多模态数据送到 GPU。如果数据读取、预处理、shuffle、缓存任何一环跟不上，GPU 就会等待。

先记住一句话：

训练数据供给的目标，是在保证数据版本、权限、shuffle 和可复现性的前提下，把数据以足够高、足够稳定的吞吐送到训练进程。

23.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，资料口径主要对齐几类公开工程资料：PyTorch `torch.utils.data` 对 `Dataset`、`IterableDataset`、`DataLoader`、`DistributedSampler`、`num_workers`、`pin_memory`、`prefetch_factor` 和 worker 初始化的稳定抽象；Hugging Face `Datasets streaming` 对超大数据集流式读取、`IterableDataset`、`shuffle buffer`、`set_epoch`、

sharding 和 Parquet 列裁剪 / 过滤的公开口径; NVIDIA DALI 对高吞吐数据加载、CPU / GPU preprocessing、pipeline、reader、sharding 和预取的工程边界; 前文存储、checkpoint 和可观测性章节对对象存储、缓存、manifest、checksum、恢复状态和指标采集的稳定抽象。

这里不把某个数据湖、对象存储、分布式缓存产品、训练框架、文件格式、内部平台或具体吞吐数值写成通用标准。正文只抽象训练数据供给里的稳定问题: manifest、shard、格式、小文件、离线 / 在线预处理、流式读取、预取、本地缓存、分布式缓存、shuffle、rank 切分、数据局部性、DataLoader、H2D copy、坏样本、监控、checkpoint 恢复和权限版本治理。

第二轮精修重点放在三个方面:

1. 把“数据读得快”拆成可计算的吞吐、等待比例、缓存命中、rank lag、坏样本率和恢复连续性。
2. 补充 GitHub MathJax 兼容公式, 解释 shard、shuffle、缓存、DataLoader 和 checkpoint 数据状态之间的关系。
3. 增加一个 0 依赖 Python demo, 把训练数据供给从经验 checklist 变成可运行门禁检查。

23.1 为什么数据供给重要

大模型训练通常很贵。

如果 GPU 每秒能处理 10000 token, 但数据 pipeline 只能提供 6000 token, 那么 GPU 会空等。

数据供给问题会导致:

1. GPU 利用率低。
2. step time 抖动。
3. tokens/s 下降。
4. dataloader time 占比高。
5. 多个 rank 不同步。
6. 存储压力暴涨。
7. 训练任务偶发失败。

所以训练数据供给是 AI Infra 的主路径, 不是数据工程的边角问题。

23.2 训练数据供给链路

一个典型链路:

Raw Data

- > Data Lake / Object Storage
- > Cleaning / Dedup / Filtering
- > Tokenization / Transform
- > Sharding / Packing
- > Dataset Manifest
- > Cache / Local Disk
- > DataLoader
- > CPU Memory
- > GPU

每一层都可能成为瓶颈。

平台需要知道:

1. 数据从哪里来。
2. 使用哪个版本。
3. 预处理规则是什么。
4. shard 如何组织。
5. cache 是否命中。
6. dataloader 是否够快。
7. 数据是否正确送到 GPU。

23.3 离线预处理和在线预处理

数据预处理可以分为离线和在线。

离线预处理：训练前完成。

包括：

1. 清洗。
2. 去重。
3. 过滤。
4. tokenization。
5. packing。
6. shard 生成。
7. index 生成。

优点：

1. 训练时更快。
2. 数据版本更稳定。
3. 便于复现。
4. 减少 CPU 压力。

缺点：

1. 预处理耗时。
2. 需要额外存储。
3. 数据更新不够实时。

在线预处理：训练时动态处理。

优点：

1. 灵活。
2. 节省预处理产物存储。
3. 适合快速实验。

缺点：

1. CPU 容易成为瓶颈。
2. 训练吞吐不稳定。
3. 可复现性更难。
4. 错误更晚暴露。

大规模训练通常倾向离线预处理，小实验可以保留在线灵活性。

23.4 数据格式和 shard

训练数据格式会影响吞吐。

常见格式：

1. JSONL。
2. Parquet。
3. Arrow。
4. WebDataset。
5. TFRecord。
6. 自定义二进制格式。

小文件太多会严重拖慢训练。

因此大规模训练常把数据打成 shard。

Shard 的好处：

1. 减少文件数量。
2. 提高顺序读取效率。
3. 降低 metadata 压力。
4. 便于分布式切分。
5. 便于缓存和预取。

Shard 设计要考虑：

1. 单个 shard 大小。
2. shard 数量。
3. 每个 shard 样本数。
4. 是否压缩。
5. 是否带 index。
6. 是否可随机访问。

Shard 太小会有小文件问题，太大又不利于并行和恢复。

23.5 Dataset Manifest

Dataset manifest 是数据集的目录和说明书。

它记录：

1. dataset_id。
2. version。
3. shard 列表。
4. 每个 shard 大小。
5. 样本数。
6. token 数。
7. checksum。
8. 数据来源。
9. preprocessing_version。
10. 权限和数据等级。

示例：

```
{
  "dataset_id": "web_text_pretrain",
  "version": "2026-05-30",
  "preprocessing_version": "dedup_v4_tokenize_v2",
  "shards": [
    {
      "uri": "s3://datasets/web_text/shard_00001.bin",
      "num_tokens": 120000000,
      "checksum": "sha256:..."
    }
  ]
}
```

没有 manifest，训练平台很难保证数据版本可复现。

23.6 流式读取

流式读取是从远端存储或数据服务边读边训。

适合：

1. 数据集很大。
2. 本地盘放不下。
3. 数据持续更新。

4. 多任务共享数据。

挑战：

1. 网络波动影响训练。
2. 存储吞吐成为瓶颈。
3. 随机读取效率低。
4. cache 命中率关键。
5. 错误恢复复杂。

流式读取要配合：

1. 预取。
2. 本地缓存。
3. 分布式缓存。
4. 重试。
5. shard 顺序规划。
6. 吞吐监控。

否则很容易拖慢 GPU。

23.7 本地缓存

本地缓存是把热点数据放到训练节点本地盘。

优点：

1. 读取快。
2. 减少远端存储压力。
3. 降低网络波动影响。
4. 提高训练稳定性。

挑战：

1. 本地盘容量有限。
2. 缓存一致性管理复杂。
3. 节点故障后缓存丢失。
4. 多任务缓存竞争。

常见策略：

1. 任务启动前预热数据。
2. 按 shard 缓存。
3. LRU 清理。
4. 按数据版本隔离缓存。
5. 缓存命中率监控。

本地缓存适合重复使用的数据集和热点 shard。

23.8 分布式缓存

分布式缓存位于远端存储和训练节点之间。

它可以：

1. 缓解对象存储压力。
2. 提高多节点读取吞吐。
3. 减少重复下载。
4. 支持跨任务共享热点数据。

挑战：

1. 部署和运维复杂。

2. 容量规划困难。
3. 一致性和版本隔离要做好。
4. 可能成为新的瓶颈。

分布式缓存适合大规模预训练和多任务共享数据场景。

23.9 Shuffle

Shuffle 是训练数据随机化的重要环节。

Shuffle 不好会影响训练质量。

常见 shuffle 层次：

1. shard 级 shuffle。
2. shard 内样本 shuffle。
3. batch 内 shuffle。
4. 多数据源采样 shuffle。

大规模流式读取中，全局完全随机 shuffle 成本很高。

常见做法是近似 shuffle：

1. 随机打乱 shard 顺序。
2. 使用 shuffle buffer。
3. 每个 epoch 使用不同 seed。
4. 多 rank 分配不同 shard。

Shuffle 要兼顾：

1. 随机性。
2. 吞吐。
3. 可复现。
4. 数据均衡。

23.10 多 rank 数据切分

分布式训练中，不同 rank 不能重复处理同一批数据，除非算法允许。

需要处理：

1. rank 到 shard 的分配。
2. epoch 边界。
3. 数据不足时是否 drop。
4. 断点恢复时数据位置。
5. world size 变化时重新切分。

常见问题：

1. 不同 rank 读到重复数据。
2. 某些 shard 被漏读。
3. 某个 rank 数据读取慢。
4. 恢复后重复训练一段数据。
5. 数据分布不均导致训练偏差。

平台要记录 dataloader state，支持恢复。

23.11 数据局部性

数据局部性是指计算尽量靠近数据。

在训练中，数据局部性包括：

1. 数据是否在本地缓存。
2. 数据是否在同机柜存储节点。
3. 数据是否在同区域对象存储。
4. 训练节点到数据路径是否拥塞。

数据局部性差会导致：

1. 读取延迟高。
2. 跨区域流量贵。
3. 网络拥塞。
4. step time 抖动。

调度系统可以利用数据局部性：

1. 优先调度到有缓存的节点。
2. 把同一数据集任务放在缓存附近。
3. 避免跨区域读取训练数据。
4. 对热点数据提前预热。

23.12 Dataloader 性能

Dataloader 是训练数据到模型之间的最后一段。

常见瓶颈：

1. worker 数太少。
2. CPU 预处理太重。
3. Python 解析慢。
4. 解压耗时。
5. tokenization 在线执行。
6. batch padding 浪费。
7. CPU 到 GPU 拷贝慢。

优化方法：

1. 增加 worker。
2. 预处理离线化。
3. 使用二进制格式。
4. 使用 pinned memory。
5. 异步预取。
6. 减少动态解析。
7. 合理 batch packing。

数据 pipeline 优化常常比改模型更能提升 GPU 利用率。

23.13 坏样本处理

训练数据中可能有坏样本。

例如：

1. 格式错误。
2. 解码失败。
3. 空文本。
4. token 超长。
5. 图片损坏。
6. 视频无法读取。
7. 标签缺失。

处理策略：

1. 离线校验。

2. 训练中跳过并记录。
3. 超过阈值失败。
4. 记录坏样本位置。
5. 生成数据质量报告。

不能无限跳过坏样本，否则训练数据分布可能被悄悄改变。

23.14 数据供给监控

关键指标：

1. dataloader time。
2. data wait time。
3. storage read throughput。
4. cache hit rate。
5. shard read latency。
6. bad sample count。
7. tokens loaded per second。
8. CPU preprocessing time。
9. H2D copy time。
10. per-rank data lag。

告警场景：

1. dataloader time 占 step time 过高。
2. cache hit rate 下降。
3. 某个 rank 数据读取明显慢。
4. 坏样本比例过高。
5. 存储吞吐接近上限。
6. 数据读取错误率上升。

这些指标能帮助判断 GPU 是不是在等数据。

23.15 数据供给和 checkpoint 恢复

断点恢复时，不只要恢复模型，也要恢复数据位置。

否则可能：

1. 重复训练一段数据。
2. 跳过一段数据。
3. shuffle 顺序变化。
4. 多 rank 数据分配变化。

Checkpoint 应记录：

1. epoch。
2. global step。
3. shard index。
4. sample offset。
5. shuffle seed。
6. dataloader worker state。

严格复现要求更高，平台要根据任务需求选择支持程度。

23.16 训练数据供给审计指标与最小 demo

这一节把训练数据供给改写成可审计指标。真实平台可以从 DataLoader profile、对象存储日志、缓存服务、dataset manifest、训练事件、rank trace 和 checkpoint metadata 中取数；教学 demo 里先用 toy 字段模拟。

一个训练数据供给样本可以抽象成：

$d_i = (m_i, s_i, a_i, u_i, c_i, q_i, r_i, \ell_i, p_i, h_i, b_i, o_i, k_i, v_i, z_i)$

其中, m_i 是 manifest 证据, s_i 是 shard 和格式, a_i 是小文件和 metadata 压力, u_i 是流式读取和预取, c_i 是缓存, q_i 是 shuffle, r_i 是 rank 切分, ℓ_i 是数据局部性, p_i 是 DataLoader 并行与预处理, h_i 是 H2D copy, b_i 是坏样本, o_i 是观测指标, k_i 是 checkpoint 数据状态, v_i 是权限和版本治理, z_i 是最终门禁。

统一覆盖率可以写成:

$$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(d_i)=1]$$

数据供给吞吐可以写成:

$$Q_{\{\mathrm{data}\}} = \frac{N_{\{\mathrm{tok}\}}}{T_{\{\mathrm{load}\}}}$$

其中, N_{tok} 是数据 pipeline 在时间窗口内真正送到训练循环的 token 数, T_{load} 是对应的数据加载耗时。

GPU 等数据的比例可以写成:

$$R_{\{\mathrm{wait}\}} = \frac{T_{\{\mathrm{data_wait}\}}}{T_{\{\mathrm{step}\}}}$$

如果 R_{wait} 长期很高, 说明瓶颈可能在对象存储、缓存、解析、预处理、DataLoader worker、H2D copy 或 rank 不均衡, 而不一定在模型计算。

缓存命中率可以写成:

$$R_{\{\mathrm{cache}\}} = \frac{B_{\{\mathrm{cache}\}}}{B_{\{\mathrm{cache}\}} + B_{\{\mathrm{remote}\}}}$$

其中, B_{cache} 是从本地或分布式缓存读取的字节数, B_{remote} 是从远端存储读取的字节数。

rank 间数据滞后可以写成:

$$A_{\{\mathrm{rank}\}} = \max_r T_{\{\mathrm{data}\},r} - \min_r T_{\{\mathrm{data}\},r}$$

如果某个 rank 数据读取慢, 其他 rank 可能在 collective 前等待, 最后表现成 GPU 利用率低或 NCCL 同步变慢。

坏样本率可以写成:

$$R_{\{\mathrm{bad}\}} = \frac{N_{\{\mathrm{bad}\}}}{N_{\{\mathrm{seen}\}}}$$

坏样本不是越过越好。超过阈值时应该失败并输出数据质量报告, 否则训练分布会被悄悄改变。

近似 shuffle 的桶分布熵可以写成:

$$H_{\{\mathrm{shuffle}\}} = -\frac{1}{\log K} \sum_{k=1}^K p_k \log p_k$$

其中, K 是桶数, p_k 是第 k 个来源桶在一个窗口中的比例。这个指标不能证明全局完全随机, 但能发现明显的 shard 顺序偏置或单来源连续读取。

最终训练数据供给门禁可以写成:

$$G_{\{\mathrm{data}\}} = \mathbf{1}[\left[\min_j C_j \geq \tau_j \ \wedge \ Q_{\{\mathrm{data}\}} \geq \gamma \ Q_{\{\mathrm{train}\}} \ \wedge \ R_{\{\mathrm{m}\}} \geq \delta \right]]$$

这个门禁背后的含义是: 数据供给不是“能读到文件”, 而是 manifest、shard、缓存、shuffle、rank 切分、DataLoader、H2D、坏样本、观测指标和 checkpoint 数据状态都要可审计。

下面的 0 依赖 Python demo 演示一个最小训练数据供给审计器。

```
from copy import deepcopy
```

```
GATES = [
    "dataset_manifest_integrity",
    "shard_format_size_fit",
    "small_file_amplification_control",
    "streaming_prefetch_retry",
    "local_cache_policy",
```

```

    "distributed_cache_policy",
    "shuffle_reproducibility",
    "rank_shard_assignment",
    "data_locality_fit",
    "dataloader_parallelism_fit",
    "h2d_copy_readiness",
    "bad_sample_threshold",
    "data_supply_observability",
    "checkpoint_dataloader_state",
    "permission_version_governance",
    "training_data_supply_gate",
]

```

```

def base_case():
    return {
        "name": "complete_data_supply_case",
        "manifest": {
            "dataset_id": "web_text_pretrain",
            "version": "2026-05-30",
            "immutable_version": True,
            "preprocessing_version": "dedup_v4_tokenize_v2_pack_v3",
            "shard_count": 64,
            "checksum_coverage": 1.0,
            "permission_metadata": True,
            "token_count": 320_000_000_000,
        },
        "shards": {
            "format": "packed_binary",
            "avg_size_mib": 512,
            "min_size_mib": 420,
            "max_size_mib": 620,
            "indexed": True,
            "compressionSplittable": True,
            "total_files": 64,
            "metadata_open_qps": 32,
        },
        "streaming": {
            "enabled": True,
            "prefetch_mib": 2048,
            "retry_policy": True,
            "timeout_s": 60,
            "observed_mib_s": 640,
            "required_mib_s": 512,
        },
        "local_cache": {
            "enabled": True,
            "hit_bytes": 800,
            "remote_bytes": 200,
            "version_isolation": True,
            "lru_eviction": True,
            "capacity_ok": True,
        },
        "distributed_cache": {
            "enabled": True,

```

```

    "hot_shard_shared": True,
    "version_consistent": True,
    "capacity_ok": True,
    "not_single_bottleneck": True,
},
"shuffle": {
    "shard_shuffle": True,
    "buffer_size": 100_000,
    "seed_recorded": True,
    "set_epoch": True,
    "bucket_entropy": 0.96,
},
"rank_assignment": {
    "world_size": 8,
    "rank_shard_counts": [8, 8, 8, 8, 8, 8, 8, 8],
    "duplicate_shards": 0,
    "missing_shards": 0,
    "resume_cursor_saved": True,
},
"locality": {
    "same_region": True,
    "cache_affinity": True,
    "cross_region_read": False,
    "topology_aware_schedule": True,
    "locality_score": 0.9,
},
"data_loader": {
    "num_workers": 8,
    "persistent_workers": True,
    "prefetch_factor": 4,
    "cpu_preprocess_ms": 80,
    "data_loader_ms": 140,
    "step_ms": 1200,
},
"h2d": {
    "pin_memory": True,
    "async_copy": True,
    "copy_ms": 18,
    "copy_budget_ms": 40,
},
"bad_samples": {
    "seen": 1_000_000,
    "bad": 1000,
    "threshold": 0.002,
    "logged_locations": True,
    "fail_over_threshold": True,
},
"observability": {
    "metrics": [
        "data_loader_time",
        "data_wait_time",
        "storage_read_throughput",
        "cache_hit_rate",
        "shard_read_latency",
        "bad_sample_count",
    ],

```

```

        "tokens_loaded_per_second",
        "cpu_preprocess_time",
        "h2d_copy_time",
        "per_rank_data_lag",
    ]
},
"checkpoint_data_state": {
    "epoch": 2,
    "global_step": 1200,
    "shard_index": 41,
    "sample_offset": 9984,
    "shuffle_seed": 20260612,
    "worker_state": True,
},
"permission_version": {
    "access_checked": True,
    "data_classification": "internal",
    "dataset_version_bound": True,
    "preprocessing_version_bound": True,
},
"tokens_loaded": 524_288,
"load_seconds": 1.2,
"train_tokens_per_second": 400_000,
"data_wait_ms": 100,
"training_data_supply_gate": True,
}

```

```

def contains_all(values, required):
    return set(required).issubset(set(values))

```

```

def rate(num, den):
    return num / max(den, 1e-9)

```

```

def evaluate(case):
    manifest = case["manifest"]
    shards = case["shards"]
    streaming = case["streaming"]
    local_cache = case["local_cache"]
    distributed_cache = case["distributed_cache"]
    shuffle = case["shuffle"]
    rank_assignment = case["rank_assignment"]
    locality = case["locality"]
    dataloader = case["dataloader"]
    h2d = case["h2d"]
    bad_samples = case["bad_samples"]
    checkpoint_state = case["checkpoint_data_state"]
    permission_version = case["permission_version"]

    shard_counts = rank_assignment["rank_shard_counts"]
    rank_balance = max(shard_counts) / max(min(shard_counts), 1)
    data_wait_ratio = rate(case["data_wait_ms"], dataloader["step_ms"])
    cache_hit = rate(local_cache["hit_bytes"], local_cache["hit_bytes"] + local_cache["remote_bytes"])

```

```

dataloader_ratio = rate(dataloader["dataloader_ms"], dataloader["step_ms"])
bad_rate = rate(bad_samples["bad"], bad_samples["seen"])

required_metrics = [
    "dataloader_time",
    "data_wait_time",
    "storage_read_throughput",
    "cache_hit_rate",
    "shard_read_latency",
    "bad_sample_count",
    "tokens_loaded_per_second",
    "cpu_preprocess_time",
    "h2d_copy_time",
    "per_rank_data_lag",
]
required_checkpoint_fields = ["epoch", "global_step", "shard_index", "sample_offset", "shuffle_seed", "worker_sta

checks = {}
checks["dataset_manifest_integrity"] = (
    manifest["immutable_version"]
    and manifest["shard_count"] > 0
    and manifest["checksum_coverage"] >= 1.0
    and bool(manifest["preprocessing_version"])
    and manifest["permission_metadata"]
)
checks["shard_format_size_fit"] = (
    shards["format"] in ["packed_binary", "parquet", "arrow", "webdataset"]
    and 128 <= shards["avg_size_mib"] <= 4096
    and shards["indexed"]
    and shards["compressionSplittable"]
)
checks["small_file_amplification_control"] = (
    rate(shards["total_files"], manifest["shard_count"]) <= 2
    and shards["metadata_open_qps"] <= 200
)
checks["streaming_prefetch_retry"] = (
    streaming["enabled"]
    and streaming["prefetch_mib"] >= 1024
    and streaming["retry_policy"]
    and streaming["timeout_s"] > 0
    and streaming["observed_mib_s"] >= streaming["required_mib_s"]
)
checks["local_cache_policy"] = (
    local_cache["enabled"]
    and local_cache["version_isolation"]
    and local_cache["lru_eviction"]
    and local_cache["capacity_ok"]
    and cache_hit >= 0.6
)
checks["distributed_cache_policy"] = (
    distributed_cache["enabled"]
    and distributed_cache["hot_shard_shared"]
    and distributed_cache["version_consistent"]
    and distributed_cache["capacity_ok"]
    and distributed_cache["not_single_bottleneck"]
)

```

```

)
checks["shuffle_reproducibility"] = (
    shuffle["shard_shuffle"]
    and shuffle["buffer_size"] >= 10_000
    and shuffle["seed_recorded"]
    and shuffle["set_epoch"]
    and shuffle["bucket_entropy"] >= 0.8
)
checks["rank_shard_assignment"] = (
    rank_assignment["duplicate_shards"] == 0
    and rank_assignment["missing_shards"] == 0
    and rank_balance <= 1.25
    and rank_assignment["resume_cursor_saved"]
)
checks["data_locality_fit"] = (
    locality["same_region"]
    and locality["cache_affinity"]
    and not locality["cross_region_read"]
    and locality["topology_aware_schedule"]
    and locality["locality_score"] >= 0.8
)
checks["dataloader_parallelism_fit"] = (
    data_loader["num_workers"] >= 4
    and data_loader["persistent_workers"]
    and data_loader["prefetch_factor"] >= 2
    and data_loader_ratio <= 0.2
)
checks["h2d_copy_readiness"] = (
    h2d["pin_memory"]
    and h2d["async_copy"]
    and h2d["copy_ms"] <= h2d["copy_budget_ms"]
)
checks["bad_sample_threshold"] = (
    bad_rate <= bad_samples["threshold"]
    and bad_samples["logged_locations"]
    and bad_samples["fail_over_threshold"]
)
checks["data_supply_observability"] = contains_all(case["observability"]["metrics"], required_metrics)
checks["checkpoint_data_loader_state"] = (
    contains_all(checkpoint_state.keys(), required_checkpoint_fields)
    and checkpoint_state["worker_state"]
)
checks["permission_version_governance"] = (
    permission_version["access_checked"]
    and permission_version["dataset_version_bound"]
    and permission_version["preprocessing_version_bound"]
    and bool(permission_version["data_classification"])
)
checks["training_data_supply_gate"] = (
    bool(case["training_data_supply_gate"])
    and data_wait_ratio <= 0.1
    and case["tokens_loaded"] / case["load_seconds"] >= case["train_tokens_per_second"]
)
return checks

```

```

def summarize_examples(case):
    shard_counts = case["rank_assignment"]["rank_shard_counts"]
    return {
        "data_tokens_per_second": round(case["tokens_loaded"] / case["load_seconds"]),
        "observed_data_mib_s": case["streaming"]["observed_mib_s"],
        "data_wait_ratio": round(rate(case["data_wait_ms"], case["data_loader"]["step_ms"]), 3),
        "cache_hit_rate": round(rate(case["local_cache"]["hit_bytes"], case["local_cache"]["hit_bytes"] + case["local_cache"]["miss_bytes"]), 3),
        "bad_sample_rate": round(rate(case["bad_samples"]["bad"], case["bad_samples"]["seen"]), 4),
        "rank_balance": round(max(shard_counts) / max(min(shard_counts), 1), 3),
    }

def mutate(name, editor):
    case = deepcopy(base_case())
    case["name"] = name
    editor(case)
    return case

def build_cases():
    cases = [base_case()]
    cases.append(mutate("manifest_missing_bad", lambda c: c["manifest"].update({"checksum_coverage": 0.5, "immutable": True})))
    cases.append(mutate("shard_format_bad", lambda c: c["shards"].update({"format": "raw_jsonl", "indexed": False})))
    cases.append(mutate("small_file_amplification_bad", lambda c: c["shards"].update({"total_files": 20000, "metadata_size": 1000000})))
    cases.append(mutate("streaming_prefetch_missing_bad", lambda c: c["streaming"].update({"prefetch_mib": 0, "retry_mib": 100})))
    cases.append(mutate("local_cache_policy_bad", lambda c: c["local_cache"].update({"hit_bytes": 100, "remote_bytes": 1000000})))
    cases.append(mutate("distributed_cache_policy_bad", lambda c: c["distributed_cache"].update({"version_consistent": False})))
    cases.append(mutate("shuffle_not_reproducible_bad", lambda c: c["shuffle"].update({"seed_recorded": False, "set_seed": True})))
    cases.append(mutate("rank_assignment_duplicate_bad", lambda c: c["rank_assignment"].update({"duplicate_shards": 2})))
    cases.append(mutate("data_locality_cross_region_bad", lambda c: c["locality"].update({"same_region": False, "cross_region": True})))
    cases.append(mutate("data_loader_parallelism_bad", lambda c: c["data_loader"].update({"num_workers": 1, "persistent_workers": True})))
    cases.append(mutate("h2d_copy_blocking_bad", lambda c: c["h2d"].update({"pin_memory": False, "async_copy": False})))
    cases.append(mutate("bad_sample_threshold_bad", lambda c: c["bad_samples"].update({"bad": 20000, "logged_location": True})))
    cases.append(mutate("observability_missing_bad", lambda c: c["observability"].update({"metrics": ["data_loader_time", "data_wait_time", "local_cache_hit_rate", "local_cache_miss_rate", "distributed_cache_hit_rate", "distributed_cache_miss_rate", "bad_sample_rate", "rank_balance"]})))
    cases.append(mutate("checkpoint_data_loader_state_missing_bad", lambda c: c["checkpoint_data_state"].pop("worker_state")))
    cases.append(mutate("permission_version_unbound_bad", lambda c: c["permission_version"].update({"access_checked": False})))
    cases.append(mutate("data_supply_gate_missing_bad", lambda c: c.update({"training_data_supply_gate": False})))
    return cases

def main():
    cases = build_cases()
    evaluations = {case["name"]: evaluate(case) for case in cases}
    metrics = {
        gate: round(sum(1 for case in cases if evaluations[case["name"]][gate]) / len(cases), 3)
        for gate in GATES
    }
    failed_cases = [
        case["name"]
        for case in cases
        if not all(evaluations[case["name"]].values())
    ]
    failed_gates = [
        gate

```

```

    for gate in GATES
    if any(not evaluations[case["name"]][gate] for case in cases)
]
complete = cases[0]
smoke = {
    "complete_case_passes": all(evaluations[complete["name"]].values()),
    "caught_small_files": not evaluations["small_file_amplification_bad"]["small_file_amplification_control"],
    "caught_cache_gap": not evaluations["local_cache_policy_bad"]["local_cache_policy"],
    "caught_shuffle_gap": not evaluations["shuffle_not_reproducible_bad"]["shuffle_reproducibility"],
    "caught_rank_overlap": not evaluations["rank_assignment_duplicate_bad"]["rank_shard_assignment"],
    "caught_checkpoint_state_gap": not evaluations["checkpoint_dataloader_state_missing_bad"]["checkpoint_dataloader_state_missing"]
}
training_data_supply_gate_pass = not failed_cases and all(v >= 1.0 for v in metrics.values())

print("training_data_supply_examples=", summarize_examples(complete))
print("smoke=", smoke)
print("metrics=", metrics)
print("hard_blocker_count=", len(failed_cases))
print("failed_cases=", failed_cases)
print("failed_gates=", failed_gates)
print("training_data_supply_gate_pass=", training_data_supply_gate_pass)

if __name__ == "__main__":
    main()

```

预期输出类似：

```

training_data_supply_examples= {'data_tokens_per_second': 436907, 'observed_data_mib_s': 640, 'data_wait_ratio': 0.083,
smoke= {'complete_case_passes': True, 'caught_small_files': True, 'caught_cache_gap': True, 'caught_shuffle_gap': True,
metrics= {'dataset_manifest_integrity': 0.941, 'shard_format_size_fit': 0.941, 'small_file_amplification_control': 0.941,
hard_blocker_count= 16
failed_cases= ['manifest_missing_bad', 'shard_format_bad', 'small_file_amplification_bad', 'streaming_prefetch_missing',
failed_gates= ['dataset_manifest_integrity', 'shard_format_size_fit', 'small_file_amplification_control', 'streaming_prefetch_missing'],
training_data_supply_gate_pass= False

```

面试里可以把这个 demo 压缩成一句话：训练数据供给要证明数据 “版本正确、读得够快、shuffle 可复现、rank 不重不漏、缓存有效、坏样本可控、恢复不乱序”，而不是只证明路径能打开。

23.17 面试中如何回答训练数据供给

如果面试官问：

如何设计大模型训练的数据供给系统？

可以这样回答：

我会把训练数据供给设计成从数据湖或对象存储到训练进程的高吞吐链路。原始数据先离线清洗、去重、过滤、tokenize、packing，

训练时根据任务规模选择流式读取、本地缓存或分布式缓存。大规模预训练会用 shard 顺序读取、预取和缓存来减少对象存储压力；

Shuffle 采用 shard 级 shuffle、shuffle buffer 和固定 seed，平衡随机性、吞吐和可复现。分布式训练中要按 rank 切分数据，避免

平台还要监控 dataloader time、data wait time、cache hit rate、storage throughput、bad sample count 和 per-rank data lag。如果 GPU 利用率低，要能判断是不是数据供给跟不上。

23.18 常见误区

误区一：数据在对象存储里，训练直接读就行。

大规模训练直接读对象存储可能受延迟、请求开销和吞吐限制，需要 shard、预取和缓存。

误区二：shuffle 越随机越好。

完全全局随机成本很高，要在随机性、吞吐和可复现性之间平衡。

误区三：跳过坏样本没影响。

坏样本比例高会改变数据分布，必须记录和设阈值。

误区四：训练恢复只需要模型 checkpoint。

严格恢复还需要数据位置、shuffle seed 和 dataloader state。

误区五：GPU 利用率低一定是模型代码问题。

也可能是数据读取、预处理、缓存、存储或 CPU 到 GPU 拷贝问题。

23.19 面试题

题 1：为什么大规模训练要做数据 shard？

答：Shard 可以减少小文件数量，提高顺序读取效率，降低 metadata 压力，便于分布式切分、缓存、预取和恢复。小文件过多会严重拖慢 dataloader。

题 2：Dataset manifest 有什么作用？

答：Manifest 记录数据集版本、shard 列表、样本数、token 数、checksum、预处理版本、数据来源和权限信息，是数据可复现、完整性校验和分布式读取的基础。

题 3：流式读取和本地缓存如何取舍？

答：流式读取适合超大数据和动态数据，但依赖网络和远端存储吞吐。本地缓存速度快、稳定，但容量有限且管理复杂。大规模训练常用对象存储作为源，本地或分布式缓存做加速。

题 4：shuffle 在流式训练中如何做？

答：通常采用近似 shuffle，例如 shard 级随机顺序、shuffle buffer、固定 seed、多 rank 不同 shard 分配。要平衡随机性、吞吐和可复现性。

题 5：如何判断 GPU 在等数据？

答：看 dataloader time、data wait time、storage throughput、cache hit rate、CPU preprocessing time、H2D copy time、per-rank data lag 和 GPU utilization。如果 dataloader 占 step time 高，可能是数据供给瓶颈。

23.20 小练习

练习一：设计一个 dataset manifest。

要求：包含 dataset_id、version、preprocessing_version、shards、num_tokens、checksum 和 data_classification。

练习二：设计一个数据缓存策略。

要求：说明哪些数据预热到本地 NVMe，哪些走流式读取，如何清理缓存。

练习三：分析 GPU 利用率低。

假设 GPU utilization 只有 45%，dataloader time 占 step time 40%，请从 shard、小文件、缓存、CPU 预处理、H2D 拷贝和存储吞吐角度排查。

练习四：设计断点恢复的数据状态。

要求：说明 checkpoint 中应该保存哪些 dataloader 和 shuffle 状态。

23.21 本章小结

本章讲了训练数据供给。

你需要掌握：

1. 数据供给目标是在保证版本、权限、shuffle 和可复现性的前提下高吞吐喂给 GPU。
2. 数据链路包括数据湖、清洗、去重、tokenization、shard、manifest、缓存、dataloader 和 GPU。
3. 大规模训练倾向离线预处理，减少训练时 CPU 和解析开销。
4. Shard 和 manifest 是训练数据供给的基础。
5. 流式读取需要预取、缓存和重试配合。
6. 本地缓存和分布式缓存能提升吞吐，但需要容量和一致性管理。
7. Shuffle 要平衡随机性、吞吐和可复现性。
8. 分布式训练要按 rank 切分数据，并支持断点恢复。
9. 坏样本要记录、限阈值和生成质量报告。
10. 数据供给监控能帮助判断 GPU 是否在等数据。

下一章我们会讲训练平台权限、安全和审计。

第 24 章 训练平台权限、安全和审计

上一章讲了训练数据供给。本章讲训练平台的权限、安全和审计。

训练平台连接了最贵的 GPU、最重要的数据、最敏感的模型权重和最复杂的代码执行环境。如果权限和审计没做好，轻则资源被滥用、成本失控，重则数据泄露、模型泄露、供应链攻击或合规事故。

先记住一句话：

训练平台安全不是给入口加登录，而是控制谁能用什么资源、读什么数据、运行什么镜像、产生什么模型、查看什么日志，并能事后追踪全过程。

24.0 本讲资料边界与第二轮精修口径

本讲第二轮精修时，资料口径主要对齐几类公开工程资料：Kubernetes RBAC 对 Role、ClusterRole、RoleBinding、ClusterRoleBinding 和 least privilege 的授权模型；Kubernetes ServiceAccount、Secrets、NetworkPolicy、Pod Security Standards 和 Auditing 对任务运行身份、密钥、网络边界、Pod 运行时约束和审计事件的稳定抽象；SLSA supply-chain security 对构建 provenance、依赖来源、构建完整性和供应链分层成熟度的通用口径；前文多租户隔离、容器环境、训练任务提交、训练可观测性和模型产物管理章节对 namespace、quota、镜像 digest、日志脱敏、artifact lineage 和权限继承的稳定抽象。

这里不把某个云厂商 IAM、某个内部权限系统、某个 Secret Manager、某个镜像扫描产品或某个合规框架写成唯一标准。正文只抽象训练平台安全里的稳定问题：身份模型、RBAC / ABAC、任务运行身份、数据权限、用途授权、产物等级传播、资源权限、镜像供应链、代码运行时隔离、密钥管理、网络 egress、日志脱敏、模型产物权限、高风险审批、审计日志、防篡改留存和安全事件响应。

第二轮精修重点放在三个方面：

1. 把“安全”从登录鉴权扩展为身份、资源、数据、运行时、供应链、密钥、网络、日志、artifact 和审计的完整控制面。
2. 补充可计算指标：越权阻断率、过度授权率、产物等级传播一致性、密钥泄露率、审计覆盖率和审批覆盖率。
3. 增加一个 0 依赖 Python demo，把训练平台安全从原则清单变成可运行门禁检查。

24.1 训练平台为什么高风险

训练平台有几个天然风险。

第一，它能访问大量数据。

训练数据可能包含用户数据、业务机密、代码、文档、图片、音频、日志和标注信息。

第二，它能运行用户代码。

训练脚本本质上是任意代码执行。如果没有限制，用户代码可以访问网络、文件系统、密钥和其他服务。

第三，它能使用昂贵资源。

GPU 成本高，资源滥用会直接造成损失。

第四，它会产生模型产物。

模型权重、checkpoint、adapter、评估报告都可能继承训练数据的敏感性。

第五，它有复杂供应链。

镜像、Python 包、CUDA kernel、训练框架、数据处理脚本都可能引入风险。

因此训练平台必须从设计上内置权限、安全和审计。

24.2 安全目标

训练平台安全要实现：

1. 身份可信。
2. 权限最小化。
3. 数据访问可控。
4. 资源使用可控。
5. 镜像和代码可信。
6. 密钥不泄露。
7. 日志和产物不越权。
8. 高风险操作可审批。
9. 全流程可审计。
10. 事故可追溯。

这些目标分别对应身份、权限、数据、运行时、供应链、密钥、审计和治理模块。

24.3 身份模型

训练平台里至少有几类身份：

1. 用户身份。
2. 团队或项目身份。
3. Service Account。
4. TrainingJob 身份。
5. 平台服务身份。
6. 管理员身份。

用户身份用于提交任务和查看结果。

TrainingJob 身份用于任务运行时访问数据、checkpoint、日志和模型仓库。

平台服务身份用于调度、创建 Pod、写审计日志、注册 artifact。

不要让所有训练任务共享一个高权限平台身份。

更好的方式是为每个任务分配最小权限的运行身份。

24.4 RBAC 和 ABAC

RBAC 是基于角色的访问控制。

例如：

1. platform_admin。
2. project_admin。
3. trainer。
4. viewer。
5. auditor。

ABAC 是基于属性的访问控制。

属性可以包括：

1. project。
2. team。
3. environment。
4. data_classification。
5. resource_pool。
6. job_type。
7. model_stage。

训练平台通常需要 RBAC + ABAC。

例如：

用户是 `trainer`，但只能在 `project_a` 下提交 `normal priority` 任务，且只能访问 `internal` 等级数据，不能访问 `restricted` 数据。只用 RBAC 很难表达这种细粒度策略。

24.5 数据权限

训练平台必须控制数据访问。

数据权限要回答：

1. 用户能不能看到数据集？
2. 任务能不能读取数据？
3. 数据能不能用于训练？
4. 数据能不能跨区域读取？
5. 数据是否需要脱敏？
6. 训练出的模型是否继承数据等级？

数据权限不应该只靠文件系统 ACL。

平台需要理解 `dataset_id`、`version`、`classification`、`owner`、`allowed_projects`、`allowed_purpose`。

训练任务提交时必须校验数据权限。

运行时也要用任务身份访问数据，而不是用户把长期密钥写进脚本。

24.6 数据等级传播

使用敏感数据训练出的产物可能也敏感。

产物包括：

1. checkpoint。
2. final model。
3. LoRA / adapter。
4. eval report。
5. logs。

6. sample outputs。

规则示例：

如果训练使用 `restricted` 数据，则 `checkpoint`、模型权重和 `eval report` 默认标记为 `restricted`。

这叫数据等级传播。

没有这条规则，用户可能无法读取原始数据，却可以下载用该数据训练出的模型。

24.7 资源权限

资源权限控制谁能用哪些 GPU 和队列。

常见策略：

1. 普通用户只能使用 `normal` 队列。
2. 高优先级队列需要审批。
3. H100 资源池只对关键项目开放。
4. 生产推理资源池禁止训练任务使用。
5. 可抢占资源允许实验任务使用。
6. 每个团队有 GPU-hour 预算。

资源权限和成本治理相关。

如果没有资源权限，昂贵 GPU 很容易被低价值任务占满。

24.8 镜像安全

训练任务运行镜像，因此镜像是安全入口。

风险：

1. 镜像包含漏洞。
2. 镜像包含恶意代码。
3. 镜像包含密钥。
4. 镜像来源不可信。
5. tag 被覆盖。
6. 基础镜像过旧。

平台应要求：

1. 只允许可信 registry。
2. 镜像扫描通过。
3. 记录 digest。
4. 高风险漏洞阻断。
5. 禁止密钥进入镜像。
6. 镜像签名。
7. 基础镜像白名单。

提交任务时就应该校验镜像，不要等 Pod 启动后才发现风险。

24.9 代码安全

训练平台运行用户代码。

代码风险包括：

1. 访问未授权数据。
2. 向外发送数据。
3. 读取宿主机路径。
4. 打印敏感信息。

5. 下载未知依赖。
6. 执行危险命令。

治理措施：

1. 限制运行时权限。
2. 禁止 privileged 容器。
3. 限制 hostPath。
4. 控制出网。
5. 代码仓库权限校验。
6. 高风险任务审批。
7. 日志脱敏。

训练平台不能假设所有训练代码都是安全的。

24.10 密钥管理

训练任务可能需要访问对象存储、数据服务、模型仓库和实验追踪系统。

不要把密钥写入：

1. 镜像。
2. 代码。
3. 配置文件。
4. 日志。
5. checkpoint。

更好的方式：

1. 使用 Secret Manager。
2. 按任务注入短期凭证。
3. 使用 Service Account。
4. 最小权限。
5. 自动轮换。
6. 访问审计。
7. 日志脱敏。

密钥泄露是训练平台重大事故。

24.11 网络安全

训练任务是否能访问外网、生产数据库、内部服务，需要严格控制。

网络策略包括：

1. 默认禁止跨租户访问。
2. egress 白名单。
3. 禁止访问生产敏感服务。
4. 只允许访问必要数据 endpoint。
5. 限制调试端口暴露。
6. 记录外部访问日志。

很多数据泄露不是从存储权限发生，而是训练代码把数据发到了外部 endpoint。

24.12 日志安全

训练日志可能包含敏感信息。

平台要控制：

1. 谁能看日志。

2. 日志保留多久。
3. 日志是否脱敏。
4. 是否允许下载。
5. 是否记录查看行为。

日志脱敏对象包括：

1. token。
2. 密钥。
3. 邮箱。
4. 手机号。
5. 用户输入。
6. 数据样本。
7. 连接串。

日志系统本身也是安全系统的一部分。

24.13 模型产物权限

模型产物包括：

1. checkpoint。
2. final model。
3. adapter。
4. tokenizer。
5. config。
6. eval report。
7. deployment package。

产物权限要根据：

1. 项目。
2. 数据等级。
3. 模型阶段。
4. owner。
5. 发布状态。
6. 审批结果。

例如训练中的 checkpoint 只能项目成员读，发布后的模型可以给更多应用调用，但下载权仍需限制。

模型权重是资产，不是普通文件。

24.14 高风险操作审批

高风险操作包括：

1. 使用 restricted 数据训练。
2. 导出模型权重。
3. 发布到生产。
4. 使用高优先级队列。
5. 关闭日志或 checkpoint。
6. 使用未审核镜像。
7. 访问外部网络。
8. 删除重要 checkpoint。

这些操作应该触发审批或策略阻断。

审批记录要进入审计日志。

24.15 审计日志

审计日志要记录谁在什么时候做了什么。

训练平台审计对象包括：

1. 任务提交。
2. 任务取消。
3. 数据访问。
4. 镜像使用。
5. checkpoint 创建和删除。
6. 模型注册和下载。
7. 权限修改。
8. 配额修改。
9. 高风险审批。
10. 日志查看。

审计日志字段：

1. actor。
2. action。
3. resource。
4. timestamp。
5. decision。
6. reason。
7. request_id。
8. ip / source。
9. metadata。

审计日志要防篡改，并有留存策略。

24.16 安全事件响应

训练平台要能响应安全事件。

例如：

1. 密钥泄露。
2. 未授权数据访问。
3. 镜像漏洞。
4. 模型权重误下载。
5. 日志泄露。
6. 异常外联。

响应流程：

1. 发现事件。
2. 冻结相关任务。
3. 撤销密钥。
4. 隔离产物。
5. 查询审计日志。
6. 通知 owner。
7. 修复策略。
8. 复盘。

没有审计，就无法做安全响应。

24.17 训练平台安全审计指标与最小 demo

这一节把训练平台安全改写为可审计门禁。真实平台可以把 IdP / IAM、Kubernetes RBAC、admission controller、NetworkPolicy、Secret Manager、registry、audit log、artifact registry 和 incident response

system 串起来；教学 demo 先用 toy 字段模拟。

一个训练平台安全样本可以抽象成：

$s_i = (u_i, r_i, d_i, a_i, q_i, m_i, c_i, e_i, n_i, l_i, w_i, h_i, o_i, t_i, z_i)$

其中， u_i 是身份和运行身份， r_i 是 RBAC / ABAC， d_i 是数据权限， a_i 是 artifact 等级传播， q_i 是资源和队列权限， m_i 是镜像供应链， c_i 是代码运行时隔离， e_i 是密钥， n_i 是网络， l_i 是日志， w_i 是模型产物权限， h_i 是高风险审批， o_i 是审计日志， t_i 是事件响应， z_i 是最终安全门禁。

统一覆盖率可以写成：

$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(s_i) = 1]$

越权阻断率可以写成：

$R_{\{\mathrm{block}\}} = \frac{N_{\{\mathrm{denied_unauth}\}}}{N_{\{\mathrm{unauth}\}}}$

过度授权率可以写成：

$R_{\{\mathrm{over}\}} = \frac{N_{\{\mathrm{granted}\}} - N_{\{\mathrm{used}\}}}{N_{\{\mathrm{granted}\}}}$

如果 R_{over} 很高，说明权限授予远大于真实使用，最小权限原则没有落地。

数据等级传播可以写成：

$V_{\{\mathrm{artifact}\}} = \mathbf{1}[\mathrm{class}(A) \geq \max_k \mathrm{class}(D_k)]$

其中， D_k 是训练使用的第 k 个数据集， A 是产物。使用 restricted 数据训练出的 checkpoint、模型、adapter、eval report 和日志不能自动降级。

密钥泄露率可以写成：

$R_{\{\mathrm{secret}\}} = \frac{N_{\{\mathrm{secret_leak}\}}}{N_{\{\mathrm{secret_event}\}}}$

审计覆盖率可以写成：

$R_{\{\mathrm{audit}\}} = \frac{N_{\{\mathrm{audited}\}}}{N_{\{\mathrm{required}\}}}$

最终训练平台安全门禁可以写成：

$G_{\{\mathrm{security}\}} = \mathbf{1}[\min_j C_j \geq \tau_j \ \& \ R_{\{\mathrm{block}\}} = 1 \ \& \ R_{\{\mathrm{over}\}} \leq \rho_{\{\mathrm{over}\}}]$

这个门禁背后的含义是：训练平台安全不是一个登录页，而是每个任务、数据、镜像、密钥、日志、artifact 和审批动作都有可证明的权限决策和审计证据。

下面的 0 依赖 Python demo 演示一个最小训练平台安全审计器。

```
from copy import deepcopy
```

```
GATES = [
    "identity_runtime_binding",
    "rbac_abac_policy_fit",
    "dataset_permission_purpose",
    "artifact_classification_propagation",
    "resource_quota_priority_governance",
    "image_supply_chain_trust",
    "runtime_code_isolation",
    "secret_scope_rotation",
    "network_egress_control",
    "log_redaction_access",
    "model_artifact_permission",
    "high_risk_approval",
    "audit_log_completeness",
    "audit_integrity_retention",
```

```

    "incident_response_readiness",
    "training_platform_security_gate",
]

```

```

LEVEL = {"public": 1, "internal": 2, "restricted": 3}

```

```

def base_case():
    return {
        "name": "complete_training_security_case",
        "identity": {
            "user_idp": True,
            "project_bound": True,
            "service_account_per_job": True,
            "shared_admin_identity": False,
            "workload_identity": True,
        },
        "permission": {
            "rbac_roles": ["trainer", "viewer"],
            "abac_attributes": ["project", "team", "data_classification", "resource_pool", "purpose"],
            "wildcard_permissions": False,
            "granted_permissions": 20,
            "used_permissions": 18,
            "unauthorized_attempts": 7,
            "denied_unauthorized": 7,
        },
        "data": {
            "dataset_id": "restricted_sales_dialog",
            "version": "v2026_05",
            "classification": "restricted",
            "allowed_project": True,
            "allowed_purpose": "training",
            "requested_purpose": "training",
            "runtime_identity_checked": True,
        },
        "artifact": {
            "input_classes": ["internal", "restricted"],
            "checkpoint_class": "restricted",
            "model_class": "restricted",
            "eval_report_class": "restricted",
            "log_class": "restricted",
        },
        "resource": {
            "quota_checked": True,
            "priority_approved": True,
            "gpu_pool_allowed": True,
            "gpu_hour_budget_ok": True,
        },
        "image": {
            "trusted_registry": True,
            "digest_pinned": True,
            "signed": True,
            "scan_passed": True,
            "critical_vulns": 0,
        }
    }

```

```

    "secret_in_image": False,
    "provenance_attested": True,
},
"runtime": {
    "privileged": False,
    "hostpath_allowed": False,
    "root_user": False,
    "egress_policy_enforced": True,
    "pod_security_restricted": True,
},
"secret": {
    "secret_manager": True,
    "short_lived": True,
    "least_privilege_scope": True,
    "rotation_enabled": True,
    "leak_events": 0,
    "secret_events": 5,
    "log_redaction": True,
},
"network": {
    "default_deny": True,
    "egress_allowlist": True,
    "prod_service_blocked": True,
    "external_access_logged": True,
},
"logs": {
    "rbac_enforced": True,
    "redaction_enabled": True,
    "download_audited": True,
    "retention_policy": True,
    "sensitive_pattern_scan": True,
},
"model_artifact": {
    "registry_rbac": True,
    "download_restricted": True,
    "stage_policy": True,
    "approval_before_release": True,
    "lineage_bound": True,
},
"approval": {
    "required_actions": ["restricted_data_train", "model_export", "prod_release", "external_egress"],
    "approved_actions": ["restricted_data_train", "model_export", "prod_release", "external_egress"],
    "approval_logged": True,
},
"audit": {
    "required_events": [
        "job_submit",
        "data_access",
        "image_use",
        "checkpoint_create",
        "model_download",
        "permission_change",
        "quota_change",
        "approval_decision",
        "log_view",

```

```

    ],
    "audited_events": [
        "job_submit",
        "data_access",
        "image_use",
        "checkpoint_create",
        "model_download",
        "permission_change",
        "quota_change",
        "approval_decision",
        "log_view",
    ],
    "actor_action_resource": True,
    "request_id": True,
    "tamper_evident": True,
    "retention_days": 365,
},
"incident": {
    "freeze_job_ready": True,
    "revoke_secret_ready": True,
    "isolate_artifact_ready": True,
    "audit_query_ready": True,
    "owner_notify_ready": True,
    "postmortem_ready": True,
},
"training_platform_security_gate": True,
}

```

```

def contains_all(values, required):
    return set(required).issubset(set(values))

```

```

def rate(num, den):
    return num / max(den, 1e-9)

```

```

def artifact_propagates(case):
    max_input = max(LEVEL[x] for x in case["artifact"]["input_classes"])
    outputs = [
        case["artifact"]["checkpoint_class"],
        case["artifact"]["model_class"],
        case["artifact"]["eval_report_class"],
        case["artifact"]["log_class"],
    ]
    return all(LEVEL[x] >= max_input for x in outputs)

```

```

def evaluate(case):
    identity = case["identity"]
    permission = case["permission"]
    data = case["data"]
    resource = case["resource"]
    image = case["image"]
    runtime = case["runtime"]

```

```

secret = case["secret"]
network = case["network"]
logs = case["logs"]
model_artifact = case["model_artifact"]
approval = case["approval"]
audit = case["audit"]
incident = case["incident"]

overprivilege = rate(
    permission["granted_permissions"] - permission["used_permissions"],
    permission["granted_permissions"],
)
unauthorized_block = rate(permission["denied_unauthorized"], permission["unauthorized_attempts"])
secret_leak = rate(secret["leak_events"], secret["secret_events"])
audit_coverage = rate(len(audit["audited_events"]), len(audit["required_events"]))

checks = {}
checks["identity_runtime_binding"] = (
    identity["user_idp"]
    and identity["project_bound"]
    and identity["service_account_per_job"]
    and identity["workload_identity"]
    and not identity["shared_admin_identity"]
)
checks["rbac_abac_policy_fit"] = (
    "trainer" in permission["rbac_roles"]
    and contains_all(permission["abac_attributes"], ["project", "data_classification", "resource_pool", "purpose"])
    and not permission["wildcard_permissions"]
    and overprivilege <= 0.2
    and unauthorized_block == 1.0
)
checks["dataset_permission_purpose"] = (
    data["allowed_project"]
    and data["allowed_purpose"] == data["requested_purpose"]
    and data["runtime_identity_checked"]
    and data["classification"] in LEVEL
    and bool(data["version"])
)
checks["artifact_classification_propagation"] = artifact_propagates(case)
checks["resource_quota_priority_governance"] = (
    resource["quota_checked"]
    and resource["priority_approved"]
    and resource["gpu_pool_allowed"]
    and resource["gpu_hour_budget_ok"]
)
checks["image_supply_chain_trust"] = (
    image["trusted_registry"]
    and image["digest_pinned"]
    and image["signed"]
    and image["scan_passed"]
    and image["critical_vulns"] == 0
    and not image["secret_in_image"]
    and image["provenance_attested"]
)
checks["runtime_code_isolation"] = (

```

```

    not runtime["privileged"]
    and not runtime["hostpath_allowed"]
    and not runtime["root_user"]
    and runtime["egress_policy_enforced"]
    and runtime["pod_security_restricted"]
)
checks["secret_scope_rotation"] = (
    secret["secret_manager"]
    and secret["short_lived"]
    and secret["least_privilege_scope"]
    and secret["rotation_enabled"]
    and secret["log_redaction"]
    and secret_leak == 0
)
checks["network_egress_control"] = (
    network["default_deny"]
    and network["egress_allowlist"]
    and network["prod_service_blocked"]
    and network["external_access_logged"]
)
checks["log_redaction_access"] = (
    logs["rbac_enforced"]
    and logs["redaction_enabled"]
    and logs["download_audited"]
    and logs["retention_policy"]
    and logs["sensitive_pattern_scan"]
)
checks["model_artifact_permission"] = (
    model_artifact["registry_rbac"]
    and model_artifact["download_restricted"]
    and model_artifact["stage_policy"]
    and model_artifact["approval_before_release"]
    and model_artifact["lineage_bound"]
)
checks["high_risk_approval"] = (
    contains_all(approval["approved_actions"], approval["required_actions"])
    and approval["approval_logged"]
)
checks["audit_log_completeness"] = (
    contains_all(audit["audited_events"], audit["required_events"])
    and audit["actor_action_resource"]
    and audit["request_id"]
    and audit_coverage >= 1.0
)
checks["audit_integrity_retention"] = audit["tamper_evident"] and audit["retention_days"] >= 180
checks["incident_response_readiness"] = all(incident.values())
checks["training_platform_security_gate"] = bool(case["training_platform_security_gate"])
return checks

```

```

def summarize_examples(case):
    permission = case["permission"]
    secret = case["secret"]
    audit = case["audit"]
    approval = case["approval"]

```

```

return {
    "unauthorized_block_rate": round(rate(permission["denied_unauthorized"], permission["unauthorized_attempts"])),
    "overprivilege_rate": round(rate(permission["granted_permissions"] - permission["used_permissions"], permission["granted_permissions"])),
    "artifact_level_propagation": artifact_propagates(case),
    "secret_exposure_rate": round(rate(secret["leak_events"], secret["secret_events"]), 3),
    "audit_coverage": round(rate(len(audit["audited_events"]), len(audit["required_events"])), 3),
    "approval_coverage": round(rate(len(approval["approved_actions"], len(approval["required_actions"])), 3),
}

def mutate(name, editor):
    case = deepcopy(base_case())
    case["name"] = name
    editor(case)
    return case

def build_cases():
    cases = [base_case()]
    cases.append(mutate("identity_shared_admin_bad", lambda c: c["identity"].update({"service_account_per_job": False})))
    cases.append(mutate("rbac_abac_wildcard_bad", lambda c: c["permission"].update({"wildcard_permissions": True, "granted_permissions": []})))
    cases.append(mutate("dataset_permission_purpose_bad", lambda c: c["data"].update({"allowed_purpose": "evaluation"})))
    cases.append(mutate("artifact_classification_downgrade_bad", lambda c: c["artifact"].update({"model_class": "intermediate"})))
    cases.append(mutate("resource_priority_unapproved_bad", lambda c: c["resource"].update({"priority_approved": False})))
    cases.append(mutate("image_supply_chain_untrusted_bad", lambda c: c["image"].update({"digest_pinned": False, "signature_verified": False})))
    cases.append(mutate("runtime_privileged_bad", lambda c: c["runtime"].update({"privileged": True, "pod_security_restrictions": "baseline"})))
    cases.append(mutate("secret_scope_rotation_bad", lambda c: c["secret"].update({"short_lived": False, "leak_events": []})))
    cases.append(mutate("network_egress_open_bad", lambda c: c["network"].update({"default_deny": False, "egress_allowed": True})))
    cases.append(mutate("log_redaction_missing_bad", lambda c: c["logs"].update({"redaction_enabled": False, "download_restricted": False})))
    cases.append(mutate("model_artifact_permission_bad", lambda c: c["model_artifact"].update({"download_restricted": False})))
    cases.append(mutate("high_risk_approval_missing_bad", lambda c: c["approval"].update({"approved_actions": ["restricted"]}))
    cases.append(mutate("audit_log_incomplete_bad", lambda c: c["audit"].update({"audited_events": ["job_submit", "data_upload"]}))
    cases.append(mutate("audit_integrity_retention_bad", lambda c: c["audit"].update({"tamper_evident": False, "retention_policy": "none"})))
    cases.append(mutate("incident_response_missing_bad", lambda c: c["incident"].update({"revoke_secret_ready": False})))
    cases.append(mutate("security_gate_missing_bad", lambda c: c.update({"training_platform_security_gate": False})))
    return cases

def main():
    cases = build_cases()
    evaluations = {case["name"]: evaluate(case) for case in cases}
    metrics = {
        gate: round(sum(1 for case in cases if evaluations[case["name"]][gate]) / len(cases), 3)
        for gate in GATES
    }
    failed_cases = [
        case["name"]
        for case in cases
        if not all(evaluations[case["name"]].values())
    ]
    failed_gates = [
        gate
        for gate in GATES
        if any(not evaluations[case["name"]][gate] for case in cases)
    ]

```

```

complete = cases[0]
smoke = {
    "complete_case_passes": all(evaluations[complete["name"]].values()),
    "caught_shared_admin": not evaluations["identity_shared_admin_bad"]["identity_runtime_binding"],
    "caught_artifact_downgrade": not evaluations["artifact_classification_downgrade_bad"]["artifact_classification"],
    "caught_secret_leak": not evaluations["secret_scope_rotation_bad"]["secret_scope_rotation"],
    "caught_audit_gap": not evaluations["audit_log_incomplete_bad"]["audit_log_completeness"],
    "caught_gate_gap": not evaluations["security_gate_missing_bad"]["training_platform_security_gate"],
}
training_platform_security_gate_pass = not failed_cases and all(v >= 1.0 for v in metrics.values())

print("training_platform_security_examples=", summarize_examples(complete))
print("smoke=", smoke)
print("metrics=", metrics)
print("hard_blocker_count=", len(failed_cases))
print("failed_cases=", failed_cases)
print("failed_gates=", failed_gates)
print("training_platform_security_gate_pass=", training_platform_security_gate_pass)

```

```

if __name__ == "__main__":
    main()

```

预期输出类似：

```

training_platform_security_examples= {'unauthorized_block_rate': 1.0, 'overprivilege_rate': 0.1, 'artifact_level_propaganda': 0.1, 'dataset_permission_purpose': 0.941, 'artifact_classification': 0.941, 'rbac_abac_policy_fit': 0.941, 'identity_runtime_binding': 0.941, 'secret_scope_rotation': 0.941, 'audit_log_completeness': 0.941, 'security_gate_missing': 0.941}
smoke= {'complete_case_passes': True, 'caught_shared_admin': True, 'caught_artifact_downgrade': True, 'caught_secret_leak': True, 'caught_audit_gap': True, 'caught_gate_gap': True}
metrics= {'identity_runtime_binding': 0.941, 'rbac_abac_policy_fit': 0.941, 'dataset_permission_purpose': 0.941, 'artifact_classification': 0.941, 'secret_scope_rotation': 0.941, 'audit_log_completeness': 0.941, 'security_gate_missing': 0.941}
hard_blocker_count= 16
failed_cases= ['identity_shared_admin_bad', 'rbac_abac_wildcard_bad', 'dataset_permission_purpose_bad', 'artifact_classification_bad', 'secret_scope_rotation_bad', 'audit_log_incomplete_bad', 'security_gate_missing_bad']
failed_gates= ['identity_runtime_binding', 'rbac_abac_policy_fit', 'dataset_permission_purpose', 'artifact_classification', 'secret_scope_rotation', 'audit_log_completeness', 'security_gate_missing']
training_platform_security_gate_pass= False

```

面试里可以把这个 demo 压缩成一句话：训练平台安全要证明 “谁、以什么运行身份、因为什么策略、访问了什么数据和产物、运行了什么镜像和代码、用了什么密钥、留下了什么审计证据”，而不是只证明用户已经登录。

24.18 面试中如何回答训练平台安全

如果面试官问：

大模型训练平台如何做权限、安全和审计？

可以这样回答：

我会从身份、资源、数据、镜像、代码、密钥、网络、日志、模型产物和审计几个层面设计。

身份层接入统一 IdP，区分用户身份、TrainingJob 运行身份、Service Account 和平台服务身份。权限层用 RBAC + ABAC，控制谁能做什么。

数据层按 dataset_id、version、classification 和 allowed_purpose 做授权，并做数据等级传播：使用敏感数据训练出的 checkpoint 也是敏感数据。

运行层只允许可信镜像 registry，做镜像扫描、签名和 digest 记录；禁止 privileged、限制 hostPath、控制 egress，密钥通过 Secrets 管理。

审计层记录任务提交、数据访问、镜像使用、checkpoint 创建删除、模型注册下载、权限变更、高风险审批和日志查看。高风险操作需要审批。

24.19 常见误区

误区一：训练平台安全就是登录鉴权。

登录只是身份入口，还要控制数据、资源、镜像、密钥、网络、日志和模型产物。

误区二：用户能读数据，就能随便训练。

数据使用目的也要授权，训练产物可能继承数据敏感等级。

误区三：平台管理员可以看所有日志。

日志可能包含敏感数据，管理员访问也应最小权限和审计。

误区四：镜像能跑就行。

镜像必须可信、扫描、签名、记录 digest，并防止密钥泄露。

误区五：审计只在事故后需要。

审计是日常治理、合规、成本和安全响应的基础。

24.20 面试题

题 1：训练平台有哪些身份？

答：包括用户身份、团队或项目身份、TrainingJob 运行身份、Service Account、平台服务身份和管理员身份。任务运行时应使用最小权限的任务身份，而不是共享高权限身份。

题 2：为什么训练产物要继承数据等级？

答：模型权重、checkpoint、adapter 和评估报告可能包含或泄露训练数据的信息。如果使用敏感数据训练，产物也应继承相应敏感等级，避免用户绕过原始数据权限访问模型产物。

题 3：镜像安全要做哪些事？

答：使用可信 registry、镜像扫描、漏洞阻断、镜像签名、digest 记录、基础镜像白名单、禁止密钥进入镜像，并在任务提交时校验镜像策略。

题 4：哪些训练操作属于高风险操作？

答：使用敏感数据训练、导出模型权重、发布到生产、使用高优先级队列、关闭日志或 checkpoint、使用未审核镜像、访问外部网络、删除重要 checkpoint 等。

题 5：审计日志应该记录什么？

答：应记录 actor、action、resource、timestamp、decision、reason、request_id、source 和 metadata。覆盖任务提交、数据访问、镜像使用、checkpoint、模型产物、权限变更、审批和日志查看。

24.21 小练习

练习一：设计一个训练平台权限模型。

要求：包含 admin、project_admin、trainer、viewer、auditor 五类角色和对应权限。

练习二：设计数据等级传播规则。

要求：说明 public、internal、restricted 数据训练出的 checkpoint、model、eval report 应如何定级。

练习三：设计高风险操作审批流程。

要求：以“使用 restricted 数据训练并导出模型权重”为例，列出审批、审计和产物权限规则。

练习四：设计审计日志 schema。

要求：包含 actor、action、resource、decision、reason、timestamp、request_id 和 metadata。

24.22 本章小结

本章讲了训练平台权限、安全和审计。

你需要掌握：

1. 训练平台安全覆盖身份、资源、数据、镜像、代码、密钥、网络、日志、模型产物和审计。
2. 训练任务运行时应使用最小权限任务身份。
3. RBAC + ABAC 比单纯角色权限更适合训练平台。
4. 数据权限要控制使用目的，并处理产物敏感等级传播。
5. 资源权限和成本治理相关，高价值 GPU 和高优先级队列需要控制。
6. 镜像安全和代码运行时安全是供应链和执行安全的关键。
7. 密钥必须通过 Secret Manager 和短期凭证管理，不能写进镜像、代码或日志。
8. 日志和模型产物都需要权限和审计。
9. 高风险操作需要审批。
10. 审计日志是安全响应、合规和复盘的基础。

下一章我们会用系统设计题的方式，把第三部分串起来：训练平台系统设计面试题。

第 25 章 训练平台系统设计面试题

前面我们已经讲了训练平台总览、任务提交、分布式启动、配置管理、日志指标事件、checkpoint、容错、数据供给、权限安全和审计。本章把这些内容整合成一道系统设计题：如何设计一个大模型训练平台？

这是一道非常高频的 AI Infra 面试题。它不是问你会不会写训练脚本，而是考你是否理解大模型训练从提交、调度、启动、运行、监控、恢复到产物注册的完整生命周期。

先记住一句话：

设计大模型训练平台，要围绕 TrainingJob 生命周期展开：提交、校验、排队、调度、启动、运行、观测、checkpoint、恢复、评估、产物注册、审计和成本归因。

25.0 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 做第二轮精修时，重点参考 Kubernetes Job、Kueue、Kubeflow Trainer、PyTorch torchrun、OpenTelemetry 和 MLflow Tracking 等公开文档来校准系统边界：Kubernetes Job 更接近通用批任务控制器，Kueue 解决队列、配额、准入和资源共享，Kubeflow Trainer / Training Operator 更接近训练任务的高层语义，torchrun 负责把节点、rank、rendezvous 和进程数转成分布式启动参数，OpenTelemetry 把 logs、metrics、traces 等信号放进统一观测模型，MLflow Tracking 则帮助理解 experiment、run、params、metrics 和 artifacts 的记录边界。

这不意味着面试必须背某一个工具的 API。面试官真正想看的是你能不能把训练平台拆成稳定抽象：

1. 控制面：TrainingJob schema、准入校验、配额、状态机和审计。
2. 调度面：queue、priority、quota、gang scheduling、topology-aware placement、preemption 和 backfilling。
3. 启动面：launcher adapter、rank、local rank、world size、rendezvous、GPU binding 和 NCCL 环境。
4. 运行面：数据版本、checkpoint、日志指标事件、故障分类、重试和恢复。
5. 治理面：权限、安全、成本、artifact lineage、experiment tracking 和开发者自助。

所以本章中的 TrainingJob 是教学用抽象，不等同于某个厂商或开源项目的单一 CRD。你可以在实现中选择 Kubernetes + Kueue + Kubeflow Trainer，也可以选择 Slurm / Ray / 自研控制面，但系统设计回答必须证明这些模块之间的输入、输出、状态、指标和失败边界都清楚。

25.1 面试题描述

题目可能这样问：

请设计一个企业级大模型训练平台。平台需要支持多个团队提交预训练、SFT、RLHF、评估等任务，能够管理 GPU 资源、分布式训练、

这道题不要一上来就说“用 Kubernetes”。Kubernetes 是底座，不是完整答案。

面试官想听的是：你能不能把训练平台拆成模块，并解释关键 trade-off。

25.2 需求澄清

可以先问：

1. 平台主要支持预训练，还是 SFT/微调为主？
2. GPU 集群规模多大？
3. 是否多租户？
4. 是否需要抢占式调度？
5. 是否需要支持多种 launcher？
6. 数据是否敏感？
7. 是否需要自动恢复？
8. 是否需要模型注册和发布？
9. 是否有成本预算要求？

如果要自己假设，可以说：

我假设这是企业内部大模型训练平台，支持预训练、SFT、评估和 RLHF。平台服务多个团队，底层有 Kubernetes + GPU 集群。系统需

25.3 核心目标

训练平台要实现：

1. 标准化任务提交。
2. GPU 资源调度。
3. 分布式训练启动。
4. 数据版本和权限管理。
5. 配置和环境可复现。
6. 日志、指标、事件和 trace。
7. checkpoint 保存和恢复。
8. 失败自动诊断和重试。
9. 模型 artifact 注册。
10. 权限、安全、审计和成本治理。

这些目标对应平台的核心模块。

25.4 总体架构

一个合理架构：

```
User / Portal / CLI / SDK
-> Training Job API
-> Config Validator
-> Policy / Quota Engine
-> Queue / Scheduler
-> Launcher / Training Operator
-> Kubernetes / GPU Cluster
    -> Worker Pods
    -> GPU / Network / Storage
-> Data Access Layer
-> Checkpoint Manager
-> Observability: Logs / Metrics / Events / Trace
-> Experiment Tracker
-> Model Registry / Artifact Store
-> Audit / Cost / Notification
```

模块说明：

1. Portal / CLI / SDK：用户提交和管理任务。
2. Training Job API：统一任务入口。
3. Config Validator：校验配置、镜像、代码、数据和资源。
4. Policy / Quota Engine：权限、配额、安全策略。
5. Queue / Scheduler：队列、优先级、gang scheduling、拓扑感知。
6. Launcher / Operator：启动 torchrun、DeepSpeed、Megatron、Ray。
7. Kubernetes / GPU Cluster：运行训练容器。
8. Data Access Layer：数据集、manifest、缓存和权限。
9. Checkpoint Manager：保存、恢复、保留和清理。
10. Observability：日志、指标、事件、trace、告警。
11. Experiment Tracker：参数、配置、数据、指标、环境。
12. Model Registry：模型权重、tokenizer、config、eval report。
13. Audit / Cost：审计日志和 GPU-hour 成本归因。

25.5 TrainingJob 设计

TrainingJob 是核心对象。

它应该包含：

1. metadata: job_name、owner、project、tags。
2. image: 镜像 digest。
3. code: repo、commit。
4. data: dataset_id、version、manifest。
5. command: entrypoint、args。
6. config: 最终生效配置。
7. resources: gpu_type、gpu_count、gpu_per_node、cpu、memory。
8. distributed: launcher、nnodes、nproc_per_node、parallelism。
9. checkpoint: save policy、resume_from、output_uri。
10. logging: logs、metrics、events。
11. scheduling: queue、priority、preemptible。
12. security: service_account、data_policy。

面试中要强调：TrainingJob 不是普通 Job，它是训练生命周期对象。

25.6 提交流程

任务提交流程：

1. 用户通过 Portal / CLI / SDK 提交 TrainingJob。
2. API 做 schema 校验。
3. 校验镜像 digest 和安全扫描状态。
4. 校验代码 commit。
5. 校验数据版本和访问权限。
6. 合并配置并生成最终生效配置。
7. 校验资源申请和配额。
8. 校验 checkpoint 路径。
9. 写入 metadata store。
10. 进入调度队列。

提交系统要支持幂等，避免重复创建任务。

25.7 调度设计

训练平台调度需要：

1. 队列。

2. 优先级。
3. 配额。
4. 公平性。
5. gang scheduling。
6. 拓扑感知。
7. 抢占。
8. backfilling。
9. GPU 碎片治理。

重点解释：

1. 分布式训练需要 worker 同时启动，所以需要 gang scheduling。
2. 大模型训练通信密集，所以需要拓扑感知。
3. 低优先级任务可以使用空闲资源，但要支持抢占和 checkpoint。
4. 多团队共用集群，需要配额和公平分享。

25.8 分布式启动设计

调度分配资源后，launcher 负责启动训练。

它要生成：

1. master_addr。
2. master_port。
3. rank。
4. local_rank。
5. world_size。
6. CUDA_VISIBLE_DEVICES。
7. NCCL 环境变量。
8. 启动命令。

平台应支持多种 launcher adapter：

1. torchrun。
2. DeepSpeed。
3. Megatron。
4. Ray。
5. MPI。

统一 adapter 可以降低用户使用复杂度。

25.9 数据层设计

数据层负责：

1. 数据集注册。
2. 数据版本。
3. dataset manifest。
4. 数据权限。
5. shard 管理。
6. 流式读取。
7. 本地缓存。
8. 分布式缓存。
9. shuffle 和 rank 切分。
10. 数据质量监控。

训练平台应保证：

1. 训练使用的数据版本可追踪。
2. 用户只能访问授权数据。

3. 数据读取吞吐足够。
4. 断点恢复时能恢复数据位置。

25.10 Checkpoint 设计

Checkpoint Manager 负责：

1. 保存频率。
2. resume checkpoint。
3. eval checkpoint。
4. release checkpoint。
5. 分片保存。
6. 异步保存。
7. manifest 和 checksum。
8. committed 状态。
9. 生命周期策略。
10. 从 checkpoint 恢复。

和调度联动：

1. 可抢占任务必须配置 checkpoint。
2. 抢占前尽量保存。
3. 重试时从最新 committed checkpoint 恢复。

25.11 可观测性设计

训练可观测性包括：

1. Logs。
2. Metrics。
3. Events。
4. Traces。
5. Dashboard。
6. Alerts。

关键指标：

1. loss。
2. learning rate。
3. tokens/s。
4. step time。
5. GPU utilization。
6. memory。
7. communication time。
8. dataloader time。
9. checkpoint time。
10. queue wait time。

关键事件：

1. JobSubmitted。
2. JobScheduled。
3. TrainingStarted。
4. CheckpointSaved。
5. WorkerFailed。
6. RetryStarted。
7. JobCompleted。

可观测性要支持 rank、node、attempt 维度。

25.12 容错设计

训练平台要做故障分类。

可重试：

1. 节点失败。
2. GPU 短暂故障。
3. 临时网络故障。
4. 存储短暂不可用。
5. Pod 被驱逐。

不可盲目重试：

1. 配置错误。
2. 权限错误。
3. 代码 bug。
4. 持续 OOM。
5. checkpoint 不兼容。
6. loss NaN。

容错流程：

1. 识别错误类型。
2. 生成诊断报告。
3. 判断是否可重试。
4. 选择健康节点。
5. 从 checkpoint 恢复。
6. 记录 attempt。
7. 通知用户。

25.13 安全和审计设计

训练平台安全包括：

1. 身份认证。
2. RBAC + ABAC。
3. 数据权限。
4. 数据等级传播。
5. 镜像扫描和签名。
6. 密钥管理。
7. 网络 egress 控制。
8. 日志脱敏。
9. 模型产物权限。
10. 审计日志。

审计要记录：

1. 谁提交任务。
2. 用了什么镜像。
3. 读了什么数据。
4. 生成了什么 checkpoint。
5. 谁下载了模型。
6. 谁修改了配额。
7. 谁审批了高风险操作。

25.14 成本治理

训练平台要记录：

1. GPU-hours。

2. CPU 和内存。
3. 存储。
4. checkpoint。
5. 网络。
6. 日志。
7. 评估推理成本。

成本维度：

1. team。
2. project。
3. user。
4. job。
5. queue。
6. model。

平台可以做：

1. 预算告警。
2. 低效任务告警。
3. 闲置 GPU 检测。
4. checkpoint 成本报告。
5. 队列利用率分析。

成本治理是训练平台生产化的重要标志。

25.15 一个完整流程

完整流程如下：

1. 用户提交 TrainingJob。
2. 平台校验镜像、代码、数据、配置和权限。
3. 生成最终生效配置。
4. 任务进入队列。
5. 调度器根据配额、优先级和拓扑分配 GPU。
6. Launcher 生成 rank、world size 和启动命令。
7. Worker Pod 启动训练。
8. Data Layer 提供训练数据。
9. Observability 收集日志、指标和事件。
10. Checkpoint Manager 周期保存。
11. 失败时故障分类并自动恢复。
12. 训练结束后注册模型和 artifact。
13. 评估平台运行评估。
14. Audit 和 Cost 系统记录全过程。

这个流程就是系统设计题的主线。

25.16 面试回答模板

可以这样回答：

我会把大模型训练平台设计成 TrainingJob 生命周期管理系统。

入口层提供 Portal、CLI、SDK 和 API，用户提交 TrainingJob，包含镜像 digest、代码 commit、数据版本、训练配置、分布式配置、

控制面先做 schema、镜像、代码、数据权限、资源配额和安全策略校验，生成最终生效配置并写入 metadata store。

调度层负责队列、优先级、配额、公平性、gang scheduling、拓扑感知、抢占和 backfilling。大模型训练需要 worker 同时启动，

启动层通过 launcher adapter 支持 torchrun、DeepSpeed、Megatron、Ray，自动生成 master_addr、rank、world_size、NCCL 配置和运行层接入数据平台，使用 dataset manifest、shard、缓存和权限控制保证数据高吞吐和可复现。Checkpoint Manager 负责分片保存。可观测性层采集 logs、metrics、events、traces，展示 loss、tokens/s、step time、GPU、通信、dataloader、checkpoint 和 rank。容错层对节点失败、通信失败、OOM、数据错误、配置错误做分类，可恢复故障从 checkpoint 自动恢复，不可恢复错误快速失败并生。安全治理层提供身份、RBAC/ABAC、数据权限、镜像安全、密钥管理、日志脱敏、模型产物权限和审计。成本层按 team/project/job hours、存储和网络成本。

25.17 系统设计审计指标与最小 demo

系统设计题最容易失分的地方，是把答案讲成“模块名列表”。更好的方式是把设计转成一张审计表：每个模块有什么输入、输出、状态、指标、失败模式和门禁。

训练平台系统设计样本可以抽象成：

$p_i = (r_i, j_i, s_i, a_i, q_i, l_i, d_i, c_i, o_i, f_i, g_i, k_i, u_i, m_i, b_i, t_i)$

其中， r_i 是需求澄清， j_i 是 TrainingJob 契约， s_i 是生命周期状态机， a_i 是准入和配额， q_i 是调度， l_i 是 launcher， d_i 是数据和配置血缘， c_i 是 checkpoint， o_i 是可观测性和实验追踪， f_i 是容错恢复， g_i 是安全审计， k_i 是成本容量， u_i 是开发者自助， m_i 是接口和 artifact 契约， b_i 是 trade-off 边界， t_i 是最终设计门禁。

第 j 个设计维度的覆盖率可以写成：

$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(p_i) = 1]$

分布式训练的总进程数和 effective batch 可以先用两个简单公式验算：

$W = n_{\{\mathrm{node}\}} n_{\{\mathrm{proc}\}}, \quad B_{\{\mathrm{eff}\}} = B_{\{\mathrm{micro}\}} A_{\{\mathrm{grad}\}} W$

其中， W 是 world size， n_{node} 是节点数， n_{proc} 是每节点进程数， B_{micro} 是每进程 micro batch， A_{grad} 是 gradient accumulation steps。系统设计回答如果连这些基本一致性都没有，就很难证明 launcher、资源申请和训练配置是一致的。

调度层可以用 P95 排队时间约束体验：

$T_{\{\mathrm{queue}, p95\}} = Q_{\{0.95\}}(\{t_{\{\mathrm{start}\}, i} - t_{\{\mathrm{submit}\}, i}\}_{i=1}^N)$

checkpoint 和抢占的损失可以用最大丢失 GPU-hours 估算：

$L_{\{\mathrm{lost}\}} = G \frac{t_{\{\mathrm{fail}\}} - t_{\{\mathrm{ckpt}\}}}{60}$

其中， G 是本次任务占用 GPU 数，时间单位是分钟。experiment、checkpoint、model registry 和 audit 的串联程度可以用 trace 覆盖率表示：

$R_{\{\mathrm{trace}\}} = \frac{N_{\{\mathrm{linked}\}}}{N_{\{\mathrm{required}\}}}$

最终可以把设计门禁写成：

$G_{\{\mathrm{design}\}} = \mathbf{1}[\min_j C_j \geq \tau_j \wedge T_{\{\mathrm{queue}, p95\}} \leq B_{\{\mathrm{queue}\}} \wedge L_{\{\mathrm{lost}\}} \leq \tau_{\mathrm{lost}} \wedge R_{\{\mathrm{trace}\}} \geq \tau_{\mathrm{trace}}]$

下面这个 demo 用 16 个门禁检查一个训练平台系统设计回答。它不是生产评审器，而是帮助你把面试回答从“我会用 Kubernetes”升级成“我能证明训练生命周期闭环可运行、可恢复、可复现、可治理”。

```
from math import ceil
```

```
GATES = [
    "requirement_clarification",
    "trainingjob_contract",
    "lifecycle_state_machine",
```

```

"admission_quota_policy",
"scheduler_resource_fit",
"distributed_launcher_fit",
"data_config_lineage",
"checkpoint_recovery_coupling",
"observability_experiment_linkage",
"fault_recovery_flow",
"security_audit_governance",
"cost_capacity_governance",
"developer_self_service",
"interface_artifact_contract",
"tradeoff_boundary_reasoning",
"training_platform_design_gate",
]

```

```

def p95(values):
    ordered = sorted(values)
    index = max(0, ceil(0.95 * len(ordered)) - 1)
    return ordered[index]

```

```

def all_present(mapping, names):
    return all(mapping.get(name) for name in names)

```

```

def evaluate_case(case):
    expected_states = [
        "submitted",
        "validated",
        "queued",
        "scheduled",
        "starting",
        "running",
        "checkpointing",
        "retrying",
        "failed",
        "completed",
        "cancelled",
    ]
    required_job_fields = [
        "owner",
        "project",
        "image_digest",
        "code_commit",
        "dataset_version",
        "final_config_hash",
        "resources",
        "distributed",
        "checkpoint_policy",
        "observability",
        "security",
        "queue",
    ]
    required_requirements = [

```

```

        "workload_scope",
        "cluster_scale",
        "multi_tenant",
        "data_sensitivity",
        "preemption",
        "recovery_slo",
        "artifact_release",
        "cost_budget",
    ]
    required_tradeoffs = [
        "kubernetes_vs_training_operator",
        "fairness_vs_utilization",
        "topology_vs_queue_time",
        "preemption_vs_lost_work",
        "self_service_vs_guardrail",
        "cost_vs_reliability",
    ]

    resources = case["resources"]
    world_size = resources["nodes"] * resources["processes_per_node"]
    effective_batch = resources["micro_batch"] * resources["grad_accum"] * world_size

    lineage = case["lineage"]
    checkpoint = case["checkpoint"]
    observability = case["observability"]
    cost = case["cost"]

    results = {}
    results["requirement_clarification"] = all_present(case["requirements"], required_requirements)
    results["trainingjob_contract"] = all_present(case["trainingjob"], required_job_fields)
    results["lifecycle_state_machine"] = (
        set(expected_states).issubset(case["states"])
        and case["state_transitions_valid"]
    )
    results["admission_quota_policy"] = all_present(
        case["admission"],
        ["schema", "quota_dry_run", "priority_approval", "image_policy", "dataset_permission", "idempotency"],
    )
    results["scheduler_resource_fit"] = all_present(
        case["scheduler"],
        ["queue", "quota", "fair_share", "gang", "topology", "backfilling", "preemption", "fragmentation"],
    )
    results["distributed_launcher_fit"] = (
        case["launcher"].get("world_size") == world_size
        and all_present(case["launcher"], ["rank_mapping", "rendezvous", "gpu_binding", "nccl", "adapter"])
    )
    results["data_config_lineage"] = all_present(
        lineage,
        ["image_digest", "code_commit", "dataset_version", "manifest", "final_config_hash", "env_versions", "seed", ],
    )
    results["checkpoint_recovery_coupling"] = (
        all_present(checkpoint, ["policy", "committed_manifest", "checksum", "restore_drill", "scheduler_preemption_h
        and checkpoint["max_stale_minutes"] <= 30
    )
    results["observability_experiment_linkage"] = all_present(

```

```

    observability,
    ["logs", "metrics", "events", "traces", "rank_dimensions", "experiment_link", "dashboard", "alerts"],
)
results["fault_recovery_flow"] = all_present(
    case["fault"],
    ["taxonomy", "retry_budget", "backoff", "checkpoint_resume", "fast_fail_nonretryable", "diagnosis_report"],
)
results["security_audit_governance"] = all_present(
    case["security"],
    ["identity", "rbac_abac", "data_purpose", "runtime_isolation", "image_supply_chain", "audit_log", "incident_r
)
results["cost_capacity_governance"] = all_present(
    cost,
    ["gpu_hours", "storage", "network", "owner_project_job", "budget_alert", "utilization_report"],
) and cost["attribution_rate"] >= 0.9
results["developer_self_service"] = all_present(
    case["developer"],
    ["portal", "cli", "sdk", "templates", "status_page", "error_report", "rerun_from_checkpoint"],
)
results["interface_artifact_contract"] = all_present(
    case["interfaces"],
    ["api_contract", "scheduler_handoff", "launcher_handoff", "data_contract", "artifact_registry", "audit_event
)
results["tradeoff_boundary_reasoning"] = all_present(case["tradeoffs"], required_tradeoffs)

trace_coverage = observability["linked_artifacts"] / observability["required_artifacts"]
max_lost_gpu_hours = resources["gpus"] * checkpoint["max_stale_minutes"] / 60
queue_p95_minutes = p95(case["queue_wait_minutes"])

results["training_platform_design_gate"] = (
    case["design_review_passed"]
    and case["p0_risks"] == 0
    and queue_p95_minutes <= case["queue_slo_minutes"]
    and max_lost_gpu_hours <= case["lost_gpu_hour_budget"]
    and trace_coverage >= 0.95
    and effective_batch == case["expected_effective_batch"]
)
return results

```

```

def make_complete_case():
    return {
        "name": "complete_training_platform_design",
        "requirements": {
            "workload_scope": True,
            "cluster_scale": True,
            "multi_tenant": True,
            "data_sensitivity": True,
            "preemption": True,
            "recovery_slo": True,
            "artifact_release": True,
            "cost_budget": True,
        },
        "trainingjob": {
            "owner": True,

```

```

    "project": True,
    "image_digest": True,
    "code_commit": True,
    "dataset_version": True,
    "final_config_hash": True,
    "resources": True,
    "distributed": True,
    "checkpoint_policy": True,
    "observability": True,
    "security": True,
    "queue": True,
  },
  "states": {
    "submitted",
    "validated",
    "queued",
    "scheduled",
    "starting",
    "running",
    "checkpointing",
    "retrying",
    "failed",
    "completed",
    "cancelled",
  },
  "state_transitions_valid": True,
  "admission": {
    "schema": True,
    "quota_dry_run": True,
    "priority_approval": True,
    "image_policy": True,
    "dataset_permission": True,
    "idempotency": True,
  },
  "scheduler": {
    "queue": True,
    "quota": True,
    "fair_share": True,
    "gang": True,
    "topology": True,
    "backfilling": True,
    "preemption": True,
    "fragmentation": True,
  },
  "resources": {
    "nodes": 8,
    "processes_per_node": 8,
    "gpus": 64,
    "micro_batch": 2,
    "grad_accum": 8,
  },
  "launcher": {
    "world_size": 64,
    "rank_mapping": True,
    "rendezvous": True,
  },

```

```

    "gpu_binding": True,
    "nccl": True,
    "adapter": True,
},
"lineage": {
    "image_digest": True,
    "code_commit": True,
    "dataset_version": True,
    "manifest": True,
    "final_config_hash": True,
    "env_versions": True,
    "seed": True,
    "experiment_run": True,
},
"checkpoint": {
    "policy": True,
    "committed_manifest": True,
    "checksum": True,
    "restore_drill": True,
    "scheduler_preemption_hook": True,
    "max_stale_minutes": 12,
},
"observability": {
    "logs": True,
    "metrics": True,
    "events": True,
    "traces": True,
    "rank_dimensions": True,
    "experiment_link": True,
    "dashboard": True,
    "alerts": True,
    "linked_artifacts": 8,
    "required_artifacts": 8,
},
"fault": {
    "taxonomy": True,
    "retry_budget": True,
    "backoff": True,
    "checkpoint_resume": True,
    "fast_fail_nonretryable": True,
    "diagnosis_report": True,
},
"security": {
    "identity": True,
    "rbac_abac": True,
    "data_purpose": True,
    "runtime_isolation": True,
    "image_supply_chain": True,
    "audit_log": True,
    "incident_response": True,
},
"cost": {
    "gpu_hours": True,
    "storage": True,
    "network": True,

```

```

        "owner_project_job": True,
        "budget_alert": True,
        "utilization_report": True,
        "attribution_rate": 0.96,
    },
    "developer": {
        "portal": True,
        "cli": True,
        "sdk": True,
        "templates": True,
        "status_page": True,
        "error_report": True,
        "rerun_from_checkpoint": True,
    },
    "interfaces": {
        "api_contract": True,
        "scheduler_handoff": True,
        "launcher_handoff": True,
        "data_contract": True,
        "artifact_registry": True,
        "audit_event_schema": True,
    },
    "tradeoffs": {
        "kubernetes_vs_training_operator": True,
        "fairness_vs_utilization": True,
        "topology_vs_queue_time": True,
        "preemption_vs_lost_work": True,
        "self_service_vs_guardrail": True,
        "cost_vs_reliability": True,
    },
    "queue_wait_minutes": [6, 10, 12, 18, 20, 24],
    "queue_slo_minutes": 30,
    "lost_gpu_hour_budget": 16,
    "expected_effective_batch": 1024,
    "design_review_passed": True,
    "p0_risks": 0,
}

```

```

def clone_case(case, name):
    copied = {}
    for key, value in case.items():
        if isinstance(value, dict):
            copied[key] = dict(value)
        elif isinstance(value, set):
            copied[key] = set(value)
        elif isinstance(value, list):
            copied[key] = list(value)
        else:
            copied[key] = value
    copied["name"] = name
    return copied

```

```

def make_bad_cases(complete):

```

```

bad_cases = []

case = clone_case(complete, "requirement_scope_missing_bad")
case["requirements"]["recovery_slo"] = False
bad_cases.append(case)

case = clone_case(complete, "trainingjob_contract_missing_bad")
case["trainingjob"]["final_config_hash"] = False
bad_cases.append(case)

case = clone_case(complete, "state_machine_invalid_bad")
case["states"].remove("retrying")
case["state_transitions_valid"] = False
bad_cases.append(case)

case = clone_case(complete, "admission_quota_policy_missing_bad")
case["admission"]["quota_dry_run"] = False
bad_cases.append(case)

case = clone_case(complete, "scheduler_gang_topology_missing_bad")
case["scheduler"]["gang"] = False
bad_cases.append(case)

case = clone_case(complete, "launcher_world_size_mismatch_bad")
case["launcher"]["world_size"] = 32
bad_cases.append(case)

case = clone_case(complete, "data_config_lineage_gap_bad")
case["lineage"]["dataset_version"] = False
bad_cases.append(case)

case = clone_case(complete, "checkpoint_recovery_uncoupled_bad")
case["checkpoint"]["restore_drill"] = False
bad_cases.append(case)

case = clone_case(complete, "observability_experiment_unlinked_bad")
case["observability"]["traces"] = False
bad_cases.append(case)

case = clone_case(complete, "fault_recovery_blind_retry_bad")
case["fault"]["fast_fail_nonretryable"] = False
bad_cases.append(case)

case = clone_case(complete, "security_audit_gap_bad")
case["security"]["audit_log"] = False
bad_cases.append(case)

case = clone_case(complete, "cost_capacity_unowned_bad")
case["cost"]["owner_project_job"] = False
bad_cases.append(case)

case = clone_case(complete, "developer_self_service_gap_bad")
case["developer"]["status_page"] = False
bad_cases.append(case)

```



```

case = clone_case(complete, "interface_artifact_contract_gap_bad")
case["interfaces"]["artifact_registry"] = False
bad_cases.append(case)

case = clone_case(complete, "tradeoff_boundary_missing_bad")
case["tradeoffs"]["topology_vs_queue_time"] = False
bad_cases.append(case)

case = clone_case(complete, "design_gate_missing_bad")
case["design_review_passed"] = False
case["p0_risks"] = 1
case["queue_wait_minutes"] = [45, 60, 90]
bad_cases.append(case)

return bad_cases

def summarize(cases):
    all_results = {case["name"]: evaluate_case(case) for case in cases}
    metrics = {}
    for gate in GATES:
        passed = sum(1 for result in all_results.values() if result[gate])
        metrics[gate] = round(passed / len(cases), 3)
    failed_cases = [
        name for name, result in all_results.items()
        if not all(result[gate] for gate in GATES)
    ]
    failed_gates = [gate for gate in GATES if metrics[gate] < 1.0]
    return all_results, metrics, failed_cases, failed_gates

complete_case = make_complete_case()
cases = [complete_case] + make_bad_cases(complete_case)
results, metrics, failed_cases, failed_gates = summarize(cases)

world_size = complete_case["resources"]["nodes"] * complete_case["resources"]["processes_per_node"]
effective_batch = world_size * complete_case["resources"]["micro_batch"] * complete_case["resources"]["grad_accum"]
queue_p95_minutes = p95(complete_case["queue_wait_minutes"])
max_lost_gpu_hours = complete_case["resources"]["gpus"] * complete_case["checkpoint"]["max_stale_minutes"] / 60
trace_coverage = complete_case["observability"]["linked_artifacts"] / complete_case["observability"]["required_artifacts"]

examples = {
    "world_size": world_size,
    "effective_batch": effective_batch,
    "queue_p95_minutes": queue_p95_minutes,
    "max_lost_gpu_hours": round(max_lost_gpu_hours, 1),
    "trace_coverage": round(trace_coverage, 3),
    "cost_attribution_rate": complete_case["cost"]["attribution_rate"],
}

smoke = {
    "complete_case_passes": all(results["complete_training_platform_design"].values()),
    "caught_missing_requirements": not results["requirement_scope_missing_bad"]["requirement_clarification"],
    "caught_scheduler_gap": not results["scheduler_gang_topology_missing_bad"]["scheduler_resource_fit"],
    "caught_lineage_gap": not results["data_config_lineage_gap_bad"]["data_config_lineage"],
}

```

```

    "caught_security_gap": not results["security_audit_gap_bad"]["security_audit_governance"],
    "caught_gate_gap": not results["design_gate_missing_bad"]["training_platform_design_gate"],
}

print(f"training_platform_design_examples={examples}")
print(f"smoke={smoke}")
print(f"metrics={metrics}")
print(f"hard_blocker_count={len(failed_cases)}")
print(f"failed_cases={failed_cases}")
print(f"failed_gates={failed_gates}")
print(f"training_platform_design_gate_pass={len(failed_cases) == 0}")

```

运行后应看到类似输出：

```

training_platform_design_examples={'world_size': 64, 'effective_batch': 1024, 'queue_p95_minutes': 24, 'max_lost_gpu_ho
smoke={'complete_case_passes': True, 'caught_missing_requirements': True, 'caught_scheduler_gap': True, 'caught_lineage
metrics={'requirement_clarification': 0.941, 'trainingjob_contract': 0.941, 'lifecycle_state_machine': 0.941, 'admission
hard_blocker_count=16
failed_cases=['requirement_scope_missing_bad', 'trainingjob_contract_missing_bad', 'state_machine_invalid_bad', 'admis
failed_gates=['requirement_clarification', 'trainingjob_contract', 'lifecycle_state_machine', 'admission_quota_policy'
training_platform_design_gate_pass=False

```

这段输出的含义是：完整设计可以通过，但 16 个 bad case 分别暴露了需求、契约、状态机、准入、调度、launcher、血缘、checkpoint、观测、容错、安全、成本、自助、接口、trade-off 和最终门禁的缺口。面试时你不需要把 demo 背下来，但应该能用同样口径说明每个模块为什么存在、怎么验收、失败时怎么定位。

25.18 常见扣分回答

扣分回答一：只说“用 Kubernetes 跑 Job”。

问题是没有讲训练语义、分布式启动、checkpoint、数据、实验和容错。

扣分回答二：没有 TrainingJob 抽象。

没有核心对象，系统会变成脚本堆积。

扣分回答三：没有 gang scheduling。

分布式训练不能只靠普通 Pod 调度。

扣分回答四：没有数据版本和配置复现。

训练平台必须能回答模型怎么训出来。

扣分回答五：没有 checkpoint 和恢复。

大模型训练长周期，没有恢复能力风险很高。

扣分回答六：没有安全和审计。

训练平台连接数据、GPU、代码和模型产物，安全必不可少。

25.19 面试题

题 1：训练平台的核心对象是什么？

答：TrainingJob。它描述一次训练任务的镜像、代码、数据、配置、资源、分布式启动、checkpoint、日志、队列、优先级、安全和 metadata，是训练生命周期管理的核心对象。

题 2：为什么不能只用 Kubernetes Job？

答：Kubernetes Job 只提供通用任务运行能力，不理解训练语义。大模型训练还需要分布式启动、gang scheduling、拓扑感知、checkpoint、实验追踪、数据版本、容错、安全审计和成本归因。

题 3：训练平台如何保证可复现？

答：记录镜像 digest、代码 commit、数据版本、最终生效配置、环境版本、随机种子、启动命令、checkpoint、评估配置和平台策略版本。

题 4：训练平台如何处理失败？

答：先分类故障。节点、临时网络、Pod 驱逐等可恢复故障可以从 checkpoint 自动重试；配置错误、权限错误、代码 bug、持续 OOM 和数值发散应快速失败并生成诊断报告。

题 5：训练平台如何做多租户治理？

答：通过身份认证、RBAC/ABAC、队列、配额、优先级、资源池、数据权限、镜像策略、密钥管理、日志权限、模型产物权限、审计和成本归因实现。

25.20 小练习

练习一：画训练平台架构图。

要求：包含入口、TrainingJob API、校验、调度、launcher、K8s、数据层、checkpoint、观测、实验追踪、模型仓库、审计和成本。

练习二：设计 TrainingJob 状态机。

要求：包含 submitted、validated、queued、scheduled、starting、running、checkpointing、retrying、failed、completed、cancelled。

练习三：设计训练平台容错策略。

要求：区分 node failure、NCCL timeout、OOM、permission denied、loss NaN 的处理方式。

练习四：设计训练平台权限模型。

要求：说明用户、TrainingJob、Service Account、数据集、checkpoint、模型产物之间的权限关系。

25.21 本章小结

本章用系统设计题串起了大模型训练平台。

你需要掌握：

1. 训练平台是 TrainingJob 生命周期管理系统。
2. 系统要覆盖提交、校验、调度、启动、运行、观测、checkpoint、恢复、产物、审计和成本。
3. TrainingJob 是核心抽象。
4. 调度要支持队列、优先级、配额、gang scheduling、拓扑感知和抢占。
5. Launcher 要统一封装 torchrun、DeepSpeed、Megatron、Ray。
6. 数据层要支持版本、manifest、缓存、shuffle 和权限。
7. Checkpoint 是容错、抢占和发布的基础。
8. 可观测性要包括 logs、metrics、events、traces。
9. 容错要做故障分类，而不是盲目重试。
10. 安全、审计和成本治理是企业训练平台必须能力。

下一章开始进入第四部分：大模型推理平台工程。

第 26 章 推理平台总览：从单模型服务到多模型平台

前面第三部分讲完了大模型训练平台。从本章开始进入第四部分：大模型推理平台工程。

训练平台解决“模型怎么训出来”，推理平台解决“模型怎么稳定、低延迟、低成本地服务线上请求”。推理平台和训练平台同样重要，因为模型效果再好，如果推理延迟高、吞吐低、成本失控、版本不可回滚，也无法真正落地。

先记住一句话：

大模型推理平台不是把模型包成一个 HTTP 服务，而是围绕延迟、吞吐、并发、成本、版本、安全和可用性构建的在线模型服务系统。

26.0 本讲资料边界与第二轮精修口径

本讲按 `WRITING_PLAN.md` 做第二轮精修时，重点参考 vLLM 对 PagedAttention、KV cache block 和 continuous batching 的公开口径，Hugging Face TGI 对 streaming、Prometheus / OpenTelemetry 指标、continuous batching 和生产推理服务的口径，NVIDIA Triton Inference Server 对 model repository、dynamic batching、metrics 和多模型推理服务的口径，MLflow Model Registry 对模型版本、stage / alias、artifact 管理的口径，以及 OpenTelemetry 对 traces、metrics、logs 等 signals 的通用可观测性边界。

这些资料说明：推理平台不是单一 runtime，也不是 Kubernetes 上的一个 Deployment。runtime 解决高性能执行，平台解决从请求进入到 token 返回的完整生命周期、模型版本治理、SLO、灰度、限流、缓存、观测、安全和成本归因。

本章只讲稳定抽象，不绑定某个框架版本或厂商实现。你可以用 vLLM、TGI、Triton、TensorRT-LLM、SGLang 或自研 runtime，但面试回答必须能解释以下边界：

1. 请求生命周期：认证、限流、路由、排队、prefill、decode、streaming、计费 and 审计。
2. 性能指标：TTFT、TPOT、P95 / P99、tokens/s、queue time、KV cache pressure 和错误率。
3. 模型治理：Model Registry、ModelVersion、Endpoint、Deployment、RoutePolicy、ScalingPolicy 和 RolloutPolicy。
4. 运行时职责：模型加载、显存管理、KV cache、continuous batching、streaming 和 runtime metrics。
5. 平台职责：SLO、灰度、回滚、限流、降级、安全策略、成本归因和开发者自助。

26.1 从单模型服务开始

最简单的推理服务长这样：

Client -> HTTP API -> Model Server -> GPU -> Response

一个模型，一个服务，一个 GPU 或一组 GPU。

这种方式适合：

1. Demo。
2. 小规模内部服务。
3. 单一模型场景。
4. 流量很低的实验。

但生产环境很快会遇到问题：

1. 多个模型如何部署？
2. 请求如何路由到合适模型？
3. 高峰流量如何扩容？
4. 延迟如何控制？
5. GPU 利用率如何提高？
6. 模型版本如何灰度和回滚？
7. 成本如何统计？
8. 某个模型故障如何降级？

这就需要推理平台。

26.2 推理平台要解决什么问题

推理平台要解决十个核心问题：

1. 模型部署。
2. 请求接入。
3. 模型路由。
4. 推理运行时。
5. batching 和队列调度。
6. KV cache 管理。
7. 自动扩缩容。
8. 限流、熔断、超时和降级。
9. 灰度、A/B 测试和回滚。
10. 监控、安全和成本治理。

这些问题组合起来，才是完整推理平台。

26.3 推理和训练的差异

训练关注：

1. 长时间运行。
2. 吞吐。
3. checkpoint。
4. 分布式通信。
5. 收敛稳定性。
6. 实验可复现。

推理关注：

1. 在线延迟。
2. QPS。
3. 并发。
4. 可用性。
5. 弹性扩缩容。
6. 成本。
7. 版本发布。
8. 用户体验。

训练任务可以排队几小时，线上推理请求不行。

训练失败可以从 checkpoint 恢复，推理服务故障要快速切流或降级。

所以推理平台是在线服务系统，不是训练平台的附属品。

26.4 推理平台总体架构

一个典型推理平台架构：

Client / App / Agent

- > API Gateway
- > Auth / Rate Limit / Safety
- > Model Router
- > Request Queue / Scheduler
- > Inference Runtime
 - > Model Worker Pods
 - > GPU / KV Cache / Batching
- > Model Registry
- > Cache Layer
- > Observability: Metrics / Logs / Traces

- > Autoscaler
- > Deployment Controller
- > Cost / Audit

模块说明:

1. API Gateway: 统一入口、认证、限流。
2. Safety: 输入输出安全策略。
3. Model Router: 选择模型、版本和实例。
4. Request Queue: 请求排队和 batching。
5. Inference Runtime: 执行 prefill、decode 和生成。
6. Model Registry: 管理模型版本、权重、tokenizer 和配置。
7. Cache Layer: prompt cache、KV cache、语义缓存和结果缓存。
8. Observability: TTFT、TPOT、QPS、错误率、GPU、成本。
9. Autoscaler: 自动扩缩容。
10. Deployment Controller: 灰度、回滚、A/B 测试。
11. Cost / Audit: 成本和访问审计。

26.5 推理平台的核心指标

推理平台常见指标:

1. QPS。
2. 并发数。
3. TTFT。
4. TPOT。
5. end-to-end latency。
6. p50 / p95 / p99 latency。
7. input tokens/s。
8. output tokens/s。
9. GPU utilization。
10. KV cache utilization。
11. queue length。
12. error rate。
13. timeout rate。
14. cost per 1k tokens。

TTFT 是 Time To First Token。

TPOT 是 Time Per Output Token。

这两个指标非常重要。用户体感通常受 TTFT 和后续 token 生成速度共同影响。

26.6 Prefill 和 Decode

大模型推理可以粗略分为两个阶段。

Prefill: 处理输入 prompt, 计算所有输入 token 的上下文表示, 生成 KV cache。

Decode: 逐 token 生成输出, 每一步使用已有 KV cache 生成下一个 token。

两阶段资源画像不同:

1. Prefill 更偏计算密集。
2. Decode 更偏显存带宽和 KV cache 访问。
3. 长 prompt 会拉高 prefill 成本。
4. 长输出会拉高 decode 成本。

推理平台必须理解 prefill 和 decode, 否则无法解释延迟和吞吐。

26.7 KV Cache 为什么关键

KV cache 保存历史 token 的 Key 和 Value。它能避免每次生成 token 都重新计算全部上下文。

但 KV cache 会占用大量显存，尤其在：

1. 长上下文。
2. 高并发。
3. 大 batch。
4. 多层大模型。
5. 多用户长对话。

KV cache 管不好，会导致：

1. 显存不足。
2. 请求被驱逐。
3. 延迟抖动。
4. GPU 利用率下降。
5. p99 延迟变差。

因此现代推理 runtime 会重点优化 KV cache，例如分页管理、复用、驱逐、前缀缓存和分离式 prefill/decode。

26.8 从单模型服务到多模型平台

单模型服务只需要回答：这个模型能不能跑起来？

多模型平台要回答更多问题：

1. 哪个业务能调用哪些模型？
2. 哪个模型版本是默认版本？
3. 哪些请求走大模型，哪些请求走小模型？
4. 哪些请求需要更强模型，哪些请求可以降级？
5. 模型服务异常时如何切换？
6. 不同模型的成本如何归因？
7. 模型更新是否影响线上质量？

多模型平台的核心抽象通常包括：

1. Model。
2. ModelVersion。
3. Endpoint。
4. Deployment。
5. Runtime。
6. RoutePolicy。
7. ScalingPolicy。
8. QuotaPolicy。
9. SafetyPolicy。
10. CostPolicy。

这些抽象让平台从“部署一个服务”升级为“管理一组模型能力”。

26.9 Model Registry 的作用

Model Registry 管理模型资产。

它通常记录：

1. 模型名称。
2. 模型版本。
3. 权重路径。
4. tokenizer。

5. 推理配置。
6. 上下文长度。
7. 支持的任务类型。
8. 量化方式。
9. 依赖 runtime。
10. 安全等级。
11. 评估结果。
12. 发布状态。

没有 Model Registry，推理平台很容易变成一堆没人说得清的服务。

26.10 Model Router 的作用

Model Router 负责决定请求去哪。

常见路由依据：

1. 模型名称。
2. 模型版本。
3. 用户或租户。
4. 业务线。
5. 请求类型。
6. prompt 长度。
7. 成本预算。
8. 延迟 SLO。
9. 安全策略。
10. 当前实例负载。

路由不是简单的负载均衡。大模型请求之间差异巨大，一个 200 token 请求和一个 100k token 请求不是同一种负载。

26.11 Runtime 的作用

Inference Runtime 是真正执行推理的组件。

常见 runtime 包括：

1. Triton。
2. vLLM。
3. TGI。
4. SGLang。
5. TensorRT-LLM。
6. 自研 runtime。

Runtime 负责：

1. 加载模型权重。
2. 管理显存。
3. 处理 tokenizer。
4. 执行 prefill。
5. 执行 decode。
6. 管理 KV cache。
7. 支持 batching。
8. 支持 streaming response。
9. 暴露指标。

平台不要把所有逻辑塞进 runtime。更合理的方式是：平台负责治理和编排，runtime 负责高性能执行。

26.12 Batching 为什么重要

大模型推理如果一个请求一个请求跑，GPU 利用率通常很低。

Batching 把多个请求合并执行，可以提高吞吐。

但 batching 会带来延迟权衡：

1. batch 越大，吞吐越高。
2. 等待 batch 的时间越长，TTFT 可能越差。
3. 请求长度差异越大，padding 或调度浪费越明显。
4. 长请求可能拖慢短请求。

所以推理平台常常需要 continuous batching，而不是静态 batch。

Continuous batching 允许新请求在 decode 过程中动态加入，也允许完成的请求动态退出，更适合生成式模型。

26.13 自动扩缩容不是只看 GPU 利用率

传统服务扩缩容经常看 CPU 或 QPS。

大模型推理不够简单。

常见扩缩容信号包括：

1. QPS。
2. 并发请求数。
3. 队列长度。
4. TTFT。
5. p95 / p99 延迟。
6. output tokens/s。
7. GPU 利用率。
8. 显存占用。
9. KV cache 占用。
10. 失败率和超时率。

只看 GPU 利用率可能误判。比如 decode 阶段可能 GPU 利用率不高，但显存带宽或 KV cache 已经成为瓶颈。

26.14 冷启动和热启动

大模型服务冷启动很慢，因为需要：

1. 拉取镜像。
2. 加载模型权重。
3. 初始化 runtime。
4. 分配显存。
5. 加载 tokenizer。
6. 预热 kernel。

冷启动可能耗时几十秒到数分钟。

常见优化方法：

1. 权重本地缓存。
2. 镜像预拉取。
3. 常用模型保留 warm pool。
4. runtime 预热。
5. 按业务峰谷提前扩容。
6. 权重格式优化。

推理平台的扩缩容不能只考虑“要不要扩”，还要考虑“扩出来多久能服务”。

26.15 灰度、A/B 测试和回滚

模型上线比普通服务上线更复杂。

因为模型版本变化会影响：

1. 输出质量。
2. 风格。
3. 安全性。
4. 延迟。
5. 成本。
6. 工具调用行为。
7. 下游业务指标。

推理平台需要支持：

1. 按流量比例灰度。
2. 按租户灰度。
3. 按用户分桶。
4. 按场景灰度。
5. 指标对比。
6. 一键回滚。

没有灰度和回滚，模型发布风险会非常高。

26.16 限流、熔断、超时和降级

推理平台必须保护自己。

常见保护机制：

1. 租户级限流。
2. 模型级限流。
3. endpoint 级限流。
4. 最大上下文长度限制。
5. 最大输出长度限制。
6. 排队超时。
7. 请求执行超时。
8. 熔断异常模型。
9. 降级到小模型。
10. 降级到缓存结果。

大模型推理成本高，一个异常客户端可能迅速打满 GPU 资源。所以限流和配额不是附加功能，而是平台基本能力。

26.17 安全和治理

推理平台安全至少包括：

1. 身份认证。
2. 租户鉴权。
3. 模型访问控制。
4. 输入内容策略。
5. 输出内容策略。
6. 敏感信息保护。
7. 日志脱敏。
8. 密钥保护。
9. 审计追踪。
10. 数据留存策略。

推理请求可能包含用户隐私、企业知识、代码、合同、财务数据和业务决策信息。平台必须明确哪些数据可以记录，哪些数据不能记录，哪些数据需要脱敏。

26.18 成本治理

推理成本通常由这些因素决定：

1. 模型大小。
2. 输入 token 数。
3. 输出 token 数。
4. 并发量。
5. GPU 单价。
6. GPU 利用率。
7. cache 命中率。
8. batch 效率。
9. 量化方式。
10. SLO 要求。

成本治理常见手段：

1. 小模型优先。
2. 分层模型路由。
3. 语义缓存。
4. prompt 压缩。
5. 输出长度控制。
6. 低优先级请求降级。
7. 离峰批处理。
8. endpoint 成本归因。

一个成熟推理平台必须能回答：哪个业务、哪个模型、哪个版本、哪类请求消耗了多少成本。

26.19 一个完整请求流程

一个线上请求的完整流程可能是：

1. Client 发起请求。
2. API Gateway 做认证和基础限流。
3. Safety 模块检查输入策略。
4. Router 根据模型、租户、成本和负载选择 endpoint。
5. Scheduler 将请求放入队列。
6. Runtime 将请求加入 continuous batching。
7. Worker 执行 prefill。
8. Worker 生成 KV cache。
9. Worker 执行 decode。
10. 服务端以 streaming 方式返回 token。
11. Safety 模块检查输出策略。
12. Metrics 记录 TTFT、TPOT、tokens 和错误码。
13. Trace 记录路由、排队、runtime 和返回阶段耗时。
14. Cost 模块做成本归因。
15. Audit 模块记录访问事件。

这才是线上推理平台的一次完整服务链路。

26.20 推理平台总览审计指标与最小 demo

系统设计里最危险的回答，是把推理平台说成“API Gateway + Model Server + GPU”。真正的推理平台要证明请求生命周期、SLO、runtime、缓存、发布、安全和成本都能被观测和治理。

推理平台审计样本可以抽象成：

$u_i=(a_i,r_i,s_i,m_i,p_i,e_i,b_i,k_i,h_i,w_i,g_i,c_i,o_i,z_i)$

其中, a_i 是接入和鉴权, r_i 是路由, s_i 是 SLO 指标, m_i 是 Model Registry, p_i 是 prefill / decode runtime, e_i 是 endpoint 和扩缩容, b_i 是 batching, k_i 是 KV cache, h_i 是灰度和回滚, w_i 是保护机制, g_i 是安全治理, c_i 是成本归因, o_i 是可观测性, z_i 是最终推理平台门禁。

第 j 个维度的覆盖率可以写成:

$$C_j=\frac{1}{N}\sum_{i=1}^N\mathbf{1}[g_j(u_i)=1]$$

一次请求的端到端延迟可以拆成:

$$T_{\mathrm{e2e}}=T_{\mathrm{auth}}+T_{\mathrm{route}}+T_{\mathrm{queue}}+T_{\mathrm{tokenize}}+T_{\mathrm{prefill}}+T_{\mathrm{decode}}$$

TTFT 与 TPOT 可以近似写成:

$$T_{\mathrm{ttft}}=T_{\mathrm{auth}}+T_{\mathrm{route}}+T_{\mathrm{queue}}+T_{\mathrm{tokenize}}+T_{\mathrm{prefill}}+T_{\mathrm{decode}}$$

$$T_{\mathrm{tpot}}=\frac{T_{\mathrm{decode}}}{N_{\mathrm{out}}}$$

KV cache 显存可以用常见估算式:

$$M_{\mathrm{kv}}=2\mathrm{LBSH}_{\mathrm{kv}}D_{\mathrm{h}}B_{\mathrm{elem}}$$

其中, L 是层数, B 是活跃请求数, S 是每个请求的保留 token 数, H_{kv} 是 KV head 数, D_{h} 是 head dimension, B_{elem} 是每个元素字节数。

单位成本可以先按 token 粗算:

$$K_{\mathrm{1k}}=\frac{1000K_{\mathrm{total}}}{N_{\mathrm{in}}+N_{\mathrm{out}}}$$

推理平台上线门禁可以写成:

$$G_{\mathrm{infer}}=\mathbf{1}\left[\min_j C_j\geq \tau_j \wedge T_{\mathrm{ttft},p95}\leq B_{\mathrm{ttft}} \wedge T_{\mathrm{tpot}}\leq K_{\mathrm{1k}}\right]$$

下面的 0 依赖 demo 模拟一个推理平台总览审计。它不替代压测, 但能帮助你把 “推理平台要有哪些模块” 转成可验收的指标表。

```
from math import ceil
```

```
GATES = [
```

```
    "request_lifecycle_coverage",
    "slo_metric_contract",
    "model_registry_readiness",
    "model_router_policy_fit",
    "runtime_prefill_decode_fit",
    "continuous_batching_readiness",
    "kv_cache_capacity_governance",
    "autoscaling_warm_pool_fit",
    "release_rollback_control",
    "rate_limit_degrade_protection",
    "safety_governance_fit",
    "cost_attribution_optimization",
    "observability_trace_coverage",
    "multi_model_governance",
    "developer_api_self_service",
    "inference_platform_gate",
```

```
]
```

```
def p95(values):
```

```
    ordered = sorted(values)
```

```
    index = max(0, ceil(0.95 * len(ordered)) - 1)
```

```

return ordered[index]

def all_present(mapping, names):
    return all(
        name in mapping
        and mapping[name] is not None
        and mapping[name] is not False
        for name in names
    )

def mib(bytes_value):
    return bytes_value / (1024 ** 2)

def evaluate_case(case):
    request = case["request_lifecycle"]
    slo = case["slo"]
    model = case["model_registry"]
    router = case["router"]
    runtime = case["runtime"]
    batching = case["batching"]
    kv = case["kv_cache"]
    autoscaling = case["autoscaling"]
    release = case["release"]
    protection = case["protection"]
    safety = case["safety"]
    cost = case["cost"]
    observability = case["observability"]
    governance = case["multi_model"]
    developer = case["developer"]

    layer_count = kv["layers"]
    active_requests = kv["active_requests"]
    sequence_length = kv["sequence_length"]
    kv_heads = kv["kv_heads"]
    head_dim = kv["head_dim"]
    elem_bytes = kv["elem_bytes"]
    kv_bytes = 2 * layer_count * active_requests * sequence_length * kv_heads * head_dim * elem_bytes
    kv_pressure = kv_bytes / kv["budget_bytes"]

    total_tokens = sum(slo["input_tokens"]) + sum(slo["output_tokens"])
    output_tokens = sum(slo["output_tokens"])
    wall_time_s = max(slo["finish_times_s"]) - min(slo["arrival_times_s"])
    tokens_per_second = total_tokens / wall_time_s
    output_tokens_per_second = output_tokens / wall_time_s
    cost_per_1k = 1000 * cost["total_cost_usd"] / total_tokens
    ttft_p95 = p95(slo["ttft_ms"])
    tpot_p95 = p95(slo["tpot_ms"])
    e2e_p99 = max(slo["e2e_ms"])

    results = {}
    results["request_lifecycle_coverage"] = all_present(
        request,

```

```

    ["auth", "rate_limit", "safety_in", "routing", "queue", "prefill", "decode", "streaming", "safety_out", "metrics"]
)
results["slo_metric_contract"] = all_present(
    slo,
    ["ttft_ms", "tpot_ms", "e2e_ms", "queue_ms", "input_tokens", "output_tokens", "error_count", "timeout_count"],
)
results["model_registry_readiness"] = all_present(
    model,
    ["model_name", "version", "weight_uri", "tokenizer", "runtime", "context_limit", "quantization", "eval_report"],
)
results["model_router_policy_fit"] = all_present(
    router,
    ["tenant_policy", "model_version", "latency_slo", "cost_budget", "load_aware", "fallback", "safety_tier"],
)
results["runtime_prefill_decode_fit"] = all_present(
    runtime,
    ["model_load", "tokenizer", "prefill", "decode", "kv_cache", "streaming", "runtime_metrics"],
)
results["continuous_batching_readiness"] = all_present(
    batching,
    ["dynamic_join", "dynamic_leave", "max_wait_ms", "max_batch_tokens", "priority_queue", "long_request_isolation"],
)
results["kv_cache_capacity_governance"] = (
    all_present(kv, ["block_manager", "eviction", "prefix_cache", "max_context", "admission_control", "metrics"])
    and kv_pressure <= kv["pressure_limit"]
)
results["autoscaling_warm_pool_fit"] = all_present(
    autoscaling,
    ["queue_signal", "ttft_signal", "kv_signal", "tokens_signal", "cold_start_s", "warm_pool", "scale_down_policy"],
)
results["release_rollback_control"] = all_present(
    release,
    ["canary", "ab_test", "traffic_split", "quality_guardrail", "latency_guardrail", "cost_guardrail", "rollback"],
)
results["rate_limit_degrade_protection"] = all_present(
    protection,
    ["tenant_quota", "model_quota", "max_input", "max_output", "queue_timeout", "execution_timeout", "circuit_breaker"],
)
results["safety_governance_fit"] = all_present(
    safety,
    ["authz", "input_policy", "output_policy", "pii_redaction", "log_redaction", "retention", "audit"],
)
results["cost_attribution_optimization"] = (
    all_present(cost, ["tenant", "model", "version", "input_tokens", "output_tokens", "gpu_seconds", "cache_savings"])
    and cost["attribution_rate"] >= 0.9
)
results["observability_trace_coverage"] = all_present(
    observability,
    ["metrics", "logs", "traces", "request_id", "route_span", "queue_span", "prefill_span", "decode_span", "streaming"],
)
results["multi_model_governance"] = all_present(
    governance,
    ["model", "model_version", "endpoint", "deployment", "route_policy", "scaling_policy", "quota_policy", "safety"],
)
results["developer_api_self_service"] = all_present(

```

```

    developer,
    ["openapi", "sdk", "playground", "usage_dashboard", "slo_dashboard", "error_report", "quota_request"],
)

results["inference_platform_gate"] = (
    case["platform_review_passed"]
    and case["p0_risks"] == 0
    and ttft_p95 <= slo["ttft_slo_ms"]
    and tpot_p95 <= slo["tpot_slo_ms"]
    and e2e_p99 <= slo["e2e_slo_ms"]
    and kv_pressure <= kv["pressure_limit"]
    and slo["error_count"] / len(slo["ttft_ms"]) <= slo["error_rate_limit"]
    and tokens_per_second >= slo["tokens_per_second_min"]
)

examples = {
    "ttft_p95_ms": ttft_p95,
    "tpot_p95_ms": tpot_p95,
    "e2e_p99_ms": e2e_p99,
    "tokens_per_second": round(tokens_per_second, 1),
    "output_tokens_per_second": round(output_tokens_per_second, 1),
    "kv_cache_mib": round(mib(kv_bytes), 1),
    "kv_pressure": round(kv_pressure, 3),
    "cost_per_1k_tokens": round(cost_per_1k, 4),
}
return results, examples

```

```

def make_complete_case():
    return {
        "name": "complete_inference_platform",
        "request_lifecycle": {
            "auth": True,
            "rate_limit": True,
            "safety_in": True,
            "routing": True,
            "queue": True,
            "prefill": True,
            "decode": True,
            "streaming": True,
            "safety_out": True,
            "metrics": True,
            "cost": True,
            "audit": True,
        },
        "slo": {
            "ttft_ms": [180, 210, 240, 260, 290, 320],
            "tpot_ms": [18, 19, 20, 22, 24, 26],
            "e2e_ms": [1500, 1680, 1720, 1900, 2100, 2350],
            "queue_ms": [10, 20, 25, 30, 35, 45],
            "input_tokens": [800, 640, 1200, 300, 900, 500],
            "output_tokens": [180, 220, 160, 120, 260, 140],
            "arrival_times_s": [0, 0.2, 0.4, 0.8, 1.0, 1.4],
            "finish_times_s": [1.5, 1.9, 2.1, 2.2, 2.7, 3.0],
            "error_count": 0,
            "timeout_count": 0,
        }
    }

```

```

    "slo_doc": True,
    "ttft_slo_ms": 400,
    "tpot_slo_ms": 35,
    "e2e_slo_ms": 2600,
    "error_rate_limit": 0.01,
    "tokens_per_second_min": 1200,
},
"model_registry": {
    "model_name": True,
    "version": True,
    "weight_uri": True,
    "tokenizer": True,
    "runtime": True,
    "context_limit": True,
    "quantization": True,
    "eval_report": True,
    "stage": True,
    "rollback_version": True,
},
"router": {
    "tenant_policy": True,
    "model_version": True,
    "latency_slo": True,
    "cost_budget": True,
    "load_aware": True,
    "fallback": True,
    "safety_tier": True,
},
"runtime": {
    "model_load": True,
    "tokenizer": True,
    "prefill": True,
    "decode": True,
    "kv_cache": True,
    "streaming": True,
    "runtime_metrics": True,
},
"batching": {
    "dynamic_join": True,
    "dynamic_leave": True,
    "max_wait_ms": True,
    "max_batch_tokens": True,
    "priority_queue": True,
    "long_request_isolation": True,
},
"kv_cache": {
    "layers": 32,
    "active_requests": 6,
    "sequence_length": 4096,
    "kv_heads": 8,
    "head_dim": 128,
    "elem_bytes": 2,
    "budget_bytes": 16 * 1024 ** 3,
    "pressure_limit": 0.8,
    "block_manager": True,

```



```

    "eviction": True,
    "prefix_cache": True,
    "max_context": True,
    "admission_control": True,
    "metrics": True,
},
"autoscaling": {
    "queue_signal": True,
    "ttft_signal": True,
    "kv_signal": True,
    "tokens_signal": True,
    "cold_start_s": True,
    "warm_pool": True,
    "scale_down_policy": True,
},
"release": {
    "canary": True,
    "ab_test": True,
    "traffic_split": True,
    "quality_guardrail": True,
    "latency_guardrail": True,
    "cost_guardrail": True,
    "rollback": True,
},
"protection": {
    "tenant_quota": True,
    "model_quota": True,
    "max_input": True,
    "max_output": True,
    "queue_timeout": True,
    "execution_timeout": True,
    "circuit_breaker": True,
    "fallback_cache": True,
},
"safety": {
    "authz": True,
    "input_policy": True,
    "output_policy": True,
    "pii_redaction": True,
    "log_redaction": True,
    "retention": True,
    "audit": True,
},
"cost": {
    "tenant": True,
    "model": True,
    "version": True,
    "input_tokens": True,
    "output_tokens": True,
    "gpu_seconds": True,
    "cache_savings": True,
    "budget_alert": True,
    "total_cost_usd": 0.19,
    "attribution_rate": 0.97,
},

```

```

"observability": {
    "metrics": True,
    "logs": True,
    "traces": True,
    "request_id": True,
    "route_span": True,
    "queue_span": True,
    "prefill_span": True,
    "decode_span": True,
    "stream_span": True,
    "dashboard": True,
    "alerts": True,
},
"multi_model": {
    "model": True,
    "model_version": True,
    "endpoint": True,
    "deployment": True,
    "route_policy": True,
    "scaling_policy": True,
    "quota_policy": True,
    "safety_policy": True,
    "cost_policy": True,
},
"developer": {
    "openapi": True,
    "sdk": True,
    "playground": True,
    "usage_dashboard": True,
    "slo_dashboard": True,
    "error_report": True,
    "quota_request": True,
},
"platform_review_passed": True,
"p0_risks": 0,
}

```

```

def clone_case(case, name):
    copied = {}
    for key, value in case.items():
        if isinstance(value, dict):
            copied[key] = dict(value)
        elif isinstance(value, list):
            copied[key] = list(value)
        else:
            copied[key] = value
    copied["name"] = name
    return copied

```

```

def make_bad_cases(complete):
    bad_cases = []

    case = clone_case(complete, "request_lifecycle_missing_bad")

```

```

case["request_lifecycle"]["audit"] = False
bad_cases.append(case)

case = clone_case(complete, "slo_metric_contract_missing_bad")
case["slo"]["slo_doc"] = False
bad_cases.append(case)

case = clone_case(complete, "model_registry_gap_bad")
case["model_registry"]["rollback_version"] = False
bad_cases.append(case)

case = clone_case(complete, "router_policy_gap_bad")
case["router"]["cost_budget"] = False
bad_cases.append(case)

case = clone_case(complete, "runtime_prefill_decode_gap_bad")
case["runtime"]["prefill"] = False
bad_cases.append(case)

case = clone_case(complete, "continuous_batching_gap_bad")
case["batching"]["dynamic_leave"] = False
bad_cases.append(case)

case = clone_case(complete, "kv_cache_capacity_gap_bad")
case["kv_cache"]["admission_control"] = False
bad_cases.append(case)

case = clone_case(complete, "autoscaling_warm_pool_gap_bad")
case["autoscaling"]["warm_pool"] = False
bad_cases.append(case)

case = clone_case(complete, "release_rollback_gap_bad")
case["release"]["rollback"] = False
bad_cases.append(case)

case = clone_case(complete, "rate_limit_degrade_gap_bad")
case["protection"]["circuit_breaker"] = False
bad_cases.append(case)

case = clone_case(complete, "safety_governance_gap_bad")
case["safety"]["log_redaction"] = False
bad_cases.append(case)

case = clone_case(complete, "cost_attribution_gap_bad")
case["cost"]["model"] = False
bad_cases.append(case)

case = clone_case(complete, "observability_trace_gap_bad")
case["observability"]["decode_span"] = False
bad_cases.append(case)

case = clone_case(complete, "multi_model_governance_gap_bad")
case["multi_model"]["quota_policy"] = False
bad_cases.append(case)

```

```

case = clone_case(complete, "developer_self_service_gap_bad")
case["developer"]["usage_dashboard"] = False
bad_cases.append(case)

case = clone_case(complete, "inference_platform_gate_missing_bad")
case["platform_review_passed"] = False
case["p0_risks"] = 1
case["slo"]["ttft_ms"] = [600, 650, 700]
bad_cases.append(case)

return bad_cases

def summarize(cases):
    all_results = {}
    examples_by_case = {}
    for case in cases:
        result, examples = evaluate_case(case)
        all_results[case["name"]] = result
        examples_by_case[case["name"]] = examples

    metrics = {}
    for gate in GATES:
        passed = sum(1 for result in all_results.values() if result[gate])
        metrics[gate] = round(passed / len(cases), 3)

    failed_cases = [
        name for name, result in all_results.items()
        if not all(result[gate] for gate in GATES)
    ]
    failed_gates = [gate for gate in GATES if metrics[gate] < 1.0]
    return all_results, examples_by_case, metrics, failed_cases, failed_gates

complete_case = make_complete_case()
cases = [complete_case] + make_bad_cases(complete_case)
results, examples_by_case, metrics, failed_cases, failed_gates = summarize(cases)
examples = examples_by_case["complete_inference_platform"]

smoke = {
    "complete_case_passes": all(results["complete_inference_platform"].values()),
    "caught_missing_lifecycle": not results["request_lifecycle_missing_bad"]["request_lifecycle_coverage"],
    "caught_router_gap": not results["router_policy_gap_bad"]["model_router_policy_fit"],
    "caught_kv_gap": not results["kv_cache_capacity_gap_bad"]["kv_cache_capacity_governance"],
    "caught_observability_gap": not results["observability_trace_gap_bad"]["observability_trace_coverage"],
    "caught_gate_gap": not results["inference_platform_gate_missing_bad"]["inference_platform_gate"],
}

print(f"inference_platform_examples={examples}")
print(f"smoke={smoke}")
print(f"metrics={metrics}")
print(f"hard_blocker_count={len(failed_cases)}")
print(f"failed_cases={failed_cases}")
print(f"failed_gates={failed_gates}")
print(f"inference_platform_gate_pass={len(failed_cases) == 0}")

```

运行后应看到类似输出：

```
inference_platform_examples={'ttft_p95_ms': 320, 'tpot_p95_ms': 26, 'e2e_p99_ms': 2350, 'tokens_per_second': 1806.7, 'or
smoke={'complete_case_passes': True, 'caught_missing_lifecycle': True, 'caught_router_gap': True, 'caught_kv_gap': True
metrics={'request_lifecycle_coverage': 0.941, 'slo_metric_contract': 0.941, 'model_registry_readiness': 0.941, 'model_r
hard_blocker_count=16
failed_cases=['request_lifecycle_missing_bad', 'slo_metric_contract_missing_bad', 'model_registry_gap_bad', 'router_po
failed_gates=['request_lifecycle_coverage', 'slo_metric_contract', 'model_registry_readiness', 'model_router_policy_f
inference_platform_gate_pass=False
```

这段输出的含义是：完整样本通过，但 16 个 bad case 分别暴露请求生命周期、SLO、Model Registry、Router、runtime、batching、KV cache、扩缩容、发布回滚、保护机制、安全、成本、观测、多模型治理、开发者自助和最终门禁的缺口。面试时不需要背代码，但要能把这些维度讲成一条可观测、可回滚、可归因的在线服务链路。

26.21 常见误区

误区一：推理平台就是 K8s 部署模型服务。

K8s 只是底座。推理平台还需要路由、runtime、batching、cache、扩缩容、灰度、限流、观测和成本治理。

误区二：QPS 越高越好。

推理平台不能只看 QPS，还要看 token 吞吐、TTFT、TPOT、p99 和成本。

误区三：GPU 利用率高就是好。

GPU 利用率高但 p99 延迟崩掉，也不是好系统。在线服务要在吞吐和延迟之间平衡。

误区四：大模型统一上最大模型。

很多请求不需要最大模型。分层路由、小模型优先和按需升级是成本治理的关键。

误区五：灰度只看错误率。

模型灰度还要看质量、安全、延迟、成本和业务指标。

26.22 面试回答模板

如果面试官问：“你怎么设计一个大模型推理平台？”

可以这样回答：

我会先澄清业务需求，包括模型数量、QPS、上下文长度、延迟 SLO、流式返回、租户隔离和成本目标。

架构上，我会把平台拆成 API Gateway、鉴权限流、Model Router、Inference Runtime、Model Registry、缓存层、自动扩缩容、发布

请求进入后先经过认证和限流，再由 Router 根据模型版本、租户、负载、SLO 和成本选择 endpoint。Runtime 负责模型加载、prefill

发布上支持灰度、A/B 测试和回滚。稳定性上支持超时、熔断、降级和缓存。成本上按租户、模型、版本和 token 用量做归因，并通

这个回答覆盖了需求、架构、关键指标、稳定性和成本，比较完整。

26.23 小练习

1. 为什么大模型推理平台不能只看 QPS？
2. TTFT 和 TPOT 分别代表什么？
3. Prefill 和 decode 的资源瓶颈有什么不同？
4. KV cache 为什么会成为显存瓶颈？
5. 为什么推理平台需要 Model Router？
6. 自动扩缩容为什么不能只看 GPU 利用率？
7. 大模型服务冷启动慢的原因是什么？
8. 模型灰度发布应该观察哪些指标？

26.24 本章小结

本章从单模型服务讲到多模型推理平台。

你需要记住：

1. 推理平台是在线服务系统，不只是模型 HTTP 包装。
2. 推理平台核心目标是低延迟、高吞吐、高可用和低成本。
3. TTFT、TPOT、p95/p99、tokens/s、队列长度和成本是关键指标。
4. Prefill、decode 和 KV cache 是理解大模型推理性能的基础。
5. 多模型平台需要 Model Registry、Model Router、Runtime、缓存、扩缩容、灰度和治理能力。
6. 成熟推理平台必须同时管理性能、稳定性、安全和成本。

下一章我们会继续深入模型服务运行时，讲 Triton、vLLM、TGI、SGLang 和自研 runtime 的差异。

第 27 章 模型服务运行时：Triton、vLLM、TGI、SGLang 与自研 runtime

上一章讲了推理平台总览。本章聚焦一个更底层也更关键的问题：模型服务运行时，也就是 inference runtime。

推理平台负责把请求接进来、路由、限流、扩缩容、灰度和治理；runtime 负责真正把模型跑起来，把 token 生成出来。

先记住一句话：

推理平台管“服务化和治理”，模型服务运行时管“高性能执行”。

如果 runtime 选型错误，平台层做得再漂亮，也很难获得好的延迟、吞吐和成本。

27.0 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做过资料校准。重点参考的是 NVIDIA Triton Inference Server 官方文档中 model repository、dynamic batching、statistics / metrics、backend 与多模型推理服务的边界；vLLM 官方文档中 PagedAttention、automatic prefix caching、production metrics、OpenAI-compatible server、量化和分布式 serving 的边界；Hugging Face Text Generation Inference 官方文档中 streaming、Prometheus 指标、PagedAttention 和当前 maintenance mode 的说明；SGLang 官方文档中 structured output、RadixAttention / cache、multi-GPU serving、PD disaggregation 和 serving API 的边界；以及 TensorRT-LLM 官方文档中 NVIDIA GPU 上的高性能 LLM inference、batching、KV cache 和 TensorRT / Triton 生态集成边界。

这些资料说明一件事：runtime 选型不是“哪个框架名气最大”，而是要把模型结构、权重格式、tokenizer / chat template、硬件拓扑、prefill / decode、continuous batching、KV cache、streaming、量化、分布式、指标、发布回滚、维护状态和成本放到同一张表里判断。

本章只抽象截至 2026-06 仍稳定的工程口径，不把某个 benchmark 排名、某个版本的默认参数、某个云厂商镜像或某次社区测评写成通用结论。Triton 更偏通用多模型推理服务框架；vLLM、SGLang 更偏现代 LLM serving runtime；TGI 对 Hugging Face 生态仍有学习价值，但当前官方维护状态会影响新项目选型；TensorRT-LLM 更偏 NVIDIA 生态下的深度性能优化；自研 runtime 只有在规模、模型结构、硬件或调度模式足够特殊时才值得投入。

27.1 什么是模型服务运行时

模型服务运行时是一个负责执行模型推理的系统组件。

它通常负责：

1. 加载模型权重。
2. 加载 tokenizer。
3. 管理 GPU 显存。
4. 接收推理请求。
5. 执行 prefill。

6. 执行 decode。
7. 管理 KV cache。
8. 做 batching。
9. 输出流式 token。
10. 暴露指标和健康检查。

从外部看，runtime 可能只是一个 HTTP 或 gRPC 服务。

从内部看，它是一个复杂的 GPU 执行引擎。

27.2 Runtime 和平台层的边界

很多团队会犯一个错误：把平台治理逻辑塞进 runtime，或者把 runtime 的高性能执行逻辑塞进平台层。

更合理的边界是：

平台层负责：

1. 模型注册。
2. 版本发布。
3. 请求鉴权。
4. 限流配额。
5. 模型路由。
6. 灰度回滚。
7. 自动扩缩容。
8. 成本归因。
9. 审计治理。

Runtime 负责：

1. 模型加载。
2. GPU kernel 执行。
3. batching。
4. KV cache。
5. token 生成。
6. streaming。
7. 单实例指标。

这条边界很重要。平台层要能替换 runtime，runtime 也不要依赖某个业务平台才能运行。

27.3 一个 Runtime 的内部结构

一个典型 LLM runtime 内部可能包含：

API Server

- > Request Parser
- > Tokenizer
- > Scheduler
- > KV Cache Manager
- > Batch Manager
- > Model Executor
- > Sampler
- > Streamer
- > Metrics Exporter

每个模块的作用：

1. API Server：接收请求。
2. Request Parser：解析 prompt、参数和停止条件。
3. Tokenizer：把文本转成 token。
4. Scheduler：决定哪些请求进入下一步执行。

5. KV Cache Manager: 分配和回收 KV cache。
6. Batch Manager: 组织 batch 或 continuous batch。
7. Model Executor: 执行模型前向计算。
8. Sampler: 执行 temperature、top-p、top-k 等采样。
9. Streamer: 流式返回 token。
10. Metrics Exporter: 输出延迟、吞吐、显存等指标。

面试中如果能讲出这些内部模块，说明你不是只会“部署模型服务”。

27.4 Triton 的定位

Triton Inference Server 是一个通用推理服务框架。

它的优势是：

1. 支持多种模型格式。
2. 支持多后端。
3. 支持 HTTP 和 gRPC。
4. 支持模型管理。
5. 支持动态 batching。
6. 支持 Prometheus 指标。
7. 适合传统深度学习模型服务。

Triton 对以下场景很常见：

1. CV 模型。
2. 推荐模型。
3. embedding 模型。
4. 多模型统一 serving。
5. TensorRT 优化模型。

但对生成式大模型，Triton 通常需要结合 TensorRT-LLM 或其他后端使用，否则很难直接覆盖现代 LLM 推理所需的 continuous batching、KV cache 管理和长上下文优化。

简单说：Triton 是通用 serving 框架，不是只为 LLM 生成推理设计的 runtime。

27.5 vLLM 的定位

vLLM 是面向大语言模型生成推理的 runtime。

它最知名的能力之一是 PagedAttention。

PagedAttention 的核心思想是：像操作系统管理虚拟内存一样管理 KV cache，把 KV cache 拆成块，减少显存碎片，提高并发能力。

vLLM 的常见优势：

1. 适合 LLM 文本生成。
2. 支持 continuous batching。
3. KV cache 管理能力强。
4. OpenAI-compatible API 生态友好。
5. 部署和使用门槛相对低。
6. 社区活跃。

vLLM 适合：

1. 通用文本生成服务。
2. Chat API。
3. 多并发在线推理。
4. 对吞吐和易用性都有要求的团队。

但 vLLM 也不是万能的。遇到极致性能、特定硬件、深度定制调度、复杂多模态或特殊模型结构时，可能需要额外工程改造或选择其他 runtime。

27.6 TGI 的定位

TGI 是 Text Generation Inference，常用于 Hugging Face 模型生态的文本生成服务。

它的优势是：

1. Hugging Face 模型生态集成好。
2. 文本生成 API 完整。
3. 支持 streaming。
4. 支持 batching。
5. 支持常见生成参数。
6. 工程化程度较高。

TGI 适合：

1. Hugging Face 模型快速上线。
2. 标准文本生成服务。
3. 团队希望少做底层改造。
4. 与 HF 生态深度结合的场景。

如果团队的模型、权重、tokenizer、部署流程大量围绕 Hugging Face，TGI 是一个自然选项。

但如果你需要更细粒度的调度控制、极致吞吐优化或复杂 agent runtime 集成，仍然要评估它和其他 runtime 的差异。尤其要注意版本边界：当前官方文档已把 TGI 标注为维护模式，新项目选型时不能只看历史生态成熟度，还要看后续维护、兼容性和安全更新责任。

27.7 SGLang 的定位

SGLang 既可以看作推理 runtime，也可以看作面向结构化生成和复杂 LLM 程序的执行系统。

它关注的不仅是单次 chat completion，还包括更复杂的 LLM 调用模式，例如：

1. 多轮生成。
2. 结构化输出。
3. 分支推理。
4. 并行采样。
5. 受约束生成。
6. agent 或 workflow 式调用。

SGLang 的价值在于：当上层应用不是简单的一问一答，而是有复杂控制流和生成模式时，runtime 可以更理解这些调用结构，从而做更好的执行优化。

适合场景包括：

1. 复杂 agent 服务。
2. 结构化生成服务。
3. 多阶段推理任务。
4. 需要高效复用上下文的任务。

如果说 vLLM 更像高性能 LLM serving engine，那么 SGLang 更强调 LLM program 的执行效率和表达能力。

27.8 TensorRT-LLM 的定位

TensorRT-LLM 更偏极致性能优化。

它常用于 NVIDIA GPU 上的大模型推理优化，关注 kernel、图优化、量化、并行和推理吞吐。

它的优势是：

1. 性能潜力高。
2. 与 NVIDIA 生态结合深。
3. 支持多种优化策略。
4. 适合大规模生产优化。
5. 可结合 Triton 做服务化。

但它的使用门槛通常更高：

1. 构建流程更复杂。
2. 模型转换成本更高。
3. 调试难度更大。
4. 对硬件和版本依赖更强。

如果目标是快速上线，可能优先考虑 vLLM 或 TGI。如果目标是大规模稳定流量下压低单位 token 成本，TensorRT-LLM 可能更有价值。

27.9 自研 Runtime 什么时候值得做

大多数团队不应该一开始就自研 runtime。

自研 runtime 成本很高，因为你要处理：

1. 模型加载。
2. 分布式推理。
3. kernel 优化。
4. KV cache 管理。
5. continuous batching。
6. speculative decoding。
7. 量化。
8. 多模型并发。
9. 长上下文。
10. 故障恢复。
11. 指标和调试。

但在一些场景下，自研是合理的：

1. 流量规模巨大，单位成本下降收益明显。
2. 模型结构高度定制。
3. 硬件不是主流 GPU。
4. 需要特殊调度策略。
5. 需要深度结合业务调用模式。
6. 开源 runtime 无法满足稳定性或性能要求。

自研 runtime 的前提是团队具备系统、CUDA、编译优化、分布式和模型结构理解能力。

27.10 Runtime 选型维度

选择 runtime 时，不要只问“哪个最快”。

应该看这些维度：

1. 支持哪些模型结构。
2. 是否支持目标硬件。
3. 是否支持 continuous batching。
4. KV cache 管理能力如何。
5. 长上下文支持如何。
6. streaming 能力如何。
7. OpenAI-compatible API 支持如何。
8. 量化支持如何。
9. 多 GPU 和多节点支持如何。

10. 指标是否完整。
11. 部署复杂度。
12. 社区活跃度。
13. 调试难度。
14. 与平台层集成成本。
15. 单位 token 成本。

runtime 选型是工程决策，不是排行榜决策。

27.11 性能指标怎么比较

比较 runtime 性能时，至少要固定以下条件：

1. 同一模型。
2. 同一权重格式。
3. 同一硬件。
4. 同一精度或量化方式。
5. 同一输入长度分布。
6. 同一输出长度分布。
7. 同一并发模型。
8. 同一 SLO。
9. 同一采样参数。
10. 同一 streaming 设置。

否则比较没有意义。

常见错误是只测一个短 prompt 的 tokens/s，然后宣称某个 runtime 更快。

生产环境中真正重要的是：

1. p95 TTFT。
2. p95 TPOT。
3. p99 latency。
4. 达到 SLO 时的最大吞吐。
5. 显存占用。
6. 失败率。
7. 成本 per 1k tokens。

27.12 Runtime 的平台集成方式

平台接入 runtime 时，不应该依赖某个 runtime 的所有细节。

常见方式是定义统一推理接口：

```
InferenceEndpoint
  model
  version
  runtime
  endpoint_url
  max_context_length
  supported_features
  health_status
  metrics
```

请求层统一抽象：

```
GenerateRequest
  prompt/messages
  max_tokens
  temperature
```

```
top_p
stop
stream
user/tenant
trace_id
```

响应层统一抽象：

```
GenerateResponse
  text/tokens
  finish_reason
  usage
  latency
  model_version
  runtime_instance
```

这样平台可以在 vLLM、TGI、SGLang、自研 runtime 之间切换，而不需要重写全部上层逻辑。

27.13 Runtime 指标必须暴露什么

一个可生产使用的 runtime 至少要暴露：

1. 请求数。
2. 成功率。
3. 错误率。
4. TTFT。
5. TPOT。
6. p95 / p99 latency。
7. queue length。
8. active requests。
9. waiting requests。
10. input tokens/s。
11. output tokens/s。
12. GPU utilization。
13. 显存占用。
14. KV cache 使用率。
15. batch size 分布。
16. OOM 次数。
17. timeout 次数。

没有这些指标，线上问题会很难排查。

比如用户说“模型变慢了”，你需要判断慢在哪里：

1. 网关慢？
2. 路由慢？
3. 排队慢？
4. prefill 慢？
5. decode 慢？
6. streaming 慢？
7. 下游客户端慢？

runtime 指标是定位问题的关键。

27.14 Runtime 的稳定性问题

Runtime 常见稳定性问题包括：

1. 模型加载失败。
2. tokenizer 不匹配。

3. 权重格式错误。
4. CUDA 版本不兼容。
5. 显存不足。
6. KV cache 碎片化。
7. 长请求拖慢短请求。
8. streaming 连接异常。
9. batch 调度饥饿。
10. 特定 prompt 触发异常。
11. 指标失真。
12. 版本升级引入性能退化。

因此 runtime 上线前要做：

1. 冷启动测试。
2. 压测。
3. 长上下文测试。
4. 高并发测试。
5. streaming 测试。
6. OOM 测试。
7. 升级回滚测试。

27.15 Runtime 和量化

量化可以降低显存占用和推理成本。

常见量化方向包括：

1. FP16。
2. BF16。
3. INT8。
4. INT4。
5. FP8。
6. weight-only quantization。
7. KV cache quantization。

量化的收益：

1. 显存占用更低。
2. 可部署更大模型。
3. batch 或并发更高。
4. 单位 token 成本更低。

量化的风险：

1. 质量下降。
2. 特定任务退化。
3. 兼容性问题。
4. 调试难度增加。
5. 与 runtime 支持强相关。

所以量化不是“越低越好”，而是要在质量、延迟、吞吐和成本之间权衡。

27.16 Runtime 和分布式推理

大模型太大时，单张 GPU 放不下，或者单机吞吐不够，就需要分布式推理。

常见方式包括：

1. Tensor Parallelism。
2. Pipeline Parallelism。
3. Expert Parallelism。

4. Data Parallel serving。
5. Prefill/decode 分离。

分布式推理会引入新的问题：

1. 通信开销。
2. 拓扑感知。
3. 多卡故障影响。
4. 实例启动变慢。
5. 调度复杂度上升。
6. 成本归因更复杂。

推理平台要知道 runtime 是否支持这些能力，以及需要什么 GPU 拓扑。

27.17 模型服务运行时审计指标与最小 demo

面试里谈 runtime 选型，最好不要直接回答“用 vLLM”或“用 Triton”。更稳的回答是：先定义 workload 和 SLO，再把候选 runtime 放进同一套审计表。

可以把一个 runtime 选型样本写成：

$r_i = (m_i, h_i, f_i, t_i, p_i, b_i, k_i, s_i, q_i, d_i, o_i, e_i, c_i, a_i, u_i, z_i)$

其中 m_i 表示模型与任务类型， h_i 表示硬件和拓扑， f_i 表示权重格式和 tokenizer， t_i 表示 runtime 与平台边界， p_i 表示 prefill / decode 和调度， b_i 表示 batching， k_i 表示 KV cache 管理， s_i 表示 streaming， q_i 表示量化， d_i 表示分布式推理， o_i 表示 observability， e_i 表示 benchmark 口径， c_i 表示成本容量， a_i 表示发布回滚， u_i 表示自研门槛， z_i 表示最终门禁。

对第 j 个审计维度，统一覆盖率可以写成：

$$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(r_i) = 1]$$

runtime 内部延迟可以拆成：

$$T_{\mathrm{runtime}} = T_{\mathrm{tokenize}} + T_{\mathrm{queue}} + T_{\mathrm{prefill}} + T_{\mathrm{decode}} + T_{\mathrm{sample}}$$

如果只看 tokens/s，就会漏掉用户最敏感的首 token 等待和流式卡顿。因此 TTFT 与 TPOT 仍然要单独看：

$$T_{\mathrm{ttft}} = T_{\mathrm{tokenize}} + T_{\mathrm{queue}} + T_{\mathrm{prefill}} + T_{\mathrm{first}}$$

$$T_{\mathrm{tpot}} = \frac{T_{\mathrm{decode}} + T_{\mathrm{sample}} + T_{\mathrm{stream}}}{N_{\mathrm{out}}}$$

KV cache 的单请求显存近似可以写成：

$$M_{\mathrm{kv, req}} = 2LSH_{\mathrm{kv}} D_{\mathrm{elem}}$$

给定可用于 KV cache 的预算 B_{kv} ，粗略并发上限是：

$$N_{\mathrm{kv}} = \left\lfloor \frac{B_{\mathrm{kv}}}{M_{\mathrm{kv, req}}} \right\rfloor$$

把功能、SLO、成本和风险放在一起，runtime 选型门禁可以写成：

$$G_{\mathrm{runtime}} = \mathbf{1}[\min_j C_j \geq \tau_j \wedge T_{\mathrm{ttft}, p95} \leq B_{\mathrm{ttft}} \wedge T_{\mathrm{tpot}} \leq B_{\mathrm{tpot}}]$$

下面的 0 依赖 demo 不是为了复现真实 runtime benchmark，而是演示如何把 runtime 选型变成可审计指标。你可以把 Triton、vLLM、TGI、SGLang、TensorRT-LLM 或自研 runtime 都抽象成同样的 profile，再用统一门禁比较。

Runtime Selection Audit: 0-dependency teaching demo.

from copy import deepcopy

```
GATES = [
    "model_hardware_fit",
    "runtime_boundary_clarity",
    "model_format_tokenizer_fit",
```

```

    "prefill_decode_scheduler_fit",
    "continuous_batching_fit",
    "kv_cache_memory_manager_fit",
    "streaming_api_fit",
    "quantization_accuracy_fit",
    "distributed_inference_fit",
    "observability_metrics_fit",
    "benchmarking_method_fit",
    "deployment_rollback_fit",
    "ecosystem_maintenance_fit",
    "cost_capacity_fit",
    "self_development_threshold",
    "runtime_selection_gate",
]

```

```

def has_keys(obj, keys):
    return isinstance(obj, dict) and all(obj.get(k) for k in keys)

```

```

def percentile(values, p):
    values = sorted(values)
    idx = max(0, min(len(values) - 1, int(len(values) * p + 0.999999) - 1))
    return values[idx]

```

```

def kv_cache_mib(layers, batch, seq, kv_heads, head_dim, dtype_bytes):
    bytes_total = 2 * layers * batch * seq * kv_heads * head_dim * dtype_bytes
    return bytes_total / (1024 * 1024)

```

```

def gate_results(profile):
    metrics = set(profile.get("metrics", []))
    benchmark = profile.get("benchmark", {})
    ecosystem = profile.get("ecosystem", {})
    self_dev = profile.get("self_development", {})
    kv = profile.get("kv_cache", {})

```

```

    results = {
        "model_hardware_fit": has_keys(profile, ["model_class", "hardware", "runtime_name"]),
        "runtime_boundary_clarity": has_keys(profile.get("boundary"), ["platform_controls", "runtime_controls"]),
        "model_format_tokenizer_fit": has_keys(profile.get("model_artifact"), ["weight_format", "tokenizer_version", "tokenizer_type"]),
        "prefill_decode_scheduler_fit": has_keys(profile.get("scheduler"), ["prefill_decode_split", "queue_policy", "scheduler_type"]),
        "continuous_batching_fit": has_keys(profile.get("continuous_batching"), ["dynamic_admit", "dynamic_evict", "dynamic_preempt"]),
        "kv_cache_memory_manager_fit": has_keys(kv, ["block_size", "admission_control", "prefix_cache", "eviction_policy", "kv_cache_type"]),
        "streaming_api_fit": has_keys(profile.get("streaming"), ["transport", "backpressure", "cancellation", "finish_policy", "streaming_type"]),
        "quantization_accuracy_fit": has_keys(profile.get("quantization"), ["dtype", "quality_eval", "rollback_threshold", "quantization_type"]),
        "distributed_inference_fit": has_keys(profile.get("distributed"), ["parallel_mode", "topology", "rank_health", "distributed_type"]),
        "observability_metrics_fit": {"ttft", "tpot", "queue", "prefill", "decode", "kv", "oom", "error", "version"},
        "benchmarking_method_fit": has_keys(benchmark, ["same_model", "same_hardware", "same_precision", "input_distribution", "benchmarking_type"]),
        "deployment_rollback_fit": has_keys(profile.get("deployment"), ["image_digest", "runtime_version", "canary", "deployment_type"]),
        "ecosystem_maintenance_fit": has_keys(ecosystem, ["status", "release_tracking", "compatibility_matrix", "ownership", "ecosystem_type"]),
        "cost_capacity_fit": has_keys(profile.get("cost"), ["gpu_hour_usd", "tokens_in", "tokens_out", "kv_budget_mib", "cost_type"]),
        "self_development_threshold": (not self_dev.get("self_development")) or has_keys(self_dev, ["scale_benefit", "system_development"]),
        "runtime_selection_gate": bool(profile.get("final_gate")),
    }

```

```

    }
    return results

def audit_runtime_selection(profiles):
    failed_cases = []
    failed_gates = set()
    for profile in profiles:
        results = gate_results(profile)
        bad = [name for name, ok in results.items() if not ok]
        if bad:
            failed_cases.append(profile["case"])
            failed_gates.update(bad)

    n = len(profiles)
    metrics = {}
    for gate in GATES:
        metrics[gate] = round(sum(gate_results(p)[gate] for p in profiles) / n, 3)

    return {
        "metrics": metrics,
        "failed_cases": failed_cases,
        "failed_gates": [g for g in GATES if g in failed_gates],
        "hard_blocker_count": len(failed_gates),
        "runtime_selection_gate_pass": not failed_cases,
    }

complete = {
    "case": "complete_runtime_profile_ok",
    "runtime_name": "vllm_or_sglang_style_llm_runtime",
    "model_class": "decoder_llm",
    "hardware": "8x_gpu_nvlink",
    "boundary": {
        "platform_controls": ["auth", "routing", "quota", "release"],
        "runtime_controls": ["prefill", "decode", "kv_cache", "sampling"],
    },
    "model_artifact": {
        "weight_format": "safetensors",
        "tokenizer_version": "tok-v3",
        "chat_template": "chatml-v2",
        "stop_tokens": ["eos"],
    },
    "scheduler": {
        "prefill_decode_split": True,
        "queue_policy": "priority_with_token_budget",
        "phase_metrics": ["queue", "prefill", "decode"],
    },
    "continuous_batching": {
        "dynamic_admit": True,
        "dynamic_evict": True,
        "max_batch_tokens": 8192,
        "long_request_isolation": True,
    },
    "kv_cache": {

```



```

    "block_size": 16,
    "admission_control": True,
    "prefix_cache": True,
    "eviction_policy": "lru_per_tenant",
    "kv_pressure_metric": True,
},
"streaming": {
    "transport": "sse",
    "backpressure": True,
    "cancellation": True,
    "finish_reason": True,
},
"quantization": {
    "dtype": "bf16_or_fp8",
    "quality_eval": "slice_regression",
    "rollback_threshold": "quality_drop_lt_1pct",
},
"distributed": {
    "parallel_mode": "tensor_parallel",
    "topology": "nvlink_aware",
    "rank_health": "per_rank_heartbeat",
},
"metrics": ["ttft", "tpot", "queue", "prefill", "decode", "kv", "oom", "error", "version"],
"benchmark": {
    "same_model": True,
    "same_hardware": True,
    "same_precision": True,
    "input_distribution": "prod_prompt_histogram",
    "output_distribution": "prod_output_histogram",
    "concurrency": "prod_qps_sweep",
    "slo": "ttft_tpot_p99",
},
"deployment": {
    "image_digest": "sha256:runtime",
    "runtime_version": "pinned",
    "canary": True,
    "rollback": True,
    "health_probe": True,
},
"ecosystem": {
    "status": "active_or_risk_accepted",
    "release_tracking": "monthly",
    "compatibility_matrix": "gpu_cuda_driver_model",
    "owner": "serving_team",
},
"cost": {
    "gpu_hour_usd": 0.90,
    "tokens_in": 18000,
    "tokens_out": 24000,
    "kv_budget_mib": 32768,
    "target_cost_per_1k": 0.03,
},
"self_development": {
    "self_develop": False,
    "fallback_runtime": "open_source_runtime",

```

```

    },
    "final_gate": True,
}

def make_bad(case, mutator):
    item = deepcopy(complete)
    item["case"] = case
    mutator(item)
    return item

profiles = [complete]
profiles.append(make_bad("model_hardware_missing_bad", lambda p: p.pop("hardware")))
profiles.append(make_bad("runtime_boundary_blurred_bad", lambda p: p["boundary"].pop("runtime_controls")))
profiles.append(make_bad("model_format_tokenizer_gap_bad", lambda p: p["model_artifact"].pop("chat_template")))
profiles.append(make_bad("prefill_decode_scheduler_gap_bad", lambda p: p["scheduler"].pop("phase_metrics")))
profiles.append(make_bad("continuous_batching_gap_bad", lambda p: p["continuous_batching"].pop("dynamic_evict")))
profiles.append(make_bad("kv_cache_manager_gap_bad", lambda p: p["kv_cache"].pop("admission_control")))
profiles.append(make_bad("streaming_api_gap_bad", lambda p: p["streaming"].pop("backpressure")))
profiles.append(make_bad("quantization_eval_gap_bad", lambda p: p["quantization"].pop("quality_eval")))
profiles.append(make_bad("distributed_inference_gap_bad", lambda p: p["distributed"].pop("topology")))
profiles.append(make_bad("observability_metrics_gap_bad", lambda p: p["metrics"].remove("decode")))
profiles.append(make_bad("benchmark_method_gap_bad", lambda p: p["benchmark"].pop("output_distribution")))
profiles.append(make_bad("deployment_rollback_gap_bad", lambda p: p["deployment"].pop("rollback")))
profiles.append(make_bad("ecosystem_maintenance_unknown_bad", lambda p: p["ecosystem"].pop("status")))
profiles.append(make_bad("cost_capacity_gap_bad", lambda p: p["cost"].pop("kv_budget_mib")))
profiles.append(make_bad("self_development_threshold_gap_bad", lambda p: p.update({"self_development": {"self_development": 0}})))
profiles.append(make_bad("runtime_selection_gate_missing_bad", lambda p: p.update({"final_gate": False})))

latency_samples = [
    {"ttft_ms": 180, "tpot_ms": 18, "e2e_ms": 1280},
    {"ttft_ms": 220, "tpot_ms": 22, "e2e_ms": 1500},
    {"ttft_ms": 270, "tpot_ms": 24, "e2e_ms": 1720},
    {"ttft_ms": 310, "tpot_ms": 26, "e2e_ms": 1980},
    {"ttft_ms": 320, "tpot_ms": 28, "e2e_ms": 2300},
]

feature_score = sum(gate_results(complete)[g] for g in GATES[:-1]) / (len(GATES) - 1)
kv_mib = kv_cache_mib(layers=32, batch=16, seq=4096, kv_heads=8, head_dim=128, dtype_bytes=2)
per_request_kv_mib = kv_cache_mib(layers=32, batch=1, seq=4096, kv_heads=8, head_dim=128, dtype_bytes=2)
cost_per_1k = 1000 * (0.90 + 0.03 + 0.02) / (complete["cost"]["tokens_in"] + complete["cost"]["tokens_out"])
audit = audit_runtime_selection(profiles)

runtime_selection_examples = {
    "ttft_p95_ms": percentile([x["ttft_ms"] for x in latency_samples], 0.95),
    "tpot_p95_ms": percentile([x["tpot_ms"] for x in latency_samples], 0.95),
    "e2e_p99_ms": percentile([x["e2e_ms"] for x in latency_samples], 0.99),
    "kv_cache_mib": round(kv_mib, 1),
    "kv_concurrency_limit": int(complete["cost"]["kv_budget_mib"] // per_request_kv_mib),
    "feature_score": round(feature_score, 3),
    "cost_per_1k_tokens": round(cost_per_1k, 4),
}

smoke = {
    "complete_case_passes": not any(v is False for v in gate_results(complete).values()),
    "caught_boundary_gap": "runtime_boundary_blurred_bad" in audit["failed_cases"],
}

```

```

    "caught_kv_gap": "kv_cache_manager_gap_bad" in audit["failed_cases"],
    "caught_observability_gap": "observability_metrics_gap_bad" in audit["failed_cases"],
    "caught_self_dev_gap": "self_development_threshold_gap_bad" in audit["failed_cases"],
    "caught_gate_gap": "runtime_selection_gate_missing_bad" in audit["failed_cases"],
}

print(f"runtime_selection_examples={runtime_selection_examples}")
print(f"smoke={smoke}")
print(f"metrics={audit['metrics']}")
print(f"hard_blocker_count={audit['hard_blocker_count']}")
print(f"failed_cases={audit['failed_cases']}")
print(f"failed_gates={audit['failed_gates']}")
print(f"runtime_selection_gate_pass={audit['runtime_selection_gate_pass']}")

```

这段 demo 想强调三点：

1. Triton、vLLM、TGI、SGLang、TensorRT-LLM 和自研 runtime 不能只按名称比较，要统一成 profile。
2. runtime benchmark 必须固定模型、硬件、精度、输入输出长度、并发和 SLO，并同时看 TTFT、TPOT、E2E、KV、功能覆盖和成本。
3. 自研 runtime 不是“更高级”的默认答案，只有规模收益、系统能力、CUDA / kernel 能力和 fallback 方案都成立时才进入候选。

27.18 面试常见追问

问题一：vLLM 和 Triton 有什么区别？

可以回答：Triton 是通用推理服务框架，支持多模型和多后端；vLLM 更专注于 LLM 生成推理，强调 continuous batching 和 KV cache 管理。生成式大模型在线服务常优先评估 vLLM，传统模型或 TensorRT 生态可能更多使用 Triton。

问题二：为什么 runtime 需要 continuous batching？

可以回答：生成式推理每个请求输出长度不同，静态 batch 会出现等待和浪费。continuous batching 允许请求动态加入和退出，提高 GPU 利用率，同时控制延迟。

问题三：为什么不能只用 OpenAI-compatible API 判断 runtime 能力？

可以回答：API 兼容只说明接口像，不说明性能、KV cache、batching、长上下文、量化、分布式、指标和稳定性满足生产要求。

问题四：什么时候考虑自研 runtime？

可以回答：当流量规模足够大、单位成本优化收益明显，或者模型结构、硬件、调度和业务调用模式高度特殊，开源 runtime 无法满足要求时，才考虑自研。

27.19 小练习

1. Runtime 和推理平台层的边界是什么？
2. Triton 更适合哪些场景？
3. vLLM 的核心优势是什么？
4. TGI 为什么适合 Hugging Face 生态？
5. SGLang 和普通 LLM serving engine 的差异是什么？
6. 为什么 runtime benchmark 必须固定输入和输出长度分布？
7. 一个生产 runtime 至少要暴露哪些指标？
8. 什么时候才值得自研 runtime？
9. 写一个 runtime profile，至少覆盖模型、硬件、权重格式、tokenizer、prefill / decode、batching、KV cache、streaming、量化、分布式、指标、发布和成本。
10. 为什么 TGI 的维护状态、TensorRT-LLM 的硬件绑定和自研 runtime 的团队能力都应该进入选型门禁？

27.20 本章小结

本章讲了模型服务运行时的核心概念和选型。

你需要记住：

1. Runtime 负责高性能推理执行，平台层负责服务化和治理。
2. Triton 是通用推理服务框架，vLLM 更专注 LLM 生成推理。
3. TGI 适合 Hugging Face 文本生成生态，SGLang 更强调复杂 LLM 程序执行。
4. TensorRT-LLM 适合追求极致性能和深度 NVIDIA 生态优化的场景。
5. 自研 runtime 成本很高，只有在规模、硬件或业务模式足够特殊时才值得做。
6. runtime 选型要综合模型、硬件、batching、KV cache、长上下文、量化、指标、稳定性和成本。
7. 生产选型要用统一审计表比较候选 runtime，避免只看单点 benchmark 或框架流行度。

下一章我们会深入 Prefill、Decode、KV Cache 与推理资源画像，理解为什么大模型推理和普通在线服务完全不同。

第 28 章 Prefill、Decode、KV Cache 与推理资源画像

上一章讲了模型服务运行时。本章继续往下拆，讲大模型推理中最关键的三个概念：Prefill、Decode 和 KV Cache。

很多人说推理慢、吞吐低、GPU 利用率不高，但说不清慢在哪里。真正要理解推理平台，必须把一次生成请求拆成阶段，分别分析它们消耗什么资源。

先记住一句话：

大模型推理不是一次普通函数调用，而是由 prefill、decode 和 KV cache 管理共同决定的在线计算过程。

28.0 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做过资料校准。重点参考的是 vLLM 官方文档中 PagedAttention、KV cache block、prefix caching、TTFT / TPOT / prefill / decode / KV 指标的公开口径；TensorRT-LLM 官方文档中 paged KV cache、in-flight batching、chunked prefill、KV cache reuse、KV cache offload 和 disaggregated serving 的工程边界；SGLang 官方文档中 RadixAttention / cache、continuous batching、PD disaggregation 和 structured serving 的边界；以及 Hugging Face TGI 文档中 streaming、PagedAttention 和 Prometheus 指标的说明。

这些资料共同指向一个稳定事实：LLM 推理不是“一个 batch forward”这么简单。Prefill 处理输入上下文，通常决定首 token 等待；decode 逐 token 生成，通常决定输出流畅度；KV cache 是连接两者的运行时状态，决定显存容量、并发上限、长上下文成本和调度风险。

本章只抽象截至 2026-06 仍稳定的资源画像口径，不把某个框架的具体指标名、默认 block size、benchmark 数值、显卡型号或云实例配置写成通用标准。正文中的公式用于容量估算和面试表达，真实上线仍要用目标模型、tokenizer、runtime、硬件、量化方式和流量分布实测校准。

28.1 一次大模型生成请求发生了什么

用户发起一个请求：

请解释 Transformer 的 attention 机制。

模型返回一段文本：

Attention 机制可以理解为...

从 runtime 内部看，大致发生了这些步骤：

1. 接收请求。
2. tokenizer 把文本转成 token。

- 3. 模型处理输入 token。
- 4. 生成第一枚输出 token。
- 5. 继续逐个生成后续 token。
- 6. 达到停止条件后结束。
- 7. 回收 KV cache。

其中第 3 步主要对应 prefill，第 4 到第 6 步主要对应 decode。

28.2 Prefill 是什么

Prefill 是处理输入 prompt 的阶段。

假设输入有 2000 个 token，模型需要一次性处理这 2000 个 token，并为每一层 attention 生成对应的 KV cache。

Prefill 的输入是完整 prompt。

Prefill 的输出包括：

- 1. 下一 token 的 logits。
- 2. 每一层的 KV cache。

可以把 prefill 理解为：模型先读完整个题目，建立上下文。

28.3 Decode 是什么

Decode 是逐 token 生成输出的阶段。

生成第一个 token 后，模型会把这个 token 加入上下文，再生成下一个 token。这个过程不断重复，直到遇到停止符、达到最大输出长度，或者请求超时。

Decode 的特点是：

- 1. 每一步通常只生成一个 token。
- 2. 每一步都要读取已有 KV cache。
- 3. 输出越长，decode 步数越多。
- 4. streaming 返回主要发生在 decode 阶段。

可以把 decode 理解为：模型一边想，一边一个词一个词往外说。

28.4 Prefill 和 Decode 的核心差异

Prefill 和 decode 的资源特征不同。

阶段	主要处理对象	主要瓶颈	延迟影响	吞吐影响
Prefill	输入 token	计算	影响 TTFT	影响输入吞吐
Decode	输出 token	显存带宽、KV cache 访问	影响 TPOT	影响输出吞吐

这张表非常重要。

如果输入很长，TTFT 可能很高。

如果输出很长，整体延迟和 GPU 占用时间会很高。

如果并发很高，KV cache 会成为主要瓶颈。

28.5 TTFT 和 TPOT

TTFT 是 Time To First Token，即从请求进入到第一个 token 返回的时间。

TTFT 通常包括：

1. 网关耗时。
2. 路由耗时。
3. 排队耗时。
4. tokenizer 耗时。
5. prefill 耗时。
6. 第一个 token 采样耗时。

TPOT 是 Time Per Output Token，即平均每生成一个输出 token 的耗时。

TPOT 通常受 decode 阶段影响更大。

用户体验上：

1. TTFT 决定 “多久开始响应”。
2. TPOT 决定 “输出流不流畅”。
3. 总延迟约等于 TTFT 加上输出 token 数乘以 TPOT。

粗略表达：

$$T_{\mathrm{ttft}} = T_{\mathrm{gateway}} + T_{\mathrm{route}} + T_{\mathrm{queue}} + T_{\mathrm{tokenize}} + T_{\mathrm{prefill}}$$

$$T_{\mathrm{tpot}} = \frac{T_{\mathrm{decode}} + T_{\mathrm{sample}} + T_{\mathrm{stream}}}{N_{\mathrm{out}}}$$

端到端延迟可以粗略写成：

$$T_{\mathrm{e2e}} \approx T_{\mathrm{ttft}} + N_{\mathrm{out}} T_{\mathrm{tpot}}$$

真实系统还要考虑网络、队列、sampling、后处理和 streaming flush。

28.6 KV Cache 是什么

Transformer 每层 attention 会计算 Query、Key、Value。

在自回归生成中，历史 token 的 Key 和 Value 不需要每一步重复计算，可以缓存起来。这就是 KV cache。

KV cache 的作用是：

1. 减少重复计算。
2. 加速 decode。
3. 支持长上下文生成。

没有 KV cache，每生成一个 token 都要重新处理全部历史上下文，成本会高得不可接受。

28.7 KV Cache 为什么占显存

KV cache 会保存每一层、每个 token、每个 head 的 Key 和 Value。

它的规模大致和这些因素相关：

1. batch size。
2. 序列长度。
3. 模型层数。
4. hidden size。
5. attention head 数。
6. 数据类型精度。

粗略理解：

$$M_{\mathrm{kv}} = 2LBSH_{\mathrm{kv}} D_{\mathrm{hB}} \mathrm{elem}$$

其中 L 是层数, B 是活跃 batch 或活跃序列数, S 是当前上下文 token 数, H_{kv} 是 KV head 数, D_h 是每个 head 的维度, B_{elem} 是每个元素的字节数。前面的 2 来自 Key 和 Value 两份缓存。

如果只看单个请求, 可以写成:

$$M_{\{\mathrm{kv}, \mathrm{req}\}} = 2 L S H_{\mathrm{kv}} D_h B_{\mathrm{elem}}$$

如果可用于 KV cache 的显存预算为 B_{kv} , 粗略并发上限是:

$$N_{\{\mathrm{kv}\}} = \left\lfloor \frac{B_{\{\mathrm{kv}\}}}{M_{\{\mathrm{kv}, \mathrm{req}\}}} \right\rfloor$$

这仍然是容量估算, 不是精确 profiler。真实系统还要考虑 block size、碎片、prefix cache、请求取消、padding、量化、MLA / GQA / MQA 结构差异和 runtime 预留显存。

长上下文和高并发都会快速放大 KV cache 显存占用。

28.8 一个直观例子

假设有两个请求:

请求 A: 输入 200 token, 输出 100 token。

请求 B: 输入 20000 token, 输出 100 token。

它们的输出长度一样, 但 B 的 TTFT 会明显更高, 因为 prefill 要处理更长输入。

再看两个请求:

请求 C: 输入 500 token, 输出 50 token。

请求 D: 输入 500 token, 输出 2000 token。

C 和 D 的 prefill 成本接近, 但 D 的 decode 时间远长于 C, 还会更长时间占用 KV cache 和 GPU 资源。

所以不能只用 QPS 衡量推理负载。

必须看 token 维度。

28.9 推理资源画像

推理资源画像就是描述不同请求消耗哪些资源。

至少要看:

1. 输入 token 数。
2. 输出 token 数。
3. 上下文长度。
4. 并发数。
5. batch size。
6. prefill latency。
7. decode latency。
8. KV cache 占用。
9. GPU 利用率。
10. 显存带宽。
11. 网络和 streaming 开销。

一个推理平台如果不知道请求画像, 就无法做正确调度。

28.10 为什么 QPS 不够用

两个系统都处理 100 QPS, 但负载可能完全不同。

系统 A: 每个请求输入 100 token, 输出 50 token。

系统 B: 每个请求输入 8000 token, 输出 1000 token。

它们的 QPS 一样，但系统 B 的计算量、显存占用和响应时间远高于系统 A。

所以大模型推理更应该关注：

1. input tokens/s。
2. output tokens/s。
3. active sequences。
4. total tokens in KV cache。
5. cost per token。
6. SLO 下可承载吞吐。

QPS 只是表层指标。

28.11 Prefill 的优化方向

Prefill 优化常见方向：

1. 提高矩阵计算效率。
2. 使用更高效 kernel。
3. 优化 attention 实现。
4. prompt cache。
5. prefix cache。
6. 输入截断或压缩。
7. 分离 prefill 和 decode 资源池。
8. 对长 prompt 请求单独调度。

长上下文场景中，prefill 往往是 TTFT 的主要来源。

如果业务强依赖长文档、RAG、代码仓库上下文或多轮对话，prefill 优化会非常关键。

28.12 Decode 的优化方向

Decode 优化常见方向：

1. continuous batching。
2. PagedAttention。
3. KV cache 复用。
4. KV cache 量化。
5. speculative decoding。
6. 更高效 sampling。
7. 限制最大输出长度。
8. 分离短输出和长输出请求。
9. 优化 streaming flush 策略。

Decode 阶段的特点是每步计算粒度小、步骤多、频繁访问 KV cache。

这也是为什么 decode 不一定能把 GPU 算力打满，显存带宽和调度开销可能成为瓶颈。

28.13 KV Cache 的管理策略

KV cache 管理常见问题：

1. 如何分配。
2. 如何释放。
3. 如何避免碎片。
4. 如何处理长上下文。
5. 如何处理请求取消。
6. 如何处理超时。
7. 如何做前缀复用。
8. 如何在显存不足时驱逐。

常见策略包括：

1. 分块管理。
2. 分页管理。
3. 引用计数。
4. prefix cache。
5. LRU 驱逐。
6. 按租户或优先级隔离。
7. 显存水位控制。

KV cache 管理不好，系统会出现延迟抖动和 OOM。

28.14 PagedAttention 的直觉

PagedAttention 的直觉类似操作系统分页。

传统方式可能为每个请求预留连续 KV cache 空间，容易浪费和碎片化。

PagedAttention 把 KV cache 拆成固定大小的块，通过映射表管理逻辑序列和物理块。

好处是：

1. 减少显存浪费。
2. 支持更高并发。
3. 更容易复用前缀。
4. 更好处理不同长度请求。

你不需要一开始就掌握所有实现细节，但要理解它解决的是 KV cache 显存管理问题。

28.15 长上下文的特殊问题

长上下文会同时放大 prefill 和 KV cache 压力。

它会带来：

1. TTFT 增加。
2. KV cache 显存占用增加。
3. batch 中长度差异更大。
4. 短请求被长请求影响。
5. 成本显著上升。
6. p99 延迟变差。

长上下文优化手段包括：

1. prompt 压缩。
2. RAG 召回控制。
3. sliding window attention。
4. prefix cache。
5. 长短请求分队列。
6. 长上下文专用实例。
7. 限制上下文配额。

长上下文不是免费能力，平台必须显式治理。

28.16 多租户下的 KV Cache 问题

多租户环境中，KV cache 还涉及公平性和隔离。

问题包括：

1. 一个租户的长请求占满显存。
2. 高优先级业务被低优先级业务阻塞。

3. KV cache 驱逐影响用户体验。
4. cache 复用可能带来数据隔离风险。
5. 成本归因不清。

平台应该支持：

1. 租户级上下文长度限制。
2. 租户级并发限制。
3. 租户级 token 配额。
4. 优先级队列。
5. cache 隔离策略。
6. 详细用量统计。

多租户推理平台不能只追求整体吞吐，还要保证公平性和安全隔离。

28.17 Prefill/Decode 分离

有些系统会把 prefill 和 decode 分离到不同资源池。

原因是两者资源特征不同：

1. Prefill 更偏计算密集。
2. Decode 更偏显存带宽和 KV cache。
3. Prefill 请求长度差异大。
4. Decode 持续时间受输出长度影响。

分离后的可能架构：

Request -> Prefill Pool -> KV Transfer -> Decode Pool -> Streaming Response

好处：

1. 更细粒度调度资源。
2. 长 prompt 不一定阻塞 decode。
3. 可按阶段独立扩缩容。
4. 更适合超大规模推理集群。

代价：

1. 系统复杂度增加。
2. KV cache 传输成本增加。
3. 调度和一致性更难。
4. 故障排查更复杂。

所以它更适合大规模和高优化场景，不一定适合所有团队。

28.18 如何定位推理慢

当用户说“模型慢”时，不要直接说加 GPU。

应该按阶段排查：

1. 请求是否在网关排队？
2. router 是否选择了拥塞实例？
3. runtime 队列长度是否过高？
4. 输入 token 是否变长？
5. prefill latency 是否上升？
6. output token 是否变长？
7. TPOT 是否上升？
8. KV cache 是否接近水位？
9. GPU 显存是否不足？
10. batch 策略是否变化？

11. streaming 是否被网络或客户端拖慢?

只有定位到阶段, 才能做正确优化。

28.19 Prefill/Decode/KV 资源画像审计指标与最小 demo

如果要把这一章落到平台工程, 最核心的动作是建立“推理资源画像”: 每个请求不只记录 QPS, 还要记录输入 token、输出 token、TTFT、TPOT、prefill、decode、KV cache、prefix cache、长上下文、租户和成本。

可以把一个推理资源画像样本写成:

$p_i = (x_i, y_i, c_i, q_i, f_i, d_i, k_i, b_i, r_i, s_i, t_i, o_i, z_i)$

其中 x_i 是输入 token 数, y_i 是输出 token 数, c_i 是上下文长度, q_i 是 queue 和 scheduler 状态, f_i 是 prefill 阶段, d_i 是 decode 阶段, k_i 是 KV cache 状态, b_i 是 batching 状态, r_i 是 prefix / prompt cache 复用, s_i 是 streaming 状态, t_i 是租户和优先级, o_i 是观测指标, z_i 是最终门禁。

统一覆盖率可以写成:

$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(p_i) = 1]$

Prefill 阶段的 attention 交互量可以粗略理解为:

$A_{\{\mathrm{prefill}\}} \propto \mathrm{LBH_qS}_{\{\mathrm{in}\}}^2$

Decode 阶段对历史上下文的访问可以粗略理解为:

$A_{\{\mathrm{decode}\}} \propto \mathrm{LBH_qS}_{\{\mathrm{ctx}\}} N_{\{\mathrm{out}\}}$

其中 H_q 是 query head 数。这个表达不是 kernel 级 FLOPs 公式, 而是帮助理解: 长输入会显著推高 prefill, 长输出会让 decode 访问 KV cache 的次数变多。

KV pressure 可以写成:

$R_{\{\mathrm{kv}\}} = \frac{M_{\{\mathrm{kv}\}}}{B_{\{\mathrm{kv}\}}}$

prefix cache 的收益可以用节省的 prefill token 或 prefill 时间表示:

$R_{\{\mathrm{prefix}\}} = \frac{N_{\{\mathrm{hit_tokens}\}}}{N_{\{\mathrm{input_tokens}\}}}$

最终资源画像门禁可以写成:

$G_{\{\mathrm{resource}\}} = \mathbf{1}[\min_j C_j \geq \tau_j \ \& \ T_{\{\mathrm{ttft}, p95\}} \leq B_{\{\mathrm{ttft}\}} \ \& \ T_{\{\mathrm{tpot}\}} \leq \dots]$

下面是一个 0 依赖 demo。它不会模拟真实 GPU kernel, 而是把 Prefill、Decode、KV cache 和资源画像门禁变成可运行的审计表。

Prefill / Decode / KV Resource Profile Audit: 0-dependency teaching demo.

from copy import deepcopy

```
GATES = [
    "request_token_profile",
    "prefill_phase_accounting",
    "decode_phase_accounting",
    "ttft_tpot_contract",
    "kv_cache_formula_fit",
    "kv_capacity_admission",
    "paged_block_management",
    "prefix_cache_reuse",
    "long_context_policy",
    "tenant_kv_isolation",
    "continuous_batching_policy",
    "pd_disaggregation_fit",
    "streaming_backpressure_fit",
```

```

    "observability_phase_metrics",
    "cost_capacity_model",
    "resource_profile_gate",
]

def has_keys(obj, keys):
    return isinstance(obj, dict) and all(obj.get(k) for k in keys)

def percentile(values, p):
    values = sorted(values)
    idx = max(0, min(len(values) - 1, int(len(values) * p + 0.999999) - 1))
    return values[idx]

def kv_cache_mib(layers, batch, seq, kv_heads, head_dim, dtype_bytes):
    total_bytes = 2 * layers * batch * seq * kv_heads * head_dim * dtype_bytes
    return total_bytes / (1024 * 1024)

def per_token_kib(layers, kv_heads, head_dim, dtype_bytes):
    return 2 * layers * kv_heads * head_dim * dtype_bytes / 1024

def gate_results(profile):
    metrics = set(profile.get("metrics", []))
    kv = profile.get("kv_cache", {})
    results = {
        "request_token_profile": has_keys(profile.get("request_tokens"), ["input_distribution", "output_distribution", "prefill_phase_accounting": has_keys(profile.get("prefill"), ["measured", "input_tokens", "chunking_policy", "decode_phase_accounting": has_keys(profile.get("decode"), ["measured", "output_tokens", "step_metrics", "decoding_metrics"]), "ttft_tpot_contract": has_keys(profile.get("slo"), ["ttft_p95", "tpot_p95", "e2e_p99"]), "kv_cache_formula_fit": has_keys(kv, ["layers", "kv_heads", "head_dim", "dtype_bytes", "active_sequences"]), "kv_capacity_admission": has_keys(kv, ["kv_budget_mib", "admission_control"]), "paged_block_management": has_keys(profile.get("paged"), ["block_size", "block_table", "free_list", "fragmentation"]), "prefix_cache_reuse": has_keys(profile.get("prefix_cache"), ["cache_key", "hit_tokens_metric", "tenant_isolation"]), "long_context_policy": has_keys(profile.get("long_context"), ["max_context", "long_short_queue", "truncation"]), "tenant_kv_isolation": has_keys(profile.get("tenant"), ["tenant_quota", "priority", "cache_isolation", "cost_model"]), "continuous_batching_policy": has_keys(profile.get("batching"), ["continuous", "max_batch_tokens", "dynamic_batching"]), "pd_disaggregation_fit": has_keys(profile.get("pd"), ["enabled_or_rejected", "kv_transfer_budget", "failure_rate"]), "streaming_backpressure_fit": has_keys(profile.get("streaming"), ["backpressure", "cancel", "flush_metric"]), "observability_phase_metrics": {"ttft", "tpot", "prefill", "decode", "queue", "kv_mib", "kv_pressure", "prefill_rate", "decode_rate", "queue_size", "kv_mib", "kv_pressure", "prefill_rate", "decode_rate", "queue_size"}, "cost_capacity_model": has_keys(profile.get("cost"), ["gpu_hour_usd", "tokens_in", "tokens_out", "kv_mib", "cost_model"]), "resource_profile_gate": bool(profile.get("final_gate")),
    }
    return results

def audit_resource_profile(profiles):
    failed_cases = []
    failed_gates = set()
    for profile in profiles:
        bad = [name for name, ok in gate_results(profile).items() if not ok]
        if bad:

```

```

        failed_cases.append(profile["case"])
        failed_gates.update(bad)

n = len(profiles)
metrics = {}
for gate in GATES:
    metrics[gate] = round(sum(gate_results(p)[gate] for p in profiles) / n, 3)

return {
    "metrics": metrics,
    "failed_cases": failed_cases,
    "failed_gates": [g for g in GATES if g in failed_gates],
    "hard_blocker_count": len(failed_gates),
    "resource_profile_gate_pass": not failed_cases,
}

complete = {
    "case": "complete_resource_profile_ok",
    "request_tokens": {
        "input_distribution": "prod_prompt_histogram",
        "output_distribution": "prod_output_histogram",
        "context_length": 4096,
        "token_budget": "tenant_model_budget",
    },
    "prefill": {
        "measured": True,
        "input_tokens": 8192,
        "chunking_policy": "chunk_long_prefill",
        "prefill_queue": "separate_or_weighted",
    },
    "decode": {
        "measured": True,
        "output_tokens": 512,
        "step_metrics": "per_token_latency",
        "decode_queue": "continuous_batching",
    },
    "slo": {"ttft_p95": 1800, "tpot_p95": 35, "e2e_p99": 6000},
    "kv_cache": {
        "layers": 32,
        "kv_heads": 8,
        "head_dim": 128,
        "dtype_bytes": 2,
        "active_sequences": 12,
        "kv_budget_mib": 24576,
        "admission_control": True,
    },
    "paged": {
        "block_size": 16,
        "block_table": True,
        "free_list": True,
        "fragmentation_metric": True,
    },
    "prefix_cache": {
        "cache_key": "tenant_model_template_prefix",

```

```

        "hit_tokens_metric": True,
        "tenant_isolation": True,
    },
    "long_context": {
        "max_context": 32768,
        "long_short_queue": True,
        "truncation_or_compression": True,
        "rag_budget": True,
    },
    "tenant": {
        "tenant_quota": True,
        "priority": True,
        "cache_isolation": True,
        "cost_attribution": True,
    },
    "batching": {
        "continuous": True,
        "max_batch_tokens": 8192,
        "dynamic_join": True,
        "dynamic_exit": True,
    },
    "pd": {
        "enabled_or_rejected": "rejected_for_small_scale",
        "kv_transfer_budget": True,
        "failure_boundary": True,
    },
    "streaming": {"backpressure": True, "cancel": True, "flush_metric": True},
    "metrics": ["ttft", "tpot", "prefill", "decode", "queue", "kv_mib", "kv_pressure", "prefix_hit", "oom", "stream_t"],
    "cost": {
        "gpu_hour_usd": 1.20,
        "tokens_in": 55000,
        "tokens_out": 12000,
        "kv_mib": 6144,
        "cache_saved_tokens": 8000,
    },
    "final_gate": True,
}

```

```

def make_bad(case, mutator):
    item = deepcopy(complete)
    item["case"] = case
    mutator(item)
    return item

```

```

profiles = [complete]
profiles.append(make_bad("token_profile_missing_bad", lambda p: p["request_tokens"].pop("output_distribution")))
profiles.append(make_bad("prefill_unmeasured_bad", lambda p: p["prefill"].pop("chunking_policy")))
profiles.append(make_bad("decode_unmeasured_bad", lambda p: p["decode"].pop("step_metrics")))
profiles.append(make_bad("slo_missing_tpot_bad", lambda p: p["slo"].pop("tpot_p95")))
profiles.append(make_bad("kv_formula_gap_bad", lambda p: p["kv_cache"].pop("kv_heads")))
profiles.append(make_bad("kv_admission_missing_bad", lambda p: p["kv_cache"].pop("admission_control")))
profiles.append(make_bad("paged_block_missing_bad", lambda p: p["paged"].pop("block_table")))
profiles.append(make_bad("prefix_cache_unsafe_bad", lambda p: p["prefix_cache"].pop("tenant_isolation")))

```

```

profiles.append(make_bad("long_context_unbounded_bad", Lambda p: p["long_context"].pop("max_context")))
profiles.append(make_bad("tenant_quota_missing_bad", Lambda p: p["tenant"].pop("tenant_quota")))
profiles.append(make_bad("continuous_batching_gap_bad", Lambda p: p["batching"].pop("dynamic_exit")))
profiles.append(make_bad("pd_disaggregation_unbounded_bad", Lambda p: p["pd"].pop("kv_transfer_budget")))
profiles.append(make_bad("streaming_backpressure_gap_bad", Lambda p: p["streaming"].pop("backpressure")))
profiles.append(make_bad("observability_phase_gap_bad", Lambda p: p["metrics"].remove("prefill")))
profiles.append(make_bad("cost_capacity_gap_bad", Lambda p: p["cost"].pop("kv_mib")))
profiles.append(make_bad("resource_profile_gate_missing_bad", Lambda p: p.update({"final_gate": False})))

latency_samples = [
    {"queue_ms": 40, "tokenize_ms": 20, "prefill_ms": 260, "first_ms": 30, "decode_ms": 900, "sample_ms": 90, "stream_ms": 120},
    {"queue_ms": 65, "tokenize_ms": 30, "prefill_ms": 420, "first_ms": 35, "decode_ms": 1250, "sample_ms": 120, "stream_ms": 150},
    {"queue_ms": 85, "tokenize_ms": 42, "prefill_ms": 610, "first_ms": 40, "decode_ms": 1680, "sample_ms": 150, "stream_ms": 180},
    {"queue_ms": 115, "tokenize_ms": 55, "prefill_ms": 950, "first_ms": 50, "decode_ms": 2100, "sample_ms": 200, "stream_ms": 220},
    {"queue_ms": 140, "tokenize_ms": 70, "prefill_ms": 1300, "first_ms": 55, "decode_ms": 2600, "sample_ms": 240, "stream_ms": 270}
]

ttfts = [x["queue_ms"] + x["tokenize_ms"] + x["prefill_ms"] + x["first_ms"] for x in latency_samples]
tpots = [(x["decode_ms"] + x["sample_ms"] + x["stream_ms"]) / x["output_tokens"] for x in latency_samples]
e2e = [ttfts[i] + latency_samples[i]["output_tokens"] * tpots[i] for i in range(len(latency_samples))]
prefill_tps = 1000 * sum(x["input_tokens"] for x in latency_samples) / sum(x["prefill_ms"] for x in latency_samples)
decode_tps = 1000 * sum(x["output_tokens"] for x in latency_samples) / sum(x["decode_ms"] for x in latency_samples)
kv_mib = kv_cache_mib(32, 12, 4096, 8, 128, 2)
per_request_kv_mib = kv_cache_mib(32, 1, 4096, 8, 128, 2)
cache_saved_prefill_ms = 1000 * complete["cost"]["cache_saved_tokens"] / prefill_tps
cost_per_1k = 1000 * (1.20 + 0.05 + 0.03) / (complete["cost"]["tokens_in"] + complete["cost"]["tokens_out"])
audit = audit_resource_profile(profiles)

resource_profile_examples = {
    "ttft_p95_ms": percentile(ttfts, 0.95),
    "tpot_p95_ms": round(percentile(tpots, 0.95), 1),
    "e2e_p99_ms": round(percentile(e2e, 0.99), 1),
    "prefill_tokens_per_second": round(prefill_tps, 1),
    "decode_output_tokens_per_second": round(decode_tps, 1),
    "kv_cache_mib": round(kv_mib, 1),
    "kv_pressure": round(kv_mib / complete["kv_cache"]["kv_budget_mib"], 3),
    "per_token_kv_kib": round(per_token_kib(32, 8, 128, 2), 1),
    "kv_concurrency_limit": int(complete["kv_cache"]["kv_budget_mib"] // per_request_kv_mib),
    "prefix_saved_prefill_ms": round(cache_saved_prefill_ms, 1),
    "cost_per_1k_tokens": round(cost_per_1k, 4),
}

smoke = {
    "complete_case_passes": not any(v is False for v in gate_results(complete).values()),
    "caught_prefill_gap": "prefill_unmeasured_bad" in audit["failed_cases"],
    "caught_decode_gap": "decode_unmeasured_bad" in audit["failed_cases"],
    "caught_kv_gap": "kv_admission_missing_bad" in audit["failed_cases"],
    "caught_long_context_gap": "long_context_unbounded_bad" in audit["failed_cases"],
    "caught_gate_gap": "resource_profile_gate_missing_bad" in audit["failed_cases"],
}

print(f"resource_profile_examples={resource_profile_examples}")
print(f"smoke={smoke}")
print(f"metrics={audit['metrics']}")
print(f"hard_blocker_count={audit['hard_blocker_count']}")
print(f"failed_cases={audit['failed_cases']}")

```

```
print(f"failed_gates={audit['failed_gates']}")
print(f"resource_profile_gate_pass={audit['resource_profile_gate_pass']}")
```

这个 demo 想说明：推理慢不是一个单点指标问题。你必须同时证明输入 / 输出 token 分布、prefill、decode、TTFT、TPOT、KV cache、分页块管理、prefix cache、长上下文、多租户、continuous batching、PD 分离、streaming、observability 和成本容量都可观测、可解释、可门禁。

28.20 面试常见追问

问题一：为什么长 prompt 会影响 TTFT？

可以回答：长 prompt 会增加 prefill 计算量，模型需要处理全部输入 token 并生成 KV cache，所以第一个 token 返回前的时间会变长。

问题二：为什么长输出会影响总延迟？

可以回答：输出是逐 token decode 的，输出越长 decode 步数越多，整体延迟约等于 TTFT 加 output tokens 乘以 TPOT。

问题三：为什么 KV cache 会成为瓶颈？

可以回答：KV cache 与并发数、序列长度、层数和 hidden size 相关，会占用大量显存。高并发和长上下文下，显存容量、显存带宽和 cache 管理都会成为瓶颈。

问题四：为什么不能只用 QPS 评估推理系统？

可以回答：不同请求 token 数差异巨大，同样 QPS 下输入输出 token 数可能相差几十倍。应该结合 input tokens/s、output tokens/s、TTFT、TPOT、KV cache 和成本来评估。

28.21 小练习

1. Prefill 和 decode 的区别是什么？
2. TTFT 和 TPOT 分别受哪些因素影响？
3. 为什么 KV cache 能加速 decode？
4. 为什么 KV cache 会占大量显存？
5. 长上下文会带来哪些平台问题？
6. 为什么高 QPS 不一定代表高负载？
7. Prefill/decode 分离有什么收益和代价？
8. 用户反馈“模型慢”时，你会按什么顺序排查？
9. 写一个推理资源画像表，至少包含 input tokens、output tokens、TTFT、TPOT、prefill、decode、KV cache、prefix cache、tenant、streaming 和 cost。
10. 为什么 prefix cache 命中率不能单独证明成本下降，还要看命中 token 数、节省 prefill 时间、权限隔离和错误复用风险？

28.22 本章小结

本章讲了 Prefill、Decode、KV Cache 和推理资源画像。

你需要记住：

1. Prefill 处理输入 prompt，主要影响 TTFT。
2. Decode 逐 token 生成输出，主要影响 TPOT 和总延迟。
3. KV cache 避免重复计算，但会消耗大量显存。
4. 长上下文会同时放大 prefill 成本和 KV cache 压力。
5. 大模型推理不能只看 QPS，要看 token 吞吐、延迟、cache 和成本。
6. 推理优化必须按阶段定位瓶颈，而不是盲目加机器。
7. 生产推理平台需要用资源画像门禁同时约束 prefill、decode、KV cache、长上下文、多租户、streaming 和成本。

下一章我们会继续讲 Continuous Batching、PagedAttention 和队列调度，理解 runtime 如何在高并发下提升吞吐并控制延迟。

第 29 章 Continuous Batching、PagedAttention 和队列调度

上一章讲了 Prefill、Decode 和 KV Cache。本章继续深入高并发推理中的三个关键机制：Continuous Batching、PagedAttention 和队列调度。

如果说上一章回答的是“推理请求消耗什么资源”，本章回答的是“runtime 怎样把很多请求组织起来，提高吞吐并控制延迟”。

先记住一句话：

大模型推理的核心挑战不是单个请求能不能跑，而是大量长短不一的请求如何在有限 GPU 和显存下公平、高效、低延迟地一起跑。

29.0 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做过资料校准。重点参考的是 vLLM 的 PagedAttention / KV cache block / scheduler 公开资料和论文口径，TensorRT-LLM 对 in-flight batching、paged KV cache、chunked prefill 和调度策略的工程说明，SGLang 对 continuous batching、RadixAttention / cache 和 serving runtime 的公开说明，以及 Hugging Face TGI 对 streaming、PagedAttention 和 Prometheus 指标的公开说明。

这些资料共同指向一个稳定事实：LLM serving 的 batching 不是传统“凑一批请求然后一起跑完”，而是围绕 decode iteration 动态维护 running set、waiting queue、prefill chunk、KV block、streaming 连接和租户配额。PagedAttention 也不是新的模型结构或新的 attention 数学公式，而是把逻辑连续的 KV cache 映射到物理上可分散管理的 blocks，从而降低碎片、支持动态调度和 prefix / block 级复用。

本章只抽象截至 2026-06 仍稳定的调度和容量画像口径，不把某个 runtime 的默认 block size、调度参数、指标字段名、benchmark 数值或云实例配置写成通用标准。正文中的公式用于面试表达、容量估算和系统设计推导；真实上线仍要用目标模型、tokenizer、runtime、硬件、量化方式、请求 token 分布和 SLO 实测校准。

29.1 为什么需要 batching

GPU 擅长并行计算。如果一次只处理一个请求，GPU 很可能吃不满。

Batching 的基本思想是：把多个请求合并成一个 batch，一起执行模型前向计算。

这样可以：

1. 提高 GPU 利用率。
2. 提高 tokens/s。
3. 降低单位 token 成本。
4. 减少调度开销。

但 batching 不是越大越好。batch 越大，请求可能等待越久，TTFT 可能变差，显存占用也会增加。

推理系统要在吞吐和延迟之间做权衡。

29.2 静态 batching 的问题

传统 batching 可以理解为：收集一批请求，凑够 batch 后一起执行。

例如：

等待 8 个请求 -> 组成 batch -> 一起执行 -> 全部结束 -> 下一批

这种方式对分类模型、embedding 模型、CV 模型比较常见。

但对生成式大模型有明显问题。

原因是每个请求输出长度不同：

1. 有的请求生成 20 token 就结束。
2. 有的请求生成 2000 token 才结束。
3. 有的请求中途被用户取消。

4. 有的请求遇到 stop sequence 提前结束。

如果静态 batch 要等所有请求结束，短请求会被长请求拖住。

29.3 Continuous Batching 的直觉

Continuous Batching 也叫动态 batching 或 iteration-level batching。

它的核心思想是：batch 不是一次固定到结束，而是在每个 decode step 动态调整。

可以理解为：

step 1: A B C D 在 batch 中

step 2: A B C D 继续生成

step 3: A 完成, E 加入

step 4: B 完成, F 加入

step 5: C D E F 继续生成

请求可以动态进入，也可以动态退出。

这非常适合大模型生成，因为输出长度天然不一致。

29.4 Continuous Batching 的收益

Continuous Batching 的收益包括：

1. 减少 GPU 空转。
2. 短请求不用一直等长请求。
3. 提高 decode 阶段吞吐。
4. 更好处理不同输出长度。
5. 支持高并发在线服务。

它把“按请求批处理”变成了“按生成步调度”。

这就是 LLM runtime 和传统模型 serving 很不一样的地方。

29.5 Continuous Batching 的代价

Continuous Batching 不是免费的。

它会带来：

1. 调度器更复杂。
2. KV cache 管理更复杂。
3. 每个 step 都要维护活跃请求集合。
4. 请求取消和超时处理更复杂。
5. streaming 返回顺序管理更复杂。
6. 指标统计更复杂。

因此 runtime 需要一个成熟的 scheduler 和 KV cache manager。

29.6 PagedAttention 解决什么问题

上一章已经讲过，KV cache 会占用大量显存。

不同请求长度不同，如果每个请求都预留连续显存空间，会出现两个问题：

1. 显存浪费。
2. 显存碎片。

PagedAttention 的直觉是：把 KV cache 拆成固定大小的 block，用类似页表的结构管理。

逻辑序列不需要对应一段连续物理显存，而是映射到多个 KV block。

如果 block size 是 s_{blk} ，请求 i 当前上下文长度是 L_i ，则它大约需要：

$$N_{\{\mathrm{blk}, i\}} = \left\lceil \frac{L_i}{s_{\{\mathrm{blk}\}}} \right\rceil$$

个 KV blocks。若所有活跃请求集合为 $\mathcal{R}$ ，则总 block 占用为：

$$N_{\{\mathrm{blk}, \text{total}\}} = \sum_{i \in \mathcal{R}} N_{\{\mathrm{blk}, i\}}$$

分页管理仍然会有尾部浪费，粗略浪费率可以写成：

$$R_{\{\mathrm{waste}\}} = \frac{\sum_{i \in \mathcal{R}} \left(N_{\{\mathrm{blk}, i\}} s_{\{\mathrm{blk}\}} - L_i \right)}{\sum_{i \in \mathcal{R}} N_{\{\mathrm{blk}, i\}} s_{\{\mathrm{blk}\}}}$$

这说明 block size 不是越小越好，也不是越大越好。小 block 降低尾部浪费，但 block table、元数据和访存管理更复杂；大 block 管理简单，但短请求和请求尾部会浪费更多 KV 空间。

这样能更好支持：

1. 不同长度请求。
2. 动态进入和退出。
3. prefix 共享。
4. 更高并发。
5. 更低显存浪费。

29.7 PagedAttention 和 Continuous Batching 的关系

Continuous Batching 要求请求可以动态加入和退出。

这意味着 KV cache 也要能动态分配、释放和复用。

PagedAttention 提供了更灵活的 KV cache 管理方式，让 continuous batching 更容易高效运行。

可以这样理解：

1. Continuous Batching 解决“请求如何动态组 batch”。
2. PagedAttention 解决“动态 batch 下 KV cache 如何高效管理”。
3. Scheduler 解决“谁先执行、谁等待、谁被拒绝”。

三者经常一起出现。

29.8 队列调度为什么重要

高并发推理服务不是请求来了就立刻执行。

请求通常会进入队列。

队列调度决定：

1. 哪些请求先执行。
2. 哪些请求进入 prefill。
3. 哪些请求进入 decode。
4. batch 里放多少请求。
5. 长请求是否要限速。
6. 高优先级请求是否插队。
7. 低优先级请求是否等待或拒绝。

调度策略会直接影响 TTFT、TPOT、吞吐、公平性和成本。

29.9 一个简单调度流程

一个 runtime scheduler 可能这样工作：

1. 接收新请求
2. 估算输入 token 和最大输出 token
3. 检查上下文长度限制
4. 检查 KV cache 是否有空间
5. 将请求放入 waiting queue
6. 选择部分请求进入 prefill
7. 将 prefill 完成的请求放入 running set
8. 对 running set 执行 decode step
9. 完成的请求释放 KV cache
10. 新请求补入 batch

这个流程看似简单，但每一步都有工程细节。

29.10 调度目标之间的冲突

推理调度有多个目标：

1. 高吞吐。
2. 低 TTFT。
3. 低 TPOT。
4. 低 p99 延迟。
5. 高 GPU 利用率。
6. 高公平性。
7. 低拒绝率。
8. 低成本。

这些目标经常冲突。

例如：

1. 更大 batch 提高吞吐，但可能增加等待时间。
2. 优先短请求降低平均延迟，但可能饿死长请求。
3. 优先高价值租户提升业务收益，但影响公平性。
4. 尽量填满 GPU 提高利用率，但可能拉高 p99。

所以调度不是一个“最佳参数”，而是一组策略权衡。

29.11 常见队列策略

常见调度策略包括：

1. FIFO。
2. shortest-job-first。
3. priority queue。
4. deadline-aware scheduling。
5. tenant-aware scheduling。
6. prefill/decode 分队列。
7. 长短请求分队列。
8. token budget scheduling。
9. fairness scheduling。

FIFO 最简单，但容易被长请求拖慢。

shortest-job-first 可以降低平均延迟，但可能伤害长请求公平性。

priority queue 适合多租户和业务优先级，但要防止低优先级长期饥饿。

token budget scheduling 更适合 LLM，因为 token 数比请求数更能代表负载。

29.12 Token Budget 调度

Token budget 调度的核心是：每轮执行不只限制请求数，还限制 token 预算。

例如：

```
max_batch_tokens = 8192
max_active_sequences = 64
```

可以把一轮调度写成：

$$\sum_{i \in \mathcal{P}_t} n_i^{\text{prefill}} + \sum_{i \in \mathcal{D}_t} n_i^{\text{decode}} \leq B_{\text{tok}}$$

其中 $\mathcal{P}_t$ 是第 t 轮被选中做 prefill 的请求集合， $\mathcal{D}_t$ 是第 t 轮被选中做 decode 的请求集合， n_i^{prefill} 是本轮给请求 i 处理的 prompt token 数， n_i^{decode} 通常是 1 或少量输出 token， B_{tok} 是本轮 token budget。

同时还要限制活跃序列数：

$$|\mathcal{R}_t| \leq B_{\text{seq}}$$

其中 $\mathcal{R}_t$ 是 running set， B_{seq} 是最大活跃 sequence 数。直觉上，max_batch_tokens 控制本轮算力和排队压力，max_active_sequences 控制 decode 并发和 KV cache 压力。只看 batch size 会低估长 prompt 和长上下文请求的成本。

调度器选择请求时同时考虑：

1. 输入 token 数。
2. 当前上下文长度。
3. 预计输出 token 数。
4. KV cache 剩余空间。
5. 活跃 sequence 数。

这样比单纯按 batch size 更合理。

因为一个长上下文请求可能比几十个短请求还重。

29.13 长短请求混跑的问题

长请求和短请求混跑会带来 head-of-line blocking。

问题表现：

1. 短请求 TTFT 被拉高。
2. 长请求占用 KV cache 很久。
3. batch 内长度差异导致调度效率下降。
4. p99 延迟变差。

常见解决方案：

1. 长短请求分队列。
2. 长上下文专用实例。
3. 对长请求限流。
4. 对短请求保留容量。
5. 按 token 预算调度。
6. 设置最大上下文长度和最大输出长度。

生产系统通常不会让所有请求无差别混在一个队列里。

29.14 Prefill 和 Decode 的调度冲突

Prefill 和 decode 会争用 GPU。

如果 prefill 太多，decode 可能变慢，TPOT 变差。

如果 decode 占满资源，新请求 prefill 进不来，TTFT 变差。

调度器要在两者之间平衡。

常见方法包括：

1. 限制每轮 prefill token 数。
2. 限制新请求进入速度。
3. 给 decode 保留执行窗口。
4. prefill/decode 分离资源池。
5. 对长 prefill 请求单独调度。

如果业务要求流式输出顺滑，decode 不能被 prefill 长时间阻塞。

29.15 公平性调度

多租户推理平台必须考虑公平性。

否则一个租户可能用长上下文、大并发、大输出占满所有资源。

公平性可以从多个维度定义：

1. 请求数公平。
2. token 数公平。
3. GPU 时间公平。
4. 成本公平。
5. SLO 公平。

对于大模型，token 数和 GPU 时间通常比请求数更合理。

平台可以设置：

1. 租户级并发上限。
2. 租户级 token/s 上限。
3. 租户级上下文长度上限。
4. 租户级优先级。
5. 租户级预算。

如果用 token 作为公平性口径，租户 u 在一个时间窗口内的资源占比可以粗略写成：

$$S_u = \frac{N_u^{\mathrm{in}} + \alpha N_u^{\mathrm{out}}}{\sum_v (N_v^{\mathrm{in}} + \alpha N_v^{\mathrm{out}})}$$

其中 N_u^{in} 是租户 u 的输入 token 数， N_u^{out} 是输出 token 数， α 是输出 token 相对输入 token 的成本权重。实际系统还会把 KV cache、GPU seconds、优先级和 SLO 合进来，但这个公式能提醒你：请求数公平通常不等于资源公平。

公平性不是道德问题，而是稳定性问题。

29.16 Admission Control

Admission control 是决定请求是否允许进入系统。

如果系统已经没有足够资源，盲目接收请求只会让所有请求都变慢。

Admission control 可以检查：

1. 队列长度。
2. 等待时间。
3. KV cache 剩余空间。
4. GPU 负载。
5. 租户配额。
6. 请求 token 数。
7. 当前 p99 延迟。
8. 错误率和超时率。

处理方式包括：

1. 接收。
2. 排队。
3. 降级。
4. 重定向到其他实例。
5. 返回限流错误。

对在线系统来说，及时拒绝比无限排队更健康。

29.17 Streaming 对调度的影响

大模型服务常用 streaming 返回。

Streaming 会影响调度和资源管理：

1. 请求连接持续时间更长。
2. decode 过程需要持续输出 token。
3. 客户端断开要及时释放资源。
4. 慢客户端可能影响发送缓冲。
5. 指标要区分生成耗时和传输耗时。

runtime 要能处理请求取消和客户端断开，否则 KV cache 和运行资源可能泄漏。

29.18 指标设计

调度相关指标至少包括：

1. waiting queue length。
2. running requests。
3. active sequences。
4. scheduled tokens per step。
5. prefill tokens/s。
6. decode tokens/s。
7. batch size 分布。
8. batch token 分布。
9. KV cache 使用率。
10. admission reject count。
11. queue wait time。
12. TTFT。
13. TPOT。
14. p95 / p99 latency。
15. tenant-level token usage。

没有这些指标，调度策略只能靠猜。

29.19 常见故障模式

Continuous batching 和队列调度常见故障包括：

1. 长请求拖慢所有短请求。
2. batch 太大导致 TTFT 升高。
3. batch 太小导致 GPU 利用率低。
4. KV cache 接近水位后频繁拒绝请求。
5. 高优先级请求插队导致低优先级饥饿。
6. prefill 抢占 decode, streaming 输出卡顿。
7. 请求取消后资源没有释放。
8. 指标只看 QPS，无法解释 p99 抖动。

这些问题都不是简单加 GPU 就能解决的。

29.20 Continuous Batching / PagedAttention 调度审计指标与最小 demo

做推理调度系统设计时，可以把一个请求样本抽象成：

$s_i = (a_i, u_i, p_i, x_i, y_i, q_i, r_i, b_i, k_i, c_i, m_i, z_i)$

其中 a_i 是到达时间， u_i 是租户， p_i 是优先级， x_i 是输入 token 数， y_i 是最大输出 token 数， q_i 是队列等待策略， r_i 是 running / waiting / finished 状态， b_i 是 KV block 占用， k_i 是是否命中 prefix / KV 复用， c_i 是取消或超时处理， m_i 是指标记录， z_i 是最终门禁结果。

对每个审计维度 j 定义一个检查函数 $g_j(s_i) \in \{0, 1\}$ ，覆盖率可以写成：

$$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(s_i) = 1]$$

最终可以用一个调度门禁收束：

$$G_{\{\mathrm{sched}\}} = \mathbf{1}[\min_j C_j \geq \tau_j \wedge T_{\{\mathrm{ttft}, p95\}} \leq B_{\{\mathrm{ttft}\}} \wedge T_{\{\mathrm{math}\}}$$

下面这个 0 依赖 demo 做两件事：第一，模拟一个小型 continuous batching scheduler，展示请求动态到达、短请求优先、长 prompt 被 chunked prefill、running set 动态退出、KV block 被释放；第二，用审计表检查一个推理调度系统是否覆盖 arrival profile、continuous batching、token budget、prefill/decode 平衡、PagedAttention block、KV admission、长短请求隔离、多租户公平、取消释放、streaming 背压、指标和最终门禁。

Continuous batching / PagedAttention scheduler audit: 0-dependency teaching demo.

```
from copy import deepcopy
from math import ceil
```

```
GATES = [
    "request_arrival_profile",
    "continuous_batching_core",
    "token_budget_policy",
    "prefill_decode_balance",
    "paged_kv_block_management",
    "kv_capacity_admission",
    "long_short_isolation",
    "tenant_priority_fairness",
    "cancellation_cleanup",
    "streaming_backpressure",
    "observability_metrics",
    "scheduler_gate",
]
```

```
def has_keys(obj, keys):
    return isinstance(obj, dict) and all(obj.get(k) for k in keys)
```

```
def percentile(values, p):
    values = sorted(values)
    idx = max(0, min(len(values) - 1, int(len(values) * p + 0.999999) - 1))
    return values[idx]
```

```
def gate_results(profile):
    metrics = set(profile.get("metrics", []))
    results = {
        "request_arrival_profile": has_keys(profile.get("arrival"), ["input_tokens", "output_tokens", "arrival_time",
        "continuous_batching_core": has_keys(profile.get("batching"), ["dynamic_join", "dynamic_exit", "iteration_level"])
```



```

        "token_budget_policy": has_keys(profile.get("token_budget"), ["max_batch_tokens", "max_prefill_chunk", "decode_token_budget"]),
        "prefill_decode_balance": has_keys(profile.get("phase_policy"), ["decode_first", "chunked_prefill", "prefill_queue"]),
        "paged_kv_block_management": has_keys(profile.get("paged_kv"), ["block_size", "block_table", "free_list", "fragmentation_metric"]),
        "kv_capacity_admission": has_keys(profile.get("admission"), ["kv_block_budget", "queue_timeout", "reject_policy", "watermark"]),
        "long_short_isolation": has_keys(profile.get("long_short"), ["long_prompt_threshold", "separate_queue", "reserved_short_capacity"]),
        "tenant_priority_fairness": has_keys(profile.get("tenant"), ["quota", "priority", "fairness_metric", "cost_attribution"]),
        "cancellation_cleanup": has_keys(profile.get("cancel"), ["cancel_signal", "release_blocks", "timeout_release"]),
        "streaming_backpressure": has_keys(profile.get("streaming"), ["flush_metric", "backpressure", "client_disconnect"]),
        "observability_metrics": {"queue_wait", "ttft", "tpot", "scheduled_tokens", "active_sequences", "kv_blocks", "kv_waste", "reject_policy"},
        "scheduler_gate": bool(profile.get("final_gate")),
    }
}
return results

def audit_scheduler_profiles(profiles):
    failed_cases = []
    failed_gates = set()
    for profile in profiles:
        bad = [name for name, ok in gate_results(profile).items() if not ok]
        if bad:
            failed_cases.append(profile["case"])
            failed_gates.update(bad)

    n = len(profiles)
    metrics = {gate: round(sum(gate_results(p)[gate] for p in profiles) / n, 3) for gate in GATES}
    return {
        "metrics": metrics,
        "failed_cases": failed_cases,
        "failed_gates": [g for g in GATES if g in failed_gates],
        "hard_blocker_count": len(failed_gates),
        "scheduler_gate_pass": not failed_cases,
    }

complete = {
    "case": "complete_scheduler_profile_ok",
    "arrival": {"input_tokens": "histogram", "output_tokens": "histogram", "arrival_time": "timestamp", "tenant": "tenant"},
    "batching": {"dynamic_join": True, "dynamic_exit": True, "iteration_level": True, "max_active_sequences": 64},
    "token_budget": {"max_batch_tokens": 4096, "max_prefill_chunk": 1024, "decode_token_budget": 256},
    "phase_policy": {"decode_first": True, "chunked_prefill": True, "prefill_queue": "weighted", "decode_queue": "round_robin"},
    "paged_kv": {"block_size": 16, "block_table": True, "free_list": True, "fragmentation_metric": True},
    "admission": {"kv_block_budget": 4096, "queue_timeout": 30, "reject_policy": "fast_fail_or_degrade", "watermark": 0.9},
    "long_short": {"long_prompt_threshold": 4096, "separate_queue": True, "reserved_short_capacity": True},
    "tenant": {"quota": True, "priority": True, "fairness_metric": "token_share", "cost_attribution": True},
    "cancel": {"cancel_signal": True, "release_blocks": True, "timeout_release": True},
    "streaming": {"flush_metric": True, "backpressure": True, "client_disconnect": True},
    "metrics": ["queue_wait", "ttft", "tpot", "scheduled_tokens", "active_sequences", "kv_blocks", "kv_waste", "reject_policy"],
    "final_gate": True,
}

def make_bad(case, mutator):
    item = deepcopy(complete)
    item["case"] = case
    mutator(item)

```

```
return item
```

```
profiles = [complete]
profiles.append(make_bad("arrival_profile_missing_bad", lambda p: p["arrival"].pop("output_tokens")))
profiles.append(make_bad("static_batching_bad", lambda p: p["batching"].pop("dynamic_exit")))
profiles.append(make_bad("token_budget_missing_bad", lambda p: p["token_budget"].pop("max_batch_tokens")))
profiles.append(make_bad("prefill_decode_unbalanced_bad", lambda p: p["phase_policy"].pop("chunked_prefill")))
profiles.append(make_bad("paged_kv_no_freelist_bad", lambda p: p["paged_kv"].pop("free_list")))
profiles.append(make_bad("kv_admission_missing_bad", lambda p: p["admission"].pop("watermark")))
profiles.append(make_bad("long_short_mixed_bad", lambda p: p["long_short"].pop("separate_queue")))
profiles.append(make_bad("tenant_fairness_missing_bad", lambda p: p["tenant"].pop("fairness_metric")))
profiles.append(make_bad("cancel_leaks_blocks_bad", lambda p: p["cancel"].pop("release_blocks")))
profiles.append(make_bad("streaming_backpressure_missing_bad", lambda p: p["streaming"].pop("backpressure")))
profiles.append(make_bad("scheduler_metrics_missing_bad", lambda p: p["metrics"].remove("kv_waste")))
profiles.append(make_bad("scheduler_gate_missing_bad", lambda p: p.update({"final_gate": False})))
```

```
def blocks_for(tokens, block_size):
    return int(ceil(tokens / block_size)) if tokens > 0 else 0
```

```
def simulate_continuous_batching(requests, max_steps=14, max_batch_tokens=1024, max_active=3, block_size=16, kv_block_budget=1024):
    reqs = {r["id"]: dict(r, prompt_done=0, output_done=0, first_scheduled=None, first_decode=None, finish_step=None)}
    waiting = []
    running = []
    free_blocks = kv_block_budget
    trace = []
    finished_order = []

    def allocate(req, new_tokens):
        nonlocal free_blocks
        need = blocks_for(new_tokens, block_size)
        delta = need - req["blocks"]
        if delta <= 0:
            return True
        if delta > free_blocks:
            return False
        req["blocks"] = need
        free_blocks -= delta
        return True

    for step in range(max_steps):
        for req in reqs.values():
            if req["arrival"] == step:
                waiting.append(req["id"])

        budget = max_batch_tokens
        actions = []

        for rid in list(running):
            if budget <= 0:
                break
            req = reqs[rid]
            if not allocate(req, req["prompt_done"] + req["output_done"] + 1):
```

```

        continue
    req["output_done"] += 1
    req["first_decode"] = req["first_decode"] if req["first_decode"] is not None else step
    budget -= 1
    actions.append(f"decode:{rid}")
    if req["output_done"] >= req["max_output"]:
        running.remove(rid)
        free_blocks += req["blocks"]
        req["blocks"] = 0
        req["finish_step"] = step
        finished_order.append(rid)
        actions.append(f"finish:{rid}")

def waiting_key(rid):
    req = reqs[rid]
    is_long = req["input_tokens"] >= 2048
    return (req["priority"], is_long, req["arrival"], req["input_tokens"])

for rid in sorted(list(waiting), key=waiting_key):
    if budget <= 0 or len(running) >= max_active:
        break
    req = reqs[rid]
    remaining = req["input_tokens"] - req["prompt_done"]
    chunk = min(remaining, max_prefill_chunk, budget)
    if chunk <= 0:
        continue
    if not allocate(req, req["prompt_done"] + chunk):
        continue
    req["prompt_done"] += chunk
    req["first_scheduled"] = req["first_scheduled"] if req["first_scheduled"] is not None else step
    budget -= chunk
    actions.append(f"prefill:{rid}:{chunk}")
    if req["prompt_done"] >= req["input_tokens"]:
        waiting.remove(rid)
        running.append(rid)
        actions.append(f"running:{rid}")

active_ids = waiting + running
used_blocks = kv_block_budget - free_blocks
capacity_tokens = sum(reqs[rid]["blocks"] * block_size for rid in active_ids)
actual_tokens = sum(reqs[rid]["prompt_done"] + reqs[rid]["output_done"] for rid in active_ids)
waste = (capacity_tokens - actual_tokens) / capacity_tokens if capacity_tokens else 0.0
trace.append({
    "step": step,
    "actions": actions,
    "running": list(running),
    "waiting": list(waiting),
    "used_blocks": used_blocks,
    "kv_waste": round(waste, 3),
})
if not waiting and not running and all(r["arrival"] <= step for r in reqs.values()):
    break

queue_waits = [r["first_scheduled"] - r["arrival"] for r in reqs.values() if r["first_scheduled"] is not None]
ttft_steps = [r["first_decode"] - r["arrival"] for r in reqs.values() if r["first_decode"] is not None]

```

```

active_counts = [len(x["running"]) + len(x["waiting"]) for x in trace]
long_prompt_chunks = sum(1 for x in trace for a in x["actions"] if a.startswith("prefill:B") or a.startswith("pre
return {
    "iterations": len(trace),
    "finished_order": finished_order,
    "avg_queue_wait_steps": round(sum(queue_waits) / len(queue_waits), 2),
    "p95_ttft_steps": percentile(ttft_steps, 0.95),
    "p95_active_sequences": percentile(active_counts, 0.95),
    "max_kv_blocks_used": max(x["used_blocks"] for x in trace),
    "max_kv_waste_ratio": max(x["kv_waste"] for x in trace),
    "long_prompt_chunks": long_prompt_chunks,
    "trace_tail": trace[-4:],
}

requests = [
    {"id": "A", "tenant": "free", "arrival": 0, "input_tokens": 512, "max_output": 3, "priority": 2},
    {"id": "B", "tenant": "pro", "arrival": 0, "input_tokens": 2304, "max_output": 4, "priority": 2},
    {"id": "C", "tenant": "free", "arrival": 1, "input_tokens": 384, "max_output": 2, "priority": 2},
    {"id": "D", "tenant": "enterprise", "arrival": 1, "input_tokens": 768, "max_output": 3, "priority": 0},
    {"id": "E", "tenant": "pro", "arrival": 2, "input_tokens": 2048, "max_output": 2, "priority": 1},
]

scheduler_examples = simulate_continuous_batching(requests)
audit = audit_scheduler_profiles(profiles)
smoke = {
    "complete_case_passes": not any(v is False for v in gate_results(complete).values()),
    "caught_static_batching": "static_batching_bad" in audit["failed_cases"],
    "caught_paged_kv_gap": "paged_kv_no_freelist_bad" in audit["failed_cases"],
    "caught_cancel_leak": "cancel_leaks_blocks_bad" in audit["failed_cases"],
    "caught_metrics_gap": "scheduler_metrics_missing_bad" in audit["failed_cases"],
    "caught_gate_gap": "scheduler_gate_missing_bad" in audit["failed_cases"],
}

print(f"scheduler_examples={scheduler_examples}")
print(f"smoke={smoke}")
print(f"metrics={audit['metrics']}")
print(f"hard_blocker_count={audit['hard_blocker_count']}")
print(f"failed_cases={audit['failed_cases']}")
print(f"failed_gates={audit['failed_gates']}")
print(f"scheduler_gate_pass={audit['scheduler_gate_pass']}")

```

这个 demo 想说明：Continuous Batching、PagedAttention 和队列调度不是三个孤立名词。请求动态加入退出需要 scheduler；scheduler 要靠 token budget 和 phase policy 平衡 TTFT / TPOT；动态请求又要求 KV block 能按需分配、释放和统计碎片；多租户环境还必须把长短请求隔离、公平性、取消释放、streaming 背压和最终上线门禁放进同一个审计表。

29.21 面试常见追问

问题一：Continuous Batching 和普通 batching 有什么区别？

可以回答：普通 batching 通常一批请求固定执行到结束；Continuous Batching 在每个 decode step 动态调整活跃请求集合，完成的请求退出，新请求加入，更适合输出长度不一致的生成式模型。

问题二：PagedAttention 解决什么问题？

可以回答：它通过类似分页的方式管理 KV cache，减少显存浪费和碎片，提高长短不一请求下的并发能力，也

更适合 dynamic batching。

问题三：为什么调度不能只按请求数？

可以回答：大模型请求的 token 数差异巨大，长 prompt 或长输出请求的资源消耗远高于短请求。按 token budget、KV cache 和 SLO 调度更合理。

问题四：如何避免长请求拖慢短请求？

可以回答：可以做长短请求分队列、长上下文专用实例、token budget 调度、限制最大上下文和输出长度，并为短请求保留容量。

29.22 小练习

1. 为什么生成式模型不适合简单静态 batching？
2. Continuous Batching 的核心思想是什么？
3. PagedAttention 和 KV cache 管理有什么关系？
4. 为什么 token budget 比 batch size 更适合 LLM 调度？
5. Prefill 和 decode 会如何互相影响？
6. 多租户推理平台如何设计公平性？
7. Admission control 为什么重要？
8. 如果 p99 延迟突然升高，你会查看哪些调度指标？

29.23 本章小结

本章讲了 Continuous Batching、PagedAttention 和队列调度。

你需要记住：

1. Batching 提高吞吐，但会影响延迟。
2. 静态 batching 不适合输出长度差异大的生成式模型。
3. Continuous Batching 允许请求按 decode step 动态加入和退出。
4. PagedAttention 通过分页式 KV cache 管理减少显存浪费和碎片。
5. 调度策略要同时考虑 TTFT、TPOT、吞吐、KV cache、公平性和成本。
6. 大模型调度应更多关注 token budget，而不是只看请求数。
7. Admission control、长短请求分队列和多租户公平性是生产推理平台的关键能力。

下一章我们会讲模型路由：模型选择、能力路由、成本路由和降级路由。

第 30 章 模型路由：模型选择、能力路由、成本路由和降级路由

上一章讲了 runtime 内部如何通过 batching、PagedAttention 和队列调度提升吞吐。本章把视角上移到平台层，讲模型路由。

在单模型服务里，请求只有一个去处。但在真实大模型平台里，通常同时存在多个模型、多个版本、多个 endpoint、多个租户和多种业务目标。此时，请求到底应该发给谁，就变成了核心问题。

先记住一句话：

模型路由不是简单负载均衡，而是在质量、能力、延迟、成本、稳定性和安全策略之间做选择。

30.0 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做过资料校准。重点参考的是 OpenAI 对模型能力、成本、延迟优化和生产检查项的公开文档，LiteLLM Router 对模型组、负载均衡、fallback、budget 和重试的工程抽象，OpenTelemetry 对 trace / span / attributes 的可观测性口径，以及 Google SRE 关于过载保护、配额、重试预算和 graceful degradation 的系统设计原则。

这些资料共同指向一个稳定事实：模型路由不是“把请求随机分给某个 endpoint”，也不是“永远使用最强模型”。生产路由要先做权限和能力过滤，再把质量、延迟 SLO、成本预算、实时健康度、灰度策略、fallback chain、降级策略和 route trace 放进同一张策略表。

本章只抽象截至 2026-06 仍稳定的多模型平台路由口径，不把某个闭源模型 ID、价格、上下文长度、SDK 字段名或第三方框架默认参数写成通用标准。正文中的公式用于面试表达、容量估算和策略审计；真实上线仍要用目标模型、真实 token 分布、在线评估、业务 SLO、成本账单和事故复盘数据校准。

30.1 为什么需要模型路由

如果平台只有一个模型、一个版本、一个实例，路由很简单。

Request -> Model Server

但生产环境通常是这样：

Request

-> Router

-> gpt-like-large-v3

-> gpt-like-small-v2

-> code-model-v1

-> embedding-model-v4

-> vision-language-model-v2

-> fallback-model-v1

平台需要决定：

1. 用哪个模型？
2. 用哪个版本？
3. 用哪个 endpoint？
4. 是否走灰度版本？
5. 是否降级到小模型？
6. 是否拒绝请求？
7. 是否从缓存返回？

这就是模型路由。

30.2 模型路由和负载均衡的区别

传统负载均衡通常在一组等价实例之间选择一个实例。

例如：

service-a: instance-1 / instance-2 / instance-3

这些实例能力基本相同。

模型路由面对的是不等价目标：

1. 模型能力不同。
2. 模型成本不同。
3. 模型延迟不同。
4. 上下文长度不同。
5. 安全等级不同。
6. 支持的任务类型不同。
7. 版本质量不同。
8. 当前负载不同。

所以模型路由先做“选模型和版本”，再做“选实例”。

30.3 模型路由的输入信息

Router 通常会使用这些信息：

1. 请求指定的模型名。
2. 业务场景。
3. 租户身份。
4. 用户等级。
5. 输入 token 数。
6. 预估输出 token 数。
7. 是否需要流式返回。
8. 是否需要工具调用。
9. 是否需要多模态能力。
10. 延迟 SLO。
11. 成本预算。
12. 安全策略。
13. 灰度规则。
14. endpoint 负载。
15. 健康状态。

路由不是只看一个字段，而是综合决策。

30.4 模型路由的输出结果

Router 的输出不应该只是一个 URL。

更完整的结果包括：

```
RouteDecision
  model
  model_version
  endpoint
  runtime
  route_reason
  fallback_chain
  timeout_budget
  max_tokens
  safety_policy
  cost_policy
  trace_id
```

这样做的好处是：

1. 便于审计。
2. 便于 debug。
3. 便于成本归因。
4. 便于回放问题请求。
5. 便于分析路由策略效果。

线上系统里，“为什么这个请求路由到这个模型” 必须可解释。

30.5 显式模型选择

最简单的路由方式是用户显式指定模型。

例如：

```
{
  "model": "large-chat-v3",
```

```
"messages": [...]  
}
```

平台根据 `model` 字段找到对应 endpoint。

这种方式简单直接，适合：

1. 开发者 API。
2. 内部平台。
3. 用户明确知道模型差异的场景。
4. 需要稳定复现的实验。

但缺点也明显：

1. 用户要理解模型能力。
2. 成本优化依赖用户自觉。
3. 模型升级需要用户迁移。
4. 难以自动降级。
5. 难以按业务目标做智能选择。

所以多模型平台通常还需要能力路由和策略路由。

30.6 能力路由

能力路由是根据任务能力选择模型。

例如：

1. 普通对话 -> 通用 chat 模型。
2. 代码补全 -> code 模型。
3. 文档问答 -> 长上下文模型。
4. 图片理解 -> 多模态模型。
5. 向量检索 -> embedding 模型。
6. 函数调用 -> tool-use 能力强的模型。
7. 复杂推理 -> reasoning 模型。

能力路由的关键是维护模型能力画像。

Model Registry 中需要记录：

1. 支持任务类型。
2. 最大上下文长度。
3. 支持语言。
4. 支持多模态类型。
5. tool calling 能力。
6. 结构化输出能力。
7. 推理能力等级。
8. 安全等级。
9. 评估分数。

没有能力画像，能力路由只能靠硬编码。

30.7 成本路由

成本路由是根据成本目标选择模型。

不是所有请求都需要最大模型。

例如：

1. 简单分类用小模型。
2. 简单改写用中模型。
3. 复杂推理用大模型。

4. 高价值用户用更强模型。
5. 低优先级后台任务用便宜模型。

成本路由常见策略：

1. 小模型优先。
2. 失败再升级到大模型。
3. 根据置信度升级。
4. 根据用户等级选择模型。
5. 根据预算选择模型。
6. 根据时间段选择模型。

成本路由不是单纯省钱，而是在满足质量和 SLO 的前提下降低成本。

30.8 质量路由

质量路由是根据模型效果选择路径。

平台可以基于：

1. 离线评估结果。
2. 在线反馈。
3. A/B 测试结果。
4. 用户满意度。
5. 任务成功率。
6. 工具调用成功率。
7. 安全拦截率。

来调整路由策略。

例如某个新模型在数学题上更好，但在代码生成上退化，那么可以只让数学任务路由到新模型，而代码任务继续使用旧模型。

成熟平台不会把“新模型”直接替换“旧模型”，而是按任务和指标逐步迁移。

30.9 延迟路由

有些请求对延迟敏感，有些请求对质量更敏感。

例如：

1. 在线聊天要求低 TTFT。
2. 搜索改写要求几十毫秒到几百毫秒。
3. 后台报告生成可以等待更久。
4. Agent 长任务可以接受较长执行时间。

延迟路由可以根据 SLO 选择：

1. 更小模型。
2. 更近地域。
3. 更空闲 endpoint。
4. 更短上下文策略。
5. 非流式或流式返回。
6. 是否启用复杂推理。

延迟路由要结合实时负载，否则静态模型选择不够。

30.10 降级路由

降级路由用于系统压力过大、模型故障或成本超预算时。

常见降级路径：

1. 大模型 -> 小模型。
2. reasoning 模型 -> 普通 chat 模型。
3. 长上下文 -> 截断上下文。
4. 实时生成 -> 缓存结果。
5. 多模态模型 -> 文本模型。
6. 工具调用流程 -> 简化回答。
7. 高温采样 -> 确定性生成。

降级不能随便做，因为它可能影响质量和用户体验。

平台需要明确：

1. 哪些业务允许降级。
2. 哪些用户允许降级。
3. 降级后如何标记。
4. 降级指标如何统计。
5. 是否需要用户可见提示。

30.11 Fallback Chain

Fallback chain 是一组备用路径。

例如：

```
primary: large-chat-v3
fallback-1: large-chat-v2
fallback-2: medium-chat-v3
fallback-3: cached-response
```

触发 fallback 的原因可能是：

1. endpoint 不健康。
2. 请求超时。
3. 队列过长。
4. 错误率过高。
5. 配额不足。
6. 成本预算耗尽。

Fallback 要避免无限重试。

每次 fallback 都要消耗时间预算和成本预算。

30.12 灰度路由

模型发布需要灰度。

灰度路由可以按：

1. 流量比例。
2. 租户。
3. 用户分桶。
4. 地域。
5. 业务场景。
6. prompt 类型。
7. 请求风险等级。

把部分请求导向新模型。

灰度期间要对比：

1. 质量指标。
2. 延迟指标。

3. 成本指标。
4. 错误率。
5. 安全指标。
6. 用户反馈。

灰度路由必须支持快速回滚。

30.13 多地域路由

如果平台跨地域部署，还要考虑地域路由。

目标包括：

1. 降低网络延迟。
2. 满足数据合规要求。
3. 提高可用性。
4. 均衡资源。
5. 支持灾备。

但大模型多地域部署成本高，因为模型权重巨大，GPU 资源昂贵，流量分布也不均匀。

多地域路由通常要考虑：

1. 用户所在地域。
2. 数据是否允许跨境或跨区。
3. 模型是否在该地域部署。
4. endpoint 当前负载。
5. 灾备策略。

30.14 Router 的实时信号

Router 不应该只看静态配置。

还应该接入实时信号：

1. endpoint 健康状态。
2. 当前 QPS。
3. queue length。
4. TTFT。
5. TPOT。
6. p95 / p99 延迟。
7. 错误率。
8. timeout rate。
9. GPU 显存水位。
10. KV cache 使用率。
11. admission reject rate。

如果某个 endpoint 队列已经很长，router 还继续导流，就会放大故障。

30.15 Router 的配置管理

Router 配置通常包括：

1. 模型到 endpoint 的映射。
2. 版本权重。
3. 灰度规则。
4. fallback chain。
5. 租户权限。
6. 成本策略。
7. 限流策略。

8. 降级策略。
9. 地域策略。
10. 安全策略。

这些配置必须：

1. 可版本化。
2. 可审计。
3. 可回滚。
4. 可灰度。
5. 支持 dry-run。

Router 配置错误可能导致大面积线上事故。

30.16 Router 的可观测性

模型路由必须记录 route trace。

至少包括：

1. 请求 ID。
2. 租户。
3. 输入 token 数。
4. 选择的模型。
5. 选择的版本。
6. 选择的 endpoint。
7. 路由原因。
8. 是否灰度。
9. 是否降级。
10. 是否 fallback。
11. 路由耗时。
12. 最终延迟和错误码。
13. 成本估算。

否则线上出现质量问题时，你很难回答：为什么这个用户被路由到了这个模型？

30.17 Router 的风险

模型路由会引入新的风险：

1. 路由规则过复杂，难以维护。
2. 静态规则和实时负载冲突。
3. 降级导致质量不可控。
4. fallback 造成重复成本。
5. 灰度分桶不稳定。
6. 缓存和路由策略不一致。
7. 多租户权限配置错误。
8. route trace 缺失导致问题无法复盘。

工程上要避免“一堆 if else 写死模型选择”。

更好的做法是将策略配置化、版本化、可观测化。

30.18 一个 Router 决策流程

一个完整路由流程可以是：

1. 解析请求和租户身份
2. 检查模型访问权限
3. 识别任务类型和能力需求

4. 估算 input/output token
5. 读取成本和延迟 SLO
6. 筛选候选模型
7. 应用灰度规则
8. 结合实时负载选择 endpoint
9. 生成 fallback chain
10. 写入 route trace
11. 转发请求到 runtime

这个流程体现了模型路由的本质：先筛选可用能力，再按策略和实时状态做选择。

30.19 模型路由审计指标与最小 demo

第二轮精修时，需要把模型路由从“规则列表”升级成可解释、可回放、可门禁的决策模型。

可以把一次请求写成：

$$r_i = (q_i, u_i, a_i, n_i, o_i, s_i, b_i, \ell_i, \rho_i, d_i, z_i)$$

其中， q_i 是任务类型， u_i 是租户或用户等级， a_i 是能力需求， n_i 是输入 token 数， o_i 是预估输出 token 数， s_i 是延迟 SLO， b_i 是成本预算， ℓ_i 是实时负载信号， ρ_i 是风险等级， d_i 是是否允许降级， z_i 是 trace 和策略版本字段。

对一组模型候选 $\mathcal{M}$ ，路由器先做硬过滤：

$$\mathcal{M}_i = \{m \in \mathcal{M} : A(m, u_i) = 1 \wedge K(m, a_i) = 1 \wedge H(m) = 1 \wedge C(i, m) \leq b_i \wedge T(i, m) \leq s_i\}$$

这里 $A(m, u_i)$ 表示租户是否有权限访问模型 m ， $K(m, a_i)$ 表示模型能力是否满足请求， $H(m)$ 表示 endpoint 健康和负载是否可接受， $C(i, m)$ 是预估成本， $T(i, m)$ 是预估延迟。硬过滤不通过的模型不能靠打分“补回来”，否则会把权限、预算或 SLO 风险藏进平均分。

成本可以按输入和输出 token 估算：

$$C(i, m) = \frac{n_i p_{\{\mathrm{in}\}, m} + o_i p_{\{\mathrm{out}\}, m}}{1000}$$

其中， $p_{\{\mathrm{in}\}, m}$ 和 $p_{\{\mathrm{out}\}, m}$ 分别是模型 m 的每千输入 token 和每千输出 token 单价或内部成本口径。线上成本还可能加入 GPU seconds、cache hit savings、region、priority 和 reserved capacity，但面试表达先把 token 成本讲清楚。

延迟可以拆成：

$$T(i, m) = T_{\{\mathrm{route}\}} + T_{\{\mathrm{queue}\}, m} + T_{\{\mathrm{ttft}\}, m} + o_i T_{\{\mathrm{tpot}\}, m}$$

其中， $T_{\{\mathrm{route}\}}$ 是网关和路由开销， $T_{\{\mathrm{queue}\}, m}$ 是当前 endpoint 排队开销， $T_{\{\mathrm{ttft}\}, m}$ 是模型首 token 延迟基线， $T_{\{\mathrm{tpot}\}, m}$ 是每个输出 token 的平均生成时间。这个公式的重点不是精确预测每个请求，而是提醒面试时不要只按模型名字选路由。

硬过滤后，可以用 utility score 排序：

$$U(i, m) = \alpha Q(i, m) - \beta \frac{C(i, m)}{b_i} - \gamma \max\left(0, \frac{T(i, m)}{s_i} - 1\right) - \delta R(i, m)$$

其中， $Q(i, m)$ 是任务质量或 eval 分数， $R(i, m)$ 是安全、版本或业务风险惩罚， $\alpha, \beta, \gamma, \delta$ 是策略权重。这个打分只应该在权限、能力、健康、预算和 SLO 已经通过之后使用。

Fallback chain 也要受预算约束：

$$\sum_{k=1}^{K_i} T(i, m_k) \leq s_i, \quad \sum_{k=1}^{K_i} C(i, m_k) \leq b_i$$

这表示一次请求最多尝试 K_i 个候选模型，总时间和总成本仍不能无限膨胀。否则 fallback 会把一次失败放大成多次排队、多次计费 and 更差的尾延迟。

灰度路由要稳定分桶：

$$h_i = H(u_i, v) \bmod 10000, \quad G(i, v) = \mathbf{1}[h_i < 10000 w_v]$$

其中，H 是稳定哈希，v 是候选版本，w_v 是灰度权重。稳定分桶可以避免同一用户在新旧模型之间来回跳，方便对比质量、延迟和安全指标。

Route trace 覆盖率可以写成：

$$C_{\{\mathrm{trace}\}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\{\mathrm{trace}\}(i) \supseteq F_{\{\mathrm{required}\}}]$$

其中， $F_{\{\mathrm{required}\}}$ 是必填 trace 字段集合，例如 request id、tenant、task、input/output tokens、selected model、version、route reason、fallback、degrade、cost、latency 和 config version。没有 trace 的路由系统无法复盘质量回归和成本异常。

最终模型路由门禁可以写成：

$$G_{\{\mathrm{route}\}} = \mathbf{1}[\left[\min_j C_j \geq \tau_j \ \& \ C_{\{\mathrm{trace}\}} \geq \tau_{\{\mathrm{trace}\}} \ \& \ R_{\{\mathrm{fallback}\}} \leq \tau_{\{\mathrm{fallback}\}} \ \& \ R_{\{\mathrm{degrade}\}} \leq \tau_{\{\mathrm{degrade}\}} \ \& \ P_0 \leq \tau_{\{\mathrm{P0}\}}\right]]$$

其中， C_j 是各审计维度覆盖率， $R_{\{\mathrm{fallback}\}}$ 是 fallback 率， $R_{\{\mathrm{degrade}\}}$ 是降级率， P_0 是未关闭的 P0 风险数。这个门禁强调：路由策略要能解释、能回滚、能观测，而不是只在 demo 流量上看起来可用。

下面这个 0 依赖 Python demo 演示一个简化模型路由器：先按租户、能力、健康、预算和 SLO 过滤候选，再按质量、成本、延迟和风险打分，最后输出 fallback chain、降级请求、trace 覆盖率和路由审计门禁。

```
from hashlib import sha256
```

```
CHECKS = [
    "request_intent_profile",
    "model_capability_profile",
    "permission_candidate_filter",
    "cost_budget_policy",
    "latency_slo_policy",
    "quality_safety_policy",
    "fallback_chain_governance",
    "canary_routing_stability",
    "realtime_health_load",
    "route_trace_coverage",
    "degrade_policy_control",
    "route_config_versioning",
    "model_routing_gate",
]

MODELS = {
    "small_chat": {
        "tasks": {"chat", "summary"},
        "modalities": {"text"},
        "max_context": 8192,
        "tool_calling": False,
        "structured_output": True,
        "tenants": {"free", "pro", "enterprise"},
        "quality": {"chat": 0.78, "summary": 0.80, "reasoning": 0.50, "code": 0.35},
        "safety_score": 0.76,
        "input_cost": 0.10,
        "output_cost": 0.40,
        "ttft_ms": 180,
        "tpot_ms": 9,
        "health": 0.99,
        "load": 0.42,
        "version": "v2",
    },
    "large_chat": {
```

```

    "tasks": {"chat", "summary", "reasoning", "tool"},
    "modalities": {"text"},
    "max_context": 128000,
    "tool_calling": True,
    "structured_output": True,
    "tenants": {"pro", "enterprise"},
    "quality": {"chat": 0.90, "summary": 0.91, "reasoning": 0.94, "code": 0.70},
    "safety_score": 0.90,
    "input_cost": 1.00,
    "output_cost": 3.00,
    "ttft_ms": 520,
    "tpot_ms": 24,
    "health": 0.98,
    "load": 0.61,
    "version": "v3",
  },
  "large_chat_v4_canary": {
    "tasks": {"chat", "summary", "reasoning", "tool"},
    "modalities": {"text"},
    "max_context": 128000,
    "tool_calling": True,
    "structured_output": True,
    "tenants": {"enterprise"},
    "quality": {"chat": 0.91, "summary": 0.93, "reasoning": 0.95, "code": 0.72},
    "safety_score": 0.90,
    "input_cost": 1.10,
    "output_cost": 3.20,
    "ttft_ms": 560,
    "tpot_ms": 25,
    "health": 0.97,
    "load": 0.57,
    "version": "v4-canary",
    "canary_weight": 0.15,
  },
  "code_model": {
    "tasks": {"code", "tool"},
    "modalities": {"text"},
    "max_context": 64000,
    "tool_calling": True,
    "structured_output": True,
    "tenants": {"pro", "enterprise"},
    "quality": {"chat": 0.60, "summary": 0.55, "reasoning": 0.70, "code": 0.93},
    "safety_score": 0.84,
    "input_cost": 0.80,
    "output_cost": 2.40,
    "ttft_ms": 430,
    "tpot_ms": 21,
    "health": 0.96,
    "load": 0.50,
    "version": "v1",
  },
  "vlm": {
    "tasks": {"vision", "chat", "summary"},
    "modalities": {"text", "image"},
    "max_context": 32000,

```

```

    "tool_calling": False,
    "structured_output": True,
    "tenants": {"enterprise"},
    "quality": {"vision": 0.92, "chat": 0.82, "summary": 0.83},
    "safety_score": 0.86,
    "input_cost": 1.20,
    "output_cost": 3.60,
    "ttft_ms": 760,
    "tpot_ms": 27,
    "health": 0.95,
    "load": 0.48,
    "version": "v2",
  },
  "fallback_chat": {
    "tasks": {"chat", "summary"},
    "modalities": {"text"},
    "max_context": 16000,
    "tool_calling": False,
    "structured_output": True,
    "tenants": {"free", "pro", "enterprise"},
    "quality": {"chat": 0.70, "summary": 0.72, "reasoning": 0.42, "code": 0.30},
    "safety_score": 0.74,
    "input_cost": 0.06,
    "output_cost": 0.20,
    "ttft_ms": 140,
    "tpot_ms": 8,
    "health": 0.995,
    "load": 0.38,
    "version": "v1",
  },
}

```

```

REQUESTS = [
  {
    "id": "chat_001",
    "tenant": "free",
    "user": "alice",
    "task": "summary",
    "modalities": {"text"},
    "input_tokens": 900,
    "expected_output_tokens": 120,
    "needs_tool": False,
    "slo_ms": 1800,
    "budget": 0.18,
    "risk": 0.20,
    "allow_degrade": True,
  },
  {
    "id": "code_002",
    "tenant": "pro",
    "user": "bob",
    "task": "code",
    "modalities": {"text"},
    "input_tokens": 1400,
    "expected_output_tokens": 260,
  },
]

```



```

    "needs_tool": True,
    "slo_ms": 7000,
    "budget": 2.20,
    "risk": 0.35,
    "allow_degrade": False,
  },
  {
    "id": "image_003",
    "tenant": "enterprise",
    "user": "carol",
    "task": "vision",
    "modalities": {"text", "image"},
    "input_tokens": 1800,
    "expected_output_tokens": 220,
    "needs_tool": False,
    "slo_ms": 8000,
    "budget": 3.60,
    "risk": 0.30,
    "allow_degrade": False,
  },
  {
    "id": "reason_004",
    "tenant": "enterprise",
    "user": "dave",
    "task": "reasoning",
    "modalities": {"text"},
    "input_tokens": 2200,
    "expected_output_tokens": 520,
    "needs_tool": True,
    "slo_ms": 15000,
    "budget": 4.20,
    "risk": 0.45,
    "allow_degrade": False,
  },
  {
    "id": "batch_005",
    "tenant": "free",
    "user": "erin",
    "task": "chat",
    "modalities": {"text"},
    "input_tokens": 300,
    "expected_output_tokens": 40,
    "needs_tool": False,
    "slo_ms": 2600,
    "budget": 0.05,
    "risk": 0.10,
    "allow_degrade": True,
  },
  {
    "id": "free_image_006",
    "tenant": "free",
    "user": "frank",
    "task": "vision",
    "modalities": {"text", "image"},
    "input_tokens": 1100,

```

```

        "expected_output_tokens": 90,
        "needs_tool": False,
        "slo_ms": 3000,
        "budget": 0.10,
        "risk": 0.20,
        "allow_degrade": True,
        "degrade_to_text": True,
    },
]

def stable_bucket(*parts):
    digest = sha256("".join(map(str, parts)).encode()).hexdigest()
    return int(digest[:8], 16) % 10000

def estimate_cost(req, model):
    return (
        req["input_tokens"] * model["input_cost"]
        + req["expected_output_tokens"] * model["output_cost"]
    ) / 1000.0

def estimate_latency(req, model):
    route_overhead_ms = 24
    queue_penalty_ms = int(200 * model["load"])
    return route_overhead_ms + queue_penalty_ms + model["tft_ms"] + req["expected_output_tokens"] * model["tpot_ms"]

def capability_match(req, model):
    context = req["input_tokens"] + req["expected_output_tokens"]
    return (
        req["task"] in model["tasks"]
        and req["modalities"].issubset(model["modalities"])
        and context <= model["max_context"]
        and (not req["needs_tool"] or model["tool_calling"])
    )

def trace_record(req, selected, decision, cost, latency):
    return {
        "request_id": req["id"],
        "tenant": req["tenant"],
        "task": req["task"],
        "input_tokens": req["input_tokens"],
        "expected_output_tokens": req["expected_output_tokens"],
        "selected_model": selected,
        "decision": decision,
        "cost": round(cost, 4),
        "latency_ms": int(latency),
        "config_version": "route-policy-2026-06-12",
        "reason": "highest_utility_after_policy_filter",
    }

```

```

def route_request(req):
    candidates = []
    rejected = []
    for name, model in MODELS.items():
        if "canary_weight" in model:
            bucket = stable_bucket(req["tenant"], req["user"], model["version"])
            if bucket >= int(model["canary_weight"] * 10000):
                rejected.append((name, "canary_bucket"))
                continue
        if req["tenant"] not in model["tenants"]:
            rejected.append((name, "permission"))
            continue
        if not capability_match(req, model):
            rejected.append((name, "capability"))
            continue
        if model["health"] < 0.90 or model["load"] > 0.90:
            rejected.append((name, "health_or_load"))
            continue
        cost = estimate_cost(req, model)
        if cost > req["budget"]:
            rejected.append((name, "budget"))
            continue
        latency = estimate_latency(req, model)
        if latency > req["slo_ms"]:
            rejected.append((name, "latency"))
            continue
        quality = model["quality"].get(req["task"], 0.0)
        cost_ratio = cost / max(req["budget"], 1e-9)
        risk_penalty = max(0.0, req["risk"] - model["safety_score"])
        score = 1.8 * quality - 0.55 * cost_ratio - 0.35 * risk_penalty
        candidates.append((score, name, cost, latency, quality))

    if not candidates and req.get("allow_degrade") and req.get("degrade_to_text"):
        text_req = dict(req)
        text_req["task"] = "summary"
        text_req["modalities"] = {"text"}
        text_req["needs_tool"] = False
        text_req["expected_output_tokens"] = min(req["expected_output_tokens"], 80)
        text_req["budget"] = max(req["budget"], 0.10)
        degraded_decision = route_request(text_req)
        degraded_decision["request_id"] = req["id"]
        degraded_decision["degraded"] = True
        degraded_decision["degrade_reason"] = "drop_image_and_answer_text_only"
        degraded_decision["rejected_candidates"] = rejected[:4]
        return degraded_decision

    if not candidates:
        return {
            "request_id": req["id"],
            "selected_model": None,
            "decision": "reject",
            "fallback_chain": [],
            "degraded": False,
            "rejected_candidates": rejected[:4],
            "trace": trace_record(req, None, "reject", 0.0, 0),

```

```

    }

    candidates.sort(reverse=True)
    score, selected, cost, latency, quality = candidates[0]
    chain = [selected]
    for fallback in ("small_chat", "fallback_chat"):
        if fallback != selected and req["tenant"] in MODELS[fallback]["tenants"]:
            if req["task"] in MODELS[fallback]["tasks"] and req["modalities"].issubset(MODELS[fallback]["modalities"]):
                chain.append(fallback)
    return {
        "request_id": req["id"],
        "selected_model": selected,
        "decision": "route",
        "fallback_chain": chain[:3],
        "score": round(score, 3),
        "estimated_cost": round(cost, 4),
        "estimated_latency_ms": int(latency),
        "quality": quality,
        "degraded": False,
        "rejected_candidates": rejected[:4],
        "trace": trace_record(req, selected, "route", cost, latency),
    }

def build_profile_cases():
    complete = {"name": "complete_model_routing_case"}
    complete.update({check: True for check in CHECKS})
    cases = [complete]
    bad_names = [
        "request_intent_missing_bad",
        "model_capability_missing_bad",
        "permission_filter_missing_bad",
        "cost_budget_missing_bad",
        "latency_slo_missing_bad",
        "quality_safety_missing_bad",
        "fallback_chain_unbounded_bad",
        "canary_bucket_unstable_bad",
        "health_load_ignored_bad",
        "route_trace_missing_bad",
        "degrade_policy_uncontrolled_bad",
        "config_version_missing_bad",
        "model_routing_gate_missing_bad",
    ]
    for bad_name, failed_check in zip(bad_names, CHECKS):
        case = {"name": bad_name}
        case.update({check: True for check in CHECKS})
        case[failed_check] = False
        cases.append(case)
    return cases

def audit_profiles(cases):
    metrics = {}
    for check in CHECKS:
        metrics[check] = round(sum(1 for case in cases if case.get(check)) / len(cases), 3)

```

```

failed_cases = [case["name"] for case in cases if not all(case.get(check) for check in CHECKS)]
failed_gates = [check for check, value in metrics.items() if value < 1.0]
hard_blockers = sum(1 for case in cases if case["name"].endswith("_bad"))
gate_pass = min(metrics.values()) >= 0.95 and hard_blockers == 0
return metrics, failed_cases, failed_gates, hard_blockers, gate_pass

decisions = [route_request(req) for req in REQUESTS]
required_trace_fields = {
    "request_id",
    "tenant",
    "task",
    "input_tokens",
    "expected_output_tokens",
    "selected_model",
    "decision",
    "cost",
    "latency_ms",
    "config_version",
    "reason",
}
complete_traces = sum(1 for item in decisions if required_trace_fields.issubset(item["trace"]))
trace_coverage = complete_traces / len(decisions)
metrics, failed_cases, failed_gates, hard_blockers, gate_pass = audit_profiles(build_profile_cases())

print("routing_decisions=", {item["request_id"]: item["selected_model"] for item in decisions})
print("fallback_chains=", {item["request_id"]: item["fallback_chain"] for item in decisions})
print("degraded_requests=", [item["request_id"] for item in decisions if item["degraded"]])
print("estimated_costs=", {item["request_id"]: item.get("estimated_cost", 0.0) for item in decisions})
print("estimated_latency_ms=", {item["request_id"]: item.get("estimated_latency_ms", 0) for item in decisions})
print("route_trace_coverage=", round(trace_coverage, 3))
print("metrics=", metrics)
print("hard_blocker_count=", hard_blockers)
print("failed_cases=", failed_cases)
print("failed_gates=", failed_gates)
print("model_routing_gate_pass=", gate_pass)

```

输出示例:

```

routing_decisions= {'chat_001': 'fallback_chat', 'code_002': 'code_model', 'image_003': 'vlm', 'reason_004': 'large_chat'}
fallback_chains= {'chat_001': ['fallback_chat', 'small_chat'], 'code_002': ['code_model'], 'image_003': ['vlm'], 'reason_004': ['reason_model']}
degraded_requests= ['free_image_006']
estimated_costs= {'chat_001': 0.078, 'code_002': 1.744, 'image_003': 2.952, 'reason_004': 3.76, 'batch_005': 0.026, 'free_image_006': 0.01}
estimated_latency_ms= {'chat_001': 1200, 'code_002': 6014, 'image_003': 6820, 'reason_004': 13146, 'batch_005': 560, 'free_image_006': 100}
route_trace_coverage= 1.0
metrics= {'request_intent_profile': 0.929, 'model_capability_profile': 0.929, 'permission_candidate_filter': 0.929, 'cost_efficiency': 0.929}
hard_blocker_count= 13
model_routing_gate_pass= False

```

这个 demo 的重点不是模拟真实线上价格或模型能力，而是训练一套面试思维：路由器必须先拒绝不合规候选，再在合规候选里优化质量、成本和延迟；fallback、灰度、降级和 trace 都要可审计。

30.20 面试常见追问

问题一：模型路由和负载均衡有什么区别？

可以回答：负载均衡通常在等价实例之间分流；模型路由要先在不同能力、成本、延迟、版本和安全等级的模型

之间选择，再选择具体 endpoint。

问题二：如何设计成本路由？

可以回答：可以建立模型成本和能力画像，对简单任务优先使用小模型，对复杂任务或低置信度结果升级到大模型，同时结合租户预算、SLO 和质量反馈做策略控制。

问题三：降级路由要注意什么？

可以回答：要明确哪些业务允许降级，降级到什么模型或缓存，是否影响用户体验，如何标记降级结果，并记录降级率、质量和成本影响。

问题四：Router 为什么需要实时指标？

可以回答：因为 endpoint 负载和健康状态动态变化。如果只按静态配置路由，可能继续把请求打到拥塞或故障实例，导致延迟和错误进一步扩大。

问题五：如何避免 fallback 变成雪崩放大器？

可以回答：fallback chain 要有最大尝试次数、总时间预算、总成本预算、错误类型分类、幂等控制和熔断规则。不能对所有错误无脑重试，也不能在主模型超时后继续走一串同样拥塞的模型。

问题六：模型灰度为什么要稳定分桶？

可以回答：稳定分桶能保证同一用户或同一租户在一段时间内稳定进入同一模型版本，方便比较质量、延迟和安全指标，也避免用户在新旧模型之间来回跳导致体验不一致。

30.21 小练习

1. 模型路由为什么不是普通负载均衡？
2. Model Registry 中需要记录哪些能力信息？
3. 成本路由如何在质量和成本之间平衡？
4. 降级路由有哪些常见路径？
5. 灰度路由应该按哪些维度分流？
6. Router 应该接入哪些实时指标？
7. route trace 应该记录哪些字段？
8. 为什么 Router 配置必须可版本化和可回滚？
9. 写出一个路由 utility score，并解释每个权重如何影响质量、成本和延迟。
10. 为一个多模型平台设计 fallback chain 的时间预算和成本预算。
11. 用稳定哈希设计一个 10% 灰度路由规则，并说明为什么不能每次随机。
12. 设计一个 route trace schema，要求能复盘模型版本、降级、fallback、成本和延迟。

30.22 本章小结

本章讲了模型路由。

你需要记住：

1. 模型路由不是简单负载均衡，而是模型、版本、endpoint 和策略的综合决策。
2. 路由依据包括能力、质量、成本、延迟、安全、租户、灰度和实时负载。
3. 成本路由的目标是在满足质量和 SLO 的前提下降低单位成本。
4. 降级和 fallback 可以提升可用性，但必须可控、可观测、可审计。
5. Router 配置必须版本化、可回滚，并记录完整 route trace。
6. 多模型平台的核心能力之一，就是把请求稳定地送到“最合适”而不是“最强”或“最便宜”的模型。
7. 模型路由门禁要同时检查请求画像、能力画像、权限过滤、成本预算、延迟 SLO、质量安全、fallback、灰度、实时健康、trace、降级策略和配置版本。

下一章我们会讲缓存体系：prompt cache、KV cache、语义缓存和结果缓存。

第 31 章 缓存体系：prompt cache、KV cache、语义缓存和结果缓存

上一章讲了模型路由。本章讲推理平台中另一个非常关键的能力：缓存体系。

大模型推理成本高、延迟高，很多请求又存在重复、相似或共享前缀。如果完全每次从头算，GPU 成本会迅速失控。缓存的目标就是：在不明显损害正确性和安全性的前提下，减少重复计算、降低延迟和成本。

先记住一句话：

推理缓存不是一个缓存，而是一组位于不同层次、解决不同问题的缓存机制。

31.0 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做过资料校准。重点参考的是 OpenAI Prompt Caching 对长 prompt 前缀复用的公开说明，vLLM Automatic Prefix Caching 对 KV block hash、prefix reuse 和 cache hit 的工程口径，TensorRT-LLM 对 paged KV cache、KV cache reuse、retention 和 eviction 的公开说明，以及 RedisVL SemanticCache 对 embedding 相似检索、阈值、TTL 和过滤条件的工程抽象。

这些资料共同指向一个稳定事实：大模型推理缓存不是单个 Redis key-value 缓存，而是跨平台层、runtime 层和业务层的一组缓存策略。prompt / prefix cache 主要减少重复 prefill，KV cache 主要管理生成过程中的 attention 状态，语义缓存用相似检索减少近似重复请求，结果缓存则复用完整响应。四者的 key、生命周期、隔离风险和收益指标都不同。

本章只抽象截至 2026-06 仍稳定的缓存体系设计口径，不把某个云服务的具体折扣、最小 token 门槛、框架默认 block size、向量库参数或缓存字段名写成通用标准。正文公式用于面试表达、容量估算和策略审计；真实上线仍要用目标模型、tokenizer、runtime、请求分布、租户权限、知识库版本、采样参数和安全策略实测校准。

31.1 为什么大模型推理需要缓存

推理平台里的缓存主要解决四类问题：

1. 降低 TTFT。
2. 降低 TPOT。
3. 降低 GPU 计算成本。
4. 降低重复请求的端到端延迟。

常见重复来源包括：

1. 相同系统 prompt。
2. 相同文档前缀。
3. 相同 RAG 检索结果。
4. 相似用户问题。
5. 热门 FAQ。
6. 重试请求。
7. 多 Agent 复用同一上下文。

没有缓存，平台会反复为相同或相似内容付费。

31.2 四类常见缓存

大模型推理平台常见缓存包括：

1. prompt cache。
2. KV cache。
3. 语义缓存。
4. 结果缓存。

它们的位置不同，命中条件不同，风险也不同。

缓存类型	缓存内容	主要收益	主要风险
prompt cache	prompt 前缀或 tokenized prompt	降低 prefill 成本	前缀匹配和权限隔离
KV cache	attention 的 Key/Value	加速 decode 和上下文复用	显存占用和隔离风险
语义缓存	相似问题到答案或中间结果	减少近似重复请求	误命中导致答非所问
结果缓存	完整请求到完整结果	最快返回	过期、个性化和安全风险

31.3 Prompt Cache 是什么

Prompt cache 缓存的是 prompt 的可复用部分。

例如很多请求都有相同系统提示词：

你是一个企业客服助手，请遵守以下规则...

或者 RAG 场景中多个请求共享相同文档前缀：

以下是产品手册内容：...

请基于手册回答用户问题：...

如果前缀相同，runtime 或平台可以复用前缀处理结果，从而减少 prefill 开销。

Prompt cache 主要改善 TTFT。

31.4 Prompt Cache 的命中条件

Prompt cache 通常要求：

1. prompt 前缀完全一致，或满足某种规范化后一致。
2. tokenizer 一致。
3. 模型版本一致。
4. 推理配置兼容。
5. 权限边界一致。
6. 安全策略一致。

模型版本不同，KV 或 prompt 处理结果通常不能直接复用。

租户不同，也不能随便复用，因为 prompt 里可能包含私有信息。

31.5 Prefix Cache

Prefix cache 是 prompt cache 的常见形式，缓存共享前缀。

适合场景：

1. 固定 system prompt。
2. 固定工具说明。
3. 固定长文档。
4. 固定代码上下文。
5. 多轮对话共享历史。

例如两个请求：

prefix: 公司产品手册全文

query A: 这个产品如何退款？

query B: 这个产品支持哪些地区？

如果产品手册部分相同，就可以复用前缀计算。

Prefix cache 对长上下文尤其有价值。

31.6 KV Cache 回顾

KV cache 是 Transformer attention 中 Key 和 Value 的缓存。

在生成过程中，历史 token 的 Key 和 Value 不需要每一步重复计算，所以缓存起来。

KV cache 的主要作用是：

1. 加速 decode。
2. 支持长上下文。
3. 支持 continuous batching。
4. 支持 prefix 复用。

但它也会带来显存压力。

KV cache 更接近 runtime 内部缓存，而不是传统意义上的业务缓存。

31.7 KV Cache 的生命周期

一个请求的 KV cache 生命周期通常是：

请求进入 -> prefill 生成 KV cache -> decode 持续追加 -> 请求结束 -> 释放 KV cache

如果启用 prefix cache，部分 KV cache 可能在请求结束后继续保留，供后续请求复用。

生命周期管理要处理：

1. 正常结束。
2. 超时结束。
3. 用户取消。
4. 客户端断开。
5. runtime 异常。
6. 显存水位过高。
7. cache 驱逐。

如果释放不及时，会造成显存泄漏。

31.8 语义缓存是什么

语义缓存不是要求请求完全相同，而是判断问题语义相似。

例如：

问题 A：怎么申请退款？

问题 B：我想退货退款，流程是什么？

这两个问题字面不同，但语义接近。语义缓存可能直接返回已有答案，或者复用中间结果。

实现方式通常是：

1. 对用户问题做 embedding。
2. 在向量索引中检索相似历史请求。
3. 如果相似度超过阈值，返回缓存答案或候选答案。
4. 如果不够相似，走正常模型推理。

31.9 语义缓存的收益和风险

语义缓存的收益：

1. 降低重复问题成本。
2. 显著降低延迟。
3. 对 FAQ 类场景很有效。
4. 可以减少大模型调用次数。

语义缓存的风险：

1. 相似但不等价。
2. 答案过期。
3. 用户上下文不同。
4. 权限不同。
5. 个性化需求不同。
6. 安全策略不同。

例如“我的订单能退吗”和“退款规则是什么”很相似，但前者可能需要用户订单信息，不能直接复用通用答案。

31.10 结果缓存是什么

结果缓存缓存完整请求到完整输出。

例如：

```
cache_key = hash(model + version + prompt + parameters)
cache_value = generated_answer
```

如果下次完全相同的请求到来，可以直接返回结果。

结果缓存适合：

1. 确定性生成。
2. FAQ。
3. 后台批处理。
4. 重试请求。
5. 固定 prompt 的分类或抽取任务。

如果 temperature 较高、输出本身要求多样性，结果缓存就不一定适合。

31.11 缓存 Key 如何设计

缓存 key 设计非常关键。

常见字段包括：

1. 模型名称。
2. 模型版本。
3. tokenizer 版本。
4. prompt 或 prompt hash。
5. system prompt hash。
6. 参数配置。
7. temperature。
8. top_p。
9. max_tokens。
10. stop sequence。
11. 租户 ID。
12. 权限域。
13. 安全策略版本。
14. 数据版本。

不要只用 prompt 文本做 key。

否则模型版本、参数、权限或安全策略变化后，可能错误命中旧结果。

31.12 缓存粒度

缓存粒度可以从粗到细：

1. 完整响应缓存。
2. 中间检索结果缓存。
3. prompt 前缀缓存。
4. tokenizer 结果缓存。
5. KV cache block 缓存。

粒度越粗，命中后收益越大，但命中条件更严格，错误风险也更高。

粒度越细，复用机会更多，但管理复杂度更高。

平台通常会组合多层缓存，而不是只依赖一种。

31.13 缓存与安全隔离

缓存最容易被忽视的是安全隔离。

必须避免：

1. A 租户命中 B 租户缓存。
2. 普通用户命中管理员上下文缓存。
3. 低权限用户拿到高权限检索结果。
4. 私有文档前缀被跨用户复用。
5. 敏感输出被结果缓存长期保存。

安全策略：

1. cache key 包含租户和权限域。
2. 私有数据默认不跨租户复用。
3. 敏感数据设置短 TTL 或不缓存。
4. 缓存内容加密存储。
5. 访问缓存也要审计。
6. 明确 cache purge 机制。

缓存是性能优化，但不能破坏数据边界。

31.14 缓存与一致性

缓存可能返回过期答案。

常见过期来源：

1. 模型版本升级。
2. prompt 模板升级。
3. 文档知识库更新。
4. 安全策略更新。
5. 业务规则更新。
6. 用户权限变化。

应对方式：

1. cache key 加版本。
2. 设置 TTL。
3. 主动失效。
4. 按数据版本隔离。
5. 高风险场景禁用结果缓存。
6. 对命中结果做二次校验。

缓存一致性不是数据库一致性，但同样需要治理。

31.15 缓存与随机性

大模型生成可能有随机性。

如果 temperature 大于 0，同一个 prompt 可能生成不同答案。

这时结果缓存要谨慎。

可选策略：

1. 只缓存 temperature 为 0 的请求。
2. cache key 包含 sampling 参数。
3. 对创意生成禁用结果缓存。
4. 对分类、抽取、改写等任务启用缓存。
5. 对缓存命中返回标记。

缓存适合确定性或近似确定性任务，不适合所有生成任务。

31.16 缓存与流式输出

流式输出下，结果缓存有两种方式：

1. 缓存完整结果，下次模拟 streaming 分片返回。
2. 不缓存完整结果，只缓存前缀或 KV。

第一种对用户体验更一致，但要注意：

1. chunk 节奏是否需要模拟。
2. 是否暴露缓存命中。
3. 客户端取消时如何计费。
4. trace 如何记录。

流式场景中，prompt cache 和 KV cache 通常比完整结果缓存更常用。

31.17 缓存驱逐策略

缓存空间有限，需要驱逐。

常见策略：

1. LRU。
2. LFU。
3. TTL。
4. 按成本收益驱逐。
5. 按租户配额驱逐。
6. 按安全等级驱逐。
7. 按模型版本驱逐。

KV cache 驱逐尤其复杂，因为它占的是 GPU 显存，资源昂贵且变化快。

结果缓存和语义缓存通常可以放在内存、Redis、向量库或对象存储中，驱逐策略相对灵活。

31.18 缓存指标

缓存体系至少要观察：

1. hit rate。
2. miss rate。
3. cache latency。
4. saved tokens。
5. saved GPU time。
6. cache memory usage。

7. KV cache usage。
8. eviction count。
9. stale hit count。
10. semantic false hit rate。
11. tenant-level hit rate。
12. cost saved。

只看 hit rate 不够。

一个缓存命中率很高，但命中的都是低成本短请求，实际收益可能不大。

更应该关注节省了 token、GPU 时间和成本。

31.19 缓存策略如何落地

一个实际平台可以这样设计缓存策略：

1. 对 system prompt 和工具说明启用 prefix cache。
2. 对固定知识库版本启用 prompt cache。
3. 对 FAQ 场景启用语义缓存。
4. 对 temperature 为 0 的分类和抽取任务启用结果缓存。
5. 对私有数据默认只在租户内缓存。
6. 对高风险请求禁用跨请求缓存。
7. 对所有缓存命中记录 trace。
8. 定期评估缓存误命中和成本收益。

缓存策略应该按场景开启，而不是全局一刀切。

31.20 缓存体系审计指标与最小 demo

第二轮精修时，需要把缓存体系从“哪些地方可以缓存”升级成“哪些缓存可以安全命中、命中后节省多少、错误命中如何阻断、最终能否过门禁”。

可以把一次缓存查找写成：

$c_i = (r_i, m_i, v_i, t_i, p_i, d_i, u_i, a_i, s_i, \tau_i, z_i)$

其中， r_i 是请求内容或请求 hash， m_i 是模型名， v_i 是模型版本， t_i 是 tokenizer 版本， p_i 是生成参数， d_i 是数据或知识库版本， u_i 是租户， a_i 是权限域， s_i 是安全策略版本， τ_i 是 TTL / 时间戳信息， z_i 是 trace 和审计字段。

缓存命中率可以写成：

$H_k = \frac{N_{\{\mathrm{hit}\}, k}}{N_{\{\mathrm{lookup}\}, k}}$

其中， k 表示缓存层，例如 result、semantic、prefix 或 KV。只看 H_k 不够，因为一个高命中率的短请求缓存可能节省不了多少成本。

更有意义的是 token 节省量：

$S_{\{\mathrm{tok}\}} = \sum_{i=1}^N \mathbf{1}[h_i=1] (n_i^{\{\mathrm{save}\}} + o_i^{\{\mathrm{save}\}})$

其中， h_i 表示第 i 次查找是否命中， $n_i^{\{\mathrm{save}\}}$ 是节省的输入 token 或 prefill token， $o_i^{\{\mathrm{save}\}}$ 是节省的输出 token。prefix cache 主要节省输入 token 的 prefill，结果缓存和安全的语义缓存可能同时节省输入和输出。

成本收益可以写成：

$K_{\{\mathrm{save}\}} = \sum_i \frac{n_i^{\{\mathrm{save}\}} p_{\{\mathrm{in}\}} + o_i^{\{\mathrm{save}\}} p_{\{\mathrm{out}\}}}{1000} - K_{\{\mathrm{cache}\}}$

其中， $p_{\{\mathrm{in}\}}$ 和 $p_{\{\mathrm{out}\}}$ 是每千 input / output token 成本， $K_{\{\mathrm{cache}\}}$ 是缓存存储、向量检索、网络、反序列化和运维成本。这个公式强调：缓存收益要扣掉缓存系统自身成本。

prefix cache 的 prefill 时间收益可以粗略写成：

$$T_{\{\mathrm{prefill}, \mathrm{save}\}} = \frac{N_{\{\mathrm{prefix}, \mathrm{hit}\}}}{Q_{\{\mathrm{prefill}\}}}$$

其中， $N_{\{\mathrm{prefix}, \mathrm{hit}\}}$ 是命中的前缀 token 数， $Q_{\{\mathrm{prefill}\}}$ 是 prefill tokens/s。真实系统还要考虑 batch、硬件、KV block 布局、cache read latency 和调度等待。

语义缓存需要单独监控误命中率：

$$R_{\{\mathrm{false}\}} = \frac{N_{\{\mathrm{false_semantic_hit}\}}}{N_{\{\mathrm{semantic_hit}\}}}$$

其中， $N_{\{\mathrm{false_semantic_hit}\}}$ 是相似但不等价、权限不一致、上下文不同或知识过期的语义命中数。语义缓存的关键不是“相似度越高越好”，而是相似度、权限、数据版本、任务类型和风险等级共同过线。

过期命中率可以写成：

$$R_{\{\mathrm{stale}\}} = \frac{N_{\{\mathrm{stale_hit}\}}}{N_{\{\mathrm{hit}\}}}$$

租户或权限隔离违规率可以写成：

$$R_{\{\mathrm{iso}\}} = \frac{N_{\{\mathrm{cross_domain_hit}\}}}{N_{\{\mathrm{hit}\}}}$$

这两个指标通常应该接近 0。缓存事故里，过期答案和跨权限命中比低命中率更危险。

最终缓存门禁可以写成：

$$G_{\{\mathrm{cache}\}} = \mathbf{1} \setminus \left[\min_j C_j \geq \tau_j \ \wedge \ R_{\{\mathrm{false}\}} \leq \rho_{\{\mathrm{false}\}} \ \wedge \ R_{\{\mathrm{stale}\}} \leq \rho_{\{\mathrm{stale}\}} \right]$$

其中， C_j 是各审计维度覆盖率， P_0 是未关闭的 P0 风险数。缓存门禁强调：缓存必须同时证明收益、正确性、隔离、一致性和可审计。

下面这个 0 依赖 Python demo 演示一个简化缓存审计器：先查 result cache，再查 prefix cache，最后查 semantic cache；同时阻断跨租户语义命中、版本过期结果命中和非确定性生成的结果缓存。

```
from math import sqrt
```

```
CHECKS = [
```

```
    "request_cache_profile",
    "cache_layer_boundary",
    "key_version_fingerprint",
    "prompt_prefix_reuse",
    "kv_lifecycle_management",
    "semantic_similarity_guard",
    "result_cache_determinism",
    "tenant_permission_isolation",
    "ttl_staleness_control",
    "eviction_quota_policy",
    "streaming_cache_policy",
    "cache_observability_metrics",
    "cost_latency_savings",
    "cache_governance_gate",
```

```
]
```

```
POLICY = {
```

```
    "now_min": 1000,
    "semantic_threshold": 0.92,
    "prefill_tokens_per_second": 12000,
    "input_cost_per_1k": 0.6,
    "output_cost_per_1k": 1.8,
    "cache_read_ms": {"result": 12, "prefix": 5, "semantic": 18},
```

```
}
```

```
RESULT_CACHE = [
```

```

{
  "prompt_hash": "faq_refund_v1",
  "model": "chat-large",
  "model_version": "v3",
  "tokenizer": "tok-a",
  "params_hash": "temp0_top1_max80",
  "tenant": "public",
  "permission_domain": "faq_public",
  "safety_policy": "safe-2026-06",
  "data_version": "kb-42",
  "created_min": 980,
  "ttl_min": 60,
  "answer_tokens": 76,
},
{
  "prompt_hash": "policy_shipping_v1",
  "model": "chat-large",
  "model_version": "v3",
  "tokenizer": "tok-a",
  "params_hash": "temp0_top1_max120",
  "tenant": "public",
  "permission_domain": "faq_public",
  "safety_policy": "safe-2026-06",
  "data_version": "kb-41",
  "created_min": 950,
  "ttl_min": 30,
  "answer_tokens": 90,
},
]

PREFIX_CACHE = [
  {
    "prefix_hash": "handbook_v7_prefix",
    "model": "chat-large",
    "model_version": "v3",
    "tokenizer": "tok-a",
    "tenant": "enterprise-a",
    "permission_domain": "handbook_reader",
    "safety_policy": "safe-2026-06",
    "data_version": "doc-v7",
    "cached_tokens": 1600,
    "created_min": 990,
    "ttl_min": 120,
  }
]

SEMANTIC_CACHE = [
  {
    "query_id": "refund_process",
    "embedding": [1.0, 0.0, 0.0],
    "tenant": "public",
    "permission_domain": "faq_public",
    "data_version": "kb-42",
    "created_min": 970,
    "ttl_min": 90,
  }
]

```

```

        "saved_input_tokens": 140,
        "saved_output_tokens": 70,
    },
    {
        "query_id": "enterprise_salary_policy",
        "embedding": [0.0, 1.0, 0.0],
        "tenant": "enterprise-a",
        "permission_domain": "hr_private",
        "data_version": "hr-9",
        "created_min": 990,
        "ttl_min": 45,
        "saved_input_tokens": 900,
        "saved_output_tokens": 120,
    },
]

```

```

REQUESTS = [
    {
        "id": "exact_faq_retry",
        "prompt_hash": "faq_refund_v1",
        "prefix_hash": None,
        "embedding": [0.97, 0.03, 0.0],
        "model": "chat-large",
        "model_version": "v3",
        "tokenizer": "tok-a",
        "params_hash": "temp0_top1_max80",
        "temperature": 0.0,
        "tenant": "public",
        "permission_domain": "faq_public",
        "safety_policy": "safe-2026-06",
        "data_version": "kb-42",
        "input_tokens": 180,
        "expected_output_tokens": 80,
        "stream": False,
        "sensitive": False,
    },
    {
        "id": "shared_doc_query",
        "prompt_hash": "handbook_query_a",
        "prefix_hash": "handbook_v7_prefix",
        "embedding": [0.20, 0.10, 0.97],
        "model": "chat-large",
        "model_version": "v3",
        "tokenizer": "tok-a",
        "params_hash": "temp0_top1_max200",
        "temperature": 0.0,
        "tenant": "enterprise-a",
        "permission_domain": "handbook_reader",
        "safety_policy": "safe-2026-06",
        "data_version": "doc-v7",
        "input_tokens": 2100,
        "expected_output_tokens": 160,
        "stream": True,
        "sensitive": False,
    },
]

```



```

{
  "id": "semantic_faq",
  "prompt_hash": "refund_wording_new",
  "prefix_hash": None,
  "embedding": [0.96, 0.05, 0.0],
  "model": "chat-large",
  "model_version": "v3",
  "tokenizer": "tok-a",
  "params_hash": "temp0_top1_max80",
  "temperature": 0.0,
  "tenant": "public",
  "permission_domain": "faq_public",
  "safety_policy": "safe-2026-06",
  "data_version": "kb-42",
  "input_tokens": 130,
  "expected_output_tokens": 70,
  "stream": False,
  "sensitive": False,
},
{
  "id": "private_cross_tenant",
  "prompt_hash": "salary_question",
  "prefix_hash": None,
  "embedding": [0.01, 0.99, 0.0],
  "model": "chat-large",
  "model_version": "v3",
  "tokenizer": "tok-a",
  "params_hash": "temp0_top1_max120",
  "temperature": 0.0,
  "tenant": "enterprise-b",
  "permission_domain": "hr_private",
  "safety_policy": "safe-2026-06",
  "data_version": "hr-9",
  "input_tokens": 760,
  "expected_output_tokens": 110,
  "stream": False,
  "sensitive": True,
},
{
  "id": "stale_policy",
  "prompt_hash": "policy_shipping_v1",
  "prefix_hash": None,
  "embedding": [0.1, 0.2, 0.9],
  "model": "chat-large",
  "model_version": "v3",
  "tokenizer": "tok-a",
  "params_hash": "temp0_top1_max120",
  "temperature": 0.0,
  "tenant": "public",
  "permission_domain": "faq_public",
  "safety_policy": "safe-2026-06",
  "data_version": "kb-42",
  "input_tokens": 220,
  "expected_output_tokens": 95,
  "stream": False,
}

```

```

        "sensitive": False,
    },
    {
        "id": "creative_generation",
        "prompt_hash": "creative_copy_variant",
        "prefix_hash": None,
        "embedding": [0.2, 0.2, 0.8],
        "model": "chat-large",
        "model_version": "v3",
        "tokenizer": "tok-a",
        "params_hash": "temp08_top09_max200",
        "temperature": 0.8,
        "tenant": "public",
        "permission_domain": "marketing_public",
        "safety_policy": "safe-2026-06",
        "data_version": "kb-42",
        "input_tokens": 240,
        "expected_output_tokens": 160,
        "stream": False,
        "sensitive": False,
    },
]

def cosine(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    na = sqrt(sum(x * x for x in a))
    nb = sqrt(sum(y * y for y in b))
    return dot / max(na * nb, 1e-9)

def is_fresh(entry):
    return POLICY["now_min"] - entry["created_min"] <= entry["ttl_min"]

def result_key_match(req, entry):
    fields = [
        "prompt_hash",
        "model",
        "model_version",
        "tokenizer",
        "params_hash",
        "tenant",
        "permission_domain",
        "safety_policy",
        "data_version",
    ]
    return all(req[field] == entry[field] for field in fields)

def stale_result_candidate(req, entry):
    return (
        req["prompt_hash"] == entry["prompt_hash"]
        and req["model"] == entry["model"]
        and req["model_version"] == entry["model_version"]
    )

```

```

        and req["tenant"] == entry["tenant"]
        and req["permission_domain"] == entry["permission_domain"]
        and req["data_version"] != entry["data_version"]
    )

def prefix_match(req, entry):
    fields = [
        "prefix_hash",
        "model",
        "model_version",
        "tokenizer",
        "tenant",
        "permission_domain",
        "safety_policy",
        "data_version",
    ]
    return req["prefix_hash"] and all(req[field] == entry[field] for field in fields)

def saved_cost(input_tokens, output_tokens):
    return (
        input_tokens * POLICY["input_cost_per_1k"]
        + output_tokens * POLICY["output_cost_per_1k"]
    ) / 1000.0

def evaluate_cache(req):
    trace = {
        "request_id": req["id"],
        "tenant": req["tenant"],
        "permission_domain": req["permission_domain"],
        "model_version": req["model_version"],
        "data_version": req["data_version"],
        "decision": None,
        "layer": None,
        "saved_input_tokens": 0,
        "saved_output_tokens": 0,
        "cache_latency_ms": 0,
        "blocked_reason": None,
    }

    if req["temperature"] == 0.0:
        for entry in RESULT_CACHE:
            if result_key_match(req, entry):
                if is_fresh(entry):
                    trace.update({
                        "decision": "result_cache_hit",
                        "layer": "result",
                        "saved_input_tokens": req["input_tokens"],
                        "saved_output_tokens": min(req["expected_output_tokens"], entry["answer_tokens"]),
                        "cache_latency_ms": POLICY["cache_read_ms"]["result"],
                    })
                return trace
            trace.update({"decision": "stale_result_blocked", "layer": "result", "blocked_reason": "ttl_expired"})

```

```

        return trace
    if stale_result_candidate(req, entry):
        trace.update({"decision": "stale_result_blocked", "layer": "result", "blocked_reason": "data_version_mismatch"})
        return trace
    else:
        trace.update({"blocked_reason": "non_deterministic_sampling"})

    for entry in PREFIX_CACHE:
        if prefix_match(req, entry):
            if is_fresh(entry):
                trace.update({
                    "decision": "prefix_cache_hit",
                    "layer": "prefix",
                    "saved_input_tokens": min(req["input_tokens"], entry["cached_tokens"]),
                    "cache_latency_ms": POLICY["cache_read_ms"]["prefix"],
                })
                return trace
            trace.update({"decision": "stale_prefix_blocked", "layer": "prefix", "blocked_reason": "ttl_expired"})
            return trace

    best = None
    for entry in SEMANTIC_CACHE:
        sim = cosine(req["embedding"], entry["embedding"])
        if sim >= POLICY["semantic_threshold"]:
            if req["tenant"] != entry["tenant"] or req["permission_domain"] != entry["permission_domain"]:
                trace.update({
                    "decision": "semantic_isolation_blocked",
                    "layer": "semantic",
                    "blocked_reason": "tenant_or_permission_mismatch",
                })
                return trace
            if req["data_version"] != entry["data_version"] or not is_fresh(entry):
                trace.update({"decision": "semantic_stale_blocked", "layer": "semantic", "blocked_reason": "stale_semantic_cache"})
                return trace
            if best is None or sim > best[0]:
                best = (sim, entry)
    if best is not None and not req["sensitive"]:
        _, entry = best
        trace.update({
            "decision": "semantic_cache_hit",
            "layer": "semantic",
            "saved_input_tokens": min(req["input_tokens"], entry["saved_input_tokens"]),
            "saved_output_tokens": min(req["expected_output_tokens"], entry["saved_output_tokens"]),
            "cache_latency_ms": POLICY["cache_read_ms"]["semantic"],
        })
        return trace

    trace["decision"] = "cache_miss"
    trace["layer"] = "none"
    return trace

```

```

def build_profile_cases():
    complete = {"name": "complete_cache_case"}
    complete.update({check: True for check in CHECKS})

```

```

cases = [complete]
bad_names = [
    "request_cache_profile_missing_bad",
    "cache_layer_blurred_bad",
    "key_version_missing_bad",
    "prompt_prefix_disabled_bad",
    "kv_lifecycle_leak_bad",
    "semantic_threshold_missing_bad",
    "result_cache_random_bad",
    "tenant_isolation_missing_bad",
    "ttl_staleness_missing_bad",
    "eviction_quota_missing_bad",
    "streaming_policy_missing_bad",
    "cache_metrics_missing_bad",
    "savings_model_missing_bad",
    "cache_governance_gate_missing_bad",
]
for bad_name, failed_check in zip(bad_names, CHECKS):
    case = {"name": bad_name}
    case.update({check: True for check in CHECKS})
    case[failed_check] = False
    cases.append(case)
return cases

def audit_profiles(cases):
    metrics = {check: round(sum(1 for case in cases if case.get(check)) / len(cases), 3) for check in CHECKS}
    failed_cases = [case["name"] for case in cases if not all(case.get(check) for check in CHECKS)]
    failed_gates = [check for check, value in metrics.items() if value < 1.0]
    hard_blockers = sum(1 for case in cases if case["name"].endswith("_bad"))
    gate_pass = min(metrics.values()) >= 0.95 and hard_blockers == 0
    return metrics, failed_cases, failed_gates, hard_blockers, gate_pass

traces = [evaluate_cache(req) for req in REQUESTS]
decisions = {trace["request_id"]: trace["decision"] for trace in traces}
layer_hits = {
    "result": sum(1 for trace in traces if trace["layer"] == "result" and trace["decision"].endswith("hit")),
    "prefix": sum(1 for trace in traces if trace["layer"] == "prefix" and trace["decision"].endswith("hit")),
    "semantic": sum(1 for trace in traces if trace["layer"] == "semantic" and trace["decision"].endswith("hit")),
}
saved_input = sum(trace["saved_input_tokens"] for trace in traces)
saved_output = sum(trace["saved_output_tokens"] for trace in traces)
cache_cost_usd = 0.003
net_saved_cost = round(saved_cost(saved_input, saved_output) - cache_cost_usd, 4)
saved_prefill_ms = round(saved_input / POLICY["prefill_tokens_per_second"] * 1000, 1)
required_trace = {
    "request_id",
    "tenant",
    "permission_domain",
    "model_version",
    "data_version",
    "decision",
    "layer",
    "saved_input_tokens",
}

```

```

    "saved_output_tokens",
    "cache_latency_ms",
    "blocked_reason",
}
trace_coverage = sum(1 for trace in traces if required_trace.issubset(trace)) / len(traces)
blocked = [trace["request_id"] for trace in traces if "blocked" in trace["decision"]]
metrics, failed_cases, failed_gates, hard_blockers, gate_pass = audit_profiles(build_profile_cases())

print("cache_decisions=", decisions)
print("cache_layer_hits=", layer_hits)
print("blocked_cache_reuses=", blocked)
print("saved_tokens=", {"input": saved_input, "output": saved_output})
print("saved_prefill_ms=", saved_prefill_ms)
print("net_saved_cost=", net_saved_cost)
print("cache_trace_coverage=", round(trace_coverage, 3))
print("metrics=", metrics)
print("hard_blocker_count=", hard_blockers)
print("failed_cases=", failed_cases)
print("failed_gates=", failed_gates)
print("cache_governance_gate_pass=", gate_pass)

```

输出示例：

```

cache_decisions= {'exact_faq_retry': 'result_cache_hit', 'shared_doc_query': 'prefix_cache_hit', 'semantic_faq': 'semantic_cache_hit'}
cache_layer_hits= {'result': 1, 'prefix': 1, 'semantic': 1}
blocked_cache_reuses= ['private_cross_tenant', 'stale_policy']
saved_tokens= {'input': 1910, 'output': 146}
saved_prefill_ms= 159.2
net_saved_cost= 1.4058
cache_trace_coverage= 1.0
metrics= {'request_cache_profile': 0.933, 'cache_layer_boundary': 0.933, 'key_version_fingerprint': 0.933, 'prompt_prefill_ratio': 0.933}
hard_blocker_count= 14
cache_governance_gate_pass= False

```

这个 demo 的重点不是复刻某个生产缓存系统，而是训练一套面试思维：缓存命中前必须检查模型、版本、tokenizer、参数、租户、权限、安全策略、数据版本和 TTL；语义相似不能绕过权限；随机生成不能随便结果缓存；缓存收益要同时看 token、TTFT、成本和误命中风险。

31.21 常见误区

误区一：缓存命中率越高越好。

高命中率不一定等于高收益，还要看节省的 token 和成本。

误区二：语义相似就可以直接返回。

语义相似不代表答案等价，尤其涉及用户、权限、时间和业务状态时。

误区三：KV cache 和结果缓存是一回事。

KV cache 是 runtime 内部 attention 缓存，结果缓存是业务层响应缓存，完全不是同一层东西。

误区四：缓存只是性能问题。

缓存同时是安全、权限、一致性和审计问题。

31.22 面试常见追问

问题一：prompt cache 和 KV cache 有什么区别？

可以回答：prompt cache 更关注复用 prompt 前缀处理结果，主要降低 prefill 成本；KV cache 是 attention 中 Key/Value 的缓存，主要用于加速 decode 和上下文复用，通常由 runtime 管理。

问题二：语义缓存有什么风险？

可以回答：语义缓存可能把相似但不等价的问题当成同一问题，导致答非所问；还可能因为权限、用户上下文、知识过期或安全策略不同而错误返回。

问题三：缓存 key 应该包含什么？

可以回答：至少包含模型和版本、prompt hash、参数、tokenizer、租户或权限域、安全策略版本和数据版本，避免跨模型、跨权限或跨版本误命中。

问题四：如何评估缓存收益？

可以回答：不能只看命中率，还要看节省的 input/output tokens、GPU 时间、TTFT、端到端延迟、成本，以及误命中和过期命中风险。

问题五：语义缓存上线前要设置哪些门禁？

可以回答：至少要设置相似度阈值、任务类型一致、租户和权限域一致、数据版本一致、TTL 未过期、敏感请求禁用或二次校验、误命中率评估、trace 完整和人工复核样本。

问题六：为什么结果缓存通常要求采样参数进入 key？

可以回答：temperature、top_p、max_tokens、stop sequence 等参数会改变输出分布和截断边界。如果 key 不包含这些参数，同一个 prompt 可能错误命中不同生成配置下的结果。

31.23 小练习

1. Prompt cache、KV cache、语义缓存和结果缓存分别缓存什么？
2. 为什么 prompt cache 通常要求模型版本和 tokenizer 一致？
3. 语义缓存为什么可能误命中？
4. 结果缓存适合哪些任务？
5. 缓存 key 为什么要包含租户和权限域？
6. 缓存如何处理模型版本升级？
7. 为什么只看 cache hit rate 不够？
8. 如何为企业知识库问答设计缓存策略？
9. 写出一个缓存 key schema，要求覆盖模型、参数、租户、权限、安全策略和数据版本。
10. 设计一个语义缓存误命中评估集，至少包含相似但不等价、跨权限、过期知识和个性化上下文四类样本。
11. 用公式估算 prefix cache 节省的 prefill token、TTFT 和成本。
12. 为一个多租户推理平台设计缓存 trace 字段。

31.24 本章小结

本章讲了大模型推理平台的缓存体系。

你需要记住：

1. 推理缓存是一组多层机制，不是单个 Redis 缓存。
2. Prompt cache 主要降低 prefill 成本，KV cache 主要加速 decode 和上下文复用。
3. 语义缓存能降低相似请求成本，但有误命中风险。
4. 结果缓存返回最快，但对版本、参数、权限、一致性和随机性要求最严格。
5. 缓存 key 必须包含模型、版本、参数、权限、安全和数据版本等信息。
6. 缓存必须同时考虑性能、成本、安全、隔离、一致性和审计。
7. 缓存门禁要同时检查请求画像、缓存层边界、key 版本指纹、prefix / KV 生命周期、语义相似阈值、结果缓存确定性、租户隔离、TTL、驱逐、streaming、指标和成本收益。

下一章我们会讲自动扩缩容：QPS、延迟、队列长度和 GPU 利用率。

第 32 章 自动扩缩容：QPS、延迟、队列长度和 GPU 利用率

上一章讲了缓存体系。本章讲推理平台的自动扩缩容。

自动扩缩容看起来像一个很成熟的问题：流量高了扩容，流量低了缩容。但大模型推理和普通 Web 服务不同，不能简单照搬 CPU 利用率或 QPS 驱动的 HPA。

先记住一句话：

大模型推理扩缩容的核心不是“实例数够不够”，而是“在冷启动、显存、队列、token 负载和 SLO 约束下，容量是否及时、稳定、低成本地匹配需求”。

32.0 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做过资料校准。重点参考的是 Kubernetes Horizontal Pod Autoscaling 对按指标比例计算期望副本数的公开算法，KEDA 对事件驱动和队列型指标扩缩容的工程抽象，Triton Inference Server 对 queue、compute、request、batch、GPU 等推理指标的公开口径，KServe / Knative 对并发、请求量和冷启动相关的 serverless 推理服务口径，以及 Google SRE 关于过载保护、容量、错误预算和控制回路稳定性的系统设计原则。

这些资料共同指向一个稳定事实：普通 HPA 的比例缩放思想有价值，但 LLM serving 不能只用 CPU、QPS 或 GPU utilization 做单指标扩缩容。推理 autoscaler 必须同时看 input / output tokens、TTFT、TPOT、queue wait、KV pressure、active sequences、error / timeout、冷启动、warm pool、draining、路由降级、租户配额和成本预算。

本章只抽象截至 2026-06 仍稳定的 LLM-aware autoscaling 设计口径，不把某个云厂商、Kubernetes 插件、serverless 平台、GPU 型号、runtime 指标字段名或内部平台阈值写成通用标准。正文公式用于面试表达、容量估算和策略审计；真实上线仍要用目标模型、runtime、硬件、流量分布、冷启动时间、warm pool 策略和 SLO 实测校准。

32.1 为什么推理服务需要自动扩缩容

推理流量通常有明显波动：

1. 白天和夜晚流量不同。
2. 工作日和周末不同。
3. 活动期间突增。
4. 某个产品功能上线后突增。
5. Agent 批量任务触发突增。
6. 某个租户异常调用导致突增。

如果容量固定，会出现两个问题：

1. 容量不足时延迟升高、排队增加、超时变多。
2. 容量过剩时 GPU 空闲、成本浪费。

自动扩缩容的目标是在服务质量和成本之间平衡。

32.2 普通 HPA 为什么不够

Kubernetes HPA 常见依据是 CPU、内存或自定义指标。

普通 Web 服务里，CPU 利用率常常能反映负载。

但 LLM 推理不一样：

1. GPU 利用率不等于用户体验。
2. QPS 不等于 token 负载。
3. Decode 可能显存带宽受限，而不是算力受限。
4. KV cache 接近满时，GPU 利用率可能还不高。
5. 冷启动时间很长，扩容不是立即生效。

6. 长请求和短请求对资源消耗差异巨大。

所以推理平台需要更贴近 LLM 的扩缩容指标。

32.3 扩缩容目标

推理服务扩缩容通常要同时满足：

1. TTFT SLO。
2. TPOT SLO。
3. p95 / p99 延迟目标。
4. 错误率和超时率目标。
5. 队列等待时间目标。
6. 成本预算。
7. GPU 利用率目标。
8. 多租户配额目标。

这些目标会冲突。

例如，更激进扩容可以降低延迟，但成本上升。更保守扩容可以省钱，但容易排队和超时。

32.4 QPS 指标的局限

QPS 是最常见的流量指标，但对大模型推理不够。

两个 endpoint 都是 50 QPS：

1. Endpoint A 每个请求输入 100 token，输出 50 token。
2. Endpoint B 每个请求输入 8000 token，输出 1000 token。

它们的 QPS 一样，但 B 的资源消耗可能高出几十倍。

所以 QPS 只能作为辅助指标。

更关键的是：

1. input tokens/s。
2. output tokens/s。
3. active sequences。
4. total context tokens。
5. KV cache usage。
6. queue wait time。

32.5 延迟指标

延迟指标直接反映用户体验。

常见指标：

1. TTFT。
2. TPOT。
3. end-to-end latency。
4. p50 latency。
5. p95 latency。
6. p99 latency。

扩容触发可以基于：

1. p95 TTFT 超过阈值。
2. p99 延迟持续升高。
3. TPOT 明显退化。
4. 超时率升高。

但延迟是滞后指标。等 p99 已经升高再扩容，可能已经影响用户。
所以需要结合队列和负载指标提前判断。

32.6 队列长度和等待时间

队列指标是推理扩缩容非常重要的信号。

常见指标：

1. waiting queue length。
2. queue wait time。
3. admission reject count。
4. active requests。
5. pending prefill requests。
6. pending decode requests。

队列增长通常说明当前处理能力跟不上到达速度。

相比延迟，队列是更早的拥塞信号。

例如：

queue length 持续增长 -> TTFT 开始变差 -> timeout rate 上升

扩容系统应该在队列持续增长阶段就行动，而不是等超时爆发。

32.7 GPU 利用率的误区

GPU 利用率常被用于扩缩容，但它不是万能指标。

问题包括：

1. GPU 利用率高不一定坏，可能只是吞吐高。
2. GPU 利用率低不一定轻松，可能卡在显存带宽或 KV cache。
3. GPU 利用率不能反映队列等待。
4. GPU 利用率不能反映 p99 延迟。
5. 不同 runtime 指标口径可能不同。

更合理的方式是把 GPU 利用率和其他指标一起看：

1. GPU 利用率。
2. GPU 显存占用。
3. KV cache 使用率。
4. tokens/s。
5. queue wait time。
6. TTFT / TPOT。

32.8 KV Cache 水位

KV cache 是 LLM 推理扩缩容中特别重要的资源。

如果 KV cache 水位接近上限，系统可能出现：

1. 新请求无法进入。
2. 长上下文请求被拒绝。
3. cache 驱逐增加。
4. OOM 风险上升。
5. p99 延迟抖动。

因此扩容信号应包括：

1. KV cache usage。

2. free KV blocks。
3. eviction rate。
4. OOM count。
5. max active sequences。

对于长上下文业务，KV cache 水位往往比 GPU 利用率更能反映容量压力。

32.9 Token 吞吐指标

Token 吞吐是推理服务容量的重要描述。

常见指标：

1. input tokens/s。
2. output tokens/s。
3. total tokens/s。
4. per-GPU tokens/s。
5. per-model tokens/s。
6. per-tenant tokens/s。

扩容可以基于 tokens/s 接近实例可承载上限。

例如某模型在指定 SLO 下每张 GPU 可稳定处理 3000 output tokens/s，如果当前已经长期接近 2800，并且队列增长，就应该扩容。

注意：最大 tokens/s 不是固定值，会受输入长度、输出长度、batch、上下文长度和 sampling 参数影响。

32.10 冷启动问题

大模型扩容的最大难点之一是冷启动慢。

冷启动包括：

1. 调度 Pod。
2. 拉取镜像。
3. 挂载权重。
4. 加载模型。
5. 初始化 CUDA。
6. 分配显存。
7. 预热 kernel。
8. 注册 endpoint。

这个过程可能从几十秒到数分钟。

所以扩容决策必须提前。

如果等流量打满再扩，新的实例可能来不及承接流量。

32.11 热池和预热

为了解决冷启动，平台常用 warm pool。

Warm pool 是预先启动好的一组可用实例或半可用实例。

常见方式：

1. 常用模型常驻。
2. 低流量时保留最小副本。
3. 预拉取镜像。
4. 本地缓存权重。
5. 提前加载模型但不接流量。
6. 定时预热 kernel。

Warm pool 能降低扩容响应时间，但会增加空闲成本。
这是典型的成本和可用性权衡。

32.12 扩容策略

常见扩容策略：

1. 阈值扩容。
2. 预测扩容。
3. 队列驱动扩容。
4. SLO 驱动扩容。
5. 定时扩容。
6. 手动兜底扩容。

阈值扩容简单，例如 queue wait time 超过 500ms 持续 3 分钟就扩容。

预测扩容根据历史流量提前扩，例如每天上午 9 点流量上涨前先扩容。

SLO 驱动扩容更关注用户体验，但要避免指标滞后。

实际系统通常组合使用。

32.13 缩容策略

缩容比扩容更容易出问题。

如果缩得太快，会导致：

1. 刚释放资源又要扩容。
2. 连接被中断。
3. streaming 请求被影响。
4. KV cache 丢失。
5. 热缓存失效。
6. 延迟抖动。

缩容要考虑：

1. 当前是否有 active requests。
2. 是否有长 streaming 请求。
3. 实例是否处于 draining 状态。
4. 是否是 warm pool 实例。
5. 最近流量是否稳定下降。
6. 是否处于业务高峰前。

缩容通常要比扩容更保守。

32.14 Draining 机制

缩容前应先让实例进入 draining 状态。

Draining 表示：

1. 不再接收新请求。
2. 继续处理已有请求。
3. 等 active requests 结束。
4. 超过最长等待时间后强制终止或迁移。

没有 draining，缩容可能直接中断用户请求。

对 streaming 生成尤其要小心，因为连接可能持续很久。

32.15 多模型扩缩容

多模型平台里，扩缩容更复杂。

问题包括：

1. 每个模型的流量不同。
2. 每个模型的冷启动时间不同。
3. 每个模型需要的 GPU 类型不同。
4. 小模型和大模型的资源利用率不同。
5. 模型之间能否共享节点或实例。
6. 路由策略会影响流量分布。

平台要决定：

1. 给哪个模型扩容？
2. 扩多少副本？
3. 用什么 GPU？
4. 是否从其他模型释放资源？
5. 是否通过路由降级缓解压力？

扩缩容不能和模型路由割裂。

32.16 多租户扩缩容

多租户场景还要考虑配额和优先级。

例如：

1. 高优先级租户 SLO 要优先保证。
2. 低优先级批量任务可以延迟。
3. 某个租户突增不能拖垮全平台。
4. 扩容成本要能归因到租户或业务。

平台可以按：

1. 租户级队列。
2. 租户级 token/s。
3. 租户级 p95 延迟。
4. 租户级预算。
5. 租户级优先级。

做扩缩容和限流决策。

32.17 扩缩容和路由联动

扩缩容和路由应该联动。

当某个模型过载时，可以：

1. 扩容该模型。
2. 将部分流量路由到其他 endpoint。
3. 将低优先级流量降级到小模型。
4. 开启更严格限流。
5. 使用缓存或结果复用。

如果只扩容不改路由，可能来不及。

如果只改路由不扩容，可能把压力转移到另一个模型。

成熟平台会把 autoscaler、router、admission controller 和 cost controller 放在同一个控制闭环里。

32.18 扩缩容控制闭环

一个完整控制闭环：

Metrics -> Autoscaler -> Scheduler -> Deployment -> Router -> Runtime -> Metrics

步骤如下：

1. Metrics 收集流量、延迟、队列、GPU 和 KV cache 指标。
2. Autoscaler 判断是否扩缩容。
3. Scheduler 分配 GPU 节点。
4. Deployment 启动或停止 runtime 实例。
5. Router 更新可用 endpoint。
6. Runtime 承接流量并继续上报指标。

这个闭环要避免振荡。

32.19 如何避免扩缩容振荡

振荡指系统频繁扩容、缩容、再扩容。

常见原因：

1. 指标窗口太短。
2. 阈值太敏感。
3. 冷启动未纳入考虑。
4. 缩容太激进。
5. 流量本身周期性波动。

缓解方法：

1. 使用滑动窗口。
2. 设置 cooldown 时间。
3. 扩容快、缩容慢。
4. 最小副本和最大副本限制。
5. 预测扩容。
6. 缩容前 draining。
7. 对异常指标做平滑。

扩缩容系统本身也是一个控制系统。

32.20 自动扩缩容审计指标与最小 demo

第二轮精修时，需要把自动扩缩容从“看哪个指标扩副本”升级成可解释的控制闭环。

可以把一个观测窗口写成：

$$a_i = (\lambda_i, x_i, y_i, q_i, w_i, t_i, p_i, g_i, k_i, c_i, h_i, u_i, z_i)$$

其中， λ_i 是请求到达率， x_i 是 input tokens/s， y_i 是 output tokens/s， q_i 是队列长度， w_i 是 queue wait， t_i 是 TTFT / TPOT / E2E 延迟， p_i 是 p95 / p99 和 timeout 等尾部风险， g_i 是 GPU 指标， k_i 是 KV cache 水位， c_i 是冷启动和 warm pool 状态， h_i 是租户或优先级， u_i 是成本预算， z_i 是 trace 和配置版本。

普通 HPA 的核心比例思想可以抽象成：

$$N_{\{\mathrm{des}\}} = \left\lceil N_{\{\mathrm{cur}\}} \max_j \frac{M_j}{B_j} \right\rceil$$

其中， $N_{\{\mathrm{cur}\}}$ 是当前副本数， M_j 是第 j 个观测指标， B_j 是该指标的目标阈值。这个公式的价值是简单稳定；它的问题是如果只选 CPU 或 GPU utilization，就会漏掉 LLM 的 token 负载、队列和 KV 压力。

LLM 推理应该先把 QPS 转成 token 负载：

$$\begin{aligned} \Lambda_{\{\mathrm{in}\}} &= \lambda_{\{\mathrm{req}\}} \mathbb{E}[n_{\{\mathrm{in}\}}], \\ \Lambda_{\{\mathrm{out}\}} &= \lambda_{\{\mathrm{req}\}} \mathbb{E}[n_{\{\mathrm{out}\}}] \end{aligned}$$

其中, Λ_{in} 是 input tokens/s, Λ_{out} 是 output tokens/s, n_{in} 和 n_{out} 是单请求输入 / 输出 token 数。两个 endpoint 的 QPS 一样时, Λ_{in} 和 Λ_{out} 可能完全不同。

基于 token 的副本需求可以写成:

$$N_{\mathrm{tok}} = \max \left(\left\lceil \frac{\Lambda_{\mathrm{in}}}{\mu_{\mathrm{in}}} \right\rceil, \left\lceil \frac{\Lambda_{\mathrm{out}}}{\mu_{\mathrm{out}}} \right\rceil \right)$$

其中, μ_{in} 是单副本在目标 SLO 下可承载的 input tokens/s, μ_{out} 是单副本在目标 SLO 下可承载的 output tokens/s。它们不是理论峰值, 而应该来自压测和线上回放。

队列等待可以用 Little's Law 做直觉估算:

$$W_q \approx \frac{L_q}{\lambda_{\mathrm{req}}}$$

其中, L_q 是等待队列长度, W_q 是平均等待时间。线上扩容不应等 timeout 已经爆发, 而应在 L_q 和 W_q 持续增长时提前行动。

冷启动时间可以拆成:

$$T_{\mathrm{ready}} = T_{\mathrm{sched}} + T_{\mathrm{image}} + T_{\mathrm{weight}} + T_{\mathrm{init}} + T_{\mathrm{warm}} + T_{\mathrm{register}}$$

其中, T_{sched} 是调度 GPU 节点时间, T_{image} 是镜像拉取和解包时间, T_{weight} 是权重加载时间, T_{init} 是 runtime / CUDA 初始化时间, T_{warm} 是 kernel 和 cache 预热时间, T_{register} 是 endpoint 注册和路由生效时间。

因此扩容真正生效的容量不是当前副本, 而是:

$$N_{\mathrm{eff}}(t + T_{\mathrm{ready}}) = N_{\mathrm{cur}} + N_{\mathrm{warm}} + N_{\mathrm{cold}}(t + T_{\mathrm{ready}})$$

其中, N_{warm} 是 warm pool 中可以快速接流量的副本, N_{cold} 是冷启动完成后的新增副本。大模型扩容如果不考虑 T_{ready} , 就会在流量尖峰期间一直追不上需求。

缩容要比扩容更保守。一个简单的缩容安全条件可以写成:

$$S_{\mathrm{down}} = \mathbf{1} \left[L_q = 0 \wedge A_{\mathrm{stream}} = 0 \wedge T_{\mathrm{cool}} \geq B_{\mathrm{cool}} \wedge D = 1 \right]$$

其中, A_{stream} 是活跃流式请求数, T_{cool} 是距离上次缩容或扩容的 cooldown 时间, B_{cool} 是 cooldown 阈值, D 表示待缩容实例已经进入 draining 并停止接新请求。没有这个条件, 缩容很容易中断流式连接或造成振荡。

最终 autoscaling 门禁可以写成:

$$G_{\mathrm{autoscale}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \wedge T_{\mathrm{ttft}, p95} \leq B_{\mathrm{ttft}} \wedge T_{\mathrm{p0}} \leq P_0 \right]$$

其中, C_j 是各审计维度覆盖率, R_{kv} 是 KV pressure, N_{need} 是多指标推导出的需求副本数, P_0 是未关闭的 P0 风险数。这个门禁强调: autoscaler 不是单独的 K8s 配置, 而是推理平台容量控制闭环。

下面这个 0 依赖 Python demo 演示一个简化 LLM-aware autoscaler: 分别按 QPS、input tokens/s、output tokens/s、queue wait、TTFT、TPOT、KV pressure 和 GPU utilization 计算建议副本数, 再结合 warm pool、冷启动、预算上限和 draining 安全做动作判断。

```
from math import ceil
```

```
CHECKS = [
    "traffic_token_profile",
    "slo_latency_contract",
    "queue_backlog_signal",
    "gpu_kv_capacity_signal",
```

```

    "cold_start_lead_time",
    "warm_pool_readiness",
    "multi_metric_recommendation",
    "scale_up_down_policy",
    "draining_streaming_safety",
    "router_admission_coupling",
    "tenant_quota_priority",
    "cost_budget_guard",
    "observability_trace_coverage",
    "autoscaling_gate",
]

```

```

CAPACITY = {
    "per_replica_qps": 7.0,
    "per_replica_input_tps": 9000,
    "per_replica_output_tps": 900,
    "queue_wait_target_s": 0.8,
    "ttft_target_s": 1.8,
    "tpot_target_ms": 45,
    "kv_pressure_target": 0.78,
    "gpu_util_target": 0.82,
    "max_replicas": 12,
    "min_replicas": 2,
    "current_replicas": 4,
    "warm_pool_ready": 2,
    "cold_start_s": 180,
    "budget_max_replicas": 10,
    "scale_down_cooldown_s": 300,
}

```

```

WINDOWS = [
    {
        "id": "normal",
        "qps": 18,
        "input_tps": 42000,
        "output_tps": 2600,
        "queue_len": 12,
        "queue_wait_p95_s": 0.55,
        "ttft_p95_s": 1.3,
        "tpot_p95_ms": 32,
        "kv_pressure": 0.52,
        "gpu_util": 0.68,
        "timeout_rate": 0.002,
        "tenant_burst": False,
    },
    {
        "id": "prefill_spike",
        "qps": 24,
        "input_tps": 82000,
        "output_tps": 3300,
        "queue_len": 45,
        "queue_wait_p95_s": 1.35,
        "ttft_p95_s": 2.7,
        "tpot_p95_ms": 38,
        "kv_pressure": 0.64,
    }
]

```



```

        "gpu_util": 0.74,
        "timeout_rate": 0.01,
        "tenant_burst": False,
    },
    {
        "id": "decode_kv_pressure",
        "qps": 21,
        "input_tps": 50000,
        "output_tps": 6100,
        "queue_len": 52,
        "queue_wait_p95_s": 1.1,
        "ttft_p95_s": 1.9,
        "tpot_p95_ms": 68,
        "kv_pressure": 0.88,
        "gpu_util": 0.79,
        "timeout_rate": 0.018,
        "tenant_burst": False,
    },
    {
        "id": "low_traffic_but_long_streams",
        "qps": 8,
        "input_tps": 12000,
        "output_tps": 1300,
        "queue_len": 4,
        "queue_wait_p95_s": 0.30,
        "ttft_p95_s": 1.0,
        "tpot_p95_ms": 30,
        "kv_pressure": 0.42,
        "gpu_util": 0.28,
        "timeout_rate": 0.002,
        "tenant_burst": False,
        "active_streams": 9,
    },
]

```

```

def desired_by_ratio(current, current_value, target_value):
    if target_value <= 0:
        return current
    return ceil(current * current_value / target_value)

```

```

def recommendations(window):
    current = CAPACITY["current_replicas"]
    rec = {
        "qps": ceil(window["qps"] / CAPACITY["per_replica_qps"]),
        "input_tokens": ceil(window["input_tps"] / CAPACITY["per_replica_input_tps"]),
        "output_tokens": ceil(window["output_tps"] / CAPACITY["per_replica_output_tps"]),
        "queue_wait": desired_by_ratio(current, window["queue_wait_p95_s"], CAPACITY["queue_wait_target_s"]),
        "ttft": desired_by_ratio(current, window["ttft_p95_s"], CAPACITY["ttft_target_s"]),
        "tpot": desired_by_ratio(current, window["tpot_p95_ms"], CAPACITY["tpot_target_ms"]),
        "kv": desired_by_ratio(current, window["kv_pressure"], CAPACITY["kv_pressure_target"]),
        "gpu": desired_by_ratio(current, window["gpu_util"], CAPACITY["gpu_util_target"]),
    }
    raw = max(rec.values())

```

```

capped = min(
    max(raw, CAPACITY["min_replicas"]),
    CAPACITY["max_replicas"],
    CAPACITY["budget_max_replicas"],
)
warm_deficit = max(0, capped - current - CAPACITY["warm_pool_ready"])
ready_s = 20 if capped <= current + CAPACITY["warm_pool_ready"] else CAPACITY["cold_start_s"]
if capped > current:
    action = "scale_up"
elif capped < current and window.get("active_streams", 0) == 0 and window["queue_wait_p95_s"] < 0.4:
    action = "scale_down_candidate"
elif capped < current:
    action = "hold_for_draining"
else:
    action = "hold"
return rec, raw, capped, action, warm_deficit, ready_s

def build_profile_cases():
    complete = {"name": "complete_autoscaling_case"}
    complete.update({check: True for check in CHECKS})
    cases = [complete]
    bad_names = [
        "traffic_token_profile_missing_bad",
        "slo_latency_contract_missing_bad",
        "queue_signal_missing_bad",
        "gpu_kv_signal_missing_bad",
        "cold_start_ignored_bad",
        "warm_pool_missing_bad",
        "single_metric_hpa_bad",
        "scale_policy_missing_bad",
        "draining_missing_bad",
        "router_not_coupled_bad",
        "tenant_quota_missing_bad",
        "cost_guard_missing_bad",
        "autoscaling_trace_missing_bad",
        "autoscaling_gate_missing_bad",
    ]
    for bad_name, failed_check in zip(bad_names, CHECKS):
        case = {"name": bad_name}
        case.update({check: True for check in CHECKS})
        case[failed_check] = False
        cases.append(case)
    return cases

def audit_profiles(cases):
    metrics = {check: round(sum(1 for case in cases if case.get(check)) / len(cases), 3) for check in CHECKS}
    failed_cases = [case["name"] for case in cases if not all(case.get(check) for check in CHECKS)]
    failed_gates = [check for check, value in metrics.items() if value < 1.0]
    hard_blockers = sum(1 for case in cases if case["name"].endswith("_bad"))
    gate_pass = min(metrics.values()) >= 0.95 and hard_blockers == 0
    return metrics, failed_cases, failed_gates, hard_blockers, gate_pass

```

```
summary = {}
for window in WINDOWS:
    rec, raw, capped, action, warm_deficit, ready_s = recommendations(window)
    summary[window["id"]] = {
        "max_signal": max(rec, key=rec.get),
        "raw": raw,
        "capped": capped,
        "action": action,
        "warm_deficit": warm_deficit,
        "ready_s": ready_s,
    }

metrics, failed_cases, failed_gates, hard_blockers, gate_pass = audit_profiles(build_profile_cases())
```

```
print("autoscaling_recommendations=", summary)
print("normal_metric_replicas=", recommendations(WINDOWS[0])[0])
print("spike_metric_replicas=", recommendations(WINDOWS[1])[0])
print("kv_metric_replicas=", recommendations(WINDOWS[2])[0])
print("draining_decision=", summary["low_traffic_but_long_streams"]["action"])
print("warm_pool_ready=", CAPACITY["warm_pool_ready"])
print("cold_start_s=", CAPACITY["cold_start_s"])
print("metrics=", metrics)
print("hard_blocker_count=", hard_blockers)
print("failed_cases=", failed_cases)
print("failed_gates=", failed_gates)
print("autoscaling_gate_pass=", gate_pass)
```

输出示例:

```
autoscaling_recommendations= {'normal': {'max_signal': 'input_tokens', 'raw': 5, 'capped': 5, 'action': 'scale_up', 'war
normal_metric_replicas= {'qps': 3, 'input_tokens': 5, 'output_tokens': 3, 'queue_wait': 3, 'ttft': 3, 'tpot': 3, 'kv': 3,
spike_metric_replicas= {'qps': 4, 'input_tokens': 10, 'output_tokens': 4, 'queue_wait': 7, 'ttft': 6, 'tpot': 4, 'kv': 4,
kv_metric_replicas= {'qps': 3, 'input_tokens': 6, 'output_tokens': 7, 'queue_wait': 6, 'ttft': 5, 'tpot': 7, 'kv': 5, 'gpu
draining_decision= hold_for_draining
warm_pool_ready= 2
cold_start_s= 180
metrics= {'traffic_token_profile': 0.933, 'slo_latency_contract': 0.933, 'queue_backlog_signal': 0.933, 'gpu_kv_capacit
hard_blocker_count= 14
autoscaling_gate_pass= False
```

这个 demo 的重点不是给出生产阈值，而是训练一套判断方式：LLM 扩缩容要从 QPS 走向 token 负载，从平均延迟走向 TTFT / TPOT / queue / KV，从副本数走向冷启动和 warm pool，从缩容动作走向 draining 和 streaming 安全。

32.21 常见误区

误区一：GPU 利用率高就扩容。

GPU 利用率高可能是健康高吞吐，也可能是过载。必须结合队列、延迟、KV cache 和错误率。

误区二：QPS 低就可以缩容。

QPS 低但请求很长，token 负载可能仍然高。

误区三：扩容立即生效。

大模型冷启动慢，扩容需要提前量。

误区四：缩容只是减少副本。

缩容要处理 active request、streaming、KV cache、draining 和路由摘除。

误区五：扩缩容只属于 K8s。

K8s 只是执行层，推理平台需要 LLM-aware autoscaler。

32.22 面试常见追问

问题一：为什么 LLM 推理不能只按 QPS 扩容？

可以回答：因为不同请求的输入输出 token 数差异很大，同样 QPS 资源消耗可能相差几十倍。应该结合 input/output tokens/s、队列、TTFT、TPOT、KV cache 和 GPU 指标。

问题二：为什么不能只看 GPU 利用率？

可以回答：GPU 利用率不能反映队列等待、p99 延迟、KV cache 水位和显存带宽瓶颈。decode 阶段可能不是算力瓶颈，因此需要多指标综合判断。

问题三：大模型扩容为什么慢？

可以回答：需要调度 GPU、拉镜像、加载权重、初始化 runtime、分配显存和预热 kernel，模型越大冷启动越慢。

问题四：如何安全缩容？

可以回答：先将实例从路由摘除并进入 draining，不再接新请求，等待已有请求完成，处理 streaming 和超时，再释放实例，同时设置 cooldown 避免振荡。

问题五：为什么 warm pool 是成本和 SLO 的权衡？

可以回答：warm pool 可以把扩容生效时间从模型冷启动时间降到路由接入时间，但会长期占用 GPU、显存或至少占用已加载权重的资源。关键是用历史峰值、SLO、冷启动时间和预算来确定最小热池，而不是无限常驻。

问题六：扩缩容如何和模型路由联动？

可以回答：autoscaler 发现某模型过载时，router 可以临时把低优先级流量降级、转移到健康 endpoint、启用缓存或限流；同时 autoscaler 扩容目标模型。只改路由可能转移故障，只扩容可能来不及，二者要共享指标和策略版本。

32.23 小练习

1. 为什么普通 HPA 不适合直接管理 LLM 推理服务？
2. QPS 和 tokens/s 的区别是什么？
3. TTFT 和队列长度哪个更早反映拥塞？
4. KV cache 水位为什么能作为扩容信号？
5. 大模型冷启动包括哪些步骤？
6. Warm pool 的收益和代价是什么？
7. 缩容为什么要 draining？
8. 如何避免扩缩容振荡？
9. 写出一个多指标 autoscaling 公式，至少包含 input tokens/s、output tokens/s、queue wait、TTFT、TPOT 和 KV pressure。
10. 估算一个模型的 T_{ready} ，把调度、镜像、权重加载、runtime 初始化、kernel 预热和路由注册拆开。
11. 设计一个缩容 draining 状态机，覆盖 active requests、streaming、timeout、router 摘除和 cooldown。
12. 为多租户推理平台设计 autoscaling trace 字段，要求能解释一次扩容是哪个指标触发的。

32.24 本章小结

本章讲了大模型推理平台的自动扩缩容。

你需要记住：

1. LLM 推理扩缩容不能只看 CPU、QPS 或 GPU 利用率。
2. 更关键的指标包括 TTFT、TPOT、队列长度、tokens/s、KV cache 和错误率。

3. 大模型冷启动慢，扩容必须提前，常需要 warm pool 和预测扩容。
4. 缩容要处理 draining、streaming、active requests 和缓存失效。
5. 多模型、多租户场景下，扩缩容必须和路由、配额、限流、成本治理联动。
6. 成熟的推理 autoscaler 是 LLM-aware 的控制闭环，不只是 K8s HPA 配置。
7. Autoscaling 门禁要同时检查 token 画像、SLO、队列、GPU / KV、冷启动、warm pool、多指标推荐、扩缩容策略、draining、路由准入、租户配额、成本和 trace。

下一章我们会讲限流、熔断、重试、超时和降级。

第 33 章 限流、熔断、重试、超时和降级

上一章讲了自动扩缩容。本章继续讲推理平台的稳定性保护机制：限流、熔断、重试、超时和降级。

大模型推理服务很贵，也很容易被长请求、高并发、异常重试和下游拥塞拖垮。稳定性设计的核心不是让每个请求都尽量执行，而是在系统压力过大时保护整体服务质量。

先记住一句话：

推理平台的稳定性不是“永不失败”，而是能控制失败范围、避免故障扩散，并在压力下优先保证关键请求。

33.0 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做过资料校准。重点参考的是 Google SRE 对 overload、criticality、per-customer limits 和 retry budget 的系统设计口径，AWS Builders Library 对 timeout、retry、exponential backoff、jitter、token bucket 和 idempotency 的工程经验，Envoy 对 rate limit、circuit breaking、outlier detection、timeout、retry 和 retry budget 的代理层抽象，gRPC 对重试提交点、退避、重试节流和服务配置的说明，以及 Resilience4j 对 circuit breaker 状态机、滑动窗口、failure rate、slow call rate 和 half-open 探测的公开文档。

这些资料共同指向一个稳定事实：保护机制不能只靠某一个开关。限流解决“谁可以用多少”，admission control 解决“系统现在接不接得住”，熔断解决“环依赖是否继续放大故障”，重试解决“哪些临时失败值得再试”，超时解决“请求最多占用多久”，降级解决“压力下保哪些能力、退哪些能力”。

本章只抽象截至 2026-06 仍稳定的推理平台保护策略，不把某个 API Gateway、service mesh、云厂商、SDK、runtime 或内部平台的字段名写成通用标准。正文公式用于面试表达、策略审计和最小 demo；真实上线仍要用目标模型、租户协议、流量分布、SLO、成本预算、runtime 行为和事故复盘来校准阈值。

33.1 为什么推理平台需要保护机制

大模型推理有几个特点：

1. 单请求成本高。
2. 长请求会占用资源很久。
3. streaming 连接持续时间长。
4. KV cache 和显存是硬约束。
5. 冷启动慢，不能瞬间扩容。
6. 多租户之间会相互影响。
7. 重试可能放大流量。

如果没有保护机制，一个异常业务、一个超长 prompt、一个重试风暴，就可能拖垮整个平台。

33.2 五类保护机制

本章讨论五类机制：

1. 限流：控制进入系统的请求量。
2. 熔断：发现某个依赖或模型异常时临时切断调用。

3. 重试：在可恢复错误下重新尝试。
4. 超时：限制请求最多占用多久。
5. 降级：在资源紧张或故障时使用更低成本或更简单路径。

它们不是孤立的，而是组合使用。

33.3 限流解决什么问题

限流是控制流量进入系统。

目标是：

1. 防止某个租户占满资源。
2. 防止突发流量压垮 runtime。
3. 防止长请求耗尽 KV cache。
4. 保证高优先级请求。
5. 控制成本。

限流不是为了拒绝用户，而是为了避免所有用户一起变慢或失败。

33.4 限流维度

推理平台限流可以按多个维度设计：

1. 租户级。
2. 用户级。
3. API key 级。
4. 模型级。
5. endpoint 级。
6. 任务类型级。
7. token 级。
8. 并发级。
9. 成本预算级。
10. 上下文长度级。

普通服务常按 QPS 限流。

大模型推理还必须按 token 和并发限流。

33.5 QPS 限流为什么不够

两个用户都是 10 QPS：

1. 用户 A 每次输入 100 token，输出 50 token。
2. 用户 B 每次输入 50000 token，输出 2000 token。

如果只看 QPS，它们一样。

但资源消耗完全不同。

所以推理平台需要：

1. request rate limit。
2. input token rate limit。
3. output token rate limit。
4. concurrent request limit。
5. max context length。
6. max output tokens。

这才是 LLM-aware 限流。

33.6 Token Bucket 和 Leaky Bucket

常见限流算法包括 token bucket 和 leaky bucket。

Token bucket: 系统以固定速率生成令牌, 请求消耗令牌。桶里有令牌就允许通过, 没有就拒绝或排队。它允许一定突发。

Leaky bucket: 请求进入桶后以固定速率流出, 更平滑, 但对突发更严格。

大模型平台可以把令牌从“请求数”扩展成“token 数”或“成本额度”。

例如:

tenant_a: 100k input tokens/min, 50k output tokens/min

tenant_b: 10 concurrent requests, max 8k context

这样比单纯 QPS 更符合资源消耗。

33.7 Admission Control 和限流的区别

限流通常基于配额和速率。

Admission control 更关注当前系统是否有能力接收这个请求。

例如:

1. KV cache 是否足够?
2. 队列是否过长?
3. 当前 p99 是否已经超标?
4. endpoint 是否健康?
5. 请求是否过长?

限流回答: “这个租户是否允许这么用?”

Admission control 回答: “系统现在是否接得住?”

两者都需要。

33.8 熔断解决什么问题

熔断用于阻止故障扩散。

如果某个模型 endpoint 持续超时、错误率升高或队列爆炸, router 应该暂时停止向它发送请求。

熔断对象可以是:

1. 某个 runtime 实例。
2. 某个 endpoint。
3. 某个模型版本。
4. 某个下游工具。
5. 某个外部 API。
6. 某个地域。

熔断的目标是快速隔离坏节点或坏依赖。

33.9 熔断状态

经典熔断器有三种状态:

1. Closed: 正常调用。
2. Open: 熔断, 直接拒绝或走 fallback。
3. Half-open: 放少量探测请求, 判断是否恢复。

在推理平台中，Half-open 很重要。

因为模型加载、GPU OOM、runtime 抖动可能是短暂问题，也可能是持续问题。

恢复时不能立刻把全部流量打回去，要逐步探测。

33.10 熔断触发条件

常见触发条件：

1. 错误率超过阈值。
2. 超时率超过阈值。
3. p99 延迟超过阈值。
4. queue length 持续过高。
5. OOM 次数过多。
6. health check 失败。
7. admission reject rate 过高。
8. runtime crash loop。

熔断要避免误伤。

比如某个租户发来大量超长请求导致超时，不能简单认为整个模型坏了，可能应该对该租户限流。

33.11 重试的危险

重试看似能提升成功率，但在大模型推理中很危险。

原因是：

1. 请求成本高。
2. 重试会放大流量。
3. 非幂等输出可能不同。
4. 原请求可能还在执行。
5. streaming 请求重试体验复杂。
6. 多层服务都重试会造成重试风暴。

所以推理平台不能无脑重试。

33.12 什么错误可以重试

适合重试的情况：

1. 连接建立失败。
2. endpoint 临时不可达。
3. 请求尚未进入 runtime 执行。
4. 明确的 transient error。
5. router 选到不健康实例但尚未开始生成。

不适合重试的情况：

1. 请求已经生成部分 token。
2. 用户取消请求。
3. 输入超过限制。
4. 权限失败。
5. 安全策略拒绝。
6. 配额不足。
7. 业务参数错误。

重试前必须知道请求执行到了哪个阶段。

33.13 重试策略

好的重试策略包括：

1. 最大重试次数。
2. 指数退避。
3. jitter 随机抖动。
4. 总超时预算。
5. 幂等键。
6. retry budget。
7. 只对特定错误重试。
8. 不跨越安全和权限边界。

retry budget 很重要。

它限制系统整体用于重试的额外流量，避免重试把平台打垮。

33.14 超时为什么重要

超时限制请求最多占用资源多久。

没有超时会导致：

1. 长请求长期占用 KV cache。
2. streaming 连接不释放。
3. 上游一直等待。
4. 队列持续堆积。
5. 成本不可控。

推理平台必须设计多层超时。

33.15 多层超时

常见超时包括：

1. 网关接收超时。
2. 排队超时。
3. prefill 超时。
4. decode 总超时。
5. 单 token 生成超时。
6. streaming idle 超时。
7. 下游工具调用超时。
8. 客户端总超时。

不同超时要有层次。

例如客户端总超时 60 秒，服务端不能设置 120 秒才返回，否则结果没有意义。

33.16 Timeout Budget

Timeout budget 是把总时间预算分配给各阶段。

例如：

```
total timeout: 30s
gateway + auth: 0.5s
queue wait: 2s
prefill: 5s
decode: 20s
postprocess: 0.5s
buffer: 2s
```

如果队列阶段已经消耗太多预算，后面可能应该拒绝、降级或减少 max_tokens，而不是继续完整生成。Timeout budget 能避免“请求虽然成功但用户已经不需要”的情况。

33.17 降级解决什么问题

降级是在资源不足、依赖异常或成本超预算时，用更低成本、更快或更稳定的路径返回。

降级目标是：

1. 保持核心功能可用。
2. 控制延迟。
3. 控制成本。
4. 避免故障扩散。
5. 给用户一个可接受结果。

降级不是失败，而是有控制的能力退让。

33.18 常见降级方式

常见降级方式：

1. 大模型降到小模型。
2. reasoning 模型降到普通模型。
3. 长上下文截断。
4. 降低最大输出长度。
5. 关闭工具调用。
6. 关闭多轮深度推理。
7. 返回缓存结果。
8. 返回模板化答案。
9. 后台异步处理。
10. 提示用户稍后再试。

降级策略必须按业务场景设计。

对客服 FAQ，降级到缓存可能可接受。

对金融决策，错误降级可能不可接受。

33.19 降级策略要可观测

每次降级都要记录：

1. 请求 ID。
2. 租户。
3. 原计划模型。
4. 降级后模型或路径。
5. 降级原因。
6. 降级耗时。
7. 用户是否可见。
8. 质量反馈。
9. 成本影响。

没有记录，降级会变成隐形质量问题。

33.20 保护机制之间的组合

一个请求可能经历这样的保护链路：

1. API Gateway 做认证和基础限流
2. Quota Service 检查租户 token 配额
3. Admission Controller 检查队列和 KV cache
4. Router 选择健康 endpoint
5. Circuit Breaker 跳过异常实例
6. Timeout Budget 控制各阶段耗时
7. Retry Policy 只对安全错误重试
8. Degradation Policy 在压力过大时降级
9. Metrics 和 Trace 记录全过程

稳定性设计要从入口一直贯穿到 runtime。

33.21 保护策略审计指标与最小 demo

把限流、熔断、重试、超时和降级放到同一章，容易让人误以为它们只是五个配置项。真正上线时，更重要的是把它们变成一条可审计的请求保护链路。

一个请求保护样本可以写成：

$$p_i=(r_i,u_i,m_i,x_i,y_i,c_i,q_i,k_i,e_i,t_i,s_i,a_i,d_i,z_i)$$

其中， r_i 是请求 ID， u_i 是租户或用户， m_i 是目标模型或 endpoint， x_i 是 input token 数， y_i 是计划 output token 数， c_i 是当前并发， q_i 是队列等待， k_i 是 KV pressure， e_i 是错误类型， t_i 是 timeout budget， s_i 是执行阶段， a_i 是 retry attempt， d_i 是降级动作， z_i 是 trace / audit 字段。

Token bucket 可以从请求数扩展成 token 和成本。一个简化的桶余额更新是：

$$B_{\{u,t+1\}}=\min(B_{\{\max\}},B_{\{u,t\}}+\rho_u\Delta t)-w_i$$

其中， $B_{\{u,t\}}$ 是租户 u 在时刻 t 的余额， $B_{\{\max\}}$ 是桶容量， ρ_u 是补充速率， Δt 是时间间隔， w_i 是请求消耗的权重。对 LLM 来说， w_i 不应该只是 1，而可以写成：

$$w_i=\alpha_x x_i+\alpha_y y_i+\alpha_c c_i+\alpha_k k_i$$

其中， α_x 、 α_y 、 α_c 和 α_k 分别控制输入 token、输出 token、并发占用和 KV 压力的权重。这样能解释为什么“同样 1 个请求”可能消耗完全不同的资源。

Admission control 可以写成一个即时门禁：

$$A_i=\mathbf{1}\left[B_{\{\mathrm{req}\}}\geq 1\wedge B_{\{\mathrm{tok}\}}\geq x_i+y_i\wedge C_u<C_{\{u,\max\}}\wedge Q\leq Q_{\{\max\}}\wedge R_{\{\mathrm{kv}\}}\leq R_{\{\mathrm{open}\}}\right]$$

其中， $B_{\{\mathrm{req}\}}$ 是请求数配额余额， $B_{\{\mathrm{tok}\}}$ 是 token 配额余额， C_u 是租户当前并发， Q 是队列水位， $R_{\{\mathrm{kv}\}}$ 是 KV pressure， $T_{\{\mathrm{rem}\}}$ 是剩余 timeout budget。限流偏静态配额，admission 偏动态容量；两者缺一不可。

熔断要基于滑动窗口，而不是单个错误。对 endpoint j ，可以定义：

$$E_j=\frac{N_{\{\mathrm{error}\},j}}{N_{\{\mathrm{total}\},j}},\quad S_j=\frac{N_{\{\mathrm{slow}\},j}}{N_{\{\mathrm{total}\},j}}$$

其中， E_j 是错误率， S_j 是慢调用率。一个简化的熔断触发条件是：

$$O_j=\mathbf{1}\left[E_j\geq\theta_e\vee S_j\geq\theta_s\vee T_{\{j,\mathrm{p99}\}}\geq B_{\{99\}}\vee Q_j\geq Q_{\{\mathrm{open}\}}\vee R_{\{\mathrm{kv}\},j}\geq R_{\{\mathrm{open}\}}\right]$$

其中， $O_j=1$ 表示进入 open 状态。half-open 阶段不应该立刻放全量流量，而应该只允许 $H_{\{\max\}}$ 个探测请求：

$$H_j=\min(H_{\{\max\}},N_{\{\mathrm{probe}\},j})$$

重试要有预算。若初始请求数为 $N_{\{\mathrm{init}\}}$ ，重试请求数为 $N_{\{\mathrm{retry}\}}$ ，一个简单的 retry budget 是：

$$R_{\{\mathrm{retry}\}} = \frac{N_{\{\mathrm{retry}\}}}{N_{\{\mathrm{init}\}} + N_{\{\mathrm{retry}\}}} \leq \beta$$

其中, β 是允许的重试流量占比。指数退避加 jitter 可以写成:

$$D_a = \min(D_{\{\max\}}, D_0 \cdot 2^{a-1}) + \epsilon_a, \epsilon_a \in [0, \epsilon_{\max}]$$

其中, a 是第几次 retry, D_0 是初始退避, $D_{\{\max\}}$ 是最大退避, ϵ_a 是随机抖动。重试还必须满足阶段和幂等条件:

$$G_{\{\mathrm{retry}\}} = \mathbf{1} \left[e_i \in \mathcal{E}_{\{\mathrm{transient}\}} \wedge s_i = \mathrm{not_started} \wedge a_i < a_{\{\max\}} \wedge I_i = 1 \wedge R_{\{\mathrm{streaming}\}} \right]$$

其中, $\mathcal{E}_{\{\mathrm{transient}\}}$ 是可重试错误集合, I_i 表示请求有幂等键或可去重。对已经开始 streaming、已经产生 side effect、权限失败、配额不足或安全拒绝的请求, $G_{\{\mathrm{retry}\}}$ 必须为 0。

Timeout budget 要保证每一层都受总 deadline 约束:

$$T_{\{\mathrm{total}\}} = T_{\{\mathrm{gateway}\}} + T_{\{\mathrm{queue}\}} + T_{\{\mathrm{prefill}\}} + T_{\{\mathrm{decode}\}} + T_{\{\mathrm{stream}\}}$$

剩余预算为:

$$T_{\{\mathrm{rem}\}} = T_{\{\mathrm{deadline}\}} - \sum_h T_h$$

如果 $T_{\{\mathrm{rem}\}}$ 已经不足以完成完整 decode, 就应该拒绝、减少输出长度或降级, 而不是继续让请求占住 KV cache。

降级也要有约束。设可选降级集合为 $\mathcal{D}_i$, 每个降级动作 d 有质量估计 $Q(d)$ 、成本 $K(d)$ 和延迟 $L(d)$, 可以写成:

$$d_i^* = \arg\max_{d \in \mathcal{D}_i} Q(d)$$

并满足:

$$L(d) \leq B_{\{\mathrm{lat}\}} \wedge K(d) \leq B_{\{\mathrm{cost}\}} \wedge P(d) = 1$$

其中, $P(d)$ 表示该业务、租户和风险等级允许这个降级动作。客服 FAQ 可以降级到缓存, 金融决策可能只能拒绝或转人工; 这不是技术偏好, 而是业务风险边界。

最终保护策略门禁可以写成:

$$G_{\{\mathrm{protect}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \wedge R_{\{\mathrm{retry}\}} \leq \beta \wedge R_{\{\mathrm{timeout}\}} \leq \omega \wedge R_{\{\mathrm{open_miss}\}} \leq p_0 \right]$$

其中, C_j 是各审计维度覆盖率, $R_{\{\mathrm{timeout}\}}$ 是超时率, $R_{\{\mathrm{open_miss}\}}$ 是应该熔断却继续放量的比例, p_0 是未关闭的 P0 风险数。这个门禁强调: 保护机制不是单个中间件配置, 而是入口、路由、runtime、缓存、降级、trace 和事故治理共同组成的控制面。

下面这个 0 依赖 Python demo 演示一个简化的推理保护策略审计器: 它会对 toy 请求依次执行租户限流、动态准入、熔断、重试、timeout budget 和降级决策, 并用 bad case 检查保护链路是否完整。

```
from math import ceil
```

```
CHECKS = [
    "request_token_rate_limit",
    "concurrency_quota_control",
    "admission_capacity_guard",
    "circuit_breaker_state_machine",
    "retry_error_classification",
    "retry_budget_bound",
    "idempotency_stage_guard",
    "timeout_budget_allocation",
    "degradation_policy_control",
    "priority_tenant_fairness",
```

```

    "streaming_cancellation_cleanup",
    "observability_trace_coverage",
    "cost_budget_guard",
    "protection_gate",
]

TENANT_LIMITS = {
    "free": {
        "requests_per_min": 3,
        "input_tpm": 4000,
        "output_tpm": 1200,
        "concurrency": 2,
        "context_limit": 4096,
        "cost_budget": 0.30,
        "priority": 0,
        "allow_degrade": True,
    },
    "pro": {
        "requests_per_min": 30,
        "input_tpm": 50000,
        "output_tpm": 12000,
        "concurrency": 8,
        "context_limit": 16000,
        "cost_budget": 12.00,
        "priority": 1,
        "allow_degrade": True,
    },
    "enterprise": {
        "requests_per_min": 200,
        "input_tpm": 240000,
        "output_tpm": 60000,
        "concurrency": 60,
        "context_limit": 64000,
        "cost_budget": 100.00,
        "priority": 2,
        "allow_degrade": False,
    },
},

USAGE = {
    "free": {"requests": 3, "input_tokens": 3500, "output_tokens": 200, "active": 2, "cost": 0.20},
    "pro": {"requests": 8, "input_tokens": 30000, "output_tokens": 6000, "active": 4, "cost": 4.00},
    "enterprise": {"requests": 28, "input_tokens": 70000, "output_tokens": 18000, "active": 20, "cost": 25.00},
}

ENDPOINTS = {
    "large": {"error_rate": 0.03, "slow_rate": 0.05, "p99_s": 2.8, "queue": 18, "kv": 0.68, "state": "closed"},
    "sick_large": {"error_rate": 0.26, "slow_rate": 0.31, "p99_s": 10.5, "queue": 90, "kv": 0.93, "state": "closed"},
    "small": {"error_rate": 0.01, "slow_rate": 0.03, "p99_s": 1.1, "queue": 9, "kv": 0.38, "state": "closed"},
}

POLICY = {
    "queue_limit": 60,
    "kv_limit": 0.88,
    "timeout_min_s": 3.0,

```

```

    "error_open": 0.20,
    "slow_open": 0.25,
    "p99_open_s": 8.0,
    "queue_open": 80,
    "kv_open": 0.90,
    "retry_max_attempts": 2,
    "retry_budget_ratio": 0.10,
    "retry_initial": 1000,
    "retry_used": 96,
    "retry_base_delay_s": 0.2,
    "retry_max_delay_s": 2.0,
    "retry_jitter_s": 0.05,
    "input_cost": 0.0004,
    "output_cost": 0.0012,
}

TRANSIENT_ERRORS = {"connect_reset", "endpoint_unavailable", "gateway_timeout_before_start"}
NON_RETRYABLE_ERRORS = {"auth_failed", "quota_exceeded", "safety_block", "bad_request", "client_cancelled"}

REQUESTS = [
    {
        "id": "healthy_chat",
        "tenant": "pro",
        "endpoint": "large",
        "input_tokens": 800,
        "max_output_tokens": 200,
        "timeout_budget_s": 15.0,
        "elapsed_s": 2.0,
        "stage": "not_started",
        "attempt": 0,
        "idempotent": True,
        "error": None,
    },
    {
        "id": "free_long_context",
        "tenant": "free",
        "endpoint": "large",
        "input_tokens": 9000,
        "max_output_tokens": 1000,
        "timeout_budget_s": 20.0,
        "elapsed_s": 1.0,
        "stage": "not_started",
        "attempt": 0,
        "idempotent": True,
        "error": None,
    },
    {
        "id": "bad_endpoint",
        "tenant": "enterprise",
        "endpoint": "sick_large",
        "input_tokens": 1200,
        "max_output_tokens": 240,
        "timeout_budget_s": 12.0,
        "elapsed_s": 1.5,
        "stage": "not_started",
    },

```

```

    "attempt": 0,
    "idempotent": True,
    "error": None,
    "fallback": "small",
},
{
    "id": "kv_pressure_degrade",
    "tenant": "pro",
    "endpoint": "large",
    "input_tokens": 6000,
    "max_output_tokens": 1000,
    "timeout_budget_s": 12.0,
    "elapsed_s": 7.5,
    "stage": "not_started",
    "attempt": 0,
    "idempotent": True,
    "error": None,
    "observed_kv": 0.92,
},
{
    "id": "transient_connect",
    "tenant": "pro",
    "endpoint": "large",
    "input_tokens": 500,
    "max_output_tokens": 120,
    "timeout_budget_s": 10.0,
    "elapsed_s": 1.0,
    "stage": "not_started",
    "attempt": 0,
    "idempotent": True,
    "error": "connect_reset",
},
{
    "id": "partial_stream_timeout",
    "tenant": "pro",
    "endpoint": "large",
    "input_tokens": 900,
    "max_output_tokens": 400,
    "timeout_budget_s": 12.0,
    "elapsed_s": 9.5,
    "stage": "streaming",
    "attempt": 0,
    "idempotent": True,
    "partial_tokens": 38,
    "error": "gateway_timeout_before_start",
},
{
    "id": "retry_budget_exhausted",
    "tenant": "enterprise",
    "endpoint": "large",
    "input_tokens": 700,
    "max_output_tokens": 160,
    "timeout_budget_s": 10.0,
    "elapsed_s": 1.0,
    "stage": "not_started",

```

```

        "attempt": 0,
        "idempotent": True,
        "error": "endpoint_unavailable",
        "retry_used_override": 112,
    },
]

def estimate_cost(request):
    return round(
        request["input_tokens"] * POLICY["input_cost"]
        + request["max_output_tokens"] * POLICY["output_cost"],
        4,
    )

def endpoint_state(name):
    endpoint = ENDPOINTS[name]
    should_open = (
        endpoint["error_rate"] >= POLICY["error_open"]
        or endpoint["slow_rate"] >= POLICY["slow_open"]
        or endpoint["p99_s"] >= POLICY["p99_open_s"]
        or endpoint["queue"] >= POLICY["queue_open"]
        or endpoint["kv"] >= POLICY["kv_open"]
    )
    return "open" if should_open else endpoint["state"]

def rate_limit_decision(request):
    tenant = request["tenant"]
    limits = TENANT_LIMITS[tenant]
    usage = USAGE[tenant]
    if usage["requests"] + 1 > limits["requests_per_min"]:
        return False, "request_rate_limit"
    if usage["active"] + 1 > limits["concurrency"]:
        return False, "concurrency_limit"
    if request["input_tokens"] > limits["context_limit"]:
        return False, "context_limit"
    if usage["input_tokens"] + request["input_tokens"] > limits["input_tpm"]:
        return False, "input_token_limit"
    if usage["output_tokens"] + request["max_output_tokens"] > limits["output_tpm"]:
        return False, "output_token_limit"
    if usage["cost"] + estimate_cost(request) > limits["cost_budget"]:
        return False, "cost_budget_limit"
    return True, "pass"

def remaining_timeout(request):
    return round(request["timeout_budget_s"] - request["elapsed_s"], 3)

def retry_delay(attempt):
    base = min(POLICY["retry_max_delay_s"], POLICY["retry_base_delay_s"] * (2 ** attempt))
    return round(base + POLICY["retry_jitter_s"], 3)

```



```

def retry_budget_ok(request):
    used = request.get("retry_used_override", POLICY["retry_used"])
    initial = POLICY["retry_initial"]
    retry_ratio = (used + 1) / (initial + used + 1)
    return retry_ratio <= POLICY["retry_budget_ratio"], round(retry_ratio, 3)

def can_retry(request):
    error = request.get("error")
    if error is None:
        return False, "no_error", None
    if error in NON_RETRYABLE_ERRORS:
        return False, "non_retryable_error", None
    if error not in TRANSIENT_ERRORS:
        return False, "unknown_error_class", None
    if request["stage"] != "not_started" or request.get("partial_tokens", 0) > 0:
        return False, "already_started_or_streaming", None
    if request["attempt"] >= POLICY["retry_max_attempts"]:
        return False, "max_attempts", None
    if not request["idempotent"]:
        return False, "missing_idempotency", None
    budget_ok, retry_ratio = retry_budget_ok(request)
    if not budget_ok:
        return False, "retry_budget_exhausted", retry_ratio
    delay = retry_delay(request["attempt"])
    if remaining_timeout(request) < delay + POLICY["timeout_min_s"]:
        return False, "timeout_budget_too_small", retry_ratio
    return True, "retry_scheduled", retry_ratio

def admission_decision(request, endpoint_name):
    endpoint = ENDPOINTS[endpoint_name]
    observed_kv = request.get("observed_kv", endpoint["kv"])
    if endpoint["queue"] > POLICY["queue_limit"] or observed_kv > POLICY["kv_limit"]:
        return False, "capacity_pressure"
    if remaining_timeout(request) < POLICY["timeout_min_s"]:
        return False, "timeout_budget_too_small"
    return True, "pass"

def degrade(request, reason):
    tenant_policy = TENANT_LIMITS[request["tenant"]]
    if not tenant_policy["allow_degrade"]:
        return "reject_" + reason
    if reason == "capacity_pressure":
        reduced_output = max(64, request["max_output_tokens"] // 2)
        return "degrade_small_model_output_%d" % reduced_output
    if reason == "circuit_open" and request.get("fallback"):
        return "fallback_" + request["fallback"]
    return "degrade_cached_or_async"

def protect_request(request):
    trace = []

```

```

rate_ok, rate_reason = rate_limit_decision(request)
trace.append(("rate_limit", rate_reason))
if not rate_ok:
    return "rate_limited_" + rate_reason, trace

retry_ok, retry_reason, retry_ratio = can_retry(request)
if request.get("error") is not None:
    trace.append(("retry", retry_reason if retry_ratio is None else "%s@%s" % (retry_reason, retry_ratio)))
    if retry_ok:
        return "retry_scheduled", trace
    return "no_retry_" + retry_reason, trace

state = endpoint_state(request["endpoint"])
trace.append(("circuit", state))
if state == "open":
    return degrade(request, "circuit_open"), trace

admission_ok, admission_reason = admission_decision(request, request["endpoint"])
trace.append(("admission", admission_reason))
if not admission_ok:
    return degrade(request, admission_reason), trace

return "admitted_" + request["endpoint"], trace

def build_audit_cases():
    complete = {"name": "complete_protection_case"}
    complete.update({check: True for check in CHECKS})
    cases = [complete]
    bad_names = [
        "request_token_rate_limit_missing_bad",
        "concurrency_quota_missing_bad",
        "admission_capacity_missing_bad",
        "circuit_state_missing_bad",
        "retry_error_classification_missing_bad",
        "retry_budget_missing_bad",
        "idempotency_stage_missing_bad",
        "timeout_budget_missing_bad",
        "degradation_policy_missing_bad",
        "priority_fairness_missing_bad",
        "streaming_cleanup_missing_bad",
        "protection_trace_missing_bad",
        "cost_budget_missing_bad",
        "protection_gate_missing_bad",
    ]
    for bad_name, failed_check in zip(bad_names, CHECKS):
        case = {"name": bad_name}
        case.update({check: True for check in CHECKS})
        case[failed_check] = False
        cases.append(case)
    return cases

def audit_cases(cases):
    metrics = {

```

```

        check: round(sum(1 for case in cases if case.get(check)) / len(cases), 3)
    for check in CHECKS
}
failed_cases = [case["name"] for case in cases if not all(case.get(check) for check in CHECKS)]
failed_gates = [check for check, value in metrics.items() if value < 1.0]
hard_blockers = sum(1 for case in cases if case["name"].endswith("_bad"))
gate_pass = min(metrics.values()) >= 0.95 and hard_blockers == 0
return metrics, failed_cases, failed_gates, hard_blockers, gate_pass

decisions = {}
traces = {}
for request in REQUESTS:
    decision, trace = protect_request(request)
    decisions[request["id"]] = decision
    traces[request["id"]] = trace

open_endpoints = [name for name in ENDPOINTS if endpoint_state(name) == "open"]
retryable = {
    request["id"]: can_retry(request)[1]
    for request in REQUESTS
    if request.get("error") is not None
}
metrics, failed_cases, failed_gates, hard_blockers, gate_pass = audit_cases(build_audit_cases())

print("protection_decisions=", decisions)
print("open_endpoints=", open_endpoints)
print("retry_classification=", retryable)
print("estimated_costs=", {request["id"]: estimate_cost(request) for request in REQUESTS})
print("remaining_timeout_s=", {request["id"]: remaining_timeout(request) for request in REQUESTS})
print("trace_coverage=", round(sum(1 for trace in traces.values() if len(trace) >= 1) / len(traces), 3))
print("metrics=", metrics)
print("hard_blocker_count=", hard_blockers)
print("failed_cases=", failed_cases)
print("failed_gates=", failed_gates)
print("protection_gate_pass=", gate_pass)

```

输出示例:

```

protection_decisions= {'healthy_chat': 'admitted_large', 'free_long_context': 'rate_limited_request_rate_limit', 'bad_
open_endpoints= ['sick_large']
retry_classification= {'transient_connect': 'retry_scheduled', 'partial_stream_timeout': 'already_started_or_streaming
estimated_costs= {'healthy_chat': 0.56, 'free_long_context': 4.8, 'bad_endpoint': 0.768, 'kv_pressure_degrade': 3.6, 'tr
remaining_timeout_s= {'healthy_chat': 13.0, 'free_long_context': 19.0, 'bad_endpoint': 10.5, 'kv_pressure_degrade': 4.5
trace_coverage= 1.0
metrics= {'request_token_rate_limit': 0.933, 'concurrency_quota_control': 0.933, 'admission_capacity_guard': 0.933, 'ci
hard_blocker_count= 14
protection_gate_pass= False

```

这个 demo 的重点是训练保护策略的判断顺序: 先看租户和 token 配额, 再看错误是否可重试, 再看 endpoint 是否应熔断, 再看当前容量是否接得住, 最后才决定执行、拒绝、重试或降级。面试中能讲清这条链路, 比单独背 token bucket 或 circuit breaker 更有说服力。

33.22 常见误区

误区一: 失败就重试。

大模型请求成本高，重试可能放大故障，必须限制重试条件和预算。

误区二：限流只按 QPS。

LLM 推理要按 token、并发、上下文长度和成本预算限流。

误区三：超时只设一个总超时。

推理链路很长，需要排队、prefill、decode、streaming 和工具调用等多层超时。

误区四：降级就是换小模型。

降级还可以是缓存、截断、减少输出、异步处理或模板化响应。

误区五：熔断只看错误率。

对推理服务，队列长度、超时率、OOM、KV cache 水位和 p99 也可能触发熔断。

33.23 面试常见追问

问题一：LLM 服务怎么做限流？

可以回答：不能只按 QPS，要按租户、API key、模型、并发、input tokens/s、output tokens/s、最大上下文、最大输出和成本预算做多维限流。

问题二：为什么推理请求不能无脑重试？

可以回答：因为请求成本高，可能已经在 runtime 执行或已返回部分 token，重试会放大流量且输出非幂等。应只对尚未执行的临时错误重试，并设置重试预算。

问题三：如何设计超时？

可以回答：使用 timeout budget，把总超时拆到网关、排队、prefill、decode、streaming、工具调用和后处理阶段，并根据剩余预算决定是否继续、降级或拒绝。

问题四：降级策略怎么设计？

可以回答：根据业务可接受范围设计降级路径，例如小模型、缓存、截断上下文、减少输出、关闭工具调用或异步处理，同时记录降级原因和质量影响。

问题五：如何把限流、熔断、重试、超时和降级串成一条链路？

可以回答：入口先做认证、租户限流、token / 并发 / 成本配额；admission control 再看队列、KV、剩余 timeout 和 runtime 健康；router 跳过 open endpoint；可重试错误必须满足 not started、幂等、retry budget 和剩余 deadline；资源不足或依赖异常时按业务策略降级；全程记录 request id、tenant、policy version、decision reason、fallback、retry attempt、timeout 和成本。

33.24 小练习

1. 为什么 LLM 推理限流不能只看 QPS？
2. Admission control 和限流有什么区别？
3. 熔断器的 Closed、Open、Half-open 状态分别是什么意思？
4. 哪些错误适合重试，哪些不适合？
5. Timeout budget 为什么比单个总超时更合理？
6. 大模型服务有哪些常见降级方式？
7. 为什么降级必须记录 trace？
8. 如何防止重试风暴？
9. 写一个保护策略审计器，输入请求的租户、token、并发、错误类型、执行阶段、剩余 timeout 和 endpoint 健康状态，输出 admit、reject、retry、fallback 或 degrade。

33.25 本章小结

本章讲了限流、熔断、重试、超时和降级。

你需要记住：

1. 推理平台稳定性设计的目标是控制故障范围，而不是保证每个请求都成功。
2. 限流要按请求、token、并发、上下文、租户和成本多维设计。
3. 熔断用于隔离异常模型、实例、工具或地域，防止故障扩散。
4. 重试必须谨慎，只对安全的临时错误重试，并设置 retry budget。
5. 超时要分阶段设计，使用 timeout budget 管理总耗时。
6. 降级是有控制的能力退让，必须可观测、可审计、可评估质量影响。
7. 五类保护机制要串成同一条 request protection trace，否则很难解释一次拒绝、重试或降级到底是否正确。

下一章我们会讲灰度发布、A/B 测试、回滚和模型版本管理。

第 34 章 灰度发布、A/B 测试、回滚和模型版本管理

上一章讲了限流、熔断、重试、超时和降级。本章讲模型上线过程中的风险控制：灰度发布、A/B 测试、回滚和模型版本管理。

在传统后端服务里，新版本上线主要担心功能 bug、性能退化和错误率上升。大模型上线还要担心输出质量、风格、安全、成本、延迟、工具调用行为和业务指标变化。

先记住一句话：

模型发布不是把新权重部署上去，而是一个围绕版本、流量、指标、风险和回滚建立的发布控制系统。

34.0 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做过资料校准。重点参考的是 Kubernetes Deployment 对 rolling update、revision history、rollout status 和 rollback 的稳定控制面口径，Argo Rollouts 对 canary steps、traffic weight、pause、analysis、abort 和 rollback 的发布治理抽象，MLflow Model Registry 对 model、model version、alias、tags、description 和 lineage metadata 的版本管理口径，Google SRE 对 canary release、发布风险控制和指标观测的工程经验，以及在线受控实验 / A/B testing 文献对随机分桶、样本比例、显著性、guardrail 指标和连续看数风险的实验口径。

这些资料共同指向一个稳定事实：模型发布不是“把新模型切成 100% 流量”。发布控制面要同时证明版本包完整、离线评估通过、契约兼容、灰度分桶稳定、A/B 实验设计可信、guardrail 不破、回滚路径可用、trace 可审计，并且任何一个硬门禁失败都能停止放量。

本章只抽象截至 2026-06 仍稳定的模型发布治理方法，不把某个发布平台、模型仓库、service mesh、实验平台或云厂商字段名写成通用标准。正文公式用于面试表达、发布策略审计和最小 demo；真实上线仍要结合业务风险、模型能力、历史事故、实验样本量、SLO、人工审核流程和合规要求校准阈值。

34.1 为什么模型发布更复杂

模型版本变化可能带来多种影响：

1. 回答质量变化。
2. 语言风格变化。
3. 安全拒答率变化。
4. 幻觉率变化。
5. 工具调用格式变化。
6. 结构化输出稳定性变化。
7. 延迟变化。
8. 成本变化。
9. 下游业务转化率变化。

10. 用户满意度变化。

有些变化不是错误率能捕获的。

模型服务可能 200 OK，但答案质量下降，这就是模型发布的特殊风险。

34.2 模型版本管理的基本对象

一个成熟平台至少要管理这些对象：

1. Model。
2. ModelVersion。
3. ModelArtifact。
4. TokenizerVersion。
5. RuntimeConfig。
6. PromptTemplateVersion。
7. EvalReport。
8. Deployment。
9. Endpoint。
10. RoutePolicy。

不要只管理权重文件。

模型上线依赖的是一组版本：权重、tokenizer、prompt、runtime、推理参数、路由策略和安全策略。

34.3 ModelVersion 应该记录什么

ModelVersion 中通常记录：

1. 模型名称。
2. 版本号。
3. 权重路径。
4. 权重 checksum。
5. tokenizer 版本。
6. 基座模型。
7. 训练数据版本。
8. SFT / RLHF / DPO 信息。
9. 推理默认参数。
10. 最大上下文长度。
11. 支持能力。
12. 评估报告。
13. 安全报告。
14. 发布状态。
15. 创建人和审批人。

这些信息是可追溯性的基础。

当线上出现问题时，你要能回答：这个输出来自哪个模型、哪个版本、哪个 tokenizer、哪个 prompt、哪个 runtime。

34.4 发布状态机

模型版本可以有状态机：

created -> validated -> staged -> canary -> production -> deprecated -> archived

状态说明：

1. created：版本已注册但未验证。
2. validated：通过基础校验和离线评估。

3. staged: 部署到预发环境。
4. canary: 小流量灰度。
5. production: 正式生产。
6. deprecated: 不再推荐使用, 但仍可回滚或兼容。
7. archived: 归档, 不再服务线上请求。

状态机可以防止未经验证的模型直接上线。

34.5 发布前校验

模型发布前至少要校验:

1. 权重文件完整性。
2. tokenizer 是否匹配。
3. runtime 是否兼容。
4. 推理参数是否有效。
5. 上下文长度是否符合声明。
6. 基础 benchmark 是否通过。
7. 离线 eval 是否达标。
8. 安全 eval 是否达标。
9. 输出格式是否兼容。
10. 成本和延迟是否可接受。

很多事故来自 “模型能加载” 但 “不符合线上契约”。

34.6 灰度发布是什么

灰度发布是让一小部分流量先使用新版本, 观察指标后逐步扩大。

典型流程:

0% -> 1% -> 5% -> 10% -> 25% -> 50% -> 100%

每一步都要观察指标。

如果指标异常, 停止灰度或回滚。

灰度的核心不是 “慢慢放量”, 而是 “在可控范围内暴露风险”。

34.7 灰度维度

灰度可以按多种维度:

1. 流量比例。
2. 租户。
3. 用户分桶。
4. 地域。
5. 业务场景。
6. prompt 类型。
7. 模型能力。
8. 风险等级。
9. 内部用户。
10. 白名单客户。

不同维度适合不同场景。

例如新模型先给内部员工、低风险 FAQ、非关键租户使用, 比直接全量随机灰度更安全。

34.8 稳定分桶

灰度要使用稳定分桶。

稳定分桶指同一个用户或请求实体在同一实验中总是落到同一个版本。

常见方式：

```
bucket = hash(user_id + experiment_id) % 100
```

稳定分桶的好处：

1. 用户体验一致。
2. 指标统计稳定。
3. 避免同一用户来回切版本。
4. 便于问题复现。

如果每次请求随机分流，用户可能一会儿看到旧模型，一会儿看到新模型，体验和评估都会变差。

34.9 A/B 测试是什么

A/B 测试是对比两个或多个版本在真实流量下的表现。

例如：

1. A：旧模型。
2. B：新模型。

比较它们的：

1. 任务成功率。
2. 用户满意度。
3. 点击率或转化率。
4. 安全拦截率。
5. 人工反馈。
6. 延迟。
7. 成本。
8. 工具调用成功率。

A/B 测试不只是模型质量评估，也是业务效果评估。

34.10 A/B 测试和灰度的区别

灰度发布关注风险控制。

A/B 测试关注效果比较。

灰度问的是：新版本能不能安全上线？

A/B 测试问的是：新版本是不是比旧版本更好？

它们经常一起使用，但目标不同。

一个模型可以通过灰度稳定性检查，但 A/B 测试发现业务效果没有提升。

34.11 模型发布指标

模型发布要观察多类指标。

质量指标：

1. 任务成功率。
2. 用户满意度。
3. 人工偏好胜率。

4. 工具调用成功率。
5. 结构化输出解析成功率。

安全指标：

1. 拒答率。
2. 违规输出率。
3. 敏感信息泄露率。
4. 越权回答率。

性能指标：

1. TTFT。
2. TPOT。
3. p95 / p99 延迟。
4. tokens/s。
5. 错误率。
6. 超时率。

成本指标：

1. cost per 1k tokens。
2. 平均输入 token。
3. 平均输出 token。
4. GPU 利用率。
5. cache 命中率。

业务指标：

1. 转化率。
2. 留存。
3. 工单解决率。
4. 用户投诉率。
5. 人工接管率。

模型发布不能只看一个指标。

34.12 Guardrail 指标

Guardrail 指标是发布过程中不能突破的红线。

例如：

1. p99 延迟不能上升超过 20%。
2. 成本不能上升超过 15%。
3. 安全违规率不能上升。
4. 工具调用失败率不能超过阈值。
5. 用户投诉不能显著增加。

即使主指标提升，只要 guardrail 破了，也要停止发布。

例如新模型回答更详细，但成本翻倍、延迟翻倍，就未必适合全量。

34.13 回滚机制

回滚是发布系统的生命线。

回滚不应该依赖人工临时改配置。

平台应该支持：

1. 一键回滚模型版本。
2. 回滚路由权重。

3. 回滚 prompt 模板。
4. 回滚 runtime 配置。
5. 回滚安全策略。
6. 回滚灰度实验。

回滚要快，且要可验证。

34.14 回滚触发条件

常见回滚触发条件：

1. 错误率上升。
2. 超时率上升。
3. p99 延迟突破阈值。
4. 安全指标异常。
5. 质量指标显著下降。
6. 成本异常上升。
7. 工具调用失败。
8. 结构化输出解析失败。
9. 用户投诉上升。
10. 人工审核发现严重问题。

回滚可以人工触发，也可以自动触发。

自动回滚要谨慎，避免指标波动导致频繁切换。

34.15 兼容性问题

模型版本升级可能破坏兼容性。

例如：

1. JSON 输出字段变化。
2. tool call 参数格式变化。
3. 拒答风格变化。
4. prompt 模板不再适配。
5. tokenizer 不兼容。
6. function calling 稳定性下降。
7. 下游 parser 解析失败。

所以模型发布要做契约测试。

契约测试检查模型输出是否满足下游依赖的格式和行为要求。

34.16 Prompt 版本也要管理

很多线上变化不是模型权重变了，而是 prompt 变了。

Prompt 模板也应该版本化：

1. system prompt。
2. instruction 模板。
3. tool 描述。
4. RAG 拼接模板。
5. 输出格式约束。
6. 安全提示词。

Prompt 版本变化同样需要灰度、A/B 测试和回滚。

不要把 prompt 当成代码里的字符串随便改。

34.17 Runtime 配置也要管理

Runtime 配置变化也可能影响输出和性能。

例如：

1. temperature。
2. top_p。
3. max_tokens。
4. stop sequence。
5. batch 参数。
6. KV cache 参数。
7. quantization。
8. speculative decoding。
9. runtime 版本。

这些变化都应该纳入发布管理。

否则你可能以为是模型质量变差，其实是推理参数或 runtime 变化导致。

34.18 发布审计

模型发布必须可审计。

审计内容包括：

1. 谁创建了版本。
2. 谁提交了发布。
3. 谁批准了发布。
4. 发布了什么模型和配置。
5. 灰度比例如何变化。
6. 何时全量。
7. 是否回滚。
8. 回滚原因是什么。
9. 发布期间指标如何变化。

对企业平台来说，模型发布审计和代码发布审计一样重要。

34.19 一个完整发布流程

一个完整模型发布流程可以是：

1. 注册 ModelVersion
2. 校验权重、tokenizer、runtime 和配置
3. 运行离线 eval 和安全 eval
4. 部署到 staging
5. 执行契约测试和压测
6. 创建灰度发布计划
7. 对内部或低风险流量放量 1%
8. 观察质量、安全、延迟、成本和业务指标
9. 逐步扩大到 5%、10%、25%、50%
10. 达标后全量
11. 持续监控 guardrail
12. 异常时自动或人工回滚
13. 记录完整审计和复盘信息

这个流程看起来繁琐，但这是生产模型平台的必要复杂度。

34.20 发布治理审计指标与最小 demo

灰度发布、A/B 测试、回滚和版本管理如果分散在不同系统里，线上事故时很难回答“到底哪一个版本、哪一个 prompt、哪一个 runtime 参数导致了问题”。因此需要把模型发布抽象成一个可审计的 release bundle。

一个发布样本可以写成：

$r_i = (m_i, w_i, t_i, p_i, c_i, s_i, e_i, g_i, b_i, k_i, l_i, a_i, o_i, z_i)$

其中， m_i 是模型版本， w_i 是权重 artifact， t_i 是 tokenizer 版本， p_i 是 prompt 模板版本， c_i 是 runtime config， s_i 是安全策略版本， e_i 是离线 eval 报告， g_i 是灰度计划， b_i 是稳定分桶配置， k_i 是 guardrail 指标， l_i 是回滚目标， a_i 是审批信息， o_i 是上线观测结果， z_i 是 audit trace。

发布版本完整性可以写成覆盖率：

$C_{\{\mathrm{bundle}\}} = \frac{K_{\{\mathrm{captured}\}}}{K_{\{\mathrm{required}\}}}$

其中， $K_{\{\mathrm{required}\}}$ 包括权重 checksum、tokenizer、prompt、runtime、推理参数、安全策略、eval report、contract test、owner、approver、rollback target 和 policy version。只记录权重路径时， $C_{\{\mathrm{bundle}\}}$ 远小于 1。

稳定分桶可以写成：

$b_u = h(u, e) \bmod 100$

其中， u 是用户或租户实体， e 是 experiment id， h 是稳定 hash。若灰度权重为 q ，则进入 treatment 的条件是：

$Z_u = \mathbf{1}[b_u < 100q]$

这能保证同一用户在同一实验中稳定看到同一版本，避免每次请求随机切换导致体验和指标污染。

灰度放量可以看成是一个单调步骤序列：

$0 = q_0 < q_1 < \dots < q_K \leq 1$

第 k 步是否继续放量由指标门禁决定：

$G_k = \mathbf{1} \left[\Delta Q_k \geq -\epsilon_{q_k} \wedge R_{\{\mathrm{safe}\},k} \leq \rho_s \wedge L_{99,k} \leq B_{99} \wedge C_k \leq B_c \wedge P_{0,k} = 0 \right]$

其中， ΔQ_k 是候选版本相对 baseline 的质量变化， $R_{\{\mathrm{safe}\},k}$ 是安全风险率， $L_{99,k}$ 是 p99 延迟， C_k 是单位成本， $P_{0,k}$ 是 P0 风险数。只要某一步 $G_k = 0$ ，发布就应该 pause、abort 或 rollback。

A/B 测试需要同时看主指标和 guardrail。对二值主指标，例如任务成功率，候选版本 B 和基线版本 A 的差异可以写成：

$\Delta_{\{\mathrm{success}\}} = \hat{p}_B - \hat{p}_A$

一个常见的二比例近似统计量是：

$z = \frac{\hat{p}_B - \hat{p}_A}{\sqrt{\hat{p}(1-\hat{p})(1/n_A + 1/n_B)}}$

其中， $\hat{p}$ 是 pooled success rate， n_A 和 n_B 是样本量。这个公式不是让面试中手算 p-value，而是提醒你：A/B 测试要有样本量、随机分桶、显著性和停止规则，不能看到一天数据好就全量。

样本比例异常可以用 sample ratio mismatch 的思想检查。若期望 treatment 占比为 q ，实际 treatment 数为 n_B ，总样本为 n ，可以写成：

$\chi^2_{\{\mathrm{srm}\}} = \frac{(n_B - nq)^2}{nq} + \frac{(n_A - n(1-q))^2}{n(1-q)}$

若这个值明显偏大，说明分流、日志或过滤可能有问题，A/B 结果不能直接信。

回滚准备度可以写成：

$R_{\{\mathrm{rollback}\}} = \mathbf{1} \left[l_i \neq \text{varnothing} \wedge T_{\{\mathrm{rollback}\}} \leq B_{\{\mathrm{rollback}\}} \wedge C_{\{\mathrm{compat}\}} = 1 \wedge D_{\{\mathrm{stable}\}} \right]$

其中, L_i 是回滚目标, $T_{\{\mathrm{rollback}\}}$ 是回滚生效时间, $C_{\{\mathrm{compat}\}}$ 表示旧版本仍兼容当前路由、prompt、tokenizer 和安全策略, $D_{\{\mathrm{state}\}}$ 表示是否存在无法回滚的状态迁移。模型发布如果没有已验证回滚目标, 就不应该进入大流量灰度。

最终发布治理门禁可以写成:

$$G_{\{\mathrm{release}\}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \wedge C_{\{\mathrm{bundle}\}} = 1 \wedge G_{\{\mathrm{offline}\}} = 1 \wedge G_{\{\mathrm{canary}\}} = 1 \wedge G_{\{\mathrm{ab}\}} = 1 \right]$$

其中, C_j 是各审计维度覆盖率, $G_{\{\mathrm{offline}\}}$ 是离线质量 / 安全 / 契约测试门禁, $G_{\{\mathrm{canary}\}}$ 是灰度门禁, $G_{\{\mathrm{ab}\}}$ 是在线实验门禁。这个门禁强调: 模型发布系统不仅要能部署, 还要能证明为什么放量、为什么停止、为什么回滚。

下面这个 0 依赖 Python demo 演示一个简化的模型发布治理审计器: 它会检查 release bundle 完整性、稳定分桶、canary 放量、A/B 结果、guardrail、rollback readiness 和审计覆盖, 并用 bad case 检查发布系统的硬阻断。

```
import hashlib
from math import erfc, sqrt

CHECKS = [
    "version_bundle_trace",
    "model_artifact_integrity",
    "tokenizer_prompt_runtime_binding",
    "offline_eval_quality_gate",
    "safety_eval_gate",
    "contract_test_compatibility",
    "stable_bucket_assignment",
    "canary_ramp_control",
    "ab_experiment_design",
    "sample_ratio_health",
    "guardrail_metric_control",
    "rollback_readiness",
    "release_audit_trace",
    "post_release_monitoring",
    "release_governance_gate",
]

BASELINE = {
    "version": "chat-prod@champion:v33",
    "task_success": 0.812,
    "unsafe_rate": 0.0015,
    "parse_fail_rate": 0.012,
    "p99_latency_ms": 3200,
    "cost_per_1k": 0.0065,
    "complaint_rate": 0.0040,
}

CANDIDATE = {
    "version": "chat-prod@candidate:v34",
    "weight_checksum": "sha256:weights-v34",
    "tokenizer_version": "tok-2026-05",
    "prompt_version": "prompt-support-19",
    "runtime_config": "vllm-conf-27",
    "safety_policy": "safe-policy-11",
    "eval_report": "eval-2026-06-15",
    "owner": "serving-platform",
}
```

```

    "approver": "release-review",
    "rollback_target": BASELINE["version"],
    "offline_task_success": 0.838,
    "offline_safety_pass": 0.998,
    "contract_parse_pass": 0.992,
}

GUARDRAILS = {
    "min_task_success_delta": -0.005,
    "max_unsafe_rate": BASELINE["unsafe_rate"] + 0.0005,
    "max_parse_fail": BASELINE["parse_fail_rate"] + 0.004,
    "max_p99_latency_ms": int(BASELINE["p99_latency_ms"] * 1.20),
    "max_cost_per_1k": round(BASELINE["cost_per_1k"] * 1.15, 6),
    "max_complaint_rate": BASELINE["complaint_rate"] + 0.0010,
    "max_rollback_s": 120,
}

CANARY_STEPS = [
    {
        "weight": 1,
        "task_success": 0.842,
        "unsafe_rate": 0.0014,
        "parse_fail_rate": 0.010,
        "p99_latency_ms": 3310,
        "cost_per_1k": 0.0068,
        "complaint_rate": 0.0038,
        "p0": 0,
    },
    {
        "weight": 5,
        "task_success": 0.839,
        "unsafe_rate": 0.0016,
        "parse_fail_rate": 0.011,
        "p99_latency_ms": 3420,
        "cost_per_1k": 0.0070,
        "complaint_rate": 0.0041,
        "p0": 0,
    },
    {
        "weight": 10,
        "task_success": 0.836,
        "unsafe_rate": 0.0018,
        "parse_fail_rate": 0.012,
        "p99_latency_ms": 3650,
        "cost_per_1k": 0.0072,
        "complaint_rate": 0.0044,
        "p0": 0,
    },
    {
        "weight": 25,
        "task_success": 0.829,
        "unsafe_rate": 0.0029,
        "parse_fail_rate": 0.018,
        "p99_latency_ms": 3920,
        "cost_per_1k": 0.0077,
    }
]

```

```

        "complaint_rate": 0.0057,
        "p0": 0,
    },
]

AB_COUNTS = {
    "control_success": 8120,
    "control_total": 10000,
    "treatment_success": 8390,
    "treatment_total": 10000,
    "expected_treatment_ratio": 0.10,
    "observed_treatment": 1985,
    "observed_total": 20000,
}

def stable_bucket(entity_id, experiment_id):
    raw = (entity_id + ":" + experiment_id).encode("utf-8")
    digest = hashlib.sha256(raw).hexdigest()
    return int(digest[:8], 16) % 100

def assign_variant(entity_id, experiment_id, treatment_percent):
    return "candidate" if stable_bucket(entity_id, experiment_id) < treatment_percent else "baseline"

def bundle_completeness(candidate):
    required = [
        "version",
        "weight_checksum",
        "tokenizer_version",
        "prompt_version",
        "runtime_config",
        "safety_policy",
        "eval_report",
        "owner",
        "approver",
        "rollback_target",
    ]
    captured = sum(1 for key in required if candidate.get(key))
    return round(captured / len(required), 3)

def guardrail_failures(step):
    failures = []
    if step["task_success"] - BASELINE["task_success"] < GUARDRAILS["min_task_success_delta"]:
        failures.append("task_success")
    if step["unsafe_rate"] > GUARDRAILS["max_unsafe_rate"]:
        failures.append("unsafe_rate")
    if step["parse_fail_rate"] > GUARDRAILS["max_parse_fail"]:
        failures.append("parse_fail_rate")
    if step["p99_latency_ms"] > GUARDRAILS["max_p99_latency_ms"]:
        failures.append("p99_latency_ms")
    if step["cost_per_1k"] > GUARDRAILS["max_cost_per_1k"]:
        failures.append("cost_per_1k")

```

```

if step["complaint_rate"] > GUARDRAILS["max_complaint_rate"]:
    failures.append("complaint_rate")
if step["p0"] > 0:
    failures.append("p0")
return failures

def canary_decisions(steps):
    decisions = {}
    failures = {}
    for step in steps:
        failed = guardrail_failures(step)
        if failed:
            decisions[step["weight"]] = "rollback"
            failures[step["weight"]] = failed
            break
        decisions[step["weight"]] = "promote"
    return decisions, failures

def two_proportion_test(success_a, total_a, success_b, total_b):
    rate_a = success_a / total_a
    rate_b = success_b / total_b
    pooled = (success_a + success_b) / (total_a + total_b)
    se = sqrt(pooled * (1 - pooled) * (1 / total_a + 1 / total_b))
    z = (rate_b - rate_a) / se
    p_value = erfc(abs(z) / sqrt(2))
    return round(rate_b - rate_a, 4), round(z, 3), round(p_value, 6)

def sample_ratio_chi2(observed_treatment, observed_total, expected_ratio):
    observed_control = observed_total - observed_treatment
    expected_treatment = observed_total * expected_ratio
    expected_control = observed_total * (1 - expected_ratio)
    chi2 = ((observed_treatment - expected_treatment) ** 2) / expected_treatment
    chi2 += ((observed_control - expected_control) ** 2) / expected_control
    return round(chi2, 3)

def rollback_ready(candidate, rollback_seconds):
    return bool(candidate.get("rollback_target")) and rollback_seconds <= GUARDRAILS["max_rollback_s"]

def build_audit_cases():
    complete = {"name": "complete_release_case"}
    complete.update({check: True for check in CHECKS})
    cases = [complete]
    bad_names = [
        "version_bundle_missing_bad",
        "artifact_checksum_missing_bad",
        "tokenizer_prompt_runtime_unbound_bad",
        "offline_eval_missing_bad",
        "safety_eval_missing_bad",
        "contract_test_missing_bad",
        "random_bucket_bad",
    ]

```



```

        "canary_ramp_uncontrolled_bad",
        "ab_design_missing_bad",
        "sample_ratio_unchecked_bad",
        "guardrail_missing_bad",
        "rollback_target_missing_bad",
        "release_trace_missing_bad",
        "post_release_monitoring_missing_bad",
        "release_gate_missing_bad",
    ]
    for bad_name, failed_check in zip(bad_names, CHECKS):
        case = {"name": bad_name}
        case.update({check: True for check in CHECKS})
        case[failed_check] = False
        cases.append(case)
    return cases

def audit_cases(cases):
    metrics = {
        check: round(sum(1 for case in cases if case.get(check)) / len(cases), 3)
        for check in CHECKS
    }
    failed_cases = [case["name"] for case in cases if not all(case.get(check) for check in CHECKS)]
    failed_gates = [check for check, value in metrics.items() if value < 1.0]
    hard_blockers = sum(1 for case in cases if case["name"].endswith("_bad"))
    gate_pass = min(metrics.values()) >= 0.95 and hard_blockers == 0
    return metrics, failed_cases, failed_gates, hard_blockers, gate_pass

decisions, failures = canary_decisions(CANARY_STEPS)
delta, z_score, p_value = two_proportion_test(
    AB_COUNTS["control_success"],
    AB_COUNTS["control_total"],
    AB_COUNTS["treatment_success"],
    AB_COUNTS["treatment_total"],
)
chi2_srm = sample_ratio_chi2(
    AB_COUNTS["observed_treatment"],
    AB_COUNTS["observed_total"],
    AB_COUNTS["expected_treatment_ratio"],
)
bucket_examples = {
    user: assign_variant(user, "exp-release-v34", 10)
    for user in ["u001", "u014", "u201", "u333", "u777"]
}
metrics, failed_cases, failed_gates, hard_blockers, gate_pass = audit_cases(build_audit_cases())

print("release_bundle_completeness=", bundle_completeness(CANDIDATE))
print("bucket_examples=", bucket_examples)
print("canary_decisions=", decisions)
print("guardrail_failures=", failures)
print("ab_result=", {
    "delta_success": delta,
    "z_score": z_score,
    "p_value": p_value,

```

```

    "sample_ratio_chi2": chi2_srm,
    "sample_ratio_pass": chi2_srm < 3.84,
})
print("rollback_ready=", rollback_ready(CANDIDATE, rollback_seconds=45))
print("metrics=", metrics)
print("hard_blocker_count=", hard_blockers)
print("failed_cases=", failed_cases)
print("failed_gates=", failed_gates)
print("release_governance_gate_pass=", gate_pass)

```

输出示例：

```

release_bundle_completeness= 1.0
bucket_examples= {'u001': 'candidate', 'u014': 'baseline', 'u201': 'baseline', 'u333': 'baseline', 'u777': 'candidate'}
canary_decisions= {1: 'promote', 5: 'promote', 10: 'promote', 25: 'rollback'}
guardrail_failures= {25: ['unsafe_rate', 'parse_fail_rate', 'p99_latency_ms', 'cost_per_1k', 'complaint_rate']}
ab_result= {'delta_success': 0.027, 'z_score': 5.03, 'p_value': 0.0, 'sample_ratio_chi2': 0.125, 'sample_ratio_pass': True}
rollback_ready= True
metrics= {'version_bundle_trace': 0.938, 'model_artifact_integrity': 0.938, 'tokenizer_prompt_runtime_binding': 0.938,
hard_blocker_count= 15
release_governance_gate_pass= False

```

这个 demo 的重点是训练发布治理的判断方式：候选版本离线指标更好，不代表可以直接全量；A/B 主指标显著提升，也不能绕过安全、延迟、成本、结构化输出和投诉率 guardrail。发布控制面必须能在 25% 灰度发现风险后暂停或回滚，并把版本、分桶、指标、审批和回滚证据串成 audit trace。

34.21 常见误区

误区一：模型 eval 分数高就可以全量。

离线 eval 不能覆盖所有线上场景，还需要灰度和在线指标。

误区二：发布只管理权重。

权重、tokenizer、prompt、runtime、参数、安全策略和路由都要版本化。

误区三：HTTP 200 就代表发布成功。

模型可能稳定返回错误答案。质量指标和业务指标同样重要。

误区四：回滚就是切回旧模型。

有时需要回滚 prompt、runtime 参数、路由策略或安全策略。

误区五：A/B 测试只看满意度。

A/B 测试还要看延迟、成本、安全和下游业务指标。

34.22 面试常见追问

问题一：模型发布和普通服务发布有什么不同？

可以回答：模型发布不仅关注错误率和性能，还要关注输出质量、安全性、工具调用、结构化输出、成本、延迟、用户体验和业务指标，而且很多问题不会表现为 HTTP 错误。

问题二：如何设计模型灰度发布？

可以回答：先注册版本并完成离线 eval、安全 eval、契约测试和压测，再按租户、用户分桶、场景或流量比例小流量灰度，观察质量、安全、延迟、成本和业务指标，达标后逐步放量，异常则回滚。

问题三：为什么 prompt 也要版本管理？

可以回答：prompt 会影响模型行为、输出格式、安全策略和工具调用，改 prompt 可能像改模型一样影响线上结果，因此需要版本、灰度、审计和回滚。

问题四：回滚应该回滚什么？

可以回答：不仅包括模型权重版本，还包括 tokenizer、prompt 模板、runtime 配置、推理参数、路由权重、安全策略和灰度实验配置。

问题五：为什么 A/B 主指标显著提升也不能直接全量？

可以回答：因为主指标只代表一个目标，模型发布还必须满足安全、延迟、成本、结构化输出、工具调用、投诉率、样本比例健康、分桶稳定、回滚准备和审计 trace 等 guardrail。任何硬 guardrail 失败都应该暂停或回滚。

34.23 小练习

1. 为什么模型发布不能只看错误率？
2. ModelVersion 应该记录哪些字段？
3. 灰度发布和 A/B 测试有什么区别？
4. 什么是稳定分桶，为什么重要？
5. 模型发布应该观察哪些 guardrail 指标？
6. 为什么 prompt 和 runtime 配置也要版本化？
7. 模型回滚除了切模型版本，还可能回滚什么？
8. 如何设计一个企业级模型发布流程？
9. 写一个 0 依赖发布治理审计器，输入 release bundle、canary steps、A/B 样本数、guardrail 阈值和 rollback target，输出 promote、pause、rollback 或 release blocked。

34.24 本章小结

本章讲了灰度发布、A/B 测试、回滚和模型版本管理。

你需要记住：

1. 模型发布管理的是一组版本，而不只是权重文件。
2. 模型上线风险包括质量、安全、延迟、成本、工具调用和业务指标变化。
3. 灰度发布用于控制风险，A/B 测试用于比较效果。
4. 发布指标要覆盖质量、安全、性能、成本和业务结果。
5. 回滚能力必须覆盖模型、prompt、runtime、路由和安全策略。
6. 企业级模型平台必须具备版本化、灰度、审计、回滚和复盘能力。
7. 发布治理要把 release bundle、稳定分桶、A/B 实验、guardrail、回滚目标和 audit trace 串成同一条证据链。

下一章我们会进入本部分收尾，讲推理平台系统设计面试题。

第 35 章 推理平台系统设计面试题

前面第 26 到第 34 章讲了大模型推理平台工程：推理平台总览、runtime、prefill/decode、KV cache、continuous batching、模型路由、缓存、扩缩容、稳定性保护和模型发布。本章作为第四部分收尾，专门讲推理平台系统设计面试题。

面试中常见题目是：

请你设计一个大模型推理平台，支持多模型、多租户、高并发、低延迟、灰度发布和成本治理。

这类题不是让你背组件名，而是考你能否把在线服务、GPU runtime、大模型推理特性和平台治理串起来。

先记住一句话：

推理平台系统的核心，是在模型质量、延迟、吞吐、成本、安全和可用性之间建立一个可观测、可治理、可演进的在线服务体系。

35.0 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做过资料校准。重点参考的是 Google SRE 对 SLI / SLO / error budget、长尾延迟和发布风险控制的系统设计口径，OpenTelemetry 对 traces、metrics、logs 和 baggage 等观测信号的抽象，Triton Inference Server 对 request、queue、compute、batch 和 GPU 指标的推理服务指标口径，vLLM 对 PagedAttention、KV cache、OpenAI-compatible serving、observability 和 metrics 的公开文档，以及前面第 26 到第 34 章已经校准过的 runtime、prefill / decode、KV cache、continuous batching、模型路由、缓存、扩缩容、保护策略和发布治理口径。

这些资料共同指向一个稳定事实：推理平台系统设计不是 “API Gateway + K8s + vLLM” 三个组件名，而是要证明在线请求生命周期、SLO、模型注册、路由、runtime、KV 容量、调度、缓存、扩缩容、保护机制、发布治理、多租户、安全、观测和成本都能被同一套控制面管理。

本章只抽象截至 2026-06 仍稳定的大模型推理平台系统设计方法，不把某个 runtime、云厂商、Kubernetes 发行版、service mesh、模型仓库或内部平台写成通用标准。正文公式用于面试表达、容量估算和系统设计自查；真实上线仍要用目标模型、硬件、runtime、流量分布、SLO、租户协议、合规要求和历史事故复盘来校准。

35.1 面试题拆解

题目通常会这样问：

1. 如何部署多个大模型？
2. 如何支持高并发推理？
3. 如何控制 TTFT 和 TPOT？
4. 如何做模型路由？
5. 如何做缓存？
6. 如何自动扩缩容？
7. 如何限流和降级？
8. 如何灰度和回滚？
9. 如何做多租户隔离？
10. 如何控制成本？

如果你直接回答 “K8s + vLLM + 网关”，是不够的。

你需要从需求、架构、核心链路、关键瓶颈、稳定性和治理几个层面展开。

35.2 先做需求澄清

系统设计题第一步是澄清需求。

可以问：

1. 支持哪些模型类型？文本、embedding、多模态、rerank？
2. 模型规模多大？7B、70B、MoE、长上下文？
3. QPS 和并发目标是多少？
4. 输入输出 token 分布是什么？
5. 延迟 SLO 是多少？TTFT、TPOT、p99？
6. 是否需要 streaming？
7. 是否多租户？租户之间是否隔离？
8. 是否需要 OpenAI-compatible API？
9. 是否需要灰度、A/B 测试和回滚？
10. 是否有成本预算？
11. 是否有安全和审计要求？

这些问题能体现你知道推理平台不是普通 HTTP 服务。

35.3 定义核心目标

一个生产级推理平台的核心目标可以定义为：

1. 支持多模型、多版本、多 runtime。
2. 支持高并发和流式生成。
3. 控制 TTFT、TPOT 和 p99 延迟。
4. 提高 GPU 利用率和 token 吞吐。
5. 支持模型路由、灰度和回滚。
6. 支持限流、熔断、超时和降级。
7. 支持多租户权限、配额和隔离。
8. 支持缓存和成本治理。
9. 支持完整指标、日志、trace 和审计。

面试回答时要把目标讲清楚，再讲架构。

35.4 总体架构

可以画出这样的架构：

Client / App / Agent

- > API Gateway
- > Auth / Quota / Rate Limit
- > Safety Filter
- > Model Router
- > Cache Layer
- > Request Scheduler / Admission Controller
- > Inference Runtime Cluster
 - > vLLM / TGI / Triton / SGLang / TensorRT-LLM
 - > GPU Workers
 - > KV Cache / Continuous Batching
- > Model Registry
- > Deployment Controller
- > Autoscaler
- > Observability Platform
- > Cost / Audit / Billing

模块职责：

1. API Gateway: 统一入口、协议转换、认证入口。
2. Auth / Quota / Rate Limit: 身份、权限、配额和限流。
3. Safety Filter: 输入输出安全策略。
4. Model Router: 模型、版本和 endpoint 选择。
5. Cache Layer: prompt cache、语义缓存、结果缓存。
6. Admission Controller: 判断系统当前是否接得住。
7. Runtime Cluster: 执行 prefill、decode、batching、KV cache。
8. Model Registry: 管理模型元数据和版本。
9. Deployment Controller: 发布、灰度、回滚。
10. Autoscaler: 基于 LLM 指标扩缩容。
11. Observability: 指标、日志、trace、告警。
12. Cost / Audit: 成本归因和审计。

35.5 请求链路

一个完整请求链路：

1. Client 发送请求
2. Gateway 做认证、协议校验和基础限流
3. Quota Service 检查租户配额和 token 预算
4. Safety Filter 检查输入策略
5. Router 根据模型、能力、成本、SLO 和负载选择 endpoint
6. Cache Layer 尝试命中结果缓存或语义缓存

7. Admission Controller 检查队列、KV cache 和实例健康
8. Runtime Scheduler 将请求加入 waiting queue
9. Runtime 执行 prefill, 生成 KV cache
10. Runtime 执行 decode, 通过 streaming 返回 token
11. Safety Filter 检查输出策略
12. Observability 记录 TTFT、TPOT、tokens、trace 和错误码
13. Cost 模块按租户、模型和 token 做归因
14. Audit 模块记录访问事件

这条链路可以帮助面试官看到你对线上服务全流程的理解。

35.6 Runtime 设计重点

Runtime 是执行推理的核心。

需要讲清楚：

1. 使用 vLLM、TGI、Triton、SGLang 或 TensorRT-LLM。
2. 支持 continuous batching。
3. 支持 KV cache 管理。
4. 支持 streaming response。
5. 支持长上下文。
6. 暴露 runtime 指标。
7. 支持健康检查。
8. 支持多 GPU 或分布式推理。

关键指标：

1. TTFT。
2. TPOT。
3. input tokens/s。
4. output tokens/s。
5. queue length。
6. active sequences。
7. KV cache usage。
8. GPU utilization。
9. OOM count。

面试中要强调：runtime 不是普通 Python 服务，而是 GPU 执行引擎。

35.7 Prefill、Decode 和 KV Cache 怎么讲

可以这样讲：

1. Prefill 处理输入 prompt，主要影响 TTFT。
2. Decode 逐 token 生成，主要影响 TPOT 和总延迟。
3. KV cache 用于避免重复计算，但会占用大量显存。
4. 长上下文和高并发会放大 KV cache 压力。
5. 调度时不能只看 QPS，要看 input/output tokens、active sequences 和 KV cache。

这段是推理平台系统设计的核心区分点。

如果候选人完全不提 prefill/decode/KV cache，说明对 LLM serving 理解不足。

35.8 模型路由设计

Router 要做两层选择：

1. 选择模型和版本。
2. 选择 endpoint 和实例。

路由依据包括：

1. 请求指定模型。
2. 任务能力。
3. 租户权限。
4. 延迟 SLO。
5. 成本预算。
6. 灰度规则。
7. endpoint 健康状态。
8. queue length。
9. TTFT / TPOT。
10. KV cache 水位。

Router 输出要包含 route decision 和 route trace，方便审计和排障。

35.9 缓存设计

缓存层可以设计为多级缓存：

1. prompt cache：复用系统 prompt、工具说明、长文档前缀。
2. KV cache：由 runtime 管理，复用 attention KV。
3. 语义缓存：相似问题命中历史答案或中间结果。
4. 结果缓存：完全相同请求直接返回。

缓存 key 要包含：

1. 模型版本。
2. tokenizer 版本。
3. prompt hash。
4. 参数。
5. 租户和权限域。
6. 安全策略版本。
7. 数据版本。

要强调缓存不是纯性能问题，还涉及安全隔离、一致性和审计。

35.10 自动扩缩容设计

Autoscaler 不能只看 CPU 或 QPS。

应该看：

1. QPS。
2. input tokens/s。
3. output tokens/s。
4. waiting queue length。
5. queue wait time。
6. TTFT。
7. TPOT。
8. p95 / p99 latency。
9. GPU utilization。
10. GPU memory usage。
11. KV cache usage。
12. timeout rate。

还要考虑冷启动：

1. 镜像预拉取。
2. 权重本地缓存。
3. warm pool。

4. 模型预热。
5. 预测扩容。

缩容要有 draining, 不再接收新请求, 等待 active request 和 streaming 请求完成。

35.11 限流和稳定性设计

稳定性保护包括:

1. 租户级限流。
2. API key 限流。
3. token/s 限流。
4. 并发限制。
5. 最大上下文长度。
6. 最大输出长度。
7. admission control。
8. 熔断异常 endpoint。
9. 有预算的重试。
10. 分阶段超时。
11. 降级路由。

要强调重试不能无脑做。

如果请求已经开始生成或已经返回部分 token, 重试可能导致重复成本和不一致体验。

35.12 灰度发布和回滚设计

模型发布要管理:

1. 权重版本。
2. tokenizer 版本。
3. prompt 模板版本。
4. runtime 配置。
5. 推理参数。
6. 路由策略。
7. 安全策略。

灰度方式:

1. 按流量比例。
2. 按租户。
3. 按用户分桶。
4. 按业务场景。
5. 按风险等级。

观察指标:

1. 质量。
2. 安全。
3. TTFT / TPOT。
4. p99 延迟。
5. 错误率。
6. 成本。
7. 工具调用成功率。
8. 业务指标。

回滚要一键化, 并且能回滚模型、prompt、runtime、路由和安全策略。

35.13 多租户设计

多租户要考虑：

1. 身份认证。
2. 模型访问权限。
3. 租户级配额。
4. 租户级限流。
5. 租户级成本归因。
6. 数据隔离。
7. 缓存隔离。
8. 日志脱敏。
9. 审计记录。

不要只说 namespace 隔离。

推理平台多租户更关注模型访问权、token 配额、上下文数据和缓存边界。

35.14 可观测性设计

可观测性要覆盖请求全链路。

指标：

1. QPS。
2. TTFT。
3. TPOT。
4. p95 / p99 latency。
5. input/output tokens/s。
6. queue length。
7. GPU utilization。
8. KV cache usage。
9. error rate。
10. timeout rate。
11. cache hit rate。
12. cost per 1k tokens。

Trace 阶段：

1. gateway。
2. auth。
3. router。
4. cache。
5. queue。
6. prefill。
7. decode。
8. safety。
9. response streaming。

日志要脱敏，审计要记录谁在何时调用了哪个模型版本。

35.15 成本治理设计

成本治理包括：

1. 按租户统计 token 用量。
2. 按模型和版本统计成本。
3. 按 endpoint 统计 GPU 成本。
4. 按请求记录 input/output tokens。
5. 小模型优先路由。

6. 语义缓存和结果缓存。
7. prompt 压缩。
8. 输出长度限制。
9. 自动扩缩容。
10. 离峰批处理。

成本治理的目标不是单纯省钱，而是在满足质量和 SLO 的前提下降低单位 token 成本。

35.16 安全设计

安全设计至少包括：

1. API 鉴权。
2. 租户权限。
3. 模型访问控制。
4. 输入安全检查。
5. 输出安全检查。
6. 敏感信息脱敏。
7. 日志和 trace 脱敏。
8. 缓存隔离。
9. 数据留存策略。
10. 审计追踪。

对于企业平台，还要考虑模型输出是否可能泄露私有知识、缓存是否跨租户误命中、日志是否记录敏感 prompt。

35.17 系统瓶颈分析

常见瓶颈包括：

1. Prefill 计算瓶颈。
2. Decode 显存带宽瓶颈。
3. KV cache 显存瓶颈。
4. 队列排队瓶颈。
5. 长请求拖慢短请求。
6. 冷启动慢。
7. 路由不均衡。
8. 下游工具调用慢。
9. streaming 连接过多。
10. 缓存命中率低。

优化也要对应瓶颈：

1. Prefill 慢，考虑 prompt cache、长短请求分队列、prefill/decode 分离。
2. Decode 慢，考虑 continuous batching、KV cache 优化、speculative decoding。
3. KV cache 满，考虑限制上下文、PagedAttention、扩容、驱逐策略。
4. 冷启动慢，考虑 warm pool、权重缓存和预热。

35.18 面试回答模板

可以这样组织答案：

我会先澄清需求，包括模型类型、QPS、token 分布、延迟 SLO、是否 streaming、多租户、安全和成本要求。

整体架构上，我会设计 API Gateway、鉴权限流、Model Router、Cache Layer、Admission Controller、Inference Runtime Cluster、

请求进来后，先做认证、配额和安全检查，再由 Router 根据模型能力、租户权限、成本、SLO、灰度规则和 endpoint 实时负载选择模型。

扩缩容上，我不会只看 QPS 或 GPU 利用率，而会结合 input/output tokens/s、TTFT、TPOT、queue length、KV cache usage、p99 和

稳定性上，会做多维限流、admission control、熔断、分阶段超时、谨慎重试和降级路由。发布上支持模型、prompt、runtime 和路
这个回答覆盖需求、架构、请求链路、runtime 特性、扩缩容、稳定性、发布和治理。

35.19 系统设计审计指标与最小 demo

系统设计题最容易犯的错误是“列组件”。高分答案要能证明每个组件为什么存在、解决什么瓶颈、用什么指标验收、失败时如何回滚或降级。可以把一份推理平台系统设计答案抽象为：

$d_i = (q_i, a_i, l_i, r_i, k_i, c_i, s_i, p_i, g_i, o_i, u_i, h_i, z_i)$

其中， q_i 是需求澄清， a_i 是总体架构， l_i 是请求 lifecycle， r_i 是模型路由和发布， k_i 是 runtime / KV / scheduler， c_i 是缓存和成本， s_i 是安全和多租户， p_i 是保护策略， g_i 是扩缩容和容量， o_i 是观测和审计， u_i 是开发者自助， h_i 是 trade-off 和瓶颈分析， z_i 是证据、指标和门禁。

需求覆盖率可以写成：

$$C_{\{\mathrm{req}\}} = \frac{K_{\{\mathrm{answered}\}}}{K_{\{\mathrm{required}\}}}$$

其中， $K_{\{\mathrm{required}\}}$ 至少包括模型类型、QPS、input / output token 分布、上下文长度、TTFT、TPOT、p99、streaming、多租户、安全、成本、发布频率和可用性目标。需求没澄清，后面的架构很容易变成套模板。

端到端延迟可以拆成：

$$T_{\{\mathrm{e2e}\}} = T_{\{\mathrm{gateway}\}} + T_{\{\mathrm{auth}\}} + T_{\{\mathrm{route}\}} + T_{\{\mathrm{cache}\}} + T_{\{\mathrm{queue}\}} + T_{\{\mathrm{decode}\}}$$

其中 decode 时间可以近似写成：

$$T_{\{\mathrm{decode}\}} \approx y_i T_{\{\mathrm{tpot}\}}$$

这里 y_i 是输出 token 数， $T_{\{\mathrm{tpot}\}}$ 是每个输出 token 的生成时间。这个拆分能解释为什么同样 p99 变慢，可能是 queue、prefill、decode、streaming 或 safety 的问题，不能只看一个总 latency。

Token 负载可以写成：

$$\Lambda_{\{\mathrm{tok}\}} = \lambda_{\{\mathrm{req}\}} (\bar{x} + \bar{y})$$

其中， $\lambda_{\{\mathrm{req}\}}$ 是请求速率， $\bar{x}$ 是平均输入 token， $\bar{y}$ 是平均输出 token。系统设计里只说 QPS 而不说 token 分布，通常说明没有抓住 LLM serving 的核心瓶颈。

KV cache 单请求显存可以估算为：

$$M_{\{\mathrm{kv}, \mathrm{req}\}} = 2L H_{\{\mathrm{kv}\}} d_h b T$$

其中， L 是层数， $H_{\{\mathrm{kv}\}}$ 是 KV heads， d_h 是 head dimension， b 是每个元素字节数， T 是上下文 token 数。KV 容量门禁可以写成：

$$G_{\{\mathrm{kv}\}} = \mathbf{1} \left[N_{\{\mathrm{active}\}} M_{\{\mathrm{kv}, \mathrm{req}\}} \leq \eta_{\{\mathrm{kv}\}} M_{\{\mathrm{gpu}\}} \right]$$

其中， $N_{\{\mathrm{active}\}}$ 是活跃请求数， $M_{\{\mathrm{gpu}\}}$ 是可用显存， $\eta_{\{\mathrm{kv}\}}$ 是留给 KV 的比例。注意实际平台还要扣除权重、workspace、fragmentation 和 runtime overhead。

多指标容量需求可以用最大值收束：

$$N_{\{\mathrm{need}\}} = \max \left(\left\lceil \frac{\Lambda_{\{\mathrm{req}\}}}{\mu_{\{\mathrm{req}\}}} \right\rceil, \left\lceil \frac{\Lambda_{\{\mathrm{in}\}}}{\mu_{\{\mathrm{in}\}}} \right\rceil, \left\lceil \frac{\Lambda_{\{\mathrm{out}\}}}{\mu_{\{\mathrm{out}\}}} \right\rceil, \left\lceil \frac{M_{\{\mathrm{kv}, \mathrm{req}\}}}{N_{\{\mathrm{kv}, \mathrm{one}\}}} \right\rceil \right)$$

其中， $\mu_{\{\mathrm{req}\}}$ 、 $\mu_{\{\mathrm{in}\}}$ 、 $\mu_{\{\mathrm{out}\}}$ 是单实例请求、输入 token 和输出 token 能力， $N_{\{\mathrm{kv}\}}$ 是需要承载的 KV 并发， $N_{\{\mathrm{kv}, \mathrm{one}\}}$ 是单实例 KV 并发上限。

最终系统设计门禁可以写成：

$$G_{\{\mathrm{design}\}} = \mathbf{1} \setminus \left[\min_j C_j \setminus \mathrm{ge} \tau_j \setminus \mathrm{and} T_{\{\mathrm{ttft}, p95\}} \setminus \mathrm{le} B_{\{\mathrm{ttft}\}} \setminus \mathrm{and} T_{\{\mathrm{tpot}, p95\}} \setminus \mathrm{le} B_{\{\mathrm{tpot}\}} \setminus \right]$$

其中, C_j 是各设计维度覆盖率, $R_{\{\mathrm{trace}\}}$ 是请求链路 trace 覆盖率, P_0 是未关闭 P0 风险。这个门禁强调: 系统设计不是组件列表, 而是可验收的工程闭环。

下面这个 0 依赖 Python demo 演示一个 “推理平台系统设计答案审计器”: 输入 toy design answers, 检查是否覆盖需求、请求链路、SLO、路由、runtime、KV、缓存、扩缩容、保护、发布、多租户、观测、成本和 trade-off, 并给出容量估算样例。

```
from math import ceil

CHECKS = [
    "requirement_clarification",
    "request_lifecycle_coverage",
    "slo_token_contract",
    "model_registry_release",
    "router_policy_fit",
    "runtime_prefill_decode",
    "kv_scheduler_capacity",
    "cache_isolation_strategy",
    "autoscaling_capacity_plan",
    "protection_policy_design",
    "release_governance_design",
    "tenant_security_governance",
    "observability_trace_readiness",
    "cost_capacity_governance",
    "tradeoff_boundary_reasoning",
    "inference_platform_design_gate",
]

TRAFFIC = {
    "peak_qps": 42,
    "avg_input_tokens": 1400,
    "avg_output_tokens": 180,
    "p95_input_tokens": 6000,
    "p95_output_tokens": 640,
    "active_sequences": 96,
    "ttft_budget_s": 1.8,
    "tpot_budget_ms": 45,
    "p99_budget_s": 12.0,
}

INSTANCE = {
    "req_capacity": 7,
    "input_tps": 9000,
    "output_tps": 900,
    "kv_concurrency": 18,
    "gpu_memory_gib": 80,
    "kv_memory_fraction": 0.30,
}

MODEL = {
    "layers": 40,
    "kv_heads": 8,
    "head_dim": 128,
```

```

    "bytes_per_element": 2,
    "context_tokens": TRAFFIC["p95_input_tokens"] + TRAFFIC["p95_output_tokens"],
}

```

```

LATENCY = {
    "gateway_s": 0.03,
    "auth_s": 0.02,
    "route_s": 0.04,
    "cache_s": 0.03,
    "queue_s": 0.45,
    "prefill_s": 0.78,
    "tpot_ms": 34,
    "safety_s": 0.08,
    "stream_s": 0.15,
}

```

```

def kv_per_request_mib(model):
    total_bytes = (
        2
        * model["layers"]
        * model["kv_heads"]
        * model["head_dim"]
        * model["bytes_per_element"]
        * model["context_tokens"]
    )
    return round(total_bytes / (1024 ** 2), 1)

```

```

def latency_summary(traffic, latency):
    decode_s = traffic["p95_output_tokens"] * latency["tpot_ms"] / 1000
    ttft_s = (
        latency["gateway_s"]
        + latency["auth_s"]
        + latency["route_s"]
        + latency["cache_s"]
        + latency["queue_s"]
        + latency["prefill_s"]
    )
    e2e_s = ttft_s + decode_s + latency["safety_s"] + latency["stream_s"]
    return round(ttft_s, 3), round(decode_s, 3), round(e2e_s, 3)

```

```

def capacity_plan(traffic, instance):
    input_tps = traffic["peak_qps"] * traffic["avg_input_tokens"]
    output_tps = traffic["peak_qps"] * traffic["avg_output_tokens"]
    by_req = ceil(traffic["peak_qps"] / instance["req_capacity"])
    by_input = ceil(input_tps / instance["input_tps"])
    by_output = ceil(output_tps / instance["output_tps"])
    by_kv = ceil(traffic["active_sequences"] / instance["kv_concurrency"])
    need = max(by_req, by_input, by_output, by_kv)
    return {
        "input_tps": input_tps,
        "output_tps": output_tps,
        "by_req": by_req,
    }

```

```

        "by_input": by_input,
        "by_output": by_output,
        "by_kv": by_kv,
        "replicas_needed": need,
    }

def kv_gate(model, instance, active_sequences_per_replica):
    kv_mib = kv_per_request_mib(model)
    available_mib = instance["gpu_memory_gib"] * 1024 * instance["kv_memory_fraction"]
    used_mib = kv_mib * active_sequences_per_replica
    return {
        "kv_per_request_mib": kv_mib,
        "available_kv_mib": round(available_mib, 1),
        "used_kv_mib": round(used_mib, 1),
        "kv_gate_pass": used_mib <= available_mib,
    }

def build_design_cases():
    complete = {"name": "complete_inference_platform_design"}
    complete.update({check: True for check in CHECKS})
    cases = [complete]
    bad_names = [
        "requirements_missing_bad",
        "request_lifecycle_missing_bad",
        "slo_token_contract_missing_bad",
        "model_registry_release_missing_bad",
        "router_policy_missing_bad",
        "runtime_phase_missing_bad",
        "kv_scheduler_missing_bad",
        "cache_isolation_missing_bad",
        "autoscaling_single_metric_bad",
        "protection_policy_missing_bad",
        "release_governance_missing_bad",
        "tenant_security_missing_bad",
        "observability_trace_missing_bad",
        "cost_capacity_missing_bad",
        "tradeoff_boundary_missing_bad",
        "design_gate_missing_bad",
    ]
    for bad_name, failed_check in zip(bad_names, CHECKS):
        case = {"name": bad_name}
        case.update({check: True for check in CHECKS})
        case[failed_check] = False
        cases.append(case)
    return cases

def audit_cases(cases):
    metrics = {
        check: round(sum(1 for case in cases if case.get(check)) / len(cases), 3)
        for check in CHECKS
    }
    failed_cases = [case["name"] for case in cases if not all(case.get(check) for check in CHECKS)]

```

```

failed_gates = [check for check, value in metrics.items() if value < 1.0]
hard_blockers = sum(1 for case in cases if case["name"].endswith("_bad"))
gate_pass = min(metrics.values()) >= 0.95 and hard_blockers == 0
return metrics, failed_cases, failed_gates, hard_blockers, gate_pass

ttft_s, decode_s, e2e_s = latency_summary(TRAFFIC, LATENCY)
capacity = capacity_plan(TRAFFIC, INSTANCE)
kv = kv_gate(MODEL, INSTANCE, active_sequences_per_replica=ceil(TRAFFIC["active_sequences"] / capacity["replicas_needed"]))
metrics, failed_cases, failed_gates, hard_blockers, gate_pass = audit_cases(build_design_cases())

print("latency_summary=", {"ttft_s": ttft_s, "decode_s": decode_s, "e2e_s": e2e_s})
print("capacity_plan=", capacity)
print("kv_gate=", kv)
print("slo_pass=", {
    "ttft": ttft_s <= TRAFFIC["ttft_budget_s"],
    "tpot": LATENCY["tpot_ms"] <= TRAFFIC["tpot_budget_ms"],
    "p99": e2e_s <= TRAFFIC["p99_budget_s"],
})
print("metrics=", metrics)
print("hard_blocker_count=", hard_blockers)
print("failed_case_count=", len(failed_cases))
print("failed_gate_count=", len(failed_gates))
print("failed_case_sample=", failed_cases[:4])
print("failed_gate_sample=", failed_gates[:4])
print("inference_platform_design_gate_pass=", gate_pass)

```

输出示例:

```

latency_summary= {'ttft_s': 1.35, 'decode_s': 21.76, 'e2e_s': 23.34}
capacity_plan= {'input_tps': 58800, 'output_tps': 7560, 'by_req': 6, 'by_input': 7, 'by_output': 9, 'by_kv': 6, 'replicas_needed': 10}
kv_gate= {'kv_per_request_mib': 1037.5, 'available_kv_mib': 24576.0, 'used_kv_mib': 11412.5, 'kv_gate_pass': True}
slo_pass= {'ttft': True, 'tpot': True, 'p99': False}
metrics= {'requirement_clarification': 0.941, 'request_lifecycle_coverage': 0.941, 'slo_token_contract': 0.941, 'model_registry': 0.941}
hard_blocker_count= 16
failed_case_count= 16
failed_gate_count= 16
failed_case_sample= ['requirements_missing_bad', 'request_lifecycle_missing_bad', 'slo_token_contract_missing_bad', 'model_registry_missing_bad']
failed_gate_sample= ['requirement_clarification', 'request_lifecycle_coverage', 'slo_token_contract', 'model_registry']
inference_platform_design_gate_pass= False

```

这个 demo 的重点不是给生产容量结论, 而是训练面试中的自查习惯: 如果答案没有 token 分布、TTFT/TPOT、KV 容量、端到端 trace、扩缩容、保护策略、发布治理和成本归因, 就不是一个完整的推理平台系统设计答案。示例里 TTFT 和 TPOT 过线, 但 p99 不过线, 说明可以继续追问长输出、decode 并发、输出长度限制、流式体验、speculative decoding 或能力路由, 而不是只说 “加机器”。

35.20 常见扣分回答

扣分回答一: 只说 K8s 部署模型。

问题: 忽略 runtime、KV cache、路由、缓存、SLO、扩缩容和成本治理。

扣分回答二: 只看 QPS 扩容。

问题: 不理解 token 负载、长上下文和 decode 瓶颈。

扣分回答三: 失败就重试。

问题: 不理解大模型请求成本、非幂等输出和 streaming 特性。

扣分回答四：灰度只看错误率。

问题：模型发布还要看质量、安全、成本、延迟和业务指标。

扣分回答五：缓存不考虑权限。

问题：语义缓存、结果缓存和 prompt cache 都可能引入跨租户数据泄露。

35.21 面试官可能继续追问

追问一：如果 p99 延迟突然升高，你怎么排查？

回答方向：先拆分 trace，看 gateway、router、queue、prefill、decode、streaming 哪一段变慢；再看 input/output token 分布、queue length、KV cache、GPU、runtime 错误和是否有长请求或租户突增。

追问二：如果 GPU 利用率不高但请求很慢，可能是什么原因？

回答方向：可能是队列调度、KV cache、显存带宽、长上下文、下游工具、streaming 网络、batch 配置或 admission control 问题，不一定是算力不足。

追问三：如何降低推理成本？

回答方向：小模型优先、能力路由、缓存、prompt 压缩、输出长度控制、量化、batching、自动扩缩容、离峰任务和按租户成本归因。

追问四：如何支持企业多租户？

回答方向：认证鉴权、模型访问控制、租户配额、限流、数据隔离、缓存隔离、日志脱敏、审计、成本归因和租户级 SLO。

35.22 小练习

1. 设计推理平台时，需求澄清应该问哪些问题？
2. 为什么推理平台系统设计必须讲 prefill、decode 和 KV cache？
3. Model Router 的决策依据有哪些？
4. 推理平台为什么不能只靠 K8s HPA？
5. 缓存层如何避免跨租户数据泄露？
6. 大模型发布为什么要同时管理 prompt 和 runtime 配置？
7. 如果 TTFT 升高，你会如何排查？
8. 如果成本突然升高，你会如何定位原因？
9. 写一个 0 依赖推理平台系统设计审计 demo，输入 toy design answers，输出需求覆盖、链路覆盖、SLO、容量估算、KV 门禁、失败设计和最终 design gate。

35.23 本章小结

本章用系统设计面试题串起了整个推理平台工程。

你需要记住：

1. 推理平台不是一个模型 HTTP 服务，而是在线模型服务、GPU runtime 和平台治理的组合。
2. 系统设计要从需求、架构、请求链路、runtime、路由、缓存、扩缩容、稳定性、发布、安全、观测和成本展开。
3. Prefill、decode、KV cache、continuous batching 和 token 负载是 LLM serving 的核心差异点。
4. 多模型平台必须支持模型路由、灰度、回滚、缓存、多租户和成本归因。
5. 好的面试答案要体现 trade-off：质量、延迟、吞吐、成本、安全和可用性不能同时无条件最优。
6. 系统设计答案要能通过 design gate：需求、链路、SLO、容量、KV、路由、缓存、扩缩容、保护、发布、租户、安全、观测、成本和取舍都要可验证。

下一章我们会进入第五部分：数据、模型与实验平台，先讲模型注册中心与模型资产管理。

第 36 章 数据平台与训练数据供给链路

前面第四部分讲完了大模型推理平台工程。从本章开始进入第五部分：数据、模型与实验平台。

训练平台和推理平台解决的是“算力怎么用、服务怎么跑”。但大模型能力的底层基础仍然是数据。没有稳定的数据平台，训练任务会因为数据找不到、版本不清、质量不稳、吞吐不足和权限不明而反复失败。

先记住一句话：

大模型数据平台不是简单存放数据的目录，而是围绕数据采集、清洗、治理、版本、质量、权限、供给和血缘建立的工程系统。

36.0 本讲资料边界与第二轮精修口径

本讲按截至 2026-06 的稳定公开资料校准：Apache Iceberg specification 里的 snapshot、manifest 和 meta-data model, Apache Parquet file format 对列式文件、schema 和 metadata 的抽象, Hugging Face Datasets 对 streaming / IterableDataset / sharding / shuffle 的说明, PyTorch DataLoader、IterableDataset 和 DistributedSampler 的数据读取接口，以及 OpenLineage 的 Job、Run、Dataset 事件模型。

本讲只抽象大模型数据平台与训练数据供给链路的通用工程问题，不绑定某个湖仓表格式、对象存储、调度器、数据仓库、数据编排系统或内部平台实现。

还要区分两个层次：

1. 第 23 章训练数据供给侧重训练任务前后的 manifest、shard、streaming、cache、shuffle、rank 切分、DataLoader 和 H2D copy。
2. 本章数据平台侧重更上游的数据源注册、原始湖、清洗策略、去重污染隔离、PII 脱敏、质量评分、dataset builder、版本、权限、血缘和供给门禁。

也就是说，第 23 章回答“训练任务怎样稳定读到这批数据”，本章回答“这批数据为什么可信、可复现、可审计、可授权、可追溯，并且能被训练平台稳定消费”。

36.1 为什么需要数据平台

在小实验中，数据可能只是一个本地文件：

data/train.jsonl

但大模型训练中，数据规模和复杂度完全不同：

1. 数据量从 GB 到 TB、PB。
2. 数据来源包括网页、代码、文档、对话、图片、音频、视频、业务数据。
3. 数据需要清洗、去重、过滤、脱敏和标注。
4. 数据版本要可复现。
5. 数据质量要持续监控。
6. 数据权限要严格控制。
7. 训练时要高吞吐供给 GPU。

如果没有数据平台，训练会变成“谁知道这批数据从哪里来”的黑盒流程。

36.2 数据平台要解决什么问题

数据平台要解决八类问题：

1. 数据从哪里来。
2. 数据如何存。
3. 数据如何清洗。
4. 数据如何标注。
5. 数据如何版本化。
6. 数据质量如何监控。
7. 训练如何高吞吐读取。

8. 数据权限和血缘如何管理。

大模型训练中的很多问题看似是训练问题，本质上是数据问题。

比如 loss 异常，可能不是优化器坏了，而是某批数据重复、污染、格式错误或分布变化。

36.3 训练数据供给链路总览

一个典型训练数据供给链路：

Raw Data Sources

- > Ingestion
- > Raw Data Lake
- > Cleaning / Filtering
- > Deduplication
- > PII Redaction
- > Quality Scoring
- > Dataset Builder
- > Dataset Version Registry
- > Object Storage / File System
- > Streaming / Sharding / Cache
- > Training Job

每一步都可能影响最终模型质量。

数据平台的目标是让这条链路可复现、可观测、可治理。

36.4 数据来源

大模型数据来源常见包括：

1. 公开网页。
2. 书籍和百科。
3. 代码仓库。
4. 学术论文。
5. 对话数据。
6. 人工标注数据。
7. 合成数据。
8. 企业文档。
9. 产品日志。
10. 多模态数据。

不同来源的风险不同。

公开网页规模大但噪声多。

企业文档价值高但权限和隐私敏感。

合成数据可控但可能放大模型偏差。

代码数据结构清晰但许可证复杂。

进入平台后的一个数据资产可以抽象为：

$d_i = (s_i, r_i, c_i, v_i, m_i, q_i, p_i, g_i, l_i, t_i, h_i, o_i, z_i)$

其中 s_i 是 source, r_i 是 raw object, c_i 是清洗策略, v_i 是 dataset version, m_i 是 manifest, q_i 是质量统计, p_i 是权限和用途, g_i 是治理规则, l_i 是 lineage, t_i 是 token 或媒体统计, h_i 是 checksum, o_i 是 owner, z_i 是最终供给状态。

36.5 数据接入 Ingestion

数据接入负责把外部数据进入平台。

接入系统要记录：

1. 数据来源。
2. 抓取时间。
3. 原始路径。
4. 数据格式。
5. 许可证或使用约束。
6. 所属租户或业务。
7. 权限等级。
8. 接入任务 ID。
9. 校验和。

不要只把文件丢进对象存储。

数据从进入平台开始就要有元数据。

数据源注册覆盖率可以写成：

$$C_{\{\mathrm{source}\}} = \frac{K_{\{\mathrm{source}, \mathrm{ok}\}}}{K_{\{\mathrm{source}, \mathrm{total}\}}}$$

$K_{\{\mathrm{source}, \mathrm{total}\}}$ 是必须登记的来源字段数量， $K_{\{\mathrm{source}, \mathrm{ok}\}}$ 是已经完整登记并通过校验的字段数量。来源、owner、许可证、用途、分类、接入时间和 checksum 任一缺失，都会让后续版本与血缘失真。

36.6 Raw Data Lake

Raw Data Lake 保存原始数据。

原始数据通常不直接用于训练，但必须保留，因为后续清洗策略可能变化，需要回溯。

Raw Data Lake 的要求：

1. 原始数据不可随意覆盖。
2. 写入后有校验。
3. 支持按来源、时间、租户检索。
4. 支持权限控制。
5. 支持生命周期管理。
6. 支持审计。

保留 raw data 的价值是：当清洗规则或质量问题出现时，可以重新构建数据集。

Raw Data Lake 的关键不是“有一份原始文件”，而是原始对象、checksum、写入事件和权限策略不可随意漂移。可以用 raw lake 完整率描述：

$$C_{\{\mathrm{raw}\}} = \frac{N_{\{\mathrm{raw}, \mathrm{verified}\}}}{N_{\{\mathrm{raw}, \mathrm{total}\}}}$$

36.7 数据清洗

数据清洗是把原始数据变成可训练数据的重要步骤。

常见清洗包括：

1. 格式解析。
2. 编码规范化。
3. HTML / boilerplate 去除。
4. 语言识别。
5. 低质量文本过滤。
6. 重复内容去除。
7. 乱码过滤。
8. 过短或过长样本过滤。

9. 敏感信息脱敏。
10. 不合规内容过滤。

清洗不是越狠越好。

过度清洗可能损失多样性；清洗不足会带来噪声和安全风险。

清洗保留率可以写成：

$$R_{\{\mathrm{keep}\}} = \frac{N_{\{\mathrm{kept}\}}}{N_{\{\mathrm{raw}\}}}$$

这个指标本身没有固定“越高越好”或“越低越好”。它要和语言、领域、来源、质量分、风险命中率一起看。如果某个来源保留率突然从 0.7 变成 0.1，可能是数据分布变了，也可能是清洗规则误伤。

36.8 去重 Deduplication

去重大模型非常关键。

重复数据会导致：

1. 训练效率下降。
2. 模型记忆重复内容。
3. 评估泄漏风险增加。
4. 数据分布被重复样本扭曲。
5. 成本浪费。

去重可以分为：

1. 精确去重。
2. 近似去重。
3. 文档级去重。
4. 段落级去重。
5. 样本级去重。
6. train/eval 跨集合去重。

跨集合去重非常重要，否则 eval 集出现在 train 集里，会让评估结果虚高。

重复率可以写成：

$$R_{\{\mathrm{dup}\}} = \frac{N_{\{\mathrm{duplicate}\}}}{N_{\{\mathrm{raw}\}}}$$

污染隔离还要单独看 train / eval overlap：

$$R_{\{\mathrm{overlap}\}} = \frac{N_{\{\mathrm{train,eval,overlap}\}}}{N_{\{\mathrm{eval}\}}}$$

预训练数据很大时，重复率下降不等于污染风险清零。评估集、benchmark、线上灰度样本和高价值私有集都要单独做隔离。

36.9 PII 脱敏和安全过滤

PII 是 personally identifiable information，也就是个人可识别信息。

数据平台要检测和处理：

1. 手机号。
2. 邮箱。
3. 身份证件号。
4. 地址。
5. 银行卡。
6. 账号密钥。
7. 企业内部敏感信息。
8. 私有代码或凭据。

处理方式包括：

1. 删除样本。
2. 脱敏替换。
3. 权限隔离。
4. 标记敏感等级。
5. 禁止进入特定训练集。

敏感数据一旦进入模型训练，后续风险很难彻底消除。

PII 泄露率可以写成：

$$R_{\{\mathrm{pii}, \text{leak}\}} = \frac{N_{\{\mathrm{pii}, \text{unlocked}\}}}{N_{\{\mathrm{pii}, \text{total}\}}}$$

对 restricted 或 regulated 数据， $R_{\{\mathrm{pii}, \text{leak}\}}$ 的目标通常不是“尽量低”，而是进入训练集前必须为 0 或触发人工审批和隔离流程。

36.10 数据质量评分

数据质量不是一个单一指标。

常见质量维度：

1. 可读性。
2. 信息密度。
3. 语言质量。
4. 格式完整性。
5. 重复率。
6. 毒性和安全风险。
7. 主题分布。
8. 领域覆盖。
9. 标注一致性。
10. 许可证风险。

平台可以为样本或文档打质量分，并在 dataset builder 中按策略采样。

质量评分的目标不是把数据分成好坏两类，而是支持可控的数据配方。

质量评分要版本化。一个样本的质量分可以写成：

$$q_i = f_{\{\phi\}}(d_i, u_i)$$

其中 u_i 是用途或训练阶段， $f_{\{\phi\}}$ 是带版本的质量评分策略。预训练、SFT、安全、代码、数学、多模态数据的质量函数不应混成一个不可解释总分。

36.11 Dataset Builder

Dataset Builder 负责把清洗后的数据组合成训练数据集。

它通常支持：

1. 选择数据来源。
2. 设置采样比例。
3. 按质量分过滤。
4. 按语言过滤。
5. 按领域过滤。
6. 设置最大样本数。
7. 设置 train/val/test 切分。
8. 输出统一格式。
9. 生成 dataset manifest。

数据集不是一堆文件，而是一个有配方、有版本、有 manifest 的训练输入。

Dataset Builder 的配方可以抽象为：

$D_v = B(R, C, A, S, \omega)$

R 是原始或中间数据集，C 是清洗策略，A 是审计和权限约束，S 是采样与切分策略， ω 是各来源或各 slice 的混合权重。只要其中任一项变化，就应该生成新的 dataset version。

36.12 Dataset Manifest

Dataset manifest 是描述数据集内容的元数据文件。

它可以包含：

1. dataset 名称。
2. dataset 版本。
3. 数据来源列表。
4. 文件路径列表。
5. 样本数量。
6. token 数量估计。
7. 采样比例。
8. 清洗规则版本。
9. 去重规则版本。
10. 数据质量统计。
11. 权限等级。
12. checksum。

训练任务应该引用 dataset version 或 manifest，而不是随便引用一个目录。

Manifest 完整率可以写成：

$$C_{\{\mathrm{manifest}\}} = \frac{K_{\{\mathrm{manifest}, \mathrm{filled}\}}}{K_{\{\mathrm{manifest}, \mathrm{required}\}}}$$

如果 manifest 里没有来源、清洗版本、去重版本、shard checksum、token 数、权限等级和 lineage run id，训练任务即使跑通，也很难复现或审计。

36.13 数据格式

大模型训练常见格式：

1. JSONL。
2. Parquet。
3. Arrow。
4. WebDataset。
5. TFRecord。
6. 自定义二进制格式。

JSONL 易读易调试，但大规模读取效率不一定高。

Parquet 和 Arrow 更适合列式存储和大规模处理。

WebDataset 常用于多模态数据。

格式选择要考虑读取吞吐、压缩、随机访问、schema 演进和调试便利性。

36.14 Tokenization 在数据链路中的位置

Tokenization 可以在线做，也可以离线做。

在线 tokenization：训练时读取文本再分词。

优点：灵活，tokenizer 更新方便。

缺点：训练时 CPU 开销和延迟更高。

离线 tokenization：提前把文本转成 token IDs。

优点：训练读取更快。

缺点：tokenizer 版本绑定强，存储成本增加。

大规模预训练常会考虑离线 tokenization 或半离线 tokenization，以降低训练时数据处理瓶颈。

36.15 Sharding

Sharding 是把大数据集切成多个 shard。

它的作用：

1. 支持并行读取。
2. 降低单文件压力。
3. 提高分布式训练吞吐。
4. 支持 shuffle。
5. 便于断点恢复。

Shard 设计要避免：

1. shard 大小差异太大。
2. 样本分布不均。
3. 热点 shard。
4. 小文件过多。
5. shuffle 不充分。

训练数据供给不是只要能读，而是要持续喂饱 GPU。

36.16 数据读取链路

训练时的数据读取链路可能是：

Object Storage / Data Lake

- > Data Loader Workers
- > Local Cache
- > Shuffle Buffer
- > Tokenizer / Decoder
- > Batch Collator
- > GPU Training Process

瓶颈可能出现在：

1. 对象存储吞吐。
2. 网络带宽。
3. 小文件过多。
4. 解压缩。
5. tokenization。
6. shuffle buffer。
7. CPU dataloader。
8. 本地缓存容量。

如果数据供给跟不上，GPU 会等待数据，训练效率下降。

有效读取吞吐可以写成：

$$Q_{\mathrm{read}} = \frac{N_{\mathrm{tok}}}{T_{\mathrm{read}}}$$

GPU 等待数据比例可以写成：

$$R_{\mathrm{wait}} = \frac{T_{\mathrm{wait, data}}}{T_{\mathrm{step}}}$$

数据平台不能只承诺“对象存储能读”，还要证明 Q_{read} 高于训练消耗速度，并且 R_{wait} 不会让 GPU 长时间空转。

36.17 数据缓存

训练数据缓存可以放在：

1. 本地 NVMe。
2. 分布式缓存。
3. 节点级缓存。
4. 任务级缓存。
5. 共享文件系统。

缓存目标：

1. 降低对象存储重复读取。
2. 提高 epoch 内读取速度。
3. 降低网络压力。
4. 支持任务重启。

但缓存要注意一致性和版本。

不要让训练任务误读旧版本数据。

36.18 Shuffle 设计

Shuffle 对训练很重要。

Shuffle 不充分会导致：

1. batch 分布偏移。
2. 训练不稳定。
3. loss 曲线异常。
4. 模型过度适应数据顺序。

大规模数据下，完全全局 shuffle 成本很高。

常见做法：

1. shard-level shuffle。
2. shard 内 shuffle。
3. sliding shuffle buffer。
4. 多 epoch 改变 shard 顺序。
5. 按 worker 分片后局部 shuffle。

Shuffle 是训练质量和数据吞吐之间的权衡。

36.19 数据权限

数据平台必须做权限控制。

权限维度包括：

1. 谁能看 raw data。
2. 谁能使用某个 dataset。
3. 哪些训练任务可以读取敏感数据。
4. 哪些模型可以用某类数据训练。
5. 数据是否允许导出。
6. 数据是否允许跨租户使用。

训练任务提交时，平台应该校验用户或服务账号是否有 dataset 访问权限。

不能因为训练跑在集群里，就默认能读所有数据。

36.20 数据血缘

数据血缘回答：某个模型是由哪些数据训练出来的。

至少记录：

1. 原始数据来源。
2. 清洗任务。
3. 清洗规则版本。
4. 去重规则版本。
5. dataset builder 配方。
6. dataset version。
7. 训练任务 ID。
8. checkpoint。
9. eval report。
10. deployment version。

当模型出现安全问题、版权问题或质量问题时，血缘可以帮助追溯原因。

数据血缘覆盖率可以写成：

$$C_{\{\mathrm{lineage}\}} = \frac{N_{\{\mathrm{samples, linked}\}}}{N_{\{\mathrm{samples, total}\}}}$$

覆盖率低时，平台可能知道某个 dataset version 的路径，却无法回答某个样本来自哪个源、经过哪版清洗规则、进入哪个 shard、被哪些训练任务消费。

36.21 数据供给和训练平台的接口

训练平台不应该直接拼文件路径。

更好的接口是：

```
TrainingJob
dataset:
  name: pretrain_mix
  version: 2026-05-30-v1
  split: train
  access_mode: streaming
  shuffle: true
```

训练平台通过 dataset registry 解析出实际文件、权限、格式和读取方式。

这样训练任务更可复现，也更容易审计。

36.22 数据平台供给链路审计指标与最小 demo

把前面的概念收束成门禁，可以写成：

$$G_{\{\mathrm{data}\}} = \mathbf{1} \setminus \left[\min_j C_j \geq \tau_j \ \wedge \ R_{\{\mathrm{pii, leak}\}} = 0 \ \wedge \ C_{\{\mathrm{lineage}\}} \geq \rho \right]$$

这里 C_j 是各个审计项覆盖率， $Q_{\{\mathrm{train}\}}$ 是训练侧 token 消耗速度， P_0 是 P0 级硬阻断数量。这个门禁强调：数据平台合格不只是“文件存在”，还要同时证明来源完整、原始数据可回溯、清洗去重可复现、隐私风险被阻断、manifest 完整、版本不可变、权限和用途绑定、血缘覆盖、读取吞吐和训练平台接口都可验收。

下面是一个 0 依赖 toy demo。它故意构造一个完整样本和 16 个一项失败的 bad case，用来把数据平台供给链路拆成可审计指标。

```
from copy import deepcopy
from pprint import pprint
```

```
SOURCE_FIELDS = [
```

```

    "source_id",
    "owner",
    "license",
    "classification",
    "purpose",
    "checksum",
    "ingested_at",
    "raw_uri",
]

MANIFEST_FIELDS = [
    "dataset_id",
    "version",
    "sources",
    "recipe_version",
    "cleaning_version",
    "dedup_version",
    "pii_policy_version",
    "shards",
    "sample_count",
    "token_count",
    "checksum",
    "permission_policy_version",
    "lineage_run_ids",
    "quality_stats",
]

def complete_case():
    return {
        "name": "complete_case",
        "sources": [
            {
                "source_id": "web_2026_05",
                "owner": "data-platform",
                "license": "allow_train",
                "classification": "public",
                "purpose": ["pretrain"],
                "checksum": "sha256:raw-web",
                "ingested_at": "2026-05-30",
                "raw_uri": "lake/raw/web/2026-05",
            },
            {
                "source_id": "docs_2026_05",
                "owner": "corp-data",
                "license": "internal_train",
                "classification": "restricted",
                "purpose": ["sft"],
                "checksum": "sha256:raw-docs",
                "ingested_at": "2026-05-30",
                "raw_uri": "lake/raw/docs/2026-05",
            },
        ],
        "raw_lake": {
            "immutable": True,

```

```

    "checksum_coverage": 1.0,
    "retention_policy": "versioned",
    "audit_log": True,
},
"cleaning": {
    "policy_version": "clean_policy_v3",
    "recipe_version": "clean_recipe_v7",
    "raw_count": 2000,
    "kept_count": 1600,
},
"dedup": {
    "rule_version": "dedup_v4",
    "duplicate_count": 200,
    "train_eval_overlap_count": 0,
},
"pii": {
    "policy_version": "pii_v5",
    "detected": 48,
    "blocked_or_redacted": 48,
    "unblocked": 0,
},
"quality": {
    "score_version": "quality_v6",
    "calibration_report": True,
    "slice_stats": ["language", "domain", "source"],
},
"builder": {
    "dataset_id": "pretrain_mix",
    "version": "2026-05-30-v1",
    "recipe_version": "mix_recipe_v2",
    "frozen": True,
},
"manifest": {
    "dataset_id": "pretrain_mix",
    "version": "2026-05-30-v1",
    "sources": ["web_2026_05", "docs_2026_05"],
    "recipe_version": "mix_recipe_v2",
    "cleaning_version": "clean_recipe_v7",
    "dedup_version": "dedup_v4",
    "pii_policy_version": "pii_v5",
    "shards": ["shard-000.parquet", "shard-001.parquet"],
    "sample_count": 1600,
    "token_count": 1200000,
    "checksum": "sha256:manifest",
    "permission_policy_version": "perm_v2",
    "lineage_run_ids": ["ingest_1", "clean_1", "dedup_1", "builder_1"],
    "quality_stats": {"avg_score": 0.86},
},
"shards": [
    {
        "path": "s3://data/pretrain_mix/shard-000.parquet",
        "format": "parquet",
        "size_mib": 512,
        "checksum": "sha256:shard0",
        "sample_count": 800,
    }
]

```

```

        "token_count": 600000,
    },
    {
        "path": "s3://data/pretrain_mix/shard-001.parquet",
        "format": "parquet",
        "size_mib": 512,
        "checksum": "sha256:shard1",
        "sample_count": 800,
        "token_count": 600000,
    },
],
"reader": {
    "streaming": True,
    "shuffle_seed": 3407,
    "prefetch": 4,
    "retry": 3,
    "local_cache_versioned": True,
    "tokens_to_read": 1200000,
    "estimated_read_seconds": 1.2,
    "train_tokens_per_s": 800000,
    "data_wait_seconds": 0.06,
    "step_seconds": 1.0,
},
"permissions": [
    {
        "actor": "trainer",
        "purpose": "pretrain",
        "dataset_version": "2026-05-30-v1",
        "allowed": True,
    },
    {
        "actor": "analyst",
        "purpose": "export_raw",
        "dataset_version": "2026-05-30-v1",
        "allowed": False,
    },
],
"lineage": {
    "sample_count": 1600,
    "linked_samples": 1568,
    "run_ids": ["ingest_1", "clean_1", "dedup_1", "builder_1"],
    "openlineage_events": True,
},
"observability": {
    "metrics": [
        "source_coverage",
        "manifest_completeness",
        "pii_leak_rate",
        "lineage_coverage",
        "read_tokens_per_s",
        "data_wait_ratio",
    ],
    "alerts": ["pii_leak", "throughput_drop"],
    "dashboard": True,
},

```

```

        "data_platform_gate": True,
    }

def ratio(num, den):
    return 0.0 if den == 0 else num / den

def source_registry_coverage(case):
    required = len(case["sources"]) * len(SOURCE_FIELDS)
    filled = sum(1 for src in case["sources"] for field in SOURCE_FIELDS if src.get(field))
    return ratio(filled, required) == 1.0

def raw_lake_integrity(case):
    raw = case["raw_lake"]
    return raw["immutable"] and raw["checksum_coverage"] == 1.0 and raw["audit_log"]

def cleaning_policy_versioning(case):
    clean = case["cleaning"]
    keep_rate = ratio(clean["kept_count"], clean["raw_count"])
    return bool(clean["policy_version"]) and bool(clean["recipe_version"]) and 0.2 <= keep_rate <= 0.95

def dedup_contamination_guard(case):
    dedup = case["dedup"]
    dup_rate = ratio(dedup["duplicate_count"], case["cleaning"]["raw_count"])
    return bool(dedup["rule_version"]) and dup_rate <= 0.3 and dedup["train_eval_overlap_count"] == 0

def pii_redaction_coverage(case):
    pii = case["pii"]
    return bool(pii["policy_version"]) and pii["unblocked"] == 0 and pii["blocked_or_redacted"] == pii["detected"]

def quality_scoring_calibration(case):
    quality = case["quality"]
    return bool(quality["score_version"]) and quality["calibration_report"] and len(quality["slice_stats"]) >= 3

def dataset_builder_recipe(case):
    builder = case["builder"]
    return bool(builder["dataset_id"]) and bool(builder["version"]) and bool(builder["recipe_version"])

def dataset_manifest_completeness(case):
    manifest = case["manifest"]
    return all(manifest.get(field) for field in MANIFEST_FIELDS)

def dataset_version_immutability(case):
    builder = case["builder"]
    manifest = case["manifest"]
    return builder["frozen"] and builder["version"] == manifest["version"] and bool(manifest.get("checksum"))

```

```

def streaming_shard_readiness(case):
    reader = case["reader"]
    shards = case["shards"]
    shard_ok = all(s["format"] in {"parquet", "arrow", "webdataset"} and s["size_mib"] >= 128 and s["checksum"] for s in shards)
    return reader["streaming"] and reader["prefetch"] > 0 and reader["retry"] > 0 and reader["local_cache_versioned"]

def permission_purpose_binding(case):
    version = case["builder"]["version"]
    allowed = [p for p in case["permissions"] if p["allowed"]]
    denied = [p for p in case["permissions"] if not p["allowed"]]
    allowed_ok = all(p["dataset_version"] == version and p["purpose"] in {"pretrain", "sft"} for p in allowed)
    return allowed_ok and len(denied) >= 1

def data_lineage_coverage(case):
    lineage = case["lineage"]
    coverage = ratio(lineage["linked_samples"], lineage["sample_count"])
    return coverage >= 0.95 and lineage["openlineage_events"] and len(lineage["run_ids"]) >= 4

def supply_observability(case):
    obs = case["observability"]
    needed = {"pii_leak_rate", "lineage_coverage", "read_tokens_per_s", "data_wait_ratio"}
    return obs["dashboard"] and needed.issubset(set(obs["metrics"])) and len(obs["alerts"]) >= 2

def read_throughput_fit(case):
    reader = case["reader"]
    read_tps = ratio(reader["tokens_to_read"], reader["estimated_read_seconds"])
    return read_tps >= 1.1 * reader["train_tokens_per_s"]

def data_wait_fit(case):
    reader = case["reader"]
    return ratio(reader["data_wait_seconds"], reader["step_seconds"]) <= 0.1

def data_platform_supply_gate(case):
    return case["data_platform_gate"] is True

```

```

CHECKS = [
    ("source_registry_coverage", source_registry_coverage),
    ("raw_lake_integrity", raw_lake_integrity),
    ("cleaning_policy_versioning", cleaning_policy_versioning),
    ("dedup_contamination_guard", dedup_contamination_guard),
    ("pii_redaction_coverage", pii_redaction_coverage),
    ("quality_scoring_calibration", quality_scoring_calibration),
    ("dataset_builder_recipe", dataset_builder_recipe),
    ("dataset_manifest_completeness", dataset_manifest_completeness),
    ("dataset_version_immutability", dataset_version_immutability),
    ("streaming_shard_readiness", streaming_shard_readiness),

```

```

("permission_purpose_binding", permission_purpose_binding),
("data_lineage_coverage", data_lineage_coverage),
("supply_observability", supply_observability),
("read_throughput_fit", read_throughput_fit),
("data_wait_fit", data_wait_fit),
("data_platform_supply_gate", data_platform_supply_gate),
]

def make_bad_cases(base):
    mutations = []

    def add(name, change):
        case = deepcopy(base)
        case["name"] = name
        change(case)
        mutations.append(case)

    add("source_registry_missing_bad", lambda c: c["sources"][0].pop("license"))
    add("raw_lake_mutable_bad", lambda c: c["raw_lake"].update({"immutable": False}))
    add("cleaning_policy_unversioned_bad", lambda c: c["cleaning"].update({"policy_version": ""}))
    add("dedup_overlap_bad", lambda c: c["dedup"].update({"train_eval_overlap_count": 3}))
    add("pii_leak_bad", lambda c: c["pii"].update({"unblocked": 2}))
    add("quality_uncalibrated_bad", lambda c: c["quality"].update({"calibration_report": False}))
    add("builder_recipe_missing_bad", lambda c: c["builder"].update({"recipe_version": ""}))
    add("manifest_incomplete_bad", lambda c: c["manifest"].pop("quality_stats"))
    add("dataset_version_mutable_bad", lambda c: c["builder"].update({"frozen": False}))
    add("streaming_shard_not_ready_bad", lambda c: c["reader"].update({"streaming": False}))
    add("permission_purpose_gap_bad", lambda c: c["permissions"][1].update({"allowed": True}))
    add("lineage_gap_bad", lambda c: c["lineage"].update({"linked_samples": 1200}))
    add("observability_missing_bad", lambda c: c["observability"].update({"metrics": ["source_coverage"]})))
    add("read_throughput_low_bad", lambda c: c["reader"].update({"estimated_read_seconds": 2.4}))
    add("data_wait_high_bad", lambda c: c["reader"].update({"data_wait_seconds": 0.2}))
    add("data_platform_gate_missing_bad", lambda c: c.update({"data_platform_gate": False}))
    return mutations

def summarize_examples(case):
    clean = case["cleaning"]
    dedup = case["dedup"]
    pii = case["pii"]
    lineage = case["lineage"]
    reader = case["reader"]
    manifest = case["manifest"]
    permissions = case["permissions"]
    source_total = len(case["sources"]) * len(SOURCE_FIELDS)
    manifest_total = len(MANIFEST_FIELDS)

    return {
        "source_registry_coverage": round(source_total / source_total, 3),
        "manifest_completeness": round(manifest_total / manifest_total, 3),
        "keep_rate": round(ratio(clean["kept_count"], clean["raw_count"]), 3),
        "duplicate_rate": round(ratio(dedup["duplicate_count"], clean["raw_count"]), 3),
        "pii_leak_rate": round(ratio(pii["unblocked"], pii["detected"]), 3),
        "lineage_coverage": round(ratio(lineage["linked_samples"], lineage["sample_count"]), 3),
    }

```

```

        "read_throughput_tokens_s": round(ratio(reader["tokens_to_read"], reader["estimated_read_seconds"]), 1),
        "train_tokens_s": reader["train_tokens_per_s"],
        "data_wait_ratio": round(ratio(reader["data_wait_seconds"], reader["step_seconds"]), 3),
        "permission_allowed_count": sum(1 for p in permissions if p["allowed"]),
        "permission_denied_count": sum(1 for p in permissions if not p["allowed"]),
        "dataset_version": manifest["version"],
    }

```

```

def audit_data_platform(cases):
    per_case = {}
    for case in cases:
        per_case[case["name"]] = {name: check(case) for name, check in CHECKS}

    metrics = {
        name: round(sum(result[name] for result in per_case.values()) / len(cases), 3)
        for name, _ in CHECKS
    }
    failed_cases = [name for name, result in per_case.items() if not all(result.values())]
    failed_gates = [name for name, score in metrics.items() if score < 1.0]

    return {
        "smoke": {
            "complete_case_passes": all(per_case["complete_case"].values()),
            "caught_missing_source": "source_registry_missing_bad" in failed_cases,
            "caught_pii_leak": "pii_leak_bad" in failed_cases,
            "caught_permission_gap": "permission_purpose_gap_bad" in failed_cases,
            "caught_lineage_gap": "lineage_gap_bad" in failed_cases,
            "caught_throughput_gap": "read_throughput_low_bad" in failed_cases,
        },
        "metrics": metrics,
        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "failed_gates": failed_gates,
        "data_platform_gate_pass": len(failed_cases) == 0,
    }

```

```

base = complete_case()
cases = [base] + make_bad_cases(base)
report = audit_data_platform(cases)

pprint({"data_platform_examples": summarize_examples(base)})
pprint(report["smoke"])
pprint(report["metrics"])
print("hard_blocker_count=", report["hard_blocker_count"])
print("failed_cases=", report["failed_cases"])
print("failed_gates=", report["failed_gates"])
print("data_platform_gate_pass=", report["data_platform_gate_pass"])

assert report["smoke"]["complete_case_passes"]
assert report["smoke"]["caught_pii_leak"]
assert report["metrics"]["source_registry_coverage"] == 0.941
assert report["hard_blocker_count"] == 16
assert report["data_platform_gate_pass"] is False

```


这个 demo 的重点不是模拟真实湖仓系统，而是把面试和方案评审里的抽象要求落成检查表：source registry、raw lake、cleaning、dedup、PII、quality、builder、manifest、version、streaming shard、permission、lineage、observability、throughput、data wait 和最终 data platform gate，每一项都能给出 bad case。

36.23 常见故障

数据供给链路常见故障：

1. 数据路径错误。
2. 文件缺失。
3. schema 不一致。
4. tokenizer 版本不匹配。
5. 小文件太多导致读取慢。
6. 对象存储限流。
7. shuffle 不充分。
8. 数据权限失败。
9. 某个 shard 损坏。
10. 数据分布突然变化。
11. eval 数据泄漏到 train。

这些问题都需要指标、日志和数据血缘来定位。

36.24 面试常见追问

问题一：为什么训练任务不应该直接引用数据目录？

可以回答：直接引用目录缺少版本、权限、质量统计和血缘信息，难以复现和审计。应该引用 dataset version 或 manifest。

问题二：数据供给为什么会影响 GPU 利用率？

可以回答：如果数据读取、解压、tokenization、shuffle 或网络吞吐跟不上，GPU 会等待 batch，导致利用率下降。

问题三：为什么要保留 raw data？

可以回答：清洗规则和数据配方会变化，保留 raw data 才能重建数据集、追溯问题和验证数据血缘。

问题四：大模型数据平台为什么要做血缘？

可以回答：模型质量、安全、合规和版权问题都需要追溯训练数据来源、清洗规则、dataset version、训练任务和 checkpoint。

问题五：数据平台供给链路和第 23 章训练数据供给有什么区别？

可以回答：训练数据供给偏训练运行时，重点是 manifest、shard、streaming、cache、shuffle、rank、DataLoader、H2D、坏样本和 checkpoint 数据状态；数据平台供给链路更上游，重点是 source registry、raw lake、清洗策略、去重污染隔离、PII、质量评分、dataset builder、版本不可变、权限用途绑定和全链路 lineage。

36.25 小练习

1. 数据平台和普通文件存储有什么区别？
2. 大模型训练数据供给链路包含哪些步骤？
3. 为什么去重对大模型训练重要？
4. Dataset manifest 应该记录哪些信息？
5. 在线 tokenization 和离线 tokenization 各有什么优缺点？
6. Sharding 和 shuffle 如何影响训练吞吐和质量？
7. 数据权限应该在训练任务哪个阶段校验？
8. 如何追溯一个线上模型使用了哪些训练数据？

9. 写一个 0 依赖数据平台供给链路审计 demo，至少覆盖 source registry、raw lake、cleaning、dedup、PII、quality、builder、manifest、version、streaming shard、permission、lineage、observability、throughput、data wait 和最终 gate。

36.26 本章小结

本章讲了数据平台与训练数据供给链路。

你需要记住：

1. 大模型数据平台不是文件目录，而是数据采集、清洗、治理、版本、质量、权限和供给系统。
2. 训练数据供给链路包括 ingestion、raw data lake、清洗、去重、脱敏、质量评分、dataset builder、版本注册和训练读取。
3. 训练任务应该引用 dataset version 或 manifest，而不是直接引用路径。
4. 数据读取吞吐会直接影响 GPU 利用率和训练效率。
5. 数据权限、血缘和质量监控是企业级数据平台的核心能力。
6. 数据平台供给门禁要同时检查来源、原始湖、清洗、去重、PII、质量、manifest、版本、权限、血缘、吞吐、等待比例和最终硬阻断。

下一章我们会继续讲数据版本管理、数据血缘和数据质量监控。

第 37 章 数据版本管理、数据血缘和数据质量监控

上一章讲了数据平台与训练数据供给链路。本章继续深入三个关键能力：数据版本管理、数据血缘和数据质量监控。

这三个能力决定了一个训练数据平台是否可复现、可追溯、可治理。没有它们，大模型训练就会变成“这次效果好像变差了，但没人知道为什么”。

先记住一句话：

数据平台的核心不是存更多数据，而是让每一次数据使用都能被复现、解释、追溯和监控。

37.0 本讲资料边界与第二轮精修口径

本讲按截至 2026-06 的稳定公开资料校准：Apache Iceberg specification 对 snapshot、manifest list、manifest file 和 metadata 的版本管理抽象，W3C PROV Data Model 对 Entity、Activity、Agent 以及它们关系的 provenance 抽象，OpenLineage 对 Job、Run、Dataset 和 RunEvent 的数据血缘事件模型，Great Expectations 对 expectation、validation 和 checkpoint 的数据质量检查抽象，以及第九册数据治理章节已经整理过的数据版本、datasheet、dataset card 和删除请求口径。

本讲只抽象大模型数据平台里的数据版本、血缘和质量监控能力，不绑定某个湖仓表格式、数据版本工具、数据质量框架、血缘平台、数据目录产品或内部平台实现。

和上一章的分工是：

1. 第 36 章回答“数据如何从来源进入平台，并最终稳定供给训练”。
2. 本章回答“每个数据版本如何被冻结、如何比较差异、如何追踪影响范围、如何持续监控质量并阻止坏版本发布”。

37.1 为什么数据版本管理重要

大模型训练结果高度依赖数据。

如果数据发生变化，即使代码、模型结构和超参都不变，训练结果也可能变化。

常见变化包括：

1. 新增数据源。
2. 删除某批低质量数据。

3. 修改清洗规则。
4. 修改去重规则。
5. 修改采样比例。
6. 修改 train/val/test 切分。
7. 修改 tokenizer。
8. 修复数据格式错误。

如果没有数据版本，就无法判断模型效果变化来自代码、参数还是数据。

37.2 数据版本和代码版本的区别

代码版本通常是 Git commit。

数据版本更复杂，因为数据：

1. 体积巨大。
2. 来源多样。
3. 处理链路长。
4. 可能包含权限和合规约束。
5. 可能持续增量更新。
6. 不适合全部放进 Git。

所以数据版本通常由 manifest、元数据、文件 checksum、处理规则版本和存储路径共同定义。

37.3 一个 Dataset Version 应该包含什么

一个 dataset version 至少要记录：

1. dataset 名称。
2. 版本号。
3. 创建时间。
4. 创建人或任务。
5. 数据来源列表。
6. 清洗规则版本。
7. 去重规则版本。
8. 过滤规则版本。
9. 采样配方。
10. 文件 manifest。
11. 样本数量。
12. token 数量估计。
13. train/val/test split。
14. 数据质量统计。
15. 权限等级。
16. checksum。

训练任务引用 dataset version，而不是引用一个随时可能变化的目录。

一个 dataset version 可以抽象为：

$V_k = (D_k, M_k, R_k, S_k, P_k, Q_k, L_k, H_k, A_k)$

其中 D_k 是数据样本或 shard 集合， M_k 是 manifest， R_k 是清洗、去重、过滤和采样规则版本， S_k 是 schema 与 split， P_k 是权限和用途约束， Q_k 是质量统计， L_k 是 lineage 事件集合， H_k 是 checksum 或内容指纹， A_k 是审计记录。

注意： V_k 不是目录名，而是一组不可变输入和治理证据的组合。

37.4 Manifest 版本化

Manifest 是数据版本管理的核心。

它可以描述：

```
dataset: pretrain_mix
version: 2026-05-30-v1
files:
  - path: s3://bucket/datasets/pretrain_mix/v1/shard-0001.parquet
    checksum: xxx
    samples: 1000000
    tokens: 850000000
  - path: s3://bucket/datasets/pretrain_mix/v1/shard-0002.parquet
    checksum: yyy
    samples: 980000
    tokens: 820000000
recipe:
  web: 40%
  code: 30%
  books: 20%
  qa: 10%
cleaning_rule_version: clean-v17
dedup_rule_version: dedup-v9
```

有了 manifest，训练任务才能精确知道自己读了哪些文件和规则。

Manifest 完整率可以写成：

$$C_{\{\mathrm{manifest}\}} = \frac{K_{\{\mathrm{filled}\}}}{K_{\{\mathrm{required}\}}}$$

Shard checksum 覆盖率可以写成：

$$C_{\{\mathrm{checksum}\}} = \frac{N_{\{\mathrm{shard,checked}\}}}{N_{\{\mathrm{shard,total}\}}}$$

当 $C_{\{\mathrm{manifest}\}} < 1$ 或 $C_{\{\mathrm{checksum}\}} < 1$ 时，训练即使能启动，也不能证明读到的是预期数据。

37.5 不可变数据版本

生产级 dataset version 应该尽量不可变。

也就是说：一旦 pretrain_mix:v1 被发布，就不要悄悄修改它的文件内容。

如果需要修复，发布新版本：

```
pretrain_mix:v1 -> pretrain_mix:v2
```

不可变版本的好处：

1. 训练可复现。
2. 问题可追溯。
3. 审计更清晰。
4. cache 更安全。
5. 多任务共享更可靠。

如果版本名不变但内容变了，后续所有分析都会混乱。

不可变版本可以用内容指纹约束：

$$H_k = h(M_k, D_k, R_k, S_k, P_k)$$

同一个 dataset version 发布后， H_k 不应变化。如果内容、规则、schema、split 或权限快照发生变化，应发布新版本 V_{k+1} ，而不是覆盖 V_k 。

37.6 增量版本

有些数据是持续增长的，例如日志、对话、文档更新和用户反馈。

这时可以使用增量版本：

```
dataset:v1 = base data
dataset:v2 = v1 + delta_2026_05
dataset:v3 = v2 + delta_2026_06 - bad_source_x
```

增量版本要记录：

1. base version。
2. 增加了哪些数据。
3. 删除了哪些数据。
4. 修改了哪些规则。
5. 新版本质量差异。

增量不是简单追加文件，而是受治理的版本演进。

两个版本之间的样本变化可以写成：

$$A_{\{k\}} = \left| D_{\{k\}} \setminus D_{\{k-1\}} \right|$$
$$R_{\{k\}} = \left| D_{\{k-1\}} \setminus D_{\{k\}} \right|$$

新增率和删除率可以写成：

$$r_{\{\mathrm{add}\}} = \frac{|A_{\{k\}}|}{|D_{\{k-1\}}|}, \quad r_{\{\mathrm{remove}\}} = \frac{|R_{\{k\}}|}{|D_{\{k-1\}}|}$$

版本 diff report 至少要解释这些变化来自新增来源、删除请求、清洗规则变化、去重策略变化还是采样配方变化。

37.7 数据血缘是什么

数据血缘是记录数据从来源到使用的完整链路。

它回答这些问题：

1. 某个 dataset 来自哪些原始数据？
2. 经过了哪些清洗和过滤任务？
3. 使用了哪些规则版本？
4. 被哪些训练任务使用？
5. 产生了哪些 checkpoint？
6. checkpoint 经过了哪些评估？
7. 最终部署到了哪些线上模型？

血缘把 raw data、dataset、training job、checkpoint、eval report 和 deployment 串起来。

37.8 数据血缘图

可以把血缘理解为一张图：

```
RawSource
-> IngestionJob
-> RawDataset
-> CleaningJob
-> CleanDataset
-> DedupJob
-> DatasetVersion
-> TrainingJob
-> Checkpoint
-> EvalReport
-> ModelVersion
-> Deployment
```

每个节点都有元数据，每条边表示依赖关系。

有了这张图，平台才能回答“这个线上模型到底吃过哪些数据”。

血缘图可以抽象为有向图：

$$G_{\{\mathrm{lin}\}}=(N,E)$$

节点 N 包含 source、job、dataset version、training run、checkpoint、eval report、model version 和 deployment。边 E 表示“由谁产生”“依赖谁”“被谁消费”。

血缘覆盖率可以写成：

$$C_{\{\mathrm{lineage}\}}=\frac{N_{\{\mathrm{linked}\}}}{N_{\{\mathrm{expected}\}}}$$

这里的 N_{expected} 不是全平台所有节点，而是某个 dataset version 或 model version 追踪链路里应出现的关键节点数量。

37.9 为什么血缘重要

血缘的价值包括：

1. 复现训练。
2. 定位质量问题。
3. 追踪安全问题。
4. 支持合规审计。
5. 评估数据删除影响。
6. 分析模型退化原因。
7. 做成本归因。
8. 支持数据资产管理。

例如发现某个数据源有许可证风险，就需要知道哪些 dataset、checkpoint 和线上模型使用了它。

没有血缘，只能人工翻日志，几乎不可维护。

37.10 数据删除和影响分析

企业场景中经常会遇到数据删除或撤回需求。

例如：

1. 某个文档不再允许训练。
2. 某批用户数据需要删除。
3. 某个数据源许可证变化。
4. 某个业务线撤回授权。

血缘可以帮助做影响分析：

这个数据源 -> 哪些 dataset version -> 哪些 training job -> 哪些 checkpoint -> 哪些 deployment

注意：模型已经训练过的数据不一定能简单从权重中删除。血缘至少能帮助判断影响范围，并决定是否重训、下线或标记风险。

给定一个待删除或撤权的数据源 s ，影响集合可以写成图可达集合：

$$I(s)=\{n\in N\mid s\leadsto n\}$$

$I(s)$ 中如果包含线上 deployment，通常需要进入风险评估、重训、下线、补充声明或权限隔离流程。

37.11 数据质量监控为什么重要

数据质量不是构建数据集时检查一次就结束。

数据源会变化，清洗规则会变化，业务数据分布会变化。

质量监控要持续进行。

常见问题：

1. 某个数据源突然乱码增多。
2. 某个采集任务重复抓取。
3. 某个清洗规则误删大量样本。
4. 某类语言比例异常变化。
5. 敏感信息过滤失效。
6. train/eval 泄漏。
7. 标注质量下降。
8. 合成数据比例过高。

这些都会影响训练结果。

37.12 数据质量指标

常见数据质量指标包括：

基础统计：

1. 样本数。
2. token 数。
3. 文件数。
4. shard 大小。
5. 平均样本长度。
6. 长度分布。

内容质量：

1. 语言分布。
2. 领域分布。
3. 重复率。
4. 低质量文本比例。
5. 乱码比例。
6. 毒性或风险内容比例。

结构质量：

1. schema 合法率。
2. JSON 解析成功率。
3. 空字段比例。
4. 标签缺失率。
5. 多模态文件缺失率。

安全合规：

1. PII 命中率。
2. 密钥或凭据命中率。
3. 许可证风险比例。
4. 敏感等级分布。

常见质量指标可以统一写成比例。

Schema 合法率：

$$R_{\{\mathrm{schema}\}} = \frac{N_{\{\mathrm{schema}, \mathrm{ok}\}}}{N_{\{\mathrm{sample}\}}}$$

重复率：

$$R_{\{\mathrm{dup}\}} = \frac{N_{\{\mathrm{duplicate}\}}}{N_{\{\mathrm{sample}\}}}$$

PII 未阻断率：

$$R_{\{\mathrm{pii},\mathrm{open}\}}=\frac{N_{\{\mathrm{pii},\mathrm{open}\}}}{N_{\{\mathrm{pii},\mathrm{total}\}}}$$

train / eval overlap 率:

$$R_{\{\mathrm{overlap}\}}=\frac{N_{\{\mathrm{train},\mathrm{eval},\mathrm{overlap}\}}}{N_{\{\mathrm{eval}\}}}$$

这些指标要按来源、语言、领域、数据用途、清洗版本和时间窗口切片看，不能只看全局平均。

37.13 分布漂移监控

数据分布漂移是指数据分布和历史版本相比发生明显变化。

例如:

1. 英文比例从 50% 变成 80%。
2. 代码数据比例突然下降。
3. 某个领域数据突然增多。
4. 平均样本长度翻倍。
5. 合成数据比例过高。

漂移不一定是坏事，但必须被发现和解释。

平台可以对比当前 dataset version 和上一个 version 的统计差异，并生成 diff report。

离散分布漂移可以用 total variation distance 粗略衡量:

$$D_{\{\mathrm{tv}\}}(p,q)=\frac{1}{2}\sum_{c\in C}|p_c-q_c|$$

其中 p 是旧版本在语言、领域或来源上的分布， q 是新版本分布。 D_{tv} 高不一定说明新版本错误，但必须解释变化来源，并检查是否符合数据配方。

37.14 Dataset Diff Report

Dataset diff report 用于比较两个数据版本。

它可以包含:

1. 新增样本数。
2. 删除样本数。
3. token 数变化。
4. 来源比例变化。
5. 语言分布变化。
6. 领域分布变化。
7. 重复率变化。
8. PII 命中率变化。
9. 质量分分布变化。
10. 采样配方变化。

训练效果变化时，diff report 是排查入口之一。

如果模型突然在代码任务上退化，先看代码数据比例、质量和去重策略有没有变化。

37.15 数据质量门禁

数据质量门禁是 dataset 发布前的检查。

例如:

1. schema 合法率必须大于 99.9%。
2. 重复率不能超过阈值。
3. PII 命中率不能超过阈值。
4. train/eval overlap 必须低于阈值。
5. 关键来源比例不能异常变化。

6. token 数必须达到预期。
7. shard checksum 必须通过。

不通过门禁的数据集不能进入训练。

这和代码 CI 类似，只不过检查对象变成数据。

数据质量门禁可以写成：

```
G_{\mathrm{quality}}=\mathbf{1}\left[
C_{\mathrm{manifest}}=1
\land C_{\mathrm{checksum}}=1
\land C_{\mathrm{lineage}}\geq\rho
\land R_{\mathrm{schema}}\geq\tau_{\mathrm{schema}}
\land R_{\mathrm{dup}}\leq\rho_{\mathrm{dup}}
\land R_{\mathrm{pii,open}}=0
\land R_{\mathrm{overlap}}\leq\rho_{\mathrm{overlap}}
\land D_{\mathrm{tv}}\leq\rho_{\mathrm{drift}}
\land P_0=0
\right]
```

其中 P_0 是 P0 级硬阻断数量，例如 manifest 丢失、checksum 失败、评估泄漏、PII 未阻断或权限不匹配。

37.16 Train/Eval 泄漏监控

Train/eval 泄漏是大模型评估中的严重问题。

如果 eval 样本出现在 train 数据中，评估结果会虚高。

泄漏形式包括：

1. 完全重复。
2. 近似重复。
3. 同一问题不同表述。
4. benchmark 数据被网页爬取后进入训练。
5. 标注数据同时进入训练和评估。

监控方法包括：

1. hash 去重。
2. MinHash / SimHash。
3. embedding 相似度。
4. benchmark contamination 检查。
5. 数据来源黑名单。

评估可信度依赖数据隔离。

37.17 标注数据质量监控

SFT、RLHF、偏好数据和人工评测都依赖标注质量。

标注数据要监控：

1. 标注员一致性。
2. 标签分布。
3. 标注耗时。
4. 低质量标注比例。
5. 冲突样本比例。
6. gold set 准确率。
7. 审核通过率。
8. 标注指南版本。

如果标注指南变化，也要记录版本。

否则模型行为变化可能来自标注标准变化，而不是算法改进。

37.18 合成数据质量监控

合成数据越来越常见，但也有风险。

需要监控：

1. 生成模型版本。
2. 生成 prompt 版本。
3. 采样参数。
4. 去重率。
5. 多样性。
6. 事实性。
7. 难度分布。
8. 与真实数据比例。
9. 是否引入偏差。
10. 是否被目标模型过度拟合。

合成数据不是免费质量提升，必须有质量评估和版本追踪。

37.19 数据质量告警

数据质量监控应该产生告警。

例如：

1. 样本量低于预期。
2. 某来源数据突然为 0。
3. PII 命中率升高。
4. JSON 解析失败率升高。
5. 重复率异常升高。
6. 语言分布漂移。
7. shard checksum 失败。
8. eval overlap 超阈值。

告警要能定位到：

1. 哪个 dataset。
2. 哪个 version。
3. 哪个数据源。
4. 哪个处理任务。
5. 哪个规则版本。

否则告警只会制造噪音。

37.20 数据版本和训练任务联动

训练任务提交时应记录：

1. dataset name。
2. dataset version。
3. manifest checksum。
4. tokenizer version。
5. data loader version。
6. sampling seed。
7. shuffle config。
8. split 信息。

这样才能复现一次训练。

如果训练只记录了路径，路径下内容又变化了，就无法复现。

37.21 数据版本和模型版本联动

模型版本应该记录训练所用数据。

例如：

```
ModelVersion: chat-7b-sft-v12
base_model: base-7b-v3
sft_dataset: sft_mix:2026-05-30-v1
preference_dataset: pref_mix:2026-05-20-v2
eval_report: eval-2026-05-31
```

这样线上模型出问题时，可以沿模型版本追溯到 dataset version。

模型仓库和数据仓库不能割裂。

训练任务与模型版本联动可以写成：

$T_j = (V_k, H_k, \theta_j, e_j, c_j)$

其中 T_j 是训练任务， V_k 是数据版本， H_k 是 manifest / dataset 内容指纹， θ_j 是训练配置， e_j 是环境版本， c_j 是代码版本。模型版本必须能反向找到这个元组，否则线上事故无法可靠复盘。

37.22 常见系统设计

一个数据版本、血缘和质量平台可以包含：

```
Data Catalog
-> Dataset Registry
-> Manifest Store
-> Lineage Graph
-> Quality Metrics Store
-> Quality Gate Service
-> Diff Report Service
-> Access Control
-> Audit Log
```

模块职责：

1. Data Catalog：查找数据资产。
2. Dataset Registry：注册 dataset 和版本。
3. Manifest Store：保存文件列表和 checksum。
4. Lineage Graph：记录依赖关系。
5. Quality Metrics Store：保存质量指标。
6. Quality Gate Service：发布前门禁。
7. Diff Report Service：版本对比。
8. Access Control：权限控制。
9. Audit Log：记录访问和变更。

37.23 数据版本、血缘与质量监控审计指标和最小 demo

下面是一个 0 依赖 toy demo。它把第 37 章的三类能力合到同一张审计表：dataset version 是否完整，manifest 和 checksum 是否可靠，snapshot 是否不可变，diff report 是否能解释增量，lineage graph 是否完整，删除或撤权能否做影响分析，质量指标和告警是否能阻止坏数据版本进入训练。

```
from copy import deepcopy
from pprint import pprint
```

```

REQUIRED_VERSION_FIELDS = [
    "dataset_id",
    "version",
    "parent_version",
    "created_at",
    "owner",
    "state",
    "schema_version",
    "tokenizer_version",
    "recipe_version",
    "split_seed",
]

REQUIRED_LINEAGE_NODES = {
    "raw_source",
    "ingestion_job",
    "raw_dataset",
    "cleaning_job",
    "dedup_job",
    "dataset_version",
    "training_job",
    "checkpoint",
    "eval_report",
    "model_version",
    "deployment",
}

REQUIRED_EDGES = [
    ("raw_source", "ingestion_job"),
    ("ingestion_job", "raw_dataset"),
    ("raw_dataset", "cleaning_job"),
    ("cleaning_job", "dedup_job"),
    ("dedup_job", "dataset_version"),
    ("dataset_version", "training_job"),
    ("training_job", "checkpoint"),
    ("checkpoint", "eval_report"),
    ("eval_report", "model_version"),
    ("model_version", "deployment"),
]

def complete_case():
    return {
        "name": "complete_case",
        "version": {
            "dataset_id": "pretrain_mix",
            "version": "2026-06-01-v2",
            "parent_version": "2026-05-30-v1",
            "created_at": "2026-06-01T08:00:00Z",
            "owner": "data-platform",
            "state": "published",
            "schema_version": "schema_v3",
            "tokenizer_version": "tok_v4",
        }
    }

```

```

    "recipe_version": "mix_v5",
    "split_seed": 3407,
    "frozen": True,
},
"manifest": {
    "dataset_id": "pretrain_mix",
    "version": "2026-06-01-v2",
    "manifest_checksum": "sha256:manifest-v2",
    "immutable": True,
    "sample_count": 2000,
    "token_count": 1320000,
    "files": [
        {
            "path": "s3://data/pretrain_mix/v2/shard-000.parquet",
            "checksum": "sha256:shard0",
            "samples": 1000,
            "tokens": 700000,
        },
        {
            "path": "s3://data/pretrain_mix/v2/shard-001.parquet",
            "checksum": "sha256:shard1",
            "samples": 1000,
            "tokens": 620000,
        },
    ],
},
"diff": {
    "generated": True,
    "base_samples": 1800,
    "current_samples": 2000,
    "added_samples": 300,
    "removed_samples": 100,
    "token_delta": 120000,
    "reason": "added_code_and_removed_bad_source",
},
"lineage": {
    "nodes": sorted(REQUIRED_LINEAGE_NODES),
    "edges": list(REQUIRED_EDGES),
    "events": [
        {"job": "ingest", "run_id": "run_ingest_1", "inputs": ["raw_source"], "outputs": ["raw_dataset"]},
        {"job": "clean", "run_id": "run_clean_1", "inputs": ["raw_dataset"], "outputs": ["clean_dataset"]},
        {"job": "dedup", "run_id": "run_dedup_1", "inputs": ["clean_dataset"], "outputs": ["dataset_version"]},
        {"job": "train", "run_id": "run_train_1", "inputs": ["dataset_version"], "outputs": ["checkpoint"]},
    ],
    "linked_samples": 1940,
    "expected_samples": 2000,
},
"impact": {
    "revoked_source": "raw_source",
    "ready": True,
    "impacted_dataset_versions": ["pretrain_mix:2026-06-01-v2"],
    "impacted_training_jobs": ["train_001"],
    "impacted_models": ["chat-7b-sft-v12"],
    "impacted_deployments": ["chat-prod-us"],
},

```

```

"quality": {
  "sample_count": 2000,
  "schema_ok": 1999,
  "parse_ok": 1999,
  "duplicate_count": 40,
  "low_quality_count": 80,
  "pii_total": 10,
  "pii_open": 0,
  "license_risk": 0,
  "train_eval_overlap": 0,
  "eval_count": 500,
  "current_distribution": {"en": 0.55, "zh": 0.25, "code": 0.20},
  "baseline_distribution": {"en": 0.52, "zh": 0.26, "code": 0.22},
  "annotation_agreement": 0.91,
  "gold_accuracy": 0.96,
  "synthetic_count": 300,
  "synthetic_factual_pass": 0.93,
  "synthetic_diversity": 0.82,
},
"alerts": {
  "dashboard": True,
  "rules": [
    "schema_valid_rate",
    "duplicate_rate",
    "pii_open_rate",
    "train_eval_overlap",
    "distribution_drift",
    "annotation_agreement",
    "synthetic_ratio",
  ],
  "routed_to": ["data-platform-oncall"],
  "context_fields": ["dataset_id", "version", "source", "job", "rule_version"],
},
"training_links": {
  "training_jobs": [
    {
      "job_id": "train_001",
      "dataset_version": "2026-06-01-v2",
      "manifest_checksum": "sha256:manifest-v2",
      "tokenizer_version": "tok_v4",
      "sampling_seed": 3407,
    }
  ],
  "model_versions": [
    {
      "model_id": "chat-7b-sft-v12",
      "train_job_id": "train_001",
      "dataset_versions": ["pretrain_mix:2026-06-01-v2"],
      "eval_report": "eval_001",
    }
  ],
},
"data_governance_gate": True,
}

```

```

def ratio(num, den):
    return 0.0 if den == 0 else num / den

def total_variation(p, q):
    keys = set(p) | set(q)
    return 0.5 * sum(abs(p.get(k, 0.0) - q.get(k, 0.0)) for k in keys)

def dataset_version_contract(case):
    version = case["version"]
    return all(version.get(field) for field in REQUIRED_VERSION_FIELDS) and version["state"] == "published"

def manifest_checksum_integrity(case):
    manifest = case["manifest"]
    files = manifest["files"]
    samples_match = sum(f["samples"] for f in files) == manifest["sample_count"]
    tokens_match = sum(f["tokens"] for f in files) == manifest["token_count"]
    checksums = bool(manifest.get("manifest_checksum")) and all(f.get("checksum") for f in files)
    return samples_match and tokens_match and checksums

def immutable_snapshot_policy(case):
    return case["version"]["frozen"] and case["manifest"]["immutable"]

def delta_version_diff(case):
    diff = case["diff"]
    expected_current = diff["base_samples"] + diff["added_samples"] - diff["removed_samples"]
    return diff["generated"] and expected_current == diff["current_samples"] and bool(diff["reason"])

def lineage_graph_completeness(case):
    lineage = case["lineage"]
    nodes_ok = REQUIRED_LINEAGE_NODES.issubset(set(lineage["nodes"]))
    edges_ok = all(edge in lineage["edges"] for edge in REQUIRED_EDGES)
    events_ok = all(e.get("job") and e.get("run_id") and e.get("inputs") and e.get("outputs") for e in lineage["event"])
    sample_coverage = ratio(lineage["linked_samples"], lineage["expected_samples"]) >= 0.95
    return nodes_ok and edges_ok and events_ok and sample_coverage

def impact_analysis_readiness(case):
    impact = case["impact"]
    return (
        impact["ready"]
        and bool(impact["revoked_source"])
        and bool(impact["impacted_dataset_versions"])
        and bool(impact["impacted_training_jobs"])
        and bool(impact["impacted_models"])
        and bool(impact["impacted_deployments"])
    )

```

```

def schema_quality_gate(case):
    quality = case["quality"]
    return ratio(quality["schema_ok"], quality["sample_count"]) >= 0.999 and ratio(quality["parse_ok"], quality["sample_count"]) >= 0.999

def content_quality_gate(case):
    quality = case["quality"]
    return ratio(quality["duplicate_count"], quality["sample_count"]) <= 0.05 and ratio(quality["low_quality_count"], quality["sample_count"]) <= 0.05

def safety_compliance_gate(case):
    quality = case["quality"]
    return quality["pii_open"] == 0 and quality["license_risk"] == 0

def distribution_drift_monitoring(case):
    quality = case["quality"]
    drift = total_variation(quality["baseline_distribution"], quality["current_distribution"])
    return drift <= 0.08

def train_eval_leakage_monitoring(case):
    quality = case["quality"]
    return ratio(quality["train_eval_overlap"], quality["eval_count"]) == 0

def annotation_quality_monitoring(case):
    quality = case["quality"]
    return quality["annotation_agreement"] >= 0.85 and quality["gold_accuracy"] >= 0.90

def synthetic_data_monitoring(case):
    quality = case["quality"]
    synthetic_ratio = ratio(quality["synthetic_count"], quality["sample_count"])
    return synthetic_ratio <= 0.25 and quality["synthetic_factual_pass"] >= 0.90 and quality["synthetic_diversity"] > 0.90

def quality_alert_routing(case):
    alerts = case["alerts"]
    needed_context = {"dataset_id", "version", "source", "job", "rule_version"}
    return alerts["dashboard"] and len(alerts["rules"]) >= 6 and bool(alerts["routed_to"]) and needed_context.issubset(set(alerts["rules"].keys()))

def training_model_version_linkage(case):
    version = case["version"]
    manifest = case["manifest"]
    links = case["training_links"]
    jobs = links["training_jobs"]
    models = links["model_versions"]
    job_ids = {job["job_id"] for job in jobs}
    job_ok = all(
        job["dataset_version"] == version["version"]
        and job["manifest_checksum"] == manifest["manifest_checksum"]
        and job["tokenizer_version"] == version["tokenizer_version"]
        for job in jobs
    )

```



```

)
model_ok = all(model["train_job_id"] in job_ids and model["dataset_versions"] for model in models)
return bool(jobs) and bool(models) and job_ok and model_ok

def data_version_quality_gate(case):
    return case["data_governance_gate"] is True

CHECKS = [
    ("dataset_version_contract", dataset_version_contract),
    ("manifest_checksum_integrity", manifest_checksum_integrity),
    ("immutable_snapshot_policy", immutable_snapshot_policy),
    ("delta_version_diff", delta_version_diff),
    ("lineage_graph_completeness", lineage_graph_completeness),
    ("impact_analysis_readiness", impact_analysis_readiness),
    ("schema_quality_gate", schema_quality_gate),
    ("content_quality_gate", content_quality_gate),
    ("safety_compliance_gate", safety_compliance_gate),
    ("distribution_drift_monitoring", distribution_drift_monitoring),
    ("train_eval_leakage_monitoring", train_eval_leakage_monitoring),
    ("annotation_quality_monitoring", annotation_quality_monitoring),
    ("synthetic_data_monitoring", synthetic_data_monitoring),
    ("quality_alert_routing", quality_alert_routing),
    ("training_model_version_linkage", training_model_version_linkage),
    ("data_version_quality_gate", data_version_quality_gate),
]

def make_bad_cases(base):
    cases = []

    def add(name, change):
        case = deepcopy(base)
        case["name"] = name
        change(case)
        cases.append(case)

    add("version_contract_missing_bad", lambda c: c["version"].update({"owner": ""}))
    add("manifest_checksum_missing_bad", lambda c: c["manifest"]["files"][0].update({"checksum": ""}))
    add("snapshot_mutated_bad", lambda c: c["version"].update({"frozen": False}))
    add("diff_report_missing_bad", lambda c: c["diff"].update({"generated": False}))
    add("lineage_edge_missing_bad", lambda c: c["lineage"]["edges"].remove(("cleaning_job", "dedup_job")))
    add("impact_analysis_missing_bad", lambda c: c["impact"].update({"ready": False}))
    add("schema_failure_bad", lambda c: c["quality"].update({"schema_ok": 1980, "parse_ok": 1980}))
    add("duplicate_spike_bad", lambda c: c["quality"].update({"duplicate_count": 180}))
    add("pii_spike_bad", lambda c: c["quality"].update({"pii_open": 2}))
    add("distribution_drift_bad", lambda c: c["quality"].update({"current_distribution": {"en": 0.80, "zh": 0.10, "co": 0.10}}))
    add("eval_leakage_bad", lambda c: c["quality"].update({"train_eval_overlap": 3}))
    add("annotation_agreement_low_bad", lambda c: c["quality"].update({"annotation_agreement": 0.70}))
    add("synthetic_ratio_high_bad", lambda c: c["quality"].update({"synthetic_count": 800}))
    add("alert_unrouted_bad", lambda c: c["alerts"].update({"routed_to": []}))
    add("model_linkage_missing_bad", lambda c: c["training_links"].update({"model_versions": []}))
    add("governance_gate_missing_bad", lambda c: c.update({"data_governance_gate": False}))
    return cases

```

```

def summarize_examples(case):
    manifest = case["manifest"]
    quality = case["quality"]
    diff = case["diff"]
    lineage = case["lineage"]
    drift = total_variation(quality["baseline_distribution"], quality["current_distribution"])
    return {
        "manifest_completeness": 1.0,
        "checksum_coverage": round(ratio(sum(1 for f in manifest["files"] if f["checksum"]), len(manifest["files"])), 3),
        "added_sample_ratio": round(ratio(diff["added_samples"], diff["base_samples"]), 3),
        "removed_sample_ratio": round(ratio(diff["removed_samples"], diff["base_samples"]), 3),
        "lineage_coverage": round(ratio(lineage["linked_samples"], lineage["expected_samples"]), 3),
        "distribution_drift_tv": round(drift, 3),
        "schema_valid_rate": round(ratio(quality["schema_ok"], quality["sample_count"]), 4),
        "duplicate_rate": round(ratio(quality["duplicate_count"], quality["sample_count"]), 3),
        "pii_open_rate": round(ratio(quality["pii_open"], quality["pii_total"]), 3),
        "train_eval_overlap_rate": round(ratio(quality["train_eval_overlap"], quality["eval_count"]), 3),
        "annotation_agreement": quality["annotation_agreement"],
        "synthetic_ratio": round(ratio(quality["synthetic_count"], quality["sample_count"]), 3),
        "impacted_models": case["impact"]["impacted_models"],
    }

```

```

def audit_data_version_lineage_quality(cases):
    per_case = {
        case["name"]: {name: check(case) for name, check in CHECKS}
        for case in cases
    }
    metrics = {
        name: round(sum(result[name] for result in per_case.values()) / len(cases), 3)
        for name, _ in CHECKS
    }
    failed_cases = [name for name, result in per_case.items() if not all(result.values())]
    failed_gates = [name for name, score in metrics.items() if score < 1.0]
    return {
        "smoke": {
            "complete_case_passes": all(per_case["complete_case"].values()),
            "caught_manifest_gap": "manifest_checksum_missing_bad" in failed_cases,
            "caught_lineage_gap": "lineage_edge_missing_bad" in failed_cases,
            "caught_drift": "distribution_drift_bad" in failed_cases,
            "caught_eval_leakage": "eval_leakage_bad" in failed_cases,
            "caught_model_linkage_gap": "model_linkage_missing_bad" in failed_cases,
        },
        "metrics": metrics,
        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "failed_gates": failed_gates,
        "data_version_quality_gate_pass": len(failed_cases) == 0,
    }

```

```

base = complete_case()
cases = [base] + make_bad_cases(base)

```

```

report = audit_data_version_lineage_quality(cases)

pprint({"data_version_examples": summarize_examples(base)})
pprint(report["smoke"])
pprint(report["metrics"])
print("hard_blocker_count=", report["hard_blocker_count"])
print("failed_cases=", report["failed_cases"])
print("failed_gates=", report["failed_gates"])
print("data_version_quality_gate_pass=", report["data_version_quality_gate_pass"])

assert report["smoke"]["complete_case_passes"]
assert report["smoke"]["caught_eval_leakage"]
assert report["metrics"]["dataset_version_contract"] == 0.941
assert report["hard_blocker_count"] == 16
assert report["data_version_quality_gate_pass"] is False

```

这个 demo 的重点不是还原真实数据平台，而是把系统设计题里的“数据版本、血缘和质量监控”拆成可落地证据：版本契约、manifest checksum、不可变 snapshot、diff report、lineage graph、影响分析、schema、内容质量、安全合规、分布漂移、train/eval 泄漏、标注质量、合成数据、告警路由、训练 / 模型版本联动和最终治理门禁。

37.24 常见误区

误区一：数据版本就是目录名。

目录名不够，版本还要包含 manifest、规则版本、checksum、配方和质量统计。

误区二：数据血缘只是合规需要。

血缘也用于训练复现、质量排查、删除影响分析和成本归因。

误区三：质量检查只在构建时做一次。

数据源和规则会变化，质量需要持续监控和版本对比。

误区四：eval 分数可信。

如果 train/eval 泄漏，eval 分数可能严重虚高。

误区五：合成数据越多越好。

合成数据需要监控多样性、事实性、难度和偏差。

37.25 面试常见追问

问题一：如何保证训练数据可复现？

可以回答：训练任务引用不可变 dataset version，版本中包含 manifest、文件 checksum、清洗规则、去重规则、采样配方、split、tokenizer 和质量统计，并在训练任务中记录 manifest checksum 和随机种子。

问题二：数据血缘怎么设计？

可以回答：用 lineage graph 记录 raw source、ingestion、cleaning、dedup、dataset version、training job、checkpoint、eval report、model version 和 deployment 的依赖关系，支持从线上模型反查训练数据。

问题三：数据质量监控看哪些指标？

可以回答：看样本数、token 数、长度分布、语言/领域分布、重复率、schema 合法率、PII 命中率、许可证风险、train/eval overlap、标注一致性和版本 diff。

问题四：如何发现 train/eval 泄漏？

可以回答：使用 hash 精确匹配、MinHash/SimHash 近似匹配、embedding 相似度、benchmark contamination 检查和数据来源黑名单，检查 train 与 eval 之间的重复和近似重复。

问题五：数据版本、血缘和质量监控如何串成一个发布门禁？

可以回答：dataset 发布前先校验 version contract、manifest checksum、不可变 snapshot 和 diff report，再检查 lineage graph、删除或撤权影响分析、schema / 重复 / PII / overlap / drift / 标注 / 合成数据指标，最后确认告警能路由到 source、job 和 rule version，并把 dataset version、manifest checksum、tokenizer、seed 绑定到 TrainingJob 和 ModelVersion。

37.26 小练习

1. 为什么 dataset version 应该尽量不可变？
2. Dataset manifest 和普通文件目录有什么区别？
3. 数据血缘图中应该包含哪些节点？
4. 如何做两个 dataset version 的 diff？
5. 数据质量门禁应该检查哪些指标？
6. Train/eval 泄漏为什么会让评估虚高？
7. 合成数据需要监控哪些质量问题？
8. 如何从一个线上模型追溯到它使用过的数据源？
9. 写一个 0 依赖数据版本、血缘和质量监控审计 demo，覆盖 version contract、manifest checksum、immutable snapshot、diff report、lineage graph、impact analysis、schema、duplicate、PII、drift、train/eval overlap、annotation、synthetic data、alert routing、TrainingJob / ModelVersion linkage 和 final gate。

37.27 本章小结

本章讲了数据版本管理、数据血缘和数据质量监控。

你需要记住：

1. 数据版本管理的目标是训练可复现，不是简单给目录起名字。
2. Dataset version 应包含 manifest、规则版本、checksum、采样配方、质量统计和权限信息。
3. 数据血缘把 raw data、dataset、training job、checkpoint、eval report、model version 和 deployment 串起来。
4. 数据质量监控要覆盖基础统计、内容质量、结构质量、安全合规、分布漂移和 train/eval 泄漏。
5. 数据质量门禁和 diff report 是数据平台工程化治理的重要工具。
6. 真正可用的数据治理平台要把 dataset version、manifest checksum、lineage event、impact analysis、quality metrics、alert routing、TrainingJob 和 ModelVersion 串成闭环。

下一章我们会讲模型仓库：权重、tokenizer、配置、adapter 和量化版本。

第 38 章 模型仓库：权重、tokenizer、配置、adapter 和量化版本

上一章讲了数据版本、血缘和质量监控。本章继续讲模型资产管理中的核心系统：模型仓库。

训练平台产出 checkpoint，评估平台产出 eval report，推理平台需要部署模型版本。中间必须有一个可信的模型仓库，把权重、tokenizer、配置、adapter、量化版本和元数据统一管理起来。

先记住一句话：

模型仓库不是存权重文件的普通目录，而是模型资产的版本、元数据、权限、血缘、评估和发布状态管理系统。

38.0 本讲资料边界与第二轮精修口径

本讲按截至 2026-06 的稳定公开资料校准：MLflow Model Registry 对 registered model、model version、alias、tag 和 stage 的模型生命周期抽象，Hugging Face Hub 对模型仓库、revision、model card 和文件版本的资产管理口径，Safetensors 对安全权重格式和避免 pickle 反序列化风险的说明，Hugging Face PEFT 对

adapter、base model、merge 的关系管理口径，以及 vLLM 等推理 runtime 对权重格式、tokenizer、量化和加载兼容性的公开工程边界。

本章只抽象大模型平台里的模型产物仓库能力，不绑定某个 registry 产品、对象存储、推理引擎、量化工具、模型卡模板或内部平台实现。

和前面章节的分工是：

1. 第 36 章和第 37 章回答 “训练数据如何被版本化、追踪和治理”。
2. 本章回答 “训练、评估和部署之间的模型产物如何被冻结、校验、发布、回滚和审计”。

38.1 为什么需要模型仓库

小实验中，模型可能只是一个目录：

```
./outputs/checkpoint-10000/
```

但生产环境里会有大量模型资产：

1. base model。
2. SFT model。
3. RLHF / DPO model。
4. LoRA / adapter。
5. merge 后模型。
6. 量化模型。
7. embedding model。
8. reranker。
9. 多模态模型。
10. 不同 runtime 格式的模型。

如果没有模型仓库，团队很快会搞不清哪个模型能用、哪个模型评估过、哪个模型线上在跑、哪个模型可以删除。

38.2 模型仓库解决什么问题

模型仓库要解决：

1. 模型资产注册。
2. 权重和相关文件存储。
3. 模型版本管理。
4. tokenizer 和配置绑定。
5. adapter 和 base model 关系管理。
6. 量化版本管理。
7. 模型血缘追踪。
8. 评估报告关联。
9. 权限和安全管理。
10. 发布状态管理。
11. 生命周期管理。

它是训练平台、评估平台和推理平台之间的资产枢纽。

38.3 模型仓库中的核心对象

模型仓库通常包含这些对象：

1. Model。
2. ModelVersion。
3. ModelArtifact。
4. TokenizerVersion。
5. ModelConfig。
6. Adapter。

7. QuantizedVersion。
8. EvalReport。
9. DeploymentRecord。
10. LineageRecord。

其中 Model 是逻辑模型，ModelVersion 是具体版本，Artifact 是实际文件集合。

不要把 Model 和 ModelVersion 混在一起。

38.4 Model 和 ModelVersion

Model 表示一个模型系列，例如：

```
company-chat-model  
company-code-model  
company-embedding-model
```

ModelVersion 表示具体版本，例如：

```
company-chat-model:v1.0.0  
company-chat-model:v1.1.0-sft  
company-chat-model:v1.2.0-dpo
```

一个 Model 可以有多个 ModelVersion。

线上部署、评估、回滚都应该指向具体 ModelVersion。

38.5 ModelVersion 应该记录什么

ModelVersion 至少记录：

1. 模型名称。
2. 版本号。
3. 模型类型。
4. 参数规模。
5. 架构类型。
6. 权重路径。
7. 权重 checksum。
8. tokenizer 版本。
9. model config。
10. 上下文长度。
11. 支持精度。
12. 支持 runtime。
13. 训练任务 ID。
14. 训练数据版本。
15. 评估报告。
16. 安全等级。
17. 发布状态。
18. 创建人和审批人。

这些字段决定模型是否可复现、可部署、可审计。

一个模型版本可以抽象为：

$a_i = (m_i, v_i, w_i, t_i, c_i, b_i, p_i, q_i, r_i, e_i, s_i, l_i, o_i, z_i)$

其中 m_i 是逻辑 model id, v_i 是 model version, w_i 是权重 manifest, t_i 是 tokenizer version, c_i 是 model config, b_i 是 base model 或训练 checkpoint, p_i 是权限与安全策略, q_i 是量化配置, r_i 是 runtime 兼容性, e_i 是评估报告, s_i 是发布状态, l_i 是 lineage, o_i 是 owner / approver, z_i 是审计记录。

元数据完整率可以写成：

$$C_{\mathrm{artifact}} = \frac{K_{\mathrm{filled}}}{K_{\mathrm{required}}}$$

其中 K_{filled} 是已填写的必要字段数， K_{required} 是模型发布前必须存在的字段数。这个公式的直觉是：模型版本不是文件路径，而是一组发布证据；字段缺失越多，复现和回滚风险越高。

38.6 权重文件管理

权重文件是模型仓库最核心的 artifact。

常见权重格式包括：

1. PyTorch .bin。
2. safetensors。
3. TensorRT engine。
4. GGUF。
5. ONNX。
6. 自定义 runtime 格式。

大模型权重通常很大，管理时要考虑：

1. 分片文件。
2. checksum 校验。
3. 上传断点续传。
4. 下载加速。
5. 本地缓存。
6. 跨地域复制。
7. 生命周期和冷存储。

权重文件损坏或不一致，会导致模型加载失败或线上行为异常。

权重 checksum 覆盖率可以写成：

$$C_{\mathrm{weight}} = \frac{N_{\mathrm{checked}}}{N_{\mathrm{weight,total}}}$$

其中 N_{checked} 是已经校验过 checksum 的权重分片数， $N_{\mathrm{weight,total}}$ 是 manifest 中声明的权重分片总数。生产发布时通常要求 $C_{\mathrm{weight}}=1$ ，否则同一个 ModelVersion 在不同机器上可能加载到不一致内容。

38.7 Safetensors 为什么常见

Safetensors 是常见权重格式之一。

它的优点包括：

1. 加载更安全。
2. 避免 pickle 反序列化风险。
3. 支持快速加载。
4. 适合大模型分片存储。

模型仓库可以要求生产模型优先使用 safetensors 或经过安全校验的格式。

权重格式不是细枝末节，它关系到安全、加载速度和 runtime 兼容性。

38.8 Tokenizer 管理

Tokenizer 是模型不可分割的一部分。

同一个权重配错 tokenizer，会导致：

1. 输出乱码。
2. 特殊 token 错误。
3. 上下文长度异常。
4. stop token 失效。

5. 工具调用格式异常。
6. 多语言效果退化。

模型仓库必须记录 tokenizer 版本，并和 ModelVersion 绑定。

tokenizer 和 config 的绑定可以用一个二值门禁表示：

$$B_{\{\mathrm{tok}\}}=\mathbf{1}[h_{\{\mathrm{tok}\}}=h_{\{\mathrm{tok},\mathrm{expected}\}}]\wedge h_{\{\mathrm{cfg}\}}=h_{\{\mathrm{cfg},\mathrm{expected}\}}$$

其中 h_{tok} 是实际 tokenizer artifact 的 hash, $h_{\mathrm{tok},\mathrm{expected}}$ 是 ModelVersion 记录的期望 tokenizer hash, h_{cfg} 是实际 config hash, $h_{\mathrm{cfg},\mathrm{expected}}$ 是期望 config hash。这个门禁的直觉是：权重、tokenizer、chat template、special token 和 config 是同一个行为契约，不能只靠文件名匹配。

Tokenizer artifact 通常包括：

1. tokenizer config。
2. vocab。
3. merges。
4. special tokens map。
5. chat template。

不要把 tokenizer 当成可随意替换的小文件。

38.9 Model Config 管理

Model config 描述模型结构和推理参数。

常见字段：

1. hidden size。
2. layer 数。
3. attention head 数。
4. vocab size。
5. rope 参数。
6. max position embeddings。
7. bos/eos token。
8. dtype。
9. quantization config。
10. architecture name。

runtime 加载模型时依赖这些配置。

配置错误会导致加载失败、性能异常或输出错误。

38.10 Adapter 管理

Adapter 常见于 LoRA、QLoRA 和其他 PEFT 方法。

Adapter 不是完整模型，而是依赖 base model 的增量参数。

模型仓库要记录：

1. adapter 名称。
2. adapter 版本。
3. base model 版本。
4. adapter 权重路径。
5. 训练任务 ID。
6. 训练数据版本。
7. merge 状态。
8. 兼容性约束。
9. 评估报告。

如果 adapter 和 base model 不匹配，结果可能完全不可用。

adapter 与 base model 的最小兼容门禁可以写成：

$$G_{\{\mathrm{adapter}\}}=\mathbf{1}[v_{\{\mathrm{base}\}}=v_{\{\mathrm{base,expected}\}} \wedge d_{\{\mathrm{rank}\}}\leq d_{\{\mathrm{hidden}\}}]$$

其中 v_{base} 是 adapter 实际绑定的 base model 版本， $v_{\mathrm{base,expected}}$ 是仓库记录的期望 base model 版本， d_{rank} 是 adapter rank， d_{hidden} 是 base model hidden size。真实系统还会检查 target modules、dtype、RoPE / vocab / tokenizer、PEFT 方法和训练配置。

38.11 Merge 后模型

LoRA adapter 可以在推理时动态加载，也可以 merge 到 base model 中生成完整权重。

两种方式各有利弊。

动态加载 adapter：

1. 存储更省。
2. 方便切换多个 adapter。
3. 对 runtime 支持有要求。
4. 可能增加推理复杂度。

Merge 后模型：

1. 部署更简单。
2. runtime 兼容性更好。
3. 存储占用更大。
4. 需要记录 merge 过程。

模型仓库应记录 merge lineage：哪个 base model 加哪个 adapter 生成了哪个 merged model。

38.12 量化版本管理

量化版本是同一模型的低精度部署形态。

例如：

1. FP16。
2. BF16。
3. FP8。
4. INT8。
5. INT4。
6. GPTQ。
7. AWQ。
8. GGUF variants。

量化版本要记录：

1. 原始模型版本。
2. 量化方法。
3. 校准数据。
4. 量化工具版本。
5. 量化参数。
6. 目标 runtime。
7. 质量评估。
8. 延迟和吞吐 benchmark。

量化不是简单转换格式，必须评估质量和性能。

量化前后的质量退化可以写成：

$$\Delta Q=Q_{\{\mathrm{base}\}}-Q_{\{\mathrm{quant}\}}$$

其中 Q_{base} 是原始模型在同一评估集上的质量指标， Q_{quant} 是量化版本质量指标。上线门禁不应该只看吞吐提升，还要约束 ΔQ 不超过业务可接受阈值。

38.13 Runtime 兼容性

不同 runtime 支持的格式和能力不同。

模型仓库要记录：

1. 是否支持 vLLM。
2. 是否支持 TGI。
3. 是否支持 Triton。
4. 是否支持 TensorRT-LLM。
5. 是否支持 SGLang。
6. 需要的 CUDA 版本。
7. 需要的 driver 版本。
8. 需要的 tokenizer 配置。
9. 是否支持 tensor parallel。
10. 是否支持特定量化格式。

推理平台部署前应检查 runtime compatibility。

不能让部署阶段才发现模型格式不支持。

模型加载耗时可以粗略拆成：

$$T_{\mathrm{load}} = \frac{S_{\mathrm{artifact}}}{B_{\mathrm{read}}} + T_{\mathrm{verify}} + T_{\mathrm{init}}$$

其中 S_{artifact} 是模型产物大小， B_{read} 是有效读取带宽， T_{verify} 是 checksum / manifest / 安全扫描校验时间， T_{init} 是 runtime 初始化、权重映射、kernel 准备和缓存预热时间。这个公式用于解释：模型仓库的缓存、manifest 和格式选择会直接影响推理平台冷启动。

38.14 模型血缘

模型血缘记录模型从哪里来。

常见血缘关系：

1. base model -> SFT model。
2. SFT model -> DPO model。
3. base model + adapter -> merged model。
4. FP16 model -> INT4 quantized model。
5. checkpoint -> model version。
6. model version -> deployment。

血缘还要关联训练数据、训练代码、训练配置和评估报告。

当某个模型出现问题时，平台需要追溯到训练任务和数据版本。

38.15 评估报告关联

模型仓库不应该只保存模型文件，还要关联 eval report。

评估报告可以包括：

1. 通用能力评估。
2. 领域评估。
3. 安全评估。
4. 工具调用评估。
5. 结构化输出评估。
6. 长上下文评估。

7. 延迟和吞吐 benchmark。
8. 成本估算。

模型发布前，平台应检查是否有必要评估报告，且指标满足门禁。

38.16 发布状态管理

模型版本可以有状态：

registered -> validated -> evaluated -> staged -> production -> deprecated -> archived

状态变化应该有权限控制和审计。

例如：

1. 未评估模型不能进入 production。
2. 安全评估未通过不能发布。
3. deprecated 模型不能新建部署。
4. archived 模型只能保留审计，不提供线上服务。

模型仓库和发布系统要联动。

38.17 权限控制

模型本身也有权限。

权限问题包括：

1. 谁能下载权重。
2. 谁能部署模型。
3. 谁能查看评估报告。
4. 谁能使用敏感数据训练出的模型。
5. 哪个租户能调用哪个模型。
6. 哪些模型不能导出。

企业内部训练出的模型可能包含敏感知识，不能默认所有人可见和可用。

38.18 安全扫描

模型仓库可以对 artifact 做安全扫描：

1. 文件完整性校验。
2. 反序列化风险检查。
3. 可疑文件检查。
4. 权重格式检查。
5. tokenizer 特殊 token 检查。
6. 配置合法性检查。
7. 许可证元数据检查。

模型 artifact 是供应链的一部分，也需要安全治理。

38.19 生命周期管理

大模型 artifact 很占存储。

生命周期管理要回答：

1. 哪些 checkpoint 保留？
2. 哪些中间版本可以删除？
3. 哪些 production 模型必须长期保留？
4. 哪些模型可以转冷存储？

5. 删除前是否有部署依赖?
6. 删除是否影响回滚?

策略示例:

1. production 版本长期保留。
2. 最近 N 个候选版本保留。
3. 失败实验 checkpoint 定期清理。
4. 有审计或合规要求的版本冻结。
5. 删除前检查 lineage 和 deployment。

模型仓库如果不做生命周期管理, 存储成本会快速增长。

38.20 模型仓库和推理平台的接口

推理平台部署模型时, 应该从模型仓库读取:

1. 权重路径。
2. tokenizer 路径。
3. model config。
4. runtime 兼容性。
5. 默认推理参数。
6. 上下文长度。
7. 量化版本。
8. 安全等级。
9. 发布状态。

如果模型状态不是 production 或 staged, 推理平台应该拒绝部署或要求审批。

38.21 模型仓库和训练平台的接口

训练平台完成训练后, 可以将 checkpoint 注册成候选模型版本。

注册过程包括:

1. 上传或引用权重。
2. 计算 checksum。
3. 绑定 tokenizer。
4. 绑定训练配置。
5. 绑定 dataset version。
6. 绑定训练任务 ID。
7. 创建 ModelVersion。
8. 触发评估任务。

这样训练产物才能进入标准化资产管理流程。

38.22 模型仓库和评估平台的接口

评估平台从模型仓库读取候选模型, 执行评估后写回 eval report。

流程:

ModelVersion -> EvalJob -> EvalReport -> Quality Gate -> Release Candidate

评估报告写回后, 模型状态可以从 registered 变成 evaluated 或 validated。

没有评估报告的模型不应该直接上线。

38.23 一个模型仓库系统架构

可以这样设计：

Model Registry API

- > Metadata DB
- > Artifact Store
- > Checksum Service
- > Access Control
- > Lineage Graph
- > Eval Report Store
- > Release State Machine
- > Audit Log

Artifact Store 存大文件，Metadata DB 存模型元数据，Lineage Graph 存依赖关系，Audit Log 记录变更。

38.24 模型仓库审计指标和最小 demo

模型仓库的最终门禁可以写成：

$$G_{\{\mathrm{registry}\}} = \mathbf{1}[\min_j C_j \geq \tau_j \wedge C_{\{\mathrm{weight}\}} = 1 \wedge B_{\{\mathrm{tok}\}} = 1 \wedge \Delta$$

其中 C_j 是每个审计维度的覆盖率， τ_j 是该维度阈值， ρ_Q 是可接受量化质量退化， τ_{load} 是加载耗时阈值， P_0 是未解决的 P_0 级发布风险数量。这个门禁强调：模型仓库不是“文件可下载”就合格，而是权重、tokenizer、config、adapter、merge、量化、runtime、评估、安全、发布、回滚和生命周期都能被平台验证。

下面是一个 0 依赖 toy demo。它把本章的模型仓库治理拆成 16 个可审计门禁，并构造 16 个 bad case，覆盖权重 manifest 缺失、unsafe 权重格式、tokenizer / config 绑定错误、adapter base 不匹配、merge lineage 缺失、量化质量退化、runtime 不兼容、评估报告缺失、权限安全不足、发布治理缺失、回滚别名缺失、lineage 断边、加载缓存不达标、生命周期策略缺失和最终 registry gate 缺失。

```
from copy import deepcopy
from pprint import pprint
```

```
REQUIRED_MODEL_FIELDS = [
```

```
    "model_id",
    "version",
    "artifact_type",
    "architecture",
    "parameter_count",
    "owner",
    "stage",
    "created_at",
    "train_job_id",
    "dataset_version",
    "eval_report_id",
    "safety_report_id",
    "release_alias",
    "rollback_alias",
```

```
]
```

```
REQUIRED_LINEAGE_NODES = {
```

```
    "dataset_version",
    "training_job",
    "checkpoint",
    "model_version",
    "adapter",
    "merged_model",
```

```

    "quantized_model",
    "eval_report",
    "deployment",
}

```

```

REQUIRED_LINEAGE_EDGES = [
    ("dataset_version", "training_job"),
    ("training_job", "checkpoint"),
    ("checkpoint", "model_version"),
    ("model_version", "adapter"),
    ("adapter", "merged_model"),
    ("merged_model", "quantized_model"),
    ("quantized_model", "eval_report"),
    ("eval_report", "deployment"),
]

```

```

def complete_case():
    return {
        "name": "complete_case",
        "model_version": {
            "model_id": "company-chat-model",
            "version": "v1.2.0-dpo-int4",
            "artifact_type": "quantized_merged_model",
            "architecture": "decoder_only_transformer",
            "parameter_count": 7000000000,
            "owner": "model-platform",
            "stage": "production",
            "created_at": "2026-06-01T09:00:00Z",
            "train_job_id": "train_sft_042",
            "dataset_version": "pretrain_mix:2026-06-01-v2",
            "eval_report_id": "eval_chat_v120",
            "safety_report_id": "safety_chat_v120",
            "release_alias": "chat-prod",
            "rollback_alias": "chat-prod-previous",
        },
        "weights": {
            "format": "safetensors",
            "pickle_required": False,
            "security_scan_passed": True,
            "manifest_checksum": "sha256:manifest-model-v120",
            "total_size_gib": 14.0,
            "files": [
                {"path": "model-00001-of-00002.safetensors", "checksum": "sha256:w0", "size_gib": 7.0},
                {"path": "model-00002-of-00002.safetensors", "checksum": "sha256:w1", "size_gib": 7.0},
            ],
        },
        "tokenizer_config": {
            "tokenizer_hash": "sha256:tok-v4",
            "expected_tokenizer_hash": "sha256:tok-v4",
            "config_hash": "sha256:cfg-v120",
            "expected_config_hash": "sha256:cfg-v120",
            "chat_template_hash": "sha256:chat-template-v3",
            "special_tokens": ["bos", "eos", "pad"],
            "stop_tokens": ["<eos>"],
        },
    }

```

```

},
"adapter": {
  "method": "lora",
  "adapter_version": "adapter-dpo-v7",
  "base_version": "company-chat-model:v1.1.0-sft",
  "expected_base_version": "company-chat-model:v1.1.0-sft",
  "rank": 16,
  "hidden_size": 4096,
  "target_modules": ["q_proj", "v_proj"],
  "train_job_id": "train_dpo_007",
},
"merge": {
  "base_version": "company-chat-model:v1.1.0-sft",
  "adapter_version": "adapter-dpo-v7",
  "merged_model_version": "company-chat-model:v1.2.0-dpo",
  "merge_tool_version": "peft-merge-v2",
  "lineage_recorded": True,
  "eval_after_merge": True,
},
"quantization": {
  "source_version": "company-chat-model:v1.2.0-dpo",
  "quant_method": "awq",
  "bits": 4,
  "calibration_dataset": "calib_chat_2026_06",
  "tool_version": "awq-0.2",
  "runtime_target": "vllm",
  "base_quality": 0.842,
  "quant_quality": 0.834,
  "max_quality_drop": 0.015,
  "latency_speedup": 1.7,
  "eval_report_id": "eval_quant_v120",
},
"runtime": {
  "target_runtime": "vllm",
  "supported_runtimes": ["vllm", "tgi"],
  "quantization_supported": True,
  "required_context": 8192,
  "context_length": 8192,
  "tensor_parallel": 2,
  "available_gpus": 2,
  "artifact_size_gib": 14.0,
  "read_bandwidth_gib_s": 2.0,
  "verify_time_s": 2.0,
  "init_time_s": 11.0,
  "load_time_slo": 25.0,
  "cache_prepared": True,
  "prewarm_ready": True,
  "smoke_loaded": True,
},
"eval": {
  "required_tasks": ["chat", "tool_call", "safety", "latency"],
  "reports": {
    "chat": {"report_id": "eval_chat_v120", "score": 0.86, "threshold": 0.82},
    "tool_call": {"report_id": "eval_tool_v120", "score": 0.91, "threshold": 0.88},
    "safety": {"report_id": "safety_chat_v120", "score": 0.99, "threshold": 0.98},
  }
}

```

```

        "latency": {"report_id": "bench_chat_v120", "score": 0.94, "threshold": 0.90},
    },
    "security_permission": {
        "safety_approved": True,
        "license_ok": True,
        "classification": "internal",
        "authorized_projects": ["assistant-prod"],
        "download_roles": ["model-platform-admin"],
        "export_public": False,
        "audit_log": True,
    },
    "release": {
        "stage": "production",
        "approvers": ["model-owner", "safety-owner"],
        "state_transition_audit": True,
        "no_mutable_latest": True,
        "versions": ["company-chat-model:v1.1.0-sft", "company-chat-model:v1.2.0-dpo-int4"],
        "rollback_version": "company-chat-model:v1.1.0-sft",
        "rollback_load_test": True,
    },
    "lineage": {
        "nodes": sorted(REQUIRED_LINEAGE_NODES),
        "edges": list(REQUIRED_LINEAGE_EDGES),
        "events": [
            {"job": "train", "run_id": "train_sft_042", "inputs": ["dataset_version"], "outputs": ["checkpoint"]},
            {"job": "adapt", "run_id": "train_dpo_007", "inputs": ["model_version"], "outputs": ["adapter"]},
            {"job": "merge", "run_id": "merge_003", "inputs": ["model_version", "adapter"], "outputs": ["merged_model"]},
            {"job": "quantize", "run_id": "quant_002", "inputs": ["merged_model"], "outputs": ["quantized_model"]},
            {"job": "eval", "run_id": "eval_009", "inputs": ["quantized_model"], "outputs": ["eval_report"]},
        ],
    },
    "lifecycle": {
        "production_retention_years": 3,
        "candidate_retention_days": 30,
        "dependency_check_before_delete": True,
        "protected_from_delete": True,
        "archive_policy": "retain_manifest_and_eval",
    },
    "model_registry_gate": True,
}

```

```

def ratio(num, den):
    return 0.0 if den == 0 else num / den

```

```

def load_time(runtime):
    return runtime["artifact_size_gib"] / runtime["read_bandwidth_gib_s"] + runtime["verify_time_s"] + runtime["init_time_s"]

```

```

def model_version_contract(case):
    version = case["model_version"]
    return all(version.get(field) for field in REQUIRED_MODEL_FIELDS) and version["stage"] in {"registered", "evaluated"}

```



```

def weight_manifest_integrity(case):
    weights = case["weights"]
    files = weights["files"]
    unique_paths = len({f["path"] for f in files}) == len(files)
    size_match = abs(sum(f["size_gib"] for f in files) - weights["total_size_gib"]) < 1e-9
    checksums = bool(weights["manifest_checksum"]) and all(f.get("checksum") for f in files)
    return unique_paths and size_match and checksums

def safe_weight_format(case):
    weights = case["weights"]
    return weights["format"] == "safetensors" and not weights["pickle_required"] and weights["security_scan_passed"]

def tokenizer_config_binding(case):
    binding = case["tokenizer_config"]
    token_ok = binding["tokenizer_hash"] == binding["expected_tokenizer_hash"]
    config_ok = binding["config_hash"] == binding["expected_config_hash"]
    special_ok = {"bos", "eos"}.issubset(set(binding["special_tokens"])) and bool(binding["stop_tokens"])
    return token_ok and config_ok and bool(binding["chat_template_hash"]) and special_ok

def adapter_base_compatibility(case):
    adapter = case["adapter"]
    return (
        adapter["base_version"] == adapter["expected_base_version"]
        and adapter["rank"] <= adapter["hidden_size"]
        and bool(adapter["target_modules"])
        and bool(adapter["train_job_id"])
    )

def merge_lineage_completeness(case):
    merge = case["merge"]
    return (
        bool(merge["base_version"])
        and bool(merge["adapter_version"])
        and bool(merge["merged_model_version"])
        and bool(merge["merge_tool_version"])
        and merge["lineage_recorded"]
        and merge["eval_after_merge"]
    )

def quantization_eval_gate(case):
    quant = case["quantization"]
    quality_drop = quant["base_quality"] - quant["quant_quality"]
    return (
        bool(quant["source_version"])
        and bool(quant["calibration_dataset"])
        and bool(quant["tool_version"])
        and bool(quant["runtime_target"])
        and quality_drop <= quant["max_quality_drop"]
        and quant["latency_speedup"] > 1.0
    )

```

```

        and bool(quant["eval_report_id"])
    )

def runtime_compatibility_fit(case):
    runtime = case["runtime"]
    return (
        runtime["target_runtime"] in runtime["supported_runtimes"]
        and runtime["quantization_supported"]
        and runtime["context_length"] >= runtime["required_context"]
        and runtime["available_gpus"] >= runtime["tensor_parallel"]
        and runtime["smoke_loaded"]
    )

def eval_report_linkage(case):
    eval_info = case["eval"]
    reports = eval_info["reports"]
    required = eval_info["required_tasks"]
    task_ok = all(task in reports for task in required)
    score_ok = all(report["score"] >= report["threshold"] for report in reports.values())
    ids = {report["report_id"] for report in reports.values()}
    version = case["model_version"]
    linked = version["eval_report_id"] in ids and version["safety_report_id"] in ids
    return task_ok and score_ok and linked

def safety_permission_gate(case):
    policy = case["security_permission"]
    return (
        policy["safety_approved"]
        and policy["license_ok"]
        and policy["classification"] in {"internal", "public"}
        and bool(policy["authorized_projects"])
        and bool(policy["download_roles"])
        and not policy["export_public"]
        and policy["audit_log"]
    )

def release_status_governance(case):
    release = case["release"]
    return (
        release["stage"] == "production"
        and len(release["approvers"]) >= 2
        and release["state_transition_audit"]
        and release["no_mutable_latest"]
    )

def rollback_alias_readiness(case):
    release = case["release"]
    version = case["model_version"]
    return (
        bool(version["rollback_alias"])
    )

```

```

        and release["rollback_version"] in release["versions"]
        and release["rollback_load_test"]
    )

def artifact_lineage_completeness(case):
    lineage = case["lineage"]
    nodes_ok = REQUIRED_LINEAGE_NODES.issubset(set(lineage["nodes"]))
    edges_ok = all(edge in lineage["edges"] for edge in REQUIRED_LINEAGE_EDGES)
    events_ok = all(e.get("job") and e.get("run_id") and e.get("inputs") and e.get("outputs") for e in lineage["events"])
    return nodes_ok and edges_ok and events_ok

def load_cache_readiness(case):
    runtime = case["runtime"]
    return (
        load_time(runtime) <= runtime["load_time_slo"]
        and runtime["cache_prepared"]
        and runtime["prewarm_ready"]
    )

def lifecycle_retention_policy(case):
    lifecycle = case["lifecycle"]
    return (
        lifecycle["production_retention_years"] >= 1
        and lifecycle["candidate_retention_days"] >= 7
        and lifecycle["dependency_check_before_delete"]
        and lifecycle["protected_from_delete"]
        and bool(lifecycle["archive_policy"])
    )

def model_registry_gate(case):
    return case["model_registry_gate"] is True

CHECKS = [
    ("model_version_contract", model_version_contract),
    ("weight_manifest_integrity", weight_manifest_integrity),
    ("safe_weight_format", safe_weight_format),
    ("tokenizer_config_binding", tokenizer_config_binding),
    ("adapter_base_compatibility", adapter_base_compatibility),
    ("merge_lineage_completeness", merge_lineage_completeness),
    ("quantization_eval_gate", quantization_eval_gate),
    ("runtime_compatibility_fit", runtime_compatibility_fit),
    ("eval_report_linkage", eval_report_linkage),
    ("safety_permission_gate", safety_permission_gate),
    ("release_status_governance", release_status_governance),
    ("rollback_alias_readiness", rollback_alias_readiness),
    ("artifact_lineage_completeness", artifact_lineage_completeness),
    ("load_cache_readiness", load_cache_readiness),
    ("lifecycle_retention_policy", lifecycle_retention_policy),
    ("model_registry_gate", model_registry_gate),
]

```

```

def make_bad_cases(base):
    cases = []

    def add(name, change):
        case = deepcopy(base)
        case["name"] = name
        change(case)
        cases.append(case)

    add("model_contract_missing_bad", lambda c: c["model_version"].update({"owner": ""}))
    add("weight_checksum_missing_bad", lambda c: c["weights"]["files"][0].update({"checksum": ""}))
    add("unsafe_weight_format_bad", lambda c: c["weights"].update({"format": "bin", "pickle_required": True}))
    add("tokenizer_hash_mismatch_bad", lambda c: c["tokenizer_config"].update({"tokenizer_hash": "sha256:wrong"}))
    add("adapter_base_mismatch_bad", lambda c: c["adapter"].update({"base_version": "company-chat-model:v0.9.0"}))
    add("merge_lineage_missing_bad", lambda c: c["merge"].update({"lineage_recorded": False}))
    add("quant_quality_drop_bad", lambda c: c["quantization"].update({"quant_quality": 0.80}))
    add("runtime_incompatible_bad", lambda c: c["runtime"].update({"supported_runtimes": ["tgi"]}))
    add("eval_report_missing_bad", lambda c: c["eval"]["reports"].pop("tool_call"))
    add("permission_open_bad", lambda c: c["security_permission"].update({"export_public": True}))
    add("release_unapproved_bad", lambda c: c["release"].update({"approvers": ["model-owner"]}))
    add("rollback_alias_missing_bad", lambda c: c["release"].update({"rollback_version": "company-chat-model:missing"}))
    add("artifact_lineage_edge_missing_bad", lambda c: c["lineage"]["edges"].remove(("merged_model", "quantized_model")))
    add("load_cache_not_ready_bad", lambda c: c["runtime"].update({"cache_prepared": False}))
    add("lifecycle_policy_missing_bad", lambda c: c["lifecycle"].update({"protected_from_delete": False}))
    add("registry_gate_missing_bad", lambda c: c.update({"model_registry_gate": False}))
    return cases

def summarize_examples(case):
    weights = case["weights"]
    quant = case["quantization"]
    runtime = case["runtime"]
    required_fields = sum(1 for field in REQUIRED_MODEL_FIELDS if case["model_version"].get(field))
    return {
        "artifact_metadata_completeness": round(ratio(required_fields, len(REQUIRED_MODEL_FIELDS)), 3),
        "weight_checksum_coverage": round(ratio(sum(1 for f in weights["files"] if f["checksum"]), len(weights["files"])), 3),
        "weight_total_size_gib": weights["total_size_gib"],
        "tokenizer_config_binding": tokenizer_config_binding(case),
        "adapter_rank_ratio": round(ratio(case["adapter"]["rank"], case["adapter"]["hidden_size"]), 4),
        "quant_quality_drop": round(quant["base_quality"] - quant["quant_quality"], 3),
        "quant_latency_speedup": quant["latency_speedup"],
        "estimated_load_time_s": round(load_time(runtime), 2),
        "load_time_slo": runtime["load_time_slo"],
        "lineage_node_count": len(case["lineage"]["nodes"]),
        "release_alias": case["model_version"]["release_alias"],
        "rollback_alias": case["model_version"]["rollback_alias"],
    }

def audit_model_artifact_registry(cases):
    per_case = {
        case["name"]: {name: check(case) for name, check in CHECKS}
    }
    for case in cases

```

```

}
metrics = {
    name: round(sum(result[name] for result in per_case.values()) / len(cases), 3)
    for name, _ in CHECKS
}
failed_cases = [name for name, result in per_case.items() if not all(result.values())]
failed_gates = [name for name, score in metrics.items() if score < 1.0]
return {
    "smoke": {
        "complete_case_passes": all(per_case["complete_case"].values()),
        "caught_weight_gap": "weight_checksum_missing_bad" in failed_cases,
        "caught_tokenizer_gap": "tokenizer_hash_mismatch_bad" in failed_cases,
        "caught_adapter_gap": "adapter_base_mismatch_bad" in failed_cases,
        "caught_quant_gap": "quant_quality_drop_bad" in failed_cases,
        "caught_runtime_gap": "runtime_incompatible_bad" in failed_cases,
        "caught_release_gap": "release_unapproved_bad" in failed_cases,
    },
    "metrics": metrics,
    "hard_blocker_count": len(failed_cases),
    "failed_cases": failed_cases,
    "failed_gates": failed_gates,
    "model_registry_gate_pass": len(failed_cases) == 0,
}

```

```

base = complete_case()
cases = [base] + make_bad_cases(base)
report = audit_model_artifact_registry(cases)

pprint({"model_registry_examples": summarize_examples(base)})
pprint(report["smoke"])
pprint(report["metrics"])
print("hard_blocker_count=", report["hard_blocker_count"])
print("failed_cases=", report["failed_cases"])
print("failed_gates=", report["failed_gates"])
print("model_registry_gate_pass=", report["model_registry_gate_pass"])

assert report["smoke"]["complete_case_passes"]
assert report["smoke"]["caught_runtime_gap"]
assert report["metrics"]["model_version_contract"] == 0.941
assert report["metrics"]["model_registry_gate"] == 0.941
assert report["hard_blocker_count"] == 16
assert report["model_registry_gate_pass"] is False

```

这个 demo 的重点不是复刻真实模型平台，而是把“模型仓库”拆成可检查证据：ModelVersion 契约、权重 manifest、safetensors 安全格式、tokenizer / config 绑定、adapter 和 base model 兼容、merge lineage、量化评估、runtime 兼容、评估报告、安全权限、发布状态、回滚 alias、artifact lineage、加载缓存、生命周期和最终 registry gate。

38.25 常见误区

误区一：模型仓库就是对象存储目录。

对象存储只能存文件，模型仓库还要管理版本、元数据、血缘、评估、权限和发布状态。

误区二：tokenizer 可以随便换。

tokenizer 和模型权重强绑定，换错会导致严重输出问题。

误区三：量化版本只看速度。

量化还要看质量退化、runtime 兼容性和目标场景。

误区四：adapter 是独立模型。

adapter 依赖 base model，必须记录兼容关系。

误区五：模型删除只看文件是否没人用。

还要看 deployment、回滚、审计、合规和 lineage 依赖。

38.26 面试常见追问

问题一：模型仓库和对象存储有什么区别？

可以回答：对象存储只负责保存文件，模型仓库还管理 ModelVersion、tokenizer、config、adapter、量化版本、评估报告、权限、血缘、发布状态和审计。

问题二：为什么 tokenizer 要版本化？

可以回答：tokenizer 决定文本和 token 的映射，特殊 token、chat template 和 stop token 都会影响模型行为。tokenizer 与权重不匹配会导致乱码、格式错误或能力退化。

问题三：如何管理 LoRA adapter？

可以回答：记录 adapter 版本、base model 版本、训练数据、训练任务、权重路径、兼容性、是否 merge 以及评估报告。adapter 不能脱离 base model 单独管理。

问题四：量化模型上线前要检查什么？

可以回答：检查原始模型版本、量化方法、校准数据、量化工具版本、runtime 兼容性、质量评估、延迟吞吐 benchmark 和成本收益。

38.27 小练习

1. Model 和 ModelVersion 有什么区别？
2. ModelVersion 应该记录哪些元数据？
3. 为什么权重文件需要 checksum？
4. tokenizer 和 chat template 为什么要绑定模型版本？
5. LoRA adapter 和 merge 后模型如何管理血缘？
6. 量化版本为什么需要单独评估？
7. 模型仓库如何和训练平台、评估平台、推理平台联动？
8. 模型生命周期管理要考虑哪些依赖？

38.28 本章小结

本章讲了模型仓库。

你需要记住：

1. 模型仓库不是普通目录，而是模型资产的版本、元数据、权限、血缘、评估和发布状态管理系统。
2. Model 是逻辑模型，ModelVersion 是具体版本，Artifact 是实际文件集合。
3. 权重、tokenizer、config、adapter 和量化版本都必须被纳入版本管理。
4. Adapter 依赖 base model，量化版本依赖原始模型和量化配置，它们都需要血缘关系。
5. 模型仓库连接训练、评估和推理平台，是模型资产生命周期的中心。

下一章我们会讲 Artifact 管理：dataset、checkpoint、eval report 和 deployment package。

第 39 章 Artifact 管理: dataset、checkpoint、eval report 和 deployment package

上一章讲了模型仓库。本章讲一个更通用的概念: Artifact 管理。

在 AI Infra 中, 训练、评估、部署、回滚、审计都围绕各种产物流转。dataset、checkpoint、eval report、deployment package、日志、配置、模型权重都可以看作 artifact。

先记住一句话:

Artifact 管理的目标, 是让 AI 研发过程中的每个关键产物都可定位、可校验、可复现、可追溯、可治理。

39.0 本讲资料边界与第二轮精修口径

本讲按截至 2026-06 的稳定公开资料校准: MLflow Tracking 对 run、artifact store、backend store、参数、指标和 artifact 的实验追踪抽象, Weights & Biases Artifacts 对数据集、模型、文件集合、版本和 lineage 的产物管理口径, OpenLineage 对 Job、Run、Dataset 和事件血缘的抽象, Amazon S3 object integrity 对 checksum 的对象完整性口径, PyTorch Distributed Checkpoint 对分布式 checkpoint 保存 / 加载的状态管理边界, 以及 OCI image / manifest 对 deployment package 中容器镜像与内容寻址的工程边界。

本章只抽象大模型平台里的通用 artifact 管理能力, 不绑定某个实验追踪产品、对象存储、镜像仓库、模型仓库、评估平台或内部发布系统。

和上一章的分工是:

1. 第 38 章聚焦模型仓库, 回答 “权重、tokenizer、config、adapter 和量化版本如何作为模型资产被管理”。
2. 本章扩展到更通用的 artifact 管理, 回答 “dataset、checkpoint、eval report、deployment package、日志、配置和 release manifest 如何被统一注册、校验、追踪、晋级、回滚和清理”。

39.1 什么是 Artifact

Artifact 可以理解为一次计算过程产生或依赖的可保存产物。

在 AI Infra 中常见 artifact 包括:

1. dataset。
2. dataset manifest。
3. checkpoint。
4. model weights。
5. tokenizer。
6. config。
7. adapter。
8. eval report。
9. benchmark report。
10. deployment package。
11. container image。
12. training log。
13. trace。
14. experiment summary。

Artifact 不是简单文件, 它还应该带有元数据、版本、校验和血缘。

39.2 为什么 Artifact 管理重要

没有 Artifact 管理, 会出现这些问题:

1. 不知道某个 checkpoint 是哪个训练任务产出的。

2. 不知道某个模型用了哪个 dataset。
3. eval report 和模型版本对不上。
4. 部署包里 tokenizer 和权重不匹配。
5. 线上出问题无法回滚。
6. 存储里堆满没人敢删的文件。
7. 审计时无法证明模型来源。

Artifact 管理是 AI 工程可复现性的基础。

39.3 Artifact 的基本元数据

一个 artifact 至少要有：

1. artifact ID。
2. artifact type。
3. name。
4. version。
5. storage URI。
6. checksum。
7. size。
8. created time。
9. created by。
10. producer job。
11. input dependencies。
12. permissions。
13. lifecycle state。
14. tags。

有了这些字段，平台才能进行查询、校验、权限控制和生命周期管理。

一个 artifact 记录可以抽象为：

$A_i = (u_i, \tau_i, n_i, v_i, r_i, h_i, s_i, o_i, d_i, p_i, l_i, g_i, e_i, z_i)$

其中 u_i 是 artifact id, τ_i 是 artifact type, n_i 是名称, v_i 是版本, r_i 是存储 URI, h_i 是 checksum 或 digest, s_i 是大小, o_i 是 owner / producer job, d_i 是输入依赖, p_i 是权限策略, l_i 是 lifecycle state, g_i 是 lineage 事件, e_i 是评估、发布或审计证据, z_i 是 tag 和审计日志。

元数据完整率可以写成：

$C_{\{\mathrm{meta}\}} = \frac{K_{\{\mathrm{filled}\}}}{K_{\{\mathrm{required}\}}}$

其中 K_{filled} 是已填写的必需字段数, K_{required} 是该 artifact type 的必需字段总数。这个公式的直觉是：artifact 不是“有路径就够”，而是要有足够证据支撑复现、权限、血缘、回滚和删除判断。

39.4 Artifact 类型

常见类型可以分为几类。

数据类：

1. raw dataset。
2. cleaned dataset。
3. dataset manifest。
4. annotation file。
5. synthetic data。

模型类：

1. checkpoint。
2. final weights。
3. adapter。

4. tokenizer。
5. model config。
6. quantized weights。

评估类：

1. eval report。
2. benchmark result。
3. safety report。
4. human review result。

部署类：

1. deployment package。
2. container image。
3. runtime config。
4. release manifest。

39.5 Dataset Artifact

Dataset artifact 是训练和评估的输入。

它不一定是数据本体，也可以是 manifest。

Dataset artifact 应记录：

1. dataset name。
2. version。
3. split。
4. file list。
5. schema。
6. sample count。
7. token count。
8. source composition。
9. cleaning rule version。
10. dedup rule version。
11. quality metrics。
12. permissions。
13. checksum。

训练任务引用 dataset artifact，而不是裸路径。

Dataset manifest 完整率可以写成：

$$C_{\{\mathrm{dataset}\}} = \frac{N_{\{\mathrm{manifest}, \mathrm{filled}\}}}{N_{\{\mathrm{manifest}, \mathrm{required}\}}}$$

其中 $N_{\mathrm{manifest}, \mathrm{filled}}$ 是 dataset manifest 中已填写的必要字段数， $N_{\mathrm{manifest}, \mathrm{required}}$ 是 dataset artifact 发布前必须记录的字段总数。对训练输入来说，manifest 比目录路径更重要，因为它冻结了文件集合、schema、样本数、token 数、来源组成、规则版本和权限快照。

39.6 Checkpoint Artifact

Checkpoint artifact 是训练过程中保存的模型状态。

它可能包含：

1. model weights。
2. optimizer state。
3. scheduler state。
4. dataloader state。
5. random state。
6. training step。

7. config。
8. distributed training metadata。

Checkpoint 有两种用途：

1. 断点恢复。
2. 作为候选模型版本。

两者要求不同。

断点恢复 checkpoint 需要 optimizer 和随机状态。

候选模型 checkpoint 更关注权重、tokenizer、config 和评估。

39.7 Checkpoint 元数据

Checkpoint artifact 应记录：

1. training job ID。
2. global step。
3. epoch。
4. loss。
5. learning rate。
6. dataset version。
7. code version。
8. config version。
9. parent checkpoint。
10. file shards。
11. checksum。
12. save time。
13. retention policy。

这样才能判断某个 checkpoint 是否可恢复、是否值得评估、是否可以清理。

Checkpoint 恢复完整率可以写成：

$$C_{\{\mathrm{ckpt}\}} = \frac{N_{\{\mathrm{state}, \mathrm{present}\}}}{N_{\{\mathrm{state}, \mathrm{required}\}}}$$

其中 $N_{\mathrm{state}, \mathrm{present}}$ 是 checkpoint 中实际保存的状态项数， $N_{\mathrm{state}, \mathrm{required}}$ 是恢复训练所需状态项数。用于断点恢复时，模型权重、optimizer、scheduler、dataloader cursor、随机数状态和分布式 metadata 都应进入检查；用于候选模型时，权重、tokenizer、config、eval report 和 release metadata 更关键。

39.8 Eval Report Artifact

Eval report artifact 是模型评估结果。

它应绑定：

1. model version 或 checkpoint。
2. eval dataset version。
3. eval code version。
4. metric definitions。
5. prompt template version。
6. inference config。
7. runtime version。
8. result summary。
9. sample-level outputs。
10. failure cases。
11. evaluator version。

如果 eval report 不记录这些信息，评估结果不可复现。

同一个模型用不同 prompt、不同 temperature、不同 eval 集，分数可能完全不同。

Eval report 可复现门禁可以写成：

$$G_{\{\mathrm{eval}\}}=\mathbf{1}[h_{\{\mathrm{model}\}}=h_{\{\mathrm{model},\mathrm{expected}\}}\wedge h_{\{\mathrm{data}\}}=h_{\{\mathrm{data},\mathrm{expected}\}}]$$

其中四个 hash 分别对应模型、评估数据、评估代码和 prompt / template。这个门禁强调：eval report 不是一个孤立分数，而是一组可复现实验条件和样本级证据。

39.9 Deployment Package Artifact

Deployment package 是部署推理服务所需的一组文件和配置。

它可以包含：

1. model weights。
2. tokenizer。
3. model config。
4. runtime config。
5. container image。
6. health check config。
7. default generation config。
8. safety policy。
9. route config。
10. release manifest。

部署包的目标是保证线上部署可复现。

不要让线上服务临时拼装权重、tokenizer 和配置。

Deployment package 完整率可以写成：

$$C_{\{\mathrm{release}\}}=\frac{N_{\{\mathrm{artifact},\mathrm{present}\}}}{N_{\{\mathrm{artifact},\mathrm{required}\}}}$$

其中 $N_{\mathrm{artifact},\mathrm{present}}$ 是部署包中实际存在并通过校验的 artifact 数， $N_{\mathrm{artifact},\mathrm{required}}$ 是上线所需 artifact 总数。生产部署通常要求权重、tokenizer、model config、runtime config、image digest、health check、generation config、安全策略、route policy 和 release manifest 都可定位、可校验。

39.10 Release Manifest

Release manifest 描述一次发布使用了哪些 artifact。

例如：

```
release: chat-model-prod-2026-05-30
model_version: chat-model:v1.3.0
weights: artifact://weights/abc
tokenizer: artifact://tokenizer/def
runtime_image: registry/runtime:v0.8.1
runtime_config: artifact://config/ghi
safety_policy: policy:v12
route_policy: route:v7
eval_report: eval:v1.3.0-prod-gate
```

有了 release manifest，回滚和审计都会简单很多。

39.11 Artifact 血缘

Artifact 之间应该有依赖关系。

例如：

```
dataset:v1 + config:v3 + code:commit_x
-> training_job:123
-> checkpoint:step_10000
-> eval_report:abc
-> model_version:v2
-> deployment_package:p1
-> deployment:prod
```

这条链路就是 artifact lineage。

它能帮助你从线上部署追溯到训练数据，也能从某个数据问题找到受影响的模型。

Artifact 血缘图可以写成：

$$G_{\{\mathrm{art}\}}=(V_{\{\mathrm{art}\}},E_{\{\mathrm{dep}\}})$$

其中 V_{art} 是 artifact、job、run、deployment 等节点集合， E_{dep} 是输入、输出、派生、评估、发布和部署依赖边。血缘覆盖率可以写成：

$$C_{\{\mathrm{lineage}\}}=\frac{N_{\{\mathrm{linked}\}}}{N_{\{\mathrm{expected}\}}}$$

其中 N_{linked} 是已经能追踪的依赖节点或边数量， N_{expected} 是按产物类型应追踪的依赖数量。

39.12 Artifact Store 和 Metadata Store

Artifact 管理通常分成两层：

1. Artifact Store：存大文件。
2. Metadata Store：存元数据和索引。

Artifact Store 可以是对象存储、共享文件系统、镜像仓库或专用模型存储。

Metadata Store 存：

1. artifact ID。
2. 类型。
3. 版本。
4. URI。
5. checksum。
6. 依赖关系。
7. 权限。
8. 生命周期状态。

不要试图把大模型权重直接塞进数据库。

39.13 Checksum 和完整性校验

Artifact 必须支持完整性校验。

原因：

1. 大文件传输可能失败。
2. 分片文件可能缺失。
3. 存储可能出现不一致。
4. 部署前需要确认文件没被篡改。
5. 审计需要证明 artifact 未变化。

常见做法：

1. 每个文件有 checksum。
2. manifest 有整体 checksum。
3. 上传完成后校验。
4. 下载或部署前校验。

5. 校验失败则拒绝使用。

checksum 是 AI 资产可信的底线。

文件级 checksum 覆盖率可以写成：

$$C_{\{\mathrm{checksum}\}} = \frac{N_{\{\mathrm{checked}\}}}{N_{\{\mathrm{files, total}\}}}$$

其中 N_{checked} 是已经完成 checksum 校验的文件或 shard 数， $N_{\mathrm{files, total}}$ 是 manifest 中声明的文件总数。大文件、分片 checkpoint、deployment package 和 eval sample outputs 都应该进入这个口径。

39.14 Artifact 权限

Artifact 权限和数据权限、模型权限一样重要。

权限问题包括：

1. 谁能读取 dataset artifact。
2. 谁能下载 checkpoint。
3. 谁能查看 eval sample outputs。
4. 谁能部署 deployment package。
5. 谁能删除 artifact。
6. 哪些 artifact 不能跨租户使用。

Eval report 中的 sample-level outputs 可能包含敏感 prompt 或模型输出，也需要权限控制。

39.15 Artifact 生命周期

Artifact 生命周期可以是：

created -> validated -> active -> deprecated -> archived -> deleted

不同 artifact 生命周期策略不同。

Dataset：生产数据版本需要长期保留。

Checkpoint：中间 checkpoint 可以按策略清理。

Eval report：与发布相关的报告要保留。

Deployment package：线上和可回滚版本必须保留。

删除前要做依赖检查。

39.16 Retention Policy

保留策略可以按类型制定。

示例：

1. production deployment package 永久保留或长期保留。
2. 最近 5 个 production model version 保留。
3. 每个训练任务保留 best checkpoint 和 latest checkpoint。
4. 中间 checkpoint 保留 30 天。
5. failed job artifact 保留 7 天。
6. 安全审计相关 artifact 冻结删除。

没有 retention policy，存储成本会不可控。

策略太激进，又可能破坏复现和回滚能力。

存储保留成本可以粗略写成：

$$K_{\{\mathrm{store}\}} = \sum_{a=1}^A S_a P_a T_a + K_{\{\mathrm{request}\}} + K_{\{\mathrm{egress}\}} + K_{\{\mathrm{ops}\}}$$

其中 S_a 是第 a 类 artifact 的存储大小, P_a 是单位存储价格, T_a 是保留时长, 后面三项分别是请求、跨区传输和运维成本。这个公式说明 retention policy 是工程和成本问题, 不是“全部保留”或“按时间删除”二选一。

删除安全门禁可以写成:

$$G_{\{\mathrm{delete}\}}=\mathbf{1}[N_{\{\mathrm{dependents}\}}=0 \wedge T_{\{\mathrm{age}\}}\geq T_{\{\mathrm{retention}\}} \wedge H_{\{\mathrm{legal}\}}=1]$$

其中 $N_{\mathrm{dependents}}$ 是依赖该 artifact 的下游对象数量, T_{age} 是 artifact 年龄, $T_{\mathrm{retention}}$ 是保留策略要求, H_{legal} 是合规或审计冻结标记, A_{audit} 是删除审计记录是否完整。

39.17 Artifact Promotion

Artifact promotion 是把产物从低级状态提升到高级状态。

例如:

checkpoint -> candidate model -> evaluated model -> release candidate -> production package

每次 promotion 都应有门禁:

1. checkpoint 完整性校验。
2. eval 通过。
3. safety 通过。
4. runtime 兼容性通过。
5. 审批通过。
6. 发布状态更新。

Promotion 让产物流转有流程, 而不是谁都能拿一个 checkpoint 去部署。

Artifact 晋级门禁可以写成:

$$G_{\{\mathrm{promo}\}}=\mathbf{1}[\min_j C_j\geq\tau_j \wedge C_{\{\mathrm{checksum}\}}=1 \wedge C_{\{\mathrm{lineage}\}}\geq\rho \wedge P_0=0]$$

其中 C_j 是每个 artifact 检查维度的覆盖率, τ_j 是阈值, $R_{\mathrm{permission}}$ 是权限违规率, P_0 是未解决的 P0 风险数量。这个门禁适用于 checkpoint 晋级为候选模型、eval report 晋级为发布证据、deployment package 晋级为生产发布包。

39.18 Artifact 和实验追踪的关系

实验追踪记录一次实验的参数、指标和日志。

Artifact 管理记录实验产生和依赖的产物。

二者应该关联。

例如一个 experiment run 关联:

1. 输入 dataset artifact。
2. 输入 config artifact。
3. 输出 checkpoint artifact。
4. 输出 eval report artifact。
5. 输出 logs artifact。

这样才能从实验结果直接定位产物。

39.19 Artifact 和回滚

回滚依赖 artifact 可用。

如果旧版本 deployment package 被删除, 回滚就会失败。

回滚需要:

1. 旧模型权重。
2. 旧 tokenizer。
3. 旧 runtime image。
4. 旧配置。
5. 旧安全策略。
6. 旧路由策略。
7. 旧 release manifest。

所以 production artifact 不能随便清理。

39.20 Artifact 命名和路径规范

命名规范能降低混乱。

示例：

```
artifact://model/chat-model/v1.3.0/weights
artifact://dataset/pretrain-mix/2026-05-30-v1/manifest
artifact://eval/chat-model/v1.3.0/safety-eval-2026-05-30
artifact://release/chat-model/prod/2026-05-30-r1
```

路径中应体现类型、名称、版本和用途。

但不要只依赖路径表达元数据，核心元数据仍要进入 metadata store。

39.21 Artifact 管理系统架构

一个 Artifact 管理系统可以这样设计：

```
Artifact API
-> Metadata DB
-> Artifact Store
-> Manifest Store
-> Checksum Service
-> Lineage Graph
-> Access Control
-> Lifecycle Manager
-> Audit Log
```

Artifact API 提供注册、查询、下载、校验、promote、archive 和删除接口。

Lifecycle Manager 负责 retention 和依赖检查。

39.22 Artifact 管理审计指标和最小 demo

Artifact 管理的最终门禁可以写成：

$$G_{\{\mathrm{artifact}\}} = \mathbf{1}[\min_j C_j \geq \tau_j \wedge C_{\{\mathrm{checksum}\}} = 1 \wedge C_{\{\mathrm{lineage}\}} \geq \rho]$$

其中 C_j 是各类 artifact 的审计覆盖率， C_{checksum} 是文件级完整性校验， C_{lineage} 是血缘覆盖率， C_{release} 是 deployment package / release manifest 完整率， G_{delete} 是删除安全门禁，P0 是未解决的高风险问题。这个门禁强调：artifact 管理不是文件系统整理，而是训练、评估、发布、回滚、审计和成本治理的共同基础。

下面是一个 0 依赖 toy demo。它把 dataset、checkpoint、eval report、deployment package 和 release manifest 放进同一张审计表，并构造 16 个 bad case，覆盖元数据缺失、dataset manifest 错误、checkpoint 不可恢复、eval report 不可复现、部署包缺文件、release manifest 可变、checksum 未验证、血缘断边、metadata / blob 分层错误、权限放开、retention 缺失、promotion 缺门禁、实验追踪未绑定、回滚 artifact 缺失、删除依赖不安全和最终 artifact gate 缺失。

```
from copy import deepcopy
from pprint import pprint
```

```

REQUIRED_ARTIFACT_FIELDS = [
    "artifact_id",
    "artifact_type",
    "name",
    "version",
    "uri",
    "checksum",
    "size_gib",
    "created_at",
    "created_by",
    "producer_job",
    "dependencies",
    "permissions",
    "lifecycle_state",
    "tags",
]

REQUIRED_RELEASE_REFS = {
    "weights",
    "tokenizer",
    "model_config",
    "runtime_config",
    "runtime_image",
    "safety_policy",
    "route_policy",
    "eval_report",
    "deployment_package",
}

REQUIRED_LINEAGE_NODES = {
    "dataset_artifact",
    "config_artifact",
    "training_job",
    "checkpoint_artifact",
    "eval_job",
    "eval_report_artifact",
    "model_version",
    "deployment_package",
    "release_manifest",
    "deployment",
}

REQUIRED_LINEAGE_EDGES = [
    ("dataset_artifact", "training_job"),
    ("config_artifact", "training_job"),
    ("training_job", "checkpoint_artifact"),
    ("checkpoint_artifact", "eval_job"),
    ("eval_job", "eval_report_artifact"),
    ("eval_report_artifact", "model_version"),
    ("model_version", "deployment_package"),
    ("deployment_package", "release_manifest"),
    ("release_manifest", "deployment"),
]

```



```

def complete_case():
    return {
        "name": "complete_case",
        "artifact": {
            "artifact_id": "artifact_release_chat_v120",
            "artifact_type": "deployment_package",
            "name": "chat-model-prod-package",
            "version": "2026-06-01-r1",
            "uri": "artifact://release/chat-model/prod/2026-06-01-r1",
            "checksum": "sha256:release-package",
            "size_gib": 18.0,
            "created_at": "2026-06-01T10:00:00Z",
            "created_by": "release-bot",
            "producer_job": "release_job_009",
            "dependencies": ["weights", "tokenizer", "runtime_image", "eval_report"],
            "permissions": ["assistant-prod", "model-platform"],
            "lifecycle_state": "active",
            "tags": ["prod", "chat", "rollback-ready"],
        },
        "dataset": {
            "dataset_id": "pretrain_mix",
            "version": "2026-06-01-v2",
            "split": "train",
            "schema_version": "schema_v3",
            "sample_count": 2000,
            "token_count": 1320000,
            "source_composition": {"web": 0.5, "code": 0.3, "books": 0.2},
            "cleaning_rule_version": "clean_v17",
            "dedup_rule_version": "dedup_v9",
            "quality_metrics": {"schema_valid_rate": 0.9995, "duplicate_rate": 0.02},
            "permissions": {"classification": "internal", "allowed_purpose": ["training"]},
            "checksum": "sha256:dataset-manifest",
            "files": [
                {"path": "shard-000.parquet", "checksum": "sha256:ds0", "samples": 1000, "tokens": 700000},
                {"path": "shard-001.parquet", "checksum": "sha256:ds1", "samples": 1000, "tokens": 620000},
            ],
        },
        "checkpoint": {
            "training_job_id": "train_042",
            "global_step": 12000,
            "epoch": 2,
            "loss": 1.92,
            "learning_rate": 0.00012,
            "dataset_version": "pretrain_mix:2026-06-01-v2",
            "code_version": "commit_abc123",
            "config_version": "cfg_v8",
            "parent_checkpoint": "ckpt_11000",
            "model_state": True,
            "optimizer_state": True,
            "scheduler_state": True,
            "dataloader_state": True,
            "rng_state": True,
            "distributed_metadata": True,
        }
    }

```

```

    "shards": ["rank0.pt", "rank1.pt"],
    "checksums": {"rank0.pt": "sha256:c0", "rank1.pt": "sha256:c1"},
    "restore_tested": True,
    "purpose": ["resume", "candidate_model"],
  },
  "eval_report": {
    "model_ref": "checkpoint:train_042:12000",
    "eval_dataset_version": "eval_chat_2026_06",
    "eval_code_version": "eval_commit_777",
    "metric_definitions": ["chat_quality", "tool_call", "safety", "latency"],
    "prompt_template_version": "prompt_v5",
    "inference_config": {"temperature": 0.0, "max_tokens": 512},
    "runtime_version": "vllm_0_9",
    "result_summary": {"chat_quality": 0.86, "tool_call": 0.91, "safety": 0.99, "latency": 0.94},
    "sample_outputs_uri": "artifact://eval/chat/v120/samples",
    "failure_cases": ["tool_arg_schema_edge"],
    "evaluator_version": "judge_v4",
    "report_checksum": "sha256:eval-report",
  },
  "deployment_package": {
    "weights": "artifact://model/chat/v120/weights",
    "tokenizer": "artifact://model/chat/v120/tokenizer",
    "model_config": "artifact://model/chat/v120/config",
    "runtime_config": "artifact://runtime/chat/v120/config",
    "container_image_digest": "sha256:runtime-image-v120",
    "health_check": "artifact://release/chat/v120/health",
    "generation_config": "artifact://release/chat/v120/generation",
    "safety_policy": "policy:v12",
    "route_config": "route:v7",
    "release_manifest": "artifact://release/chat/prod/2026-06-01-r1/manifest",
  },
  "release_manifest": {
    "release_id": "chat-model-prod-2026-06-01-r1",
    "artifact_refs": {
      "weights": "artifact://model/chat/v120/weights",
      "tokenizer": "artifact://model/chat/v120/tokenizer",
      "model_config": "artifact://model/chat/v120/config",
      "runtime_config": "artifact://runtime/chat/v120/config",
      "runtime_image": "sha256:runtime-image-v120",
      "safety_policy": "policy:v12",
      "route_policy": "route:v7",
      "eval_report": "artifact://eval/chat/v120/report",
      "deployment_package": "artifact://release/chat/prod/2026-06-01-r1",
    },
    "checksum": "sha256:release-manifest",
    "immutable": True,
    "rollback_target": "chat-model-prod-2026-05-25-r3",
  },
  "integrity": {
    "checksums_verified": True,
    "manifest_checksum_verified": True,
    "download_verify_required": True,
    "failed_files": [],
  },
  "lineage": {

```

```

    "nodes": sorted(REQUIRED_LINEAGE_NODES),
    "edges": list(REQUIRED_LINEAGE_EDGES),
    "events": [
        {"job": "train", "run_id": "train_042", "inputs": ["dataset_artifact", "config_artifact"], "outputs": []},
        {"job": "eval", "run_id": "eval_120", "inputs": ["checkpoint_artifact"], "outputs": ["eval_report_artifact"]},
        {"job": "package", "run_id": "release_job_009", "inputs": ["model_version", "eval_report_artifact"], "outputs": []},
    ],
},
"store": {
    "artifact_store_for_blobs": True,
    "metadata_store_for_index": True,
    "large_blob_in_metadata_db": False,
    "indexed_fields": ["artifact_id", "artifact_type", "version", "checksum", "lifecycle_state", "owner"],
},
"access": {
    "rbac_enabled": True,
    "sample_outputs_restricted": True,
    "cross_tenant_blocked": True,
    "delete_requires_approval": True,
    "public_download_allowed": False,
    "audit_log": True,
},
"lifecycle": {
    "policies": {
        "dataset": {"retention_days": 1095, "dependency_check": True},
        "checkpoint": {"retention_days": 30, "keep_best": True, "keep_latest": True},
        "eval_report": {"retention_days": 1095, "dependency_check": True},
        "deployment_package": {"retention_days": 1095, "dependency_check": True},
    },
    "legal_hold": False,
    "archive_tier": "warm_then_cold",
},
"promotion": {
    "sequence": ["checkpoint", "candidate_model", "evaluated_model", "release_candidate", "production_package"],
    "checks": {"integrity": True, "eval": True, "safety": True, "runtime": True, "approval": True},
    "approvers": ["model-owner", "safety-owner"],
},
"experiment": {
    "run_id": "run_train_042",
    "params_logged": True,
    "metrics_logged": True,
    "logs_uri": "artifact://logs/run_train_042",
    "input_artifacts": ["dataset_artifact", "config_artifact"],
    "output_artifacts": ["checkpoint_artifact", "eval_report_artifact"],
},
"rollback": {
    "rollback_package": "artifact://release/chat/prod/2026-05-25-r3",
    "required_artifacts_available": True,
    "rollback_manifest_checksum": "sha256:rollback-manifest",
    "load_test_passed": True,
},
"delete_plan": {
    "candidate_artifact": "checkpoint:train_042:step_8000",
    "dependent_count": 0,
    "retention_met": True,
}

```

```

        "legal_hold": False,
        "audit_record": True,
        "protected_artifact_blocked": True,
    },
    "inventory": [
        {"artifact_type": "dataset", "size_gib": 1200, "price_per_gib_month": 0.012, "retention_months": 36},
        {"artifact_type": "checkpoint", "size_gib": 600, "price_per_gib_month": 0.012, "retention_months": 1},
        {"artifact_type": "eval_report", "size_gib": 2, "price_per_gib_month": 0.012, "retention_months": 36},
        {"artifact_type": "deployment_package", "size_gib": 18, "price_per_gib_month": 0.012, "retention_months": 36},
    ],
    "artifact_management_gate": True,
}

```

```

def ratio(num, den):
    return 0.0 if den == 0 else num / den

```

```

def artifact_storage_cost(case):
    return sum(item["size_gib"] * item["price_per_gib_month"] * item["retention_months"] for item in case["inventory"])

```

```

def artifact_metadata_contract(case):
    artifact = case["artifact"]
    fields_ok = all(artifact.get(field) for field in REQUIRED_ARTIFACT_FIELDS)
    return fields_ok and artifact["uri"].startswith("artifact://") and artifact["lifecycle_state"] in {"created", "valid"}

```

```

def dataset_artifact_manifest(case):
    dataset = case["dataset"]
    files = dataset["files"]
    samples_match = sum(f["samples"] for f in files) == dataset["sample_count"]
    tokens_match = sum(f["tokens"] for f in files) == dataset["token_count"]
    fields = ["dataset_id", "version", "split", "schema_version", "source_composition", "cleaning_rule_version", "deployment_package"]
    return all(dataset.get(field) for field in fields) and samples_match and tokens_match

```

```

def checkpoint_artifact_recoverability(case):
    checkpoint = case["checkpoint"]
    required_states = ["model_state", "optimizer_state", "scheduler_state", "dataloader_state", "rng_state", "distributed_state"]
    state_ok = all(checkpoint[state] for state in required_states)
    checksum_ok = all(shard in checkpoint["checksums"] for shard in checkpoint["shards"])
    return state_ok and checksum_ok and checkpoint["restore_tested"] and "resume" in checkpoint["purpose"]

```

```

def eval_report_reproducibility(case):
    report = case["eval_report"]
    required = ["model_ref", "eval_dataset_version", "eval_code_version", "metric_definitions", "prompt_template_version"]
    metrics_ok = set(report["metric_definitions"]) == set(report["result_summary"])
    return all(report.get(field) for field in required) and metrics_ok

```

```

def deployment_package_completeness(case):
    package = case["deployment_package"]
    required = ["weights", "tokenizer", "model_config", "runtime_config", "container_image_digest", "health_check", "deployment_package"]

```

```

    return all(package.get(field) for field in required) and package["container_image_digest"].startswith("sha256:")

def release_manifest_integrity(case):
    manifest = case["release_manifest"]
    refs_ok = REQUIRED_RELEASE_REFS.issubset(set(manifest["artifact_refs"]))
    return refs_ok and bool(manifest["checksum"]) and manifest["immutable"] and bool(manifest["rollback_target"])

def checksum_integrity_gate(case):
    integrity = case["integrity"]
    dataset_files = all(f.get("checksum") for f in case["dataset"]["files"])
    checkpoint_files = all(case["checkpoint"]["checksums"].get(shard) for shard in case["checkpoint"]["shards"])
    return integrity["checksums_verified"] and integrity["manifest_checksum_verified"] and integrity["download_verified"]

def artifact_lineage_completeness(case):
    lineage = case["lineage"]
    nodes_ok = REQUIRED_LINEAGE_NODES.issubset(set(lineage["nodes"]))
    edges_ok = all(edge in lineage["edges"] for edge in REQUIRED_LINEAGE_EDGES)
    events_ok = all(e.get("job") and e.get("run_id") and e.get("inputs") and e.get("outputs") for e in lineage["events"])
    return nodes_ok and edges_ok and events_ok

def artifact_store_metadata_split(case):
    store = case["store"]
    required_index = {"artifact_id", "artifact_type", "version", "checksum", "lifecycle_state"}
    return store["artifact_store_for_blobs"] and store["metadata_store_for_index"] and not store["large_blob_in_metadata"]

def permission_access_control(case):
    access = case["access"]
    return access["rbac_enabled"] and access["sample_outputs_restricted"] and access["cross_tenant_blocked"] and access["cross_tenant_blocked"]

def lifecycle_retention_policy(case):
    lifecycle = case["lifecycle"]
    policies = lifecycle["policies"]
    needed = {"dataset", "checkpoint", "eval_report", "deployment_package"}
    deps_ok = all(policies[name].get("dependency_check", True) for name in ["dataset", "eval_report", "deployment_package"])
    checkpoint_ok = policies["checkpoint"]["keep_best"] and policies["checkpoint"]["keep_latest"] and policies["checkpoint"]["keep_latest"]
    return needed.issubset(set(policies)) and deps_ok and checkpoint_ok and bool(lifecycle["archive_tier"])

def promotion_gate_readiness(case):
    promotion = case["promotion"]
    checks = promotion["checks"]
    expected_sequence = ["checkpoint", "candidate_model", "evaluated_model", "release_candidate", "production_package"]
    return promotion["sequence"] == expected_sequence and all(checks.values()) and len(promotion["approvers"]) >= 2

def experiment_tracking_linkage(case):
    experiment = case["experiment"]
    return (
        bool(experiment["run_id"])
    )

```

```

    and experiment["params_logged"]
    and experiment["metrics_logged"]
    and bool(experiment["logs_uri"])
    and {"dataset_artifact", "config_artifact"}.issubset(set(experiment["input_artifacts"]))
    and {"checkpoint_artifact", "eval_report_artifact"}.issubset(set(experiment["output_artifacts"]))
)

def rollback_artifact_readiness(case):
    rollback = case["rollback"]
    return bool(rollback["rollback_package"]) and rollback["required_artifacts_available"] and bool(rollback["rollback_package"])

def deletion_dependency_safety(case):
    plan = case["delete_plan"]
    return plan["dependent_count"] == 0 and plan["retention_met"] and not plan["legal_hold"] and plan["audit_record"]

def artifact_management_gate(case):
    return case["artifact_management_gate"] is True

CHECKS = [
    ("artifact_metadata_contract", artifact_metadata_contract),
    ("dataset_artifact_manifest", dataset_artifact_manifest),
    ("checkpoint_artifact_recoverability", checkpoint_artifact_recoverability),
    ("eval_report_reproducibility", eval_report_reproducibility),
    ("deployment_package_completeness", deployment_package_completeness),
    ("release_manifest_integrity", release_manifest_integrity),
    ("checksum_integrity_gate", checksum_integrity_gate),
    ("artifact_lineage_completeness", artifact_lineage_completeness),
    ("artifact_store_metadata_split", artifact_store_metadata_split),
    ("permission_access_control", permission_access_control),
    ("lifecycle_retention_policy", lifecycle_retention_policy),
    ("promotion_gate_readiness", promotion_gate_readiness),
    ("experiment_tracking_linkage", experiment_tracking_linkage),
    ("rollback_artifact_readiness", rollback_artifact_readiness),
    ("deletion_dependency_safety", deletion_dependency_safety),
    ("artifact_management_gate", artifact_management_gate),
]

def make_bad_cases(base):
    cases = []

    def add(name, change):
        case = deepcopy(base)
        case["name"] = name
        change(case)
        cases.append(case)

    add("artifact_metadata_missing_bad", lambda c: c["artifact"].update({"created_by": ""}))
    add("dataset_manifest_mismatch_bad", lambda c: c["dataset"].update({"sample_count": 1999}))
    add("checkpoint_restore_state_missing_bad", lambda c: c["checkpoint"].update({"rng_state": False}))
    add("eval_report_prompt_missing_bad", lambda c: c["eval_report"].update({"prompt_template_version": ""}))

```

```

add("deployment_package_missing_bad", lambda c: c["deployment_package"].update({"health_check": ""}))
add("release_manifest_mutable_bad", lambda c: c["release_manifest"].update({"immutable": False}))
add("checksum_not_verified_bad", lambda c: c["integrity"].update({"checksums_verified": False}))
add("artifact_lineage_edge_missing_bad", lambda c: c["lineage"]["edges"].remove(("model_version", "deployment_package")))
add("metadata_blob_mixed_bad", lambda c: c["store"].update({"large_blob_in_metadata_db": True}))
add("permission_public_bad", lambda c: c["access"].update({"public_download_allowed": True}))
add("retention_policy_missing_bad", lambda c: c["lifecycle"]["policies"]["deployment_package"].update({"dependencies": []}))
add("promotion_safety_missing_bad", lambda c: c["promotion"]["checks"].update({"safety": False}))
add("experiment_outputs_missing_bad", lambda c: c["experiment"].update({"output_artifacts": ["checkpoint_artifact"]}))
add("rollback_artifact_missing_bad", lambda c: c["rollback"].update({"required_artifacts_available": False}))
add("delete_dependency_unsafe_bad", lambda c: c["delete_plan"].update({"dependent_count": 3}))
add("artifact_gate_missing_bad", lambda c: c.update({"artifact_management_gate": False}))
return cases

```

```

def summarize_examples(case):
    artifact = case["artifact"]
    dataset = case["dataset"]
    package = case["deployment_package"]
    release = case["release_manifest"]
    checksum_files = len(dataset["files"]) + len(case["checkpoint"]["shards"])
    checksum_present = sum(1 for f in dataset["files"] if f["checksum"]) + sum(1 for s in case["checkpoint"]["shards"] if s["checksum"])
    package_required = ["weights", "tokenizer", "model_config", "runtime_config", "container_image_digest", "health_check"]
    return {
        "metadata_completeness": round(ratio(sum(1 for f in REQUIRED_ARTIFACT_FIELDS if artifact.get(f)), len(REQUIRED_ARTIFACT_FIELDS))),
        "dataset_manifest_completeness": 1.0,
        "checksum_coverage": round(ratio(checksum_present, checksum_files), 3),
        "checkpoint_recoverable": checkpoint_artifact_recoverability(case),
        "eval_metric_count": len(case["eval_report"]["metric_definitions"]),
        "deployment_package_completeness": round(ratio(sum(1 for f in package_required if package.get(f)), len(package_required))),
        "release_ref_coverage": round(ratio(len(set(release["artifact_refs"]) & REQUIRED_RELEASE_REFS), len(REQUIRED_RELEASE_REFS))),
        "lineage_node_count": len(case["lineage"]["nodes"]),
        "monthly_storage_cost_usd": round(artifact_storage_cost(case), 2),
        "candidate_delete_safe": deletion_dependency_safety(case),
        "artifact_size_gib": artifact["size_gib"],
    }

```

```

def audit_artifact_management(cases):
    per_case = {
        case["name"]: {name: check(case) for name, check in CHECKS}
        for case in cases
    }
    metrics = {
        name: round(sum(result[name] for result in per_case.values()) / len(cases), 3)
        for name, _ in CHECKS
    }
    failed_cases = [name for name, result in per_case.items() if not all(result.values())]
    failed_gates = [name for name, score in metrics.items() if score < 1.0]
    return {
        "smoke": {
            "complete_case_passes": all(per_case["complete_case"].values()),
            "caught_dataset_gap": "dataset_manifest_mismatch_bad" in failed_cases,
            "caught_checkpoint_gap": "checkpoint_restore_state_missing_bad" in failed_cases,
            "caught_eval_gap": "eval_report_prompt_missing_bad" in failed_cases,
        }
    }

```

```

        "caught_release_gap": "release_manifest_mutable_bad" in failed_cases,
        "caught_delete_gap": "delete_dependency_unsafe_bad" in failed_cases,
    },
    "metrics": metrics,
    "hard_blocker_count": len(failed_cases),
    "failed_cases": failed_cases,
    "failed_gates": failed_gates,
    "artifact_management_gate_pass": len(failed_cases) == 0,
}

```

```

base = complete_case()
cases = [base] + make_bad_cases(base)
report = audit_artifact_management(cases)

```

```

pprint({"artifact_examples": summarize_examples(base)})
pprint(report["smoke"])
pprint(report["metrics"])
print("hard_blocker_count=", report["hard_blocker_count"])
print("failed_cases=", report["failed_cases"])
print("failed_gates=", report["failed_gates"])
print("artifact_management_gate_pass=", report["artifact_management_gate_pass"])

```

```

assert report["smoke"]["complete_case_passes"]
assert report["smoke"]["caught_checkpoint_gap"]
assert report["smoke"]["caught_delete_gap"]
assert report["metrics"]["artifact_metadata_contract"] == 0.941
assert report["metrics"]["artifact_management_gate"] == 0.941
assert report["hard_blocker_count"] == 16
assert report["artifact_management_gate_pass"] is False

```

这个 demo 的重点不是复刻真实平台，而是把 Artifact 管理拆成可检查证据：元数据契约、dataset manifest、checkpoint recoverability、eval report 可复现性、deployment package 完整性、release manifest、checksum、lineage、store / metadata 分层、权限、生命周期、promotion、experiment tracking、rollback、删除依赖安全和最终 artifact gate。

39.23 常见误区

误区一：Artifact 就是文件路径。

文件路径只是定位方式，artifact 还需要类型、版本、checksum、权限、血缘和生命周期。

误区二：checkpoint 都应该长期保留。

中间 checkpoint 成本很高，应按策略保留 best、latest 和关键版本。

误区三：eval report 只是一个分数表。

Eval report 还要记录 eval 数据版本、代码版本、prompt、runtime、样本输出和失败案例。

误区四：部署时现场拼装 artifact。

生产部署应使用 release manifest 或 deployment package，保证可复现和可回滚。

误区五：删除 artifact 只看时间。

删除前必须检查 lineage、deployment、回滚和合规依赖。

39.24 面试常见追问

问题一：Artifact 管理和模型仓库有什么区别？

可以回答：模型仓库专注模型资产，如权重、tokenizer、config、adapter 和量化版本；Artifact 管理更通用，覆盖 dataset、checkpoint、eval report、deployment package、日志和配置等所有关键产物。

问题二：为什么 checkpoint artifact 要记录 optimizer state？

可以回答：如果用于断点恢复，除了模型权重，还需要 optimizer、scheduler、dataloader 和随机状态，否则恢复后的训练轨迹可能不一致。

问题三：Eval report 为什么要版本化？

可以回答：评估结果依赖模型版本、eval 数据、评估代码、prompt、推理参数和 runtime。缺少版本信息，评估结果不可复现，也无法比较。

问题四：如何安全删除 artifact？

可以回答：先检查 artifact 是否被 dataset、model version、eval report、deployment、release manifest 或审计流程依赖；如果无依赖且超过 retention policy，再归档或删除，并记录审计。

39.25 小练习

1. Artifact 和普通文件有什么区别？
2. Dataset artifact 应该记录哪些元数据？
3. Checkpoint 用于断点恢复和候选模型时有什么不同？
4. Eval report 为什么必须绑定 eval dataset 和代码版本？
5. Deployment package 应该包含哪些内容？
6. Release manifest 对回滚有什么价值？
7. Artifact retention policy 如何设计？
8. 删除 artifact 前应该检查哪些依赖？

39.26 本章小结

本章讲了 Artifact 管理。

你需要记住：

1. Artifact 是 AI 研发和生产过程中的关键产物，不只是文件路径。
2. Artifact 应该有类型、版本、URI、checksum、权限、血缘和生命周期状态。
3. Dataset、checkpoint、eval report 和 deployment package 是 AI Infra 中最重要的 artifact 类型。
4. Release manifest 能保证部署可复现和可回滚。
5. Artifact 管理连接训练、评估、部署、审计和成本治理，是平台可复现性的基础设施。

下一章我们会讲实验追踪：参数、指标、日志、样本和代码版本。

第 40 章 实验追踪：参数、指标、日志、样本和代码版本

上一章讲了 Artifact 管理。本章讲实验追踪，也就是 experiment tracking。

大模型研发中，每天可能有大量训练、微调、评估、推理压测和 prompt 实验。如果没有实验追踪，团队很快会陷入“这个结果是哪次跑出来的”“为什么这次比上次好”“这个 checkpoint 对应哪个参数”的混乱。

先记住一句话：

实验追踪的目标，是把一次实验的输入、过程、输出和结论完整记录下来，让结果可比较、可复现、可解释。

40.0 本讲资料边界与第二轮精修口径

本讲按截至 2026-06 的稳定公开资料校准：MLflow Tracking 对 experiment、run、parameters、metrics、tags、artifacts、backend store 和 artifact store 的实验追踪抽象，Weights & Biases Experiments / Tables / Artifacts 对 metrics、hyperparameters、media、sample table 和 lineage 的记录口径，TensorBoard 对 scalar、histogram、image、text、graph 和 hparams 的可视化日志口径，DVC Experiments 对 Git commit、params、metrics、plots 和复现实验的工程边界，以及 OpenLineage 对 Job、Run、Dataset 和运行事件血缘的抽象。

本章只抽象大模型平台里的通用实验追踪能力，不绑定某个 tracking 产品、UI、时序库、日志系统、训练框架或内部平台字段名。

和上一章的分工是：

1. 第 39 章聚焦 Artifact 管理，回答 “dataset、checkpoint、eval report 和 deployment package 这些产物如何被统一注册、校验和流转”。
2. 本章聚焦 Experiment Tracking，回答 “一次训练、评估、prompt 实验或压测 run 的参数、指标、日志、样本、代码版本、成本和结论如何被记录、比较和复现”。

40.1 为什么需要实验追踪

在小实验里，你可能只看终端输出：

```
loss=1.83 acc=0.72
```

但团队研发中会有很多变量：

1. 模型结构。
2. 数据版本。
3. 学习率。
4. batch size。
5. optimizer。
6. 训练步数。
7. seed。
8. prompt 模板。
9. eval 数据。
10. runtime 配置。
11. 代码版本。
12. 硬件环境。

如果这些没有记录，实验结果就很难复现和比较。

40.2 实验追踪记录什么

一次实验至少应该记录：

1. 实验 ID。
2. 实验名称。
3. 运行人。
4. 运行时间。
5. 代码版本。
6. 配置参数。
7. 数据版本。
8. 模型版本。
9. 环境信息。
10. 指标曲线。
11. 日志。
12. 输出 artifact。
13. 样本级结果。

14. 备注和结论。

实验追踪不是只保存最终分数，而是保存整个实验上下文。

40.3 Run 的概念

Experiment 是一组相关实验。

Run 是一次具体执行。

例如：

Experiment: chat-sft-learning-rate-search

Run 1: lr=1e-5

Run 2: lr=2e-5

Run 3: lr=5e-6

每个 run 都有自己的参数、指标、日志和 artifact。

对比实验时，比较的是多个 run。

一个 run 可以抽象为：

$R_i = (e_i, u_i, a_i, t_i, p_i, m_i, l_i, s_i, c_i, d_i, q_i, o_i, z_i)$

其中 e_i 是 experiment id, u_i 是 run id, a_i 是 actor / owner, t_i 是时间与状态, p_i 是参数, m_i 是指标曲线, l_i 是日志, s_i 是样本级结果, c_i 是代码与环境版本, d_i 是数据和模型版本, q_i 是成本和资源, o_i 是输出 artifact, z_i 是 tags、notes 和 lineage 事件。

40.4 参数追踪

参数包括所有会影响结果的配置。

常见训练参数：

1. learning rate。
2. batch size。
3. global batch size。
4. optimizer。
5. weight decay。
6. warmup steps。
7. max steps。
8. gradient accumulation。
9. sequence length。
10. dropout。
11. seed。

大模型训练还要记录：

1. tensor parallel。
2. pipeline parallel。
3. data parallel。
4. ZeRO stage。
5. precision。
6. activation checkpointing。
7. flash attention。
8. tokenizer version。

没有参数追踪，就无法解释结果差异。

参数覆盖率可以写成：

$$C_{\{\mathrm{param}\}} = \frac{1}{N_{\{\mathrm{param}\}} \sum_{j=1}^{N_{\{\mathrm{param}\}}} \mathbf{1}[p_j \setminus \mathrm{recorded}]}$$

这里 `N_param` 是当前任务必须记录的参数数量。训练 run 里, learning rate、global batch size、seed、optimizer、数据版本、并行策略和精度通常都应该进入这个集合。

40.5 指标追踪

指标包括训练过程指标和最终评估指标。

训练指标:

1. train loss。
2. eval loss。
3. learning rate。
4. gradient norm。
5. throughput。
6. tokens/s。
7. GPU utilization。
8. memory usage。
9. step time。

评估指标:

1. accuracy。
2. pass@k。
3. win rate。
4. reward score。
5. safety score。
6. tool call success rate。
7. JSON parse success rate。
8. human preference score。

指标要带 step 或 timestamp, 否则无法画曲线和定位变化。

指标曲线可以表示为:

$$M_k = \{(r_i, k, t, v) \mid r_i \in \mathcal{R}, t \in \mathcal{T}\}$$

其中 r_i 是 run, k 是指标名, t 是 step 或 timestamp, v 是指标值。两个 run 的指标要可比, 至少需要 metric definition、eval dataset、evaluator version 和 aggregation rule 一致。

40.6 日志追踪

日志用于排查问题。

常见日志包括:

1. training stdout/stderr。
2. framework 日志。
3. dataloader 日志。
4. distributed training 日志。
5. checkpoint 保存日志。
6. eval 日志。
7. 错误堆栈。
8. 系统事件。

日志要和 run ID 关联。

当某个 run loss 异常时, 可以直接查看对应日志, 而不是去集群里找已消失的 Pod。

40.7 样本级结果追踪

大模型实验不能只看聚合指标。

很多问题需要看样本级结果。

例如：

1. 哪些题答错了？
2. 哪些 prompt 触发拒答？
3. 哪些样本 JSON 解析失败？
4. 哪些代码题编译失败？
5. 哪些工具调用参数错了？
6. 哪些安全样本越界了？

样本级结果可以包括：

1. input。
2. prediction。
3. reference。
4. score。
5. error type。
6. model version。
7. prompt version。
8. evaluator version。

样本级追踪是模型诊断的基础。

样本级错误覆盖率可以写成：

$$C_{\{\mathrm{sample}\}} = \frac{N_{\{\mathrm{sampldiag}\}}}{N_{\{\mathrm{sample}\}}}$$

如果只存 aggregate score，不存 input hash、prediction、reference、score、error type、prompt version 和 evaluator version，就很难定位模型到底在哪类样本上退化。

40.8 代码版本追踪

代码版本非常重要。

实验应该记录：

1. Git commit。
2. branch。
3. 是否有未提交修改。
4. diff patch。
5. 依赖版本。
6. Docker image。
7. entrypoint。

如果代码没有固定，即使参数和数据一样，也可能无法复现。

生产平台通常要求训练任务使用镜像和 commit 绑定的代码版本。

代码可复现门禁可以写成：

$$G_{\{\mathrm{code}\}} = \mathbf{1}[C_{\{\mathrm{commit}\}} = 1 \wedge C_{\{\mathrm{dirty}\}} = 0 \wedge C_{\{\mathrm{image}\}} = 1 \wedge C_{\{\mathrm{entry}\}} = 1]$$

其中 C_{commit} 表示 commit 已记录， C_{dirty} 表示运行时存在未提交改动， C_{image} 表示容器镜像 digest 已记录， C_{entry} 表示 entrypoint 和配置文件已记录。实验追踪里最危险的情况，是指标很好但无法证明当时跑的代码是什么。

40.9 数据版本追踪

实验必须记录使用的数据。

包括：

1. dataset name。

2. dataset version。
3. split。
4. manifest checksum。
5. sampling recipe。
6. shuffle seed。
7. tokenizer version。

数据版本变化是实验结果变化的常见原因。

如果只记录 `s3://bucket/train/`，而目录内容变了，就无法复现。

数据可复现门禁可以写成：

$G_{\{\mathrm{data}\}}=\mathbf{1}[C_{\{\mathrm{dataset}\}}=1 \wedge C_{\{\mathrm{manifest}\}}=1 \wedge C_{\{\mathrm{recipe}\}}=1 \wedge C_{\{\mathrm{seed}\}}=1]$

其中 `C_dataset` 表示 dataset id / version 已记录，`C_manifest` 表示 manifest checksum 已记录，`C_recipe` 表示采样和清洗配方已记录，`C_seed` 表示 split、shuffle 和 sampling seed 已记录。

40.10 模型和 checkpoint 追踪

如果实验从某个模型继续训练，需要记录：

1. base model version。
2. init checkpoint。
3. adapter version。
4. tokenizer version。
5. resume step。
6. parent run。

输出也要记录：

1. checkpoint artifact。
2. final model artifact。
3. eval report artifact。
4. logs artifact。

实验追踪和 artifact 管理应该打通。

run 与 artifact 的血缘图可以写成：

$G_{\{\mathrm{run}\}}=(V_{\{\mathrm{run}\}}\cup V_{\{\mathrm{artifact}\}},E_{\{\mathrm{in}\}}\cup E_{\{\mathrm{out}\}})$

其中输入边 `E_in` 连接 dataset、base model、checkpoint、config 和 code 到 run，输出边 `E_out` 连接 run 到 checkpoint、eval report、model artifact 和 deployment package。

40.11 环境追踪

环境也会影响实验。

应该记录：

1. Python 版本。
2. PyTorch 版本。
3. CUDA 版本。
4. driver 版本。
5. NCCL 版本。
6. transformers 版本。
7. deepspeed 版本。
8. GPU 类型。
9. GPU 数量。
10. 节点拓扑。
11. 容器镜像。

分布式训练中，环境差异可能导致性能和稳定性差异。

环境完整率可以写成：

$$C_{\{\mathrm{env}\}} = \frac{N_{\{\mathrm{envrec}\}}}{N_{\{\mathrm{envreq}\}}}$$

对大模型训练， N_{envreq} 至少应覆盖 Python、框架、CUDA、driver、NCCL、容器镜像、GPU 类型、GPU 数量和并行拓扑。

40.12 实验对比

实验追踪平台必须支持对比多个 run。

常见对比维度：

1. 参数差异。
2. 数据版本差异。
3. 指标曲线。
4. 最终评估分数。
5. 训练耗时。
6. 资源消耗。
7. 成本。
8. 样本级错误差异。

例如你要回答： $\mathrm{lr}=2\mathrm{e}-5$ 为什么比 $\mathrm{lr}=1\mathrm{e}-5$ 好？

不仅要看最终分数，还要看 loss 曲线、eval 样本、训练稳定性和成本。

在 eval 口径一致时，两个 run 的指标差异可以写成：

$$\Delta_k(a,b) = m_k(R_a) - m_k(R_b)$$

如果 eval dataset、metric definition、judge model、prompt template 或 aggregation rule 不一致，这个差异就只是一种提示，不能直接作为模型能力提升的证据。

40.13 实验搜索和筛选

实验多了以后，需要检索。

常见筛选条件：

1. 模型名称。
2. 数据版本。
3. 运行人。
4. 时间范围。
5. 参数范围。
6. 指标阈值。
7. 标签。
8. 状态。
9. 是否产出模型。
10. 是否进入发布候选。

如果找不到实验，记录再多也没用。

搜索可用性可以用结构化字段覆盖率近似：

$$C_{\{\mathrm{index}\}} = \frac{N_{\{\mathrm{indexed}\}}}{N_{\{\mathrm{query}\}}}$$

常见 query field 包括 owner、experiment、model、dataset、code commit、metric threshold、tag、status、created time 和 artifact id。

40.14 Tags 和 Notes

实验平台通常支持 tags 和 notes。

Tags 用于结构化标记：

1. `sft`。
2. `dpo`。
3. `long-context`。
4. `debug`。
5. `release-candidate`。

Notes 用于记录人的判断：

1. 为什么跑这个实验。
2. 观察到什么现象。
3. 是否值得继续。
4. 下一步计划。

不要低估 notes 的价值。很多实验结论无法从指标自动推断。

实验结论完整率可以写成：

$$C_{\{\mathrm{decision}\}} = \frac{N_{\{\mathrm{decision}\}}}{N_{\{\mathrm{candidate}\}}}$$

候选实验如果没有 hypothesis、观察、结论和 next action，后续团队只能重新读曲线和日志，协作效率会很低。

40.15 大模型实验的特殊性

大模型实验比传统 ML 实验更复杂。

原因：

1. 单次实验成本高。
2. 运行时间长。
3. 分布式环境复杂。
4. 数据版本巨大。
5. 评估指标多维。
6. 人工评估和自动评估并存。
7. `prompt` 和推理参数影响结果。
8. 样本级诊断很重要。

所以实验追踪必须和训练平台、评估平台、artifact 管理、成本系统打通。

大模型实验总成本可以粗略写成：

$$K_{\{\mathrm{run}\}} = H_{\{\mathrm{gpu}\}} P_{\{\mathrm{gpu}\}} + K_{\{\mathrm{storage}\}} + K_{\{\mathrm{eval}\}} + K_{\{\mathrm{network}\}} + K_{\{\mathrm{mat}\}}$$

这里 H_{gpu} 是 GPU 小时， P_{gpu} 是单 GPU 小时价格或内部折算价，后几项分别表示存储、评估调用、网络 and 运维成本。成本必须和质量指标一起看，否则容易用巨大资源换很小提升。

40.16 Prompt 实验追踪

大模型应用中，很多实验不是训练，而是 prompt 改动。

Prompt 实验也要记录：

1. `prompt template version`。
2. `system prompt`。
3. `tool description`。
4. `few-shot examples`。
5. `generation config`。
6. `eval dataset`。

7. model version。
8. 输出结果。
9. 解析成功率。
10. 人工偏好。

Prompt 是模型行为的一部分，也要像代码和模型一样版本化。

40.17 Eval 实验追踪

评估实验要记录：

1. 被评模型。
2. eval dataset。
3. evaluator。
4. prompt 模板。
5. 推理参数。
6. judge model 版本。
7. metric 定义。
8. sample outputs。
9. aggregate metrics。

如果 judge model 或评估 prompt 变了，分数就可能不可比较。

评估实验追踪可以避免“分数涨了但评估口径变了”的误判。

40.18 成本追踪

大模型实验成本高，实验追踪应记录成本。

包括：

1. GPU 小时。
2. GPU 类型。
3. 节点数量。
4. 存储读写量。
5. 网络流量。
6. 训练 tokens。
7. 推理 tokens。
8. eval 调用成本。
9. 总成本估算。

这样团队才能比较“分数提升 0.2 是否值得多花 3 倍成本”。

成本收益比可以写成：

$$E_{\{\mathrm{gain}\}} = \frac{\Delta Q}{K_{\{\mathrm{run}\}} - K_{\{\mathrm{baseline}\}}}$$

其中 ΔQ 是在同一 eval 口径下的质量提升。如果成本增加但质量提升落在噪声范围内，这个 run 不应该进入 release candidate。

40.19 实验状态管理

Run 可以有状态：

created -> running -> succeeded -> failed -> killed -> archived

还可以有业务状态：

debug -> candidate -> evaluated -> release_candidate -> rejected

状态管理帮助团队区分：

1. 哪些只是调试实验。
2. 哪些值得评估。
3. 哪些进入发布候选。
4. 哪些已经废弃。

状态转移应是受控的：

$G_{\{\mathrm{state}\}}=\mathbf{1}[\mathrm{transition}(s_{\{\mathrm{old}\}},s_{\{\mathrm{new}\}})\in \mathcal{A}]$

其中 A 是允许的状态转移集合。比如 debug -> release_candidate 通常不应跳过 eval、safety 和 artifact integrity。

40.20 实验追踪系统架构

一个实验追踪系统可以这样设计：

Experiment Tracking API

- > Run Metadata Store
- > Metrics Time Series Store
- > Log Store
- > Artifact Store
- > Sample Result Store
- > Search Index
- > Lineage Graph
- > UI / Dashboard

不同数据用不同存储：

1. 元数据适合关系型数据库。
2. 指标曲线适合时序存储。
3. 日志适合日志系统。
4. 大文件适合 artifact store。
5. 样本级结果适合对象存储或分析型数据库。

40.21 和训练平台的集成

训练平台提交任务时应自动创建 run。

运行过程中自动上报：

1. 参数。
2. 指标。
3. 日志。
4. checkpoint artifact。
5. 资源使用。
6. 错误事件。

任务完成后自动更新 run 状态。

不要依赖研究员手动复制结果到表格。

40.22 和评估平台的集成

评估平台应把每次 eval 也记录为 run，或作为训练 run 的子 run。

记录：

1. eval config。
2. eval dataset。
3. metrics。
4. sample outputs。
5. failure cases。

6. eval report artifact。

这样训练结果和评估结果可以关联。

实验可复现门禁可以写成：

$G_{\mathrm{repro}} = \mathbf{1}[G_{\mathrm{code}} = 1 \wedge G_{\mathrm{data}} = 1 \wedge C_{\mathrm{param}} \geq \tau_p \wedge C_{\mathrm{track}} = \mathbf{1}[\min_j C_j \geq \tau_j \wedge G_{\mathrm{repro}} = 1 \wedge G_{\mathrm{state}} = 1 \wedge P_0 = 0]$

实验追踪总门禁可以写成：

$G_{\mathrm{track}} = \mathbf{1}[\min_j C_j \geq \tau_j \wedge G_{\mathrm{repro}} = 1 \wedge G_{\mathrm{state}} = 1 \wedge P_0 = 0]$

其中 C_j 包括参数、指标、日志、样本、代码、数据、模型、环境、artifact、成本、搜索索引、lineage 和结论记录等覆盖率， P_0 表示会导致不可复现、无法审计或错误发布的硬阻断问题数量。

40.23 实验追踪审计指标和最小 demo

下面这个 demo 不依赖第三方库，用一个 toy run 说明：实验追踪不是“保存几个分数”，而是要同时检查参数、指标、日志、样本、代码版本、数据版本、环境、artifact、成本、状态、搜索、血缘和实验结论。

```
from copy import deepcopy
from pprint import pprint
```

```
class MiniExperimentTrackingAudit:
    def __init__(self, required):
        self.required = required

    @staticmethod
    def coverage(items, required):
        present = sum(1 for key in required if items.get(key))
        return round(present / len(required), 3)

    def metadata_completeness(self, run):
        return self.coverage(run, self.required["metadata"])

    def parameter_coverage(self, run):
        return self.coverage(run.get("params", {}), self.required["params"])

    @staticmethod
    def metric_step_coverage(run):
        metrics = run.get("metrics", [])
        if not metrics:
            return 0.0
        with_step = [m for m in metrics if "step" in m and "name" in m and "value" in m]
        return round(len(with_step) / len(metrics), 3)

    @staticmethod
    def log_linkage_coverage(run):
        logs = run.get("logs", [])
        if not logs:
            return 0.0
        linked = [log for log in logs if log.get("run_id") == run.get("run_id") and log.get("uri")]
        return round(len(linked) / len(logs), 3)

    def sample_result_coverage(self, run):
        samples = run.get("sample_results", [])
        if not samples:
            return 0.0
```

```

good = []
for sample in samples:
    if all(sample.get(key) is not None for key in self.required["sample"]):
        good.append(sample)
return round(len(good) / len(samples), 3)

@staticmethod
def code_version_ready(run):
    code = run.get("code", {})
    return bool(
        code.get("commit")
        and not code.get("dirty")
        and code.get("image_digest", "").startswith("sha256:")
        and code.get("entrypoint")
    )

@staticmethod
def data_version_ready(run):
    data = run.get("data", {})
    return bool(
        data.get("dataset_version")
        and data.get("manifest_checksum")
        and data.get("sampling_recipe")
        and data.get("shuffle_seed") is not None
    )

@staticmethod
def model_checkpoint_linkage(run):
    model = run.get("model", {})
    artifacts = run.get("artifacts", [])
    artifact_types = {artifact.get("type") for artifact in artifacts}
    return bool(model.get("base_model_version") and model.get("parent_run") and "checkpoint" in artifact_types)

def environment_coverage(self, run):
    return self.coverage(run.get("environment", {}), self.required["environment"])

@staticmethod
def prompt_eval_config_ready(run):
    prompt = run.get("prompt", {})
    evaluation = run.get("evaluation", {})
    return bool(
        prompt.get("template_version")
        and prompt.get("generation_config")
        and evaluation.get("dataset_version")
        and evaluation.get("metric_definitions")
        and evaluation.get("evaluator_version")
    )

@staticmethod
def artifact_linkage_ready(run):
    artifacts = run.get("artifacts", [])
    if not artifacts:
        return False
    for artifact in artifacts:
        if artifact.get("run_id") != run.get("run_id"):

```

```

        return False
    if not artifact.get("checksum") or not artifact.get("uri"):
        return False
    return True

@staticmethod
def cost_attribution_ready(run):
    cost = run.get("cost", {})
    return bool(cost.get("gpu_hours", 0) > 0 and cost.get("total_usd", 0) > 0 and cost.get("project"))

@staticmethod
def state_transition_ready(run):
    allowed = {
        ("created", "running"),
        ("running", "succeeded"),
        ("running", "failed"),
        ("succeeded", "evaluated"),
        ("evaluated", "release_candidate"),
        ("release_candidate", "rejected"),
    }
    transitions = run.get("state_transitions", [])
    return all(tuple(item) in allowed for item in transitions)

@staticmethod
def search_index_ready(run):
    indexed = set(run.get("indexed_fields", []))
    needed = {
        "owner",
        "experiment_id",
        "model_version",
        "dataset_version",
        "code_commit",
        "status",
        "tag",
        "metric",
    }
    return needed.issubset(indexed)

@staticmethod
def lineage_ready(run):
    nodes = set(run.get("lineage_nodes", []))
    edges = {tuple(edge) for edge in run.get("lineage_edges", [])}
    expected_edges = {
        ("dataset:sft-v4", "run:run-040-a"),
        ("code:9f3a12b", "run:run-040-a"),
        ("run:run-040-a", "artifact:ckpt-9200"),
        ("run:run-040-a", "artifact:eval-report-9200"),
    }
    return expected_edges.issubset(edges) and len(nodes) >= 6

@staticmethod
def decision_note_ready(run):
    return bool(run.get("hypothesis") and run.get("decision") and run.get("notes") and run.get("next_action"))

def reproducibility_ready(self, run):

```

```

    return (
        self.code_version_ready(run)
        and self.data_version_ready(run)
        and self.parameter_coverage(run) == 1.0
        and self.environment_coverage(run) == 1.0
        and self.sample_result_coverage(run) >= 0.8
    )

def audit_case(self, run):
    checks = {
        "run_metadata_contract": self.metadata_completeness(run) == 1.0,
        "parameter_config_capture": self.parameter_coverage(run) == 1.0,
        "metric_curve_step_capture": self.metric_step_coverage(run) == 1.0,
        "log_run_linkage": self.log_linkage_coverage(run) == 1.0,
        "sample_result_diagnosis": self.sample_result_coverage(run) == 1.0,
        "code_version_reproducibility": self.code_version_ready(run),
        "data_version_reproducibility": self.data_version_ready(run),
        "model_checkpoint_linkage": self.model_checkpoint_linkage(run),
        "environment_capture": self.environment_coverage(run) == 1.0,
        "prompt_eval_config_capture": self.prompt_eval_config_ready(run),
        "artifact_output_linkage": self.artifact_linkage_ready(run),
        "cost_attribution": self.cost_attribution_ready(run),
        "run_state_transition": self.state_transition_ready(run),
        "search_index_readiness": self.search_index_ready(run),
        "lineage_graph_readiness": self.lineage_ready(run),
        "decision_note_capture": self.decision_note_ready(run),
    }
    checks["experiment_tracking_gate"] = all(checks.values()) and self.reproducibility_ready(run)
    return checks

def audit_suite(self, cases):
    metric_names = [name for name in self.audit_case(cases[0]) if name != "experiment_tracking_gate"]
    scores = {}
    for metric in metric_names:
        passed = sum(1 for case in cases if self.audit_case(case)[metric])
        scores[metric] = round(passed / len(cases), 3)

    failed_cases = []
    failed_gates = []
    for case in cases[1:]:
        checks = self.audit_case(case)
        if not checks["experiment_tracking_gate"]:
            failed_cases.append(case["case"])
        failed_gates.extend([name for name, passed in checks.items() if not passed])

    return {
        "scores": scores,
        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "failed_gates": sorted(set(failed_gates)),
        "experiment_tracking_gate_pass": all(
            self.audit_case(case)["experiment_tracking_gate"] for case in cases
        ),
    }

```

```

def build_complete_run():
    return {
        "case": "complete_experiment_tracking",
        "run_id": "run-040-a",
        "experiment_id": "chat-sft-ablation",
        "owner": "llm-platform",
        "status": "release_candidate",
        "created_at": "2026-06-10T08:00:00Z",
        "ended_at": "2026-06-10T13:20:00Z",
        "tags": ["sft", "long-context", "candidate"],
        "hypothesis": "A slightly larger learning rate improves win rate without raising unsafe rate.",
        "notes": "lr 2e-5 improves eval win rate with acceptable cost.",
        "decision": "promote_to_release_candidate",
        "next_action": "run canary eval and safety regression",
        "params": {
            "learning_rate": 0.00002,
            "global_batch_size": 256,
            "optimizer": "adamw",
            "weight_decay": 0.1,
            "warmup_steps": 200,
            "max_steps": 9200,
            "seed": 42,
            "sequence_length": 8192,
            "precision": "bf16",
            "tensor_parallel": 4,
            "pipeline_parallel": 2,
            "zero_stage": 3,
        },
        "metrics": [
            {"name": "train_loss", "step": 1000, "value": 1.92},
            {"name": "train_loss", "step": 9000, "value": 1.41},
            {"name": "eval_loss", "step": 9000, "value": 1.55},
            {"name": "win_rate", "step": 9200, "value": 0.584},
            {"name": "json_parse_success", "step": 9200, "value": 0.992},
            {"name": "unsafe_rate", "step": 9200, "value": 0.001},
        ],
        "logs": [
            {"run_id": "run-040-a", "uri": "logs/run-040-a/train.log"},
            {"run_id": "run-040-a", "uri": "logs/run-040-a/eval.log"},
        ],
        "sample_results": [
            {"input_hash": "s1", "prediction": "ok", "reference": "ok", "score": 1.0, "error_type": "none"},
            {"input_hash": "s2", "prediction": "bad_json", "reference": "json", "score": 0.0, "error_type": "parse"},
            {"input_hash": "s3", "prediction": "ok", "reference": "ok", "score": 1.0, "error_type": "none"},
            {"input_hash": "s4", "prediction": "refusal", "reference": "answer", "score": 0.0, "error_type": "over_re"},
            {"input_hash": "s5", "prediction": "ok", "reference": "ok", "score": 1.0, "error_type": "none"},
        ],
        "code": {
            "commit": "9f3a12b",
            "branch": "main",
            "dirty": False,
            "image_digest": "sha256:abc123",
            "entrypoint": "train_sft.py",
        },
    },

```

```

"data": {
  "dataset_version": "sft-v4",
  "manifest_checksum": "sha256:data456",
  "split": "train",
  "sampling_recipe": "mix-v7",
  "shuffle_seed": 2026,
},
"model": {
  "base_model_version": "chat-7b-base-v3",
  "init_checkpoint": "ckpt-8000",
  "parent_run": "run-039-parent",
},
"environment": {
  "python": "3.11",
  "pytorch": "2.7",
  "cuda": "12.4",
  "driver": "550",
  "nccl": "2.21",
  "gpu_type": "H100",
  "gpu_count": 64,
  "container_image": "sha256:abc123",
},
"prompt": {
  "template_version": "chat-template-v12",
  "generation_config": {"temperature": 0.2, "top_p": 0.95},
},
"evaluation": {
  "dataset_version": "eval-v9",
  "evaluator_version": "judge-v5",
  "metric_definitions": ["win_rate", "json_parse_success", "unsafe_rate"],
},
"artifacts": [
  {"type": "checkpoint", "id": "ckpt-9200", "run_id": "run-040-a", "uri": "artifacts/ckpt-9200", "checksum": "sha256:abc123"},
  {"type": "eval_report", "id": "eval-report-9200", "run_id": "run-040-a", "uri": "artifacts/eval-9200", "checksum": "sha256:abc123"},
  {"type": "model", "id": "model-sft-v12", "run_id": "run-040-a", "uri": "artifacts/model-v12", "checksum": "sha256:abc123"},
],
"cost": {"gpu_hours": 341.3, "storage_gib_month": 820, "eval_calls": 2400, "total_usd": 963.4, "project": "chatbot"},
"state_transitions": [
  ("created", "running"),
  ("running", "succeeded"),
  ("succeeded", "evaluated"),
  ("evaluated", "release_candidate"),
],
"indexed_fields": [
  "owner",
  "experiment_id",
  "model_version",
  "dataset_version",
  "code_commit",
  "status",
  "tag",
  "metric",
],
"lineage_nodes": [
  "dataset:sft-v4",

```



```

        "code:9f3a12b",
        "run:run-040-a",
        "artifact:ckpt-9200",
        "artifact:eval-report-9200",
        "artifact:model-sft-v12",
    ],
    "lineage_edges": [
        ("dataset:sft-v4", "run:run-040-a"),
        ("code:9f3a12b", "run:run-040-a"),
        ("run:run-040-a", "artifact:ckpt-9200"),
        ("run:run-040-a", "artifact:eval-report-9200"),
        ("run:run-040-a", "artifact:model-sft-v12"),
    ],
}

```

```

def make_bad_case(name, mutate):
    run = deepcopy(build_complete_run())
    run["case"] = name
    mutate(run)
    return run

```

```

def summarize_examples(run, auditor):
    successful = sum(1 for sample in run["sample_results"] if sample["score"] >= 1.0)
    error_types = sorted({sample["error_type"] for sample in run["sample_results"] if sample["error_type"] != "none"})
    return {
        "metadata_completeness": auditor.metadata_completeness(run),
        "parameter_coverage": auditor.parameter_coverage(run),
        "metric_step_coverage": auditor.metric_step_coverage(run),
        "sample_result_coverage": auditor.sample_result_coverage(run),
        "environment_coverage": auditor.environment_coverage(run),
        "sample_error_types": error_types,
        "success_rate": round(successful / len(run["sample_results"]), 3),
        "total_cost_usd": run["cost"]["total_usd"],
        "cost_per_success_usd": round(run["cost"]["total_usd"] / successful, 2),
        "lineage_node_count": len(run["lineage_nodes"]),
        "reproducibility_ready": auditor.reproducibility_ready(run),
    }

```

```

required = {
    "metadata": ["run_id", "experiment_id", "owner", "status", "created_at", "ended_at", "tags", "notes"],
    "params": [
        "learning_rate",
        "global_batch_size",
        "optimizer",
        "weight_decay",
        "warmup_steps",
        "max_steps",
        "seed",
        "sequence_length",
        "precision",
        "tensor_parallel",
        "pipeline_parallel",
    ],
}

```

```

        "zero_stage",
    ],
    "sample": ["input_hash", "prediction", "reference", "score", "error_type"],
    "environment": ["python", "pytorch", "cuda", "driver", "nccl", "gpu_type", "gpu_count", "container_image"],
}

auditor = MiniExperimentTrackingAudit(required)
base = build_complete_run()
cases = [
    base,
    make_bad_case("metadata_missing_bad", lambda r: r.pop("owner")),
    make_bad_case("parameter_missing_bad", lambda r: r["params"].pop("seed")),
    make_bad_case("metric_step_missing_bad", lambda r: r["metrics"][0].pop("step")),
    make_bad_case("log_unlinked_bad", lambda r: r["logs"][0].update({"run_id": "other-run"})),
    make_bad_case("sample_diagnosis_missing_bad", lambda r: r["sample_results"][1].pop("error_type")),
    make_bad_case("code_dirty_bad", lambda r: r["code"].update({"dirty": True})),
    make_bad_case("data_manifest_missing_bad", lambda r: r["data"].pop("manifest_checksum")),
    make_bad_case("checkpoint_link_missing_bad", lambda r: r["artifacts"].pop(0)),
    make_bad_case("environment_missing_bad", lambda r: r["environment"].pop("nccl")),
    make_bad_case("prompt_eval_config_missing_bad", lambda r: r["evaluation"].pop("metric_definitions")),
    make_bad_case("artifact_checksum_missing_bad", lambda r: r["artifacts"][1].pop("checksum")),
    make_bad_case("cost_unattributed_bad", lambda r: r["cost"].pop("project")),
    make_bad_case("state_jump_bad", lambda r: r["state_transitions"].append(("debug", "release_candidate"))),
    make_bad_case("search_index_missing_bad", lambda r: r["indexed_fields"].remove("metric")),
    make_bad_case("lineage_edge_missing_bad", lambda r: r["lineage_edges"].pop(0)),
    make_bad_case("decision_note_missing_bad", lambda r: r.pop("decision")),
]

report = auditor.audit_suite(cases)

pprint({"experiment_tracking_examples": summarize_examples(base, auditor)})
pprint(report["scores"])
print("hard_blocker_count=", report["hard_blocker_count"])
print("failed_cases=", report["failed_cases"])
print("failed_gates=", report["failed_gates"])
print("experiment_tracking_gate_pass=", report["experiment_tracking_gate_pass"])

```

这段代码的输出会显示：完整 run 的元数据、参数、指标 step、样本诊断和环境覆盖率都为 1.0；样本成功率是 0.6；错误类型包括 over_refusal 和 parse；总成本是 963.4；每个成功样本折算成本是 321.13；lineage 节点数是 6；复现门禁为 True。随后 16 个坏例子分别破坏 metadata、参数、指标 step、日志关联、样本诊断、代码干净性、数据 manifest、checkpoint linkage、环境、评估配置、artifact checksum、成本归因、状态流转、搜索索引、血缘边和实验结论，审计分数会稳定落在 0.941，最终 experiment_tracking_gate_pass=False。

面试中讲实验追踪，不能只说“用 MLflow 或 W&B 记录实验”。更好的回答是：run metadata 是控制面，metrics / logs / samples 是观测面，code / data / environment / artifact / lineage 是复现面，cost / status / tags / notes 是协作面，最后用可复现门禁和实验追踪门禁把它们收束。

40.24 常见误区

误区一：实验追踪只记录最终指标。

最终指标不够，还要记录参数、数据、代码、环境、日志、样本和 artifact。

误区二：日志在训练机器上就行。

训练机器可能销毁，日志必须集中保存并和 run ID 关联。

误区三：prompt 实验不需要追踪。

Prompt 变化会显著影响模型行为，必须版本化和追踪。

误区四：实验分数可以直接横向比较。

只有 eval 数据、评估代码、prompt、judge model 和 metric 一致时，分数才可比。

误区五：成本不是实验指标。

大模型实验成本高，成本本身就是重要指标。

40.25 面试常见追问

问题一：实验追踪系统要记录哪些信息？

可以回答：记录参数、指标、日志、数据版本、代码版本、模型版本、环境、样本级结果、artifact、成本、运行状态和备注结论。

问题二：为什么样本级结果重要？

可以回答：聚合指标只能说明总体变化，样本级结果能定位具体失败类型，例如 JSON 解析失败、工具调用错误、安全越界或某类题退化。

问题三：如何保证实验可复现？

可以回答：固定代码 commit、镜像、依赖、数据版本、manifest checksum、模型版本、参数、seed、环境和输出 artifact，并记录完整 run metadata。

问题四：为什么 eval run 也要追踪？

可以回答：评估结果依赖 eval 数据、评估代码、prompt、judge model、推理参数和 metric 定义，任何一项变化都会影响分数。

40.26 小练习

1. Experiment 和 Run 有什么区别？
2. 大模型训练 run 应该记录哪些参数？
3. 为什么实验要记录代码 commit 和 Docker image？
4. 样本级结果能帮助定位哪些问题？
5. Prompt 实验应该记录哪些内容？
6. 为什么不同 eval 配置下的分数不能直接比较？
7. 实验追踪如何和 artifact 管理联动？
8. 如何设计一个实验对比页面？

40.27 本章小结

本章讲了实验追踪。

你需要记住：

1. 实验追踪的目标是让实验结果可比较、可复现、可解释。
2. 一次 run 应记录参数、指标、日志、样本、数据版本、模型版本、代码版本、环境和 artifact。
3. 大模型实验必须重视样本级结果、prompt 版本、eval 配置和成本。
4. 实验追踪系统要和训练平台、评估平台、artifact store、lineage graph 和成本系统打通。
5. 没有实验追踪，团队无法可靠地判断模型改进来自哪里。

下一章我们会讲评估平台：离线批量评测、人工评测和在线评测。

第 41 章 评估平台：离线批量评测、人工评测和在线评测

上一章讲了实验追踪。本章讲评估平台。

大模型研发中，训练出一个模型只是开始。它是否真的更好，是否更安全，是否能上线，是否比旧版本成本更低，都需要评估平台来回答。

先记住一句话：

评估平台不是跑 benchmark 的脚本，而是连接模型、数据、指标、人工判断、在线反馈和发布决策的质量控制系统。

41.0 本讲资料边界与第二轮精修口径

本讲按通用大模型评估平台抽象来写，不绑定某个内部平台、单一 benchmark、单一 LLM judge、单一标注系统或单一 A/B 实验工具。

资料校准时参考的公开口径包括：OpenAI 官方评估接口迁移说明、Hugging Face Evaluate 对 metric / comparison / measurement 的抽象、EleutherAI lm-evaluation-harness 对多任务离线评测的执行方式、HELM 对 scenario / metric / slice 的评估组织方式、Label Studio / HumanSignal 对标注质量与一致性的说明，以及 MLflow / Weave 这类实验评估工具对 scorer、trace、report 和 artifact 的工程抽象。

这些资料给本章一个共同边界：评估平台不是“把模型跑一遍数据集”，而是把模型版本、评估数据、prompt、推理配置、指标定义、judge、人评、在线指标、样本诊断、报告、门禁和发布证据统一成可复现对象。

本轮精修重点补三类内容：

1. 用稳定数学符号描述评估任务、数据覆盖、指标聚合、人评一致性、pairwise win rate、在线指标差异、切片退化、缓存命中和最终发布门禁。
2. 增加一个 0 依赖 MiniEvaluationPlatformAudit demo，用 toy case 演示评估平台如何检查 16 个关键审计维度。
3. 同步第四册百科、面试题、练习、术语表、项目任务和知识图谱，让“评估平台”成为 AI Infra 生命周期的一环，而不是孤立章节。

41.1 为什么需要评估平台

小实验可以手动跑几个样例。

生产级模型需要回答：

1. 新模型比旧模型好吗？
2. 哪些能力提升，哪些能力退化？
3. 安全性是否达标？
4. 工具调用是否稳定？
5. 结构化输出是否可解析？
6. 长上下文是否可靠？
7. 延迟和成本是否可接受？
8. 是否能进入灰度发布？

这些问题不能靠主观感觉回答。

评估平台让评估流程标准化、可复现、可比较、可审计。

可以把一次评估任务抽象成：

$J_i = (m_i, d_i, p_i, c_i, q_i, g_i, h_i, o_i, r_i, z_i)$

其中 m_i 是模型版本， d_i 是评估数据版本， p_i 是 prompt / template 版本， c_i 是推理配置， q_i 是评估代码版本， g_i 是指标定义集合， h_i 是 judge 或人评配置， o_i 是样本级输出， r_i 是评估报告， z_i 是发布或回归状态。

如果这些字段没有一起记录，后续就只能看到一个分数，不能回答“这个分数从哪里来、能不能复现、能不能和旧版本比较”。

41.2 评估平台的核心目标

评估平台要做到：

1. 支持多模型评估。
2. 支持多数据集评估。
3. 支持多指标评估。
4. 支持离线批量评测。
5. 支持人工评测。
6. 支持在线评测。
7. 支持样本级诊断。
8. 支持评估报告版本化。
9. 支持发布门禁。
10. 支持评估结果对比。

一个成熟评估平台既服务研究，也服务生产发布。

因此它的最低可用标准不是“支持某个指标”，而是覆盖完整证据链：

$$C_{\mathrm{eval}} = \frac{1}{N_{\mathrm{cap}}} \sum_{j=1}^{N_{\mathrm{cap}}} \mathbf{1}[a_j \setminus \mathrm{ready}]$$

其中 a_j 表示模型版本、数据版本、指标、样本诊断、报告、门禁、artifact、可观测性等平台能力。 C_{eval} 越接近 1，说明评估平台越能支撑研发和发布闭环；如果只覆盖离线跑分， C_{eval} 会很低。

41.3 评估类型总览

常见评估可以分为三类：

1. 离线批量评测。
2. 人工评测。
3. 在线评测。

离线批量评测：在固定数据集上批量跑模型，计算指标。

人工评测：由人类标注员或专家判断模型输出质量。

在线评测：在真实流量中观察模型表现和业务指标。

三者互补。

离线评测快、可复现，但不一定覆盖真实场景。

人工评测更接近人类偏好，但成本高。

在线评测最真实，但风险和噪声也最大。

三类评估最好不要互相替代，而要做成分层证据：

$$E_{\mathrm{all}} = (E_{\mathrm{offline}}, E_{\mathrm{human}}, E_{\mathrm{online}})$$

离线评估证明候选版本在固定口径下没有明显退化，人评证明开放式质量和偏好更可信，在线评估证明真实流量下的质量、延迟、成本和安全护栏没有失控。

41.4 离线批量评测

离线评测通常流程：

```
ModelVersion
-> Eval Dataset
-> Inference Job
-> Prediction Outputs
-> Metric Computation
-> Eval Report
```

关键是固定：

1. 模型版本。
2. eval 数据版本。

3. prompt 模板版本。
4. 推理参数。
5. 评估代码版本。
6. 指标定义。

否则不同评估结果不可比较。

离线批量评测的可复现门禁可以写成：

$$G_{\{\mathrm{offline}\}}=\mathbf{1}[C_m=1 \ \& \ C_d=1 \ \& \ C_p=1 \ \& \ C_c=1 \ \& \ C_q=1 \ \& \ C_g=1]$$

其中 C_m 、 C_d 、 C_p 、 C_c 、 C_q 、 C_g 分别表示模型版本、数据版本、prompt 版本、推理配置、评估代码和指标定义是否固定。任何一项缺失，离线分数都不应直接用于发布判断。

41.5 Eval Dataset 管理

评估数据集必须版本化。

Eval dataset 应记录：

1. 数据集名称。
2. 版本。
3. 任务类型。
4. 样本数量。
5. 领域分布。
6. 难度分布。
7. 标签或参考答案。
8. 数据来源。
9. 构造方式。
10. 是否允许训练使用。

Eval 数据必须和训练数据隔离。

如果 eval 数据泄漏到训练集，评估结果会虚高。

数据覆盖率可以按必需切片统计：

$$C_{\{\mathrm{dataset}\}}=\frac{1}{N_{\{\mathrm{slice}\}}}\sum_{s=1}^{N_{\{\mathrm{slice}\}}}\mathbf{1}[n_s\geq n_{\{\min\}}]$$

其中 n_s 是第 s 个切片的样本数， $n_{\min}$ 是该切片的最低样本量要求。覆盖率只看有没有样本还不够，还要看是否达到能支持比较的最小规模。

训练 / 评估污染率可以写成：

$$R_{\{\mathrm{leak}\}}=\frac{N_{\{\mathrm{overlap}\}}}{N_{\{\mathrm{eval}\}}}$$

其中 N_{overlap} 是训练集和评估集 exact duplicate、near duplicate 或来源交叉的样本数。 R_{leak} 越高，评估越可能虚高；重要 benchmark 或内部回归集要把这个值纳入门禁。

41.6 指标定义

不同任务需要不同指标。

分类任务：

1. accuracy。
2. precision。
3. recall。
4. F1。

代码任务：

1. pass@k。
2. 编译成功率。

3. 单测通过率。

工具调用：

1. tool selection accuracy。
2. argument correctness。
3. execution success rate。

结构化输出：

1. JSON parse success rate。
2. schema validation pass rate。
3. field accuracy。

安全评估：

1. unsafe response rate。
2. over-refusal rate。
3. policy compliance rate。

不要用一个总分掩盖所有问题。

如果必须给一个汇总分，也要把权重和方向写清楚：

$$S_{\{\mathrm{agg}\}} = \frac{\sum_{k=1}^K w_k \tilde{M}_k}{\sum_{k=1}^K w_k}$$

其中 M_k 是第 k 个指标， $\tilde{M}_k$ 是统一到“越大越好”后的归一化分数， w_k 是业务或安全权重。面试中要强调：汇总分只能做导航，真正决策仍要看分项指标、切片指标和失败样本。

候选模型和 baseline 的指标差异可以写成：

$$\Delta_k = M_k(\mathrm{candidate}) - M_k(\mathrm{baseline})$$

每个 Δ_k 都应该记录统计口径、样本数、置信区间或至少稳定性说明，不能只写“提升 2 分”。

41.7 LLM-as-Judge

LLM-as-Judge 是用模型评估模型输出。

它适合：

1. 开放式问答。
2. 摘要质量。
3. 风格偏好。
4. 多轮对话质量。
5. 指令遵循。

但它有风险：

1. judge model 有偏好。
2. prompt 会影响评估。
3. 可能偏向更长答案。
4. 对事实性判断不可靠。
5. 不同 judge 版本不可直接比较。

因此要记录 judge model、judge prompt、评分规则和版本。

重要发布不能只依赖 LLM-as-Judge。

judge 和人工标注的一致率可以写成：

$$A_{\{\mathrm{judge}\}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i^{\{\mathrm{judge}\}} = y_i^{\{\mathrm{human}\}}]$$

其中 y_{human} 是人工或专家标签， $\hat{y}_{\mathrm{judge}}$ 是 judge 输出。这个值低时，judge 只能用于探索性分析，不能作为发布硬门禁。

如果怀疑 judge 偏向长答案，可以记录长度偏差：

$B_{\{\mathrm{len}\}} = \mathrm{mean}(\ell_i \mid \hat{y}^{\{\mathrm{judge}\}}_i = 1) - \mathrm{mean}(\ell_i \mid \hat{y}^{\{\mathrm{judge}\}}_i = 0)$
 其中 ℓ_i 是回答长度。如果被判优的答案显著更长，要检查 rubric 是否把“啰嗦”误当成“质量高”。

41.8 人工评测

人工评测适合评估：

1. 主观质量。
2. 风格。
3. 有用性。
4. 安全边界。
5. 复杂推理。
6. 多轮对话体验。
7. 业务可接受性。

人工评测流程：

Sample Selection
 -> Annotation Guideline
 -> Human Labeling
 -> Review / QA
 -> Aggregation
 -> Report

人工评测不是随便找人打分，需要标注指南、质量控制和一致性检查。

一次人工评测任务也应该有版本化对象：

$H_i = (s_i, u_i, v_i, a_i, g_i, r_i, t_i, z_i)$

其中 s_i 是样本， u_i 是标注员或专家， v_i 是标注指南版本， a_i 是标注结果， g_i 是 gold set 命中情况， r_i 是复审或仲裁结果， t_i 是标注耗时， z_i 是质量状态。

41.9 人工评测质量控制

人工评测要控制质量。

常见方法：

1. 标注指南版本化。
2. 标注员培训。
3. gold set 检查。
4. 多人重复标注。
5. 一致性指标。
6. 仲裁机制。
7. 抽样复审。
8. 标注耗时监控。

如果人工评测本身不稳定，模型对比结论也不可靠。

gold set 准确率可以写成：

$A_{\{\mathrm{gold}\}} = \frac{1}{N_{\{\mathrm{gold}\}}} \sum_{i=1}^{N_{\{\mathrm{gold}\}}} \mathbf{1}[\hat{y}^{\{\mathrm{ann}\}}_i = y^{\{\mathrm{gold}\}}_i]$

多人重复标注的一致率可以用简单 pair agreement 表达：

$A_{\{\mathrm{pair}\}} = \frac{2}{N_{\{\mathrm{ann}\}}(N_{\{\mathrm{ann}\}} - 1)} \sum_{a < b} \mathbf{1}[y_a = y_b]$

其中 N_{ann} 是同一样本上的标注人数。真实系统还可以使用更复杂的一致性指标，但面试里先把 gold set、重复标注、复审和仲裁讲清楚更重要。

41.10 Pairwise 评测

大模型常用 pairwise 评测。

给同一个输入，展示模型 A 和模型 B 的输出，让人或 judge 判断哪个更好。

结果可以统计 win rate。

Pairwise 的优点：

1. 比绝对打分更稳定。
2. 适合开放式生成。
3. 能直接比较新旧模型。

注意事项：

1. 隐藏模型身份。
2. 随机输出顺序。
3. 控制长度偏差。
4. 记录 tie。
5. 按任务类型分层统计。

Win rate 高不代表所有任务都好，要看分项结果。

带 tie 的 pairwise win rate 常写成：

$$W = \frac{N_{\mathrm{win}} + 0.5N_{\mathrm{tie}}}{N_{\mathrm{win}} + N_{\mathrm{loss}} + N_{\mathrm{tie}}}$$

为了避免把小样本波动误判成真实提升，可以同时给出近似标准误：

$$\mathrm{SE}(W) = \sqrt{\frac{W(1-W)}{N}}$$

一个粗略 95% 区间可以写成 $W \pm 1.96 \mathrm{SE}(W)$ 。这不是严谨统计课里的全部细节，但足以提醒面试官：pairwise 评测必须关注样本量、随机顺序、盲评、tie 和切片稳定性。

41.11 在线评测

在线评测是在真实流量中观察模型表现。

常见形式：

1. A/B 测试。
2. 灰度发布指标监控。
3. 用户反馈。
4. 隐式行为指标。
5. 人工抽检线上样本。

在线评测最接近真实使用，但受噪声影响大。

例如用户群体、时间段、业务活动、路由策略变化都会影响指标。

在线指标差异可以写成：

$$\Delta_{\mathrm{online}} = M_{\mathrm{treatment}} - M_{\mathrm{control}}$$

如果指标是比例，可以用近似标准误做基本 sanity check：

$$\mathrm{SE}_{\Delta} = \sqrt{\frac{p_t(1-p_t)}{n_t} + \frac{p_c(1-p_c)}{n_c}}$$

其中 p_t 、 n_t 是 treatment 分桶的比例指标和样本数， p_c 、 n_c 是 control 分桶的比例指标和样本数。上线决策不能只看均值，还要看分桶稳定性、样本比例、护栏指标和回滚准备。

41.12 在线指标

在线指标包括：

质量和体验：

1. 用户评分。
2. 点赞/点踩。
3. 追问率。
4. 人工接管率。
5. 任务完成率。
6. 投诉率。

性能：

1. TTFT。
2. TPOT。
3. p99 延迟。
4. 错误率。
5. 超时率。

成本：

1. 平均输入 token。
2. 平均输出 token。
3. cost per request。
4. cache hit rate。

安全：

1. 拦截率。
2. 违规输出率。
3. 过度拒答率。

在线护栏门禁可以写成：

$$G_{\{\mathrm{online}\}}=\mathbf{1}[\Delta_Q\geq-\epsilon_Q\wedge\Delta_L\leq\epsilon_L\wedge\Delta_K\leq\epsilon_K\wedge P_0\leq P_{00}]$$

其中 Δ_Q 是质量或任务完成率变化， Δ_L 是延迟变化， Δ_K 是成本变化， P_0 是 P0 风险事件数量。这个公式的直觉是：即使质量略升，只要延迟、成本或安全事件失控，也不能继续放量。

41.13 发布门禁

评估平台应支持发布门禁。

例如模型要进入灰度，必须满足：

1. 核心能力不低于旧版本。
2. 安全评估通过。
3. 工具调用成功率达标。
4. JSON 解析成功率达标。
5. p99 延迟不超过阈值。
6. 成本不超过预算。
7. 人工评测 win rate 达标。

门禁不是为了阻碍发布，而是让发布风险显式化。

发布门禁可以把离线、人评、在线、性能、成本和安全串起来：

$$G_{\{\mathrm{release}\}}=\mathbf{1}[G_{\{\mathrm{offline}\}}=1\wedge G_{\{\mathrm{human}\}}=1\wedge G_{\{\mathrm{online}\}}=1\wedge G_{\{\mathrm{performance}\}}=1\wedge G_{\{\mathrm{cost}\}}=1\wedge G_{\{\mathrm{security}\}}=1]$$

其中 G_{rollback} 表示回滚版本、回滚包、回滚路由和回滚监控都准备好。评估平台如果只负责产出分数，不负责把分数接到发布门禁，就仍然不是完整平台。

41.14 评估报告

Eval report 应包含：

1. model version。

2. eval dataset version。
3. eval code version。
4. prompt version。
5. inference config。
6. metric definitions。
7. aggregate metrics。
8. slice metrics。
9. sample-level results。
10. failure cases。
11. 与 baseline 对比。
12. 结论和建议。

评估报告应该作为 artifact 进入模型仓库和发布流程。

报告完整率可以写成：

$$C_{\{\mathrm{report}\}} = \frac{1}{N_{\{\mathrm{field}\}}} \sum_{j=1}^{N_{\{\mathrm{field}\}}} \mathbf{1}[f_j \setminus \mathrm{present}]$$

其中 f_j 是报告必须字段。低完整率报告不能作为发布证据，因为它无法解释模型为什么被批准、拒绝或回滚。

41.15 Slice Evaluation

只看总体指标很危险。

Slice evaluation 是按维度切片评估。

常见切片：

1. 语言。
2. 领域。
3. 难度。
4. 输入长度。
5. 输出格式。
6. 用户类型。
7. 安全类别。
8. 工具类型。
9. 多轮轮数。

新模型总分提升，可能只是在简单样本上提升，而复杂样本退化。

切片评估能暴露隐藏退化。

切片退化可以按每个切片单独比较：

$$D_s = M_s(\mathrm{candidate}) - M_s(\mathrm{baseline})$$

然后定义切片回归门禁：

$$G_{\{\mathrm{slice}\}} = \mathbf{1}[\min_s D_s \geq -\epsilon_s]$$

其中 ϵ_s 是允许的最大退化。这个门禁能防止“总体提升 1%，但高风险切片退化 10%”被平均数掩盖。

41.16 回归评测

回归评测用于防止旧能力退化。

每次新模型或新 prompt 发布前，都应跑回归集。

回归集通常包含：

1. 历史线上问题样本。
2. 关键业务样本。
3. 安全边界样本。
4. 工具调用样本。

5. 格式约束样本。
6. 高价值客户场景。

回归集应该持续更新。

线上出过问题的样本，修复后应加入回归集。

回归集通过率可以写成：

$$C_{\{\mathrm{reg}\}} = \frac{N_{\{\mathrm{pass}\}}}{N_{\{\mathrm{reg}\}}}$$

回归集新鲜度可以写成：

$$F_{\{\mathrm{reg}\}} = \frac{N_{\{\mathrm{recent}\}}}{N_{\{\mathrm{reg}\}}}$$

其中 N_{recent} 是来自最近线上事故、客户反馈、红队发现或高价值业务场景的样本数。只有固定老样本的回归集会逐渐失去覆盖能力。

41.17 评估任务调度

评估平台也需要调度系统。

评估任务可能包括：

1. 批量推理。
2. 多模型对比。
3. 多数据集评测。
4. LLM judge。
5. 人工标注任务。
6. 安全扫描。

评估调度要考虑：

1. GPU 资源。
2. API 调用成本。
3. 优先级。
4. 依赖关系。
5. 结果缓存。
6. 失败重试。

评估平台本身也需要成本治理。

评估任务成本可以粗略写成：

$$K_{\{\mathrm{eval}\}} = K_{\{\mathrm{infer}\}} + K_{\{\mathrm{judge}\}} + K_{\{\mathrm{human}\}} + K_{\{\mathrm{store}\}} + K_{\{\mathrm{ops}\}}$$

其中 K_{infer} 是候选模型推理成本， K_{judge} 是 judge 调用成本， K_{human} 是人工标注成本， K_{store} 是样本和报告存储成本， K_{ops} 是调度、重试、监控和运维成本。评估任务也要有预算、优先级和失败重试策略，否则评估平台会变成隐形成本中心。

41.18 评估缓存

评估中很多推理可以复用。

例如同一个 model version、eval dataset、prompt version 和 inference config，如果已经跑过，可以复用 prediction outputs。

缓存 key 应包含：

1. model version。
2. eval dataset version。
3. prompt version。
4. inference config。
5. runtime version。

这样可以避免重复评估浪费成本。

但如果任何一个关键版本变化，就不能直接复用。

缓存 key 可以写成：

$$k_{\{\mathrm{cache}\}}=H(m,d,p,c,q,r)$$

其中 H 是稳定 hash， m 是模型版本， d 是数据版本， p 是 prompt 版本， c 是推理配置， q 是评估代码版本， r 是 runtime 版本。

缓存命中率可以写成：

$$R_{\{\mathrm{cache}\}}=\frac{N_{\{\mathrm{hit}\}}}{N_{\{\mathrm{request}\}}}$$

高缓存命中率能降低成本，但不能以牺牲正确性为代价。只要 key 少了一个版本字段，就可能复用错误输出。

41.19 评估可观测性

评估任务也需要观测：

1. 评估任务状态。
2. 每秒处理样本数。
3. 推理错误率。
4. judge 错误率。
5. 成本消耗。
6. 样本失败原因。
7. 队列等待时间。
8. 结果写入状态。

如果评估任务失败，需要知道是模型推理失败、数据问题、评估代码问题，还是 judge 服务问题。

可观测性覆盖率可以写成：

$$C_{\{\mathrm{obs}\}}=\frac{1}{N_{\{\mathrm{signal}\}}}\sum_{j=1}^{N_{\{\mathrm{signal}\}}}\mathbf{1}[\sigma_j\setminus\mathrm{captured}]$$

其中 σ_j 表示任务状态、吞吐、错误率、成本、样本失败原因、队列等待、结果写入、trace id 等信号。评估失败时， C_{obs} 越低，越难判断是模型差、数据坏、judge 不稳定还是平台故障。

41.20 评估平台架构

一个评估平台可以这样设计：

Eval API / UI

- > Eval Dataset Registry
- > Eval Job Scheduler
- > Inference Runner
- > Metric Engine
- > LLM Judge Service
- > Human Review System
- > Eval Result Store
- > Eval Report Generator
- > Quality Gate Service
- > Artifact Store

评估平台应和模型仓库、实验追踪、artifact 管理、发布系统打通。

可以把关键对象连接成血缘图：

$$G_{\{\mathrm{eval_lineage}\}}=(V,E)$$

其中节点 V 包括 ModelVersion、EvalDataset、PromptVersion、EvalJob、PredictionArtifact、JudgeRun、HumanReviewTask、EvalReport、ReleaseDecision；边 E 表示 “使用、生成、比较、批准、拒绝、发布、回

滚”等关系。面试里讲评估平台架构时，要强调这张图能从线上模型反查到离线评估证据，也能从一个失败样本追到模型、prompt、judge 和发布决策。

41.21 评估平台审计指标和最小 demo

把前面的内容合在一起，评估平台可以用 16 个维度做最小审计：

1. eval dataset contract: 数据集版本、样本量、切片、污染检查和使用权限是否完整。
2. offline batch reproducibility: 模型、数据、prompt、推理配置、评估代码和指标定义是否固定。
3. metric definition versioning: 指标名称、方向、权重、阈值和定义版本是否记录。
4. llm judge calibration: judge model、prompt、rubric、人类校准集和偏差检查是否完整。
5. human review quality control: 标注指南、gold set、重复标注、复审和仲裁是否完整。
6. pairwise blinding randomization: pairwise 是否隐藏模型身份、随机顺序、记录 tie 并按切片统计。
7. online eval guardrail: A/B 或灰度是否有稳定分桶、样本比例、质量、延迟、成本和安全护栏。
8. release gate readiness: 发布门禁是否连接离线、人评、在线、安全、性能、成本和回滚。
9. eval report completeness: 评估报告是否含模型、数据、代码、prompt、配置、指标、切片、样本和结论。
10. slice regression detection: 是否能发现语言、领域、难度、长度、安全、工具等切片退化。
11. regression suite coverage: 回归集是否覆盖历史事故、关键业务、安全边界、工具和格式约束。
12. eval job scheduler reliability: 评估调度是否有优先级、依赖、重试、预算和失败分类。
13. eval cache key integrity: 缓存 key 是否包含所有会影响输出和指标的版本字段。
14. eval observability: 评估任务是否有状态、吞吐、错误、成本、失败原因、trace 和结果写入信号。
15. artifact tracking linkage: prediction outputs、sample table、judge run、人评结果、报告和 release decision 是否进入 artifact / lineage。
16. evaluation platform gate: 最终门禁是否能证明无 P0 风险、关键覆盖率达标、可复现、可比较、可发布或可回滚。

下面的 demo 不依赖外部库。它故意构造 1 个完整评估平台样本和 16 个只破坏一个维度的 bad case，帮助你理解“评估平台审计”不是看一个总分，而是逐项证明评估数据、指标、人评、judge、在线实验、报告、调度、缓存、可观测性和发布门禁都可信。

```
from math import sqrt
```

```
class MiniEvaluationPlatformAudit:
    REQUIRED_DATASET_FIELDS = {
        "dataset_id",
        "version",
        "task_type",
        "sample_count",
        "slices",
        "source",
        "construction",
        "labels",
        "train_usage_allowed",
        "contamination_checked",
    }

    REQUIRED_OFFLINE_FIELDS = {
        "model_version",
        "dataset_version",
        "prompt_version",
        "inference_config_hash",
        "eval_code_version",
        "metric_definition_version",
        "runtime_version",
    }
```

```

REQUIRED_REPORT_FIELDS = {
    "model_version",
    "dataset_version",
    "eval_code_version",
    "prompt_version",
    "inference_config",
    "metric_definitions",
    "aggregate_metrics",
    "slice_metrics",
    "sample_results",
    "failure_cases",
    "baseline_comparison",
    "decision",
}

REQUIRED_OBS_SIGNALS = {
    "state",
    "throughput",
    "inference_error_rate",
    "judge_error_rate",
    "cost",
    "sample_failure_reason",
    "queue_wait",
    "result_write_state",
    "trace_id",
}

CACHE_KEY_FIELDS = {
    "model_version",
    "dataset_version",
    "prompt_version",
    "inference_config_hash",
    "eval_code_version",
    "runtime_version",
}

def __init__(self, cases):
    self.cases = cases
    self.checks = [
        ("eval_dataset_contract", self.eval_dataset_contract),
        ("offline_batch_reproducibility", self.offline_batch_reproducibility),
        ("metric_definition_versioning", self.metric_definition_versioning),
        ("llm_judge_calibration", self.llm_judge_calibration),
        ("human_review_quality_control", self.human_review_quality_control),
        ("pairwise_blinding_randomization", self.pairwise_blinding_randomization),
        ("online_eval_guardrail", self.online_eval_guardrail),
        ("release_gate_readiness", self.release_gate_readiness),
        ("eval_report_completeness", self.eval_report_completeness),
        ("slice_regression_detection", self.slice_regression_detection),
        ("regression_suite_coverage", self.regression_suite_coverage),
        ("eval_job_scheduler_reliability", self.eval_job_scheduler_reliability),
        ("eval_cache_key_integrity", self.eval_cache_key_integrity),
        ("eval_observability", self.eval_observability),
        ("artifact_tracking_linkage", self.artifact_tracking_linkage),
        ("evaluation_platform_gate", self.evaluation_platform_gate),
    ]

```

```

]

def eval_dataset_contract(self, case):
    dataset = case["dataset"]
    slices = dataset.get("slices", {})
    required_slices = {"language:zh", "language:en", "domain:code", "domain:math", "difficulty:hard"}
    enough_slice_samples = all(slices.get(name, 0) >= 20 for name in required_slices)
    required_fields_present = self.REQUIRED_DATASET_FIELDS <= set(dataset)
    return (
        required_fields_present
        and dataset["sample_count"] >= 100
        and enough_slice_samples
        and dataset["labels"]
        and dataset["contamination_checked"]
        and dataset["train_usage_allowed"] is False
    )

def offline_batch_reproducibility(self, case):
    offline = case["offline"]
    return self.REQUIRED_OFFLINE_FIELDS <= set(offline)

def metric_definition_versioning(self, case):
    metrics = case["metrics"]
    if not metrics:
        return False
    for metric in metrics:
        required = {"name", "version", "direction", "weight", "threshold"}
        if not required <= set(metric):
            return False
        if metric["direction"] not in {"higher_is_better", "lower_is_better"}:
            return False
        if metric["weight"] <= 0:
            return False
    return True

def llm_judge_calibration(self, case):
    judge = case["judge"]
    required = {"model", "prompt_version", "rubric_version", "human_calibration_score", "bias_checks"}
    return (
        required <= set(judge)
        and judge["human_calibration_score"] >= 0.8
        and {"length_bias", "position_bias"} <= set(judge["bias_checks"])
    )

def human_review_quality_control(self, case):
    human = case["human_review"]
    required = {"guideline_version", "gold_accuracy", "duplicate_annotation_rate", "review_rate", "arbitration"}
    return (
        required <= set(human)
        and human["gold_accuracy"] >= 0.9
        and human["duplicate_annotation_rate"] >= 0.2
        and human["review_rate"] >= 0.1
        and human["arbitration"]
    )

```



```

def pairwise_blinding_randomization(self, case):
    pairwise = case["pairwise"]
    required = {"blind_model_ids", "randomized_order", "tie_recorded", "slice_breakdown", "wins", "losses", "ties"}
    return (
        required <= set(pairwise)
        and pairwise["blind_model_ids"]
        and pairwise["randomized_order"]
        and pairwise["tie_recorded"]
        and bool(pairwise["slice_breakdown"])
    )

def online_eval_guardrail(self, case):
    online = case["online"]
    required = {
        "stable_bucket",
        "sample_ratio_health",
        "quality_delta",
        "latency_delta",
        "cost_delta",
        "safety_events",
        "rollback_ready",
    }
    return (
        required <= set(online)
        and online["stable_bucket"]
        and online["sample_ratio_health"] >= 0.95
        and online["quality_delta"] >= -0.01
        and online["latency_delta"] <= 0.05
        and online["cost_delta"] <= 0.08
        and online["safety_events"] == 0
        and online["rollback_ready"]
    )

def release_gate_readiness(self, case):
    gate = case["release_gate"]
    required = {
        "offline_passed",
        "human_passed",
        "online_passed",
        "safety_passed",
        "performance_passed",
        "cost_passed",
        "rollback_ready",
        "approver",
    }
    return required <= set(gate) and all(gate[key] for key in required)

def eval_report_completeness(self, case):
    report = case["report"]
    return self.REQUIRED_REPORT_FIELDS <= set(report)

def slice_regression_detection(self, case):
    slices = case["slice_metrics"]
    if not slices:
        return False

```

```

    return all(item["candidate"] - item["baseline"] >= -0.02 for item in slices.values())

def regression_suite_coverage(self, case):
    suite = case["regression_suite"]
    required = {"incident", "business", "safety", "tool", "format"}
    return required <= set(suite.get("categories", set())) and suite.get("fresh_case_ratio", 0.0) >= 0.2

def eval_job_scheduler_reliability(self, case):
    scheduler = case["scheduler"]
    required = {"priority", "dependencies", "retry_policy", "budget_limit", "failure_classification", "owner"}
    return required <= set(scheduler) and scheduler["retry_policy"]["max_retries"] >= 1 and scheduler["budget_limit"] >= 0.2

def eval_cache_key_integrity(self, case):
    cache = case["cache"]
    return self.CACHE_KEY_FIELDS <= set(cache.get("key_fields", set())) and cache.get("hit_rate", 0.0) >= 0.2

def eval_observability(self, case):
    observed = case["observability"].get("signals", set())
    return self.REQUIRED_OBS_SIGNALS <= observed

def artifact_tracking_linkage(self, case):
    artifacts = case["artifacts"]
    required = {
        "prediction_outputs",
        "sample_table",
        "judge_run",
        "human_review",
        "eval_report",
        "release_decision",
        "lineage_edges",
    }
    return required <= set(artifacts) and len(artifacts["lineage_edges"]) >= 6

def evaluation_platform_gate(self, case):
    gate = case["platform_gate"]
    return gate.get("enabled") and gate.get("min_coverage", 0.0) >= 0.95 and gate.get("p0_risks", 1) == 0

def metric_scores(self):
    total = len(self.cases)
    return {
        name: round(sum(1 for case in self.cases if check(case)) / total, 3)
        for name, check in self.checks
    }

def failed_cases(self):
    failures = []
    for case in self.cases:
        failed = [name for name, check in self.checks if not check(case)]
        if failed:
            failures.append((case["case_id"], failed))
    return failures

def examples(self):
    case = self.cases[0]
    dataset = case["dataset"]

```

```

pairwise = case["pairwise"]
human = case["human_review"]
judge = case["judge"]
online = case["online"]
report = case["report"]
cache = case["cache"]

required_slice_names = {"language:zh", "language:en", "domain:code", "domain:math", "difficulty:hard"}
dataset_coverage = sum(
    1 for name in required_slice_names if dataset["slices"].get(name, 0) >= 20
) / len(required_slice_names)
contamination_rate = dataset["overlap_count"] / dataset["sample_count"]
offline_accuracy = report["aggregate_metrics"]["accuracy"]

total_pairwise = pairwise["wins"] + pairwise["losses"] + pairwise["ties"]
win_rate = (pairwise["wins"] + 0.5 * pairwise["ties"]) / total_pairwise
win_rate_se = sqrt(win_rate * (1.0 - win_rate) / total_pairwise)

online_pass = (
    online["quality_delta"] >= -0.01
    and online["latency_delta"] <= 0.05
    and online["cost_delta"] <= 0.08
    and online["safety_events"] == 0
)

return {
    "dataset_coverage": round(dataset_coverage, 3),
    "contamination_rate": round(contamination_rate, 4),
    "offline_accuracy": offline_accuracy,
    "pairwise_win_rate": round(win_rate, 3),
    "pairwise_win_rate_se": round(win_rate_se, 4),
    "human_gold_accuracy": human["gold_accuracy"],
    "human_pair_agreement": human["pair_agreement"],
    "judge_human_agreement": judge["human_calibration_score"],
    "judge_length_bias": judge["bias_checks"]["length_bias"],
    "online_guardrail_pass": online_pass,
    "release_ready": self.release_gate_readiness(case),
    "cache_hit_rate": cache["hit_rate"],
    "report_field_count": len(report),
    "lineage_edge_count": len(case["artifacts"]["lineage_edges"]),
    "estimated_eval_cost_usd": case["cost"]["total_usd"],
}

```

```

def complete_case():
    return {
        "case_id": "complete_eval_platform_ok",
        "dataset": {
            "dataset_id": "eval-general-zh-en",
            "version": "2026-06-01",
            "task_type": "mixed_llm_eval",
            "sample_count": 1000,
            "slices": {
                "language:zh": 420,
                "language:en": 360,
            }
        }
    }

```

```

        "domain:code": 120,
        "domain:math": 110,
        "difficulty:hard": 90,
    },
    "source": "internal_regression_and_public_benchmark",
    "construction": "stratified_sampling",
    "labels": True,
    "train_usage_allowed": False,
    "contamination_checked": True,
    "overlap_count": 2,
},
"offline": {
    "model_version": "chat-2026-06-candidate",
    "dataset_version": "2026-06-01",
    "prompt_version": "prompt-v17",
    "inference_config_hash": "cfg-b7",
    "eval_code_version": "eval-commit-a1b2",
    "metric_definition_version": "metrics-v9",
    "runtime_version": "runtime-2026-06",
},
"metrics": [
    {"name": "accuracy", "version": "v3", "direction": "higher_is_better", "weight": 0.3, "threshold": 0.78},
    {"name": "unsafe_rate", "version": "v2", "direction": "lower_is_better", "weight": 0.4, "threshold": 0.01},
    {"name": "tool_success", "version": "v5", "direction": "higher_is_better", "weight": 0.3, "threshold": 0.01},
],
"judge": {
    "model": "judge-strong-2026-05",
    "prompt_version": "judge-prompt-v12",
    "rubric_version": "rubric-v4",
    "human_calibration_score": 0.86,
    "bias_checks": {"length_bias": 0.04, "position_bias": 0.02},
},
"human_review": {
    "guideline_version": "guide-v8",
    "gold_accuracy": 0.94,
    "duplicate_annotation_rate": 0.25,
    "review_rate": 0.15,
    "arbitration": True,
    "pair_agreement": 0.88,
},
"pairwise": {
    "blind_model_ids": True,
    "randomized_order": True,
    "tie_recorded": True,
    "slice_breakdown": {"code": 0.61, "math": 0.57, "safety": 0.53},
    "wins": 62,
    "losses": 32,
    "ties": 6,
},
"online": {
    "stable_bucket": True,
    "sample_ratio_health": 0.99,
    "quality_delta": 0.018,
    "latency_delta": 0.012,
    "cost_delta": 0.035,

```

```

    "safety_events": 0,
    "rollback_ready": True,
},
"release_gate": {
    "offline_passed": True,
    "human_passed": True,
    "online_passed": True,
    "safety_passed": True,
    "performance_passed": True,
    "cost_passed": True,
    "rollback_ready": True,
    "approver": True,
},
"report": {
    "model_version": "chat-2026-06-candidate",
    "dataset_version": "2026-06-01",
    "eval_code_version": "eval-commit-a1b2",
    "prompt_version": "prompt-v17",
    "inference_config": {"temperature": 0.2, "max_tokens": 1024},
    "metric_definitions": "metrics-v9",
    "aggregate_metrics": {"accuracy": 0.812, "unsafe_rate": 0.006, "tool_success": 0.94},
    "slice_metrics": {"code": 0.77, "math": 0.74, "hard": 0.69},
    "sample_results": "sample-table-artifact",
    "failure_cases": ["tool_arg_missing", "math_reasoning_error"],
    "baseline_comparison": {"accuracy_delta": 0.021, "unsafe_rate_delta": -0.002},
    "decision": "approve_canary",
},
"slice_metrics": {
    "zh": {"candidate": 0.82, "baseline": 0.80},
    "code": {"candidate": 0.77, "baseline": 0.76},
    "math": {"candidate": 0.74, "baseline": 0.735},
    "hard": {"candidate": 0.69, "baseline": 0.70},
},
"regression_suite": {
    "categories": {"incident", "business", "safety", "tool", "format"},
    "fresh_case_ratio": 0.31,
},
"scheduler": {
    "priority": "release_blocking",
    "dependencies": ["offline_predictions", "judge", "human_review"],
    "retry_policy": {"max_retries": 2, "backoff_s": 60},
    "budget_limit": 2500.0,
    "failure_classification": True,
    "owner": "eval-platform",
},
"cache": {
    "key_fields": {
        "model_version",
        "dataset_version",
        "prompt_version",
        "inference_config_hash",
        "eval_code_version",
        "runtime_version",
    },
    "hit_rate": 0.43,

```

```

    },
    "observability": {
        "signals": {
            "state",
            "throughput",
            "inference_error_rate",
            "judge_error_rate",
            "cost",
            "sample_failure_reason",
            "queue_wait",
            "result_write_state",
            "trace_id",
        }
    },
    "artifacts": {
        "prediction_outputs": "predictions.parquet",
        "sample_table": "samples.parquet",
        "judge_run": "judge-run-2026-06",
        "human_review": "human-review-2026-06",
        "eval_report": "eval-report.json",
        "release_decision": "release-decision.json",
        "lineage_edges": [
            ("model", "prediction_outputs"),
            ("dataset", "prediction_outputs"),
            ("prediction_outputs", "judge_run"),
            ("prediction_outputs", "sample_table"),
            ("judge_run", "eval_report"),
            ("human_review", "eval_report"),
            ("eval_report", "release_decision"),
        ],
    },
    "platform_gate": {"enabled": True, "min_coverage": 0.96, "p0_risks": 0},
    "cost": {"total_usd": 1487.35},
}

```

```

def make_bad_case(case_id, mutate):
    case = complete_case()
    case["case_id"] = case_id
    mutate(case)
    return case

```

```

cases = [
    complete_case(),
    make_bad_case("dataset_contract_missing_bad", lambda c: c["dataset"].pop("version")),
    make_bad_case("offline_config_unpinned_bad", lambda c: c["offline"].pop("eval_code_version")),
    make_bad_case("metric_definition_unversioned_bad", lambda c: c["metrics"][0].pop("version")),
    make_bad_case("judge_calibration_missing_bad", lambda c: c["judge"].pop("human_calibration_score")),
    make_bad_case("human_qc_missing_bad", lambda c: c["human_review"].update({"gold_accuracy": 0.62})),
    make_bad_case("pairwise_unblinded_bad", lambda c: c["pairwise"].update({"blind_model_ids": False})),
    make_bad_case("online_guardrail_unhealthy_bad", lambda c: c["online"].update({"safety_events": 1})),
    make_bad_case("release_gate_missing_bad", lambda c: c["release_gate"].update({"rollback_ready": False})),
    make_bad_case("eval_report_incomplete_bad", lambda c: c["report"].pop("sample_results")),
    make_bad_case("slice_regression_hidden_bad", lambda c: c["slice_metrics"]["math"].update({"candidate": 0.66})),

```

```

    make_bad_case("regression_suite_thin_bad", lambda c: c["regression_suite"].update({"categories": {"business", "to
    make_bad_case("scheduler_retry_missing_bad", lambda c: c["scheduler"]["retry_policy"].update({"max_retries": 0})),
    make_bad_case("cache_key_incomplete_bad", lambda c: c["cache"]["key_fields"].remove("prompt_version")),
    make_bad_case("observability_signal_missing_bad", lambda c: c["observability"]["signals"].remove("judge_error_rat
    make_bad_case("artifact_lineage_missing_bad", lambda c: c["artifacts"].pop("lineage_edges")),
    make_bad_case("platform_gate_disabled_bad", lambda c: c["platform_gate"].update({"enabled": False})),
]

audit = MiniEvaluationPlatformAudit(cases)
examples = audit.examples()
failures = audit.failed_cases()
failed_cases = [case_id for case_id, _ in failures]
failed_gates = [gates[0] for _, gates in failures]

print("evaluation_platform_examples=" + repr(examples))
print("metrics=" + repr(audit.metric_scores()))
print("hard_blocker_count=" + str(len(failures)))
print("failed_cases=" + repr(failed_cases))
print("failed_gates=" + repr(failed_gates))
print("evaluation_platform_gate_pass=" + str(not failures))

```

参考输出：

```

evaluation_platform_examples={'dataset_coverage': 1.0, 'contamination_rate': 0.002, 'offline_accuracy': 0.812, 'pairwis
metrics={'eval_dataset_contract': 0.941, 'offline_batch_reproducibility': 0.941, 'metric_definition_versioning': 0.941
hard_blocker_count=16
failed_cases=['dataset_contract_missing_bad', 'offline_config_unpinned_bad', 'metric_definition_unversioned_bad', 'ju
failed_gates=['eval_dataset_contract', 'offline_batch_reproducibility', 'metric_definition_versioning', 'llm_judge_cal
evaluation_platform_gate_pass=False

```

这段 demo 对面试最有用的地方是：它把评估平台拆成可检查对象，而不是抽象地说“我们有一套 eval”。真正的平台要能证明每一次模型比较都绑定模型版本、数据版本、prompt、推理配置、指标、人评、judge、在线分桶、报告、artifact、trace、发布门禁和回滚证据。

41.22 常见误区

误区一：跑几个公开 benchmark 就够了。

公开 benchmark 不能覆盖企业业务场景，还需要内部数据、回归集和在线指标。

误区二：总分提升就能上线。

总分可能掩盖安全、工具调用、长上下文或特定业务退化。

误区三：LLM-as-Judge 完全可靠。

Judge model 有偏差，评估 prompt 会影响结论，重要场景需要人工和规则校验。

误区四：人工评测就是找人看看。

人工评测需要标注指南、质量控制、一致性检查和审计。

误区五：离线评估通过就代表线上好。

线上真实流量、用户行为和业务指标可能不同，还需要灰度和在线评测。

41.23 面试常见追问

问题一：如何设计一个大模型评估平台？

可以回答：设计 eval dataset registry、eval job scheduler、inference runner、metric engine、LLM judge、human review、result store、report generator 和 quality gate，并与模型仓库、实验追踪和发布系统打通。

问题二：离线评测、人工评测和在线评测有什么区别？

可以回答：离线评测可复现、速度快；人工评测适合主观质量和复杂场景；在线评测最接近真实用户，但风险和噪声更高。三者互补。

问题三：为什么要做 slice evaluation？

可以回答：总体指标可能掩盖局部退化。按语言、领域、难度、输入长度、安全类别和工具类型切片，能发现某些场景退化。

问题四：评估报告为什么要版本化？

可以回答：评估结果依赖模型、数据、prompt、推理参数、评估代码、judge model 和指标定义。版本化才能复现和比较。

问题五：评估平台为什么必须和实验追踪、artifact 管理和发布系统打通？

可以回答：实验追踪记录 run 参数和指标，artifact 管理保存 prediction outputs、sample table、eval report 和 release bundle，发布系统执行灰度、A/B、回滚和在线监控。评估平台如果不和这些系统打通，就无法从线上模型反查评估证据，也无法从失败样本追到数据、prompt、judge、指标和发布决策。

41.24 小练习

1. 评估平台和 benchmark 脚本有什么区别？
2. Eval dataset 应该记录哪些元数据？
3. LLM-as-Judge 有哪些风险？
4. Pairwise 评测为什么常用于大模型？
5. 发布门禁应该包含哪些指标？
6. Slice evaluation 能发现哪些问题？
7. 如何构建回归评测集？
8. 在线评测为什么有噪声？
9. 写一个 0 依赖脚本，检查一次评估任务是否记录 model version、eval dataset version、prompt version、inference config、metric definition、judge config、sample outputs 和 eval report。
10. 扩展本章 demo，加入一个“线上 A/B 样本比例异常”的 bad case，并让它触发 online eval guardrail。

41.25 本章小结

本章讲了评估平台。

你需要记住：

1. 评估平台是模型质量控制系统，不是单个 benchmark 脚本。
2. 离线批量评测、人工评测和在线评测各有优势和局限，需要组合使用。
3. Eval dataset、prompt、metric、judge model、评估代码和推理参数都要版本化。
4. 评估报告要包含总体指标、切片指标、样本级结果、失败案例和 baseline 对比。
5. 发布门禁把评估结果接入模型发布流程，是模型上线风险控制的关键。
6. 成熟评估平台还要记录人评质量、judge 校准、pairwise 盲评、在线护栏、调度、缓存、可观测性、artifact 血缘和回滚证据。

下一章我们会讲 Feature Store、Embedding Store 与向量索引基础设施。

第 42 章 Feature Store、Embedding Store 与向量索引基础设施

上一章讲了评估平台。本章讲 Feature Store、Embedding Store 与向量索引基础设施。

这些系统在推荐、搜索、RAG、Agent、风控和个性化模型中都很常见。它们的共同点是：把模型需要的特征、向量和索引从临时计算变成可管理、可复用、可服务化的基础设施。

先记住一句话：

Feature Store 管结构化特征, Embedding Store 管向量资产, 向量索引负责高效相似度检索; 三者共同支撑在线智能系统的数据访问层。

42.0 本讲资料边界与第二轮精修口径

本讲按通用 AI Infra 数据访问层来写, 不绑定某个 Feature Store、向量数据库、RAG 框架、搜索引擎、对象存储或内部平台实现。

资料校准时参考的公开口径包括: Feast 对 entity、feature view、offline store、online store、historical feature retrieval 和 point-in-time join 的抽象; Faiss 对相似度搜索、IVF、PQ、HNSW 等索引类型的公开说明; Milvus / Qdrant / Weaviate / Pinecone 等向量数据库文档对 ANN、metadata filter、payload、collection、多租户和索引参数的工程口径; 以及前文数据版本、artifact、实验追踪和评估平台章节对版本、血缘、门禁和可观测性的要求。

本轮精修重点不是介绍某个产品 API, 而是把 Feature Store、Embedding Store 和 Vector Index 抽象成可审计基础设施:

1. 用稳定数学符号描述特征定义、训练线上一致性、point-in-time correctness、embedding 版本、chunk 血缘、ANN 召回、过滤、索引切换、成本和最终门禁。
2. 增加一个 0 依赖 MiniFeatureEmbeddingIndexAudit demo, 用 toy case 检查 16 个关键治理维度。
3. 同步第四册百科、题库、练习、术语表、项目任务和知识图谱, 让本章和 RAG / Agent 平台章节自然衔接。

42.1 为什么需要这些基础设施

在小实验中, 特征和 embedding 可能直接存在一个文件里:

```
features.csv  
embeddings.npy
```

但生产系统需要面对:

1. 特征来源多。
2. 特征更新频率不同。
3. 训练和线上特征要一致。
4. embedding 规模巨大。
5. 向量检索要低延迟。
6. 索引需要定期更新。
7. 多租户和权限隔离。
8. 特征和向量版本要可追溯。

没有基础设施, 系统会变成各业务自己造一套特征和向量管道, 难以治理。

可以把本章讨论的基础设施抽象成一条数据访问链:

$$A_i = (f_i, e_i, c_i, v_i, s_i, r_i, p_i, l_i, z_i)$$

其中 f_i 是 feature definition, e_i 是 embedding record, c_i 是 chunk / object metadata, v_i 是 vector index, s_i 是 serving request, r_i 是 retrieval result, p_i 是 permission context, l_i 是 lineage, z_i 是当前状态。

基础设施能力覆盖率可以写成:

$$C_{\{\mathrm{infra}\}} = \frac{1}{N_{\{\mathrm{cap}\}}} \sum_{j=1}^{N_{\{\mathrm{cap}\}}} \mathbf{1}[a_j \setminus \mathrm{ready}]$$

如果只存了 embeddings.npy, 但没有版本、权限、索引状态、trace 和回滚, C_{infra} 会很低。

42.2 Feature Store 是什么

Feature Store 是特征管理系统。

它负责:

1. 定义特征。
2. 计算特征。
3. 存储特征。
4. 版本化特征。
5. 在线服务特征。
6. 离线训练读取特征。
7. 保证训练和线上一致。
8. 记录特征血缘。

Feature Store 最初常见于推荐、排序、风控等机器学习系统。

在大模型时代，它仍然重要，尤其在个性化、用户画像、工具路由、Agent 记忆和检索增强中。

一个特征定义可以写成：

$F_j = (n_j, v_j, k_j, s_j, t_j, w_j, \tau_j, o_j, p_j)$

其中 n_j 是 feature name, v_j 是 definition version, k_j 是 entity key, s_j 是 source, t_j 是 transform version, w_j 是时间窗口, τ_j 是 TTL 或 freshness SLA, o_j 是 owner, p_j 是 permission policy。

特征定义完整率可以写成：

$C_{\{\mathrm{feature}\}} = \frac{1}{N_{\{\mathrm{feature}\}}} \sum_{j=1}^{N_{\{\mathrm{feature}\}}} \mathbf{1}[F_j \setminus \mathrm{complete}]$

面试里不要只说“有特征表”，要能说清每个特征的定义、版本、实体键、窗口、刷新频率和线上读取方式。

42.3 什么是特征

特征是模型或业务逻辑使用的结构化信号。

例如：

1. 用户最近 7 天活跃次数。
2. 商品点击率。
3. 文档热度。
4. 用户等级。
5. 历史购买次数。
6. 风险评分。
7. 会话长度。
8. 工具调用成功率。
9. 模型响应满意度。

特征可以用于：

1. 训练传统 ML 模型。
2. 训练排序模型。
3. 做模型路由。
4. 做个性化 prompt。
5. 做风控和安全策略。
6. 做实验分层。

42.4 Offline Store 和 Online Store

Feature Store 通常分为两层：

1. Offline Store。
2. Online Store。

Offline Store 用于训练和分析，通常存放历史特征，支持大规模扫描。

Online Store 用于线上实时服务，要求低延迟读取。

例如：

Offline Store: Hive / Iceberg / Parquet / Data Lake
Online Store: Redis / RocksDB / Cassandra / DynamoDB / HBase

训练读 Offline Store, 推理读 Online Store。

关键是两者特征定义要一致。

offline / online 一致性可以按同一批实体抽样检查：

$$R_{\{\mathrm{skew}\}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1} \left[|x^{\{\mathrm{off}\}}_i - x^{\{\mathrm{on}\}}_i| > \epsilon_i \right]$$

其中 x_{off} 是离线训练读取到的特征值, x_{on} 是同一实体线上读取到的特征值, ϵ_i 是允许的数值误差。

R_{skew} 越高, 说明训练和线上逻辑越不一致。

42.5 Training-Serving Skew

Training-serving skew 指训练时用的特征和线上服务时用的特征不一致。

例如：

1. 训练用 7 天点击率, 线上用 3 天点击率。
2. 训练特征有延迟, 线上实时更新。
3. 训练时缺失值填 0, 线上填 -1。
4. 特征计算代码不同。
5. 时间窗口切错导致未来信息泄漏。

Feature Store 的核心价值之一就是减少 training-serving skew。

它通过统一特征定义、版本和计算逻辑来保证一致性。

训练线上一致门禁可以写成：

$$G_{\{\mathrm{skew}\}} = \mathbf{1} [R_{\{\mathrm{skew}\}} \leq \rho \wedge C_{\{\mathrm{transform}\}} = 1 \wedge C_{\{\mathrm{default}\}} = 1]$$

其中 $C_{\mathrm{transform}}$ 表示离线和线上使用同一份或等价的 transform 版本, C_{default} 表示缺失值、截断、类型转换和默认值策略一致。

42.6 Point-in-Time Correctness

特征训练中必须注意 point-in-time correctness。

意思是：训练样本在某个时间点, 只能使用那个时间点之前可获得的特征。

如果使用未来信息, 就会数据泄漏。

例如预测用户明天是否购买, 不能用明天之后的点击数据。

Feature Store 要支持按时间回放历史特征。

这对推荐、风控和在线学习非常关键。

Point-in-time correctness 的核心约束是：

$$t_{\{\mathrm{feature_available}\}}(i, j) \leq t_{\{\mathrm{event}\}}(i)$$

也就是训练样本 i 在事件时间 t_{event} 上, 只能使用当时已经可获得的第 j 个特征。

泄漏率可以写成：

$$R_{\{\mathrm{pit_leak}\}} = \frac{1}{N_{\{\mathrm{join}\}}} \sum_{i, j} \mathbf{1} [t_{\{\mathrm{feature_available}\}}(i, j) > t_{\{\mathrm{event}\}}(i)]$$

$R_{\mathrm{pit_leak}}$ 必须接近 0, 否则离线训练指标会被未来信息污染。

42.7 Embedding Store 是什么

Embedding Store 是向量资产管理系统。

它存储对象的 embedding，例如：

1. 文档 embedding。
2. 段落 embedding。
3. 图片 embedding。
4. 商品 embedding。
5. 用户 embedding。
6. 工具 embedding。
7. API embedding。
8. 代码片段 embedding。

Embedding Store 关心的不只是向量本身，还包括：

1. embedding model version。
2. 生成时间。
3. 原始对象 ID。
4. 元数据。
5. 权限。
6. 版本。
7. 索引状态。

一个 embedding 记录可以写成：

$E_i = (o_i, m_i, v_i, d_i, c_i, g_i, u_i, h_i, z_i)$

其中 o_i 是原始对象， m_i 是 embedding model， v_i 是 model version， d_i 是 dimension， c_i 是 chunking version， g_i 是生成时间， u_i 是权限和租户， h_i 是向量或源内容 checksum， z_i 是索引状态。

Embedding Store 的价值在于能回答：这个向量由哪个对象、哪个 chunk、哪个模型、哪个预处理和哪个权限上下文生成。

42.8 Embedding 为什么要版本化

Embedding 是由模型生成的。

如果 embedding model 变了，同一个文本的向量也会变。

因此必须记录：

1. embedding model name。
2. embedding model version。
3. pooling 方法。
4. normalize 策略。
5. dimension。
6. chunking 策略。
7. preprocessing 规则。

不同版本的 embedding 通常不能混在同一个索引里直接比较。

否则检索结果会不可控。

embedding 版本门禁可以写成：

$G_{\{\mathrm{emb}\}} = \mathbf{1} [C_{\{\mathrm{model}\}} = 1 \wedge C_{\{\mathrm{dim}\}} = 1 \wedge C_{\{\mathrm{chunk}\}} = 1 \wedge C_{\{\mathrm{norm}\}} = 1]$

其中 C_{model} 、 C_{dim} 、 C_{chunk} 、 C_{norm} 分别表示 embedding 模型版本、向量维度、chunking 版本和 normalize 策略一致。只要其中一项变化，就要重新评估是否需要新索引或双索引切换。

42.9 向量索引是什么

向量索引用于高效相似度检索。

给定 query embedding, 找到最相似的 top-k 向量。

常见应用：

1. RAG 文档召回。
2. 相似问题匹配。
3. 图片检索。
4. 推荐召回。
5. 工具选择。
6. 代码搜索。
7. 语义缓存。

如果向量数量很少，可以暴力计算相似度。

如果有百万、千万、十亿级向量，就需要索引。

暴力检索的计算量可以粗略写成：

$$T_{\{\mathrm{brute}\}} \propto N d$$

其中 N 是向量数， d 是向量维度。 N 很大时，即使用矩阵乘法优化，也会遇到延迟、内存带宽和成本压力。

常见余弦相似度写成：

$$\mathrm{cos}(q, x) = \frac{q^{\mathrm{top} \ x}}{\|q\|_2 \|x\|_2}$$

如果向量已经归一化，余弦相似度等价于点积排序。

42.10 ANN 检索

ANN 是 Approximate Nearest Neighbor, 近似最近邻检索。

它牺牲一点召回精度，换取更快查询。

常见索引类型：

1. HNSW。
2. IVF。
3. PQ。
4. ScaNN。
5. DiskANN。
6. Faiss 索引。

选型要考虑：

1. 数据规模。
2. 向量维度。
3. 查询延迟。
4. 召回率。
5. 内存成本。
6. 更新频率。
7. 是否支持过滤。

向量索引是性能、成本和召回率之间的 trade-off。

ANN 质量通常用召回率对比精确检索：

$$R@k = \frac{|S_{\{\mathrm{ann}\}}(k) \cap S_{\{\mathrm{exact}\}}(k)|}{|S_{\{\mathrm{exact}\}}(k)|}$$

其中 $S_{\mathrm{ann}}(k)$ 是 ANN 返回的 top-k 集合， $S_{\mathrm{exact}}(k)$ 是暴力精确检索 top-k 集合。ANN 不是“结果随便近似”，而是在明确召回阈值下换取延迟和成本收益。

42.11 HNSW 的直觉

HNSW 可以直观理解为多层图索引。

高层图用于快速跳转到大致区域，底层图用于精细搜索。

优点：

1. 查询速度快。
2. 召回率高。
3. 适合内存索引。
4. 支持增量插入。

缺点：

1. 内存占用较高。
2. 构建成本不低。
3. 删除和压缩复杂。

HNSW 常用于中大规模低延迟向量检索。

HNSW 的内存可以粗略理解为向量本体加图边：

$$M_{\{\mathrm{hnsw}\}} \approx Ndb + NM b_{\{\mathrm{id}\}} + M_{\{\mathrm{overhead}\}}$$

其中 N 是向量数， d 是维度， b 是每个向量元素字节数， M 是每个节点近邻连接数， b_{id} 是邻居 id 字节数。HNSW 召回高、延迟低，但内存通常比压缩索引更贵。

42.12 IVF 和 PQ 的直觉

IVF 是先把向量聚类到多个桶里，查询时只搜索部分桶。

PQ 是 Product Quantization，用压缩表示降低存储和计算成本。

IVF/PQ 更适合大规模向量和成本敏感场景。

代价是：

1. 召回率可能下降。
2. 参数调优复杂。
3. 构建和重建需要成本。

大规模向量系统常常在内存、磁盘、召回率和延迟之间做组合优化。

IVF 查询只扫描部分桶，扫描比例可以写成：

$$R_{\{\mathrm{scan}\}} = \frac{n_{\{\mathrm{probe}\}}}{n_{\{\mathrm{list}\}}}$$

其中 n_{probe} 是查询时探测的桶数， n_{list} 是总桶数。 n_{probe} 越大，召回通常越高，延迟也越高。

PQ 压缩后的存储可以粗略写成：

$$M_{\{\mathrm{pq}\}} \approx \frac{Nm b_{\{\mathrm{code}\}}}{8} + M_{\{\mathrm{codebook}\}}$$

其中 m 是子向量段数， b_{code} 是每段编码 bit 数。PQ 适合降低存储和内存成本，但需要接受量化误差。

42.13 向量索引的元数据过滤

实际检索往往不只是向量相似。

还要按元数据过滤：

1. 租户。
2. 权限。
3. 文档类型。
4. 时间范围。

5. 语言。
6. 产品线。
7. 安全等级。

例如企业 RAG 中，用户只能检索自己有权限的文档。

如果只做向量相似而不做权限过滤，会造成数据泄露。

过滤选择率可以写成：

$$R_{\{\mathrm{filter}\}} = \frac{N_{\{\mathrm{after}\}}}{N_{\{\mathrm{before}\}}}$$

其中 N_{before} 是向量相似召回候选数， N_{after} 是 metadata / permission filter 后的候选数。企业 RAG 中，过滤不是可选优化，而是安全边界。

权限泄露率可以写成：

$$R_{\{\mathrm{leak}\}} = \frac{N_{\{\mathrm{unauth_returned}\}}}{N_{\{\mathrm{returned}\}}}$$

这个指标必须为 0。

42.14 Chunking 和 Embedding

文档通常不能整体做 embedding，需要切 chunk。

Chunking 策略会影响检索质量。

常见策略：

1. 固定长度切分。
2. 按段落切分。
3. 按标题结构切分。
4. sliding window。
5. 语义切分。
6. 代码按函数或类切分。

需要记录：

1. chunking 规则版本。
2. chunk size。
3. overlap。
4. 原文档 ID。
5. chunk offset。

否则检索结果无法复现和调优。

固定长度 sliding window 的 chunk 数可以粗略写成：

$$N_{\{\mathrm{chunk}\}} = \left\lceil \frac{\max(0, L-s)}{s-o} \right\rceil + 1$$

其中 L 是文档 token 长度， s 是 chunk size， o 是 overlap。增大 overlap 可能提升跨段召回，但也会增加 embedding 成本、索引规模和重复召回。

42.15 Re-indexing

当 embedding 模型、chunking 策略或数据更新后，需要重新建索引。

触发原因：

1. 文档新增。
2. 文档删除。
3. 文档更新。
4. embedding model 升级。
5. chunking 策略变化。
6. 索引参数变化。

Re-indexing 要考虑：

1. 增量更新还是全量重建。
2. 旧索引和新索引切换。
3. 查询一致性。
4. 回滚。
5. 构建成本。

大规模索引重建可能需要数小时甚至更久。

索引重建成本可以拆成：

$$T_{\{\mathrm{reindex}\}} = T_{\{\mathrm{extract}\}} + T_{\{\mathrm{embed}\}} + T_{\{\mathrm{build}\}} + T_{\{\mathrm{validate}\}} + T_{\{\mathrm{switch}\}}$$

向量重建成本可以粗略写成：

$$K_{\{\mathrm{embed}\}} = N_{\{\mathrm{chunk}\}} K_{\{\mathrm{per_chunk}\}}$$

其中 N_{chunk} 是需要重新 embedding 的 chunk 数， $K_{\mathrm{per_chunk}}$ 是单 chunk embedding 成本。embedding model 或 chunking 变更通常比单纯新增文档更贵，因为可能触发全量重建。

42.16 双索引切换

为了降低索引更新风险，可以使用双索引切换。

流程：

```
active index: v1
build shadow index: v2
validate v2
canary traffic to v2
switch active to v2
keep v1 for rollback
```

这样可以避免直接在生产索引上修改导致不可控风险。

向量索引也需要灰度和回滚。

双索引切换门禁可以写成：

$$G_{\{\mathrm{switch}\}} = \mathbf{1}[C_{\{\mathrm{shadow}\}} = 1 \wedge R@k \geq \tau_R \wedge L_{\{\mathrm{p95}\}} \leq \tau_L \wedge C_{\{\mathrm{filter}\}} = 1]$$

其中 C_{shadow} 表示 shadow index 构建完成， $R@k$ 是召回率， L_{p95} 是检索 P95 延迟， C_{filter} 表示权限过滤一致， C_{rollback} 表示旧索引仍可回滚。

42.17 Embedding Pipeline

一个 embedding 生产链路：

```
Raw Object
-> Preprocess
-> Chunking
-> Embedding Model
-> Embedding Store
-> Index Builder
-> Vector Index
-> Retrieval Service
```

每一步都要记录版本和状态。

如果检索效果变差，要能判断是文档变了、chunking 变了、embedding 模型变了，还是索引参数变了。

Embedding pipeline 的版本指纹可以写成：

$$h_{\{\mathrm{pipe}\}} = H(d_{\{\mathrm{src}\}}, v_{\{\mathrm{prep}\}}, v_{\{\mathrm{chunk}\}}, v_{\{\mathrm{emb}\}}, v_{\{\mathrm{index}\}})$$

其中 H 是稳定 hash。只要原始数据、预处理、chunking、embedding model 或索引参数变化，指纹就应该变化，避免把新旧产物混在一起。

42.18 Retrieval Service

Retrieval Service 对外提供检索接口。

请求可能包括：

1. query text。
2. query embedding。
3. top-k。
4. filters。
5. rerank 配置。
6. tenant。
7. permission context。

返回包括：

1. object ID。
2. chunk ID。
3. score。
4. metadata。
5. source pointer。
6. index version。

服务必须记录 trace，方便排查 “为什么召回了这些文档”。

一次检索请求可以写成：

$Q_i = (u_i, t_i, q_i, k_i, \phi_i, \rho_i, \eta_i, z_i)$

其中 u_i 是用户或租户， t_i 是时间， q_i 是 query， k_i 是 top-k， ϕ_i 是 metadata filter， ρ_i 是 rerank 配置， η_i 是权限上下文， z_i 是 trace id。

检索 trace 完整率可以写成：

$C_{\{\mathrm{trace}\}} = \frac{1}{N_{\{\mathrm{trace}\}}} \sum_{j=1}^{N_{\{\mathrm{trace}\}}} \mathbf{1}[\sigma_j \setminus \mathrm{captured}]$

σ_j 可以包括 query hash、embedding model、index version、filters、candidate ids、scores、rerank ids、permission decision、latency 和 source pointers。

42.19 向量检索质量指标

常见指标：

1. recall@k。
2. precision@k。
3. MRR。
4. NDCG。
5. hit rate。
6. latency p95/p99。
7. index build time。
8. index size。
9. filter selectivity。
10. empty result rate。

RAG 场景还要看：

1. answer correctness。
2. citation correctness。
3. groundedness。

4. hallucination rate。

检索质量不能只看向量相似分数。

常见排序指标可以写成：

$$\mathrm{precision@k} = \frac{N_{\{\mathrm{relevant@k}\}}}{k}$$

$$\mathrm{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\mathrm{rank}_i}$$

$$\mathrm{NDCG@k} = \frac{\mathrm{DCG@k}}{\mathrm{IDCG@k}}$$

上线门禁通常要同时看召回、排序、权限、延迟、空结果、引用支持和最终回答质量。

42.20 Feature / Embedding 血缘

Feature 和 embedding 都需要血缘。

Feature 血缘：

raw event -> feature transform -> feature version -> model training / online serving

Embedding 血缘：

document -> chunk -> embedding model version -> embedding version -> index version -> retrieval result

血缘可以帮助定位线上效果变化。

例如 RAG 变差，可能是索引版本变化，而不是大模型退化。

Feature / embedding 血缘图可以写成：

$$G_{\{\mathrm{fe}\}} = (V, E)$$

其中节点包括 raw event、feature transform、feature version、document、chunk、embedding model、embedding record、index version、retrieval request、retrieval result、RAG answer；边表示“生成、读取、索引、检索、引用、服务”。这张图能把线上 bad case 追到具体数据和索引版本。

42.21 多租户和权限

向量和特征基础设施必须支持权限隔离。

风险包括：

1. A 租户检索到 B 租户文档。
2. 用户检索到无权限知识库。
3. embedding 泄露敏感语义。
4. 特征被越权读取。
5. 索引构建时混入私有数据。

治理方式：

1. tenant ID 进入 metadata。
2. 检索时强制权限过滤。
3. index 按租户隔离或逻辑隔离。
4. 敏感数据单独索引。
5. 访问日志和审计。

不要把向量看成“不可还原所以不敏感”。embedding 仍然可能泄露信息。

多租户隔离门禁可以写成：

$$G_{\{\mathrm{tenant}\}} = \mathbf{1}[C_{\{\mathrm{tenant}\}} = 1 \wedge C_{\{\mathrm{acl}\}} = 1 \wedge R_{\{\mathrm{leak}\}} = 0 \wedge C_{\{\mathrm{audit}\}} = 1]$$

其中 C_{tenant} 表示 tenant metadata 必填， C_{acl} 表示检索时强制 ACL 过滤， R_{leak} 是无权返回率， C_{audit} 表示访问日志和审计 trace 完整。

42.22 Feature Store、Embedding Store 和 RAG

RAG 系统通常依赖：

1. 文档存储。
2. chunk 存储。
3. embedding store。
4. vector index。
5. metadata filter。
6. reranker。
7. retrieval trace。

Feature Store 也可能参与 RAG：

1. 用户画像特征影响检索策略。
2. 文档热度特征影响排序。
3. 权限特征影响过滤。
4. 历史反馈特征影响 rerank。

RAG 不只是一个向量数据库，而是一条完整检索和治理链路。

RAG 检索门禁可以写成：

$$G_{\{\mathrm{rag_retrieval}\}}=\mathbf{1}[R@k\geq\tau_R\ \&\ C_{\{\mathrm{citation}\}}\geq\tau_C\ \&\ R_{\{\mathrm{Leak}\}}=0\ \&\ \mathrm{La}...$$

这里 C_{citation} 是引用能否追到 source pointer、chunk id、doc version 和权限范围的覆盖率。RAG / Agent 平台如果丢了这些元数据，就很难解释答案来源。

42.23 常见系统架构

一个基础设施架构：

Data Sources

- > Feature Pipeline / Embedding Pipeline
- > Offline Store / Embedding Store
- > Online Store / Vector Index
- > Feature Serving / Retrieval Service
- > Model Serving / RAG / Agent
- > Monitoring / Lineage / Access Control

关键是把 batch 构建、在线服务、版本管理、权限和监控打通。

42.24 特征向量基础设施审计指标和最小 demo

把本章内容合在一起，可以用 16 个维度审计 Feature Store、Embedding Store 和 Vector Index：

1. feature definition contract：特征名称、版本、实体键、来源、transform、窗口、TTL、owner 和权限是否完整。
2. offline online consistency：offline store 和 online store 是否使用同一特征定义、transform、默认值和类型策略。
3. point in time correctness：离线训练样本是否只读取当时已经可获得的历史特征。
4. embedding version contract：embedding model、版本、维度、pooling、normalize、预处理和 chunking 版本是否记录。
5. chunk embedding lineage：文档、chunk、offset、checksum、embedding record 和权限元数据是否可追踪。
6. vector index build readiness：索引版本、维度、metric、参数、构建状态、构建输入和 shadow index 是否完整。
7. ANN quality latency gate：ANN recall@k、P95/P99 延迟、空结果率和过滤选择率是否达标。
8. metadata permission filter：tenant、ACL、数据等级和敏感索引隔离是否在检索阶段强制执行。
9. shadow index switch readiness：双索引构建、验证、canary、切换和回滚是否完整。

10. retrieval trace completeness: query、embedding model、index version、filters、candidate ids、scores、rerank、权限决策和 source pointer 是否进入 trace。
11. feature embedding lineage graph: feature、document、chunk、embedding、index、retrieval、RAG answer 是否可双向追踪。
12. multi tenant isolation: 租户元数据、配额、物理或逻辑隔离、跨租户命中阻断和审计是否完整。
13. quality monitoring metrics: recall、precision、latency、stale feature、skew、permission leak 和 empty result 是否监控。
14. cost capacity governance: 向量内存、索引构建成本、embedding 成本、查询成本、预算和 owner 是否可治理。
15. RAG Agent integration: retrieval trace、rerank、citation、Agent memory scope 和 permission context 是否打通。
16. feature embedding index gate: 最终门禁是否能证明覆盖率达标、无 P0 风险、可回滚、可观测、可审计。

下面的 demo 不依赖外部库。它用一个完整样本和 16 个单点坏样本说明：这类基础设施不能只看“能查向量”，而要同时证明特征、embedding、chunk、索引、过滤、trace、血缘、租户和 RAG / Agent 集成都可信。

```
from math import sqrt
```

```
class MiniFeatureEmbeddingIndexAudit:
```

```
    REQUIRED_FEATURE_FIELDS = {
```

```
        "name",
        "version",
        "owner",
        "entity_key",
        "source",
        "transform_version",
        "dtype",
        "window",
        "ttl_minutes",
        "permission",
    }
```

```
    REQUIRED_EMBEDDING_MODEL_FIELDS = {
```

```
        "model_name",
        "model_version",
        "dimension",
        "pooling",
        "normalize",
        "preprocess_version",
        "chunking_version",
    }
```

```
    REQUIRED_TRACE_SIGNALS = {
```

```
        "query_hash",
        "embedding_model_version",
        "index_version",
        "filters",
        "candidate_ids",
        "scores",
        "rerank_ids",
        "permission_decision",
        "latency_ms",
        "source_pointers",
    }
```

```

REQUIRED_QUALITY_METRICS = {
    "recall_at_10",
    "precision_at_5",
    "p95_ms",
    "empty_result_rate",
    "stale_feature_rate",
    "skew_rate",
    "permission_leak_rate",
}

def __init__(self, cases):
    self.cases = cases
    self.checks = [
        ("feature_definition_contract", self.feature_definition_contract),
        ("offline_online_consistency", self.offline_online_consistency),
        ("point_in_time_correctness", self.point_in_time_correctness),
        ("embedding_version_contract", self.embedding_version_contract),
        ("chunk_embedding_lineage", self.chunk_embedding_lineage),
        ("vector_index_build_readiness", self.vector_index_build_readiness),
        ("ann_quality_latency_gate", self.ann_quality_latency_gate),
        ("metadata_permission_filter", self.metadata_permission_filter),
        ("shadow_index_switch_readiness", self.shadow_index_switch_readiness),
        ("retrieval_trace_completeness", self.retrieval_trace_completeness),
        ("feature_embedding_lineage_graph", self.feature_embedding_lineage_graph),
        ("multi_tenant_isolation", self.multi_tenant_isolation),
        ("quality_monitoring_metrics", self.quality_monitoring_metrics),
        ("cost_capacity_governance", self.cost_capacity_governance),
        ("rag_agent_integration", self.rag_agent_integration),
        ("feature_embedding_index_gate", self.feature_embedding_index_gate),
    ]

def feature_definition_contract(self, case):
    features = case["features"]
    return bool(features) and all(self.REQUIRED_FEATURE_FIELDS <= set(item) and item["ttl_minutes"] > 0 for item

def offline_online_consistency(self, case):
    consistency = case["offline_online"]
    offline = consistency["offline_values"]
    online = consistency["online_values"]
    if not consistency.get("same_transform_version") or not consistency.get("same_default_policy"):
        return False
    return all(abs(offline[name] - online.get(name, float("inf")))) <= 1e-9 for name in offline)

def point_in_time_correctness(self, case):
    if not case["point_in_time"].get("point_in_time_join"):
        return False
    return all(item["feature_available_ts"] <= item["event_ts"] for item in case["point_in_time"]["training_rows"])

def embedding_version_contract(self, case):
    model = case["embedding_model"]
    if not self.REQUIRED_EMBEDDING_MODEL_FIELDS <= set(model):
        return False
    dimension = model["dimension"]
    vectors = case["embedding_records"]
    if any(len(item["vector"]) != dimension for item in vectors):

```

```

        return False
    if model["normalize"]:
        return all(abs(vector_norm(item["vector"]) - 1.0) <= 1e-3 for item in vectors)
    return True

def chunk_embedding_lineage(self, case):
    embedding_ids = {item["embedding_id"] for item in case["embedding_records"]}
    for chunk in case["chunks"]:
        required = {"doc_id", "chunk_id", "offset_start", "offset_end", "chunking_version", "embedding_id", "checked"}
        if not required <= set(chunk):
            return False
        if chunk["offset_start"] >= chunk["offset_end"] or chunk["embedding_id"] not in embedding_ids:
            return False
    return True

def vector_index_build_readiness(self, case):
    index = case["index"]
    required = {"index_id", "version", "dimension", "metric", "index_type", "build_status", "index_params", "embedding_id"}
    return required <= set(index) and index["build_status"] == "ready" and index["dimension"] == case["embedding_dimension"]

def ann_quality_latency_gate(self, case):
    ann = case["ann_quality"]
    return (
        ann["recall_at_10"] >= 0.9
        and ann["p95_ms"] <= 50
        and ann["p99_ms"] <= 100
        and ann["empty_result_rate"] <= 0.05
    )

def metadata_permission_filter(self, case):
    filt = case["permission_filter"]
    return (
        filt.get("tenant_filter")
        and filt.get("acl_filter")
        and filt.get("data_classification_filter")
        and filt.get("unauthorized_returned") == 0
        and filt.get("audit_log")
    )

def shadow_index_switch_readiness(self, case):
    switch = case["shadow_switch"]
    return (
        switch.get("shadow_built")
        and switch.get("shadow_validated")
        and switch.get("canary_passed")
        and bool(switch.get("rollback_index"))
        and bool(switch.get("switch_plan"))
    )

def retrieval_trace_completeness(self, case):
    return self.REQUIRED_TRACE_SIGNALS <= case["trace"].get("signals", set())

def feature_embedding_lineage_graph(self, case):
    lineage = case["lineage"]
    required_nodes = {"feature", "document", "chunk", "embedding", "index", "retrieval", "rag_answer"}

```

```

    return required_nodes <= set(lineage["nodes"]) and len(lineage["edges"]) >= 9

def multi_tenant_isolation(self, case):
    tenant = case["multi_tenant"]
    return (
        tenant.get("tenant_key_in_metadata")
        and tenant.get("per_tenant_quota")
        and tenant.get("sensitive_index_isolated")
        and tenant.get("no_cross_tenant_hits")
        and tenant.get("audit_ready")
    )

def quality_monitoring_metrics(self, case):
    metrics = case["quality_monitoring"]["metrics"]
    return (
        self.REQUIRED_QUALITY_METRICS <= set(metrics)
        and metrics["recall_at_10"] >= 0.9
        and metrics["p95_ms"] <= 50
        and metrics["skew_rate"] <= 0.01
        and metrics["permission_leak_rate"] == 0
    )

def cost_capacity_governance(self, case):
    cost = case["cost"]
    return (
        cost.get("owner")
        and cost["memory_gib"] <= cost["memory_budget_gib"]
        and cost["build_cost_usd"] <= cost["build_budget_usd"]
        and cost["query_cost_per_1k"] <= cost["query_budget_per_1k"]
    )

def rag_agent_integration(self, case):
    integration = case["rag_agent"]
    required = {"retrieval_trace", "rerank_linked", "citation_source_pointers", "agent_memory_scope", "permission"}
    return required <= set(integration) and all(integration[name] for name in required)

def feature_embedding_index_gate(self, case):
    gate = case["gate"]
    return gate.get("enabled") and gate.get("min_coverage", 0.0) >= 0.95 and gate.get("p0_risks", 1) == 0

def metric_scores(self):
    total = len(self.cases)
    return {
        name: round(sum(1 for case in self.cases if check(case)) / total, 3)
        for name, check in self.checks
    }

def failed_cases(self):
    failures = []
    for case in self.cases:
        failed = [name for name, check in self.checks if not check(case)]
        if failed:
            failures.append((case["case_id"], failed))
    return failures

```

```

def examples(self):
    case = self.cases[0]
    offline = case["offline_online"]["offline_values"]
    online = case["offline_online"]["online_values"]
    skew_rate = sum(1 for name in offline if abs(offline[name] - online[name]) > 1e-9) / len(offline)
    pit_rows = case["point_in_time"]["training_rows"]
    pit_leak_rate = sum(1 for row in pit_rows if row["feature_available_ts"] > row["event_ts"]) / len(pit_rows)
    chunk_embedding_coverage = sum(1 for chunk in case["chunks"] if chunk.get("embedding_id")) / len(case["chunks"])
    brute_force = brute_force_search(
        query=case["query"]["vector"],
        records=case["embedding_records"],
        top_k=3,
        tenant=None,
        role=None,
    )
    filtered = brute_force_search(
        query=case["query"]["vector"],
        records=case["embedding_records"],
        top_k=3,
        tenant=case["query"]["tenant"],
        role=case["query"]["role"],
    )
    vector_bytes = case["index"]["vector_count"] * case["index"]["dimension"] * case["index"]["dtype_bytes"]
    vector_memory_gib = vector_bytes / (1024 ** 3)
    return {
        "feature_contract_coverage": round(sum(1 for item in case["features"] if self.REQUIRED_FEATURE_FIELDS <=
        "offline_online_skew_rate": round(skew_rate, 3),
        "point_in_time_leak_rate": round(pit_leak_rate, 3),
        "embedding_dimension": case["embedding_model"]["dimension"],
        "embedding_norm_ok": self.embedding_version_contract(case),
        "chunk_embedding_coverage": round(chunk_embedding_coverage, 3),
        "bruteforce_top3": [item["chunk_id"] for item in brute_force],
        "permission_filtered_top3": [item["chunk_id"] for item in filtered],
        "cross_tenant_blocked": all(item["tenant"] == case["query"]["tenant"] for item in filtered),
        "ann_recall_at_10": case["ann_quality"]["recall_at_10"],
        "retrieval_p95_ms": case["ann_quality"]["p95_ms"],
        "vector_memory_gib": round(vector_memory_gib, 2),
        "shadow_index_ready": self.shadow_index_switch_readiness(case),
        "lineage_edge_count": len(case["lineage"]["edges"]),
        "estimated_build_cost_usd": case["cost"]["build_cost_usd"],
    }

```

```

def vector_norm(vector):
    return sqrt(sum(value * value for value in vector))

```

```

def dot(a, b):
    return sum(x * y for x, y in zip(a, b))

```

```

def brute_force_search(query, records, top_k, tenant=None, role=None):
    candidates = []
    for record in records:
        if tenant is not None and record["tenant"] != tenant:

```



```

        continue
    if role is not None and role not in record["acl"]:
        continue
    candidates.append((dot(query, record["vector"]), record))
candidates.sort(key=lambda item: item[0], reverse=True)
return [record for _, record in candidates[:top_k]]

def complete_case():
    return {
        "case_id": "complete_feature_embedding_index_ok",
        "features": [
            {
                "name": "user_activity_7d",
                "version": "v3",
                "owner": "growth",
                "entity_key": "user_id",
                "source": "events.daily_user_activity",
                "transform_version": "tx-activity-v3",
                "dtype": "float",
                "window": "7d",
                "ttl_minutes": 120,
                "permission": "tenant_private",
            },
            {
                "name": "doc_hotness",
                "version": "v2",
                "owner": "search",
                "entity_key": "doc_id",
                "source": "logs.doc_views",
                "transform_version": "tx-doc-hotness-v2",
                "dtype": "float",
                "window": "1d",
                "ttl_minutes": 60,
                "permission": "tenant_private",
            },
            {
                "name": "tool_success_rate",
                "version": "v5",
                "owner": "agent-platform",
                "entity_key": "tool_id",
                "source": "agent.tool_calls",
                "transform_version": "tx-tool-success-v5",
                "dtype": "float",
                "window": "30d",
                "ttl_minutes": 180,
                "permission": "platform_internal",
            },
        ],
        "offline_online": {
            "same_transform_version": True,
            "same_default_policy": True,
            "offline_values": {"user_activity_7d": 12.0, "doc_hotness": 0.82, "tool_success_rate": 0.91},
            "online_values": {"user_activity_7d": 12.0, "doc_hotness": 0.82, "tool_success_rate": 0.91},
        },
    }

```

```

"point_in_time": {
  "point_in_time_join": True,
  "training_rows": [
    {"entity": "user_1", "event_ts": 100, "feature_available_ts": 94},
    {"entity": "user_2", "event_ts": 130, "feature_available_ts": 125},
    {"entity": "doc_8", "event_ts": 180, "feature_available_ts": 170},
  ],
},
"embedding_model": {
  "model_name": "text-embedder",
  "model_version": "embed-v4",
  "dimension": 4,
  "pooling": "mean",
  "normalize": True,
  "preprocess_version": "prep-v2",
  "chunking_version": "chunk-v6",
},
"embedding_records": [
  {"embedding_id": "emb_policy", "chunk_id": "chunk_policy", "tenant": "acme", "acl": {"analyst"}, "vector"},
  {"embedding_id": "emb_pricing", "chunk_id": "chunk_pricing", "tenant": "acme", "acl": {"analyst"}, "vector"},
  {"embedding_id": "emb_beta", "chunk_id": "chunk_beta_private", "tenant": "beta", "acl": {"admin"}, "vector"},
  {"embedding_id": "emb_sensitive", "chunk_id": "chunk_sensitive", "tenant": "acme", "acl": {"security"}, "vector"},
],
"chunks": [
  {"doc_id": "doc_policy", "chunk_id": "chunk_policy", "offset_start": 0, "offset_end": 480, "chunking_version": "chunk-v6"},
  {"doc_id": "doc_pricing", "chunk_id": "chunk_pricing", "offset_start": 480, "offset_end": 960, "chunking_version": "chunk-v6"},
  {"doc_id": "doc_beta", "chunk_id": "chunk_beta_private", "offset_start": 0, "offset_end": 420, "chunking_version": "chunk-v6"},
  {"doc_id": "doc_sensitive", "chunk_id": "chunk_sensitive", "offset_start": 0, "offset_end": 390, "chunking_version": "chunk-v6"},
],
"index": {
  "index_id": "kb-acme-shadow",
  "version": "index-v12",
  "dimension": 4,
  "metric": "cosine",
  "index_type": "hns",
  "build_status": "ready",
  "index_params": {"M": 32, "ef_search": 128},
  "embedding_model_version": "embed-v4",
  "chunking_version": "chunk-v6",
  "vector_count": 20_000_000,
  "dtype_bytes": 2,
},
"ann_quality": {"recall_at_10": 0.93, "p95_ms": 35, "p99_ms": 78, "empty_result_rate": 0.018},
"permission_filter": {
  "tenant_filter": True,
  "acl_filter": True,
  "data_classification_filter": True,
  "unauthorized_returned": 0,
  "audit_log": True,
},
"shadow_switch": {
  "active_index": "index-v11",
  "shadow_index": "index-v12",
  "shadow_built": True,
  "shadow_validated": True,
}

```

```

    "canary_passed": True,
    "rollback_index": "index-v11",
    "switch_plan": "canary-then-promote",
},
"trace": {
    "signals": {
        "query_hash",
        "embedding_model_version",
        "index_version",
        "filters",
        "candidate_ids",
        "scores",
        "rerank_ids",
        "permission_decision",
        "latency_ms",
        "source_pointers",
    }
},
"lineage": {
    "nodes": {"feature", "document", "chunk", "embedding", "index", "retrieval", "rag_answer"},
    "edges": [
        ("raw_event", "feature"),
        ("document", "chunk"),
        ("chunk", "embedding"),
        ("embedding", "index"),
        ("index", "retrieval"),
        ("feature", "retrieval"),
        ("retrieval", "rag_answer"),
        ("permission", "retrieval"),
        ("rag_answer", "evaluation"),
    ],
},
"multi_tenant": {
    "tenant_key_in_metadata": True,
    "per_tenant_quota": True,
    "sensitive_index_isolated": True,
    "no_cross_tenant_hits": True,
    "audit_ready": True,
},
"quality_monitoring": {
    "metrics": {
        "recall_at_10": 0.93,
        "precision_at_5": 0.82,
        "p95_ms": 35,
        "empty_result_rate": 0.018,
        "stale_feature_rate": 0.004,
        "skew_rate": 0.0,
        "permission_leak_rate": 0,
    }
},
"cost": {
    "owner": "retrieval-platform",
    "memory_gib": 24.0,
    "memory_budget_gib": 64.0,
    "build_cost_usd": 780.0,
}

```

```

        "build_budget_usd": 1200.0,
        "query_cost_per_1k": 0.19,
        "query_budget_per_1k": 0.3,
    },
    "rag_agent": {
        "retrieval_trace": True,
        "rerank_linked": True,
        "citation_source_pointers": True,
        "agent_memory_scope": True,
        "permission_context": True,
    },
    "gate": {"enabled": True, "min_coverage": 0.96, "p0_risks": 0},
    "query": {"tenant": "acme", "role": "analyst", "vector": [1.0, 0.0, 0.0, 0.0]},
}

```

```

def make_bad_case(case_id, mutate):
    case = complete_case()
    case["case_id"] = case_id
    mutate(case)
    return case

```

```

cases = [
    complete_case(),
    make_bad_case("feature_definition_missing_bad", lambda c: c["features"][0].pop("version")),
    make_bad_case("offline_online_skew_bad", lambda c: c["offline_online"]["online_values"].update({"doc_hotness": 0.5})),
    make_bad_case("point_in_time_leak_bad", lambda c: c["point_in_time"]["training_rows"][0].update({"feature_availability": 0.5})),
    make_bad_case("embedding_version_missing_bad", lambda c: c["embedding_model"].pop("model_version")),
    make_bad_case("chunk_lineage_missing_bad", lambda c: c["chunks"][0].pop("embedding_id")),
    make_bad_case("index_not_ready_bad", lambda c: c["index"].update({"build_status": "building"})),
    make_bad_case("ann_recall_low_bad", lambda c: c["ann_quality"].update({"recall_at_10": 0.82})),
    make_bad_case("permission_filter_missing_bad", lambda c: c["permission_filter"].update({"acl_filter": False})),
    make_bad_case("shadow_switch_missing_bad", lambda c: c["shadow_switch"].update({"rollback_index": None})),
    make_bad_case("retrieval_trace_missing_bad", lambda c: c["trace"]["signals"].remove("index_version")),
    make_bad_case("lineage_edge_missing_bad", lambda c: c["lineage"]["edges"].pop()),
    make_bad_case("multi_tenant_isolation_bad", lambda c: c["multi_tenant"].update({"no_cross_tenant_hits": False})),
    make_bad_case("quality_monitoring_missing_bad", lambda c: c["quality_monitoring"]["metrics"].pop("permission_leak")),
    make_bad_case("cost_capacity_unbounded_bad", lambda c: c["cost"].update({"build_cost_usd": 1800.0})),
    make_bad_case("rag_agent_trace_missing_bad", lambda c: c["rag_agent"].update({"citation_source_pointers": False})),
    make_bad_case("infrastructure_gate_missing_bad", lambda c: c["gate"].update({"enabled": False})),
]

```

```

audit = MiniFeatureEmbeddingIndexAudit(cases)
examples = audit.examples()
failures = audit.failed_cases()
failed_cases = [case_id for case_id, _ in failures]
failed_gates = [gates[0] for _, gates in failures]

print("feature_embedding_index_examples=" + repr(examples))
print("metrics=" + repr(audit.metric_scores()))
print("hard_blocker_count=" + str(len(failures)))
print("failed_cases=" + repr(failed_cases))
print("failed_gates=" + repr(failed_gates))
print("feature_embedding_index_gate_pass=" + str(not failures))

```

```
feature_embedding_index_examples={'feature_contract_coverage': 1.0, 'offline_online_skew_rate': 0.0, 'point_in_time_leak_rate': 0.0, 'point_in_time_correctness': 0.941, 'embedding_version': 'v1'}
metrics={'feature_definition_contract': 0.941, 'offline_online_consistency': 0.941, 'point_in_time_correctness': 0.941, 'embedding_version': 'v1'}
hard_blocker_count=16
failed_cases=['feature_definition_missing_bad', 'offline_online_skew_bad', 'point_in_time_leak_bad', 'embedding_version_bad']
failed_gates=['feature_definition_contract', 'offline_online_consistency', 'point_in_time_correctness', 'embedding_version']
feature_embedding_index_gate_pass=False
```

42.25 常见误区

向量数据库只是检索组件，RAG 还需要文档处理、chunking、embedding、权限、rerank、prompt 组装和 trace。

embedding 强依赖模型版本、预处理和 chunking, 不版本化会导致检索不可复现。

这会带来 training-serving skew, 应该统一特征定义和版本。

相似度只是召回信号，还需要 rerank、metadata filter 和任务评估。

embedding 可能泄露语义和私有信息，必须做权限和审计。

42.26 面试常见追问

可以回答：它统一特征定义、计算、存储、版本和在线服务，减少训练和线上特征不一致，支持特征复用、血缘和权限治理。

可以回答：Embedding Store 管向量资产及其元数据、版本、来源和权限；向量索引负责高效相似度检索。前者偏资产管理，后者偏检索性能。

可以回答: embedding 依赖 embedding model、预处理、chunking、pooling 和 normalize 策略。版本变了, 向量空间也可能变, 不能混用。

可以回答：文档、chunk、embedding 和索引都记录租户和权限元数据，检索时强制权限过滤，敏感数据可独立索引，并记录 retrieval trace 和审计日志。

可以回答: embedding model、chunking、索引参数、metadata filter 或数据更新都会改变召回集合。直接替换生产索引可能导致召回退化、权限过滤失效、延迟上涨或空结果率升高。shadow index 先构建和评估, canary 小流量验证, 保留旧索引回滚, 才能把检索基础设施当成可治理生产系统。

42.27 小练习

1. Feature Store 的 offline store 和 online store 有什么区别?
2. 什么是 training-serving skew?
3. Point-in-time correctness 为什么重要?
4. Embedding Store 应记录哪些元数据?
5. HNSW、IVF、PQ 分别适合什么场景?
6. Chunking 策略如何影响 RAG 质量?
7. 为什么向量索引更新需要灰度和回滚?
8. 如何设计一个支持权限过滤的企业向量检索服务?
9. 写一个 0 依赖脚本, 检查 embedding model version、chunking version、index version 和 retrieval trace 是否完整。
10. 扩展本章 demo, 加入一个 “旧索引 metadata filter 漏掉 tenant” 的 bad case, 并让它触发 metadata permission filter。

42.28 本章小结

本章讲了 Feature Store、Embedding Store 与向量索引基础设施。

你需要记住:

1. Feature Store 管理结构化特征, 核心价值是复用、版本、训练线上一致和血缘。
2. Embedding Store 管理向量资产, 必须记录 embedding model、chunking、预处理、版本和权限。
3. 向量索引负责高效 ANN 检索, 需要在召回、延迟、成本和更新复杂度之间权衡。
4. RAG 不是向量数据库本身, 而是文档处理、embedding、索引、过滤、rerank、prompt 和 trace 的完整链路。
5. 特征和向量基础设施必须支持多租户、权限控制、版本化、血缘和质量监控。
6. 生产级特征和向量平台必须有 point-in-time、training-serving skew、embedding 版本、shadow index、permission filter、retrieval trace、lineage、成本和回滚门禁。

下一章我们会讲 RAG/Agent 平台中的知识库、工具和 trace 存储。

第 43 章 RAG/Agent 平台中的知识库、工具和 trace 存储

上一章讲了 Feature Store、Embedding Store 与向量索引基础设施。本章作为第五部分收尾, 讲 RAG/Agent 平台中的知识库、工具和 trace 存储。

RAG 和 Agent 是大模型应用平台中的常见形态。它们不只依赖模型本身, 还依赖知识库、工具、权限、记忆、调用链路和可回放 trace。如果这些资产没有统一管理, 应用很快会变成难以调试和治理的黑盒。

先记住一句话:

RAG/Agent 平台的核心数据资产, 不只是 prompt 和向量库, 还包括知识库、工具定义、权限、检索记录、工具调用记录和完整执行 trace。

43.0 本讲资料边界与第二轮精修口径

本章按通用 RAG / Agent 平台基础设施抽象来写, 不绑定某个向量数据库、Agent 框架、工具协议、可观测性产品、trace SaaS 或内部平台实现。资料校准时, 主要参考 OpenTelemetry 对 trace / span / event / attribute 的通用可观测性模型、W3C Trace Context 对跨服务 trace 上下文传播的标准化口径、LangSmith / LlamaIndex 对 RAG / Agent 应用中的 run、document、node、metadata、retrieval 和 eval 记录方式、OpenAI Agents SDK 对 agent workflow tracing 的工程边界, 以及 MCP 对 tool 定义、schema 和 server 能力声明的协议化表达。

第二轮精修只做三件事:

1. 把知识库、文档、chunk、citation、tool、memory、run state 和 trace 统一成可审计对象。
2. 补齐权限、版本、回放、隐私、采样、成本和删除传播的公式化门禁。

3. 增加一个 0 依赖 Python demo, 用 toy cases 证明 RAG / Agent 平台存储不能只靠向量库或日志表。

43.1 为什么需要 RAG/Agent 平台存储

一个简单 RAG demo 可能只需要:

PDF -> chunk -> embedding -> vector search -> prompt -> LLM

一个简单 Agent demo 可能只需要:

LLM -> tool call -> tool result -> LLM

但生产平台要处理:

1. 多知识库。
2. 多租户权限。
3. 文档版本。
4. 工具版本。
5. 工具权限。
6. 检索 trace。
7. 工具调用 trace。
8. 多轮状态。
9. 失败回放。
10. 安全审计。

这些都需要专门的数据和存储设计。

可以把一次 RAG / Agent 平台运行抽象成:

$$P_i = (k_i, d_i, c_i, e_i, v_i, q_i, p_i, a_i, u_i, m_i, r_i, s_i, z_i)$$

其中 k_i 是知识库版本, d_i 是文档版本集合, c_i 是 chunk 集合, e_i 是 embedding 版本, v_i 是向量索引版本, q_i 是检索请求, p_i 是 prompt assembly, a_i 是 agent definition, u_i 是 tool 调用集合, m_i 是 memory / run state, r_i 是执行 trace, s_i 是安全与权限决策, z_i 是成本和审计记录。

平台存储覆盖率可以写成:

$$C_{\text{storage}} = \frac{1}{N} \sum_{i=1}^N I(k_i, d_i, c_i, e_i, v_i, q_i, p_i, a_i, u_i, m_i, r_i, s_i, z_i \text{ complete})$$

直觉上, 只有 prompt 或向量库记录完整不够; RAG / Agent 的一次回答必须能追溯到知识、工具、权限、trace、成本和审计证据。

43.2 RAG 平台中的核心资产

RAG 平台常见资产包括:

1. Knowledge Base。
2. Document。
3. Chunk。
4. Embedding。
5. Vector Index。
6. Retrieval Config。
7. Rerank Config。
8. Prompt Template。
9. Citation。
10. Retrieval Trace。

其中知识库是逻辑集合，文档是原始内容，chunk 是检索单元，embedding 和 index 负责召回，trace 负责解释和排障。

RAG 资产完整率可以写成：

$$C_{\text{rag}} = \frac{|\{a \in A_{\text{rag}} : a \text{ versioned} \wedge a \text{ traceable}\}|}{|A_{\text{rag}}|}$$

A_{rag} 是知识库、文档、chunk、embedding、index、retrieval config、rerank config、prompt template、citation 和 retrieval trace 的集合。这个指标强调每类资产都要有版本和血缘，而不是只要求向量可查。

43.3 Knowledge Base 设计

Knowledge Base 是知识库对象。

它通常记录：

1. 知识库名称。
2. 所属租户。
3. 描述。
4. 数据来源。
5. 权限策略。
6. embedding 模型版本。
7. chunking 规则版本。
8. 索引版本。
9. 同步状态。
10. 创建人和维护人。

知识库不是一个向量索引名，而是一组文档、版本、权限、索引和策略的集合。

一个 Knowledge Base 契约可以写成：

$$K_j = (\text{kb_id}, \text{tenant}, \text{owner}, S_j, \pi_j, e_j, g_j, v_j, \rho_j, \tau_j)$$

其中 S_j 是来源连接器集合， π_j 是权限策略， e_j 是 embedding 版本， g_j 是 chunking 规则版本， v_j 是索引版本， ρ_j 是同步状态， τ_j 是保留和删除策略。

知识库契约完整率：

$$C_{\text{kb}} = \frac{\sum_j I(K_j \text{ has required fields})}{|\{K_j\}|}$$

43.4 Document 设计

Document 是知识库中的原始文档。

它可以来自：

1. PDF。
2. Markdown。
3. HTML。
4. Word 文档。
5. Wiki 页面。
6. 工单。
7. 数据库记录。
8. 代码文件。

9. API 文档。

Document 元数据包括：

1. document ID。
2. knowledge base ID。
3. source URI。
4. title。
5. author。
6. created time。
7. updated time。
8. content hash。
9. permission labels。
10. document version。
11. parsing status。

文档必须版本化，否则检索结果无法复现。

文档版本对象可以写成：

$$D_j = (\text{doc_id}, \text{kb_id}, u_j, t_j, h_j, \ell_j, v_j, \sigma_j)$$

其中 u_j 是 source URI, t_j 是更新时间, h_j 是内容 hash, ℓ_j 是权限标签, v_j 是 document version, σ_j 是解析状态。文档 hash 和版本一起决定 “当时被检索的到底是哪份内容”。

43.5 Chunk 设计

Chunk 是向量检索的基本单元。

Chunk 元数据包括：

1. chunk ID。
2. document ID。
3. chunk index。
4. text。
5. token count。
6. offset。
7. heading path。
8. chunking rule version。
9. embedding ID。
10. permission labels。

Chunk 要能回溯到原文位置。

这样模型回答引用内容时，才能给出 citation，也方便用户验证来源。

Chunk 记录可以写成：

$$C_j = (\text{chunk_id}, \text{doc_id}, v_j, o_j, n_j, g_j, e_j, h_j, \ell_j)$$

其中 o_j 是原文 offset, n_j 是 token 数, g_j 是 chunking rule version, e_j 是 embedding ID, h_j 是 chunk checksum, ℓ_j 是权限标签。

chunk 到原文的可引用覆盖率：

$$C_{\text{cite_lineage}} = \frac{\sum_j I(\text{doc_id}_j, v_j, o_j, h_j \text{ present})}{|\{C_j\}|}$$

如果 chunk 没有 offset、document version 或 checksum, citation 就只能展示一个看似正确的链接, 不能证明生成时引用的原文片段。

43.6 文档解析和清洗存储

RAG 平台需要保存解析状态。

文档解析可能包括:

1. OCR。
2. PDF 结构解析。
3. 表格抽取。
4. 标题层级识别。
5. 代码块识别。
6. 图片说明生成。
7. 噪声去除。

解析结果应作为 artifact 或 document version 的一部分保存。

否则每次重建索引都可能得到不同 chunk。

43.7 知识库同步

企业知识库经常来自外部系统, 例如 Wiki、网盘、工单系统和代码仓库。

同步系统要记录:

1. source connector。
2. sync job ID。
3. sync time。
4. 新增文档。
5. 更新文档。
6. 删除文档。
7. 失败文档。
8. 权限变化。
9. 索引更新状态。

知识库同步不是只导入文本, 还要同步权限和删除状态。

如果源文档删除了, RAG 索引也必须处理删除或失效。

删除传播率可以写成:

$$R_{\text{delete}} = \frac{|\{d : d \in D_{\text{deleted}} \wedge d \in D_{\text{tombstone}}\}|}{|D_{\text{deleted}}|}$$

D_{deleted} 是源系统删除或撤权的文档集合, $D_{\text{tombstone}}$ 是索引、chunk store 和 citation store 中已标记失效的集合。 $R_{\text{delete}} < 1$ 时, 旧文档可能继续被召回。

43.8 权限和 ACL

企业 RAG 最容易出事故的是权限。

ACL 是 access control list, 访问控制列表。

文档、chunk 和检索结果都要带权限信息。

检索时必须根据用户身份过滤:

1. 用户所属租户。
2. 用户所在部门。

3. 用户角色。
4. 文档权限标签。
5. 文档安全等级。
6. 数据地域约束。

不能先检索再让模型 “不要回答无权限内容”。

权限过滤必须在检索阶段强制执行。

检索权限门禁可以写成：

$$G_{\text{acl}}(u, c) = I(\text{tenant}(u) = \text{tenant}(c)) \cdot I(\ell_c \subseteq L_u) \cdot I(r_c \leq r_u)$$

其中 u 是用户或 agent 运行身份, c 是候选 chunk, ℓ_c 是 chunk 权限标签, L_u 是用户可访问标签集合, r_c 和 r_u 分别是数据等级和用户授权等级。只有 $G_{\text{acl}}=1$ 的 chunk 才能进入候选、rerank、prompt 和日志。

43.9 Retrieval Trace

Retrieval trace 记录一次检索发生了什么。

至少包括：

1. query。
2. query rewrite 结果。
3. embedding model version。
4. index version。
5. filters。
6. top-k 参数。
7. candidate chunks。
8. similarity scores。
9. rerank scores。
10. 最终进入 prompt 的 chunks。
11. 被过滤掉的原因。
12. latency。

当用户问 “为什么回答引用了这个文档”，retrieval trace 是排查入口。

Retrieval trace 完整率：

$$C_{\text{retr_trace}} = \frac{\sum_i I(q_i, e_i, v_i, f_i, k_i, s_i, r_i, l_i \text{ present})}{N}$$

其中 q_i 是 query / rewrite, e_i 是 query embedding 版本, v_i 是 index version, f_i 是 filter, k_i 是 top-k, s_i 是 candidate scores, r_i 是 rerank 结果, l_i 是 latency。缺任一关键字段，线上 bad case 就很难复盘。

43.10 Prompt Assembly Trace

RAG 不是检索完就结束，还要组装 prompt。

Prompt assembly trace 记录：

1. system prompt version。
2. user query。
3. retrieved chunks。
4. chunk 排序。
5. token budget。
6. 被截断的内容。
7. 引用格式。

8. 最终 prompt hash。

很多 RAG 问题不是检索错了，而是 prompt 组装时把关键 chunk 截掉了。

Prompt assembly 可以用稳定 hash 绑定：

$$h_{\text{prompt}} = H(t_{\text{sys}}, t_{\text{user}}, O_{\text{chunk}}, B, \Gamma)$$

其中 t_{sys} 是 system prompt version, t_{user} 是用户输入, O_{chunk} 是 chunk 排序, B 是 token budget, Γ 是截断和引用格式策略。只记录最终 prompt 文本容易泄露隐私，记录版本、排序、预算和 hash 更适合审计。

43.11 Citation 存储

Citation 是回答引用的来源。

Citation 应记录：

1. answer span。
2. chunk ID。
3. document ID。
4. source URI。
5. offset。
6. score。
7. index version。
8. document version。

好的 citation 不只是展示链接，还要能追溯到当时的文档版本。

否则文档更新后，用户看到的引用可能和生成时不一致。

Citation 版本绑定率：

$$C_{\text{citation}} = \frac{\sum_j I(\text{span}_j, \text{chunk_id}_j, \text{doc_id}_j, \text{doc_version}_j, o_j, v_{\text{index},j} \text{ present})}{|\{\text{citation}_j\}|}$$

这个指标衡量引用是否能从答案 span 回到当时的 chunk、文档版本、offset 和索引版本。

43.12 Agent 平台中的核心资产

Agent 平台常见资产包括：

1. Agent Definition。
2. Prompt Template。
3. Tool Definition。
4. Tool Permission。
5. Memory。
6. Plan。
7. Tool Call Trace。
8. Execution Trace。
9. Run State。
10. Evaluation Result。

Agent 平台比 RAG 更复杂，因为它有多步决策、工具调用、状态变化和副作用。

43.13 Agent Definition

Agent Definition 描述一个 agent 的配置。

包括：

1. agent 名称。
2. 版本。
3. system prompt。
4. 可用工具列表。
5. 模型配置。
6. 记忆策略。
7. 最大步骤数。
8. 超时设置。
9. 权限策略。
10. 安全策略。

Agent Definition 必须版本化。

否则同一个 agent 今天和明天行为不同，trace 无法复现。

Agent definition 可以写成：

$$A_j = (\text{agent_id}, v_j, p_j, T_j, m_j, b_j, \tau_j, \pi_j, \gamma_j)$$

其中 v_j 是 agent version, p_j 是 system prompt version, T_j 是可用工具及版本集合, m_j 是模型配置, b_j 是预算和最大步数, τ_j 是超时策略, π_j 是权限策略, γ_j 是安全策略。

43.14 Tool Definition

Tool Definition 描述工具能力。

包括：

1. tool name。
2. version。
3. description。
4. input schema。
5. output schema。
6. endpoint。
7. timeout。
8. retry policy。
9. permission requirement。
10. side effect level。

工具描述会影响模型是否调用工具以及如何填参数。

所以工具定义变化也要版本化、灰度和回滚。

工具定义契约完整率：

$$C_{\text{tool}} = \frac{\sum_j I(n_j, v_j, d_j, x_j, y_j, \pi_j, \eta_j, \rho_j \text{ present})}{|\{T_j\}|}$$

其中 n_j 是工具名, v_j 是版本, d_j 是描述, x_j 是输入 schema, y_j 是输出 schema, π_j 是权限要求, η_j 是副作用等级, ρ_j 是 retry / timeout 策略。

43.15 Tool Permission

工具权限比普通 API 权限更复杂。

因为调用者可能是 agent，不是人直接点击按钮。

平台需要判断：

1. 用户是否有权限使用该工具。
2. agent 是否被授权调用该工具。
3. 工具是否会产生副作用。
4. 是否需要用户确认。
5. 是否需要审批。
6. 是否允许在当前租户和地域调用。

例如查询工具和删除工具的风险完全不同。

高风险工具调用必须记录审计，必要时需要 human-in-the-loop。

工具权限门禁：

$$G_{\text{tool}} = I(p_{\text{user}} \supseteq p_{\text{tool}}) \cdot I(p_{\text{agent}} \supseteq p_{\text{tool}}) \cdot I(\text{tenant ok}) \cdot I(\text{confirm ok} \vee \eta \leq \eta_{\text{safe}})$$

p_{user} 是用户权限， p_{agent} 是 agent 被授权权限， p_{tool} 是工具要求权限， η 是工具副作用等级。查询类工具和删除、转账、发信这类高风险工具必须有不同门禁。

43.16 Tool Call Trace

Tool call trace 记录每一次工具调用。

包括：

1. run ID。
2. step ID。
3. tool name。
4. tool version。
5. input arguments。
6. output result。
7. start time。
8. end time。
9. latency。
10. status。
11. error message。
12. permission decision。
13. retry count。

工具输入和输出可能包含敏感信息，要做脱敏和权限控制。

Tool call trace 完整率：

$$C_{\text{tool_trace}} = \frac{\sum_i I(r_i, s_i, n_i, v_i, x_i, y_i, t_i, \sigma_i, \pi_i \text{ present})}{N_{\text{tool}}}$$

其中 r_i 是 run ID， s_i 是 step ID， n_i 是 tool name， v_i 是 tool version， x_i 是输入参数， y_i 是输出引用或结果摘要， t_i 是耗时， σ_i 是状态， π_i 是权限决策。

43.17 Execution Trace

Execution trace 是 Agent 一次完整运行的记录。

它可以包含：

1. user input。
2. agent version。
3. model version。
4. prompt version。
5. planning steps。
6. tool calls。
7. observations。
8. intermediate messages。
9. final answer。
10. token usage。
11. cost。
12. errors。
13. safety events。

Execution trace 是 Agent 调试、评估、审计和回放的基础。

Execution trace 可以看成 span 图：

$$G_{\text{trace}} = (V_{\text{span}}, E_{\text{parent}}, R_{\text{artifact}})$$

V_{span} 是检索、LLM、工具、memory、rerank、安全检查等 span, E_{parent} 是父子关系, R_{artifact} 是 prompt、tool output、citation、eval result 等 artifact 引用。trace 不是一串日志，而是一棵可回放的证据树。

43.18 Trace 回放

Trace 回放用于复现一次 RAG/Agent 行为。

回放需要固定：

1. 模型版本。
2. prompt 版本。
3. agent definition。
4. tool definition。
5. tool outputs。
6. knowledge base version。
7. index version。
8. retrieval results。
9. sampling 参数。

如果外部工具结果已经变化，回放需要使用当时记录的 tool output，或者标记为不可完全复现。

回放就绪门禁：

$$G_{\text{replay}} = I(m, p, a, T, o, k, v, \theta \text{ fixed})$$

其中 m 是 model version, p 是 prompt version, a 是 agent definition, T 是 tool definition, o 是 tool output 或外部观察, k 是 knowledge base version, v 是 index version, θ 是 sampling 参数。任一关键项不可固定，都只能做近似回放。

43.19 Memory 存储

Agent 常有 memory。

Memory 可以分为：

1. 会话记忆。
2. 用户长期记忆。
3. 任务状态。
4. 工具结果缓存。
5. 偏好信息。

Memory 存储要考虑：

1. 生命周期。
2. 用户可见性。
3. 删除机制。
4. 权限。
5. 隐私。
6. 是否允许进入 prompt。
7. 是否允许用于训练。

记忆不是随便保存聊天记录。它是敏感数据资产。

Memory 写入门禁：

$$G_{\text{mem}} = I(\text{purpose ok}) \cdot I(\text{scope ok}) \cdot I(\text{sensitive} = 0 \vee \text{explicit approval}) \cdot I(\text{ttl set}) \cdot I(\text{delete supported})$$

记忆命中率不能单独作为成功指标。更重要的是写入是否有目的、作用域、权限、TTL、删除机制和用户可见性。

43.20 Run State 存储

长任务 Agent 需要保存 run state。

例如：

1. 当前步骤。
2. 已完成任务。
3. 待执行工具。
4. 中间结果。
5. 用户确认状态。
6. 超时状态。
7. cancellation 状态。

Run state 支持：

1. 任务恢复。
2. 暂停和继续。
3. 人工接管。
4. 失败诊断。
5. 审计。

长任务 Agent 不能只依赖内存变量。

43.21 安全与审计

RAG/Agent 平台要记录安全事件：

1. 无权限文档过滤。
2. 高风险工具调用。

3. 用户确认。
4. 安全策略拦截。
5. 敏感信息脱敏。
6. 越权访问尝试。
7. 工具调用失败。
8. 异常高成本运行。

审计日志要能回答：谁在什么时间，通过哪个 agent，用了哪个模型，检索了哪些知识，调用了哪些工具，产生了什么结果。

43.22 成本追踪

RAG/Agent 的成本来源包括：

1. embedding 生成。
2. 向量检索。
3. rerank。
4. LLM tokens。
5. tool API 调用。
6. 长任务多轮循环。
7. trace 存储。

成本追踪要按：

1. tenant。
2. application。
3. agent。
4. user。
5. run。
6. model。
7. tool。

进行归因。

Agent 特别容易成本失控，因为它可能循环调用模型和工具。

一次 run 的成本可以拆成：

$$K_{\text{run}} = K_{\text{llm}} + K_{\text{retr}} + K_{\text{rerank}} + K_{\text{tool}} + K_{\text{embed}} + K_{\text{trace}}$$

成本归因完整率：

$$C_{\text{cost}} = \frac{\sum_i I(\text{tenant}_i, \text{app}_i, \text{agent}_i, \text{run}_i, \text{component}_i, K_i \text{ present})}{N_{\text{cost}}}$$

如果成本只按总账记录，Agent 循环、工具 API、trace 存储和重排成本都无法定位。

43.23 Trace 存储的隐私问题

Trace 里可能包含大量敏感信息：

1. 用户输入。
2. 检索文档片段。
3. 工具参数。
4. 工具返回结果。
5. 模型中间推理内容。
6. 最终输出。

因此 trace 存储要支持：

1. 字段级脱敏。
2. 访问控制。
3. TTL。
4. 加密。
5. 审计。
6. 用户删除请求。
7. 采样存储。

不是所有 trace 都应该永久保存完整内容。

Trace 脱敏覆盖率：

$$C_{\text{redact}} = \frac{N_{\text{redacted fields}}}{N_{\text{sensitive fields}}}$$

Trace 保留门禁：

$$G_{\text{retention}} = I(C_{\text{redact}} \geq \alpha) \cdot I(\text{ttl} \leq \tau_{\text{max}}) \cdot I(\text{access scoped}) \cdot I(\text{delete supported})$$

其中 α 是脱敏阈值， τ_{max} 是最大保留时长。trace 里如果保存工具参数和检索片段，就必须按敏感数据资产治理。

43.24 Trace 采样

完整保存所有 trace 成本高，也有隐私风险。

可以按策略采样：

1. 错误请求全量保存。
2. 高价值业务全量保存。
3. 普通成功请求采样保存。
4. 敏感请求只保存摘要。
5. 高成本 run 保存完整 trace。
6. 用户反馈差的请求保存完整 trace。

采样策略要兼顾排障、评估、成本和隐私。

分层采样率可以写成：

$$p_{\text{store}}(r) = \begin{cases} 1, & r \in R_{\text{error}} \cup R_{\text{high value}} \cup R_{\text{feedback bad}} \\ p_s, & r \in R_{\text{success}} \\ p_{\text{summary}}, & r \in R_{\text{sensitive}} \end{cases}$$

采样不是随便丢日志，而是用错误、价值、反馈、敏感等级和成本共同决定保存粒度。

43.25 RAG/Agent 平台存储架构

一个可能的架构：

Knowledge Base Store

-> Document Store

-> Chunk Store

-> Embedding Store

-> Vector Index

Agent Registry

- > Prompt Registry
- > Tool Registry
- > Permission Store
- > Memory Store

Trace Platform

- > Retrieval Trace Store
- > Tool Call Trace Store
- > Execution Trace Store
- > Audit Log
- > Cost Store

不同存储承担不同职责，不要把所有东西塞进一个向量数据库。

43.26 RAG/Agent 平台存储审计指标和最小 demo

把本章落到平台审计时，可以用 16 个门禁：

1. Knowledge Base Contract: 知识库是否有租户、owner、来源、权限、embedding、chunking、索引、同步和保留策略。
2. Document Chunk Version Contract: document、chunk、offset、checksum、document version 和 embedding ID 是否完整。
3. Sync Delete Propagation: 源系统删除、撤权和权限变化是否传播到 chunk、index 和 citation。
4. ACL Permission Enforcement: 检索、rerank、prompt 和 trace 是否都强制 tenant / ACL filter。
5. Retrieval Trace Completeness: query、rewrite、embedding version、index version、filter、candidate、rerank、final chunk 和 latency 是否完整。
6. Prompt Assembly Trace: system prompt、template、chunk 顺序、token budget、截断和 prompt hash 是否可复现。
7. Citation Version Binding: answer span 是否绑定 chunk、document version、offset、source URI 和 index version。
8. Agent Definition Versioning: agent definition、prompt、model、工具集合、预算、权限和安全策略是否版本化。
9. Tool Definition Contract: tool name、version、description、input/output schema、endpoint、timeout、权限和副作用等级是否完整。
10. Tool Permission Gate: 用户权限、agent 权限、工具权限、副作用等级和确认策略是否一起判断。
11. Tool Call Trace Completeness: 每次工具调用是否记录 run、step、tool version、参数、结果引用、耗时、状态、错误、权限决策和重试。
12. Execution Trace Replay Readiness: span 树、artifact 引用、模型 / prompt / tool / KB / index / sampling 参数是否足够回放。
13. Memory Privacy Lifecycle: memory 写入是否有 purpose、scope、TTL、可见性、删除机制和敏感信息阻断。
14. Trace Privacy Retention: trace 是否支持字段级脱敏、TTL、加密、访问控制、删除请求和采样。
15. Cost Attribution Governance: LLM、retrieval、rerank、tool、embedding 和 trace 成本是否能按 tenant / app / agent / run / component 归因。
16. RAG Agent Storage Gate: 以上证据是否有 owner、门禁、回滚和 P0 风险阻断。

综合门禁可以写成：

$$G_{\text{rag_agent_storage}} = \prod_{j=1}^{16} G_j$$

其中每个 G_j 是上面一个子门禁。面试里不要只说“我们把 trace 存起来”，而要说明 trace 是否能解释、回放、评估、审计、脱敏、删除和归因。

下面是一个 0 依赖 demo。它不是生产实现，而是展示如何把 RAG/Agent 平台存储设计变成可检查的结构化审计。

```
from copy import deepcopy
```

```
class MiniRAGAgentStorageAudit:
    GATES = [
        "knowledge_base_contract",
        "document_chunk_version_contract",
        "sync_delete_propagation",
        "acl_permission_enforcement",
        "retrieval_trace_completeness",
        "prompt_assembly_trace",
        "citation_version_binding",
        "agent_definition_versioning",
        "tool_definition_contract",
        "tool_permission_gate",
        "tool_call_trace_completeness",
        "execution_trace_replay_readiness",
        "memory_privacy_lifecycle",
        "trace_privacy_retention",
        "cost_attribution_governance",
        "rag_agent_storage_gate",
    ]

    KB_FIELDS = [
        "kb_id",
        "tenant",
        "owner",
        "source_connectors",
        "permission_policy",
        "embedding_version",
        "chunking_version",
        "index_version",
        "sync_state",
        "retention_policy",
    ]

    DOC_FIELDS = [
        "doc_id",
        "kb_id",
        "version",
        "source_uri",
        "content_hash",
        "permission_labels",
        "parsing_status",
    ]

    CHUNK_FIELDS = [
        "chunk_id",
        "doc_id",
        "doc_version",
        "offset_start",
        "offset_end",
        "embedding_id",
        "permission_labels",
        "checksum",
    ]
```

```

        "tenant",
    ]
    RETRIEVAL_FIELDS = [
        "query_id",
        "query",
        "query_embedding_version",
        "index_version",
        "filters",
        "top_k",
        "candidate_chunks",
        "rerank_scores",
        "final_chunks",
        "latency_ms",
    ]
    PROMPT_FIELDS = [
        "system_prompt_version",
        "template_version",
        "chunk_order",
        "token_budget",
        "prompt_hash",
        "context_tokens",
    ]
    CITATION_FIELDS = [
        "answer_span",
        "chunk_id",
        "doc_id",
        "doc_version",
        "source_uri",
        "offset_start",
        "offset_end",
        "index_version",
    ]
    AGENT_FIELDS = [
        "agent_id",
        "version",
        "system_prompt_version",
        "tool_versions",
        "model_version",
        "max_steps",
        "timeout_s",
        "permission_policy",
        "memory_policy",
        "safety_policy",
    ]
    TOOL_FIELDS = [
        "name",
        "version",
        "description",
        "input_schema",
        "output_schema",
        "endpoint",
        "timeout_s",
        "retry_policy",
        "permission_requirement",
        "side_effect_level",
    ]

```

```

]
TOOL_TRACE_FIELDS = [
    "run_id",
    "step_id",
    "tool_name",
    "tool_version",
    "input_args",
    "output_ref",
    "status",
    "latency_ms",
    "permission_decision",
    "retry_count",
    "redacted",
]
EXEC_TRACE_FIELDS = [
    "run_id",
    "agent_version",
    "model_version",
    "prompt_version",
    "span_ids",
    "parent_map",
    "retrieval_trace_ids",
    "tool_trace_ids",
    "final_answer_hash",
    "token_usage",
    "artifact_refs",
]
COST_FIELDS = ["tenant", "application", "agent", "run_id", "component", "usr"]

@staticmethod
def present(record, key):
    return key in record and record[key] is not None and record[key] != ""

def coverage(self, record, fields):
    if not record:
        return 0.0
    return sum(1 for field in fields if self.present(record, field)) / len(fields)

def chunk_map(self, case):
    return {chunk["chunk_id"]: chunk for chunk in case.get("chunks", [])}

def doc_map(self, case):
    return {doc["doc_id"]: doc for doc in case.get("documents", [])}

def tool_map(self, case):
    return {tool["name"]: tool for tool in case.get("tools", [])}

def chunk_allowed(self, case, chunk_id):
    chunks = self.chunk_map(case)
    chunk = chunks.get(chunk_id)
    if not chunk:
        return False
    user = case.get("user", {})
    user_labels = set(user.get("labels", []))
    chunk_labels = set(chunk.get("permission_labels", []))

```

```

    return (
        chunk.get("tenant") == user.get("tenant")
        and chunk_labels.issubset(user_labels)
    )

def knowledge_base_contract(self, case):
    kb = case.get("knowledge_base", {})
    return self.coverage(kb, self.KB_FIELDS) == 1.0

def document_chunk_version_contract(self, case):
    documents = case.get("documents", [])
    chunks = case.get("chunks", [])
    docs_ok = documents and all(
        self.coverage(document, self.DOC_FIELDS) == 1.0
        for document in documents
    )
    chunks_ok = chunks and all(
        self.coverage(chunk, self.CHUNK_FIELDS) == 1.0
        and chunk["offset_end"] > chunk["offset_start"]
        for chunk in chunks
    )
    return bool(docs_ok and chunks_ok)

def sync_delete_propagation(self, case):
    sync = case.get("sync_job", {})
    required = ["sync_id", "source_connector", "deleted", "tombstone_applied", "permission_changes"]
    if self.coverage(sync, required) < 1.0:
        return False
    deleted = set(sync.get("deleted", []))
    tombstone = set(sync.get("tombstone_applied", []))
    return deleted.issubset(tombstone)

def acl_permission_enforcement(self, case):
    trace = case.get("retrieval_trace", {})
    for candidate in trace.get("candidate_chunks", []):
        allowed = self.chunk_allowed(case, candidate.get("chunk_id"))
        if not allowed and not candidate.get("filtered", False):
            return False
    return all(self.chunk_allowed(case, chunk_id) for chunk_id in trace.get("final_chunks", []))

def retrieval_trace_completeness(self, case):
    trace = case.get("retrieval_trace", {})
    if self.coverage(trace, self.RETRIEVAL_FIELDS) < 1.0:
        return False
    candidate_ids = {candidate.get("chunk_id") for candidate in trace["candidate_chunks"]}
    final_ids = set(trace["final_chunks"])
    candidate_scores_ok = all("score" in candidate for candidate in trace["candidate_chunks"])
    return final_ids.issubset(candidate_ids) and candidate_scores_ok

def prompt_assembly_trace(self, case):
    prompt = case.get("prompt_trace", {})
    trace = case.get("retrieval_trace", {})
    if self.coverage(prompt, self.PROMPT_FIELDS) < 1.0:
        return False
    return set(trace.get("final_chunks", [])).issubset(set(prompt.get("chunk_order", [])))

```

```

def citation_version_binding(self, case):
    citations = case.get("citations", [])
    chunks = self.chunk_map(case)
    if not citations:
        return False
    for citation in citations:
        if self.coverage(citation, self.CITATION_FIELDS) < 1.0:
            return False
        chunk = chunks.get(citation["chunk_id"])
        if not chunk:
            return False
        if chunk["doc_id"] != citation["doc_id"]:
            return False
        if chunk["doc_version"] != citation["doc_version"]:
            return False
    return True

def agent_definition_versioning(self, case):
    agent = case.get("agent_definition", {})
    if self.coverage(agent, self.AGENT_FIELDS) < 1.0:
        return False
    return bool(agent.get("tool_versions"))

def tool_definition_contract(self, case):
    tools = case.get("tools", [])
    if not tools:
        return False
    for tool in tools:
        if self.coverage(tool, self.TOOL_FIELDS) < 1.0:
            return False
        if not tool.get("input_schema") or not tool.get("output_schema"):
            return False
    return True

def tool_permission_gate(self, case):
    tools = self.tool_map(case)
    policy = case.get("tool_policy", {})
    user_permissions = set(policy.get("user_permissions", []))
    agent_permissions = set(policy.get("agent_permissions", []))
    confirmations = set(policy.get("confirmed_actions", []))
    high_risk = {"write", "destructive", "external"}
    for trace in case.get("tool_traces", []):
        tool = tools.get(trace.get("tool_name"))
        if not tool:
            return False
        required = tool["permission_requirement"]
        has_permission = required in user_permissions and required in agent_permissions
        needs_confirmation = tool["side_effect_level"] in high_risk
        confirmed = f"{trace.get('tool_name')}:{trace.get('step_id')}" in confirmations
        if trace.get("status") == "ok" and not has_permission:
            return False
        if trace.get("status") == "ok" and needs_confirmation and not confirmed:
            return False
    return True

```



```

def tool_call_trace_completeness(self, case):
    tools = self.tool_map(case)
    traces = case.get("tool_traces", [])
    if not traces:
        return False
    for trace in traces:
        if self.coverage(trace, self.TOOL_TRACE_FIELDS) < 1.0:
            return False
        tool = tools.get(trace["tool_name"])
        if not tool or tool["version"] != trace["tool_version"]:
            return False
        if trace.get("redacted") is not True:
            return False
    return True

def execution_trace_replay_readiness(self, case):
    trace = case.get("execution_trace", {})
    replay = case.get("replay", {})
    if self.coverage(trace, self.EXEC_TRACE_FIELDS) < 1.0:
        return False
    spans = set(trace.get("span_ids", []))
    parent_map = trace.get("parent_map", {})
    non_root_spans = spans - {"root"}
    parents_ok = all(span in parent_map and parent_map[span] in spans for span in non_root_spans)
    replay_ok = replay and all(replay.values())
    return parents_ok and replay_ok

def memory_privacy_lifecycle(self, case):
    memory = case.get("memory", {})
    if not memory.get("deletion_supported"):
        return False
    for write in memory.get("writes", []):
        if not write.get("ttl_days"):
            return False
        if not write.get("visible_to_user"):
            return False
        sensitive = write.get("sensitive", False)
        blocked = write.get("blocked", False)
        approved = write.get("explicit_approval", False)
        if sensitive and not blocked and not approved:
            return False
    return True

def trace_privacy_retention(self, case):
    privacy = case.get("trace_privacy", {})
    total = privacy.get("fields_total", 0)
    redacted = privacy.get("redacted_fields", 0)
    if total <= 0:
        return False
    redaction_coverage = redacted / total
    return (
        redaction_coverage >= 0.95
        and privacy.get("ttl_days", 9999) <= 90
        and bool(privacy.get("encryption"))
    )

```

```

        and bool(privacy.get("access_policy"))
        and privacy.get("raw_cot_stored") is False
    )

def cost_attribution_governance(self, case):
    costs = case.get("costs", [])
    if not costs:
        return False
    total = 0.0
    for cost in costs:
        if self.coverage(cost, self.COST_FIELDS) < 1.0:
            return False
        total += float(cost["usd"])
    budget = case.get("platform_gate", {}).get("cost_budget_usd", 9999.0)
    return total <= budget

def rag_agent_storage_gate(self, case):
    gate = case.get("platform_gate", {})
    return (
        gate.get("enabled") is True
        and bool(gate.get("owner"))
        and gate.get("p0_open") is False
        and gate.get("rollback_ready") is True
    )

def audit_case(self, case):
    return {gate: getattr(self, gate)(case) for gate in self.GATES}

def run_all(self, cases):
    results = {case["case_id"]: self.audit_case(case) for case in cases}
    metrics = {}
    for gate in self.GATES:
        passed = sum(1 for result in results.values() if result[gate])
        metrics[gate] = round(passed / len(cases), 3)
    failed_cases = [
        case_id
        for case_id, result in results.items()
        if not all(result.values())
    ]
    failed_gates = [
        gate
        for gate in self.GATES
        if any(not result[gate] for result in results.values())
    ]
    return {
        "metrics": metrics,
        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "failed_gates": failed_gates,
        "rag_agent_storage_gate_pass": metrics["rag_agent_storage_gate"] == 1.0,
    }

def example_outputs(self, case):
    trace = case["retrieval_trace"]
    prompt = case["prompt_trace"]

```

```

costs = case["costs"]
privacy = case["trace_privacy"]
return {
    "kb_contract_coverage": round(self.coverage(case["knowledge_base"], self.KB_FIELDS), 3),
    "document_version_coverage": round(
        sum(self.coverage(doc, self.DOC_FIELDS) for doc in case["documents"])
        / len(case["documents"]),
        3,
    ),
    "chunk_citation_coverage": round(
        sum(self.coverage(chunk, self.CHUNK_FIELDS) for chunk in case["chunks"])
        / len(case["chunks"]),
        3,
    ),
    "permission_filtered_chunks": [
        candidate["chunk_id"]
        for candidate in trace["candidate_chunks"]
        if candidate.get("filtered")
    ],
    "retrieval_trace_complete": self.retrieval_trace_completeness(case),
    "prompt_hash_ready": bool(prompt.get("prompt_hash")),
    "citation_version_bound": self.citation_version_binding(case),
    "tool_schema_coverage": round(
        sum(self.coverage(tool, self.TOOL_FIELDS) for tool in case["tools"])
        / len(case["tools"]),
        3,
    ),
    "tool_permission_allowed": case["tool_traces"][0]["status"] == "ok",
    "dangerous_tool_blocked": case["tool_traces"][1]["status"].startswith("blocked"),
    "tool_trace_complete": self.tool_call_trace_completeness(case),
    "trace_replay_ready": self.execution_trace_replay_readiness(case),
    "memory_private_blocked": case["memory"]["writes"][1]["blocked"],
    "trace_redaction_coverage": round(
        privacy["redacted_fields"] / privacy["fields_total"],
        3,
    ),
    "estimated_run_cost_usd": round(sum(cost["usd"] for cost in costs), 3),
    "audit_event_count": len(case["audit_logs"]),
}

```

```

def build_good_case():
    return {
        "case_id": "production_ready",
        "knowledge_base": {
            "kb_id": "kb_support",
            "tenant": "acme",
            "owner": "ai-infra",
            "source_connectors": ["wiki", "tickets"],
            "permission_policy": "tenant_acl_labels",
            "embedding_version": "emb_v4",
            "chunking_version": "chunk_v3",
            "index_version": "idx_7",
            "sync_state": "green",
            "retention_policy": "90d_trace_immutable_doc_versions",

```

```

},
"documents": [
  {
    "doc_id": "doc_policy",
    "kb_id": "kb_support",
    "version": "v3",
    "source_uri": "wiki://policy",
    "content_hash": "sha256:policy",
    "permission_labels": ["public"],
    "parsing_status": "parsed",
  },
  {
    "doc_id": "doc_salary",
    "kb_id": "kb_support",
    "version": "v5",
    "source_uri": "wiki://salary",
    "content_hash": "sha256:salary",
    "permission_labels": ["finance_private"],
    "parsing_status": "parsed",
  },
],
"chunks": [
  {
    "chunk_id": "chunk_policy_public",
    "doc_id": "doc_policy",
    "doc_version": "v3",
    "offset_start": 0,
    "offset_end": 120,
    "embedding_id": "emb_001",
    "permission_labels": ["public"],
    "checksum": "sha256:chunk-policy",
    "tenant": "acme",
  },
  {
    "chunk_id": "chunk_salary_private",
    "doc_id": "doc_salary",
    "doc_version": "v5",
    "offset_start": 20,
    "offset_end": 180,
    "embedding_id": "emb_002",
    "permission_labels": ["finance_private"],
    "checksum": "sha256:chunk-salary",
    "tenant": "acme",
  },
],
"sync_job": {
  "sync_id": "sync_2026_06_15",
  "source_connector": "wiki",
  "added": ["doc_policy"],
  "updated": [],
  "deleted": ["doc_old"],
  "tombstone_applied": ["doc_old"],
  "permission_changes": ["doc_salary"],
  "failed": [],
},

```

```

"user": {
  "tenant": "acme",
  "roles": ["employee"],
  "labels": ["public"],
},
"retrieval_trace": {
  "query_id": "query_1",
  "query": "What is the refund policy?",
  "query_embedding_version": "emb_v4",
  "index_version": "idx_7",
  "filters": {"tenant": "acme", "labels": ["public"]},
  "top_k": 3,
  "candidate_chunks": [
    {"chunk_id": "chunk_policy_public", "score": 0.91, "filtered": False},
    {
      "chunk_id": "chunk_salary_private",
      "score": 0.88,
      "filtered": True,
      "reason": "acl",
    },
  ],
  "rerank_scores": {"chunk_policy_public": 0.97},
  "final_chunks": ["chunk_policy_public"],
  "latency_ms": 42,
},
"prompt_trace": {
  "system_prompt_version": "sys_v5",
  "template_version": "rag_template_v2",
  "chunk_order": ["chunk_policy_public"],
  "token_budget": 4000,
  "truncated_chunks": [],
  "prompt_hash": "sha256:prompt",
  "context_tokens": 512,
},
"citations": [
  {
    "answer_span": "refunds are handled within 7 days",
    "chunk_id": "chunk_policy_public",
    "doc_id": "doc_policy",
    "doc_version": "v3",
    "source_uri": "wiki://policy",
    "offset_start": 0,
    "offset_end": 120,
    "index_version": "idx_7",
  }
],
"agent_definition": {
  "agent_id": "support_agent",
  "version": "agent_2026_06_01",
  "system_prompt_version": "sys_v5",
  "tool_versions": {"kb_search": "1.2", "ticket_delete": "1.0"},
  "model_version": "chat_prod_7",
  "max_steps": 6,
  "timeout_s": 60,
  "permission_policy": "least_privilege",
}

```

```

    "memory_policy": "scoped_user_memory",
    "safety_policy": "confirm_high_risk_tools",
},
"tools": [
    {
        "name": "kb_search",
        "version": "1.2",
        "description": "Search approved knowledge base chunks.",
        "input_schema": {"query": "string", "kb_id": "string"},
        "output_schema": {"chunks": "list"},
        "endpoint": "internal://kb_search",
        "timeout_s": 5,
        "retry_policy": "safe_read_retry",
        "permission_requirement": "kb.read",
        "side_effect_level": "read",
    },
    {
        "name": "ticket_delete",
        "version": "1.0",
        "description": "Delete a ticket after explicit approval.",
        "input_schema": {"ticket_id": "string"},
        "output_schema": {"deleted": "bool"},
        "endpoint": "internal://ticket_delete",
        "timeout_s": 10,
        "retry_policy": "no_auto_retry",
        "permission_requirement": "ticket.delete",
        "side_effect_level": "destructive",
    },
],
"tool_policy": {
    "user_permissions": ["kb.read", "ticket.read"],
    "agent_permissions": ["kb.read", "ticket.read"],
    "confirmed_actions": [],
    "tenant": "acme",
    "region": "us",
},
"tool_traces": [
    {
        "run_id": "run_1",
        "step_id": "s1",
        "tool_name": "kb_search",
        "tool_version": "1.2",
        "input_args": {"query": "refund policy", "kb_id": "kb_support"},
        "output_ref": "artifact://tool/run_1/s1",
        "status": "ok",
        "latency_ms": 32,
        "permission_decision": "allowed",
        "retry_count": 0,
        "redacted": True,
    },
    {
        "run_id": "run_1",
        "step_id": "s2",
        "tool_name": "ticket_delete",
        "tool_version": "1.0",
    },

```

```

        "input_args": {"ticket_id": "T-7"},
        "output_ref": "blocked:no_output",
        "status": "blocked_requires_permission",
        "latency_ms": 0,
        "permission_decision": "blocked_requires_permission",
        "retry_count": 0,
        "redacted": True,
    },
],
"execution_trace": {
    "run_id": "run_1",
    "agent_version": "agent_2026_06_01",
    "model_version": "chat_prod_7",
    "prompt_version": "sys_v5",
    "span_ids": ["root", "retrieval", "tool_s1", "final"],
    "parent_map": {"retrieval": "root", "tool_s1": "root", "final": "root"},
    "retrieval_trace_ids": ["query_1"],
    "tool_trace_ids": ["s1", "s2"],
    "final_answer_hash": "sha256:answer",
    "token_usage": {"input": 900, "output": 140},
    "errors": [],
    "safety_events": ["ticket_delete_blocked"],
    "artifact_refs": ["artifact://prompt/run_1", "artifact://tool/run_1/s1"],
},
"replay": {
    "model_version_fixed": True,
    "prompt_version_fixed": True,
    "agent_definition_fixed": True,
    "tool_definitions_fixed": True,
    "tool_outputs_recorded": True,
    "kb_version_fixed": True,
    "index_version_fixed": True,
    "sampling_params_fixed": True,
},
"memory": {
    "writes": [
        {
            "key": "preferred_language",
            "scope": "user",
            "value": "zh",
            "sensitive": False,
            "blocked": False,
            "ttl_days": 180,
            "visible_to_user": True,
            "training_allowed": False,
        },
        {
            "key": "private_card_number",
            "scope": "user",
            "value": "redacted",
            "sensitive": True,
            "blocked": True,
            "ttl_days": 1,
            "visible_to_user": True,
            "training_allowed": False,
        }
    ]
}

```

```

    },
  ],
  "reads": [{"key": "preferred_language", "authorized": True, "used_in_prompt": True}],
  "deletion_supported": True,
},
"trace_privacy": {
  "fields_total": 8,
  "redacted_fields": 8,
  "ttl_days": 30,
  "encryption": "kms",
  "access_policy": "trace.read.scoped",
  "sample_policy": "error_full_success_sampled",
  "deletion_request_ids": ["del_1"],
  "raw_cot_stored": False,
},
"costs": [
  {
    "tenant": "acme",
    "application": "support",
    "agent": "support_agent",
    "run_id": "run_1",
    "component": "llm",
    "usd": 0.15,
  },
  {
    "tenant": "acme",
    "application": "support",
    "agent": "support_agent",
    "run_id": "run_1",
    "component": "retrieval",
    "usd": 0.02,
  },
  {
    "tenant": "acme",
    "application": "support",
    "agent": "support_agent",
    "run_id": "run_1",
    "component": "tool",
    "usd": 0.047,
  },
],
"audit_logs": [
  {"event": "retrieval_allowed", "run_id": "run_1"},
  {"event": "acl_filtered_chunk", "chunk_id": "chunk_salary_private"},
  {"event": "tool_allowed", "tool": "kb_search"},
  {"event": "tool_blocked", "tool": "ticket_delete"},
  {"event": "trace_redacted", "run_id": "run_1"},
],
"platform_gate": {
  "enabled": True,
  "owner": "ai-infra",
  "p0_open": False,
  "rollback_ready": True,
  "cost_budget_usd": 1.0,
},

```



```
}
```

```
def build_bad_cases(good_case):
    cases = []

    case = deepcopy(good_case)
    case["case_id"] = "knowledge_base_contract_missing_bad"
    case["knowledge_base"].pop("owner")
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "document_chunk_version_missing_bad"
    case["chunks"][0].pop("checksum")
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "sync_delete_not_propagated_bad"
    case["sync_job"]["tombstone_applied"] = []
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "acl_filter_missing_bad"
    case["retrieval_trace"]["candidate_chunks"][1]["filtered"] = False
    case["retrieval_trace"]["final_chunks"] = ["chunk_policy_public", "chunk_salary_private"]
    case["prompt_trace"]["chunk_order"] = ["chunk_policy_public", "chunk_salary_private"]
    case["citations"].append(
        {
            "answer_span": "private salary rule",
            "chunk_id": "chunk_salary_private",
            "doc_id": "doc_salary",
            "doc_version": "v5",
            "source_uri": "wiki://salary",
            "offset_start": 20,
            "offset_end": 180,
            "index_version": "idx_7",
        }
    )
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "retrieval_trace_incomplete_bad"
    case["retrieval_trace"].pop("query_embedding_version")
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "prompt_trace_hash_missing_bad"
    case["prompt_trace"].pop("prompt_hash")
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "citation_version_missing_bad"
    case["citations"][0].pop("doc_version")
    cases.append(case)
```

```

case = deepcopy(good_case)
case["case_id"] = "agent_definition_unversioned_bad"
case["agent_definition"].pop("version")
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "tool_definition_schema_missing_bad"
case["tools"][0]["input_schema"] = {}
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "tool_permission_bypass_bad"
case["tool_traces"][1]["status"] = "ok"
case["tool_traces"][1]["permission_decision"] = "allowed"
case["tool_traces"][1]["output_ref"] = "artifact://tool/run_1/s2"
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "tool_call_trace_incomplete_bad"
case["tool_traces"][0].pop("permission_decision")
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "execution_replay_not_ready_bad"
case["replay"]["tool_outputs_recorded"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "memory_sensitive_write_bad"
case["memory"]["writes"][1]["blocked"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "trace_privacy_retention_bad"
case["trace_privacy"]["redacted_fields"] = 5
case["trace_privacy"]["ttl_days"] = 365
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "cost_attribution_missing_bad"
case["costs"][0].pop("tenant")
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "rag_agent_storage_gate_missing_bad"
case["platform_gate"]["enabled"] = False
cases.append(case)

return cases

```

```

audit = MiniRAGAgentStorageAudit()
good = build_good_case()
cases = [good] + build_bad_cases(good)
summary = audit.run_all(cases)

```

```

print("rag_agent_storage_examples=" + repr(audit.example_outputs(good)))
print("metrics=" + repr(summary["metrics"]))
print("hard_blocker_count=" + repr(summary["hard_blocker_count"]))
print("failed_cases=" + repr(summary["failed_cases"]))
print("failed_gates=" + repr(summary["failed_gates"]))
print("rag_agent_storage_gate_pass=" + repr(summary["rag_agent_storage_gate_pass"]))

```

参考输出应类似：

```

rag_agent_storage_examples={'kb_contract_coverage': 1.0, 'document_version_coverage': 1.0, 'chunk_citation_coverage': 1.0,
metrics={'knowledge_base_contract': 0.941, 'document_chunk_version_contract': 0.941, 'sync_delete_propagation': 0.941,
hard_blocker_count=16
failed_cases=['knowledge_base_contract_missing_bad', 'document_chunk_version_missing_bad', 'sync_delete_not_propagated',
failed_gates=['knowledge_base_contract', 'document_chunk_version_contract', 'sync_delete_propagation', 'acl_permission',
rag_agent_storage_gate_pass=False

```

这个 demo 的核心不是把所有存储组件写成一个类，而是展示平台验收方式：每个 bad case 都应该触发一个明确门禁，最后能回答“为什么这次回答、工具调用或 Agent run 能被解释、复现、审计和删除”。

43.27 常见误区

误区一：RAG 平台就是向量库。

向量库只是检索组件，RAG 还需要知识库、文档版本、chunk、权限、citation、prompt assembly 和 trace。

误区二：Agent trace 只用于 debug。

Trace 还用于评估、审计、成本归因、回放、质量改进和安全分析。

误区三：工具定义不需要版本化。

工具描述、schema、权限和 endpoint 变化都会影响 agent 行为。

误区四：Memory 越多越好。

Memory 会带来隐私、污染、成本和错误引用风险，必须有生命周期和权限治理。

误区五：权限可以交给模型判断。

权限必须由平台在检索和工具调用阶段强制执行，不能依赖模型自觉。

43.28 面试常见追问

问题一：企业 RAG 平台要存哪些东西？

可以回答：要存 knowledge base、document、chunk、embedding、index version、权限元数据、retrieval config、rerank config、prompt template、citation 和 retrieval trace。

问题二：Agent 平台为什么需要 trace？

可以回答：Agent 是多步执行系统，trace 记录模型决策、工具调用、中间结果、错误、安全事件和成本，用于调试、回放、审计和评估。

问题三：如何避免 RAG 越权检索？

可以回答：文档和 chunk 存权限标签，检索时根据用户身份做强制 metadata filter 或物理索引隔离，不能把无权限内容交给模型再要求它不回答。

问题四：工具调用如何做权限控制？

可以回答：同时检查用户权限、agent 权限、工具风险等级、租户策略和是否需要用户确认，高风险工具调用要审计并支持 human-in-the-loop。

问题五：如何设计 RAG / Agent 平台存储审计？

可以回答：先把对象拆成 knowledge base、document、chunk、citation、agent definition、tool definition、memory、run state、retrieval trace、tool call trace、execution trace、audit log 和 cost record；再按版本、权限、回放、隐私、删除、采样、成本和 owner 做门禁；最后用 trace span tree 串起检索、prompt、工具、memory、安全事件和最终回答，证明线上 bad case 可解释、可复现、可回滚。

43.29 小练习

1. Knowledge Base、Document 和 Chunk 有什么区别？
2. Retrieval trace 应该记录哪些字段？
3. 为什么 citation 要绑定 document version 和 chunk ID？
4. Agent Definition 应该包含哪些内容？
5. Tool Definition 为什么要版本化？
6. Agent memory 存储有哪些隐私风险？
7. Trace 采样策略如何设计？
8. 如何设计一个支持回放的 Agent trace 系统？
9. 用纯 Python 实现一个最小 RAG / Agent 平台存储审计 demo，覆盖知识库契约、ACL、retrieval trace、tool permission、execution replay、memory 隐私和成本归因。
10. 构造 5 个线上事故样本：越权 chunk 进入 prompt、tool schema 变更未版本化、trace 缺 tool output、memory 写入敏感字段、源文档删除未传播，并说明各自应触发的门禁。

43.30 本章小结

本章讲了 RAG/Agent 平台中的知识库、工具和 trace 存储。

你需要记住：

1. RAG 平台不只是向量库，还包括知识库、文档、chunk、embedding、索引、权限、citation 和 retrieval trace。
2. Agent 平台要管理 agent definition、tool definition、permission、memory、run state 和 execution trace。
3. 权限必须在检索和工具调用阶段由平台强制执行，不能依赖模型判断。
4. Trace 是调试、回放、评估、审计和成本治理的核心数据。
5. RAG/Agent 存储系统必须同时考虑版本、权限、隐私、成本、生命周期和可观测性。
6. 第二轮精修后，本章新增的核心抓手是 RAG / Agent Storage Gate：让每次回答、检索、工具调用、memory 读写和成本记录都有版本、权限、trace、回放、隐私和审计证据。

下一章我们会进入第六部分：可观测性、可靠性与成本治理，先讲 AI Infra 可观测性总览。

第 44 章 AI Infra 可观测性总览：metrics、logs、traces、events

前面第五部分讲完了数据、模型与实验平台。从本章开始进入第六部分：可观测性、可靠性与成本治理。

AI Infra 的系统链路很长：数据、训练、评估、推理、RAG、Agent、工具、GPU 集群、存储、网络和调度都可能出问题。没有可观测性，平台只能靠猜。

先记住一句话：

AI Infra 可观测性的目标，不是收集更多指标，而是让训练、推理、数据、模型和平台问题能够被快速发现、定位、解释和复盘。

44.0 本讲资料边界与第二轮精修口径

本章按通用可观测性平台抽象来写，不绑定 Prometheus、Grafana、OpenTelemetry Collector、Datadog、Loki、Jaeger、Tempo、云厂商监控或内部平台实现。资料校准时，主要参考 OpenTelemetry 对 traces、metrics、logs、baggage、events / profiles 的信号划分和 metric instrument / aggregation / attribute 口径，参考 Prometheus 对 time series、metric name、labels 和 label 变化生成新时间序列的模型，参考 Google SRE

Book 对 SLI、SLO、SLA、error budget 和可行动监控的定义，也结合 AI Infra 前文中的训练、推理、RAG、Agent、评估、成本和安全治理场景。

第二轮精修只做三件事：

1. 把 metrics、logs、traces、events 和 cost signal 变成可审计对象，而不是概念解释。
2. 补齐 SLO、error budget、p95 / p99、标签基数、trace 覆盖、日志脱敏、告警噪音和根因定位公式。
3. 增加一个 0 依赖 Python demo，用 toy telemetry cases 检查 AI Infra 可观测性平台是否真的可排障、可告警、可复盘。

44.1 为什么 AI Infra 更需要可观测性

普通后端服务主要关注请求、延迟、错误率和资源使用。

AI Infra 还要额外关注：

1. GPU 利用率。
2. 显存和 KV cache。
3. 训练 loss。
4. gradient norm。
5. checkpoint。
6. 数据读取吞吐。
7. NCCL 通信。
8. 推理 TTFT 和 TPOT。
9. token 吞吐。
10. 模型版本。
11. prompt 版本。
12. eval 分数。
13. RAG 检索 trace。
14. tool call trace。
15. 成本。

AI 系统的问题常常跨越模型、数据、系统和业务多个层面。

一次 AI Infra 运行的可观测对象可以抽象成：

$$O_i = (m_i, l_i, t_i, e_i, s_i, c_i, a_i, d_i)$$

其中 m_i 是 metrics, l_i 是 logs, t_i 是 traces, e_i 是 events, s_i 是 SLO / error budget, c_i 是 cost signal, a_i 是 alert / dashboard, d_i 是 data / model / artifact 版本上下文。

整体覆盖率：

$$C_{\text{obs}} = \frac{1}{N} \sum_{i=1}^N I(m_i, l_i, t_i, e_i, s_i, c_i, a_i, d_i \text{ present})$$

这说明可观测性不是“有监控页面”，而是每类关键任务都要有能互相关联的信号。

44.2 可观测性的四类信号

本章重点讲四类信号：

1. Metrics。
2. Logs。
3. Traces。
4. Events。

Metrics 是数值指标，适合监控趋势和告警。

Logs 是文本或结构化日志，适合排查细节。

Traces 是一次请求或任务的链路，适合定位阶段耗时和调用关系。

Events 是状态变化和关键事件，适合理解系统发生了什么。

四者结合，才能形成完整可观测性。

四类信号的关联可以写成：

$$R_i = (\text{trace_id}_i, \text{request_id}_i, \text{tenant}_i, \text{model}_i, \text{run}_i, \text{time}_i)$$

如果 metrics、logs、traces 和 events 里没有共享的 R_i ，事故复盘就只能靠人工猜测时间线。

44.3 Metrics 是什么

Metrics 是时间序列指标。

例如：

1. GPU utilization。
2. training loss。
3. QPS。
4. TTFT p95。
5. checkpoint save latency。
6. data loader throughput。
7. queue length。
8. error rate。
9. cost per hour。

Metrics 适合回答：

1. 系统现在是否健康？
2. 指标是否超过阈值？
3. 最近是否有趋势变化？
4. 哪个租户或模型消耗最多？

一个时间序列可以写成：

$$x_{m,L}(t)$$

其中 m 是 metric name， L 是 label 集合， t 是时间。窗口均值：

$$\bar{x}_{[a,b]} = \frac{1}{b-a} \int_a^b x(t) dt$$

p 分位延迟可以写成：

$$Q_p = \inf\{q : F(q) \geq p\}$$

AI 推理里平均延迟不够，通常要同时看 $Q_{0.95}$ 、 $Q_{0.99}$ 、TTFT 和 TPOT。

44.4 Logs 是什么

Logs 记录具体发生了什么。

例如：

1. 训练脚本输出。
2. runtime 错误堆栈。
3. 数据加载失败。
4. checkpoint 保存失败。
5. 模型加载日志。
6. 工具调用错误。
7. 权限拒绝原因。
8. 路由决策日志。

Logs 适合回答：

1. 具体错误是什么？
2. 哪个文件失败了？
3. 哪个参数不合法？
4. 哪个节点报错？

日志最好结构化，否则很难检索和聚合。

结构化日志覆盖率：

$$C_{\log} = \frac{\sum_i I(\text{ts}_i, \text{level}_i, \text{service}_i, \text{trace_id}_i, \text{event}_i, \text{status}_i \text{ present})}{N_{\log}}$$

日志脱敏覆盖率：

$$C_{\text{redact}} = \frac{N_{\text{redacted fields}}}{N_{\text{sensitive fields}}}$$

AI 系统里，prompt、检索片段、tool 参数和输出经常含敏感信息，不能只追求“日志越全越好”。

44.5 Traces 是什么

Trace 记录一次请求或任务经过哪些阶段，每个阶段耗时多少，调用了哪些组件。

推理请求 trace 可能包括：

gateway -> auth -> router -> cache -> queue -> prefill -> decode -> safety -> response

训练任务 trace 可能包括：

submit -> schedule -> image pull -> data load -> train step -> checkpoint -> eval -> finish

Trace 适合回答：

1. 慢在哪里？
2. 哪个组件失败？
3. 调用链路是什么？
4. 哪个模型或工具参与了这次请求？

端到端推理延迟可以拆成：

$$T_{\text{e2e}} = T_{\text{gateway}} + T_{\text{auth}} + T_{\text{router}} + T_{\text{queue}} + T_{\text{prefill}} + T_{\text{decode}} + T_{\text{safety}} + T_{\text{stream}}$$

Trace span 覆盖率：

$$C_{\text{span}} = \frac{|S_{\text{observed}} \cap S_{\text{expected}}|}{|S_{\text{expected}}|}$$

如果没有 queue、prefill、decode 这些 span，TTFT / TPOT 的退化就只能靠猜。

44.6 Events 是什么

Events 是离散事件，通常表示状态变化或关键动作。

例如：

1. TrainingJob created。
2. TrainingJob scheduled。
3. Pod OOMKilled。
4. Checkpoint saved。
5. ModelVersion promoted。
6. Deployment rolled back。
7. Autoscaler scaled out。
8. Router degraded request。
9. Safety policy blocked output。
10. Dataset quality gate failed。

Events 能帮助复盘时间线。

当系统出问题时，你需要知道指标变化前后发生了哪些事件。

事件时间线完整率：

$$C_{\text{event}} = \frac{\sum_i I(\text{event_type}_i, \text{entity}_i, \text{version}_i, \text{actor}_i, \text{ts}_i \text{ present})}{N_{\text{event}}}$$

事件要能和指标突变对齐，比如发布、回滚、扩容、降级、质量门禁失败和数据版本切换。

44.7 四类信号如何协作

假设推理 p99 突然升高。

Metrics 告诉你：p99 高了，queue length 也高了。

Logs 告诉你：某些 runtime 出现 KV cache allocation failed。

Traces 告诉你：请求主要慢在 queue 和 prefill。

Events 告诉你：10 分钟前某个大租户开始灰度新功能，流量突增。

单独看任何一种信号都不够。

可观测性的价值在于把它们关联起来。

信号关联率：

$$C_{\text{corr}} = \frac{N_{\text{records with trace_or run_id}}}{N_{\text{records}}}$$

定位根因时，通常先用 metrics 发现异常，再用 events 找时间点，用 traces 定位阶段，用 logs 看细节，用版本和成本信号解释影响面。

44.8 训练可观测性

训练场景需要观察：

训练指标：

1. train loss。
2. eval loss。
3. learning rate。
4. gradient norm。
5. step time。
6. tokens/s。
7. samples/s。

系统指标：

1. GPU utilization。
2. GPU memory。
3. CPU utilization。
4. host memory。
5. disk I/O。
6. network bandwidth。
7. NCCL communication time。

平台指标：

1. queue time。
2. scheduling time。
3. checkpoint save time。
4. data load throughput。
5. retry count。
6. node failure count。

训练可观测性既要看模型收敛，也要看系统效率。

训练效率可以用 token 吞吐和 GPU 利用率联合衡量：

$$E_{\text{train}} = \frac{\text{tokens/s}}{N_{\text{gpu}} \cdot U_{\text{gpu}}}$$

其中 U_{gpu} 是平均 GPU 利用率。 E_{train} 下降但 loss 正常，通常是系统瓶颈；loss 异常但系统指标正常，通常要回到数据、配置或优化器排查。

44.9 推理可观测性

推理场景需要观察：

请求指标：

1. QPS。
2. TTFT。
3. TPOT。
4. p95 / p99 latency。
5. error rate。
6. timeout rate。

token 指标：

1. input tokens/s。
2. output tokens/s。
3. average input tokens。

4. average output tokens。

runtime 指标:

1. queue length。
2. active sequences。
3. batch size。
4. KV cache usage。
5. GPU utilization。
6. GPU memory。

平台指标:

1. route decision。
2. cache hit rate。
3. fallback rate。
4. degradation rate。
5. admission reject rate。

推理可观测性要能把端到端延迟拆到 queue、prefill、decode、streaming 等阶段。

推理 SLO 可以写成:

$$G_{\text{infer}} = I(Q_{0.99}(T_{\text{e2e}}) \leq S_{\text{e2e}}) \cdot I(Q_{0.95}(T_{\text{tft}}) \leq S_{\text{tft}}) \cdot I(Q_{0.95}(T_{\text{tpot}}) \leq S_{\text{tpot}}) \cdot I(R_{\text{err}} \leq \epsilon)$$

这比只看 QPS 更接近用户体验，因为用户会先感知 TTFT，再感知流式输出间隔。

44.10 数据平台可观测性

数据平台需要观察:

1. ingestion 成功率。
2. 清洗任务成功率。
3. 样本数量变化。
4. token 数变化。
5. 重复率。
6. PII 命中率。
7. schema 错误率。
8. shard 损坏率。
9. 数据读取吞吐。
10. data loader waiting time。
11. dataset quality gate 状态。

数据问题常常在训练中表现为 loss 异常或效果退化。

所以数据质量指标要和训练结果关联起来。

数据质量异常率:

$$R_{\text{data_bad}} = \frac{N_{\text{schema fail}} + N_{\text{pii hit}} + N_{\text{corrupt}} + N_{\text{quality fail}}}{N_{\text{sample}}}$$

数据平台可观测性要能把 $R_{\text{data_bad}}$ 、读取吞吐、data wait 和训练 loss / eval 退化连起来。

44.11 模型和实验可观测性

模型和实验层需要观察:

1. experiment run 状态。

2. 指标曲线。
3. eval score。
4. eval failure cases。
5. model version 状态。
6. release gate 状态。
7. 灰度指标。
8. A/B 测试结果。
9. 回滚事件。

模型质量问题不一定表现为系统错误。

模型稳定返回 200，但回答质量下降，这也必须被观测。

模型质量监控可以写成：

$$\Delta Q = Q_{\text{online}} - Q_{\text{baseline}}$$

当 ΔQ 在关键切片上低于阈值时，即使错误率为 0，也应该触发质量告警或发布门禁。

44.12 RAG/Agent 可观测性

RAG/Agent 需要观察：

RAG 指标：

1. retrieval latency。
2. top-k hit rate。
3. empty result rate。
4. rerank latency。
5. citation correctness。
6. groundedness。

Agent 指标：

1. step count。
2. tool call count。
3. tool success rate。
4. tool latency。
5. loop detection count。
6. user confirmation count。
7. run success rate。
8. cost per run。

Trace：

1. retrieval trace。
2. prompt assembly trace。
3. tool call trace。
4. execution trace。

Agent 是多步系统，不看 trace 基本无法排查问题。

RAG / Agent 可观测性门禁：

$$G_{\text{rag_agent_obs}} = I(C_{\text{retr_trace}} \geq \alpha) \cdot I(C_{\text{tool_trace}} \geq \beta) \cdot I(C_{\text{exec_trace}} \geq \gamma) \cdot I(R_{\text{loop}} \leq \lambda)$$

其中 R_{loop} 是循环或重复工具调用异常率。Agent 成功率高但 trace 缺失，仍然不可上线。

44.13 成本可观测性

AI Infra 成本高，成本本身也是可观测性的一部分。

需要观察：

1. GPU hour。
2. GPU utilization。
3. storage cost。
4. network cost。
5. training cost per run。
6. inference cost per 1k tokens。
7. cost per tenant。
8. cost per model。
9. cost per endpoint。
10. idle GPU cost。

如果只能看到总账单，无法做成本治理。

成本必须和模型、租户、任务、endpoint、run 关联。

单位 token 成本：

$$K_{1k} = \frac{1000 \cdot K_{\text{total}}}{N_{\text{input token}} + N_{\text{output token}}}$$

闲置 GPU 成本：

$$K_{\text{idle}} = H_{\text{gpu}} \cdot p_{\text{gpu}} \cdot (1 - U_{\text{gpu}})$$

成本可观测性要能定位 “哪个模型、租户、endpoint、run 或工具” 造成成本变化。

44.14 统一 ID 体系

可观测性要靠统一 ID 串联。

常见 ID：

1. request ID。
2. trace ID。
3. tenant ID。
4. user ID。
5. training job ID。
6. experiment run ID。
7. model version ID。
8. dataset version ID。
9. checkpoint ID。
10. deployment ID。

如果日志、指标、trace、事件里的 ID 不一致，就很难关联。

统一 ID 是平台可观测性的基础。

ID 覆盖率：

$$C_{\text{id}} = \frac{\sum_i I(\text{trace_id}_i \vee \text{run_id}_i \vee \text{job_id}_i)}{N_{\text{record}}}$$

训练、评估、推理、RAG 和 Agent 不一定都有 request ID，但至少要有可关联的 job / run / trace / model / dataset 版本 ID。

44.15 标签体系

Metrics 要有合理标签。

常见标签：

1. cluster。
2. namespace。
3. tenant。
4. model。
5. model_version。
6. endpoint。
7. runtime。
8. gpu_type。
9. job_type。
10. priority。

标签过少，无法定位。

标签过多，会导致指标基数爆炸。

设计标签要平衡查询能力和存储成本。

时间序列数量近似为：

$$N_{\text{series}} \approx \sum_{m \in M} \prod_{\ell \in L_m} |V_\ell|$$

V_ℓ 是某个 label 的取值集合。把 request_id、user_id、prompt_hash 放进 metrics label，会让 N_{series} 爆炸。

44.16 高基数问题

高基数是 metrics 系统常见问题。

例如把 request ID 作为 metrics label，会导致时间序列数量爆炸。

高基数字段适合放在 logs 或 traces 中，不适合放在 metrics label 中。

Metrics label 应选择有限枚举字段，例如 model、tenant、endpoint、status。

高基数字段如 request ID、prompt hash、user ID，通常进入 trace 或日志。

标签基数门禁：

$$G_{\text{card}} = I(N_{\text{series}} \leq B_{\text{series}}) \cdot I(L_{\text{high card}} \cap L_{\text{metric}} = \emptyset)$$

其中 B_{series} 是序列预算， $L_{\text{high card}}$ 是高基数字段集合。

44.17 告警设计

告警不是阈值越多越好。

好的告警应该：

1. 指向用户影响。
2. 可行动。
3. 有上下文。

4. 避免重复。
5. 有优先级。

常见告警：

1. 训练任务 hang。
2. GPU 利用率长期过低。
3. checkpoint 保存失败。
4. 推理 p99 超阈值。
5. timeout rate 上升。
6. KV cache 水位过高。
7. eval gate 失败。
8. cost 异常上升。

不要为每个小波动都告警，否则值班会被噪音淹没。

告警可行动率：

$$A_{\text{action}} = \frac{N_{\text{actionable alerts}}}{N_{\text{alerts}}}$$

告警噪音率：

$$R_{\text{noise}} = \frac{N_{\text{duplicate}} + N_{\text{non action}}}{N_{\text{alerts}}}$$

好的告警要能指向 runbook、owner、影响面和可能根因，而不是只告诉你某条曲线抖了一下。

44.18 Dashboard 设计

不同角色需要不同 dashboard。

训练工程师关注：

1. loss。
2. tokens/s。
3. GPU。
4. data loader。
5. checkpoint。

推理工程师关注：

1. QPS。
2. TTFT。
3. TPOT。
4. queue。
5. KV cache。
6. runtime errors。

平台负责人关注：

1. 集群利用率。
2. 成本。
3. SLO。
4. 租户配额。
5. 发布状态。

不要试图用一个 dashboard 服务所有人。

44.19 日志脱敏

AI 系统日志可能包含敏感信息：

1. prompt。
2. 用户输入。
3. 检索文档。
4. 工具参数。
5. 模型输出。
6. 密钥。
7. 私有数据。

日志系统必须支持：

1. 脱敏。
2. 字段级权限。
3. TTL。
4. 加密。
5. 审计。

不能为了排障，把所有 prompt 和输出无保护地永久保存。

日志与 trace 隐私门禁：

$$G_{\text{privacy}} = I(C_{\text{redact}} \geq \alpha) \cdot I(\text{ttl} \leq \tau_{\text{max}}) \cdot I(\text{access scoped}) \cdot I(\text{audit enabled})$$

44.20 Trace 采样

Trace 全量保存成本高，尤其是 Agent 和 RAG。

常见策略：

1. 错误请求全量保存。
2. 慢请求全量保存。
3. 高成本请求全量保存。
4. 普通成功请求采样。
5. 敏感请求只保存结构化摘要。

采样策略要兼顾排障能力、成本和隐私。

Trace 保存概率可以写成：

$$p_{\text{trace}}(r) = \begin{cases} 1, & r \in R_{\text{error}} \cup R_{\text{slow}} \cup R_{\text{high cost}} \\ p_s, & r \in R_{\text{success}} \\ p_{\text{summary}}, & r \in R_{\text{sensitive}} \end{cases}$$

这和上一章 RAG / Agent trace 的隐私策略一致：错误、慢请求和高成本 run 优先保留，普通成功请求采样，敏感请求保留摘要。

44.21 可观测性平台架构

一个 AI Infra 可观测性平台可以这样设计：

Collectors / SDKs / Agents

- > Metrics Pipeline
- > Logs Pipeline
- > Traces Pipeline
- > Events Pipeline
- > Correlation / Indexing

- > Alerting
- > Dashboards
- > Incident / Postmortem

数据来源包括训练任务、推理 runtime、K8s、GPU exporter、存储系统、网络系统、评估平台、RAG/Agent runtime 和成本系统。

44.22 AI Infra 可观测性审计指标和最小 demo

把本章落到平台验收时，可以用 16 个门禁：

1. Signal Inventory Coverage: metrics、logs、traces、events、costs、alerts、dashboards、versions 是否都有入口。
2. Metric Contract Completeness: metric name、unit、labels、aggregation、owner、SLO / threshold 是否完整。
3. SLO Error Budget Readiness: 可用性 SLI、SLO 目标、错误预算、burn rate 是否可计算。
4. Latency Quantile Guard: p95 / p99、TTFT、TPOT 是否有分位指标和阈值。
5. Trace Span Coverage: 关键链路 span 是否覆盖 gateway、auth、router、queue、prefill、decode、safety、response。
6. Log Event Structure: 日志和事件是否结构化，有时间、级别、服务、trace id、状态、事件类型、actor 和版本。
7. Correlation ID Coverage: metrics、logs、traces、events、costs 是否能通过 trace / run / job / model / dataset ID 关联。
8. Cardinality Budget Control: metrics label 是否避免 request ID、user ID、prompt hash 等高基数字段。
9. Training Observability: loss、learning rate、gradient norm、tokens/s、GPU、NCCL、data wait、checkpoint 是否完整。
10. Inference Observability: QPS、TTFT、TPOT、error、timeout、queue、KV cache、GPU、cache、fallback、degrade 是否完整。
11. Data Quality Observability: ingestion、schema、PII、duplicate、corrupt shard、quality gate、data loader wait 是否完整。
12. RAG Agent Observability: retrieval trace、prompt assembly trace、tool call trace、execution trace、loop rate 和 groundedness 是否完整。
13. Cost Observability: GPU、storage、network、LLM token、tool、trace 成本是否能按 tenant / model / endpoint / run 归因。
14. Privacy Redaction Retention: prompt、检索片段、tool 参数、输出和日志是否有脱敏、TTL、访问控制和审计。
15. Alert Actionability: 告警是否有 owner、runbook、severity、影响面、去重和可行动上下文。
16. Observability Platform Gate: 最终是否有 owner、dashboard、incident process、postmortem、回滚和 P0 风险阻断。

综合门禁：

$$G_{\text{observability}} = \prod_{j=1}^{16} G_j$$

下面是一个 0 依赖 demo，用 toy telemetry cases 检查一个 AI Infra 可观测性平台是否只是“收集了一堆数据”，还是能真正发现、定位、解释和复盘问题。

from copy import deepcopy

```
class MiniAIInfraObservabilityAudit:
    GATES = [
        "signal_inventory_coverage",
        "metric_contract_completeness",
```



```

        "slo_error_budget_readiness",
        "latency_quantile_guard",
        "trace_span_coverage",
        "log_event_structure",
        "correlation_id_coverage",
        "cardinality_budget_control",
        "training_observability",
        "inference_observability",
        "data_quality_observability",
        "rag_agent_observability",
        "cost_observability",
        "privacy_redaction_retention",
        "alert_actionability",
        "observability_platform_gate",
    ]

    METRIC_FIELDS = ["name", "unit", "labels", "aggregation", "owner", "value"]
    LOG_FIELDS = ["ts", "level", "service", "trace_id", "event", "status", "redacted"]
    EVENT_FIELDS = ["ts", "event_type", "entity", "version", "actor"]
    REQUIRED_TRAINING = [
        "train_loss",
        "eval_loss",
        "learning_rate",
        "gradient_norm",
        "tokens_per_s",
        "gpu_util",
        "nccl_time_ratio",
        "data_wait_ratio",
        "checkpoint_failures",
    ]
    REQUIRED_INFERENCE = [
        "qps",
        "ttft_p95_ms",
        "tpot_p95_ms",
        "error_rate",
        "timeout_rate",
        "queue_p95_ms",
        "kv_cache_pressure",
        "gpu_util",
        "cache_hit_rate",
        "fallback_rate",
        "degrade_rate",
    ]
    REQUIRED_DATA = [
        "ingestion_success_rate",
        "schema_error_rate",
        "pii_hit_rate",
        "duplicate_rate",
        "corrupt_shards",
        "quality_gate_pass",
        "data_loader_wait_ratio",
    ]

    @staticmethod
    def present(record, key):

```

```

    return key in record and record[key] is not None and record[key] != ""

def coverage(self, record, fields):
    if not record:
        return 0.0
    return sum(1 for field in fields if self.present(record, field)) / len(fields)

def signal_inventory_coverage(self, case):
    required = {"metrics", "logs", "traces", "events", "costs", "alerts", "dashboards", "versions"}
    return required.issubset(set(case.get("signals", [])))

def metric_contract_completeness(self, case):
    metrics = case.get("metrics", [])
    return bool(metrics) and all(
        self.coverage(metric, self.METRIC_FIELDS) == 1.0
        and isinstance(metric.get("labels"), dict)
        for metric in metrics
    )

def slo_error_budget_readiness(self, case):
    slo = case.get("slo", {})
    total = slo.get("requests_total", 0)
    good = slo.get("good_requests", 0)
    target = slo.get("availability_target", 1.0)
    if total <= 0 or target >= 1.0:
        return False
    availability = good / total
    budget = 1 - target
    used = 1 - availability
    return availability >= target and (budget - used) >= 0 and "window_minutes" in slo

def latency_quantile_guard(self, case):
    latency = case.get("latency", {})
    return (
        latency.get("e2e_p99_ms", 999999) <= latency.get("e2e_slo_ms", 0)
        and latency.get("ttft_p95_ms", 999999) <= latency.get("ttft_slo_ms", 0)
        and latency.get("tpot_p95_ms", 999999) <= latency.get("tpot_slo_ms", 0)
    )

def trace_span_coverage(self, case):
    traces = case.get("traces", {})
    expected = set(traces.get("expected_spans", []))
    observed = set(traces.get("observed_spans", []))
    if not expected:
        return False
    return len(expected & observed) / len(expected) >= 0.95

def log_event_structure(self, case):
    logs = case.get("logs", [])
    events = case.get("events", [])
    logs_ok = logs and all(self.coverage(log, self.LOG_FIELDS) == 1.0 for log in logs)
    events_ok = events and all(self.coverage(event, self.EVENT_FIELDS) == 1.0 for event in events)
    return bool(logs_ok and events_ok)

def correlation_id_coverage(self, case):

```

```

corr = case.get("correlation", {})
total = corr.get("records_total", 0)
with_id = corr.get("records_with_id", 0)
return total > 0 and with_id / total >= 0.95

def cardinality_budget_control(self, case):
    labels = case.get("labels", {})
    metric_count = labels.get("metric_count", 0)
    values = labels.get("label_value_counts", {})
    series = metric_count
    for count in values.values():
        series *= count
    return (
        series <= labels.get("series_budget", 0)
        and not labels.get("high_cardinality_metric_labels", [])
    )

def training_observability(self, case):
    training = case.get("training", {})
    return (
        self.coverage(training, self.REQUIRED_TRAINING) == 1.0
        and training["data_wait_ratio"] <= 0.15
        and training["checkpoint_failures"] == 0
    )

def inference_observability(self, case):
    inference = case.get("inference", {})
    return (
        self.coverage(inference, self.REQUIRED_INFERENCE) == 1.0
        and inference["error_rate"] <= 0.01
        and inference["timeout_rate"] <= 0.005
        and inference["kv_cache_pressure"] <= 0.85
    )

def data_quality_observability(self, case):
    data = case.get("data_quality", {})
    return (
        self.coverage(data, self.REQUIRED_DATA) == 1.0
        and data["schema_error_rate"] <= 0.01
        and data["pii_hit_rate"] <= 0.001
        and data["corrupt_shards"] == 0
        and data["quality_gate_pass"] is True
    )

def rag_agent_observability(self, case):
    rag_agent = case.get("rag_agent", {})
    return (
        rag_agent.get("retrieval_trace_coverage", 0.0) >= 0.95
        and rag_agent.get("tool_trace_coverage", 0.0) >= 0.95
        and rag_agent.get("execution_trace_coverage", 0.0) >= 0.95
        and rag_agent.get("loop_rate", 1.0) <= 0.05
        and rag_agent.get("groundedness", 0.0) >= 0.8
    )

def cost_observability(self, case):

```

```

cost = case.get("cost", {})
if cost.get("total_tokens", 0) <= 0:
    return False
cost_per_1k = 1000 * cost["total_cost_usd"] / cost["total_tokens"]
return (
    cost.get("by_tenant") is True
    and cost.get("by_model") is True
    and cost.get("by_endpoint") is True
    and cost.get("by_run") is True
    and cost_per_1k <= cost.get("max_cost_per_1k_usd", 9999)
)

def privacy_redaction_retention(self, case):
    privacy = case.get("privacy", {})
    total = privacy.get("sensitive_fields_total", 0)
    redacted = privacy.get("redacted_fields", 0)
    if total <= 0:
        return False
    return (
        redacted / total >= 0.95
        and privacy.get("ttl_days", 9999) <= 90
        and privacy.get("access_scoped") is True
        and privacy.get("audit_enabled") is True
    )

def alert_actionability(self, case):
    alerts = case.get("alerts", [])
    if not alerts:
        return False
    actionable = [alert for alert in alerts if alert.get("actionable")]
    duplicates = [alert for alert in alerts if alert.get("duplicate")]
    metadata_ok = all(alert.get("owner") and alert.get("runbook") and alert.get("severity") for alert in alerts)
    return len(actionable) / len(alerts) >= 0.8 and len(duplicates) / len(alerts) <= 0.2 and metadata_ok

def observability_platform_gate(self, case):
    gate = case.get("platform_gate", {})
    return (
        gate.get("enabled") is True
        and bool(gate.get("owner"))
        and gate.get("dashboards_ready") is True
        and gate.get("incident_process_ready") is True
        and gate.get("p0_open") is False
    )

def audit_case(self, case):
    return {gate: getattr(self, gate)(case) for gate in self.GATES}

def run_all(self, cases):
    results = {case["case_id"]: self.audit_case(case) for case in cases}
    metrics = {}
    for gate in self.GATES:
        passed = sum(1 for result in results.values() if result[gate])
        metrics[gate] = round(passed / len(cases), 3)
    failed_cases = [
        case_id

```

```

        for case_id, result in results.items()
        if not all(result.values())
    ]
    failed_gates = [
        gate
        for gate in self.GATES
        if any(not result[gate] for result in results.values())
    ]
    return {
        "metrics": metrics,
        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "failed_gates": failed_gates,
        "observability_gate_pass": metrics["observability_platform_gate"] == 1.0,
    }

def example_outputs(self, case):
    slo = case["slo"]
    availability = slo["good_requests"] / slo["requests_total"]
    budget = 1 - slo["availability_target"]
    used = 1 - availability
    labels = case["labels"]
    series = labels["metric_count"]
    for count in labels["label_value_counts"].values():
        series *= count
    cost = case["cost"]
    privacy = case["privacy"]
    logs = case["logs"]
    events = case["events"]
    traces = case["traces"]
    span_coverage = len(set(traces["expected_spans"]) & set(traces["observed_spans"])) / len(traces["expected_spans"])
    return {
        "signal_coverage": round(len(case["signals"]) / 8, 3),
        "metric_contract_coverage": round(
            sum(self.coverage(metric, self.METRIC_FIELDS) for metric in case["metrics"])
            / len(case["metrics"]),
            3,
        ),
        "availability": round(availability, 4),
        "error_budget_remaining": round((budget - used) / budget, 3),
        "latency_p99_ms": case["latency"]["e2e_p99_ms"],
        "trace_span_coverage": round(span_coverage, 3),
        "log_structure_coverage": round(
            sum(self.coverage(log, self.LOG_FIELDS) for log in logs) / len(logs),
            3,
        ),
        "event_timeline_coverage": round(
            sum(self.coverage(event, self.EVENT_FIELDS) for event in events) / len(events),
            3,
        ),
        "correlation_id_coverage": round(case["correlation"]["records_with_id"] / case["correlation"]["records_total"], 3),
        "cardinality_estimate": series,
        "training_tokens_per_s": case["training"]["tokens_per_s"],
        "inference_ttft_p95_ms": case["inference"]["ttft_p95_ms"],
        "rag_agent_trace_ready": self.rag_agent_observability(case),
    }

```

```

    "cost_per_1k_tokens_usd": round(1000 * cost["total_cost_usd"] / cost["total_tokens"], 3),
    "redaction_coverage": round(privacy["redacted_fields"] / privacy["sensitive_fields_total"], 3),
    "actionable_alerts": sum(1 for alert in case["alerts"] if alert["actionable"]),
}

```

```
def build_good_case():
```

```

    return {
        "case_id": "production_ready",
        "signals": ["metrics", "logs", "traces", "events", "costs", "alerts", "dashboards", "versions"],
        "metrics": [
            {
                "name": "inference_e2e_latency_ms",
                "unit": "ms",
                "labels": {"tenant": "acme", "model": "chat", "endpoint": "chat", "status": "ok"},
                "aggregation": "histogram",
                "owner": "serving",
                "value": 890,
            },
            {
                "name": "training_tokens_per_s",
                "unit": "tokens/s",
                "labels": {"cluster": "train-a", "job_type": "pretrain", "gpu_type": "h100"},
                "aggregation": "gauge",
                "owner": "training",
                "value": 178000,
            },
            {
                "name": "rag_groundedness",
                "unit": "ratio",
                "labels": {"tenant": "acme", "app": "support"},
                "aggregation": "gauge",
                "owner": "rag-platform",
                "value": 0.88,
            },
        ],
        "slo": {
            "requests_total": 10000,
            "good_requests": 9970,
            "availability_target": 0.99,
            "window_minutes": 60,
        },
        "latency": {
            "e2e_p99_ms": 890,
            "e2e_slo_ms": 1200,
            "ttft_p95_ms": 620,
            "ttft_slo_ms": 800,
            "tpot_p95_ms": 45,
            "tpot_slo_ms": 80,
        },
        "traces": {
            "expected_spans": ["gateway", "auth", "router", "queue", "prefill", "decode", "safety", "response"],
            "observed_spans": ["gateway", "auth", "router", "queue", "prefill", "decode", "safety", "response"],
        },
        "logs": [

```

```

{
  "ts": "2026-06-15T10:00:00Z",
  "level": "INFO",
  "service": "router",
  "trace_id": "tr_1",
  "event": "route_decision",
  "status": "ok",
  "redacted": True,
},
{
  "ts": "2026-06-15T10:00:01Z",
  "level": "WARN",
  "service": "runtime",
  "trace_id": "tr_1",
  "event": "kv_pressure_high",
  "status": "warn",
  "redacted": True,
},
],
"events": [
  {
    "ts": "2026-06-15T09:55:00Z",
    "event_type": "deployment_promoted",
    "entity": "chat-serving",
    "version": "deploy_7",
    "actor": "release-bot",
  },
  {
    "ts": "2026-06-15T09:58:00Z",
    "event_type": "autoscaler_scaled_out",
    "entity": "chat-serving",
    "version": "hpa_3",
    "actor": "autoscaler",
  },
],
"correlation": {"records_total": 50, "records_with_id": 50},
"labels": {
  "metric_count": 10,
  "label_value_counts": {"tenant": 12, "model": 20, "endpoint": 10, "status": 4},
  "series_budget": 100000,
  "high_cardinality_metric_labels": [],
},
"training": {
  "train_loss": 1.92,
  "eval_loss": 1.98,
  "learning_rate": 0.0002,
  "gradient_norm": 0.74,
  "tokens_per_s": 178000,
  "gpu_util": 0.72,
  "nccl_time_ratio": 0.12,
  "data_wait_ratio": 0.08,
  "checkpoint_failures": 0,
},
"inference": {
  "qps": 320,

```

```

    "ttft_p95_ms": 620,
    "tpot_p95_ms": 45,
    "error_rate": 0.003,
    "timeout_rate": 0.001,
    "queue_p95_ms": 110,
    "kv_cache_pressure": 0.72,
    "gpu_util": 0.77,
    "cache_hit_rate": 0.34,
    "fallback_rate": 0.02,
    "degrade_rate": 0.01,
  },
  "data_quality": {
    "ingestion_success_rate": 0.998,
    "schema_error_rate": 0.004,
    "pii_hit_rate": 0.0,
    "duplicate_rate": 0.018,
    "corrupt_shards": 0,
    "quality_gate_pass": True,
    "data_loader_wait_ratio": 0.06,
  },
  "rag_agent": {
    "retrieval_trace_coverage": 1.0,
    "tool_trace_coverage": 1.0,
    "execution_trace_coverage": 1.0,
    "loop_rate": 0.01,
    "groundedness": 0.88,
  },
  "cost": {
    "total_cost_usd": 240.0,
    "total_tokens": 20_000_000,
    "max_cost_per_1k_usd": 0.02,
    "by_tenant": True,
    "by_model": True,
    "by_endpoint": True,
    "by_run": True,
    "idle_gpu_cost_usd": 38.4,
  },
  "privacy": {
    "sensitive_fields_total": 5,
    "redacted_fields": 5,
    "ttl_days": 30,
    "access_scoped": True,
    "audit_enabled": True,
  },
  "alerts": [
    {
      "name": "inference_p99_burn",
      "owner": "serving",
      "runbook": "runbook://serving-latency",
      "severity": "page",
      "actionable": True,
      "duplicate": False,
    },
    {
      "name": "checkpoint_failure",

```



```

        "owner": "training",
        "runbook": "runbook://checkpoint",
        "severity": "ticket",
        "actionable": True,
        "duplicate": False,
    },
    {
        "name": "cost_spike",
        "owner": "platform-finops",
        "runbook": "runbook://cost-spike",
        "severity": "ticket",
        "actionable": True,
        "duplicate": False,
    },
],
"platform_gate": {
    "enabled": True,
    "owner": "ai-infra-observability",
    "dashboards_ready": True,
    "incident_process_ready": True,
    "p0_open": False,
},
}

```

```

def build_bad_cases(good_case):
    cases = []

    case = deepcopy(good_case)
    case["case_id"] = "signal_inventory_missing_bad"
    case["signals"].remove("events")
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "metric_contract_missing_bad"
    case["metrics"][0].pop("owner")
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "slo_error_budget_burn_bad"
    case["slo"]["good_requests"] = 9890
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "latency_quantile_bad"
    case["latency"]["e2e_p99_ms"] = 1800
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "trace_span_missing_bad"
    case["traces"]["observed_spans"].remove("decode")
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "log_event_structure_bad"

```

```

case["logs"][0].pop("trace_id")
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "correlation_id_missing_bad"
case["correlation"]["records_with_id"] = 40
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "cardinality_explosion_bad"
case["labels"]["high_cardinality_metric_labels"] = ["request_id"]
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "training_observability_gap_bad"
case["training"]["data_wait_ratio"] = 0.31
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "inference_observability_gap_bad"
case["inference"]["kv_cache_pressure"] = 0.94
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "data_quality_observability_bad"
case["data_quality"]["schema_error_rate"] = 0.08
case["data_quality"]["quality_gate_pass"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "rag_agent_trace_gap_bad"
case["rag_agent"]["tool_trace_coverage"] = 0.5
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "cost_observability_missing_bad"
case["cost"]["by_tenant"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "privacy_redaction_retention_bad"
case["privacy"]["redacted_fields"] = 2
case["privacy"]["ttl_days"] = 365
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "alert_noise_bad"
case["alerts"][0]["actionable"] = False
case["alerts"][0]["duplicate"] = True
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "observability_gate_missing_bad"
case["platform_gate"]["enabled"] = False
cases.append(case)

```

```
return cases
```

```
audit = MiniAIInfraObservabilityAudit()
good = build_good_case()
cases = [good] + build_bad_cases(good)
summary = audit.run_all(cases)

print("observability_examples=" + repr(audit.example_outputs(good)))
print("metrics=" + repr(summary["metrics"]))
print("hard_blocker_count=" + repr(summary["hard_blocker_count"]))
print("failed_cases=" + repr(summary["failed_cases"]))
print("failed_gates=" + repr(summary["failed_gates"]))
print("observability_gate_pass=" + repr(summary["observability_gate_pass"]))
```

参考输出应类似：

```
observability_examples={'signal_coverage': 1.0, 'metric_contract_coverage': 1.0, 'availability': 0.997, 'error_budget_readiness': 0.997, 'latency_quantile': 0.997}
metrics={'signal_inventory_coverage': 0.941, 'metric_contract_completeness': 0.941, 'slo_error_budget_readiness': 0.941, 'latency_quantile': 0.941}
hard_blocker_count=16
failed_cases=['signal_inventory_missing_bad', 'metric_contract_missing_bad', 'slo_error_budget_burn_bad', 'latency_quantile_burn_bad']
failed_gates=['signal_inventory_coverage', 'metric_contract_completeness', 'slo_error_budget_readiness', 'latency_quantile']
observability_gate_pass=False
```

这个 demo 的面试价值是把“可观测性”从泛泛的监控系统，变成可验收的工程能力：指标有契约，SLO 能算，trace 能拆阶段，日志和事件能关联，标签基数受控，AI 专属训练 / 推理 / RAG / Agent / 成本 / 隐私信号都能进入同一个排障闭环。

44.23 常见误区

误区一：有 GPU 指标就够了。

GPU 指标只能说明资源状态，不能解释模型质量、数据问题、trace 链路和成本。

误区二：日志越多越好。

日志太多会增加成本和隐私风险，关键是结构化、可检索和有上下文。

误区三：只看平均延迟。

AI 在线服务更要看 p95/p99、TTFT、TPOT 和队列。

误区四：模型质量不属于可观测性。

模型质量、eval 分数、线上反馈和安全指标都应该进入可观测体系。

误区五：可观测性只服务故障排查。

它还服务性能优化、成本治理、发布决策和合规审计。

44.24 面试常见追问

问题一：AI Infra 可观测性和普通后端可观测性有什么不同？

可以回答：除了 QPS、延迟、错误率和资源，还要关注 GPU、显存、KV cache、training loss、tokens/s、TTFT、TPOT、数据质量、模型版本、eval 分数、RAG trace、tool trace 和成本。

问题二：Metrics、logs、traces、events 分别解决什么问题？

可以回答：metrics 用于趋势和告警，logs 用于细节排查，traces 用于链路和阶段耗时定位，events 用于状态变化和复盘时间线。

问题三：为什么 request ID 不适合做 metrics label?

可以回答：request ID 基数极高，会导致时间序列爆炸。它适合放在 trace 或日志里，而不是 metrics label。

问题四：如何设计推理服务的可观测性?

可以回答：记录 QPS、TTFT、TPOT、p99、input/output tokens/s、queue length、active sequences、KV cache、GPU、cache hit、fallback、degradation、error/timeout，并通过 trace 拆分 gateway、router、queue、prefill、decode 和 streaming。

问题五：如何设计 AI Infra 可观测性平台的上线门禁?

可以回答：按 signal inventory、metric contract、SLO / error budget、latency quantile、trace span、structured log / event、correlation ID、label cardinality、training、inference、data quality、RAG / Agent、cost、privacy、alert actionability 和 platform gate 逐项验收。重点不是收集更多指标，而是证明每个关键事故都能被发现、定位、解释、复盘和改进。

44.25 小练习

1. AI Infra 为什么比普通后端更依赖可观测性?
2. Metrics、logs、traces、events 各适合回答什么问题?
3. 训练任务需要观察哪些模型指标和系统指标?
4. 推理服务为什么要同时看 TTFT 和 TPOT?
5. RAG/Agent 为什么必须保存 trace?
6. 什么是 metrics 高基数问题?
7. 日志脱敏为什么在 AI 系统中特别重要?
8. 如何设计一个推理 p99 延迟告警?
9. 写一个 0 依赖 AI Infra 可观测性审计 demo，覆盖 metrics、logs、traces、events、SLO、标签基数、训练、推理、RAG / Agent、成本、隐私和告警。
10. 构造 5 个可观测性失败样本：request ID 进入 metrics label、trace 缺 decode span、日志未脱敏、告警无 runbook、成本只按总账记录，并说明各自如何修复。

44.26 本章小结

本章讲了 AI Infra 可观测性总览。

你需要记住：

1. AI Infra 可观测性要覆盖训练、推理、数据、模型、RAG、Agent、成本和平台资源。
2. Metrics、logs、traces、events 各有作用，必须关联起来使用。
3. 训练可观测性要同时看 loss、tokens/s、GPU、通信、数据读取和 checkpoint。
4. 推理可观测性要关注 TTFT、TPOT、tokens/s、queue、KV cache、runtime 和路由。
5. RAG/Agent 可观测性离不开 retrieval trace、tool call trace 和 execution trace。
6. 可观测性还必须考虑成本、隐私、脱敏、高基数和告警噪音。
7. 第二轮精修后，本章新增的核心抓手是 Observability Platform Gate：让 signals、SLO、trace、日志、事件、成本、隐私、告警和 dashboard 都能被结构化验收。

下一章我们会讲训练故障定位：loss 异常、hang、OOM、通信慢和 I/O 慢。

第 45 章 训练故障定位：loss 异常、hang、OOM、通信慢和 I/O 慢

上一章讲了 AI Infra 可观测性总览。本章进入具体故障定位：训练任务出问题时，如何系统排查。

大模型训练任务通常运行时间长、成本高、节点多、依赖复杂。一次训练故障可能浪费数百张 GPU 数小时。因此，训练故障定位是 AI Infra 工程师必须掌握的能力。

先记住一句话：

训练故障定位不要先猜模型或框架，而要按数据、代码、配置、资源、通信、存储和平台链路逐层排查。

45.0 本讲资料边界与第二轮精修口径

本章按通用大模型训练平台故障定位抽象来写，不绑定 PyTorch DDP、FSDP、DeepSpeed、Megatron、Kubernetes、Slurm、NCCL 具体版本、GPU 型号或云厂商平台实现。资料校准时，主要参考 PyTorch autograd anomaly detection、CUDA memory management、PyTorch profiler 对算子 / CPU / CUDA 耗时定位的口径，参考 NVIDIA NCCL troubleshooting 对通信、环境变量、网络和异步错误排查的工程边界，也结合前文训练可观测性、checkpoint、数据供给和实验追踪章节中的版本、manifest、rank 日志和事件时间线要求。

第二轮精修只做三件事：

1. 把 loss 异常、NaN、hang、OOM、通信慢、I/O 慢、checkpoint 和 resume 问题统一成故障样本。
2. 补齐 step time、loss spike、NaN / Inf、gradient health、显存预算、通信占比、data wait、checkpoint 完整性和最小复现公式。
3. 增加一个 0 依赖 Python demo，用 toy training incident cases 检查训练故障定位是否有足够证据链。

45.1 训练故障的常见类型

大模型训练常见故障包括：

1. loss 异常。
2. loss 不下降。
3. loss 突然 NaN。
4. 训练 hang 住。
5. OOM。
6. 节点失败。
7. NCCL 通信错误。
8. step time 变慢。
9. 数据读取慢。
10. checkpoint 保存失败。
11. 任务反复重试。
12. GPU 利用率低。

这些问题可能互相影响。

例如数据读取慢会导致 GPU 利用率低，通信慢会导致 step time 变长，OOM 可能来自 batch、sequence length 或显存碎片。

一个训练故障样本可以写成：

$$F_i = (j_i, e_i, r_i, l_i, g_i, m_i, c_i, d_i, k_i, \Delta_i, p_i, z_i)$$

其中 j_i 是 TrainingJob, e_i 是生命周期事件, r_i 是 rank 日志, l_i 是 loss / eval 曲线, g_i 是梯度和 optimizer 证据, m_i 是显存与 OOM 证据, c_i 是通信证据, d_i 是数据和 I/O 证据, k_i 是 checkpoint / resume 证据, Δ_i 是代码 / 配置 / 数据版本 diff, p_i 是最小复现证据, z_i 是最终根因和修复记录。

故障定位证据覆盖率：

$$C_{\text{fault}} = \frac{1}{N} \sum_{i=1}^N I(e_i, r_i, l_i, g_i, m_i, c_i, d_i, k_i, \Delta_i, p_i, z_i \text{ present})$$

45.2 排查总原则

训练故障排查可以遵循这个顺序：

1. 看任务状态。
2. 看最近事件。
3. 看错误日志。
4. 看训练指标。

5. 看系统资源。
6. 看数据读取。
7. 看分布式通信。
8. 看 checkpoint。
9. 看代码和配置变更。
10. 看数据版本变更。

不要只盯着最后一行报错。

分布式训练中，真正原因可能出现在另一个 rank、另一个节点或更早的事件里。

排查优先级可以写成：

$$\text{priority}(s) = w_t T_s + w_i I_s + w_r R_s + w_c C_s$$

其中 T_s 是时间上是否更早， I_s 是影响面， R_s 是是否可复现， C_s 是证据可信度。训练事故不要只看最后一个报错，而要优先看最早、影响最大、可复现且证据最完整的线索。

45.3 先看任务生命周期事件

训练平台应该记录事件：

1. Job submitted。
2. Job scheduled。
3. Pods created。
4. Image pulled。
5. Dataset mounted。
6. Training started。
7. Checkpoint saved。
8. Node lost。
9. Pod restarted。
10. Job failed。

这些事件能快速判断问题发生在哪个阶段。

如果任务连训练都没开始，就不要先看模型代码，应该看调度、镜像、权限和数据挂载。

45.4 Loss 异常怎么排查

Loss 异常包括：

1. loss 不下降。
2. loss 波动异常。
3. loss 突然升高。
4. loss 变 NaN。
5. eval loss 和 train loss 背离。

排查维度：

1. 数据是否变化。
2. 学习率是否过大。
3. batch size 是否变化。
4. mixed precision 是否稳定。
5. loss scale 是否异常。
6. 标签或 mask 是否错误。
7. tokenizer 是否匹配。
8. padding 和 attention mask 是否正确。
9. 梯度是否爆炸。
10. checkpoint resume 是否正确。

Loss 是模型、数据和训练配置共同作用的结果。

loss spike 可以用相对变化衡量：

$$S_t = \frac{L_t - \text{median}(L_{t-w:t-1})}{\max(\epsilon, \text{median}(L_{t-w:t-1}))}$$

当 S_t 超过阈值时，再结合数据 batch、学习率、梯度、precision 和 resume 证据判断根因。

45.5 Loss NaN

Loss NaN 是常见严重问题。

可能原因：

1. learning rate 过大。
2. 梯度爆炸。
3. FP16 溢出。
4. loss scale 不合适。
5. 输入数据包含异常值。
6. labels 全是 ignore index 或异常。
7. logits 出现 inf。
8. 自定义 loss 有除零。
9. resume checkpoint 状态损坏。

排查方法：

1. 查看 NaN 出现的 step。
2. 查看该 step 数据 batch。
3. 查看 gradient norm。
4. 降低学习率。
5. 开启 gradient clipping。
6. 切换 BF16 或调整 precision。
7. 检查 loss scale 日志。
8. 单卡复现最小 batch。

NaN 不要只靠跳过 batch 掩盖，必须找到触发条件。

NaN / Inf 命中率：

$$R_{\text{nan}} = \frac{N_{\text{nan loss}} + N_{\text{inf logit}} + N_{\text{nan grad}}}{N_{\text{checked}}}$$

$R_{\text{nan}} > 0$ 时，至少保存触发 batch、loss scale、precision、gradient norm、logits finite check 和最小复现配置。

45.6 Loss 不下降

Loss 不下降可能原因：

1. 学习率太小。
2. 数据标签错误。
3. 模型参数没有更新。
4. optimizer 没有正确加载。
5. 训练样本被 mask 掉。
6. tokenizer 不匹配。
7. 数据重复或质量差。
8. 冻结了不该冻结的层。

9. LoRA target modules 配置错误。
10. 梯度为 0。

排查方法：

1. 看参数梯度是否非零。
2. 看 optimizer step 是否执行。
3. 看 learning rate 曲线。
4. 看有效 token 数。
5. 看样本和标签是否合理。
6. 小数据集 overfit 测试。

如果模型连小数据集都无法 overfit，通常是训练代码或配置问题。

梯度更新健康度：

$$G_{\text{update}} = I(R_{\text{nonzero_grad}} \geq \tau_g) \cdot I(N_{\text{optimizer_step}} > 0) \cdot I(\text{lr}_t > 0)$$

其中 $R_{\text{nonzero_grad}}$ 是非零梯度参数占比。loss 不下降时，先确认模型真的在更新。

45.7 Hang 住怎么排查

训练 hang 指任务没有退出，但 step 不再推进。

常见原因：

1. 某个 rank 卡在 dataloader。
2. 某个 rank OOM 或崩溃但其他 rank 等待。
3. NCCL collective 不一致。
4. 数据读取阻塞。
5. checkpoint 保存阻塞。
6. 节点网络异常。
7. barrier 使用不当。
8. 死锁。

排查方法：

1. 看所有 rank 最后一条日志。
2. 看 step 是否停止增长。
3. 看 GPU 利用率是否为 0 或部分为 0。
4. 看 dataloader worker 状态。
5. 看 NCCL 日志。
6. 看节点网络和存储指标。
7. 看是否有一个 rank 先失败。

分布式训练 hang 往往不是所有 rank 同时出错，而是一个 rank 出问题拖住所有 rank。

hang 判定可以写成：

$$G_{\text{hang}} = I(\Delta\text{step} = 0) \cdot I(\Delta t > \tau) \cdot I(\text{job status} = \text{running})$$

如果某些 rank GPU 利用率为 0、某些 rank 还在等待 collective，优先找最早停止推进的 rank。

45.8 OOM 怎么排查

OOM 可以发生在：

1. GPU 显存。
2. CPU 内存。

3. shared memory。
4. dataloader worker。
5. checkpoint 保存阶段。

GPU OOM 常见原因：

1. batch size 太大。
2. sequence length 太长。
3. activation 占用过高。
4. optimizer state 过大。
5. ZeRO 配置不当。
6. gradient checkpointing 未开启。
7. flash attention 未生效。
8. 显存碎片。
9. eval 阶段未关闭缓存或梯度。

OOM 排查要看发生阶段：forward、backward、optimizer step、eval、checkpoint。

不同阶段对应不同优化。

训练显存可以粗略拆成：

$$M_{\text{train}} = M_{\text{param}} + M_{\text{grad}} + M_{\text{optim}} + M_{\text{act}} + M_{\text{temp}} + M_{\text{fragment}}$$

OOM 阶段门禁：

$$G_{\text{oom_phase}} = I(\text{phase} \in \{\text{forward}, \text{backward}, \text{optimizer}, \text{eval}, \text{checkpoint}\}) \cdot I(M_{\text{peak}} \text{ recorded})$$

不知道 OOM 发生阶段，就很难选择正确修复策略。

45.9 GPU OOM 的常见解决方案

解决方案包括：

1. 降低 micro batch size。
2. 使用 gradient accumulation 保持 global batch。
3. 降低 sequence length。
4. 开启 activation checkpointing。
5. 使用 ZeRO/FSDP。
6. 使用 BF16/FP16。
7. 使用 flash attention。
8. 优化 optimizer state。
9. eval 时使用 no_grad。
10. 清理不必要缓存。

不要盲目降低 global batch size，否则训练动态也会变化。

优先区分 micro batch 和 global batch。

45.10 通信慢怎么排查

分布式训练中，通信慢会导致 step time 变长。

常见原因：

1. 网络带宽不足。
2. IB/RoCE 配置错误。
3. NCCL 参数不合适。

4. 拓扑不佳。
5. 跨机通信过多。
6. 某个节点慢。
7. 网卡错误。
8. all-reduce 数据量过大。

排查指标：

1. step time。
2. compute time。
3. communication time。
4. NCCL all-reduce latency。
5. 网络吞吐。
6. 重传和错误包。
7. GPU 等待时间。

通信慢通常需要结合 profiler、NCCL 日志和网络指标。

通信占比：

$$R_{\text{comm}} = \frac{T_{\text{comm}}}{T_{\text{step}}}$$

straggler 比例：

$$R_{\text{straggle}} = \frac{\max_r T_r - \text{median}_r(T_r)}{\max(\epsilon, \text{median}_r(T_r))}$$

通信慢要同时看 R_{comm} 、rank 间 step time 分布、NCCL 日志和网络错误。

45.11 NCCL 错误

NCCL 错误常见表现：

1. timeout。
2. unhandled system error。
3. connection refused。
4. rank mismatch。
5. collective hang。

可能原因：

1. 某个 rank 崩溃。
2. 节点网络不通。
3. 端口冲突。
4. 环境变量错误。
5. world size 配置错误。
6. GPU 拓扑问题。
7. IB/RoCE 驱动问题。

排查时要找到最早报错的 rank。

最后报 NCCL timeout 的 rank 未必是根因。

45.12 I/O 慢怎么排查

训练 I/O 慢会导致 GPU 等数据。

常见原因：

1. 对象存储吞吐不足。
2. 小文件过多。
3. 网络带宽不足。
4. 解压缩 CPU 瓶颈。
5. tokenizer 在线计算慢。
6. shuffle buffer 太小或太大。
7. dataloader worker 不足。
8. 本地缓存未命中。
9. 共享文件系统抖动。

排查指标：

1. dataloader time。
2. batch ready time。
3. GPU idle time。
4. storage read throughput。
5. cache hit rate。
6. CPU utilization。
7. worker queue length。

I/O 慢往往表现为 GPU 利用率周期性下降。

data wait 比例：

$$R_{\text{data}} = \frac{T_{\text{data}}}{T_{\text{step}}}$$

缓存命中率：

$$H_{\text{cache}} = \frac{N_{\text{cache hit}}}{N_{\text{read}}}$$

R_{data} 高且 H_{cache} 低时，优先排查对象存储、小文件、预处理、worker 和本地缓存。

45.13 GPU 利用率低

GPU 利用率低不一定是 GPU 问题。

可能原因：

1. 数据读取慢。
2. CPU preprocessing 慢。
3. batch 太小。
4. 通信等待。
5. checkpoint 太频繁。
6. evaluation 太频繁。
7. 负载不均衡。
8. kernel 效率低。
9. dataloader hang。

排查要拆分 step time：

step time = data time + forward + backward + optimizer + communication + checkpoint overhead

只有知道哪一段慢，才能优化。

更正式地写：

$$T_{\text{step}} = T_{\text{data}} + T_{\text{fwd}} + T_{\text{bwd}} + T_{\text{optim}} + T_{\text{comm}} + T_{\text{ckpt}} + T_{\text{other}}$$

GPU 利用率低的根因应该落到上面某一项，而不是直接归因成 “GPU 有问题”。

45.14 Checkpoint 故障

Checkpoint 相关故障包括：

1. 保存失败。
2. 保存太慢。
3. 文件不完整。
4. 恢复失败。
5. 状态不一致。
6. 存储空间不足。
7. 多 rank 写入冲突。

排查维度：

1. checkpoint 大小。
2. 保存频率。
3. 存储吞吐。
4. rank 写入策略。
5. manifest 是否完整。
6. checksum 是否通过。
7. optimizer state 是否保存。
8. resume 配置是否匹配。

Checkpoint 是训练容错基础，不能只靠 “目录里看起来有文件”。

checkpoint 完整性门禁：

$$G_{\text{ckpt}} = I(M_{\text{manifest}} = 1) \cdot I(C_{\text{checksum}} = 1) \cdot I(S_{\text{model}} = 1) \cdot I(S_{\text{optim}} = 1) \cdot I(S_{\text{sched}} = 1) \cdot I(S_{\text{rng}} = 1)$$

45.15 Resume 后异常

从 checkpoint 恢复后可能出现：

1. loss 突然跳变。
2. learning rate 不连续。
3. optimizer 状态丢失。
4. dataloader 重复或漏读。
5. random seed 不一致。
6. global step 错误。
7. 分布式状态不一致。

恢复时要校验：

1. model state。
2. optimizer state。
3. scheduler state。
4. global step。
5. random state。
6. dataloader state。
7. config 是否一致。

只加载模型权重，不等于完整恢复训练。

resume 连续性：

$$G_{\text{resume}} = I(\Delta\text{step} = 1) \cdot I(|\text{lr}_{t+1} - \text{lr}_{t+1}^{\text{expected}}| \leq \epsilon) \cdot I(|L_{t+1} - L_t| \leq \rho)$$

45.16 数据问题导致的训练异常

数据问题包括：

1. 空样本。
2. 超长样本。
3. 非法 JSON。
4. 标签错位。
5. tokenizer 结果异常。
6. 特殊 token 错误。
7. 重复样本过多。
8. 数据分布变化。
9. train/eval 泄漏。
10. 敏感数据混入。

排查方法：

1. 保存触发异常的 batch。
2. 打印样本 token 长度分布。
3. 校验 schema。
4. 对比 dataset diff report。
5. 回滚数据版本验证。

数据问题是训练异常中最容易被低估的一类。

数据异常率：

$$R_{\text{data_fault}} = \frac{N_{\text{empty}} + N_{\text{too_long}} + N_{\text{schema_bad}} + N_{\text{label_bad}} + N_{\text{tokenizer_bad}}}{N_{\text{sample}}}$$

45.17 配置变更排查

训练异常常来自配置变更。

需要对比：

1. learning rate。
2. batch size。
3. sequence length。
4. precision。
5. ZeRO/FSDP config。
6. tokenizer。
7. data config。
8. model config。
9. checkpoint config。
10. distributed config。

实验追踪系统应该能显示两个 run 的 config diff。

没有 config diff，排查会非常低效。

配置差异集合：

$$\Delta_{\text{cfg}} = \{k : \text{cfg}_k^{(a)} \neq \text{cfg}_k^{(b)}\}$$

排查时要把 Delta_cfg 和 dataset diff、code commit diff、environment diff 放在同一张证据表里。

45.18 最小复现

复杂故障要尽量最小复现。

方法：

1. 从多机多卡缩到单机多卡。
2. 从大数据缩到小数据。
3. 固定 seed。
4. 固定某个异常 batch。
5. 关闭不必要优化。
6. 简化配置。

如果单卡小数据能复现，多半是代码、数据或配置问题。

如果只有多机才复现，更可能是通信、调度或环境问题。

最小复现就绪门禁：

$$G_{\text{repro}} = I(\text{seed fixed}) \cdot I(\text{batch saved}) \cdot I(\text{config frozen}) \cdot I(\text{data version frozen}) \cdot I(\text{single node trial recorded})$$

45.19 故障定位需要的工具

常用工具和能力：

1. 训练指标 dashboard。
2. GPU metrics。
3. NCCL logs。
4. profiler。
5. dataloader timing。
6. config diff。
7. dataset diff。
8. checkpoint manifest。
9. per-rank logs。
10. event timeline。

平台应把这些整合到 TrainingJob 页面，而不是让工程师到处找。

45.20 故障复盘

训练故障修复后要复盘。

复盘应记录：

1. 故障现象。
2. 影响范围。
3. 根因。
4. 发现方式。
5. 修复方式。
6. 是否可提前告警。
7. 是否需要平台自动防护。
8. 是否需要文档或 runbook。

成熟平台会把常见故障变成自动检测和 runbook。

45.21 训练故障定位审计指标和最小 demo

把本章落到平台验收时，可以用 16 个门禁：

1. Training Fault Evidence Coverage: 事件、rank 日志、loss、梯度、显存、通信、数据 I/O、checkpoint、配置 diff、最小复现和诊断结论是否齐全。
2. Lifecycle Event Timeline: 任务提交、调度、启动、最后推进、异常检测等事件是否按时间线记录。
3. Rank Log Completeness: 所有 rank 是否都有 node、last step、last event、status 和 heartbeat。
4. Loss Anomaly Diagnosis: loss spike 是否绑定触发 step、异常 batch、学习率、precision、梯度和 tokenizer 证据。
5. NaN Inf Guard: loss、logits、gradient 的非有限值检查是否覆盖，并且能在触发时停止或回滚。
6. Update Health Check: 非零梯度比例、optimizer step、learning rate 和参数更新是否证明训练真的在更新。
7. Hang Straggler Detection: step 停止推进、rank heartbeat、collective wait、dataloader 状态和可疑 rank 是否定位清楚。
8. OOM Phase Memory Evidence: OOM 阶段、显存峰值、显存上限、显存拆分和修复建议是否完整。
9. Communication Bottleneck Diagnosis: 通信占比、rank step time 分布、NCCL 日志、网络指标和首个错误 rank 是否齐全。
10. IO Data Bottleneck Diagnosis: data wait、cache hit、worker queue、存储吞吐和坏样本隔离是否齐全。
11. Checkpoint Integrity: manifest、checksum、model、optimizer、scheduler、RNG 和 committed 状态是否完整。
12. Resume Continuity: step、learning rate、loss、optimizer、scheduler 和 dataloader cursor 是否连续。
13. Data Fault Isolation: 空样本、超长样本、schema、label、tokenizer 错误是否统计，并且触发 batch 可隔离。
14. Config Code Diff Readiness: 配置 diff、代码 commit diff、数据 diff 和环境 diff 是否能一起对比。
15. Minimal Reproduction Readiness: seed、异常 batch、配置、数据版本、单机试验和复现命令是否固定。
16. Training Fault Diagnosis Gate: 最终是否有 owner、runbook、postmortem、P0 风险阻断和训练故障定位门禁。

综合门禁可以写成：

$$G_{\text{training_fault}} = \prod_{j=1}^{16} G_j$$

其中每个 G_j 对应上面一个子门禁。训练故障定位题不要只说“看日志”，而要证明证据链能从现象走到根因、修复和复盘。

下面是一个 0 依赖 demo。它不模拟真实分布式训练，而是用 toy incident cases 检查一个训练平台是否能把 loss 异常、NaN、hang、OOM、通信慢、I/O 慢、checkpoint、resume、数据问题、配置变更和最小复现放进同一套诊断门禁。

```
from copy import deepcopy
```

```
class MiniTrainingFaultDiagnosisAudit:
    GATES = [
        "training_fault_evidence_coverage",
        "lifecycle_event_timeline",
        "rank_log_completeness",
        "loss_anomaly_diagnosis",
        "nan_inf_guard",
        "update_health_check",
        "hang_straggler_detection",
        "oom_phase_memory_evidence",
        "communication_bottleneck_diagnosis",
        "io_data_bottleneck_diagnosis",
        "checkpoint_integrity",
```

```

        "resume_continuity",
        "data_fault_isolation",
        "config_code_diff_readiness",
        "minimal_reproduction_readiness",
        "training_fault_diagnosis_gate",
    ]

    EVIDENCE_FIELDS = [
        "events",
        "rank_logs",
        "loss",
        "gradient",
        "memory",
        "communication",
        "data_io",
        "checkpoint",
        "config_diff",
        "reproduction",
        "diagnosis",
    ]

    RANK_LOG_FIELDS = ["rank", "node", "last_step", "last_event", "status", "heartbeat_age_s"]
    VALID_OOM_PHASES = {"forward", "backward", "optimizer", "eval", "checkpoint"}

    @staticmethod
    def present(record, key):
        return key in record and record[key] is not None and record[key] != ""

    def coverage(self, record, fields):
        if not record:
            return 0.0
        return sum(1 for field in fields if self.present(record, field)) / len(fields)

    @staticmethod
    def median(values):
        ordered = sorted(values)
        middle = len(ordered) // 2
        if len(ordered) % 2 == 1:
            return ordered[middle]
        return (ordered[middle - 1] + ordered[middle]) / 2

    def loss_spike(self, case):
        loss = case["loss"]
        values = loss["values"]
        window = values[-loss["window"] - 1:-1]
        baseline = self.median(window)
        return (values[-1] - baseline) / max(1e-12, baseline)

    def nan_inf_rate(self, case):
        item = case["nan_inf"]
        total = item["checked_tensors"]
        bad = item["nan_loss"] + item["inf_logit"] + item["nan_grad"]
        return bad / total if total else 1.0

    def comm_ratio(self, case):

```



```

comm = case["communication"]
return comm["comm_time_s"] / comm["step_time_s"]

def straggler_ratio(self, case):
    times = case["communication"]["rank_step_times_s"]
    med = self.median(times)
    return (max(times) - med) / max(1e-12, med)

def data_wait_ratio(self, case):
    data = case["data_io"]
    return data["data_time_s"] / data["step_time_s"]

def cache_hit_rate(self, case):
    data = case["data_io"]
    return data["cache_hits"] / data["reads"]

def data_fault_rate(self, case):
    data = case["data_fault"]
    bad = data["empty"] + data["too_long"] + data["schema_bad"] + data["label_bad"] + data["tokenizer_bad"]
    return bad / data["samples"]

def training_fault_evidence_coverage(self, case):
    evidence = case.get("evidence", {})
    return self.coverage(evidence, self.EVIDENCE_FIELDS) == 1.0 and all(
        evidence[field] is True for field in self.EVIDENCE_FIELDS
    )

def lifecycle_event_timeline(self, case):
    events = case.get("lifecycle", [])
    names = [event.get("event") for event in events]
    times = [event.get("ts") for event in events]
    required = ["job_submitted", "job_scheduled", "training_started", "last_progress", "incident_detected"]
    return all(name in names for name in required) and times == sorted(times)

def rank_log_completeness(self, case):
    ranks = case.get("ranks", {})
    logs = ranks.get("logs", [])
    logged = {log.get("rank") for log in logs}
    expected = set(range(ranks.get("world_size", 0)))
    fields_ok = all(self.coverage(log, self.RANK_LOG_FIELDS) == 1.0 for log in logs)
    return bool(expected) and logged == expected and fields_ok

def loss_anomaly_diagnosis(self, case):
    loss = case.get("loss", {})
    spike = self.loss_spike(case)
    required = [
        "trigger_step",
        "batch_saved",
        "lr_checked",
        "precision_checked",
        "gradient_checked",
        "tokenizer_checked",
    ]
    if self.coverage(loss, required) < 1.0:
        return False

```

```

    return spike <= loss["spike_threshold"] or all(loss[field] is True for field in required[1:])

def nan_inf_guard(self, case):
    item = case.get("nan_inf", {})
    if item.get("checked_tensors", 0) <= 0:
        return False
    nonfinite = self.nan_inf_rate(case)
    checks_ready = item.get("finite_logits_check") is True and item.get("loss_scale_logged") is True
    return checks_ready and (nonfinite == 0.0 or item.get("stop_on_nonfinite") is True)

def update_health_check(self, case):
    update = case.get("update", {})
    return (
        update.get("nonzero_grad_ratio", 0.0) >= 0.9
        and update.get("optimizer_steps", 0) > 0
        and update.get("lr", 0.0) > 0
        and update.get("parameters_changed") is True
    )

def hang_straggler_detection(self, case):
    hang = case.get("hang", {})
    detected = (
        hang.get("step_delta") == 0
        and hang.get("seconds_since_progress", 0) > hang.get("threshold_seconds", 10**9)
        and hang.get("job_status") == "running"
    )
    heartbeats = hang.get("rank_heartbeats", {})
    return (
        detected
        and bool(heartbeats)
        and hang.get("culprit_rank") in heartbeats
        and hang.get("collective_wait_seen") is True
        and hang.get("dataloader_state_seen") is True
    )

def oom_phase_memory_evidence(self, case):
    memory = case.get("memory", {})
    components = memory.get("components", {})
    peak = memory.get("peak_gib", 0.0)
    limit = memory.get("limit_gib", 0.0)
    return (
        memory.get("oom_phase") in self.VALID_OOM_PHASES
        and peak > 0
        and limit > 0
        and peak <= limit
        and all(value >= 0 for value in components.values())
        and sum(components.values()) <= peak
        and bool(memory.get("remediation"))
    )

def communication_bottleneck_diagnosis(self, case):
    comm = case.get("communication", {})
    return (
        self.comm_ratio(case) >= 0.2
        and len(comm.get("rank_step_times_s", [])) == case["ranks"]["world_size"]
    )

```

```

        and comm.get("nccl_logs_collected") is True
        and comm.get("network_metrics") is True
        and comm.get("first_error_rank") in range(case["ranks"]["world_size"])
    )

def io_data_bottleneck_diagnosis(self, case):
    data = case.get("data_io", {})
    return (
        self.data_wait_ratio(case) >= 0.1
        and self.cache_hit_rate(case) >= 0.8
        and data.get("worker_queue_observed") is True
        and data.get("storage_throughput_mibs", 0) > 0
        and data.get("bad_sample_quarantined") is True
    )

def checkpoint_integrity(self, case):
    checkpoint = case.get("checkpoint", {})
    required = ["manifest", "checksum", "model", "optimizer", "scheduler", "rng", "committed"]
    return all(checkpoint.get(field) is True for field in required)

def resume_continuity(self, case):
    resume = case.get("resume", {})
    lr_ok = abs(resume.get("lr_actual", 0.0) - resume.get("lr_expected", 1.0)) <= 1e-6
    loss_ok = abs(resume.get("loss_after", 0.0) - resume.get("loss_before", 0.0)) <= resume.get("rho", 0.0)
    return (
        resume.get("step_delta") == 1
        and lr_ok
        and loss_ok
        and resume.get("optimizer") is True
        and resume.get("scheduler") is True
        and resume.get("dataloader") is True
    )

def data_fault_isolation(self, case):
    data = case.get("data_fault", {})
    return (
        data.get("samples", 0) > 0
        and self.data_fault_rate(case) <= 0.01
        and data.get("trigger_batch_saved") is True
        and data.get("dataset_diff_report") is True
        and data.get("quarantine") is True
    )

def config_code_diff_readiness(self, case):
    diff = case.get("diff", {})
    return (
        bool(diff.get("config_diff_keys"))
        and diff.get("code_commit_diff") is True
        and diff.get("dataset_diff") is True
        and diff.get("environment_diff") is True
        and bool(diff.get("owner"))
    )

def minimal_reproduction_readiness(self, case):
    repro = case.get("repro", {})

```

```

required = [
    "seed_fixed",
    "batch_saved",
    "config_frozen",
    "data_version_frozen",
    "single_node_trial_recorded",
    "command_captured",
]
return all(repro.get(field) is True for field in required)

def training_fault_diagnosis_gate(self, case):
    gate = case.get("platform_gate", {})
    return (
        gate.get("enabled") is True
        and bool(gate.get("owner"))
        and bool(gate.get("runbook"))
        and gate.get("postmortem_required") is True
        and gate.get("p0_open") is False
    )

def audit_case(self, case):
    return {gate: getattr(self, gate)(case) for gate in self.GATES}

def run_all(self, cases):
    results = {case["case_id"]: self.audit_case(case) for case in cases}
    metrics = {}
    for gate in self.GATES:
        passed = sum(1 for result in results.values() if result[gate])
        metrics[gate] = round(passed / len(cases), 3)
    failed_cases = [
        case_id
        for case_id, result in results.items()
        if not all(result.values())
    ]
    failed_gates = [
        gate
        for gate in self.GATES
        if any(not result[gate] for result in results.values())
    ]
    return {
        "metrics": metrics,
        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "failed_gates": failed_gates,
        "training_fault_diagnosis_gate_pass": metrics["training_fault_diagnosis_gate"] == 1.0,
    }

def example_outputs(self, case):
    ranks = case["ranks"]
    return {
        "evidence_coverage": round(self.coverage(case["evidence"], self.EVIDENCE_FIELDS), 3),
        "timeline_events": len(case["lifecycle"]),
        "rank_log_coverage": round(len(ranks["logs"]) / ranks["world_size"], 3),
        "loss_spike": round(self.loss_spike(case), 3),
        "nan_inf_rate": round(self.nan_inf_rate(case), 3),
    }

```

```

        "nonzero_grad_ratio": case["update"]["nonzero_grad_ratio"],
        "hang_detected": self.hang_straggler_detection(case),
        "oom_phase": case["memory"]["oom_phase"],
        "memory_peak_gib": case["memory"]["peak_gib"],
        "comm_ratio": round(self.comm_ratio(case), 3),
        "straggler_ratio": round(self.straggler_ratio(case), 3),
        "data_wait_ratio": round(self.data_wait_ratio(case), 3),
        "cache_hit_rate": round(self.cache_hit_rate(case), 3),
        "checkpoint_integrity": self.checkpoint_integrity(case),
        "resume_ready": self.resume_continuity(case),
        "data_fault_rate": round(self.data_fault_rate(case), 3),
        "config_diff_keys": case["diff"]["config_diff_keys"],
        "minimal_repro_ready": self.minimal_reproduction_readiness(case),
    }

```

```
def build_good_case():
```

```

    return {
        "case_id": "full_training_fault_diagnosis",
        "evidence": {
            "events": True,
            "rank_logs": True,
            "loss": True,
            "gradient": True,
            "memory": True,
            "communication": True,
            "data_io": True,
            "checkpoint": True,
            "config_diff": True,
            "reproduction": True,
            "diagnosis": True,
        },
        "lifecycle": [
            {"ts": 1, "event": "job_submitted"},
            {"ts": 2, "event": "job_scheduled"},
            {"ts": 3, "event": "training_started"},
            {"ts": 205, "event": "last_progress"},
            {"ts": 236, "event": "incident_detected"},
        ],
        "ranks": {
            "world_size": 4,
            "logs": [
                {"rank": 0, "node": "node-a", "last_step": 205, "last_event": "all_reduce_wait", "status": "running"},
                {"rank": 1, "node": "node-a", "last_step": 205, "last_event": "all_reduce_wait", "status": "running"},
                {"rank": 2, "node": "node-b", "last_step": 205, "last_event": "all_reduce_wait", "status": "running"},
                {"rank": 3, "node": "node-b", "last_step": 204, "last_event": "dataloader_wait", "status": "running"},
            ],
        },
        "loss": {
            "values": [2.4, 2.1, 1.9, 1.8, 3.2],
            "window": 4,
            "spike_threshold": 0.3,
            "trigger_step": 205,
            "batch_saved": True,
            "lr_checked": True,
        }
    }

```

```

    "precision_checked": True,
    "gradient_checked": True,
    "tokenizer_checked": True,
},
"nan_inf": {
    "checked_tensors": 1000,
    "nan_loss": 0,
    "inf_logit": 0,
    "nan_grad": 0,
    "finite_logits_check": True,
    "loss_scale_logged": True,
    "stop_on_nonfinite": True,
},
"update": {
    "nonzero_grad_ratio": 0.94,
    "optimizer_steps": 205,
    "lr": 0.0002,
    "parameters_changed": True,
},
"hang": {
    "step_delta": 0,
    "seconds_since_progress": 900,
    "threshold_seconds": 600,
    "job_status": "running",
    "rank_heartbeats": {0: 20, 1: 22, 2: 18, 3: 840},
    "culprit_rank": 3,
    "collective_wait_seen": True,
    "dataloader_state_seen": True,
},
"memory": {
    "oom_phase": "backward",
    "peak_gib": 75.0,
    "limit_gib": 80.0,
    "components": {"param": 14.0, "grad": 14.0, "optim": 28.0, "activation": 16.0, "temp": 1.5},
    "remediation": ["activation_checkpointing", "micro_batch_down"],
},
"communication": {
    "step_time_s": 8.0,
    "comm_time_s": 2.2,
    "rank_step_times_s": [7.8, 7.9, 8.0, 11.2],
    "nccl_logs_collected": True,
    "network_metrics": True,
    "first_error_rank": 3,
},
"data_io": {
    "data_time_s": 1.4,
    "step_time_s": 8.0,
    "cache_hits": 930,
    "reads": 1000,
    "worker_queue_observed": True,
    "storage_throughput_mibs": 780,
    "bad_sample_quarantined": True,
},
"checkpoint": {
    "manifest": True,

```

```

        "checksum": True,
        "model": True,
        "optimizer": True,
        "scheduler": True,
        "rng": True,
        "committed": True,
    },
    "resume": {
        "step_delta": 1,
        "lr_expected": 0.0002,
        "lr_actual": 0.00020001,
        "loss_before": 1.88,
        "loss_after": 1.91,
        "rho": 0.1,
        "optimizer": True,
        "scheduler": True,
        "data_loader": True,
    },
    "data_fault": {
        "samples": 10000,
        "empty": 2,
        "too_long": 5,
        "schema_bad": 8,
        "label_bad": 4,
        "tokenizer_bad": 1,
        "trigger_batch_saved": True,
        "dataset_diff_report": True,
        "quarantine": True,
    },
    "diff": {
        "config_diff_keys": ["learning_rate", "precision"],
        "code_commit_diff": True,
        "dataset_diff": True,
        "environment_diff": True,
        "owner": "training-platform",
    },
    "repro": {
        "seed_fixed": True,
        "batch_saved": True,
        "config_frozen": True,
        "data_version_frozen": True,
        "single_node_trial_recorded": True,
        "command_captured": True,
    },
    "platform_gate": {
        "enabled": True,
        "owner": "train-debug-oncall",
        "runbook": "runbook://training-fault",
        "postmortem_required": True,
        "p0_open": False,
    },
}

```

```
def build_bad_cases(good_case):
```

```

cases = []

case = deepcopy(good_case)
case["case_id"] = "fault_evidence_missing_bad"
case["evidence"]["diagnosis"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "lifecycle_timeline_bad"
case["lifecycle"][3]["ts"] = 260
case["lifecycle"][4]["ts"] = 236
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "rank_log_missing_bad"
case["ranks"]["logs"].pop()
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "loss_anomaly_diagnosis_bad"
case["loss"]["batch_saved"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "nan_inf_guard_bad"
case["nan_inf"]["finite_logits_check"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "update_health_bad"
case["update"]["nonzero_grad_ratio"] = 0.4
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "hang_straggler_detection_bad"
case["hang"]["culprit_rank"] = None
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "oom_phase_memory_bad"
case["memory"]["oom_phase"] = "unknown"
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "communication_bottleneck_bad"
case["communication"]["nccl_logs_collected"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "io_data_bottleneck_bad"
case["data_io"]["worker_queue_observed"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "checkpoint_integrity_bad"

```



```

case["checkpoint"]["checksum"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "resume_continuity_bad"
case["resume"]["step_delta"] = 5
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "data_fault_isolation_bad"
case["data_fault"]["dataset_diff_report"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "config_code_diff_bad"
case["diff"]["config_diff_keys"] = []
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "minimal_reproduction_bad"
case["repro"]["single_node_trial_recorded"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "training_fault_gate_missing_bad"
case["platform_gate"]["enabled"] = False
cases.append(case)

```

return cases

```

audit = MiniTrainingFaultDiagnosisAudit()
good = build_good_case()
cases = [good] + build_bad_cases(good)
summary = audit.run_all(cases)

print("training_fault_examples=" + repr(audit.example_outputs(good)))
print("metrics=" + repr(summary["metrics"]))
print("hard_blocker_count=" + repr(summary["hard_blocker_count"]))
print("failed_cases=" + repr(summary["failed_cases"]))
print("failed_gates=" + repr(summary["failed_gates"]))
print("training_fault_diagnosis_gate_pass=" + repr(summary["training_fault_diagnosis_gate_pass"]))

```

参考输出应类似：

```

training_fault_examples={'evidence_coverage': 1.0, 'timeline_events': 5, 'rank_log_coverage': 1.0, 'loss_spike': 0.6, 'l
metrics={'training_fault_evidence_coverage': 0.941, 'lifecycle_event_timeline': 0.941, 'rank_log_completeness': 0.941,
hard_blocker_count=16
failed_cases=['fault_evidence_missing_bad', 'lifecycle_timeline_bad', 'rank_log_missing_bad', 'loss_anomaly_diagnosis
failed_gates=['training_fault_evidence_coverage', 'lifecycle_event_timeline', 'rank_log_completeness', 'loss_anomaly_c
training_fault_diagnosis_gate_pass=False

```

这个 demo 的核心是把 “训练故障排查” 从经验性 checklist 变成可验收证据链：现象要有指标，指标要能关联 rank / step / batch / version / checkpoint，修复要能绑定最小复现和复盘，平台门禁要能阻断没有证据的结论。

45.22 常见误区

误区一：loss 异常就是模型结构问题。

也可能是数据、标签、tokenizer、学习率、precision 或 resume 状态问题。

误区二：NCCL timeout 的 rank 就是根因。

根因常常是另一个 rank 先崩溃或卡住。

误区三：GPU 利用率低就加 batch。

如果瓶颈是数据、通信或 checkpoint，加 batch 可能无效甚至 OOM。

误区四：OOM 只靠减 batch。

还可以优化 sequence length、activation checkpointing、ZeRO/FSDP、precision 和 eval 行为。

误区五：checkpoint 能加载就说明恢复正确。

完整恢复还要 optimizer、scheduler、random state、dataloader state 和 global step。

45.23 面试常见追问

问题一：训练 loss NaN 怎么排查？

可以回答：先定位 NaN 出现 step，检查数据 batch、学习率、gradient norm、precision、loss scale、logits/grad 是否 inf，再尝试降学习率、gradient clipping、BF16、保存异常 batch 做最小复现。

问题二：分布式训练 hang 怎么排查？

可以回答：看所有 rank 最后一条日志、step 是否推进、GPU 利用率、NCCL 日志、dataloader 状态、节点事件，重点找最早异常 rank，而不是最后 timeout 的 rank。

问题三：GPU 利用率低可能是什么原因？

可以回答：可能是数据读取慢、CPU preprocessing、batch 太小、通信等待、checkpoint/eval 太频繁、负载不均衡或 kernel 效率低，需要拆分 step time。

问题四：checkpoint resume 后 loss 跳变怎么办？

可以回答：检查是否恢复了 optimizer、scheduler、global step、random state 和 dataloader state，确认 config、数据版本和学习率曲线一致。

45.24 小练习

1. Loss NaN 常见原因有哪些？
2. 为什么小数据 overfit 测试能帮助排查训练问题？
3. 分布式训练 hang 时为什么要看所有 rank 日志？
4. GPU OOM 发生在 forward 和 optimizer step，排查方向有什么不同？
5. 通信慢需要看哪些指标？
6. I/O 慢为什么会导致 GPU 利用率低？
7. 完整 checkpoint resume 需要恢复哪些状态？
8. 如何设计 TrainingJob 故障排查页面？

45.25 本章小结

本章讲了训练故障定位。

你需要记住：

1. 训练故障要按任务事件、日志、指标、资源、数据、通信、checkpoint 和配置逐层排查。
2. Loss 异常可能来自数据、标签、tokenizer、学习率、precision、梯度、代码或 resume 状态。
3. Hang 通常要看所有 rank 日志和分布式通信状态，不能只看最后报错。

4. OOM 要区分发生阶段，优化手段不只有减 batch。
5. GPU 利用率低常常是数据、通信、checkpoint 或调度问题。
6. 完整可观测性和实验追踪能显著降低训练故障定位成本。

下一章我们会讲推理故障定位：TTFT、TPOT、吞吐下降和错误率上升。

第 46 章 推理故障定位：TTFT、TPOT、吞吐下降和错误率上升

上一章讲了训练故障定位。本章讲推理故障定位。

推理服务是在线系统，用户直接感知延迟、失败和质量变化。训练任务可以排队和重试，线上推理不行。推理故障定位的关键，是把端到端请求拆成阶段，并结合模型、runtime、路由、缓存、GPU、KV cache、网络和业务流量一起分析。

先记住一句话：

推理故障定位不要只看 QPS 和 GPU 利用率，而要按 gateway、router、queue、prefill、decode、streaming、cache、runtime 和下游依赖逐段拆解。

46.0 本讲资料边界与第二轮精修口径

本章按通用大模型推理平台故障定位抽象来写，不绑定 vLLM、SGLang、TensorRT-LLM、TGI、Triton Inference Server、Kubernetes、某个云厂商网关或具体 GPU 型号。资料校准时，主要参考 vLLM production metrics 对 TTFT、TPOT、queue、prefill、decode、KV cache、prefix cache 和调度状态的指标口径，参考 Triton Inference Server metrics 对 request、failure、pending、queue、compute 和 first response 的服务端指标拆分，参考 OpenTelemetry trace / span 对跨服务阶段拆解的通用模型，并结合前文推理平台、runtime 选型、Prefill / Decode / KV 资源画像、可观测性和发布治理章节中的 trace、版本、SLO、回滚和成本门禁。

第二轮精修只做三件事：

1. 把 TTFT、TPOT、p99、吞吐下降、错误率、timeout、KV cache、模型加载、streaming、cache、route、tool 和质量异常统一成推理故障样本。
2. 补齐端到端延迟拆分、TTFT、TPOT、尾延迟、token 吞吐、错误分类、timeout stage、KV 压力、stream abort、cache hit、tool failure 和质量漂移公式。
3. 增加一个 0 依赖 Python demo，用 toy inference incident cases 检查推理故障定位是否有完整请求 trace、版本 diff、容量证据和应急动作。

46.1 推理故障的常见类型

推理故障常见包括：

1. TTFT 升高。
2. TPOT 升高。
3. p99 延迟升高。
4. 吞吐下降。
5. 错误率上升。
6. 超时率上升。
7. OOM。
8. KV cache 不足。
9. 模型加载失败。
10. streaming 中断。
11. 路由异常。
12. cache 命中率下降。
13. 工具调用失败。
14. 质量或安全指标异常。

推理故障不仅是系统故障，也可能是模型版本、prompt、路由或数据依赖变化导致。

一个推理故障样本可以写成：

$$I_i = (q_i, s_i, t_i, k_i, m_i, r_i, c_i, e_i, u_i, d_i, p_i, z_i)$$

其中 q_i 是请求与 token 画像, s_i 是影响范围切片, t_i 是阶段 trace, k_i 是 KV cache 证据, m_i 是模型和 runtime 证据, r_i 是路由证据, c_i 是 cache 证据, e_i 是错误和 timeout 证据, u_i 是下游工具证据, d_i 是发布 / 配置 diff, p_i 是应急动作, z_i 是根因和复盘结论。

推理故障证据覆盖率:

$$C_{\text{infer_fault}} = \frac{1}{N} \sum_{i=1}^N I(q_i, s_i, t_i, k_i, m_i, r_i, c_i, e_i, u_i, d_i, p_i, z_i \text{ present})$$

46.2 推理请求链路回顾

一次典型推理请求链路:

Client

- > API Gateway
- > Auth / Rate Limit
- > Safety Input Check
- > Model Router
- > Cache Layer
- > Admission Controller
- > Runtime Queue
- > Prefill
- > Decode
- > Safety Output Check
- > Streaming Response

故障定位时, 要先判断问题发生在哪一段。

端到端慢, 不代表模型慢。

可能是网关排队、路由异常、cache miss、runtime queue 堆积、prefill 变长、decode 变慢或客户端接收慢。

端到端延迟可以拆成:

$$T_{\text{e2e}} = T_{\text{gw}} + T_{\text{auth}} + T_{\text{router}} + T_{\text{cache}} + T_{\text{queue}} + T_{\text{tok}} + T_{\text{prefill}} + T_{\text{decode}} + T_{\text{stream}} + T_{\text{safety}} + T_{\text{down}}$$

故障定位时, 至少要把一次慢请求的每个阶段挂到同一个 trace id 上。

46.3 先看故障范围

排查第一步是确定影响范围。

要问:

1. 是所有模型还是某个模型?
2. 是所有租户还是某个租户?
3. 是所有地域还是某个集群?
4. 是所有请求还是长上下文请求?
5. 是 streaming 还是 non-streaming?
6. 是新版本还是旧版本?
7. 是某个 runtime 类型还是全部 runtime?
8. 是高峰期才发生还是持续发生?

范围判断能快速缩小根因。

如果只有某个模型版本出问题，优先看模型、runtime 配置和路由。

如果全平台都慢，优先看集群、网关、存储、网络或流量突增。

范围切片覆盖率：

$$C_{\text{scope}} = \frac{N_{\text{slice fields}}}{N_{\text{required slice fields}}}$$

必要切片至少包括 model、tenant、region、endpoint、runtime、streaming、input length bucket、output length bucket 和 release version。

46.4 TTFT 升高怎么排查

TTFT 是从请求进入到第一个 token 返回的时间。

TTFT 升高可能来自：

1. 网关排队。
2. 鉴权或限流服务慢。
3. Router 慢。
4. Cache 查询慢或 miss。
5. runtime queue 等待。
6. prefill 输入变长。
7. tokenizer 慢。
8. KV cache 分配慢。
9. 模型实例冷启动。
10. safety input check 慢。

排查指标：

1. gateway latency。
2. router latency。
3. queue wait time。
4. prefill latency。
5. input token length。
6. active requests。
7. KV cache usage。
8. cold start count。

TTFT 是前半段链路的综合指标，必须拆阶段看。

TTFT 可以写成：

$$T_{\text{TTFT}} = T_{\text{gw}} + T_{\text{auth}} + T_{\text{router}} + T_{\text{cache}} + T_{\text{queue}} + T_{\text{tok}} + T_{\text{prefill}} + T_{\text{first_decode}} + T_{\text{first_flush}}$$

如果 T_{TTFT} 升高，但 T_{prefill} 没变，根因更可能在 gateway、router、cache、queue、cold start 或 safety input check。

46.5 TPOT 升高怎么排查

TPOT 是每生成一个输出 token 的平均耗时。

TPOT 升高通常与 decode 阶段有关。

可能原因：

1. decode batch 过大。

2. KV cache 访问变慢。
3. 显存带宽瓶颈。
4. GPU 负载过高。
5. batch 中长序列过多。
6. continuous batching 策略变化。
7. 量化或 runtime 版本变化。
8. streaming flush 策略变化。
9. sampling 参数变化。

排查指标：

1. decode latency。
2. output tokens/s。
3. active sequences。
4. batch size。
5. batch token count。
6. KV cache usage。
7. GPU memory bandwidth 相关指标。
8. runtime version。

TPOT 高，用户会感到输出一顿一顿。

TPOT 可以写成：

$$T_{\text{TPOT}} = \frac{T_{\text{last token}} - T_{\text{first token}}}{\max(1, N_{\text{out}} - 1)}$$

其中 N_{out} 是输出 token 数。TTFT 高不一定 TPOT 高；前者更多受 prefill 和排队影响，后者更多受 decode、KV cache、显存带宽和调度影响。

46.6 p99 延迟升高

p99 升高表示尾部请求变慢。

常见原因：

1. 少量超长请求混入。
2. 某个租户突增。
3. 长短请求混跑。
4. 个别实例拥塞。
5. 某个 endpoint OOM 或重启。
6. cache 命中率下降。
7. downstream tool 慢。
8. 网络抖动。
9. autoscaling 来不及。

排查方法：

1. 按模型切分 p99。
2. 按租户切分 p99。
3. 按 endpoint 切分 p99。
4. 按输入长度切分 p99。
5. 按输出长度切分 p99。
6. 查看慢请求 trace。

平均延迟正常不代表系统健康，线上更要关注尾延迟。

尾延迟放大系数：

$$A_{\text{tail}} = \frac{Q_{0.99}(T_{e2e})}{\max(\epsilon, Q_{0.50}(T_{e2e}))}$$

A_{tail} 明显升高时, 优先按长上下文、租户、endpoint、region、runtime instance 和 tool call 切片找尾部请求。

46.7 吞吐下降怎么排查

吞吐下降可能表现为 QPS 下降或 tokens/s 下降。

要区分:

1. 请求数下降。
2. input tokens/s 下降。
3. output tokens/s 下降。
4. 每 GPU tokens/s 下降。
5. 有效吞吐下降但流量没变。

可能原因:

1. batch 变小。
2. queue 策略变化。
3. GPU 利用率下降。
4. runtime 版本退化。
5. 模型版本变大。
6. 输入输出长度变化。
7. cache 命中率下降。
8. 实例数量减少。
9. 某些实例不健康。

吞吐必须按 token 看, 不要只看 QPS。

token 吞吐可以写成:

$$X_{\text{tok}} = \frac{N_{\text{in}} + N_{\text{out}}}{\Delta t}$$

每 GPU 输出吞吐:

$$X_{\text{out/gpu}} = \frac{N_{\text{out}}}{G \Delta t}$$

其中 G 是 GPU 数。请求 QPS 不变但 $X_{\text{out/gpu}}$ 下降, 通常说明输出长度、decode、KV cache、batch 或 runtime 发生变化。

46.8 错误率上升怎么排查

错误率上升先分类错误。

常见错误类型:

1. 4xx 参数错误。
2. 401/403 权限错误。
3. 429 限流。
4. 5xx runtime 错误。
5. timeout。
6. OOM。

7. model load failed。
8. tokenizer error。
9. safety blocked。
10. tool call failed。

不同错误对应不同处理。

429 上升可能是限流配置或流量突增。

OOM 上升可能是上下文变长或并发过高。

5xx 上升可能是 runtime、模型或依赖异常。

按类别错误率：

$$R_c = \frac{N_c}{N_{\text{request}}}$$

总错误率：

$$R_{\text{err}} = \frac{\sum_c N_c}{N_{\text{request}}}$$

其中 c 可以是 4xx、429、5xx、timeout、OOM、model load、tool failure 或 safety block。错误率上升要先分类，再决定是限流、回滚、扩容、摘除实例还是修复依赖。

46.9 Timeout 上升

Timeout 可能发生在多个阶段：

1. 网关超时。
2. 排队超时。
3. prefill 超时。
4. decode 超时。
5. streaming idle 超时。
6. tool call 超时。
7. safety check 超时。

排查时要看 timeout stage。

如果所有 timeout 都只记录成一个错误码，就很难定位。

Timeout budget 应该在 trace 中记录每个阶段消耗。

timeout stage 覆盖率：

$$C_{\text{timeout}} = \frac{N_{\text{timeout with stage}}}{N_{\text{timeout}}}$$

如果 timeout 都只有一个错误码，C_timeout 很低，系统就无法区分 queue timeout、prefill timeout、decode timeout、streaming idle timeout 和 tool timeout。

46.10 KV Cache 不足

KV cache 不足会导致：

1. admission reject 上升。
2. OOM。

3. p99 抖动。
4. 长上下文请求失败。
5. active sequence 上限提前触达。

可能原因：

1. 输入上下文变长。
2. 输出变长。
3. 并发变高。
4. batch 策略变化。
5. prefix cache 保留太多。
6. KV block 碎片。
7. 模型层数或 hidden size 变大。

排查指标：

1. KV cache usage。
2. free blocks。
3. active sequences。
4. average context length。
5. eviction count。
6. OOM count。

KV cache 是大模型推理特有的重要瓶颈。

KV cache 粗略显存预算：

$$M_{kv} = 2L_{layer}H_{kv}d_hdN_{token}$$

其中 2 表示 K 和 V, L_{layer} 是层数, H_{kv} 是 KV head 数, d_h 是 head dimension, b 是每个元素字节数, N_{token} 是当前保留的 KV token 数。

KV 压力：

$$P_{kv} = \frac{M_{kv,used}}{M_{kv,budget}}$$

当 P_{kv} 接近 1 时, admission reject、OOM、eviction 和 p99 抖动都会变多。

46.11 模型加载失败

模型加载失败常见原因：

1. 权重文件缺失。
2. checksum 不匹配。
3. tokenizer 缺失或不匹配。
4. config 错误。
5. runtime 不支持该模型结构。
6. 量化格式不兼容。
7. CUDA 或 driver 版本不匹配。
8. 显存不足。
9. 权限不足。
10. 权重下载超时。

排查要看 deployment package、model registry、runtime logs 和节点环境。

不要只看 Pod restart。

模型加载门禁：

$$G_{\text{load}} = I(M_{\text{manifest}} = 1) \cdot I(C_{\text{weight}} = 1) \cdot I(C_{\text{tokenizer}} = 1) \cdot I(C_{\text{runtime}} = 1) \cdot I(C_{\text{permission}} = 1)$$

如果模型加载失败，排查要从 model registry、deployment package、checksum、tokenizer、runtime compatibility、量化格式、权限和节点环境一起看。

46.12 Streaming 中断

Streaming 中断可能来自：

1. 客户端断开。
2. 网关 idle timeout。
3. 后端 decode 超时。
4. 网络抖动。
5. runtime 崩溃。
6. safety output check 中断。
7. 代理层缓冲问题。

排查指标：

1. stream open count。
2. stream duration。
3. client disconnect count。
4. server abort count。
5. tokens sent before abort。
6. idle timeout count。

Streaming 问题要区分服务端生成慢和客户端消费慢。

stream abort rate：

$$R_{\text{abort}} = \frac{N_{\text{server abort}} + N_{\text{client disconnect}} + N_{\text{idle timeout}}}{N_{\text{stream}}}$$

如果 abort 主要来自 client disconnect, 优先看客户端网络和消费速度; 如果来自 server abort 或 idle timeout, 优先看 decode、gateway、proxy buffer、timeout budget 和 runtime。

46.13 Cache 命中率下降

Cache 命中率下降会导致延迟和成本上升。

可能原因：

1. prompt 模板变化。
2. model version 变化。
3. tokenizer 变化。
4. cache key 变化。
5. TTL 过短。
6. cache 被驱逐。
7. 流量分布变化。
8. 权限域变化。

排查要看：

1. prompt cache hit rate。
2. semantic cache hit rate。
3. result cache hit rate。
4. cache key diff。

5. eviction rate。
6. latency before/after cache。

缓存问题常常在发布后出现，因为 prompt、模型或安全策略版本变化会影响 key。

cache hit rate:

$$H_{\text{cache}} = \frac{N_{\text{hit}}}{N_{\text{lookup}}}$$

缓存节省的 prefill 时间可以近似写成：

$$S_{\text{cache}} = N_{\text{hit}} \bar{T}_{\text{prefill saved}}$$

Cache 命中率下降时，要看 cache key 是否绑定 model、tokenizer、prompt、safety policy、tenant 和 permission 版本。

46.14 路由异常

路由异常可能表现为：

1. 请求打到错误模型。
2. 灰度比例异常。
3. 高优先级租户被降级。
4. 过载实例仍然接流量。
5. fallback 频繁触发。
6. 成本突然上升。

排查需要 route trace。

Route trace 应记录：

1. 候选模型。
2. 选择模型。
3. endpoint。
4. route reason。
5. 灰度规则。
6. fallback chain。
7. 实时负载。
8. 租户策略。

没有 route trace，路由问题很难复盘。

路由正确性门禁：

$$G_{\text{route}} = I(m_{\text{selected}} \in M_{\text{candidate}}) \cdot I(P_{\text{tenant}} = 1) \cdot I(H_{\text{endpoint}} = 1) \cdot I(C_{\text{trace}} = 1)$$

其中 P_tenant 是租户策略满足，H_endpoint 是 endpoint 健康，C_trace 是 route trace 完整。

46.15 工具调用失败

Agent 或 tool-use 场景中，错误率可能来自工具。

常见原因：

1. tool schema 变化。
2. 参数生成错误。
3. 工具权限失败。

4. 工具超时。
5. 工具返回格式变化。
6. 下游服务不可用。
7. 重试策略错误。

排查要看 tool call trace:

1. tool name/version。
2. input arguments。
3. output。
4. status。
5. latency。
6. error message。
7. retry count。

不要把所有 Agent 失败都归因于模型。

工具失败率:

$$R_{\text{tool}} = \frac{N_{\text{tool fail}}}{N_{\text{tool call}}}$$

工具失败率上升时, 要看 schema 版本、权限决策、timeout、retry、下游状态和 tool call trace, 而不是只看最终 HTTP 状态码。

46.16 质量异常怎么排查

推理服务可能系统指标正常, 但质量下降。

可能原因:

1. 模型版本变化。
2. prompt 版本变化。
3. retrieval index 变化。
4. reranker 变化。
5. tool description 变化。
6. safety 策略变化。
7. sampling 参数变化。
8. fallback 到小模型。

排查需要:

1. model version。
2. prompt version。
3. route trace。
4. retrieval trace。
5. tool trace。
6. eval/online feedback。

HTTP 200 不代表模型服务质量正常。

质量漂移可以写成:

$$\Delta Q = Q_{\text{current}} - Q_{\text{baseline}}$$

当 ΔQ 为负且超过阈值时, 要把 model、prompt、retrieval、rerank、tool、safety、sampling、fallback 和 release event 放进同一张 diff 表。

46.17 发布后故障

发布后常见故障：

1. 新模型延迟更高。
2. 新 tokenizer 不兼容。
3. prompt cache 失效。
4. 输出格式变化。
5. tool call 参数变化。
6. 安全拒答率变化。
7. 成本上升。
8. 灰度分桶异常。

发布后排查应先看变更：

1. model version。
2. tokenizer。
3. prompt。
4. runtime。
5. route policy。
6. safety policy。
7. cache policy。

变更时间线和事件记录非常重要。

发布差异集合：

$$\Delta_{\text{release}} = \{k : v_k^{\text{new}} \neq v_k^{\text{old}}\}$$

推理故障常常是多个小变更共同导致，例如 prompt 变化让 cache miss，runtime 变化让 TPOT 升高，route policy 变化又把长请求集中到同一组实例。

46.18 推理故障排查 Runbook

一个通用 runbook：

1. 确认影响范围。
2. 查看最近变更和发布事件。
3. 按模型、租户、endpoint、地域切分指标。
4. 查看 TTFT、TPOT、p99、error、timeout。
5. 查看 queue、KV cache、GPU、active sequences。
6. 抽样慢请求 trace。
7. 查看 runtime logs。
8. 查看 route/cache/tool/retrieval trace。
9. 判断是否需要限流、降级、回滚或扩容。
10. 记录复盘。

Runbook 的价值是减少线上故障时的临场猜测。

46.19 应急处理手段

常见应急手段：

1. 限流异常租户。
2. 降级到小模型。
3. 回滚模型版本。
4. 关闭有问题的灰度。
5. 摘除异常 endpoint。
6. 扩容实例。

7. 降低 max output tokens。
8. 限制最大上下文长度。
9. 启用缓存兜底。
10. 暂停高风险工具调用。

应急处理要优先恢复服务，再深入定位根因。

46.20 推理故障定位审计指标和最小 demo

把本章落到平台验收时，可以用 16 个门禁：

1. Inference Fault Evidence Coverage: 请求画像、影响范围、阶段 trace、KV、模型、路由、cache、错误、工具、发布 diff、应急动作和诊断是否齐全。
2. Request Scope Slice: 是否按 model、tenant、region、endpoint、runtime、streaming、输入长度、输出长度和版本切片。
3. Trace Stage Coverage: gateway、auth、router、cache、queue、tokenize、prefill、first decode、decode、stream、safety 和 downstream 是否有 span。
4. TTFT Decomposition: TTFT 是否能拆到 queue、tokenize、prefill、first decode 和 first flush，并有 SLO 与根因解释。
5. TPOT Decode Health: TPOT 是否能绑定 decode time、输出 token、active sequences、KV 和 runtime 版本。
6. Tail Latency Attribution: p99 / p50 是否可计算，尾部慢请求是否有 trace sample 和切片归因。
7. Throughput Token Capacity: QPS、input tokens/s、output tokens/s、per GPU output tokens/s 是否都达标。
8. Error Taxonomy Timeout Stage: 错误是否按 4xx、429、5xx、timeout、OOM、tool、safety 分类，timeout 是否有阶段。
9. KV Cache Pressure Guard: KV pressure、free blocks、eviction、admission reject、fragmentation 是否可判断。
10. Model Artifact Load Readiness: manifest、checksum、tokenizer、config、runtime、quantization、权限和 warmup 是否完整。
11. Streaming Reliability: server abort、client disconnect、idle timeout、tokens before abort 和 cancellation cleanup 是否可观测。
12. Cache Hit Key Governance: cache hit、key version、prompt / tokenizer / safety / tenant 版本和 eviction 是否可解释。
13. Route Trace Correctness: 候选模型、选择模型、路由原因、灰度比例、租户策略、endpoint 健康和 fallback 是否在 trace 中。
14. Tool Dependency Trace: tool schema、权限、timeout、retry、下游错误和 tool call trace 是否完整。
15. Quality Release Diff Readiness: 模型、prompt、runtime、route、cache、safety、sampling diff 与质量指标是否关联。
16. Inference Fault Diagnosis Gate: 最终是否有 owner、runbook、应急动作、回滚、postmortem 和 P0 风险阻断。

综合门禁：

$$G_{\text{inference_fault}} = \prod_{j=1}^{16} G_j$$

下面是一个 0 依赖 demo，用 toy inference incident cases 检查推理故障定位是否只是“看几条 dashboard 曲线”，还是能真正把慢请求、错误、容量、版本和应急动作串成证据链。

from copy import deepcopy

```
class MiniInferenceFaultDiagnosisAudit:
    GATES = [
        "inference_fault_evidence_coverage",
```

```

        "request_scope_slice",
        "trace_stage_coverage",
        "ttft_decomposition",
        "tpot_decode_health",
        "tail_latency_attribution",
        "throughput_token_capacity",
        "error_taxonomy_timeout_stage",
        "kv_cache_pressure_guard",
        "model_artifact_load_readiness",
        "streaming_reliability",
        "cache_hit_key_governance",
        "route_trace_correctness",
        "tool_dependency_trace",
        "quality_release_diff_readiness",
        "inference_fault_diagnosis_gate",
    ]

```

```

EVIDENCE_FIELDS = [
    "request_profile",
    "scope",
    "trace",
    "kv_cache",
    "model_runtime",
    "route",
    "cache",
    "errors",
    "tool",
    "release_diff",
    "mitigation",
    "diagnosis",
]

```

```

REQUIRED_SCOPE_FIELDS = [
    "model",
    "tenant",
    "region",
    "endpoint",
    "runtime",
    "streaming",
    "input_length_bucket",
    "output_length_bucket",
    "release_version",
]

```

```

REQUIRED_STAGES = [
    "gateway",
    "auth",
    "router",
    "cache",
    "queue",
    "tokenize",
    "prefill",
    "first_decode",
    "first_flush",
    "decode",
    "stream",
    "safety",
]

```

```

        "downstream",
    ]

    @staticmethod
    def present(record, key):
        return key in record and record[key] is not None and record[key] != ""

    def coverage(self, record, fields):
        if not record:
            return 0.0
        return sum(1 for field in fields if self.present(record, field)) / len(fields)

    def stage_map(self, case):
        return {stage["name"]: stage["ms"] for stage in case.get("trace", {}).get("stages", [])}

    def ttft_ms(self, case):
        stages = self.stage_map(case)
        names = [
            "gateway",
            "auth",
            "router",
            "cache",
            "queue",
            "tokenize",
            "prefill",
            "first_decode",
            "first_flush",
        ]
        return sum(stages[name] for name in names)

    def tpot_ms(self, case):
        trace = case["trace"]
        return trace["decode_total_ms"] / max(1, trace["output_tokens"] - 1)

    def tail_amplification(self, case):
        latency = case["latency"]
        return latency["p99_ms"] / max(1e-12, latency["p50_ms"])

    def total_tokens_per_s(self, case):
        traffic = case["traffic"]
        return (traffic["input_tokens"] + traffic["output_tokens"]) / traffic["window_s"]

    def per_gpu_output_tokens_s(self, case):
        traffic = case["traffic"]
        return traffic["output_tokens"] / (traffic["gpu_count"] * traffic["window_s"])

    def error_rate(self, case):
        errors = case["errors"]
        return sum(errors["counts"].values()) / errors["requests"]

    def timeout_stage_coverage(self, case):
        errors = case["errors"]
        timeout_count = errors["counts"].get("timeout", 0)
        if timeout_count == 0:
            return 1.0

```



```

    return sum(errors["timeout_stages"].values()) / timeout_count

def kv_pressure(self, case):
    kv = case["kv_cache"]
    return kv["used_gib"] / kv["budget_gib"]

def stream_abort_rate(self, case):
    stream = case["streaming"]
    aborts = stream["server_abort"] + stream["client_disconnect"] + stream["idle_timeout"]
    return aborts / stream["open_streams"]

def cache_hit_rate(self, case):
    cache = case["cache"]
    return cache["hits"] / cache["lookups"]

def tool_failure_rate(self, case):
    tool = case["tool"]
    return tool["failures"] / tool["calls"]

def quality_delta(self, case):
    quality = case["quality"]
    return quality["current_score"] - quality["baseline_score"]

def inference_fault_evidence_coverage(self, case):
    evidence = case.get("evidence", {})
    return self.coverage(evidence, self.EVIDENCE_FIELDS) == 1.0 and all(
        evidence[field] is True for field in self.EVIDENCE_FIELDS
    )

def request_scope_slice(self, case):
    scope = case.get("scope", {})
    return self.coverage(scope, self.REQUIRED_SCOPE_FIELDS) == 1.0

def trace_stage_coverage(self, case):
    stages = set(self.stage_map(case))
    return set(self.REQUIRED_STAGES).issubset(stages) and bool(case.get("trace", {}).get("trace_id"))

def ttft_decomposition(self, case):
    trace = case.get("trace", {})
    ttft = self.ttft_ms(case)
    return (
        ttft <= trace.get("ttft_slo_ms", 0)
        or (
            trace.get("ttft_root_cause_ready") is True
            and trace.get("queue_ms_explained") is True
            and trace.get("prefill_ms_explained") is True
        )
    )

def tpot_decode_health(self, case):
    decode = case.get("decode", {})
    tpot = self.tpot_ms(case)
    return (
        tpot <= decode.get("tpot_slo_ms", 0)
        or (

```

```

        decode.get("decode_root_cause_ready") is True
        and decode.get("kv_evidence_ready") is True
        and decode.get("runtime_version_bound") is True
    )
)

def tail_latency_attribution(self, case):
    latency = case.get("latency", {})
    return (
        latency.get("p99_ms", 10**9) <= latency.get("p99_slo_ms", 0)
        or (
            latency.get("slow_trace_sampled") is True
            and latency.get("slice_attribution_ready") is True
            and self.tail_amplification(case) <= latency.get("max_tail_amplification", 10**9)
        )
    )

def throughput_token_capacity(self, case):
    traffic = case.get("traffic", {})
    return (
        traffic.get("qps", 0.0) >= traffic.get("qps_target", 10**9)
        and self.total_tokens_per_s(case) >= traffic.get("total_tokens_s_target", 10**9)
        and self.per_gpu_output_tokens_s(case) >= traffic.get("per_gpu_output_tokens_s_target", 10**9)
    )

def error_taxonomy_timeout_stage(self, case):
    errors = case.get("errors", {})
    return (
        errors.get("taxonomy_ready") is True
        and self.error_rate(case) <= errors.get("max_error_rate", 0.0)
        and self.timeout_stage_coverage(case) >= 0.95
    )

def kv_cache_pressure_guard(self, case):
    kv = case.get("kv_cache", {})
    return (
        self.kv_pressure(case) <= kv.get("max_pressure", 0.0)
        and kv.get("free_blocks", 0) > kv.get("min_free_blocks", 10**9)
        and kv.get("eviction_count", 10**9) <= kv.get("max_evictions", 0)
        and kv.get("admission_reject_rate", 1.0) <= kv.get("max_admission_reject_rate", 0.0)
        and kv.get("fragmentation", 1.0) <= kv.get("max_fragmentation", 0.0)
    )

def model_artifact_load_readiness(self, case):
    model = case.get("model_runtime", {})
    required = [
        "manifest",
        "weight_checksum",
        "tokenizer",
        "config",
        "runtime_compatible",
        "quantization_compatible",
        "permission",
        "warmup_passed",
    ]

```

```

        return all(model.get(field) is True for field in required) and model.get("load_latency_ms", 10**9) <= model.g

def streaming_reliability(self, case):
    stream = case.get("streaming", {})
    return (
        stream.get("open_streams", 0) > 0
        and self.stream_abort_rate(case) <= stream.get("max_abort_rate", 0.0)
        and stream.get("cancellation_cleanup") is True
        and stream.get("tokens_before_abort_median", 0) >= stream.get("min_tokens_before_abort_median", 0)
    )

def cache_hit_key_governance(self, case):
    cache = case.get("cache", {})
    return (
        self.cache_hit_rate(case) >= cache.get("min_hit_rate", 1.0)
        and cache.get("key_versions_bound") is True
        and cache.get("prompt_version_bound") is True
        and cache.get("tokenizer_version_bound") is True
        and cache.get("safety_version_bound") is True
        and cache.get("tenant_permission_bound") is True
        and cache.get("eviction_rate", 1.0) <= cache.get("max_eviction_rate", 0.0)
    )

def route_trace_correctness(self, case):
    route = case.get("route", {})
    canary_gap = abs(route.get("actual_canary_percent", 0.0) - route.get("expected_canary_percent", 0.0))
    return (
        route.get("selected_model") in route.get("candidate_models", [])
        and route.get("selected_model") == route.get("expected_model")
        and bool(route.get("route_reason"))
        and canary_gap <= route.get("max_canary_gap", 0.0)
        and route.get("tenant_policy_ok") is True
        and route.get("endpoint_health_ok") is True
        and route.get("trace_complete") is True
    )

def tool_dependency_trace(self, case):
    tool = case.get("tool", {})
    return (
        tool.get("calls", 0) > 0
        and self.tool_failure_rate(case) <= tool.get("max_failure_rate", 0.0)
        and tool.get("schema_version_bound") is True
        and tool.get("permission_decisions_recorded") is True
        and tool.get("timeout_budget_recorded") is True
        and tool.get("retry_policy_recorded") is True
        and tool.get("trace_complete") is True
    )

def quality_release_diff_readiness(self, case):
    quality = case.get("quality", {})
    diff = case.get("release_diff", {})
    required = ["model", "prompt", "runtime", "route", "cache", "safety", "sampling"]
    return (
        self.quality_delta(case) >= -quality.get("max_allowed_drop", 0.0)
        and all(field in diff.get("changed_fields", []) for field in required)
    )

```

```

        and quality.get("eval_feedback_linked") is True
        and diff.get("release_event_timeline") is True
        and diff.get("rollback_ready") is True
    )

def inference_fault_diagnosis_gate(self, case):
    gate = case.get("platform_gate", {})
    return (
        gate.get("enabled") is True
        and bool(gate.get("owner"))
        and bool(gate.get("runbook"))
        and bool(gate.get("mitigation_actions"))
        and gate.get("rollback_ready") is True
        and gate.get("postmortem_required") is True
        and gate.get("p0_open") is False
    )

def audit_case(self, case):
    return {gate: getattr(self, gate)(case) for gate in self.GATES}

def run_all(self, cases):
    results = {case["case_id"]: self.audit_case(case) for case in cases}
    metrics = {}
    for gate in self.GATES:
        passed = sum(1 for result in results.values() if result[gate])
        metrics[gate] = round(passed / len(cases), 3)
    failed_cases = [
        case_id
        for case_id, result in results.items()
        if not all(result.values())
    ]
    failed_gates = [
        gate
        for gate in self.GATES
        if any(not result[gate] for result in results.values())
    ]
    return {
        "metrics": metrics,
        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "failed_gates": failed_gates,
        "inference_fault_diagnosis_gate_pass": metrics["inference_fault_diagnosis_gate"] == 1.0,
    }

def example_outputs(self, case):
    return {
        "evidence_coverage": round(self.coverage(case["evidence"], self.EVIDENCE_FIELDS), 3),
        "scope_coverage": round(self.coverage(case["scope"], self.REQUIRED_SCOPE_FIELDS), 3),
        "stage_coverage": round(len(self.stage_map(case)) / len(self.REQUIRED_STAGES), 3),
        "ttft_ms": self.ttft_ms(case),
        "tpot_ms": round(self.tpot_ms(case), 3),
        "p99_ms": case["latency"]["p99_ms"],
        "tail_amplification": round(self.tail_amplification(case), 3),
        "tokens_per_s": round(self.total_tokens_per_s(case), 3),
        "per_gpu_output_tokens_s": round(self.per_gpu_output_tokens_s(case), 3),
    }

```

```

        "error_rate": round(self.error_rate(case), 3),
        "timeout_stage_coverage": round(self.timeout_stage_coverage(case), 3),
        "kv_pressure": round(self.kv_pressure(case), 3),
        "model_load_ready": self.model_artifact_load_readiness(case),
        "stream_abort_rate": round(self.stream_abort_rate(case), 3),
        "cache_hit_rate": round(self.cache_hit_rate(case), 3),
        "route_correct": self.route_trace_correctness(case),
        "tool_failure_rate": round(self.tool_failure_rate(case), 3),
        "quality_delta": round(self.quality_delta(case), 3),
    }

```

```

def build_good_case():
    return {
        "case_id": "full_inference_fault_diagnosis",
        "evidence": {
            "request_profile": True,
            "scope": True,
            "trace": True,
            "kv_cache": True,
            "model_runtime": True,
            "route": True,
            "cache": True,
            "errors": True,
            "tool": True,
            "release_diff": True,
            "mitigation": True,
            "diagnosis": True,
        },
        "scope": {
            "model": "chat-v8",
            "tenant": "all",
            "region": "us-east",
            "endpoint": "chat",
            "runtime": "vllm",
            "streaming": True,
            "input_length_bucket": "4k-8k",
            "output_length_bucket": "256-512",
            "release_version": "rel_2026_06_15",
        },
        "trace": {
            "trace_id": "tr_infer_1",
            "stages": [
                {"name": "gateway", "ms": 40},
                {"name": "auth", "ms": 20},
                {"name": "router", "ms": 35},
                {"name": "cache", "ms": 45},
                {"name": "queue", "ms": 180},
                {"name": "tokenize", "ms": 30},
                {"name": "prefill", "ms": 280},
                {"name": "first_decode", "ms": 45},
                {"name": "first_flush", "ms": 45},
                {"name": "decode", "ms": 580},
                {"name": "stream", "ms": 60},
                {"name": "safety", "ms": 35},
            ]
        }
    }

```

```

        {"name": "downstream", "ms": 20},
    ],
    "decode_total_ms": 580,
    "output_tokens": 11,
    "ttft_slo_ms": 900,
    "ttft_root_cause_ready": True,
    "queue_ms_explained": True,
    "prefill_ms_explained": True,
},
"decode": {
    "tpot_slo_ms": 80,
    "active_sequences": 96,
    "decode_root_cause_ready": True,
    "kv_evidence_ready": True,
    "runtime_version_bound": True,
},
"latency": {
    "p50_ms": 720,
    "p99_ms": 1900,
    "p99_slo_ms": 2000,
    "max_tail_amplification": 3.0,
    "slow_trace_sampled": True,
    "slice_attribution_ready": True,
},
"traffic": {
    "qps": 10.0,
    "qps_target": 9.0,
    "window_s": 100,
    "input_tokens": 300000,
    "output_tokens": 220000,
    "gpu_count": 8,
    "total_tokens_s_target": 5000,
    "per_gpu_output_tokens_s_target": 250,
},
"errors": {
    "requests": 10000,
    "counts": {"4xx": 40, "429": 60, "5xx": 35, "timeout": 30, "oom": 10, "tool": 5},
    "timeout_stages": {"queue": 12, "decode": 18},
    "taxonomy_ready": True,
    "max_error_rate": 0.02,
},
"kv_cache": {
    "used_gib": 65.6,
    "budget_gib": 80.0,
    "max_pressure": 0.9,
    "free_blocks": 320,
    "min_free_blocks": 200,
    "eviction_count": 2,
    "max_evictions": 5,
    "admission_reject_rate": 0.01,
    "max_admission_reject_rate": 0.02,
    "fragmentation": 0.08,
    "max_fragmentation": 0.15,
},
"model_runtime": {

```

```

    "manifest": True,
    "weight_checksum": True,
    "tokenizer": True,
    "config": True,
    "runtime_compatible": True,
    "quantization_compatible": True,
    "permission": True,
    "warmup_passed": True,
    "load_latency_ms": 90000,
    "load_slo_ms": 120000,
  },
  "streaming": {
    "open_streams": 5000,
    "server_abort": 40,
    "client_disconnect": 20,
    "idle_timeout": 0,
    "max_abort_rate": 0.02,
    "tokens_before_abort_median": 64,
    "min_tokens_before_abort_median": 32,
    "cancellation_cleanup": True,
  },
  "cache": {
    "hits": 4200,
    "lookups": 10000,
    "min_hit_rate": 0.4,
    "key_versions_bound": True,
    "prompt_version_bound": True,
    "tokenizer_version_bound": True,
    "safety_version_bound": True,
    "tenant_permission_bound": True,
    "eviction_rate": 0.03,
    "max_eviction_rate": 0.05,
    "latency_saved_ms": 180,
  },
  "route": {
    "candidate_models": ["chat-v8", "chat-v7-small"],
    "selected_model": "chat-v8",
    "expected_model": "chat-v8",
    "route_reason": "quality_slo_primary",
    "expected_canary_percent": 5.0,
    "actual_canary_percent": 5.2,
    "max_canary_gap": 0.5,
    "tenant_policy_ok": True,
    "endpoint_health_ok": True,
    "fallback_chain": ["chat-v7-small"],
    "trace_complete": True,
  },
  "tool": {
    "calls": 500,
    "failures": 10,
    "max_failure_rate": 0.03,
    "schema_version_bound": True,
    "permission_decisions_recorded": True,
    "timeout_budget_recorded": True,
    "retry_policy_recorded": True,
  },

```

```

        "trace_complete": True,
    },
    "quality": {
        "baseline_score": 0.86,
        "current_score": 0.845,
        "max_allowed_drop": 0.03,
        "eval_feedback_linked": True,
    },
    "release_diff": {
        "changed_fields": ["model", "prompt", "runtime", "route", "cache", "safety", "sampling"],
        "release_event_timeline": True,
        "rollback_ready": True,
    },
    "platform_gate": {
        "enabled": True,
        "owner": "serving-oncall",
        "runbook": "runbook://inference-fault",
        "mitigation_actions": ["rate_limit_spike", "rollback_cache_policy"],
        "rollback_ready": True,
        "postmortem_required": True,
        "p0_open": False,
    },
}

```

```

def build_bad_cases(good_case):
    cases = []

    case = deepcopy(good_case)
    case["case_id"] = "inference_evidence_missing_bad"
    case["evidence"]["diagnosis"] = False
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "scope_slice_missing_bad"
    case["scope"].pop("runtime")
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "trace_stage_missing_bad"
    case["trace"]["stages"] = [stage for stage in case["trace"]["stages"] if stage["name"] != "downstream"]
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "ttft_decomposition_bad"
    for stage in case["trace"]["stages"]:
        if stage["name"] == "queue":
            stage["ms"] = 520
    case["trace"]["ttft_root_cause_ready"] = False
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "tpot_decode_bad"
    case["trace"]["decode_total_ms"] = 980
    case["decode"]["decode_root_cause_ready"] = False

```



```

cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "tail_latency_unattributed_bad"
case["latency"]["p99_ms"] = 2600
case["latency"]["slow_trace_sampled"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "throughput_capacity_bad"
case["traffic"]["output_tokens"] = 120000
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "error_taxonomy_bad"
case["errors"]["taxonomy_ready"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "kv_pressure_bad"
case["kv_cache"]["used_gib"] = 77.5
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "model_artifact_load_bad"
case["model_runtime"]["weight_checksum"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "streaming_reliability_bad"
case["streaming"]["cancellation_cleanup"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "cache_key_governance_bad"
case["cache"]["key_versions_bound"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "route_trace_bad"
case["route"]["selected_model"] = "chat-v7-small"
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "tool_trace_bad"
case["tool"]["trace_complete"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "quality_release_diff_bad"
case["quality"]["current_score"] = 0.79
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "inference_fault_gate_missing_bad"

```

```
case["platform_gate"]["enabled"] = False
cases.append(case)
```

```
return cases
```

```
audit = MiniInferenceFaultDiagnosisAudit()
good = build_good_case()
cases = [good] + build_bad_cases(good)
summary = audit.run_all(cases)
```

```
print("inference_fault_examples=" + repr(audit.example_outputs(good)))
print("metrics=" + repr(summary["metrics"]))
print("hard_blocker_count=" + repr(summary["hard_blocker_count"]))
print("failed_cases=" + repr(summary["failed_cases"]))
print("failed_gates=" + repr(summary["failed_gates"]))
print("inference_fault_diagnosis_gate_pass=" + repr(summary["inference_fault_diagnosis_gate_pass"]))
```

参考输出应类似：

```
inference_fault_examples={'evidence_coverage': 1.0, 'scope_coverage': 1.0, 'stage_coverage': 1.0, 'ttft_ms': 720, 'tpot': 0.015}
metrics={'inference_fault_evidence_coverage': 0.941, 'request_scope_slice': 0.941, 'trace_stage_coverage': 0.941, 'ttft_decomposition': 0.941}
hard_blocker_count=16
failed_cases=['inference_evidence_missing_bad', 'scope_slice_missing_bad', 'trace_stage_missing_bad', 'ttft_decomposition_missing_bad']
failed_gates=['inference_fault_evidence_coverage', 'request_scope_slice', 'trace_stage_coverage', 'ttft_decomposition']
inference_fault_diagnosis_gate_pass=False
```

这个 demo 的面试价值是把推理故障定位从“看 QPS 和 GPU utilization”升级为请求级证据链：TTFT 拆前半段，TPOT 拆 decode，p99 拆尾部切片，吞吐按 token 归一化，错误按类别和阶段归因，KV、cache、route、tool、模型版本和 release diff 都能落到同一个 trace 里。

46.21 常见误区

误区一：GPU 利用率高就是过载。

要结合 queue、TTFT、TPOT、KV cache 和错误率判断。

误区二：TTFT 高就是模型慢。

TTFT 可能来自网关、路由、排队、cache miss、prefill 或冷启动。

误区三：错误率上升就是 runtime bug。

可能是限流、权限、输入参数、工具、模型加载或安全策略。

误区四：吞吐下降只看 QPS。

要看 input/output tokens/s 和每 GPU tokens/s。

误区五：质量问题不属于故障。

模型输出质量、安全、工具调用和 RAG 引用错误都是生产故障的一部分。

46.22 面试常见追问

问题一：TTFT 突然升高怎么排查？

可以回答：拆分 gateway、auth、router、cache、queue、tokenizer、prefill 和 safety 阶段，重点看 queue wait、input tokens、prefill latency、KV cache、cold start 和流量变化。

问题二：TPOT 升高说明什么？

可以回答：通常说明 decode 阶段变慢，可能与 batch、KV cache、显存带宽、active sequences、runtime 版本、sampling 参数或 GPU 负载有关。

问题三：p99 升高但平均延迟没变怎么办？

可以回答：按模型、租户、endpoint、输入长度、输出长度切分，查看慢请求 trace，排查长请求、局部实例拥塞、cache miss、工具调用和网络抖动。

问题四：推理错误率上升如何分类？

可以回答：先按 4xx、429、5xx、timeout、OOM、model load failed、tool failed、safety blocked 分类，再分别定位权限、限流、runtime、资源、模型或下游依赖。

46.23 小练习

1. TTFT 和 TPOT 分别对应推理链路中的哪些阶段？
2. 为什么 p99 延迟比平均延迟更重要？
3. KV cache 不足会导致哪些现象？
4. 模型加载失败应检查哪些 artifact 和环境？
5. Cache 命中率下降可能由哪些发布变更导致？
6. Route trace 应该记录哪些信息？
7. Agent 工具调用失败如何排查？
8. 如何设计一个推理故障 runbook？

46.24 本章小结

本章讲了推理故障定位。

你需要记住：

1. 推理故障要先判断影响范围，再按请求链路拆阶段。
2. TTFT 主要受前置链路、排队和 prefill 影响，TPOT 主要受 decode 和 KV cache 影响。
3. p99、tokens/s、queue、KV cache、active sequences 和 route trace 比单纯 QPS 更关键。
4. 错误率要先分类，timeout 要记录发生阶段。
5. RAG/Agent 场景还要看 retrieval trace、prompt assembly trace 和 tool call trace。
6. 生产故障处理要结合限流、降级、回滚、扩容和摘除异常 endpoint。

下一章我们会讲 SLO、SLA、错误预算和生产值班体系。

第 47 章 SLO、SLA、错误预算和生产值班体系

上一章讲了推理故障定位。本章继续讲可靠性治理中的核心概念：SLO、SLA、错误预算和生产值班体系。

AI Infra 不只是把训练和推理跑起来，还要稳定地服务研发和线上业务。稳定性不能只靠救火，而要通过目标、指标、预算、告警、值班和复盘形成体系。

先记住一句话：

可靠性治理的核心，不是追求永不失败，而是明确什么程度的失败可接受，并围绕这个目标组织工程、发布和值班。

47.0 本讲资料边界与第二轮精修口径

本章按通用 AI Infra 生产可靠性治理来写，不绑定某个云厂商、监控系统、告警平台、工单系统、Kubernetes 发行版或内部 SRE 组织形态。资料校准时，主要参考 Google SRE Book 对 SLI、SLO、SLA、用户体验指标、错误预算和“不追求 100% SLO”的定义，参考 Google SRE Workbook 对 burn rate、多窗口多 burn rate 告警、on-call、incident response 和 postmortem action item 的实践口径，并结合前文可观测性、训练故障定位、推理故障定位、发布治理、回滚降级、成本治理和事故复盘章节。

第二轮精修只做三件事：

1. 把 SLI / SLO / SLA、错误预算、burn rate、告警、值班、runbook、事故分级、变更、复盘和成本取舍统一成可审计的可靠性治理样本。
2. 补齐 availability、latency pass rate、平台失败率、数据构建 SLO、错误预算、预算消耗、burn rate、告警可行率、MTTA / MTTR、事故影响、runbook 覆盖、变更关联、行动项关闭率和 SLO 成本收益公式。
3. 增加一个 0 依赖 Python demo，用 toy SLO / on-call cases 检查生产值班体系是否只是“有人收告警”，还是能真正把用户体验、错误预算、告警路由、止损权限、复盘行动项和发布策略串成闭环。

47.1 SLI、SLO、SLA 的区别

先区分三个概念。

SLI 是 Service Level Indicator，服务水平指标。

它是实际观测到的指标。

例如：

1. 推理请求成功率。
2. p99 latency。
3. TTFT p95。
4. 训练任务成功率。
5. checkpoint 恢复成功率。

SLO 是 Service Level Objective，服务水平目标。

它是内部设定的目标。

例如：

99.9% 推理请求在 3 秒内返回首 token

SLA 是 Service Level Agreement，服务水平协议。

它是对客户或业务承诺的协议，通常有赔偿或约束。

简单说：SLI 是指标，SLO 是目标，SLA 是承诺。

一个可靠性治理样本可以抽象成：

$$R_i = (s_i, o_i, a_i, b_i, g_i, p_i, r_i, c_i, m_i, z_i)$$

其中 s_i 是 SLI 定义， o_i 是 SLO 目标， a_i 是 SLA 或业务承诺边界， b_i 是错误预算， g_i 是告警和 burn rate 规则， p_i 是值班排班和升级路径， r_i 是 runbook， c_i 是变更事件， m_i 是止损 / 回滚 / 降级动作， z_i 是复盘和行动项。

这组变量的核心要求是“可测量、可行动、可归责、可复盘”。如果一个 SLO 不能落到实际观测数据和明确动作，就只是口号。

47.2 为什么 AI Infra 需要 SLO

没有 SLO，平台团队很难回答：

1. 推理慢到什么程度算故障？
2. 训练任务失败率多少可接受？
3. 评估平台排队多久算异常？
4. GPU 集群利用率低是否要处理？
5. 发布导致 p99 上升多少必须回滚？
6. 成本上升多少需要告警？

SLO 给工程决策提供边界。

没有目标，所有问题都变成“看情况”。

SLO 的控制闭环可以写成：

$$G_{\text{slo}} = \mathbf{1} \left[\min_j C_j \geq \tau_j \wedge B_{\text{remain}} > 0 \wedge P_0 = 0 \right]$$

其中 C_j 是第 j 个关键 SLO 或治理门禁覆盖率， τ_j 是最低阈值， B_{remain} 是剩余错误预算比例， P_0 是未关闭的最高优先级可靠性阻断项数量。这个式子表达的是：SLO 不是单个 dashboard 数字，而是目标、预算和阻断项共同组成的上线控制条件。

47.3 AI Infra 常见 SLI

训练平台 SLI：

1. 任务提交成功率。
2. 任务调度等待时间。
3. 训练任务成功率。
4. 节点失败恢复时间。
5. checkpoint 保存成功率。
6. checkpoint 恢复成功率。
7. GPU 利用率。

推理平台 SLI：

1. 请求成功率。
2. TTFT。
3. TPOT。
4. p95 / p99 延迟。
5. timeout rate。
6. error rate。
7. admission reject rate。

数据和评估平台 SLI：

1. dataset 构建成功率。
2. 数据质量门禁通过率。
3. eval job 成功率。
4. eval report 生成时延。

RAG/Agent SLI：

1. retrieval success rate。
2. tool call success rate。
3. agent run success rate。
4. citation available rate。

统一的 SLI 覆盖率可以写成：

$$C_{\text{sli}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[s_i \text{ defined} \wedge s_i \text{ measured} \wedge s_i \text{ owned}]$$

其中 N 是关键服务或关键工作流数量。这个指标要求每个重要工作流不仅有指标名，还要有采集口径、时间窗口、owner 和对应动作。

47.4 推理 SLO 示例

推理平台可以定义：

Availability SLO:

99.9% 请求返回非 5xx 和非 timeout

Latency SLO:

95% 请求 TTFT < 1s

99% 请求 end-to-end latency < 10s

Streaming SLO:

95% 请求 TPOT < 80ms

Quality Guardrail:

线上安全违规率低于阈值

注意：推理 SLO 不只看可用性，还要看 TTFT、TPOT 和质量 guardrail。

大模型服务如果 100% 返回但全部很慢，用户仍然认为不可用。

可用性可以定义为好请求占有效请求的比例：

$$A = \frac{N_{\mathrm{good}}}{N_{\mathrm{valid}}}$$

其中 N_{good} 是非 5xx、非 timeout、非平台拒绝且满足基本响应契约的请求数， N_{valid} 是纳入 SLO 统计的有效请求数。

延迟 SLO 常用通过率，而不是只看平均值：

$$C_{\mathrm{lat}}(\tau) = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[T_i \leq \tau]$$

其中 T_i 可以是 TTFT、TPOT 或 end-to-end latency， τ 是目标阈值。推理平台的上线门禁可以写成：

$$G_{\mathrm{infer_slo}} = \mathbf{1} \left[A \geq a_0 \wedge C_{\mathrm{tft}}(\tau_f) \geq c_f \wedge C_{\mathrm{tpot}}(\tau_p) \geq c_p \wedge R_{\mathrm{err}} \leq r_0 \wedge Q_{\mathrm{guard}} \geq q_0 \right]$$

其中 a_0 是可用性目标， c_f 是 TTFT 通过率目标， c_p 是 TPOT 通过率目标， R_{err} 是错误率， Q_{guard} 是质量 / 安全 guardrail 得分。这个门禁能防止 “请求都返回了，但慢、错或不安全” 的假可用。

47.5 训练 SLO 示例

训练平台可以定义：

Submission SLO:

99.5% 训练任务提交请求在 5s 内完成校验

Scheduling SLO:

高优先级任务 95% 在 10 分钟内开始运行

Reliability SLO:

训练任务因平台原因失败率 < 1%

Checkpoint SLO:

99% checkpoint 保存成功，恢复成功率 > 99%

训练平台 SLO 和推理平台不同。

训练任务本身可能因为用户代码失败，这不一定算平台错误。

因此要区分用户错误和平台错误。

平台原因失败率可以写成：

$$R_{\text{platform_fail}} = \frac{N_{\text{platform_fail}}}{N_{\text{train_job}}}$$

其中 $N_{\text{platform_fail}}$ 只统计调度、节点、镜像、存储、网络、权限、checkpoint、平台 launcher 等平台责任导致的失败，用户代码、用户数据格式错误和显式超配资源不应直接算进平台失败率。

训练平台的可靠性门禁可以写成：

$$G_{\text{train_slo}} = \mathbf{1} [R_{\text{platform_fail}} \leq r_t \wedge C_{\text{submit}} \geq c_s \wedge C_{\text{schedule}} \geq c_q \wedge C_{\text{ckpt}} \geq c_k]$$

其中 C_{submit} 是提交校验通过率, C_{schedule} 是调度等待时间达标率, C_{ckpt} 是 checkpoint 保存 / 恢复达标率。

47.6 数据平台 SLO 示例

数据平台可以定义：

Dataset Build SLO:

95% dataset build 在预期时间内完成

Data Quality SLO:

生产 dataset 必须通过质量门禁

Access SLO:

99.9% dataset manifest 查询成功

Serving SLO:

训练数据读取吞吐满足任务声明需求

数据平台 SLO 不只是 API 可用，还要看数据质量和训练供给能力。

数据构建 SLO 可以写成：

$$C_{\text{build}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1} [T_{\text{build},i} \leq \tau_i \wedge Q_i \geq q_i \wedge M_i = 1]$$

其中 $T_{\text{build},i}$ 是第 i 个 dataset build 的耗时, τ_i 是预期完成时间, Q_i 是数据质量门禁得分, M_i 表示 manifest、checksum、lineage 和权限记录是否完整。这个定义提醒面试时不要把数据平台 SLO 讲成 “API 还能访问”，而要包含数据是否可训练、可追溯、可治理。

47.7 错误预算是什么

错误预算是 SLO 允许的失败空间。

例如 SLO 是 99.9% 可用性，那么一个周期内允许 0.1% 请求失败。

如果一个月有 1000 万请求：

允许失败请求 = 1000 万 * 0.1% = 1 万

这 1 万就是错误预算。

错误预算用来平衡稳定性和迭代速度。

如果 SLO 目标是 S_{slo} ，错误预算比例是：

$$B = 1 - S_{\mathrm{slo}}$$

给定统计窗口内有效请求数 N_{valid} ，允许的坏事件数是：

$$N_{\mathrm{bad,allowed}} = N_{\mathrm{valid}} B$$

如果统计对象不是请求，也可以把 N_{valid} 换成训练任务数、dataset build 数、eval job 数或 agent run 数。关键是先定义“什么算坏事件”，再谈预算。

47.8 错误预算怎么用

错误预算可以指导决策：

1. 预算充足，可以正常发布新功能。
2. 预算消耗过快，减少高风险发布。
3. 预算耗尽，进入稳定性优先模式。
4. 某个模型持续消耗预算，必须修复或降级。
5. 某个租户异常消耗预算，可能需要限流或隔离。

错误预算不是财务预算，而是可靠性预算。

它让发布速度和稳定性之间有量化边界。

预算已使用比例和剩余比例可以写成：

$$U_{\mathrm{budget}} = \frac{N_{\mathrm{bad}}}{\max(1, N_{\mathrm{bad,allowed}})}$$

$$B_{\mathrm{remain}} = \max(0, 1 - U_{\mathrm{budget}})$$

其中 N_{bad} 是当前窗口已经发生的坏事件数。发布策略可以由 B_{remain} 驱动：剩余预算高时允许常规灰度，剩余预算低时减速或冻结高风险发布，预算耗尽时只允许可靠性修复。

47.9 Burn Rate

Burn rate 是错误预算消耗速度。

如果一天就消耗了一个月 50% 的错误预算，说明问题很严重。

常见告警方式：

1. 快速 burn rate 告警：短时间内大量消耗预算。
2. 慢速 burn rate 告警：长期小幅超标。

快速 burn 适合发现事故。

慢速 burn 适合发现慢性退化。

AI Infra 中，p99 慢性上升、成本慢性上涨、某个模型错误率小幅升高，都可能是慢速 burn。

某个时间窗口 w 的 burn rate 可以写成：

$$\beta_w = \frac{R_{\mathrm{bad},w}}{B}$$

其中 $R_{\mathrm{bad},w}$ 是窗口内坏事件比例, B 是错误预算比例。若 99.9% SLO 的预算比例是 0.1%, 窗口内错误率是 0.6%, 则 $\beta_w=6$, 意味着当前错误消耗速度是预算允许速度的 6 倍。

多窗口告警通常同时看快窗口和慢窗口:

$$G_{\mathrm{burn}} = \mathbf{1} [(\beta_{1h} \geq \beta_f \wedge \beta_{5m} \geq \beta_f) \vee (\beta_{6h} \geq \beta_s \wedge \beta_{30m} \geq \beta_s)]$$

其中 β_f 是快速 burn 阈值, β_s 是慢速 burn 阈值。这样既能发现突发事故, 也能降低已经恢复后的告警残留。

47.10 SLO 不能太多

很多团队会定义几十个 SLO, 最后没人看。

好的 SLO 应该:

1. 反映用户体验。
2. 可测量。
3. 可行动。
4. 能区分平台责任和用户责任。
5. 数量有限。

例如推理平台优先选择:

1. Availability。
2. TTFT。
3. TPOT。
4. p99 latency。
5. timeout rate。

不要把所有内部指标都升级成 SLO。

SLO 数量本身也可以治理:

$$G_{\mathrm{slo_set}} = \mathbf{1} [N_{\mathrm{slo}} \leq n_0 \wedge C_{\mathrm{user}} \geq c_u \wedge C_{\mathrm{action}} \geq c_a \wedge C_{\mathrm{owner}} \geq c_o]$$

其中 N_{slo} 是某个服务的正式 SLO 数量, C_{user} 是用户体验相关覆盖率, C_{action} 是 SLO 触发后有明确动作的比例, C_{owner} 是有 owner 的比例。这个门禁的直觉是: SLO 太多会稀释注意力, SLO 太虚会无法驱动工程动作。

47.11 告警和 SLO 的关系

告警应围绕 SLO 和用户影响设计。

不好的告警:

GPU utilization > 90%

因为 GPU 高利用率可能是正常高吞吐。

更好的告警:

TTFT p95 超过 SLO 且 queue wait 持续上升

告警要告诉值班人员：用户体验是否受影响，可能原因是什么，下一步看哪里。

告警可行动率可以写成：

$$A_{\text{action}} = \frac{N_{\text{actionable}}}{N_{\text{alert}}}$$

告警噪音率可以写成：

$$R_{\text{noise}} = \frac{N_{\text{no_action}} + N_{\text{duplicate}}}{N_{\text{alert}}}$$

其中 $N_{\text{actionable}}$ 是触发明确排查、止损、回滚、扩容、限流或修复动作的告警数, $N_{\text{no_action}}$ 是无人需要处理的告警数, $N_{\text{duplicate}}$ 是重复告警数。好的告警体系应提高 A_{action} , 降低 R_{noise} 。

47.12 生产值班体系

生产值班是保障线上系统的组织机制。

值班体系至少包括：

1. 值班排班。
2. 告警路由。
3. 升级机制。
4. Runbook。
5. 事故分级。
6. 状态页或内部通报。
7. 事故复盘。
8. 后续行动项跟踪。

没有值班体系，再好的监控也没人处理。

值班响应的两个基本指标是 MTTA 和 MTTR：

$$\text{MTTA} = \frac{1}{N} \sum_{i=1}^N (t_{\text{ack},i} - t_{\text{detect},i})$$

$$\text{MTTR} = \frac{1}{N} \sum_{i=1}^N (t_{\text{recover},i} - t_{\text{detect},i})$$

其中 $t_{\text{detect},i}$ 是第 i 个事故被检测到的时间, $t_{\text{ack},i}$ 是值班确认时间, $t_{\text{recover},i}$ 是用户影响恢复时间。MTTR 不等于根因完全修复时间，线上事故通常先止损恢复，再做完整根因修复。

47.13 事故分级

事故可以分级。

示例：

1. SEV1：核心推理服务大面积不可用。
2. SEV2：关键租户或关键模型严重受影响。
3. SEV3：部分功能退化，有降级方案。
4. SEV4：低影响问题或内部平台异常。

分级决定响应速度、通知范围和复盘要求。

AI Infra 中，训练平台大面积不可用和线上推理不可用影响不同，分级要结合业务场景。

事故影响分数可以写成：

$$S_{\text{incident}} = w_u U + w_d D + w_s S + w_r R$$

其中 U 是受影响用户或租户比例， D 是持续时间， S 是服务重要性权重， R 是风险权重，例如安全、隐私、收入、合规或关键客户影响。事故分级不应只看错误率，还要结合业务影响和风险。

47.14 On-call Runbook

Runbook 是值班手册。

一个好的 runbook 包括：

1. 告警含义。
2. 影响范围判断。
3. 需要查看的 dashboard。
4. 常见根因。
5. 应急处理步骤。
6. 回滚或降级方法。
7. 升级联系人。
8. 复盘记录模板。

Runbook 要实际可执行，不是概念文档。

Runbook 覆盖率可以写成：

$$C_{\text{runbook}} = \frac{N_{\text{alert_with_runbook}}}{N_{\text{critical_alert}}}$$

执行就绪门禁可以写成：

$$G_{\text{runbook}} = \mathbf{1} [C_{\text{runbook}} \geq c_r \wedge C_{\text{step}} \geq c_s \wedge C_{\text{rollback}} \geq c_b \wedge C_{\text{owner}} \geq c_o]$$

其中 C_{step} 是 runbook 步骤可执行覆盖率， C_{rollback} 是回滚 / 降级动作覆盖率， C_{owner} 是升级联系人覆盖率。

47.15 推理事故处理流程

推理事故可以这样处理：

1. 确认告警和影响范围
2. 查看是否有发布或流量事件
3. 按模型、租户、endpoint、地域切分指标
4. 查看 TTFT、TPOT、queue、KV cache、error、timeout
5. 决定是否限流、降级、摘除实例、扩容或回滚
6. 恢复服务后定位根因
7. 记录复盘和行动项

线上事故中，先恢复服务，再做完整根因分析。

推理事故的止损门禁可以写成：

$$G_{\text{mitigation}} = \mathbf{1} [M_{\text{rate}} \vee M_{\text{degrade}} \vee M_{\text{rollback}} \vee M_{\text{scale}} \vee M_{\text{isolate}}]$$

其中 M_{rate} 是限流, M_{degrade} 是降级, M_{rollback} 是回滚, M_{scale} 是扩容, M_{isolate} 是摘除异常实例、租户、模型或路由。这个公式不是说所有动作都要做, 而是要求至少存在一条已验证的止损路径。

47.16 训练事故处理流程

训练事故可以这样处理:

1. 判断是单任务失败还是平台级故障
2. 区分用户代码错误和平台错误
3. 查看调度、节点、镜像、数据、通信和存储事件
4. 如果影响多个任务, 暂停相关队列或迁移资源
5. 恢复 **checkpoint** 或重试任务
6. 修复根因并补充检测

训练平台要避免某个节点、存储或网络问题影响大量任务。

训练事故造成的丢失 GPU 小时可以写成:

$$K_{\mathrm{lost}} = \sum_{j=1}^J G_j T_j$$

其中 G_j 是第 j 个受影响训练任务占用的 GPU 数, T_j 是从故障发生到恢复或重试成功之间损失的小时数。这个指标能把“训练任务失败”转成资源浪费和调度影响, 便于决定是否要升级事故级别。

47.17 值班中的权限

值班人员需要足够权限处理事故, 但权限不能无限大。

常见值班权限:

1. 查看 dashboard。
2. 查看脱敏日志。
3. 摘除 endpoint。
4. 触发回滚。
5. 修改限流配置。
6. 暂停队列。
7. 扩缩容。

高风险操作要有审计, 必要时需要双人确认。

值班权限门禁可以写成:

$$G_{\mathrm{oncall_perm}} = \mathbf{1} [C_{\mathrm{least}} \geq c_l \wedge C_{\mathrm{audit}} \geq c_a \wedge C_{\mathrm{breakglass}} \geq c_b \wedge H_{\mathrm{confirm}} = 1]$$

其中 C_{least} 是最小权限覆盖率, C_{audit} 是操作审计覆盖率, $C_{\mathrm{breakglass}}$ 是紧急授权流程覆盖率, H_{confirm} 表示高风险动作是否需要确认或审批。值班权限的目标不是让人“什么都不能做”, 而是让值班能止损, 同时让高风险动作可追踪、可回放、可复盘。

47.18 变更管理

很多事故来自变更。

变更包括:

1. 模型版本发布。
2. prompt 变更。

3. runtime 升级。
4. 路由规则变更。
5. 数据版本变更。
6. 安全策略变更。
7. 集群配置变更。
8. 网络或存储升级。

值班系统应该能看到最近变更时间线。

排查事故时，第一问题通常是：最近变了什么？

变更关联覆盖率可以写成：

$$C_{\text{change}} = \frac{N_{\text{incident_with_change_timeline}}}{N_{\text{incident}}}$$

一次 AI Infra 事故至少应能关联 model、prompt、runtime、route、data、cluster、cache、safety 和 owner 等字段。没有变更时间线，事故排查会退化为猜测。

47.19 事故复盘

事故复盘不是追责，而是改进系统。

复盘应包括：

1. 时间线。
2. 影响范围。
3. 用户影响。
4. 检测方式。
5. 响应过程。
6. 根因。
7. 哪些防线失效。
8. 修复措施。
9. 后续行动项。
10. 负责人和截止时间。

AI Infra 复盘还应包括：模型、数据、runtime、GPU、路由、缓存、成本和安全是否相关。

行动项关闭率可以写成：

$$C_{\text{action}} = \frac{N_{\text{action_closed}}}{N_{\text{action_total}}}$$

复盘质量门禁可以写成：

$$G_{\text{postmortem}} = \mathbf{1} [C_{\text{timeline}} \geq c_t \wedge C_{\text{impact}} \geq c_i \wedge C_{\text{root}} \geq c_r \wedge C_{\text{action}} \geq c_a \wedge C_{\text{regression}} \geq c_g]$$

其中 C_{timeline} 是时间线完整率， C_{impact} 是影响量化覆盖率， C_{root} 是根因分类覆盖率， $C_{\text{regression}}$ 是修复后回归验证覆盖率。复盘的价值不在于文档写完，而在于行动项真正关闭并转成监控、回归、runbook 或上线门禁。

47.20 错误预算和发布策略

错误预算可以影响发布策略。

例如：

1. 预算充足：正常灰度发布。

2. 预算消耗较快：降低灰度速度。
3. 预算接近耗尽：冻结高风险发布。
4. 预算耗尽：只允许修复稳定性问题。

对 AI 平台来说，模型发布、runtime 升级和路由策略变更都应该受错误预算约束。

基于错误预算的发布门禁可以写成：

$$G_{\text{release_by_budget}} = 1 \left[B_{\text{remain}} \geq b_0 \wedge \beta_{\text{fast}} < \beta_f \wedge \beta_{\text{slow}} < \beta_s \wedge R_{\text{rollback}} = 1 \right]$$

其中 b_0 是发布所需的最低剩余预算， β_{fast} 和 β_{slow} 是快慢窗口 burn rate, R_{rollback} 表示是否有可执行回滚。这个门禁能把“最近稳定吗”转成可执行发布条件。

47.21 SLO 和成本的关系

更高 SLO 通常意味着更高成本。

例如：

1. 更低 TTFT 需要更多 warm pool。
2. 更低 p99 需要更多冗余。
3. 更高可用性需要多地域容灾。
4. 更快训练调度需要预留 GPU。

所以 SLO 不是越高越好。

要根据业务价值设定合理目标。

对低优先级批处理任务，99.99% 可用性可能没有意义。

SLO 提升的价值 / 成本比可以粗略写成：

$$V_{\text{slo}} = \frac{\Delta U_{\text{user}} + \Delta R_{\text{risk}}}{\Delta K_{\text{infra}} + \Delta K_{\text{ops}}}$$

其中 ΔU_{user} 是用户体验或业务收益提升， ΔR_{risk} 是风险降低收益， ΔK_{infra} 是额外基础设施成本， ΔK_{ops} 是额外运维和值班成本。面试中要能说明：99.99% 并不天然比 99.9% 更好，只有当价值 / 成本比足够高时才值得追求。

47.22 SLO 值班体系审计指标和最小 demo

把本章落到平台验收时，可以用 16 个门禁：

1. SLI Contract Completeness: 关键服务的 SLI 是否有定义、采集、窗口、owner 和坏事件口径。
2. SLO Target Measurability: SLO 是否可测量、可回放、可解释，并能对应具体用户体验。
3. SLA Boundary Clarity: 对外承诺、内部目标、赔偿或业务责任边界是否清楚。
4. Inference SLO Coverage: 推理是否覆盖 availability、TTFT、TPOT、p99、error、timeout 和质量 / 安全 guardrail。
5. Training SLO Coverage: 训练是否区分用户错误和平台错误，并覆盖提交、调度、平台失败、checkpoint 和恢复。
6. Data Eval SLO Coverage: 数据、评估、RAG / Agent 是否覆盖 build、manifest、质量、eval report、retrieval / tool / run success。
7. Error Budget Accounting: 错误预算是否按窗口、坏事件和服务维度计算，并能驱动发布策略。
8. Burn Rate Alerting: 是否有快慢窗口 burn rate 告警，并能减少恢复后的告警残留。
9. Alert Actionability: 告警是否可行动、低噪音、有 owner、有 runbook、有下一步。
10. Oncall Ownership Escalation: 值班排班、升级链路、ACK SLO、负责人和状态沟通是否完整。
11. Incident Severity Routing: 事故分级是否结合用户影响、持续时间、服务重要性和风险。

12. Runbook Executability: runbook 是否包含影响判断、dashboard、根因路径、止损步骤、回滚和升级联系人。
13. Change Event Linkage: 事故系统是否能关联模型、prompt、runtime、route、data、cluster、cache 和 owner 变更。
14. Mitigation Rollback Authority: 值班是否有可审计的限流、降级、扩容、摘除、回滚和暂停队列权限。
15. Postmortem Action Closure: 复盘是否包含时间线、影响、根因、防线失效、行动项 owner、截止时间和回归验证。
16. SLO Cost Tradeoff Gate: SLO 目标是否经过用户价值、风险降低、基础设施成本和值班成本取舍。

综合门禁:

$$G_{\text{slo_oncall}} = \prod_{j=1}^{16} G_j$$

下面是一个 0 依赖 demo, 用 toy SLO / on-call cases 检查生产可靠性治理是否只是 “有人收告警”, 还是能真正把 SLI、SLO、SLA、错误预算、burn rate、值班、runbook、变更、止损和复盘行动项串成闭环。

```
from copy import deepcopy
```

```
class MiniSLOoncallAudit:
```

```
    GATES = [
        "sli_contract_completeness",
        "slo_target_measurability",
        "sla_boundary_clarity",
        "inference_slo_coverage",
        "training_slo_coverage",
        "data_eval_slo_coverage",
        "error_budget_accounting",
        "burn_rate_alerting",
        "alert_actionability",
        "oncall_ownership_escalation",
        "incident_severity_routing",
        "runbook_executability",
        "change_event_linkage",
        "mitigation_rollback_authority",
        "postmortem_action_closure",
        "slo_cost_tradeoff_gate",
    ]
```

```
    SLI_FIELDS = ["name", "definition", "source", "window", "owner", "bad_event_rule"]
```

```
    SLO_FIELDS = ["target", "window", "threshold", "query", "business_reason", "action"]
```

```
    SLA_FIELDS = ["customer_scope", "promise", "exclusion", "remedy", "owner"]
```

```
    INFERENCE_FIELDS = ["availability", "ttft", "tpot", "p99", "error", "timeout", "quality_guardrail"]
```

```
    TRAINING_FIELDS = ["submission", "scheduling", "platform_fail", "checkpoint_save", "checkpoint_restore"]
```

```
    DATA_EVAL_FIELDS = ["dataset_build", "manifest_access", "quality_gate", "eval_report", "rag_agent_success"]
```

```
    CHANGE_FIELDS = ["model", "prompt", "runtime", "route", "data", "cluster", "cache", "owner"]
```

```
    RUNBOOK_FIELDS = ["meaning", "impact", "dashboards", "root_causes", "mitigation", "rollback", "escalation"]
```

```
    POSTMORTEM_FIELDS = ["timeline", "impact", "detection", "response", "root_cause", "failed_defenses", "actions", ]
```

```
    @staticmethod
```

```
    def present(record, key):
```

```
        return key in record and record[key] is not None and record[key] != ""
```

```
    def coverage(self, record, fields):
```

```

    if not record:
        return 0.0
    return sum(1 for field in fields if self.present(record, field)) / len(fields)

def availability(self, case):
    infer = case["inference"]
    return infer["good_requests"] / infer["valid_requests"]

def latency_pass_rate(self, case):
    infer = case["inference"]
    return infer["latency_pass"] / infer["valid_requests"]

def bad_allowed(self, case):
    budget = case["error_budget"]
    return int(round(case["inference"]["valid_requests"] * (1.0 - budget["slo_target"])))

def budget_used(self, case):
    budget = case["error_budget"]
    return budget["bad_events"] / max(1, self.bad_allowed(case))

def budget_remaining(self, case):
    return max(0.0, 1.0 - self.budget_used(case))

def burn_rate(self, case, key):
    budget = case["error_budget"]
    return budget[key] / max(1e-12, 1.0 - budget["slo_target"])

def actionability(self, case):
    alerts = case["alerts"]
    return alerts["actionable"] / alerts["total"]

def noise_rate(self, case):
    alerts = case["alerts"]
    return (alerts["no_action"] + alerts["duplicate"]) / alerts["total"]

def mttm_minutes(self, case):
    oncall = case["oncall"]
    return oncall["ack_minutes_total"] / oncall["incidents"]

def action_item_closure(self, case):
    postmortem = case["postmortem"]
    return postmortem["action_closed"] / postmortem["action_total"]

def value_cost_ratio(self, case):
    tradeoff = case["cost_tradeoff"]
    value = tradeoff["user_value_delta"] + tradeoff["risk_reduction_value"]
    cost = tradeoff["infra_cost_delta"] + tradeoff["ops_cost_delta"]
    return value / cost

def sli_contract_completeness(self, case):
    return self.coverage(case.get("sli_contract", {}), self.SLI_FIELDS) == 1.0

def slo_target_measurability(self, case):
    slo = case.get("slo_target", {})
    return (

```



```

        self.coverage(slo, self.SLO_FIELDS) == 1.0
        and slo.get("replayable") is True
        and slo.get("user_visible") is True
    )

def sla_boundary_clarity(self, case):
    return self.coverage(case.get("sla_boundary", {}), self.SLA_FIELDS) == 1.0

def inference_slo_coverage(self, case):
    infer = case.get("inference", {})
    return (
        self.coverage(infer.get("slo_fields", {}), self.INFERENCE_FIELDS) == 1.0
        and self.availability(case) >= infer.get("availability_target", 1.0)
        and self.latency_pass_rate(case) >= infer.get("latency_pass_target", 1.0)
        and infer.get("quality_guardrail_pass") is True
    )

def training_slo_coverage(self, case):
    train = case.get("training", {})
    return (
        self.coverage(train.get("slo_fields", {}), self.TRAINING_FIELDS) == 1.0
        and train.get("user_platform_error_split") is True
        and train.get("platform_fail_rate", 1.0) <= train.get("max_platform_fail_rate", 0.0)
        and train.get("checkpoint_restore_success", 0.0) >= train.get("checkpoint_restore_target", 1.0)
    )

def data_eval_slo_coverage(self, case):
    data_eval = case.get("data_eval", {})
    return (
        self.coverage(data_eval.get("slo_fields", {}), self.DATA_EVAL_FIELDS) == 1.0
        and data_eval.get("build_success_rate", 0.0) >= data_eval.get("build_target", 1.0)
        and data_eval.get("eval_report_latency_pass", 0.0) >= data_eval.get("eval_report_target", 1.0)
    )

def error_budget_accounting(self, case):
    budget = case.get("error_budget", {})
    return (
        budget.get("window_days", 0) > 0
        and budget.get("bad_event_rule") is True
        and budget.get("service_dimension") is True
        and self.budget_used(case) <= budget.get("max_budget_used", 0.0)
        and budget.get("release_policy_bound") is True
    )

def burn_rate_alerting(self, case):
    burn = case.get("burn_alerts", {})
    return (
        self.burn_rate(case, "fast_window_bad_rate") <= burn.get("fast_threshold", 0.0)
        and self.burn_rate(case, "slow_window_bad_rate") <= burn.get("slow_threshold", 0.0)
        and burn.get("multi_window") is True
        and burn.get("recovery_suppression") is True
    )

def alert_actionability(self, case):
    alerts = case.get("alerts", {})

```

```

    return (
        self.actionability(case) >= alerts.get("min_actionability", 1.0)
        and self.noise_rate(case) <= alerts.get("max_noise_rate", 0.0)
        and alerts.get("owner") is True
        and alerts.get("runbook") is True
        and alerts.get("next_step") is True
    )

def oncall_ownership_escalation(self, case):
    oncall = case.get("oncall", {})
    return (
        oncall.get("schedule") is True
        and oncall.get("primary") is True
        and oncall.get("secondary") is True
        and oncall.get("escalation_policy") is True
        and self.mtta_minutes(case) <= oncall.get("ack_slo_minutes", 0.0)
        and oncall.get("status_comms") is True
    )

def incident_severity_routing(self, case):
    severity = case.get("severity", {})
    score = (
        severity["user_impact"] * severity["w_user"]
        + severity["duration_minutes"] * severity["w_duration"]
        + severity["service_weight"] * severity["w_service"]
        + severity["risk_weight"] * severity["w_risk"]
    )
    return (
        score >= severity.get("sev_threshold", 10**9)
        and severity.get("sev_level") == severity.get("expected_sev")
        and severity.get("notification_scope") is True
        and severity.get("review_required") is True
    )

def runbook_executability(self, case):
    runbook = case.get("runbook", {})
    return (
        self.coverage(runbook, self.RUNBOOK_FIELDS) == 1.0
        and runbook.get("last_drill_days", 10**9) <= runbook.get("max_drill_age_days", 0)
        and runbook.get("tested_by_oncall") is True
    )

def change_event_linkage(self, case):
    change = case.get("change_events", {})
    return (
        self.coverage(change.get("fields", {}), self.CHANGE_FIELDS) == 1.0
        and change.get("timeline_linked") is True
        and change.get("diff_ready") is True
        and change.get("owner_present") is True
    )

def mitigation_rollback_authority(self, case):
    mitigation = case.get("mitigation", {})
    required = ["rate_limit", "degrade", "scale", "isolate", "rollback", "pause_queue"]
    return (

```

```

        all(mitigation.get(field) is True for field in required)
        and mitigation.get("audit_log") is True
        and mitigation.get("high_risk_confirmed") is True
        and mitigation.get("rollback_tested") is True
    )

def postmortem_action_closure(self, case):
    postmortem = case.get("postmortem", {})
    return (
        self.coverage(postmortem.get("fields", {}), self.POSTMORTEM_FIELDS) == 1.0
        and self.action_item_closure(case) >= postmortem.get("min_action_closure", 1.0)
        and postmortem.get("owners_and_due_dates") is True
        and postmortem.get("regression_verified") is True
    )

def slo_cost_tradeoff_gate(self, case):
    tradeoff = case.get("cost_tradeoff", {})
    return (
        self.value_cost_ratio(case) >= tradeoff.get("min_value_cost_ratio", 1.0)
        and tradeoff.get("business_priority_bound") is True
        and tradeoff.get("capacity_plan_ready") is True
        and tradeoff.get("ops_load_accepted") is True
    )

def audit_case(self, case):
    return {gate: getattr(self, gate)(case) for gate in self.GATES}

def run_all(self, cases):
    results = {case["case_id"]: self.audit_case(case) for case in cases}
    metrics = {}
    for gate in self.GATES:
        passed = sum(1 for result in results.values() if result[gate])
        metrics[gate] = round(passed / len(cases), 3)
    failed_cases = [
        case_id
        for case_id, result in results.items()
        if not all(result.values())
    ]
    failed_gates = [
        gate
        for gate in self.GATES
        if any(not result[gate] for result in results.values())
    ]
    return {
        "metrics": metrics,
        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "failed_gates": failed_gates,
        "slo_oncall_gate_pass": metrics["slo_cost_tradeoff_gate"] == 1.0,
    }

def example_outputs(self, case):
    return {
        "availability": round(self.availability(case), 4),
        "latency_pass_rate": round(self.latency_pass_rate(case), 4),
    }

```

```

        "bad_allowed": self.bad_allowed(case),
        "budget_used": round(self.budget_used(case), 3),
        "budget_remaining": round(self.budget_remaining(case), 3),
        "fast_burn": round(self.burn_rate(case, "fast_window_bad_rate"), 3),
        "slow_burn": round(self.burn_rate(case, "slow_window_bad_rate"), 3),
        "actionable_alerts": case["alerts"]["actionable"],
        "ack_slo_minutes": case["oncall"]["ack_slo_minutes"],
        "sev1_response_minutes": case["oncall"]["sev1_response_minutes"],
        "runbook_count": case["runbook"]["count"],
        "change_fields": sorted(case["change_events"]["fields"]),
        "mitigation_ready": self.mitigation_rollback_authority(case),
        "action_item_closure": round(self.action_item_closure(case), 3),
        "value_cost_ratio": round(self.value_cost_ratio(case), 3),
    }

```

```

def build_good_case():
    return {
        "case_id": "full_slo_oncall_system",
        "sli_contract": {
            "name": "chat_availability",
            "definition": "non_5xx_non_timeout",
            "source": "request_metrics",
            "window": "30d",
            "owner": "serving-sre",
            "bad_event_rule": "5xx_or_timeout_or_contract_violation",
        },
        "slo_target": {
            "target": "99.9 availability and 95 TTFT under 1s",
            "window": "30d",
            "threshold": 0.999,
            "query": "slo_query",
            "business_reason": "interactive chat user experience",
            "action": "release_freeze_or_rollback",
            "replayable": True,
            "user_visible": True,
        },
        "sla_boundary": {
            "customer_scope": "paid_enterprise_chat",
            "promise": "monthly_availability_commitment",
            "exclusion": "customer_code_and_policy_blocks",
            "remedy": "service_credit",
            "owner": "platform-owner",
        },
        "inference": {
            "slo_fields": {
                "availability": True,
                "ttft": True,
                "tpot": True,
                "p99": True,
                "error": True,
                "timeout": True,
                "quality_guardrail": True,
            },
            "good_requests": 9992,
        }
    }

```

```

    "valid_requests": 10000,
    "latency_pass": 9700,
    "availability_target": 0.999,
    "latency_pass_target": 0.95,
    "quality_guardrail_pass": True,
},
"training": {
    "slo_fields": {
        "submission": True,
        "scheduling": True,
        "platform_fail": True,
        "checkpoint_save": True,
        "checkpoint_restore": True,
    },
    "user_platform_error_split": True,
    "platform_fail_rate": 0.006,
    "max_platform_fail_rate": 0.01,
    "checkpoint_restore_success": 0.995,
    "checkpoint_restore_target": 0.99,
},
"data_eval": {
    "slo_fields": {
        "dataset_build": True,
        "manifest_access": True,
        "quality_gate": True,
        "eval_report": True,
        "rag_agent_success": True,
    },
    "build_success_rate": 0.97,
    "build_target": 0.95,
    "eval_report_latency_pass": 0.96,
    "eval_report_target": 0.95,
},
"error_budget": {
    "slo_target": 0.9,
    "bad_events": 800,
    "window_days": 30,
    "bad_event_rule": True,
    "service_dimension": True,
    "max_budget_used": 0.85,
    "release_policy_bound": True,
    "fast_window_bad_rate": 0.6,
    "slow_window_bad_rate": 0.15,
},
"burn_alerts": {
    "fast_threshold": 10.0,
    "slow_threshold": 2.0,
    "multi_window": True,
    "recovery_suppression": True,
},
>alerts": {
    "total": 4,
    "actionable": 3,
    "no_action": 0,
    "duplicate": 1,

```

```

    "min_actionability": 0.7,
    "max_noise_rate": 0.3,
    "owner": True,
    "runbook": True,
    "next_step": True,
},
"oncall": {
    "schedule": True,
    "primary": True,
    "secondary": True,
    "escalation_policy": True,
    "ack_minutes_total": 15,
    "incidents": 3,
    "ack_slo_minutes": 5,
    "sev1_response_minutes": 5,
    "status_comms": True,
},
"severity": {
    "user_impact": 0.65,
    "duration_minutes": 30,
    "service_weight": 4,
    "risk_weight": 5,
    "w_user": 10,
    "w_duration": 0.1,
    "w_service": 2,
    "w_risk": 2,
    "sev_threshold": 20,
    "sev_level": "SEV1",
    "expected_sev": "SEV1",
    "notification_scope": True,
    "review_required": True,
},
"runbook": {
    "meaning": True,
    "impact": True,
    "dashboards": True,
    "root_causes": True,
    "mitigation": True,
    "rollback": True,
    "escalation": True,
    "last_drill_days": 20,
    "max_drill_age_days": 90,
    "tested_by_oncall": True,
    "count": 1,
},
"change_events": {
    "fields": {
        "model": True,
        "prompt": True,
        "runtime": True,
        "route": True,
        "data": True,
        "cluster": True,
        "cache": True,
        "owner": True,

```

```

    },
    "timeline_linked": True,
    "diff_ready": True,
    "owner_present": True,
},
"mitigation": {
    "rate_limit": True,
    "degrade": True,
    "scale": True,
    "isolate": True,
    "rollback": True,
    "pause_queue": True,
    "audit_log": True,
    "high_risk_confirmed": True,
    "rollback_tested": True,
},
"postmortem": {
    "fields": {
        "timeline": True,
        "impact": True,
        "detection": True,
        "response": True,
        "root_cause": True,
        "failed_defenses": True,
        "actions": True,
        "regression": True,
    },
    "action_closed": 9,
    "action_total": 10,
    "min_action_closure": 0.85,
    "owners_and_due_dates": True,
    "regression_verified": True,
},
"cost_tradeoff": {
    "user_value_delta": 160,
    "risk_reduction_value": 110,
    "infra_cost_delta": 70,
    "ops_cost_delta": 30,
    "min_value_cost_ratio": 2.0,
    "business_priority_bound": True,
    "capacity_plan_ready": True,
    "ops_load_accepted": True,
},
}

```

```

def build_bad_cases(good_case):
    cases = []

    case = deepcopy(good_case)
    case["case_id"] = "sli_contract_missing_bad"
    case["sli_contract"].pop("bad_event_rule")
    cases.append(case)

    case = deepcopy(good_case)

```

```

case["case_id"] = "slo_target_unmeasurable_bad"
case["slo_target"]["query"] = ""
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "sla_boundary_unclear_bad"
case["sla_boundary"].pop("remedy")
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "inference_slo_missing_bad"
case["inference"]["slo_fields"]["tpot"] = None
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "training_slo_missing_bad"
case["training"]["user_platform_error_split"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "data_eval_slo_missing_bad"
case["data_eval"]["slo_fields"].pop("eval_report")
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "error_budget_exhausted_bad"
case["error_budget"]["bad_events"] = 1100
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "burn_rate_alerting_bad"
case["burn_alerts"]["multi_window"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "alert_actionability_bad"
case["alerts"]["actionable"] = 1
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "oncall_escalation_bad"
case["oncall"]["secondary"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "severity_routing_bad"
case["severity"]["sev_level"] = "SEV3"
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "runbook_executability_bad"
case["runbook"]["rollback"] = None
cases.append(case)

case = deepcopy(good_case)

```



```

case["case_id"] = "change_event_linkage_bad"
case["change_events"]["fields"].pop("runtime")
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "mitigation_rollback_bad"
case["mitigation"]["rollback"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "postmortem_action_bad"
case["postmortem"]["action_closed"] = 5
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "slo_cost_tradeoff_bad"
case["cost_tradeoff"]["infra_cost_delta"] = 250
cases.append(case)

return cases

```

```

audit = MiniSLOncallAudit()
good = build_good_case()
cases = [good] + build_bad_cases(good)
summary = audit.run_all(cases)

print("slo_onscall_examples=" + repr(audit.example_outputs(good)))
print("metrics=" + repr(summary["metrics"]))
print("hard_blocker_count=" + repr(summary["hard_blocker_count"]))
print("failed_cases=" + repr(summary["failed_cases"]))
print("failed_gates=" + repr(summary["failed_gates"]))
print("slo_onscall_gate_pass=" + repr(summary["slo_onscall_gate_pass"]))

```

参考输出应类似：

```

slo_onscall_examples={'availability': 0.9992, 'latency_pass_rate': 0.97, 'bad_allowed': 1000, 'budget_used': 0.8, 'budget_remaining': 0.2}
metrics={'sli_contract_completeness': 0.941, 'slo_target_measurability': 0.941, 'sla_boundary_clarity': 0.941, 'inference_slo_coverage': 0.941}
hard_blocker_count=16
failed_cases=['sli_contract_missing_bad', 'slo_target_unmeasurable_bad', 'sla_boundary_unclear_bad', 'inference_slo_coverage_bad']
failed_gates=['sli_contract_completeness', 'slo_target_measurability', 'sla_boundary_clarity', 'inference_slo_coverage']
slo_onscall_gate_pass=False

```

这个 demo 的面试价值是把 SLO 和值班体系从“定义几个百分比指标”升级为生产可靠性治理闭环：SLI 必须有契约，SLO 必须可测量，SLA 边界必须清楚，推理、训练、数据和评估要各自有用户体验指标，错误预算要能影响发布，burn rate 告警要可行动，值班要能升级和止损，事故复盘要把失败转成行动项、回归样本、runbook 和下一次发布门禁。

47.23 常见误区

误区一：SLA、SLO、SLI 是一回事。

SLI 是指标，SLO 是目标，SLA 是对外承诺。

误区二：SLO 越高越好。

更高 SLO 需要更高成本和更慢发布速度，要和业务价值匹配。

误区三：错误预算只是 SRE 概念，AI 平台不用。

AI 平台同样需要平衡模型迭代、runtime 升级和线上稳定性。

误区四：告警越多越安全。

告警太多会造成疲劳。告警应该围绕用户影响和 SLO。

误区五：事故复盘就是找人背锅。

复盘目标是改进系统防线、检测能力和响应流程。

47.24 面试常见追问

问题一：SLI、SLO、SLA 有什么区别？

可以回答：SLI 是实际观测指标，SLO 是内部服务目标，SLA 是对客户或业务承诺的协议，通常带有责任或赔偿。

问题二：推理平台如何定义 SLO？

可以回答：可以定义请求成功率、TTFT、TPOT、p99 延迟、timeout rate、错误率，并结合安全和质量 guardrail。SLO 要反映用户体验。

问题三：错误预算有什么用？

可以回答：错误预算量化了 SLO 允许的失败空间，用来平衡稳定性和迭代速度。预算消耗过快时应降低发布风险或优先修复可靠性问题。

问题四：AI Infra 值班体系应该包含什么？

可以回答：包括告警、排班、事故分级、runbook、升级机制、回滚/降级权限、变更时间线、事故复盘和行动项跟踪。

47.25 小练习

1. 为什么 SLO 不能只看平均延迟？
2. 为推理服务设计 3 个 SLI 和 3 个 SLO。
3. 为训练平台设计一个 checkpoint SLO。
4. 错误预算耗尽后应该如何调整发布策略？
5. Burn rate 告警有什么价值？
6. 一个推理 p99 告警的 runbook 应包含什么？
7. 为什么值班操作需要审计？
8. 事故复盘应该记录哪些内容？

47.26 本章小结

本章讲了 SLO、SLA、错误预算和生产值班体系。

你需要记住：

1. SLI 是指标，SLO 是目标，SLA 是承诺。
2. AI Infra 的 SLO 要覆盖推理、训练、数据、评估、RAG/Agent 和成本相关体验。
3. 错误预算用于平衡稳定性和迭代速度，不是追求零故障。
4. 告警应围绕 SLO 和用户影响设计，避免噪音。
5. 生产值班体系需要告警、runbook、事故分级、权限、变更管理和复盘机制。
6. 更高可靠性通常意味着更高成本，SLO 要和业务价值匹配。

下一章我们会讲成本治理：GPU 成本、存储成本、网络成本和推理成本。

第 48 章 成本治理：GPU 成本、存储成本、网络成本和推理成本

上一章讲了 SLO、SLA、错误预算和生产值班体系。本章讲 AI Infra 成本治理。

AI Infra 的成本非常高，尤其是 GPU、存储、网络和推理服务。成本治理不是财务部门月底看账单，而是平台工程中的实时能力：能看清成本从哪里来，能归因到租户、模型和任务，能在不破坏质量和 SLO 的前提下降低浪费。

先记住一句话：

AI Infra 成本治理的核心，是把不可见的资源消耗变成可观测、可归因、可预算、可优化的工程指标。

48.0 本讲资料边界与第二轮精修口径

本章按通用 AI Infra 成本治理抽象来写，不绑定某个云厂商账单、GPU 型号、Kubernetes 成本插件、FinOps 组织形态或内部计费系统。资料校准时，主要参考 FinOps 对 allocation、tagging、预算、showback / chargeback 和单位经济账的通用口径，参考 OpenCost / Kubernetes 成本分摊对 workload、namespace、label 和资源用量计量的抽象，并结合前文集群容量规划、训练调度、推理平台、缓存、可观测性、SLO 值班体系和 artifact 生命周期章节。

第二轮精修只做三件事：

1. 把 GPU、训练、推理、存储、网络、评估、RAG / Agent、日志 trace 和平台运维成本统一成可计量、可归因、可下钻的成本样本。
2. 补齐 GPU-hours、tokens per GPU hour、失败任务浪费、推理 cost per 1k tokens、缓存节省、存储生命周期、网络出站、标签归因、预算使用率、异常成本、单位经济账和 SLO / 成本权衡公式。
3. 增加一个 0 依赖 Python demo，用 toy cost cases 检查成本治理是否只是“月底看账单”，还是能把 usage、attribution、budget、quota、dashboard、告警和优化建议串成闭环。

48.1 为什么 AI Infra 成本治理重要

AI Infra 的成本特点：

1. GPU 单价高。
2. 训练任务持续时间长。
3. 推理流量可能快速增长。
4. 模型权重和 checkpoint 占用大量存储。
5. 数据读取和跨地域传输产生网络成本。
6. RAG/Agent 多轮调用会放大成本。
7. 实验数量多，很多实验不会进入生产。

如果不治理，成本会随着模型规模、用户增长和实验数量快速失控。

一个成本治理样本可以写成：

$$C_i = (u_i, g_i, t_i, p_i, s_i, n_i, a_i, b_i, q_i, d_i, o_i, z_i)$$

其中 u_i 是 usage 记录， g_i 是 GPU / accelerator 消耗， t_i 是训练任务与实验， p_i 是推理请求与 token 画像， s_i 是存储 artifact， n_i 是网络流量， a_i 是归因标签， b_i 是预算， q_i 是配额， d_i 是 dashboard 下钻证据， o_i 是优化建议， z_i 是成本治理结论。

48.2 成本治理解决什么问题

成本治理要回答：

1. 钱花在哪里？
2. 哪个团队、租户、项目、模型消耗最多？
3. 哪些资源空闲？
4. 哪些训练任务性价比低？
5. 哪些推理请求成本异常？
6. 哪些 checkpoint 可以清理？
7. 哪些模型应该量化或降级？
8. 哪些 SLO 目标导致成本过高？

没有成本归因，就只能粗暴砍资源，容易伤害核心业务。

成本治理的总门禁可以写成：

$$G_{\text{cost}} = 1 [C_{\text{usage}} \geq \tau_u \wedge C_{\text{attr}} \geq \tau_a \wedge U_{\text{budget}} \leq 1 \wedge R_{\text{waste}} \leq \rho_w \wedge P_0 = 0]$$

其中 C_{usage} 是用量计量覆盖率， C_{attr} 是成本归因覆盖率， U_{budget} 是预算使用率， R_{waste} 是浪费成本比例， P_0 是未关闭的成本治理硬阻断。

48.3 成本分类

AI Infra 成本大致分为：

1. GPU 成本。
2. CPU 和内存成本。
3. 存储成本。
4. 网络成本。
5. 推理运行成本。
6. 数据处理成本。
7. 评估成本。
8. 人工标注成本。
9. 运维和平台成本。

其中 GPU、存储、网络 and 推理成本最常被关注。

总成本可以按类型分解：

$$K_{\text{total}} = K_{\text{gpu}} + K_{\text{cpu}} + K_{\text{mem}} + K_{\text{storage}} + K_{\text{network}} + K_{\text{infer}} + K_{\text{data}} + K_{\text{eval}} + K_{\text{ops}}$$

面试中要强调：成本分类必须能回到资源用量和 owner，否则只是财务账单重命名。

48.4 GPU 成本

GPU 成本通常是最大头。

它来自：

1. 训练任务。
2. 推理服务。
3. 评估任务。
4. embedding 生成。
5. 数据处理中的 GPU 加速。
6. 空闲预留资源。

GPU 成本不能只看总卡时，还要看利用率和有效产出。

例如同样消耗 1000 GPU 小时，一个任务训练出可上线模型，另一个任务因为配置错误失败，价值完全不同。

GPU 任务成本可以粗略写成：

$$K_{\text{gpu}} = G \cdot T \cdot P_{\text{gpu}} \cdot \eta_{\text{bill}}$$

其中 G 是 GPU 数， T 是运行小时数， P_{gpu} 是单 GPU 小时价格， η_{bill} 是计费修正系数，例如预留、竞价、包年包月或云上折扣。成本治理不能只看 K_{gpu} ，还要看这笔钱产生了多少有效训练 token、模型版本、评估提升或业务收益。

48.5 GPU 成本指标

常见 GPU 成本指标：

1. GPU hours。
2. GPU cost per job。
3. GPU cost per model version。
4. GPU cost per tenant。
5. GPU utilization。
6. MFU。
7. tokens per GPU hour。
8. idle GPU hours。
9. preempted GPU hours。
10. failed job GPU hours。

其中 failed job GPU hours 很重要。

失败任务消耗的 GPU 时间通常是明显浪费。

训练吞吐效率可以写成：

$$E_{\text{tok/gpu_h}} = \frac{N_{\text{token}}}{G \cdot T}$$

失败任务 GPU 成本可以写成：

$$K_{\text{failed}} = \sum_{j=1}^J G_j T_j P_j$$

其中 J 是失败任务数量。成本 dashboard 至少要把 K_{failed} 下钻到 owner、project、queue、failure type、node、image、dataset 和 checkpoint 状态。

48.6 训练成本治理

训练成本优化方向：

1. 提高 GPU 利用率。
2. 减少失败任务。
3. 减少无效实验。
4. 使用低优先级队列。
5. 支持抢占和恢复。
6. 合理 checkpoint 频率。
7. 数据供给优化。
8. 分布式通信优化。
9. 实验预算控制。
10. 任务级成本预估。

训练平台可以在任务提交时预估成本：

`estimated_cost = gpu_count * expected_hours * gpu_hour_price`

虽然不精确，但可以防止用户无意识提交超大任务。

训练任务提交前的估算可以写成：

$$\hat{K}_{\text{train}} = G_{\text{req}} \cdot \hat{T} \cdot P_{\text{gpu}} + K_{\text{storage}} + K_{\text{network}} + K_{\text{eval}}$$

如果 $\hat{K}_{\mathrm{train}}$ 超过项目预算或配额，平台应要求审批、降级队列、降低资源、拆分实验或补充业务理由。训练成本治理的目标不是阻止训练，而是让高成本实验有清楚假设、预算和失败止损条件。

48.7 推理成本

推理成本由多个因素决定：

1. 模型大小。
2. GPU 类型。
3. 输入 token 数。
4. 输出 token 数。
5. 并发量。
6. batch 效率。
7. cache 命中率。
8. 量化方式。
9. SLO 要求。
10. warm pool 规模。

推理成本常用指标：

1. cost per request。
2. cost per 1k tokens。
3. cost per output token。
4. cost per tenant。
5. cost per model。
6. cost per endpoint。
7. GPU utilization。
8. cache saved cost。

推理成本治理要和模型路由、缓存、限流、扩缩容联动。

推理单位成本常写成每 1k token 成本：

$$K_{1k} = \frac{1000K_{\mathrm{infer}}}{N_{\mathrm{in}} + N_{\mathrm{out}}}$$

其中 K_{infer} 是窗口内推理运行成本， N_{in} 和 N_{out} 分别是输入和输出 token 数。对产品和平台更有用的是分租户、分模型、分 endpoint、分请求类型计算这个指标。

48.8 推理成本优化

常见优化方式：

1. 小模型优先。
2. 分层模型路由。
3. 语义缓存。
4. prompt cache。
5. 结果缓存。
6. 输出长度限制。
7. prompt 压缩。
8. 量化。
9. continuous batching。
10. 自动扩缩容。
11. 降低 warm pool 空闲成本。
12. 低优先级请求离峰处理。

优化要注意质量和 SLO。

例如把所有请求都降到小模型，成本降了，但用户体验可能崩掉。

缓存节省成本可以粗略写成：

$$S_{\text{cache}} = N_{\text{hit}} \cdot K_{\text{prefill_saved}} - K_{\text{cache}}$$

其中 N_{hit} 是缓存命中次数， $K_{\text{prefill_saved}}$ 是单次命中节省的 prefill / retrieval / tool 成本， K_{cache} 是缓存存储、维护和一致性成本。只有 $S_{\text{cache}} > 0$ 且质量和权限不受损时，缓存才是真正的成本优化。

48.9 存储成本

AI Infra 存储成本来自：

1. raw data。
2. cleaned dataset。
3. dataset shards。
4. checkpoint。
5. model weights。
6. quantized weights。
7. eval outputs。
8. logs。
9. traces。
10. vector index。

大模型 checkpoint 和数据集非常占空间。

如果每个实验都长期保留全部 checkpoint，存储成本会迅速上升。

存储月成本可以按 artifact 类型分层：

$$K_{\text{storage}} = \sum_{a=1}^A S_a P_a T_a + K_{\text{request}} + K_{\text{restore}}$$

其中 S_a 是第 a 类 artifact 的存储大小， P_a 是单位存储价格， T_a 是保留时间， K_{request} 是请求成本， K_{restore} 是冷归档恢复成本。

48.10 存储成本指标

常见指标：

1. storage by artifact type。
2. storage by tenant。
3. storage by project。
4. checkpoint storage growth。
5. dataset storage growth。
6. hot/cold storage ratio。
7. unused artifact size。
8. duplicate artifact size。
9. log retention size。
10. trace storage size。

存储成本治理的关键是生命周期策略。

可删除候选比例可以写成：

$$R_{\text{delete}} = \frac{S_{\text{unused}} + S_{\text{duplicate}} + S_{\text{expired}}}{S_{\text{total}}}$$

但 R_{delete} 只能作为候选筛选，真正删除前必须通过血缘和回滚依赖检查。

48.11 存储成本优化

优化手段：

1. Checkpoint retention policy。
2. 删除失败实验中间产物。
3. 只保留 best/latest checkpoint。
4. 冷热分层存储。
5. 压缩。
6. 去重。
7. 权重分片复用。
8. eval output 采样保存。
9. trace 采样。
10. log TTL。

删除前必须检查血缘和部署依赖。

不能因为省存储，把可回滚的生产 artifact 删除。

删除安全门禁可以写成：

$$G_{\mathrm{delete}} = 1 [D_{\mathrm{deploy}} = 0 \wedge D_{\mathrm{rollback}} = 0 \wedge D_{\mathrm{lineage}} = 0 \wedge A_{\mathrm{owner}} = 1]$$

其中 D_{deploy} 表示是否仍被线上部署依赖， D_{rollback} 表示是否仍是回滚目标， D_{lineage} 表示是否仍被训练、评估或审计血缘依赖， A_{owner} 表示 owner 是否批准。

48.12 网络成本

网络成本常被低估。

来源包括：

1. 训练数据从对象存储读取。
2. checkpoint 上传下载。
3. 跨地域复制模型权重。
4. 分布式训练通信。
5. 推理服务跨地域调用。
6. RAG 检索访问远端索引。
7. 日志和 trace 上报。

网络成本不仅是费用，也影响延迟和稳定性。

网络成本可以写成：

$$K_{\mathrm{network}} = \sum_{r=1}^R D_r P_r$$

其中 D_r 是第 r 类数据传输量，例如跨地域 checkpoint、模型权重分发、对象存储读取、日志 trace 上报， P_r 是对应单价或内部成本权重。

48.13 网络成本优化

优化方法：

1. 数据本地性调度。
2. 本地 NVMe 缓存。

3. 权重预分发。
4. 避免跨地域训练读取。
5. checkpoint 增量上传。
6. 压缩日志和 trace。
7. 就近推理。
8. 向量索引地域化。

数据在哪里，计算就尽量在哪里。

跨地域传大模型权重和数据集通常代价很高。

数据本地性得分可以写成：

$$S_{\text{locality}} = \frac{D_{\text{local}}}{D_{\text{local}} + D_{\text{remote}}}$$

其中 D_{local} 是本地域或同集群读取的数据量， D_{remote} 是跨地域或远端读取的数据量。成本优化通常先提升 S_{locality} ，再考虑压缩、增量同步和缓存。

48.14 成本归因

成本归因是成本治理的基础。

要能按以下维度归因：

1. 租户。
2. 团队。
3. 项目。
4. 用户。
5. 模型。
6. 模型版本。
7. 训练任务。
8. 推理 endpoint。
9. dataset。
10. experiment run。

没有归因，就没有治理。

所有资源申请和任务提交都应该带 owner、project、tenant、cost center 等标签。

归因覆盖率可以写成：

$$C_{\text{attr}} = \frac{N_{\text{tagged}}}{N_{\text{resource}}}$$

未归因成本比例可以写成：

$$R_{\text{unallocated}} = \frac{K_{\text{unallocated}}}{K_{\text{total}}}$$

成熟平台应把 $R_{\text{unallocated}}$ 控制在很低水平，并对无 owner、无 cost center、无 project 的资源触发拦截或升级。

48.15 标签和成本中心

成本标签包括：

1. tenant。
2. project。
3. team。
4. owner。
5. environment。
6. job_type。
7. model。
8. priority。
9. cost_center。

平台应该强制关键标签存在。

没有标签的资源容易变成无人认领成本。

标签完整率可以写成：

$$C_{\text{tag}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{tenant}_i \wedge \text{project}_i \wedge \text{owner}_i \wedge \text{cost_center}_i]$$

这里 tenant、project、owner 和 cost_center 是正文标签字段，不建议在公式里展开成复杂变量。成本标签要在资源创建时强制，而不是月底人工补账。

48.16 预算和配额

成本治理需要预算和配额。

预算控制的是花多少钱。

配额控制的是能用多少资源。

例如：

1. 每个团队每月 GPU 预算。
2. 每个租户推理 token 预算。
3. 每个项目最大 checkpoint 存储。
4. 每个用户最大并发训练任务。
5. 每个 endpoint 最大 warm pool 副本。

预算和配额可以软限制，也可以硬限制。

软限制用于提醒，硬限制用于保护平台。

预算使用率和配额使用率分别可以写成：

$$U_{\text{budget}} = \frac{K_{\text{actual}}}{K_{\text{budget}}}$$

$$U_{\text{quota}} = \frac{R_{\text{used}}}{R_{\text{quota}}}$$

其中 K_{actual} 是实际成本， K_{budget} 是预算， R_{used} 是资源使用量， R_{quota} 是资源配额。预算回答“花了多少钱”，配额回答“用了多少资源”，两者都要有 owner 和例外审批。

48.17 成本告警

常见成本告警：

1. 日成本超过预算。
2. 某租户成本突增。
3. idle GPU hours 过高。
4. failed job cost 过高。
5. 推理 cost per 1k tokens 上升。
6. checkpoint 存储增长异常。
7. 跨地域流量异常。
8. 某 Agent run 成本异常。

成本告警要能定位责任方和资源来源。

否则只能知道“贵了”，不知道谁导致。

成本异常放大系数可以写成：

$$A_t = \frac{K_t}{\max(\epsilon, \bar{K}_{t-w:t-1})}$$

其中 K_t 是当前窗口成本， $\bar{K}_{t-w:t-1}$ 是历史窗口均值， ϵ 是防止除零的小常数。一个可行的成本告警还要附带 tenant、project、model、job、endpoint、artifact、recent change 和建议动作。

48.18 成本和 SLO 的权衡

成本优化不能孤立进行。

例如：

1. 减少 warm pool 会降低成本，但增加冷启动风险。
2. 降低副本数会省钱，但 p99 可能变差。
3. 减少 checkpoint 会省存储，但恢复成本上升。
4. 使用小模型会省推理成本，但质量可能下降。
5. 降低 trace 留存会省存储，但排障能力下降。

因此成本治理必须和 SLO、质量、安全一起决策。

成本优化门禁可以写成：

$$G_{\text{opt}} = \mathbf{1} [\Delta K < 0 \wedge \Delta Q \geq -q_0 \wedge \Delta S_{\text{slo}} \geq -s_0 \wedge R_{\text{risk}} \leq r_0]$$

其中 ΔK 是成本变化， ΔQ 是质量变化， ΔS_{slo} 是 SLO 变化， R_{risk} 是新增风险。只有成本下降且质量、SLO 和风险都在可接受范围内，优化才应该扩大使用。

48.19 Unit Economics

Unit economics 是单位经济性。

AI Infra 常见单位成本：

1. cost per 1k tokens。
2. cost per request。
3. cost per successful task。
4. cost per eval sample。
5. cost per training token。
6. cost per model version。
7. cost per RAG query。

8. cost per Agent run。

单位成本比总成本更有用。

总成本上升可能是业务增长，单位成本上升才可能说明效率变差。

单位毛利可以写成：

$$M_{\text{unit}} = V_{\text{unit}} - K_{\text{unit}}$$

其中 V_{unit} 是一次请求、一次成功任务或 1k token 带来的价值， K_{unit} 是对应单位成本。若 $M_{\text{unit}} < 0$ ，规模越大亏损越大，不能只靠扩大流量解决。

48.20 成本 Dashboard

成本 dashboard 应该展示：

1. 总成本趋势。
2. GPU 成本分布。
3. 存储成本分布。
4. 网络成本分布。
5. 推理单位成本。
6. 训练任务成本排行。
7. 租户成本排行。
8. idle 和 waste。
9. failed job cost。
10. 预算使用率。

Dashboard 要能下钻到 job、model、tenant 和 artifact。

dashboard 下钻覆盖率可以写成：

$$C_{\text{drill}} = \frac{N_{\text{cost_item_drillable}}}{N_{\text{cost_item}}}$$

如果只能看到总成本曲线，不能下钻到 job、tenant、model、endpoint、dataset、artifact 和 change event，就无法支持真正治理。

48.21 成本治理系统架构

一个成本治理系统可以这样设计：

Resource Usage Collectors

- > Usage Metering
- > Cost Attribution
- > Budget / Quota Service
- > Cost Dashboard
- > Cost Alerts
- > Optimization Recommendations

Usage 来源包括：

1. Kubernetes。
2. GPU exporter。
3. training platform。
4. inference platform。
5. object storage。
6. network billing。
7. artifact store。

8. trace/log system。

成本治理依赖统一标签和资源用量采集。

48.22 成本治理审计指标和最小 demo

把本章落到平台验收时，可以用 16 个门禁：

1. Usage Metering Coverage: GPU、CPU、memory、storage、network、request、token、artifact、logs 和 traces 是否都有用量记录。
2. Cost Attribution Labels: tenant、project、team、owner、model、job type、environment 和 cost center 是否强制存在。
3. GPU Cost Efficiency: GPU-hours、tokens per GPU hour、MFU / utilization 和空闲 GPU 小时是否可计算。
4. Training Waste Control: 失败任务、抢占丢失、无效实验和 checkpoint / resume 成本是否可定位。
5. Inference Unit Cost: cost per request、cost per 1k tokens、cost per output token、endpoint / model / tenant 成本是否可计算。
6. Cache Savings Accounting: result cache、prompt cache、KV / prefix cache 和 semantic cache 的节省与维护成本是否可量化。
7. Storage Lifecycle Governance: checkpoint、dataset、model、eval output、logs、traces 和 vector index 是否有保留、归档、删除和冷存储策略。
8. Network Egress Governance: 跨地域数据、权重、checkpoint、日志 trace 和远端索引访问是否可计量和优化。
9. Artifact Dependency Safety: 删除或降冷前是否检查部署、回滚、训练、评估、审计和血缘依赖。
10. Budget Quota Enforcement: 团队、租户、项目、endpoint、训练任务和 artifact 是否有预算 / 配额 / 例外审批。
11. Cost Anomaly Alerting: 日成本、单位成本、idle、failed job、storage growth、egress 和 agent run 异常是否能告警到 owner。
12. Tenant Model Chargeback: 成本是否能按租户、模型、模型版本、endpoint、job 和 experiment 做 showback / chargeback。
13. Dashboard Drilldown Readiness: dashboard 是否能从总成本下钻到 tenant、model、job、artifact、request slice 和 recent change。
14. Optimization Recommendation Trace: 优化建议是否带成本节省、质量 / SLO 风险、owner、证据和回滚条件。
15. SLO Quality Cost Tradeoff: 降本动作是否同时检查质量、SLO、安全、回滚和用户体验。
16. Cost Governance Gate: 最终是否有 owner、预算策略、优化 backlog、审批记录、审计和 P0 风险阻断。

综合门禁：

$$G_{\text{cost_governance}} = \prod_{j=1}^{16} G_j$$

下面是一个 0 依赖 demo，用 toy cost cases 检查成本治理是否只是“月底看账单”，还是能把用量、归因、预算、配额、告警、dashboard 和优化建议串成闭环。

```
from copy import deepcopy
```

```
class MiniAICostGovernanceAudit:
    GATES = [
        "usage_metering_coverage",
        "cost_attribution_labels",
        "gpu_cost_efficiency",
        "training_waste_control",
        "inference_unit_cost",
        "cache_savings_accounting",
```

```

        "storage_lifecycle_governance",
        "network_egress_governance",
        "artifact_dependency_safety",
        "budget_quota_enforcement",
        "cost_anomaly_alerting",
        "tenant_model_chargeback",
        "dashboard_drilldown_readiness",
        "optimization_recommendation_trace",
        "slo_quality_cost_tradeoff",
        "cost_governance_gate",
    ]

USAGE_FIELDS = ["gpu", "cpu", "memory", "storage", "network", "request", "token", "artifact", "logs", "traces"]
LABEL_FIELDS = ["tenant", "project", "team", "owner", "model", "job_type", "environment", "cost_center"]
DASHBOARD_FIELDS = ["tenant", "model", "job", "artifact", "endpoint", "request_slice", "recent_change"]

@staticmethod
def present(record, key):
    return key in record and record[key] is not None and record[key] != ""

def coverage(self, record, fields):
    if not record:
        return 0.0
    return sum(1 for field in fields if self.present(record, field)) / len(fields)

def gpu_job_cost(self, case):
    gpu = case["gpu"]
    return gpu["count"] * gpu["hours"] * gpu["price_per_hour"] * gpu["billing_factor"]

def tokens_per_gpu_hour(self, case):
    gpu = case["gpu"]
    return gpu["train_tokens"] / max(1e-12, gpu["count"] * gpu["hours"])

def failed_job_cost(self, case):
    return sum(job["gpu_count"] * job["hours"] * job["price_per_hour"] for job in case["training_waste"]["failed_jobs"])

def inference_cost_per_1k_tokens(self, case):
    inference = case["inference"]
    total_tokens = inference["input_tokens"] + inference["output_tokens"]
    return 1000.0 * inference["cost"] / total_tokens

def cache_saved_cost(self, case):
    cache = case["cache"]
    return cache["hits"] * cache["saved_cost_per_hit"] - cache["maintenance_cost"]

def storage_monthly_cost(self, case):
    storage = case["storage"]
    return sum(item["gib"] * item["price_per_gib_month"] for item in storage["artifacts"])

def network_egress_cost(self, case):
    return sum(flow["gib"] * flow["price_per_gib"] for flow in case["network"]["flows"])

def budget_used(self, case):
    budget = case["budget"]
    return budget["actual_cost"] / budget["monthly_budget"]

```

```

def unit_margin(self, case):
    unit = case["unit_economics"]
    return unit["value_per_unit"] - unit["cost_per_unit"]

def usage_metering_coverage(self, case):
    usage = case.get("usage_metering", {})
    return self.coverage(usage, self.USAGE_FIELDS) == 1.0 and usage.get("freshness_minutes", 10**9) <= usage.get(

def cost_attribution_labels(self, case):
    labels = case.get("labels", {})
    return self.coverage(labels, self.LABEL_FIELDS) == 1.0 and labels.get("unallocated_cost_ratio", 1.0) <= label

def gpu_cost_efficiency(self, case):
    gpu = case.get("gpu", {})
    return (
        self.gpu_job_cost(case) <= gpu.get("max_job_cost", 0.0)
        and self.tokens_per_gpu_hour(case) >= gpu.get("min_tokens_per_gpu_hour", 10**18)
        and gpu.get("mfu", 0.0) >= gpu.get("min_mfu", 1.0)
        and gpu.get("idle_gpu_hours", 10**9) <= gpu.get("max_idle_gpu_hours", 0)
    )

def training_waste_control(self, case):
    waste = case.get("training_waste", {})
    return (
        self.failed_job_cost(case) <= waste.get("max_failed_job_cost", 0.0)
        and waste.get("invalid_experiments", 10**9) <= waste.get("max_invalid_experiments", 0)
        and waste.get("resume_ready") is True
        and waste.get("failure_owner_ready") is True
    )

def inference_unit_cost(self, case):
    inference = case.get("inference", {})
    return (
        self.inference_cost_per_1k_tokens(case) <= inference.get("max_cost_per_1k_tokens", 0.0)
        and inference.get("cost_per_request", 10**9) <= inference.get("max_cost_per_request", 0.0)
        and inference.get("tenant_breakdown") is True
        and inference.get("model_endpoint_breakdown") is True
    )

def cache_savings_accounting(self, case):
    cache = case.get("cache", {})
    return (
        self.cache_saved_cost(case) >= cache.get("min_saved_cost", 10**9)
        and cache.get("permission_safe") is True
        and cache.get("quality_regression") <= cache.get("max_quality_regression", 0.0)
        and cache.get("versioned_keys") is True
    )

def storage_lifecycle_governance(self, case):
    storage = case.get("storage", {})
    return (
        self.storage_monthly_cost(case) <= storage.get("max_monthly_cost", 0.0)
        and storage.get("retention_policy") is True
        and storage.get("hot_cold_tiering") is True
    )

```

```

        and storage.get("duplicate_ratio") <= storage.get("max_duplicate_ratio", 0.0)
        and storage.get("log_trace_ttl") is True
    )

def network_egress_governance(self, case):
    network = case.get("network", {})
    local = network["local_gib"]
    remote = network["remote_gib"]
    locality = local / max(1e-12, local + remote)
    return (
        self.network_egress_cost(case) <= network.get("max_egress_cost", 0.0)
        and locality >= network.get("min_locality", 1.0)
        and network.get("cross_region_reason") is True
        and network.get("compression_or_incremental") is True
    )

def artifact_dependency_safety(self, case):
    artifact = case.get("artifact_delete", {})
    return (
        artifact.get("deployment_dependency") is False
        and artifact.get("rollback_dependency") is False
        and artifact.get("lineage_dependency") is False
        and artifact.get("owner_approved") is True
        and artifact.get("delete_plan_audited") is True
    )

def budget_quota_enforcement(self, case):
    budget = case.get("budget", {})
    return (
        self.budget_used(case) <= budget.get("max_budget_used", 0.0)
        and budget.get("quota_used") <= budget.get("quota_limit")
        and budget.get("exception_approval") is True
        and budget.get("hard_limit_for_p0") is True
    )

def cost_anomaly_alerting(self, case):
    alerts = case.get("alerts", {})
    return (
        alerts.get("daily_cost_ratio", 10**9) <= alerts.get("max_daily_cost_ratio", 0.0)
        and alerts.get("unit_cost_ratio", 10**9) <= alerts.get("max_unit_cost_ratio", 0.0)
        and alerts.get("owner_routed") is True
        and alerts.get("recent_change_attached") is True
        and alerts.get("recommendation_attached") is True
    )

def tenant_model_chargeback(self, case):
    chargeback = case.get("chargeback", {})
    return (
        chargeback.get("tenant") is True
        and chargeback.get("model") is True
        and chargeback.get("model_version") is True
        and chargeback.get("endpoint") is True
        and chargeback.get("job") is True
        and chargeback.get("experiment") is True
        and chargeback.get("cost_center_count", 0) >= chargeback.get("min_cost_center_count", 10**9)
    )

```



```

    )

def dashboard_drilldown_readiness(self, case):
    dashboard = case.get("dashboard", {})
    return self.coverage(dashboard.get("drilldown", {}), self.DASHBOARD_FIELDS) == 1.0 and dashboard.get("fresh")

def optimization_recommendation_trace(self, case):
    rec = case.get("recommendation", {})
    return (
        rec.get("estimated_savings", 0.0) >= rec.get("min_estimated_savings", 10**9)
        and rec.get("evidence") is True
        and rec.get("owner") is True
        and rec.get("rollback_condition") is True
        and rec.get("quality_slo_risk_reviewed") is True
    )

def slo_quality_cost_tradeoff(self, case):
    tradeoff = case.get("tradeoff", {})
    return (
        tradeoff.get("cost_delta", 0.0) < 0
        and tradeoff.get("quality_delta", -1.0) >= -tradeoff.get("max_quality_drop", 0.0)
        and tradeoff.get("slo_delta", -1.0) >= -tradeoff.get("max_slo_drop", 0.0)
        and tradeoff.get("risk_score", 1.0) <= tradeoff.get("max_risk_score", 0.0)
    )

def cost_governance_gate(self, case):
    gate = case.get("platform_gate", {})
    return (
        gate.get("enabled") is True
        and bool(gate.get("owner"))
        and gate.get("budget_policy") is True
        and gate.get("optimization_backlog") is True
        and gate.get("approval_records") is True
        and gate.get("audit_ready") is True
        and gate.get("p0_open") is False
    )

def audit_case(self, case):
    return {gate: getattr(self, gate)(case) for gate in self.GATES}

def run_all(self, cases):
    results = {case["case_id"]: self.audit_case(case) for case in cases}
    metrics = {}
    for gate in self.GATES:
        passed = sum(1 for result in results.values() if result[gate])
        metrics[gate] = round(passed / len(cases), 3)
    failed_cases = [
        case_id
        for case_id, result in results.items()
        if not all(result.values())
    ]
    failed_gates = [
        gate
        for gate in self.GATES
        if any(not result[gate] for result in results.values())
    ]

```

```

]
return {
    "metrics": metrics,
    "hard_blocker_count": len(failed_cases),
    "failed_cases": failed_cases,
    "failed_gates": failed_gates,
    "cost_governance_gate_pass": metrics["cost_governance_gate"] == 1.0,
}

def example_outputs(self, case):
    return {
        "gpu_job_cost_usd": round(self.gpu_job_cost(case), 3),
        "tokens_per_gpu_hour": round(self.tokens_per_gpu_hour(case), 3),
        "failed_job_cost_usd": round(self.failed_job_cost(case), 3),
        "cost_per_1k_tokens_usd": round(self.inference_cost_per_1k_tokens(case), 3),
        "cache_saved_cost_usd": round(self.cache_saved_cost(case), 3),
        "storage_monthly_cost_usd": round(self.storage_monthly_cost(case), 3),
        "network_egress_cost_usd": round(self.network_egress_cost(case), 3),
        "budget_used": round(self.budget_used(case), 3),
        "idle_gpu_hours": case["gpu"]["idle_gpu_hours"],
        "unit_margin": round(self.unit_margin(case), 3),
        "cost_center_count": case["chargeback"]["cost_center_count"],
    }

def build_good_case():
    return {
        "case_id": "full_cost_governance",
        "usage_metering": {
            "gpu": True,
            "cpu": True,
            "memory": True,
            "storage": True,
            "network": True,
            "request": True,
            "token": True,
            "artifact": True,
            "logs": True,
            "traces": True,
            "freshness_minutes": 10,
            "max_freshness_minutes": 30,
        },
        "labels": {
            "tenant": "enterprise-a",
            "project": "assistant",
            "team": "infra",
            "owner": "cost-owner",
            "model": "chat-v8",
            "job_type": "sft",
            "environment": "prod",
            "cost_center": "cc-ai-01",
            "unallocated_cost_ratio": 0.02,
            "max_unallocated_cost_ratio": 0.05,
        },
        "gpu": {

```

```

    "count": 8,
    "hours": 10,
    "price_per_hour": 32.0,
    "billing_factor": 1.0,
    "train_tokens": 400_000_000,
    "max_job_cost": 3000.0,
    "min_tokens_per_gpu_hour": 4_000_000,
    "mfu": 0.42,
    "min_mfu": 0.35,
    "idle_gpu_hours": 12,
    "max_idle_gpu_hours": 16,
  },
  "training_waste": {
    "failed_jobs": [
      {"gpu_count": 2, "hours": 2, "price_per_hour": 32.0},
      {"gpu_count": 1, "hours": 1, "price_per_hour": 32.0},
    ],
    "max_failed_job_cost": 200.0,
    "invalid_experiments": 1,
    "max_invalid_experiments": 1,
    "resume_ready": True,
    "failure_owner_ready": True,
  },
  "inference": {
    "cost": 90.0,
    "input_tokens": 3_000_000,
    "output_tokens": 2_000_000,
    "max_cost_per_1k_tokens": 0.02,
    "cost_per_request": 0.012,
    "max_cost_per_request": 0.02,
    "tenant_breakdown": True,
    "model_endpoint_breakdown": True,
  },
  "cache": {
    "hits": 1000,
    "saved_cost_per_hit": 0.06,
    "maintenance_cost": 6.0,
    "min_saved_cost": 40.0,
    "permission_safe": True,
    "quality_regression": 0.0,
    "max_quality_regression": 0.01,
    "versioned_keys": True,
  },
  "storage": {
    "artifacts": [
      {"type": "checkpoint", "gib": 5000, "price_per_gib_month": 0.4},
      {"type": "dataset", "gib": 3000, "price_per_gib_month": 0.25},
      {"type": "trace", "gib": 700, "price_per_gib_month": 0.2},
    ],
    "max_monthly_cost": 3200.0,
    "retention_policy": True,
    "hot_cold_tiering": True,
    "duplicate_ratio": 0.04,
    "max_duplicate_ratio": 0.08,
    "log_trace_ttl": True,
  },

```

```

},
"network": {
  "flows": [
    {"name": "weights_cross_region", "gib": 400, "price_per_gib": 0.12},
    {"name": "logs_trace_export", "gib": 600, "price_per_gib": 0.10},
  ],
  "max_egress_cost": 120.0,
  "local_gib": 9000,
  "remote_gib": 1000,
  "min_locality": 0.85,
  "cross_region_reason": True,
  "compression_or_incremental": True,
},
"artifact_delete": {
  "deployment_dependency": False,
  "rollback_dependency": False,
  "lineage_dependency": False,
  "owner_approved": True,
  "delete_plan_audited": True,
},
"budget": {
  "actual_cost": 7200.0,
  "monthly_budget": 10000.0,
  "max_budget_used": 0.8,
  "quota_used": 80,
  "quota_limit": 100,
  "exception_approval": True,
  "hard_limit_for_p0": True,
},
>alerts": {
  "daily_cost_ratio": 1.2,
  "max_daily_cost_ratio": 1.5,
  "unit_cost_ratio": 1.1,
  "max_unit_cost_ratio": 1.25,
  "owner_routed": True,
  "recent_change_attached": True,
  "recommendation_attached": True,
},
"chargeback": {
  "tenant": True,
  "model": True,
  "model_version": True,
  "endpoint": True,
  "job": True,
  "experiment": True,
  "cost_center_count": 3,
  "min_cost_center_count": 3,
},
"dashboard": {
  "drilldown": {
    "tenant": True,
    "model": True,
    "job": True,
    "artifact": True,
    "endpoint": True,
  }
}

```

```

        "request_slice": True,
        "recent_change": True,
    },
    "fresh": True,
},
"recommendation": {
    "estimated_savings": 650.0,
    "min_estimated_savings": 500.0,
    "evidence": True,
    "owner": True,
    "rollback_condition": True,
    "quality_slo_risk_reviewed": True,
},
"tradeoff": {
    "cost_delta": -0.18,
    "quality_delta": -0.005,
    "slo_delta": -0.002,
    "risk_score": 0.1,
    "max_quality_drop": 0.01,
    "max_slo_drop": 0.005,
    "max_risk_score": 0.2,
},
"unit_economics": {
    "value_per_unit": 0.10,
    "cost_per_unit": 0.018,
},
"platform_gate": {
    "enabled": True,
    "owner": "finops-ai-infra",
    "budget_policy": True,
    "optimization_backlog": True,
    "approval_records": True,
    "audit_ready": True,
    "p0_open": False,
},
}

```

```

def build_bad_cases(good_case):
    cases = []

    case = deepcopy(good_case)
    case["case_id"] = "usage_metering_missing_bad"
    case["usage_metering"]["token"] = None
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "cost_labels_missing_bad"
    case["labels"].pop("cost_center")
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "gpu_efficiency_bad"
    case["gpu"]["idle_gpu_hours"] = 40
    cases.append(case)

```

```

case = deepcopy(good_case)
case["case_id"] = "training_waste_bad"
case["training_waste"]["failed_jobs"].append({"gpu_count": 4, "hours": 4, "price_per_hour": 32.0})
cases.append(case)

```

```

case = deepcopy(good_case)
case["case_id"] = "inference_unit_cost_bad"
case["inference"]["cost"] = 130.0
cases.append(case)

```

```

case = deepcopy(good_case)
case["case_id"] = "cache_savings_bad"
case["cache"]["permission_safe"] = False
cases.append(case)

```

```

case = deepcopy(good_case)
case["case_id"] = "storage_lifecycle_bad"
case["storage"]["retention_policy"] = False
cases.append(case)

```

```

case = deepcopy(good_case)
case["case_id"] = "network_egress_bad"
case["network"]["remote_gib"] = 4000
cases.append(case)

```

```

case = deepcopy(good_case)
case["case_id"] = "artifact_dependency_bad"
case["artifact_delete"]["rollback_dependency"] = True
cases.append(case)

```

```

case = deepcopy(good_case)
case["case_id"] = "budget_quota_bad"
case["budget"]["actual_cost"] = 9300.0
cases.append(case)

```

```

case = deepcopy(good_case)
case["case_id"] = "cost_anomaly_alerting_bad"
case["alerts"]["owner_routed"] = False
cases.append(case)

```

```

case = deepcopy(good_case)
case["case_id"] = "chargeback_bad"
case["chargeback"]["model_version"] = False
cases.append(case)

```

```

case = deepcopy(good_case)
case["case_id"] = "dashboard_drilldown_bad"
case["dashboard"]["drilldown"].pop("request_slice")
cases.append(case)

```

```

case = deepcopy(good_case)
case["case_id"] = "recommendation_trace_bad"
case["recommendation"]["quality_slo_risk_reviewed"] = False
cases.append(case)

```

```

case = deepcopy(good_case)
case["case_id"] = "slo_quality_tradeoff_bad"
case["tradeoff"]["quality_delta"] = -0.05
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "cost_governance_gate_missing_bad"
case["platform_gate"]["enabled"] = False
cases.append(case)

return cases

```

```

audit = MiniAICostGovernanceAudit()
good = build_good_case()
cases = [good] + build_bad_cases(good)
summary = audit.run_all(cases)

print("cost_governance_examples=" + repr(audit.example_outputs(good)))
print("metrics=" + repr(summary["metrics"]))
print("hard_blocker_count=" + repr(summary["hard_blocker_count"]))
print("failed_cases=" + repr(summary["failed_cases"]))
print("failed_gates=" + repr(summary["failed_gates"]))
print("cost_governance_gate_pass=" + repr(summary["cost_governance_gate_pass"]))

```

参考输出应类似：

```

cost_governance_examples={'gpu_job_cost_usd': 2560.0, 'tokens_per_gpu_hour': 5000000.0, 'failed_job_cost_usd': 160.0, '
metrics={'usage_metering_coverage': 0.941, 'cost_attribution_labels': 0.941, 'gpu_cost_efficiency': 0.941, 'training_wa
hard_blocker_count=16
failed_cases=['usage_metering_missing_bad', 'cost_labels_missing_bad', 'gpu_efficiency_bad', 'training_waste_bad', 'in
failed_gates=['usage_metering_coverage', 'cost_attribution_labels', 'gpu_cost_efficiency', 'training_waste_control', '
cost_governance_gate_pass=False

```

这个 demo 的面试价值是把成本治理从“月底账单分析”升级为工程控制闭环：先采集 usage，再强制 attribution，然后按 GPU、训练、推理、缓存、存储、网络 and artifact 拆成本，接着用预算、配额、异常告警和 dashboard 下钻定位责任方，最后用质量 / SLO / 风险门禁决定优化建议能不能落地。

48.23 常见误区

误区一：成本治理就是月底看账单。

月底账单只能事后分析，平台需要实时或准实时用量和预算监控。

误区二：GPU 利用率高就成本健康。

如果任务无效、失败或模型不可用，高利用率也可能是浪费。

误区三：只优化训练成本。

推理、存储、网络、评估和 trace 都可能成为大头。

误区四：成本优化就是砍资源。

真正优化是提高单位产出，例如更高 tokens/GPU hour、更低 cost per 1k tokens。

误区五：存储便宜，不用治理。

大模型 checkpoint、数据集和 trace 规模很大，长期看存储成本不可忽视。

48.24 面试常见追问

问题一：AI Infra 成本主要来自哪里？

可以回答：主要来自 GPU 训练和推理、模型和 checkpoint 存储、数据和权重传输网络成本、评估和 embedding 生成、日志 trace 留存，以及人工标注和平台运维。

问题二：如何降低推理成本？

可以回答：通过小模型优先、模型路由、缓存、prompt 压缩、输出长度控制、量化、continuous batching、自动扩缩容、warm pool 优化和单位 token 成本监控降低成本。

问题三：如何做成本归因？

可以回答：所有资源、任务、模型、endpoint、artifact 都必须带 tenant、team、project、owner、model、job_type、cost_center 等标签，再结合 usage metering 归因到租户、团队、模型和任务。

问题四：成本优化和 SLO 冲突怎么办？

可以回答：成本优化必须在质量和 SLO 约束下进行。可以分层服务，不同业务使用不同 SLO、模型、缓存、配额和降级策略，而不是一刀切省钱。

48.25 小练习

1. GPU hours 和 tokens per GPU hour 分别说明什么？
2. 为什么 failed job GPU hours 是重要成本指标？
3. 推理 cost per 1k tokens 受哪些因素影响？
4. Checkpoint retention policy 如何设计？
5. 网络成本在 AI Infra 中有哪些来源？
6. 为什么成本标签必须强制填写？
7. 如何设计租户级推理 token 预算？
8. 成本优化如何避免伤害 SLO？

48.26 本章小结

本章讲了 AI Infra 成本治理。

你需要记住：

1. 成本治理的目标是让资源消耗可观测、可归因、可预算、可优化。
2. GPU、推理、存储、网络、评估和 trace 都可能产生显著成本。
3. 成本归因依赖统一标签、用量采集和成本中心。
4. 训练成本要关注 GPU hours、失败任务、tokens/GPU hour 和资源利用率。
5. 推理成本要关注 cost per request、cost per 1k tokens、cache 命中率和模型路由。
6. 成本优化必须和 SLO、质量、安全和回滚能力一起权衡。

下一章我们会讲资源利用率优化：混部、抢占、弹性训练和低优先级队列。

第 49 章 资源利用率优化：混部、抢占、弹性训练和低优先级队列

上一章讲了成本治理。本章继续讲资源利用率优化。

AI Infra 中最昂贵的资源通常是 GPU。GPU 买了或租了以后，如果长期空闲，就是直接浪费。但为了保证训练任务和线上推理的稳定性，又不能简单把所有资源塞满。因此资源利用率优化的核心，是在稳定性、隔离、公平性和成本之间做工程权衡。

先记住一句话：

GPU 利用率优化不是把集群打满，而是在满足优先级、SLO 和隔离约束下，减少空闲、减少碎片、减少失败和提高有效产出。

49.0 本讲资料边界与第二轮精修口径

本章按 WRITING_PLAN.md 做第二轮精修时，主要参考 Kubernetes 对 Pod 资源请求 / 限制、调度、PriorityClass、preemption、ResourceQuota、bin packing 和 overprovisioning 的官方口径，参考 Kueue 对 queue、ClusterQueue、cohort borrowing、preemption 和 fair sharing 的调度治理模型，参考 PyTorch Elastic / torchrun 对弹性训练、rendezvous 和 worker 重启的说明，也参考 NVIDIA Kubernetes device plugin 对 GPU sharing、time-slicing 和 MPS 的边界说明。

需要强调三点：

1. 本章讨论的是 AI Infra 资源利用率治理的稳定工程问题，不绑定某个云厂商、调度器插件、Kubernetes 发行版、GPU 型号或内部平台实现。
2. 利用率指标不能脱离 SLO、隔离、公平性、checkpoint、恢复成本和业务价值单独优化。
3. 文中 demo 是教学级审计脚本，只用于解释资源治理门禁，不代表真实调度器、预测器或成本系统。

49.1 为什么资源利用率难

GPU 集群利用率难优化，原因包括：

1. 任务资源需求差异大。
2. 大模型训练需要 gang scheduling。
3. 多机多卡任务容易产生资源碎片。
4. 推理服务需要保留冗余和 warm pool。
5. 训练任务运行时间长。
6. 低优先级任务可能被抢占。
7. 不同 GPU 型号不可完全替代。
8. 数据、网络、存储也会限制 GPU 使用。

所以利用率优化不是单纯提高 GPU utilization，而是整体资源调度问题。

49.2 利用率指标

常见指标包括：

1. GPU allocation rate。
2. GPU utilization。
3. GPU memory utilization。
4. MFU。
5. tokens per GPU hour。
6. idle GPU hours。
7. pending jobs。
8. queue wait time。
9. preempted job count。
10. failed job GPU hours。
11. fragmentation rate。

GPU allocation rate 表示 GPU 是否被分配。

GPU utilization 表示 GPU 分配后是否真的忙。

两者都要看。

GPU 被分配但利用率很低，可能是数据、通信或配置问题。

可以先把一个时间窗口内的资源记录抽象为：

$$R_i = (g_i, t_i, u_i, m_i, q_i, p_i, s_i, c_i, z_i)$$

其中 g_i 是 GPU 数， t_i 是占用时长， u_i 是平均 GPU 利用率， m_i 是显存利用率， q_i 是队列， p_i 是优先级， s_i 是 SLO 通过状态， c_i 是 checkpoint / resume 状态， z_i 是标签和 owner 证据。

GPU 分配率:

$$U_{\text{alloc}} = \frac{G_{\text{allocated}}}{G_{\text{total}}}$$

GPU 活跃利用率可以按 GPU-hours 加权:

$$U_{\text{active}} = \frac{\sum_i g_i t_i u_i}{\sum_i g_i t_i}$$

空闲 GPU-hours:

$$H_{\text{idle}} = \sum_{n=1}^N g_{n,\text{idle}} T$$

其中 T 是统计窗口长度。面试中要特别说明: U_{alloc} 高只能说明资源被占用, U_{active} 高才说明占用期间较忙, 但二者都不等于 “产出有效”。

49.3 有效利用率

有效利用率比表面利用率更重要。

例如:

1. GPU 满载跑失败任务, 不是有效利用。
2. GPU 满载跑错误配置实验, 不是有效利用。
3. GPU 满载但训练数据重复严重, 产出价值低。
4. 推理 GPU 满载但 p99 崩掉, 也不是好状态。

有效利用率要结合产出:

1. 成功训练 step。
2. 有效 tokens。
3. 成功请求。
4. 符合 SLO 的吞吐。
5. 可发布模型产物。

可以用一个教学版有效利用率表达:

$$U_{\text{eff}} = \frac{\sum_i h_i v_i s_i}{\sum_i h_i}$$

其中 $h_i = g_i t_i$ 是任务 GPU-hours, v_i 是有效产出门禁, 例如训练成功、评估可用、模型可发布或请求成功, s_i 是 SLO / 质量门禁。这个式子提醒你: 失败任务、无效实验和破坏 SLO 的推理吞吐, 都不能算成有效利用率。

49.4 资源碎片

资源碎片是 GPU 集群常见问题。

例如集群中有 8 台机器, 每台 8 卡, 但空闲 GPU 分散在不同机器上:

```
node1: 2 free
node2: 1 free
node3: 3 free
node4: 2 free
```

总共有 8 张空闲 GPU，但没有一台机器有 8 张连续 GPU，无法启动一个单机 8 卡任务。

多机多卡任务还需要网络拓扑匹配，碎片问题更严重。

对于一个需要每台节点至少 g 张同型 GPU 的任务，可以定义碎片比例：

$$R_{\text{frag}}(g) = 1 - \frac{\sum_{n=1}^N \lfloor f_n/g \rfloor g}{\sum_{n=1}^N f_n}$$

其中 f_n 是第 n 个节点上的空闲 GPU 数。上面的例子里 $g=8$ 时，没有任何节点能放下单机 8 卡任务，所以可用于该任务的连续空闲 GPU 为 0， $R_{\text{frag}}(8)=1$ 。

49.5 减少碎片的方法

常见方法：

1. 拓扑感知调度。
2. bin packing。
3. 大任务优先或预留。
4. 队列重排。
5. 低优先级任务可抢占。
6. 任务资源规格标准化。
7. 避免小任务随意占用整机关键位置。
8. 定期 defragmentation。

调度器要知道 GPU 拓扑，而不是只看剩余卡数。

一个简单的 bin packing 打分可以写成：

$$S_{\text{pack}}(n) = \sum_r w_r u_{n,r}$$

其中 $u_{\{n,r\}}$ 是节点 n 上资源 r 的已使用比例， w_r 是权重。Most-allocated 类策略会倾向把新任务放到已经较满但仍能满足约束的节点，从而保留更完整的空闲节点给大任务。AI Infra 里还要叠加拓扑和干扰风险：

$$S_{\text{node}} = \alpha S_{\text{pack}} + \beta S_{\text{topo}} - \gamma R_{\text{interfere}}$$

其中 S_{topo} 衡量 GPU、NIC、机柜和存储局部性， $R_{\text{interfere}}$ 衡量混部干扰风险。

49.6 混部是什么

混部是把不同类型任务放在同一资源池或同一节点上运行，以提高资源利用率。

常见混部：

1. 训练和推理混部。
2. 在线和离线混部。
3. 高优先级和低优先级混部。
4. GPU 任务和 CPU 任务混部。
5. embedding 任务和训练任务混部。

混部目标是利用资源空隙。

但混部会带来干扰。

混部收益可以先用闲置资源填充率表示：

$$F_{\text{fill}} = \frac{H_{\text{reclaimed}}}{H_{\text{idle,before}}}$$

其中 $H_{\text{reclaimed}}$ 是被低优先级或可中断任务填充的 GPU-hours, $H_{\text{idle,before}}$ 是混部前预计空闲 GPU-hours。这个指标只说明填洞能力，不说明混部是否安全。

49.7 混部风险

混部风险包括：

1. GPU 显存竞争。
2. CPU 竞争。
3. 内存竞争。
4. 网络竞争。
5. 磁盘 I/O 竞争。
6. 推理 p99 延迟抖动。
7. 训练 step time 抖动。
8. 故障影响扩大。

在线推理任务和离线训练混部尤其要谨慎。

如果离线任务导致在线推理 p99 上升，节省的成本可能不值得。

混部干扰可以按核心 SLO 指标估计：

$$D_{\text{lat}} = \frac{L_{\text{p99,mix}} - L_{\text{p99,base}}}{L_{\text{p99,base}}}$$

$$D_{\text{step}} = \frac{T_{\text{step,mix}} - T_{\text{step,base}}}{T_{\text{step,base}}}$$

其中 D_{lat} 是在线推理尾延迟放大， D_{step} 是训练 step time 放大。只要混部节省没有覆盖 SLO 风险、故障风险和排障成本，就不应该扩大使用。

49.8 安全混部策略

更安全的混部方式：

1. 按优先级隔离。
2. 按 GPU 型号隔离。
3. 在线服务保留资源。
4. 离线任务只使用可抢占资源。
5. 限制低优先级任务资源上限。
6. 使用 node taint/toleration。
7. 使用 QoS class。
8. 对在线服务设置 latency guardrail。
9. 异常时自动驱逐低优先级任务。

混部必须有监控和自动保护，不应靠人工观察。

安全混部门禁可以写成：

$$G_{\text{mix}} = \mathbf{1} [D_{\text{lat}} \leq \epsilon_{\text{lat}} \wedge D_{\text{step}} \leq \epsilon_{\text{step}} \wedge R_{\text{evict}} \leq \rho \wedge P_0 = 0]$$

其中 R_{evict} 是驱逐或抢占导致的异常比例， P_0 是高严重度风险数量。这个门禁强调：混部不是资源利用率技巧，而是带自动保护的调度策略。

49.9 抢占是什么

抢占是高优先级任务需要资源时，平台停止或迁移低优先级任务，释放资源。

抢占适合：

1. 低优先级训练。
2. 探索性实验。
3. 离线 embedding。
4. 批量评估。
5. 可恢复任务。

不适合：

1. 关键线上推理。
2. 无 checkpoint 的长任务。
3. 强实时任务。
4. 不可重复执行任务。

抢占的前提是任务能恢复。

抢占损失可以用 GPU-hours 估计：

$$L_j = g_j(t_{\text{now}} - t_{\text{ckpt},j})$$

其中 g_j 是被抢占任务的 GPU 数, t_{now} 是抢占时间, $t_{\text{ckpt},j}$ 是最近可恢复 checkpoint 时间。没有 checkpoint 或 checkpoint 太旧时，抢占的真实成本会非常高。

49.10 抢占设计

抢占系统要考虑：

1. 谁可以抢占谁。
2. 抢占前是否保存 checkpoint。
3. 给任务多少 graceful shutdown 时间。
4. 被抢占任务如何重新排队。
5. 抢占次数是否有限制。
6. 是否补偿低优先级队列。
7. 抢占事件如何通知用户。

抢占不能粗暴 kill。

否则会浪费大量已完成训练进度。

一次抢占是否值得，可以用收益减损失判断：

$$B_{\text{preempt}} = V_{\text{high}} - L_{\text{low}} - K_{\text{resume}} - K_{\text{noise}}$$

其中 V_{high} 是高优先级任务按时启动的价值， L_{low} 是低优先级任务丢失 GPU-hours， K_{resume} 是恢复成本， K_{noise} 是用户通知、队列抖动和排障成本。生产系统通常只允许 $B_{\text{preempt}} > 0$ 且恢复证据完整的抢占。

49.11 低优先级队列

低优先级队列用于吸收闲置资源。

适合放：

1. 探索实验。
2. 批量评估。
3. embedding 生成。

4. 数据处理。
5. 非紧急微调。
6. 可中断任务。

低优先级队列的特点：

1. 等待时间不保证。
2. 可被抢占。
3. 成本较低。
4. 适合填充资源空洞。

这能提升集群整体利用率。

低优先级队列可用容量可以近似为：

$$C_{\text{low}} = \max(0, C_{\text{total}} - C_{\text{reserved}} - C_{\text{high,pending}})$$

其中 C_{total} 是总容量， C_{reserved} 是 SLO、故障恢复和关键任务预留， $C_{\text{high,pending}}$ 是高优先级待满足需求。低优先级任务只能吃这个“可回收余量”，不能挤占硬预留。

49.12 弹性训练是什么

弹性训练是训练任务可以在不同资源规模下运行。

例如一个任务可以从 64 卡扩到 128 卡，也可以从 128 卡缩到 64 卡。

弹性训练的价值：

1. 提高资源利用率。
2. 减少排队等待。
3. 支持抢占后继续运行。
4. 利用临时空闲 GPU。
5. 高峰期释放资源给更重要任务。

但弹性训练对框架和训练逻辑要求高。

弹性训练的基本批量关系是：

$$B_{\text{global}} = b_{\text{micro}} a_{\text{grad}} w$$

其中 b_{micro} 是单 worker micro batch， a_{grad} 是梯度累积步数， w 是 worker 数。如果希望 B_{global} 不变，worker 数变化后通常要调整 a_{grad} ：

$$a_{\text{grad}} = \frac{B_{\text{target}}}{b_{\text{micro}} w}$$

49.13 弹性训练的难点

难点包括：

1. world size 变化。
2. global batch size 变化。
3. learning rate 调整。
4. optimizer state 重分片。
5. dataloader 分片变化。
6. checkpoint 格式兼容。
7. 通信拓扑变化。
8. 收敛稳定性影响。

弹性不是简单增加或减少 Pod。

如果 global batch 变化没有处理好，训练动态会改变。

弹性事件的风险可以用批量漂移表示：

$$\Delta B = \left| \frac{B_{\text{global,new}} - B_{\text{global,old}}}{B_{\text{global,old}}} \right|$$

如果 Delta B 超过阈值，就需要重新检查学习率、warmup、loss scale、数据分片和评估可比性。

49.14 弹性训练策略

常见策略：

1. 固定 global batch，改变 gradient accumulation。
2. 固定 learning rate schedule。
3. 支持 elastic dataloader。
4. checkpoint 支持不同 world size restore。
5. 只在安全边界内扩缩容。
6. 对弹性事件记录 trace。

弹性训练适合成熟平台，不适合一开始就强行支持所有任务。

弹性训练门禁可以写成：

$$G_{\text{elastic}} = 1 \left[w_{\min} \leq w \leq w_{\max} \wedge \Delta B \leq \epsilon_B \wedge C_{\text{ckpt}} = 1 \wedge C_{\text{data}} = 1 \right]$$

其中 C_{ckpt} 表示 checkpoint 能跨 world size 恢复， C_{data} 表示 dataloader 分片、shuffle 和去重不会因为 worker 数变化而破坏训练语义。

49.15 推理资源利用率优化

推理资源优化方向：

1. continuous batching。
2. KV cache 管理。
3. 模型量化。
4. 多模型路由。
5. 小模型优先。
6. 自动扩缩容。
7. warm pool 优化。
8. cache 提升命中。
9. 低峰缩容。
10. 分离 prefill/decode。

推理资源利用率不能只追求 GPU 打满。

在线服务还要满足 TTFT、TPOT 和 p99。

推理有效吞吐应扣掉失败请求和 SLO 失败请求：

$$X_{\text{good}} = \frac{N_{\text{out,ok}}}{T} C_{\text{slo}}$$

其中 $N_{\text{out,ok}}$ 是成功输出 token 数， C_{slo} 是请求级 SLO 通过率。对于在线推理， X_{good} 比单纯 tokens/s 更适合和资源利用率一起看。

49.16 Warm Pool 优化

Warm pool 可以降低冷启动，但会增加空闲成本。

优化方式：

1. 按模型热度设置最小副本。
2. 按时间预测扩容。
3. 热模型常驻。
4. 冷模型按需加载。
5. 权重本地缓存。
6. 多模型共享基础镜像。
7. 对低优先级模型允许冷启动。

不同业务需要不同 warm pool 策略。

高价值在线业务需要更高可用性，低频内部模型可以接受冷启动。

Warm pool 的取舍可以写成两类成本比较：

$$K_{\text{warm}} = r_{\text{min}} c_{\text{gpu}} T$$

$$K_{\text{cold}} = \lambda_{\text{cold}} p_{\text{miss}} V_{\text{lat}}$$

其中 r_{min} 是常驻副本数， c_{gpu} 是单位时间成本， λ_{cold} 是冷启动请求量， p_{miss} 是冷启动概率， V_{lat} 是一次冷启动延迟对业务或 SLO 的损失估计。当 K_{cold} 大于 K_{warm} 时，warm pool 更合理。

49.17 资源预留

资源预留用于保障关键任务。

常见预留：

1. 线上推理保留 GPU。
2. 关键训练项目保留配额。
3. 高优先级队列保留容量。
4. 故障恢复保留冗余。
5. 峰值流量前预留资源。

预留会降低表面利用率，但提高稳定性。

因此要平衡：预留太少影响 SLO，预留太多造成浪费。

资源预留可以按多个约束取最大值：

$$C_{\text{reserve}} = \max(C_{\text{slo}}, C_{\text{recovery}}, C_{\text{priority}}, C_{\text{burst}})$$

其中 C_{slo} 是在线 SLO 所需容量， C_{recovery} 是故障恢复容量， C_{priority} 是关键训练或客户预留， C_{burst} 是短期峰值容量。Headroom 可以写成：

$$H_{\text{headroom}} = 1 - \frac{C_{\text{used}}}{C_{\text{total}}}$$

49.18 配额和公平性

资源利用率优化不能牺牲公平性。

配额维度包括：

1. 团队。
2. 租户。
3. 项目。
4. 队列。
5. 优先级。
6. GPU 类型。

公平性策略：

1. dominant resource fairness。
2. weighted fair sharing。
3. quota borrowing。
4. priority-based scheduling。
5. preemption compensation。

允许借用空闲配额，可以提高利用率。

但借用资源要能在原 owner 需要时归还。

Dominant resource share 可用于多资源公平性：

$$D_u = \max_r \frac{A_{u,r}}{C_r}$$

其中 $A_{u,r}$ 是用户或租户 u 占用的资源 r ， C_r 是集群总资源。借用量可以写成：

$$B_u = \max(0, A_u - Q_u)$$

其中 Q_u 是租户配额。允许 $B_u > 0$ 可以提高利用率，但必须配合可回收、可抢占和补偿策略。

49.19 利用率优化和可靠性

利用率越高，系统余量越少。

风险包括：

1. 故障恢复空间不足。
2. 突发流量承接困难。
3. 训练任务排队增加。
4. 推理 p99 抖动。
5. 抢占频繁。

所以不能追求 100% 利用率。

平台要根据 SLO 保留合理 headroom。

Headroom 是为了应对波动和故障的安全余量。

可靠性风险可以用尾延迟放大和抢占放大一起观察：

$$A_{\text{tail}} = \frac{L_{\text{p99,current}}}{L_{\text{p99,base}}}$$

$$A_{\text{preempt}} = \frac{N_{\text{preempt,current}}}{N_{\text{preempt,base}} + 1}$$

当利用率上升伴随 A_{tail} 和 A_{preempt} 同时上升时，说明系统已经从“填洞”进入“挤压可靠性”的区域。

49.20 调度器能力

高利用率需要强调度器能力：

1. gang scheduling。
2. 拓扑感知。
3. 队列和优先级。
4. 抢占。
5. 配额借用。
6. 资源碎片感知。
7. 多资源调度。
8. 弹性任务支持。
9. 调度模拟。
10. 调度事件可观测。

调度器是 AI Infra 利用率优化的核心组件之一。

调度可执行性可以用覆盖率表达：

$$C_{\text{sched}} = \frac{1}{M} \sum_{j=1}^M \mathbf{1}[G_j = 1]$$

其中 G_j 是 gang、拓扑、抢占、配额、公平性、弹性、可观测性等调度能力门禁。只有 dashboard 指标而没有调度执行能力，利用率优化会停留在报表层。

49.21 利用率 Dashboard

Dashboard 应包括：

1. GPU allocation rate。
2. GPU utilization。
3. idle GPU hours。
4. queue wait time。
5. pending jobs。
6. fragmentation。
7. preemption count。
8. failed job GPU hours。
9. per-team usage。
10. per-priority usage。
11. tokens/GPU hour。

同时要能按 GPU 型号、集群、队列、团队和任务类型切分。

Dashboard 下钻覆盖率：

$$C_{\text{dash}} = \frac{N_{\text{drill,ready}}}{N_{\text{drill,total}}}$$

其中下钻维度至少包括 cluster、GPU type、node、queue、tenant、team、priority、job type、model、endpoint 和 time window。没有下钻能力时，很难判断空闲、碎片、抢占和 SLO 抖动是谁造成的。

49.22 资源利用率治理审计指标和最小 demo

把本章落到平台验收时，可以用 16 个门禁：

1. Resource Metric Contract: GPU allocation、GPU utilization、memory、MFU、tokens、idle、pending、queue wait、preemption、failed GPU-hours 和 fragmentation 是否有统一口径。
2. Effective Utilization Accounting: 是否把失败任务、无效实验、SLO 失败请求和不可发布产物从有效利用率中扣除。
3. Fragmentation Control: 是否能按节点、GPU 型号、拓扑和任务规格计算碎片并执行 defragmentation。
4. Topology Binpacking Readiness: bin packing 是否同时考虑 GPU、NIC、rack、storage、gang 和 score trace。
5. Colocation SLO Guard: 混部是否有 latency、step time、驱逐、低优先级上限和关键任务隔离保护。
6. Preemption Checkpoint Safety: 抢占是否绑定 checkpoint 新鲜度、graceful shutdown、requeue、频率限制和用户通知。
7. Low Priority Reclaim Policy: 低优先级队列是否只使用可回收余量，并能在高优先级需求出现时归还。
8. Elastic Training Safety: 弹性训练是否控制 global batch 漂移、checkpoint world size、dataloader、rendezvous 和事件 trace。
9. Inference Capacity Utilization: 推理利用率是否同时满足 TTFT、TPOT、p99、KV pressure、continuous batching 和 autoscaling。
10. Warm Pool Headroom Policy: warm pool 是否基于模型热度、冷启动损失、成本、headroom 策略和容量上限决策。
11. Quota Fairness Borrowing: 配额借用是否受 dominant share、公平性、可回收和补偿机制约束。
12. Scheduler Observability: 调度器是否暴露 gang、queue、priority、preemption、quota、topology、elastic 和 event trace。
13. Dashboard Drilldown Readiness: dashboard 是否能按 cluster、GPU type、node、queue、tenant、team、priority、job type、model、endpoint 和时间窗口下钻。
14. Reliability Headroom Guard: 高利用率是否仍保留故障恢复、突发流量、高优任务和线上 SLO 余量。
15. Cost SLO Tradeoff Control: 优化建议是否同时检查成本节省、质量、SLO、风险、owner 和回滚条件。
16. Resource Utilization Gate: 最终是否有 owner、资源策略、例外审批、审计证据和 P0 风险阻断。

综合门禁：

$$G_{\text{resource_utilization}} = \prod_{j=1}^{16} G_j$$

下面是一个 0 依赖 demo，用 toy cases 检查资源利用率优化是否只是“把 GPU 打满”，还是能把指标、碎片、混部、抢占、低优先级队列、弹性训练、推理容量、warm pool、配额公平和可靠性余量连成闭环。

```
from copy import deepcopy
```

```
class MiniResourceUtilizationAudit:
    GATES = [
        "resource_metric_contract",
        "effective_utilization_accounting",
        "fragmentation_control",
        "topology_binpacking_readiness",
        "colocation_slo_guard",
        "preemption_checkpoint_safety",
        "low_priority_reclaim_policy",
        "elastic_training_safety",
        "inference_capacity_utilization",
        "warm_pool_headroom_policy",
        "quota_fairness_borrowing",
        "scheduler_observability",
        "dashboard_drilldown_readiness",
```

```

        "reliability_headroom_guard",
        "cost_slo_tradeoff_control",
        "resource_utilization_gate",
    ]

    METRIC_FIELDS = [
        "allocation",
        "utilization",
        "memory",
        "mfu",
        "tokens",
        "idle",
        "pending",
        "queue_wait",
        "preemption",
        "failed_gpu_hours",
        "fragmentation",
    ]

    DASHBOARD_FIELDS = [
        "cluster",
        "gpu_type",
        "node",
        "queue",
        "tenant",
        "team",
        "priority",
        "job_type",
        "model",
        "endpoint",
        "time_window",
    ]

    SCHEDULER_FIELDS = [
        "gang",
        "queue",
        "priority",
        "preemption",
        "quota",
        "topology",
        "elastic",
        "event_trace",
    ]

    @staticmethod
    def present(record, key):
        return key in record and record[key] is not None and record[key] != ""

    def coverage(self, record, fields):
        if not record:
            return 0.0
        return sum(1 for field in fields if self.present(record, field)) / len(fields)

    def allocation_rate(self, case):
        capacity = case["capacity"]
        return capacity["allocated_gpu"] / max(1e-12, capacity["total_gpu"])

```

```

def active_utilization(self, case):
    records = case["utilization_records"]
    used = sum(item["gpu_hours"] * item["gpu_util"] for item in records)
    total = sum(item["gpu_hours"] for item in records)
    return used / max(1e-12, total)

def effective_utilization(self, case):
    records = case["utilization_records"]
    useful = sum(
        item["gpu_hours"] * float(item["value_gate"]) * float(item["slo_gate"])
        for item in records
    )
    total = sum(item["gpu_hours"] for item in records)
    return useful / max(1e-12, total)

def fragmentation_ratio(self, case):
    frag = case["fragmentation"]
    target = frag["target_per_node"]
    free_by_node = frag["free_by_node"]
    total_free = sum(free_by_node)
    packable = sum((free // target) * target for free in free_by_node)
    if total_free == 0:
        return 0.0
    return 1.0 - packable / total_free

def colocation_latency_delta(self, case):
    coloc = case["colocation"]
    return (coloc["p99_ms_mix"] - coloc["p99_ms_base"]) / max(1e-12, coloc["p99_ms_base"])

def colocation_step_delta(self, case):
    coloc = case["colocation"]
    return (coloc["step_ms_mix"] - coloc["step_ms_base"]) / max(1e-12, coloc["step_ms_base"])

def preemption_loss_gpu_min(self, case):
    preempt = case["preemption"]
    return preempt["gpu_count"] * (preempt["now_min"] - preempt["last_checkpoint_min"])

def global_batch(self, elastic):
    return elastic["micro_batch"] * elastic["grad_accum"] * elastic["workers"]

def adjusted_grad_accum(self, case):
    elastic = case["elastic"]
    return elastic["target_global_batch"] / max(1e-12, elastic["micro_batch"] * elastic["new_workers"])

def batch_delta(self, case):
    elastic = case["elastic"]
    old_batch = self.global_batch(elastic)
    new_batch = elastic["micro_batch"] * elastic["new_grad_accum"] * elastic["new_workers"]
    return abs(new_batch - old_batch) / max(1e-12, old_batch)

def good_output_tokens_s(self, case):
    inference = case["inference"]
    return inference["successful_output_tokens"] / max(1e-12, inference["window_seconds"]) * inference["slo_pass_

def warm_pool_cost(self, case):

```

```

warm = case["warm_pool"]
return warm["min_replicas"] * warm["gpu_cost_per_hour"] * warm["hours"]

def cold_start_penalty(self, case):
    warm = case["warm_pool"]
    return warm["cold_requests"] * warm["cold_miss_prob"] * warm["latency_penalty_value"]

def headroom(self, case):
    reliability = case["reliability"]
    return 1.0 - reliability["used_capacity"] / max(1e-12, reliability["total_capacity"])

def dominant_share(self, case):
    fairness = case["fairness"]
    shares = []
    for resource, used in fairness["tenant_usage"].items():
        shares.append(used / max(1e-12, fairness["cluster_capacity"][resource]))
    return max(shares)

def borrowed_gpu(self, case):
    fairness = case["fairness"]
    return max(0, fairness["tenant_usage"]["gpu"] - fairness["tenant_quota"]["gpu"])

def resource_metric_contract(self, case):
    metrics = case.get("metrics", {})
    capacity = case.get("capacity", {})
    return (
        self.coverage(metrics, self.METRIC_FIELDS) == 1.0
        and metrics.get("freshness_minutes", 10**9) <= metrics.get("max_freshness_minutes", 0)
        and capacity.get("total_gpu", 0) > 0
        and 0.0 <= self.allocation_rate(case) <= 1.0
    )

def effective_utilization_accounting(self, case):
    eff = case.get("effective", {})
    return (
        self.effective_utilization(case) >= eff.get("min_effective_utilization", 1.0)
        and eff.get("failed_gpu_hours", 10**9) <= eff.get("max_failed_gpu_hours", 0)
        and eff.get("invalid_experiments", 10**9) <= eff.get("max_invalid_experiments", 0)
        and eff.get("publishable_output_tracked") is True
    )

def fragmentation_control(self, case):
    frag = case.get("fragmentation", {})
    return (
        self.fragmentation_ratio(case) <= frag.get("max_fragmentation_ratio", 0.0)
        and frag.get("defragmentation_plan") is True
        and frag.get("standard_shapes") is True
        and frag.get("gpu_type_aware") is True
    )

def topology_binpacking_readiness(self, case):
    topo = case.get("topology", {})
    return (
        topo.get("binpacking_enabled") is True
        and topo.get("gpu_type") is True
    )

```

```

        and topo.get("nic_locality") is True
        and topo.get("rack_locality") is True
        and topo.get("storage_locality") is True
        and topo.get("gang_aware") is True
        and topo.get("score_trace") is True
    )

def colocation_slo_guard(self, case):
    coloc = case.get("colocation", {})
    return (
        self.colocation_latency_delta(case) <= coloc.get("max_latency_delta", 0.0)
        and self.colocation_step_delta(case) <= coloc.get("max_step_delta", 0.0)
        and coloc.get("low_priority_limit") is True
        and coloc.get("auto_evict") is True
        and coloc.get("critical_isolation") is True
    )

def preemption_checkpoint_safety(self, case):
    preempt = case.get("preemption", {})
    return (
        preempt.get("enabled") is True
        and preempt.get("checkpoint_ready") is True
        and preempt.get("graceful_shutdown_seconds", 0) >= preempt.get("min_graceful_shutdown_seconds", 10**9)
        and preempt.get("requeue_ready") is True
        and preempt.get("notify_owner") is True
        and preempt.get("max_preemptions_per_day", 10**9) <= preempt.get("limit_preemptions_per_day", 0)
        and self.preemption_loss_gpu_min(case) <= preempt.get("max_loss_gpu_min", 0)
    )

def low_priority_reclaim_policy(self, case):
    low = case.get("low_priority", {})
    capacity = case.get("capacity", {})
    reclaimable = max(0, capacity["total_gpu"] - capacity["reserved_gpu"] - capacity["high_pending_gpu"])
    return (
        low.get("enabled") is True
        and low.get("preemptible") is True
        and low.get("uses_reclaimable_only") is True
        and low.get("owner_notice") is True
        and low.get("compensation_policy") is True
        and low.get("current_gpu") <= reclaimable
        and low.get("current_gpu") <= low.get("max_gpu_share") * capacity["total_gpu"]
    )

def elastic_training_safety(self, case):
    elastic = case.get("elastic", {})
    adjusted = self.adjusted_grad_accum(case)
    return (
        elastic.get("min_workers", 0) <= elastic.get("new_workers", -1) <= elastic.get("max_workers", 0)
        and self.batch_delta(case) <= elastic.get("max_batch_delta", 0.0)
        and abs(round(adjusted) - adjusted) <= 1e-9
        and elastic.get("checkpoint_world_size_safe") is True
        and elastic.get("elastic_data_loader") is True
        and elastic.get("rendezvous_ready") is True
        and elastic.get("event_trace") is True
    )

```

```

def inference_capacity_utilization(self, case):
    inference = case.get("inference", {})
    return (
        self.good_output_tokens_s(case) >= inference.get("min_good_output_tokens_s", 10**9)
        and inference.get("ttft_p95_ms", 10**9) <= inference.get("ttft_slo_ms", 0)
        and inference.get("tpot_p95_ms", 10**9) <= inference.get("tpot_slo_ms", 0)
        and inference.get("p99_ms", 10**9) <= inference.get("p99_slo_ms", 0)
        and inference.get("kv_pressure", 1.0) <= inference.get("max_kv_pressure", 0.0)
        and inference.get("continuous_batching") is True
        and inference.get("autoscaling_ready") is True
    )

def warm_pool_headroom_policy(self, case):
    warm = case.get("warm_pool", {})
    return (
        warm.get("policy_ready") is True
        and warm.get("heat_based") is True
        and warm.get("weight_cache_ready") is True
        and warm.get("cold_start_allowed_for_low_priority") is True
        and warm.get("max_replicas") >= warm.get("min_replicas")
        and self.warm_pool_cost(case) <= warm.get("max_warm_pool_cost", 0.0)
        and self.cold_start_penalty(case) >= self.warm_pool_cost(case)
    )

def quota_fairness_borrowing(self, case):
    fairness = case.get("fairness", {})
    return (
        fairness.get("drf_ready") is True
        and fairness.get("weighted_share_ready") is True
        and fairness.get("borrow_returnable") is True
        and fairness.get("preemption_compensation") is True
        and self.dominant_share(case) <= fairness.get("max_dominant_share", 0.0)
        and self.borrowed_gpu(case) <= fairness.get("max_borrowed_gpu", 0)
    )

def scheduler_observability(self, case):
    scheduler = case.get("scheduler", {})
    return self.coverage(scheduler, self.SCHEDULER_FIELDS) == 1.0 and scheduler.get("fresh") is True

def dashboard_drilldown_readiness(self, case):
    dashboard = case.get("dashboard", {})
    return self.coverage(dashboard.get("drilldown", {}), self.DASHBOARD_FIELDS) == 1.0 and dashboard.get("fresh")

def reliability_headroom_guard(self, case):
    reliability = case.get("reliability", {})
    return (
        self.headroom(case) >= reliability.get("min_headroom", 1.0)
        and reliability.get("tail_amplification", 10**9) <= reliability.get("max_tail_amplification", 0.0)
        and reliability.get("preemption_amplification", 10**9) <= reliability.get("max_preemption_amplification", 0.0)
        and reliability.get("recovery_capacity") is True
        and reliability.get("p0_open") is False
    )

def cost_slo_tradeoff_control(self, case):

```



```

tradeoff = case.get("tradeoff", {})
return (
    tradeoff.get("estimated_savings", 0.0) >= tradeoff.get("min_estimated_savings", 10**9)
    and tradeoff.get("quality_delta", -1.0) >= -tradeoff.get("max_quality_drop", 0.0)
    and tradeoff.get("slo_delta", -1.0) >= -tradeoff.get("max_slo_drop", 0.0)
    and tradeoff.get("risk_score", 1.0) <= tradeoff.get("max_risk_score", 0.0)
    and tradeoff.get("owner") is True
    and tradeoff.get("rollback_condition") is True
)

def resource_utilization_gate(self, case):
    gate = case.get("platform_gate", {})
    return (
        gate.get("enabled") is True
        and bool(gate.get("owner"))
        and gate.get("resource_policy") is True
        and gate.get("exception_approval") is True
        and gate.get("audit_ready") is True
        and gate.get("p0_open") is False
    )

def audit_case(self, case):
    return {gate: getattr(self, gate)(case) for gate in self.GATES}

def run_all(self, cases):
    results = {case["case_id"]: self.audit_case(case) for case in cases}
    metrics = {}
    for gate in self.GATES:
        passed = sum(1 for result in results.values() if result[gate])
        metrics[gate] = round(passed / len(cases), 3)
    failed_cases = [
        case_id
        for case_id, result in results.items()
        if not all(result.values())
    ]
    failed_gates = [
        gate
        for gate in self.GATES
        if any(not result[gate] for result in results.values())
    ]
    return {
        "metrics": metrics,
        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "failed_gates": failed_gates,
        "resource_utilization_gate_pass": metrics["resource_utilization_gate"] == 1.0,
    }

def example_outputs(self, case):
    return {
        "allocation_rate": round(self.allocation_rate(case), 3),
        "active_utilization": round(self.active_utilization(case), 3),
        "effective_utilization": round(self.effective_utilization(case), 3),
        "fragmentation_ratio": round(self.fragmentation_ratio(case), 3),
        "colocation_latency_delta": round(self.colocation_latency_delta(case), 3),
    }

```

```

        "colocation_step_delta": round(self.colocation_step_delta(case), 3),
        "preemption_loss_gpu_min": round(self.preemption_loss_gpu_min(case), 3),
        "global_batch": self.global_batch(case["elastic"]),
        "adjusted_grad_accum": round(self.adjusted_grad_accum(case), 3),
        "good_output_tokens_s": round(self.good_output_tokens_s(case), 3),
        "warm_pool_cost": round(self.warm_pool_cost(case), 3),
        "cold_start_penalty": round(self.cold_start_penalty(case), 3),
        "headroom": round(self.headroom(case), 3),
        "dominant_share": round(self.dominant_share(case), 3),
        "borrowed_gpu": self.borrowed_gpu(case),
    }

```

```

def build_good_case():
    return {
        "case_id": "full_resource_utilization",
        "metrics": {
            "allocation": True,
            "utilization": True,
            "memory": True,
            "mfu": True,
            "tokens": True,
            "idle": True,
            "pending": True,
            "queue_wait": True,
            "preemption": True,
            "failed_gpu_hours": True,
            "fragmentation": True,
            "freshness_minutes": 5,
            "max_freshness_minutes": 15,
        },
        "capacity": {
            "total_gpu": 100,
            "allocated_gpu": 75,
            "reserved_gpu": 20,
            "high_pending_gpu": 15,
        },
        "utilization_records": [
            {"gpu_hours": 80, "gpu_util": 0.72, "value_gate": True, "slo_gate": True},
            {"gpu_hours": 40, "gpu_util": 0.55, "value_gate": True, "slo_gate": True},
            {"gpu_hours": 30, "gpu_util": 0.85, "value_gate": True, "slo_gate": 0.95},
        ],
        "effective": {
            "min_effective_utilization": 0.85,
            "failed_gpu_hours": 4,
            "max_failed_gpu_hours": 8,
            "invalid_experiments": 1,
            "max_invalid_experiments": 1,
            "publishable_output_tracked": True,
        },
        "fragmentation": {
            "free_by_node": [8, 4, 4, 2],
            "target_per_node": 4,
            "max_fragmentation_ratio": 0.2,
            "defragmentation_plan": True,
        },
    }

```

```

    "standard_shapes": True,
    "gpu_type_aware": True,
},
"topology": {
    "binpacking_enabled": True,
    "gpu_type": True,
    "nic_locality": True,
    "rack_locality": True,
    "storage_locality": True,
    "gang_aware": True,
    "score_trace": True,
},
"colocation": {
    "p99_ms_base": 100,
    "p99_ms_mix": 110,
    "max_latency_delta": 0.15,
    "step_ms_base": 600,
    "step_ms_mix": 610,
    "max_step_delta": 0.05,
    "low_priority_limit": True,
    "auto_evict": True,
    "critical_isolation": True,
},
"preemption": {
    "enabled": True,
    "gpu_count": 8,
    "now_min": 120,
    "last_checkpoint_min": 115,
    "max_loss_gpu_min": 60,
    "checkpoint_ready": True,
    "graceful_shutdown_seconds": 90,
    "min_graceful_shutdown_seconds": 60,
    "requeue_ready": True,
    "notify_owner": True,
    "max_preemptions_per_day": 2,
    "limit_preemptions_per_day": 4,
},
"low_priority": {
    "enabled": True,
    "preemptible": True,
    "uses_reclaimable_only": True,
    "owner_notice": True,
    "compensation_policy": True,
    "current_gpu": 30,
    "max_gpu_share": 0.4,
},
"elastic": {
    "micro_batch": 4,
    "grad_accum": 8,
    "workers": 16,
    "target_global_batch": 512,
    "new_workers": 8,
    "new_grad_accum": 16,
    "min_workers": 4,
    "max_workers": 32,

```

```

    "max_batch_delta": 0.01,
    "checkpoint_world_size_safe": True,
    "elastic_dataloader": True,
    "rendezvous_ready": True,
    "event_trace": True,
},
"inference": {
    "successful_output_tokens": 540000,
    "window_seconds": 100,
    "slo_pass_rate": 0.9,
    "min_good_output_tokens_s": 4500,
    "ttft_p95_ms": 650,
    "ttft_slo_ms": 800,
    "tpot_p95_ms": 45,
    "tpot_slo_ms": 60,
    "p99_ms": 1600,
    "p99_slo_ms": 2000,
    "kv_pressure": 0.78,
    "max_kv_pressure": 0.85,
    "continuous_batching": True,
    "autoscaling_ready": True,
},
"warm_pool": {
    "min_replicas": 2,
    "max_replicas": 8,
    "gpu_cost_per_hour": 32.0,
    "hours": 24,
    "max_warm_pool_cost": 1800.0,
    "cold_requests": 100,
    "cold_miss_prob": 0.3,
    "latency_penalty_value": 80.0,
    "policy_ready": True,
    "heat_based": True,
    "weight_cache_ready": True,
    "cold_start_allowed_for_low_priority": True,
},
"fairness": {
    "tenant_usage": {"gpu": 70, "cpu": 500, "memory": 700},
    "tenant_quota": {"gpu": 60, "cpu": 600, "memory": 1000},
    "cluster_capacity": {"gpu": 100, "cpu": 1000, "memory": 2000},
    "max_dominant_share": 0.8,
    "max_borrowed_gpu": 16,
    "drf_ready": True,
    "weighted_share_ready": True,
    "borrow_returnable": True,
    "preemption_compensation": True,
},
"scheduler": {
    "gang": True,
    "queue": True,
    "priority": True,
    "preemption": True,
    "quota": True,
    "topology": True,
    "elastic": True,

```

```

        "event_trace": True,
        "fresh": True,
    },
    "dashboard": {
        "drilldown": {
            "cluster": True,
            "gpu_type": True,
            "node": True,
            "queue": True,
            "tenant": True,
            "team": True,
            "priority": True,
            "job_type": True,
            "model": True,
            "endpoint": True,
            "time_window": True,
        },
        "fresh": True,
    },
    "reliability": {
        "used_capacity": 720,
        "total_capacity": 1000,
        "min_headroom": 0.2,
        "tail_amplification": 1.08,
        "max_tail_amplification": 1.2,
        "preemption_amplification": 1.1,
        "max_preemption_amplification": 1.5,
        "recovery_capacity": True,
        "p0_open": False,
    },
    "tradeoff": {
        "estimated_savings": 900.0,
        "min_estimated_savings": 500.0,
        "quality_delta": -0.003,
        "max_quality_drop": 0.01,
        "slo_delta": -0.002,
        "max_slo_drop": 0.005,
        "risk_score": 0.12,
        "max_risk_score": 0.2,
        "owner": True,
        "rollback_condition": True,
    },
    "platform_gate": {
        "enabled": True,
        "owner": "resource-governance",
        "resource_policy": True,
        "exception_approval": True,
        "audit_ready": True,
        "p0_open": False,
    },
}

```

```

def build_bad_cases(good_case):
    cases = []

```

```

case = deepcopy(good_case)
case["case_id"] = "metric_contract_missing_bad"
case["metrics"]["fragmentation"] = None
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "effective_utilization_bad"
case["utilization_records"][2]["value_gate"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "fragmentation_uncontrolled_bad"
case["fragmentation"]["free_by_node"] = [2, 1, 3, 2]
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "topology_binpacking_bad"
case["topology"]["score_trace"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "colocation_slo_bad"
case["colocation"]["p99_ms_mix"] = 135
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "preemption_checkpoint_bad"
case["preemption"]["checkpoint_ready"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "low_priority_reclaim_bad"
case["low_priority"]["uses_reclaimable_only"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "elastic_training_bad"
case["elastic"]["new_grad_accum"] = 8
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "inference_capacity_bad"
case["inference"]["p99_ms"] = 2600
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "warm_pool_policy_bad"
case["warm_pool"]["policy_ready"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "quota_fairness_bad"
case["fairness"]["borrow_returnable"] = False
cases.append(case)

```

```

case = deepcopy(good_case)
case["case_id"] = "scheduler_observability_bad"
case["scheduler"]["event_trace"] = None
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "dashboard_drilldown_bad"
case["dashboard"]["drilldown"].pop("endpoint")
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "reliability_headroom_bad"
case["reliability"]["used_capacity"] = 880
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "cost_slo_tradeoff_bad"
case["tradeoff"]["slo_delta"] = -0.03
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "resource_utilization_gate_missing_bad"
case["platform_gate"]["enabled"] = False
cases.append(case)

return cases

```

```

audit = MiniResourceUtilizationAudit()
good = build_good_case()
cases = [good] + build_bad_cases(good)
summary = audit.run_all(cases)

print("resource_utilization_examples=" + repr(audit.example_outputs(good)))
print("metrics=" + repr(summary["metrics"]))
print("hard_blocker_count=" + repr(summary["hard_blocker_count"]))
print("failed_cases=" + repr(summary["failed_cases"]))
print("failed_gates=" + repr(summary["failed_gates"]))
print("resource_utilization_gate_pass=" + repr(summary["resource_utilization_gate_pass"]))

```

参考输出应类似：

```

resource_utilization_examples={'allocation_rate': 0.75, 'active_utilization': 0.701, 'effective_utilization': 0.99, 'fr
metrics={'resource_metric_contract': 0.941, 'effective_utilization_accounting': 0.941, 'fragmentation_control': 0.941,
hard_blocker_count=16
failed_cases=['metric_contract_missing_bad', 'effective_utilization_bad', 'fragmentation_uncontrolled_bad', 'topology
failed_gates=['resource_metric_contract', 'effective_utilization_accounting', 'fragmentation_control', 'topology_binpa
resource_utilization_gate_pass=False

```

这个 demo 的面试价值是把资源利用率优化从 “GPU 忙不忙” 升级为工程控制闭环：先证明指标口径一致，再扣除无效产出，然后治理碎片、混部、抢占、低优先级队列、弹性训练、推理容量、warm pool、配额公平和可靠性 headroom，最后用成本 / SLO / 风险门禁决定优化动作能不能落地。

49.23 常见误区

误区一：GPU 利用率越高越好。

过高可能导致没有故障余量、p99 抖动和排队增加。

误区二：混部一定省钱。

如果混部影响在线 SLO 或导致训练抖动，收益可能小于风险。

误区三：抢占就是 kill 低优先级任务。

生产抢占需要 checkpoint、graceful shutdown、重新排队和事件通知。

误区四：弹性训练就是改副本数。

弹性训练涉及 world size、batch、学习率、optimizer state、dataloader 和 checkpoint。

误区五：资源预留是浪费。

合理预留是保证 SLO 和故障恢复能力的必要成本。

49.24 面试常见追问

问题一：如何提高 GPU 集群利用率？

可以回答：从减少碎片、拓扑感知调度、低优先级队列、抢占、配额借用、弹性训练、数据供给优化、checkpoint 策略、推理 batching 和自动扩缩容等方面综合优化。

问题二：训练和推理能混部吗？

可以回答：可以但要谨慎。要考虑 GPU、CPU、内存、网络和 I/O 干扰，在线推理要有 SLO guardrail，低优先级离线任务应可抢占，必要时物理或逻辑隔离。

问题三：抢占机制怎么设计？

可以回答：定义优先级和抢占关系，抢占前给任务保存 checkpoint 的时间，被抢占任务重新入队，限制抢占频率，记录事件并通知用户。

问题四：为什么不能追求 100% GPU 利用率？

可以回答：需要保留 headroom 应对突发流量、故障恢复和高优先级任务。过高利用率会增加排队、抢占和线上延迟风险。

49.25 小练习

1. GPU allocation rate 和 GPU utilization 有什么区别？
2. 什么是资源碎片？为什么多机多卡任务更容易受影响？
3. 混部有哪些收益和风险？
4. 抢占低优先级任务前应该做什么？
5. 弹性训练为什么会影响 global batch 和学习率？
6. Warm pool 如何影响成本和延迟？
7. Headroom 为什么不是浪费？
8. 如何设计一个 GPU 利用率 dashboard？

49.26 本章小结

本章讲了资源利用率优化。

你需要记住：

1. GPU 利用率优化不是简单打满集群，而是提高有效产出。
2. 资源碎片、拓扑、gang scheduling 和多机多卡需求是 AI 集群调度的核心难点。
3. 混部、抢占、低优先级队列和配额借用可以提高利用率，但必须有隔离和 SLO guardrail。
4. 弹性训练能提升资源灵活性，但涉及 world size、batch、optimizer、dataloader 和 checkpoint 的复杂变化。
5. 利用率优化必须保留 headroom，不能牺牲可靠性和线上 SLO。

下一章我们会讲安全治理：密钥、权限、数据合规、供应链和模型访问控制。

第 50 章 安全治理：密钥、权限、数据合规、供应链和模型访问控制

上一章讲了资源利用率优化。本章讲 AI Infra 安全治理。

AI Infra 的安全边界非常长：数据进入平台、训练任务读取数据、模型产物保存、推理服务暴露 API、RAG 检索企业知识、Agent 调用工具、日志和 trace 保存用户输入，每一处都可能出问题。

先记住一句话：

AI Infra 安全治理不是给服务加一个鉴权中间件，而是覆盖身份、权限、密钥、数据、模型、供应链、运行时、审计和合规的系统工程。

50.0 本讲资料边界与第二轮精修口径

本章按 WRITING_PLAN.md 做第二轮精修时，主要参考 Kubernetes RBAC、Secret、Pod Security Standards、NetworkPolicy 和审计日志的官方口径，参考 NIST AI RMF / Generative AI Profile 对 AI 风险治理、数据治理、测量和管理的通用框架，参考 SLSA 对软件供应链 provenance、build integrity 和 artifact 可追溯的定义，也参考 OWASP LLM Top 10 对 prompt injection、sensitive information disclosure、supply chain、excessive agency 和 overreliance 等应用风险的分类。

需要强调三点：

1. 本章只讨论防御性安全治理、审计和平台门禁，不提供漏洞利用、越权访问、绕过审计或规避安全控制的方法。
2. 安全治理不能只落在推理 API，也必须进入训练任务、数据平台、模型仓库、RAG / Agent、运行时、供应链、日志和应急响应。
3. 文中 demo 是 0 依赖教学审计脚本，用于解释安全门禁如何串联，不代表生产级安全扫描器、合规判断或法律建议。

50.1 为什么 AI Infra 安全更复杂

AI Infra 安全复杂，原因包括：

1. 数据类型敏感。
2. 模型可能记忆训练数据。
3. prompt 和 trace 可能包含隐私。
4. 模型权重是高价值资产。
5. Agent 工具调用可能产生副作用。
6. RAG 可能越权检索知识。
7. 训练任务运行任意代码。
8. 镜像和依赖供应链复杂。
9. 多租户共享 GPU 集群。

传统 Web 安全仍然需要，但不够。

AI Infra 还要面对模型、数据和工具调用带来的新风险。

50.2 安全治理范围

AI Infra 安全治理至少包括：

1. 身份认证。
2. 权限控制。
3. 密钥管理。
4. 数据合规。
5. 多租户隔离。

6. 模型访问控制。
7. 供应链安全。
8. 运行时安全。
9. 日志和 trace 安全。
10. 审计和响应。

安全治理要贯穿训练、评估、推理、RAG、Agent 和平台管理。

50.3 身份认证

身份认证回答：你是谁。

常见身份包括：

1. 人类用户。
2. 服务账号。
3. 训练任务。
4. 推理服务。
5. Agent。
6. CI/CD 流水线。
7. 外部应用。

AI Infra 中不能只认证人，还要认证任务和服务。

例如训练任务读取数据时，应该使用任务绑定的 service account，而不是平台管理员的长期凭据。

可以把一次安全审计样本抽象为：

$$S_i = (a_i, w_i, r_i, d_i, m_i, t_i, k_i, p_i, l_i, o_i, z_i)$$

其中 a_i 是 actor, w_i 是 workload, r_i 是 resource, d_i 是 data classification, m_i 是 model / artifact, t_i 是 tool, k_i 是 secret, p_i 是 policy decision, l_i 是 log / trace, o_i 是 owner, z_i 是审计证据。

身份绑定覆盖率：

$$C_{id} = \frac{N_{bound}}{N_{workload}}$$

其中 N_{bound} 是绑定了人类用户、service account、任务身份和租户的 workload 数, $N_{workload}$ 是总 workload 数。

50.4 权限控制

权限控制回答：你能做什么。

权限对象包括：

1. Dataset。
2. Model。
3. ModelVersion。
4. Checkpoint。
5. Eval report。
6. Endpoint。
7. Tool。
8. Knowledge Base。
9. Trace。
10. Secret。

权限动作包括：

1. read。
2. write。
3. execute。
4. deploy。
5. delete。
6. approve。
7. export。

权限要细到资源和动作，不能只有“管理员”和“普通用户”。

访问决策可以写成门禁：

$$G_{\text{access}} = \mathbf{1} [I(a) = 1 \wedge P(a, r, e) = \text{allow} \wedge L(r) \leq L(a) \wedge U(r) \subseteq U(a)]$$

其中 $I(a)$ 表示身份认证通过， $P(a, r, e)$ 是在环境 e 下 actor a 对资源 r 的策略决策， $L(r)$ 是资源敏感等级， $L(a)$ 是 actor 可访问等级， $U(r)$ 是资源允许用途集合， $U(a)$ 是 actor 本次任务声明用途集合。

50.5 RBAC 和 ABAC

RBAC 是 role-based access control，基于角色的权限控制。

例如：

1. data scientist。
2. platform admin。
3. model reviewer。
4. security auditor。

ABAC 是 attribute-based access control，基于属性的权限控制。

例如根据：

1. 租户。
2. 数据等级。
3. 地域。
4. 项目。
5. 模型安全等级。
6. 用户部门。

做动态判断。

AI Infra 通常需要 RBAC + ABAC。

例如某用户是研究员，但只能访问自己项目下的低敏数据。

过度授权率：

$$R_{\text{over}} = \frac{N_{\text{overprivileged}}}{N_{\text{binding}}}$$

其中 $N_{\text{overprivileged}}$ 是拥有超出当前任务需要权限的绑定数， N_{binding} 是权限绑定总数。安全治理要让 R_{over} 趋近 0，而不是让“方便”覆盖最小权限原则。

50.6 密钥管理

密钥包括：

1. API key。
2. 数据库密码。
3. 对象存储凭据。

4. 外部工具 token。
5. SSH key。
6. 模型服务访问 token。
7. KMS key。

密钥管理原则：

1. 不写入代码。
2. 不写入镜像。
3. 不写入日志。
4. 使用 secret manager。
5. 最小权限。
6. 定期轮换。
7. 支持吊销。
8. 访问审计。

Agent 工具调用尤其要注意，不要把高权限密钥暴露给模型或 prompt。

密钥暴露率：

$$R_{\text{secret}} = \frac{N_{\text{secret,exposed}}}{N_{\text{secret,total}}}$$

密钥轮换新鲜度可以写成：

$$C_{\text{rotate}} = \frac{N_{\text{rotated_within_ttl}}}{N_{\text{secret,total}}}$$

其中 $N_{\text{secret,exposed}}$ 是出现在代码、镜像、日志、trace、prompt 或无权限环境中的密钥数， $N_{\text{rotated_within_ttl}}$ 是在 TTL 内轮换的密钥数。

50.7 训练任务中的密钥

训练任务可能需要访问数据和 artifact。

错误做法：

1. 在启动命令中传密钥。
2. 把密钥写进环境变量后打印日志。
3. 把密钥打进镜像。
4. 多租户共享同一个高权限 token。

更好的方式：

1. 使用短期凭证。
2. 使用 service account。
3. 按任务注入最小权限 secret。
4. 任务结束后自动失效。
5. 日志脱敏。

训练任务是运行任意代码的环境，密钥暴露风险很高。

训练任务密钥门禁：

$$G_{\text{job_secret}} = 1 [C_{\text{short}} \geq \tau \wedge R_{\text{secret}} = 0 \wedge A_{\text{secret}} = 1 \wedge E_{\text{log}} = 1]$$

其中 C_{short} 是短期凭证覆盖率， A_{secret} 是 secret 访问审计是否开启， E_{log} 是日志脱敏是否开启。

50.8 数据合规

数据合规关注数据是否允许被使用。

问题包括：

1. 数据是否包含个人信息。
2. 数据是否有使用授权。
3. 数据是否允许训练模型。
4. 数据是否允许跨地域处理。
5. 数据是否允许长期保存。
6. 数据是否允许用于评估或日志。
7. 数据删除请求如何处理。

数据平台要记录数据来源、授权、敏感等级、地域和使用限制。

不能把合规信息放在文档里靠人工记忆。

数据合规覆盖率：

$$C_{\text{compliance}} = \frac{N_{\text{classified_authorized}}}{N_{\text{dataset}}}$$

其中 $N_{\text{classified_authorized}}$ 是同时有来源、授权、敏感等级、地域、用途和保留策略记录的数据集数。没有这些字段的数据，不应该进入训练、评估或日志闭环。

50.9 数据分级

数据可以分级：

1. Public。
2. Internal。
3. Confidential。
4. Restricted。

不同等级对应不同控制：

1. 谁能访问。
2. 是否能训练。
3. 是否能导出。
4. 是否能进入日志。
5. 是否能跨地域。
6. 是否需要脱敏。
7. 是否需要审批。

训练平台提交任务时，应检查 dataset 的数据等级和任务权限。

数据分级传播门禁：

$$G_{\text{class}} = \mathbf{1} \left[L_{\text{artifact}} \geq \max_{d \in D_{\text{input}}} L_d \right]$$

其中 L_{artifact} 是训练产物、模型、checkpoint 或 eval report 的等级， D_{input} 是输入数据集合。产物等级不能低于输入数据最高等级。

50.10 PII 和敏感信息

PII 和敏感信息要在多个阶段处理：

1. 数据接入。

2. 数据清洗。
3. 训练数据构建。
4. 推理请求。
5. 日志和 trace。
6. 评估样本。
7. Agent 工具结果。

处理方式包括：

1. 检测。
2. 脱敏。
3. 删除。
4. 权限隔离。
5. 短期留存。
6. 审计。

敏感数据进入模型训练后，风险很难彻底消除，所以源头治理很重要。

PII 脱敏覆盖率：

$$C_{\text{pii}} = \frac{N_{\text{pii,redacted}}}{N_{\text{pii,detected}}}$$

敏感数据阻断率：

$$B_{\text{sensitive}} = \frac{N_{\text{sensitive,blocked}}}{N_{\text{sensitive,total}}}$$

其中 PII 和敏感信息要在数据、请求、日志、trace、评估样本和工具结果中统一处理。

50.11 多租户隔离

多租户隔离包括：

1. 身份隔离。
2. 数据隔离。
3. 模型隔离。
4. 资源隔离。
5. 网络隔离。
6. 日志隔离。
7. 缓存隔离。
8. trace 隔离。
9. 成本隔离。

不要只靠 Kubernetes namespace。

例如 RAG 缓存跨租户命中，或者向量索引没有权限过滤，都会造成严重数据泄露。

多租户越权命中率：

$$R_{\text{tenant_leak}} = \frac{N_{\text{cross_tenant_hit}}}{N_{\text{access_check}}}$$

多租户隔离目标不是“命中后不展示”，而是让检索、缓存、日志、trace、工具和模型 artifact 在命中前就被权限和租户边界过滤。

50.12 模型访问控制

模型本身也是受保护资产。

需要控制：

1. 谁能调用模型。
2. 谁能下载权重。
3. 谁能部署模型。
4. 谁能查看 eval report。
5. 谁能访问敏感模型输出。
6. 哪些模型可以跨租户使用。
7. 哪些模型禁止导出。

企业自训模型可能包含内部知识、业务策略或专有知识，不能当普通文件公开。

模型访问门禁：

$$G_{\text{model_access}} = 1 \left[P(a, m, \text{action}) = \text{allow} \wedge L(m) \leq L(a) \wedge E_{\text{download}} = 1 \wedge E_{\text{deploy}} = 1 \right]$$

其中 m 是模型或模型版本， action 可以是 call、download、deploy、export 或 delete， E_{download} 和 E_{deploy} 是下载和部署审计证据。

50.13 模型输出安全

模型输出也要治理。

风险包括：

1. 泄露敏感信息。
2. 生成不安全内容。
3. 输出越权知识。
4. 错误调用工具。
5. 生成违规代码。
6. 幻觉导致错误决策。

治理手段：

1. 输出安全检查。
2. 高风险场景人工确认。
3. 工具调用权限校验。
4. 安全评估。
5. 线上监控。
6. 违规样本回流。

输出安全通过率：

$$C_{\text{output}} = \frac{N_{\text{safe_output}}}{N_{\text{checked_output}}}$$

高风险输出人工确认覆盖率：

$$C_{\text{review}} = \frac{N_{\text{high_risk_reviewed}}}{N_{\text{high_risk}}}$$

输出安全不是只靠拒答；它还要支持越权知识阻断、工具动作确认、风险分级、监控和回归样本回流。

50.14 RAG 权限安全

RAG 安全的核心是检索阶段权限过滤。

要保证：

1. 文档有权限标签。
2. chunk 继承文档权限。
3. embedding 和索引带权限元数据。
4. 检索时强制过滤。
5. citation 不暴露无权限来源。
6. trace 存储脱敏。

错误做法是把无权限文档交给模型，再要求模型不要说。

权限必须由平台强制执行。

RAG 权限阻断率：

$$B_{\text{rag}} = \frac{N_{\text{unauthorized_chunk_blocked}}}{N_{\text{unauthorized_chunk}}}$$

引用安全覆盖率：

$$C_{\text{citation}} = \frac{N_{\text{citation_authorized}}}{N_{\text{citation}}}$$

其中权限过滤必须在检索和上下文组装前完成，不能把无权限 chunk 交给模型后再要求模型保密。

50.15 Agent 工具安全

Agent 工具调用有副作用风险。

工具可以分级：

1. 只读工具。
2. 低风险写工具。
3. 高风险写工具。
4. 外部支付或删除类工具。

高风险工具需要：

1. 用户确认。
2. 权限校验。
3. 参数校验。
4. 审计日志。
5. 限流。
6. 超时。
7. 回滚或补偿机制。

Agent 不能因为模型“想调用”就直接调用高权限工具。

高风险工具确认覆盖率：

$$C_{\text{tool_confirm}} = \frac{N_{\text{high_risk_confirmed}}}{N_{\text{high_risk_tool_call}}}$$

工具越权率：

$$R_{\text{tool_unauth}} = \frac{N_{\text{unauthorized_tool_call}}}{N_{\text{tool_call}}}$$

Agent 工具安全的目标是让工具调用满足权限、参数、确认、限流、超时、审计和补偿条件。

50.16 供应链安全

AI Infra 供应链包括：

1. 容器镜像。
2. Python 包。
3. CUDA 库。
4. 模型权重。
5. tokenizer 文件。
6. 数据集。
7. 第三方工具。
8. 开源 runtime。

风险包括：

1. 恶意依赖。
2. 漏洞镜像。
3. 被篡改权重。
4. 不安全反序列化。
5. 数据污染。
6. 未知许可证风险。

供应链安全要覆盖从构建到部署的全过程。

供应链完整性覆盖率：

$$C_{\text{supply}} = \frac{N_{\text{artifact_with_provenance}}}{N_{\text{artifact_total}}}$$

高危依赖阻断率：

$$B_{\text{critical}} = \frac{N_{\text{critical_blocked}}}{N_{\text{critical_found}}}$$

其中 provenance、checksum、签名、依赖锁定、漏洞扫描和许可证记录要从构建延续到部署。

50.17 镜像安全

镜像安全要求：

1. 基础镜像受控。
2. 依赖版本锁定。
3. 漏洞扫描。
4. 镜像签名。
5. 禁止 root 运行。
6. 最小化镜像。
7. 不包含密钥。
8. 镜像来源可追溯。

训练任务镜像尤其危险，因为用户可能提交自定义代码和依赖。

平台应对镜像来源和权限做控制。

镜像安全门禁：

$$G_{\text{image}} = 1 [D_{\text{digest}} = 1 \wedge S_{\text{signed}} = 1 \wedge V_{\text{critical}} = 0 \wedge R_{\text{root}} = 0 \wedge K_{\text{embedded}} = 0]$$

其中 D_{digest} 表示 digest 固定， S_{signed} 表示签名存在， V_{critical} 是未处理高危漏洞数， R_{root} 表示 root 运行风险， K_{embedded} 是镜像内嵌密钥数。

50.18 模型 artifact 安全

模型 artifact 安全包括：

1. 权重 checksum。
2. artifact 签名。
3. 安全格式。
4. tokenizer 文件校验。
5. model config 校验。
6. 下载权限。
7. 部署前校验。

不安全的模型文件可能在加载时触发风险。

生产环境应避免加载来源不明的模型 artifact。

模型 artifact 完整性门禁：

$$G_{\text{artifact}} = 1 [C_{\text{checksum}} = 1 \wedge C_{\text{signature}} = 1 \wedge C_{\text{format}} = 1 \wedge C_{\text{tokenizer}} = 1 \wedge C_{\text{permission}} = 1]$$

其中 checksum、signature、安全格式、tokenizer / config 绑定和下载权限必须同时满足。

50.19 运行时安全

运行时安全包括：

1. 容器隔离。
2. 非 root 用户。
3. 只读文件系统。
4. 网络策略。
5. syscall 限制。
6. GPU 设备隔离。
7. secret 最小注入。
8. egress 控制。
9. 资源限制。

训练任务和推理服务都应该有运行时安全边界。

多租户环境中尤其重要。

运行时加固覆盖率：

$$C_{\text{runtime}} = \frac{N_{\text{hardened}}}{N_{\text{workload}}}$$

其中 hardened 至少包括非 root、只读文件系统、最小 secret 注入、egress 控制、网络策略、资源限制和 GPU 设备隔离。

50.20 日志和 Trace 安全

日志和 trace 可能包含：

1. prompt。
2. 用户输入。
3. 文档片段。
4. 工具参数。
5. 工具输出。
6. 模型输出。
7. 错误堆栈中的密钥。

治理要求：

1. 脱敏。
2. 字段级权限。
3. TTL。
4. 加密存储。
5. 访问审计。
6. 采样策略。

排障需要日志，但不能以牺牲隐私为代价。

日志 Trace 脱敏覆盖率：

$$C_{\log_redact} = \frac{N_{\text{redacted_field}}}{N_{\text{sensitive_field}}}$$

留存合规率：

$$C_{\text{retention}} = \frac{N_{\text{within_ttl}}}{N_{\text{log_item}}}$$

排障 trace 要能回答问题，但不能无限期保存原始 prompt、工具参数、文档片段或密钥。

50.21 审计

审计要记录关键动作：

1. 数据访问。
2. 模型下载。
3. 模型部署。
4. endpoint 调用。
5. 权限变更。
6. secret 访问。
7. 工具调用。
8. 高风险 Agent 操作。
9. 发布和回滚。
10. 数据删除和导出。

审计日志要不可随意篡改，并能按用户、租户、模型、数据集、时间查询。

审计覆盖率：

$$C_{\text{audit}} = \frac{N_{\text{audited_event}}}{N_{\text{critical_event}}}$$

不可篡改覆盖率：

$$C_{\text{immut}} = \frac{N_{\text{immutable_event}}}{N_{\text{audited_event}}}$$

审计日志至少要覆盖数据、模型、endpoint、secret、工具、权限、发布、回滚、删除和导出。

50.22 安全事件响应

安全事件包括：

1. 密钥泄露。
2. 越权访问。
3. 数据误用。
4. 模型权重泄露。
5. 供应链漏洞。
6. 高风险工具误调用。
7. 日志泄露敏感信息。

响应流程：

1. 识别事件。
2. 隔离影响。
3. 吊销密钥。
4. 停止相关服务。
5. 审计访问范围。
6. 修复根因。
7. 复盘和补充控制。

安全事件响应需要预案，不能等事故发生再想。

安全响应就绪门禁：

$$G_{\text{response}} = \mathbf{1} [C_{\text{detect}} \geq \tau_d \wedge C_{\text{revoke}} \geq \tau_r \wedge C_{\text{scope}} \geq \tau_s \wedge C_{\text{postmortem}} \geq \tau_p]$$

其中 C_{detect} 是检测覆盖率， C_{revoke} 是吊销和隔离动作覆盖率， C_{scope} 是影响范围审计覆盖率， $C_{\text{postmortem}}$ 是复盘行动项覆盖率。

50.23 安全治理系统架构

一个 AI Infra 安全治理架构：

Identity Provider

- > Policy Engine
- > Secret Manager
- > Data Catalog / Classification
- > Model Registry Access Control
- > Runtime Security
- > Supply Chain Scanner
- > Audit Log
- > Security Monitoring

Policy Engine 负责统一判断权限。

Secret Manager 管密钥。

Data Catalog 管数据等级和合规信息。

Audit Log 支撑追溯。

安全治理总门禁：

$$G_{\text{security}} = \prod_{j=1}^{16} G_j$$

其中 16 个 G_j 分别覆盖身份、权限、密钥、数据合规、PII、多租户、模型访问、输出安全、RAG、Agent、供应链、artifact、运行时、日志 trace、审计响应和最终平台门禁。

50.24 安全治理审计指标和最小 demo

把本章落到平台验收时，可以用 16 个门禁：

1. Identity Workload Binding: 人类用户、service account、训练任务、推理服务、Agent、CI/CD 和外部应用身份是否绑定到 workload。
2. RBAC ABAC Policy Fit: RBAC、ABAC、deny by default、最小权限、用途和数据等级是否进入统一策略引擎。
3. Secret Lifecycle Management: 密钥是否使用 secret manager、短期凭证、最小权限、轮换、吊销、访问审计和日志脱敏。
4. Data Compliance Classification: 数据来源、授权、敏感等级、地域、用途、保留和删除请求是否结构化记录。
5. PII Sensitive Data Control: PII / 敏感信息是否在数据、请求、日志、trace、评估和工具结果中检测、脱敏、阻断和审计。
6. Tenant Isolation Boundary: 租户的身份、数据、模型、资源、网络、缓存、日志、trace 和成本是否隔离。
7. Model Access Control: 模型调用、权重下载、部署、导出、eval report 和敏感输出是否受权限与审计控制。
8. Model Output Safety Gate: 模型输出是否经过安全策略、高风险人工确认、工具动作保护、监控和违规样本回流。
9. RAG Permission Enforcement: RAG 是否在检索和上下文组装前强制权限过滤，并保证 citation 和 trace 不泄露无权限来源。
10. Agent Tool Safety Gate: Agent 工具调用是否有权限、参数校验、高风险确认、限流、超时、审计和补偿机制。
11. Supply Chain Integrity: 镜像、依赖、runtime、数据集、模型权重、tokenizer 和工具是否有 provenance、checksum、签名、扫描和许可证记录。
12. Model Artifact Integrity: 模型权重、tokenizer、config、adapter、量化包和 deployment package 是否有 checksum、签名、安全格式和权限。
13. Runtime Security Boundary: 训练 / 推理 workload 是否非 root、只读文件系统、网络策略、egress 控制、最小 secret、GPU 隔离和资源限制。
14. Log Trace Privacy Governance: 日志和 trace 是否脱敏、字段级限权、TTL、加密、采样、访问审计和删除覆盖。
15. Audit Incident Response Readiness: 审计日志是否覆盖关键事件且不可篡改，安全事件响应是否能检测、隔离、吊销、定界、修复和复盘。
16. Security Governance Gate: 最终是否有 owner、安全策略、例外审批、审计证据、响应预案和 P0 风险阻断。

综合门禁：

$$G_{\text{security}} = \prod_{j=1}^{16} G_j$$

下面是一个 0 依赖 demo，用 toy security cases 检查 AI Infra 安全治理是否只是“API 鉴权”，还是能把身份、权限、密钥、数据、模型、RAG、Agent、供应链、运行时、日志、审计和响应串成闭环。

```
from copy import deepcopy
```

```
class MiniAIInfraSecurityGovernanceAudit:
```

```

GATES = [
    "identity_workload_binding",
    "rbac_abac_policy_fit",
    "secret_lifecycle_management",
    "data_compliance_classification",
    "pii_sensitive_data_control",
    "tenant_isolation_boundary",
    "model_access_control",
    "model_output_safety_gate",
    "rag_permission_enforcement",
    "agent_tool_safety_gate",
    "supply_chain_integrity",
    "model_artifact_integrity",
    "runtime_security_boundary",
    "log_trace_privacy_governance",
    "audit_incident_response_readiness",
    "security_governance_gate",
]

def identity_coverage(self, case):
    identity = case["identity"]
    return identity["bound_workloads"] / max(1e-12, identity["total_workloads"])

def overprivilege_rate(self, case):
    permission = case["permission"]
    return permission["overprivileged_bindings"] / max(1e-12, permission["total_bindings"])

def secret_exposure_rate(self, case):
    secret = case["secret"]
    return secret["exposed"] / max(1e-12, secret["total"])

def secret_rotation_coverage(self, case):
    secret = case["secret"]
    return secret["rotated_within_ttl"] / max(1e-12, secret["total"])

def compliance_coverage(self, case):
    data = case["data"]
    return data["classified_authorized"] / max(1e-12, data["datasets"])

def pii_redaction_coverage(self, case):
    pii = case["pii"]
    return pii["redacted"] / max(1e-12, pii["detected"])

def sensitive_block_rate(self, case):
    pii = case["pii"]
    return pii["sensitive_blocked"] / max(1e-12, pii["sensitive_total"])

def tenant_leak_rate(self, case):
    tenant = case["tenant"]
    return tenant["cross_tenant_hits"] / max(1e-12, tenant["access_checks"])

def output_safety_pass_rate(self, case):
    output = case["output"]
    return output["safe_outputs"] / max(1e-12, output["checked_outputs"])

```

```

def high_risk_review_coverage(self, case):
    output = case["output"]
    return output["high_risk_reviewed"] / max(1e-12, output["high_risk_outputs"])

def rag_block_rate(self, case):
    rag = case["rag"]
    return rag["unauthorized_chunks_blocked"] / max(1e-12, rag["unauthorized_chunks"])

def citation_authorized_rate(self, case):
    rag = case["rag"]
    return rag["authorized_citations"] / max(1e-12, rag["citations"])

def tool_confirm_coverage(self, case):
    agent = case["agent"]
    return agent["high_risk_confirmed"] / max(1e-12, agent["high_risk_calls"])

def unauthorized_tool_rate(self, case):
    agent = case["agent"]
    return agent["unauthorized_calls"] / max(1e-12, agent["tool_calls"])

def supply_chain_coverage(self, case):
    supply = case["supply_chain"]
    return supply["with_provenance"] / max(1e-12, supply["artifacts"])

def critical_block_rate(self, case):
    supply = case["supply_chain"]
    return supply["critical_blocked"] / max(1e-12, supply["critical_found"])

def runtime_hardening_coverage(self, case):
    runtime = case["runtime"]
    return runtime["hardened_workloads"] / max(1e-12, runtime["workloads"])

def log_redaction_coverage(self, case):
    logs = case["logs"]
    return logs["redacted_fields"] / max(1e-12, logs["sensitive_fields"])

def retention_coverage(self, case):
    logs = case["logs"]
    return logs["within_ttl"] / max(1e-12, logs["log_items"])

def audit_coverage(self, case):
    audit = case["audit_response"]
    return audit["audited_events"] / max(1e-12, audit["critical_events"])

def immutable_audit_coverage(self, case):
    audit = case["audit_response"]
    return audit["immutable_events"] / max(1e-12, audit["audited_events"])

def identity_workload_binding(self, case):
    identity = case.get("identity", {})
    return (
        self.identity_coverage(case) >= identity.get("min_identity_coverage", 1.0)
        and identity.get("service_account_binding") is True
        and identity.get("no_shared_admin") is True
        and identity.get("external_app_scoped") is True
    )

```

```

)

def rbac_abac_policy_fit(self, case):
    permission = case.get("permission", {})
    return (
        self.overprivilege_rate(case) <= permission.get("max_overprivilege_rate", 0.0)
        and permission.get("policy_engine") is True
        and permission.get("rbac_ready") is True
        and permission.get("abac_ready") is True
        and permission.get("deny_by_default") is True
        and permission.get("purpose_bound") is True
    )

def secret_lifecycle_management(self, case):
    secret = case.get("secret", {})
    return (
        self.secret_exposure_rate(case) == 0.0
        and self.secret_rotation_coverage(case) >= secret.get("min_rotation_coverage", 1.0)
        and secret.get("secret_manager") is True
        and secret.get("short_lived") is True
        and secret.get("revocation_ready") is True
        and secret.get("log_redaction") is True
        and secret.get("not_in_image") is True
    )

def data_compliance_classification(self, case):
    data = case.get("data", {})
    return (
        self.compliance_coverage(case) >= data.get("min_compliance_coverage", 1.0)
        and data.get("training_consent") is True
        and data.get("region_policy") is True
        and data.get("retention_policy") is True
        and data.get("deletion_sla") is True
        and data.get("lineage_ready") is True
    )

def pii_sensitive_data_control(self, case):
    pii = case.get("pii", {})
    return (
        self.pii_redaction_coverage(case) >= pii.get("min_redaction_coverage", 1.0)
        and self.sensitive_block_rate(case) >= pii.get("min_sensitive_block_rate", 1.0)
        and pii.get("request_scan") is True
        and pii.get("trace_scan") is True
        and pii.get("tool_result_scan") is True
    )

def tenant_isolation_boundary(self, case):
    tenant = case.get("tenant", {})
    return (
        self.tenant_leak_rate(case) <= tenant.get("max_tenant_leak_rate", 0.0)
        and tenant.get("namespace_quota") is True
        and tenant.get("network_policy") is True
        and tenant.get("cache_isolation") is True
        and tenant.get("trace_isolation") is True
        and tenant.get("cost_isolation") is True
    )

```



```

)

def model_access_control(self, case):
    model = case.get("model_access", {})
    return (
        model.get("call_policy") is True
        and model.get("download_policy") is True
        and model.get("deploy_policy") is True
        and model.get("export_block_for_restricted") is True
        and model.get("eval_report_acl") is True
        and model.get("access_audit") is True
    )

def model_output_safety_gate(self, case):
    output = case.get("output", {})
    return (
        self.output_safety_pass_rate(case) >= output.get("min_safe_output_rate", 1.0)
        and self.high_risk_review_coverage(case) >= output.get("min_review_coverage", 1.0)
        and output.get("unsafe_block") is True
        and output.get("tool_action_guard") is True
        and output.get("monitoring") is True
        and output.get("badcase_feedback") is True
    )

def rag_permission_enforcement(self, case):
    rag = case.get("rag", {})
    return (
        self.rag_block_rate(case) >= rag.get("min_block_rate", 1.0)
        and self.citation_authorized_rate(case) >= rag.get("min_citation_authorized_rate", 1.0)
        and rag.get("metadata_filter") is True
        and rag.get("chunk_acl_propagation") is True
        and rag.get("prompt_assembly_filter") is True
        and rag.get("trace_redaction") is True
    )

def agent_tool_safety_gate(self, case):
    agent = case.get("agent", {})
    return (
        self.tool_confirm_coverage(case) >= agent.get("min_confirm_coverage", 1.0)
        and self.unauthorized_tool_rate(case) <= agent.get("max_unauthorized_tool_rate", 0.0)
        and agent.get("permission_check") is True
        and agent.get("parameter_validation") is True
        and agent.get("rate_limit_timeout") is True
        and agent.get("rollback_compensation") is True
        and agent.get("tool_audit") is True
    )

def supply_chain_integrity(self, case):
    supply = case.get("supply_chain", {})
    return (
        self.supply_chain_coverage(case) >= supply.get("min_supply_chain_coverage", 1.0)
        and self.critical_block_rate(case) >= supply.get("min_critical_block_rate", 1.0)
        and supply.get("signed_artifacts") is True
        and supply.get("locked_dependencies") is True
        and supply.get("license_record") is True
    )

```

```

        and supply.get("trusted_build") is True
    )

def model_artifact_integrity(self, case):
    artifact = case.get("artifact", {})
    return (
        artifact.get("checksum") is True
        and artifact.get("signature") is True
        and artifact.get("safe_format") is True
        and artifact.get("tokenizer_config_bound") is True
        and artifact.get("download_acl") is True
        and artifact.get("deployment_smoke_test") is True
    )

def runtime_security_boundary(self, case):
    runtime = case.get("runtime", {})
    return (
        self.runtime_hardening_coverage(case) >= runtime.get("min_runtime_hardening", 1.0)
        and runtime.get("non_root") is True
        and runtime.get("read_only_fs") is True
        and runtime.get("network_policy") is True
        and runtime.get("egress_control") is True
        and runtime.get("gpu_device_isolation") is True
        and runtime.get("resource_limit") is True
    )

def log_trace_privacy_governance(self, case):
    logs = case.get("logs", {})
    return (
        self.log_redaction_coverage(case) >= logs.get("min_redaction_coverage", 1.0)
        and self.retention_coverage(case) >= logs.get("min_retention_coverage", 1.0)
        and logs.get("field_level_acl") is True
        and logs.get("encrypted_storage") is True
        and logs.get("access_audit") is True
        and logs.get("sampling_policy") is True
    )

def audit_incident_response_readiness(self, case):
    audit = case.get("audit_response", {})
    return (
        self.audit_coverage(case) >= audit.get("min_audit_coverage", 1.0)
        and self.immutable_audit_coverage(case) >= audit.get("min_immutable_coverage", 1.0)
        and audit.get("detect_coverage", 0.0) >= audit.get("min_detect_coverage", 1.0)
        and audit.get("revoke_coverage", 0.0) >= audit.get("min_revoke_coverage", 1.0)
        and audit.get("scope_coverage", 0.0) >= audit.get("min_scope_coverage", 1.0)
        and audit.get("postmortem_coverage", 0.0) >= audit.get("min_postmortem_coverage", 1.0)
        and audit.get("runbook_ready") is True
    )

def security_governance_gate(self, case):
    gate = case.get("platform_gate", {})
    return (
        gate.get("enabled") is True
        and bool(gate.get("owner"))
        and gate.get("security_policy") is True
    )

```

```

        and gate.get("exception_approval") is True
        and gate.get("audit_ready") is True
        and gate.get("incident_response_ready") is True
        and gate.get("p0_open") is False
    )

def audit_case(self, case):
    return {gate: getattr(self, gate)(case) for gate in self.GATES}

def run_all(self, cases):
    results = {case["case_id"]: self.audit_case(case) for case in cases}
    metrics = {}
    for gate in self.GATES:
        passed = sum(1 for result in results.values() if result[gate])
        metrics[gate] = round(passed / len(cases), 3)
    failed_cases = [
        case_id
        for case_id, result in results.items()
        if not all(result.values())
    ]
    failed_gates = [
        gate
        for gate in self.GATES
        if any(not result[gate] for result in results.values())
    ]
    return {
        "metrics": metrics,
        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "failed_gates": failed_gates,
        "security_governance_gate_pass": metrics["security_governance_gate"] == 1.0,
    }

def example_outputs(self, case):
    return {
        "identity_coverage": round(self.identity_coverage(case), 3),
        "overprivilege_rate": round(self.overprivilege_rate(case), 3),
        "secret_exposure_rate": round(self.secret_exposure_rate(case), 3),
        "secret_rotation_coverage": round(self.secret_rotation_coverage(case), 3),
        "compliance_coverage": round(self.compliance_coverage(case), 3),
        "pii_redaction_coverage": round(self.pii_redaction_coverage(case), 3),
        "sensitive_block_rate": round(self.sensitive_block_rate(case), 3),
        "tenant_leak_rate": round(self.tenant_leak_rate(case), 3),
        "output_safety_pass_rate": round(self.output_safety_pass_rate(case), 3),
        "rag_block_rate": round(self.rag_block_rate(case), 3),
        "tool_confirm_coverage": round(self.tool_confirm_coverage(case), 3),
        "supply_chain_coverage": round(self.supply_chain_coverage(case), 3),
        "runtime_hardening_coverage": round(self.runtime_hardening_coverage(case), 3),
        "log_redaction_coverage": round(self.log_redaction_coverage(case), 3),
        "audit_coverage": round(self.audit_coverage(case), 3),
        "incident_response_ready": self.audit_incident_response_readiness(case),
    }

```

```

def build_good_case():

```

```

return {
  "case_id": "full_security_governance",
  "identity": {
    "bound_workloads": 49,
    "total_workloads": 50,
    "min_identity_coverage": 0.95,
    "service_account_binding": True,
    "no_shared_admin": True,
    "external_app_scoped": True,
  },
  "permission": {
    "overprivileged_bindings": 1,
    "total_bindings": 100,
    "max_overprivilege_rate": 0.02,
    "policy_engine": True,
    "rbac_ready": True,
    "abac_ready": True,
    "deny_by_default": True,
    "purpose_bound": True,
  },
  "secret": {
    "exposed": 0,
    "total": 50,
    "rotated_within_ttl": 48,
    "min_rotation_coverage": 0.9,
    "secret_manager": True,
    "short_lived": True,
    "revocation_ready": True,
    "log_redaction": True,
    "not_in_image": True,
  },
  "data": {
    "classified_authorized": 38,
    "datasets": 40,
    "min_compliance_coverage": 0.9,
    "training_consent": True,
    "region_policy": True,
    "retention_policy": True,
    "deletion_sla": True,
    "lineage_ready": True,
  },
  "pii": {
    "detected": 100,
    "redacted": 98,
    "min_redaction_coverage": 0.95,
    "sensitive_total": 30,
    "sensitive_blocked": 30,
    "min_sensitive_block_rate": 1.0,
    "request_scan": True,
    "trace_scan": True,
    "tool_result_scan": True,
  },
  "tenant": {
    "cross_tenant_hits": 0,
    "access_checks": 200,
  }
}

```

```

    "max_tenant_leak_rate": 0.0,
    "namespace_quota": True,
    "network_policy": True,
    "cache_isolation": True,
    "trace_isolation": True,
    "cost_isolation": True,
},
"model_access": {
    "call_policy": True,
    "download_policy": True,
    "deploy_policy": True,
    "export_block_for_restricted": True,
    "eval_report_acl": True,
    "access_audit": True,
},
"output": {
    "checked_outputs": 1000,
    "safe_outputs": 992,
    "min_safe_output_rate": 0.98,
    "high_risk_outputs": 40,
    "high_risk_reviewed": 40,
    "min_review_coverage": 1.0,
    "unsafe_block": True,
    "tool_action_guard": True,
    "monitoring": True,
    "badcase_feedback": True,
},
"rag": {
    "unauthorized_chunks": 12,
    "unauthorized_chunks_blocked": 12,
    "min_block_rate": 1.0,
    "citations": 90,
    "authorized_citations": 90,
    "min_citation_authorized_rate": 1.0,
    "metadata_filter": True,
    "chunk_acl_propagation": True,
    "prompt_assembly_filter": True,
    "trace_redaction": True,
},
"agent": {
    "tool_calls": 200,
    "unauthorized_calls": 0,
    "max_unauthorized_tool_rate": 0.0,
    "high_risk_calls": 20,
    "high_risk_confirmed": 19,
    "min_confirm_coverage": 0.9,
    "permission_check": True,
    "parameter_validation": True,
    "rate_limit_timeout": True,
    "rollback_compensation": True,
    "tool_audit": True,
},
"supply_chain": {
    "artifacts": 100,
    "with_provenance": 94,

```

```

    "min_supply_chain_coverage": 0.9,
    "critical_found": 5,
    "critical_blocked": 5,
    "min_critical_block_rate": 1.0,
    "signed_artifacts": True,
    "locked_dependencies": True,
    "license_record": True,
    "trusted_build": True,
  },
  "artifact": {
    "checksum": True,
    "signature": True,
    "safe_format": True,
    "tokenizer_config_bound": True,
    "download_acl": True,
    "deployment_smoke_test": True,
  },
  "runtime": {
    "hardened_workloads": 38,
    "workloads": 40,
    "min_runtime_hardening": 0.9,
    "non_root": True,
    "read_only_fs": True,
    "network_policy": True,
    "egress_control": True,
    "gpu_device_isolation": True,
    "resource_limit": True,
  },
  "logs": {
    "sensitive_fields": 100,
    "redacted_fields": 98,
    "min_redaction_coverage": 0.95,
    "log_items": 1000,
    "within_ttl": 970,
    "min_retention_coverage": 0.95,
    "field_level_acl": True,
    "encrypted_storage": True,
    "access_audit": True,
    "sampling_policy": True,
  },
  "audit_response": {
    "critical_events": 1000,
    "audited_events": 990,
    "min_audit_coverage": 0.98,
    "immutable_events": 990,
    "min_immutable_coverage": 0.99,
    "detect_coverage": 0.96,
    "min_detect_coverage": 0.9,
    "revoke_coverage": 0.95,
    "min_revoke_coverage": 0.9,
    "scope_coverage": 0.94,
    "min_scope_coverage": 0.9,
    "postmortem_coverage": 0.92,
    "min_postmortem_coverage": 0.9,
    "runbook_ready": True,
  },

```

```

    },
    "platform_gate": {
        "enabled": True,
        "owner": "security-governance",
        "security_policy": True,
        "exception_approval": True,
        "audit_ready": True,
        "incident_response_ready": True,
        "p0_open": False,
    },
}

```

```

def build_bad_cases(good_case):
    cases = []

    case = deepcopy(good_case)
    case["case_id"] = "identity_binding_bad"
    case["identity"]["no_shared_admin"] = False
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "rbac_abac_policy_bad"
    case["permission"]["deny_by_default"] = False
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "secret_lifecycle_bad"
    case["secret"]["exposed"] = 1
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "data_compliance_bad"
    case["data"]["training_consent"] = False
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "pii_sensitive_control_bad"
    case["pii"]["redacted"] = 80
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "tenant_isolation_bad"
    case["tenant"]["cross_tenant_hits"] = 1
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "model_access_bad"
    case["model_access"]["download_policy"] = False
    cases.append(case)

    case = deepcopy(good_case)
    case["case_id"] = "model_output_safety_bad"
    case["output"]["high_risk_reviewed"] = 20
    cases.append(case)

```

```

case = deepcopy(good_case)
case["case_id"] = "rag_permission_bad"
case["rag"]["unauthorized_chunks_blocked"] = 10
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "agent_tool_safety_bad"
case["agent"]["unauthorized_calls"] = 1
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "supply_chain_bad"
case["supply_chain"]["signed_artifacts"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "model_artifact_bad"
case["artifact"]["signature"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "runtime_security_bad"
case["runtime"]["non_root"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "log_trace_privacy_bad"
case["logs"]["redacted_fields"] = 70
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "audit_response_bad"
case["audit_response"]["runbook_ready"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "security_governance_gate_missing_bad"
case["platform_gate"]["enabled"] = False
cases.append(case)

```

return cases

```

audit = MiniAIInfraSecurityGovernanceAudit()
good = build_good_case()
cases = [good] + build_bad_cases(good)
summary = audit.run_all(cases)

print("security_governance_examples=" + repr(audit.example_outputs(good)))
print("metrics=" + repr(summary["metrics"]))
print("hard_blocker_count=" + repr(summary["hard_blocker_count"]))
print("failed_cases=" + repr(summary["failed_cases"]))
print("failed_gates=" + repr(summary["failed_gates"]))
print("security_governance_gate_pass=" + repr(summary["security_governance_gate_pass"]))

```


参考输出应类似：

```
security_governance_examples={'identity_coverage': 0.98, 'overprivilege_rate': 0.01, 'secret_exposure_rate': 0.0, 'secret_metrics':{'identity_workload_binding': 0.941, 'rbac_abac_policy_fit': 0.941, 'secret_lifecycle_management': 0.941, 'data_compliance_cla': 0.941, 'pii_sensitivity': 0.941, 'hard_blocker_count': 16}, 'failed_cases':['identity_binding_bad', 'rbac_abac_policy_bad', 'secret_lifecycle_bad', 'data_compliance_bad', 'pii_sensitivity_bad'], 'failed_gates':['identity_workload_binding', 'rbac_abac_policy_fit', 'secret_lifecycle_management', 'data_compliance_cla'], 'security_governance_gate_pass': False}
```

这个 demo 的面试价值是把 AI Infra 安全从 “入口鉴权” 升级为平台控制面：身份、权限、密钥、数据等级、PII、多租户、模型、RAG、Agent、供应链、artifact、运行时、日志、审计和响应都要有可计算指标、硬阻断和复盘证据。

50.25 常见误区

误区一：AI Infra 安全就是 API 鉴权。

还需要数据、模型、工具、trace、供应链和运行时安全。

误区二：embedding 不敏感。

embedding 可能泄露语义信息，仍需权限和隔离。

误区三：模型权重只是普通文件。

模型权重是高价值资产，可能包含专有能力和训练数据风险。

误区四：Agent 工具调用和普通 API 调用一样。

Agent 调用由模型决策，必须加强权限、确认、审计和副作用控制。

误区五：日志越完整越好。

日志和 trace 可能包含敏感信息，必须脱敏、限权和设置留存策略。

50.26 面试常见追问

问题一：AI Infra 安全治理包括哪些方面？

可以回答：包括身份认证、权限控制、密钥管理、数据合规、多租户隔离、模型访问控制、供应链安全、运行时安全、日志 trace 安全、审计和事件响应。

问题二：如何保护训练任务中的密钥？

可以回答：使用 service account 和短期凭证，按任务最小权限注入 secret，不写入代码、镜像、命令行或日志，任务结束后失效，并记录访问审计。

问题三：企业 RAG 如何防止越权？

可以回答：文档和 chunk 带权限标签，embedding 和索引保存权限元数据，检索时平台强制 metadata filter 或物理隔离，citation 和 trace 也要做权限控制。

问题四：模型供应链安全怎么做？

可以回答：控制模型来源，对权重、tokenizer、config 做 checksum 和签名校验，使用安全格式，扫描镜像和依赖漏洞，部署前做兼容和安全检查。

50.27 小练习

1. AI Infra 中有哪些身份需要认证？
2. Dataset、Model、Endpoint、Tool 分别需要哪些权限动作？
3. 为什么训练任务不能使用长期高权限密钥？
4. 数据分级如何影响训练和推理？
5. RAG 检索为什么必须在检索阶段做权限过滤？

6. Agent 高风险工具调用应该有哪些保护?
7. 模型 artifact 安全检查包括哪些内容?
8. 日志和 trace 安全如何兼顾排障和隐私?

50.28 本章小结

本章讲了 AI Infra 安全治理。

你需要记住:

1. AI Infra 安全覆盖身份、权限、密钥、数据、模型、供应链、运行时、日志、trace 和审计。
2. 数据和模型都是高价值资产, 必须有分级、权限和血缘。
3. 训练任务和 Agent 工具调用是高风险场景, 需要最小权限、短期凭证和审计。
4. RAG 权限必须在检索阶段强制执行, 不能依赖模型判断。
5. 供应链安全要覆盖镜像、依赖、模型 artifact、tokenizer、数据集和 runtime。
6. 安全治理必须产品化和平台化, 不能只靠人工规范。

下一章我们会讲审计、日志留存、事故复盘和变更管理。

第 51 章 审计、日志留存、事故复盘和变更管理

上一章讲了安全治理。本章继续讲 AI Infra 的治理体系: 审计、日志留存、事故复盘和变更管理。

如果说监控和告警帮助我们发现问题, 审计和复盘帮助我们解释问题, 变更管理帮助我们减少问题。成熟的 AI Infra 平台, 不能只靠工程师临场排查, 而要让关键操作、关键数据、关键事件都可追溯。

先记住一句话:

审计、留存、复盘和变更管理的目标, 是让平台上的每一次重要操作都可追溯, 每一次事故都能学习, 每一次变更都能控制风险。

51.0 本讲资料边界与第二轮精修口径

本章按 WRITING_PLAN.md 做第二轮精修时, 主要参考 Kubernetes audit logging 对结构化审计事件、audit policy、stage 和元数据的官方口径, 参考 Google SRE Workbook / SRE Book 对 blameless postmortem、timeline、action item 和事故学习文化的说明, 参考 NIST SP 800-61 对 incident handling 生命周期的通用框架, 参考 OpenTelemetry 对 logs、traces、spans 和事件关联的观测模型, 也参考 DORA 对 change failure rate、恢复时间和交付稳定性的指标口径。

需要强调三点:

1. 本章只讨论防御性的审计、留存、复盘和变更治理, 不提供规避审计、删除痕迹、逃避留存或绕过审批的方法。
2. AI Infra 的变更不仅是代码发布, 还包括模型、prompt、runtime、路由、数据版本、权限策略、工具定义、RAG 知识库和 GPU 集群配置。
3. 文中 demo 是教学级治理审计脚本, 用于解释可追溯和可复盘的门禁, 不代表生产级合规系统或法律建议。

51.1 为什么 AI Infra 需要审计

AI Infra 中的关键资产很多:

1. 数据集。
2. 模型权重。
3. checkpoint。
4. endpoint。
5. prompt 模板。
6. 工具定义。
7. 知识库。

8. trace。
9. secret。
10. 发布配置。

任何一个资产被误改、误删、误用，都可能造成严重后果。

审计要回答：谁在什么时候，对什么资源，做了什么操作，结果是什么。

可以把一条审计事件抽象为：

$$E_i = (t_i, a_i, u_i, r_i, o_i, d_i, s_i, h_i, z_i)$$

其中 t_i 是时间， a_i 是 actor， u_i 是租户或项目， r_i 是资源， o_i 是操作， d_i 是权限决策， s_i 是执行状态， h_i 是 before / after diff 或 hash， z_i 是 request id、trace id、来源 IP 和审计证据。

51.2 审计和普通日志的区别

普通日志用于排查系统运行问题。

审计日志用于追踪关键操作和安全责任。

普通日志可以比较详细，保留时间可能较短。

审计日志通常要求：

1. 结构化。
2. 不可随意篡改。
3. 保留更久。
4. 可按主体和资源查询。
5. 记录操作结果。
6. 支持合规和调查。

不要把审计日志和应用 debug 日志混为一谈。

审计 schema 完整率：

$$C_{\text{schema}} = \frac{N_{\text{event_complete}}}{N_{\text{audit_event}}}$$

其中 complete 表示 timestamp、actor、tenant、action、resource、decision、status、request id 和 trace id 等必需字段齐全。

51.3 审计日志应该记录什么

审计日志基本字段：

1. timestamp。
2. actor。
3. actor type。
4. tenant。
5. action。
6. resource type。
7. resource ID。
8. request ID。
9. source IP。
10. user agent。
11. decision。
12. reason。
13. before/after diff。

14. status。

actor 可以是人、服务账号、训练任务、Agent 或自动化系统。

decision 表示权限判断结果，例如 allow 或 deny。

51.4 需要审计的操作

AI Infra 中需要审计的操作包括：

数据相关：

1. 访问敏感 dataset。
2. 导出数据。
3. 删除数据。
4. 修改数据权限。

模型相关：

1. 下载模型权重。
2. 注册模型版本。
3. 发布模型。
4. 回滚模型。
5. 删除模型 artifact。

平台相关：

1. 修改队列配额。
2. 修改限流策略。
3. 修改路由规则。
4. 修改安全策略。
5. 访问 secret。

Agent/RAG 相关：

1. 调用高风险工具。
2. 修改工具定义。
3. 修改知识库权限。
4. 查看敏感 trace。

关键操作审计覆盖率：

$$C_{\text{audit}} = \frac{N_{\text{critical_audited}}}{N_{\text{critical_operation}}}$$

这里的 critical operation 至少包括数据访问 / 导出、模型下载 / 发布、权限变更、secret 访问、高风险工具调用、知识库权限变更、发布、回滚、删除和跨地域导出。

51.5 审计日志存储要求

审计日志存储要满足：

1. 可靠写入。
2. 防篡改。
3. 可查询。
4. 可导出。
5. 权限隔离。
6. 长期留存。
7. 加密存储。
8. 生命周期管理。

防篡改可以通过 append-only 存储、对象锁、签名、专用审计系统等方式实现。

审计日志本身也可能包含敏感信息，所以访问审计日志也要被审计。

审计完整性覆盖率：

$$C_{\text{immut}} = \frac{N_{\text{immutable}}}{N_{\text{audit_event}}}$$

审计查询覆盖率：

$$C_{\text{query}} = \frac{N_{\text{queryable}}}{N_{\text{audit_event}}}$$

其中 immutable 表示事件写入后不可随意修改或删除，queryable 表示可按 actor、tenant、resource、action、request id 和时间窗口检索。

51.6 日志留存策略

日志留存不是越久越好，也不是越短越好。

要按日志类型制定策略：

1. Debug 日志。
2. 训练日志。
3. 推理请求日志。
4. RAG/Agent trace。
5. 安全日志。
6. 审计日志。
7. 成本日志。
8. 发布事件日志。

不同日志有不同价值、成本和风险。

例如审计日志可能需要长期保存，而包含用户 prompt 的 trace 可能需要短 TTL 或脱敏保存。

日志留存策略可以用收益和风险权衡：

$$V_{\text{retain}}(T) = B_{\text{debug}}(T) + B_{\text{security}}(T) + B_{\text{compliance}}(T) - K_{\text{storage}}(T) - R_{\text{privacy}}(T)$$

其中 T 是留存时间，B_debug 是排障收益，B_security 是安全调查收益，B_compliance 是合规收益，K_storage 是存储成本，R_privacy 是隐私风险。

51.7 日志留存的权衡

留存时间受多个因素影响：

1. 排障需求。
2. 安全调查需求。
3. 合规要求。
4. 存储成本。
5. 隐私风险。
6. 数据删除请求。
7. 业务价值。

完整保存所有请求内容很方便排障，但成本和隐私风险很高。

常见策略是：结构化摘要长期保存，敏感内容短期保存或采样保存。

留存合规率：

$$C_{\text{retention}} = \frac{N_{\text{within_policy}}}{N_{\text{log_item}}}$$

其中 within policy 表示日志类型、敏感等级、TTL、脱敏、加密、权限和删除请求处理都满足策略。

51.8 日志分级

日志可以分级：

1. Operational logs：运行日志。
2. Security logs：安全日志。
3. Audit logs：审计日志。
4. Diagnostic traces：诊断 trace。
5. Business events：业务事件。

不同等级决定：

1. 谁能看。
2. 保存多久。
3. 是否脱敏。
4. 是否加密。
5. 是否允许导出。

AI 系统日志经常包含用户输入和模型输出，分级非常重要。

日志分级覆盖率：

$$C_{\text{level}} = \frac{N_{\text{classified_log}}}{N_{\text{log_stream}}}$$

没有分级的日志不应该默认进入长期留存，因为无法判断访问权限、脱敏要求和导出边界。

51.9 日志脱敏和最小化

日志留存应遵循最小化原则。

能记录摘要，就不要默认记录完整敏感内容。

常见脱敏对象：

1. API key。
2. access token。
3. 手机号。
4. 邮箱。
5. 身份证件号。
6. 私有文档片段。
7. 工具调用参数。
8. 用户 prompt。

脱敏要在写入前做，而不是写入后再清洗。

写入前脱敏覆盖率：

$$C_{\text{redact}} = \frac{N_{\text{redacted_before_write}}}{N_{\text{sensitive_field}}}$$

日志最小化率：

$$M_{\text{log}} = 1 - \frac{N_{\text{raw_sensitive_saved}}}{N_{\text{sensitive_field}}}$$

51.10 Trace 留存

RAG/Agent trace 很有价值，但也敏感。

Trace 留存策略可以是：

1. 错误 trace 保留更久。
2. 慢请求 trace 保留更久。
3. 普通成功 trace 采样。
4. 高敏请求只保留结构化元数据。
5. 工具参数脱敏。
6. 检索内容按权限控制。
7. 用户可请求删除。

Trace 是调试资产，也是隐私资产。

Trace 采样率：

$$R_{\text{trace_sample}} = \frac{N_{\text{trace_kept}}}{N_{\text{trace_total}}}$$

敏感 trace 结构化留存率：

$$C_{\text{trace_meta}} = \frac{N_{\text{sensitive_trace_metadata_only}}}{N_{\text{sensitive_trace}}}$$

这两个指标要一起看：普通 trace 可采样，错误和慢请求可以保留更久，高敏 trace 应优先保留结构化元数据。

51.11 事故复盘的价值

事故复盘的目标不是追责，而是让系统变得更可靠。

复盘要回答：

1. 发生了什么？
2. 影响了谁？
3. 为什么发生？
4. 为什么没有更早发现？
5. 为什么没有自动恢复？
6. 如何防止再次发生？

如果每次事故只是临时修复，下次还会发生类似问题。

复盘完整率：

$$C_{\text{postmortem}} = \frac{N_{\text{required_section_filled}}}{N_{\text{required_section}}}$$

复盘不仅要有标题和根因，还要有影响范围、时间线、检测方式、响应过程、防线失效、行动项、owner、deadline 和验证证据。

51.12 事故复盘模板

一个复盘模板可以包括：

1. 事故标题。
2. 事故等级。
3. 时间线。
4. 影响范围。
5. 用户影响。
6. 检测方式。
7. 响应过程。
8. 根因分析。
9. 触发因素。
10. 哪些防线失效。
11. 修复措施。
12. 后续行动项。
13. 负责人。
14. 截止时间。

AI Infra 复盘还应记录相关模型、数据、runtime、集群、路由和安全策略版本。

AI Infra 版本证据覆盖率：

$$C_{\text{version}} = \frac{N_{\text{linked_version}}}{N_{\text{required_version}}}$$

其中 required version 至少包括 model、prompt、runtime、route、dataset、feature / index、tool、security policy 和 cluster config。

51.13 时间线很重要

事故时间线应该包含：

1. 首次异常发生时间。
2. 告警触发时间。
3. 值班响应时间。
4. 影响扩大时间。
5. 应急操作时间。
6. 服务恢复时间。
7. 根因确认时间。

时间线能暴露检测延迟和响应延迟。

如果问题发生 30 分钟后才告警，说明监控或 SLO 告警需要改进。

常见时间指标：

$$\text{MTTD} = t_{\text{detect}} - t_{\text{start}}$$

$$\text{MTTA} = t_{\text{ack}} - t_{\text{alert}}$$

$$\text{MTTR} = t_{\text{recover}} - t_{\text{start}}$$

其中 MTTD 是平均检测时间，MTTA 是平均响应确认时间，MTTR 是平均恢复时间。

51.14 根因分析

根因分析不要停留在“某个服务挂了”。

要继续问：

1. 为什么服务挂了？
2. 为什么没有自动摘除？
3. 为什么 fallback 没生效？
4. 为什么发布前没测出来？
5. 为什么告警太晚？
6. 为什么值班 runbook 没覆盖？

常用方法是 5 Whys。

真正的根因往往是系统防线缺失，而不是某个人点错。

系统根因覆盖率：

$$C_{\text{root}} = \frac{N_{\text{systemic_root_cause}}}{N_{\text{incident}}}$$

如果复盘只写“人工误操作”，但没有继续分析权限、审批、灰度、回滚、告警、runbook 和自动保护为什么失效， C_{root} 就很低。

51.15 行动项管理

复盘没有行动项，就只是写报告。

行动项要具体：

1. 增加某个告警。
2. 修复某个自动回滚逻辑。
3. 增加发布前检查。
4. 改进 runbook。
5. 补充权限校验。
6. 增加数据质量门禁。
7. 改进 dashboard。

每个行动项要有 owner 和 deadline。

平台应跟踪行动项是否完成。

行动项关闭率：

$$C_{\text{action}} = \frac{N_{\text{action_closed_verified}}}{N_{\text{action_total}}}$$

逾期率：

$$R_{\text{overdue}} = \frac{N_{\text{action_overdue}}}{N_{\text{action_total}}}$$

只有关闭并验证的行动项才算完成，单纯把 issue 标为 done 不够。

51.16 变更管理为什么重要

很多事故来自变更。

AI Infra 变更包括：

1. 模型版本发布。
2. prompt 修改。
3. runtime 升级。
4. 路由规则调整。
5. 安全策略变更。
6. 数据版本更新。
7. checkpoint 清理。
8. 集群驱动升级。
9. 网络配置变更。
10. 调度策略变更。

变更管理的目标是让高风险变更可审批、可灰度、可回滚、可审计。

变更失败率：

$$R_{\text{change_fail}} = \frac{N_{\text{incident_after_change}}}{N_{\text{change}}}$$

高风险变更占比：

$$R_{\text{high_risk}} = \frac{N_{\text{high_risk_change}}}{N_{\text{change}}}$$

51.17 变更记录应包含什么

变更记录包括：

1. 变更人。
2. 变更时间。
3. 变更对象。
4. 变更原因。
5. 变更内容。
6. 风险评估。
7. 回滚方案。
8. 审批记录。
9. 灰度计划。
10. 关联指标。

事故排查时，最近变更记录往往是第一入口。

变更记录完整率：

$$C_{\text{change}} = \frac{N_{\text{change_complete}}}{N_{\text{change}}}$$

complete 表示 owner、目的、对象、diff、风险、审批、灰度、回滚、指标和关联 incident / release 记录齐全。

51.18 高风险变更

高风险变更包括：

1. 生产模型全量发布。
2. 安全策略放宽。
3. RAG 权限规则修改。
4. 高风险工具启用。
5. 删除生产 artifact。
6. 调度器升级。
7. GPU driver 升级。
8. 网络策略修改。
9. Secret 轮换。
10. 跨地域数据策略变化。

高风险变更应需要审批、灰度和回滚验证。

高风险变更门禁：

$$G_{\text{high_risk_change}} = 1 [C_{\text{approval}} = 1 \wedge C_{\text{canary}} = 1 \wedge C_{\text{rollback}} = 1 \wedge C_{\text{owner}} = 1]$$

其中 approval、canary、rollback 和 owner 任一缺失，高风险变更都不应直接进入生产。

51.19 变更冻结

在一些场景下需要变更冻结：

1. 大促期间。
2. 关键客户上线期间。
3. 错误预算耗尽。
4. 大规模事故恢复期间。
5. 安全事件调查期间。

冻结不是禁止所有变更，而是只允许低风险修复或经过审批的变更。

AI 平台中模型发布和 prompt 修改也应纳入冻结范围。

冻结执行率：

$$C_{\text{freeze}} = \frac{N_{\text{blocked_during_freeze}}}{N_{\text{high_risk_during_freeze}}}$$

冻结期允许的变更应该有更严格的例外审批和回滚证据。

51.20 变更和 SLO

变更管理应和 SLO、错误预算联动。

例如：

1. 错误预算充足，正常灰度。
2. 错误预算快速消耗，放慢发布。
3. 错误预算耗尽，冻结高风险变更。
4. 变更后 SLO 破坏，自动回滚。

这能避免在系统已经不稳定时继续引入风险。

基于错误预算的变更门禁：

$$G_{\text{change_budget}} = 1 [B_{\text{remain}} \geq \beta \vee R_{\text{risk}} \leq \rho_{\text{emergency}}]$$

其中 B_{remain} 是剩余错误预算, β 是常规变更最低预算, R_{risk} 是变更风险, $\rho_{\text{emergency}}$ 是紧急修复允许风险上限。

51.21 审计、复盘、变更的闭环

三者应该形成闭环:

变更记录 -> 事件和告警 -> 事故复盘 -> 行动项 -> 改进变更流程

审计日志提供事实。

事故复盘提供分析。

变更管理减少未来风险。

如果三者割裂, 治理体系会失效。

闭环覆盖率:

$$C_{\text{loop}} = \frac{N_{\text{incident_linked_to_change_and_action}}}{N_{\text{incident}}}$$

一次事故最好能从 change record 找到触发因素, 从 audit log 找到事实, 从 postmortem 找到根因, 从 action item 找到防复发改进。

51.22 系统架构

一个治理平台可以包含:

Audit Log Service

- > Log Retention Policy Engine
- > Change Management System
- > Incident Management System
- > Postmortem Repository
- > Action Item Tracker
- > Compliance Export

它应该和权限系统、发布系统、告警系统、模型仓库、数据平台和 RAG/Agent trace 平台打通。

51.23 审计变更治理指标和最小 demo

把本章落到平台验收时, 可以用 16 个门禁:

1. Audit Event Schema Completeness: 审计事件是否包含 timestamp、actor、tenant、action、resource、decision、status、request id、trace id 和 diff / hash。
2. Critical Operation Audit Coverage: 数据、模型、权限、secret、工具、知识库、发布、回滚、删除和导出等关键操作是否被审计。
3. Audit Log Integrity Retention: 审计日志是否可靠写入、防篡改、加密、可查询、可导出和长期留存。
4. Log Retention Policy Fit: 不同日志类型是否有 TTL、敏感等级、脱敏、权限、成本和删除请求策略。
5. Log Minimization Redaction: 敏感字段是否在写入前脱敏, 是否避免长期保存原始敏感内容。
6. Trace Privacy Sampling: RAG / Agent trace 是否按错误、慢请求、普通成功和高敏请求分别采样、脱敏和留存。
7. Incident Timeline Completeness: 事故时间线是否覆盖发生、检测、告警、响应、扩散、恢复和根因确认。
8. Impact Detection Response Metrics: 是否能计算 MTTD、MTTA、MTTR、影响范围和用户影响。
9. Root Cause Systemic Analysis: 根因分析是否追到系统防线, 而不是停在个人操作或单服务故障。

10. Postmortem Action Item Closure: 行动项是否有 owner、deadline、关闭证据、验证结果和逾期跟踪。
11. Change Record Completeness: 变更记录是否包含 owner、对象、原因、diff、风险、审批、灰度、回滚和指标。
12. High Risk Change Approval: 高风险变更是否有审批、灰度、回滚验证、owner 和通知。
13. Rollout Rollback Readiness: 变更是否支持 canary、自动观测、暂停、回滚和回滚演练。
14. Change Freeze Error Budget Policy: 变更冻结和错误预算是否能约束高风险变更。
15. Audit Postmortem Change Linkage: audit log、change record、incident、postmortem 和 action item 是否能互相跳转。
16. Governance Loop Gate: 最终是否有 owner、治理策略、例外审批、dashboard、审计导出和 P0 风险阻断。

综合门禁:

$$G_{\text{audit_change}} = \prod_{j=1}^{16} G_j$$

下面是一个 0 依赖 demo, 用 toy governance cases 检查审计、留存、复盘和变更管理是否形成闭环。

```
from copy import deepcopy
```

```
class MiniAuditChangeGovernanceAudit:
```

```
    GATES = [
        "audit_event_schema_completeness",
        "critical_operation_audit_coverage",
        "audit_log_integrity_retention",
        "log_retention_policy_fit",
        "log_minimization_redaction",
        "trace_privacy_sampling",
        "incident_timeline_completeness",
        "impact_detection_response_metrics",
        "root_cause_systemic_analysis",
        "postmortem_action_item_closure",
        "change_record_completeness",
        "high_risk_change_approval",
        "rollout_rollback_readiness",
        "change_freeze_error_budget_policy",
        "audit_postmortem_change_linkage",
        "governance_loop_gate",
    ]
```

```
    SCHEMA_FIELDS = ["timestamp", "actor", "tenant", "action", "resource", "decision", "status", "request_id", "trace_id"]
```

```
    CHANGE_FIELDS = ["owner", "time", "object", "reason", "diff", "risk", "approval", "canary", "rollback", "metrics"]
```

```
    TIMELINE_FIELDS = ["start", "detect", "alert", "ack", "mitigate", "recover", "root_confirmed"]
```

```
    @staticmethod
```

```
    def present(record, key):
```

```
        return key in record and record[key] is not None and record[key] != ""
```

```
    def coverage(self, record, fields):
```

```
        if not record:
```

```
            return 0.0
```

```
        return sum(1 for field in fields if self.present(record, field)) / len(fields)
```

```
    def audit_schema_coverage(self, case):
```

```

    schema = case["audit_schema"]
    return schema["complete_events"] / max(1e-12, schema["audit_events"])

def critical_audit_coverage(self, case):
    ops = case["critical_ops"]
    return ops["audited"] / max(1e-12, ops["total"])

def immutable_coverage(self, case):
    integrity = case["audit_integrity"]
    return integrity["immutable_events"] / max(1e-12, integrity["audit_events"])

def queryable_coverage(self, case):
    integrity = case["audit_integrity"]
    return integrity["queryable_events"] / max(1e-12, integrity["audit_events"])

def retention_policy_fit(self, case):
    retention = case["retention"]
    return retention["within_policy"] / max(1e-12, retention["log_items"])

def redaction_coverage(self, case):
    redact = case["redaction"]
    return redact["redacted_before_write"] / max(1e-12, redact["sensitive_fields"])

def log_minimization_rate(self, case):
    redact = case["redaction"]
    return 1.0 - redact["raw_sensitive_saved"] / max(1e-12, redact["sensitive_fields"])

def trace_sample_rate(self, case):
    trace = case["trace"]
    return trace["kept"] / max(1e-12, trace["total"])

def trace_metadata_only_rate(self, case):
    trace = case["trace"]
    return trace["sensitive_metadata_only"] / max(1e-12, trace["sensitive_trace"])

def timeline_coverage(self, case):
    return self.coverage(case["timeline"], self.TIMELINE_FIELDS)

def mttdd(self, case):
    t = case["timeline"]
    return t["detect"] - t["start"]

def mttta(self, case):
    t = case["timeline"]
    return t["ack"] - t["alert"]

def mtttr(self, case):
    t = case["timeline"]
    return t["recover"] - t["start"]

def root_cause_coverage(self, case):
    root = case["root_cause"]
    return root["systemic_root_causes"] / max(1e-12, root["incidents"])

def action_closure_rate(self, case):

```

```

    action = case["actions"]
    return action["closed_verified"] / max(1e-12, action["total"])

def action_overdue_rate(self, case):
    action = case["actions"]
    return action["overdue"] / max(1e-12, action["total"])

def change_record_coverage(self, case):
    change = case["change"]
    return change["complete_records"] / max(1e-12, change["total_changes"])

def high_risk_approval_rate(self, case):
    high = case["high_risk_change"]
    return high["approved"] / max(1e-12, high["total"])

def freeze_execution_rate(self, case):
    freeze = case["freeze_budget"]
    return freeze["blocked_during_freeze"] / max(1e-12, freeze["high_risk_during_freeze"])

def loop_coverage(self, case):
    loop = case["linkage"]
    return loop["incident_linked"] / max(1e-12, loop["incidents"])

def audit_event_schema_completeness(self, case):
    schema = case.get("audit_schema", {})
    return (
        self.audit_schema_coverage(case) >= schema.get("min_schema_coverage", 1.0)
        and self.coverage(schema.get("fields", {}), self.SCHEMA_FIELDS) == 1.0
        and schema.get("stage_recorded") is True
    )

def critical_operation_audit_coverage(self, case):
    ops = case.get("critical_ops", {})
    return (
        self.critical_audit_coverage(case) >= ops.get("min_critical_audit_coverage", 1.0)
        and ops.get("data_model_secret_tool_release") is True
        and ops.get("deny_events_included") is True
    )

def audit_log_integrity_retention(self, case):
    integrity = case.get("audit_integrity", {})
    return (
        self.immutable_coverage(case) >= integrity.get("min_immutable_coverage", 1.0)
        and self.queryable_coverage(case) >= integrity.get("min_queryable_coverage", 1.0)
        and integrity.get("reliable_write") is True
        and integrity.get("encrypted") is True
        and integrity.get("export_ready") is True
        and integrity.get("access_to_audit_is_audited") is True
    )

def log_retention_policy_fit(self, case):
    retention = case.get("retention", {})
    return (
        self.retention_policy_fit(case) >= retention.get("min_retention_fit", 1.0)
        and retention.get("type_ttl_policy") is True
    )

```

```

        and retention.get("delete_request_ready") is True
        and retention.get("cost_budget_ready") is True
        and retention.get("compliance_export_ready") is True
    )

def log_minimization_redaction(self, case):
    redact = case.get("redaction", {})
    return (
        self.redaction_coverage(case) >= redact.get("min_redaction_coverage", 1.0)
        and self.log_minimization_rate(case) >= redact.get("min_minimization_rate", 1.0)
        and redact.get("before_write") is True
        and redact.get("field_acl") is True
    )

def trace_privacy_sampling(self, case):
    trace = case.get("trace", {})
    return (
        trace.get("min_sample_rate", 0.0) <= self.trace_sample_rate(case) <= trace.get("max_sample_rate", 1.0)
        and self.trace_metadata_only_rate(case) >= trace.get("min_metadata_only_rate", 1.0)
        and trace.get("error_trace_extended") is True
        and trace.get("tool_args_redacted") is True
        and trace.get("permission_filter") is True
    )

def incident_timeline_completeness(self, case):
    timeline = case.get("timeline", {})
    return self.timeline_coverage(case) == 1.0 and timeline.get("ordered") is True

def impact_detection_response_metrics(self, case):
    metrics = case.get("incident_metrics", {})
    return (
        self.mttdd(case) <= metrics.get("max_mttdd_min", 0)
        and self.mtta(case) <= metrics.get("max_mtta_min", 0)
        and self.mttr(case) <= metrics.get("max_mttr_min", 0)
        and metrics.get("impact_quantified") is True
        and metrics.get("customer_slice_ready") is True
    )

def root_cause_systemic_analysis(self, case):
    root = case.get("root_cause", {})
    return (
        self.root_cause_coverage(case) >= root.get("min_root_cause_coverage", 1.0)
        and root.get("five_whys") is True
        and root.get("defense_failure_identified") is True
        and root.get("not_person_blame") is True
    )

def postmortem_action_item_closure(self, case):
    action = case.get("actions", {})
    return (
        self.action_closure_rate(case) >= action.get("min_closure_rate", 1.0)
        and self.action_overdue_rate(case) <= action.get("max_overdue_rate", 0.0)
        and action.get("owner_deadline") is True
        and action.get("verification_evidence") is True
    )

```



```

def change_record_completeness(self, case):
    change = case.get("change", {})
    return (
        self.change_record_coverage(case) >= change.get("min_change_record_coverage", 1.0)
        and self.coverage(change.get("fields", {}), self.CHANGE_FIELDS) == 1.0
        and change.get("recent_change_search") is True
    )

def high_risk_change_approval(self, case):
    high = case.get("high_risk_change", {})
    return (
        self.high_risk_approval_rate(case) >= high.get("min_approval_rate", 1.0)
        and high.get("canary_required") is True
        and high.get("rollback_verified") is True
        and high.get("owner_notify") is True
    )

def rollout_rollback_readiness(self, case):
    rollout = case.get("rollout", {})
    return (
        rollout.get("canary_ready") is True
        and rollout.get("auto_observe") is True
        and rollout.get("pause_ready") is True
        and rollout.get("rollback_ready") is True
        and rollout.get("rollback_drill") is True
    )

def change_freeze_error_budget_policy(self, case):
    freeze = case.get("freeze_budget", {})
    return (
        self.freeze_execution_rate(case) >= freeze.get("min_freeze_execution", 1.0)
        and freeze.get("error_budget_gate") is True
        and freeze.get("exception_approval") is True
        and freeze.get("emergency_fix_path") is True
    )

def audit_postmortem_change_linkage(self, case):
    linkage = case.get("linkage", {})
    return (
        self.loop_coverage(case) >= linkage.get("min_loop_coverage", 1.0)
        and linkage.get("audit_to_change") is True
        and linkage.get("incident_to_postmortem") is True
        and linkage.get("postmortem_to_action") is True
        and linkage.get("action_to_regression") is True
    )

def governance_loop_gate(self, case):
    gate = case.get("platform_gate", {})
    return (
        gate.get("enabled") is True
        and bool(gate.get("owner"))
        and gate.get("governance_policy") is True
        and gate.get("exception_approval") is True
        and gate.get("dashboard_ready") is True
    )

```

```

        and gate.get("compliance_export") is True
        and gate.get("p0_open") is False
    )

def audit_case(self, case):
    return {gate: getattr(self, gate)(case) for gate in self.GATES}

def run_all(self, cases):
    results = {case["case_id"]: self.audit_case(case) for case in cases}
    metrics = {}
    for gate in self.GATES:
        passed = sum(1 for result in results.values() if result[gate])
        metrics[gate] = round(passed / len(cases), 3)
    failed_cases = [
        case_id
        for case_id, result in results.items()
        if not all(result.values())
    ]
    failed_gates = [
        gate
        for gate in self.GATES
        if any(not result[gate] for result in results.values())
    ]
    return {
        "metrics": metrics,
        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "failed_gates": failed_gates,
        "governance_loop_gate_pass": metrics["governance_loop_gate"] == 1.0,
    }

def example_outputs(self, case):
    return {
        "audit_schema_coverage": round(self.audit_schema_coverage(case), 3),
        "critical_audit_coverage": round(self.critical_audit_coverage(case), 3),
        "immutable_coverage": round(self.immutable_coverage(case), 3),
        "retention_policy_fit": round(self.retention_policy_fit(case), 3),
        "redaction_coverage": round(self.redaction_coverage(case), 3),
        "log_minimization_rate": round(self.log_minimization_rate(case), 3),
        "trace_sample_rate": round(self.trace_sample_rate(case), 3),
        "timeline_coverage": round(self.timeline_coverage(case), 3),
        "mttd_min": self.mttdd(case),
        "mtta_min": self.mttta(case),
        "mttr_min": self.mttr(case),
        "root_cause_coverage": round(self.root_cause_coverage(case), 3),
        "action_closure_rate": round(self.action_closure_rate(case), 3),
        "change_record_coverage": round(self.change_record_coverage(case), 3),
        "freeze_execution_rate": round(self.freeze_execution_rate(case), 3),
        "loop_coverage": round(self.loop_coverage(case), 3),
    }

def build_good_case():
    return {
        "case_id": "full_audit_change_governance",
    }

```

```

"audit_schema": {
  "audit_events": 1000,
  "complete_events": 980,
  "min_schema_coverage": 0.95,
  "fields": {
    "timestamp": True,
    "actor": True,
    "tenant": True,
    "action": True,
    "resource": True,
    "decision": True,
    "status": True,
    "request_id": True,
    "trace_id": True,
    "diff_hash": True,
  },
  "stage_recorded": True,
},
"critical_ops": {
  "audited": 196,
  "total": 200,
  "min_critical_audit_coverage": 0.95,
  "data_model_secret_tool_release": True,
  "deny_events_included": True,
},
"audit_integrity": {
  "audit_events": 1000,
  "immutable_events": 1000,
  "min_immutable_coverage": 1.0,
  "queryable_events": 990,
  "min_queryable_coverage": 0.95,
  "reliable_write": True,
  "encrypted": True,
  "export_ready": True,
  "access_to_audit_is_audited": True,
},
"retention": {
  "log_items": 1000,
  "within_policy": 960,
  "min_retention_fit": 0.95,
  "type_ttl_policy": True,
  "delete_request_ready": True,
  "cost_budget_ready": True,
  "compliance_export_ready": True,
},
"redaction": {
  "sensitive_fields": 100,
  "redacted_before_write": 98,
  "min_redaction_coverage": 0.95,
  "raw_sensitive_saved": 1,
  "min_minimization_rate": 0.98,
  "before_write": True,
  "field_acl": True,
},
"trace": {

```

```

    "total": 1000,
    "kept": 200,
    "min_sample_rate": 0.05,
    "max_sample_rate": 0.3,
    "sensitive_trace": 100,
    "sensitive_metadata_only": 100,
    "min_metadata_only_rate": 1.0,
    "error_trace_extended": True,
    "tool_args_redacted": True,
    "permission_filter": True,
},
"timeline": {
    "start": 0,
    "detect": 4,
    "alert": 5,
    "ack": 8,
    "mitigate": 20,
    "recover": 45,
    "root_confirmed": 80,
    "ordered": True,
},
"incident_metrics": {
    "max_mttdd_min": 10,
    "max_mtta_min": 5,
    "max_mttr_min": 60,
    "impact_quantified": True,
    "customer_slice_ready": True,
},
"root_cause": {
    "incidents": 10,
    "systemic_root_causes": 9,
    "min_root_cause_coverage": 0.8,
    "five_whys": True,
    "defense_failure_identified": True,
    "not_person_blame": True,
},
"actions": {
    "total": 20,
    "closed_verified": 18,
    "min_closure_rate": 0.85,
    "overdue": 1,
    "max_overdue_rate": 0.1,
    "owner_deadline": True,
    "verification_evidence": True,
},
"change": {
    "total_changes": 100,
    "complete_records": 95,
    "min_change_record_coverage": 0.9,
    "fields": {
        "owner": True,
        "time": True,
        "object": True,
        "reason": True,
        "diff": True,
    }
}

```

```

        "risk": True,
        "approval": True,
        "canary": True,
        "rollback": True,
        "metrics": True,
    },
    "recent_change_search": True,
},
"high_risk_change": {
    "total": 20,
    "approved": 20,
    "min_approval_rate": 1.0,
    "canary_required": True,
    "rollback_verified": True,
    "owner_notify": True,
},
"rollout": {
    "canary_ready": True,
    "auto_observe": True,
    "pause_ready": True,
    "rollback_ready": True,
    "rollback_drill": True,
},
"freeze_budget": {
    "high_risk_during_freeze": 10,
    "blocked_during_freeze": 10,
    "min_freeze_execution": 1.0,
    "error_budget_gate": True,
    "exception_approval": True,
    "emergency_fix_path": True,
},
"linkage": {
    "incidents": 10,
    "incident_linked": 10,
    "min_loop_coverage": 1.0,
    "audit_to_change": True,
    "incident_to_postmortem": True,
    "postmortem_to_action": True,
    "action_to_regression": True,
},
"platform_gate": {
    "enabled": True,
    "owner": "audit-change-governance",
    "governance_policy": True,
    "exception_approval": True,
    "dashboard_ready": True,
    "compliance_export": True,
    "p0_open": False,
},
}

```

```

def build_bad_cases(good_case):
    cases = []

```

```

case = deepcopy(good_case)
case["case_id"] = "audit_schema_missing_bad"
case["audit_schema"]["fields"]["trace_id"] = None
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "critical_operation_audit_bad"
case["critical_ops"]["audited"] = 150
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "audit_integrity_bad"
case["audit_integrity"]["immutable_events"] = 900
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "retention_policy_bad"
case["retention"]["delete_request_ready"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "redaction_minimization_bad"
case["redaction"]["raw_sensitive_saved"] = 20
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "trace_privacy_sampling_bad"
case["trace"]["sensitive_metadata_only"] = 60
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "timeline_incomplete_bad"
case["timeline"]["root_confirmed"] = None
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "impact_response_metrics_bad"
case["timeline"]["ack"] = 20
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "root_cause_shallow_bad"
case["root_cause"]["defense_failure_identified"] = False
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "action_item_closure_bad"
case["actions"]["closed_verified"] = 10
cases.append(case)

case = deepcopy(good_case)
case["case_id"] = "change_record_bad"
case["change"]["fields"]["rollback"] = None
cases.append(case)

```


审计日志关注关键操作和责任追踪，要求更强完整性、留存和查询能力。

误区二：事故复盘是找人背锅。

复盘目标是修复系统防线，而不是追责个人。

误区三：Prompt 修改不是变更。

Prompt 修改会改变模型行为，必须纳入变更管理。

误区四：日志留存越久越好。

长期留存敏感内容会增加成本和隐私风险，要分级留存。

误区五：行动项写了就算完成。

行动项必须有 owner、deadline 和完成验证。

51.25 面试常见追问

问题一：审计日志应该记录什么？

可以回答：记录 actor、tenant、action、resource type、resource ID、timestamp、source、decision、reason、before/after diff 和 status，用于追踪关键操作。

问题二：AI Infra 哪些操作需要审计？

可以回答：数据访问和导出、模型下载和部署、权限变更、secret 访问、路由和安全策略修改、高风险工具调用、知识库权限修改、发布和回滚都需要审计。

问题三：事故复盘怎么做？

可以回答：记录时间线、影响范围、检测和响应过程、根因、防线失效点、修复措施和行动项，重点改进系统而不是追责个人。

问题四：为什么 prompt 修改也要变更管理？

可以回答：prompt 会影响模型行为、输出格式、安全策略和工具调用，可能造成线上质量和安全问题，因此需要版本、审批、灰度、回滚和审计。

51.26 小练习

1. 审计日志和 debug 日志有什么区别？
2. 哪些 AI Infra 操作必须审计？
3. 日志留存策略应考虑哪些因素？
4. 为什么 trace 留存要采样和脱敏？
5. 事故复盘时间线应记录哪些关键时间点？
6. 什么是高风险变更？请举 5 个 AI Infra 例子。
7. 错误预算耗尽时，变更管理应该如何调整？
8. 如何设计一个事故行动项跟踪系统？

51.27 本章小结

本章讲了审计、日志留存、事故复盘和变更管理。

你需要记住：

1. 审计用于追踪关键操作，普通日志用于排障，两者目标不同。
2. AI Infra 审计要覆盖数据、模型、权限、secret、发布、RAG/Agent 和高风险工具。
3. 日志和 trace 留存要在排障价值、合规、成本和隐私之间权衡。
4. 事故复盘要关注系统改进，不是追责个人。
5. 变更管理要覆盖模型、prompt、runtime、路由、数据、安全策略和基础设施配置。
6. 审计、复盘和变更管理应形成治理闭环。

下一章我们会进入第七部分：平台工程实践与组织协作，先讲平台 API、SDK、CLI 和 Web Console 设计。

第 52 章 设计一个大模型训练平台

前面六个部分讲了 AI Infra 的硬件、集群、训练平台、推理平台、数据模型实验平台、可观测性和治理。从本章开始进入第七部分：AI Infra 系统设计与开放题。

本章聚焦高频系统设计题：设计一个大模型训练平台。

这类题目常见于大模型平台、Infra、MLOps、LLMOps、训练系统和平台工程岗位。面试官不是想听你说“用 Kubernetes 跑训练”，而是考你是否理解训练任务从提交到调度、启动、运行、监控、checkpoint、容错、评估和产物注册的完整生命周期。

先记住一句话：

大模型训练平台不是脚本启动器，而是围绕 TrainingJob 生命周期构建的多租户、可观测、可恢复、可治理的训练任务管理系统。

52.0 本讲资料边界与第二轮精修口径

本章按 WRITING_PLAN.md 做第二轮精修时，主要参考 Kubernetes Job 对 run-to-completion workload 的通用控制器语义，参考 Kueue 对 queue、admission、quota、cohort 和 workload 调度的官方口径，参考 Kubeflow Trainer / Training Operator 对分布式训练任务 API 和 runtime 的抽象，参考 PyTorch torchrun / Elastic 对 rank、world size、rendezvous 和 worker restart 的说明，也参考 MLflow Tracking 对 experiment、run、params、metrics 和 artifacts 的实验追踪模型。

需要强调三点：

1. 本章讲的是系统设计题的稳定抽象，不绑定某个云厂商、内部平台、Kubernetes 发行版、训练 Operator 或单一 launcher。
2. 训练平台的核心不是“能创建 Pod”，而是把需求、TrainingJob 契约、准入、调度、分布式启动、数据、checkpoint、观测、容错、安全、成本和产物注册串成可审计生命周期。
3. 文中 demo 是教学级系统设计审计脚本，用于训练面试表达和设计自查，不代表生产级调度器、训练 Operator 或企业平台实现。

52.1 题目理解

题目可能这样问：

1. 请设计一个支持多机多卡的大模型训练平台。
2. 请设计一个公司内部训练平台，支持预训练、SFT、评估和 checkpoint。
3. 请设计一个多租户 GPU 训练平台。
4. 请设计一个支持 torchrun、DeepSpeed、Megatron、Ray 的训练平台。

这些题本质相同：设计 TrainingJob 生命周期管理系统。

52.2 先做需求澄清

面试时先问清楚：

1. 支持哪些训练类型？预训练、SFT、RLHF、DPO、评估？
2. 支持多大规模？单机多卡还是多机多卡？
3. GPU 集群规模多大？是否异构？
4. 是否多租户？是否有队列、配额、优先级？
5. 是否要求 checkpoint 和自动恢复？
6. 是否要求 experiment tracking？
7. 是否需要数据版本和模型仓库？
8. 是否支持抢占和低优先级队列？

9. 是否需要 Web Console、CLI、API?
10. SLO 是什么? 调度等待、任务成功率、恢复时间?

需求澄清能体现你不是背模板, 而是在做系统设计。

52.3 核心目标

训练平台核心目标:

1. 支持训练任务标准化提交。
2. 支持多机多卡分布式启动。
3. 支持 GPU 资源调度、队列和配额。
4. 支持数据、代码、镜像和配置版本化。
5. 支持日志、指标、事件和实验追踪。
6. 支持 checkpoint 保存和断点恢复。
7. 支持容错、重试和抢占恢复。
8. 支持多租户权限、安全和审计。
9. 支持模型产物注册和评估联动。
10. 支持成本归因和资源利用率治理。

这十点基本覆盖训练平台系统设计主干。

52.4 核心对象: TrainingJob

TrainingJob 是训练平台的核心抽象。

它不是普通 K8s Job。

TrainingJob 应该包含:

1. job name。
2. owner / tenant / project。
3. priority / queue。
4. resource request。
5. image。
6. code version。
7. entrypoint。
8. training config。
9. dataset version。
10. output artifact path。
11. checkpoint policy。
12. retry policy。
13. launcher type。
14. environment。
15. permissions。

TrainingJob 是平台理解训练任务的方式。

可以把一条训练任务抽象为:

$$J_i = (u_i, q_i, r_i, l_i, d_i, c_i, k_i, o_i, s_i, z_i)$$

其中 u_i 是 owner / tenant, q_i 是队列与优先级, r_i 是资源请求, l_i 是 launcher 配置, d_i 是 dataset / data manifest, c_i 是代码、镜像和最终配置, k_i 是 checkpoint policy, o_i 是 observability / experiment tracking, s_i 是 security / audit policy, z_i 是成本、产物和治理元数据。

52.5 TrainingJob 示例

可以用一个简化配置说明：

```
apiVersion: ai.company.com/v1
kind: TrainingJob
metadata:
  name: chat-sft-v12
spec:
  owner: team-a
  queue: high-priority
  resources:
    gpuType: h100
    nodes: 8
    gpusPerNode: 8
  image: registry/train:pytorch-2.4-cuda12
  code:
    repo: git@example.com:llm/train.git
    commit: abc123
  launcher:
    type: deepspeed
  dataset:
    name: sft_mix
    version: 2026-05-30-v1
  checkpoint:
    intervalSteps: 1000
    keepLast: 3
  command: python train.py --config configs/sft.yaml
```

这个对象能让平台做校验、调度、追踪、恢复和审计。

52.6 总体架构

可以设计如下架构：

```
User / CLI / Web Console
-> Training API Server
-> Auth / Quota / Validation
-> TrainingJob Controller
-> Queue / Scheduler
-> Kubernetes / GPU Cluster
-> Launcher Controller
-> Training Workers
-> Data Platform
-> Checkpoint Store
-> Experiment Tracker
-> Model Registry
-> Observability / Cost / Audit
```

各模块职责清晰：API 管提交，Controller 管生命周期，Scheduler 管资源，Launcher 管分布式启动，Observability 管可见性。

52.7 提交系统设计

提交入口可以包括：

1. Web Console。
2. CLI。

3. SDK。
4. REST/gRPC API。
5. CI/CD pipeline。

提交时平台要做校验：

1. 用户身份。
2. 队列权限。
3. GPU 配额。
4. 镜像合法性。
5. 数据集权限。
6. checkpoint 路径权限。
7. 配置合法性。
8. 资源规格合法性。

不合法任务应在提交阶段失败，而不是调度后才失败。

52.8 队列和配额

多租户训练平台必须有队列和配额。

队列维度：

1. team。
2. project。
3. priority。
4. training type。
5. GPU type。

配额维度：

1. 最大 GPU 数。
2. 最大并发任务数。
3. 最大单任务规模。
4. 存储配额。
5. 月度 GPU hour 预算。

配额既保护平台，也保护团队之间的公平性。

队列体验可以用 P95 等待时间描述：

$$T_{\text{queue,p95}} = Q_{0.95}(\{t_{\text{start},i} - t_{\text{submit},i}\}_{i=1}^N)$$

如果训练平台只说“有队列”，但不能按 team、priority、GPU type、job type 和时间窗口解释等待时间，就很难支撑资源治理。

52.9 调度系统设计

训练调度需要支持：

1. gang scheduling。
2. GPU 类型匹配。
3. 拓扑感知。
4. 队列优先级。
5. 配额限制。
6. 抢占。
7. 低优先级队列。
8. 资源碎片优化。
9. 数据本地性。

大模型训练不能只用普通 Pod 调度。

多机多卡任务需要所有 worker 同时启动，否则任务无法运行。

world size 和有效 batch 可以写成：

$$W = n_{\text{node}} n_{\text{proc}}$$

$$B_{\text{eff}} = B_{\text{micro}} A_{\text{grad}} W$$

其中 n_{node} 是节点数, n_{proc} 是每节点进程数, B_{micro} 是每进程 micro batch, A_{grad} 是 gradient accumulation steps。系统设计题里说支持分布式训练，至少要能解释这几个量如何由平台配置和 launcher 一致生成。

52.10 分布式启动器

平台应封装 launcher。

支持：

1. torchrun。
2. DeepSpeed。
3. Megatron。
4. Ray。
5. MPI。
6. 自研 launcher。

Launcher 负责：

1. 计算 world size。
2. 设置 rank。
3. 设置 master address。
4. 注入环境变量。
5. 启动 worker。
6. 收集退出状态。
7. 处理失败和重试。

平台要屏蔽启动细节，让用户关注训练逻辑。

52.11 数据供给设计

训练平台要和数据平台打通。

TrainingJob 引用 dataset version，而不是裸路径。

平台负责：

1. 校验数据权限。
2. 解析 dataset manifest。
3. 注入数据路径。
4. 选择读取模式。
5. 配置缓存。
6. 记录数据版本。
7. 监控 data loader 吞吐。

数据供给慢会直接降低 GPU 利用率。

52.12 Checkpoint 设计

Checkpoint 是训练平台容错核心。

平台要支持：

1. 定期保存。
2. 手动保存。
3. 异步保存。
4. 保留策略。
5. checksum 校验。
6. manifest。
7. 跨节点恢复。
8. 从 checkpoint resume。

Checkpoint policy 应属于 TrainingJob 配置，而不是训练脚本临时决定。

一次故障最多丢失的 GPU 小时可以粗略写成：

$$L_{\text{lost}} = G \frac{t_{\text{fail}} - t_{\text{ckpt}}}{60}$$

其中 G 是任务使用的 GPU 数， $t_{\text{fail}} - t_{\text{ckpt}}$ 是距离最近可恢复 checkpoint 的分钟数。checkpoint 频率不是纯技术偏好，而是恢复目标、存储成本和 GPU 浪费之间的取舍。

52.13 容错和重试

训练任务失败可能来自：

1. 节点故障。
2. 网络故障。
3. 存储故障。
4. OOM。
5. 用户代码错误。
6. 数据错误。

平台要区分可重试和不可重试错误。

可重试：节点丢失、临时存储错误、可恢复通信失败。

不可重试：配置错误、权限错误、代码语法错误、数据 schema 错误。

不要无限重试用户错误。

52.14 可观测性设计

训练平台可观测性包括：

1. job status。
2. queue wait time。
3. scheduling events。
4. per-rank logs。
5. train loss。
6. eval loss。
7. learning rate。
8. gradient norm。
9. tokens/s。
10. GPU utilization。
11. data loader time。
12. checkpoint events。

13. cost。

用户需要一个 TrainingJob 页面，看状态、日志、指标、事件、checkpoint 和成本。

52.15 实验追踪联动

每个 TrainingJob 应自动创建 experiment run。

记录：

1. 参数。
2. 数据版本。
3. 代码版本。
4. 镜像版本。
5. 环境信息。
6. 指标曲线。
7. checkpoint artifact。
8. eval report。

这样训练结果才能复现和比较。

不要依赖用户手动登记实验结果。

trace / experiment / artifact 链接覆盖率可以写成：

$$R_{\text{trace}} = \frac{N_{\text{linked}}}{N_{\text{required}}}$$

其中 required 至少包括 TrainingJob、final config、image digest、code commit、dataset version、checkpoint manifest、metrics、logs、eval report 和 model artifact。

52.16 模型仓库联动

训练完成后，平台可以把 checkpoint 注册到模型仓库。

流程：

```
TrainingJob finished
-> select checkpoint
-> validate artifact
-> register ModelVersion candidate
-> trigger eval job
-> attach eval report
```

这样训练产物才能进入标准发布流程。

52.17 安全和审计

安全设计包括：

1. 用户认证。
2. 队列权限。
3. 数据集权限。
4. secret 最小注入。
5. 镜像安全扫描。
6. 网络隔离。
7. artifact 访问控制。
8. 日志脱敏。
9. 审计记录。

训练任务运行用户代码，安全风险很高。

平台必须限制数据、密钥和网络访问。

52.18 成本治理

训练平台成本治理包括：

1. 提交时成本预估。
2. 运行中成本统计。
3. 按团队和项目归因。
4. failed job GPU hours。
5. idle GPU hours。
6. 低优先级队列。
7. 抢占任务。
8. checkpoint 存储成本。
9. 预算告警。

用户提交任务前应看到预估资源和成本。

平台负责人应看到集群利用率和浪费来源。

52.19 Web Console 设计

Web Console 页面可以包括：

1. 任务列表。
2. 提交任务表单。
3. 队列和配额页面。
4. TrainingJob 详情页。
5. 日志查看。
6. 指标曲线。
7. checkpoint 列表。
8. 实验对比。
9. 成本页面。
10. 审计记录。

TrainingJob 详情页是核心，应把用户排障所需信息集中展示。

52.20 API / CLI / SDK 设计

平台应提供 API、CLI 和 SDK。

CLI 示例：

```
ai train submit -f job.yaml
ai train status chat-sft-v12
ai train logs chat-sft-v12 --rank 0
ai train stop chat-sft-v12
ai train resume chat-sft-v12 --checkpoint latest
```

SDK 用于自动化实验和 CI/CD。

API 用于平台集成。

三者应该共享同一套 TrainingJob schema。

52.21 状态机设计

TrainingJob 状态机可以是：

Created -> Validating -> Queued -> Scheduling -> Running -> Succeeded
 -> Failed
 -> Preempted
 -> Retrying
 -> Cancelled

每次状态变化都应记录 event。

状态机让用户知道任务卡在哪一步。

52.22 训练平台系统设计指标和最小 demo

系统设计题不是只看架构图是否完整，也要能把设计拆成可验收门禁。可以用 16 个维度审计：

1. Requirement Clarification: 是否问清训练类型、规模、多租户、抢占、恢复、发布和成本预算。
2. TrainingJob Contract: TrainingJob 是否绑定 owner、image digest、code commit、dataset version、final config、resources、distributed、checkpoint、queue 和 security。
3. Lifecycle State Machine: 状态机是否覆盖 submitted、validated、queued、scheduled、starting、running、checkpointing、retrying、failed、completed 和 cancelled。
4. Admission Quota Policy: 提交阶段是否做 schema、权限、配额、priority、checkpoint path 和 idempotency 校验。
5. Scheduler Resource Fit: 是否支持 fair share、gang scheduling、拓扑感知、backfilling、preemption、fragmentation 和 starvation control。
6. Distributed Launcher Fit: launcher 是否能正确生成 rank、world size、rendezvous、GPU binding、NCCL 和 adapter 配置。
7. Data Config Lineage: 数据、代码、镜像、环境、seed、tokenizer 和 final config 是否可追踪。
8. Checkpoint Recovery Coupling: checkpoint 是否绑定 manifest、checksum、restore drill、preemption hook 和最大丢失进度。
9. Observability Experiment Linkage: logs、metrics、events、traces、experiment run 和 artifact registry 是否互相链接。
10. Fault Recovery Flow: 是否区分平台故障、用户错误、数据错误、OOM、通信失败和权限错误，并有 retry budget。
11. Security Audit Governance: 身份、RBAC / ABAC、数据用途、secret、runtime 隔离、镜像供应链和审计是否覆盖。
12. Cost Capacity Governance: GPU-hours、存储、网络、失败成本、idle、预算和容量计划是否能归因。
13. Developer Self Service: Portal、CLI、SDK、模板、状态页、错误诊断和 checkpoint resume 是否可自助。
14. Interface Artifact Contract: API / CLI / SDK / Web、Model Registry、Eval Report 和 Deployment Package 是否共享契约。
15. Tradeoff Boundary Reasoning: 是否能讲清资源利用率、等待时间、checkpoint、抢占、隔离、日志和自动重试的 trade-off。
16. Training Platform Design Gate: 最终是否有 owner、dashboard、例外审批、审计导出和 P0 风险阻断。

综合门禁：

$$G_{\text{training_platform_design}} = 1 \left[\min_j C_j \geq \tau_j \wedge T_{\text{queue}, p95} \leq B_{\text{queue}} \wedge L_{\text{lost}} \leq B_{\text{loss}} \wedge R_{\text{trace}} \geq \rho_{\text{trace}} \wedge P_0 = 0 \right]$$

下面是一个 0 依赖 demo，用 toy design cases 检查训练平台系统设计是否覆盖关键闭环。

```
from copy import deepcopy
```

```
class MiniTrainingPlatformSystemDesignAudit:
    GATES = [
        "requirement_clarification",
```

```

        "trainingjob_contract",
        "lifecycle_state_machine",
        "admission_quota_policy",
        "scheduler_resource_fit",
        "distributed_launcher_fit",
        "data_config_lineage",
        "checkpoint_recovery_coupling",
        "observability_experiment_linkage",
        "fault_recovery_flow",
        "security_audit_governance",
        "cost_capacity_governance",
        "developer_self_service",
        "interface_artifact_contract",
        "tradeoff_boundary_reasoning",
        "training_platform_design_gate",
    ]

REQUIRED_JOB_FIELDS = [
    "owner", "project", "image_digest", "code_commit", "dataset_version",
    "final_config_hash", "resources", "distributed", "checkpoint_policy",
    "queue", "priority", "security_policy",
]

REQUIRED_STATES = [
    "submitted", "validated", "queued", "scheduled", "starting", "running",
    "checkpointing", "retrying", "failed", "completed", "cancelled",
]

REQUIRED_TRACE_LINKS = [
    "training_job", "final_config", "image_digest", "code_commit",
    "dataset_version", "checkpoint_manifest", "metrics", "logs",
    "eval_report", "model_artifact",
]

@staticmethod
def present(record, key):
    return key in record and record[key] is not None and record[key] != ""

def coverage(self, record, fields):
    if not record:
        return 0.0
    return sum(1 for field in fields if self.present(record, field)) / len(fields)

def world_size(self, case):
    resources = case["resources"]
    return resources["nodes"] * resources["gpus_per_node"]

def effective_batch(self, case):
    batch = case["batch"]
    return batch["micro_batch"] * batch["grad_accum"] * self.world_size(case)

def queue_p95_minutes(self, case):
    waits = sorted(case["queue"]["wait_minutes"])
    index = int(0.95 * (len(waits) - 1))
    return waits[index]

def max_lost_gpu_hours(self, case):

```

```

        ckpt = case["checkpoint"]
        return round(self.world_size(case) * ckpt["max_stale_minutes"] / 60, 3)

def trace_coverage(self, case):
    trace = case["observability"]
    return self.coverage(trace["links"], self.REQUIRED_TRACE_LINKS)

def cost_attribution_rate(self, case):
    cost = case["cost"]
    return cost["attributed_gpu_hours"] / max(1e-12, cost["total_gpu_hours"])

def requirement_clarification(self, case):
    req = case.get("requirements", {})
    required = ["train_types", "scale", "gpu_types", "multi_tenant", "preemption", "recovery_slo", "release_path"]
    return self.coverage(req, required) == 1.0

def trainingjob_contract(self, case):
    return self.coverage(case.get("training_job", {}), self.REQUIRED_JOB_FIELDS) == 1.0

def lifecycle_state_machine(self, case):
    lifecycle = case.get("lifecycle", {})
    states = lifecycle.get("states", [])
    return all(state in states for state in self.REQUIRED_STATES) and lifecycle.get("legal_transitions") is True

def admission_quota_policy(self, case):
    admission = case.get("admission", {})
    return (
        admission.get("schema_validation") is True
        and admission.get("quota_dry_run") is True
        and admission.get("dataset_permission_check") is True
        and admission.get("checkpoint_path_check") is True
        and admission.get("idempotency_key") is True
    )

def scheduler_resource_fit(self, case):
    scheduler = case.get("scheduler", {})
    return (
        scheduler.get("gang") is True
        and scheduler.get("topology_aware") is True
        and scheduler.get("fair_share") is True
        and scheduler.get("preemption_policy") is True
        and scheduler.get("fragmentation_control") is True
        and self.queue_p95_minutes(case) <= scheduler.get("max_queue_p95_min", 0)
    )

def distributed_launcher_fit(self, case):
    launcher = case.get("launcher", {})
    return (
        launcher.get("type") in {"torchrun", "deepspeed", "megatron", "ray", "mpi"}
        and launcher.get("world_size") == self.world_size(case)
        and launcher.get("rendezvous") is True
        and launcher.get("rank_assignment") is True
        and launcher.get("gpu_binding") is True
    )

```

```

def data_config_lineage(self, case):
    lineage = case.get("lineage", {})
    required = ["dataset_version", "manifest_hash", "code_commit", "image_digest", "final_config_hash", "seed", ]
    return self.coverage(lineage, required) == 1.0

def checkpoint_recovery_coupling(self, case):
    ckpt = case.get("checkpoint", {})
    return (
        ckpt.get("manifest_committed") is True
        and ckpt.get("checksum") is True
        and ckpt.get("restore_drill") is True
        and ckpt.get("preemption_hook") is True
        and self.max_lost_gpu_hours(case) <= ckpt.get("max_lost_gpu_hours", 0)
    )

def observability_experiment_linkage(self, case):
    obs = case.get("observability", {})
    return (
        self.trace_coverage(case) >= obs.get("min_trace_coverage", 1.0)
        and obs.get("rank_logs") is True
        and obs.get("metrics") is True
        and obs.get("events") is True
        and obs.get("experiment_run") is True
    )

def fault_recovery_flow(self, case):
    fault = case.get("fault_recovery", {})
    return (
        fault.get("taxonomy") is True
        and fault.get("retry_budget") is True
        and fault.get("backoff") is True
        and fault.get("fast_fail_user_error") is True
        and fault.get("diagnosis_report") is True
    )

def security_audit_governance(self, case):
    security = case.get("security", {})
    return (
        security.get("identity_binding") is True
        and security.get("rbac_abac") is True
        and security.get("secret_scope") is True
        and security.get("runtime_isolation") is True
        and security.get("audit_log") is True
    )

def cost_capacity_governance(self, case):
    cost = case.get("cost", {})
    return (
        self.cost_attribution_rate(case) >= cost.get("min_attribution_rate", 1.0)
        and cost.get("budget_alert") is True
        and cost.get("idle_tracking") is True
        and cost.get("capacity_plan") is True
    )

def developer_self_service(self, case):

```

```

dev = case.get("developer", {})
return (
    dev.get("portal") is True
    and dev.get("cli") is True
    and dev.get("sdk") is True
    and dev.get("templates") is True
    and dev.get("status_page") is True
    and dev.get("resume_from_checkpoint") is True
)

def interface_artifact_contract(self, case):
    contract = case.get("interfaces", {})
    return (
        contract.get("shared_schema") is True
        and contract.get("api_cli_sdk_parity") is True
        and contract.get("model_registry_link") is True
        and contract.get("eval_report_link") is True
        and contract.get("artifact_contract") is True
    )

def tradeoff_boundary_reasoning(self, case):
    tradeoff = case.get("tradeoff", {})
    required = ["utilization_wait", "checkpoint_cost", "preemption_ux", "isolation_sharing", "log_privacy", "retr
    return self.coverage(tradeoff, required) == 1.0

def training_platform_design_gate(self, case):
    gate = case.get("platform_gate", {})
    return (
        gate.get("enabled") is True
        and bool(gate.get("owner"))
        and gate.get("dashboard_ready") is True
        and gate.get("exception_approval") is True
        and gate.get("audit_export") is True
        and gate.get("p0_open") is False
    )

def audit_case(self, case):
    return {gate: getattr(self, gate)(case) for gate in self.GATES}

def run_all(self, cases):
    results = {case["case_id"]: self.audit_case(case) for case in cases}
    metrics = {}
    for gate in self.GATES:
        passed = sum(1 for result in results.values() if result[gate])
        metrics[gate] = round(passed / len(cases), 3)
    failed_cases = [case_id for case_id, result in results.items() if not all(result.values())]
    failed_gates = [gate for gate in self.GATES if any(not result[gate] for result in results.values())]
    return {
        "metrics": metrics,
        "hard_blocker_count": len(failed_cases),
        "failed_cases": failed_cases,
        "failed_gates": failed_gates,
        "training_platform_design_gate_pass": metrics["training_platform_design_gate"] == 1.0,
    }

```

```

def example_outputs(self, case):
    return {
        "world_size": self.world_size(case),
        "effective_batch": self.effective_batch(case),
        "queue_p95_minutes": self.queue_p95_minutes(case),
        "max_lost_gpu_hours": self.max_lost_gpu_hours(case),
        "trace_coverage": round(self.trace_coverage(case), 3),
        "cost_attribution_rate": round(self.cost_attribution_rate(case), 3),
    }

def build_good_case():
    return {
        "case_id": "full_training_platform_design",
        "requirements": {
            "train_types": ["pretrain", "sft", "eval"],
            "scale": "multi-node",
            "gpu_types": ["h100"],
            "multi_tenant": True,
            "preemption": True,
            "recovery_slo": "lost <= 16 GPU-hours",
            "release_path": "model registry + eval",
            "cost_budget": "team budget",
        },
        "training_job": {
            "owner": "team-a",
            "project": "chat-sft",
            "image_digest": "sha256:train",
            "code_commit": "abc123",
            "dataset_version": "sft-2026-05-30",
            "final_config_hash": "cfg789",
            "resources": "8x8 h100",
            "distributed": "deepspeed",
            "checkpoint_policy": "every 1000 steps",
            "queue": "high-priority",
            "priority": "p1",
            "security_policy": "restricted-data",
        },
        "resources": {"nodes": 8, "gpus_per_node": 8},
        "batch": {"micro_batch": 2, "grad_accum": 8},
        "queue": {"wait_minutes": [5, 8, 12, 18, 24, 32]},
        "lifecycle": {
            "states": [
                "submitted", "validated", "queued", "scheduled", "starting", "running",
                "checkpointing", "retrying", "failed", "completed", "cancelled",
            ],
            "legal_transitions": True,
        },
        "admission": {
            "schema_validation": True,
            "quota_dry_run": True,
            "dataset_permission_check": True,
            "checkpoint_path_check": True,
            "idempotency_key": True,
        },
    }

```

```

"scheduler": {
  "gang": True,
  "topology_aware": True,
  "fair_share": True,
  "preemption_policy": True,
  "fragmentation_control": True,
  "max_queue_p95_min": 30,
},
"launcher": {
  "type": "deepspeed",
  "world_size": 64,
  "rendezvous": True,
  "rank_assignment": True,
  "gpu_binding": True,
},
"lineage": {
  "dataset_version": "sft-2026-05-30",
  "manifest_hash": "data456",
  "code_commit": "abc123",
  "image_digest": "sha256:train",
  "final_config_hash": "cfg789",
  "seed": 42,
  "tokenizer_version": "tok-v3",
},
"checkpoint": {
  "manifest_committed": True,
  "checksum": True,
  "restore_drill": True,
  "preemption_hook": True,
  "max_stale_minutes": 12,
  "max_lost_gpu_hours": 16,
},
"observability": {
  "links": {
    "training_job": True,
    "final_config": True,
    "image_digest": True,
    "code_commit": True,
    "dataset_version": True,
    "checkpoint_manifest": True,
    "metrics": True,
    "logs": True,
    "eval_report": True,
    "model_artifact": True,
  },
  "min_trace_coverage": 0.95,
  "rank_logs": True,
  "metrics": True,
  "events": True,
  "experiment_run": True,
},
"fault_recovery": {
  "taxonomy": True,
  "retry_budget": True,
  "backoff": True,
}

```

```

        "fast_fail_user_error": True,
        "diagnosis_report": True,
    },
    "security": {
        "identity_binding": True,
        "rbac_abac": True,
        "secret_scope": True,
        "runtime_isolation": True,
        "audit_log": True,
    },
    "cost": {
        "total_gpu_hours": 1000,
        "attributed_gpu_hours": 960,
        "min_attribution_rate": 0.95,
        "budget_alert": True,
        "idle_tracking": True,
        "capacity_plan": True,
    },
    "developer": {
        "portal": True,
        "cli": True,
        "sdk": True,
        "templates": True,
        "status_page": True,
        "resume_from_checkpoint": True,
    },
    "interfaces": {
        "shared_schema": True,
        "api_cli_sdk_parity": True,
        "model_registry_link": True,
        "eval_report_link": True,
        "artifact_contract": True,
    },
    "tradeoff": {
        "utilization_wait": True,
        "checkpoint_cost": True,
        "preemption_ux": True,
        "isolation_sharing": True,
        "log_privacy": True,
        "retry_masking": True,
    },
    "platform_gate": {
        "enabled": True,
        "owner": "training-platform",
        "dashboard_ready": True,
        "exception_approval": True,
        "audit_export": True,
        "p0_open": False,
    },
}

```

```

def build_bad_cases(good_case):
    cases = []
    changes = [

```



```

("requirement_scope_missing_bad", ("requirements", "recovery_slo"), None),
("trainingjob_contract_missing_bad", ("training_job", "final_config_hash"), None),
("state_machine_invalid_bad", ("lifecycle", "legal_transitions"), False),
("admission_quota_policy_missing_bad", ("admission", "quota_dry_run"), False),
("scheduler_gang_topology_missing_bad", ("scheduler", "gang"), False),
("launcher_world_size_mismatch_bad", ("launcher", "world_size"), 32),
("data_config_lineage_gap_bad", ("lineage", "dataset_version"), None),
("checkpoint_recovery_uncoupled_bad", ("checkpoint", "restore_drill"), False),
("observability_experiment_unlinked_bad", ("observability", "experiment_run"), False),
("fault_recovery_blind_retry_bad", ("fault_recovery", "fast_fail_user_error"), False),
("security_audit_gap_bad", ("security", "audit_log"), False),
("cost_capacity_unowned_bad", ("cost", "attributed_gpu_hours"), 600),
("developer_self_service_gap_bad", ("developer", "status_page"), False),
("interface_artifact_contract_gap_bad", ("interfaces", "artifact_contract"), False),
("tradeoff_boundary_missing_bad", ("tradeoff", "retry_masking"), None),
("design_gate_missing_bad", ("platform_gate", "enabled"), False),
]
for case_id, path, value in changes:
    case = deepcopy(good_case)
    case["case_id"] = case_id
    cursor = case
    for key in path[:-1]:
        cursor = cursor[key]
    cursor[path[-1]] = value
    cases.append(case)
return cases

```

```

audit = MiniTrainingPlatformSystemDesignAudit()
good = build_good_case()
cases = [good] + build_bad_cases(good)
summary = audit.run_all(cases)

```

```

print("training_platform_design_examples=" + repr(audit.example_outputs(good)))
print("metrics=" + repr(summary["metrics"]))
print("hard_blocker_count=" + repr(summary["hard_blocker_count"]))
print("failed_cases=" + repr(summary["failed_cases"]))
print("failed_gates=" + repr(summary["failed_gates"]))
print("training_platform_design_gate_pass=" + repr(summary["training_platform_design_gate_pass"]))

```

参考输出应类似：

```

training_platform_design_examples={'world_size': 64, 'effective_batch': 1024, 'queue_p95_minutes': 24, 'max_lost_gpu_ho
metrics={'requirement_clarification': 0.941, 'trainingjob_contract': 0.941, 'lifecycle_state_machine': 0.941, 'admissio
hard_blocker_count=16
failed_cases=['requirement_scope_missing_bad', 'trainingjob_contract_missing_bad', 'state_machine_invalid_bad', 'admis
failed_gates=['requirement_clarification', 'trainingjob_contract', 'lifecycle_state_machine', 'admission_quota_policy'
training_platform_design_gate_pass=False

```

这个 demo 的面试价值是把“设计训练平台”从架构图转成可验证回答：需求澄清要完整，TrainingJob 契约要冻结，状态机要可解释，准入和调度要有门禁，launcher 要和资源一致，数据 / 配置 / checkpoint / experiment / artifact 要能追踪，安全、成本、自助和 trade-off 要能被审计。

52.23 关键 trade-off

系统设计中要讲 trade-off:

1. 资源利用率 vs 调度等待时间。
2. Checkpoint 频率 vs 存储和 I/O 成本。
3. 抢占能力 vs 用户体验。
4. 平台封装 vs 用户灵活性。
5. 多租户隔离 vs 资源共享效率。
6. 日志完整性 vs 隐私和成本。
7. 自动重试 vs 隐藏用户错误。

能讲 trade-off，说明你理解生产系统。

52.24 常见故障和应对

平台要支持常见故障定位：

1. 调度等待太久：看队列、配额、资源碎片。
2. 任务启动失败：看镜像、权限、entrypoint。
3. GPU 利用率低：看 data loader、通信、batch。
4. Loss NaN：看数据、学习率、precision、gradient。
5. OOM：看 batch、sequence、ZeRO/FSDP。
6. Hang：看 rank logs、NCCL、dataloader。
7. Checkpoint 慢：看存储吞吐和保存策略。

这些能力要体现在观测和 runbook 中。

52.25 面试回答模板

可以这样回答：

我会把大模型训练平台设计成 TrainingJob 生命周期管理系统。

用户通过 Web、CLI、SDK 或 API 提交 TrainingJob，配置镜像、代码版本、数据版本、资源、launcher、checkpoint 和重试策略。API

校验通过后进入队列，Scheduler 根据 GPU 类型、队列优先级、配额、gang scheduling、拓扑和资源碎片做调度。Launcher Control

运行中平台采集日志、metrics、events 和 checkpoint 信息，并自动写入 experiment tracking。Checkpoint 支持保留策略、校验和

训练完成后，平台将候选 checkpoint 注册到模型仓库，触发评估，并关联 eval report。平台还提供多租户权限、审计、成本归因、这个答案覆盖了提交、调度、启动、运行、观测、容错、产物和治理。

52.26 常见扣分点

扣分点一：只说 Kubernetes Job。

问题：忽略 TrainingJob 抽象、分布式启动、checkpoint、数据版本和实验追踪。

扣分点二：不提 gang scheduling。

问题：多机多卡训练需要同时拿到资源，否则无法启动。

扣分点三：不提 checkpoint 和恢复。

问题：大模型训练时间长，没有容错会浪费大量 GPU 成本。

扣分点四：不提多租户和配额。

问题：内部平台一定会面对公平性和资源治理。

扣分点五：不提可观测性。

问题：训练故障复杂，没有日志、指标、事件和 trace 无法运维。

52.27 小练习

1. TrainingJob 和普通 K8s Job 有什么区别?
2. 大模型训练平台为什么需要 gang scheduling?
3. TrainingJob schema 应包含哪些字段?
4. 如何设计 checkpoint policy?
5. 如何区分可重试错误和不可重试错误?
6. TrainingJob 详情页应该展示哪些信息?
7. 如何把训练平台和模型仓库联动?
8. 如何为多租户训练平台设计队列和配额?

52.28 本章小结

本章系统设计了一个大模型训练平台。

你需要记住：

1. 大模型训练平台的核心对象是 TrainingJob。
2. 平台主链路是提交、校验、排队、调度、分布式启动、运行、观测、checkpoint、恢复、评估和模型注册。
3. 多机多卡训练需要 gang scheduling、拓扑感知和 launcher 抽象。
4. 生产训练平台必须支持数据版本、代码版本、实验追踪、checkpoint、容错、多租户、安全和成本治理。
5. 面试时要讲清楚模块、链路、状态机和 trade-off，而不是只列技术名词。

下一章我们会继续系统设计题：设计一个高并发大模型推理平台。

第 53 章 设计一个高并发大模型推理平台

上一章设计了大模型训练平台。本章继续系统设计题：设计一个高并发大模型推理平台。

这是 AI Infra 面试中最常见的开放题之一。面试官想看你是否理解在线服务、大模型 runtime、KV cache、路由、缓存、限流、扩缩容、灰度发布、可观测性和成本治理之间的关系。

先记住一句话：

高并发大模型推理平台不是一个模型 API 服务，而是围绕低延迟、高吞吐、高可用、低成本和多模型治理构建的在线服务平台。

53.0 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做过资料校准。重点参考的是 Google SRE 对 SLI / SLO、长尾延迟、错误预算和发布风险控制的系统设计口径，OpenTelemetry 对 traces、metrics、logs 和事件关联的观测抽象，Triton Inference Server 对 request、queue、compute、batch 和 GPU 指标的推理服务指标口径，vLLM 对 PagedAttention、KV cache、OpenAI-compatible serving、observability 和 metrics 的公开文档，以及前文第 26 到第 35 章已经校准过的 runtime、prefill / decode、KV cache、continuous batching、模型路由、缓存、扩缩容、保护策略和发布治理口径。

这些资料共同指向一个稳定结论：高并发大模型推理平台不是把模型封装成 HTTP API，也不是把所有逻辑塞进 runtime，而是用平台控制面把需求澄清、SLO、请求链路、模型版本、路由策略、KV 容量、调度、缓存隔离、扩缩容、限流保护、发布回滚、多租户安全、观测、成本和审计统一管理。

本章只抽象截至 2026-06 仍稳定的大模型推理平台系统设计方法，不把某个 runtime、云厂商、Kubernetes 发行版、service mesh、模型仓库或内部平台写成通用标准。正文公式用于面试表达、容量估算和系统设计自查；真实上线仍要用目标模型、硬件、runtime、流量分布、SLO、租户协议、合规要求和历史事故复盘来校准。

53.1 题目理解

题目可能这样问：

1. 请设计一个支持高并发的 ChatGPT 类推理平台。

2. 请设计一个多模型、多租户的大模型服务平台。
3. 请设计一个支持 streaming、自动扩缩容和灰度发布的 LLM serving 系统。
4. 请设计一个企业内部统一模型推理网关。

核心都是：如何把大模型稳定、高效、可控地服务线上请求。

53.2 需求澄清

先问清楚：

1. 支持哪些模型？chat、embedding、多模态、rerank？
2. 模型规模多大？7B、70B、MoE？
3. QPS 和并发目标是多少？
4. 输入和输出 token 分布是什么？
5. TTFT、TPOT、p99 SLO 是多少？
6. 是否支持 streaming？
7. 是否多租户？是否需要配额和权限？
8. 是否支持多模型路由？
9. 是否需要缓存？
10. 是否需要灰度、A/B 测试和回滚？
11. 是否有安全策略和审计要求？
12. 成本预算是什么？

推理平台设计必须从 token 分布和 SLO 开始，而不是从 QPS 开始。

53.3 核心目标

平台目标：

1. 支持多模型、多版本、多 runtime。
2. 支持高并发和 streaming。
3. 控制 TTFT、TPOT 和 p99。
4. 提高 GPU 利用率和 tokens/s。
5. 支持模型路由、灰度和回滚。
6. 支持缓存、限流、熔断和降级。
7. 支持自动扩缩容和 warm pool。
8. 支持多租户权限、配额和审计。
9. 支持可观测性和故障定位。
10. 支持成本归因和成本优化。

高并发不只是“能抗 QPS”，还要抗 token 负载和长上下文。

53.4 总体架构

可以设计如下：

Client / App / Agent

- > API Gateway
- > Auth / Quota / Rate Limit
- > Safety Input Check
- > Model Router
- > Cache Layer
- > Admission Controller
- > Inference Runtime Cluster
 - > vLLM / TGI / Triton / SGLang / TensorRT-LLM
 - > Runtime Scheduler
 - > Continuous Batching
 - > KV Cache Manager

- > Model Registry
- > Deployment Controller
- > Autoscaler
- > Observability
- > Cost / Audit

架构要体现两层：平台治理层和 runtime 执行层。

平台治理层负责鉴权、路由、限流、缓存、发布、观测和成本。

Runtime 执行层负责模型加载、prefill、decode、batching 和 KV cache。

53.5 API Gateway

API Gateway 负责统一入口。

能力包括：

1. API 认证。
2. 协议适配。
3. 请求大小限制。
4. 基础限流。
5. request ID 注入。
6. streaming 连接管理。
7. 超时控制。
8. 初步日志和审计。

Gateway 不应该做复杂模型选择。

复杂路由应交给 Model Router。

53.6 多租户鉴权和配额

多租户推理平台要控制：

1. 谁能调用哪个模型。
2. 每个租户 QPS 上限。
3. 每个租户 token/s 上限。
4. 每个租户并发上限。
5. 每个租户成本预算。
6. 最大上下文长度。
7. 最大输出长度。

LLM 平台限流不能只按 QPS。

一个长上下文请求可能比几十个短请求更贵。

53.7 Model Router

Model Router 决定请求去哪。

决策依据：

1. 请求指定模型。
2. 任务能力。
3. 租户权限。
4. 成本预算。
5. 延迟 SLO。
6. 灰度规则。
7. endpoint 健康状态。
8. queue length。

9. KV cache 水位。
10. fallback chain。

Router 输出 route decision, 并记录 route trace。

路由不是负载均衡, 它还要选模型、版本和降级路径。

53.8 Cache Layer

缓存层可以包括:

1. prompt cache。
2. KV cache。
3. semantic cache。
4. result cache。

缓存收益:

1. 降低 TTFT。
2. 减少 prefill 成本。
3. 降低重复请求成本。
4. 提升用户体验。

缓存风险:

1. 权限误命中。
2. 过期答案。
3. 模型版本不一致。
4. 安全策略不一致。

cache key 必须包含模型版本、参数、租户、权限域和数据版本。

53.9 Admission Controller

Admission Controller 判断系统当前是否接得住请求。

检查:

1. 队列长度。
2. queue wait time。
3. KV cache 剩余空间。
4. endpoint 健康。
5. 当前 p99。
6. 租户配额。
7. 请求 token 数。
8. timeout budget。

如果接不住, 可以:

1. 排队。
2. 降级。
3. 转发到其他 endpoint。
4. 返回限流。

及时拒绝比无限排队更健康。

53.10 Runtime Cluster

Runtime Cluster 负责真正执行推理。

可选 runtime:

1. vLLM。
2. TGI。
3. Triton。
4. SGLang。
5. TensorRT-LLM。
6. 自研 runtime。

Runtime 负责：

1. 模型加载。
2. tokenizer。
3. prefill。
4. decode。
5. continuous batching。
6. KV cache。
7. streaming。
8. runtime metrics。

平台层不要把所有逻辑塞进 runtime。

53.11 Prefill、Decode 和 KV Cache

系统设计中一定要讲：

1. Prefill 处理输入 prompt，影响 TTFT。
2. Decode 逐 token 生成，影响 TPOT。
3. KV cache 避免重复计算，但占显存。
4. 长上下文和高并发会放大 KV cache 压力。
5. 调度不能只看请求数，要看 token budget。

这是大模型推理平台区别于普通 Web 服务的核心。

53.12 Continuous Batching

Continuous batching 让请求在 decode step 动态加入和退出。

收益：

1. 提高 GPU 利用率。
2. 提高 output tokens/s。
3. 更好处理不同输出长度。
4. 降低短请求被长请求拖住的问题。

代价：

1. 调度复杂。
2. KV cache 管理复杂。
3. 指标和 trace 更复杂。

高并发推理平台通常离不开 continuous batching。

53.13 队列调度

Runtime queue 不能简单 FIFO。

要考虑：

1. token budget。
2. 输入长度。
3. 预计输出长度。
4. 优先级。

5. 租户公平性。
6. prefill/decode 平衡。
7. 长短请求分队列。

长请求和短请求混跑会导致 p99 变差。

可以对长上下文请求使用专门队列或专门实例。

53.14 自动扩缩容

Autoscaler 不能只看 CPU 或 QPS。

应看：

1. QPS。
2. input tokens/s。
3. output tokens/s。
4. queue length。
5. queue wait time。
6. TTFT。
7. TPOT。
8. p99。
9. KV cache usage。
10. GPU utilization。
11. timeout rate。

还要考虑冷启动：镜像、权重、runtime 初始化和 kernel 预热。

因此需要 warm pool 和预测扩容。

53.15 稳定性保护

稳定性机制包括：

1. 多维限流。
2. admission control。
3. 熔断。
4. timeout budget。
5. 谨慎重试。
6. 降级路由。
7. fallback chain。
8. endpoint health check。

重试要非常谨慎。

如果请求已经开始生成或已经返回部分 token，重试会导致成本增加和体验不一致。

53.16 灰度发布和回滚

推理平台要支持：

1. 模型版本灰度。
2. prompt 灰度。
3. runtime 灰度。
4. 路由策略灰度。
5. 安全策略灰度。
6. 一键回滚。

观察指标：

1. 质量。

2. 安全。
3. TTFT。
4. TPOT。
5. p99。
6. 错误率。
7. 成本。
8. 工具调用成功率。

模型发布不能只看 HTTP 错误率。

53.17 可观测性设计

必须记录 metrics、logs、traces、events。

核心 metrics：

1. QPS。
2. TTFT。
3. TPOT。
4. p95/p99。
5. input/output tokens/s。
6. queue length。
7. active sequences。
8. KV cache usage。
9. GPU utilization。
10. cache hit rate。
11. fallback rate。
12. cost per 1k tokens。

Trace 要拆分 gateway、auth、router、cache、queue、prefill、decode、safety 和 streaming。

53.18 安全和审计

安全设计包括：

1. API key 和身份认证。
2. 模型访问权限。
3. 租户配额。
4. 输入输出安全检查。
5. 日志脱敏。
6. trace 权限控制。
7. RAG 权限过滤。
8. tool call 权限控制。
9. 审计日志。

推理请求可能包含用户隐私和企业知识，不能把 prompt 和输出无保护地长期保存。

53.19 成本治理

成本治理包括：

1. 按租户统计 input/output tokens。
2. 按模型统计 cost per 1k tokens。
3. 记录 cache saved cost。
4. 小模型优先。
5. 分层路由。
6. 输出长度控制。
7. 量化。

8. autoscaling。
9. warm pool 优化。
10. 预算和告警。

高并发推理平台如果没有成本治理，很快会不可持续。

53.20 高可用设计

高可用手段：

1. 多副本 runtime。
2. 多 endpoint。
3. health check。
4. 熔断和摘除。
5. fallback model。
6. 多可用区部署。
7. 灰度和回滚。
8. 自动扩缩容。
9. 降级策略。

多地域部署要考虑数据合规、模型权重分发和成本。

53.21 状态和数据模型

核心对象包括：

1. Model。
2. ModelVersion。
3. Endpoint。
4. Deployment。
5. RoutePolicy。
6. QuotaPolicy。
7. CachePolicy。
8. ScalingPolicy。
9. SafetyPolicy。
10. RuntimeInstance。

这些对象让平台从“部署服务”升级为“治理模型服务”。

53.22 推理平台系统设计指标和最小 demo

面试中讲高并发推理平台，最好把“组件架构”收束成几个可计算指标。一个推理平台系统设计样本可以抽象为：

$D_i = (q_i, a_i, l_i, r_i, k_i, c_i, s_i, p_i, g_i, o_i, u_i, h_i, z_i)$

其中， q_i 是需求和流量画像， a_i 是入口、鉴权和配额， l_i 是请求生命周期， r_i 是模型路由和发布， k_i 是 runtime、prefill / decode 和 KV cache， c_i 是缓存策略， s_i 是准入和调度， p_i 是限流、熔断、超时和降级， g_i 是多租户安全治理， o_i 是观测链路， u_i 是成本容量， h_i 是高可用和回滚， z_i 是未关闭风险。

某个设计维度的覆盖率可以写成：

$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(D_i)=1]$

其中， g_j 是第 j 个检查函数，例如是否给出 token 分布、是否区分 prefill / decode、是否估算 KV 容量、是否给出 trace 和成本归因。这个公式的直觉是：系统设计不是“讲到了一个词”，而是每个关键维度都能被样本化检查。

端到端延迟可以拆成：

$$T_{\{\mathrm{e2e}\}} = T_{\{\mathrm{gateway}\}} + T_{\{\mathrm{auth}\}} + T_{\{\mathrm{route}\}} + T_{\{\mathrm{cache}\}} + T_{\{\mathrm{queue}\}} + T_{\{\mathrm{prefill}\}} + T_{\{\mathrm{decode}\}} + T_{\{\mathrm{safety}\}} + T_{\{\mathrm{stream}\}}$$

TTFT 主要到首 token 生成前为止：

$$T_{\{\mathrm{ttft}\}} = T_{\{\mathrm{gateway}\}} + T_{\{\mathrm{auth}\}} + T_{\{\mathrm{route}\}} + T_{\{\mathrm{cache}\}} + T_{\{\mathrm{queue}\}} + T_{\{\mathrm{prefill}\}}$$

Decode 阶段可以用输出 token 数近似：

$$T_{\{\mathrm{decode}\}} \approx y_i T_{\{\mathrm{tpot}\}}$$

其中， y_i 是第 i 个请求输出 token 数， $T_{\{\mathrm{tpot}\}}$ 是每个输出 token 的平均或分位延迟。这个近似提醒你：长输出会把总延迟拉长，即使 TTFT 看起来很好，p99 也可能失败。

token 负载要拆成输入和输出：

$$\begin{aligned} \Lambda_{\{\mathrm{in}\}} &= \Lambda_{\{\mathrm{req}\}} \mathbb{E}[x_i], \\ \Lambda_{\{\mathrm{out}\}} &= \Lambda_{\{\mathrm{req}\}} \mathbb{E}[y_i], \\ \Lambda_{\{\mathrm{tok}\}} &= \Lambda_{\{\mathrm{in}\}} + \Lambda_{\{\mathrm{out}\}} \end{aligned}$$

其中， $\Lambda_{\{\mathrm{req}\}}$ 是请求到达率， x_i 是输入 token 数， y_i 是输出 token 数。推理平台按 QPS 扩容会漏掉长上下文和长输出的真实负载。

KV cache 单请求显存可以粗略估算为：

$$M_{\{\mathrm{kv}, \mathrm{req}\}} = 2LH_{\{\mathrm{kv}\}}D_hB_{\{\mathrm{elem}\}}S_{\{\mathrm{ctx}\}}$$

其中， L 是层数， $H_{\{\mathrm{kv}\}}$ 是 KV head 数， D_h 是 head dimension， $B_{\{\mathrm{elem}\}}$ 是每个元素字节数， $S_{\{\mathrm{ctx}\}}$ 是输入加已生成输出的上下文 token 数。前面的 2 表示 key 和 value 两份缓存。

KV 容量门禁可以写成：

$$G_{\{\mathrm{kv}\}} = \mathbf{1} \left[N_{\{\mathrm{active}\}} M_{\{\mathrm{kv}, \mathrm{req}\}} \leq \eta_{\{\mathrm{kv}\}} M_{\{\mathrm{gpu}\}} \right]$$

其中， $N_{\{\mathrm{active}\}}$ 是活跃请求数， $M_{\{\mathrm{gpu}\}}$ 是可用显存， $\eta_{\{\mathrm{kv}\}}$ 是留给 KV 的比例。真实平台还要扣除权重、workspace、fragmentation、runtime overhead 和不同请求长度造成的 block waste。

多指标副本需求可以用最大值收束：

$$N_{\{\mathrm{need}\}} = \max \left(\left\lceil \frac{\Lambda_{\{\mathrm{req}\}}}{\mu_{\{\mathrm{req}\}}} \right\rceil, \left\lceil \frac{\Lambda_{\{\mathrm{in}\}}}{\mu_{\{\mathrm{in}\}}} \right\rceil, \left\lceil \frac{\Lambda_{\{\mathrm{out}\}}}{\mu_{\{\mathrm{out}\}}} \right\rceil, \left\lceil \frac{N_{\{\mathrm{kv}\}}}{N_{\{\mathrm{kv}, \mathrm{one}\}}} \right\rceil \right)$$

其中, μ_{req} 、 μ_{in} 、 μ_{out} 分别是单实例请求、输入 token 和输出 token 能力, N_{kv} 是需要承载的 KV 并发, $N_{\text{kv,one}}$ 是单实例 KV 并发上限。这个公式比 “QPS 除以单机 QPS” 更接近 LLM serving 的真实容量规划。

最终系统设计门禁可以写成:

$$G_{\text{design}} = \mathbf{1} \left[\begin{array}{l} \min_j C_j \geq \tau_j \quad \text{and} \\ T_{\text{ttft},p95} \leq B_{\text{ttft}} \quad \text{and} \\ T_{\text{tpot},p95} \leq B_{\text{tpot}} \quad \text{and} \\ G_{\text{kv}} = 1 \quad \text{and} \\ R_{\text{trace}} \geq \rho \quad \text{and} \\ P_0 = 0 \end{array} \right]$$

其中, C_j 是各设计维度覆盖率, R_{trace} 是请求链路 trace 覆盖率, P_0 是未关闭 P0 风险。这个门禁强调: 系统设计不是组件列表, 而是可验收的工程闭环。

下面这个 0 依赖 Python demo 演示一个 “推理平台系统设计答案审计器”: 输入 toy design answers, 检查是否覆盖需求、请求链路、SLO、模型注册、路由、runtime、KV、缓存、扩缩容、保护、发布、多租户、观测、成本和 trade-off, 并给出容量估算样例。

```
from math import ceil
```

```
class MiniInferencePlatformSystemDesignAudit:
```

```
    CHECKS = [
        "requirement_clarification",
        "request_lifecycle_coverage",
        "slo_token_contract",
        "model_registry_release",
        "router_policy_fit",
        "runtime_prefill_decode",
        "kv_scheduler_capacity",
        "cache_isolation_strategy",
        "autoscaling_capacity_plan",
        "protection_policy_design",
        "release_governance_design",
        "tenant_security_governance",
        "observability_trace_readiness",
        "cost_capacity_governance",
        "tradeoff_boundary_reasoning",
        "inference_platform_design_gate",
    ]
```

```
    def __init__(self):
        self.traffic = {
            "peak_qps": 42,
            "avg_input_tokens": 1400,
            "avg_output_tokens": 180,
            "p95_input_tokens": 6000,
            "p95_output_tokens": 640,
            "active_sequences": 96,
            "ttft_budget_s": 1.8,
            "tpot_budget_ms": 45,
            "p99_budget_s": 12.0,
        }
        self.instance = {
```

```

        "req_capacity": 7,
        "input_tps": 9000,
        "output_tps": 900,
        "kv_concurrency": 18,
        "gpu_memory_gib": 80,
        "kv_memory_fraction": 0.30,
    }
    self.model = {
        "layers": 40,
        "kv_heads": 8,
        "head_dim": 128,
        "bytes_per_element": 2,
        "context_tokens": self.traffic["p95_input_tokens"] + self.traffic["p95_output_tokens"],
    }
    self.latency = {
        "gateway_s": 0.03,
        "auth_s": 0.02,
        "route_s": 0.04,
        "cache_s": 0.03,
        "queue_s": 0.45,
        "prefill_s": 0.78,
        "tpot_ms": 34,
        "safety_s": 0.08,
        "stream_s": 0.15,
    }

def kv_per_request_mib(self):
    total_bytes = (
        2
        * self.model["layers"]
        * self.model["kv_heads"]
        * self.model["head_dim"]
        * self.model["bytes_per_element"]
        * self.model["context_tokens"]
    )
    return round(total_bytes / (1024 ** 2), 1)

def latency_summary(self):
    decode_s = self.traffic["p95_output_tokens"] * self.latency["tpot_ms"] / 1000
    ttft_s = (
        self.latency["gateway_s"]
        + self.latency["auth_s"]
        + self.latency["route_s"]
        + self.latency["cache_s"]
        + self.latency["queue_s"]
        + self.latency["prefill_s"]
    )
    e2e_s = ttft_s + decode_s + self.latency["safety_s"] + self.latency["stream_s"]
    return {"ttft_s": round(ttft_s, 3), "decode_s": round(decode_s, 3), "e2e_s": round(e2e_s, 3)}

def capacity_plan(self):
    input_tps = self.traffic["peak_qps"] * self.traffic["avg_input_tokens"]
    output_tps = self.traffic["peak_qps"] * self.traffic["avg_output_tokens"]
    by_req = ceil(self.traffic["peak_qps"] / self.instance["req_capacity"])
    by_input = ceil(input_tps / self.instance["input_tps"])

```

```

by_output = ceil(output_tps / self.instance["output_tps"])
by_kv = ceil(self.traffic["active_sequences"] / self.instance["kv_concurrency"])
return {
    "input_tps": input_tps,
    "output_tps": output_tps,
    "by_req": by_req,
    "by_input": by_input,
    "by_output": by_output,
    "by_kv": by_kv,
    "replicas_needed": max(by_req, by_input, by_output, by_kv),
}

def kv_gate(self, active_sequences_per_replica):
    kv_mib = self.kv_per_request_mib()
    available_mib = self.instance["gpu_memory_gib"] * 1024 * self.instance["kv_memory_fraction"]
    used_mib = kv_mib * active_sequences_per_replica
    return {
        "kv_per_request_mib": kv_mib,
        "available_kv_mib": round(available_mib, 1),
        "used_kv_mib": round(used_mib, 1),
        "kv_gate_pass": used_mib <= available_mib,
    }

def build_design_cases(self):
    complete = {"name": "complete_inference_platform_design"}
    complete.update({check: True for check in self.CHECKS})
    cases = [complete]
    bad_names = [
        "requirements_missing_bad",
        "request_lifecycle_missing_bad",
        "slo_token_contract_missing_bad",
        "model_registry_release_missing_bad",
        "router_policy_missing_bad",
        "runtime_phase_missing_bad",
        "kv_scheduler_missing_bad",
        "cache_isolation_missing_bad",
        "autoscaling_single_metric_bad",
        "protection_policy_missing_bad",
        "release_governance_missing_bad",
        "tenant_security_missing_bad",
        "observability_trace_missing_bad",
        "cost_capacity_missing_bad",
        "tradeoff_boundary_missing_bad",
        "design_gate_missing_bad",
    ]
    for bad_name, failed_check in zip(bad_names, self.CHECKS):
        case = {"name": bad_name}
        case.update({check: True for check in self.CHECKS})
        case[failed_check] = False
        cases.append(case)
    return cases

def audit_cases(self, cases):
    metrics = {
        check: round(sum(1 for case in cases if case.get(check)) / len(cases), 3)

```

```

        for check in self.CHECKS
    }
    failed_cases = [case["name"] for case in cases if not all(case.get(check) for check in self.CHECKS)]
    failed_gates = [check for check, value in metrics.items() if value < 1.0]
    hard_blockers = sum(1 for case in cases if case["name"].endswith("_bad"))
    gate_pass = min(metrics.values()) >= 0.95 and hard_blockers == 0
    return metrics, failed_cases, failed_gates, hard_blockers, gate_pass

def run(self):
    latency = self.latency_summary()
    capacity = self.capacity_plan()
    active_per_replica = ceil(self.traffic["active_sequences"] / capacity["replicas_needed"])
    kv = self.kv_gate(active_sequences_per_replica=active_per_replica)
    metrics, failed_cases, failed_gates, hard_blockers, gate_pass = self.audit_cases(
        self.build_design_cases()
    )
    slo_pass = {
        "ttft": latency["ttft_s"] <= self.traffic["ttft_budget_s"],
        "tpot": self.latency["tpot_ms"] <= self.traffic["tpot_budget_ms"],
        "p99": latency["e2e_s"] <= self.traffic["p99_budget_s"],
    }
    return {
        "latency_summary": latency,
        "capacity_plan": capacity,
        "kv_gate": kv,
        "slo_pass": slo_pass,
        "metrics": metrics,
        "hard_blocker_count": hard_blockers,
        "failed_case_count": len(failed_cases),
        "failed_gate_count": len(failed_gates),
        "failed_case_sample": failed_cases[:4],
        "failed_gate_sample": failed_gates[:4],
        "inference_platform_design_gate_pass": gate_pass,
    }

```

```

report = MiniInferencePlatformSystemDesignAudit().run()
print("latency_summary=", report["latency_summary"])
print("capacity_plan=", report["capacity_plan"])
print("kv_gate=", report["kv_gate"])
print("slo_pass=", report["slo_pass"])
print("metrics=", report["metrics"])
print("hard_blocker_count=", report["hard_blocker_count"])
print("failed_case_count=", report["failed_case_count"])
print("failed_gate_count=", report["failed_gate_count"])
print("failed_case_sample=", report["failed_case_sample"])
print("failed_gate_sample=", report["failed_gate_sample"])
print("inference_platform_design_gate_pass=", report["inference_platform_design_gate_pass"])

```

输出示例:

```

latency_summary= {'ttft_s': 1.35, 'decode_s': 21.76, 'e2e_s': 23.34}
capacity_plan= {'input_tps': 58800, 'output_tps': 7560, 'by_req': 6, 'by_input': 7, 'by_output': 9, 'by_kv': 6, 'replicas': 1}
kv_gate= {'kv_per_request_mib': 1037.5, 'available_kv_mib': 24576.0, 'used_kv_mib': 11412.5, 'kv_gate_pass': True}
slo_pass= {'ttft': True, 'tpot': True, 'p99': False}
metrics= {'requirement_clarification': 0.941, 'request_lifecycle_coverage': 0.941, 'slo_token_contract': 0.941, 'model_load_time': 0.941, 'model_size': 0.941, 'model_version': 0.941, 'model_weight': 0.941, 'model_weight_size': 0.941, 'model_weight_size_bytes': 0.941, 'model_weight_size_gb': 0.941, 'model_weight_size_mb': 0.941, 'model_weight_size_tb': 0.941, 'model_weight_size_pb': 0.941, 'model_weight_size_yb': 0.941, 'model_weight_size_zb': 0.941, 'model_weight_size_db': 0.941, 'model_weight_size_qb': 0.941, 'model_weight_size_sb': 0.941, 'model_weight_size_hb': 0.941, 'model_weight_size_kb': 0.941, 'model_weight_size_b': 0.941, 'model_weight_size_yb': 0.941, 'model_weight_size_zb': 0.941, 'model_weight_size_db': 0.941, 'model_weight_size_qb': 0.941, 'model_weight_size_sb': 0.941, 'model_weight_size_hb': 0.941, 'model_weight_size_kb': 0.941, 'model_weight_size_b': 0.941}

```

```
hard_blocker_count= 16
failed_case_count= 16
failed_gate_count= 16
failed_case_sample= ['requirements_missing_bad', 'request_lifecycle_missing_bad', 'slo_token_contract_missing_bad', 'm
failed_gate_sample= ['requirement_clarification', 'request_lifecycle_coverage', 'slo_token_contract', 'model_registry_
inference_platform_design_gate_pass= False
```

这个 demo 的重点不是给生产容量结论，而是训练面试中的自查习惯：如果答案没有 token 分布、TTFT/TPOT、KV 容量、端到端 trace、扩缩容、保护策略、发布治理和成本归因，就不是一个完整的推理平台系统设计答案。示例里 TTFT 和 TPOT 过线，但 p99 不过线，说明可以继续追问长输出、decode 并发、输出长度限制、流式体验、speculative decoding 或能力路由，而不是只说“加机器”。

53.23 关键 trade-off

需要讲清楚：

1. 吞吐 vs 延迟。
2. GPU 利用率 vs p99。
3. 缓存收益 vs 一致性和安全。
4. 小模型成本低 vs 质量可能下降。
5. warm pool 延迟低 vs 空闲成本高。
6. 灰度速度 vs 发布风险。
7. 完整 trace vs 隐私和成本。
8. 自动重试 vs 重试风暴。

系统设计题中，trade-off 比组件名更重要。

53.24 面试回答模板

可以这样回答：

我会先澄清模型规模、QPS、token 分布、TTFT/TPOT/p99 SLO、是否 streaming、多租户、安全和成本要求。

整体架构分为平台治理层和 runtime 执行层。请求先进入 API Gateway，做认证、限流和 request ID 注入，再经过安全检查和 Model

执行层使用 vLLM/TGI/Triton/SGLang 等 runtime，支持 prefill、decode、continuous batching、KV cache 和 streaming。调度上按

扩缩容不只看 QPS，而结合 input/output tokens/s、queue wait、TTFT、TPOT、KV cache、GPU 和 timeout。稳定性上做限流、熔断、这段回答覆盖需求、架构、runtime、调度、稳定性、发布和治理。

53.25 常见扣分点

扣分点一：只说 Nginx + 模型服务。

问题：忽略模型路由、runtime、KV cache、batching 和治理。

扣分点二：只按 QPS 扩容。

问题：不理解 token 负载和长上下文差异。

扣分点三：不提 TTFT/TPOT。

问题：无法解释用户体验和推理阶段瓶颈。

扣分点四：不提缓存权限。

问题：企业场景中缓存误命中可能导致数据泄露。

扣分点五：发布只看错误率。

问题：模型发布还要看质量、安全、成本和工具调用行为。

53.26 小练习

1. 高并发大模型推理平台为什么不能只按 QPS 设计?
2. Model Router 和普通负载均衡有什么区别?
3. Admission Controller 应检查哪些指标?
4. TTFT 和 TPOT 分别如何优化?
5. Continuous batching 为什么适合生成式模型?
6. 自动扩缩容为什么要看 KV cache?
7. 推理平台如何设计 fallback chain?
8. 如何设计推理平台的成本 dashboard?

53.27 本章小结

本章系统设计了一个高并发大模型推理平台。

你需要记住：

1. 推理平台是在线模型服务平台，不只是模型 HTTP API。
2. 高并发设计要围绕 TTFT、TPOT、tokens/s、KV cache、queue 和 p99 展开。
3. 平台治理层负责鉴权、路由、缓存、限流、发布、观测和成本，runtime 层负责 prefill、decode、batching 和 KV cache。
4. 自动扩缩容、稳定性保护、灰度回滚和成本治理是生产推理平台的核心能力。
5. 面试时要讲清请求链路、核心对象、关键指标和 trade-off。

下一章我们会设计一个企业级 LLMOps 平台。

第 54 章 设计一个企业级 LLMOps 平台

上一章设计了高并发大模型推理平台。本章继续系统设计题：设计一个企业级 LLMOps 平台。

LLMOps 平台关注的是大模型应用从开发、评估、发布、观测、治理到迭代的完整生命周期。它不是单纯训练平台，也不是单纯推理平台，而是把模型、prompt、RAG、Agent、工具、安全、评估、发布和成本整合起来的企业级平台。

先记住一句话：

企业级 LLMOps 平台的核心，是让大模型应用可开发、可评估、可发布、可观测、可治理、可审计、可持续优化。

54.0 本讲资料边界与第二轮精修口径

本讲按 WRITING_PLAN.md 的第二轮要求做过资料校准。重点参考的是 OpenTelemetry 对生成式 AI trace / metric 语义、span 属性和系统观测信号的公开口径，MLflow 对 GenAI 评估、trace、run、metric、artifact 和 prompt / eval 配置管理的公开口径，OpenAI Evals 对可复用评估样例、评估逻辑和回归验证的工程边界，以及前文模型网关、推理平台、RAG / Agent 存储、评估平台、可观测性、安全治理、成本治理和审计变更治理章节已经校准过的口径。

这些资料共同指向一个稳定结论：企业级 LLMOps 平台不是“把模型 API 包一层控制台”，而是把大模型应用的模型路由、prompt、知识库、工具、Agent、评估、发布、trace、反馈、安全、成本和审计放进同一套应用生命周期控制面。

本章只抽象截至 2026-06 仍稳定的企业级 LLMOps 系统设计方法，不把某个 SaaS、SDK、模型供应商、评估框架、向量数据库或内部平台实现写成通用标准。正文公式用于面试表达、系统设计自查和 toy demo 审计；真实落地仍要结合企业身份体系、数据分级、模型供应商协议、业务流程、合规要求、可用性 SLO 和历史事故复盘校准。

54.1 题目理解

面试题可能这样问：

1. 请设计一个企业内部 LLMOps 平台。
2. 请设计一个支持 prompt、RAG、Agent、评估和发布的大模型应用平台。
3. 请设计一个多团队共享的大模型应用生命周期管理平台。
4. 请设计一个统一管理模型调用、知识库、工具和 trace 的平台。

这类题重点不是模型训练，而是大模型应用落地和治理。

54.2 LLMOps 和 MLOps 的区别

MLOps 更关注传统机器学习生命周期：

1. 数据。
2. 特征。
3. 训练。
4. 部署。
5. 监控。

LLMOps 额外关注：

1. Prompt 版本。
2. RAG 知识库。
3. Agent 工具调用。
4. 模型路由。
5. 输出安全。
6. 用户反馈。
7. LLM-as-Judge。
8. Trace 回放。
9. Token 成本。
10. 多模型能力治理。

LLMOps 更偏 “大模型应用生命周期管理”。

54.3 需求澄清

设计前先问：

1. 平台服务哪些用户？算法、应用开发、业务团队、运维？
2. 是否接入多个基础模型和外部模型？
3. 是否支持 prompt 管理？
4. 是否支持 RAG 知识库？
5. 是否支持 Agent 和工具？
6. 是否支持评估和 A/B 测试？
7. 是否需要多租户隔离？
8. 是否有安全、合规和审计要求？
9. 是否需要成本预算和计量？
10. 是否对外提供 API/SDK？

企业级平台必须明确用户角色和治理要求。

54.4 核心目标

企业级 LLMOps 平台目标：

1. 统一模型接入和调用。
2. 管理 prompt 版本和发布。
3. 管理知识库和 RAG 链路。

4. 管理 Agent 和工具定义。
5. 支持评估、实验和对比。
6. 支持灰度、A/B 测试和回滚。
7. 支持 trace、日志和可观测性。
8. 支持权限、安全和审计。
9. 支持 token 成本和预算治理。
10. 提供 Web Console、API、SDK 和 CLI。

LLMOps 平台本质上是企业大模型应用的控制平面。

54.5 总体架构

可以设计如下架构：

Users / Apps / Agents
-> LLMops API Gateway
-> Auth / Tenant / Quota
-> Model Gateway / Router
-> Prompt Registry
-> Knowledge Base / RAG Service
-> Tool Registry / Agent Runtime
-> Eval Platform
-> Release / Experiment System
-> Trace / Observability
-> Cost / Audit / Governance
-> Model Registry / Inference Platform

底层依赖推理平台、模型仓库、向量索引、评估平台和可观测性平台。

LLMOps 把这些能力包装成应用开发者能使用的产品。

54.6 Model Gateway

Model Gateway 是统一模型调用入口。

能力包括：

1. 多模型接入。
2. OpenAI-compatible API。
3. 模型路由。
4. 租户鉴权。
5. 限流配额。
6. 统一日志和 trace。
7. token 计量。
8. fallback 和降级。

它让应用不直接绑定某个模型供应商或某个内部 endpoint。

54.7 Prompt Registry

Prompt Registry 管理 prompt 模板。

它应支持：

1. prompt 创建。
2. prompt 版本。
3. 变量 schema。
4. prompt 预览。
5. 测试样例。

6. 评估结果。
7. 发布状态。
8. 灰度和回滚。
9. 权限控制。
10. 审计记录。

Prompt 是大模型应用的重要资产。

不要把 prompt 当成代码里的硬编码字符串。

54.8 Prompt 发布流程

Prompt 发布可以类似模型发布：

draft -> reviewed -> evaluated -> staged -> canary -> production -> deprecated

发布前检查：

1. 变量是否完整。
2. 是否通过格式测试。
3. 是否通过安全测试。
4. 是否通过评估集。
5. 是否影响工具调用。
6. 是否有回滚版本。

Prompt 修改可能导致输出格式、拒答率和工具调用行为变化，因此必须可灰度和回滚。

54.9 Knowledge Base 管理

RAG 知识库管理包括：

1. 知识库创建。
2. 文档导入。
3. 文档解析。
4. chunking。
5. embedding。
6. 向量索引。
7. 权限同步。
8. 索引版本。
9. 检索配置。
10. citation。

企业知识库必须支持权限过滤。

不能让用户检索到自己无权访问的文档。

54.10 RAG Pipeline

RAG 请求链路：

```
user query
-> query rewrite
-> retrieval
-> metadata permission filter
-> rerank
-> prompt assembly
-> model generation
-> citation
-> trace
```

平台要记录 retrieval trace 和 prompt assembly trace。
否则无法解释为什么模型引用了某些文档，或者为什么没有召回正确知识。

54.11 Tool Registry

Tool Registry 管理可供 Agent 调用的工具。

Tool Definition 包括：

1. tool name。
2. version。
3. description。
4. input schema。
5. output schema。
6. endpoint。
7. timeout。
8. retry policy。
9. permission requirement。
10. side effect level。

工具描述会直接影响模型调用行为，因此工具也需要版本化、评估和灰度。

54.12 Agent Runtime

Agent Runtime 负责任务执行。

它需要支持：

1. agent definition。
2. 多步执行。
3. 工具调用。
4. memory。
5. run state。
6. 超时。
7. step limit。
8. human confirmation。
9. execution trace。
10. error handling。

Agent 不是一次模型调用，而是一个多步状态机。

平台必须限制无限循环和高风险工具调用。

54.13 权限和安全

LLMOps 平台安全包括：

1. 模型访问权限。
2. prompt 修改权限。
3. 知识库访问权限。
4. 工具调用权限。
5. trace 查看权限。
6. secret 管理。
7. 输入输出安全检查。
8. 审计日志。

特别注意：应用开发者不一定能访问所有模型、所有知识库、所有工具。

权限要跟租户、项目、环境和数据等级绑定。

54.14 Eval Platform 集成

LLMOps 平台必须集成评估。

评估对象包括：

1. 模型版本。
2. prompt 版本。
3. RAG 配置。
4. 知识库版本。
5. 工具定义版本。
6. Agent 配置。

评估方式：

1. 离线评估。
2. LLM-as-Judge。
3. 人工评测。
4. 在线 A/B 测试。
5. 回归测试。

没有评估，LLMOps 只能靠感觉发布。

54.15 Release System

发布系统管理应用版本。

一个 LLM 应用版本可能包括：

1. model route policy。
2. prompt version。
3. RAG config。
4. knowledge base version。
5. tool versions。
6. safety policy。
7. generation config。

这些组合应该形成 release manifest。

发布支持：

1. 灰度。
2. A/B 测试。
3. 回滚。
4. 审批。
5. 变更记录。

54.16 Trace 和可观测性

LLMOps trace 应覆盖：

1. model call trace。
2. prompt assembly trace。
3. retrieval trace。
4. rerank trace。
5. tool call trace。
6. agent execution trace。
7. safety events。
8. cost trace。

核心指标：

1. latency。
2. TTFT。
3. TPOT。
4. token usage。
5. cost。
6. cache hit rate。
7. retrieval hit rate。
8. tool success rate。
9. user feedback。
10. safety violation rate。

大模型应用没有 trace 就很难 debug。

54.17 用户反馈闭环

企业 LLMOps 平台要支持反馈收集。

反馈类型：

1. 点赞点踩。
2. 用户评论。
3. 人工接管。
4. 任务成功失败。
5. 引用是否有用。
6. 工具调用是否正确。
7. 安全投诉。

反馈用于：

1. 评估数据构建。
2. 回归集更新。
3. prompt 优化。
4. RAG 调优。
5. 模型选择。
6. 安全规则改进。

反馈要和 trace 关联。

54.18 成本治理

LLMOps 成本治理关注：

1. token usage。
2. cost per app。
3. cost per tenant。
4. cost per user。
5. cost per model。
6. RAG 检索成本。
7. rerank 成本。
8. tool API 成本。
9. agent run 成本。

Agent 特别容易成本失控，因为它会多轮调用模型和工具。

平台要支持 budget、quota、step limit、max tokens 和 cost alert。

54.19 多环境管理

企业平台通常有多个环境：

1. dev。
2. staging。
3. production。

LLM 应用在不同环境中应有独立配置：

1. 模型版本。
2. prompt 版本。
3. 知识库版本。
4. 工具 endpoint。
5. 安全策略。
6. 成本配额。

从 dev 到 production 应通过发布流程，而不是手工复制配置。

54.20 Web Console 设计

Console 功能可以包括：

1. 应用列表。
2. 模型网关配置。
3. Prompt 管理。
4. 知识库管理。
5. 工具管理。
6. Agent 配置。
7. 评估任务。
8. 发布和灰度。
9. Trace 查看。
10. 成本和配额。
11. 权限和审计。

不同角色看到的页面不同。

应用开发者关注调试和发布，平台管理员关注安全、成本和稳定性。

54.21 API / SDK / CLI

平台应提供统一接口。

SDK 示例能力：

1. 调用模型。
2. 获取 prompt version。
3. 查询知识库。
4. 调用 agent。
5. 上传反馈。
6. 获取 trace ID。

CLI 示例：

```
llmops prompt publish customer-bot --version v12
llmops kb sync product-docs
llmops eval run customer-bot --suite regression
llmops release canary customer-bot --percent 5
llmops trace get <trace-id>
```

API、SDK、CLI 和 Console 应基于同一套资源模型。

54.22 核心数据模型

核心对象：

1. Application。
2. Environment。
3. PromptTemplate。
4. KnowledgeBase。
5. ToolDefinition。
6. AgentDefinition。
7. EvalSuite。
8. ReleaseManifest。
9. Trace。
10. Feedback。
11. Policy。
12. CostRecord。

这些对象构成 LLMOps 平台的控制平面。

54.23 LLMOps 平台系统设计指标和最小 demo

企业级 LLMOps 平台的系统设计，最好从“资源对象 + 生命周期门禁”讲起。一个应用样本可以抽象为：

$$A_i = (m_i, p_i, k_i, t_i, g_i, e_i, r_i, o_i, f_i, c_i, s_i, u_i, z_i)$$

其中， m_i 是模型网关和路由， p_i 是 prompt 版本， k_i 是知识库和 RAG 配置， t_i 是工具与 Agent runtime， g_i 是治理策略， e_i 是评估， r_i 是 release manifest， o_i 是 trace 和观测， f_i 是用户反馈， c_i 是成本预算， s_i 是安全审计， u_i 是开发者接口和用户体验， z_i 是未关闭风险。

第 j 个设计维度覆盖率可以写成：

$$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(A_i) = 1]$$

其中， g_j 是检查函数，例如是否有 prompt 版本、知识库权限、工具权限、评估回归、发布回滚、trace、成本归因和审计。这个公式的直觉是：LLMOps 不是一个功能列表，而是一组应用生命周期对象是否都可版本化、可验证、可回滚、可审计。

一次应用发布可以用 release manifest 表示：

$$M_{\{\mathrm{rel}\}} = (v_{\{\mathrm{model}\}}, v_{\{\mathrm{prompt}\}}, v_{\{\mathrm{kb}\}}, v_{\{\mathrm{tool}\}}, v_{\{\mathrm{agent}\}}, v_{\{\mathrm{gen}\}})$$

其中， $v_{\{\mathrm{model}\}}$ 是模型或路由策略版本， $v_{\{\mathrm{prompt}\}}$ 是 prompt 版本， $v_{\{\mathrm{kb}\}}$ 是知识库版本， $v_{\{\mathrm{tool}\}}$ 是工具版本集合， $v_{\{\mathrm{agent}\}}$ 是 Agent 配置版本， $v_{\{\mathrm{safety}\}}$ 是安全策略版本， $\theta_{\{\mathrm{gen}\}}$ 是生成参数。Manifest 不完整时，即使模型输出变差，也很难回放“到底变了什么”。

Manifest 完整率可以写成：

$$C_{\{\mathrm{manifest}\}} = \frac{N_{\{\mathrm{filled}\}}}{N_{\{\mathrm{required}\}}}$$

Prompt 变量覆盖率可以写成：

$$C_{\{\mathrm{prompt}\}} = \frac{|V_{\{\mathrm{provided}\}} \cap V_{\{\mathrm{required}\}}|}{|V_{\{\mathrm{required}\}}|}$$

其中， $V_{\{\mathrm{required}\}}$ 是 prompt 模板声明的变量集合， $V_{\{\mathrm{provided}\}}$ 是发布或调用时实际提供的变量集合。这个指标能阻断“模板改了但应用代码没改”的隐性事故。

RAG 权限阻断率可以写成：

$$B_{\{\mathrm{rag}\}} = \frac{N_{\{\mathrm{blocked, unauth}\}}}{N_{\{\mathrm{unauth}\}}}$$

其中， $N_{\{\mathrm{unauth}\}}$ 是无权访问但被检索候选召回的文档数， $N_{\{\mathrm{blocked, unauth}\}}$ 是最终被权限过滤阻断的文档数。企业 RAG 的核心不是召回越多越好，而是正确内容和正确权限同时成立。

Agent 高风险工具确认覆盖率可以写成：

$$C_{\{\mathrm{confirm}\}} = \frac{N_{\{\mathrm{confirmed, high}\}}}{N_{\{\mathrm{high}\}}}$$

其中, N_{high} 是高风险工具调用次数, $N_{\mathrm{confirmed,high}}$ 是经过人审、二次确认或审批策略放行的次数。这个指标提醒你: Agent 自动化能力越强, 越要把副作用和权限边界显式化。

Trace 覆盖率可以写成:

$$R_{\mathrm{trace}} = \frac{N_{\mathrm{trace,complete}}}{N_{\mathrm{runs}}}$$

完整 trace 至少要覆盖 model call、prompt assembly、retrieval、rerank、tool call、agent step、safety event、feedback 和 cost record。否则线上 bad case 很难复现, 也无法沉淀成回归集。

反馈行动率可以写成:

$$C_{\mathrm{feedback}} = \frac{N_{\mathrm{actioned}}}{N_{\mathrm{feedback}}}$$

其中, N_{actioned} 是已经进入回归集、prompt 修复、知识库更新、工具策略修复或安全规则更新的反馈数。只收集点赞点踩但不进入行动闭环, 不算真正的 LLMOps。

一次 Agent / RAG 应用运行成本可以拆成:

$$K_{\mathrm{run}} = K_{\mathrm{model}} + K_{\mathrm{retrieval}} + K_{\mathrm{rerank}} + K_{\mathrm{tool}} + K_{\mathrm{review}}$$

企业平台最终要能按应用、租户、用户、模型、知识库和工具做成本归因, 而不是只看统一账单。

评估发布门禁可以写成:

$$G_{\mathrm{eval}} = \mathbf{1} \setminus [Q_{\mathrm{offline}} \geq \tau_q \wedge S_{\mathrm{safety}} \geq \tau_s \wedge R_{\mathrm{reg}}$$

其中, Q_{offline} 是离线质量指标, S_{safety} 是安全评估通过率, R_{reg} 是回归集通过率。Prompt、RAG 配置和工具 schema 的变更都应该走类似门禁。

最终 LLMOps 平台门禁可以写成:

$$G_{\mathrm{llmops}} = \mathbf{1} \setminus [\min_j C_j \geq \tau_j \wedge C_{\mathrm{manifest}} = 1 \wedge B_{\mathrm{rag}} \geq \rho_{\mathrm{rag}} \wedge C_{\mathrm{confirm}} \geq \rho_{\mathrm{tool}} \wedge R_{\mathrm{trace}} \geq \rho_{\mathrm{trace}} \wedge G_{\mathrm{eval}} = 1 \wedge P_0 = 0]$$

其中, P_0 是未关闭 P0 风险。这个门禁强调: 企业级 LLMOps 的答案不能只讲 “有 prompt 管理和知识库管理”, 还要证明版本、权限、评估、发布、trace、反馈、成本和安全的闭环。

下面这个 0 依赖 Python demo 演示一个 “企业级 LLMOps 平台系统设计审计器”: 输入 toy applications、release manifest、RAG 候选、工具调用、trace、反馈和成本记录, 输出生命周期、成本和设计门禁结果。

```
class MiniEnterpriseLLMopsPlatformAudit:
    CHECKS = [
        "lifecycle_boundary_clarity",
        "application_resource_model",
        "model_gateway_policy",
        "prompt_registry_versioning",
        "rag_kb_permission_governance",
        "tool_agent_runtime_control",
        "eval_feedback_loop",
        "release_manifest_governance",
        "trace_observability_readiness",
        "cost_budget_governance",
        "security_audit_policy",
        "multi_environment_promotion",
        "developer_interface_readiness",
        "production_monitoring_guardrail",
```

```

        "tradeoff_boundary_reasoning",
        "llmops_platform_gate",
    ]

REQUIRED_MANIFEST_FIELDS = [
    "model_route_version",
    "prompt_version",
    "knowledge_base_version",
    "tool_versions",
    "agent_version",
    "safety_policy_version",
    "generation_config",
    "rollback_target",
]

def __init__(self):
    self.applications = [
        {"name": "customer_bot", "environment": "production", "eval_ready": True, "release_ready": True},
        {"name": "ops_copilot", "environment": "production", "eval_ready": True, "release_ready": True},
        {"name": "hr_agent", "environment": "staging", "eval_ready": False, "release_ready": False},
    ]
    self.release_manifest = {
        "model_route_version": "route-v7",
        "prompt_version": "prompt-v12",
        "knowledge_base_version": "kb-2026-06-01",
        "tool_versions": ["ticket-v3", "refund-v2"],
        "agent_version": "agent-v5",
        "safety_policy_version": "safety-v9",
        "generation_config": {"temperature": 0.2, "max_tokens": 800},
        "rollback_target": "release-v41",
    }
    self.prompt = {
        "required_variables": {"customer_tier", "issue_summary", "locale"},
        "provided_variables": {"customer_tier", "issue_summary", "locale"},
    }
    self.retrieval_candidates = [
        {"doc": "public_faq", "authorized": True, "blocked": False},
        {"doc": "enterprise_contract", "authorized": True, "blocked": False},
        {"doc": "hr_private_policy", "authorized": False, "blocked": True},
        {"doc": "finance_private_note", "authorized": False, "blocked": True},
    ]
    self.tool_calls = [
        {"tool": "search_docs", "high_risk": False, "confirmed": True},
        {"tool": "create_ticket", "high_risk": False, "confirmed": True},
        {"tool": "issue_refund", "high_risk": True, "confirmed": True},
        {"tool": "send_customer_email", "high_risk": True, "confirmed": True},
    ]
    self.trace_runs = [
        {"id": "r1", "complete": True},
        {"id": "r2", "complete": True},
        {"id": "r3", "complete": True},
        {"id": "r4", "complete": False},
        {"id": "r5", "complete": True},
        {"id": "r6", "complete": True},
        {"id": "r7", "complete": True},
    ]

```

```

        {"id": "r8", "complete": True},
        {"id": "r9", "complete": False},
    ]
    self.feedback = {"total": 64, "actioned": 48}
    self.cost_records = [
        {"app": "customer_bot", "tokens": 240000, "cost": 9.9, "success": 160, "budget": 12.0},
        {"app": "ops_copilot", "tokens": 170000, "cost": 3.1, "success": 80, "budget": 5.0},
        {"app": "hr_agent", "tokens": 50000, "cost": 2.0, "success": 20, "budget": 2.0},
    ]

def lifecycle_summary(self):
    total = len(self.applications)
    production = sum(1 for app in self.applications if app["environment"] == "production")
    eval_ready = sum(1 for app in self.applications if app["eval_ready"])
    release_ready = sum(1 for app in self.applications if app["release_ready"])
    return {
        "apps": total,
        "production_apps": production,
        "eval_ready_rate": round(eval_ready / total, 3),
        "release_ready_rate": round(release_ready / total, 3),
    }

def release_manifest_completeness(self):
    filled = sum(1 for field in self.REQUIRED_MANIFEST_FIELDS if self.release_manifest.get(field))
    return round(filled / len(self.REQUIRED_MANIFEST_FIELDS), 3)

def prompt_variable_coverage(self):
    required = self.prompt["required_variables"]
    provided = self.prompt["provided_variables"]
    return round(len(required & provided) / len(required), 3)

def rag_permission_block_rate(self):
    unauthorized = [item for item in self.retrieval_candidates if not item["authorized"]]
    blocked = [item for item in unauthorized if item["blocked"]]
    return round(len(blocked) / len(unauthorized), 3)

def tool_confirmation_coverage(self):
    high_risk = [call for call in self.tool_calls if call["high_risk"]]
    confirmed = [call for call in high_risk if call["confirmed"]]
    return round(len(confirmed) / len(high_risk), 3)

def trace_coverage(self):
    complete = sum(1 for run in self.trace_runs if run["complete"])
    return round(complete / len(self.trace_runs), 3)

def feedback_action_rate(self):
    return round(self.feedback["actioned"] / self.feedback["total"], 3)

def cost_summary(self):
    total_tokens = sum(item["tokens"] for item in self.cost_records)
    total_cost = round(sum(item["cost"] for item in self.cost_records), 2)
    total_success = sum(item["success"] for item in self.cost_records)
    budget_pass = all(item["cost"] <= item["budget"] for item in self.cost_records)
    return {
        "total_tokens": total_tokens,

```

```

        "total_cost": total_cost,
        "cost_per_success": round(total_cost / total_success, 3),
        "budget_pass": budget_pass,
    }

def llmops_examples(self):
    return {
        "release_manifest_completeness": self.release_manifest_completeness(),
        "prompt_variable_coverage": self.prompt_variable_coverage(),
        "rag_permission_block_rate": self.rag_permission_block_rate(),
        "tool_confirmation_coverage": self.tool_confirmation_coverage(),
        "trace_coverage": self.trace_coverage(),
        "feedback_action_rate": self.feedback_action_rate(),
    }

def build_design_cases(self):
    complete = {"name": "complete_enterprise_llmops_platform"}
    complete.update({check: True for check in self.CHECKS})
    cases = [complete]
    bad_names = [
        "lifecycle_boundary_missing_bad",
        "resource_model_missing_bad",
        "model_gateway_policy_missing_bad",
        "prompt_registry_missing_bad",
        "rag_permission_missing_bad",
        "agent_tool_control_missing_bad",
        "eval_feedback_loop_missing_bad",
        "release_manifest_missing_bad",
        "trace_observability_missing_bad",
        "cost_budget_missing_bad",
        "security_audit_missing_bad",
        "environment_promotion_manual_bad",
        "developer_interface_missing_bad",
        "production_guardrail_missing_bad",
        "tradeoff_boundary_missing_bad",
        "llmops_gate_missing_bad",
    ]
    for bad_name, failed_check in zip(bad_names, self.CHECKS):
        case = {"name": bad_name}
        case.update({check: True for check in self.CHECKS})
        case[failed_check] = False
        cases.append(case)
    return cases

def audit_cases(self, cases):
    metrics = {
        check: round(sum(1 for case in cases if case.get(check)) / len(cases), 3)
        for check in self.CHECKS
    }
    failed_cases = [case["name"] for case in cases if not all(case.get(check) for check in self.CHECKS)]
    failed_gates = [check for check, value in metrics.items() if value < 1.0]
    hard_blockers = sum(1 for case in cases if case["name"].endswith("_bad"))
    gate_pass = min(metrics.values()) >= 0.95 and hard_blockers == 0
    return metrics, failed_cases, failed_gates, hard_blockers, gate_pass

```

```

def run(self):
    metrics, failed_cases, failed_gates, hard_blockers, gate_pass = self.audit_cases(
        self.build_design_cases()
    )
    return {
        "lifecycle_summary": self.lifecycle_summary(),
        "llmops_examples": self.llmops_examples(),
        "cost_summary": self.cost_summary(),
        "metrics": metrics,
        "hard_blocker_count": hard_blockers,
        "failed_case_count": len(failed_cases),
        "failed_gate_count": len(failed_gates),
        "failed_case_sample": failed_cases[:4],
        "failed_gate_sample": failed_gates[:4],
        "llmops_platform_gate_pass": gate_pass,
    }

```

```

report = MiniEnterpriseLLMOpsPlatformAudit().run()
print("lifecycle_summary=", report["lifecycle_summary"])
print("llmops_examples=", report["llmops_examples"])
print("cost_summary=", report["cost_summary"])
print("metrics=", report["metrics"])
print("hard_blocker_count=", report["hard_blocker_count"])
print("failed_case_count=", report["failed_case_count"])
print("failed_gate_count=", report["failed_gate_count"])
print("failed_case_sample=", report["failed_case_sample"])
print("failed_gate_sample=", report["failed_gate_sample"])
print("llmops_platform_gate_pass=", report["llmops_platform_gate_pass"])

```

输出示例:

```

lifecycle_summary= {'apps': 3, 'production_apps': 2, 'eval_ready_rate': 0.667, 'release_ready_rate': 0.667}
llmops_examples= {'release_manifest_completeness': 1.0, 'prompt_variable_coverage': 1.0, 'rag_permission_block_rate': 1.0}
cost_summary= {'total_tokens': 460000, 'total_cost': 15.0, 'cost_per_success': 0.058, 'budget_pass': True}
metrics= {'lifecycle_boundary_clarity': 0.941, 'application_resource_model': 0.941, 'model_gateway_policy': 0.941, 'prompt_registry': 0.941}
hard_blocker_count= 16
failed_case_count= 16
failed_gate_count= 16
failed_case_sample= ['lifecycle_boundary_missing_bad', 'resource_model_missing_bad', 'model_gateway_policy_missing_bad', 'prompt_registry_missing_bad']
failed_gate_sample= ['lifecycle_boundary_clarity', 'application_resource_model', 'model_gateway_policy', 'prompt_registry']
llmops_platform_gate_pass= False

```

这个 demo 的重点不是生产系统容量结论，而是把面试答案变成可审计清单：一个企业级 LLMOps 平台如果不能说明应用资源模型、release manifest、prompt / KB / tool / agent 版本、评估回归、trace、反馈、权限、成本和发布回滚，就只是一个功能入口，不是可靠的平台控制面。

54.24 关键 trade-off

LLMOps 平台 trade-off:

1. 平台统一治理 vs 团队灵活性。
2. Trace 完整性 vs 隐私和成本。
3. Prompt 快速迭代 vs 发布风险。
4. RAG 召回更多内容 vs 权限和噪声风险。
5. Agent 自动化能力 vs 工具副作用风险。
6. 多模型路由优化成本 vs 输出一致性。

7. 强审批流程 vs 研发效率。

企业平台的难点往往不是技术组件，而是治理边界和协作流程。

54.25 面试回答模板

可以这样回答：

我会把企业级 LLMOps 平台设计成大模型应用生命周期管理平台，覆盖模型调用、prompt、RAG、Agent、评估、发布、观测、安全和成本。

架构上包括 Model Gateway、Prompt Registry、Knowledge Base/RAG Service、Tool Registry、Agent Runtime、Eval Platform、Release Management。

应用开发者通过 Web Console、API、SDK 或 CLI 管理 prompt、知识库、工具和 agent 配置。每次发布生成 release manifest，包含模型版本、工具版本、agent 配置、评估结果、发布时间和发布人。

运行中记录 model call、retrieval、prompt assembly、tool call 和 agent execution trace，并采集 token、latency、cost、feedback。

这个回答体现了企业级 LLMOps 的完整边界。

54.26 常见扣分点

扣分点一：只说模型部署。

LLMOps 不只是部署模型，还包括 prompt、RAG、Agent、评估、发布和治理。

扣分点二：不提 prompt 版本。

Prompt 是应用行为的重要组成，必须版本化和灰度。

扣分点三：不提知识库权限。

企业 RAG 的核心风险是越权检索。

扣分点四：不提 trace。

没有 trace，RAG/Agent 应用无法调试和审计。

扣分点五：不提成本。

企业平台需要 token 计量、预算、配额和成本归因。

54.27 小练习

1. LLMOps 和 MLOps 的核心区别是什么？
2. Prompt Registry 应该支持哪些能力？
3. 一个 LLM 应用 release manifest 应包含哪些内容？
4. 企业 RAG 平台如何做权限控制？
5. Agent Runtime 为什么需要 step limit 和 human confirmation？
6. LLMOps trace 应覆盖哪些阶段？
7. 用户反馈如何进入模型和应用迭代闭环？
8. 如何设计 LLMOps 平台的成本治理？

54.28 本章小结

本章系统设计了一个企业级 LLMOps 平台。

你需要记住：

1. LLMOps 关注大模型应用生命周期，而不只是模型训练或部署。
2. 企业级 LLMOps 平台要管理模型调用、prompt、RAG、Agent、工具、评估、发布、trace、安全和成本。
3. Prompt、知识库、工具和 Agent 配置都要版本化、评估、灰度和回滚。
4. RAG/Agent 场景中，trace、权限和审计是核心能力。
5. 企业平台要在研发效率、治理、安全、成本和灵活性之间做平衡。

下一章我们会设计一个多租户 GPU 集群调度系统。

第 55 章 设计一个多租户 GPU 集群调度系统

上一章设计了企业级 LLM Ops 平台。本章继续系统设计题：设计一个多租户 GPU 集群调度系统。

GPU 集群调度是 AI Infra 的核心难题之一。大模型训练需要多机多卡、推理需要低延迟和弹性、评估和数据处理也会抢资源。多租户共享集群时，还要处理公平性、优先级、配额、抢占、碎片和成本。

先记住一句话：

多租户 GPU 调度系统的核心，是在有限 GPU 资源下，同时满足公平性、优先级、拓扑约束、任务完整性、利用率和 SLO。

55.0 本讲资料边界与第二轮精修口径

第二轮精修时，本章按“稳定调度抽象 + 主流开源实现口径”校准，不绑定某个公司内部平台或某个 Kubernetes 发行版。

参考口径包括：

1. Kubernetes 的 ResourceQuota、Device Plugin / Extended Resource、PriorityClass 和 Pod preemption，用来校准资源声明、命名空间配额、设备暴露和优先级抢占边界。
2. Kueue 的 ClusterQueue、LocalQueue、cohort 借用 / 出借、preemption 和 topology-aware scheduling，用来校准队列、配额借用、准入控制和拓扑感知调度边界。
3. Volcano 的 Queue、PodGroup、gang scheduling、priority 和 preemption，用来校准批训练任务整体调度、队列治理和抢占口径。
4. Dominant Resource Fairness 的 dominant share 思想，用来解释为什么多租户公平不能只看 GPU 数，还要同时看 CPU、内存、网络、存储和稀缺 GPU 型号。

本章的目标不是教你背某个调度器参数，而是把面试题回答成可验证的系统设计：任务资源画像清楚，队列和配额可解释，借用可回收，gang scheduling 能防止半启动，拓扑放置能减少通信损耗，抢占和 checkpoint 有耦合，低优先级资源可回收，推理服务有 SLO 隔离，异构 GPU 有成本策略，调度事件能解释“为什么还没跑”。

55.1 题目理解

面试题可能这样问：

1. 请设计一个公司内部多租户 GPU 集群调度系统。
2. 请设计一个支持大模型训练和推理的 GPU 调度平台。
3. 如何设计支持队列、配额、抢占和 gang scheduling 的调度系统？
4. 如何提高 GPU 集群利用率并保证公平性？

这类题考的是资源调度和平台治理能力，不只是 Kubernetes 使用经验。

55.2 需求澄清

先问：

1. 集群规模多大？多少节点、多少 GPU？
2. GPU 是否异构？A100、H100、L40、国产加速器？
3. 任务类型有哪些？训练、推理、评估、embedding、数据处理？
4. 是否多机多卡？是否需要 gang scheduling？
5. 是否多租户？租户之间如何隔离？
6. 是否有队列和优先级？
7. 是否允许抢占？
8. 是否需要拓扑感知？
9. 是否支持低优先级队列？

10. 目标是低等待时间、高利用率还是公平性？

不同目标会导向不同调度策略。

55.3 核心目标

GPU 调度系统目标：

1. 支持多租户队列和配额。
2. 支持不同 GPU 类型。
3. 支持训练 gang scheduling。
4. 支持拓扑感知调度。
5. 支持优先级和抢占。
6. 支持低优先级任务填充空闲资源。
7. 减少资源碎片。
8. 提高 GPU 有效利用率。
9. 支持公平性和成本归因。
10. 支持可观测性和审计。

系统设计要同时讲调度算法、资源模型和治理机制。

55.4 总体架构

可以设计如下架构：

Job Submission

- > Auth / Quota Check
- > Queue Manager
- > Scheduler
 - > Resource Snapshot
 - > Policy Engine
 - > Placement Engine
 - > Preemption Engine
- > Kubernetes / Cluster Manager
- > Node / GPU Runtime
- > Observability / Cost / Audit

核心模块：

1. Queue Manager：管理队列、优先级、等待任务。
2. Resource Snapshot：维护集群资源视图。
3. Policy Engine：处理配额、公平性、优先级。
4. Placement Engine：做具体放置决策。
5. Preemption Engine：处理抢占。
6. Observability：提供调度事件和资源指标。

55.5 资源模型

资源模型要表示：

1. GPU 类型。
2. GPU 数量。
3. GPU 显存。
4. CPU。
5. 内存。
6. 本地 NVMe。
7. 网络拓扑。
8. 节点标签。

9. 可用区。
10. 设备健康状态。

大模型训练不能只看 GPU 数量。

同样 8 张 GPU，如果跨 NUMA、跨节点或网络慢，性能差别很大。

55.6 任务模型

任务要声明：

1. job type。
2. tenant。
3. queue。
4. priority。
5. GPU type。
6. nodes。
7. gpus per node。
8. CPU / memory。
9. storage。
10. expected duration。
11. preemptible。
12. checkpoint support。
13. topology requirement。

训练任务、推理服务和评估任务的调度需求不同。

调度系统要理解任务类型，而不是只看 Pod spec。

55.7 多租户队列

队列可以按：

1. 团队。
2. 项目。
3. 优先级。
4. 任务类型。
5. GPU 类型。
6. 生产/实验环境。

划分。

队列属性包括：

1. quota。
2. priority weight。
3. max running jobs。
4. max GPUs。
5. preemptible。
6. allowed GPU types。

队列是多租户公平性的基本单位。

55.8 配额设计

配额分为：

1. guaranteed quota。
2. burst quota。
3. borrowed quota。

Guaranteed quota 是保证资源。

Burst quota 是允许短期超过。

Borrowed quota 是借用别人暂时不用的资源。

借用能提高利用率，但必须可回收。

当原 owner 需要资源时，借用方低优先级任务可能被抢占。

55.9 公平性策略

常见公平性策略：

1. FIFO。
2. weighted fair sharing。
3. dominant resource fairness。
4. priority scheduling。
5. quota-based scheduling。

FIFO 简单但不公平。

Priority scheduling 能保证高优先级任务，但可能饿死低优先级任务。

Weighted fair sharing 适合多团队共享。

实际系统通常组合使用。

55.10 Gang Scheduling

Gang scheduling 是分布式训练核心需求。

一个 64 卡训练任务需要所有 worker 同时启动。

如果只调度了一部分 worker，任务无法正常运行，还会占住资源。

Gang scheduling 要求：

1. 资源够才整体调度。
2. 资源不够就继续排队。
3. 启动失败要整体回滚。
4. 运行中 worker 故障要整体处理。

没有 gang scheduling，多机多卡训练会非常不稳定。

55.11 拓扑感知调度

拓扑感知要考虑：

1. 单机内 GPU 互联。
2. NVLink / NVSwitch。
3. PCIe 拓扑。
4. NUMA。
5. 跨节点网络。
6. 机架位置。
7. IB/RoCE 网络域。

目标是让通信密集型任务尽量放在通信更好的拓扑中。

例如优先满足单机 8 卡，再考虑跨机多卡。

55.12 Placement 策略

Placement 要决定任务放到哪些节点。

常见策略：

1. pack: 尽量紧凑放置，减少碎片。
2. spread: 分散放置，提高容错。
3. topology-aware pack。
4. data-locality-aware placement。
5. reserved placement。

训练任务通常偏向拓扑感知 pack。

推理服务可能需要 spread，提高可用性。

不同任务类型策略不同。

55.13 资源碎片治理

碎片导致 “总资源够，但任务调度不了”。

治理方法：

1. bin packing。
2. 大任务预留。
3. 小任务避开整机资源。
4. 低优先级任务可抢占。
5. 队列重排。
6. 调度模拟。
7. defragmentation。

调度器需要可观测碎片情况。

例如显示有多少空闲 GPU 无法组成 8 卡节点。

55.14 抢占机制

抢占用于让高优先级任务拿到资源。

设计要点：

1. 抢占规则。
2. 候选 victim 选择。
3. graceful termination。
4. checkpoint 保存。
5. 重新入队。
6. 抢占次数限制。
7. 抢占审计。
8. 用户通知。

选择 victim 时要考虑：

1. 优先级低。
2. 可抢占。
3. 已运行时间。
4. checkpoint 状态。
5. 释放资源是否足够。

55.15 低优先级队列

低优先级队列用于填充空闲资源。

适合：

1. 探索实验。
2. 批量评估。
3. embedding 生成。
4. 数据处理。
5. 可恢复训练。

特点：

1. 不保证启动时间。
2. 可被抢占。
3. 成本更低。
4. 利用闲置 GPU。

低优先级队列是提高利用率的重要手段。

55.16 推理服务调度

推理服务调度不同于训练。

关注：

1. 长期运行。
2. 低延迟。
3. 自动扩缩容。
4. 多副本分散。
5. warm pool。
6. 滚动发布。
7. SLO guardrail。

推理服务通常不适合被随意抢占。

可以把推理和训练放在不同资源池，或设置严格优先级和隔离策略。

55.17 异构 GPU 调度

集群可能有不同 GPU。

调度要支持：

1. GPU 型号选择。
2. 显存需求。
3. compute capability。
4. driver / CUDA 兼容。
5. 价格差异。
6. 任务适配性。

例如训练 70B 模型可能需要 H100，embedding 批处理可以使用较低规格 GPU。

异构调度可以提高成本效率。

55.18 调度事件和可观测性

调度系统必须可观测。

需要记录：

1. 为什么任务排队。

2. 为什么任务无法调度。
3. 选择了哪些节点。
4. 为什么抢占某个任务。
5. 资源碎片情况。
6. 队列等待时间。
7. 配额使用情况。
8. pending job 数。
9. 调度决策耗时。

用户最常问的问题是：我的任务为什么还没跑？

调度系统必须能解释。

55.19 成本和利用率

调度系统要输出成本和利用率指标：

1. GPU allocation rate。
2. GPU utilization。
3. idle GPU hours。
4. queue wait time。
5. failed job GPU hours。
6. preempted GPU hours。
7. per-tenant GPU hours。
8. borrowed quota usage。
9. low-priority queue usage。

调度不仅要让任务跑，还要让平台负责人知道资源怎么被使用。

55.20 与 Kubernetes 的关系

可以基于 Kubernetes，但需要扩展。

K8s 提供：

1. Pod 调度。
2. 节点管理。
3. 资源声明。
4. 控制器机制。

AI 调度需要补充：

1. gang scheduling。
2. queue。
3. quota。
4. topology-aware scheduling。
5. preemption policy。
6. GPU health。
7. cost attribution。
8. TrainingJob 语义。

所以答案可以说“基于 K8s 扩展”，但不能止步于 K8s。

55.21 多租户 GPU 调度系统设计指标和最小 demo

系统设计题里，最容易失分的地方是只画出 Scheduler、Queue 和 Kubernetes，却没有说明怎样验收调度系统真的公平、可回收、可解释、可审计。可以把多租户 GPU 调度样本写成：

$S_i = (t_i, q_i, r_i, p_i, g_i, n_i, h_i, b_i, f_i, o_i, c_i, z_i)$

其中 t_i 是 tenant, q_i 是 queue, r_i 是任务资源画像, p_i 是优先级, g_i 是 gang 需求, n_i 是拓扑需求, h_i 是异构 GPU 约束, b_i 是借用策略, f_i 是公平性状态, o_i 是可观测事件, c_i 是成本标签, z_i 是最终调度状态。

统一覆盖率可以写成:

$$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{left}[g_j(S_i)=1\text{right}]]$$

其中 g_j 是第 j 个审计条件, 例如队列契约、配额借用、gang、拓扑、抢占、推理 SLO 或成本归因。

队列配额占用率可以写成:

$$U_q = \frac{G_{\{\text{used}\},q}}{G_{\{\text{quota}\},q}}$$

借用 GPU 数可以写成:

$$B_q = \max\left(0, G_{\{\text{used}\},q} - G_{\{\text{guaranteed}\},q}\right)$$

多资源公平可以用 dominant share 表达:

$$D_q = \max_{r \in \mathcal{R}} \frac{u_{q,r}}{c_r}$$

其中 $u_{q,r}$ 是队列 q 在资源 r 上的使用量, c_r 是集群总容量。带权公平偏差可以写成:

$$H_q = \frac{D_q}{w_q}, \quad \Delta_{\{\text{fair}\}} = \max_q H_q - \min_q H_q$$

其中 w_q 是队列权重。 Δ_{fair} 越大, 说明同权重归一化后的 dominant share 越不均衡。

Gang scheduling 可行性可以写成:

$$G_{\{\text{gang}\}} = \mathbf{1}[\text{left}[\sum_{n=1}^N \text{left}[\lfloor \frac{a_n}{g_{\{\text{pernode}\}}} \rfloor \text{right} \rfloor \geq n_{\{\text{worker}\}} \text{right}]]$$

其中 a_n 是节点 n 的可用 GPU 数, g_{pernode} 是每个 worker 需要的 GPU 数, n_{worker} 是需要同时启动的 worker 数。

拓扑得分可以写成:

$$S_{\{\text{topo}\}} = w_{1C_{\{\text{block}\}}} + w_{2C_{\{\text{rack}\}}} + w_{3C_{\{\text{nic}\}}} + w_{4C_{\{\text{health}\}}}$$

其中 C_{block} 、 C_{rack} 、 C_{nic} 和 C_{health} 分别表示是否处在更好的 GPU / 网络拓扑域、机架域、GPU-NIC 亲和域和健康节点集合中。

碎片率可以写成:

$$R_{\{\text{frag}\}} = 1 - \frac{\sum_n \text{left}[\lfloor \frac{a_n}{g_{\{\text{job}\}}} \rfloor \text{right} \rfloor g_{\{\text{job}\}}}{\sum_n a_n}$$

其中 g_{job} 是典型大任务每节点需要的 GPU 数。总空闲 GPU 很多但 R_{frag} 很高, 说明资源形状不适合大任务。

抢占损失可以写成:

$$L_{\{\text{preempt}\}} = \sum_{v \in \mathcal{V}} G_v \Delta t_v$$

其中 $\mathcal{V}$ 是被抢占任务集合, G_v 是 victim 占用 GPU 数, Δt_v 是距离最近 checkpoint 的分钟数。这个量表示最坏丢失 GPU-minutes。

有效利用率可以写成:

$$U_{\{\text{eff}\}} = \frac{H_{\{\text{alloc}\}} - H_{\{\text{idle}\}} - H_{\{\text{failed}\}} - H_{\{\text{blocked}\}}}{H_{\{\text{alloc}\}}}$$

这里 H_{alloc} 是已分配 GPU-hours, H_{idle} 是分配但空闲, H_{failed} 是失败重跑浪费, H_{blocked} 是碎片或错误调度导致的无效占用。它比普通 GPU utilization 更接近“真实有效产出”。

最终门禁可以写成:

$$G_{\{\text{mtgpu}\}} = \mathbf{1}[\text{left}[\min_j C_j \geq \tau_j \text{and } \Delta_{\{\text{fair}\}} \leq \delta \text{and } R_{\{\text{frag}\}} \leq \rho \text{right}]]$$

其中 P_0 是 P0 级硬阻断数量。这个门禁提醒你: 多租户 GPU 调度不是单个算法, 而是队列、配额、公平、gang、拓扑、抢占、低优先级、推理 SLO、异构成本和可观测性的综合治理。

下面是一个 0 依赖 demo。它不实现真实调度器，而是模拟面试中应具备的“系统设计审计表”：给定 toy 队列、节点和设计样本，输出配额占用、借用、dominant share、gang 可行性、拓扑得分、碎片率、抢占损失、有效利用率，并检查 16 个系统设计门禁。

```
from math import floor
```

```
class MiniMultiTenantGPUSchedulerDesignAudit:
```

```
    def __init__(self):
```

```
        self.capacity = {"gpu": 80, "cpu": 960, "memory_gib": 4096}
```

```
        self.queues = {
```

```
            "research": {
```

```
                "guaranteed_gpus": 16,
```

```
                "quota_gpus": 32,
```

```
                "weight": 0.50,
```

```
                "usage": {"gpu": 24, "cpu": 160, "memory_gib": 900},
```

```
            },
```

```
            "product": {
```

```
                "guaranteed_gpus": 16,
```

```
                "quota_gpus": 24,
```

```
                "weight": 0.30,
```

```
                "usage": {"gpu": 14, "cpu": 120, "memory_gib": 512},
```

```
            },
```

```
            "eval": {
```

```
                "guaranteed_gpus": 8,
```

```
                "quota_gpus": 16,
```

```
                "weight": 0.20,
```

```
                "usage": {"gpu": 10, "cpu": 96, "memory_gib": 256},
```

```
            },
```

```
        }
```

```
        self.nodes = [
```

```
            {"id": "node-a1", "free_gpus": 8, "block": "nv-a", "rack": "rack-1", "nic": "ib-a", "healthy": True},
```

```
            {"id": "node-a2", "free_gpus": 8, "block": "nv-a", "rack": "rack-1", "nic": "ib-a", "healthy": True},
```

```
            {"id": "node-b1", "free_gpus": 8, "block": "nv-b", "rack": "rack-2", "nic": "ib-b", "healthy": True},
```

```
            {"id": "node-b2", "free_gpus": 4, "block": "nv-b", "rack": "rack-2", "nic": "ib-b", "healthy": True},
```

```
            {"id": "node-c1", "free_gpus": 2, "block": "pcie-c", "rack": "rack-3", "nic": "eth-c", "healthy": True},
```

```
        ]
```

```
        self.gates = [
```

```
            "tenant_queue_contract",
```

```
            "quota_borrowing_policy",
```

```
            "dominant_fairness_accounting",
```

```
            "gang_scheduling_fit",
```

```
            "topology_aware_placement",
```

```
            "fragmentation_control",
```

```
            "preemption_checkpoint_safety",
```

```
            "low_priority_reclaim",
```

```
            "inference_slo_isolation",
```

```
            "heterogeneous_gpu_policy",
```

```
            "resource_snapshot_health",
```

```
            "placement_explainability",
```

```
            "cost_utilization_attribution",
```

```
            "scheduler_observability_audit",
```

```
            "tradeoff_boundary_reasoning",
```

```
            "multi_tenant_gpu_scheduler_gate",
```

```
        ]
```



```

def quota_usage(self):
    result = {}
    for name, queue in self.queues.items():
        result[name] = round(queue["usage"]["gpu"] / queue["quota_gpus"], 3)
    return result

def borrowed_gpus(self):
    result = {}
    for name, queue in self.queues.items():
        borrowed = queue["usage"]["gpu"] - queue["guaranteed_gpus"]
        result[name] = max(0, borrowed)
    return result

def dominant_shares(self):
    shares = {}
    for name, queue in self.queues.items():
        dominant = 0.0
        for resource, capacity in self.capacity.items():
            dominant = max(dominant, queue["usage"][resource] / capacity)
        shares[name] = round(dominant, 3)
    return shares

def fairness_gap(self):
    shares = self.dominant_shares()
    normalized = []
    for name, share in shares.items():
        normalized.append(share / self.queues[name]["weight"])
    return round(max(normalized) - min(normalized), 3)

def gang_feasible(self, worker_count, gpus_per_worker):
    fit_nodes = 0
    for node in self.nodes:
        fit_nodes += floor(node["free_gpus"] / gpus_per_worker)
    return fit_nodes >= worker_count

def topology_score(self, selected_node_ids):
    selected = [node for node in self.nodes if node["id"] in selected_node_ids]
    if not selected:
        return 0.0
    same_block = int(len({node["block"] for node in selected}) == 1)
    same_rack = int(len({node["rack"] for node in selected}) == 1)
    same_nic = int(len({node["nic"] for node in selected}) == 1)
    all_healthy = int(all(node["healthy"] for node in selected))
    score = 0.35 * same_block + 0.25 * same_rack + 0.20 * same_nic + 0.20 * all_healthy
    return round(score, 3)

def fragmentation_ratio(self, gpus_per_job):
    free_total = sum(node["free_gpus"] for node in self.nodes)
    if free_total == 0:
        return 0.0
    packable = sum(floor(node["free_gpus"] / gpus_per_job) * gpus_per_job for node in self.nodes)
    return round(1 - packable / free_total, 3)

def preemption_loss_gpu_minutes(self):
    victims = [

```

```

        {"job": "low-priority-train", "gpus": 8, "minutes_since_checkpoint": 12},
        {"job": "batch-eval", "gpus": 4, "minutes_since_checkpoint": 5},
    ]
    return sum(victim["gpus"] * victim["minutes_since_checkpoint"] for victim in victims)

def effective_utilization(self):
    allocated = 1000
    idle = 90
    failed = 20
    blocked = 30
    return round((allocated - idle - failed - blocked) / allocated, 3)

def scheduler_examples(self):
    return {
        "quota_usage": self.quota_usage(),
        "borrowed_gpus": self.borrowed_gpus(),
        "dominant_shares": self.dominant_shares(),
        "fairness_gap": self.fairness_gap(),
        "gang_feasible": self.gang_feasible(worker_count=3, gpus_per_worker=8),
        "topology_score": self.topology_score(["node-a1", "node-a2"]),
        "fragmentation_ratio": self.fragmentation_ratio(gpus_per_job=8),
        "preemption_loss_gpu_min": self.preemption_loss_gpu_minutes(),
        "effective_utilization": self.effective_utilization(),
    }

def design_cases(self):
    complete = {gate: True for gate in self.gates}
    bad_specs = [
        ("tenant_queue_missing_bad", "tenant_queue_contract"),
        ("quota_borrowing_unbounded_bad", "quota_borrowing_policy"),
        ("dominant_fairness_missing_bad", "dominant_fairness_accounting"),
        ("gang_partial_start_bad", "gang_scheduling_fit"),
        ("topology_ignored_bad", "topology_aware_placement"),
        ("fragmentation_uncontrolled_bad", "fragmentation_control"),
        ("preemption_without_checkpoint_bad", "preemption_checkpoint_safety"),
        ("low_priority_not_reclaimable_bad", "low_priority_reclaim"),
        ("inference_slo_mixed_with_training_bad", "inference_slo_isolation"),
        ("heterogeneous_gpu_no_policy_bad", "heterogeneous_gpu_policy"),
        ("resource_snapshot_stale_bad", "resource_snapshot_health"),
        ("placement_reason_missing_bad", "placement_explainability"),
        ("cost_utilization_unattributed_bad", "cost_utilization_attribution"),
        ("scheduler_observability_missing_bad", "scheduler_observability_audit"),
        ("tradeoff_boundary_missing_bad", "tradeoff_boundary_reasoning"),
        ("scheduler_gate_missing_bad", "multi_tenant_gpu_scheduler_gate"),
    ]
    cases = [{"name": "complete_scheduler_design", "checks": complete}]
    for name, failed_gate in bad_specs:
        checks = {gate: True for gate in self.gates}
        checks[failed_gate] = False
        cases.append({"name": name, "checks": checks})
    return cases

def evaluate_cases(self):
    cases = self.design_cases()
    metrics = {}

```

```

failed_cases = []
failed_gates = set()
for gate in self.gates:
    passed = sum(1 for case in cases if case["checks"][gate])
    metrics[gate] = round(passed / len(cases), 3)
for case in cases:
    missing = [gate for gate, ok in case["checks"].items() if not ok]
    if missing:
        failed_cases.append(case["name"])
        failed_gates.update(missing)
return {
    "metrics": metrics,
    "failed_cases": failed_cases,
    "failed_gates": sorted(failed_gates),
    "hard_blocker_count": len(failed_cases),
    "gate_pass": len(failed_cases) == 0,
}

def run(self):
    evaluated = self.evaluate_cases()
    return {
        "scheduler_examples": self.scheduler_examples(),
        "metrics": evaluated["metrics"],
        "hard_blocker_count": evaluated["hard_blocker_count"],
        "failed_case_count": len(evaluated["failed_cases"]),
        "failed_gate_count": len(evaluated["failed_gates"]),
        "multi_tenant_gpu_scheduler_gate_pass": evaluated["gate_pass"],
    }

```

```

if __name__ == "__main__":
    audit = MiniMultiTenantGPUSchedulerDesignAudit()
    result = audit.run()
    print("scheduler_examples=" + str(result["scheduler_examples"]))
    print("metrics=" + str(result["metrics"]))
    print("hard_blocker_count=" + str(result["hard_blocker_count"]))
    print("failed_case_count=" + str(result["failed_case_count"]))
    print("failed_gate_count=" + str(result["failed_gate_count"]))
    print("multi_tenant_gpu_scheduler_gate_pass=" + str(result["multi_tenant_gpu_scheduler_gate_pass"]))

```

一组典型输出如下：

```

scheduler_examples={'quota_usage': {'research': 0.75, 'product': 0.583, 'eval': 0.625}, 'borrowed_gpus': {'research': 8,
metrics={'tenant_queue_contract': 0.941, 'quota_borrowing_policy': 0.941, 'dominant_fairness_accounting': 0.941, 'gang
hard_blocker_count=16
failed_case_count=16
failed_gate_count=16
multi_tenant_gpu_scheduler_gate_pass=False

```

这里的重点不是 0.941 这个 toy 分数，而是审计思路：每个 bad case 只破坏一个设计条件，因此每个条件都能被独立解释。真实平台中，这张表可以接入调度事件、队列状态、GPU 资源快照、checkpoint metadata、推理 SLO、成本标签和审计日志，成为“为什么这个任务没跑、为什么这个任务被抢占、为什么这个队列能借用资源”的统一解释层。

55.22 关键 trade-off

调度系统 trade-off:

1. 公平性 vs 利用率。
2. 高优先级保障 vs 低优先级饥饿。
3. 紧凑放置 vs 容错。
4. 抢占效率 vs 浪费已完成进度。
5. 资源预留 vs 空闲成本。
6. 调度复杂度 vs 可解释性。
7. 异构成本优化 vs 用户体验复杂度。

系统设计题中一定要讲这些权衡。

55.23 面试回答模板

可以这样回答：

我会把多租户 GPU 调度系统设计成基于队列、配额和拓扑感知的调度控制平面。用户提交 TrainingJob 或 InferenceDeployment 后，Scheduler 维护集群 Resource Snapshot，包括 GPU 类型、数量、健康状态、拓扑、CPU、内存、网络和本地盘。调度时 Policy Engine 对于高优先级任务，如果资源不足，可以通过 Preemption Engine 抢占低优先级、可恢复任务，抢占前保存 checkpoint，任务重新入队。系统同时提供调度事件、队列等待、配额使用、资源碎片、GPU 利用率、idle GPU hours、preemption 和成本归因的可观测性，让用户可以对这个回答覆盖资源模型、队列、调度、拓扑、抢占、可观测性和成本。

55.24 常见扣分点

扣分点一：只说 Kubernetes scheduler。

问题：没有讲 gang scheduling、队列、配额、拓扑和抢占。

扣分点二：不提多租户公平性。

问题：内部平台一定需要公平性和配额。

扣分点三：不提资源碎片。

问题：GPU 集群常见问题就是总资源够但无法满足大任务。

扣分点四：抢占只说 kill 任务。

问题：生产抢占要考虑 checkpoint、重新入队、通知和审计。

扣分点五：不提可观测性。

问题：调度系统必须解释任务为什么排队和为什么被抢占。

55.25 小练习

1. 为什么大模型训练需要 gang scheduling?
2. GPU 资源碎片是什么？如何缓解？
3. Guaranteed quota、burst quota 和 borrowed quota 有什么区别？
4. 拓扑感知调度要考虑哪些因素？
5. 抢占 victim 选择时应考虑哪些指标？
6. 推理服务和训练任务调度有什么不同？
7. 如何设计“我的任务为什么没跑”的解释功能？
8. 如何用低优先级队列提高 GPU 利用率？
9. 如何用 dominant share 解释多租户 GPU、CPU 和内存的公平性？
10. 如何把抢占损失从“kill 一个任务”量化成 GPU-minutes 损失？

55.26 本章小结

本章系统设计了一个多租户 GPU 集群调度系统。

你需要记住：

1. 多租户 GPU 调度要同时考虑公平性、优先级、配额、拓扑、碎片、抢占和利用率。
2. 大模型训练需要 gang scheduling，推理服务需要低延迟和高可用，两者调度目标不同。
3. 队列和配额是多租户治理基础，借用和低优先级队列能提高利用率。
4. 拓扑感知和资源碎片治理是 GPU 集群调度的核心难点。
5. 调度系统必须可解释、可观测、可审计，否则用户和平台团队都无法信任它。
6. 第二轮新增的公式和 demo 可以把配额占用、借用、dominant share、gang 可行性、拓扑得分、碎片率、抢占损失和有效利用率统一成可验收的系统设计门禁。

下一章我们会设计一个大模型评估与实验平台。

第 56 章 设计一个大模型评估与实验平台

上一章设计了多租户 GPU 集群调度系统。本章继续系统设计题：设计一个大模型评估与实验平台。

评估与实验平台是大模型研发和发布的质量控制中心。训练得到一个模型、改了一个 prompt、更新了一个 RAG 配置、上线了一个 Agent，都需要评估和实验系统回答：它是否真的更好？是否更安全？是否更便宜？是否可以发布？

先记住一句话：

大模型评估与实验平台的核心，是把模型、prompt、数据、指标、样本、人工判断、线上反馈和发布门禁连接成可复现的质量决策系统。

56.0 本讲资料边界与第二轮精修口径

第二轮精修时，本章按“评估平台系统设计 + 质量决策闭环”校准，不绑定某个评估产品、实验平台或内部发布系统。

参考口径包括：

1. OpenAI Evals：把 eval 理解为评估 LLM 或 LLM 系统的框架，并强调可以构建自定义 eval 和私有 eval，用来衡量模型版本变化对具体用例的影响。
2. MLflow GenAI evaluation / monitoring：把 evaluation dataset、human feedback、LLM-as-a-Judge、systematic evaluation 和 production monitoring 放在 LLM / Agent 生命周期里理解。
3. OpenTelemetry 生成式 AI 语义约定：把模型调用、request / response、token、latency、span 属性和观测信号作为评估、trace 和线上监控可关联的基础。
4. Kubernetes Job / batch workload 口径：把 EvalJob 看成 run-to-completion 的批处理任务，需要状态、重试、失败分类、资源预算和 artifact 输出。

本章的目标不是教你背 benchmark 名称，而是把“模型是否可以发布”回答成一套可复现证据链：评估数据版本清楚，推理配置固定，指标定义有版本，LLM-as-a-Judge 有校准，人评有质量控制，样本级结果可回放，切片退化能发现，在线实验有 guardrail，评估报告能进入模型仓库和发布系统。

56.1 题目理解

面试题可能这样问：

1. 请设计一个大模型评估平台。
2. 请设计一个支持离线评测、人工评测和在线 A/B 的系统。
3. 请设计一个模型实验追踪与评估平台。
4. 如何设计模型发布前的质量门禁系统？

这些题的核心是：如何系统化评估模型和大模型应用，而不是手写 benchmark 脚本。

56.2 需求澄清

先问：

1. 评估对象是什么？模型、prompt、RAG、Agent 还是工具？
2. 支持哪些任务？问答、代码、工具调用、安全、长上下文？
3. 是否需要离线批量评测？
4. 是否需要人工评测？
5. 是否需要 LLM-as-Judge？
6. 是否需要在线 A/B 测试？
7. 是否需要和发布系统联动？
8. 是否需要样本级结果和失败分析？
9. 是否多租户？是否有权限和审计？
10. 评估成本和时延要求是什么？

评估平台的设计取决于评估对象和决策场景。

56.3 核心目标

平台目标：

1. 管理 eval dataset。
2. 管理 eval suite。
3. 支持多模型、多版本评估。
4. 支持 prompt/RAG/Agent 评估。
5. 支持离线批量评测。
6. 支持人工评测。
7. 支持 LLM-as-Judge。
8. 支持在线 A/B 测试。
9. 支持样本级结果和失败分析。
10. 支持评估报告、发布门禁和实验追踪。

一句话：评估平台要把“跑分”变成“质量决策”。

56.4 总体架构

可以设计如下：

User / CI / Release System
-> Eval API / Web Console
-> Eval Dataset Registry
-> Eval Suite Registry
-> Eval Job Scheduler
-> Inference Runner
-> Metric Engine
-> LLM Judge Service
-> Human Review System
-> Result Store
-> Experiment Tracker
-> Report Generator
-> Quality Gate
-> Model Registry / Release System

关键链路：选择模型和评估集，运行推理，计算指标，生成报告，决定是否进入发布。

56.5 Eval Dataset Registry

Eval dataset registry 管理评估数据集。

每个 eval dataset 记录：

1. 名称。
2. 版本。
3. 任务类型。
4. 样本数量。
5. 数据来源。
6. 标签或参考答案。
7. 难度分布。
8. 语言和领域分布。
9. 是否可用于训练。
10. 权限和敏感等级。

Eval 数据必须版本化，并与训练数据隔离。

56.6 Eval Suite

Eval suite 是一组评估任务集合。

例如：

```
chat-release-gate
- general-qa
- safety-basic
- tool-call-regression
- json-format
- long-context
```

Eval suite 用于标准化评估。

不同模型发布前跑同一套 suite，结果才可比较。

56.7 Eval Job

Eval Job 描述一次评估执行。

字段包括：

1. eval job ID。
2. target model version。
3. prompt version。
4. eval suite。
5. eval dataset versions。
6. inference config。
7. metric config。
8. judge model version。
9. runtime。
10. owner。
11. status。

Eval Job 和 TrainingJob 一样，也需要状态、日志、指标和 artifact。

56.8 Inference Runner

Inference Runner 负责批量调用模型。

它要支持：

1. 多模型 endpoint。
2. 并发控制。

3. 限流。
4. 重试。
5. 超时。
6. streaming 或非 streaming。
7. 结果缓存。
8. 失败样本记录。

评估时要固定 inference config。

temperature、max tokens、prompt 模板变化都会影响评估结果。

56.9 Metric Engine

Metric Engine 负责计算指标。

指标类型包括：

1. 规则指标。
2. 字符串匹配。
3. 结构化输出校验。
4. 单测执行。
5. 安全策略校验。
6. LLM-as-Judge 分数。
7. 人工标注结果。

Metric definition 必须版本化。

否则同一个结果在不同指标定义下可能分数不同。

56.10 LLM-as-Judge 服务

LLM-as-Judge 适合开放式生成评估。

Judge 服务要记录：

1. judge model version。
2. judge prompt version。
3. scoring rubric。
4. inference config。
5. calibration data。
6. bias checks。

还要防止：

1. 偏向长答案。
2. 偏向某种风格。
3. judge prompt 漂移。
4. judge model 升级后分数不可比。

重要结论最好结合人工评测或规则指标验证。

56.11 Human Review System

人工评测系统负责分发和收集人工判断。

能力包括：

1. 样本抽样。
2. 标注任务分发。
3. 标注指南版本。
4. 多人标注。

5. 仲裁。
6. gold set 检查。
7. 一致性统计。
8. 人工评测报告。

人工评测适合主观质量、安全边界和业务可接受性评估。

人工评测本身也要质量控制。

56.12 Sample-level Result Store

样本级结果非常重要。

每条样本应记录：

1. input。
2. reference。
3. prediction。
4. model version。
5. prompt version。
6. score。
7. error type。
8. latency。
9. token usage。
10. judge output。
11. human label。

样本级结果用于失败分析、回归集构建和模型对比。

只看平均分无法定位问题。

56.13 Slice Evaluation

平台要支持切片评估。

切片维度包括：

1. 语言。
2. 领域。
3. 难度。
4. 输入长度。
5. 输出格式。
6. 安全类别。
7. 工具类型。
8. 用户场景。

一个模型总体分数提升，但某个高价值业务场景退化，不能直接上线。

Slice evaluation 能发现隐藏退化。

56.14 Experiment Tracker 集成

每次 eval job 应关联 experiment run。

记录：

1. 参数。
2. 模型版本。
3. 数据版本。
4. prompt 版本。
5. 指标。

6. 样本级结果。
7. 报告 artifact。
8. 成本。

这样评估结果可以和训练实验、prompt 实验、RAG 实验关联。

56.15 Eval Report

Eval report 应包含：

1. 被评对象。
2. eval suite。
3. 数据版本。
4. 指标定义。
5. 总体指标。
6. 切片指标。
7. 与 baseline 对比。
8. 样本级失败案例。
9. 成本和延迟。
10. 结论和建议。

Eval report 应作为 artifact 进入模型仓库或发布系统。

56.16 Quality Gate

Quality Gate 是发布门禁。

示例规则：

1. 核心能力不低于 baseline。
2. 安全违规率低于阈值。
3. 工具调用成功率达标。
4. JSON parse success rate 达标。
5. p99 latency 不超过阈值。
6. cost per request 不超过预算。
7. 关键 slice 不退化。

Quality Gate 可以输出：

1. pass。
2. fail。
3. warning。
4. needs human review。

发布系统根据门禁结果决定是否允许灰度。

56.17 Online A/B 测试

在线 A/B 用真实流量比较版本。

平台要支持：

1. 稳定分桶。
2. 实验配置。
3. 指标采集。
4. guardrail 指标。
5. 实验停止。
6. 结果分析。
7. 回滚。

在线指标包括用户反馈、任务完成率、人工接管率、延迟、成本和安全指标。

在线 A/B 要和灰度发布区分：灰度偏风险控制，A/B 偏效果比较。

56.18 评估缓存

评估任务成本高，应支持缓存。

缓存 key 包括：

1. model version。
2. prompt version。
3. eval dataset version。
4. inference config。
5. runtime version。

如果这些都不变，可以复用 prediction outputs 或 metric results。

缓存能减少重复评估成本，但不能错误复用不同版本结果。

56.19 权限和安全

评估平台也需要安全治理。

要控制：

1. 谁能看 eval dataset。
2. 谁能运行评估。
3. 谁能查看 sample outputs。
4. 谁能查看人工标注结果。
5. 谁能修改 metric definition。
6. 谁能修改 release gate。

Eval 数据和 sample outputs 可能包含敏感信息，要做权限和脱敏。

56.20 成本治理

评估成本来自：

1. 批量推理。
2. LLM-as-Judge。
3. 人工评测。
4. 存储 sample outputs。
5. 运行单测或工具。

平台应记录：

1. cost per eval job。
2. cost per sample。
3. judge model cost。
4. human review cost。
5. 评估缓存节省成本。

评估不是免费操作，大规模评估需要预算控制。

56.21 可观测性

Eval 平台可观测性包括：

1. eval job status。
2. queue wait time。

3. samples processed per second。
4. inference error rate。
5. judge error rate。
6. metric computation error。
7. cost。
8. report generation latency。

如果评估失败，要知道是模型服务问题、数据问题、metric 问题还是 judge 服务问题。

56.22 核心数据模型

核心对象：

1. EvalDataset。
2. EvalSuite。
3. EvalJob。
4. MetricDefinition。
5. JudgeConfig。
6. HumanReviewTask。
7. SampleResult。
8. EvalReport。
9. QualityGate。
10. ABExperiment。

这些对象要有版本、权限、状态和血缘。

56.23 评估实验平台系统设计指标和最小 demo

系统设计题里，评估平台不能只讲“跑 benchmark 得分”。可以把一次评估实验抽象成：

$$E_i=(a_i,m_i,d_i,p_i,c_i,k_i,j_i,h_i,o_i,r_i,s_i,g_i,z_i)$$

其中 a_i 是被评对象，可能是模型、prompt、RAG、Agent 或工具链； m_i 是模型版本， d_i 是 eval dataset 版本， p_i 是 prompt 版本， c_i 是推理配置， k_i 是指标定义， j_i 是 judge 配置， h_i 是人评配置， o_i 是在线实验配置， r_i 是样本级结果， s_i 是切片结果， g_i 是 release gate， z_i 是最终决策。

统一覆盖率可以写成：

$$C_j=\frac{1}{N}\sum_{i=1}^N\mathbf{1}\left[g_j(E_i)=1\right]$$

其中 g_j 是第 j 个审计条件，例如数据集契约、离线复现、指标版本、judge 校准、人评质量、在线 guardrail、报告完整性或 artifact 血缘。

数据集切片覆盖率可以写成：

$$C_{\mathrm{slice}}=\frac{1}{S}\sum_{s=1}^S\mathbf{1}\left[n_s\geq n_{\min}\right]$$

训练 / 评估污染率可以写成：

$$R_{\mathrm{leak}}=\frac{N_{\mathrm{overlap}}}{N_{\mathrm{eval}}}$$

多指标加权汇总可以写成：

$$S_{\mathrm{agg}}=\frac{\sum_{k=1}^Kw_{M_k}}{\sum_{k=1}^Kw_k}$$

其中 M_k 是已经同向归一化后的指标分数。注意它只适合做候选排序，不能替代切片退化和安全门禁。

Pairwise win rate 可以写成：

$$W=\frac{N_{\mathrm{win}}+0.5N_{\mathrm{tie}}}{N_{\mathrm{win}}+N_{\mathrm{loss}}+N_{\mathrm{tie}}}$$

Judge 和人工标注一致率可以写成：

$$A_{\mathrm{jh}}=\frac{1}{N}\sum_{i=1}^N\mathbf{1}\left[y_i^{\mathrm{judge}}=y_i^{\mathrm{human}}\right]$$

在线实验 guardrail 可以写成:

$$G_{\{\mathrm{online}\}} = \mathbf{1} \setminus [\Delta_Q \geq -\epsilon_Q \wedge \Delta_L \leq \epsilon_L \wedge \Delta_K \leq \epsilon_K]$$

其中 Δ_Q 是质量变化, Δ_L 是延迟变化, Δ_K 是成本变化, P_0 是 P0 级事故数量。

缓存命中率可以写成:

$$R_{\{\mathrm{cache}\}} = \frac{N_{\{\mathrm{hit}\}}}{N_{\{\mathrm{request}\}}}$$

但缓存 key 必须包含模型版本、数据版本、prompt 版本、推理配置、评估代码版本和 runtime 版本, 否则命中率越高, 错误复用风险越大。

Artifact 血缘覆盖率可以写成:

$$R_{\{\mathrm{lineage}\}} = \frac{|E_{\{\mathrm{observed}\}}|}{|E_{\{\mathrm{required}\}}|}$$

最终评估平台门禁可以写成:

$$G_{\{\mathrm{eval}\}} = \mathbf{1} \setminus [\min_j C_j \geq \tau_j \wedge C_{\{\mathrm{slice}\}} \geq \alpha \wedge R_{\{\mathrm{leak}\}} \leq \beta]$$

下面的 0 依赖 demo 不实现完整评估系统, 而是实现一个系统设计审计器: 它给出 toy eval dataset、metric、pairwise、人评、judge、online guardrail、cache、lineage 和 design cases, 输出可用于面试解释的质量决策指标。

```
class MiniEvalExperimentPlatformSystemDesignAudit:
    def __init__(self):
        self.slice_counts = {
            "language:zh": 72,
            "language:en": 86,
            "domain:code": 64,
            "domain:math": 58,
            "difficulty:hard": 41,
        }
        self.sample_count = 500
        self.overlap_count = 1
        self.metrics = [
            {"name": "accuracy", "weight": 0.35, "score": 0.82},
            {"name": "safety_pass", "weight": 0.25, "score": 0.98},
            {"name": "tool_success", "weight": 0.20, "score": 0.76},
            {"name": "format_success", "weight": 0.10, "score": 0.93},
            {"name": "cost_budget", "weight": 0.10, "score": 0.80},
        ]
        self.pairwise = {"wins": 58, "losses": 36, "ties": 6}
        self.judge_labels = ["good", "bad", "good", "good", "bad"]
        self.human_labels = ["good", "bad", "bad", "good", "bad"]
        self.online = {
            "quality_delta": 0.025,
            "latency_delta": 0.03,
            "cost_delta": 0.04,
            "safety_events": 0,
        }
        self.cache = {"requests": 1200, "hits": 420}
        self.lineage = {"observed_edges": 13, "required_edges": 14}
        self.costs = {"inference": 18.2, "judge": 6.4, "human_review": 40.0}
        self.gates = [
            "eval_dataset_contract",
            "offline_batch_reproducibility",
            "metric_definition_versioning",
            "llm_judge_calibration",
            "human_review_quality_control",
```

```

        "pairwise_blinding_randomization",
        "online_eval_guardrail",
        "release_gate_readiness",
        "eval_report_completeness",
        "slice_regression_detection",
        "regression_suite_coverage",
        "eval_job_scheduler_reliability",
        "eval_cache_key_integrity",
        "eval_observability",
        "artifact_tracking_linkage",
        "evaluation_platform_gate",
    ]

    def dataset_slice_coverage(self, min_samples=40):
        covered = sum(1 for count in self.slice_counts.values() if count >= min_samples)
        return round(covered / len(self.slice_counts), 3)

    def contamination_rate(self):
        return round(self.overlap_count / self.sample_count, 4)

    def weighted_score(self):
        weight_sum = sum(metric["weight"] for metric in self.metrics)
        score = sum(metric["weight"] * metric["score"] for metric in self.metrics) / weight_sum
        return round(score, 3)

    def pairwise_win_rate(self):
        wins = self.pairwise["wins"]
        losses = self.pairwise["losses"]
        ties = self.pairwise["ties"]
        return round((wins + 0.5 * ties) / (wins + losses + ties), 3)

    def judge_human_agreement(self):
        matched = sum(
            1 for judge, human in zip(self.judge_labels, self.human_labels)
            if judge == human
        )
        return round(matched / len(self.human_labels), 3)

    def online_guardrail_pass(self):
        return (
            self.online["quality_delta"] >= -0.01
            and self.online["latency_delta"] <= 0.05
            and self.online["cost_delta"] <= 0.08
            and self.online["safety_events"] == 0
        )

    def cache_hit_rate(self):
        return round(self.cache["hits"] / self.cache["requests"], 3)

    def lineage_coverage(self):
        return round(self.lineage["observed_edges"] / self.lineage["required_edges"], 3)

    def estimated_eval_cost(self):
        return round(sum(self.costs.values()), 2)

```

```

def eval_examples(self):
    return {
        "dataset_slice_coverage": self.dataset_slice_coverage(),
        "contamination_rate": self.contamination_rate(),
        "weighted_score": self.weighted_score(),
        "pairwise_win_rate": self.pairwise_win_rate(),
        "judge_human_agreement": self.judge_human_agreement(),
        "online_guardrail_pass": self.online_guardrail_pass(),
        "cache_hit_rate": self.cache_hit_rate(),
        "lineage_coverage": self.lineage_coverage(),
        "estimated_eval_cost": self.estimated_eval_cost(),
    }

def design_cases(self):
    complete = {gate: True for gate in self.gates}
    bad_specs = [
        ("dataset_version_missing_bad", "eval_dataset_contract"),
        ("offline_config_unfrozen_bad", "offline_batch_reproducibility"),
        ("metric_version_missing_bad", "metric_definition_versioning"),
        ("judge_uncalibrated_bad", "llm_judge_calibration"),
        ("human_review_no_gold_bad", "human_review_quality_control"),
        ("pairwise_not_blinded_bad", "pairwise_blinding_randomization"),
        ("online_guardrail_missing_bad", "online_eval_guardrail"),
        ("release_gate_no_rollback_bad", "release_gate_readiness"),
        ("report_no_samples_bad", "eval_report_completeness"),
        ("slice_regression_hidden_bad", "slice_regression_detection"),
        ("regression_suite_thin_bad", "regression_suite_coverage"),
        ("eval_scheduler_unreliable_bad", "eval_job_scheduler_reliability"),
        ("cache_key_missing_prompt_bad", "eval_cache_key_integrity"),
        ("eval_observability_missing_bad", "eval_observability"),
        ("artifact_lineage_broken_bad", "artifact_tracking_linkage"),
        ("evaluation_gate_missing_bad", "evaluation_platform_gate"),
    ]
    cases = [{"name": "complete_eval_platform_design", "checks": complete}]
    for name, failed_gate in bad_specs:
        checks = {gate: True for gate in self.gates}
        checks[failed_gate] = False
        cases.append({"name": name, "checks": checks})
    return cases

def evaluate_cases(self):
    cases = self.design_cases()
    metrics = {}
    failed_cases = []
    failed_gates = set()
    for gate in self.gates:
        passed = sum(1 for case in cases if case["checks"][gate])
        metrics[gate] = round(passed / len(cases), 3)
    for case in cases:
        missing = [gate for gate, ok in case["checks"].items() if not ok]
        if missing:
            failed_cases.append(case["name"])
            failed_gates.update(missing)
    return {
        "metrics": metrics,

```

```

        "failed_cases": failed_cases,
        "failed_gates": sorted(failed_gates),
        "hard_blocker_count": len(failed_cases),
        "gate_pass": len(failed_cases) == 0,
    }

    def run(self):
        evaluated = self.evaluate_cases()
        return {
            "eval_examples": self.eval_examples(),
            "metrics": evaluated["metrics"],
            "hard_blocker_count": evaluated["hard_blocker_count"],
            "failed_case_count": len(evaluated["failed_cases"]),
            "failed_gate_count": len(evaluated["failed_gates"]),
            "evaluation_platform_gate_pass": evaluated["gate_pass"],
        }

if __name__ == "__main__":
    audit = MiniEvalExperimentPlatformSystemDesignAudit()
    result = audit.run()
    print("eval_examples=" + str(result["eval_examples"]))
    print("metrics=" + str(result["metrics"]))
    print("hard_blocker_count=" + str(result["hard_blocker_count"]))
    print("failed_case_count=" + str(result["failed_case_count"]))
    print("failed_gate_count=" + str(result["failed_gate_count"]))
    print("evaluation_platform_gate_pass=" + str(result["evaluation_platform_gate_pass"]))

```

一组典型输出如下：

```

eval_examples={'dataset_slice_coverage': 1.0, 'contamination_rate': 0.002, 'weighted_score': 0.857, 'pairwise_win_rate': 0.941}
metrics={'eval_dataset_contract': 0.941, 'offline_batch_reproducibility': 0.941, 'metric_definition_versioning': 0.941}
hard_blocker_count=16
failed_case_count=16
failed_gate_count=16
evaluation_platform_gate_pass=False

```

这个 demo 的重点是把“质量决策”拆成可检查证据。真实平台里，这些字段会来自 EvalDataset registry、EvalJob scheduler、Inference Runner、Metric Engine、Judge Service、Human Review、Experiment Tracker、Artifact Store、线上 A/B 和 Release Gate。面试时这样回答，比只说“我们会跑 MMLU / HumanEval / safety eval”更能体现系统设计能力。

56.24 关键 trade-off

评估平台 trade-off:

1. 评估全面性 vs 成本。
2. 离线评估速度 vs 线上真实性。
3. LLM-as-Judge 低成本 vs 人工评测可信度。
4. 样本级结果完整保存 vs 隐私和存储成本。
5. 门禁严格 vs 发布效率。
6. 公开 benchmark vs 内部业务集。

成熟平台通常组合多种评估，而不是迷信单一分数。

56.25 面试回答模板

可以这样回答：

我会把大模型评估与实验平台设计成质量决策系统。核心模块包括 Eval Dataset Registry、Eval Suite Registry、Eval Job Scheduler

用户选择模型版本、prompt 版本或 RAG/Agent 配置，并选择 eval suite 发起 EvalJob。Scheduler 执行批量推理，Inference Runner 作为 LLM-as-Judge，关键场景进入人工评测。

平台保存 sample-level results，支持按语言、领域、难度、输入长度、安全类别和工具类型做 slice evaluation。Eval report 作为发布依据。

在线阶段支持 A/B 测试和用户反馈闭环。平台同时提供权限、安全、成本、缓存和可观测性，保证评估可复现、可比较、可审计。

56.26 常见扣分点

扣分点一：只说跑 benchmark。

评估平台还要管理数据、指标、样本、人工评测、报告、门禁和发布联动。

扣分点二：不提 eval 数据版本。

没有数据版本，评估不可复现。

扣分点三：只看总体分数。

总体分数会掩盖 slice 退化。

扣分点四：盲信 LLM-as-Judge。

Judge model 有偏差，需要版本化、校准和人工验证。

扣分点五：不提发布门禁。

评估结果应该进入发布决策，而不只是生成报告。

56.27 小练习

1. Eval Dataset 和 Eval Suite 有什么区别？
2. Eval Job 应该记录哪些版本信息？
3. 为什么 MetricDefinition 要版本化？
4. LLM-as-Judge 有哪些风险？
5. 为什么样本级结果比单一总分更有价值？
6. Quality Gate 应该包含哪些规则？
7. 在线 A/B 和灰度发布有什么区别？
8. 如何设计评估缓存 key？
9. 为什么加权总分不能替代 slice regression 和 safety guardrail？
10. 如何把 judge-human agreement、pairwise win rate 和 online guardrail 放进同一个 release gate？

56.28 本章小结

本章系统设计了一个大模型评估与实验平台。

你需要记住：

1. 评估平台是质量决策系统，不是 benchmark 脚本。
2. Eval dataset、eval suite、metric、judge、prompt、模型和推理配置都要版本化。
3. 平台要支持离线批量评测、LLM-as-Judge、人工评测、在线 A/B 和用户反馈。
4. 样本级结果、slice evaluation 和 failure cases 是模型诊断关键。
5. Quality Gate 把评估结果接入发布流程，是企业级模型治理的重要环节。
6. 第二轮新增的公式和 demo 可以把切片覆盖、污染率、加权指标、pairwise、人评 / judge 一致率、在线 guardrail、缓存、血缘和发布门禁串成可验收的系统设计证据。

下一章我们会设计一个模型仓库与发布系统。

第 57 章 设计一个模型仓库与发布系统

上一章设计了大模型评估与实验平台。本章继续系统设计题：设计一个模型仓库与发布系统。

模型仓库与发布系统是训练平台、评估平台和推理平台之间的桥梁。训练产出 checkpoint，评估产出报告，发布系统决定哪些模型可以进入线上、如何灰度、如何回滚、如何审计。

先记住一句话：

模型仓库与发布系统不是存权重文件的目录，而是模型资产从训练产物到线上服务的版本化、评估、审批、发布和回滚控制系统。

57.0 本讲资料边界与第二轮精修口径

第二轮精修时，本章按“模型资产生命周期控制面 + 发布治理证据链”校准，不绑定某个模型仓库产品、对象存储、CI/CD 或推理平台实现。

参考口径包括：

1. MLflow Model Registry：把 model registry 看成集中模型仓库、API 和 UI，用来管理模型完整生命周期，强调版本、alias、metadata、lineage、stage / production workflow 和 governance。
2. Hugging Face Hub Model Cards：把模型卡看成随模型一起发布的 Markdown + metadata，记录模型用途、限制、训练参数、数据集和评估结果，说明模型仓库不能只有权重 URI。
3. Safetensors / 安全权重格式：用来校准权重格式安全、manifest、checksum 和加载风险边界，避免把任意 pickle 权重直接作为生产 artifact。
4. OpenTelemetry 生成式 AI 语义约定：用来校准线上模型调用 trace 如何反查 model id、tokenizer / config、release manifest、latency、token 和错误信号。

本章的目标不是教你搭一个文件目录，而是把“checkpoint 如何变成 production model”回答成可验证链路：ModelVersion 契约完整，权重 manifest 可校验，tokenizer / config 绑定，adapter / merge / quantization 有血缘，runtime 可加载，eval / safety 报告过线，发布状态机可审计，rollback alias 可用，release manifest 可复现。

57.1 题目理解

面试题可能这样问：

1. 请设计一个企业级模型仓库。
2. 请设计一个支持模型版本管理、评估门禁和发布回滚的系统。
3. 请设计一个从 checkpoint 到 production model 的发布流水线。
4. 如何管理模型权重、tokenizer、adapter、量化版本和发布状态？

核心是模型资产生命周期管理。

57.2 需求澄清

先问：

1. 支持哪些模型类型？LLM、embedding、rerank、多模态？
2. 模型 artifact 有哪些？权重、tokenizer、config、adapter、量化版本？
3. 是否需要关联训练任务和数据血缘？
4. 是否需要评估门禁？
5. 是否支持灰度发布和 A/B 测试？
6. 是否支持一键回滚？
7. 是否多租户和权限隔离？
8. 是否需要跨地域分发？
9. 是否需要 artifact 安全校验？
10. 生命周期和存储成本如何管理？

模型仓库和发布系统既是资产系统，也是生产变更系统。

57.3 核心目标

系统目标：

1. 管理模型和模型版本。
2. 管理权重、tokenizer、config、adapter、量化版本。
3. 记录模型血缘。
4. 关联 eval report。
5. 支持发布状态机。
6. 支持质量门禁和审批。
7. 支持灰度、A/B 和回滚。
8. 支持权限、安全和审计。
9. 支持 artifact 校验和分发。
10. 支持生命周期和成本治理。

57.4 总体架构

可以设计如下：

Training Platform

- > Model Registry API
- > Artifact Store
- > Metadata DB
- > Lineage Graph
- > Eval Report Store
- > Quality Gate
- > Release Controller
- > Deployment Controller
- > Inference Platform
- > Audit / Cost / Security

训练平台注册候选模型，评估平台写入评估报告，发布系统控制 staged、canary、production 和 rollback。

57.5 核心对象

核心对象包括：

1. Model。
2. ModelVersion。
3. ModelArtifact。
4. TokenizerVersion。
5. AdapterVersion。
6. QuantizedVersion。
7. EvalReport。
8. ReleaseManifest。
9. Deployment。
10. ApprovalRecord。
11. AuditEvent。

这些对象要有版本、状态、权限和血缘。

57.6 Model 和 ModelVersion

Model 是逻辑模型系列。

例如：

company-chat
company-code

company-embedding

ModelVersion 是具体版本。

例如：

company-chat:v1.2.0

company-chat:v1.3.0-sft

company-chat:v1.4.0-dpo

线上部署必须指向具体 ModelVersion。

不要用 latest 作为生产发布依据。

57.7 ModelVersion 元数据

ModelVersion 应记录：

1. model name。
2. version。
3. model type。
4. architecture。
5. parameter size。
6. context length。
7. weight artifact。
8. tokenizer version。
9. model config。
10. training job ID。
11. dataset versions。
12. code version。
13. eval reports。
14. safety level。
15. release state。
16. owner。
17. created time。

这些元数据让模型可追溯、可部署、可审计。

57.8 Artifact Store

Artifact Store 保存大文件：

1. 权重分片。
2. tokenizer 文件。
3. config。
4. adapter。
5. 量化权重。
6. deployment package。
7. eval report artifact。

要求：

1. 支持大文件上传下载。
2. 支持 checksum。
3. 支持权限控制。
4. 支持跨地域复制。
5. 支持冷热分层。
6. 支持生命周期管理。

Artifact Store 和 Metadata DB 要分离。

57.9 Artifact 校验

模型 artifact 必须校验：

1. 文件是否完整。
2. checksum 是否匹配。
3. 权重格式是否合法。
4. tokenizer 是否存在。
5. config 是否匹配。
6. runtime 是否兼容。
7. 是否通过安全扫描。
8. 是否有许可证风险。

校验失败的模型不能进入发布流程。

不要等推理部署时才发现模型缺文件。

57.10 模型血缘

模型仓库要记录血缘：

dataset version + training config + code version

- > training job
- > checkpoint
- > model version
- > eval report
- > release manifest
- > deployment

血缘用于：

1. 复现训练。
2. 追踪数据问题影响。
3. 审计模型来源。
4. 分析质量退化。
5. 判断 artifact 是否可删除。

57.11 Adapter 和 Merge 管理

LoRA adapter 依赖 base model。

系统要记录：

1. adapter version。
2. base model version。
3. training job。
4. dataset version。
5. merge status。
6. eval report。
7. compatibility constraints。

Merge 后模型要记录：

base model + adapter -> merged model

否则后续无法判断某个线上模型如何产生。

57.12 量化版本管理

量化版本依赖原始模型。

要记录：

1. source model version。
2. quantization method。
3. calibration dataset。
4. quantization tool version。
5. target runtime。
6. quality eval。
7. latency benchmark。
8. cost improvement。

量化版本不能只看性能，也要看质量和兼容性。

57.13 发布状态机

ModelVersion 状态机可以是：

registered -> validated -> evaluated -> approved -> staged -> canary -> production -> deprecated -> archived

状态含义：

1. registered：已注册。
2. validated：artifact 校验通过。
3. evaluated：评估完成。
4. approved：审批通过。
5. staged：部署到预发。
6. canary：小流量灰度。
7. production：正式生产。
8. deprecated：不推荐使用。
9. archived：归档。

状态变更必须记录审计。

57.14 Quality Gate

Quality Gate 决定模型是否能发布。

门禁规则：

1. eval suite 通过。
2. safety eval 通过。
3. latency benchmark 达标。
4. cost benchmark 达标。
5. tool call success rate 达标。
6. output format 兼容。
7. 关键 slice 不退化。
8. artifact 校验通过。

没有通过门禁的模型不能进入 canary。

57.15 审批流程

企业发布通常需要审批。

审批人可能包括：

1. 模型负责人。
2. 业务负责人。
3. 安全负责人。
4. 平台负责人。

5. 合规负责人。

审批记录包括：

1. 审批人。
2. 时间。
3. 结论。
4. 风险说明。
5. 关联 eval report。
6. 回滚方案。

高风险模型发布不能绕过审批。

57.16 Release Manifest

ReleaseManifest 描述一次发布用到的完整资产。

包括：

1. model version。
2. tokenizer version。
3. weight artifact。
4. runtime image。
5. runtime config。
6. generation config。
7. safety policy。
8. route policy。
9. eval report。
10. release owner。

回滚时回滚的是 release manifest，而不只是模型权重。

57.17 灰度发布

灰度发布支持：

1. 按流量比例。
2. 按租户。
3. 按用户分桶。
4. 按地域。
5. 按业务场景。

灰度过程：

1% -> 5% -> 10% -> 25% -> 50% -> 100%

每一步观察质量、安全、延迟、错误率和成本。

异常则暂停或回滚。

57.18 A/B 测试

A/B 测试用于比较新旧模型效果。

要支持：

1. 稳定分桶。
2. 实验配置。
3. 指标采集。
4. guardrail。
5. 统计分析。

6. 结果报告。

灰度关注风险控制，A/B 关注效果对比。

两者可以同时使用。

57.19 回滚设计

回滚必须快速可靠。

回滚对象包括：

1. model version。
2. tokenizer。
3. runtime config。
4. prompt。
5. route policy。
6. safety policy。
7. generation config。

如果只回滚权重，其他配置不回滚，行为可能仍然异常。

回滚前要确保旧 artifact 仍然可用。

57.20 Deployment Controller

Deployment Controller 负责把 release manifest 部署到推理平台。

能力包括：

1. 创建 endpoint。
2. 加载模型。
3. health check。
4. warmup。
5. 注册到 router。
6. 灰度流量调整。
7. 回滚。
8. 摘除异常实例。

部署不是文件复制，而是生产流量接入过程。

57.21 权限控制

模型仓库权限包括：

1. 谁能注册模型。
2. 谁能下载权重。
3. 谁能查看 eval report。
4. 谁能发布模型。
5. 谁能回滚。
6. 谁能删除 artifact。
7. 哪些租户能调用模型。

模型权重是高价值资产，必须控制下载和导出。

57.22 安全和审计

审计操作包括：

1. 模型注册。
2. artifact 上传。

3. 权重下载。
4. eval report 修改。
5. 审批。
6. 发布。
7. 灰度调整。
8. 回滚。
9. 删除。

安全检查包括：

1. artifact checksum。
2. 权重格式安全。
3. 镜像扫描。
4. secret 检查。
5. 许可证信息。
6. 数据敏感等级继承。

57.23 生命周期管理

模型 artifact 很占存储。

生命周期策略：

1. production 版本长期保留。
2. 可回滚版本保留。
3. failed candidate 定期清理。
4. 中间 checkpoint 按策略清理。
5. archived 版本转冷存储。
6. 删除前检查 deployment 和 lineage。

删除模型前必须确认没有线上依赖和合规保留要求。

57.24 可观测性

系统要观察：

1. model registration count。
2. artifact validation failure。
3. eval gate pass rate。
4. deployment success rate。
5. canary error rate。
6. rollback count。
7. model load latency。
8. storage growth。
9. download audit events。

发布系统的问题往往会直接影响线上推理。

57.25 模型仓库与发布系统设计指标和最小 demo

模型仓库与发布系统可以把一次模型资产注册和发布抽象成：

$M_i = (m_i, v_i, w_i, t_i, c_i, a_i, q_i, e_i, s_i, r_i, d_i, l_i, z_i)$

其中 m_i 是逻辑模型， v_i 是 ModelVersion， w_i 是权重 manifest， t_i 是 tokenizer， c_i 是 config， a_i 是 adapter / merge / quantization 关系， q_i 是 eval report， e_i 是安全和权限证据， s_i 是发布状态， r_i 是 release manifest， d_i 是 deployment， l_i 是 lineage， z_i 是最终治理状态。

统一覆盖率可以写成：

$$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{left}[g_j(M_i)=1]]$$

Release manifest 完整率可以写成:

$$C_{\{\text{manifest}\}} = \frac{N_{\{\text{present}\}}}{N_{\{\text{required}\}}}$$

权重分片 checksum 覆盖率可以写成:

$$C_{\{\text{checksum}\}} = \frac{N_{\{\text{checked}\}}}{N_{\{\text{shard}\}}}$$

Tokenizer / config 绑定门禁可以写成:

$$G_{\{\text{tokcfg}\}} = \mathbf{1}[\text{left}[h_{\{\text{tok}\}} = h_{\{\text{tok}\}}^{\{\text{record}\}} \wedge h_{\{\text{cfg}\}} = h_{\{\text{ma}}]$$

量化质量退化和延迟加速比可以写成:

$$\Delta Q = Q_{\{\text{source}\}} - Q_{\{\text{quant}\}}, \quad S_{\{\text{lat}\}} = \frac{L_{\{\text{source}\}}}{L_{\{\text{quant}\}}}$$

量化版本只有在 ΔQ 不超过阈值且 S_{lat} 确实带来收益时, 才适合进入发布候选。

灰度发布 guardrail 可以写成:

$$G_{\{\text{canary}\}} = \mathbf{1}[\text{left}[\Delta Q \geq -\epsilon_Q \wedge \Delta L \leq \epsilon_L \wedge \Delta E \leq \epsilon_E \wedge$$

回滚就绪门禁可以写成:

$$G_{\{\text{rollback}\}} = \mathbf{1}[\text{left}[A_{\{\text{rollback}\}} = 1 \wedge H_{\{\text{old}\}} = 1 \wedge T_{\{\text{rollback}\}} \leq$$

其中 A_{rollback} 表示 rollback alias 存在, H_{old} 表示旧 release manifest 和 artifact 已通过健康检查。

Artifact 血缘覆盖率可以写成:

$$R_{\{\text{lineage}\}} = \frac{|E_{\{\text{observed}\}}|}{|E_{\{\text{required}\}}|}$$

生命周期存储成本可以写成:

$$K_{\{\text{life}\}} = \sum_{a=1}^A S_{aP} a T_a$$

最终模型仓库发布门禁可以写成:

$$G_{\{\text{model_release}\}} = \mathbf{1}[\text{left}[\min_j C_j \geq \tau_j \wedge C_{\{\text{checksum}\}} = 1 \wedge G_{\{\text{tokcfg}\}} = 1 \wedge$$

下面的 0 依赖 demo 把模型仓库和发布系统合成一个 toy 审计器。它输出 manifest、checksum、tokenizer / config、adapter、量化、灰度、回滚、血缘和生命周期指标, 并用 16 个 bad case 模拟面试中必须覆盖的风险点。

class MiniModelRegistryReleaseSystemDesignAudit:

def __init__(self):

 self.release_manifest = {

 "model_version": "company-chat:v1.4.0-dpo",

 "tokenizer_version": "tok-2026-06",

 "weight_manifest": "weights-manifest-sha256",

 "runtime_image": "runtime@sha256:abc",

 "runtime_config": "runtime-config-v7",

 "generation_config": "gen-config-v3",

 "safety_policy": "safety-policy-v12",

 "route_policy": "route-policy-v5",

 "eval_report": "eval-report-778",

 "rollback_target": "company-chat:v1.3.2",

 }

 self.weight_shards = [

 {"name": "model-00001.safetensors", "gib": 32.0, "checksum_ok": True},

 {"name": "model-00002.safetensors", "gib": 32.0, "checksum_ok": True},

 {"name": "model-00003.safetensors", "gib": 32.0, "checksum_ok": True},

 {"name": "model-00004.safetensors", "gib": 32.0, "checksum_ok": True},

]

 self.tokenizer_hash = "tok-hash-7"

```

self.recorded_tokenizer_hash = "tok-hash-7"
self.config_hash = "cfg-hash-9"
self.recorded_config_hash = "cfg-hash-9"
self.adapter = {"rank": 16, "hidden_size": 128, "base_model_ok": True}
self.quant = {
    "source_quality": 0.842,
    "quant_quality": 0.830,
    "source_latency_ms": 145,
    "quant_latency_ms": 100,
}
self.canary = {"quality_delta": 0.006, "latency_delta": -0.12, "error_delta": 0.002, "p0": 0}
self.rollback = {"alias_ready": True, "old_health_ok": True, "rollback_seconds": 45}
self.lineage = {"observed_edges": 14, "required_edges": 15}
self.lifecycle = [
    {"tier": "hot", "gib": 160, "usd_per_gib_month": 0.03, "months": 2},
    {"tier": "warm", "gib": 400, "usd_per_gib_month": 0.012, "months": 6},
    {"tier": "cold", "gib": 900, "usd_per_gib_month": 0.004, "months": 12},
]
self.gates = [
    "model_version_contract",
    "weight_manifest_integrity",
    "safe_weight_format",
    "tokenizer_config_binding",
    "adapter_base_compatibility",
    "merge_lineage_completeness",
    "quantization_eval_gate",
    "runtime_compatibility_fit",
    "eval_report_linkage",
    "safety_permission_gate",
    "release_status_governance",
    "rollback_alias_readiness",
    "artifact_lineage_completeness",
    "load_cache_readiness",
    "lifecycle_retention_policy",
    "model_registry_gate",
]

def release_manifest_completeness(self):
    required = {
        "model_version",
        "tokenizer_version",
        "weight_manifest",
        "runtime_image",
        "runtime_config",
        "generation_config",
        "safety_policy",
        "route_policy",
        "eval_report",
        "rollback_target",
    }
    present = sum(1 for field in required if self.release_manifest.get(field))
    return round(present / len(required), 3)

def checksum_coverage(self):
    checked = sum(1 for shard in self.weight_shards if shard["checksum_ok"])

```

```

    return round(checkered / len(self.weight_shards), 3)

def weight_total_size_gib(self):
    return round(sum(shard["gib"] for shard in self.weight_shards), 1)

def tokenizer_config_binding(self):
    return (
        self.tokenizer_hash == self.recorded_tokenizer_hash
        and self.config_hash == self.recorded_config_hash
    )

def adapter_rank_ratio(self):
    return round(self.adapter["rank"] / self.adapter["hidden_size"], 3)

def quant_quality_drop(self):
    return round(self.quant["source_quality"] - self.quant["quant_quality"], 3)

def quant_latency_speedup(self):
    return round(self.quant["source_latency_ms"] / self.quant["quant_latency_ms"], 3)

def canary_pass(self):
    return (
        self.canary["quality_delta"] >= -0.01
        and self.canary["latency_delta"] <= 0.05
        and self.canary["error_delta"] <= 0.01
        and self.canary["p0"] == 0
    )

def rollback_ready(self):
    return (
        self.rollback["alias_ready"]
        and self.rollback["old_health_ok"]
        and self.rollback["rollback_seconds"] <= 60
    )

def lineage_coverage(self):
    return round(self.lineage["observed_edges"] / self.lineage["required_edges"], 3)

def lifecycle_monthly_cost(self):
    cost = 0.0
    for item in self.lifecycle:
        cost += item["gib"] * item["usd_per_gib_month"] * item["months"]
    return round(cost, 2)

def registry_examples(self):
    return {
        "release_manifest_completeness": self.release_manifest_completeness(),
        "checksum_coverage": self.checksum_coverage(),
        "weight_total_size_gib": self.weight_total_size_gib(),
        "tokenizer_config_binding": self.tokenizer_config_binding(),
        "adapter_rank_ratio": self.adapter_rank_ratio(),
        "quant_quality_drop": self.quant_quality_drop(),
        "quant_latency_speedup": self.quant_latency_speedup(),
        "canary_pass": self.canary_pass(),
        "rollback_ready": self.rollback_ready(),
    }

```

```

        "lineage_coverage": self.lineage_coverage(),
        "lifecycle_cost": self.lifecycle_monthly_cost(),
    }

def design_cases(self):
    complete = {gate: True for gate in self.gates}
    bad_specs = [
        ("model_version_contract_missing_bad", "model_version_contract"),
        ("weight_manifest_checksum_missing_bad", "weight_manifest_integrity"),
        ("unsafe_weight_format_bad", "safe_weight_format"),
        ("tokenizer_config_mismatch_bad", "tokenizer_config_binding"),
        ("adapter_base_mismatch_bad", "adapter_base_compatibility"),
        ("merge_lineage_missing_bad", "merge_lineage_completeness"),
        ("quantization_quality_drop_bad", "quantization_eval_gate"),
        ("runtime_incompatible_bad", "runtime_compatibility_fit"),
        ("eval_report_missing_bad", "eval_report_linkage"),
        ("safety_permission_gap_bad", "safety_permission_gate"),
        ("release_state_machine_missing_bad", "release_status_governance"),
        ("rollback_alias_missing_bad", "rollback_alias_readiness"),
        ("artifact_lineage_broken_bad", "artifact_lineage_completeness"),
        ("load_cache_not_ready_bad", "load_cache_readiness"),
        ("lifecycle_policy_missing_bad", "lifecycle_retention_policy"),
        ("model_registry_gate_missing_bad", "model_registry_gate"),
    ]
    cases = [{"name": "complete_model_registry_release_design", "checks": complete}]
    for name, failed_gate in bad_specs:
        checks = {gate: True for gate in self.gates}
        checks[failed_gate] = False
        cases.append({"name": name, "checks": checks})
    return cases

def evaluate_cases(self):
    cases = self.design_cases()
    metrics = {}
    failed_cases = []
    failed_gates = set()
    for gate in self.gates:
        passed = sum(1 for case in cases if case["checks"][gate])
        metrics[gate] = round(passed / len(cases), 3)
    for case in cases:
        missing = [gate for gate, ok in case["checks"].items() if not ok]
        if missing:
            failed_cases.append(case["name"])
            failed_gates.update(missing)
    return {
        "metrics": metrics,
        "failed_cases": failed_cases,
        "failed_gates": sorted(failed_gates),
        "hard_blocker_count": len(failed_cases),
        "gate_pass": len(failed_cases) == 0,
    }

def run(self):
    evaluated = self.evaluate_cases()
    return {

```

```

        "registry_examples": self.registry_examples(),
        "metrics": evaluated["metrics"],
        "hard_blocker_count": evaluated["hard_blocker_count"],
        "failed_case_count": len(evaluated["failed_cases"]),
        "failed_gate_count": len(evaluated["failed_gates"]),
        "model_registry_gate_pass": evaluated["gate_pass"],
    }

```

```

if __name__ == "__main__":
    audit = MiniModelRegistryReleaseSystemDesignAudit()
    result = audit.run()
    print("registry_examples=" + str(result["registry_examples"]))
    print("metrics=" + str(result["metrics"]))
    print("hard_blocker_count=" + str(result["hard_blocker_count"]))
    print("failed_case_count=" + str(result["failed_case_count"]))
    print("failed_gate_count=" + str(result["failed_gate_count"]))
    print("model_registry_gate_pass=" + str(result["model_registry_gate_pass"]))

```

一组典型输出如下：

```

registry_examples={'release_manifest_completeness': 1.0, 'checksum_coverage': 1.0, 'weight_total_size_gib': 128.0, 'token
metrics={'model_version_contract': 0.941, 'weight_manifest_integrity': 0.941, 'safe_weight_format': 0.941, 'tokenizer_c
hard_blocker_count=16
failed_case_count=16
failed_gate_count=16
model_registry_gate_pass=False

```

这个 demo 的重点是把模型仓库从“存储路径”提升为“发布控制面”。真实系统里，这些字段应该由训练平台、artifact store、模型仓库、评估平台、发布控制器、推理平台、审计日志和成本系统共同填充。

57.26 核心 trade-off

Trade-off 包括：

1. 发布速度 vs 审批安全。
2. 模型版本保留 vs 存储成本。
3. 评估门禁严格 vs 迭代效率。
4. 权重下载便利 vs 模型资产安全。
5. 灰度慢 vs 新模型收益延迟。
6. 自动回滚 vs 指标波动误判。

企业平台要根据风险分层管理。

57.27 面试回答模板

可以这样回答：

我会把模型仓库与发布系统设计成模型资产生命周期管理系统。训练平台完成任务后，将 checkpoint 和 metadata 注册到 Model Registry。

模型仓库记录模型血缘，包括训练任务、数据版本、代码版本、checkpoint、eval report 和 deployment。评估平台完成 eval 后写回 Registry。

发布系统通过状态机管理 registered、validated、evaluated、approved、staged、canary、production、deprecated 等状态。每次发布都需要经过这些状态。

Deployment Controller 将 release 部署到推理平台，支持灰度、A/B、health check、warmup 和一键回滚。系统还提供权限控制、审计和成本系统。

57.28 常见扣分点

扣分点一：模型仓库就是对象存储。

问题：忽略元数据、版本、血缘、评估、权限和发布状态。

扣分点二：只管理权重，不管理 tokenizer。

问题：tokenizer 不匹配会导致严重线上问题。

扣分点三：不提评估门禁。

问题：模型发布必须基于质量、安全、延迟和成本判断。

扣分点四：回滚只回滚模型文件。

问题：还要回滚 runtime、prompt、route 和 safety policy。

扣分点五：不提生命周期。

问题：模型和 checkpoint 存储成本非常高。

57.29 小练习

1. Model 和 ModelVersion 有什么区别？
2. ReleaseManifest 应包含哪些内容？
3. 为什么 tokenizer 必须和模型版本绑定？
4. 量化版本上线前要评估什么？
5. 模型发布状态机如何设计？
6. Quality Gate 应包含哪些规则？
7. 回滚为什么不能只回滚权重？
8. 删除模型 artifact 前要检查哪些依赖？
9. 为什么 ReleaseManifest 比单独的 model version 更适合作为回滚单位？
10. 如何用 checksum 覆盖率、tokenizer/config 绑定、canary guardrail 和 rollback readiness 组成发布门禁？

57.30 本章小结

本章系统设计了模型仓库与发布系统。

你需要记住：

1. 模型仓库管理的不只是权重，还有 tokenizer、config、adapter、量化版本、eval report、权限、血缘和发布状态。
2. 发布系统要通过 Quality Gate、审批、灰度、A/B 和回滚控制线上风险。
3. ReleaseManifest 是可复现发布和可靠回滚的关键。
4. 模型 artifact 要做 checksum、安全校验、runtime 兼容性检查和生命周期管理。
5. 企业级模型发布必须可审计、可追溯、可回滚、可治理。
6. 第二轮新增的公式和 demo 可以把 manifest 完整率、checksum 覆盖率、tokenizer / config 绑定、量化退化、灰度 guardrail、回滚就绪、血缘覆盖和生命周期成本统一成可验收的发布门禁。

下一章我们会设计一个 AI Infra 可观测性平台。

第 58 章 设计一个 AI Infra 可观测性平台

上一章设计了模型仓库与发布系统。本章继续系统设计题：设计一个 AI Infra 可观测性平台。

AI Infra 的问题往往跨越训练、推理、数据、模型、RAG、Agent、GPU 集群、存储、网络 and 成本。如果没有统一可观测性平台，故障定位会变成到处翻日志、猜测指标、人工拼时间线。

先记住一句话：

AI Infra 可观测性平台的核心, 是把 metrics、logs、traces、events、cost 和 model quality signals 串起来, 让问题能被发现、定位、解释、复盘和治理。

58.0 本讲资料边界与第二轮精修口径

第二轮精修时, 本章按 “AI Infra 可观测性控制面 + 事故排查证据链” 校准, 不绑定某个 APM、日志系统、Prometheus 后端、Trace 后端或 Dashboard 产品实现。

参考口径包括:

1. OpenTelemetry signals: 把 traces、metrics、logs、baggage 和 profiles 看成不同观测信号, 用统一上下文传播和语义约定把系统活动串起来。
2. OpenTelemetry GenAI semantic conventions: 生成式 AI 语义约定已经独立维护, 并覆盖 GenAI clients、MCP、provider-specific conventions、spans、metrics 和 events; 本章只吸收其 “模型调用、token、latency、span、event 和 provider 属性要可关联” 的稳定方向, 不把 development 状态字段写成永久标准。
3. Prometheus data model: metric name 与 labels 共同唯一确定 time series, 任何 label value 变化都会创建新的 series, 因此 request ID、user ID、prompt hash 等高基数字段不能随意进入 metrics label。
4. Google SRE SLO / error budget / burn-rate alerting: 告警应围绕 SLI、SLO、错误预算消耗速度和多窗口多 burn-rate 设计, 而不是只围绕单点 CPU、GPU 或平均延迟阈值。

本章的目标不是教你搭一套监控页面, 而是把 “AI Infra 怎么看得见” 回答成可验证系统: 信号清单完整, metric contract 可治理, SLO 和错误预算可计算, trace 能拆出训练 / 推理 / RAG / Agent 阶段, logs 和 events 能复盘时间线, 统一 ID 能把模型、数据、发布、请求、成本和事故串起来, 高基数、隐私、留存、告警噪音和成本都有边界。

58.1 题目理解

面试题可能这样问:

1. 请设计一个 AI Infra 可观测性平台。
2. 如何监控训练、推理、RAG 和 Agent?
3. 如何定位 TTFT、TPOT、loss 异常、GPU 利用率低等问题?
4. 如何统一 metrics、logs、traces 和 events?

这类题既考后端可观测性, 也考 AI 系统特有指标。

58.2 需求澄清

先问:

1. 观测对象包括哪些? 训练、推理、数据、评估、RAG、Agent?
2. 集群规模多大?
3. 是否多租户?
4. 是否有 SLO 和告警体系?
5. 是否需要 trace 回放?
6. 日志是否包含敏感数据?
7. 是否需要成本可观测性?
8. 是否需要模型质量指标?
9. 是否和事故复盘系统联动?
10. 数据留存周期要求是什么?

AI Infra 可观测性不能只按普通微服务指标设计。

58.3 核心目标

平台目标:

1. 收集 metrics、logs、traces、events。

2. 覆盖训练、推理、数据、模型、RAG、Agent、集群和成本。
3. 支持统一 ID 关联。
4. 支持多租户隔离和权限。
5. 支持 dashboard 和告警。
6. 支持慢请求和失败任务诊断。
7. 支持 SLO 和错误预算。
8. 支持事故复盘和变更关联。
9. 支持成本可观测性。
10. 支持日志脱敏和留存治理。

58.4 总体架构

可以设计如下：

Instrumentation SDK / Agents / Exporters

- > Metrics Pipeline
- > Logs Pipeline
- > Traces Pipeline
- > Events Pipeline
- > Cost Pipeline
- > Correlation Layer
- > Query / Dashboard / Alerting
- > Incident / Postmortem / SLO

数据来源包括：

1. Training platform。
2. Inference platform。
3. GPU cluster。
4. Kubernetes。
5. Runtime。
6. Data platform。
7. Eval platform。
8. RAG/Agent platform。
9. Cost system。
10. Release system。

58.5 Metrics Pipeline

Metrics pipeline 处理时间序列指标。

指标来源：

1. GPU exporter。
2. Kubernetes metrics。
3. training SDK。
4. inference runtime。
5. API Gateway。
6. storage system。
7. network system。
8. eval platform。

指标例子：

1. GPU utilization。
2. train loss。
3. tokens/s。
4. TTFT。
5. TPOT。

6. queue length。
7. KV cache usage。
8. error rate。
9. cost per 1k tokens。

Metrics 用于趋势、告警和 SLO。

58.6 Logs Pipeline

Logs pipeline 处理日志。

日志来源：

1. 训练脚本。
2. runtime。
3. API Gateway。
4. scheduler。
5. data loader。
6. checkpoint。
7. RAG retrieval。
8. tool call。
9. security audit。

日志要求：

1. 结构化。
2. 带 request ID / job ID。
3. 支持脱敏。
4. 支持采样。
5. 支持权限控制。
6. 支持 TTL。

日志不是越多越好，关键是可检索、可关联、可安全保存。

58.7 Traces Pipeline

Traces pipeline 记录链路。

推理 trace：

gateway -> auth -> router -> cache -> queue -> prefill -> decode -> safety -> response

RAG trace：

query rewrite -> retrieval -> filter -> rerank -> prompt assembly -> generation

Agent trace：

plan -> model call -> tool call -> observation -> next step -> final answer

Trace 适合定位慢在哪、失败在哪、为什么路由到这个模型或工具。

58.8 Events Pipeline

Events 是状态变化。

事件包括：

1. TrainingJob submitted。
2. Job scheduled。
3. Pod failed。
4. Checkpoint saved。

5. Model deployed。
6. Canary started。
7. Autoscaler scaled out。
8. Router fallback triggered。
9. Dataset quality gate failed。
10. Security policy denied。

Events 用于构建事故时间线。

指标告诉你发生了异常，事件告诉你异常前后发生了什么。

58.9 Cost Pipeline

AI Infra 成本要进入可观测性平台。

成本数据包括：

1. GPU hours。
2. idle GPU hours。
3. failed job cost。
4. storage cost。
5. network cost。
6. inference cost per 1k tokens。
7. eval cost。
8. agent run cost。

成本维度：

1. tenant。
2. team。
3. project。
4. model。
5. endpoint。
6. training job。
7. experiment run。

没有成本可观测性，就无法做成本治理。

58.10 统一 ID 体系

可观测性平台必须有统一 ID。

常见 ID：

1. request ID。
2. trace ID。
3. tenant ID。
4. user ID。
5. model version ID。
6. endpoint ID。
7. training job ID。
8. experiment run ID。
9. dataset version ID。
10. deployment ID。

这些 ID 要贯穿 metrics、logs、traces 和 events。

否则无法从一个慢请求跳到对应日志、路由、模型版本和成本记录。

58.11 Correlation Layer

Correlation Layer 负责关联不同信号。

例如：

1. 从 p99 告警跳到慢请求 trace。
2. 从 trace 跳到 runtime 日志。
3. 从 model version 跳到发布事件。
4. 从 training job 跳到 checkpoint 和成本。
5. 从 RAG 答案跳到 retrieval trace。

这层是 AI Infra 可观测性平台的关键。

没有关联，数据只是散落的监控碎片。

58.12 训练可观测性设计

Training dashboard 应展示：

1. job status。
2. queue wait time。
3. loss curve。
4. learning rate。
5. gradient norm。
6. tokens/s。
7. step time breakdown。
8. GPU utilization。
9. data loader time。
10. NCCL communication。
11. checkpoint events。
12. failed/retry events。

训练问题要能拆到数据、计算、通信、checkpoint 和调度。

58.13 推理可观测性设计

Inference dashboard 应展示：

1. QPS。
2. TTFT。
3. TPOT。
4. p95/p99 latency。
5. input/output tokens/s。
6. queue length。
7. active sequences。
8. KV cache usage。
9. GPU utilization。
10. cache hit rate。
11. fallback rate。
12. error/timeout。
13. cost per 1k tokens。

同时支持按 model、tenant、endpoint、runtime、region 切分。

58.14 RAG/Agent 可观测性设计

RAG dashboard：

1. retrieval latency。

2. empty result rate。
3. top-k hit rate。
4. rerank latency。
5. citation availability。
6. groundedness 指标。

Agent dashboard:

1. run success rate。
2. step count。
3. tool call count。
4. tool success rate。
5. loop detection。
6. cost per run。
7. human confirmation count。

RAG/Agent 必须支持 trace 查看和回放。

58.15 数据和评估可观测性

数据平台:

1. ingestion 成功率。
2. 数据质量指标。
3. dataset build 状态。
4. shard 错误。
5. train/eval overlap。
6. data loader throughput。

评估平台:

1. eval job 状态。
2. samples/s。
3. eval score。
4. judge error rate。
5. report generation time。
6. quality gate pass rate。

模型质量信号也是可观测性的一部分。

58.16 告警系统

告警应支持:

1. 阈值告警。
2. SLO burn rate 告警。
3. 异常检测。
4. 事件驱动告警。
5. 组合条件告警。

示例:

TTFT p95 > SLO 且 queue wait time 持续升高

比单纯 GPU 利用率高更有价值。

告警要有 owner、severity、runbook 和 dashboard 链接。

58.17 SLO 和错误预算

可观测性平台应支持 SLO:

1. 定义 SLI。
2. 配置 SLO。
3. 计算错误预算。
4. 展示 burn rate。
5. 触发告警。
6. 联动发布策略。

推理 SLO 可以包括成功率、TTFT、TPOT、p99。

训练 SLO 可以包括任务成功率、调度等待、checkpoint 恢复成功率。

58.18 Dashboard 设计

不同角色需要不同 dashboard。

平台 SRE：

1. SLO。
2. error rate。
3. p99。
4. cluster health。
5. incidents。

训练工程师：

1. loss。
2. GPU。
3. data loader。
4. checkpoint。

推理工程师：

1. TTFT。
2. TPOT。
3. queue。
4. KV cache。

业务负责人：

1. 成本。
2. 用户反馈。
3. 模型质量。
4. 发布状态。

不要用一个 dashboard 覆盖所有需求。

58.19 隐私和权限

可观测数据可能包含敏感信息。

要求：

1. 日志脱敏。
2. trace 字段级权限。
3. prompt 和输出 TTL。
4. 审计日志长期保存。
5. 敏感 trace 采样。
6. 多租户隔离。
7. 访问审计。

可观测性平台本身也是敏感数据平台。

58.20 高基数控制

Metrics label 不能无限增加。

高基数字段：

1. request ID。
2. user ID。
3. prompt hash。
4. document ID。

这些不适合作为 metrics label。

适合放在 trace 或 logs 中。

低基数字段如 model、endpoint、tenant、status 可以作为 label。

58.21 数据留存和成本

可观测性数据本身成本很高。

策略：

1. metrics 聚合降采样。
2. debug logs 短期保留。
3. audit logs 长期保留。
4. trace 采样。
5. 错误 trace 全量保留。
6. 敏感字段脱敏或不落盘。
7. 冷热存储分层。

留存策略要按数据价值和风险分层。

58.22 Incident 集成

可观测性平台应和事故管理集成。

告警触发后：

1. 创建 incident。
2. 关联 dashboard。
3. 关联最近变更。
4. 记录时间线。
5. 记录应急操作。
6. 生成复盘模板。
7. 跟踪行动项。

这样事故处理不是散落在聊天记录里。

58.23 变更关联

很多故障来自变更。

可观测性平台应展示：

1. 模型发布。
2. prompt 修改。
3. runtime 升级。
4. 路由策略变化。
5. 数据版本变化。
6. 调度策略变化。
7. 集群升级。

排障时第一问题是：最近变了什么？

58.24 AI Infra 可观测性平台系统设计指标和最小 demo

把本章落到系统设计验收时，可以先定义一条观测对象记录：

$O_i = (s_i, m_i, l_i, t_i, e_i, k_i, q_i, r_i, c_i, a_i, p_i, z_i)$

其中 s_i 是信号清单， m_i 是 metric contract， l_i 是 log 结构， t_i 是 trace span， e_i 是事件时间线， k_i 是统一关联 ID， q_i 是 SLO / latency 分位， r_i 是训练、推理、RAG / Agent、数据质量等 AI 专属观测项， c_i 是成本， a_i 是告警动作， p_i 是隐私和留存策略， z_i 是最终平台门禁状态。

统一覆盖率可以写成：

$C_j = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[g_j(O_i)=1]$

可用性 SLI 可以写成：

$A = \frac{N_{\{\mathrm{good}\}}}{N_{\{\mathrm{total}\}}}$

错误预算剩余比例可以写成：

$B_{\{\mathrm{remain}\}} = \frac{(1 - S_{\{\mathrm{slo}\}}) - (1 - A)}{1 - S_{\{\mathrm{slo}\}}}$

延迟分位可以写成：

$Q_p = \inf\{x: F_L(x) \geq p\}$

Trace span 覆盖率可以写成：

$C_{\{\mathrm{span}\}} = \frac{|S_{\{\mathrm{observed}\}} \cap S_{\{\mathrm{required}\}}|}{|S_{\{\mathrm{required}\}}|}$

统一关联 ID 覆盖率可以写成：

$C_{\{\mathrm{corr}\}} = \frac{N_{\{\mathrm{with_id}\}}}{N_{\{\mathrm{records}\}}}$

Metrics 基数估算可以写成：

$N_{\{\mathrm{series}\}} = M \prod_{d=1}^D n_d$

单位 token 成本可以写成：

$K_{\{1k\}} = 1000 \frac{K_{\{\mathrm{total}\}}}{N_{\{\mathrm{token}\}}}$

脱敏覆盖率可以写成：

$C_{\{\mathrm{redact}\}} = \frac{N_{\{\mathrm{redacted}\}}}{N_{\{\mathrm{sensitive}\}}}$

可行动告警率可以写成：

$A_{\{\mathrm{alert}\}} = \frac{N_{\{\mathrm{actionable}\}}}{N_{\{\mathrm{alerts}\}}}$

最终可观测性平台门禁可以写成：

$G_{\{\mathrm{obs}\}} = \mathbf{1}[\min_j C_j \geq \tau_j \wedge A \geq S_{\{\mathrm{slo}\}} \wedge B_{\{\mathrm{remain}\}} \geq 0 \wedge N$

下面的 0 依赖 demo 把 AI Infra 可观测性平台压缩成一个 toy 审计器。它计算信号覆盖、metric contract、SLO、错误预算、分位延迟、span、日志、事件、关联 ID、高基数、训练 / 推理 / RAG / Agent、成本、脱敏和告警指标，并用 16 个 bad case 模拟系统设计面试中必须主动提到的风险点。

class MiniAIInfraObservabilityPlatformDesignAudit:

def __init__(self):

 self.signals = ["metrics", "logs", "traces", "events", "costs", "alerts", "dashboards", "versions"]

 self.metric_contracts = [

 {

 "name": "inference_e2e_latency_ms",

 "unit": "ms",

 "labels": {"tenant": "acme", "model": "chat", "endpoint": "chat", "status": "ok"},


```

        "aggregation": "histogram",
        "owner": "serving",
    },
    {
        "name": "training_tokens_per_s",
        "unit": "tokens/s",
        "labels": {"cluster": "train-a", "job_type": "pretrain", "gpu_type": "h100"},
        "aggregation": "gauge",
        "owner": "training",
    },
    {
        "name": "rag_groundedness",
        "unit": "ratio",
        "labels": {"tenant": "acme", "app": "support"},
        "aggregation": "gauge",
        "owner": "rag-platform",
    },
]
self.slo = {"good_requests": 9970, "requests_total": 10000, "availability_target": 0.99}
self.latency_samples_ms = [410, 520, 620, 700, 890]
self.traces = {
    "required": ["gateway", "auth", "router", "queue", "prefill", "decode", "safety", "response"],
    "observed": ["gateway", "auth", "router", "queue", "prefill", "decode", "safety", "response"],
}
self.logs = [
    {"ts": "2026-06-15T10:00:00Z", "level": "INFO", "service": "router", "trace_id": "tr_1", "event": "route"},
    {"ts": "2026-06-15T10:00:01Z", "level": "WARN", "service": "runtime", "trace_id": "tr_1", "event": "kv_pr"},
]
self.events = [
    {"ts": "2026-06-15T09:55:00Z", "event_type": "deployment_promoted", "entity": "chat-serving", "version": "1.0.0"},
    {"ts": "2026-06-15T09:58:00Z", "event_type": "autoscaler_scaled_out", "entity": "chat-serving", "version": "1.0.0"},
]
self.correlation = {"records_with_id": 50, "records_total": 50}
self.labels = {
    "metric_count": 10,
    "label_value_counts": {"tenant": 12, "model": 20, "endpoint": 10, "status": 4},
    "series_budget": 100000,
    "high_cardinality_metric_labels": [],
}
self.training = {"tokens_per_s": 178000, "data_wait_ratio": 0.08, "checkpoint_failures": 0}
self.inference = {"ttft_p95_ms": 620, "tpot_p95_ms": 45, "error_rate": 0.003, "kv_cache_pressure": 0.72}
self.rag_agent = {"retrieval_trace_coverage": 1.0, "tool_trace_coverage": 1.0, "execution_trace_coverage": 1.0}
self.cost = {"total_cost_usd": 240.0, "total_tokens": 20_000_000}
self.privacy = {"redacted_fields": 5, "sensitive_fields_total": 5, "ttl_days": 30}
self.alerts = [
    {"owner": "serving", "runbook": "runbook://serving-latency", "severity": "page", "actionable": True, "duplicate": False},
    {"owner": "training", "runbook": "runbook://checkpoint", "severity": "ticket", "actionable": True, "duplicate": False},
    {"owner": "platform-finops", "runbook": "runbook://cost-spike", "severity": "ticket", "actionable": True, "duplicate": False},
]
self.gates = [
    "signal_inventory_coverage",
    "metric_contract_completeness",
    "slo_error_budget_readiness",
    "latency_quantile_guard",
    "trace_span_coverage",
]

```

```

        "log_event_structure",
        "correlation_id_coverage",
        "cardinality_budget_control",
        "training_observability",
        "inference_observability",
        "data_quality_observability",
        "rag_agent_observability",
        "cost_observability",
        "privacy_redaction_retention",
        "alert_actionability",
        "observability_platform_gate",
    ]

    def quantile(self, values, p):
        ordered = sorted(values)
        index = max(0, min(len(ordered) - 1, int(p * len(ordered) + 0.999999) - 1))
        return ordered[index]

    def field_coverage(self, records, required_fields):
        total = len(records) * len(required_fields)
        present = 0
        for record in records:
            present += sum(1 for field in required_fields if record.get(field) not in (None, ""))
        return round(present / total, 3)

    def availability(self):
        return self.slo["good_requests"] / self.slo["requests_total"]

    def error_budget_remaining(self):
        budget = 1 - self.slo["availability_target"]
        used = 1 - self.availability()
        return (budget - used) / budget

    def trace_span_coverage(self):
        required = set(self.traces["required"])
        observed = set(self.traces["observed"])
        return len(required & observed) / len(required)

    def cardinality_estimate(self):
        series = self.labels["metric_count"]
        for count in self.labels["label_value_counts"].values():
            series *= count
        return series

    def rag_agent_trace_ready(self):
        return (
            self.rag_agent["retrieval_trace_coverage"] >= 0.95
            and self.rag_agent["tool_trace_coverage"] >= 0.95
            and self.rag_agent["execution_trace_coverage"] >= 0.95
        )

    def observability_examples(self):
        metric_fields = ["name", "unit", "labels", "aggregation", "owner"]
        log_fields = ["ts", "level", "service", "trace_id", "event", "status", "redacted"]
        event_fields = ["ts", "event_type", "entity", "version", "actor"]

```

```

    return {
        "signal_coverage": round(len(self.signals) / 8, 3),
        "metric_contract_coverage": self.field_coverage(self.metric_contracts, metric_fields),
        "availability": round(self.availability(), 4),
        "error_budget_remaining": round(self.error_budget_remaining(), 3),
        "latency_p99_ms": self.quantile(self.latency_samples_ms, 0.99),
        "trace_span_coverage": round(self.trace_span_coverage(), 3),
        "log_structure_coverage": self.field_coverage(self.logs, log_fields),
        "event_timeline_coverage": self.field_coverage(self.events, event_fields),
        "correlation_id_coverage": round(self.correlation["records_with_id"] / self.correlation["records_total"], 3),
        "cardinality_estimate": self.cardinality_estimate(),
        "training_tokens_per_s": self.training["tokens_per_s"],
        "inference_ttft_p95_ms": self.inference["ttft_p95_ms"],
        "rag_agent_trace_ready": self.rag_agent_trace_ready(),
        "cost_per_1k_tokens_usd": round(1000 * self.cost["total_cost_usd"] / self.cost["total_tokens"], 3),
        "redaction_coverage": round(self.privacy["redacted_fields"] / self.privacy["sensitive_fields_total"], 3),
        "actionable_alerts": sum(1 for alert in self.alerts if alert["actionable"]),
    }

def design_cases(self):
    complete = {gate: True for gate in self.gates}
    bad_specs = [
        ("signal_inventory_missing_bad", "signal_inventory_coverage"),
        ("metric_contract_missing_bad", "metric_contract_completeness"),
        ("slo_error_budget_burn_bad", "slo_error_budget_readiness"),
        ("latency_quantile_bad", "latency_quantile_guard"),
        ("trace_span_missing_bad", "trace_span_coverage"),
        ("log_event_structure_bad", "log_event_structure"),
        ("correlation_id_missing_bad", "correlation_id_coverage"),
        ("cardinality_explosion_bad", "cardinality_budget_control"),
        ("training_observability_gap_bad", "training_observability"),
        ("inference_observability_gap_bad", "inference_observability"),
        ("data_quality_observability_bad", "data_quality_observability"),
        ("rag_agent_trace_gap_bad", "rag_agent_observability"),
        ("cost_observability_missing_bad", "cost_observability"),
        ("privacy_redaction_retention_bad", "privacy_redaction_retention"),
        ("alert_noise_bad", "alert_actionability"),
        ("observability_gate_missing_bad", "observability_platform_gate"),
    ]
    cases = [{"name": "complete_observability_platform_design", "checks": complete}]
    for name, failed_gate in bad_specs:
        checks = {gate: True for gate in self.gates}
        checks[failed_gate] = False
        cases.append({"name": name, "checks": checks})
    return cases

def evaluate_cases(self):
    cases = self.design_cases()
    metrics = {}
    failed_cases = []
    failed_gates = set()
    for gate in self.gates:
        passed = sum(1 for case in cases if case["checks"][gate])
        metrics[gate] = round(passed / len(cases), 3)
    for case in cases:

```

```

        missing = [gate for gate, ok in case["checks"].items() if not ok]
        if missing:
            failed_cases.append(case["name"])
            failed_gates.update(missing)
    return {
        "metrics": metrics,
        "failed_cases": failed_cases,
        "failed_gates": sorted(failed_gates),
        "hard_blocker_count": len(failed_cases),
        "gate_pass": len(failed_cases) == 0,
    }

def run(self):
    evaluated = self.evaluate_cases()
    return {
        "observability_examples": self.observability_examples(),
        "metrics": evaluated["metrics"],
        "hard_blocker_count": evaluated["hard_blocker_count"],
        "failed_case_count": len(evaluated["failed_cases"]),
        "failed_gate_count": len(evaluated["failed_gates"]),
        "observability_platform_gate_pass": evaluated["gate_pass"],
    }

if __name__ == "__main__":
    audit = MiniAIInfraObservabilityPlatformDesignAudit()
    result = audit.run()
    print("observability_examples=" + str(result["observability_examples"]))
    print("metrics=" + str(result["metrics"]))
    print("hard_blocker_count=" + str(result["hard_blocker_count"]))
    print("failed_case_count=" + str(result["failed_case_count"]))
    print("failed_gate_count=" + str(result["failed_gate_count"]))
    print("observability_platform_gate_pass=" + str(result["observability_platform_gate_pass"]))

```

一组典型输出如下：

```

observability_examples={'signal_coverage': 1.0, 'metric_contract_coverage': 1.0, 'availability': 0.997, 'error_budget_readiness': 0.941}
metrics={'signal_inventory_coverage': 0.941, 'metric_contract_completeness': 0.941, 'slo_error_budget_readiness': 0.941}
hard_blocker_count=16
failed_case_count=16
failed_gate_count=16
observability_platform_gate_pass=False

```

这个 demo 的重点是把“可观测性平台”从“监控图表集合”提升为“排障和治理控制面”。真实系统里，这些字段应该由 instrumentation SDK、agent、exporter、metrics pipeline、logs pipeline、traces pipeline、events pipeline、cost pipeline、incident system、release system 和权限审计系统共同填充。

58.25 核心 trade-off

Trade-off:

1. 数据完整性 vs 成本。
2. Trace 细节 vs 隐私。
3. 指标标签丰富度 vs 高基数风险。
4. 告警灵敏度 vs 噪音。
5. 长期留存 vs 存储成本。
6. 统一平台 vs 团队自定义灵活性。

可观测性平台设计必须考虑这些权衡。

58.26 面试回答模板

可以这样回答：

我会把 AI Infra 可观测性平台设计成统一收集 metrics、logs、traces、events、cost 和 model quality signals 的系统。数据来源

架构上通过 SDK、exporter 和 agent 采集数据，分别进入 metrics、logs、traces、events 和 cost pipeline。平台通过统一 request

训练侧观察 loss、tokens/s、GPU、data loader、NCCL、checkpoint；推理侧观察 TTFT、TPOT、p99、queue、KV cache、tokens/s、

平台提供 dashboard、SLO、burn rate 告警、incident 集成、变更关联和复盘支持，同时通过脱敏、权限、采样、TTL 和高基数控制

58.27 常见扣分点

扣分点一：只讲 Prometheus 和 Grafana。

问题：没有覆盖 logs、traces、events、AI 特有指标和关联层。

扣分点二：不提 TTFT、TPOT、KV cache。

问题：不理解推理可观测性关键指标。

扣分点三：不提训练 loss 和 data loader。

问题：训练平台可观测性不完整。

扣分点四：不提隐私和脱敏。

问题：AI 日志和 trace 可能包含敏感 prompt、文档和工具结果。

扣分点五：不提高基数。

问题：metrics 系统容易被 request ID、user ID 等高基数字段打爆。

58.28 小练习

1. AI Infra 可观测性平台和普通微服务监控有什么不同？
2. 为什么需要统一 request ID 和 training job ID？
3. 推理平台 dashboard 应展示哪些关键指标？
4. RAG trace 应该包含哪些阶段？
5. 什么字段适合作为 metrics label，什么字段不适合？
6. 如何设计 TTFT SLO 告警？
7. 可观测性数据如何做留存分层？
8. 如何把变更记录和故障排查关联起来？

58.29 本章小结

本章系统设计了一个 AI Infra 可观测性平台。

你需要记住：

1. AI Infra 可观测性要覆盖 metrics、logs、traces、events、cost 和 model quality signals。
2. 训练、推理、数据、评估、RAG、Agent 和成本都有专属指标和 trace。
3. 统一 ID 和 correlation layer 是从告警定位根因的关键。
4. SLO、错误预算、告警、incident 和变更关联让可观测性进入生产治理闭环。
5. 隐私、脱敏、采样、留存和高基数控制是 AI 可观测性平台不可忽视的工程问题。

下一章我们会整理 AI Infra 面试高频问题与标准回答。

第 59 章 AI Infra 面试高频问题与标准回答

上一章设计了 AI Infra 可观测性平台。本章整理 AI Infra 面试中的高频问题与标准回答。

这章不是重新讲知识点，而是帮助你把前面 58 章压缩成面试可用的回答模板。面试时最重要的不是背术语，而是能把问题拆开，讲清楚目标、架构、关键机制、trade-off 和生产治理。

先记住一句话：

AI Infra 面试的核心，是证明你既懂大模型任务特性，又懂分布式系统、GPU 集群、平台工程和生产治理。

59.0 本讲资料边界与第二轮精修口径

本章是第二十三册前 58 章的面试压缩版，不重新展开每个系统的完整设计。第二轮精修时按下面口径校准：

1. Kubernetes Job 只作为通用批任务控制器边界，用来解释 TrainingJob 为什么还需要训练语义、分布式 launcher、checkpoint、实验追踪、artifact 和权限成本治理。
2. vLLM / TGI / Triton / SGLang 只作为推理 runtime 例子，不把任何单个 runtime 写成唯一标准答案。回答推理题时要落到 prefill、decode、TTFT、TPOT、KV cache、continuous batching、queue 和 token throughput。
3. OpenTelemetry 的 signals 口径用于统一 metrics、logs、traces、events；Google SRE 的 SLO、错误预算和 burn-rate 思路用于约束线上可靠性回答。
4. 本章所有标准回答都要从“术语正确”升级为“可验证”：能给出指标、公式、demo 或项目证据、风险边界、trade-off 和生产治理链路。

面试回答可以抽象成一个样本：

$A_i = (q_i, t_i, m_i, d_i, r_i, u_i, s_i, c_i, o_i, p_i, z_i)$

其中 q 是问题， t 是覆盖主题， m 是指标或公式， d 是 demo / 项目证据， r 是风险， u 是 trade-off， s 是得分， c 是成本或安全约束， o 是可观测性证据， p 是生产落地证据， z 是下一步修复计划。

几个最低限度的自评指标：

$$C_{\{\mathrm{topic}\}} = \frac{|T_{\{\mathrm{covered}\}}|}{|T_{\{\mathrm{required}\}}|}$$

$$C_{\{\mathrm{formula}\}} = \frac{|F_{\{\mathrm{used}\}}|}{|F_{\{\mathrm{required}\}}|}$$

$$C_{\{\mathrm{demo}\}} = \frac{|D_{\{\mathrm{shown}\}}|}{|D_{\{\mathrm{required}\}}|}$$

$$C_{\{\mathrm{risk}\}} = \frac{|R_{\{\mathrm{covered}\}}|}{|R_{\{\mathrm{required}\}}|}$$

$$C_{\{\mathrm{trade}\}} = \frac{|U_{\{\mathrm{covered}\}}|}{|U_{\{\mathrm{required}\}}|}$$

$$S_{\{\mathrm{avg}\}} = \frac{1}{N} \sum_{i=1}^N s_i, \quad R_{\{\mathrm{red}\}} = \frac{N_{\{\mathrm{red}\}}}{N}$$

最后用一个门禁判断是否可以进入正式面试：

$$G_{\{\mathrm{interview}\}} = \mathbf{1} \left[\begin{aligned} &C_{\{\mathrm{topic}\}} \geq \tau_t \\ &\text{and } C_{\{\mathrm{formula}\}} \geq \tau_f \\ &\text{and } C_{\{\mathrm{demo}\}} \geq \tau_d \\ &\text{and } C_{\{\mathrm{risk}\}} \geq \tau_r \\ &\text{and } C_{\{\mathrm{trade}\}} \geq \tau_u \\ &\text{and } S_{\{\mathrm{avg}\}} \geq \tau_s \\ &\text{and } R_{\{\mathrm{red}\}} = 0 \end{aligned} \right]$$

直觉是：AI Infra 面试不是“我知道很多组件名”，而是“我能把训练、推理、RAG / Agent、评估、模型仓库、可观测性、成本、安全和多租户这些知识点，压缩成有指标、有公式、有证据、有边界的回答”。

59.1 AI Infra 是什么

问题：什么是 AI Infra？它和 MLOps、LLMOps 有什么区别？

标准回答：

AI Infra 是支撑大模型训练、推理、评估和应用运行的基础设施与平台工程体系，覆盖 GPU 集群、网络、存储、调度、训练平台、推理平台等。

MLOps 更关注传统机器学习生命周期管理，例如数据、训练、部署、监控。LLMOps 更关注大模型应用生命周期，例如 prompt、RAG、Agent 等。

追问重点：AI Infra 不是单个工具，而是一套平台体系。

59.2 训练平台怎么设计

问题：请设计一个大模型训练平台。

标准回答：

我会把训练平台设计成 TrainingJob 生命周期管理系统。用户通过 Web、CLI、SDK 或 API 提交 TrainingJob，配置镜像、代码版本、超参数等。

平台先做身份、权限、配额、数据访问和配置校验，然后进入队列。Scheduler 根据 GPU 类型、优先级、配额、gang scheduling、拓扑感知等进行调度。

运行中采集日志、metrics、events、loss、tokens/s、GPU 利用率、data loader time 和 checkpoint 信息，并自动写入 experiment tracking。

关键词：TrainingJob、gang scheduling、launcher、checkpoint、experiment tracking、model registry。

59.3 为什么 TrainingJob 不是普通 K8s Job

问题：TrainingJob 和 Kubernetes Job 有什么区别？

标准回答：

K8s Job 只是通用批任务抽象，而 TrainingJob 需要表达训练语义，包括数据版本、代码版本、镜像、分布式 launcher、GPU 拓扑、超参数等。

大模型训练还需要 gang scheduling、rank/world size 注入、NCCL 环境、checkpoint 恢复和多租户队列，这些不是普通 K8s Job 自带的。

扣分点：只说 “TrainingJob 是一个 YAML”。

59.4 为什么需要 Gang Scheduling

问题：大模型训练为什么需要 gang scheduling？

标准回答：

多机多卡训练要求所有 worker 同时启动并组成同一个分布式作业。如果只启动一部分 worker，任务无法正常运行，还会占住 GPU 资源。

Gang scheduling 要求资源足够时整体调度，资源不足时整体排队，启动失败时整体回滚。它能避免部分 worker 占资源但任务无法执行。

延伸：gang scheduling 和资源碎片、拓扑感知经常一起考。

59.5 GPU 集群调度怎么设计

问题：请设计一个多租户 GPU 集群调度系统。

标准回答：

我会设计队列、配额和拓扑感知的调度控制平面。用户提交任务后，平台做身份、权限、队列和配额校验，然后进入 Queue Manager。

Scheduler 维护 Resource Snapshot，包括 GPU 类型、数量、健康状态、拓扑、CPU、内存、网络和本地盘。Policy Engine 根据租户策略进行调度。

如果高优先级任务资源不足，可以通过 Preemption Engine 抢占低优先级、可恢复任务。低优先级队列用于填充空闲 GPU。系统还要处理抢占和恢复。

关键词：queue、quota、topology-aware、gang scheduling、preemption、fairness。

59.6 GPU 利用率低怎么排查

问题：训练任务 GPU 利用率低，你怎么排查？

标准回答：

我会先拆分 step time，看时间花在 data loading、forward、backward、optimizer、communication 还是 checkpoint。GPU 利用率低

具体看 dataloader time、GPU idle time、storage throughput、CPU 利用率、NCCL latency、tokens/s、batch size 和 per-rank logs。定位阶段后再做对应优化。

重点：不要直接说“加 batch”。

59.7 Loss NaN 怎么排查

问题：训练 loss 出现 NaN 怎么排查？

标准回答：

先定位 NaN 出现的 step，保存对应 batch，检查输入、label、mask、tokenizer 和异常样本。再看 learning rate、gradient norm、

常见原因包括学习率过大、梯度爆炸、FP16 溢出、loss scale 不合适、数据异常、自定义 loss 除零、checkpoint resume 状态损坏。

重点：NaN 不要只靠跳过 batch 掩盖。

59.8 推理平台怎么设计

问题：请设计一个高并发大模型推理平台。

标准回答：

我会把平台分为治理层和 runtime 执行层。请求进入 API Gateway 后做认证、限流和 request ID 注入，再经过安全检查和 Model Router

Cache Layer 尝试命中 prompt cache、semantic cache 或 result cache。Admission Controller 根据队列、KV cache 和配额判断是否

扩缩容结合 input/output tokens/s、queue wait、TTFT、TPOT、KV cache、GPU 和 timeout。稳定性上做限流、熔断、timeout budget

关键词：Model Router、KV cache、continuous batching、TTFT、TPOT、autoscaling。

59.9 为什么不能只看 QPS

问题：LLM 推理为什么不能只用 QPS 衡量负载？

标准回答：

因为大模型请求的输入和输出 token 数差异巨大。同样 10 QPS，一个请求可能是 100 input tokens + 50 output tokens，另一个可能

LLM 推理更应该看 input tokens/s、output tokens/s、active sequences、KV cache usage、TTFT、TPOT、queue length 和 cost per token

重点：token 是推理负载的基本单位。

59.10 TTFT 和 TPOT

问题：TTFT 和 TPOT 分别是什么？怎么优化？

标准回答：

TTFT 是 Time To First Token，表示从请求进入到第一个 token 返回的时间，主要受 gateway、router、queue、tokenizer、prefill

TPOT 是 Time Per Output Token，表示每生成一个输出 token 的时间，主要受 decode、KV cache、batching、显存带宽、active sequences

优化 TTFT 可以做 prompt cache、减少排队、优化 prefill、长短请求分队列。优化 TPOT 可以做 continuous batching、KV cache 优化

59.11 KV Cache 为什么重要

问题：KV cache 是什么？为什么会成为瓶颈？

标准回答：

KV cache 保存 Transformer attention 中历史 token 的 Key 和 Value，避免 decode 时每一步重新计算完整上下文。

它能加速生成，但会占用大量显存，显存占用和 batch size、sequence length、层数、hidden size、数据类型有关。长上下文、高并

59.12 Continuous Batching

问题：Continuous batching 和普通 batching 有什么区别？

标准回答：

普通 batching 通常一批请求固定执行到结束，而生成式大模型每个请求输出长度不同，短请求会被长请求拖住。

Continuous batching 在每个 decode step 动态调整 batch，完成的请求退出，新请求加入，更适合生成式模型。它能提高 GPU 利用率。

59.13 模型路由和负载均衡

问题：模型路由和普通负载均衡有什么区别？

标准回答：

普通负载均衡是在等价实例之间分流。模型路由要先在不同能力、成本、延迟、安全等级、上下文长度和版本的模型之间选择，再选择

路由依据包括任务类型、租户权限、成本预算、SL0、灰度规则、endpoint 健康、queue length、KV cache 水位和 fallback chain。

59.14 自动扩缩容怎么设计

问题：LLM 推理自动扩缩容怎么设计？

标准回答：

不能只看 CPU、QPS 或 GPU 利用率。应结合 QPS、input/output tokens/s、queue length、queue wait time、TTFT、TPOT、p99、KV c

还要考虑冷启动，因为大模型实例需要拉镜像、加载权重、初始化 runtime、分配显存和预热 kernel。可以使用 warm pool、权重本地

59.15 RAG 平台怎么设计

问题：企业 RAG 平台怎么设计？

标准回答：

RAG 平台包括 Knowledge Base、Document Store、Chunk Store、Embedding Store、Vector Index、Retrieval Service、Reranker、P

文档进入平台后经过解析、清洗、chunking、embedding 和索引构建。检索时必须根据用户身份做权限过滤，再 rerank，最后组装 pr
k、rerank scores、进入 prompt 的 chunks 和 citation。

重点：企业 RAG 的核心是权限和 trace，不只是向量数据库。

59.16 Agent 平台怎么设计

问题：Agent 平台怎么设计？

标准回答：

Agent 平台需要管理 Agent Definition、Prompt、Tool Registry、Tool Permission、Memory、Run State 和 Execution Trace。

Agent Runtime 执行多步任务，控制最大 step、超时、工具调用、用户确认和错误处理。Tool Definition 要版本化，包括 schema、e in-the-loop。

平台必须记录 tool call trace 和 execution trace，用于调试、回放、评估、审计和成本归因。

59.17 模型仓库怎么设计

问题：模型仓库与发布系统怎么设计？

标准回答：

模型仓库管理 Model、ModelVersion、权重、tokenizer、config、adapter、量化版本、eval report、权限、血缘和发布状态。训练

评估平台写回 EvalReport，Quality Gate 根据质量、安全、延迟、成本和关键 slice 决定是否允许发布。发布系统通过 registered、

59.18 评估平台怎么设计

问题：大模型评估平台怎么设计？

标准回答：

评估平台包括 Eval Dataset Registry、Eval Suite Registry、Eval Job Scheduler、Inference Runner、Metric Engine、LLM Judge、

Eval dataset、prompt、metric definition、judge model、inference config 都要版本化。平台要支持离线批量评测、LLM-as-Judge、人工评测、slice evaluation、sample-level failure analysis、在线 A/B 和发布门禁。

重点：评估平台不是 benchmark 脚本。

59.19 可观测性平台怎么设计

问题：AI Infra 可观测性平台怎么设计？

标准回答：

我会设计统一收集 metrics、logs、traces、events、cost 和 model quality signals 的平台。数据来源包括训练平台、推理平台、C

通过 request ID、job ID、model version、tenant、deployment 等统一 ID 做关联，支持从告警跳到 trace、日志、事件、发布记录

59.20 成本治理怎么做

问题：AI Infra 成本治理怎么做？

标准回答：

成本治理要先做用量采集和归因。按 tenant、team、project、model、endpoint、training job、experiment run 统计 GPU hours、s

训练看 GPU hours、failed job cost、tokens/GPU hour、idle GPU hours。推理看 cost per request、cost per 1k tokens、cache sa

59.21 安全治理怎么做

问题：AI Infra 安全治理包括什么？

标准回答：

包括身份认证、权限控制、密钥管理、数据合规、多租户隔离、模型访问控制、供应链安全、运行时安全、日志 trace 安全、审计和

训练任务使用 service account 和短期凭证，不使用长期高权限密钥。RAG 检索必须在检索阶段做权限过滤。Agent 高风险工具调用要

59.22 多租户隔离怎么做

问题：AI Infra 多租户隔离包括哪些方面？

标准回答：

多租户隔离不仅是 namespace。它包括身份隔离、资源配额、队列隔离、数据权限、模型访问权限、网络隔离、secret 隔离、日志和

例如 RAG 中 chunk 和 embedding 要带权限标签，检索时强制过滤；推理缓存 key 要包含租户和权限域；训练任务读取 dataset 前要保

59.23 训练故障定位

问题：训练任务 hang 住怎么排查？

标准回答：

先看所有 rank 最后一条日志、step 是否停止增长、GPU 利用率是否部分为 0、dataloading worker 状态、NCCL 日志、节点事件和存储

Hang 常见原因是某个 rank 卡在 dataloading、某个 rank 先 OOM 或崩溃、NCCL collective 不一致、数据读取阻塞、checkpoint 保存

59.24 推理故障定位

问题：推理 p99 突然升高怎么排查？

标准回答：

先确定影响范围：模型、租户、endpoint、地域、请求类型。再按模型、租户、endpoint、输入长度、输出长度切分 p99。查看慢请

常见原因包括长请求混入、某个租户突增、长短请求混跑、某个实例拥塞、KV cache 水位高、cache 命中率下降、tool 调用慢、发布

59.25 SLO 和错误预算

问题：AI Infra 如何设计 SLO 和错误预算？

标准回答：

先定义 SLI，例如推理成功率、TTFT、TPOT、p99、timeout rate，训练任务成功率、调度等待时间、checkpoint 恢复成功率。再定义

错误预算是 SLO 允许的失败空间，用于平衡稳定性和迭代速度。预算消耗过快时应降低发布风险、暂停高风险变更或优先修复可靠性

59.26 回答系统设计题的结构

面试中回答 AI Infra 系统设计题，可以按这个结构：

1. 需求澄清。
2. 核心目标。
3. 核心对象。
4. 总体架构。
5. 请求或任务链路。
6. 调度和资源。
7. 数据和 artifact。
8. 容错和稳定性。
9. 可观测性。
10. 安全和权限。
11. 成本治理。
12. Trade-off。

这个结构适用于训练平台、推理平台、LLMOps、评估平台、调度系统等题目。

59.27 面试常用公式速记

AI Infra 面试中的公式不需要写得很复杂，但要能把系统对象量化。下面这些公式可以作为高频题的最小工具箱。
训练平台先看有效 batch、训练步数、训练算力和故障损失：

$$B_{\mathrm{eff}} = B_{\mathrm{micro}} A_{\mathrm{grad}} W$$

其中 B_{micro} 是单卡 micro batch， A_{grad} 是梯度累积步数， W 是 data parallel world size。

$$S = \left\lceil \frac{D_{\mathrm{tok}}}{B_{\mathrm{eff}}} \right\rceil \quad C_{\mathrm{train}} \approx 6ND_{\mathrm{tok}}$$

其中 D_{tok} 是训练 token 数， N 是 dense Transformer 参数量。 $6ND$ 是粗估，适合面试中做量级判断，不适合替代真实 profiler。

$$L_{\mathrm{lost}} = G \frac{t_{\mathrm{fail}} - t_{\mathrm{ckpt}}}{3600}$$

其中 G 是占用 GPU 数， $t_{\mathrm{fail}} - t_{\mathrm{ckpt}}$ 是距离最近 checkpoint 的秒数。它回答 “checkpoint 间隔太长会浪费多少 GPU 小时”。

推理平台先拆 TTFT、TPOT 和 KV cache：

$$\mathrm{TTFT} = T_{\mathrm{gateway}} + T_{\mathrm{router}} + T_{\mathrm{queue}} + T_{\mathrm{tokenize}} + T_{\mathrm{prefill}} \\ \mathrm{TPOT} = \frac{T_{\mathrm{last}} - T_{\mathrm{first}}}{N_{\mathrm{out}} - 1}$$

其中 N_{out} 是输出 token 数；如果只生成 1 个 token，应单独报告 TTFT，不强行计算 TPOT。

$$M_{\mathrm{KV}} = 2LBTH_{\mathrm{kv}}D_{\mathrm{h}}b$$

其中 L 是层数， B 是并发序列数， T 是上下文长度， H_{kv} 是 KV head 数， D_{h} 是 head dim， b 是每个元素字节数。前面的 2 来自 Key 和 Value 两份缓存。

SLO 和成本题要能落到错误预算和单位经济账：

$$A = 1 - \frac{N_{\mathrm{bad}}}{N_{\mathrm{total}}}$$

$$B_{\mathrm{remain}} = (1 - \mathrm{SLO})N_{\mathrm{total}} - N_{\mathrm{bad}}$$

$$K_{\mathrm{1k}} = 1000 \frac{K_{\mathrm{gpu}} + K_{\mathrm{runtime}} + K_{\mathrm{storage}} + K_{\mathrm{network}}}{N_{\mathrm{input}}}$$

多租户调度题要能解释配额、公平性和资源碎片：

$$D_{\mathrm{q}} = \max_r \frac{u_{\mathrm{q},r}}{c_{\mathrm{q},r}} \quad \Delta_{\mathrm{fair}} = \max_q D_{\mathrm{q}} - \min_q D_{\mathrm{q}}$$

其中 $u_{\mathrm{q},r}$ 是队列 q 在资源 r 上的使用量， $c_{\mathrm{q},r}$ 是该队列的配额或权重归一化容量。它不是唯一公平性定义，但能帮助面试中把 “公平” 从口号变成可度量对象。

59.28 AI Infra 面试准备度指标和最小 demo

下面这个 0 依赖 demo 把一轮 mock interview 的回答抽象成结构化记录。它故意构造 1 个完整回答和 16 个只缺一个关键门禁的坏回答，用来验证哪些题目会因为缺公式、缺 demo、缺风险边界、缺生产证据而被阻断。

```
from copy import deepcopy
```

```
class MiniAIInfraInterviewReadinessAudit:
    def __init__(self):
        self.required_topics = {
            "ai_infra_scope",
            "training_platform",
            "gpu_scheduler",
            "training_debug",
            "inference_platform",
```

```

        "ttft_tpot_kv",
        "rag_agent_platform",
        "model_registry_eval",
        "observability_slo",
        "cost_security_multitenancy",
    }
    self.required_formulas = {
        "ttft_decomposition",
        "tpot",
        "kv_cache_budget",
        "availability_error_budget",
        "cost_per_1k_token",
        "quota_fairness",
    }
    self.required_demos = {
        "scheduler_audit",
        "training_fault_audit",
        "inference_platform_audit",
        "model_registry_release_audit",
        "eval_platform_audit",
        "observability_audit",
        "security_cost_audit",
    }
    self.required_risks = {
        "tool_name_confusion",
        "checkpoint_recovery",
        "kv_pressure",
        "permission_leak",
        "high_cardinality",
        "incident_no_evidence",
        "cost_slo_regression",
        "tenant_leak",
    }
    self.required_tradeoffs = {
        "latency_vs_cost",
        "fairness_vs_utilization",
        "quality_vs_latency",
        "safety_vs_autonomy",
        "cost_vs_slo",
        "self_service_vs_governance",
    }

    self.gates = {
        "scope_boundary_clarity": lambda a: "ai_infra_scope" in a["topics"],
        "training_platform_lifecycle": lambda a: "training_platform" in a["topics"],
        "gpu_scheduler_quota_fairness": lambda a: "gpu_scheduler" in a["topics"],
        "training_debug_evidence": lambda a: "training_fault_audit" in a["demos"],
        "inference_platform_token_slo": lambda a: "inference_platform" in a["topics"],
        "ttft_tpot_kv_formula": lambda a: {
            "ttft_decomposition",
            "tpot",
            "kv_cache_budget",
        }.issubset(a["formulas"]),
        "rag_agent_permission_trace": lambda a: "rag_agent_platform" in a["topics"],
        "model_registry_release_gate": lambda a: (

```

```

        "model_registry_release_audit" in a["demos"]
    ),
    "eval_platform_reproducibility": lambda a: "eval_platform_audit" in a["demos"],
    "observability_correlation": lambda a: "observability_audit" in a["demos"],
    "slo_error_budget_readiness": lambda a: (
        "availability_error_budget" in a["formulas"]
    ),
    "cost_governance": lambda a: "cost_per_1k_token" in a["formulas"],
    "security_multitenancy": lambda a: "tenant_leak" in a["risks"],
    "incident_postmortem": lambda a: "incident_no_evidence" in a["risks"],
    "tradeoff_boundary_reasoning": lambda a: "cost_vs_slo" in a["tradeoffs"],
    "interview_revision_gate": lambda a: a["revision_plan"],
}
self.remediation_actions = {
    "scope_boundary_clarity": " 用 30 秒区分 AI Infra、MLOps 和 LLMOps，避免只报工具名。",
    "training_platform_lifecycle": " 补 TrainingJob 状态机、launcher、checkpoint、artifact 和实验追踪链路。",
    "gpu_scheduler_quota_fairness": " 补 queue、quota、dominant share、gang scheduling 和拓扑放置口径。",
    "training_debug_evidence": " 补 rank log、loss 曲线、NaN 检查、NCCL / data wait 证据和最小复现。",
    "inference_platform_token_slo": " 把 QPS 改成 input tokens/s、output tokens/s、TTFT、TPOT 和 KV 水位。",
    "ttft_tpot_kv_formula": " 手写 TTFT 拆分、TPOT 和 KV cache 显存公式，并说明变量含义。",
    "rag_agent_permission_trace": " 补 RAG ACL filter、tool permission、trace / replay 和 human confirmation。",
    "model_registry_release_gate": " 补 ModelVersion、ReleaseManifest、eval / safety gate、canary 和 rollback。",
    "eval_platform_reproducibility": " 补 eval dataset、metric、judge、sample result、artifact lineage 和回归。",
    "observability_correlation": " 补 request id / job id / model version 关联 metrics、logs、traces 和 even。",
    "slo_error_budget_readiness": " 补 SLI、SLO、错误预算、burn rate alert 和发布冻结规则。",
    "cost_governance": " 补 GPU hours、tokens、cache、storage、网络和 cost per 1k tokens 归因。",
    "security_multitenancy": " 补身份、RBAC / ABAC、tenant cache key、RAG 权限和日志隔离。",
    "incident_postmortem": " 补事故时间线、影响量化、根因、回滚、action item 和回归验证。",
    "tradeoff_boundary_reasoning": " 补 latency vs cost、fairness vs utilization、self-service vs governance。",
    "interview_revision_gate": " 把每道弱题绑定到一个缺失公式、一个 toy demo 和一个修复动作。",
}

@staticmethod
def coverage(items, required):
    return round(len(set(items) & required) / len(required), 3)

def make_complete_answer(self):
    return {
        "id": "complete_ai_infra_answer",
        "topics": set(self.required_topics),
        "formulas": set(self.required_formulas),
        "demos": set(self.required_demos),
        "risks": set(self.required_risks),
        "tradeoffs": set(self.required_tradeoffs),
        "score": 0.94,
        "red_flags": [],
        "revision_plan": True,
    }

def make_bad_case(self, case_id, field, value, red_flag):
    answer = deepcopy(self.make_complete_answer())
    answer["id"] = case_id
    answer["red_flags"] = [red_flag]
    if field == "revision_plan":
        answer["revision_plan"] = False

```

```

else:
    answer[field].discard(value)
return answer

def build_cases(self):
    bad_specs = [
        ("scope_tool_name_confusion", "topics", "ai_infra_scope", "tool_only"),
        ("training_platform_lacks_lifecycle", "topics", "training_platform", "job_yaml_only"),
        ("gpu_scheduler_lacks_quota_fairness", "topics", "gpu_scheduler", "fifo_only"),
        ("training_debug_lacks_evidence", "demos", "training_fault_audit", "no_debug_evidence"),
        ("inference_platform_lacks_token_slo", "topics", "inference_platform", "qps_only"),
        ("ttft_tpot_kv_formula_missing", "formulas", "kv_cache_budget", "formula_missing"),
        ("rag_agent_permission_trace_missing", "topics", "rag_agent_platform", "vector_db_only"),
        (
            "model_registry_release_gate_missing",
            "demos",
            "model_registry_release_audit",
            "artifact_path_only",
        ),
        ("eval_platform_reproducibility_missing", "demos", "eval_platform_audit", "benchmark_only"),
        ("observability_correlation_missing", "demos", "observability_audit", "dashboard_only"),
        (
            "slo_error_budget_missing",
            "formulas",
            "availability_error_budget",
            "slo_as_slogan",
        ),
        ("cost_governance_missing", "formulas", "cost_per_1k_token", "no_unit_cost"),
        ("security_multitenancy_missing", "risks", "tenant_leak", "auth_only"),
        ("incident_postmortem_missing", "risks", "incident_no_evidence", "no_postmortem"),
        ("tradeoff_boundary_missing", "tradeoffs", "cost_vs_slo", "no_tradeoff"),
        ("interview_gate_missing", "revision_plan", None, "no_revision_plan"),
    ]
    return [self.make_complete_answer()] + [
        self.make_bad_case(*spec) for spec in bad_specs
    ]

def score_answer(self, answer):
    return {
        gate_name: bool(check(answer))
        for gate_name, check in self.gates.items()
    }

def run(self):
    cases = self.build_cases()
    complete = cases[0]
    interview_examples = {
        "topic_coverage": self.coverage(complete["topics"], self.required_topics),
        "formula_coverage": self.coverage(complete["formulas"], self.required_formulas),
        "demo_evidence_coverage": self.coverage(complete["demos"], self.required_demos),
        "risk_coverage": self.coverage(complete["risks"], self.required_risks),
        "tradeoff_coverage": self.coverage(complete["tradeoffs"], self.required_tradeoffs),
        "average_score": round(
            sum(answer["score"] for answer in [complete]) / 1,
            3,
        )
    }

```

```

    ),
    "red_flag_rate": round(
        sum(bool(answer["red_flags"]) for answer in [complete]) / 1,
        3,
    ),
}

case_results = {answer["id"]: self.score_answer(answer) for answer in cases}
metrics = {}
for gate_name in self.gates:
    pass_count = sum(result[gate_name] for result in case_results.values())
    metrics[gate_name] = round(pass_count / len(cases), 3)

failed_cases = [
    case_id
    for case_id, result in case_results.items()
    if not all(result.values())
]
failed_gates = [
    gate_name
    for gate_name in self.gates
    if any(not result[gate_name] for result in case_results.values())
]
hard_blockers = [
    (case_id, [name for name, passed in result.items() if not passed])
    for case_id, result in case_results.items()
    if not all(result.values())
]
revision_plan = {
    case_id: [
        {
            "missing_gate": gate,
            "action": self.remediation_actions[gate],
        }
        for gate in gates
    ]
    for case_id, gates in hard_blockers
}
gate_pass = not failed_cases and min(metrics.values()) >= 0.9

return {
    "interview_examples": interview_examples,
    "metrics": metrics,
    "weak_questions": failed_cases,
    "revision_plan_sample": {
        case_id: revision_plan[case_id]
        for case_id in failed_cases[:3]
    },
    "hard_blocker_count": len(hard_blockers),
    "failed_case_count": len(failed_cases),
    "failed_gate_count": len(failed_gates),
    "ai_infra_interview_gate_pass": gate_pass,
}

```



```

audit = MiniAIInfraInterviewReadinessAudit()
result = audit.run()
print("interview_examples=", result["interview_examples"], sep="")
print("metrics=", result["metrics"], sep="")
print("weak_questions=", result["weak_questions"], sep="")
print("revision_plan_sample=", result["revision_plan_sample"], sep="")
print("hard_blocker_count=", result["hard_blocker_count"], sep="")
print("failed_case_count=", result["failed_case_count"], sep="")
print("failed_gate_count=", result["failed_gate_count"], sep="")
print("ai_infra_interview_gate_pass=", result["ai_infra_interview_gate_pass"], sep="")

```

输出里的 0.941 来自 16/17: 1 个完整样本通过, 16 个坏样本各自暴露一个缺口。真实复盘时不要追求把脚本写复杂, 而是把每道弱题绑定到 “一个缺失主题、一个缺失公式、一个缺失 demo、一个风险边界和一个下一步修复动作”。

59.29 高频扣分点总结

常见扣分点:

1. 只说工具名, 不讲系统链路。
2. 只看 QPS, 不看 token。
3. 只看 GPU 利用率, 不看有效产出。
4. 不提 checkpoint 和恢复。
5. 不提 gang scheduling。
6. 不提 KV cache。
7. 不提 TTFT/TPOT。
8. 不提多租户权限。
9. 不提 trace 和可观测性。
10. 不提成本治理。
11. 不提灰度和回滚。
12. 不讲 trade-off。

面试官通常不是要你背全, 而是看你能不能抓住核心矛盾。

59.30 小练习

1. 用 2 分钟回答 “设计一个训练平台”。
2. 用 2 分钟回答 “设计一个推理平台”。
3. 用 1 分钟解释 TTFT、TPOT 和 KV cache。
4. 用 1 分钟解释 gang scheduling 和资源碎片。
5. 用 2 分钟回答 “推理 p99 升高怎么排查”。
6. 用 2 分钟回答 “AI Infra 成本治理怎么做”。
7. 用 2 分钟回答 “企业 RAG 如何防止越权”。
8. 用 3 分钟回答 “设计一个 LLMOps 平台”。

59.31 本章小结

本章整理了 AI Infra 面试高频问题与标准回答。

你需要记住:

1. AI Infra 面试重在系统性, 不是单点工具。
2. 训练平台抓 TrainingJob、gang scheduling、checkpoint、实验追踪。
3. 推理平台抓 TTFT、TPOT、KV cache、continuous batching、路由、缓存和扩缩容。
4. 企业平台抓多租户、安全、评估、发布、trace 和成本治理。
5. 回答开放题时要讲目标、架构、链路、故障、可观测性、安全、成本和 trade-off。

下一章是第二十三册最后一章，我们会讲 AI Infra 未来趋势：Serverless GPU、异构算力、边缘推理和自治运维。

第 60 章 AI Infra 未来趋势：Serverless GPU、异构算力、边缘推理和自治运维

这是第二十三册《AI Infra、大模型基础设施与平台工程》的最后一章。

前面我们从硬件、集群、训练平台、推理平台、数据模型实验平台、可观测性、成本安全治理，一直讲到系统设计题。本章不再展开某一个具体平台，而是讨论 AI Infra 的未来趋势。

先记住一句话：

AI Infra 的未来，不是简单拥有更多 GPU，而是让算力更弹性、硬件更多元、推理更靠近用户、平台更自动化、治理更内建。

60.0 本讲资料边界与第二轮精修口径

本章讨论的是 AI Infra 的趋势判断方法，不是预测某个厂商、云平台或开源项目一定会采用哪条路线。第二轮精修时，本章按稳定抽象来写：

1. Serverless GPU 参考 Knative 式自动扩缩容、scale-to-zero 和按需服务的抽象，但大模型场景必须额外考虑权重加载、GPU warm pool、KV cache 状态和冷启动尾延迟。
2. 异构算力参考 Kubernetes Dynamic Resource Allocation 对设备资源抽象的方向，以及 GPU 分区、隔离和多实例化这类硬件能力；正文只写调度和适配原则，不绑定具体 CRD、驱动版本或云厂商接口。
3. 边缘推理只讨论低延迟、隐私、离线和成本约束下的云边路由，不把“端侧模型完全替代云端大模型”当成确定结论。
4. 自治运维参考 SRE 的 SLO、error budget、burn-rate 和分阶段自动化思想；自动修复必须保留审批、回滚、爆炸半径和审计边界。
5. 可观测性和 Agent trace 参考 OpenTelemetry signals 与 GenAI 语义约定的方向，但字段命名和生态实现仍会演进，因此本章强调 trace、metrics、logs、events 的契约，不把某个字段名写成唯一标准。

所以，本章的面试表达重点不是“背趋势名词”，而是能把趋势拆成可验证的指标、门禁和失败模式。

60.1 为什么要看趋势

AI Infra 变化非常快。

过去几年，大模型基础设施经历了多次变化：

1. 从单机训练到大规模分布式训练。
2. 从脚本训练到平台化训练。
3. 从单模型服务到多模型推理平台。
4. 从普通 API 到 RAG 和 Agent。
5. 从 GPU 堆资源到成本和可靠性治理。
6. 从实验驱动到评估和发布门禁。

未来还会继续变化。

面试中，趋势题常用于考察候选人是否有系统视野。

60.2 趋势一：Serverless GPU

Serverless GPU 的目标是让用户不关心底层 GPU 实例，只提交任务或请求，由平台自动分配、启动、伸缩和计费。

理想形态：

用户提交推理或训练任务

- > 平台自动选择 GPU 类型
- > 自动加载模型或数据
- > 自动扩缩容
- > 按实际使用计费

这类似普通 serverless，但 GPU 场景更难。

60.3 Serverless GPU 的难点

难点包括：

1. 大模型冷启动慢。
2. 权重加载耗时长。
3. GPU 资源昂贵，不能无限预热。
4. 多模型切换成本高。
5. KV cache 和显存状态难迁移。
6. 训练任务运行时间长。
7. GPU 资源碎片明显。
8. 不同 GPU 型号兼容性不同。

因此 Serverless GPU 更可能先在推理、embedding、评估和短任务中成熟，再逐步扩展到更复杂训练场景。

60.4 Serverless 推理

Serverless 推理会关注：

1. 模型按需加载。
2. 权重缓存。
3. 快速冷启动。
4. warm pool。
5. 按 token 计费。
6. 多模型共享 GPU。
7. 自动扩缩容。
8. 空闲自动释放。

对长尾模型特别有价值。

高频模型可以常驻，低频模型按需加载。

这能降低多模型平台的空闲成本。

60.5 趋势二：异构算力

未来 AI Infra 不会只有一种 GPU。

异构算力包括：

1. NVIDIA GPU。
2. AMD GPU。
3. TPU。
4. NPU。
5. 国产 AI 加速器。
6. CPU 推理。
7. 边缘端芯片。
8. 专用推理芯片。

异构算力带来的核心问题是：平台如何屏蔽差异，同时利用不同硬件的成本和性能优势。

60.6 异构算力调度

异构调度要考虑：

1. 模型是否支持该硬件。
2. runtime 是否支持该硬件。
3. 精度和量化格式是否兼容。
4. 性能是否达标。
5. 成本是否更优。
6. 数据和模型是否能迁移。
7. 可观测性指标是否统一。

未来平台需要一个硬件抽象层，让用户声明目标：成本优先、延迟优先、吞吐优先或合规优先，由平台选择硬件。

60.7 Runtime 生态会继续分化

推理 runtime 会继续分化。

可能方向：

1. 通用 LLM serving runtime。
2. 极致性能 runtime。
3. 多模态 runtime。
4. Agent-oriented runtime。
5. 边缘推理 runtime。
6. 硬件厂商专用 runtime。

平台要避免被单一 runtime 锁死。

更好的方式是定义统一 inference interface，并在底层适配不同 runtime。

60.8 趋势三：边缘推理

边缘推理指模型在靠近用户或设备侧运行。

场景包括：

1. 手机端助手。
2. 车载智能系统。
3. 工业设备。
4. 本地隐私推理。
5. 低延迟交互。
6. 离线可用应用。

边缘推理的价值：

1. 降低云端成本。
2. 降低网络延迟。
3. 提升隐私保护。
4. 支持离线场景。

但边缘资源受限，需要小模型、量化、蒸馏和本地缓存。

60.9 云边协同

未来不是全部云端，也不是全部边缘，而是云边协同。

例如：

1. 简单任务在端侧小模型处理。
2. 复杂任务路由到云端大模型。
3. 敏感数据尽量本地处理。

4. 云端负责模型更新和策略下发。
5. 边缘端上传匿名反馈用于改进。

这会让模型路由从“多模型路由”扩展到“云边路由”。

60.10 趋势四：推理和训练更加一体化

未来训练平台和推理平台会更紧密联动。

原因：

1. 在线反馈会进入训练数据。
2. 推理 trace 会进入评估和回归集。
3. 线上问题样本会触发微调。
4. 模型发布会依赖训练血缘。
5. 成本和质量会共同驱动模型选择。

这意味着平台需要闭环：

训练 -> 评估 -> 发布 -> 推理 -> 反馈 -> 数据 -> 再训练

AI Infra 会从单点平台走向闭环系统。

60.11 趋势五：更强的评估与质量治理

模型能力越强，评估越重要。

未来评估会更强调：

1. 线上真实任务评估。
2. 自动评估和人工评估结合。
3. 多维安全评估。
4. Agent 行为评估。
5. 工具调用评估。
6. 长期记忆评估。
7. 成本质量联合评估。
8. 持续回归测试。

模型发布不会只看 benchmark，而会看质量、安全、成本、SLO 和业务结果。

60.12 趋势六：Agent Infra 成为新基础设施

Agent 越来越多后，会需要专门的 Agent Infra。

它包括：

1. Agent runtime。
2. Tool registry。
3. Permission system。
4. Memory store。
5. Execution trace。
6. Human-in-the-loop。
7. Agent evaluation。
8. Cost control。
9. Safety guardrail。

Agent Infra 的难点是多步状态、工具副作用、安全和可回放。

未来很多 LLMOps 平台会向 AgentOps 演进。

60.13 趋势七：自治运维

自治运维是让平台自动发现、诊断和修复问题。

可能能力：

1. 自动检测 GPU 异常。
2. 自动摘除不健康 endpoint。
3. 自动回滚异常模型发布。
4. 自动分析 p99 升高原因。
5. 自动调整扩缩容参数。
6. 自动生成事故复盘草稿。
7. 自动推荐成本优化方案。
8. 自动识别数据质量异常。

自治运维不会一开始完全取代人，而是先成为值班工程师的辅助系统。

60.14 AI Ops 在 AI Infra 中的特殊性

AI Ops 在 AI Infra 中更复杂，因为指标更多、链路更长。

它需要理解：

1. GPU。
2. 分布式训练。
3. KV cache。
4. 数据质量。
5. 模型版本。
6. prompt 版本。
7. RAG trace。
8. Agent tool trace。
9. 成本。

普通微服务 AI Ops 不能直接迁移。

AI Infra 的自治运维必须理解 AI 任务语义。

60.15 趋势八：成本成为一等公民

未来 AI 平台设计会更早考虑成本。

不是上线后再优化，而是在设计阶段就考虑：

1. cost per token。
2. cost per request。
3. cost per agent run。
4. cost per eval sample。
5. tokens per GPU hour。
6. cache saved cost。
7. failed job cost。
8. warm pool idle cost。

模型质量不再是唯一目标。

同等质量下，成本更低的平台会更有竞争力。

60.16 趋势九：安全和合规内建

未来安全不会是外置审核，而会内建在平台流程中。

例如：

1. 数据接入自动分级。
2. 训练任务自动校验数据权限。
3. 模型继承训练数据敏感等级。
4. 推理请求自动脱敏。
5. RAG 检索自动权限过滤。
6. 工具调用自动风险分级。
7. 发布自动检查安全门禁。
8. 审计自动记录。

安全治理会从“人工流程”变成“平台默认能力”。

60.17 趋势十：标准化和互操作

AI Infra 生态会逐步标准化。

可能标准化对象：

1. 模型服务 API。
2. Tool schema。
3. Agent trace。
4. Eval report。
5. Model metadata。
6. Dataset manifest。
7. Observability signals。
8. Deployment manifest。

标准化能降低平台之间迁移成本。

但短期内，不同厂商和开源生态仍会有差异。

平台设计要保留适配层。

60.18 AI Infra 工程师未来能力模型

未来 AI Infra 工程师需要同时具备：

1. 分布式系统能力。
2. GPU 和硬件理解。
3. Kubernetes 和调度能力。
4. 训练和推理机制理解。
5. 数据平台能力。
6. 可观测性能力。
7. 安全和成本治理能力。
8. 产品化平台思维。
9. 模型评估理解。
10. 跨团队协作能力。

单纯会部署模型服务已经不够。

60.19 未来趋势判断指标和最小 demo

趋势题最容易回答得很虚。更稳的方式是把每个趋势都转成“指标 + 门禁 + 失败模式”。

可以把一个未来 AI Infra 方案样本记为：

$F_i = (s_i, h_i, e_i, l_i, a_i, o_i, k_i, r_i, m_i, z_i)$

其中 s 表示 Serverless GPU 能力，h 表示异构算力适配，e 表示边缘推理，l 表示训练、评估、发布、推理和反馈闭环，a 表示 Agent Infra，o 表示自治运维，k 表示成本治理，r 表示安全合规，m 表示互操作和标准化，z 表示最终上线门禁。

Serverless GPU 最关键的是冷启动和 warm pool 成本：

$$T_{\mathrm{cold}}=T_{\mathrm{schedule}}+T_{\mathrm{image}}+T_{\mathrm{weight}}+T_{\mathrm{runtime}}+T_{\mathrm{warmup}} \\ K_{\mathrm{warm}}=N_{\mathrm{warm}}K_{\mathrm{gpu}}T_{\mathrm{idle}}$$

其中 T_{cold} 是一次冷启动总耗时， T_{schedule} 是调度时间， T_{image} 是镜像准备时间， T_{weight} 是模型权重加载时间， T_{runtime} 是 runtime 初始化时间， T_{warmup} 是模型预热时间； K_{warm} 是 warm pool 空闲成本， N_{warm} 是常驻 GPU 数量， K_{gpu} 是单 GPU 单位时间成本， T_{idle} 是空闲时长。

异构算力不是简单“支持更多硬件”，而是要做可解释选择。一个简化打分可以写成：

$$S_h=w_qQ_h-w_{LL}L_h-w_{cC}_h+w_{pP}_h+w_{mM}_h$$

其中 Q_h 是质量保持程度， L_h 是延迟归一化惩罚， C_h 是成本归一化惩罚， P_h 是可迁移性， M_h 是成熟度；权重 w_q, w_l, w_c, w_p, w_m 由业务目标决定。

边缘推理要看本地命中和云端回退：

$$R_{\mathrm{local}}=\frac{N_{\mathrm{edgeok}}}{N_{\mathrm{total}}}, \quad R_{\mathrm{fallback}}=\frac{N_{\mathrm{cloud}}}{N_{\mathrm{total}}}$$

其中 N_{edgeok} 是端侧可直接完成的请求数， N_{cloud} 是回退到云端的请求数， N_{total} 是总请求数。隐私敏感任务更看重 R_{local} ，复杂推理任务更看重可靠回退。

自治运维必须有安全门：

$$G_{\mathrm{auto}}=\mathbf{1}\left[A_{\mathrm{detect}}\right]\wedge A_{\mathrm{diagnose}}\wedge A_{\mathrm{rollback}}\wedge H_{\mathrm{approve}}\wedge B_{\mathrm{blast}}\wedge \rho_{\mathrm{blast}}\leq r_{\mathrm{blast}}\right]$$

其中 A_{detect} 、 A_{diagnose} 和 A_{rollback} 分别表示检测、诊断和回滚能力， H_{approve} 表示高风险动作需要人工审批， B_{blast} 是自动动作影响范围， ρ_{blast} 是允许的最大爆炸半径。

最终趋势判断门禁可以写成：

$$G_{\mathrm{future}}=\mathbf{1}\left[G_{\mathrm{serverless}}\wedge G_{\mathrm{hetero}}\wedge G_{\mathrm{edge}}\wedge G_{\mathrm{loop}}\wedge G_{\mathrm{agent}}\wedge G_{\mathrm{auto}}\wedge G_{\mathrm{cost}}\wedge G_{\mathrm{security}}\wedge G_{\mathrm{interop}}\right]$$

下面这个 0 依赖 demo 演示如何把趋势题做成可运行审计。它不是生产系统，而是帮助你在面试中把“未来趋势”讲成“能验收的工程能力”。

```
from copy import deepcopy
```

```
class MiniAIInfraFutureTrendAudit:
    def __init__(self):
        self.gates = [
            "serverless_gpu_readiness",
            "heterogeneous_compute_fit",
            "edge_inference_fit",
            "closed_loop_learning",
            "agent_infra_governance",
            "autonomous_ops_safety",
```



```

        "cost_first_design",
        "security_builtin",
        "interop_standardization",
    ]
    self.actions = {
        "serverless_gpu_readiness": [
            "separate model weight cache from request path",
            "size warm pool by cold-start SLO and idle cost budget",
            "record scale-to-zero and runtime warmup behavior",
        ],
        "heterogeneous_compute_fit": [
            "build hardware-runtime-precision compatibility matrix",
            "rank devices by quality, latency, cost, portability, and maturity",
        ],
        "edge_inference_fit": [
            "route privacy-sensitive and low-latency tasks to local model first",
            "keep cloud fallback for hard tasks and model update flow",
        ],
        "closed_loop_learning": [
            "link inference traces to eval sets, release gates, and feedback datasets",
            "require human review before feedback enters training data",
        ],
        "agent_infra_governance": [
            "add tool registry, permission model, replayable traces, and agent evals",
            "enforce cost budget per agent run",
        ],
        "autonomous_ops_safety": [
            "separate detection, diagnosis, and repair permissions",
            "cap blast radius and require approval for high-risk actions",
        ],
        "cost_first_design": [
            "define cost per 1k tokens and warm-pool idle budget before launch",
            "attribute cost by model, tenant, workload, and environment",
        ],
        "security_builtin": [
            "make data classification, redaction, audit, and policy gate mandatory",
            "fail closed when permissions or secrets are missing",
        ],
        "interop_standardization": [
            "standardize model, dataset, deployment, trace, and eval manifests",
            "keep adapter layer between platform API and runtime implementations",
        ],
    }

}

def cold_start_ms(self, case):
    s = case["serverless"]
    return (
        s["schedule_ms"]
        + s["image_ms"]
        + s["weight_ms"]
        + s["runtime_ms"]
        + s["warmup_ms"]
    )

def warm_pool_cost(self, case):

```

```

s = case["serverless"]
return round(s["warm_gpu_count"] * s["gpu_cost_per_hour"] * s["idle_hours"], 2)

def best_hardware(self, case):
    h = case["heterogeneous"]
    scores = []
    for candidate in h["candidates"]:
        latency_penalty = candidate["latency_ms"] / h["max_latency_ms"]
        cost_penalty = candidate["cost_per_1k"] / h["max_cost_per_1k"]
        score = (
            0.45 * candidate["quality"]
            - 0.20 * latency_penalty
            - 0.20 * cost_penalty
            + 0.10 * candidate["portability"]
            + 0.05 * candidate["maturity"]
        )
        scores.append((round(score, 3), candidate))
    return max(scores, key=lambda item: item[0])

def edge_rates(self, case):
    e = case["edge"]
    return {
        "local_hit_rate": round(e["edge_ok"] / e["total_requests"], 3),
        "fallback_rate": round(e["cloud_fallback"] / e["total_requests"], 3),
    }

def gate_serverless_gpu_readiness(self, case):
    s = case["serverless"]
    return (
        self.cold_start_ms(case) <= s["cold_start_budget_ms"]
        and s["has_weight_cache"]
        and s["has_scale_to_zero"]
        and s["runtime_warmup_profiled"]
        and self.warm_pool_cost(case) <= s["warm_pool_budget"]
    )

def gate_heterogeneous_compute_fit(self, case):
    h = case["heterogeneous"]
    _, best = self.best_hardware(case)
    return (
        h["has_resource_abstraction"]
        and h["has_runtime_matrix"]
        and h["has_metric_parity"]
        and best["runtime_supported"]
        and best["quality"] >= h["min_quality"]
        and best["latency_ms"] <= h["max_latency_ms"]
        and best["cost_per_1k"] <= h["max_cost_per_1k"]
    )

def gate_edge_inference_fit(self, case):
    e = case["edge"]
    rates = self.edge_rates(case)
    return (
        rates["local_hit_rate"] >= e["min_local_hit_rate"]
        and rates["fallback_rate"] <= e["max_fallback_rate"]
    )

```

```

        and e["has_cloud_fallback"]
        and e["privacy_route_enabled"]
        and e["thermal_budget_checked"]
    )

def gate_closed_loop_learning(self, case):
    l = case["loop"]
    return (
        l["trace_to_eval"]
        and l["eval_to_release_gate"]
        and l["feedback_dataset"]
        and l["human_review"]
        and l["rollback_policy"]
        and l["online_feedback_rate"] >= l["min_feedback_rate"]
    )

def gate_agent_infra_governance(self, case):
    a = case["agent"]
    return (
        a["tool_registry"]
        and a["permission_model"]
        and a["trace_replay"]
        and a["agent_eval_suite"]
        and a["cost_budget_per_run"]
        and a["side_effect_policy"]
    )

def gate_autonomous_ops_safety(self, case):
    o = case["ops"]
    return (
        o["detect"]
        and o["diagnose"]
        and o["rollback"]
        and o["human_approval"]
        and o["audit_log"]
        and o["blast_radius"] <= o["max_blast_radius"]
    )

def gate_cost_first_design(self, case):
    k = case["cost"]
    return (
        k["cost_per_1k_tokens"] <= k["budget_per_1k_tokens"]
        and k["attribution_enabled"]
        and k["cache_policy"]
        and k["unit_cost_slo"]
        and self.warm_pool_cost(case) <= k["warm_pool_idle_budget"]
    )

def gate_security_builtin(self, case):
    r = case["security"]
    return (
        r["data_classification"]
        and r["secret_management"]
        and r["pii_redaction"]
        and r["permission_filter"]
    )

```

```

        and r["audit_log"]
        and r["release_policy_gate"]
    )

def gate_interop_standardization(self, case):
    m = case["interop"]
    return (
        m["model_manifest"]
        and m["dataset_manifest"]
        and m["deployment_manifest"]
        and m["trace_schema"]
        and m["eval_report_schema"]
        and m["adapter_layer"]
        and m["api_versioning"]
    )

def audit_case(self, case):
    failed = []
    for gate in self.gates:
        method = getattr(self, "gate_" + gate)
        if not method(case):
            failed.append(gate)
    return failed

def audit(self, cases):
    results = {}
    failed_cases = {}
    hard_blockers = []
    for case in cases:
        failed = self.audit_case(case)
        results[case["name"]] = failed
        if failed:
            failed_cases[case["name"]] = failed
            hard_blockers.extend((case["name"], gate) for gate in failed)

    metrics = {}
    total = len(cases)
    for gate in self.gates:
        pass_count = sum(1 for failed in results.values() if gate not in failed)
        metrics[gate] = round(pass_count / total, 3)

    failed_gate_names = sorted({gate for failed in results.values() for gate in failed})
    remediation_sample = {
        name: [self.actions[gate][0] for gate in gates]
        for name, gates in list(failed_cases.items())[:3]
    }
    return {
        "metrics": metrics,
        "failed_cases": failed_cases,
        "failed_gate_names": failed_gate_names,
        "hard_blockers": hard_blockers,
        "remediation_sample": remediation_sample,
        "future_trend_gate_pass": not failed_cases,
    }

```

```

def build_cases():
    complete = {
        "name": "complete_future_platform",
        "serverless": {
            "schedule_ms": 1200,
            "image_ms": 2200,
            "weight_ms": 18000,
            "runtime_ms": 2800,
            "warmup_ms": 1800,
            "cold_start_budget_ms": 30000,
            "has_weight_cache": True,
            "has_scale_to_zero": True,
            "runtime_warmup_profiled": True,
            "warm_gpu_count": 4,
            "gpu_cost_per_hour": 3.2,
            "idle_hours": 2.5,
            "warm_pool_budget": 40.0,
        },
        "heterogeneous": {
            "has_resource_abstraction": True,
            "has_runtime_matrix": True,
            "has_metric_parity": True,
            "min_quality": 0.92,
            "max_latency_ms": 650,
            "max_cost_per_1k": 0.035,
            "candidates": [
                {
                    "name": "gpu_a",
                    "quality": 0.96,
                    "latency_ms": 420,
                    "cost_per_1k": 0.030,
                    "portability": 0.90,
                    "maturity": 0.95,
                    "runtime_supported": True,
                },
                {
                    "name": "edge_npu",
                    "quality": 0.93,
                    "latency_ms": 610,
                    "cost_per_1k": 0.018,
                    "portability": 0.70,
                    "maturity": 0.75,
                    "runtime_supported": True,
                },
            ],
        },
    },
    "edge": {
        "total_requests": 1000,
        "edge_ok": 760,
        "cloud_fallback": 190,
        "min_local_hit_rate": 0.70,
        "max_fallback_rate": 0.25,
        "has_cloud_fallback": True,
        "privacy_route_enabled": True,
    },

```

```

    "thermal_budget_checked": True,
},
"loop": {
    "trace_to_eval": True,
    "eval_to_release_gate": True,
    "feedback_dataset": True,
    "human_review": True,
    "rollback_policy": True,
    "online_feedback_rate": 0.18,
    "min_feedback_rate": 0.10,
},
"agent": {
    "tool_registry": True,
    "permission_model": True,
    "trace_replay": True,
    "agent_eval_suite": True,
    "cost_budget_per_run": True,
    "side_effect_policy": True,
},
"ops": {
    "detect": True,
    "diagnose": True,
    "rollback": True,
    "human_approval": True,
    "audit_log": True,
    "blast_radius": 0.05,
    "max_blast_radius": 0.10,
},
"cost": {
    "cost_per_1k_tokens": 0.021,
    "budget_per_1k_tokens": 0.030,
    "attribution_enabled": True,
    "cache_policy": True,
    "unit_cost_slo": True,
    "warm_pool_idle_budget": 40.0,
},
"security": {
    "data_classification": True,
    "secret_management": True,
    "pii_redaction": True,
    "permission_filter": True,
    "audit_log": True,
    "release_policy_gate": True,
},
"interop": {
    "model_manifest": True,
    "dataset_manifest": True,
    "deployment_manifest": True,
    "trace_schema": True,
    "eval_report_schema": True,
    "adapter_layer": True,
    "api_versioning": True,
},
}

```

```

cases = [complete]

bad = deepcopy(complete)
bad["name"] = "serverless_gpu_cold_start_bad"
bad["serverless"]["weight_ms"] = 45000
bad["serverless"]["has_weight_cache"] = False
cases.append(bad)

bad = deepcopy(complete)
bad["name"] = "heterogeneous_compute_runtime_gap_bad"
bad["heterogeneous"]["has_runtime_matrix"] = False
bad["heterogeneous"]["candidates"][0]["runtime_supported"] = False
bad["heterogeneous"]["candidates"][1]["quality"] = 0.86
cases.append(bad)

bad = deepcopy(complete)
bad["name"] = "edge_without_cloud_fallback_bad"
bad["edge"]["edge_ok"] = 520
bad["edge"]["cloud_fallback"] = 410
bad["edge"]["has_cloud_fallback"] = False
cases.append(bad)

bad = deepcopy(complete)
bad["name"] = "closed_loop_feedback_untrusted_bad"
bad["loop"]["trace_to_eval"] = False
bad["loop"]["human_review"] = False
cases.append(bad)

bad = deepcopy(complete)
bad["name"] = "agent_infra_permission_gap_bad"
bad["agent"]["permission_model"] = False
bad["agent"]["trace_replay"] = False
cases.append(bad)

bad = deepcopy(complete)
bad["name"] = "autonomous_ops_unbounded_action_bad"
bad["ops"]["human_approval"] = False
bad["ops"]["blast_radius"] = 0.30
cases.append(bad)

bad = deepcopy(complete)
bad["name"] = "cost_first_design_missing_bad"
bad["cost"]["cost_per_1k_tokens"] = 0.061
bad["cost"]["attribution_enabled"] = False
cases.append(bad)

bad = deepcopy(complete)
bad["name"] = "security_built_in_missing_bad"
bad["security"]["pii_redaction"] = False
bad["security"]["release_policy_gate"] = False
cases.append(bad)

bad = deepcopy(complete)
bad["name"] = "interop_standardization_missing_bad"
bad["interop"]["trace_schema"] = False

```

```

bad["interop"]["adapter_layer"] = False
cases.append(bad)

return cases

audit = MiniAIInfraFutureTrendAudit()
cases = build_cases()
example_case = cases[0]
best_score, best_hardware = audit.best_hardware(example_case)
future_examples = {
    "cold_start_ms": audit.cold_start_ms(example_case),
    "warm_pool_cost": audit.warm_pool_cost(example_case),
    "best_hardware": best_hardware["name"],
    "best_hardware_score": best_score,
    **audit.edge_rates(example_case),
}
report = audit.audit(cases)

print("future_examples=", future_examples)
print("metrics=", report["metrics"])
print("hard_blocker_count=", len(report["hard_blockers"]))
print("failed_case_count=", len(report["failed_cases"]))
print("failed_gate_count=", len(report["failed_gate_names"]))
print("remediation_sample=", report["remediation_sample"])
print("future_trend_gate_pass=", report["future_trend_gate_pass"])

```

这段代码输出的核心含义是：

1. future_examples 把趋势题变成冷启动、warm pool 成本、异构硬件选择和云边路由指标。
2. metrics 说明每类未来能力在 toy case 中的覆盖率。
3. hard_blocker_count、failed_case_count 和 failed_gate_count 能帮助你定位 “趋势判断” 里最不成熟的模块。
4. remediation_sample 给出每类失败的第一步修复动作。
5. future_trend_gate_pass=False 说明未来趋势不是口号，只要 Serverless、异构、边缘、闭环、Agent、自治运维、成本、安全和互操作任一关键门禁缺失，就不能说平台已经具备面向未来的完整能力。

60.20 面试开放题回答思路

如果面试官问 “AI Infra 未来会怎么发展”，可以这样回答：

我认为 AI Infra 会朝几个方向发展。第一是更弹性的 Serverless GPU 和按需推理，降低长尾模型的空闲成本。第二是异构算力和硬件。这个回答覆盖弹性、硬件、边缘、闭环、Agent、安全、成本和运维。

60.21 常见误区

误区一：未来只是更多 GPU。

更多 GPU 重要，但弹性、调度、成本、数据、推理、评估和治理同样关键。

误区二：Serverless GPU 会马上解决所有问题。

大模型冷启动、权重加载、显存状态和成本让 Serverless GPU 很难一蹴而就。

误区三：边缘推理会取代云端推理。

更可能是云边协同，不同任务放在不同位置。

误区四：自治运维等于完全无人值守。

短期更现实的是辅助诊断、自动告警、自动回滚和推荐修复。

误区五：标准化会消除平台差异。

标准化会降低迁移成本，但性能、成本、治理和体验仍会形成差异。

60.22 给读者的最后建议

学 AI Infra，不要只学工具。

更重要的是理解：

1. 为什么大模型训练需要特殊调度。
2. 为什么推理平台要看 token 而不是只看 QPS。
3. 为什么 KV cache 是核心瓶颈。
4. 为什么数据版本和血缘影响模型质量。
5. 为什么评估和发布门禁是生产必需。
6. 为什么 trace 对 RAG/Agent 很重要。
7. 为什么安全和成本必须内建。
8. 为什么平台工程的最终目标是提升研发效率和生产可靠性。

如果你能把这些问题讲清楚，就已经具备 AI Infra 岗位的核心思维。

60.23 第二十三册总结

本册从 AI Infra 总览讲起，覆盖了：

1. GPU、显存、网络、存储和集群基础。
2. 训练平台工程。
3. 推理平台工程。
4. 数据、模型与实验平台。
5. 可观测性、可靠性与成本治理。
6. 安全、审计和变更管理。
7. 多个 AI Infra 系统设计题。

你现在应该能系统回答：

1. 如何设计训练平台？
2. 如何设计推理平台？
3. 如何设计 GPU 调度系统？
4. 如何设计评估平台？
5. 如何设计模型仓库？
6. 如何设计可观测性平台？
7. 如何做成本、安全和可靠性治理？

这就是 AI Infra 面试和实战中的主干能力。

60.24 本章小结

本章讲了 AI Infra 的未来趋势。

你需要记住：

1. Serverless GPU 会提升弹性，但受冷启动、权重加载和显存状态限制。
2. 异构算力会成为常态，平台需要硬件抽象和 runtime 适配能力。
3. 边缘推理和云边协同会在隐私、低延迟和成本场景中变重要。
4. 训练、评估、发布、推理和反馈会形成闭环。
5. Agent Infra、自治运维、成本治理、安全合规和标准化会成为未来平台竞争重点。
6. AI Infra 工程师需要同时懂系统、硬件、模型、数据、平台、安全和成本。

第二十三册到这里正文第一版完成。